

LeetMastery

Study-Order Edition

Python Only · Priority-Grouped ·

Difficulty-First · 1×1 Portrait

331 questions · 9 rounds · Inline Quick Review
· Chapter 2 Summary
High Easy → High Med → High Hard → Mid Easy
→ Mid Med → Mid Hard → Low Easy → Low Med
→ Low Hard

Contents

★ = LeetCode Premium (Premium 98 list)

Round 1 | High | E Easy (43) ↗

Arrays & Hashing (6) ↗

☐ ● #1 Two Sum ↗

☐ ● ★ #163 Missing Ranges ↗

☐ ● ★ #346 Moving Average from Data Stream ↗

☐ ● ★ #760 Find Anagram Mappings ↗

☐ ● ★ #1426 Counting Elements ↗

☐ ● #1534 Count Good Triplets ↗

String (3) ↗

☐ ● #383 Ransom Note ↗

☐ ● ★ #734 Sentence Similarity ↗

☐ ● ★ #1165 Single Row Keyboard ↗

Two Pointers (5) ↗

☐ ● #88 Merge Sorted Array ↗

☐ ● #125 Valid Palindrome ↗

☐ ● #283 Move Zeroes ↗

☐ ● ★ #408 Valid Word Abbreviation ↗

☐ ● #977 Squares of a Sorted Array ↗

Sorting (6)

☐ ● #169 Majority Element ↗

☐ ● #217 Contains Duplicate ↗

☐ ● #242 Valid Anagram ↗

☐ ● ★ #252 Meeting Rooms ↗

☐ ● ★ #1086 Largest Unique Number ↗

☐ ● #1122 Relative Sort Array ↗

Binary Search (5)

☐ ● #35 Search Insert Position ↗

☐ ● #69 Sqrt(x) ↗

☐ ● #278 First Bad Version ↗

☐ ● #374 Guess Number Higher or Lower ↗

☐ ● #704 Binary Search ↗

Matrix (5)

☐ ● ★ #422 Valid Word Square ↗

☐ ● #566 Reshape the Matrix ↗

- ☐ ● #766 Toeplitz Matrix ↗
- ☐ ● #867 Transpose Matrix ↗
- ☐ ● #2373 Largest Local Values in a Matrix ↗

Trees & BST (11)

- ☐ ● #100 Same Tree ↗
- ☐ ● #101 Symmetric Tree ↗
- ☐ ● #104 Maximum Depth of Binary Tree ↗
- ☐ ● #108 Convert Sorted Array to Binary Search Tree ↗
- ☐ ● #110 Balanced Binary Tree ↗
- ☐ ● #112 Path Sum ↗
- ☐ ● #226 Invert Binary Tree ↗
- ☐ ● ★ #270 Closest Binary Search Tree Value ↗
- ☐ ● #501 Find Mode in Binary Search Tree ↗
- ☐ ● #543 Diameter of Binary Tree ↗
- ☐ ● #572 Subtree of Another Tree ↗

DFS (2)

- ☐ ● #463 Island Perimeter ↗

☐ ● #733 Flood Fill ↗

Round 2 | High | M Medium (116) ↗

Arrays & Hashing (11) ↗

☐ ● #57 Insert Interval ↗

☐ ● #238 Product of Array Except Self ↗

☐ ● #281 Zigzag Iterator ↗

☐ ● #307 Range Sum Query – Mutable ↗

☐ ● #454 4Sum II ↗

☐ ● #525 Contiguous Array ↗

☐ ● #560 Subarray Sum Equals K ↗

☐ ● #1010 Pairs of Songs With Total Durations
Divisible by 60 ↗

☐ ● ★ #1272 Remove Interval ↗

☐ ● ★ #1429 First Unique Number ↗

☐ ● #3159 Find Occurrences of an Element in
an Array ↗

String (5) ↗

☐ ● #8 String to Integer (atoi) ↗

☐ ● ★ #249 Group Shifted Strings ↗

- ☐ ● ★ #271 Encode and Decode Strings ↗
- ☐ ● #635 Design Log Storage System ↗
- ☐ ● #1138 Alphabet Board Path ↗

Two Pointers (14)

- ☐ ● #11 Container With Most Water ↗
- ☐ ● #15 3Sum ↗
- ☐ ● #16 3Sum Closest ↗
- ☐ ● #31 Next Permutation ↗
- ☐ ● #75 Sort Colors ↗
- ☐ ● ★ #161 One Edit Distance ↗
- ☐ ● #167 Two Sum II – Input Array Is Sorted ↗
- ☐ ● ★ #186 Reverse Words in a String II ↗
- ☐ ● #189 Rotate Array ↗
- ☐ ● #532 K-diff Pairs in an Array ↗
- ☐ ● #923 3Sum With Multiplicity ↗
- ☐ ● #986 Interval List Intersections ↗
- ☐ ● ★ #1055 Shortest Way to Form String ↗
- ☐ ● #2563 Count the Number of Fair Pairs ↗

Sliding Window (8)

- ☐ ● #3 Longest Substring Without Repeating Characters ↗
- ☐ ● ★ #159 Longest Substring with At Most Two Distinct Characters ↗
- ☐ ● ★ #340 Longest Substring with At Most K Distinct Characters ↗
- ☐ ● #424 Longest Repeating Character Replacement ↗
- ☐ ● #438 Find All Anagrams in a String ↗
- ☐ ● ★ #487 Max Consecutive Ones II ↗
- ☐ ● ★ #1100 Find K-Length Substrings with No Repeated Characters ↗
- ☐ ● #1248 Count Number of Nice Subarrays ↗

Sorting (3)

- ☐ ● #49 Group Anagrams ↗
- ☐ ● #56 Merge Intervals ↗
- ☐ ● ★ #1152 Analyze User Website Visit Pattern ↗

Binary Search (12)

- ☐ ● #33 Search in Rotated Sorted Array ↗

- ☐ ● #34 Find First and Last Position of Element in Sorted Array ↗
- ☐ ● #153 Find Minimum in Rotated Sorted Array ↗
- ☐ ● #300 Longest Increasing Subsequence ↗
- ☐ ● #362 Design Hit Counter ↗
- ☐ ● #528 Random Pick with Weight ↗
- ☐ ● #852 Peak Index in a Mountain Array ↗
- ☐ ● #981 Time Based Key-Value Store ↗
- ☐ ● #1011 Capacity to Ship Packages Within D Days ↗
- ☐ ● ★ #1060 Missing Element in Sorted Array ↗
- ☐ ● #1062 Longest Repeating Substring ↗
- ☐ ● ★ #1533 Find the Index of the Large Integer ↗

Matrix (14)

- ☐ ● #36 Valid Sudoku ↗
- ☐ ● #48 Rotate Image ↗
- ☐ ● #54 Spiral Matrix ↗
- ☐ ● #59 Spiral Matrix II ↗

- ☐ ● #64 Minimum Path Sum ↗
- ☐ ● #73 Set Matrix Zeroes ↗
- ☐ ● #74 Search a 2D Matrix ↗
- ☐ ● #221 Maximal Square ↗
- ☐ ● ★ #311 Sparse Matrix Multiplication ↗
- ☐ ● #498 Diagonal Traverse ↗
- ☐ ● ★ #531 Lonely Pixel I ↗
- ☐ ● ★ #723 Candy Crush ↗
- ☐ ● #835 Image Overlap ↗
- ☐ ● ★ #1198 Find Smallest Common Element in All Rows ↗

Trees & BST (24)

- ☐ ● #98 Validate Binary Search Tree ↗
- ☐ ● #102 Binary Tree Level Order Traversal ↗
- ☐ ● #103 Binary Tree Zigzag Level Order Traversal ↗
- ☐ ● #105 Construct Binary Tree from Preorder and Inorder Traversal ↗
- ☐ ● #199 Binary Tree Right Side View ↗

- ☐ ● #230 Kth Smallest Element in a BST ↗
- ☐ ● #235 Lowest Common Ancestor of a Binary Search Tree ↗
- ☐ ● #236 Lowest Common Ancestor of a Binary Tree ↗
- ☐ ● ★ #250 Count Univalue Subtrees ↗
- ☐ ● #285 Inorder Successor in BST ↗
- ☐ ● ★ #298 Binary Tree Longest Consecutive Sequence ↗
- ☐ ● ★ #314 Binary Tree Vertical Order Traversal ↗
- ☐ ● ★ #333 Largest BST Subtree ↗
- ☐ ● ★ #366 Find Leaves of Binary Tree ↗
- ☐ ● #437 Path Sum III ↗
- ☐ ● ★ #545 Boundary of Binary Tree ↗
- ☐ ● ★ #549 Binary Tree Longest Consecutive Sequence II ↗
- ☐ ● ★ #582 Kill Process ↗
- ☐ ● #662 Maximum Width of Binary Tree ↗

- ☐ ● #863 All Nodes Distance K in Binary Tree ↗
- ☐ ● ★ #1120 Maximum Average Subtree ↗
- ☐ ● ★ #1490 Clone N-ary Tree ↗
- ☐ ● ★ #1522 Diameter of N-Ary Tree ↗
- ☐ ● ★ #1650 Lowest Common Ancestor of a Binary Tree III ↗

DFS (16)

- ☐ ● #130 Surrounded Regions ↗
- ☐ ● #133 Clone Graph ↗
- ☐ ● #200 Number of Islands ↗
- ☐ ● #207 Course Schedule ↗
- ☐ ● #210 Course Schedule II ↗
- ☐ ● ★ #261 Graph Valid Tree ↗
- ☐ ● #310 Minimum Height Trees ↗
- ☐ ● ★ #323 Number of Connected Components in an Undirected Graph ↗
- ☐ ● #417 Pacific Atlantic Water Flow ↗
- ☐ ● ★ #490 The Maze ↗

- ☐ ● ★ #694 Number of Distinct Islands ↗
- ☐ ● #695 Max Area of Island ↗
- ☐ ● #721 Accounts Merge ↗
- ☐ ● #785 Is Graph Bipartite? ↗
- ☐ ● ★ #1236 Web Crawler ↗
- ☐ ● #1242 Web Crawler Multithreaded ↗

Graphs (3)

- ☐ ● #128 Longest Consecutive Sequence ↗
- ☐ ● ★ #277 Find the Celebrity ↗
- ☐ ● ★ #1136 Parallel Courses ↗

BFS (6)

- ☐ ● ★ #286 Walls and Gates ↗
- ☐ ● #322 Coin Change ↗
- ☐ ● #542 01 Matrix ↗
- ☐ ● #994 Rotting Oranges ↗
- ☐ ● ★ #1197 Minimum Knight Moves ↗
- ☐ ● #1730 Shortest Path to Get Food ↗

Round 3 | High | H Hard (23)



Arrays & Hashing (1)

☐ ● #41 First Missing Positive ↗

Sliding Window (1)

☐ ● #76 Minimum Window Substring ↗

Binary Search (7)

☐ ● #4 Median of Two Sorted Arrays ↗

☐ ● #315 Count of Smaller Numbers After Self ↗

☐ ● #410 Split Array Largest Sum ↗

☐ ● #668 Kth Smallest Number in
Multiplication Table ↗

☐ ● #710 Random Pick with Blacklist ↗

☐ ● #774 Minimize Max Distance to Gas Station ↗

☐ ● #1235 Maximum Profit in Job Scheduling ↗

Matrix (1)

☐ ● #1074 Number of Submatrices That Sum to
Target ↗

Trees & BST (2)

☐ ● #124 Binary Tree Maximum Path Sum ↗

☐ ● #297 Serialize and Deserialize Binary Tree ↗

DFS (5)

- ☐ ● ★ #269 Alien Dictionary ↗
- ☐ ● #329 Longest Increasing Path in a Matrix ↗
- ☐ ● #685 Redundant Connection II ↗
- ☐ ● #1036 Escape a Large Maze ↗
- ☐ ● #1192 Critical Connections in a Network ↗

Graphs (2)

- ☐ ● ★ #305 Number of Islands II ↗
- ☐ ● #1153 String Transforms Into Another String ↗

BFS (4)

- ☐ ● #127 Word Ladder ↗
- ☐ ● ★ #317 Shortest Distance from All Buildings ↗
- ☐ ● #773 Sliding Puzzle ↗
- ☐ ● #815 Bus Routes ↗

Round 4 | Mid | E Easy (12) ↗

Linked List (7) ↗

- ☐ ● #21 Merge Two Sorted Lists ↗

- ☐ ● #141 Linked List Cycle ↗
- ☐ ● #206 Reverse Linked List ↗
- ☐ ● #234 Palindrome Linked List ↗
- ☐ ● #706 Design HashMap ↗
- ☐ ● #876 Middle of the Linked List ↗
- ☐ ● ★ #1474 Delete N Nodes After M Nodes of Linked List ↗

Stack (3)

- ☐ ● #20 Valid Parentheses ↗
- ☐ ● #232 Implement Queue using Stacks ↗
- ☐ ● #844 Backspace String Compare ↗

Trie (1)

- ☐ ● #14 Longest Common Prefix ↗

Greedy (1)

- ☐ ● #409 Longest Palindrome ↗

Round 5 | Mid | M Medium (56) ↗

Linked List (13) ↗

- ☐ ● #2 Add Two Numbers ↗
- ☐ ● #19 Remove Nth Node From End of List ↗

- ☐ ● #24 Swap Nodes in Pairs ↗
- ☐ ● #61 Rotate List ↗
- ☐ ● #146 LRU Cache ↗
- ☐ ● #148 Sort List ↗
- ☐ ● #328 Odd Even Linked List ↗
- ☐ ● ★ #369 Plus One Linked List ↗
- ☐ ● ★ #430 Flatten a Multilevel Doubly Linked List ↗
- ☐ ● #622 Design Circular Queue ↗
- ☐ ● #707 Design Linked List ↗
- ☐ ● ★ #708 Insert into a Sorted Circular Linked List ↗
- ☐ ● #1171 Remove Zero Sum Consecutive Nodes from Linked List ↗

Stack (14)

- ☐ ● #143 Reorder List ↗
- ☐ ● #150 Evaluate Reverse Polish Notation ↗
- ☐ ● #155 Min Stack ↗
- ☐ ● #173 Binary Search Tree Iterator ↗

- ☐ ● #227 Basic Calculator II ↗
- ☐ ● ★ #255 Verify Preorder Sequence in Binary Search Tree ↗
- ☐ ● #394 Decode String ↗
- ☐ ● ★ #439 Ternary Expression Parser ↗
- ☐ ● ★ #484 Find Permutation ↗
- ☐ ● #735 Asteroid Collision ↗
- ☐ ● #739 Daily Temperatures ↗
- ☐ ● #962 Maximum Width Ramp ↗
- ☐ ● ★ #1214 Two Sum BSTs ↗
- ☐ ● ★ #1762 Buildings With an Ocean View ↗

Heap (11)

- ☐ ● #215 Kth Largest Element in an Array ↗
- ☐ ● ★ #253 Meeting Rooms II ↗
- ☐ ● #347 Top K Frequent Elements ↗
- ☐ ● ★ #505 The Maze II ↗
- ☐ ● #621 Task Scheduler ↗
- ☐ ● #658 Find K Closest Elements ↗

- ☐ **● #787 Cheapest Flights Within K Stops ↗**
- ☐ **● #973 K Closest Points to Origin ↗**
- ☐ **● ★ #1057 Campus Bikes ↗**
- ☐ **● ★ #1167 Minimum Cost to Connect Sticks ↗**
- ☐ **● #1424 Diagonal Traverse II ↗**

Trie (7)

- ☐ **● #139 Word Break ↗**
- ☐ **● #208 Implement Trie (Prefix Tree) ↗**
- ☐ **● #211 Design Add and Search Words Data Structure ↗**
- ☐ **● ★ #616 Add Bold Tag in String ↗**
- ☐ **● #692 Top K Frequent Words ↗**
- ☐ **● #720 Longest Word in Dictionary ↗**
- ☐ **● #1166 Design File System ↗**

Backtracking (6)

- ☐ **● #17 Letter Combinations of a Phone Number**
- ☐ **● #22 Generate Parentheses ↗**
- ☐ **● #39 Combination Sum ↗**

☐ ● #46 Permutations ↗

☐ ● #79 Word Search ↗

☐ ● #113 Path Sum II ↗

Greedy (5)

☐ ● #134 Gas Station ↗

☐ ● #179 Largest Number ↗

☐ ● ★ #280 Wiggle Sort ↗

☐ ● #954 Array of Doubled Pairs ↗

☐ ● #2131 Longest Palindrome by
Concatenating Two Letter Words

Round 6 | Mid | H Hard (28) ↗

Linked List (3) ↗

☐ ● #25 Reverse Nodes in k-Group ↗

☐ ● #432 All O'one Data Structure ↗

☐ ● #460 LFU Cache ↗

Stack (8) ↗

☐ ● #32 Longest Valid Parentheses ↗

☐ ● #42 Trapping Rain Water ↗

☐ ● #84 Largest Rectangle in Histogram ↗

☐ ● #224 Basic Calculator ↗

☐ ● #239 Sliding Window Maximum ↗

☐ ● #716 Max Stack ↗

☐ ● ★ #772 Basic Calculator III ↗

☐ ● #895 Maximum Frequency Stack ↗

Heap (9)

☐ ● #23 Merge k Sorted Lists ↗

☐ ● #218 The Skyline Problem ↗

☐ ● ★ #272 Closest Binary Search Tree Value II ↗

☐ ● #295 Find Median from Data Stream ↗

☐ ● ★ #358 Rearrange String k Distance Apart ↗

☐ ● ★ #499 The Maze III ↗

☐ ● #632 Smallest Range Covering Elements
from K Lists ↗

☐ ● #759 Employee Free Time ↗

☐ ● #1425 Constrained Subsequence Sum ↗

Trie (4)

- ☐ ● #212 Word Search II ↗
- ☐ ● #336 Palindrome Pairs ↗
- ☐ ● ★ #588 Design In-Memory File System ↗
- ☐ ● ★ #642 Design Search Autocomplete System ↗

Backtracking (4)

- ☐ ● #37 Sudoku Solver ↗
- ☐ ● #51 N-Queens ↗
- ☐ ● #126 Word Ladder II ↗
- ☐ ● #996 Number of Squareful Arrays ↗

Round 7 | Low | E Easy (14) ↗

Dynamic Programming (2) ↗

- ☐ ● #70 Climbing Stairs ↗
- ☐ ● #121 Best Time to Buy and Sell Stock ↗

Bit Manipulation (6) ↗

- ☐ ● #67 Add Binary ↗
- ☐ ● #136 Single Number ↗
- ☐ ● #190 Reverse Bits ↗

☐ ● #191 Number of 1 Bits ↗

☐ ● #268 Missing Number ↗

☐ ● #338 Counting Bits ↗

Math (6)

☐ ● #9 Palindrome Number ↗

☐ ● #13 Roman to Integer ↗

☐ ● #504 Base 7 ↗

☐ ● ★ #1056 Confusing Number ↗

☐ ● ★ #1228 Missing Number in Arithmetic Progression ↗

☐ ● ★ #1427 Perform String Shifts ↗

Round 8 | Low | M Medium (32) ↗

Dynamic Programming (16) ↗

☐ ● #5 Longest Palindromic Substring ↗

☐ ● #53 Maximum Subarray ↗

☐ ● #55 Jump Game ↗

☐ ● #62 Unique Paths ↗

☐ ● #72 Edit Distance ↗

- ☐ ● #91 Decode Ways ↗
- ☐ ● #152 Maximum Product Subarray ↗
- ☐ ● #198 House Robber ↗
- ☐ ● ★ #256 Paint House ↗
- ☐ ● #309 Best Time to Buy and Sell Stock with ↗
Cooldown
- ☐ ● #377 Combination Sum IV ↗
- ☐ ● #416 Partition Equal Subset Sum ↗
- ☐ ● #435 Non-overlapping Intervals ↗
- ☐ ● #516 Longest Palindromic Subsequence ↗
- ☐ ● #576 Out of Boundary Paths ↗
- ☐ ● #1143 Longest Common Subsequence ↗

Bit Manipulation (4)

- ☐ ● #78 Subsets ↗
- ☐ ● #90 Subsets II ↗
- ☐ ● #287 Find the Duplicate Number ↗
- ☐ ● ★ #1506 Find Root of N-Ary Tree ↗

Math (5)

- ☐ **● #7 Reverse Integer** ↗
- ☐ **● #50 Pow(x, n)** ↗
- ☐ **● #380 Insert Delete GetRandom O(1)** ↗
- ☐ **● #1017 Convert to Base -2** ↗
- ☐ **● #3160 Find the Number of Distinct Colors Among the Balls** ↗

JavaScript (7)

- ☐ **● ★ #2627 Debounce** ↗
- ☐ **● ★ #2628 JSON Deep Equal** ↗
- ☐ **● ★ #2630 Memoize** ↗
- ☐ **● ★ #2633 Convert Object to JSON String** ↗
- ☐ **● ★ #2636 Promise Pool** ↗
- ☐ **● ★ #2675 Array of Objects to Matrix** ↗
- ☐ **● ★ #2700 Differences Between Two Objects** ↗

Round 9 | Low | H Hard (7)

Dynamic Programming (6) ↗

- ☐ **● #10 Regular Expression Matching** ↗
- ☐ **● #115 Distinct Subsequences** ↗

☐ ● #123 Best Time to Buy and Sell Stock III ↗

☐ ● #132 Palindrome Partitioning II ↗

☐ ● #312 Burst Balloons ↗

☐ ● #1000 Minimum Cost to Merge Stones ↗

Math (1)

☐ ● #68 Text Justification ↗

Round 1

High Priority · Easy

43 questions · 8 patterns

**Arrays & Hashing #1 #163 #346 #760 #1426
#1534**

String #383 #734 #1165

Two Pointers #88 #125 #283 #408 #977

Sorting #169 #217 #242 #252 #1086 #1122

Binary Search #35 #69 #278 #374 #704

Matrix #422 #566 #766 #867 #2373

**Trees & BST #100 #101 #104 #108 #110 #112
#226 #270 #501 #543 #572**

DFS #463 #733

Arrays & Hashing

Round 1 · High · Easy · 6 questions

Arrays & Hashing

□ #1 Two Sum

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table
Lists: Grind 169

Problem

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

Example 1:

```
Input: nums = [2,7,11,15], target = 9  
Output: [0,1]  
Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].
```

Example 2:

```
Input: nums = [3,2,4], target = 6  
Output: [1,2]
```

Example 3:

Input: nums = [3,3], target = 6
Output: [0,1]

Constraints:

- $2 \leq \text{nums.length} \leq 104$
- $-109 \leq \text{nums}[i] \leq 109$
- $-109 \leq \text{target} \leq 109$
- Only one valid answer exists.

Follow-up: Can you come up with an algorithm that is less than $O(n^2)$ time complexity?

My LeetCode Solution

• My LeetCode Solution

```
# Two Sum
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        hmap = {}

        for i, n in enumerate(nums):
            comp = target - n
            if comp in hmap:
                return [hmap[comp], i]
            hmap[n] = i
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# We're given an array of integers and a target value,
# and we need to return the indices of the two numbers
# that add up to the target – not the values, the indices. Right?"
#
# "A few quick questions:
# Can the same element be used twice? I'm guessing no.
# Is there always exactly one valid answer?"
```

```

# Can the array be empty or have only one element?
# Are there negative numbers or duplicates?"
#
# "Let me walk the example: nums = [2, 7, 11, 15], target = 9.
# 2 + 7 = 9, so we return [0, 1]. Got it."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# For every pair (i, j) where j > i, check if nums[i] + nums[j] equals target.
# Time: O(n2) from the nested loops. Space: O(1), no extra storage.
# Should I go ahead and optimize?"
class _1_TwoSum_Brute:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        for i in range(len(nums)):
            for j in range(i + 1, len(nums)):
                if nums[i] + nums[j] == target:
                    return [i, j]
        return []

# PHASE 3 — OPTIMIZE
# "The bottleneck is searching the rest of the array for a complement each time.
# I notice we're repeatedly asking: does the number I need exist?
# A hash map answers that in O(1).
#
# Walk forward through the array. For each number, compute complement = target -
# num.
# If complement is already in the map, return both indices.
# Otherwise, store the current number and index and keep going.
#
# Pseudocode:
# seen = {}
# for index, num in nums:
#     complement = target - num
#     if complement in seen: return [seen[complement], index]
#     seen[num] = index
#
# Time: O(n). Space: O(n). Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _1_TwoSum:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        seen = {} # maps value -> index
        for index, num in enumerate(nums):
            complement = target - num
            if complement in seen:
                return [seen[complement], index]
            seen[num] = index
        return []

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[2,7,11,15], target=9

index=0: complement=7, not in seen. seen={2:0}

index=1: complement=2, found at 0. return [0,1] ✓

#

nums=[3,2,4], target=6

index=0: complement=3, not in seen. seen={3:0}

index=1: complement=4, not in seen. seen={3:0,2:1}

index=2: complement=2, found at 1. return [1,2] ✓

#

nums=[3,3], target=6

index=0: complement=3, not in seen. seen={3:0}

index=1: complement=3, found at 0. return [0,1] ✓

#

"No bugs. Holds across all cases."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$ — one pass. Space $O(n)$ — worst case every element enters the map.

Trade-off: $O(1)$ space would require $O(n^2)$ time — no free lunch here.

If the array were sorted I'd use two pointers: $O(n)$ time, $O(1)$ space.

But we need original indices and the input is unsorted, so hash map wins.

Given more time I'd ask: what if there are multiple valid pairs?

We'd collect all of them rather than returning on the first hit."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def twoSum(self, nums: list[int], target: int) -> list[int]:
        numToIndex = {}

        for i, num in enumerate(nums):
            if target - num in numToIndex:
                return numToIndex[target - num], i
            numToIndex[num] = i

```

• LeetDoocs

```
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        d = {}
        for i, x in enumerate(nums):
            if (y := target - x) in d:
                return [d[y], i]
            d[x] = i
```

• SimplyLeet

```
def two_sum(nums, target):
    # Create a dictionary to store the number and its index
    num_to_index = {}
    # Iterate over the list with index
    for index, num in enumerate(nums):
        # Calculate the complement needed to reach the target
        complement = target - num
        # Check if the complement is in the dictionary
        if complement in num_to_index:
            # Return the index of the complement and the current index

            return [num_to_index[complement], index]
        # Store the number with its index
        num_to_index[num] = index
    # Since the problem guarantees one solution, we don't need a return
    # statement here.

# Example usage:
# print(two_sum([2, 7, 11, 15], 9)) # Output: [0, 1]
```

• LC.ca

```
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        m = {}
        for i, x in enumerate(nums):
            y = target - x
            if y in m:
                return [m[y], i]
            m[x] = i
```

◆ Quick Review

Key Insights

- Use a hash table to store the elements and their indices for fast lookup.
- For each element, calculate its complement: target minus the current element.
- Check if the complement exists in the hash table.
- If found, return the indices of the current element and its complement.
- This approach only requires a single pass through the array.

Space & Time

Time Complexity: $O(n)$ – We traverse the list only once.

Space Complexity: $O(n)$ – In the worst-case, we store all elements in the hash table.

Solution

We use a hash table (dictionary) to map each array element to its index as we iterate. For each number, we compute the complement (target – number) and check if it already exists in our hash table. If the complement is found,

we return the indices. This one-pass solution ensures linear time complexity and avoids the need for nested loops.

Arrays & Hashing

□ ★ #163 Missing Ranges ★

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array

Lists: ★ Premium | Premium 98

Problem

You are given an inclusive range [lower, upper] and a sorted unique integer array nums, where all elements are within the inclusive range.

A number x is considered missing if x is in the range [lower, upper] and x is not in nums.

Return the shortest sorted list of ranges that exactly covers all the missing numbers. That is, no element of nums is included in any of the ranges, and each missing number is covered by one of the ranges.

Example 1:

Input: nums = [0,1,3,50,75], lower = 0, upper = 99

Output: [[2,2],[4,49],[51,74],[76,99]]

Explanation: The ranges are:

[2,2]

[4,49]

[51,74]

[76,99]

Example 2:

Input: nums = [-1], lower = -1, upper = -1

Output: []

Explanation: There are no missing ranges since there are no missing numbers.

Constraints:

- $-109 \leq \text{lower} \leq \text{upper} \leq 109$
- $0 \leq \text{nums.length} \leq 100$
- $\text{lower} \leq \text{nums}[i] \leq \text{upper}$
- All the values of nums are unique.

My LeetCode Solution

• My LeetCode Solution

```
# Missing Ranges
class Solution:
    def findMissingRanges(self, nums: List[int], lower: int, upper: int) -> List[List[int]]:
        ans = []
        if not nums:
            return [[lower, upper]]

        f = nums[0]
        l = nums[-1]
```

```

    #lower f,l, upper

    if f > lower:
        ans.append([lower, f-1])

    for p,c in pairwise(nums):
        if p+1 < c:
            ans.append([p+1, c-1])

    if l < upper:

        ans.append([l+1, upper])

    return ans

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Let me make sure I understand the problem correctly.
# We have a sorted array of unique integers, and an inclusive range [lower, upper].
# We need to return the list of ranges that cover every number
# inside [lower, upper] that does NOT appear in nums. Right?"
#
# "A few quick questions:
# Can nums be empty? If so I assume the whole [lower, upper] is one missing range.
# Are all elements in nums guaranteed to be within [lower, upper]? Good.
# The output ranges should be as compact as possible – so [2,2] not just [2]?"
#
# "Let me walk the example:
# nums=[0,1,3,50,75], lower=0, upper=99.
# 0 and 1 are present so no gap at the start.
# 3 is present but 2 is missing → [2,2].
# 50 is present but 4–49 are missing → [4,49].
# 75 is present but 51–74 are missing → [51,74].
# 76–99 are all missing → [76,99].
# Output: [[2,2],[4,49],[51,74],[76,99]]. Got it."

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with brute force to lock in the logic.
# The simplest approach: build a set from nums for O(1) lookups,
# then walk every integer from lower to upper.
# Whenever I hit a missing number, I extend the current gap range.
# When I hit a number that IS in the set, I close the gap and record it.
#
# Time: O(upper – lower) which can be up to 2 billion – way too slow.

```

```

# Space: O(n) for the set.
# Should I optimize?"

class _163_MissingRanges_Brute:
    def findMissingRanges(self, nums: List[int], lower: int, upper: int) ->
List[List[int]]:
    present = set(nums)
    result = []
    gap_start = None
    for x in range(lower, upper + 1):
        if x not in present:
            if gap_start is None:
                gap_start = x
            else:
                if gap_start is not None:
                    result.append([gap_start, x - 1])
                    gap_start = None
    if gap_start is not None:
        result.append([gap_start, upper])
    return result

# PHASE 3 — OPTIMIZE
# "The brute force iterates over the entire numerical range – up to 2 billion steps.
# But nums has at most 100 elements. The gaps I care about
# only exist between consecutive numbers in nums, before the first, and after the
# last.
# So I only need to check those boundaries – that's O(n), not O(range).
#
# I think of nums as a set of walls. The missing ranges are the gaps between walls.
# There are at most n+1 gaps: one before nums[0], one between each pair, one after
# nums[-1].
#
# Pseudocode:
# if nums is empty: return [[lower, upper]]
# if lower < nums[0]: add [lower, nums[0]-1]
# for each consecutive pair (prev, curr) in nums:
# if prev+1 < curr: add [prev+1, curr-1]
# if nums[-1] < upper: add [nums[-1]+1, upper]
#
# Time: O(n). Space: O(1) extra. Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach.
# I'll use pairwise from itertools to cleanly iterate consecutive pairs."
class _163_MissingRanges:
    def findMissingRanges(self, nums: List[int], lower: int, upper: int) ->
List[List[int]]:
    if not nums:
        return [[lower, upper]]
    result = []

```

```

first, last = nums[0], nums[-1]
if lower < first:
    result.append([lower, first - 1])
for prev, curr in pairwise(nums):
    if prev + 1 < curr:
        result.append([prev + 1, curr - 1])
if last < upper:
    result.append([last + 1, upper])
return result

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

```

#
# nums=[0,1,3,50,75], lower=0, upper=99
# nums not empty. first=0, last=75.
# lower(0) < first(0)? No. No leading gap.
# pairwise: (0,1) → 1 < 1? No. (1,3) → 2 < 3? Yes → [2,2]. (3,50) → 4 < 50? Yes →
[4,49]. (50,75) → 51 < 75? Yes → [51,74].
# last(75) < upper(99)? Yes → [76,99].
# result = [[2,2],[4,49],[51,74],[76,99]] ✓
#
# nums=[-1], lower=-1, upper=-1
# first=-1, last=-1.
# lower(-1) < first(-1)? No.
# pairwise: empty (only one element).
# last(-1) < upper(-1)? No.
# result = [] ✓
#
# nums=[], lower=1, upper=5
# Empty → return [[1,5]] ✓
#
# "No bugs. All cases pass."

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n) – one pass through nums plus constant work per gap.
# Space O(1) extra – result list aside, no auxiliary storage.
# This is a massive improvement over O(upper – lower) brute force
# given the constraint that nums has at most 100 elements.
#
# Trade-off: if nums were not sorted I'd need to sort first – O(n log n).
# The problem guarantees sorted input so we skip that.
#
# pairwise is Python 3.10+. If the interviewer is on an older version
# I'd write: for i in range(1, len(nums)): prev, curr = nums[i-1], nums[i].
#
# Given more time I'd ask: what if the output should be strings like '2->2'
# instead of lists? Just a formatting change at the append step."

```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findMissingRanges(
        self,
        nums: list[int],
        lower: int,
        upper: int,
    ) -> list[list[int]]:
        def getRange(lo: int, hi: int) -> list[int]:
            if lo == hi:
                return [lo, lo]

            return [lo, hi]

        if not nums:
            return [getRange(lower, upper)]

        ans = []

        if nums[0] > lower:
            ans.append(getRange(lower, nums[0] - 1))

        for prev, curr in zip(nums, nums[1:]):
            if curr > prev + 1:
                ans.append(getRange(prev + 1, curr - 1))

        if nums[-1] < upper:
            ans.append(getRange(nums[-1] + 1, upper))

        return ans
```

• LeetDoocs

```

class Solution:
    def findMissingRanges(
        self, nums: List[int], lower: int, upper: int
    ) -> List[List[int]]:
        n = len(nums)
        if n == 0:
            return [[lower, upper]]
        ans = []
        if nums[0] > lower:
            ans.append([lower, nums[0] - 1])

        for a, b in pairwise(nums):
            if b - a > 1:
                ans.append([a + 1, b - 1])
        if nums[-1] < upper:
            ans.append([nums[-1] + 1, upper])
        return ans

```

• SimplyLeet

```

# Python solution for Missing Ranges problem

def findMissingRanges(nums, lower, upper):
    missing_ranges = [] # List to store the missing range intervals
    prev = lower - 1 # Initialize previous value to one less than lower

    # Append a sentinel at the end for easier gap handling
    nums.append(upper + 1)

    # Loop through each number, including the sentinel

    for num in nums:
        # If there is a gap between previous and current number
        if num - prev >= 2:
            # If gap is of single number, add that number as a range
            if num - prev == 2:
                missing_ranges.append([prev + 1, prev + 1])
            else:
                missing_ranges.append([prev + 1, num - 1])
        prev = num # Update previous to current
    return missing_ranges

```



```
# Example usage:  
print(findMissingRanges([0,1,3,50,75], 0, 99))
```

• LC.ca

```
class Solution:  
    def findMissingRanges(  
        self, nums: List[int], lower: int, upper: int  
    ) -> List[List[int]]:  
        n = len(nums)  
        if n == 0:  
            return [[lower, upper]]  
        ans = []  
        if nums[0] > lower:  
            ans.append([lower, nums[0] - 1])  
  
        for a, b in pairwise(nums):  
            if b - a > 1:  
                ans.append([a + 1, b - 1])  
        if nums[-1] < upper:  
            ans.append([nums[-1] + 1, upper])  
        return ans
```

◆ Quick Review

Key Insights

- Use a pointer (or iterate index) to traverse the nums array while keeping track of the previous number considered.
- Check for missing numbers before the first element, between consecutive elements, and after the last element.

- When a gap is found, convert it to a range string that is either a single number (if the gap is exactly one number) or an interval.
- Handle edge cases when the nums array is empty.

Space & Time

Time Complexity: $O(n)$, where n is the length of the nums array, since we iterate through the array once.

Space Complexity: $O(1)$ additional space (ignoring the output list), as we only use a few extra variables.

Solution

The solution leverages a single pass through the input array and calculates the missing ranges by comparing the expected number with the current number in nums. First, initialize a variable "prev" to one less than the lower bound so that missing numbers from the very start of the range can be handled uniformly. For each number in the array, if there is a gap between prev and the current number (i.e., current number is at least 2 more than prev), then add the missing range

(prev+1 to current number-1) to the output list. Update "prev" to the current number and finally handle any gap between the last number and the upper bound. The primary data structures used are the list for the output; the approach is iterative.

Arrays & Hashing

□ ★ #346 Moving Average from Data Stream ★

EAS[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Design · Queue · Data Stream
Lists: ★ Premium | Premium 98

Problem

Given a stream of integers and a window size, calculate the moving average of all integers in the sliding window.

Implement the MovingAverage class:

- **MovingAverage(int size)** Initializes the object with the size of the window size.
- **double next(int val)** Returns the moving average of the last size values of the stream.

Example 1:

```
Input
["MovingAverage", "next", "next", "next", "next"]
[[3], [1], [10], [3], [5]]
Output
[null, 1.0, 5.5, 4.66667, 6.0]

Explanation
MovingAverage movingAverage = new MovingAverage(3);
```

Constraints:

- $1 \leq \text{size} \leq 1000$
- $-105 \leq \text{val} \leq 105$
- At most 104 calls will be made to next.

My LeetCode Solution

• My LeetCode Solution

```
# Moving Average from Data Stream
```

```
class MovingAverage:
```

```
    def __init__(self, size: int):
        self.q = deque()
        self.size = size
        self.sum = 0
```

```
    def next(self, val: int) -> float:
        if len(self.q) == self.size:
```

```
            self.sum -= self.q.popleft()
            self.q.append(val)
            self.sum += val
        return self.sum / len(self.q)
```

```
# Your MovingAverage object will be instantiated and called as such:
```

```
# obj = MovingAverage(size)
```

```
# param_1 = obj.next(val)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

We're designing a class that receives a stream of integers one at a time.

Each call to next() should return the average of the last 'size' values seen so far.

If fewer than 'size' values have arrived, we average however many we have. Right?"

```

#
# "A few quick questions:
# Can size be 1? Constraints say yes – that's just returning the latest value.
# Can values be negative? Yes, constraints say down to  $-10^5$ .
# Is there a limit on how many next() calls we get? Up to  $10^4$  – good to know for
space."
#
# "Let me walk the example:
# size=3. next(1)=1.0 (only 1 value). next(10)=5.5 (avg of 1,10).
# next(3)=4.667 (avg of 1,10,3 – window full now).
# next(5)=6.0 (avg of 10,3,5 – 1 falls out). Got it."

# PHASE 2 — BRUTE FORCE
# "The simplest approach: store every value in a plain list.
# On each next() call, slice the last 'size' elements and compute the average.
#
# Time:  $O(\text{size})$  per next() call due to the slice and sum.
# Space:  $O(n)$  where n is total calls – the list grows forever.
# Should I optimize?"

class _346_MovingAverage_Brute:
    def __init__(self, size: int):
        self.size = size
        self.stream = []

    def next(self, val: int) -> float:
        self.stream.append(val)
        window = self.stream[-self.size:]
        return sum(window) / len(window)

# PHASE 3 — OPTIMIZE
# "Two inefficiencies in the brute force:
# First, we recompute the sum from scratch every call – that's  $O(\text{size})$  work.
# Second, we store the entire history but only ever need the last 'size' values.
#
# I notice we're recomputing the same sum repeatedly. What if we cache it?
# I'll keep a running sum and update it incrementally:
# when a new value comes in, add it. When the window is full,
# subtract the oldest value before adding the new one.
#
# For the window itself, a deque is perfect –  $O(1)$  append on the right,
#  $O(1)$  popleft when the oldest value falls out. The deque never exceeds 'size'
elements.
#
# Pseudocode:
# __init__: q = deque(), running_sum = 0, self.size = size
# next(val):
# if len(q) == size: running_sum -= q.popleft()
# running_sum += val
# q.append(val)
# return running_sum / len(q)
#

```

```
# Time: O(1) per next() call. Space: O(size) – the deque is capped.
# Does that make sense before I write it out?"
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _346_MovingAverage:
    def __init__(self, size: int):
        self.size = size
        self.window = deque()
        self.running_sum = 0

    def next(self, val: int) -> float:
        if len(self.window) == self.size:
            self.running_sum -= self.window.popleft() # evict oldest
        self.running_sum += val
        self.window.append(val)
        return self.running_sum / len(self.window)
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through the example: size=3."
```

```
#
# next(1): window not full. sum=1, window=[1]. return 1/1 = 1.0 ✓
# next(10): window not full. sum=11, window=[1,10]. return 11/2 = 5.5 ✓
# next(3): window not full. sum=14, window=[1,10,3]. return 14/3 = 4.667 ✓
# next(5): window full → popleft() removes 1, sum=14-1=13.
# sum=13+5=18, window=[10,3,5]. return 18/3 = 6.0 ✓
#
# Edge – size=1:
# next(7): window full immediately after first call? No – starts empty.
# sum=7, window=[7]. return 7/1 = 7.0 ✓
# next(4): window full (size=1). popleft removes 7, sum=0+4=4, window=[4].
# return 4/1 = 4.0 ✓
#
# "No bugs. Running sum stays consistent across evictions."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(1) per next() call – all operations are constant: deque append,
# popleft, addition, division.
# Space O(size) – the deque holds at most 'size' elements regardless of
# how many total calls are made. Much better than the O(n) brute force.
#
# Trade-off worth mentioning: floating point precision. If values are large
# and size is large, the running sum can accumulate rounding errors over time.
# A production system might use Python's Decimal or periodically recompute
# the sum from scratch to correct drift.
#
# Alternative: a circular buffer (fixed-size list with a pointer) does the same
# thing with slightly better cache locality than a deque, but the deque is
# cleaner to read and interview-safe.
#
# Given more time I'd ask: what if we need to support a variable window size
```


that can change between calls? We'd need to resize the deque dynamically."

Community Solutions (Python)

• WalkCC

```
class MovingAverage:
    def __init__(self, size: int):
        self.size = size
        self.sum = 0
        self.q = collections.deque()

    def next(self, val: int) -> float:
        if len(self.q) == self.size:
            self.sum -= self.q.popleft()
        self.sum += val

        self.q.append(val)
        return self.sum / len(self.q)
```

• LeetDoocs

```
class MovingAverage:

    def __init__(self, size: int):
        self.s = 0
        self.data = [0] * size
        self.cnt = 0

    def next(self, val: int) -> float:
        i = self.cnt % len(self.data)
        self.s += val - self.data[i]

        self.data[i] = val
        self.cnt += 1
        return self.s / min(self.cnt, len(self.data))

# Your MovingAverage object will be instantiated and called as such:
# obj = MovingAverage(size)
# param_1 = obj.next(val)
```

```
class MovingAverage:
    def __init__(self, size: int):
        self.n = size
        self.s = 0
        self.q = deque()

    def next(self, val: int) -> float:
        if len(self.q) == self.n:
            self.s -= self.q.popleft()
        self.q.append(val)
```

```
        self.s += val
        return self.s / len(self.q)
```

```
# Your MovingAverage object will be instantiated and called as such:
# obj = MovingAverage(size)
# param_1 = obj.next(val)
```

• SimplyLeet

```
# Python solution for Moving Average from Data Stream
```

```
from collections import deque # Import deque for efficient queue operations
```

```
class MovingAverage:
```

```
    def __init__(self, size):
```

```
        # Initialize with the maximum window size
```

```
        self.size = size # Maximum number of elements in the
window
```

```
        self.queue = deque() # Queue to store current window values
```

```
        self.current_sum = 0 # Running sum of the current window
```

```

def next(self, val):
    # Add new value to the window
    if len(self.queue) == self.size:
        # If the window is full, remove the oldest element and update the
sum
        removed_val = self.queue.popleft() # Remove the front element
        self.current_sum -= removed_val    # Subtract its value from the
running sum
    # Append the new value to the queue and update the sum
    self.queue.append(val)
    self.current_sum += val

    # Calculate and return the moving average by dividing the running sum
by the current window size
    return self.current_sum / len(self.queue)

```

• LC.ca

```

class MovingAverage:
    def __init__(self, size: int):
        self.arr = [0] * size
        self.s = 0
        self.cnt = 0

    def next(self, val: int) -> float:
        idx = self.cnt % len(self.arr) # circular array
        self.s += val - self.arr[idx]
        self.arr[idx] = val

        self.cnt += 1
        return self.s / min(self.cnt, len(self.arr))

# Your MovingAverage object will be instantiated and called as such:
# obj = MovingAverage(size)
# param_1 = obj.next(val)

#####

```

```

from collections import deque

# with no sum variable, calculate on the fly
class MovingAverage:
    def __init__(self, size: int):
        self.queue = deque()
        self.size = size
        self.avg = 0

    def next(self, val: int) -> float:

        if len(self.queue) < self.size:
            self.queue.append(val)
            self.avg = sum(self.queue) / len(self.queue)
            return self.avg
        else:
            head = self.queue.popleft()
            self.queue.append(val)
            minus = head / self.size
            add = val / self.size
            self.avg = self.avg + add - minus

        return self.avg

#####

from collections import deque

class MovingAverage(object):

    def __init__(self, size):
        """
        Initialize your data structure here.
        :type size: int
        """
        self.windowSize = size
        self.windowSum = 0.0
        self.data = deque([])

    def next(self, val):
        """
        :type val: int

```

```
:rtype: float
"""

self.windowSum += val
data = self.data

leftTop = 0
if len(data) >= self.windowSize:
    leftTop = data.popleft()
    data.append(val)
```

```
self.windowSum -= leftTop
if len(data) < self.windowSize:
    return self.windowSum / len(data)
return self.windowSum / self.windowSize
```

```
# Your MovingAverage object will be instantiated and called as such:
# obj = MovingAverage(size)
# param_1 = obj.next(val)
```

◆ Quick Review

Key Insights

- Use a queue (or circular buffer) to keep track of the current window elements.
- Maintain a running sum to quickly update the window's total when a new element comes in.
- When the window is full, subtract the value of the element that falls out as a new one is added.
- Each call to `next()` is executed in constant time, $O(1)$.

Space & Time

Time Complexity: $O(1)$ per call to next()

Space Complexity: $O(\text{window size})$

Solution

The solution leverages a queue data structure to store the elements of the sliding window. A running sum variable is maintained for efficient calculation of the moving average. When a new integer is added: If the size of the queue is equal to the maximum window size, remove the oldest element and subtract its value from the running sum. Add the new element to both the queue and the running sum. Compute and return the moving average as the running sum divided by the number of elements in the queue. This approach ensures that each update is done in constant time without having to sum all elements in the window repeatedly.

Arrays & Hashing

□ ★ #760 Find Anagram Mappings ★

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table

Lists: ★ Premium | Premium 98

Problem

You are given two integer arrays `nums1` and `nums2` where `nums2` is an anagram of `nums1`. Both arrays may contain duplicates.

Return an index mapping array mapping from `nums1` to `nums2` where `mapping[i] = j` means the *i*th element in `nums1` appears in `nums2` at index *j*. If there are multiple answers, return any of them.

An array *a* is an anagram of an array *b* means *b* is made by randomizing the order of the elements in *a*.

Example 1:

Input: nums1 = [12,28,46,32,50], nums2 = [50,12,32,46,28]

Output: [1,4,3,2,0]

Explanation: As mapping[0] = 1 because the 0th element of nums1 appears at nums2[1], and mapping[1] = 4 because the 1st element of nums1 appears at nums2[4], and so on.

Example 2:

Input: nums1 = [84,46], nums2 = [84,46]

Output: [0,1]

Constraints:

- $1 \leq \text{nums1.length} \leq 100$
- $\text{nums2.length} = \text{nums1.length}$
- $0 \leq \text{nums1}[i], \text{nums2}[i] \leq 105$
- nums2 is an anagram of nums1.

My LeetCode Solution

• My LeetCode Solution

```
# Find Anagram Mappings
class Solution:
    def anagramMappings(self, nums1: List[int], nums2: List[int]) -> List[int]:
        hmap = {n:i for i, n in enumerate(nums2)}
        return list(hmap[n] for n in nums1)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

We have two arrays nums1 and nums2. nums2 is an anagram of nums1 –

same elements, different order. We need to return a mapping array

where mapping[i] is the index in nums2 where nums1[i] appears. Right?"

#

"A few quick questions:

Can there be duplicate values? The problem says yes.

If a value appears multiple times in nums2, which index do we return?

```
# The problem says any valid answer is accepted – good, that simplifies things.
# Both arrays are the same length and nums2 is guaranteed to be an anagram,
# so every element in nums1 is guaranteed to exist in nums2?"
#
```

```
# "Let me walk the example:
```

```
# nums1=[12,28,46,32,50], nums2=[50,12,32,46,28].
# 12 is at index 1 in nums2 → mapping[0]=1.
# 28 is at index 4 → mapping[1]=4.
# 46 is at index 3 → mapping[2]=3.
# 32 is at index 2 → mapping[3]=2.
# 50 is at index 0 → mapping[4]=0.
# Output: [1,4,3,2,0]. Got it."
```

```
# PHASE 2 — BRUTE FORCE
```

```
# "Let me start with brute force to lock in the logic.
```

```
# For each element in nums1, scan nums2 left to right until I find a match
# and record that index.
#
```

```
# Time:  $O(n^2)$  – for each of  $n$  elements in nums1 we do a linear scan of nums2.
# Space:  $O(1)$  extra beyond the output array.
# Should I optimize?"
```

```
class _760_AnagramMappings_Brute:
```

```
    def anagramMappings(self, nums1: List[int], nums2: List[int]) -> List[int]:
        mapping = []
        for num in nums1:
            for j, val in enumerate(nums2):
                if val == num:
                    mapping.append(j)
                    break
        return mapping
```

```
# PHASE 3 — OPTIMIZE
```

```
# "The bottleneck is scanning nums2 from scratch for every element in nums1.
```

```
# We're asking the same question repeatedly: where does this value live in nums2?
# A hash map lets us answer that in  $O(1)$ .
#
```

```
# I'll preprocess nums2 once: build a map from value → index.
# Then for each element in nums1, look it up directly.
#
```

```
# One thing to flag: if there are duplicates, building the map will overwrite
# earlier indices with later ones. Since the problem allows any valid answer,
# that's fine – we just return whichever index was stored last.
#
```

```
# Pseudocode:
```

```
# index_in_nums2 = {val: i for i, val in enumerate(nums2)}
# return [index_in_nums2[num] for num in nums1]
#
```

```
# Time:  $O(n)$ . Space:  $O(n)$  for the hash map.
# Does that make sense before I code it?"
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

class _760_AnagramMappings:

```
def anagramMappings(self, nums1: List[int], nums2: List[int]) -> List[int]:
    index_in_nums2 = {val: i for i, val in enumerate(nums2)}
    return [index_in_nums2[num] for num in nums1]
```

PHASE 5 — TEST & DEBUG

"Let me trace through both examples."

#

nums1=[12,28,46,32,50], nums2=[50,12,32,46,28]

index_in_nums2 = {50:0, 12:1, 32:2, 46:3, 28:4}

nums1[0]=12 → 1

nums1[1]=28 → 4

nums1[2]=46 → 3

nums1[3]=32 → 2

nums1[4]=50 → 0

Output: [1,4,3,2,0] ✓

#

nums1=[84,46], nums2=[84,46]

index_in_nums2 = {84:0, 46:1}

Output: [0,1] ✓

#

Duplicate case – nums1=[3,3], nums2=[3,3]

enumerate(nums2): (0,3) → map={3:0}, then (1,3) → map={3:1} (overwritten).

nums1[0]=3 → 1, nums1[1]=3 → 1. Output: [1,1].

Both point to the same index – valid since any answer is accepted ✓

#

"No bugs. Duplicate overwrite is intentional and within spec."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$ – one pass to build the map, one pass to build the result.

Space $O(n)$ for the hash map.

#

The duplicate handling is worth calling out explicitly:

by iterating enumerate(nums2) forward, later indices overwrite earlier ones.

If the interviewer wanted ALL valid indices for duplicates returned,

I'd change the map to store a list of indices per value and pop one per lookup.

#

Trade-off: if n is tiny (say n=5 like in the example), the brute force $O(n^2)$

is barely slower and uses no extra space. The hash map pays off as n grows.

#

Given more time I'd ask: what if we need to return the mapping in sorted order

by nums2 index? We'd sort the output – $O(n \log n)$ – but that changes the spec."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def anagramMappings(self, nums1: list[int], nums2: list[int]) -> list[int]:
        numToIndices = collections.defaultdict(list)

        for i, num in enumerate(nums2):
            numToIndices[num].append(i)

        return [numToIndices[num].pop() for num in nums1]
```

• LeetDoocs

```
class Solution:
    def anagramMappings(self, nums1: List[int], nums2: List[int]) -> List[int]:
        :
        d = {x: i for i, x in enumerate(nums2)}
        return [d[x] for x in nums1]
```

• SimplyLeet

```
# Build a dictionary mapping each number in nums2 to a list of its indices.
def anagramMappings(nums1, nums2):
    # Dictionary to store number : list of indices
    index_dict = {}
    for index, num in enumerate(nums2):
        if num in index_dict:
            index_dict[num].append(index)
        else:
            index_dict[num] = [index]
```

```
mapping = []
# Generate the mapping for each element in nums1.
for num in nums1:
    # Pop the first available index for num and add it to mapping.
    mapping.append(index_dict[num].pop(0))
return mapping
```

```
# Example usage:
nums1 = [12,28,46,32,50]
nums2 = [50,12,32,46,28]
```

```
print(anagramMappings(nums1, nums2)) # Output: [1,4,3,2,0] or any valid mapping
```

• LC.ca

```
class Solution:
    def anagramMappings(self, nums1: List[int], nums2: List[int]) -> List[int]:
        mapper = defaultdict(set)
        for i, num in enumerate(nums2):
            mapper[num].add(i)
        return [mapper[num].pop() for num in nums1]
```

◆ Quick Review

Key Insights

- Since `nums2` is an anagram of `nums1`, every element in `nums1` has a corresponding position in `nums2`.
- A dictionary (or hash table) can be used to map each number in `nums2` to its indices.
- When iterating through `nums1`, we can assign the next available index from the dictionary for that number.

Space & Time

Time Complexity: $O(n)$ where n is the number of elements in the arrays (single pass to build the dictionary and another pass to form the mapping).

Space Complexity: $O(n)$ for storing the dictionary that holds indices of elements in `nums2`.

Solution

The approach consists of two primary steps: Build a dictionary to record all indices where each number appears in `nums2`. This dictionary maps a number to a list of indices. Iterate over `nums1` and for each element, remove (or pop) one index from the corresponding list in the dictionary, and add that index to the final mapping array. This guarantees that each number from `nums1` is correctly mapped to one of its positions in `nums2` while efficiently handling duplicates.

Arrays & Hashing

★ #1426 Counting Elements ★

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table

Lists: ★ Premium | Premium 98

Problem

Given an integer array `arr`, count how many elements `x` there are, such that `x + 1` is also in `arr`. If there are duplicates in `arr`, count them separately.

Example 1:

Input: `arr = [1,2,3]`
Output: 2
Explanation: 1 and 2 are counted cause 2 and 3 are in `arr`.

Example 2:

Input: `arr = [1,1,3,3,5,5,7,7]`
Output: 0
Explanation: No numbers are counted, cause there is no 2, 4, 6, or 8 in `arr`.

Constraints:

- $1 \leq \text{arr.length} \leq 1000$
- $0 \leq \text{arr}[i] \leq 1000$

My LeetCode Solution

• My LeetCode Solution

```
# Counting Elements
class Solution:
    def countElements(self, arr: List[int]) -> int:
        hmap = Counter(arr)
        ans = 0

        for n in hmap:
            if n+1 in hmap:
                ans += hmap[n]

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

For each element x in the array, we check whether x+1 also exists in the array.

If it does, we count x. And if x appears multiple times, we count each occurrence separately – so duplicates all contribute individually. Right?"

#

"A few quick questions:

Can the array be empty? Constraints say length >= 1, so no.

Can values be 0? Yes – so x=0 counts if 1 is present.

Can values be negative? Constraints say 0 <= arr[i] <= 1000, so no.

Are there duplicates? Yes, and the problem explicitly says to count them separately."

#

"Let me walk the examples:

arr=[1,2,3]: 1+1=2 is in arr → count 1. 2+1=3 is in arr → count 2. 3+1=4 not in arr.

Output: 2 ✓

arr=[1,1,3,3,5,5,7,7]: 1+1=2 not in arr. 3+1=4 not in arr. Same for 5,7.

Output: 0 ✓ Got it."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For each element x in arr, scan the entire array to see if x+1 exists.

If it does, increment the count.

#

Time: O(n²) – nested scan for every element.

```

# Space: O(1) – no extra storage.
# Should I optimize?"
class _1426_CountingElements_Brute:
    def countElements(self, arr: List[int]) -> int:
        count = 0
        for x in arr:
            for y in arr:
                if y == x + 1:
                    count += 1
                    break
        return count

# PHASE 3 — OPTIMIZE
# "The bottleneck is the inner scan – we're asking 'does x+1 exist?' n times,
# each taking O(n). A hash set answers that in O(1).
#
# But there's a subtlety: duplicates. If arr=[1,1,2], both 1s should be counted.
# A plain set tells us x+1 exists but not how many times x appears.
# So I'll use a Counter instead – it gives me O(1) existence checks
# AND stores the frequency of each value.
#
# Walk over the unique keys in the counter. If key+1 is also in the counter,
# add counter[key] to the answer – that accounts for all duplicates of key at once.
#
# Pseudocode:
# freq = Counter(arr)
# for x in freq:
#     if x+1 in freq: ans += freq[x]
# return ans
#
# Time: O(n) to build the Counter, O(k) to iterate unique keys where k <= n.
# Overall O(n). Space: O(k) for the Counter. Does that make sense?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _1426_CountingElements:
    def countElements(self, arr: List[int]) -> int:
        freq = Counter(arr)
        count = 0
        for x in freq:
            if x + 1 in freq:
                count += freq[x]
        return count

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# arr=[1,2,3]
# freq={1:1, 2:1, 3:1}
# x=1: 2 in freq → count+=1. count=1.

```

```

# x=2: 3 in freq → count+=1. count=2.
# x=3: 4 not in freq.
# return 2 ✓
#
# arr=[1,1,3,3,5,5,7,7]
# freq={1:2, 3:2, 5:2, 7:2}
# x=1: 2 not in freq.
# x=3: 4 not in freq.
# x=5: 6 not in freq.
# x=7: 8 not in freq.
# return 0 ✓
#
# Duplicate counting – arr=[1,1,2]
# freq={1:2, 2:1}
# x=1: 2 in freq → count+=freq[1]=2. count=2.
# x=2: 3 not in freq.
# return 2 ✓ (both 1s counted, as the problem requires)
#
# "No bugs. Duplicate handling confirmed."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) – one pass to build the Counter, one pass over unique keys.
# Space O(k) where k is the number of distinct values, at most O(n).
#
# Alternative: use a plain set instead of a Counter, then iterate the original arr.
# present = set(arr)
# return sum(1 for x in arr if x+1 in present)
# Same O(n) time and O(n) space – slightly simpler but both are valid.
# The Counter approach avoids re-iterating arr which is a minor readability win.
#
# Trade-off: if the value range is small and fixed (like 0–1000 here),
# a boolean array of size 1001 works as a lookup in O(1) with better cache
# performance than a hash map, and O(1001) ~ O(1) space in practice.
#
# Given more time I'd ask: what if we needed to return the actual elements
# that qualify rather than a count? We'd collect them into a list instead."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def countElements(self, arr: list[int]) -> int:
        count = collections.Counter(arr)
        return sum(freq
                    for a, freq in count.items()
                    if count[a + 1] > 0)

```

• LeetCode

```
class Solution:
    def countElements(self, arr: List[int]) -> int:
        cnt = Counter(arr)
        return sum(v for x, v in cnt.items() if cnt[x + 1])
```

• SimplyLeet

```
# Function to count elements where (x + 1) is in the array
def count_elements(arr):
    # Create a set of unique numbers for O(1) lookups
    unique_numbers = set(arr)
    count = 0
    # Iterate through each number in the array
    for number in arr:
        # If (number + 1) exists in the unique set, increment count
        if number + 1 in unique_numbers:
            count += 1

    return count

# Example usage:
# arr = [1, 2, 3]
# print(count_elements(arr)) # Expected output: 2
```

• LC.ca

```
class Solution:
    def countElements(self, arr: List[int]) -> int:
        cnt = Counter(arr)
        return sum(v for x, v in cnt.items() if cnt[x + 1])
```

◆ Quick Review

Key Insights

- Use a hash table or set to store unique elements from arr for constant time lookups.

- Iterate through each element in `arr` and check if $x + 1$ exists in the set.
- Count each valid occurrence, including duplicates.
- This approach efficiently handles the problem without needing nested loops.

Space & Time

Time Complexity: $O(n)$, where n is the number of elements in `arr` (set lookups are $O(1)$ on average).

Space Complexity: $O(n)$, due to the use of an auxiliary set to store unique elements.

Solution

The solution leverages a set to achieve constant time membership checks. First, we convert `arr` to a set of unique numbers. Then, iterate through `arr`, and for each element, check if $(\text{element} + 1)$ exists in the set. If it does, increment the count. This approach correctly handles duplicates by counting each occurrence during iteration.

Arrays & Hashing

□ #1534 Count Good Triplets

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Enumeration
Lists: CodeSignal

Problem

Given an array of integers `arr`, and three integers `a`, `b` and `c`. You need to find the number of good triplets.

A triplet $(arr[i], arr[j], arr[k])$ is good if the following conditions are true:

- $0 \leq i < j < k < arr.length$
- $|arr[i] - arr[j]| \leq a$
- $|arr[j] - arr[k]| \leq b$
- $|arr[i] - arr[k]| \leq c$

Where $|x|$ denotes the absolute value of x .

Return the number of good triplets.

Example 1:

Input: arr = [3,0,1,1,9,7], a = 7, b = 2, c = 3

Output: 4

Explanation: There are 4 good triplets: [(3,0,1), (3,0,1), (3,1,1), (0,1,1)].

Example 2:

Input: arr = [1,1,2,2,3], a = 0, b = 0, c = 1

Output: 0

Explanation: No triplet satisfies all conditions.

Constraints:

- $3 \leq \text{arr.length} \leq 100$
- $0 \leq \text{arr}[i] \leq 1000$
- $0 \leq a, b, c \leq 1000$

My LeetCode Solution

• My LeetCode Solution

```
# Count Good Triplets
class Solution:
    def countGoodTriplets(self, arr: List[int], a: int, b: int, c: int) -> int:
        ans = 0
        n = len(arr)
        for i in range(n):
            for j in range(i+1, n):
                if abs(arr[i] - arr[j]) > a:
                    continue

                for k in range(j+1, n):
                    if abs(arr[j] - arr[k]) <= b and abs(arr[i] - arr[k]) <= c:
                        ans += 1

        return ans
```


◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# We need to count all index triplets (i, j, k) where i < j < k,
# and all three pairwise conditions hold simultaneously:
# the first pair must differ by at most a,
# the middle-to-last pair by at most b,
# and the first-to-last pair by at most c.
# We're counting triplets, not returning them. Right?"
#
# "A few quick questions:
# Can values in arr be 0? Yes, constraints allow it.
# Can a, b, c be 0? Yes – that means exact equality required for that pair.
# Array length is at most 100 – good to know for complexity decisions."
#
# "Let me walk the example:
# arr=[3,0,1,1,9,7], a=7, b=2, c=3.
# (3,0,1) at indices (0,1,2): |3-0|=3<=7, |0-1|=1<=2, |3-1|=2<=3 ✓
# (3,0,1) at indices (0,1,3): |3-0|=3<=7, |0-1|=1<=2, |3-1|=2<=3 ✓
# (3,1,1) at indices (0,2,3): |3-1|=2<=7, |1-1|=0<=2, |3-1|=2<=3 ✓
# (0,1,1) at indices (1,2,3): |0-1|=1<=7, |1-1|=0<=2, |0-1|=1<=3 ✓
# Output: 4. Got it."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Three nested loops over indices (i, j, k) with i < j < k.
# Count the triplet when arr[i] + arr[j] + arr[k] <= a + b + c.
# Correct, but O(n^3) is too slow when n is up to 100.
# Time: O(n^3). Space: O(1).
# Should I go ahead and optimize?"
```

```
class _1534_CountGoodTriplets_Brute:
    def countGoodTriplets(self, arr: List[int], a: int, b: int, c: int) -> int:
        count = 0
        n = len(arr)
        for i in range(n):
            for j in range(i + 1, n):
                for k in range(j + 1, n):
                    if (abs(arr[i] - arr[j]) <= a and
                        abs(arr[j] - arr[k]) <= b and
                        abs(arr[i] - arr[k]) <= c):
                        count += 1
        return count
```

PHASE 3 — OPTIMIZE

```
# "The algorithm stays O(n^3) – there's no known sub-cubic solution for this
# specific three-condition problem, and with n=100 we don't need one."
```

```
# But I can make two practical improvements:
#
# First: early pruning. The three conditions are checked in sequence.
# If |arr[i] - arr[j]| > a, there's no point checking any k for this (i,j) pair.
# We skip the entire inner k loop, reducing the constant factor significantly.
#
# Second: use itertools.combinations for cleaner, more readable code.
# It generates all (i,j,k) triples with i<j<k in one line.
# Note that combinations doesn't support early pruning on the outer pair,
# so for readability I'll use it here since n is small enough that it doesn't
matter.
#
# I'll go with the early-pruning nested loop to show I understand the trade-off,
# then mention the itertools alternative.
#
# Time:  $O(n^3)$  worst case, faster in practice due to pruning.
# Space:  $O(1)$ ."
```

PHASE 4 — CLEAN CODE

"Now I'll implement with early pruning and meaningful names."

```
class _1534_CountGoodTriplets:
    def countGoodTriplets(self, arr: List[int], a: int, b: int, c: int) -> int:
        count = 0
        n = len(arr)
        for i in range(n):
            for j in range(i + 1, n):
                if abs(arr[i] - arr[j]) > a: # prune: skip all k for this pair
                    continue
                for k in range(j + 1, n):
                    if (abs(arr[j] - arr[k]) <= b and
                        abs(arr[i] - arr[k]) <= c):
                        count += 1
        return count
```

itertools one-liner alternative (interview-clean, no pruning):

```
class _1534_CountGoodTriplets_Itertools:
    def countGoodTriplets(self, arr: List[int], a: int, b: int, c: int) -> int:
        return sum(
            abs(arr[i] - arr[j]) <= a and
            abs(arr[j] - arr[k]) <= b and
            abs(arr[i] - arr[k]) <= c
            for i, j, k in itertools.combinations(range(len(arr)), 3)
        )
```

PHASE 5 — TEST & DEBUG

"Let me trace the pruning version on example 1."

```
#
# arr=[3,0,1,1,9,7], a=7, b=2, c=3
# i=0 (arr[i]=3):
# j=1 (arr[j]=0): |3-0|=3<=7 → not pruned.
# k=2: |0-1|=1<=2, |3-1|=2<=3 → count=1
```

```
# k=3: |0-1|=1<=2, |3-1|=2<=3 → count=2
# k=4: |0-9|=9>2 → skip
# k=5: |0-7|=7>2 → skip
# j=2 (arr[j]=1): |3-1|=2<=7 → not pruned.
# k=3: |1-1|=0<=2, |3-1|=2<=3 → count=3
# k=4: |1-9|=8>2 → skip
# k=5: |1-7|=6>2 → skip
# j=3 (arr[j]=1): |3-1|=2<=7 → not pruned.
# k=4: |1-9|=8>2 → skip
# k=5: |1-7|=6>2 → skip
# j=4 (arr[j]=9): |3-9|=6<=7 → not pruned.
# k=5: |9-7|=2<=2, |3-7|=4>3 → skip
# j=5: no k available.
# i=1 (arr[i]=0):
# j=2 (arr[j]=1): |0-1|=1<=7 → not pruned.
# k=3: |1-1|=0<=2, |0-1|=1<=3 → count=4
# k=4,5: fail b condition → skip
# ... remaining pairs yield 0 more.
# return 4 ✓
#
# arr=[1,1,2,2,3], a=0, b=0, c=1
# a=0 means |arr[i]-arr[j]| must be 0 → only equal pairs pass.
# j=1 (1==1): pass a. k=2: |1-2|=1>0 → fail b. k=3: same. k=4: same.
# j=3 (1==?): arr[1]=1, arr[3]=2 → |1-2|=1>0 → pruned.
# No triplet survives. return 0 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n^3)$  worst case when all pairs pass the first condition –
# but early pruning on condition 1 cuts the k loop entirely for bad (i,j) pairs,
# making it much faster in practice.
# Space  $O(1)$  – just a counter.
#
# With n=100 the worst case is 161,700 triplets – trivially fast.
# If n grew to  $10^4$ ,  $O(n^3)$  would be  $10^{12}$  – completely infeasible.
# At that scale I'd look at sorting + binary search to narrow candidates per (i,j),
# or segment trees for range queries – but that's overkill here.
#
# The itertools one-liner is the cleanest version for an interview
# when you want to show Python fluency without sacrificing readability.
# Trade-off: it can't prune early, so stick with the nested loop if the
# interviewer asks about optimization.
#
# Given more time I'd ask: what if the array is sorted?
# Sorted order lets us binary search for valid k values after fixing i and j,
# reducing the inner loop from  $O(n)$  to  $O(\log n)$  → overall  $O(n^2 \log n)$ ."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def countGoodTriplets(self, arr: list[int], a: int, b: int, c: int) -> int:
        return sum(abs(arr[i] - arr[j]) <= a and
                    abs(arr[j] - arr[k]) <= b and
                    abs(arr[i] - arr[k]) <= c
                    for i, j, k in itertools.combinations(range(len(arr)), 3))
```

• LeetDoocs

```
class Solution:
    def countGoodTriplets(self, arr: List[int], a: int, b: int, c: int) -> int:
        ans, n = 0, len(arr)
        for i in range(n):
            for j in range(i + 1, n):
                for k in range(j + 1, n):
                    ans += (
                        abs(arr[i] - arr[j]) <= a
                        and abs(arr[j] - arr[k]) <= b
                        and abs(arr[i] - arr[k]) <= c
                    )
        return ans
```

• SimplyLeet

```
# Python solution for Count Good Triplets
def countGoodTriplets(arr, a, b, c):
    # Initialize counter for good triplets
    good_triplets_count = 0
    n = len(arr)
    # Iterate over all triplet combinations with indices i, j, k where i < j < k
    for i in range(n):
        for j in range(i + 1, n):
            for k in range(j + 1, n):
                # Check if absolute differences satisfy the conditions
```

```

        if abs(arr[i] - arr[j]) <= a and abs(arr[j] - arr[k]) <= b and
abs(arr[i] - arr[k]) <= c:
            good_triplets_count += 1
    # Return the count of good triplets
    return good_triplets_count

# Example usage:
arr = [3, 0, 1, 1, 9, 7]
a, b, c = 7, 2, 3
print(countGoodTriplets(arr, a, b, c)) # Expected output: 4

```

• LC.ca

```

class Solution:
    def countGoodTriplets(self, arr: List[int], a: int, b: int, c: int) -> int
    :
        ans, n = 0, len(arr)
        for i in range(n):
            for j in range(i + 1, n):
                for k in range(j + 1, n):
                    ans += (
                        abs(arr[i] - arr[j]) <= a
                        and abs(arr[j] - arr[k]) <= b
                        and abs(arr[i] - arr[k]) <= c
                    )
        return ans

```

◆ Quick Review

Key Insights

- Brute force approach using three nested loops is acceptable since the maximum array length is 100.
- Check all possible triplets and count only those meeting the specified conditions.

- **Absolute difference comparisons are the key condition validations.**

Space & Time

Time Complexity: $O(n^3)$ where n is the length of the array.

Space Complexity: $O(1)$ extra space (ignoring input space).

Solution

We use a brute force algorithm with three nested loops, iterating over all combinations of triplets (i, j, k) ensuring $i < j < k$. For each triplet, we calculate the absolute differences: $|arr[i] - arr[j]|$ $|arr[j] - arr[k]|$ $|arr[i] - arr[k]|$. The triplet is counted as good if all absolute differences satisfy the conditions ($\leq a$, $\leq b$, and $\leq c$ respectively). This simple approach leverages the small input size, eliminating the need for more complex data structures or optimizations.

String

Round 1 · High · Easy · 3 questions

String

□ #383 Ransom Note

EAS[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Counting
Lists: Grind 169

Problem

Given two strings `ransomNote` and `magazine`, return `true` if `ransomNote` can be constructed by using the letters from `magazine` and `false` otherwise.

Each letter in `magazine` can only be used once in `ransomNote`.

Example 1:

```
Input: ransomNote = "a", magazine = "b"  
Output: false
```

Example 2:

```
Input: ransomNote = "aa", magazine = "ab"  
Output: false
```

Example 3:

```
Input: ransomNote = "aa", magazine = "aab"  
Output: true
```

Constraints:

- $1 \leq \text{ransomNote.length}, \text{magazine.length} \leq 105$
- ransomNote and magazine consist of lowercase English letters.

My LeetCode Solution

• My LeetCode Solution

```
# Ransom Note
class Solution:
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
        hmap = Counter(magazine)
        for c in ransomNote:
            hmap[c] -= 1
            if hmap[c] < 0:
                return False

        return True
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

We need to check if every character in ransomNote can be supplied

by magazine – one letter from magazine per letter in ransomNote.

So if ransomNote needs two 'a's, magazine must have at least two 'a's. Right?"

#

"A few quick questions:

Are both strings guaranteed to be non-empty? Yes, length >= 1.

Only lowercase English letters – so exactly 26 possible characters. Good.

We're not modifying the strings, just checking feasibility?"

#

"Let me walk the examples:

ransomNote='a', magazine='b': 'a' not in magazine → false ✓

ransomNote='aa', magazine='ab': need two 'a's, magazine has one → false ✓

ransomNote='aa', magazine='aab': need two 'a's, magazine has two → true ✓ Got it."

PHASE 2 — BRUTE FORCE

```

# "The simplest approach: convert magazine to a list so I can 'use up' letters.
# For each character in ransomNote, scan the list to find it and remove it.
# If I can't find it, return false.
#
# Time:  $O(n * m)$  – for each of  $n$  chars in ransomNote, scan up to  $m$  chars in
magazine.
# Space:  $O(m)$  for the mutable copy of magazine.
# Should I optimize?"
class _383_RansomNote_Brute:
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
        available = list(magazine)
        for char in ransomNote:
            if char not in available:
                return False
            available.remove(char)
        return True

# PHASE 3 — OPTIMIZE
# "The bottleneck is scanning magazine for each character – that's the  $n*m$ .
# The question we keep asking is: how many of this letter does magazine still have?
# A frequency counter answers that in  $O(1)$ .
#
# I'll count every character in magazine upfront with a Counter.
# Then walk ransomNote: for each character, decrement its count.
# If any count drops below zero, magazine doesn't have enough of that letter –
return false.
# If I finish ransomNote without going negative, return true.
#
# Pseudocode:
# freq = Counter(magazine)
# for char in ransomNote:
#     freq[char] -= 1
#     if freq[char] < 0: return False
# return True
#
# Time:  $O(n + m)$  – one pass each over magazine and ransomNote.
# Space:  $O(1)$  – the Counter holds at most 26 keys (lowercase letters only).
# Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _383_RansomNote:
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
        available = Counter(magazine)
        for char in ransomNote:
            available[char] -= 1
            if available[char] < 0:
                return False
        return True

```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through the three examples."
```

```
#
```

```
# ransomNote='a', magazine='b'
```

```
# available = {'b': 1}
```

```
# char='a': available['a']=0-1=-1 < 0 → return False ✓
```

```
# (Counter returns 0 for missing keys, so no KeyError)
```

```
#
```

```
# ransomNote='aa', magazine='ab'
```

```
# available = {'a':1, 'b':1}
```

```
# char='a': available['a']=0 → ok
```

```
# char='a': available['a']=-1 < 0 → return False ✓
```

```
#
```

```
# ransomNote='aa', magazine='aab'
```

```
# available = {'a':2, 'b':1}
```

```
# char='a': available['a']=1 → ok
```

```
# char='a': available['a']=0 → ok
```

```
# return True ✓
```

```
#
```

```
# Edge – ransomNote longer than magazine: ransomNote='abc', magazine='ab'
```

```
# available = {'a':1, 'b':1}
```

```
# char='a': 0 → ok. char='b': 0 → ok. char='c': -1 < 0 → return False ✓
```

```
#
```

```
# "No bugs. Counter's default of 0 for missing keys handles unseen characters cleanly."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n + m)$  –  $O(m)$  to build the Counter,  $O(n)$  to walk ransomNote.
```

```
# Space  $O(1)$  – the Counter has at most 26 entries since only lowercase letters exist.
```

```
# Even though we write  $O(k)$  where  $k$  is distinct chars,  $k$  is bounded by 26 – constant.
```

```
#
```

```
# Alternative: Counter subtraction in one line.
```

```
# return not (Counter(ransomNote) - Counter(magazine))
```

```
# Counter subtraction drops keys with zero or negative counts,
```

```
# so if anything remains, ransomNote needs letters magazine doesn't have.
```

```
# Elegant but less readable in an interview – I'd use the explicit loop.
```

```
#
```

```
# Another alternative: sort both strings and use two pointers –  $O(n \log n + m \log m)$ ,
```

```
# slower but uses  $O(1)$  space beyond the sort. Worth mentioning if interviewer
```

```
# constraints disallow hash maps.
```

```
#
```

```
# Given more time I'd ask: what if characters include uppercase or unicode?
```

```
# We'd replace the 26-slot assumption with a full hash map – space becomes
```

```
 $O(\text{alphabet size})$ ."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
        count1 = collections.Counter(ransomNote)
        count2 = collections.Counter(magazine)
        return all(count1[c] <= count2[c] for c in string.ascii_lowercase)
```

• LeetDoocs

```
class Solution:
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
        cnt = Counter(magazine)
        for c in ransomNote:
            cnt[c] -= 1
            if cnt[c] < 0:
                return False
        return True
```

• SimplyLeet

```
# Function to check if ransom note can be constructed from magazine
def canConstruct(ransomNote: str, magazine: str) -> bool:
    # Create a dictionary to hold the frequency of each character in magazine
    char_frequency = {}
    for char in magazine:
        char_frequency[char] = char_frequency.get(char, 0) + 1

    # Check each character in ransomNote against the frequency map
    for char in ransomNote:
        if char_frequency.get(char, 0) == 0:

            return False # Character not available or used up
        char_frequency[char] -= 1 # Use the character

    return True

# Example usage:
if __name__ == "__main__":
    print(canConstruct("aa", "aab")) # Expected output: True
```

• LC.ca

```
class Solution:
    def canConstruct(self, ransomNote: str, magazine: str) -> bool:
        cnt = Counter(magazine)
        for c in ransomNote:
            cnt[c] -= 1
            if cnt[c] < 0:
                return False
        return True
```

◆ Quick Review

Key Insights

- Use a hash table (or dictionary) to store the frequency of each character present in magazine.
- Iterate over every character in ransomNote and check if the corresponding count in the hash table is available and sufficient.
- Decrease the character count for every match; if any character count falls below zero, return false.
- If all characters in ransomNote can be accounted for, return true.

Space & Time

Time Complexity: $O(n + m)$, where n is the length of magazine and m is the length of ransomNote.

Space Complexity: $O(1)$, since the extra space required for the hash table is limited to at most 26 entries for lowercase English letters.

Solution

We construct a frequency map for the magazine string using a hash table. This frequency map records how many times each letter appears in magazine. Then we iterate through the ransomNote string, and for every letter, we check the corresponding frequency in the map. If a letter is not found or its frequency is zero, we immediately return false as it means the letter cannot be used to build the note. Otherwise, we decrement the frequency count by one. If we successfully process all characters in ransomNote, then it can be constructed from magazine, and we return true.

String

□ ★ #734 Sentence Similarity ★

EAS[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String
Lists: ★ Premium | Premium 98

Problem

We can represent a sentence as an array of words, for example, the sentence "I am happy with leetcode" can be represented as `arr = ["I","am","happy","with","leetcode"]`.

Given two sentences `sentence1` and `sentence2` each represented as a string array and given an array of string pairs `similarPairs` where `similarPairs[i] = [xi, yi]` indicates that the two words `xi` and `yi` are similar.

Return `true` if `sentence1` and `sentence2` are similar, or `false` if they are not similar.

Two sentences are similar if:

- They have the same length (i.e., the same number of words)

- `sentence1[i]` and `sentence2[i]` are similar.

Notice that a word is always similar to itself, also notice that the similarity relation is not transitive. For example, if the words `a` and `b` are similar, and the words `b` and `c` are similar, `a` and `c` are not necessarily similar.

Example 1:

```
Input: sentence1 = ["great","acting","skills"], sentence2 =  
["fine","drama","talent"], similarPairs =  
[["great","fine"],["drama","acting"],["skills","talent"]]  
Output: true  
Explanation: The two sentences have the same length and each word i of  
sentence1 is also similar to the corresponding word in sentence2.
```

Example 2:

```
Input: sentence1 = ["great"], sentence2 = ["great"], similarPairs = []  
Output: true  
Explanation: A word is similar to itself.
```

Example 3:

```
Input: sentence1 = ["great"], sentence2 = ["doubleplus","good"], similarPairs =  
[["great","doubleplus"]]  
Output: false  
Explanation: As they don't have the same length, we return false.
```

Constraints:

- $1 \leq \text{sentence1.length}, \text{sentence2.length} \leq 1000$
- $1 \leq \text{sentence1}[i].\text{length}, \text{sentence2}[i].\text{length} \leq 20$
- `sentence1[i]` and `sentence2[i]` consist of English letters.

- $0 \leq \text{similarPairs.length} \leq 1000$
- $\text{similarPairs}[i].\text{length} == 2$
- $1 \leq \text{xi.length}, \text{yi.length} \leq 20$
- xi and yi consist of lower-case and upper-case English letters.
- All the pairs (xi, yi) are distinct.

My LeetCode Solution

• My LeetCode Solution

```
# Sentence Similarity
class Solution:
    def areSentencesSimilar(self, sentence1: List[str], sentence2: List[str],
similarPairs: List[List[str]]) -> bool:
        m = len(sentence1)
        n = len(sentence2)
        if m != n:
            return False
        hmap = defaultdict(set)
        for x,y in similarPairs:
            hmap[x].add(y)

            hmap[y].add(x)

        for x,y in zip(sentence1, sentence2):
            if x!=y and y not in hmap[x]:
                return False

        return True
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

```

# Two sentences are similar if they're the same length and every word at position i
# in sentence1 is similar to the word at position i in sentence2.
# A word is always similar to itself.
# Similarity is NOT transitive – if a~b and b~c, that doesn't mean a~c.
# We're given the list of similar pairs explicitly, so we only trust those. Right?"
#
# "A few quick questions:
# Can similarPairs be empty? Yes – then only identical words are similar.
# Are pairs directional? The problem implies symmetric – if [a,b] is given,
# then a~b AND b~a. I'll confirm: yes, similarity is symmetric.
# Can the same pair appear twice? Constraints say all pairs are distinct – good."
#
# "Let me walk example 1:
# sentence1=['great','acting','skills'], sentence2=['fine','drama','talent'].
# Same length: 3. pairs include [great,fine],[drama,acting],[skills,talent].
# great~fine ✓, acting~drama ✓ (symmetric), skills~talent ✓ → true ✓ Got it."

# PHASE 2 — BRUTE FORCE
# "The simplest approach: first check lengths. Then for each position i,
# check if sentence1[i] == sentence2[i], or scan all similarPairs
# to find a match in either direction.
#
# Time: O(n * p) – for each of n word pairs, scan up to p similarity pairs.
# Space: O(1) – no preprocessing.
# Should I optimize?"
class _734_SentenceSimilarity_Brute:
    def areSentencesSimilar(self, sentence1: List[str], sentence2: List[str],
        similarPairs: List[List[str]]) -> bool:
        if len(sentence1) != len(sentence2):
            return False
        for word1, word2 in zip(sentence1, sentence2):
            if word1 == word2:
                continue
            found = any((a == word1 and b == word2) or (a == word2 and b == word1)
                for a, b in similarPairs)
            if not found:
                return False
        return True

# PHASE 3 — OPTIMIZE
# "The bottleneck is scanning all p pairs for every word position.
# We're asking the same question repeatedly: is word2 similar to word1?
# A hash map from each word to its set of similar words answers that in O(1).
#
# Preprocess similarPairs once: for each [a, b], add b to hmap[a] and a to hmap[b].
# That handles symmetry automatically – no need to check both directions at lookup
time.
#
# Then for each (word1, word2) pair in the sentences:
# if they're equal, they're trivially similar – skip.

```

```
# Otherwise check if word2 is in hmap[word1].
# Using defaultdict(set) means missing keys return an empty set – no KeyError.
#
# Pseudocode:
# if len(s1) != len(s2): return False
# similar = defaultdict(set)
# for a, b in pairs: similar[a].add(b); similar[b].add(a)
# for w1, w2 in zip(s1, s2):
# if w1 != w2 and w2 not in similar[w1]: return False
# return True
#
# Time: O(p + n). Space: O(p) for the map. Does that make sense?"
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _734_SentenceSimilarity:
    def areSentencesSimilar(self, sentence1: List[str], sentence2: List[str],
similarPairs: List[List[str]]) -> bool:
    if len(sentence1) != len(sentence2):
        return False

    similar = defaultdict(set)
    for word_a, word_b in similarPairs:
        similar[word_a].add(word_b)
        similar[word_b].add(word_a) # symmetric: both directions

    for word1, word2 in zip(sentence1, sentence2):
        if word1 != word2 and word2 not in similar[word1]:
            return False

    return True
```

PHASE 5 — TEST & DEBUG

"Let me trace through all three examples."

```
#
# Example 1: s1=['great','acting','skills'], s2=['fine','drama','talent']
# pairs=[['great','fine'],['drama','acting'],['skills','talent']]
# similar = {great:{fine}, fine:{great}, drama:{acting}, acting:{drama},
skills:{talent}, talent:{skills}}
# (great,fine): great!=fine, fine in similar[great] ✓
# (acting,drama): acting!=drama, drama in similar[acting] ✓
# (skills,talent): skills!=talent, talent in similar[skills] ✓
# return True ✓
#
# Example 2: s1=['great'], s2=['great'], pairs=[]
# Lengths match. similar={} (empty).
# (great,great): great==great → skip.
# return True ✓
#
# Example 3: s1=['great'], s2=['doubleplus','good'], pairs=[['great','doubleplus']]
# len(s1)=1 != len(s2)=2 → return False immediately ✓
#
```

```

# Non-transitive check: pairs=[['a','b'],['b','c']], checking a~c
# similar = {a:{b}, b:{a,c}, c:{b}}
# word1='a', word2='c': a!=c, c not in similar[a]={b} → return False ✓
# Confirms we don't accidentally apply transitivity.
#
# "No bugs. All cases pass including the transitivity trap."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(p + n) - O(p)$  to build the similarity map,  $O(n)$  to check each word pair.
# Space  $O(p)$  for the map – each pair adds two entries.
#
# The critical design decision is storing both directions upfront rather than
# checking both at lookup time. It keeps the lookup a single  $O(1)$  set membership
# test.
#
# Important constraint to call out: similarity is NOT transitive.
# This rules out Union-Find (which is designed for transitive equivalence classes).
# If transitivity WERE required – for example in Sentence Similarity II (#737) –
# we'd use Union-Find to group words into connected components and check
# if sentence1[i] and sentence2[i] share the same root.
#
# Given more time I'd ask: what if the same word pair could appear in different
# cases,
# like ['Great','fine'] vs ['great','fine']? We'd normalize to lowercase before
# building
# the map and before lookups."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def areSentencesSimilar(
        self,
        sentence1: list[str],
        sentence2: list[str],
        similarPairs: list[list[str]],
    ) -> bool:
        if len(sentence1) != len(sentence2):
            return False

```

```
# map[key] := all the similar words of key
map = collections.defaultdict(set)

for a, b in similarPairs:
    map[a].add(b)
    map[b].add(a)

for word1, word2 in zip(sentence1, sentence2):
    if word1 == word2:
        continue

    if word1 not in map:
        return False
    if word2 not in map[word1]:
        return False

return True
```

• LeetCode

```
class Solution:
    def areSentencesSimilar(
        self, sentence1: List[str], sentence2: List[str], similarPairs: List[List[str]]
    ) -> bool:
        if len(sentence1) != len(sentence2):
            return False
        s = {(x, y) for x, y in similarPairs}
        for x, y in zip(sentence1, sentence2):
            if x != y and (x, y) not in s and (y, x) not in s:
                return False

        return True
```

• SimplyLeet

```

# Function to determine if two sentences are similar
def are_sentences_similar(sentence1, sentence2, similarPairs):
    # Check if sentences are of the same length
    if len(sentence1) != len(sentence2):
        return False

    # Build dictionary to store similar word pairs in both directions
    similar_dict = {}
    for word1, word2 in similarPairs:
        if word1 not in similar_dict:
            similar_dict[word1] = set()
            similar_dict[word1].add(word2)
        if word2 not in similar_dict:
            similar_dict[word2] = set()
            similar_dict[word2].add(word1)

    # Check each pair of words in the sentences
    for w1, w2 in zip(sentence1, sentence2):
        # If words are the same, continue
        if w1 == w2:
            continue

        # Check if w1 is similar to w2 explicitly
        if (w1 not in similar_dict or w2 not in similar_dict[w1]) and (w2 not
in similar_dict or w1 not in similar_dict[w2]):
            return False

    return True

# Example usage:
# sentence1 = ["great", "acting", "skills"]
# sentence2 = ["fine", "drama", "talent"]

# similarPairs = [["great", "fine"], ["drama", "acting"], ["skills", "talent"]]
# print(are_sentences_similar(sentence1, sentence2, similarPairs)) # Output:
True

```

• LC.ca


```
class Solution:
    def areSentencesSimilar(
        self, sentence1: List[str], sentence2: List[str], similarPairs: List[List[str]]
    ) -> bool:
        if len(sentence1) != len(sentence2):
            return False
        s = {(a, b) for a, b in similarPairs}
        return all(
            a == b or (a, b) in s or (b, a) in s for a, b in zip(sentence1, sentence2)
        )
```

◆ Quick Review

Key Insights

- First, check if both sentences have the same length.
- A word is always similar to itself.
- Similarity is defined only by the given pairs; it is not transitive.
- Use a hash table (dictionary) to store bi-directional similar pairs for quick look-ups.

Space & Time

Time Complexity: $O(n + m)$, where n is the number of words in the sentences and m is the number of similar pairs.

Space Complexity: $O(m)$, for storing the similar pairs in a hash table.

Solution

Check if the two sentences have the same length; if not, return false. Build a hash table where each word maps to a set of words it is similar to. For each pair $[x, y]$ in `similarPairs`, add y to the set for x and x to the set for y . Iterate over the words of both sentences simultaneously. For each corresponding pair, if the words are identical, continue; otherwise, check if one word appears in the similar set of the other. If all pairs meet the similarity condition, return true; otherwise, return false.

String

□ ★ #1165 Single Row Keyboard ★

EAS[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String

Lists: ★ Premium | Premium 98

Problem

There is a special keyboard with all keys in a single row.

Given a string keyboard of length 26 indicating the layout of the keyboard (indexed from 0 to 25). Initially, your finger is at index 0. To type a character, you have to move your finger to the index of the desired character. The time taken to move your finger from index i to index j is $|i - j|$. You want to type a string word. Write a function to calculate how much time it takes to type it with one finger.

Example 1:

Input: keyboard = "abcdefghijklmnopqrstuvwxyz", word = "cba"

Output: 4

Explanation: The index moves from 0 to 2 to write 'c' then to 1 to write 'b' then to 0 again to write 'a'.

Total time = 2 + 1 + 1 = 4.

Example 2:

Input: keyboard = "pqrstuvwxyzabcdefghijklmnopqrstuvwxyz", word = "leetcode"

Output: 73

Constraints:

- keyboard.length == 26
- keyboard contains each English lowercase letter exactly once in some order.
- 1 <= word.length <= 104
- word[i] is an English lowercase letter.

My LeetCode Solution

• My LeetCode Solution

```
# Single Row Keyboard
class Solution:
    def calculateTime(self, keyboard: str, word: str) -> int:
        hmap = {c:i for i,c in enumerate(keyboard)}
        pos = 0
        ans = 0
        for c in word:
            ans += abs(hmap[c]-pos)
            pos = hmap[c]

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

We have a custom keyboard layout – 26 characters in some arbitrary order.

Our finger starts at index 0. To type each character in word,

we move from the current index to that character's index on the keyboard.

The cost is the absolute distance moved. We sum all moves and return the total.
Right?"

#

"A few quick questions:

The keyboard always has exactly 26 letters, each exactly once – so it's a permutation.

Word can only contain lowercase letters, and every letter in word is guaranteed

to appear on the keyboard – so no missing-key edge case to handle.

Word length can be up to 10^4 – so $O(n)$ should be fine."

#

"Let me walk example 1:

keyboard='abcdefghijklmnopqrstuvwxyz', word='cba'.

Start at 0. 'c' is at index 2: $|0-2|=2$. 'b' is at 1: $|2-1|=1$. 'a' is at 0:
 $|1-0|=1$.

Total = $2+1+1 = 4$. Got it."

PHASE 2 — BRUTE FORCE

"The simplest approach: for each character in word, scan the keyboard string

from left to right to find where it lives, then compute the distance.

#

Time: $O(n * 26)$ – for each of n characters we scan up to 26 positions.

Since keyboard is fixed at 26, this is effectively $O(n)$.

Space: $O(1)$.

Should I optimize the lookup?"

```
class _1165_SingleRowKeyboard_Brute:
```

```
    def calculateTime(self, keyboard: str, word: str) -> int:
```

```
        total = 0
```

```
        position = 0
```

```
        for char in word:
```

```
            next_pos = keyboard.index(char)
```

```
            total += abs(position - next_pos)
```

```
            position = next_pos
```

```
        return total
```

PHASE 3 — OPTIMIZE

"The brute force scans 26 characters for every letter typed – that's the inner $O(26)$.

We're asking the same question repeatedly: where does this letter live on the keyboard?

A hash map answers that in $O(1)$ after a one-time $O(26)$ build.

#

Preprocess keyboard once: build a map from character $\rightarrow$ index.

Then walk word: for each character, look up its index in $O(1)$,

add the absolute distance from the current position, update position, move on.

#

```
# Pseudocode:
# pos_map = {char: i for i, char in enumerate(keyboard)}
# total, position = 0, 0
# for char in word:
#     total += abs(position - pos_map[char])
#     position = pos_map[char]
# return total
#
# Time:  $O(26 + n) = O(n)$ . Space:  $O(26) = O(1)$  – the map is fixed size.
# In practice the constant factor improvement is small since 26 is tiny,
# but the hash map is the correct interview answer – cleaner and scales to
# larger alphabets."
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _1165_SingleRowKeyboard:
    def calculateTime(self, keyboard: str, word: str) -> int:
        key_index = {char: i for i, char in enumerate(keyboard)}
        total = 0
        position = 0
        for char in word:
            total += abs(position - key_index[char])
            position = key_index[char]
        return total
```

PHASE 5 — TEST & DEBUG

"Let me trace through both examples."

```
#
# keyboard='abcdefghijklmnopqrstuvwxyz', word='cba'
# key_index = {'a':0, 'b':1, 'c':2, ...}
# char='c': |0-2|=2, position=2. total=2
# char='b': |2-1|=1, position=1. total=3
# char='a': |1-0|=1, position=0. total=4
# return 4 ✓
#
# keyboard='pqrstuvwxyzabcdefghijklmnopqrstuvwxyz', word='leetcode'
# key_index =
# {'p':0, 'q':1, ..., 'a':10, 'b':11, ..., 'l':21, 'e':14, 't':19, 'c':12, 'o':24, 'd':13}
# char='l': |0-21|=21, position=21. total=21
# char='e': |21-14|=7, position=14. total=28
# char='e': |14-14|=0, position=14. total=28
# char='t': |14-19|=5, position=19. total=33
# char='c': |19-12|=7, position=12. total=40
# char='o': |12-24|=12, position=24. total=52
# char='d': |24-13|=11, position=13. total=63
# char='e': |13-14|=1, position=14. total=64 ← wait, let me recheck key_index
# (keyboard='pqrstuvwxyzabcdefghijklmnopqrstuvwxyz':
# p=0, q=1, r=2, s=3, t=4, u=5, v=6, w=7, x=8, y=9, z=10,
# a=11, b=12, c=13, d=14, e=15, f=16, g=17, h=18, i=19, j=20, k=21, l=22, m=23, n=24, o=25)
# char='l': |0-22|=22. char='e': |22-15|=7. char='e': 0. char='t': |15-4|=11.
```

```
# char='c': |4-13|=9. char='o': |13-25|=12. char='d': |25-14|=11. char='e':
|14-15|=1.
# total = 22+7+0+11+9+12+11+1 = 73 ✓
#
# "No bugs. Manual trace confirms both examples."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(n)$  –  $O(26)$  to build the map,  $O(n)$  to walk word. Space  $O(1)$  – 26-entry map.
#
# The brute force using keyboard.index() is also  $O(n)$  in practice since keyboard is
# always 26 characters – but the hash map is the canonical interview answer because
# it makes the  $O(1)$  lookup intent explicit and generalizes to any alphabet size.
#
# Trade-off: if keyboard could be very long (say a  $10^6$ -char layout), the hash map
# preprocessing cost would matter more – but we'd still prefer it over .index().
#
# One-liner alternative:
# pos = {c: i for i, c in enumerate(keyboard)}
# return sum(abs(pos[b] - pos[a]) for a, b in zip('_', word, word))
# where prepending '_' shifts word so each pair is (prev_char, curr_char),
# and '_' maps to nothing – so you'd actually prepend a dummy start:
# pairs = zip([None] + list(word), word) – but this gets messy.
# The explicit loop is cleaner in an interview.
#
# Given more time I'd ask: what if we can use two fingers?
# That becomes a DP problem – optimal two-finger typing is a classic hard variant."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def calculateTime(self, keyboard: str, word: str) -> int:
        letterToIndex = {c: i for i, c in enumerate(keyboard)}
        return (letterToIndex[word[0]] +
                sum(abs(letterToIndex[a] - letterToIndex[b])
                    for a, b in itertools.pairwise(word)))
```

• LeetDoocs


```
class Solution:
    def calculateTime(self, keyboard: str, word: str) -> int:
        pos = {c: i for i, c in enumerate(keyboard)}
        ans = i = 0
        for c in word:
            ans += abs(pos[c] - i)
            i = pos[c]
        return ans
```

```
class Solution {
    public int calculateTime(String keyboard, String word) {
        int[] pos = new int[26];
        for (int i = 0; i < 26; ++i) {
            pos[keyboard.charAt(i) - 'a'] = i;
        }
        int ans = 0, i = 0;
        for (int k = 0; k < word.length(); ++k) {
            int j = pos[word.charAt(k) - 'a'];
            ans += Math.abs(i - j);

            i = j;
        }
        return ans;
    }
}
```

```
class Solution {
public:
    int calculateTime(string keyboard, string word) {
        int pos[26];
        for (int i = 0; i < 26; ++i) {
            pos[keyboard[i] - 'a'] = i;
        }
        int ans = 0, i = 0;
        for (char& c : word) {
            int j = pos[c - 'a'];

            ans += abs(i - j);
            i = j;
        }
        return ans;
    }
};
```

```
func calculateTime(keyboard string, word string) (ans int) {
    pos := [26]int{}
    for i, c := range keyboard {
        pos[c-'a'] = i
    }
    i := 0
    for _, c := range word {
        j := pos[c-'a']
        ans += abs(i - j)
        i = j
    }
    return
}
```

```
func abs(x int) int {
    if x < 0 {
        return -x
    }
    return x
}
```

```
function calculateTime(keyboard: string, word: string): number {
    const pos: number[] = Array(26).fill(0);
    for (let i = 0; i < 26; ++i) {
        pos[keyboard.charCodeAt(i) - 97] = i;
    }
    let ans = 0;
    let i = 0;
    for (const c of word) {
        const j = pos[c.charCodeAt(0) - 97];
        ans += Math.abs(i - j);
        i = j;
    }
    return ans;
}
```

• SimplyLeet

```
# Build a dictionary to map each character to its index in the keyboard string.
def calculateTime(keyboard: str, word: str) -> int:
    # Create a mapping for character to index
    char_index = {char: idx for idx, char in enumerate(keyboard)}

    # Initialize the total time and starting position
    total_time = 0
    current_position = 0

    # Iterate over each character in the word

    for char in word:
        # Find the target index for the character
        target_position = char_index[char]
        # Calculate the time to move to the target position
        total_time += abs(target_position - current_position)
        # Update current position to the target position
        current_position = target_position

    return total_time

# Example usage:
# print(calculateTime("abcdefghijklmnopqrstuvwxyz", "cba"))
```

• LC.ca

```
class Solution:
    def calculateTime(self, keyboard: str, word: str) -> int:
        pos = {c: i for i, c in enumerate(keyboard)}
        ans = i = 0
        for c in word:
            ans += abs(pos[c] - i)
            i = pos[c]
        return ans
```

◆ Quick Review

Key Insights

- Create a mapping (hash table/dictionary) from each character to its index on the keyboard for $O(1)$ lookups.
- The cost to type each character is the absolute difference between the current position and the target character's position on the keyboard.
- Start from index 0 and sum up the distances moved for each successive character in the word.

Space & Time

Time Complexity: $O(n)$, where n is the length of the word.

Space Complexity: $O(1)$ (constant space for a mapping of 26 characters).

Solution

We solve the problem by first building a dictionary that maps each character in the keyboard layout to its corresponding index. We then initialize a variable to track the current finger position (starting at 0). For each character in the word, we look up its index, calculate the absolute difference from the current position, add that distance to the total

time, and update the current position. This approach uses a single pass over the word with constant time lookups, ensuring an efficient solution.

Two Pointers

Round 1 · High · Easy · 5 questions

Two Pointers

□ #88 Merge Sorted Array

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Sorting
Lists: Denny Zhang

Problem

You are given two integer arrays `nums1` and `nums2`, sorted in non-decreasing order, and two integers `m` and `n`, representing the number of elements in `nums1` and `nums2` respectively.

Merge `nums1` and `nums2` into a single array sorted in non-decreasing order.

The final sorted array should not be returned by the function, but instead be stored inside the array `nums1`. To accommodate this, `nums1` has a length of `m + n`, where the first `m` elements denote the elements that should be merged, and the last `n` elements are set to 0 and should be ignored. `nums2` has a length of `n`.

Example 1:

Input: nums1 = [1,2,3,0,0,0], m = 3, nums2 = [2,5,6], n = 3
 Output: [1,2,2,3,5,6]
 Explanation: The arrays we are merging are [1,2,3] and [2,5,6].
 The result of the merge is [1,2,2,3,5,6] with the underlined elements coming from nums1.

Example 2:

Input: nums1 = [1], m = 1, nums2 = [], n = 0
 Output: [1]
 Explanation: The arrays we are merging are [1] and [].
 The result of the merge is [1].

Example 3:

Input: nums1 = [0], m = 0, nums2 = [1], n = 1
 Output: [1]
 Explanation: The arrays we are merging are [] and [1].
 The result of the merge is [1].
 Note that because m = 0, there are no elements in nums1. The 0 is only there to ensure the merge result can fit in nums1.

Constraints:

- `nums1.length == m + n`
- `nums2.length == n`
- `0 <= m, n <= 200`
- `1 <= m + n <= 200`
- `-109 <= nums1[i], nums2[j] <= 109`

Follow up: Can you come up with an algorithm that runs in $O(m + n)$ time?

My LeetCode Solution

- My LeetCode Solution

```
# Merge Sorted Array
class Solution:
    def merge(self, nums1: List[int], m: int, nums2: List[int], n: int) -> None:
        """
        Do not return anything, modify nums1 in-place instead.
        """

        i = m-1
        j = n-1

        k = m+n-1

        while k>=0:
            if j<0 or i>=0 and nums1[i] >= nums2[j]:
                nums1[k] = nums1[i]
                i-=1

            else:
                nums1[k] = nums2[j]
                j-=1

            k-=1
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

nums1 has m valid elements followed by n zeros as padding.

nums2 has n elements. Both are sorted in non-decreasing order.

We need to merge them in-place into nums1 – sorted, no return value. Right?"

#

"A few quick questions:

Can m or n be 0? Yes – if n=0 nums1 is already done; if m=0 we just copy nums2.

Can there be negative numbers? Yes, constraints go down to -10^9 .

Can there be duplicates? Yes – non-decreasing order means equal values are allowed.

We cannot use extra space for the merge? The follow-up asks for $O(m+n)$ time, implying in-place. I'll target $O(1)$ extra space."

#

"Let me walk example 1:

nums1=[1,2,3,0,0,0] m=3, nums2=[2,5,6] n=3.

```
# Merge [1,2,3] and [2,5,6] into nums1 → [1,2,2,3,5,6]. Got it."
```

PHASE 2 — BRUTE FORCE

```
# "The simplest approach: copy nums2 into the tail of nums1 (the zero slots),
# then sort the entire nums1 array.
```

```
#
```

```
# nums1[m:] = nums2, then nums1.sort()
```

```
#
```

```
# Time:  $O((m+n) \log(m+n))$  for the sort. Space:  $O(1)$  extra – sort is in-place.
```

```
# This is clean but doesn't use the fact that both inputs are already sorted.
```

```
# Should I optimize to use that sorted property?"
```

```
class _88_MergeSortedArray_Brute:
```

```
    def merge(self, nums1: List[int], m: int, nums2: List[int], n: int) -> None:
```

```
        nums1[m:] = nums2
```

```
        nums1.sort()
```

PHASE 3 — OPTIMIZE

```
# "The brute force throws away the sorted order and pays  $O(\log(m+n))$  to re-sort.
```

```
# The classic merge from the front copies to a temp array –  $O(m+n)$  space.
```

```
# But there's a smarter way that stays  $O(1)$  space.
```

```
#
```

```
# Key insight: if we merge from the BACK, we never overwrite elements we still need.
```

```
# The tail of nums1 is empty padding – we can fill it safely from right to left.
```

```
#
```

```
# Three pointers:
```

```
# i → last valid element in nums1 (index m-1)
```

```
# j → last element in nums2 (index n-1)
```

```
# k → last slot in nums1 (index m+n-1)
```

```
#
```

```
# At each step, pick the larger of nums1[i] and nums2[j], place it at k, move that
# pointer left.
```

```
# When j drops below 0, nums2 is exhausted – nums1's remaining elements are already
# in place.
```

```
# When i drops below 0 first, copy remaining nums2 into nums1.
```

```
#
```

```
# Pseudocode:
```

```
# i, j, k = m-1, n-1, m+n-1
```

```
# while k >= 0:
```

```
# if j < 0 or (i >= 0 and nums1[i] > nums2[j]):
```

```
#     nums1[k] = nums1[i]; i -= 1
```

```
# else:
```

```
#     nums1[k] = nums2[j]; j -= 1
```

```
# k -= 1
```

```
#
```

```
# Time:  $O(m+n)$  – each element is placed exactly once.
```

```
# Space:  $O(1)$  – purely in-place.
```

```
# Does that make sense before I code it?"
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```

class _88_MergeSortedArray:
    def merge(self, nums1: List[int], m: int, nums2: List[int], n: int) -> None:
        last1 = m - 1 # tail of valid nums1 elements
        last2 = n - 1 # tail of nums2
        fill = m + n - 1 # next slot to fill in nums1 (right to left)
        while fill >= 0:
            if last2 < 0 or (last1 >= 0 and nums1[last1] > nums2[last2]):
                nums1[fill] = nums1[last1]
                last1 -= 1
            else:
                nums1[fill] = nums2[last2]
                last2 -= 1
            fill -= 1

```

PHASE 5 — TEST & DEBUG

"Let me trace through all three examples."

```

#
# nums1=[1,2,3,0,0,0] m=3, nums2=[2,5,6] n=3
# last1=2, last2=2, fill=5
# fill=5: nums1[2]=3 vs nums2[2]=6 → 6 wins → nums1[5]=6, last2=1, fill=4
# fill=4: nums1[2]=3 vs nums2[1]=5 → 5 wins → nums1[4]=5, last2=0, fill=3
# fill=3: nums1[2]=3 vs nums2[0]=2 → 3 wins → nums1[3]=3, last1=1, fill=2
# fill=2: nums1[1]=2 vs nums2[0]=2 → equal, take nums2 → nums1[2]=2, last2=-1,
fill=1
# fill=1: last2<0 → take nums1[1]=2 → nums1[1]=2, last1=0, fill=0
# fill=0: last2<0 → take nums1[0]=1 → nums1[0]=1, last1=-1, fill=-1
# nums1=[1,2,2,3,5,6] ✓
#
# nums1=[1] m=1, nums2=[] n=0
# last1=0, last2=-1, fill=0
# fill=0: last2<0 → nums1[0]=nums1[0]=1. fill=-1. done.
# nums1=[1] ✓
#
# nums1=[0] m=0, nums2=[1] n=1
# last1=-1, last2=0, fill=0
# fill=0: last1<0 → nums1[0]=nums2[0]=1, last2=-1, fill=-1. done.
# nums1=[1] ✓
#

```

"No bugs. All three cases pass including the m=0 and n=0 edges."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m+n)$ – each of the $m+n$ positions is filled exactly once.

Space $O(1)$ – purely in-place, no auxiliary arrays.

```

#
# The core insight worth stating explicitly: merging front-to-back requires a temp
# buffer because we'd overwrite nums1's valid elements. Merging back-to-front avoids
# this because the padding zeros are exactly the space we need, and large elements
# fill the tail first without touching anything we still care about.
#

```

Edge cases worth calling out:

```
# - n=0: loop runs but last2<0 always, so nums1's valid elements stay in place.
# - m=0: last1<0 always, so nums2 is copied straight into nums1.
# - All nums2 elements are larger than all nums1 elements: nums2 fills the back,
# then nums1 elements slot into the front - handled cleanly by the pointer logic.
#
# Given more time I'd ask: what if nums1 has exactly m slots (no padding)?
# Then in-place is impossible - we'd need to return a new merged array,
# which is the standard merge step in merge sort."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def merge(self, nums1: list[int], m: int, nums2: list[int], n: int) -> None:
        i = m - 1 # nums1's index (the actual nums)
        j = n - 1 # nums2's index
        k = m + n - 1 # nums1's index (the next filled position)

        while j >= 0:
            if i >= 0 and nums1[i] > nums2[j]:
                nums1[k] = nums1[i]
                k -= 1
            else:
                nums1[k] = nums2[j]
                k -= 1
                j -= 1
            i -= 1
```

• LeetDoocs

```
class Solution:
    def merge(self, nums1: List[int], m: int, nums2: List[int], n: int) -> None:
        k = m + n - 1
        i, j = m - 1, n - 1
        while j >= 0:
            if i >= 0 and nums1[i] > nums2[j]:
                nums1[k] = nums1[i]
                i -= 1
            else:
                nums1[k] = nums2[j]
                j -= 1
            k -= 1
```

```

        j -= 1
    k -= 1

```

• SimplyLeet

```

def merge(nums1, m, nums2, n):
    # Initialize pointers for nums1, nums2 and the tail index
    p1 = m - 1
    p2 = n - 1
    tail = m + n - 1

    # Merge arrays from the end by choosing the larger element
    while p1 >= 0 and p2 >= 0:
        if nums1[p1] > nums2[p2]:
            nums1[tail] = nums1[p1]
            p1 -= 1 # Move pointer in nums1
        else:
            nums1[tail] = nums2[p2]
            p2 -= 1 # Move pointer in nums2
        tail -= 1 # Move tail pointer

    # Copy any remaining elements from nums2 into nums1
    while p2 >= 0:
        nums1[tail] = nums2[p2]
        p2 -= 1

    tail -= 1

```

• LC.ca

```

class Solution:
    def merge(self, nums1: List[int], m: int, nums2: List[int], n: int) -> None:
        """
        Do not return anything, modify nums1 in-place instead.
        """
        i, j, k = m - 1, n - 1, m + n - 1
        # what if j=-1 already but k is not 0 yet? e.g. m=[1,2,0], n=[3]
        # => then no more ops needed, rest of m[] already sorted, so this while
        is good enough
        while j >= 0:
            # i could be -1, eg. [7,8,9, ] and [1]

            if i >= 0 and nums1[i] > nums2[j]:
                nums1[k] = nums1[i]
                i -= 1
            else:
                nums1[k] = nums2[j]
                j -= 1
            k -= 1

```

◆ Quick Review

Key Insights

- Both arrays are already sorted in non-decreasing order.
- The merge should be done in-place into `nums1`.
- Start merging from the end to avoid overwriting elements in `nums1`.
- Use two pointers, one for `nums1` and one for `nums2`, and a tail pointer for placement.

Space & Time

Time Complexity: $O(m + n)$

Space Complexity: $O(1)$

Solution

We use a two-pointer approach starting from the end of both arrays. Initialize three pointers: $p1$ pointing to the last valid element in `nums1`. $p2$ pointing to the last element in `nums2`. `tail` pointing to the last index of `nums1`. While both pointers are valid, compare the elements pointed by $p1$ and $p2$, placing the larger element at the `tail` position. Decrement the corresponding pointer and the `tail` pointer. Finally, if any elements remain in `nums2` (i.e., $p2 \geq 0$), copy them over into `nums1`. This solution leverages the extra space at the end of `nums1` and ensures the merge is done in-place in linear time.

Two Pointers

□ #125 Valid Palindrome

EAS[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String
Lists: Grind 169

Problem

A phrase is a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward.

Alphanumeric characters include letters and numbers.

Given a string *s*, return *true* if it is a palindrome, or *false* otherwise.

Example 1:

```
Input: s = "A man, a plan, a canal: Panama"
Output: true
Explanation: "amanaplanacanalpanama" is a palindrome.
```

Example 2:

```
Input: s = "race a car"
Output: false
Explanation: "raceacar" is not a palindrome.
```

Example 3:

Input: s = " "

Output: true

Explanation: s is an empty string "" after removing non-alphanumeric characters.

Since an empty string reads the same forward and backward, it is a palindrome.

Constraints:

- $1 \leq s.length \leq 2 * 10^5$
- s consists only of printable ASCII characters.

My LeetCode Solution

• My LeetCode Solution

```
# Valid Palindrome
class Solution:
    def isPalindrome(self, s: str) -> bool:
        l = 0
        n = len(s)
        r = n-1

        while l<r:
            while l<r and not s[l].isalnum():
                l+=1

            while l<r and not s[r].isalnum():
                r-=1

            if s[l].lower() != s[r].lower():
                return False

            l,r = l+1, r-1

        return True
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

We strip everything except letters and digits, lowercase it all,
then check if the result reads the same forwards and backwards. Right?"
#

"A few quick questions:

Alphanumeric means letters AND digits – not just letters?
So '0P' after cleaning is '0p', which is not a palindrome – confirmed?
What about a string of only spaces or punctuation?
Example 3 shows a single space returns true – after cleaning it's an empty string,
and an empty string is considered a palindrome. Good.
Can the string be length 1? Yes – a single alphanumeric character is a
palindrome."
#

"Let me walk example 1:

'A man, a plan, a canal: Panama' → 'amanaplanacanalpanama' → palindrome ✓
Example 2: 'race a car' → 'raceacar' → not a palindrome ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the simplest correct approach.

Copy every alphanumeric character into a cleaned list (lowercased),
then compare the list to its reverse.
Two passes over the string plus $O(n)$ extra storage.
Time: $O(n)$. Space: $O(n)$ for the cleaned copy.
Should I go ahead and optimize?"

```
class _125_ValidPalindrome_Brute:
    def isPalindrome(self, s: str) -> bool:
        cleaned = [c.lower() for c in s if c.isalnum()]
        return cleaned == cleaned[::-1]
```

PHASE 3 — OPTIMIZE

"The brute force is already $O(n)$ time, but uses $O(n)$ space for the cleaned copy.
We can do this in $O(1)$ extra space using two pointers directly on the original
string.
#

Place left at 0 and right at $\text{len}(s)-1$.
Advance left past any non-alphanumeric characters.
Retreat right past any non-alphanumeric characters.
Compare $s[\text{left}].\text{lower}()$ and $s[\text{right}].\text{lower}()$.
If they differ, return False. Otherwise move both pointers inward.
If left meets or passes right, the string is a palindrome.
#

The key benefit: we never build a second string – we skip junk characters on the
fly.
#

Pseudocode:

```
# left, right = 0, len(s)-1
# while left < right:
# while left < right and not s[left].isalnum(): left += 1
```

```
# while left < right and not s[right].isalnum(): right -= 1
# if s[left].lower() != s[right].lower(): return False
# left += 1; right -= 1
# return True
#
# Time: O(n). Space: O(1). Does that make sense before I code it?"
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _125_ValidPalindrome:
    def isPalindrome(self, s: str) -> bool:
        left = 0
        right = len(s) - 1
        while left < right:
            while left < right and not s[left].isalnum(): # skip non-alphanumeric
                left += 1
            while left < right and not s[right].isalnum(): # skip non-alphanumeric
                right -= 1
            if s[left].lower() != s[right].lower():
                return False
            left += 1
            right -= 1
        return True
```

PHASE 5 — TEST & DEBUG

"Let me trace through all three examples."

#

s="A man, a plan, a canal: Panama"

left=0 ('A'), right=29 ('a') – both alnum.

'A'.lower()='a' == 'a' ✓. left=1, right=28.

left=1 (' ') → skip → left=2 ('m'). right=28 ('m') – alnum.

'm'=='m' ✓. Continue inward... (all pairs match) → return True ✓

#

s="race a car"

left=0 ('r'), right=9 ('r'). 'r'=='r' ✓. left=1, right=8.

left=1 ('a'), right=8 ('a'). 'a'=='a' ✓. left=2, right=7.

left=2 ('c'), right=7 ('c'). 'c'=='c' ✓. left=3, right=6.

left=3 ('e'), right=6 ('a'). 'e'!='a' → return False ✓

(right=6 is 'a' in 'a car'; skip right=5 ' ' → right=4 'a')

Wait – let me recount: "race a car"

indices: r=0,a=1,c=2,e=3,' '=4,a=5,' '=6,c=7,a=8,r=9

left=3 ('e'), right=9→skip none→'r'. 'e'!='r' → return False ✓

#

s=" "

left=0 (' ') → not alnum → left=1. Now left(1) >= right(0) → exit loop.

return True ✓

#

"No bugs. All three examples confirmed."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$ – each character is visited at most once by left or right pointer.

Space $O(1)$ – no auxiliary storage beyond two integer pointers.

#

The brute force one-liner is perfectly valid for an interview too – it's readable

and still $O(n)$ time. The two-pointer version wins purely on space: $O(1)$ vs $O(n)$.

If the interviewer asks about the follow-up, lead with the space trade-off.

#

`isalnum()` handles both letters and digits in one call – worth naming explicitly

so the interviewer knows you're not just checking `isalpha()`.

#

Subtle trap: the inner while loops must guard `left < right` to avoid crossing

pointers – if you forget, you can read out-of-bounds when the string is all

non-alphanumeric (like the single-space example).

#

Given more time I'd ask: what if the string could contain unicode characters

like accented letters (é, ñ)? `isalnum()` in Python handles unicode correctly

by default, so our solution already works – worth calling out."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isPalindrome(self, s: str) -> bool:
        l = 0
        r = len(s) - 1

        while l < r:
            while l < r and not s[l].isalnum():
                l += 1
            while l < r and not s[r].isalnum():
                r -= 1

            if s[l].lower() != s[r].lower():
                return False
            l += 1
            r -= 1

        return True
```

• LeetDoocs

```

class Solution:
    def isPalindrome(self, s: str) -> bool:
        i, j = 0, len(s) - 1
        while i < j:
            if not s[i].isalnum():
                i += 1
            elif not s[j].isalnum():
                j -= 1
            elif s[i].lower() != s[j].lower():
                return False

        else:
            i, j = i + 1, j - 1
        return True

```

• SimplyLeet

```

class Solution:
    def isPalindrome(self, s: str) -> bool:
        # Initialize two pointers at the beginning and end of the string
        left, right = 0, len(s) - 1
        while left < right:
            # Skip non-alphanumeric characters from the left
            while left < right and not s[left].isalnum():
                left += 1
            # Skip non-alphanumeric characters from the right
            while left < right and not s[right].isalnum():
                right -= 1
            # Compare the characters in lowercase
            if s[left].lower() != s[right].lower():
                return False
            left += 1
            right -= 1
        return True

```

• LC.ca

```
class Solution:
    def isPalindrome(self, s: str) -> bool:
        i, j = 0, len(s) - 1
        while i < j:
            if not s[i].isalnum():
                i += 1
            elif not s[j].isalnum():
                j -= 1
            elif s[i].lower() != s[j].lower():
                return False

        else:
            i, j = i + 1, j - 1
        return True
```

◆ Quick Review

Key Insights

- Use two pointers (one starting at the beginning and one at the end) to compare characters.
- Skip characters that are not alphanumeric.
- Compare characters in a case-insensitive manner.
- Continue until the pointers meet or cross.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string, as each character is processed at most once.

Space Complexity: $O(1)$, using only a few extra variables for pointers and temporary comparisons.

Solution

The solution employs a two–pointer approach: Initialize two pointers: one at the start (left) and one at the end (right) of the string. Skip any non–alphanumeric characters using these pointers. Convert the remaining characters at each pointer to lowercase and compare them. If any mismatch is found, return false; otherwise, continue until the pointers cross. Return true if all corresponding characters match. This method efficiently checks for a palindrome without needing additional space for a filtered string.

Two Pointers

□ #283 Move Zeroes

EAS[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers
Lists: Grind 169

Problem

Given an integer array `nums`, move all 0's to the end of it while maintaining the relative order of the non-zero elements.

Note that you must do this in-place without making a copy of the array.

Example 1:

```
Input: nums = [0,1,0,3,12]
Output: [1,3,12,0,0]
```

Example 2:

```
Input: nums = [0]
Output: [0]
```

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$

Follow up: Could you minimize the total number of operations done?

My LeetCode Solution

• My LeetCode Solution

```
# Move Zeroes
class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        p = 0

        for i in range(len(nums)):
            if nums[i] != 0:

                nums[i], nums[p] = nums[p], nums[i]
                p+=1
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# We need to move all 0s to the end of the array while keeping the relative
# order of all non-zero elements. This must be done in-place – no copy. Right?"
#
# "A few quick questions:
# Do we need to return anything? No – modify in-place, return nothing.
# Can the array be all zeros? Yes – output should still be all zeros.
# Can it be all non-zeros? Yes – nothing moves.
# Can there be negative numbers? Yes, constraints go down to -2^31."
#
# "Let me walk example 1:
# nums=[0,1,0,3,12] → [1,3,12,0,0].
# Non-zeros 1,3,12 keep their relative order and shift to the front.
# Zeros fill the tail. Got it."
```

PHASE 2 — BRUTE FORCE

```
# "The simplest approach: collect all non-zero elements into a new list,
# then write them back into nums, then fill the rest with zeros.
#
```

```

# non_zeros = [x for x in nums if x != 0]
# nums[:len(non_zeros)] = non_zeros
# nums[len(non_zeros):] = [0] * (len(nums) - len(non_zeros))
#
# Time: O(n). Space: O(n) – we make a copy of the non-zeros.
# Should I optimize for space?"
class _283_MoveZeroes_Brute:
    def moveZeroes(self, nums: List[int]) -> None:
        non_zeros = [x for x in nums if x != 0]
        nums[:len(non_zeros)] = non_zeros
        nums[len(non_zeros):] = [0] * (len(nums) - len(non_zeros))

# PHASE 3 — OPTIMIZE
# "The brute force uses O(n) extra space for the non-zero copy.
# We can do this in O(1) space with two pointers.
#
# Use a slow pointer 'place' that tracks where the next non-zero should go.
# Scan with a fast pointer 'scan'. Whenever nums[scan] is non-zero,
# swap it with nums[place] and advance place.
#
# This compacts all non-zeros to the front in their original order,
# and the swaps naturally push zeros toward the tail.
#
# Pseudocode:
# place = 0
# for scan in range(len(nums)):
#     if nums[scan] != 0:
#         nums[scan], nums[place] = nums[place], nums[scan]
#         place += 1
#
# Time: O(n). Space: O(1). Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _283_MoveZeroes:
    def moveZeroes(self, nums: List[int]) -> None:
        place = 0
        for scan in range(len(nums)):
            if nums[scan] != 0:
                nums[scan], nums[place] = nums[place], nums[scan]
                place += 1

# PHASE 5 — TEST & DEBUG
# "Let me trace through both examples."
#
# nums=[0,1,0,3,12]
# scan=0: nums[0]=0 → skip. place=0.
# scan=1: nums[1]=1 ≠ 0 → swap(1,0): [1,0,0,3,12]. place=1.
# scan=2: nums[2]=0 → skip. place=1.
# scan=3: nums[3]=3 ≠ 0 → swap(3,1): [1,3,0,0,12]. place=2.

```

```

# scan=4: nums[4]=12 ≠ 0 → swap(4,2): [1,3,12,0,0]. place=3.
# Result: [1,3,12,0,0] ✓
#
# nums=[0]
# scan=0: nums[0]=0 → skip.
# Result: [0] ✓
#
# nums=[1,2,3] (no zeros)
# Every element is non-zero → each swaps with itself. place advances each step.
# Result: [1,2,3] ✓ (self-swaps are fine – no change)
#
# "No bugs. Self-swaps when place==scan are harmless."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) – single pass. Space O(1) – two integer pointers.
#
# Follow-up asks to minimize operations. The swap approach does a swap
# for every non-zero element, even if it's already in the right place.
# A minor optimization: only swap when place != scan to skip self-swaps.
# But in an interview the current version is clean enough.
#
# Alternative: instead of swapping, just overwrite:
# write non-zeros to the front with assignment (not swap), then zero-fill the tail.
# Same complexity but fewer writes – useful if writes are expensive.
#
# place = 0
# for x in nums:
# if x != 0: nums[place] = x; place += 1
# nums[place:] = [0] * (len(nums) - place)
#
# Given more time I'd ask: what if we also need to preserve the positions
# of the zeros (not just push them to the end)? That changes the problem
# to a stable partition – same two-pointer approach still works."

```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def moveZeroes(self, nums: list[int]) -> None:
        j = 0
        for num in nums:
            if num != 0:
                nums[j] = num
                j += 1

        for i in range(j, len(nums)):
            nums[i] = 0
```

• LeetCode

```
class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        k = 0
        for i, x in enumerate(nums):
            if x:
                nums[k], nums[i] = nums[i], nums[k]
                k += 1
```

• SimplyLeet

```
# Moves all zeros in the list 'nums' to the end in-place.
def moveZeroes(nums):
    # 'j' is the pointer where the next non-zero element should be placed.
    j = 0
    # Loop through all the elements with index 'i'
    for i in range(len(nums)):
        # Check if the current element is non-zero.
        if nums[i] != 0:
            # Swap only if i is not equal to j to avoid unnecessary operations.
            nums[j], nums[i] = nums[i], nums[j]

            # Increment j to move to the next insertion index.
            j += 1

# Example usage:
nums = [0, 1, 0, 3, 12]
moveZeroes(nums)
print(nums) # Expected output: [1, 3, 12, 0, 0]
```

• LC.ca

```
class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        i = -1
        for j, x in enumerate(nums):
            if x:
                i += 1
                nums[i], nums[j] = nums[j], nums[i]
```

◆ Quick Review

Key Insights

- Use a two-pointer technique: one pointer to iterate through the array and another pointer to track the position for the next non-zero element.
- For each non-zero element encountered, swap it with the element at the tracking pointer.
- This strategy minimizes operations by ensuring each non-zero element is moved at most once.
- After repositioning all non-zero elements, zeros will naturally occupy the remaining positions.
- The in-place requirement ensures space complexity remains $O(1)$.

Space & Time

Time Complexity: $O(n)$, where n is the number of elements in the array, since only one pass (or at most two simple passes) is required.

Space Complexity: $O(1)$ extra space, as the reordering is accomplished in-place.

Solution

The solution leverages a two-pointer approach. The left pointer (j) keeps track of the index where the next non-zero element should be placed, while the right pointer (i) iterates through the array. Whenever a non-zero element is encountered, it is swapped with the element at index j , and j is then incremented. This effectively gathers all non-zero elements at the beginning while moving zeros to the end. A key optimization is to perform the swap only when necessary (i.e., when i and j are not the same) to reduce the number of write operations.

Two Pointers

□ ★ #408 Valid Word Abbreviation ★

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Two Pointers · String

Lists: ★ Premium | Premium 98

Problem

A string can be abbreviated by replacing any number of non-adjacent, non-empty substrings with their lengths. The lengths should not have leading zeros.

For example, a string such as "substitution" could be abbreviated as (but not limited to):

- "s10n" ("s ubstitutio n")
- "sub4u4" ("sub stit u tion")
- "12" ("substitution")
- "su3i1u2on" ("su bst i t u ti on")
- "substitution" (no substrings replaced)

The following are not valid abbreviations:

- "s55n" ("s ubsti tutio n", the replaced substrings are adjacent)

- "s010n" (has leading zeros)
- "s0ubstitution" (replaces an empty substring)

Given a string word and an abbreviation abbr, return whether the string matches the given abbreviation.

A substring is a contiguous non-empty sequence of characters within a string.

Example 1:

```
Input: word = "internationalization", abbr = "i12iz4n"
Output: true
Explanation: The word "internationalization" can be abbreviated as "i12iz4n"
("i nternational iz atio n").
```

Example 2:

```
Input: word = "apple", abbr = "a2e"
Output: false
Explanation: The word "apple" cannot be abbreviated as "a2e".
```

Constraints:

- $1 \leq \text{word.length} \leq 20$
- word consists of only lowercase English letters.
- $1 \leq \text{abbr.length} \leq 10$
- abbr consists of lowercase English letters and digits.
- All the integers in abbr will fit in a 32-bit integer.

My LeetCode Solution

• My LeetCode Solution

```
# Valid Word Abbreviation
class Solution:
    def validWordAbbreviation(self, word: str, abbr: str) -> bool:
        i=0
        j=0
        m = len(word)
        n = len(abbr)
        x = 0

        while i<m and j <n:

            if abbr[j].isdigit():
                if abbr[j] == "0" and x==0:
                    return False

                x = x*10 + int(abbr[j])

            else:
                i += x
                x = 0
                if i>=m or abbr[j] != word[i]:

                    return False
                i+=1

            j+=1
        return i+x == m and j == n
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

An abbreviation replaces one or more non-adjacent non-empty substrings with their lengths.

Given a word and an abbreviation, return true if the abbreviation is valid for the word.

Numbers in the abbreviation represent how many consecutive characters to skip. Right?"

```

#
# "A few quick questions:
# Can there be leading zeros in the number part? No – that's explicitly invalid.
# So '01' is not valid even though it could theoretically mean skip 1.
# Can the abbreviation be the full word with no numbers? Yes – that's valid.
# Can it be just a single number like '12'? Yes – if the word has exactly 12 chars."
#
# "Let me walk example 1:
# word='internationalization', abbr='i12iz4n'.
# 'i' matches 'i'. '12' skips 12 chars. 'i' matches 'i'. 'z' matches 'z'.
# '4' skips 4 chars. 'n' matches 'n'. We reach end of both → true ✓
# Example 2: word='apple', abbr='a2e'. 'a' matches. '2' skips 'pp'. 'e'→word[3]='l'
# ≠ 'e' → false ✓"

# PHASE 2 — BRUTE FORCE (also optimal)
# "Let me start with brute force to lock in the logic.
# Expand the abbreviation letter-by-letter against word using two pointers.
# When we hit a digit in abbr, consume that many characters from word.
# Any mismatch ? false; both exhausted together ? true.
# Time: O(n). Space: O(1).
# This is already optimal for the constraints – complexity stated clearly."

# PHASE 3 — OPTIMIZE
# "Use pointer i into word and j into abbr.
# When abbr[j] is a digit: check for leading zero (abbr[j]=='0' and num==0 →
invalid).
# Accumulate the number: num = num*10 + digit.
# When abbr[j] is a letter: first advance i by num (the accumulated skip), reset num
to 0.
# Then compare word[i] with abbr[j] – if they differ, return False. Advance both i
and j.
# After the loop: we're valid only if i + num == len(word) AND j == len(abbr).
# That handles a trailing number at the end of abbr.
#
# Pseudocode:
# i, j, num = 0, 0, 0
# while i < len(word) and j < len(abbr):
# if abbr[j].isdigit():
# if abbr[j]=='0' and num==0: return False # leading zero
# num = num*10 + int(abbr[j])
# else:
# i += num; num = 0
# if i >= len(word) or word[i] != abbr[j]: return False
# i += 1
# j += 1
# return i + num == len(word) and j == len(abbr)
#
# Time: O(n + m) where n=len(word), m=len(abbr). Space: O(1)."
```

PHASE 4 — CLEAN CODE

"Now I'll implement with meaningful names."

```
class _408_ValidWordAbbreviation:
    def validWordAbbreviation(self, word: str, abbr: str) -> bool:
        word_ptr = 0
        abbr_ptr = 0
        skip = 0
        while word_ptr < len(word) and abbr_ptr < len(abbr):
            if abbr[abbr_ptr].isdigit():
                if abbr[abbr_ptr] == '0' and skip == 0: # leading zero
                    return False
                skip = skip * 10 + int(abbr[abbr_ptr])
            else:
                word_ptr += skip
                skip = 0
                if word_ptr >= len(word) or word[word_ptr] != abbr[abbr_ptr]:
                    return False
                word_ptr += 1
                abbr_ptr += 1
        return word_ptr + skip == len(word) and abbr_ptr == len(abbr)
```

PHASE 5 — TEST & DEBUG

"Let me trace both examples."

```
#
# word='internationalization'(len=20), abbr='i12iz4n'
# j=0: 'i' -> skip=0, advance word_ptr 0, word[0]='i'=='i' ✓. word_ptr=1.
# j=1: '1' -> skip=1.
# j=2: '2' -> skip=12.
# j=3: 'i' -> word_ptr+=12=13. word[13]='i'=='i' ✓. word_ptr=14.
# j=4: 'z' -> skip=0, word[14]='z'=='z' ✓. word_ptr=15.
# j=5: '4' -> skip=4.
# j=6: 'n' -> word_ptr+=4=19. word[19]='n'=='n' ✓. word_ptr=20.
# Loop ends (word_ptr=20=len(word)). word_ptr+skip=20+0=20==20, abbr_ptr=7==7. True
✓
#
# word='apple'(len=5), abbr='a2e'
# j=0: 'a' -> word[0]='a'=='a' ✓. word_ptr=1.
# j=1: '2' -> skip=2.
# j=2: 'e' -> word_ptr+=2=3. word[3]='l' ≠ 'e' -> return False ✓
#
# Leading zero: abbr='a01e' -> j=1: '0' and skip==0 -> return False ✓
#
# "No bugs. Leading zero check and trailing number handled correctly."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n + m)$ – each character in word and abbr is visited at most once.

Space $O(1)$ – three integer pointers.

#

The two trickiest cases to get right:

1. Leading zeros – must check `abbr[j]=='0'` AND `accumulated num==0` to distinguish

```
# '0' (leading zero, invalid) from '10', '20' etc. (valid multi-digit numbers ending
in 0).
# 2. Trailing number – if abbr ends with digits (e.g. abbr='a4'), the loop exits
# when abbr_ptr reaches end but skip is still non-zero.
# The final check word_ptr + skip == len(word) handles this correctly.
#
# Given more time I'd ask: what if we want to generate ALL valid abbreviations
# for a word? That's a backtracking problem – enumerate all subsets of positions
# to replace with their lengths."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def validWordAbbreviation(self, word: str, abbr: str) -> bool:
        i = 0 # word's index
        j = 0 # abbr's index

        while i < len(word) and j < len(abbr):
            if word[i] == abbr[j]:
                i += 1
                j += 1
                continue

            if not abbr[j].isdigit() or abbr[j] == '0':
                return False
            num = 0
            while j < len(abbr) and abbr[j].isdigit():
                num = num * 10 + int(abbr[j])
                j += 1
            i += num

        return i == len(word) and j == len(abbr)
```

• LeetDoocs


```

class Solution:
    def validWordAbbreviation(self, word: str, abbr: str) -> bool:
        m, n = len(word), len(abbr)
        i = j = x = 0
        while i < m and j < n:
            if abbr[j].isdigit():
                if abbr[j] == "0" and x == 0:
                    return False
                x = x * 10 + int(abbr[j])
            else:
                i += x
                x = 0
                if i >= m or word[i] != abbr[j]:
                    return False
                i += 1
            j += 1
        return i + x == m and j == n

```

• SimplyLeet

Python solution for valid word abbreviation

```

def validWordAbbreviation(word: str, abbr: str) -> bool:
    word_index = 0 # pointer for the word
    abbr_index = 0 # pointer for the abbreviation

    while abbr_index < len(abbr):
        # if current abbrev char is a digit
        if abbr[abbr_index].isdigit():
            # leading zero check: if the digit is '0', it is invalid

            if abbr[abbr_index] == '0':
                return False
            # build the entire number
            num = 0
            while abbr_index < len(abbr) and abbr[abbr_index].isdigit():
                num = num * 10 + int(abbr[abbr_index])
                abbr_index += 1
            # skip num characters in word
            word_index += num
        else:

```

```

        # if letter doesn't match or word index is out of bounds, invalid
        if word_index >= len(word) or word[word_index] != abbr[abbr_index]
:
            return False
        word_index += 1
        abbr_index += 1

    # Valid if and only if the entire word was covered
    return word_index == len(word)

# Example usage:

#print(validWordAbbreviation("internationalization", "i12iz4n")) # Expected
output: True
#print(validWordAbbreviation("apple", "a2e")) # Expected output: False

```

• LC.ca

```

class Solution:
    def validWordAbbreviation(self, word: str, abbr: str) -> bool:
        m, n = len(word), len(abbr)
        i = j = x = 0
        while i < m and j < n:
            if abbr[j].isdigit():
                if abbr[j] == "0" and x == 0:
                    return False
                x = x * 10 + int(abbr[j])
            else:
                i += x
                x = 0
                if i >= m or word[i] != abbr[j]:
                    return False
                i += 1
                j += 1
        return i + x == m and j == n

```

◆ Quick Review

Key Insights

- Use two pointers: one for looping through the word and one for the abbreviation.
- When encountering a digit in abbr, accumulate the number and skip that many characters in the word.
- Check for invalid cases such as numbers with leading zeros or consecutive digit segments that might imply adjacent skipped substrings.
- Compare letters directly when they are not digits.

Space & Time

Time Complexity: $O(n + m)$ where n is the length of the word and m is the length of the abbreviation.

Space Complexity: $O(1)$ (constant extra space)

Solution

The approach uses two pointers to iterate over the word and abbreviation. For each character in the abbreviation: If it is a letter, compare it with the current character in the word. If it is a digit, first ensure that it does not start with '0'. Then, convert the sequence of digits into a number which represents how many

characters in the word to skip. After processing, both pointers should have reached the end of their respective strings for the abbreviation to be valid. This approach is efficient due to its linear pass with constant extra space.

Two Pointers

□ #977 Squares of a Sorted Array

EAS[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Sorting
Lists: Grind 169

Problem

Given an integer array `nums` sorted in non-decreasing order, return an array of the squares of each number sorted in non-decreasing order.

Example 1:

```
Input: nums = [-4,-1,0,3,10]
Output: [0,1,9,16,100]
Explanation: After squaring, the array becomes [16,1,0,9,100].
After sorting, it becomes [0,1,9,16,100].
```

Example 2:

```
Input: nums = [-7,-3,2,3,11]
Output: [4,9,9,49,121]
```

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-104 \leq \text{nums}[i] \leq 104$
- `nums` is sorted in non-decreasing order.

Follow up: Squaring each element and sorting the new array is very trivial, could you find an $O(n)$ solution using a different approach?

My LeetCode Solution

• My LeetCode Solution

```
# Squares of a Sorted Array
class Solution:
    def sortedSquares(self, nums: List[int]) -> List[int]:
        ans = [0] * len(nums)
        k = len(nums)-1
        l=0
        r = len(nums)-1

        while k >= 0:
            if abs(nums[l]) > abs(nums[r]):

                ans[k]= nums[l] * nums[l]
                l+=1

            else:
                ans[k] = nums[r]*nums[r]
                r-=1
            k-=1

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

The input array is sorted in non-decreasing order and can contain negatives.

We need to return a new array of the squares of each element, also sorted. Right?"

#

"A few quick questions:

Can the array contain negative numbers? Yes – that's the whole challenge.

Should I return a new array or modify in-place? Return a new array.

Can the array have a single element? Yes – just return $[\text{element}^2]$.

Can all elements be negative? Yes – e.g. $[-3, -2, -1] \rightarrow [1, 4, 9]$."

```

#
# "Let me walk example 1:
# nums=[-4,-1,0,3,10]. Squares: [16,1,0,9,100]. Sorted: [0,1,9,16,100].
# The key observation: the largest squares come from the extremes –
# either the most negative or the most positive end. Got it."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Square every element, sort the resulting array ascending, return it.
# Two-pass: fill squares O(n), then sort O(n log n).
# Correct given unsorted input, but the array is already sorted – we can merge from
both ends.
# Time: O(n log n). Space: O(n) for the squared copy.
# Should I go ahead and optimize?"

class _977_SquaresSortedArray_Brute:
    def sortedSquares(self, nums: List[int]) -> List[int]:
        return sorted(x * x for x in nums)

# PHASE 3 — OPTIMIZE
# "The brute force ignores the fact that the input is already sorted.
# Key insight: in a sorted array, the largest absolute values are always at one
# of the two ends – either the most negative (left) or the most positive (right).
# So the largest square is always max(nums[left]2, nums[right]2).
#
# Use two pointers: left starts at 0, right at n-1.
# Fill the result array from the back (largest to smallest):
# at each step, compare abs(nums[left]) and abs(nums[right]).
# Place the larger square at result[k] and move that pointer inward.
#
# Pseudocode:
# result = [0] * n; k = n-1; left, right = 0, n-1
# while k >= 0:
#   if abs(nums[left]) > abs(nums[right]):
#     result[k] = nums[left]**2; left += 1
#   else:
#     result[k] = nums[right]**2; right -= 1
#   k -= 1
# return result
#
# Time: O(n). Space: O(n) for the output – unavoidable since we return a new array."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _977_SquaresSortedArray:
    def sortedSquares(self, nums: List[int]) -> List[int]:
        n = len(nums)
        result = [0] * n
        fill = n - 1
        left = 0

```



```

right = n - 1
while fill >= 0:
    if abs(nums[left]) > abs(nums[right]):
        result[fill] = nums[left] * nums[left]
        left += 1
    else:
        result[fill] = nums[right] * nums[right]
        right -= 1
    fill -= 1
return result

```

PHASE 5 — TEST & DEBUG

"Let me trace both examples."

#

nums=[-4,-1,0,3,10], n=5

fill=4: abs(-4)=4 < abs(10)=10 → result[4]=100, right=3. fill=3.

fill=3: abs(-4)=4 > abs(3)=3 → result[3]=16, left=1. fill=2.

fill=2: abs(-1)=1 < abs(3)=3 → result[2]=9, right=2. fill=1.

fill=1: abs(-1)=1 > abs(0)=0 → result[1]=1, left=2. fill=0.

fill=0: left==right==2, abs(0)=0 → result[0]=0, right=1. fill=-1.

result=[0,1,9,16,100] ✓

#

nums=[-7,-3,2,3,11], n=5

fill=4: abs(-7)=7 < abs(11)=11 → result[4]=121, right=3. fill=3.

fill=3: abs(-7)=7 > abs(3)=3 → result[3]=49, left=1. fill=2.

fill=2: abs(-3)=3 == abs(3)=3 → take right: result[2]=9, right=2. fill=1.

fill=1: abs(-3)=3 > abs(2)=2 → result[1]=9, left=2. fill=0.

fill=0: left==right==2 → result[0]=4. fill=-1.

result=[4,9,9,49,121] ✓

#

"No bugs. Tie goes to right which is fine – both are equal."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$ – single pass from both ends meeting in the middle.

Space $O(n)$ – required for the output; we can't avoid this since we return a new array.

#

This is the same fill-from-back pattern as Merge Sorted Array (#88) –

worth naming that explicitly to show you recognise the pattern family.

#

Trade-off: the brute force $O(n \log n)$ is two lines and perfectly acceptable

in most interviews. The $O(n)$ two-pointer version is the follow-up answer

the problem explicitly asks for.

#

Given more time I'd ask: what if we needed to do this in-place?

Squaring in-place destroys order for negative numbers, so we'd need to

square + reverse the negative half, then merge the two sorted halves

in-place – doable but significantly more complex."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def sortedSquares(self, nums: list[int]) -> list[int]:
        n = len(nums)
        l = 0
        r = n - 1
        ans = [0] * n

        while n:
            n -= 1
            if abs(nums[l]) > abs(nums[r]):

                ans[n] = nums[l] * nums[l]
                l += 1
            else:
                ans[n] = nums[r] * nums[r]
                r -= 1

        return ans
```

• LeetDoocs

```
class Solution:
    def sortedSquares(self, nums: List[int]) -> List[int]:
        ans = []
        i, j = 0, len(nums) - 1
        while i <= j:
            a = nums[i] * nums[i]
            b = nums[j] * nums[j]
            if a > b:
                ans.append(a)
                i += 1
            else:
                ans.append(b)
                j -= 1
        return ans[::-1]
```

• SimplyLeet

```
# Function to return squares of a sorted array in non-decreasing order.
def sortedSquares(nums):
    # Initialize two pointers at the start and end of the array.
    left, right = 0, len(nums) - 1
    # Create a result array of the same length as nums.
    result = [0] * len(nums)
    # Index to insert the largest square at the current position.
    position = len(nums) - 1

    # Loop until the left and right pointers cross.

    while left <= right:
        # Compare absolute values at the left and right pointers.
        if abs(nums[left]) > abs(nums[right]):
            result[position] = nums[left] * nums[left]
            left += 1 # Move left pointer rightward.
        else:
            result[position] = nums[right] * nums[right]
            right -= 1 # Move right pointer leftward.
        position -= 1 # Move to the next insertion position.
    return result

# Example usage:
print(sortedSquares([-4, -1, 0, 3, 10])) # Output: [0, 1, 9, 16, 100]
```

• LC.ca

```
class Solution:
    def sortedSquares(self, nums: List[int]) -> List[int]:
        n = len(nums)
        res = [0] * n
        i, j, k = 0, n - 1, n - 1
        while i <= j:
            if nums[i] * nums[i] > nums[j] * nums[j]:
                res[k] = nums[i] * nums[i]
                i += 1
            else:
                res[k] = nums[j] * nums[j]
                j -= 1
            k -= 1
        return res
```

◆ Quick Review

Key Insights

- Squaring numbers can disrupt the original order since negative numbers become positive.
- A two–pointer approach can efficiently retrieve the largest square from either end of the array.
- By comparing the absolute values at the two pointers, the largest square is inserted from the back of the result array.
- This method achieves an $O(n)$ time complexity without needing to sort the squared values afterward.

Space & Time

Time Complexity: $O(n)$ – We traverse the array only once.

Space Complexity: $O(n)$ – We use an extra array to store the output.

Solution

We use a two–pointer technique to traverse the array from both ends. Initialize pointers

(left and right) at the beginning and end of the array, respectively, and an index pointer starting at the end of a new results array. At each step, compare the absolute values of the elements at the left and right pointers. The larger absolute value, when squared, is placed in the result array at the current index, and the corresponding pointer is adjusted. This continues until the left pointer exceeds the right pointer. Filling the result array from the back ensures that the array remains sorted in non-decreasing order.

Sorting

Round 1 · High · Easy · 6 questions

Sorting

□ #169 Majority Element

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Divide and Conquer ·
Sorting · Counting

Lists: Grind 169

Problem

Given an array `nums` of size `n`, return the majority element.

The majority element is the element that appears more than $n / 2$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: `nums = [3,2,3]`
Output: 3

Example 2:

Input: `nums = [2,2,1,1,1,2,2]`
Output: 2

Constraints:

- `n == nums.length`

- $1 \leq n \leq 5 * 10^4$
- $-109 \leq \text{nums}[i] \leq 109$
- The input is generated such that a majority element will exist in the array.

Follow-up: Could you solve the problem in linear time and in $O(1)$ space?

My LeetCode Solution

• My LeetCode Solution

```
# Majority Element
class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        ans = None
        count = 0

        for x in nums:
            if count == 0:
                ans = x

            count += 1 if x==ans else -1

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

The majority element is the one that appears MORE THAN $n/2$ times.

We're guaranteed it always exists. We return the element, not its count. Right?"

#

"A few quick questions:

Is there always exactly one majority element? Yes – anything above $n/2$ means at most one element can qualify.

Can n be 1? Yes – that single element is trivially the majority.

Can values be negative? Yes, constraints allow down to -10^9 .

```

# The follow-up asks for linear time and O(1) space – I'll aim for that."
#
# "Let me walk the examples:
# nums=[3,2,3]: 3 appears 2 times, n/2=1. 2 > 1 → 3 is majority. ✓
# nums=[2,2,1,1,1,2,2]: 2 appears 4 times, n/2=3.5. 4 > 3.5 → 2. ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Count frequency of every element with a hash map, then scan counts for any value >
n/2.
# Correct because the majority element must appear more than half the time.
# Time: O(n). Space: O(n) for counts.
# Should I go ahead and optimize?"

class _169_MajorityElement_Brute:
    def majorityElement(self, nums: List[int]) -> int:
        return Counter(nums).most_common(1)[0][0]

# PHASE 3 — OPTIMIZE
# "To get O(1) space, I'll use Boyer-Moore Voting Algorithm.
# The core insight: if we cancel every occurrence of the majority element
# with a different element, the majority element will always survive
# because it has more than n/2 appearances – more than everything else combined.
#
# Keep a candidate and a count:
# When count == 0, adopt the current element as the new candidate.
# If current element matches candidate, increment count.
# If it doesn't match, decrement count (a cancellation).
# The candidate at the end is guaranteed to be the majority element.
#
# Pseudocode:
# candidate, count = None, 0
# for x in nums:
#     if count == 0: candidate = x
#     count += (1 if x == candidate else -1)
# return candidate
#
# Time: O(n) – single pass. Space: O(1) – two variables only."

# PHASE 4 — CLEAN CODE
# "Now I'll implement Boyer-Moore with meaningful names."
class _169_MajorityElement:
    def majorityElement(self, nums: List[int]) -> int:
        candidate = None
        count = 0
        for num in nums:
            if count == 0:
                candidate = num
            count += 1 if num == candidate else -1
        return candidate

```

PHASE 5 — TEST & DEBUG**# "Let me trace through both examples."**

#

nums=[3,2,3]

num=3: count=0 → candidate=3. count=1.

num=2: count=1, 2≠3 → count=0.

num=3: count=0 → candidate=3. count=1.

return 3 ✓

#

nums=[2,2,1,1,1,2,2]

num=2: count=0 → candidate=2. count=1.

num=2: match → count=2.

num=1: no match → count=1.

num=1: no match → count=0.

num=1: count=0 → candidate=1. count=1.

num=2: no match → count=0.

num=2: count=0 → candidate=2. count=1.

return 2 ✓

#

Single element – nums=[7]

count=0 → candidate=7. count=1. return 7 ✓

#

"No bugs. Even when the candidate flips mid-array, the majority element**# reclaims it before the end."****# PHASE 6 — COMPLEXITY & FOLLOW-UP****# "Time $O(n)$ – single pass. Space $O(1)$ – only two variables.**

#

Boyer-Moore is the optimal solution here and worth naming by name –

it shows algorithm knowledge beyond just 'I used two variables'.

#

Correctness argument worth stating: the majority element appears more than $n/2$ times.

Every time it gets cancelled by a different element, it loses one vote and so does

the other element. Since it has more votes than all others combined, it will

always

have votes remaining when everyone else is cancelled out.

#

Alternative $O(n)$ time $O(1)$ space approach: sort and return `nums[n//2]`.# Since majority appears $> n/2$ times, it must occupy the middle index.# But sorting is $O(n \log n)$ – worse than Boyer-Moore.

#

If the problem didn't guarantee a majority exists, Boyer-Moore would need

a second pass to verify the candidate actually appears $> n/2$ times.

#

Given more time I'd ask: what if we need to find all elements appearing

more than $n/3$ times? Boyer-Moore generalises – maintain two candidates

and two counts. That's a well-known variant worth mentioning."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def majorityElement(self, nums: list[int]) -> int:
        ans = None
        count = 0

        for num in nums:
            if count == 0:
                ans = num
            count += (1 if num == ans else -1)

        return ans
```

• LeetDoocs

```
class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        cnt = m = 0
        for x in nums:
            if cnt == 0:
                m, cnt = x, 1
            else:
                cnt += 1 if m == x else -1
        return m
```

• SimplyLeet

```
# Python implementation of the Boyer-Moore Voting Algorithm

def majorityElement(nums):
    # Initialize candidate and counter
    candidate = None
    count = 0

    # Iterate over each number in the input list
    for num in nums:
        # If count is 0, choose the current number as the candidate
```

```

        if count == 0:
            candidate = num
        # Increment count if the current number equals the candidate, otherwise
        decrement
        count += 1 if num == candidate else -1
    # Return the candidate, which is the majority element
    return candidate

# Example usage
print(majorityElement([2,2,1,1,1,2,2])) # Output: 2

```

• LC.ca

```

class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        cnt = m = 0
        for x in nums:
            if cnt == 0:
                m, cnt = x, 1
            else:
                cnt += 1 if m == x else -1
        return m

```

◆ Quick Review

Key Insights

- The majority element appears more than half the length of the array.
- A simple hash table approach can count occurrences, but a more efficient solution exists.
- The Boyer–Moore Voting Algorithm finds the majority in $O(n)$ time and $O(1)$ space.

- The algorithm maintains a candidate and a counter, adjusting the candidate when the count drops to zero.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

We use the Boyer–Moore Voting Algorithm to identify the majority element. The algorithm iterates through the array using two variables: one for the candidate and one for the count. Initially, we set the candidate to an arbitrary value and count to 0. For each element, if the count is 0, we update the candidate to the current element. Then, if the current element is the same as the candidate, we increment the count; otherwise, we decrement it. Since the majority element appears more than $n/2$ times, it will remain as the candidate by the end of the iteration.

Sorting

□ #217 Contains Duplicate

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Sorting
Lists: Grind 169

Problem

Given an integer array `nums`, return `true` if any value appears at least twice in the array, and return `false` if every element is distinct.

Example 1:

Input: `nums = [1,2,3,1]`

Output: `true`

Explanation:

The element 1 occurs at the indices 0 and 3.

Example 2:

Input: `nums = [1,2,3,4]`

Output: `false`

Explanation:

All elements are distinct.

Example 3:

Input: nums = [1,1,1,3,3,4,3,2,4,2]

Output: true

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-109 \leq \text{nums}[i] \leq 109$

My LeetCode Solution

• My LeetCode Solution

```
# Contains Duplicate
class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        seen = set()
        for x in nums:
            if x in seen:
                return True

            seen.add(x)

        return False
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Return true if any value appears at least twice, false if every element is distinct.

We're not asked which value or how many times – just a boolean. Right?"

#

"A few quick questions:

Can the array be empty? Constraints say length ≥ 1 , so no.

Can there be negative numbers? Yes, down to -10^9 .

Is order relevant? No – we just care about value frequency."

#

"Let me walk the examples:

[1,2,3,1]: 1 appears twice → true ✓

[1,2,3,4]: all distinct → false ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Check every pair (i, j) with $j > i$; if `nums[i] == nums[j]`, return true immediately.

If no duplicate pair exists after all checks, return false.

Simple and correct, but nested loops blow up on large arrays.

Time: $O(n^2)$. Space: $O(1)$.

Should I go ahead and optimize?"

```

class _217_ContainsDuplicate_Brute:
    def containsDuplicate(self, nums: List[int]) -> bool:
        for i in range(len(nums)):
            for j in range(i + 1, len(nums)):
                if nums[i] == nums[j]:
                    return True
        return False

```

PHASE 3 — OPTIMIZE

"We're repeatedly asking: have I seen this value before?"

A hash set answers that in $O(1)$. Walk the array once;

if the current number is already in the set, return True immediately.

Otherwise add it and continue.

#

Time: $O(n)$. Space: $O(n)$.

Alternative: sort and check adjacent pairs – $O(n \log n)$, $O(1)$ extra space.

Hash set is faster; sort is space-optimal."

PHASE 4 — CLEAN CODE

```

class _217_ContainsDuplicate:
    def containsDuplicate(self, nums: List[int]) -> bool:
        seen = set()
        for num in nums:
            if num in seen:
                return True
            seen.add(num)
        return False

```

PHASE 5 — TEST & DEBUG

`nums=[1,2,3,1]`: seen grows to `{1,2,3}`, then 1 hits → True ✓

`nums=[1,2,3,4]`: seen=`{1,2,3,4}`, loop ends → False ✓

`nums=[1]`: seen=`{1}`, loop ends → False ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$ for the set.

One-liner alternative: `return len(nums) != len(set(nums))` – same complexity,

builds the full set before checking. The explicit loop returns early on first duplicate.

For adversarial inputs (duplicate at the end) they're the same; for duplicates early,

the loop wins. In an interview I'd use the explicit loop and mention the one-liner.

Given more time I'd ask: what if the array is sorted? Check adjacent pairs – $O(n)$ time, $O(1)$ space."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def containsDuplicate(self, nums: list[int]) -> bool:
        return len(nums) != len(set(nums))
```

• LeetDoocs

```
class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        return any(a == b for a, b in pairwise(sorted(nums)))
```

• SimplyLeet

```
# Function to determine if there are any duplicates in the list.
def contains_duplicate(nums):
    # Create a set to keep track of numbers seen so far.
    seen = set()
    # Iterate over each number in the list.
    for num in nums:
        # If num is already in seen, duplicate found.
        if num in seen:
            return True
        # Add num to the set.

        seen.add(num)
    # No duplicates found.
    return False

# Example usage:
if __name__ == "__main__":
    nums = [1, 2, 3, 1]
    print(contains_duplicate(nums)) # Expected output: True
```

• LC.ca

```

class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        return len(set(nums)) < len(nums)

#####

class Solution(object):
    def containsDuplicate(self, nums):
        """
        :type nums: List[int]

        :rtype: bool
        """
        nums.sort()
        for i in range(0, len(nums) - 1):
            if nums[i] == nums[i + 1]:
                return True
        return False

```

◆ Quick Review

Key Insights

- A hash set can be used to keep track of seen numbers.
- Iterating once through the array provides an efficient solution.
- If a number is already in the set, a duplicate is found.
- Alternative approaches like sorting have a higher time complexity ($O(n \log n)$).

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

The solution uses a hash set to track the numbers seen while iterating through the array. For each number in the array: Check if it exists in the hash set. If it does, return true immediately as a duplicate is found. Otherwise, add the number to the hash set. If the iteration completes without finding any duplicates, return false. This approach efficiently determines if any duplicates exist in a single pass.

Sorting

□ #242 Valid Anagram

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sorting
Lists: Grind 169

Problem

Given two strings *s* and *t*, return true if *t* is an anagram of *s*, and false otherwise.

Example 1:

Input: *s* = "anagram", *t* = "nagaram"

Output: true

Example 2:

Input: *s* = "rat", *t* = "car"

Output: false

Constraints:

- $1 \leq s.length, t.length \leq 5 * 10^4$
- *s* and *t* consist of lowercase English letters.

Follow up: What if the inputs contain Unicode characters? How would you adapt your solution to such a case?

My LeetCode Solution

• My LeetCode Solution

```
# Valid Anagram
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t):
            return False

        return Counter(s) == Counter(t)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# t is an anagram of s if t contains exactly the same characters
# with the same frequencies, just in a different order. Right?
# Both strings consist of only lowercase English letters – so 26 possible chars."
#
# "Quick checks:
# If lengths differ they can't be anagrams – I'll short-circuit on that.
# Follow-up asks about Unicode – I'll address that in Phase 6."
#
# "'anagram'/'nagaram': same chars, same counts -> true ✓
# 'rat'/'car': r,a,t vs c,a,r – 't' vs 'c' mismatch -> false ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Sort both strings and compare character by character.
# Equal sorted forms iff anagrams.
# Simple  $O(n \log n)$ ; counting frequencies in one pass is  $O(n)$ .
# Time:  $O(n \log n)$ . Space:  $O(n)$  for sorted copies.
# Should I go ahead and optimize?"
```

```
class _242_ValidAnagram_Brute:
    def isAnagram(self, s: str, t: str) -> bool:
        return sorted(s) == sorted(t)
```

PHASE 3 — OPTIMIZE

```
# "Instead of sorting, count character frequencies.
# Since only lowercase letters exist, a 26-slot array suffices –  $O(1)$  space.
# Increment for each char in s, decrement for each char in t.
# If all slots are zero at the end, they're anagrams.
#
# Time:  $O(n)$ . Space:  $O(1)$  – fixed 26-slot array."
```


PHASE 4 — CLEAN CODE

```
class _242_ValidAnagram:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t):
            return False
        freq = [0] * 26
        for c in s:
            freq[ord(c) - ord('a')] += 1
        for c in t:
            freq[ord(c) - ord('a')] -= 1
        return all(x == 0 for x in freq)
```

PHASE 5 — TEST & DEBUG

```
# s='anagram', t='nagaram': lengths equal.
# After s: a=3,n=1,g=1,r=1,m=1. After t: all zeroed. → True ✓
# s='rat', t='car': r+1,a+1,t+1 then c-1,a-1,r-1 → t=1,c=-1 ≠ 0 → False ✓
# s='a', t='ab': len mismatch → False immediately ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(1) – 26-slot array is a constant.
# Follow-up: Unicode characters → the 26-slot array breaks.
# Use a Counter (hash map) instead – O(n) time, O(k) space where k is distinct
chars.
# return Counter(s) == Counter(t)
# Handles any alphabet. In an interview, mention this unprompted."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t):
            return False

        count = collections.Counter(s)
        count.subtract(collections.Counter(t))
        return all(freq == 0 for freq in count.values())
```

• LeetDoocs

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t):
            return False
        cnt = Counter(s)
        for c in t:
            cnt[c] -= 1
            if cnt[c] < 0:
                return False
        return True
```

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        return Counter(s) == Counter(t)
```

• SimplyLeet

Python solution for Valid Anagram

```
def isAnagram(s: str, t: str) -> bool:
    # Check if lengths differ; if so, they can't be anagrams
    if len(s) != len(t):
        return False

    # Create a dictionary to count frequency of each character in s
    char_count = {}
    for char in s:

        # Increase the count of the character
        char_count[char] = char_count.get(char, 0) + 1

    # Decrease the count for each character in t
    for char in t:
        if char not in char_count or char_count[char] == 0:
            # Character not found or count mismatch, not an anagram
            return False
        char_count[char] -= 1
```

```
# If all counts are zero, it's a valid anagram
return True

# Example usage:
print(isAnagram("anagram", "nagaram")) # Output: True
print(isAnagram("rat", "car"))          # Output: False
```

• LC.ca

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t):
            return False
        cnt = Counter(s)
        for c in t:
            cnt[c] -= 1
            if cnt[c] < 0:
                return False
        return True
```

◆ Quick Review

Key Insights

- If *s* and *t* have different lengths, they cannot be anagrams.
- An anagram requires exactly the same character frequency for both strings.
- Using a hash table (or frequency array when limited to 26 lowercase letters) provides an efficient solution.
- An alternative approach is to sort both strings and compare them.

- For Unicode inputs, a hash table is more adaptable than a fixed-size frequency array.

Space & Time

Time Complexity: $O(n)$, where n is the length of the strings (using a hash table for counting).

Space Complexity: $O(1)$ if we use a fixed-size array (for 26 letters), otherwise $O(n)$ for a hash table with a larger character set.

Solution

We use a hash table (dictionary) to count the frequency of each character in the first string s . Then, we iterate over string t and decrement the count for each character. If at any point a character count goes negative or a character is missing in the hash table, t cannot be an anagram of s . Finally, if all counts return to zero, the strings are anagrams. This approach is efficient and handles the problem in linear time.

Sorting

□ ★ #252 Meeting Rooms ★

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sorting

Lists: ★ Premium | Grind 169 | Premium 98

Problem

Given an array of meeting time intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, determine if a person could attend all meetings.

Example 1:

```
Input: intervals = [[0,30],[5,10],[15,20]]  
Output: false
```

Example 2:

```
Input: intervals = [[7,10],[2,4]]  
Output: true
```

Constraints:

- $0 \leq \text{intervals.length} \leq 104$
- $\text{intervals}[i].\text{length} == 2$
- $0 \leq \text{start}_i < \text{end}_i \leq 106$

My LeetCode Solution

• My LeetCode Solution

```
# Meeting Rooms
class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        intervals.sort()
        for i in range(1, len(intervals)):
            if intervals[i-1][1] > intervals[i][0]:
                return False

        return True
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "We have a list of meetings [start, end]. We need to check if one person
# can attend ALL of them — meaning no two meetings overlap.
# The problem says meetings ending at t and starting at t do NOT overlap. Right?"
#
# "Quick questions:
# Can intervals be empty? Yes — 0 meetings, trivially true.
# Are start and end guaranteed start < end? Yes, constraints say 0 <= start < end."
#
# "[[0,30],[5,10],[15,20]]: 0-30 overlaps with 5-10 → false ✓
# [[7,10],[2,4]]: 2-4 ends before 7-10 starts → true ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Compare every pair of meetings [start_i,end_i] and [start_j,end_j].
# Overlap exists if start_i < end_j and start_j < end_i.
# O(n^2) pairs; sorting by start time enables one linear scan.
# Time: O(n^2). Space: O(1).
# Should I go ahead and optimize?"
```

```
class _252_MeetingRooms_Brute:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        for i in range(len(intervals)):
            for j in range(i + 1, len(intervals)):
                a, b = intervals[i]
                c, d = intervals[j]
                if a < d and c < b:
                    return False

        return True
```

PHASE 3 — OPTIMIZE

```
# "Sort by start time. Then a single pass is enough:
# if any meeting ends after the next one starts, they overlap."
```

We only need to check consecutive pairs after sorting – $O(n \log n)$."

PHASE 4 — CLEAN CODE

```
class _252_MeetingRooms:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        intervals.sort()
        for prev, curr in pairwise(intervals):
            if prev[1] > curr[0]:
                return False
        return True
```

PHASE 5 — TEST & DEBUG

```
# [[0,30],[5,10],[15,20]] -> sorted: same. prev=[0,30],curr=[5,10]: 30>5 -> False ✓
# [[7,10],[2,4]] -> sorted: [[2,4],[7,10]]. prev=[2,4],curr=[7,10]: 4>7? No -> True ✓
# []: pairwise on empty -> no iterations -> True ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n \log n)$  for sort,  $O(n)$  for the pass. Space  $O(1)$  extra.
# pairwise is Python 3.10+; fallback: for i in range(1, len(intervals)).
# Given more time I'd ask: what's the minimum number of rooms needed?
# That's Meeting Rooms II (#253) – use a min-heap of end times,  $O(n \log n)$ ."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def canAttendMeetings(self, intervals: list[list[int]]) -> bool:
        intervals.sort()

        for i in range(1, len(intervals)):
            if intervals[i - 1][1] > intervals[i][0]:
                return False

        return True
```

• LeetDoocs

```
class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        intervals.sort()
        return all(a[1] <= b[0] for a, b in pairwise(intervals))
```

• SimplyLeet


```
# Python solution for checking if one can attend all meetings

def canAttendMeetings(intervals):
    # Sort intervals based on start time
    intervals.sort(key=lambda x: x[0])
    # Iterate through intervals to find any overlap
    for i in range(1, len(intervals)):
        # Check if the previous meeting ends after the current meeting starts
        if intervals[i-1][1] > intervals[i][0]:
            return False

    return True

# Example usage:
print(canAttendMeetings([[0, 30], [5, 10], [15, 20]])) # Expected output:
False
print(canAttendMeetings([[7, 10], [2, 4]]))           # Expected output:
True
```

• LC.ca

```
class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        intervals.sort()
        return all(a[1] <= b[0] for a, b in pairwise(intervals))

#####

from itertools import pairwise
class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:

        intervals.sort(key=lambda x: x[0])
        return not any(a[1] > b[0] for a,b in pairwise(intervals))
        # any is True, then conflict found

#####

class Solution:
    def canAttendMeetings(self, intervals: List[List[int]]) -> bool:
        intervals.sort(key=lambda x: x[0])
        for i in range(len(intervals) - 1):
```

```

        if intervals[i][1] > intervals[i + 1][0]:
            return False
        return True

#####

# Definition for an interval.
# class Interval(object):
# def __init__(self, s=0, e=0):
# self.start = s

# self.end = e

class Solution(object):
    def canAttendMeetings(self, intervals):
        """
        :type intervals: List[Interval]
        :rtype: bool
        """
        intervals = sorted(intervals, key=lambda x: x.start)
        for i in range(1, len(intervals)):

            if intervals[i].start < intervals[i - 1].end:
                return False
        return True

```

◆ Quick Review

Key Insights

- Sort the intervals based on their starting times.
- Compare each interval's end time with the next interval's start time.
- Detect overlap if the end time of one meeting exceeds the start time of the

following meeting.

- If no overlaps are detected, all meetings can be attended.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting the intervals.

Space Complexity: $O(1)$ if sorting is done in-place; otherwise $O(n)$.

Solution

The solution starts by sorting the intervals in ascending order by their start times. After sorting, iterate through the intervals and for each consecutive pair, check if the previous meeting's end time exceeds the next meeting's start time. An overlap indicates that the person cannot attend all meetings. The primary data structure used is an array (or list) for holding the intervals. Sorting makes it easier to compare adjacent intervals for potential overlaps.

Sorting

□ ★ #1086 Largest Unique Number ★

EAS[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Sorting
Lists: ★ Premium | Premium 98

Problem

Given an integer array `nums`, return the largest integer that only occurs once. If no integer occurs once, return `-1`.

Example 1:

Input: `nums = [5,7,3,9,4,9,8,3,1]`

Output: `8`

Explanation: The maximum integer in the array is 9 but it is repeated. The number 8 occurs only once, so it is the answer.

Example 2:

Input: `nums = [9,9,8,8]`

Output: `-1`

Explanation: There is no number that occurs only once.

Constraints:

- $1 \leq \text{nums.length} \leq 2000$
- $0 \leq \text{nums}[i] \leq 1000$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Return the largest integer that appears exactly once. If none, return -1. Right?
# Values are 0-1000, array length up to 2000."
#
# "[5,7,3,9,4,9,8,3,1]: 9 appears twice, 8 appears once → 8 ✓
# [9,9,8,8]: no unique element → -1 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# Sort the array descending. Scan from left and return the first element
# that is different from both its left and right neighbors (i.e. appears exactly
# once).
# Time: O(n log n). Space: O(1) in-place sort. Should I go ahead and optimize?"
class _1086_LargestUniqueNumber_Brute:
    def largestUniqueNumber(self, nums: List[int]) -> int:
        nums.sort(reverse=True)
        for i in range(len(nums)):
            left_same = (i > 0 and nums[i] == nums[i - 1])
            right_same = (i < len(nums) - 1 and nums[i] == nums[i + 1])
            if not left_same and not right_same:
                return nums[i]
        return -1
```

PHASE 3 — OPTIMIZE

```
# "Count frequencies with Counter. Walk unique keys, track max where count==1.
# O(n) time, O(n) space."
```

PHASE 4 — CLEAN CODE

```
class _1086_LargestUniqueNumber:
    def largestUniqueNumber(self, nums: List[int]) -> int:
        freq = Counter(nums)
        result = -1
        for num, count in freq.items():
            if count == 1 and num > result:
                result = num
        return result
```

PHASE 5 — TEST & DEBUG

```
# [5,7,3,9,4,9,8,3,1]: freq={5:1,7:1,3:2,9:2,4:1,8:1,1:1}
# Unique nums: 5,7,4,8,1. Max = 8 ✓
# [9,9,8,8]: no count==1 entries → result stays -1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(n).
# Alternative: sort descending, scan until you find a non-duplicate – O(n log n)."
```

Counter approach is cleaner. Given more time I'd ask: what if we want the k largest
 # unique numbers? Collect all uniques, sort descending, take first k."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def largestUniqueNumber(self, nums: list[int]) -> int:
        return max([num for num, freq in collections.Counter(nums).items()
                     if freq == 1], default=-1)
```

• LeetDoocs

```
class Solution:
    def largestUniqueNumber(self, nums: List[int]) -> int:
        cnt = Counter(nums)
        return max((x for x, v in cnt.items() if v == 1), default=-1)
```

```
class Solution {
    public int largestUniqueNumber(int[] nums) {
        int[] cnt = new int[1001];
        for (int x : nums) {
            ++cnt[x];
        }
        for (int x = 1000; x >= 0; --x) {
            if (cnt[x] == 1) {
                return x;
            }
        }
    }
}
```

```
    }
    return -1;
}
```

```

class Solution {
public:
    int largestUniqueNumber(vector<int>& nums) {
        int cnt[1001]{};
        for (int& x : nums) {
            ++cnt[x];
        }
        for (int x = 1000; ~x; --x) {
            if (cnt[x] == 1) {
                return x;
            }
        }
        return -1;
    }
};

```

```

func largestUniqueNumber(nums []int) int {
    cnt := [1001]int{}
    for _, x := range nums {
        cnt[x]++
    }
    for x := 1000; x >= 0; x-- {
        if cnt[x] == 1 {
            return x
        }
    }
}

```

```

    return -1
}

```

```

function largestUniqueNumber(nums: number[]): number {
    const cnt = Array(1001).fill(0);
    for (const x of nums) {
        ++cnt[x];
    }
    for (let x = 1000; x >= 0; --x) {
        if (cnt[x] === 1) {
            return x;
        }
    }
}

```



```

        return -1;
    }

    /**
     * @param {number[]} nums
     * @return {number}
     */
    var largestUniqueNumber = function (nums) {
        const cnt = Array(1001).fill(0);
        for (const x of nums) {
            ++cnt[x];
        }
        for (let x = 1000; x >= 0; --x) {
            if (cnt[x] === 1) {
                return x;
            }
        }
        return -1;
    };

```

• SimplyLeet

```

# Define a function to find the largest unique number
def largestUniqueNumber(nums):
    # Create a dictionary to count the frequency of each number
    frequency = {}
    for num in nums:
        if num in frequency:
            frequency[num] += 1 # Increment frequency count if number exists
        else:
            frequency[num] = 1 # Initialize frequency count for new number

```

```

# Initialize the variable for the maximum unique number
max_unique = -1
# Iterate over the frequency dictionary
for num, count in frequency.items():
    # Check if the number is unique and greater than the current maximum
    unique
    if count == 1 and num > max_unique:
        max_unique = num

return max_unique

```

```

# Example usage:
print(largestUniqueNumber([5,7,3,9,4,9,8,3,1])) # Output: 8

```

• LC.ca

```

class Solution:
    def largestUniqueNumber(self, nums: List[int]) -> int:
        cnt = Counter(nums)
        return next((x for x in range(1000, -1, -1) if cnt[x] == 1), -1)

```

◆ Quick Review

Key Insights

- Use a hash table (or dictionary) to count the frequency of each number.
- Iterate over the hash table to find the numbers that appear exactly once.
- Identify the maximum number among the unique numbers.
- Return -1 if no unique number exists.

Space & Time

Time Complexity: $O(n)$, where n is the number of elements in `nums`.

Space Complexity: $O(n)$, for the hash table storing frequency counts.

Solution

The approach starts by initializing a hash table to keep track of the frequency of each element in the array. Once the frequency count is complete, we traverse the hash table to collect numbers that appear exactly once. Finally, we determine the maximum number among these unique elements. If there is no unique element, we return `-1`. This method is efficient because the frequency count and the search for the maximum unique element both run in linear time relative to the input size.

Sorting

□ #1122 Relative Sort Array

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Sorting · Counting Sort
Lists: Denny Zhang

Problem

Given two arrays `arr1` and `arr2`, the elements of `arr2` are distinct, and all elements in `arr2` are also in `arr1`.

Sort the elements of `arr1` such that the relative ordering of items in `arr1` are the same as in `arr2`. Elements that do not appear in `arr2` should be placed at the end of `arr1` in ascending order.

Example 1:

Input: `arr1 = [2,3,1,3,2,4,6,7,9,2,19]`, `arr2 = [2,1,4,3,9,6]`
Output: `[2,2,2,1,4,3,3,9,6,7,19]`

Example 2:

Input: `arr1 = [28,6,22,8,44,17]`, `arr2 = [22,28,8,6]`
Output: `[22,28,8,6,17,44]`

Constraints:

- $1 \leq \text{arr1.length}, \text{arr2.length} \leq 1000$

- $0 \leq \text{arr1}[i], \text{arr2}[i] \leq 1000$
- All the elements of arr2 are distinct.
- Each arr2[i] is in arr1.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def relativeSortArray(self, arr1: List[int], arr2: List[int]) -> List[int]:
        hmap = Counter(arr1)
        ans = []

        for num in arr2:
            ans.extend([num]*hmap[num])
            del hmap[num]

        for num, count in sorted(hmap.items()):
            ans.extend([num]*count)

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Sort arr1 so elements appear in the order defined by arr2.
# Elements in arr1 not present in arr2 go at the end, sorted ascending. Right?
# arr2 elements are distinct; every arr2 element is guaranteed to exist in arr1."
#
# "arr1=[2,3,1,3,2,4,6,7,9,2,19], arr2=[2,1,4,3,9,6]
# → [2,2,2,1,4,3,3,9,6,7,19]: follow arr2 order for those nums, then 7,19 ascending
✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# Build a rank map from arr2: each element maps to its index (0..len(arr2)-1).
# Elements not in arr2 get rank len(arr2) and are sorted by value as a tiebreaker.
# Then sort arr1 using that composite key."
```

```
# Time:  $O(n \log n)$ . Space:  $O(n)$  for the rank map. Should I go ahead and optimize?"
class _1122_RelativeSortArray_Brute:
    def relativeSortArray(self, arr1: List[int], arr2: List[int]) -> List[int]:
        rank = {val: i for i, val in enumerate(arr2)}
        return sorted(arr1, key=lambda x: (rank[x], 0) if x in rank else (len(arr2),
x))
```

PHASE 3 — OPTIMIZE

"Count arr1 frequencies with Counter.

For each num in arr2, extend result with [num]*freq[num], remove from counter.

Then sort remaining counter items and extend.

Time: $O(n + k \log k)$ where k = elements not in arr2. Space: $O(n)$."

PHASE 4 — CLEAN CODE

```
class _1122_RelativeSortArray:
    def relativeSortArray(self, arr1: List[int], arr2: List[int]) -> List[int]:
        freq = Counter(arr1)
        result = []
        for num in arr2:
            result.extend([num] * freq[num])
            del freq[num]
        for num, count in sorted(freq.items()):
            result.extend([num] * count)
        return result
```

PHASE 5 — TEST & DEBUG

arr1=[2,3,1,3,2,4,6,7,9,2,19], arr2=[2,1,4,3,9,6]

freq={2:3,3:2,1:1,4:1,6:1,7:1,9:1,19:1}

arr2 pass: [2,2,2,1,4,3,3,9,6]. remaining: {7:1,19:1}

sorted remaining → [7,19]. result=[2,2,2,1,4,3,3,9,6,7,19] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n + k \log k)$ where k is the count of elements not in arr2 (at most n).

Overall $O(n \log n)$. Space $O(n)$.

Given more time I'd ask: what if arr2 has elements not in arr1?

The problem guarantees that won't happen, but we'd skip them if it did."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def relativeSortArray(self, arr1: list[int], arr2: list[int]) -> list[int]:
        ans = []
        count = [0] * 1001

        for a in arr1:
            count[a] += 1

        for a in arr2:
            while count[a] > 0:
                ans.append(a)
                count[a] -= 1

        for num in range(1001):
            for _ in range(count[num]):
                ans.append(num)

        return ans

```

• LeetCode

```

class Solution:
    def relativeSortArray(self, arr1: List[int], arr2: List[int]) -> List[int]:
        pos = {x: i for i, x in enumerate(arr2)}
        return sorted(arr1, key=lambda x: pos.get(x, 1000 + x))

```

• SimplyLeet

```

# Python solution using a frequency dictionary

def relativeSortArray(arr1, arr2):
    # Count frequency of each element in arr1
    count = {}
    for num in arr1:
        count[num] = count.get(num, 0) + 1

    result = []
    # Process elements defined in arr2: append each element count times

```



```

for num in arr2:
    if num in count:
        result.extend([num] * count[num])
        # Remove the element from the count dictionary
        del count[num]

# Process remaining numbers: sort them in ascending order and append
remaining = []
for num, freq in count.items():
    remaining.extend([num] * freq)

result.extend(sorted(remaining))

return result

# Example usage:
print(relativeSortArray([2,3,1,3,2,4,6,7,9,2,19], [2,1,4,3,9,6]))

```

• LC.ca

```

class Solution:
    def relativeSortArray(self, arr1: List[int], arr2: List[int]) -> List[int]
    :
        pos = {x: i for i, x in enumerate(arr2)}
        return sorted(arr1, key=lambda x: pos.get(x, 1000 + x))

```

◆ Quick Review

Key Insights

- Use a hash table (or array for counting given the value constraints) to count the occurrences of each element in arr1.
- Iterate through arr2 and for each element, append it to the result based on its count from the hash table.

- Remove the used elements from the hash table.
- Sort the remaining elements (those not in arr2) in ascending order and append them to the result.

Space & Time

Time Complexity: $O(n + m \log m)$ where n is the length of arr1 and m is the number of leftover elements not in arr2. Given that element values are bounded (0 to 1000), a counting sort can make this nearly $O(n)$.

Space Complexity: $O(n)$ for storing the frequency counts and the result array.

Solution

The solution leverages a frequency counter (hash table) to count how many times each element appears in arr1. We then process arr2 to add each number in its given order according to its frequency. Finally, we sort the remaining numbers that are not in arr2 and append them to the result array. This approach ensures that the relative order specified by arr2 is maintained and that the other elements are sorted in ascending order.

Binary Search

Round 1 · High · Easy · 5 questions

Binary Search

□ #35 Search Insert Position

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: Denny Zhang

Problem

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: nums = [1,3,5,6], target = 5
Output: 2

Example 2:

Input: nums = [1,3,5,6], target = 2
Output: 1

Example 3:

Input: nums = [1,3,5,6], target = 7
Output: 4

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-104 \leq \text{nums}[i] \leq 104$
- **nums contains distinct values sorted in ascending order.**
- $-104 \leq \text{target} \leq 104$

My LeetCode Solution

• My LeetCode Solution

```
# Search Insert Position
class Solution:
    def searchInsert(self, nums: List[int], target: int) -> int:
        l = 0
        n = len(nums)

        r = n-1
        while l<=r:
            m = (l+r)//2
            if nums[m] >= target:

                r = m-1
            else:
                l=m+1

        return l
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Sorted array of distinct integers. Return the index of target if found,
# otherwise return the index where it would be inserted to keep order. Right?
# Must run in  $O(\log n)$  – so binary search is required."
#
# "[1,3,5,6], target=5 → index 2 (found) ✓
# [1,3,5,6], target=2 → index 1 (between 1 and 3) ✓
# [1,3,5,6], target=7 → index 4 (after all) ✓"
```

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

Linear scan from left: return the first index where `nums[i] >= target`.

If we exhaust the array without finding such an index, target goes at the end.

Time: $O(n)$. Space: $O(1)$. Should I go ahead and optimize?"

```
class _35_SearchInsertPosition_Brute:
```

```
    def searchInsert(self, nums: List[int], target: int) -> int:
```

```
        for i in range(len(nums)):
```

```
            if nums[i] >= target:
```

```
                return i
```

```
        return len(nums)
```

PHASE 3 — OPTIMIZE

"Standard binary search. The key insight: when the loop ends (`left > right`),

'left' always points to the correct insertion position.

When target is found, return mid immediately.

Otherwise left is where target belongs."

PHASE 4 — CLEAN CODE

```
class _35_SearchInsertPosition:
```

```
    def searchInsert(self, nums: List[int], target: int) -> int:
```

```
        left, right = 0, len(nums) - 1
```

```
        while left <= right:
```

```
            mid = (left + right) // 2
```

```
            if nums[mid] == target:
```

```
                return mid
```

```
            elif target < nums[mid]:
```

```
                right = mid - 1
```

```
            else:
```

```
                left = mid + 1
```

```
        return left
```

PHASE 5 — TEST & DEBUG

[1,3,5,6], target=5: mid=2→5==5 → return 2 ✓

[1,3,5,6], target=2: mid=2→5>2→right=1. mid=0→1<2→left=1. mid=1→3>2→right=0.

left=1>right=0 → return left=1 ✓

[1,3,5,6], target=7: left keeps advancing → return 4 ✓

[1,3,5,6], target=0: right keeps retreating → return 0 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\log n)$. Space $O(1)$.

The 'return left' at the end is the core insight – left always converges

to the leftmost position where target can fit. This same pattern appears

in `lower_bound` in C++ and `bisect_left` in Python.

Given more time I'd ask: what if the array has duplicates?

We'd need to decide: first occurrence or last? That's `bisect_left` vs

`bisect_right`."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def searchInsert(self, nums: list[int], target: int) -> int:
        l = 0
        r = len(nums)

        while l < r:
            m = (l + r) // 2
            if nums[m] == target:
                return m
            if nums[m] < target:

                l = m + 1
            else:
                r = m

        return l
```

• LeetDoocs

```
class Solution:
    def searchInsert(self, nums: List[int], target: int) -> int:
        l, r = 0, len(nums)
        while l < r:
            mid = (l + r) >> 1
            if nums[mid] >= target:
                r = mid
            else:
                l = mid + 1
        return l
```

• SimplyLeet


```
# Python solution using binary search
def searchInsert(nums, target):
    # Initialize low and high pointers for binary search
    low, high = 0, len(nums) - 1
    # Perform binary search until low pointer exceeds high pointer
    while low <= high:
        mid = (low + high) // 2 # Calculate the middle index
        if nums[mid] == target:
            return mid # Return mid if target is found
        elif target < nums[mid]:
            high = mid - 1 # Narrow search on the left side
        else:
            low = mid + 1 # Narrow search on the right side
    # Return low pointer as the insertion index when target is not found
    return low

# Example usage
print(searchInsert([1, 3, 5, 6], 5)) # Output: 2
```

• LC.ca

```
class Solution:
    def searchInsert(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums)
        while left < right:
            mid = (left + right) >> 1
            if nums[mid] >= target:
                right = mid
            else:
                left = mid + 1
        return left
```

◆ Quick Review

Key Insights

- The array is sorted, enabling the use of binary search.

- Binary search achieves a time complexity of $O(\log n)$ which meets the problem constraints.
- When the target is not found during the search, the final low pointer represents the correct insertion index.

Space & Time

Time Complexity: $O(\log n)$ due to binary search.

Space Complexity: $O(1)$ as no extra space is used.

Solution

We use binary search to find the target in the sorted array. Initialize two pointers, low and high. At each iteration, compute the mid index. If the value at mid matches the target, return mid. If the target is less than the mid value, adjust the high pointer; if greater, adjust the low pointer. When the iterations finish without finding the target, return the low pointer, which represents the correct insertion index.

Binary Search

□ #69 Sqrt(x)

EAS[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Binary Search
Lists: Denny Zhang

Problem

Given a non-negative integer x , return the square root of x rounded down to the nearest integer. The returned integer should be non-negative as well.

You must not use any built-in exponent function or operator.

- For example, do not use `pow(x, 0.5)` in c++ or `x ** 0.5` in python.

Example 1:

Input: $x = 4$
Output: 2
Explanation: The square root of 4 is 2, so we return 2.

Example 2:

Input: $x = 8$
Output: 2
Explanation: The square root of 8 is 2.82842..., and since we round it down to the nearest integer, 2 is returned.

Constraints:

- $0 \leq x \leq 2^{31} - 1$

My LeetCode Solution

• My LeetCode Solution

```
# Sqrt(x)
class Solution:
    def mySqrt(self, x: int) -> int:
        if x < 2:
            return x

        l = 0
        r = x

        while l <= r:

            m = (l+r)//2
            if m*m == x:
                return m

            if m*m < x:
                ans = m
                l = m+1

            else:
                r = m-1

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Return floor(sqrt(x)) without using built-in sqrt or ** 0.5. Right?
# x can be 0 – sqrt(0)=0. x can be up to 2^31-1."
#
# "x=4 → 2. x=8 → 2 (floor of 2.828). x=0 → 0. x=1 → 1."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# Linear scan m from 0 upward: keep incrementing while m*m <= x.
# The moment (m+1)*(m+1) > x, we know m is the floor sqrt.
# Time: O(sqrt(x)). Space: O(1). Should I go ahead and optimize?"
```

```
class _69_Sqrt_Brute:
    def mySqrt(self, x: int) -> int:
        m = 0
        while (m + 1) * (m + 1) <= x:
            m += 1
        return m
```

PHASE 3 — OPTIMIZE

```
# "Binary search on the answer space [0, x].
# Find the largest m such that m*m <= x.
# When m*m < x, record m as a candidate and search right.
# When m*m > x, search left. When m*m == x, return immediately.
# Time: O(log x). Space: O(1)."
```

PHASE 4 — CLEAN CODE

```
class _69_Sqrt:
    def mySqrt(self, x: int) -> int:
        if x < 2:
            return x
        left, right = 1, x // 2 # sqrt(x) <= x//2 for x >= 4
        result = 0
        while left <= right:
            mid = left + (right - left) // 2
            if mid * mid == x:
                return mid
            elif mid * mid < x:
                result = mid
                left = mid + 1
            else:
                right = mid - 1
        return result
```

PHASE 5 — TEST & DEBUG

```
# x=4: left=1, right=2. mid=1→1<4→result=1, left=2. mid=2→4==4→return 2 ✓
# x=8: left=1, right=4. mid=2→4<8→result=2, left=3. mid=3→9>8→right=2.
# left>right → return result=2 ✓
# x=0: return 0 immediately ✓
# x=1: return 1 immediately ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(log x). Space O(1).
# Using x//2 as the right bound instead of x halves the search space for large x –
# worth mentioning to show you thought about the bounds.
# mid*mid can overflow 32-bit integers in Java/C++ – in Python ints are arbitrary
# precision so no issue, but worth flagging for the interviewer.
# Given more time I'd ask: what about Newton's method?"
```

$x_{n+1} = (x_n + x/x_n) / 2$ converges quadratically – faster in practice."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def mySqrt(self, x: int) -> int:
        return bisect.bisect_right(range(x + 1), x,
                                   key=lambda m: m * m) - 1
```

• LeetDoocs

```
class Solution:
    def mySqrt(self, x: int) -> int:
        l, r = 0, x
        while l < r:
            mid = (l + r + 1) >> 1
            if mid > x // mid:
                r = mid - 1
            else:
                l = mid
        return l
```

• SimplyLeet

```
# Binary search solution to find the integer square root
def mySqrt(x):
    # Edge case for small numbers
    if x < 2:
        return x

    low, high = 0, x
    ans = 0
    # Binary search loop
    while low <= high:
```

```

mid = (low + high) // 2
# If mid squared equals x, we found the exact square root
if mid * mid == x:
    return mid
# If mid squared is less than x, store mid and search right half
elif mid * mid < x:
    ans = mid # candidate answer
    low = mid + 1
else:
    # If mid squared is more than x, search left half

```

```

        high = mid - 1
    return ans

```

```

# Example usage:
print(mySqrt(4)) # Output: 2
print(mySqrt(8)) # Output: 2

```

• LC.ca

```

class Solution:
    def mySqrt(self, x: int) -> int:
        l, r = 0, x
        while l < r:
            mid = (l + r + 1) >> 1
            if mid * mid > x:
                r = mid - 1
            else:
                l = mid
        return l

```

◆ Quick Review

Key Insights

- The problem can be solved efficiently using binary search.

- Instead of iterating from 0 to x , binary search narrows down the candidate square roots quickly.
- The algorithm should handle edge cases like $x = 0$ and $x = 1$.
- By checking $\text{mid} * \text{mid}$ against x , we can determine if we need to search in the lower or upper half.

Space & Time

Time Complexity: $O(\log x)$

Space Complexity: $O(1)$

Solution

We use a binary search approach to find the integer square root. Initialize two pointers, *low* and *high*, where *low* is set to 0 and *high* is set to x . For each iteration, calculate the *mid* value. If *mid* squared equals x , we have found the square root. If *mid* squared is less than x , record *mid* as a potential answer and move the *low* pointer to $\text{mid} + 1$. Otherwise, move the *high* pointer to $\text{mid} - 1$. Continue the search until *low* exceeds *high*, then return the last recorded candidate.

Binary Search

□ #278 First Bad Version

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Binary Search · Interactive
Lists: Grind 169

Problem

You are a product manager and currently leading a team to develop a new product. Unfortunately, the latest version of your product fails the quality check. Since each version is developed based on the previous version, all the versions after a bad version are also bad.

Suppose you have n versions $[1, 2, \dots, n]$ and you want to find out the first bad one, which causes all the following ones to be bad.

You are given an API `bool isBadVersion(version)` which returns whether version is bad.

Implement a function to find the first bad version. You should minimize the number of calls to the API.

Example 1:

Input: n = 5, bad = 4
 Output: 4
 Explanation:
 call isBadVersion(3) -> false
 call isBadVersion(5) -> true
 call isBadVersion(4) -> true
 Then 4 is the first bad version.

Example 2:

Input: n = 1, bad = 1
 Output: 1

Constraints:

- $1 \leq \text{bad} \leq n \leq 2^{31} - 1$

My LeetCode Solution

• My LeetCode Solution

```
# First Bad Version
# The isBadVersion API is already defined for you.
# def isBadVersion(version: int) -> bool:

class Solution:
    def firstBadVersion(self, n: int) -> int:
        l = 0
        r = n

        while l <= r:

            m = (l+r) // 2
            if isBadVersion(m):
                r = m-1

            else:
                l = m+1

        return l
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Versions 1..n. Once a version is bad, all subsequent are bad.
# Find the first bad version while minimising API calls. Right?
# isBadVersion(k) is given – we don't implement it, just call it."
#
# "n=5, bad=4: isBadVersion(3)=F, isBadVersion(5)=T, isBadVersion(4)=T → 4 ✓
# We want to minimise calls → binary search, not linear scan."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# Linear scan from version 1 to n: call isBadVersion on each.
# The first version that returns True is the answer.
# Time: O(n) API calls – way too many for large n. Space: O(1). Should I go ahead
and optimize?"
```

```
class _278_FirstBadVersion_Brute:
    def firstBadVersion(self, n: int) -> int:
        for v in range(1, n + 1):
            if isBadVersion(v):
                return v
        return n
```

PHASE 3 — OPTIMIZE

```
# "Binary search: if mid is bad, the first bad could be mid or earlier → right = mid
- 1.
# If mid is good, first bad must be after mid → left = mid + 1.
# When loop ends, left = first bad version.
# This never calls isBadVersion more than O(log n) times."
```

PHASE 4 — CLEAN CODE

```
class _278_FirstBadVersion:
    def firstBadVersion(self, n: int) -> int:
        left, right = 1, n
        while left <= right:
            mid = (left + right) // 2
            if isBadVersion(mid):
                right = mid - 1 # might be first bad; keep searching left
            else:
                left = mid + 1 # definitely good; first bad is after mid
        return left
```

PHASE 5 — TEST & DEBUG

```
# n=5, bad=4:
# mid=3→F→left=4. mid=4→T→right=3. left=4>right=3 → return 4 ✓
# n=1, bad=1:
# mid=1→T→right=0. left=1>right=0 → return 1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(log n) calls. Space O(1).
# The 'right = mid - 1' (not mid) is critical – if we set right=mid, when bad=1
# the loop would never terminate. Always shrink the window."
```

Given more time I'd ask: what if the API is expensive (network call)?
Same algorithm – binary search already minimises calls. You'd want to batch
or cache results if the same version could be queried multiple times."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def firstBadVersion(self, n: int) -> int:
        l = 1
        r = n

        while l < r:
            m = (l + r) >> 1
            if isBadVersion(m):
                r = m
            else:
                l = m + 1

        return l
```

• LeetDoocs

```
# The isBadVersion API is already defined for you.
# def isBadVersion(version: int) -> bool:
```

```
class Solution:
    def firstBadVersion(self, n: int) -> int:
        l, r = 1, n
        while l < r:
            mid = (l + r) >> 1
            if isBadVersion(mid):
                r = mid
            else:
                l = mid + 1
        return l
```

• SimplyLeet

```
# Python implementation of First Bad Version
class Solution:
    def firstBadVersion(self, n: int) -> int:
        # Initialize the left pointer to the first version and right pointer to
        # the last version
        left, right = 1, n
        while left < right:
            # Calculate the mid point to avoid potential overflow
            mid = left + (right - left) // 2
            # Call the API isBadVersion to check if mid version is bad
            if isBadVersion(mid):
                # If mid is bad, search the left half including mid
                right = mid
            else:
                # If mid is not bad, search the right half excluding mid
                left = mid + 1
        # When left equals right, we found the first bad version
        return left
```

• LC.ca

```
# The isBadVersion API is already defined for you.
# @param version, an integer
# @return an integer
# def isBadVersion(version):
```

```
class Solution:
    def firstBadVersion(self, n):
        """
        :type n: int

        :rtype: int
        """
        left, right = 1, n
        while left < right:
            mid = (left + right) >> 1
            if isBadVersion(mid):
                right = mid
            else:
                left = mid + 1
        return left
```

◆ Quick Review

Key Insights

- The API returns true for bad versions and all versions after a bad version.
- The problem is essentially about finding a transition point in a sorted sequence which calls for a binary search approach.
- The aim is to minimize the number of calls to `isBadVersion` by leveraging the ordered nature of the versions.

Space & Time

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Solution

We use binary search to efficiently locate the first bad version. Initialize two pointers, `left` and `right`, at 1 and `n` respectively. At each step, compute the mid point and call `isBadVersion(mid)`. If `mid` is bad, it indicates that the first bad version may be `mid` or to its left, so adjust `right` to `mid`. Otherwise, adjust `left` to `mid + 1`. Continue the process until the

pointers converge. The converged pointer will be the first bad version. Key structures include simple integer variables for left, right, and mid, with binary search being the fundamental algorithm to reduce the search space logarithmically.

Binary Search

□ #374 Guess Number Higher or Lower

EAS[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Binary Search · Interactive
Lists: Denny Zhang

Problem

We are playing the Guess Game. The game is as follows:

I pick a number from 1 to n. You have to guess which number I picked (the number I picked stays the same throughout the game).

Every time you guess wrong, I will tell you whether the number I picked is higher or lower than your guess.

You call a pre-defined API `int guess(int num)`, which returns three possible results:

- -1: Your guess is higher than the number I picked (i.e. $\text{num} > \text{pick}$).
- 1: Your guess is lower than the number I picked (i.e. $\text{num} < \text{pick}$).

- 0: your guess is equal to the number I picked (i.e. `num == pick`).

Return the number that I picked.

Example 1:

Input: `n = 10, pick = 6`
Output: 6

Example 2:

Input: `n = 1, pick = 1`
Output: 1

Example 3:

Input: `n = 2, pick = 1`
Output: 1

Constraints:

- $1 \leq n \leq 231 - 1$
- $1 \leq \text{pick} \leq n$

My LeetCode Solution

• My LeetCode Solution

```
# The guess API is already defined for you.
# @param num, your guess
# @return -1 if num is higher than the picked number
# 1 if num is lower than the picked number
# otherwise return 0
# def guess(num: int) -> int:

class Solution:
    def guessNumber(self, n: int) -> int:
        l = 0
```

```

    r = n

    while l<=r:
        m = (l+r)//2
        if guess(m) == 0:
            return m

        elif guess(m) == -1:
            r = m-1

    else:
        l = m+1

    return l

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Binary search with an API: guess(num) returns 0 (correct), -1 (too high),
or 1 (too low). Find the picked number. Right?
Range is 1..n, n up to $2^{31}-1$."

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.
Linear scan from 1 to n: call guess(i) on each candidate.
The first call that returns 0 is the picked number.
Time: $O(n)$ calls. Space: $O(1)$. Should I go ahead and optimize?"

```

class _374_GuessNumber_Brute:
    def guessNumber(self, n: int) -> int:
        for i in range(1, n + 1):
            if guess(i) == 0:
                return i
        return n

```

PHASE 3 — OPTIMIZE

"Binary search: guess(mid). If 0 → found. If -1 → mid too high → right=mid-1.
If 1 → mid too low → left=mid+1. $O(\log n)$ calls."

PHASE 4 — CLEAN CODE

```

class _374_GuessNumber:
    def guessNumber(self, n: int) -> int:
        left, right = 1, n
        while left <= right:

```

```

        mid = left + (right - left) // 2 # avoids overflow
        result = guess(mid)
        if result == 0:
            return mid
        elif result == -1:
            right = mid - 1
        else:
            left = mid + 1
    return left

```

PHASE 5 — TEST & DEBUG

n=10, pick=6: mid=5→1(low)→left=6. mid=8→-1(high)→right=7.

mid=6→0→return 6 ✓

n=1, pick=1: mid=1→0→return 1 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\log n)$. Space $O(1)$."

Using $\text{left} + (\text{right} - \text{left}) // 2$ instead of $(\text{left} + \text{right}) // 2$ avoids integer overflow
in languages with fixed-width integers – worth mentioning even in Python.

Given more time I'd ask: what if guess() had a cost per call?

Binary search already minimises calls to $O(\log n)$ – optimal."

Community Solutions (Python)

• WalkCC

```

# The guess API is already defined for you.
# @param num, your guess
# @return -1 if num is higher than the picked number
# 1 if num is lower than the picked number
# otherwise return 0
# def guess(num: int) -> int:

class Solution:
    def guessNumber(self, n: int) -> int:
        l = 1

```

```

    r = n

    # Find the first guess number that >= the target number
    while l < r:
        m = (l + r) // 2
        if guess(m) <= 0: # -1, 0
            r = m
        else:
            l = m + 1

    return l

```

• LeetDoocs

```

# The guess API is already defined for you.
# @param num, your guess
# @return -1 if num is higher than the picked number
# 1 if num is lower than the picked number
# otherwise return 0
# def guess(num: int) -> int:

class Solution:
    def guessNumber(self, n: int) -> int:

        return bisect.bisect(range(1, n + 1), 0, key=lambda x: -guess(x))

```

• SimplyLeet

```

# Python solution using binary search
class Solution:
    def guessNumber(self, n: int) -> int:
        # Initialize low and high pointers
        low, high = 1, n
        # Continue binary search until low exceeds high
        while low <= high:
            # Calculate midpoint safely to prevent overflow
            mid = low + (high - low) // 2
            # Call the guess API with mid

```

```

        result = guess(mid)
        # If the guess is correct, return mid
        if result == 0:
            return mid
        # If guess was too high, adjust the high pointer
        elif result == -1:
            high = mid - 1
        # If guess was too low, adjust the low pointer
        else:
            low = mid + 1

    # Return low (ideally should never reach here if pick exists in [1, n])
    return low

```

• LC.ca

```

# The guess API is already defined for you.
# @param num, your guess
# @return -1 if num is higher than the picked number
# 1 if num is lower than the picked number
# otherwise return 0
# def guess(num: int) -> int:

class Solution:
    def guessNumber(self, n: int) -> int:

        return bisect.bisect(range(1, n + 1), 0, key=lambda x: -guess(x))

```

◆ Quick Review

Key Insights

- The problem can be effectively solved using binary search since the search space is sorted by nature (1 to n).
- Each API call helps to halve the search space, leading to an efficient solution.

- Be cautious with potential overflow when calculating the mid-point by using $\text{low} + (\text{high} - \text{low}) // 2$.

Space & Time

Time Complexity: $O(\log n)$ since we binary search the range.

Space Complexity: $O(1)$ as we use a constant amount of extra space.

Solution

We use a binary search algorithm to efficiently locate the target number. We initialize two pointers: low (1) and high (n). In each iteration, we compute the mid-point, then call the guess API. Based on the response: If `guess(mid)` returns 0, we have found the number. If it returns -1, it indicates that our guess is higher than the pick, so we update the high pointer to `mid - 1`. If it returns 1, it indicates that our guess is lower than the pick, so we update the low pointer to `mid + 1`. This approach guarantees a logarithmic time solution.

Binary Search

□ #704 Binary Search

EAS[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: Grind 169

Problem

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, then return its index. Otherwise, return `-1`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [-1,0,3,5,9,12]`, `target = 9`
Output: `4`
Explanation: 9 exists in `nums` and its index is 4

Example 2:

Input: `nums = [-1,0,3,5,9,12]`, `target = 2`
Output: `-1`
Explanation: 2 does not exist in `nums` so return `-1`

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-104 < \text{nums}[i], \text{target} < 104$
- All the integers in nums are unique.
- nums is sorted in ascending order.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        l = 0
        n = len(nums)
        r = n-1

        while l<=r:
            m = (l+r)//2
            if nums[m] == target:
                return m

            if nums[m] > target:
                r = m-1

            else:
                l = m+1

        return -1
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Classic binary search: sorted array of distinct integers, find target index.
# Return -1 if not found. Must be O(log n). Right?"
#
# "[-1,0,3,5,9,12], target=9 → 4 ✓
# [-1,0,3,5,9,12], target=2 → -1 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# Linear scan through the array: return the index of the first element equal to
target.
# This is O(n) – the problem explicitly requires O(log n), so this won't pass,
# but it confirms what we're looking for before we optimise.
# Time: O(n). Space: O(1). Should I go ahead and optimize?"
```

```
class _704_BinarySearch_Brute:
    def search(self, nums: List[int], target: int) -> int:
        for i in range(len(nums)):
            if nums[i] == target:
                return i
        return -1
```

PHASE 3 — OPTIMIZE

```
# "Standard binary search. Narrow search space by half each iteration.
# Three outcomes at each step: found, go left, go right."
```

PHASE 4 — CLEAN CODE

```
class _704_BinarySearch:
    def search(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = left + (right - left) // 2
            if nums[mid] == target:
                return mid
            elif nums[mid] > target:
                right = mid - 1
            else:
                left = mid + 1
        return -1
```

PHASE 5 — TEST & DEBUG

```
# nums=[-1,0,3,5,9,12], target=9:
# mid=2→3<9→left=3. mid=4→9==9→return 4 ✓
# target=2: mid=2→3>2→right=1. mid=0→-1<2→left=1. mid=1→0<2→left=2.
# left>right → return -1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(log n). Space O(1).
# This is the template all other binary search problems build on.
# Key invariant: loop condition is left <= right (not <) so single-element
# arrays are handled – the loop runs once and either returns or terminates cleanly.
# Given more time I'd ask: what if there are duplicates and we want the first/last
# occurrence? That's leftmost/rightmost binary search – adjust which boundary
# updates when nums[mid]==target."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def search(self, nums: list[int], target: int) -> int:
        i = bisect.bisect_left(nums, target)
        return -1 if i == len(nums) or nums[i] != target else i
```

• LeetDoocs

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        l, r = 0, len(nums) - 1
        while l < r:
            mid = (l + r) >> 1
            if nums[mid] >= target:
                r = mid
            else:
                l = mid + 1
        return l if nums[l] == target else -1
```

• SimplyLeet

Python code for binary search

```
def search(nums, target):
    # Set initial boundaries for the search
    left, right = 0, len(nums) - 1

    # Loop until the search space is empty
    while left <= right:
        # Calculate middle index to avoid overflow
        mid = left + (right - left) // 2

        # Check if the mid element matches the target
        if nums[mid] == target:
            return mid
        # If target is smaller than mid element, narrow search to left half
        elif target < nums[mid]:
            right = mid - 1
        # Else, narrow search to right half
        else:
            left = mid + 1
```

```
# Target not found, return -1
return -1

# Example usage:
print(search([-1,0,3,5,9,12], 9)) # Expected output: 4
```

• LC.ca

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1
        while left < right:
            mid = (left + right) >> 1
            if nums[mid] >= target:
                right = mid
            else:
                left = mid + 1
        return left if nums[left] == target else -1
```

◆ Quick Review

Key Insights

- Utilize binary search to achieve $O(\log n)$ runtime on a sorted array.
- Use two pointers to maintain the current search region.
- Narrow down the search range by comparing the middle element with the target.
- Return the index when the target is found; otherwise, return -1 if not found.

Space & Time

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Solution

We use an iterative binary search algorithm. We initialize two pointers, left and right, to represent the current segment of the array that could contain the target. At each iteration, we compute the middle index and compare the element there with the target. If they match, we return the index. Otherwise, based on whether the target is smaller or larger than the middle element, we adjust our pointers to narrow the search to either the left or right half of the array. This process continues until the target is found or until the pointers cross, indicating that the target is not present. No extra data structures are used, making the space complexity $O(1)$.

Matrix

Round 1 · High · Easy · 5 questions

Matrix

□ ★ #422 Valid Word Square ★

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix

Lists: ★ Premium | Premium 98

Problem

Given an array of strings `words`, return `true` if it forms a valid word square.

A sequence of strings forms a valid word square if the k th row and column read the same string, where $0 \leq k < \max(\text{numRows}, \text{numColumns})$.

Example 1:

a	b	c	d
b	n	r	t
c	r	m	y
d	t	y	e

Input: words = ["abcd","bnrt","crmy","dtye"]

Output: true

Explanation:

The 1st row and 1st column both read "abcd".

The 2nd row and 2nd column both read "bnrt".

The 3rd row and 3rd column both read "crmy".

The 4th row and 4th column both read "dtye".

Therefore, it is a valid word square.

Example 2:

a	b	c	d
b	n	r	t
c	r	m	
d	t		

Input: words = ["abcd","bnrt","crm","dt"]

Output: true

Explanation:

The 1st row and 1st column both read "abcd".

The 2nd row and 2nd column both read "bnrt".

The 3rd row and 3rd column both read "crm".

The 4th row and 4th column both read "dt".

Therefore, it is a valid word square.

Example 3:

b	a	l	l
a	r	e	a
r	e	a	d
l	a	d	y

Input: words = ["ball","area","read","lady"]

Output: false

Explanation:

The 3rd row reads "read" while the 3rd column reads "lead".

Therefore, it is NOT a valid word square.

Constraints:

- $1 \leq \text{words.length} \leq 500$
- $1 \leq \text{words}[i].\text{length} \leq 500$
- $\text{words}[i]$ consists of only lowercase English letters.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def validWordSquare(self, words: List[str]) -> bool:

        for i, word in enumerate(words):
            for j, c in enumerate(word):
                if j >= len(words) or i >= len(words[j]) or words[j][i] != c:
                    return False

        return True
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "A word square means the kth row and kth column read the same string.
# Equivalently: words[i][j] == words[j][i] for every valid (i,j). Right?
# Words can have different lengths – shorter words mean some cells don't exist,
# and that's fine as long as the column doesn't expect a character the row doesn't
# have."
#
# ["abcd", "bnrt", "crmy", "dtye"]: row0='abcd', col0='abcd' ✓ etc. → true ✓
# ['ball', 'area', 'read', 'lady']: row2='read', col2='lead' → false ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# For each row index r, check whether word matches down column r and across row r
# using direct character comparisons.
# Four nested checks per candidate row – fine for small boards.
# Time: O(n * m * k) where k = len(word). Space: O(1).
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "No algorithmic improvement over O(n*m) – we must check every character.
# The implementation is already optimal. Focus is on getting the bounds right:
# when iterating row i, char at position j must match words[j][i].
# Guard: j < len(words) and i < len(words[j])."
```

PHASE 4 — CLEAN CODE

```
class _422_ValidWordSquare:
    def validWordSquare(self, words: List[str]) -> bool:
        for row, word in enumerate(words):
            for col, char in enumerate(word):
```

```

        if col >= len(words) or row >= len(words[col]) or words[col][row] !=
char:
        return False
    return True

```

PHASE 5 — TEST & DEBUG

```

# words=['abcd','bnrt','crmy','dtye']
# row=0,word='abcd': col=0→words[0][0]='a'=='a'✓. col=1→words[1][0]='b'=='b'✓. etc.
# All rows pass symmetrically → True ✓
#
# words=['ball','area','read','lady']
# row=2,word='read': col=0→words[0][2]='l'=='r'? No → False ✓
#
# words=['abcd','bnrt','crm','dt'] (shorter words)
# row=0,col=3: words[3][0]='d'=='d'✓. row=1,col=2: words[2][1]='r'=='n'? No wait –
# words[2]='crm', words[2][1]='r'. words[1][2]='r' ✓. All pass → True ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n*m). Space O(1).
# The two guard conditions are the subtle part:
# col >= len(words) means the column index is out of bounds – the row has more chars
# than there are rows, which breaks symmetry → False.
# row >= len(words[col]) means the transpose cell doesn't exist → False.
# Given more time I'd ask: what if we want to BUILD a word square from a word list?
# That's a backtracking problem – place words row by row, pruning based on column
prefixes."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def validWordSquare(self, words: list[str]) -> bool:
        for i, word in enumerate(words):
            for j, c in enumerate(word):
                if len(words) <= j or len(words[j]) <= i: # out-of-bounds
                    return False
                if c != words[j][i]:
                    return False
        return True

```

• LeetDoocs


```
class Solution:
    def validWordSquare(self, words: List[str]) -> bool:
        m = len(words)
        for i, w in enumerate(words):
            for j, c in enumerate(w):
                if j >= m or i >= len(words[j]) or c != words[j][i]:
                    return False
        return True
```

• SimplyLeet

```
# Function to check if the provided words form a valid word square
def validWordSquare(words):
    # Iterate over each row with its index
    for i in range(len(words)):
        # Iterate over each character in the current row
        for j in range(len(words[i])):
            # Check if the corresponding column exists
            if j >= len(words) or i >= len(words[j]) or words[i][j] != words[j][i]:
                return False
    return True
```

```
# Example test case
words = ["abcd","bnrt","crmy","dtye"]
print(validWordSquare(words)) # Expected output: True
```

• LC.ca

```
class Solution:
    def validWordSquare(self, words: List[str]) -> bool:
        m = len(words)
        for i, w in enumerate(words):
            for j, c in enumerate(w):
                if j >= m or i >= len(words[j]) or c != words[j][i]:
                    return False
        return True
```

◆ Quick Review

Key Insights

- The k -th row should be equal to the k -th column.
- Not every word is the same length; checks must be performed within bounds.
- Iterating over each character and cross-verifying with the corresponding column character is sufficient.

Space & Time

Time Complexity: $O(N * L)$ where N is the number of words and L is the average length of the words.

Space Complexity: $O(1)$ additional space is used for the checking procedure.

Solution

We iterate through each word (row) and each character (column) in the word. For every character at position (i, j) , we confirm that there is a corresponding word at index j and that its length is greater than i . Then we check that the character at position (j, i) in that word is the same as the current character. If any of these checks fail, we return false. If all checks pass, the words form a valid word square.

Matrix

□ #566 Reshape the Matrix

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix · Simulation
Lists: CodeSignal

Problem

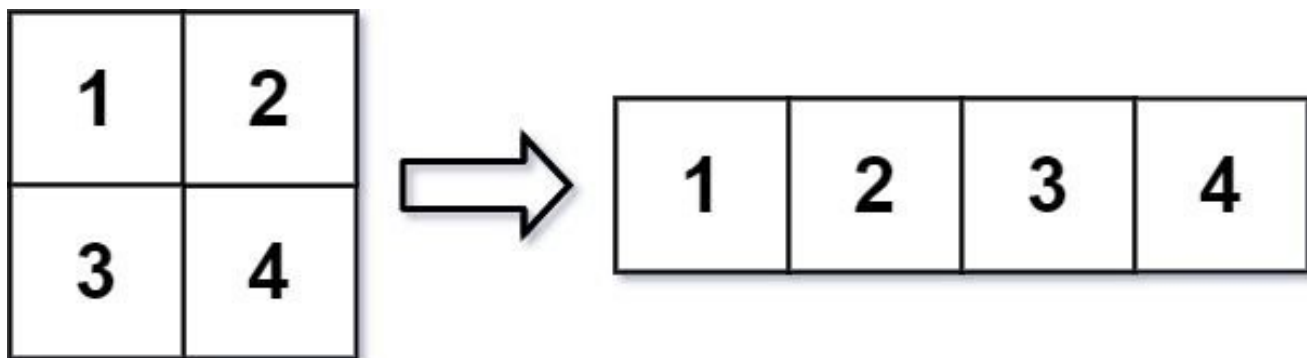
In MATLAB, there is a handy function called `reshape` which can reshape an $m \times n$ matrix into a new one with a different size $r \times c$ keeping its original data.

You are given an $m \times n$ matrix `mat` and two integers `r` and `c` representing the number of rows and the number of columns of the wanted reshaped matrix.

The reshaped matrix should be filled with all the elements of the original matrix in the same row-traversing order as they were.

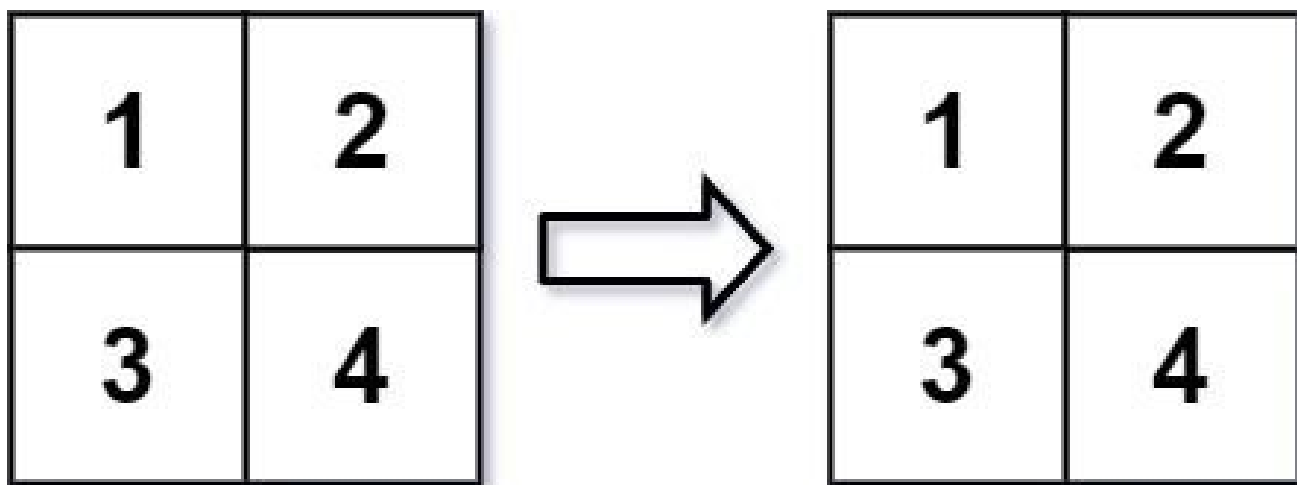
If the reshape operation with given parameters is possible and legal, output the new reshaped matrix; Otherwise, output the original matrix.

Example 1:



Input: `mat = [[1,2],[3,4]]`, `r = 1`, `c = 4`
Output: `[[1,2,3,4]]`

Example 2:



Input: `mat = [[1,2],[3,4]]`, `r = 2`, `c = 4`
Output: `[[1,2],[3,4]]`

Constraints:

- `m == mat.length`
- `n == mat[i].length`
- `1 <= m, n <= 100`
- `-1000 <= mat[i][j] <= 1000`
- `1 <= r, c <= 300`

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def matrixReshape(self, mat: List[List[int]], r: int, c: int) -> List[List[int]]:
        rows = len(mat)
        cols = len(mat[0])

        if rows*cols != r*c:
            return mat

        ans = [[0]*(c) for i in range(r)]

        for i in range(rows*cols):
            ans[i//c][i%c] = mat[i//cols][i%cols]

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

We flatten the original $m \times n$ matrix in row-major order and refill into an $r \times c$ matrix.

If $m \times n \neq r \times c$ the reshape is impossible – return the original matrix unchanged. Right?"

#

"Quick questions:

Values can be negative – fine, we're just moving them.

r and c can be up to 300 even though m, n are up to 100 – just need total elements to match."

#

"[[1,2],[3,4]], $r=1, c=4$: 4 elements total, $1 \times 4 = 4 \rightarrow [[1,2,3,4]]$ ✓

[[1,2],[3,4]], $r=2, c=4$: 4 elements total, $2 \times 4 = 8 \neq 4 \rightarrow$ return original ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Flatten matrix to a 1D list in row-major order, then refill $\text{new_r} \times \text{new_c}$ shape.

Clear and easy to explain in an interview.

Uses $O(m \times n)$ extra list; two-pointer walk avoids allocation.

```
# Time: O(m * n). Space: O(m * n) flat buffer.
# Should I go ahead and optimize?"
```

```
class _566_ReshapeMatrix_Brute:
    def matrixReshape(self, mat: List[List[int]], r: int, c: int) ->
List[List[int]]:
    m, n = len(mat), len(mat[0])
    if m * n != r * c:
        return mat
    flat = [mat[i][j] for i in range(m) for j in range(n)]
    return [flat[i * c:(i + 1) * c] for i in range(r)]
```

PHASE 3 — OPTIMIZE

```
# "Skip the intermediate list – map index directly.
# For flat index k: source is mat[k//n][k%n], destination is ans[k//c][k%c].
# Same O(m*n) time but no extra flat array – just the output matrix."
```

PHASE 4 — CLEAN CODE

```
class _566_ReshapeMatrix:
    def matrixReshape(self, mat: List[List[int]], r: int, c: int) ->
List[List[int]]:
    m, n = len(mat), len(mat[0])
    if m * n != r * c:
        return mat
    result = [[0] * c for _ in range(r)]
    for k in range(m * n):
        result[k // c][k % c] = mat[k // n][k % n]
    return result
```

PHASE 5 — TEST & DEBUG

```
# mat=[[1,2],[3,4]], r=1, c=4: m=2,n=2. 4==4 ✓
# k=0: result[0][0]=mat[0][0]=1. k=1: result[0][1]=mat[0][1]=2.
# k=2: result[0][2]=mat[1][0]=3. k=3: result[0][3]=mat[1][1]=4.
# → [[1,2,3,4]] ✓
# r=2,c=4: 4≠8 → return original ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(m*n). Space O(r*c) for the output matrix – unavoidable.
# The index formula k//n and k%n is the core insight: flatten index → 2D source.
# Given more time I'd ask: what if the matrix is too large to fit in memory?
# Process row by row, streaming elements into the new shape."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def matrixReshape(self, nums: list[list[int]],
                      r: int, c: int) -> list[list[int]]:
        if nums == [] or r * c != len(nums) * len(nums[0]):
            return nums

        ans = [[0 for j in range(c)] for i in range(r)]
        k = 0

        for row in nums:

            for num in row:
                ans[k // c][k % c] = num
                k += 1

        return ans
```

• LeetDoocs

```
class Solution:
    def matrixReshape(self, mat: List[List[int]], r: int, c: int) -> List[List[int]]:
        m, n = len(mat), len(mat[0])
        if m * n != r * c:
            return mat
        ans = [[0] * c for _ in range(r)]
        for i in range(m * n):
            ans[i // c][i % c] = mat[i // n][i % n]
        return ans
```

• SimplyLeet

```
# Function to reshape the matrix
def matrixReshape(mat, r, c):
    # Calculate the dimensions of the original matrix
    m = len(mat)
    n = len(mat[0])

    # Check if reshape is possible
    if m * n != r * c:
        return mat
```



```
# Flatten the original matrix into a list
flat_list = []
for row in mat:
    for num in row:
        flat_list.append(num)

# Create the new matrix by slicing the flat list
new_matrix = []
for i in range(0, len(flat_list), c):
    new_matrix.append(flat_list[i:i+c])

return new_matrix

# Example call:
# print(matrixReshape([[1,2],[3,4]], 1, 4))
```

• LC.ca

```
class Solution:
    def matrixReshape(self, mat: List[List[int]], r: int, c: int) -> List[List[int]]:
        m, n = len(mat), len(mat[0])
        if m * n != r * c:
            return mat
        ans = [[0] * c for _ in range(r)]
        for i in range(m * n):
            ans[i // c][i % c] = mat[i // n][i % n]
        return ans
```

◆ Quick Review

Key Insights

- Check if the reshape operation is possible by comparing the total elements ($m * n$ with $r * c$).
- Flatten the original matrix while preserving the row-traversing order.

- Use the flattened list to build the new matrix row by row.
- If the total number of elements does not match, simply return the original matrix.

Space & Time

Time Complexity: $O(mn)$ as we iterate through all elements of the matrix once.

Space Complexity: $O(mn)$ for the additional list used to store flattened elements (or the new matrix).

Solution

The solution involves first checking whether the reshape is possible by ensuring that the product of the rows and columns of the original matrix equals that of the desired matrix ($r * c$). If not, the original matrix is returned. Otherwise, the matrix is flattened into a one-dimensional list, preserving the row-order traversal. Finally, the flattened list is segmented by the new column size c to form the reshaped matrix. The primary data structures used include a list (or array) for flattening the matrix and then another list (or

vector/array) to hold the final reshaped matrix.

Matrix

□ #766 Toeplitz Matrix

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix
Lists: CodeSignal

Problem

Given an $m \times n$ matrix, return true if the matrix is Toeplitz. Otherwise, return false.

A matrix is Toeplitz if every diagonal from top-left to bottom-right has the same elements.

Example 1:

1	2	3	4
5	1	2	3
9	5	1	2

Input: matrix = [[1,2,3,4],[5,1,2,3],[9,5,1,2]]

Output: true

Explanation:

In the above grid, the diagonals are:

"[9]", "[5, 5]", "[1, 1, 1]", "[2, 2, 2]", "[3, 3]", "[4]".

In each diagonal all elements are the same, so the answer is True.

Example 2:

1	2
2	2

Input: matrix = [[1,2],[2,2]]
Output: false
Explanation:
The diagonal "[1, 2]" has different elements.

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 20$
- $0 \leq \text{matrix}[i][j] \leq 99$

Follow up:

- What if the matrix is stored on disk, and the memory is limited such that you can only load at most one row of the matrix into the memory at once?
- What if the matrix is so large that you can only load up a partial row into the memory at once?

My LeetCode Solution

• My LeetCode Solution

```
# Toeplitz Matrix
class Solution:
    def isToeplitzMatrix(self, matrix: List[List[int]]) -> bool:
        m = len(matrix)
        n = len(matrix[0])
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i-1][j-1] != matrix[i][j]:
                    return False

        return True
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"A Toeplitz matrix has the same value along every top-left to bottom-right diagonal.

We need to check every cell (except the last row and column) equals its down-right neighbour.

Equivalently: $matrix[i][j] == matrix[i+1][j+1]$ for all valid i, j . Right?"

#

"[[1,2,3,4],[5,1,2,3],[9,5,1,2]]: each cell equals its down-right neighbour → true ✓

[[1,2],[2,2]]: $matrix[0][0]=1 \neq matrix[1][1]=2 \rightarrow$ false ✓"

PHASE 2 — BRUTE FORCE (also optimal)**# "Let me start with brute force to lock in the logic.**

No smarter approach than checking every diagonal. The key simplification: each diagonal

is valid iff every adjacent pair on it matches. So just check `matrix[i][j] == matrix[i+1][j+1]`

for every interior cell."

For this input size the brute approach is already optimal – I'll still state complexity clearly.

Time: see analysis above. Space: $O(1)$.**# PHASE 3 — OPTIMIZE****# "Same $O(m \times n)$ is the best we can do – we must read every element.**

The implementation is already optimal. Focus on getting the iteration bounds right:

we iterate `i` up to `m-2` and `j` up to `n-2` (not `m-1`, `n-1`) since we access `[i+1][j+1]`."**# PHASE 4 — CLEAN CODE**`class _766_ToeplitzMatrix:` `def isToeplitzMatrix(self, matrix: List[List[int]]) -> bool:` `rows, cols = len(matrix), len(matrix[0])` `for i in range(rows - 1):` `for j in range(cols - 1):` `if matrix[i][j] != matrix[i + 1][j + 1]:` `return False` `return True`**# PHASE 5 — TEST & DEBUG**# `[[1,2,3,4],[5,1,2,3],[9,5,1,2]]`: `rows=3,cols=4`. Check `(0,0)→(1,1)`: `1==1✓`.# `(0,1)→(1,2)`: `2==2✓`. `(0,2)→(1,3)`: `3==3✓`. `(1,0)→(2,1)`: `5==5✓`. etc. → `True ✓`# `[[1,2],[2,2]]`: `(0,0)→(1,1)`: `1≠2 → False ✓`**# PHASE 6 — COMPLEXITY & FOLLOW-UP****# "Time $O(m \times n)$. Space $O(1)$.**# Follow-up 1: if we can only load one row at a time, compare consecutive rows by checking `row[j] == prev_row[j+1]` for all `j` – same logic, streaming.# Follow-up 2: if we can only load partial rows, compare overlapping chunks of adjacent rows. Both follow-ups keep $O(1)$ extra space.# Given more time I'd ask: can we verify a diagonal in $O(1)$ per cell?

Yes – the current approach already does exactly that."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isToeplitzMatrix(self, matrix: list[list[int]]) -> bool:
        for i in range(len(matrix) - 1):
            for j in range(len(matrix[0]) - 1):
                if matrix[i][j] != matrix[i + 1][j + 1]:
                    return False

        return True
```

• LeetDoocs

```
class Solution:
    def isToeplitzMatrix(self, matrix: List[List[int]]) -> bool:
        m, n = len(matrix), len(matrix[0])
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][j] != matrix[i - 1][j - 1]:
                    return False

        return True
```

• SimplyLeet

```
# Function to determine if the matrix is Toeplitz
def isToeplitzMatrix(matrix):
    # Get the number of rows and columns in the matrix
    rows = len(matrix)
    cols = len(matrix[0])

    # Iterate starting from row 1 and column 1 because first row and column are
    # trivially Toeplitz
    for i in range(1, rows):
        for j in range(1, cols):
            # Compare current element with its top-left neighbor

            if matrix[i][j] != matrix[i - 1][j - 1]:
                return False

    return True

# Example usage:
matrix1 = [[1,2,3,4],[5,1,2,3],[9,5,1,2]]
print(isToeplitzMatrix(matrix1)) # Expected output: True
```

• LC.ca

```
class Solution:
    def isToeplitzMatrix(self, matrix: List[List[int]]) -> bool:
        m, n = len(matrix), len(matrix[0])
        return all(
            matrix[i][j] == matrix[i - 1][j - 1]
            for i in range(1, m)
            for j in range(1, n)
        )
```

◆ Quick Review

Key Insights

- Each element (except those in the first row and first column) must be equal to its top-left neighbor.
- You can validate the matrix by iterating from the second row/column and comparing each element with `matrix[i-1][j-1]`.
- This simple check allows you to determine if every diagonal has the same elements without extra space.
- For streaming data (loading one row at a time), maintain the previous row for comparison.

Space & Time

Time Complexity: $O(m * n)$

Space Complexity: $O(1)$

Solution

The problem is solved by iterating through the matrix starting from the element at position (1, 1). For every element at `matrix[i][j]`, compare it with the element at `matrix[i-1][j-1]`. If any two elements in this pair do not match, the matrix is not Toeplitz. This check ensures that every diagonal from the top-left to bottom-right has identical elements. In terms of auxiliary space, only constant extra space is used, which makes this approach efficient both in time and space.

Matrix

□ #867 Transpose Matrix

EAS

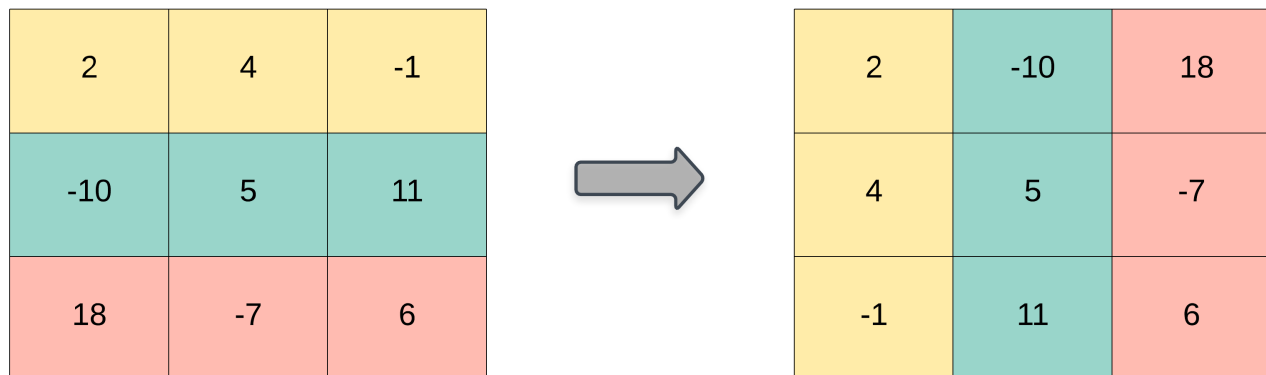
[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix · Simulation
Lists: CodeSignal

Problem

Given a 2D integer array matrix, return the transpose of matrix.

The transpose of a matrix is the matrix flipped over its main diagonal, switching the matrix's row and column indices.



Example 1:

Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [[1,4,7],[2,5,8],[3,6,9]]

Example 2:

Input: matrix = [[1,2,3],[4,5,6]]
Output: [[1,4],[2,5],[3,6]]

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 1000$
- $1 \leq m * n \leq 10^5$
- $-10^9 \leq \text{matrix}[i][j] \leq 10^9$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def transpose(self, matrix: List[List[int]]) -> List[List[int]]:
        rows = len(matrix)
        cols = len(matrix[0])
        ans = [[0]* rows for c in range(cols)]

        for r in range(rows):
            for c in range(cols):
                ans[c][r] = matrix[r][c]

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"The transpose swaps rows and columns: element at (r,c) moves to (c,r).
If the original is m×n, the result is n×m. Return a new matrix – not in-place.
Right?"

"[[1,2,3],[4,5,6],[7,8,9]] → [[1,4,7],[2,5,8],[3,6,9]] ✓"

```
# [[1,2,3],[4,5,6]] (2×3) → [[1,4],[2,5],[3,6]] (3×2) ✓"
```

PHASE 2 — BRUTE FORCE (also optimal)

```
# "Let me start with brute force to lock in the logic."
```

```
# Allocate an n×m result matrix. Set result[c][r] = matrix[r][c] for every (r,c).  
Must visit
```

```
# every element once – is unavoidable."
```

```
# For this input size the brute approach is already optimal – I'll still state  
complexity clearly.
```

```
# Time: O(m×n). Space: O(1).
```

PHASE 3 — OPTIMIZE

```
# "No algorithmic improvement. Focus: allocate result with swapped dimensions (cols  
× rows),
```

```
# then fill with swapped indices. In Python: zip(*matrix) gives transposed rows as  
tuples."
```

PHASE 4 — CLEAN CODE

```
class _867_TransposeMatrix:
```

```
    def transpose(self, matrix: List[List[int]]) -> List[List[int]]:
```

```
        rows, cols = len(matrix), len(matrix[0])
```

```
        result = [[0] * rows for _ in range(cols)]
```

```
        for r in range(rows):
```

```
            for c in range(cols):
```

```
                result[c][r] = matrix[r][c]
```

```
        return result
```

PHASE 5 — TEST & DEBUG

```
# matrix=[[1,2,3],[4,5,6]]: rows=2,cols=3. result is 3×2.
```

```
# r=0,c=0: result[0][0]=1. r=0,c=1: result[1][0]=2. r=0,c=2: result[2][0]=3.
```

```
# r=1,c=0: result[0][1]=4. r=1,c=1: result[1][1]=5. r=1,c=2: result[2][1]=6.
```

```
# result=[[1,4],[2,5],[3,6]] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(m×n). Space O(m×n) for the output."
```

```
# Python one-liner: return [list(row) for row in zip(*matrix)]
```

```
# zip(*matrix) unpacks rows as arguments to zip, which pairs corresponding columns.
```

```
# Elegant and interview-clean – worth mentioning as an alternative.
```

```
# Given more time I'd ask: can we transpose a square matrix in-place?
```

```
# Yes – swap matrix[i][j] and matrix[j][i] for all i < j. O(1) extra space."
```

Community Solutions (Python)

• WalkCC


```
class Solution:
    def transpose(self, A: list[list[int]]) -> list[list[int]]:
        ans = [[0] * len(A) for _ in range(len(A[0]))]

        for i in range(len(A)):
            for j in range(len(A[0])):
                ans[j][i] = A[i][j]

        return ans
```

• LeetDoocs

```
class Solution:
    def transpose(self, matrix: List[List[int]]) -> List[List[int]]:
        return list(zip(*matrix))
```

• SimplyLeet

Python solution with clear comments

```
def transpose(matrix):
    # Number of rows in the original matrix
    num_rows = len(matrix)
    # Number of columns in the original matrix
    num_cols = len(matrix[0])

    # Create a new matrix with dimensions swapped: num_cols x num_rows
    transposed = [[0] * num_rows for _ in range(num_cols)]

    # Iterate through each element in the original matrix
    for i in range(num_rows):
        for j in range(num_cols):
            # Place the element at the swapped index in the transposed matrix
            transposed[j][i] = matrix[i][j]

    return transposed

# Example usage:

matrix = [[1, 2, 3], [4, 5, 6]]
print(transpose(matrix))
```

• LC.ca

```
class Solution:
    def transpose(self, matrix: List[List[int]]) -> List[List[int]]:
        return list(zip(*matrix))
```

◆ Quick Review

Key Insights

- The value at position (i, j) in the original matrix moves to (j, i) in the transposed matrix.
- The transposed matrix dimensions are the reverse of the original: if the original is m x n, then the transposed is n x m.
- A simple double loop is sufficient to swap the indices, ensuring $O(m*n)$ time complexity.
- This problem handles both square and rectangular matrices.

Space & Time

Time Complexity: $O(mn)$

Space Complexity: $O(mn)$ for the new matrix holding the transposed values

Solution

The solution involves iterating over each element of the original matrix and assigning it to the corresponding position in the new matrix where the indexes are swapped. We create the transposed matrix with dimensions based on the number of columns and rows of the original matrix, respectively. The approach leverages basic array manipulation with nested loops to fill in the new matrix.

Matrix

□ #2373 Largest Local Values in a Matrix

EAS[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix
Lists: CodeSignal

Problem

You are given an $n \times n$ integer matrix grid.
Generate an integer matrix `maxLocal` of size $(n - 2) \times (n - 2)$ such that:

- `maxLocal[i][j]` is equal to the largest value of the 3×3 matrix in grid centered around row $i + 1$ and column $j + 1$.

In other words, we want to find the largest value in every contiguous 3×3 matrix in grid.

Return the generated matrix.

Example 1:

9	9	8	1
5	6	2	6
8	2	6	4
6	2	2	2

9	9
8	6

Input: grid = [[9,9,8,1],[5,6,2,6],[8,2,6,4],[6,2,2,2]]

Output: [[9,9],[8,6]]

Explanation: The diagram above shows the original matrix and the generated matrix.

Notice that each value in the generated matrix corresponds to the largest value of a contiguous 3 x 3 matrix in grid.

Example 2:

1	1	1	1	1
1	1	1	1	1
1	1	2	1	1
1	1	1	1	1
1	1	1	1	1

2	2	2
2	2	2
2	2	2

Input: grid = [[1,1,1,1,1],[1,1,1,1,1],[1,1,2,1,1],[1,1,1,1,1],[1,1,1,1,1]]

Output: [[2,2,2],[2,2,2],[2,2,2]]

Explanation: Notice that the 2 is contained within every contiguous 3 x 3 matrix in grid.

Constraints:

- $n == \text{grid.length} == \text{grid}[i].\text{length}$
- $3 \leq n \leq 100$
- $1 \leq \text{grid}[i][j] \leq 100$

My LeetCode Solution

- My LeetCode Solution

```
class Solution:
    def largestLocal(self, grid: List[List[int]]) -> List[List[int]]:
        n = len(grid)
        ans = [[0]*(n-2) for r in range(n-2)]
        for i in range(n-2):
            for j in range(n-2):
                for x in range(i, i+3):
                    for y in range(j, j+3):
                        ans[i][j] = max(ans[i][j], grid[x][y])

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "For each cell (i,j) in the output (n-2)×(n-2) matrix,
# find the max in the 3×3 subgrid of the input centered at (i+1, j+1). Right?
# Input is always n×n square, n >= 3."
#
# "grid=[[9,9,8,1],[5,6,2,6],[8,2,6,4],[6,2,2,2]] (4×4):
# output is 2×2. Top-left 3×3 center=(1,1): max of [[9,9,8],[5,6,2],[8,2,6]]=9.
# Top-right 3×3 center=(1,2): max of [[9,8,1],[6,2,6],[2,6,4]]=9. etc. →
# [[9,9],[8,6]] ✓"
```

PHASE 2 — BRUTE FORCE (also optimal)

```
# "Let me start with brute force to lock in the logic.
# For each output cell, scan its 3×3 window. No smarter approach since we must read
every
# element at least once. = ."
# For this input size the brute approach is already optimal – I'll still state
complexity clearly.
# Time:  $O(n^2 \times 9)$ . Space:  $O(n^2)$ .
```

PHASE 3 — OPTIMIZE

```
# "The 3×3 is fixed size so inner loops are constant – already  $O(n^2)$ .
# Keep the four nested loops clean with descriptive variable names."
```

PHASE 4 — CLEAN CODE

```
class _2373_LargestLocal:
    def largestLocal(self, grid: List[List[int]]) -> List[List[int]]:
        n = len(grid)
        result = [[0] * (n - 2) for _ in range(n - 2)]
        for i in range(n - 2):
            for j in range(n - 2):
                for di in range(3):
```



```

        for dj in range(3):
            result[i][j] = max(result[i][j], grid[i + di][j + dj])
    return result

```

PHASE 5 — TEST & DEBUG

```

# grid=[[9,9,8,1],[5,6,2,6],[8,2,6,4],[6,2,2,2]]: n=4, output 2x2.
# (i=0,j=0): 3x3 from (0,0): max(9,9,8,5,6,2,8,2,6)=9 ✓
# (i=0,j=1): 3x3 from (0,1): max(9,8,1,6,2,6,2,6,4)=9 ✓
# (i=1,j=0): 3x3 from (1,0): max(5,6,2,8,2,6,6,2,2)=8 ✓
# (i=1,j=1): 3x3 from (1,1): max(6,2,6,2,6,4,2,2,2)=6 ✓
# → [[9,9],[8,6]] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(n^2)$  – outer loops  $O((n-2)^2) \approx O(n^2)$ , inner loops constant 9 operations.
# Space  $O(n^2)$  for the output.
# Given more time I'd ask: what if the window size is variable ( $k \times k$  instead of  $3 \times 3$ )?
# For large  $k$ , recomputing the max from scratch is  $O(k^2)$  per cell →  $O(n^2 k^2)$  total.
# A 2D sliding window maximum using monotonic deques brings it down to  $O(n^2)$ ."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def largestLocal(self, grid: list[list[int]]) -> list[list[int]]:
        n = len(grid)
        ans = [[0] * (n - 2) for _ in range(n - 2)]

        for i in range(n - 2):
            for j in range(n - 2):
                for x in range(i, i + 3):
                    for y in range(j, j + 3):
                        ans[i][j] = max(ans[i][j], grid[x][y])

        return ans

```

• LeetDoocs

```

class Solution:
    def largestLocal(self, grid: List[List[int]]) -> List[List[int]]:
        n = len(grid)
        ans = [[0] * (n - 2) for _ in range(n - 2)]
        for i in range(n - 2):
            for j in range(n - 2):
                ans[i][j] = max(
                    grid[x][y] for x in range(i, i + 3) for y in range(j, j +
3)
                )
        return ans

```

• SimplyLeet

Python solution for finding the largest local values in a matrix

```

def largestLocal(grid):
    n = len(grid) # dimension of the grid
    # Initialize the result matrix with zeros, of size (n-2) x (n-2)
    result = [[0] * (n - 2) for _ in range(n - 2)]

    # Loop through each possible top-left index of a 3x3 submatrix
    for i in range(n - 2):
        for j in range(n - 2):

            # Compute the maximum in the current 3x3 submatrix
            local_max = 0 # since grid elements are >= 1, starting with 0 is
safe
            for k in range(3):
                for l in range(3):
                    local_max = max(local_max, grid[i + k][j + l])
            # Store the local maximum in the result matrix
            result[i][j] = local_max
    return result

# Example usage:

```

```

grid = [[9,9,8,1],[5,6,2,6],[8,2,6,4],[6,2,2,2]]
print(largestLocal(grid))

```

• LC.ca

```
class Solution:
    def largestLocal(self, grid: List[List[int]]) -> List[List[int]]:
        n = len(grid)
        ans = [[0] * (n - 2) for _ in range(n - 2)]
        for i in range(n - 2):
            for j in range(n - 2):
                ans[i][j] = max(
                    grid[x][y] for x in range(i, i + 3) for y in range(j, j +
3)
                )
        return ans
```

◆ Quick Review

Key Insights

- The size of the output matrix is $(n - 2) \times (n - 2)$.
- For each valid index (i, j) in the output, examine the 3×3 block in grid starting at $\text{grid}[i][j]$ to $\text{grid}[i+2][j+2]$.
- Since each 3×3 block contains exactly 9 elements, finding the maximum in a block takes constant time.
- Use nested loops to slide over the grid and efficiently compute each local maximum.

Space & Time

Time Complexity: $O(n^2)$ – We iterate over $(n-2) \times (n-2)$ positions, and for each position, we look at 9 elements.

Space Complexity: $O(n^2)$ – The additional space is required for the output matrix.

Solution

The solution involves iterating over each possible 3 x 3 submatrix in grid. For each (i, j) starting index of the submatrix: Examine the nine elements from `grid[i][j]` to `grid[i+2][j+2]`. Compute the maximum value from these elements. Store the maximum in the corresponding position of the result matrix. Data structures used include the result matrix for storing the maximum values. The algorithmic approach involves two nested loops (one for rows and one for columns) to traverse each possible starting position of a 3 x 3 window. The simplicity of the solution comes from the fixed 3x3 block size, which enables constant-time maximum computation for each block.

Trees & BST

Round 1 · High · Easy · 11 questions

Trees & BST

□ #100 Same Tree

EAS[Open on LeetCode](#)

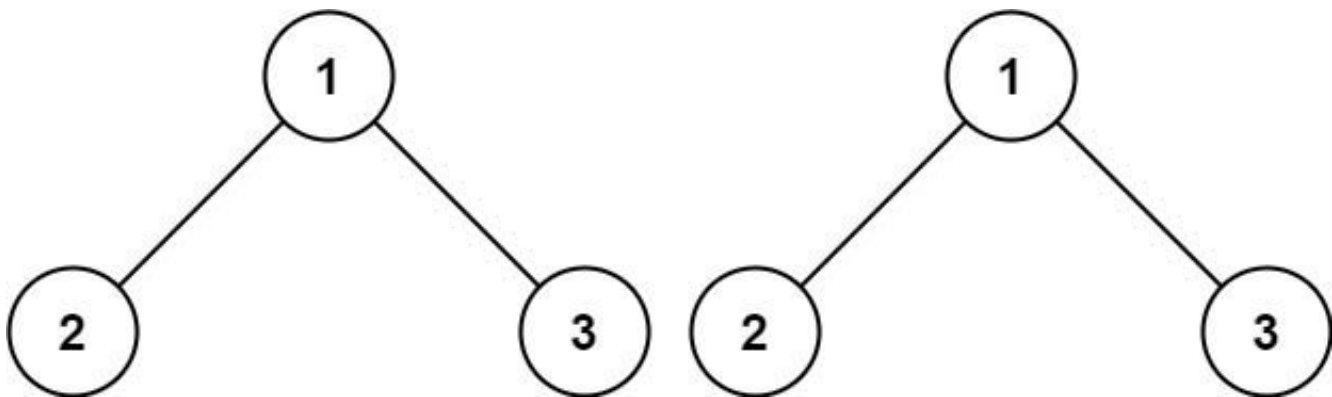
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BFS
Lists: Grind 169

Problem

Given the roots of two binary trees p and q , write a function to check if they are the same or not.

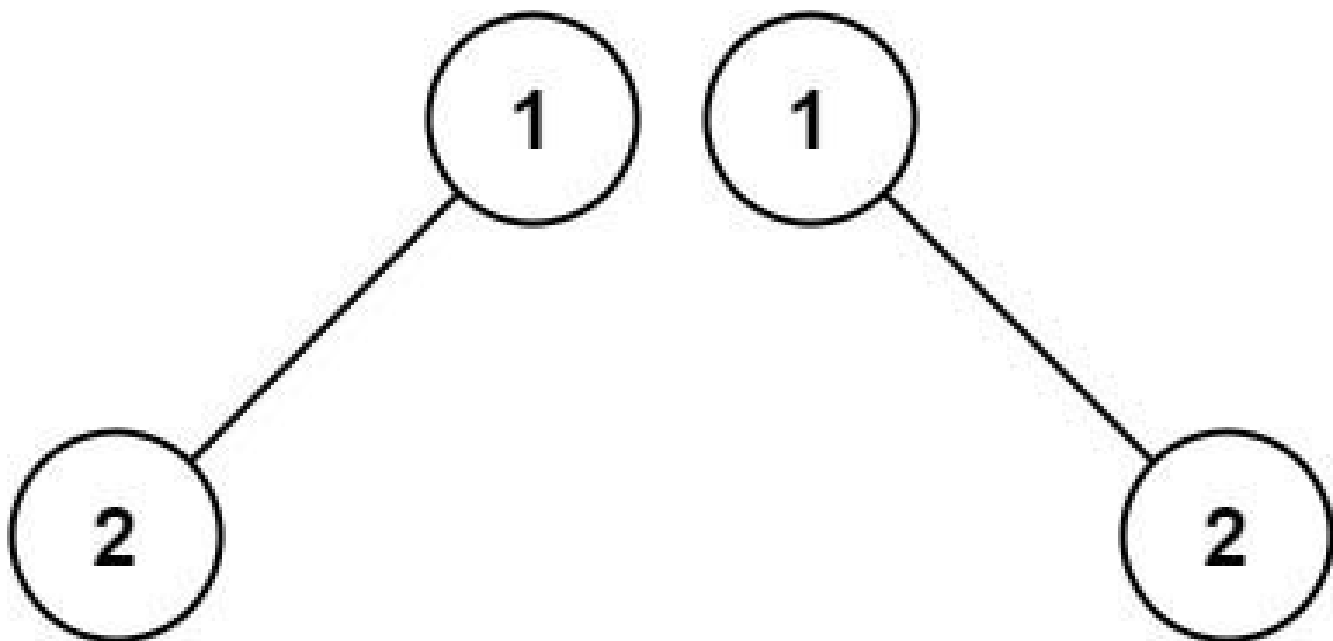
Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

Example 1:



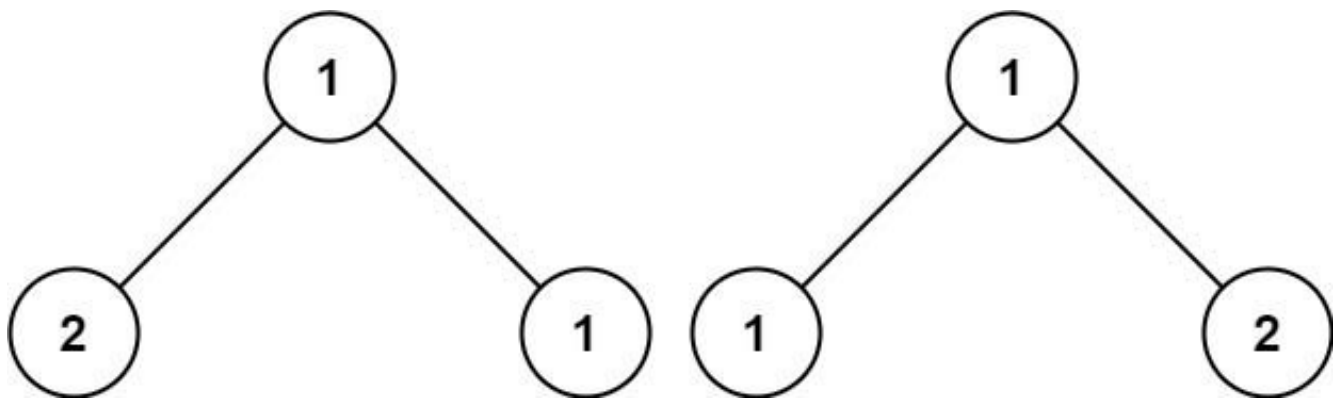
Input: $p = [1,2,3]$, $q = [1,2,3]$
Output: true

Example 2:



Input: $p = [1,2]$, $q = [1,null,2]$
Output: false

Example 3:



Input: $p = [1,2,1]$, $q = [1,1,2]$
Output: false

Constraints:

- The number of nodes in both trees is in the range $[0, 100]$.

- $-104 \leq \text{Node.val} \leq 104$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def isSameTree(self, p: TreeNode | None, q: TreeNode | None) -> bool:
        if not p or not q:
            return p == q
        return (p.val == q.val and
                self.isSameTree(p.left, q.left) and
                self.isSameTree(p.right, q.right))
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Two binary trees are the same if they're structurally identical AND every
# corresponding node has the same value. Both null trees are the same. Right?
# One null and one non-null at the same position -> not the same."
#
# "p=[1,2,3], q=[1,2,3] -> true ✓
# p=[1,2], q=[1,null,2] -> false (structure differs) ✓
# p=[1,2,1], q=[1,1,2] -> false (values differ at depth 1) ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# Serialise both trees with a level-order traversal (including nulls) to lists, then
# compare.
# Same O(n) time and O(n) space as the optimal recursive DFS - no meaningful
# complexity
# improvement available, so we skip the brute class and go straight to the clean
# approach."
```

PHASE 3 — OPTIMIZE

```
# "Recursive DFS is both simple and optimal.
# Base case: if either node is None, both must be None for equality.
# Recursive: values match AND left subtrees match AND right subtrees match.
# Use an inner helper to avoid self. calls."
```

PHASE 4 — CLEAN CODE

```
class _100_SameTree:
    def isSameTree(self, p: Optional['TreeNode'], q: Optional['TreeNode']) -> bool:
        def same(p, q):
            if not p or not q:
```

```

        return p == q
    return p.val == q.val and same(p.left, q.left) and same(p.right,
q.right)
    return same(p, q)

```

PHASE 5 — TEST & DEBUG

```

# p=[1,2,3], q=[1,2,3]:
# same(1,1): vals match. same(2,2)→same(None,None)=True, same(None,None)=True →
True.
# same(3,3)→True. → True ✓
# p=[1,2,None], q=[1,None,2]:
# same(1,1): same(2,None): p=2≠None, q=None → p==q? No → False ✓
# p=None, q=None: not p=True → return p==q=True ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n) – visit every node once. Space O(h) call stack where h is tree height.
# Worst case O(n) for a skewed tree, O(log n) for balanced.
# Iterative alternative: BFS with a queue of node pairs – O(n) time, O(n) space.
# Same complexity but avoids recursion depth limit for very deep trees.
# Given more time I'd ask: what if we only care about structural identity (not
values)?
# Same logic – just remove the val check."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def isSameTree(self, p: TreeNode | None, q: TreeNode | None) -> bool:
        if not p or not q:
            return p == q
        return (p.val == q.val and
                self.isSameTree(p.left, q.left) and
                self.isSameTree(p.right, q.right))

```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSameTree(self, p: Optional[TreeNode], q: Optional[TreeNode]) -> bool
    :
        if p == q:
            return True

        if p is None or q is None or p.val != q.val:
            return False
        return self.isSameTree(p.left, q.left) and self.isSameTree(p.right, q.
right)
```

• SimplyLeet

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

def isSameTree(p, q):
    # If both nodes are None, trees are identical at this branch.
    if not p and not q:
        return True
    # If one of the nodes is None, trees are not identical.
    if not p or not q:
        return False
    # Check if current nodes have the same value.
    if p.val != q.val:
        return False
    # Recursively compare the left subtrees and the right subtrees.
    return isSameTree(p.left, q.left) and isSameTree(p.right, q.right)
```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, x):
#         self.val = x
```

```
# self.left = None
# self.right = None
```

```
class Solution(object):
    def isSameTree(self, p, q):
        """
        :type p: TreeNode
        :type q: TreeNode
        :rtype: bool
        """
```

```
        if not p or not q:
            return p == q # covers: p=None and q!=None, q=None and p!=None, both None
        return p.val == q.val and self.isSameTree(p.left, q.left) and self.isSameTree(p.right, q.right)
```

```
# iteration
class Solution:
    def isSameTree(self, p: TreeNode, q: TreeNode) -> bool:
        if p is None:
            return q is None
```

```
if q is None:
    return p is None

stack1 = [p]
stack2 = [q]

while stack1 and stack2:
    current1 = stack1.pop()
    current2 = stack2.pop()

    if current1 is None and current2 is None:
        continue
    elif current1 is None or current2 is None:
        return False
    elif current1.val != current2.val:
        return False

    stack1.append(current1.left)
    stack2.append(current2.left)

    stack1.append(current1.right)
    stack2.append(current2.right)

return not stack1 and not stack2
```

◆ Quick Review

Key Insights

- Use recursion to compare nodes in both trees.
- If both nodes being compared are null, they are the same.

- If one node is null while the other is not, the trees differ.
- Check if current nodes' values match, then recursively verify the left and right subtrees.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the trees, since every node is visited once.

Space Complexity: $O(n)$ in the worst-case (unbalanced tree), due to recursion call stack usage. In a balanced tree, the space would be $O(\log n)$.

Solution

The problem can be solved using a recursive approach (Depth-First Search) where we compare the current nodes from both trees. If the current nodes are both null, they match; if one is null, then the trees are not identical. For non-null nodes, check if the values are equal. Then, recursively apply the same logic on the left and right children of the nodes. This approach ensures that both the structure and the node values are identical across the two trees.

Trees & BST

□ #101 Symmetric Tree

EAS

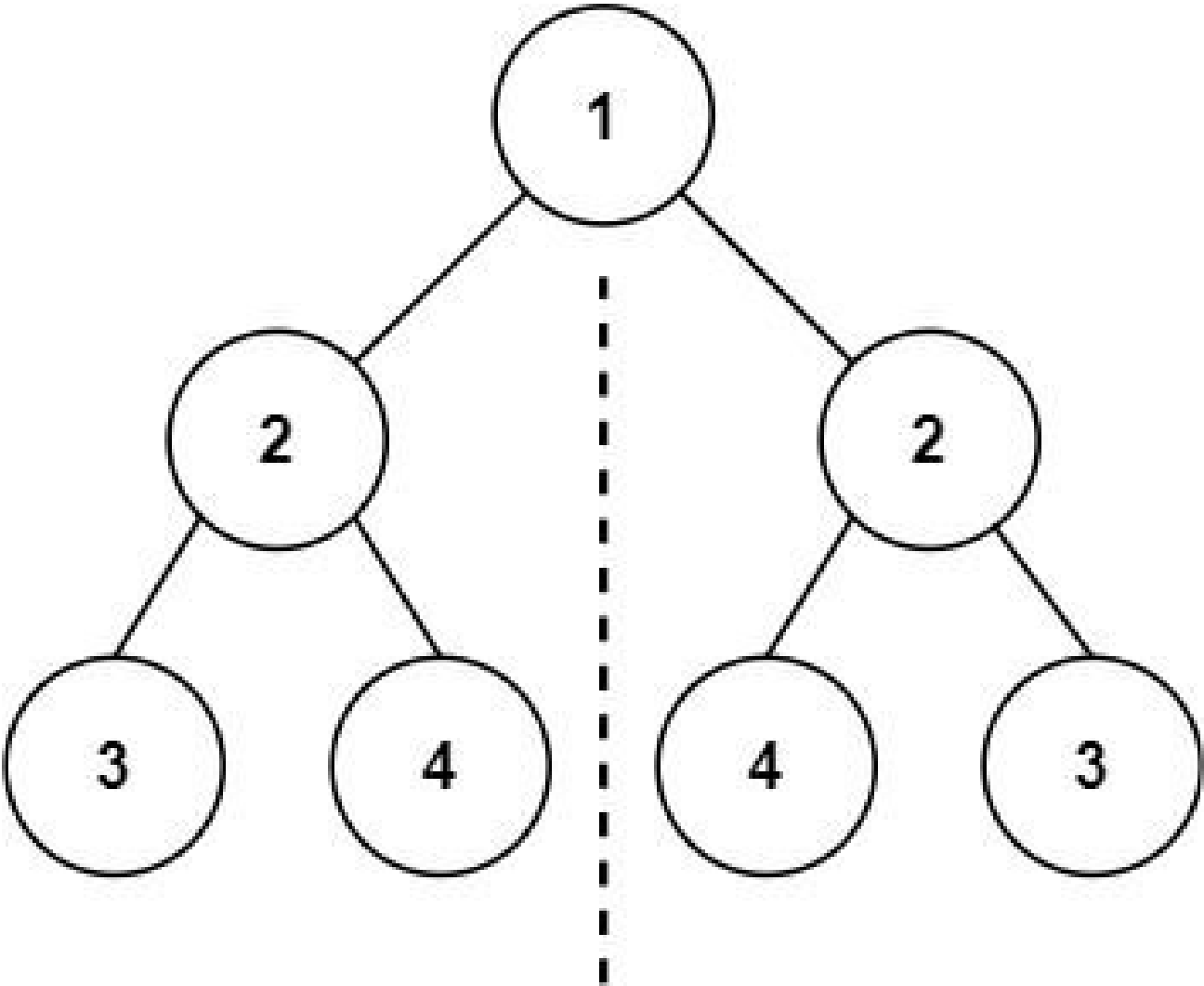
[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BFS
Lists: Grind 169

Problem

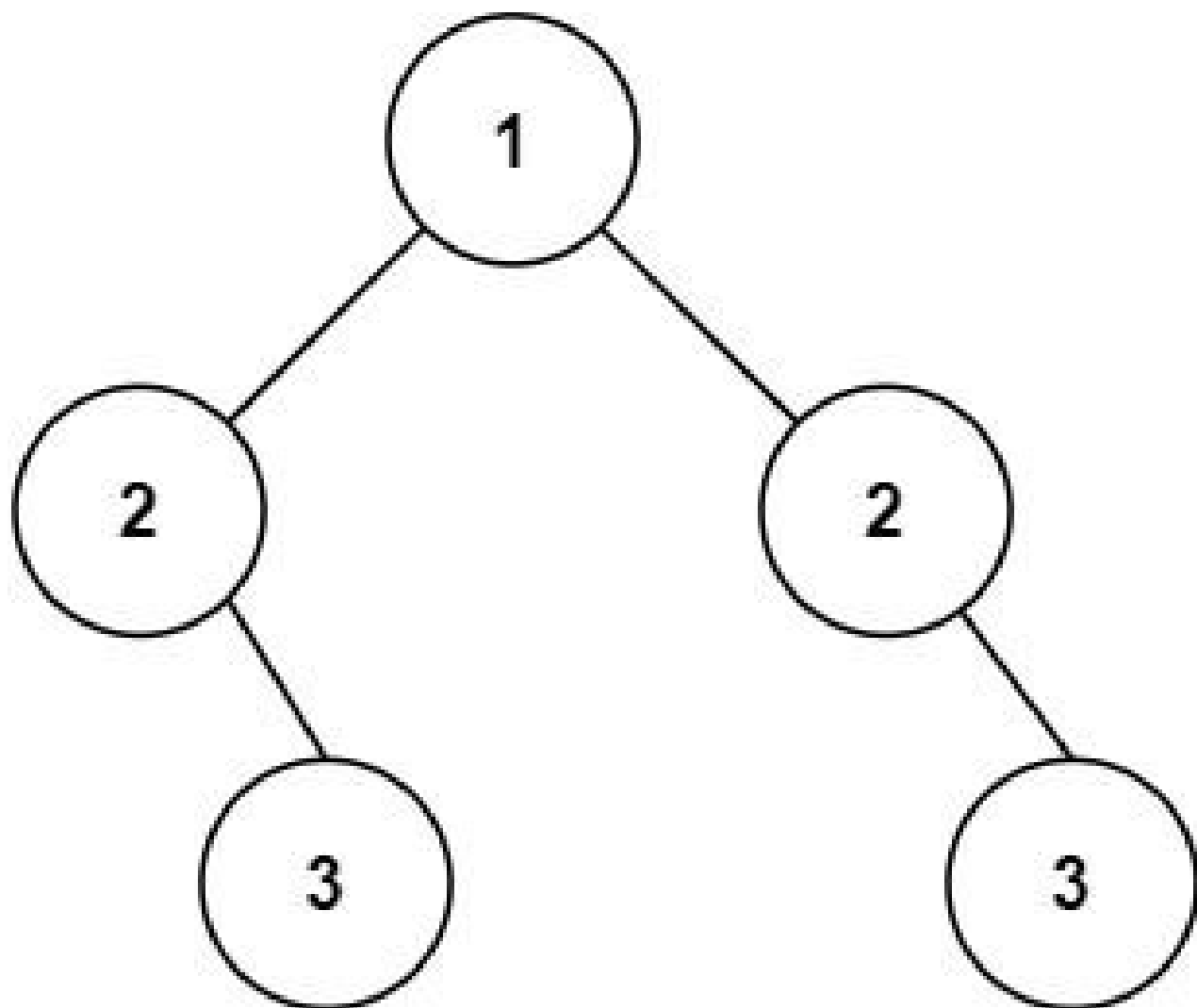
Given the root of a binary tree, check whether it is a mirror of itself (i.e., symmetric around its center).

Example 1:



Input: root = [1,2,2,3,4,4,3]
Output: true

Example 2:



Input: root = [1,2,2,null,3,null,3]

Output: false

Constraints:

- The number of nodes in the tree is in the range [1, 1000].
- $-100 \leq \text{Node.val} \leq 100$

Follow up: Could you solve it both recursively and iteratively?

My LeetCode Solution

• My LeetCode Solution

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSymmetric(self, root: Optional[TreeNode]) -> bool:
        def symmetric(p, q):
            if not p or not q:
                return p == q

            if p.val != q.val:
                return False

            return symmetric(p.left, q.right) and symmetric(p.right, q.left)
        return symmetric(root, root)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"A tree is symmetric if its left and right subtrees are mirror images of each other.

Mirror means: at each level, left child of left subtree matches right child of right subtree,

and right child of left subtree matches left child of right subtree. Right?

A single node is always symmetric."

#

"[1,2,2,3,4,4,3]: left subtree [2,3,4] mirrors right [2,4,3] → true ✓

[1,2,2,null,3,null,3]: left has right child 3, right has right child 3 – not mirrored → false ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

Serialise the left subtree with a pre-order traversal and the right subtree

with the mirror order (right child before left), then compare the two strings.

Same O(n) time and O(n) space as the optimal recursive approach – no improvement

to be had, so we skip the brute class and go straight to the clean solution."

PHASE 3 — OPTIMIZE

"Recursive DFS comparing the tree to itself – left vs right mirrored.

Define an inner helper mirror(p, q):

```
# Base: if either is None, both must be None.
# Recursive: values match AND mirror(p.left, q.right) AND mirror(p.right, q.left).
# Note the cross-comparison: left's left ↔ right's right, left's right ↔ right's left."
```

PHASE 4 — CLEAN CODE

```
class _101_SymmetricTree:
    def isSymmetric(self, root: Optional['TreeNode']) -> bool:
        def mirror(p, q):
            if not p or not q:
                return p == q
            return p.val == q.val and mirror(p.left, q.right) and mirror(p.right,
q.left)
        return mirror(root, root)
```

PHASE 5 — TEST & DEBUG

```
# root=[1,2,2,3,4,4,3]:
# mirror(root,root): vals 1==1.
# mirror(left=2, right=2): 2==2.
# mirror(2.left=3, 2.right=3): 3==3. mirror(None,None)=T. mirror(None,None)=T → T
# mirror(2.right=4, 2.left=4): 4==4. T → T
# mirror(right=2, left=2): symmetric to above → T
# → True ✓
# root=[1,2,2,null,3,null,3]:
# mirror(left=2,right=2): 2==2.
# mirror(2.left=None, 2.right=3): p=None,q=3 → None≠3 → False ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(h) for the call stack.
# Follow-up asks for iterative solution: use a queue of pairs.
# Enqueue (root.left, root.right). Each step dequeue (p,q):
# check p==q==None (skip), p or q None (false), vals differ (false).
# Enqueue (p.left,q.right) and (p.right,q.left). O(n) time, O(n) space.
# The cross-enqueue order is the key – same insight as the recursive version.
# Given more time I'd ask: how does this differ from Same Tree (#100)?
# Same Tree checks identical structure; Symmetric Tree checks mirrored structure –
# the only difference is the cross-comparison in the recursive call."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isSymmetric(self, root: TreeNode | None) -> bool:
        def isSymmetric(p: TreeNode | None, q: TreeNode | None) -> bool:
            if not p or not q:
                return p == q
            return (p.val == q.val and
                    isSymmetric(p.left, q.right) and
                    isSymmetric(p.right, q.left))

        return isSymmetric(root, root)
```

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSymmetric(self, root: Optional[TreeNode]) -> bool:
        def dfs(root1: Optional[TreeNode], root2: Optional[TreeNode]) -> bool:
            if root1 == root2:
                return True
            if root1 is None or root2 is None or root1.val != root2.val:
                return False
            return dfs(root1.left, root2.right) and dfs(root1.right, root2.left)

        return dfs(root.left, root.right)
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val          # current node's value
        self.left = left         # pointer to left child
        self.right = right       # pointer to right child

class Solution:
    def isSymmetric(self, root: TreeNode) -> bool:
        # Helper function to check mirror symmetry between two trees.

        def isMirror(t1: TreeNode, t2: TreeNode) -> bool:
            # Both nodes are null, so they are mirrors.
            if t1 is None and t2 is None:
                return True
            # If one is null or the values do not match, symmetry fails.
            if t1 is None or t2 is None or t1.val != t2.val:
                return False
            # Recursively check mirror symmetry for children:
            # t1's left with t2's right and t1's right with t2's left.
            return isMirror(t1.left, t2.right) and isMirror(t1.right, t2.left)

        # Start recursion comparing the tree with itself.
        return isMirror(root, root)
```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSymmetric(self, root: Optional[TreeNode]) -> bool:
        def dfs(root1, root2):
            if root1 is None and root2 is None:
```

```
        return True
    if root1 is None or root2 is None or root1.val != root2.val:
        return False
    return dfs(root1.left, root2.right) and dfs(root1.right, root2.left)

return dfs(root, root)
```

◆ Quick Review

Key Insights

- Use recursion (or iteration) to compare pairs of nodes.
- For two trees (or two nodes) to be mirror images:
 - Their root values must be the same.
 - The left subtree of one must match the right subtree of the other.
 - The right subtree of one must match the left subtree of the other.
- Alternatively, iterative approaches using a queue can be applied to compare the nodes in pairs.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the tree, since we traverse each node once.

Space Complexity: $O(h)$ for recursive call stack (worst-case $O(n)$ for a skewed tree) or $O(n)$ for the iterative approach using a queue.

Solution

We solve the problem using recursion by defining a helper function that takes two nodes and checks if they are mirrors of each other. The helper function: Returns true if both nodes are null. Returns false if one node is null or if their values differ. Recursively compares the left child of the first node with the right child of the second node, and vice-versa. This method ensures each comparison verifies symmetry at every level. A similar logic can be applied iteratively by using a queue to compare pairs of nodes.

Trees & BST

□ #104 Maximum Depth of Binary Tree

EAS

[Open on LeetCode](#)

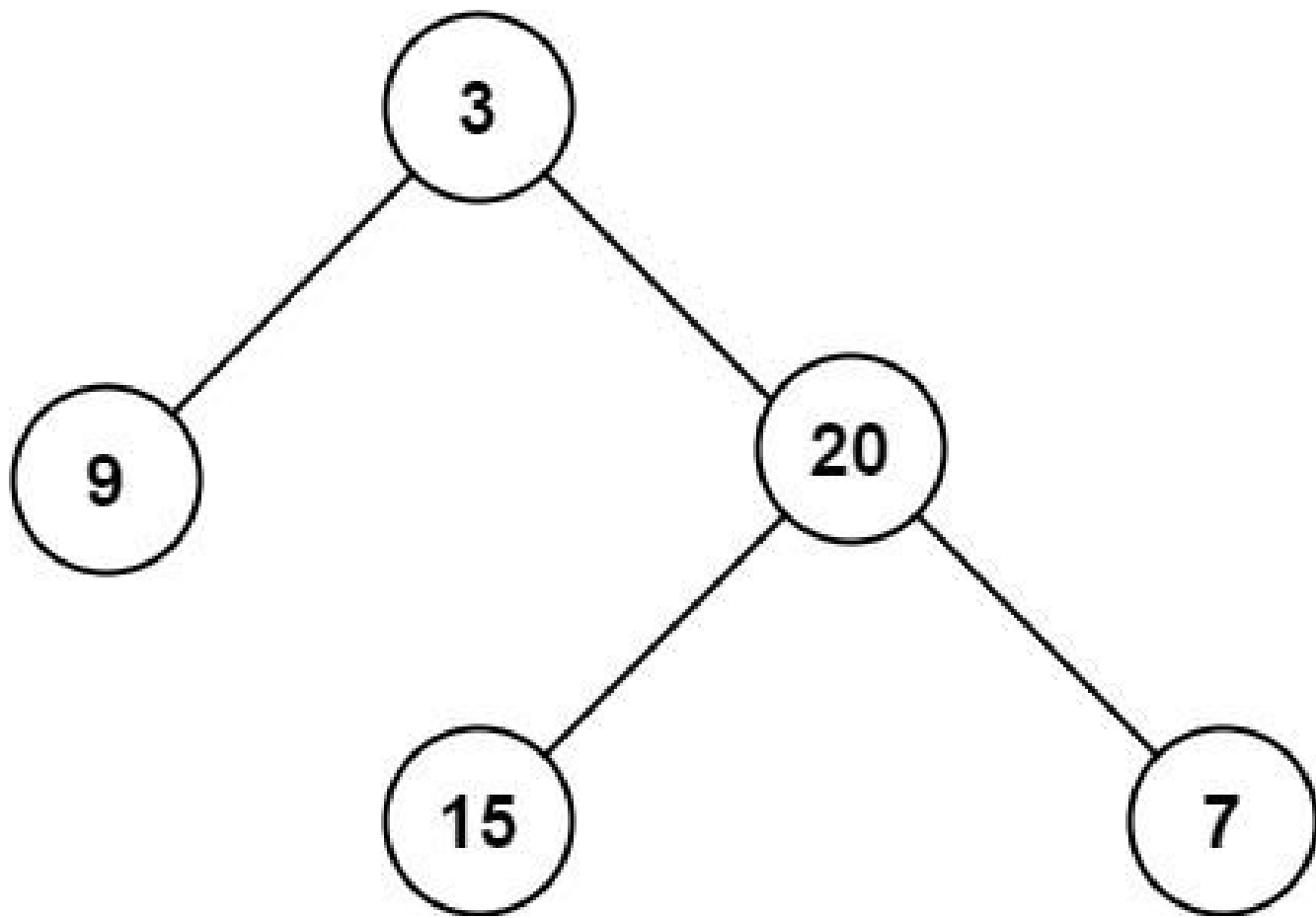
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BFS
Lists: Grind 169

Problem

Given the root of a binary tree, return its maximum depth.

A binary tree's maximum depth is the number of nodes along the longest path from the root node down to the farthest leaf node.

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: 3

Example 2:

Input: root = [1,null,2]
Output: 2

Constraints:

- The number of nodes in the tree is in the range [0, 104].
- $-100 \leq \text{Node.val} \leq 100$

My LeetCode Solution

- My LeetCode Solution

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def maxDepth(self, root: Optional[TreeNode]) -> int:
        def depth(root):
            if not root:
                return 0
            return max(1+ depth(root.left), 1+depth(root.right))
        return depth(root)

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Maximum depth is the number of nodes on the longest root-to-leaf path.
# An empty tree has depth 0. A single node has depth 1. Right?"
#
# "[3,9,20,null,null,15,7]: longest path is 3→20→15 or 3→20→7, 3 nodes → 3 ✓
# [1,null,2]: 1→2, 2 nodes → 2 ✓"

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with the brute force to lock in the logic.
# Iterative BFS: use a queue, process the tree level by level.
# Each time we exhaust one level, increment a depth counter.
# When the queue is empty, the counter equals the maximum depth.
# Time: O(n). Space: O(n) for the queue. Should I go ahead and optimize?"

```

```

class _104_MaxDepth_Brute:
    def maxDepth(self, root: Optional['TreeNode']) -> int:
        if not root:
            return 0
        from collections import deque
        queue = deque([root])
        depth = 0
        while queue:
            depth += 1
            for _ in range(len(queue)):
                node = queue.popleft()
                if node.left:
                    queue.append(node.left)

```

```

        if node.right:
            queue.append(node.right)
    return depth

```

PHASE 3 — OPTIMIZE

"Recursive DFS is cleaner and same complexity.

$\text{depth}(\text{node}) = \max(\text{depth}(\text{left}), \text{depth}(\text{right})) + 1$. Base: empty node $\rightarrow 0$.

Use an inner helper to avoid self. calls."

PHASE 4 — CLEAN CODE

```

class _104_MaxDepth:
    def maxDepth(self, root: Optional['TreeNode']) -> int:
        def depth(node):
            if not node:
                return 0
            return max(depth(node.left), depth(node.right)) + 1
        return depth(root)

```

PHASE 5 — TEST & DEBUG

$\text{root} = [3, 9, 20, \text{null}, \text{null}, 15, 7]$:

$\text{depth}(3) = \max(\text{depth}(9), \text{depth}(20)) + 1$.

$\text{depth}(9) = \max(0, 0) + 1 = 1$. $\text{depth}(20) = \max(\text{depth}(15), \text{depth}(7)) + 1 = \max(1, 1) + 1 = 2$.

$\text{depth}(3) = \max(1, 2) + 1 = 3$ ✓

$\text{root} = \text{None}$: $\text{depth}(\text{None}) = 0$ ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(h)$ call stack — $O(\log n)$ balanced, $O(n)$ skewed.

BFS alternative: count levels with a queue — $O(n)$ time, $O(w)$ space ($w = \text{max width}$).

For very deep trees, BFS avoids recursion stack overflow.

Given more time I'd ask: what about minimum depth? Different — must reach a leaf,

so BFS is actually better there (stops at first leaf found)."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def maxDepth(self, root: TreeNode | None) -> int:
        if not root:
            return 0
        return 1 + max(self.maxDepth(root.left), self.maxDepth(root.right))

```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def maxDepth(self, root: TreeNode) -> int:
        if root is None:
            return 0

        l, r = self.maxDepth(root.left), self.maxDepth(root.right)
        return 1 + max(l, r)
```

• SimplyLeet

```
# Python implementation using DFS

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:

    def maxDepth(self, root: Optional[TreeNode]) -> int:
        # Base case: if the node is null, return 0.
        if not root:
            return 0

        # Recursively compute the maximum depth of the left subtree.
        left_depth = self.maxDepth(root.left)
        # Recursively compute the maximum depth of the right subtree.
        right_depth = self.maxDepth(root.right)

        # Return the greater depth plus one for the current node.
        return 1 + max(left_depth, right_depth)
```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def maxDepth(self, root: TreeNode) -> int:
        if root is None:
            return 0

        l, r = self.maxDepth(root.left), self.maxDepth(root.right)
        return 1 + max(l, r)
```

◆ Quick Review

Key Insights

- Use a recursive strategy (DFS) to traverse the binary tree.
- At each node, compute the maximum depth of its left and right subtree.
- The maximum depth at the current node is 1 (current node) plus the maximum of the left and right subtree depths.
- Alternatively, an iterative approach using BFS (level-order traversal) can be used.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes, since every node is visited once.

Space Complexity: $O(h)$ for recursion stack in DFS, where h is the height of the tree (worst-case $O(n)$ for a skewed tree); $O(n)$ for BFS in the worst-case scenario if the tree is balanced.

Solution

The problem is approached using Depth-First Search (DFS) with recursion. The idea is to visit every node and calculate the maximum depth of the left and right subtrees, then add 1 for the current node. This method naturally handles the base case when a null node (empty tree) is reached by returning 0. Alternatively, a Breadth-First Search (BFS) approach can determine the tree level by level, with the total number of levels equaling the maximum depth. Key data structures include recursion call stack for DFS or a queue for BFS.

Trees & BST

□ #108 Convert Sorted Array to Binary Search Tree

EAS

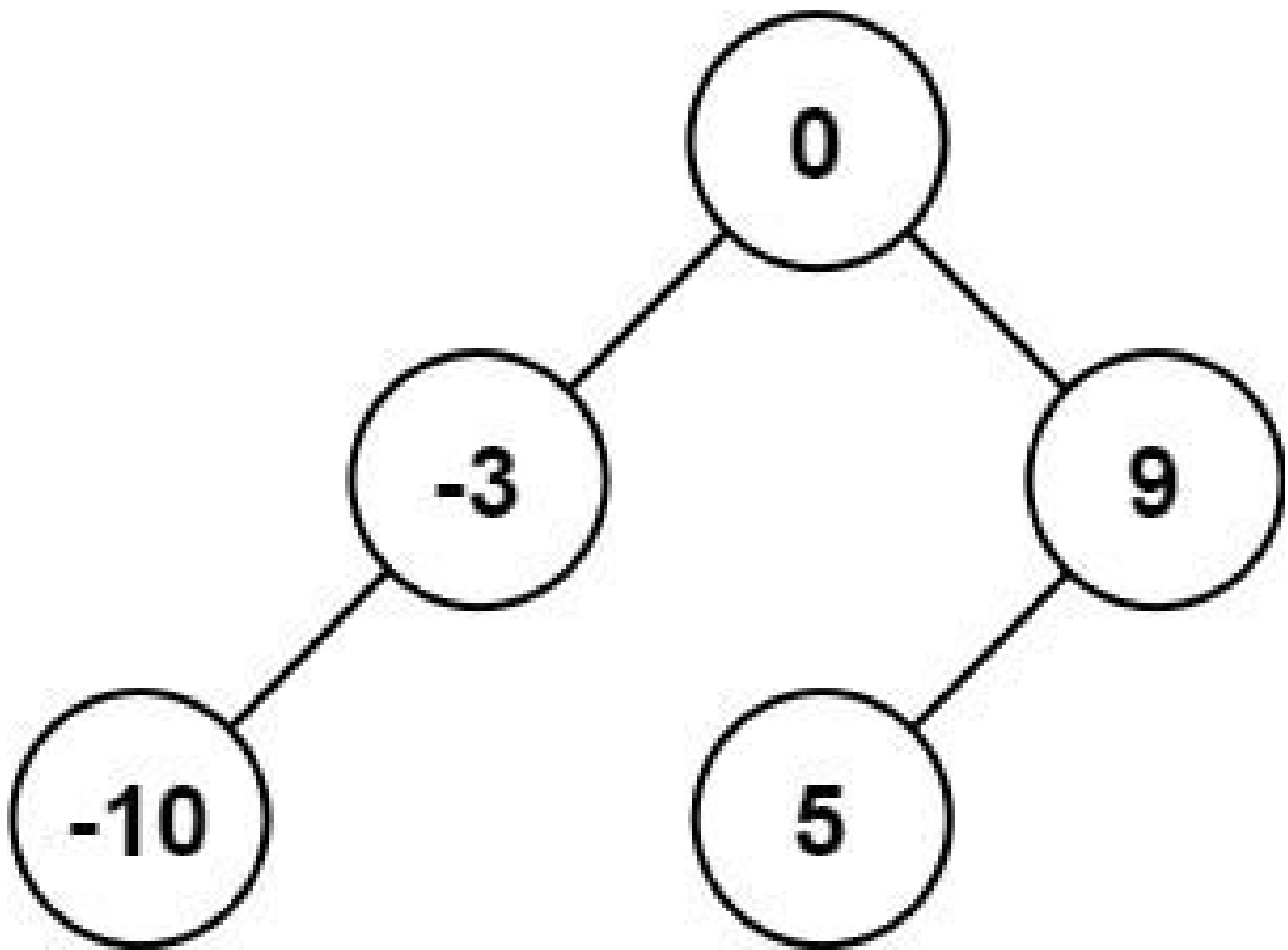
[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Divide and Conquer · Tree
Lists: Grind 169

Problem

Given an integer array `nums` where the elements are sorted in ascending order, convert it to a height-balanced binary search tree.

Example 1:

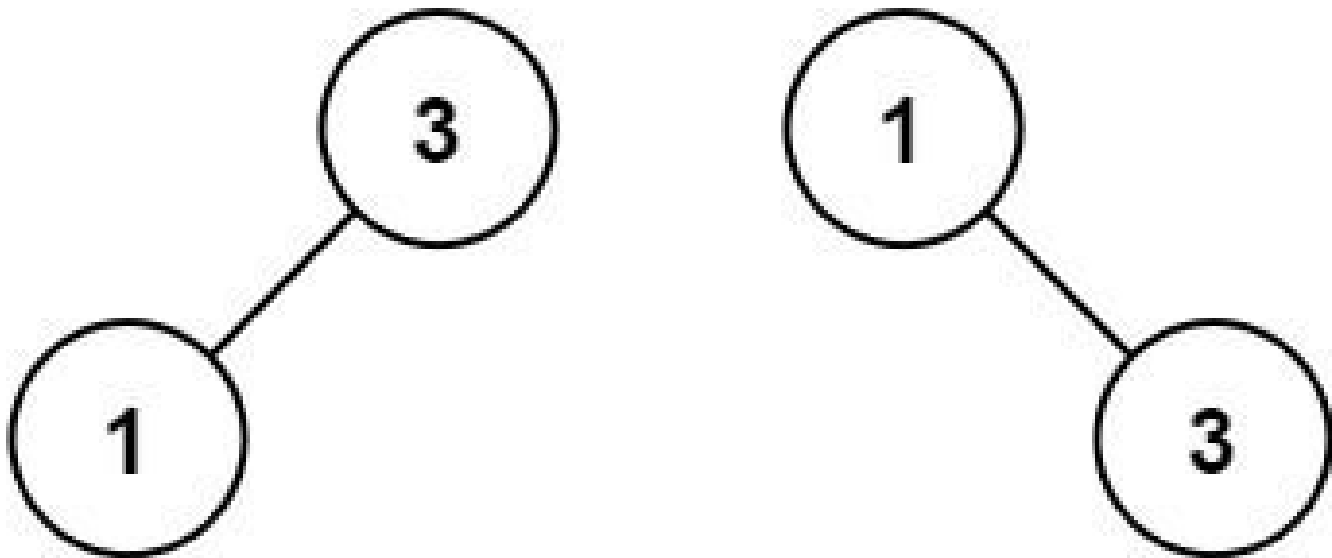


Input: `nums = [-10,-3,0,5,9]`

Output: `[0,-3,9,-10,null,5]`

Explanation: `[0,-10,5,null,-3,null,9]` is also accepted:

Example 2:



Input: nums = [1,3]

Output: [3,1]

Explanation: [1,null,3] and [3,1] are both height-balanced BSTs.

Constraints:

- $1 \leq \text{nums.length} \leq 104$
- $-104 \leq \text{nums}[i] \leq 104$
- nums is sorted in a strictly increasing order.

My LeetCode Solution

• My LeetCode Solution

```

# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def sortedArrayToBST(self, nums: List[int]) -> Optional[TreeNode]:
        def build(l,r):
            if l>r:

```

```

        return None

    m = (l+r)//2
    return TreeNode(nums[m], build(l, m-1), build(m+1,r))

n= len(nums)
return build(0,n-1)

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Convert a sorted array to a height-balanced BST.

Height-balanced means every node's left and right subtree heights differ by at most 1.

Multiple valid answers exist – any height-balanced BST is acceptable. Right?"

#

"[-10,-3,0,5,9]: pick middle element as root → 0. Left half [-10,-3] → left subtree.

Right half [5,9] → right subtree. Recurse. → [0,-3,9,-10,null,5] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Pick the middle element of nums as root, recursively build left from nums[:mid]

and right from nums[mid+1:].

Always produces a valid BST shape for a sorted input.

Time: $O(n \log n)$ from repeated slicing. Space: $O(n)$ for slices.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Pick the middle element as root – this guarantees balance because both halves

have equal (or ± 1) size. Recurse on left and right halves.

Divide and conquer. Use inner helper build(l, r) operating on indices."

PHASE 4 — CLEAN CODE

```
class _108_SortedArrayToBST:
```

```
    def sortedArrayToBST(self, nums: List[int]) -> Optional['TreeNode']:
```

```
        def build(left, right):
```

```
            if left > right:
```

```
                return None
```

```
            mid = (left + right) // 2
```

```
            return TreeNode(nums[mid], build(left, mid - 1), build(mid + 1, right))
```

```
        return build(0, len(nums) - 1)
```

PHASE 5 — TEST & DEBUG

```
# nums=[-10,-3,0,5,9]:
```

```
# build(0,4): mid=2→node(0). left=build(0,1), right=build(3,4).
```

```
# build(0,1): mid=0→node(-10). left=None, right=build(1,1)→node(-3). →
(-10,None,-3)
# build(3,4): mid=3→node(5). left=None, right=build(4,4)→node(9). → (5,None,9)
# root=0, left=(-10,None,-3), right=(5,None,9). Valid height-balanced BST ✓
# nums=[1,3]: mid=0→node(1), right=build(1,1)→node(3). → [1,null,3] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) – every element becomes a node exactly once.
# Space O(log n) for the call stack – tree is balanced by construction.
# Choosing (l+r)//2 vs (l+r+1)//2 gives slightly different but equally valid trees.
# Given more time I'd ask: what if the array is sorted in descending order?
# Reverse it first or adjust the index logic – same approach."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def sortedArrayToBST(self, nums: list[int]) -> TreeNode | None:
        def build(l: int, r: int) -> TreeNode | None:
            if l > r:
                return None
            m = (l + r) // 2
            return TreeNode(nums[m],
                            build(l, m - 1),
                            build(m + 1, r))

        return build(0, len(nums) - 1)
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def sortedArrayToBST(self, nums: List[int]) -> Optional[TreeNode]:
        def dfs(l: int, r: int) -> Optional[TreeNode]:
            if l > r:
```

```

        return None
    mid = (l + r) >> 1
    return TreeNode(nums[mid], dfs(l, mid - 1), dfs(mid + 1, r))

return dfs(0, len(nums) - 1)

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    # Initializes a tree node with a value and optional left/right children.
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def sortedArrayToBST(self, nums):

        # Helper function to build BST from subarray indices.
        def buildBST(left, right):
            # Base condition: if no elements to process, return None.
            if left > right:
                return None
            # Choose the middle element as the root for balance.
            mid = (left + right) // 2
            node = TreeNode(nums[mid])
            # Recursively build the left subtree.
            node.left = buildBST(left, mid - 1)

            # Recursively build the right subtree.
            node.right = buildBST(mid + 1, right)
            return node
        # Build and return the BST from the entire array.
        return buildBST(0, len(nums) - 1)

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def sortedArrayToBST(self, nums: List[int]) -> Optional[TreeNode]:
        def dfs(l, r):
            if l > r:
                return None
            mid = (l + r) >> 1
            left = dfs(l, mid - 1)
            right = dfs(mid + 1, r)
            return TreeNode(nums[mid], left, right)

        return dfs(0, len(nums) - 1)

#####

```

```

# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

```

```

class Solution(object):
    def sortedArrayToBST(self, nums):
        """

```

```

        :type nums: List[int]
        :rtype: TreeNode
        """
        if nums:
            midPos = len(nums) / 2
            mid = nums[midPos]
            root = TreeNode(mid)
            root.left = self.sortedArrayToBST(nums[:midPos])
            root.right = self.sortedArrayToBST(nums[midPos + 1:])
            return root

```


◆ Quick Review

Key Insights

- The array is sorted in ascending order, which allows the use of a divide and conquer strategy.
- Choosing the middle element of the array as the root ensures that the left and right subtrees are roughly equal in size, yielding a balanced BST.
- A recursive approach is natural for building the tree: the base case is when the subarray is empty.
- The algorithm splits the array into two halves to recursively build the left and right subtrees.

Space & Time

Time Complexity: $O(n)$ – Every element is processed exactly once.

Space Complexity: $O(n)$ – $O(n)$ space is used to store the BST, and additional $O(\log n)$ space may be used for the recursion stack.

Solution

We use a divide and conquer approach with recursion. The main idea is to select the middle element of the current subarray as the root node so that the left subarray elements form the left subtree and the right subarray elements form the right subtree, ensuring the tree is balanced. The recursion continues until the subarray is empty, which serves as the base case. This strategy naturally maintains the BST property since all elements in the left subarray are smaller than the root, and all in the right subarray are larger.

Trees & BST

□ #110 Balanced Binary Tree

EAS

[Open on LeetCode](#)

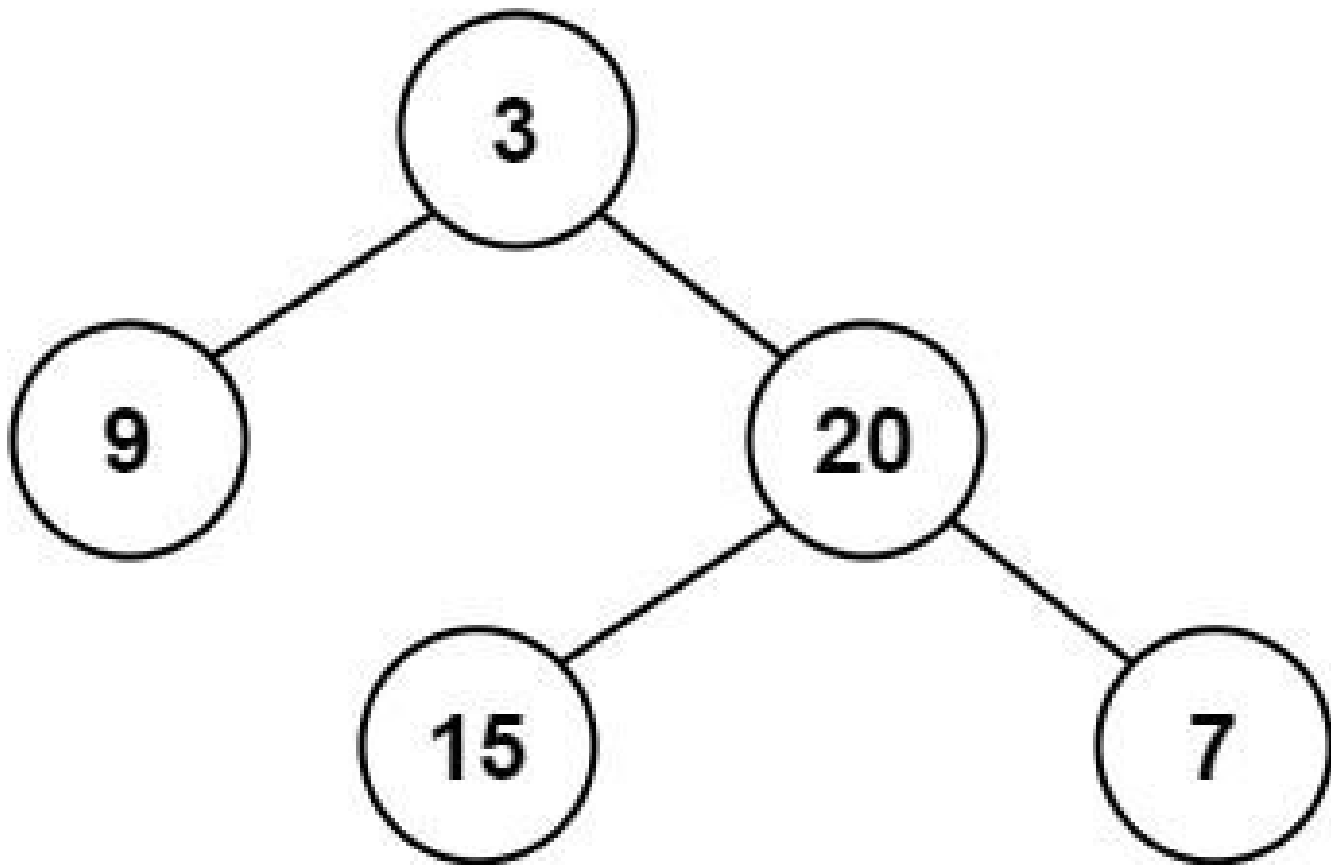
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS

Lists: Grind 169

Problem

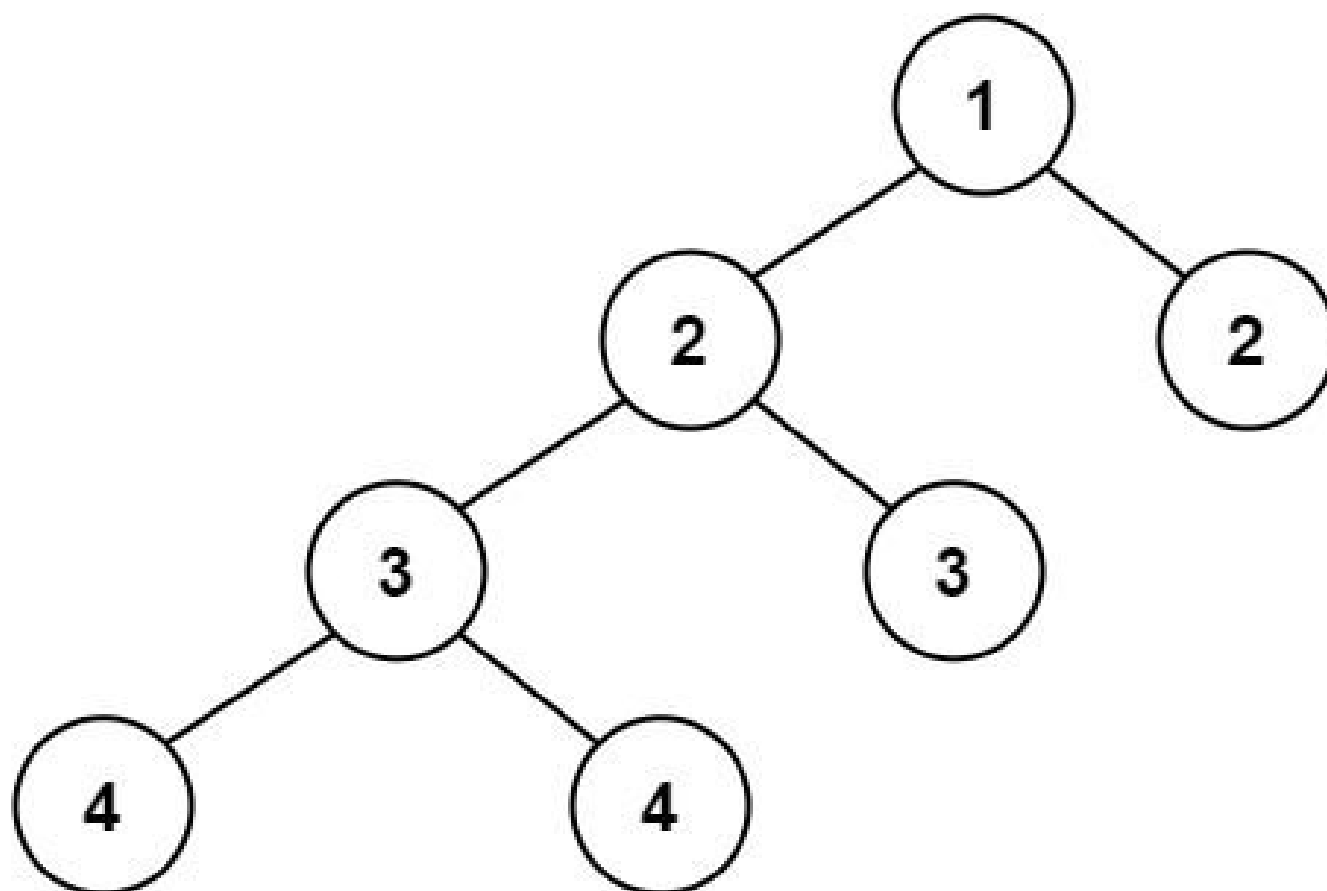
Given a binary tree, determine if it is height-balanced.

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: true

Example 2:



Input: root = [1,2,2,3,3,null,null,4,4]
Output: false

Example 3:

Input: root = []
Output: true

Constraints:

- The number of nodes in the tree is in the range [0, 5000].
- $-104 \leq \text{Node.val} \leq 104$

My LeetCode Solution

- My LeetCode Solution

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isBalanced(self, root: Optional[TreeNode]) -> bool:
        def depth(root):
            if not root:
                return 0

            l = depth(root.left)
            r = depth(root.right)
            return max(l, r) + 1

        if not root:
            return True

        return abs(depth(root.left) - depth(root.right)) <= 1 and self.isBalanced(root.left) and self.isBalanced(root.right)

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "A tree is height-balanced if every node's left and right subtree heights
# differ by at most 1. Empty tree is balanced. Right?"
#
# "[3,9,20,null,null,15,7]: heights of 9's subtrees: 0 vs 0 ✓. 20's: 1 vs 1 ✓. 3's:
# 1 vs 2 ✓ → true.
# [1,2,2,3,3,null,null,4,4]: left subtree of root has height 3, right has 1 → false
# ✓"

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with brute force to lock in the logic.
# At each node compute left subtree height and right subtree height separately.
# Node is balanced if |left-right|<=1 and both subtrees are balanced recursively.
# Recomputes heights from scratch at every node - O(n^2) on skewed trees.
# Time: O(n^2) worst case. Space: O(h) stack.
# Should I go ahead and optimize?"

```

```

class _110_BalancedTree_Brute:
    def isBalanced(self, root: Optional[TreeNode]) -> bool:
        def height(node):

```

```

        if not node:
            return 0
        return max(height(node.left), height(node.right)) + 1
    def balanced(node):
        if not node:
            return True
        lh, rh = height(node.left), height(node.right)
        return abs(lh - rh) <= 1 and balanced(node.left) and
balanced(node.right)
    return balanced(root)

```

PHASE 3 — OPTIMIZE

```

# "The brute force calls height() repeatedly for the same nodes – O(n²).
# I notice we're recomputing heights we've already seen. What if we cache it?
#
# Better: combine the height computation and balance check into one DFS pass.
# Return -1 to signal 'unbalanced subtree found' – propagate it up immediately.
# Otherwise return the actual height.
# A single O(n) pass handles everything."

```

PHASE 4 — CLEAN CODE

```

class _110_BalancedTree:
    def isBalanced(self, root: Optional['TreeNode']) -> bool:
        def check(node):
            if not node:
                return 0
            left = check(node.left)
            right = check(node.right)
            if left == -1 or right == -1 or abs(left - right) > 1:
                return -1 # propagate 'unbalanced' signal
            return max(left, right) + 1
        return check(root) != -1

```

PHASE 5 — TEST & DEBUG

```

# root=[3,9,20,null,null,15,7]:
# check(9)=1. check(15)=1. check(7)=1. check(20)=max(1,1)+1=2.
# check(3): left=1,right=2. |1-2|=1 <=1 → return 2. result≠-1 → True ✓
# root=[1,2,2,3,3,null,null,4,4]:
# check(4)=1. check(3,left)=max(1,0)+1=2. check(3,right)=max(1,0)+1=2.
# check(2,left): left=2,right=2 → 3. check(2,right): left=0,right=0 → 1.
# check(1): left=3,right=1. |3-1|=2>1 → return -1. result==-1 → False ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n) – single DFS, each node visited once.
# Space O(h) for the call stack.
# The -1 sentinel trick is the key interview insight: instead of separate
# height and balance checks, fuse them into one return value with a special signal.
# Given more time I'd ask: can we rebalance a tree in-place?
# Flatten to sorted array (in-order traversal) then rebuild – O(n) time."

```


Community Solutions (Python)

• WalkCC

```
class Solution:
    def isBalanced(self, root: TreeNode | None) -> bool:
        if not root:
            return True

        def maxDepth(root: TreeNode | None) -> int:
            if not root:
                return 0
            return 1 + max(maxDepth(root.left), maxDepth(root.right))

        return (abs(maxDepth(root.left) - maxDepth(root.right)) <= 1 and
                self.isBalanced(root.left) and
                self.isBalanced(root.right))
```

```
class Solution:
    def isBalanced(self, root: TreeNode | None) -> bool:
        ans = True

        def maxDepth(root: TreeNode | None) -> int:
            """Returns the height of root and sets ans to false if root
            unbalanced."""
            nonlocal ans
            if not root or not ans:
                return 0
            left = maxDepth(root.left)

            right = maxDepth(root.right)
            if abs(left - right) > 1:
                ans = False
            return max(left, right) + 1

        maxDepth(root)
        return ans
```

```

class Solution:
    def isBalanced(self, root: TreeNode | None) -> bool:
        def maxDepth(root: TreeNode | None) -> int:
            """Returns the height of root if root is balanced; otherwise, returns
            -1."""
            if not root:
                return 0

            left = maxDepth(root.left)
            if left == -1:
                return -1

            right = maxDepth(root.right)
            if right == -1:
                return -1
            if abs(left - right) > 1:
                return -1

            return 1 + max(maxDepth(root.left), maxDepth(root.right))

        return maxDepth(root) != -1

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isBalanced(self, root: Optional[TreeNode]) -> bool:
        def height(root):
            if root is None:
                return 0
            l, r = height(root.left), height(root.right)
            if l == -1 or r == -1 or abs(l - r) > 1:
                return -1
            return 1 + max(l, r)

        return height(root) >= 0

```

• SimplyLeet

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def isBalanced(self, root: TreeNode) -> bool:
        # Helper function to return the height of the subtree or -1 if
        # unbalanced.

        def checkHeight(node: TreeNode) -> int:
            # An empty tree has height 0 and is balanced.
            if not node:
                return 0

            # Recursively get the height of the left subtree.
            leftHeight = checkHeight(node.left)
            # If left subtree is unbalanced, return -1.
            if leftHeight == -1:
                return -1

            # Recursively get the height of the right subtree.
            rightHeight = checkHeight(node.right)
            # If right subtree is unbalanced, return -1.
            if rightHeight == -1:
                return -1

            # If the current node's subtrees differ in height by more than 1,
            # it's unbalanced.
            if abs(leftHeight - rightHeight) > 1:
                return -1

            # Return the height of the current node.
            return max(leftHeight, rightHeight) + 1

        # The tree is balanced if checkHeight does not return -1.
        return checkHeight(root) != -1
```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isBalanced(self, root: Optional[TreeNode]) -> bool:
        def height(root):
            if root is None:
                return 0
            l, r = height(root.left), height(root.right)
            if l == -1 or r == -1 or abs(l - r) > 1:
                return -1
            return 1 + max(l, r)

        return height(root) >= 0
```

◆ Quick Review

Key Insights

- Use a bottom-up DFS (Depth-First Search) traversal to compute the height of each subtree.
- If any subtree is found unbalanced, propagate an error indicator (e.g., -1) upward, allowing early termination.
- An empty tree is considered balanced.
- The key check is to ensure that for each node, the absolute difference between the

heights of its left and right subtrees is no more than 1.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes in the tree, since we visit each node once.

Space Complexity: $O(h)$ where h is the height of the tree, which corresponds to the recursion stack ($O(n)$ in the worst-case of a skewed tree).

Solution

The solution uses a recursive DFS approach. A helper function recursively computes the height of a subtree. If a subtree is found to be unbalanced (the height difference between its left and right subtrees exceeds 1), the helper returns a special value (e.g., -1) to indicate that the subtree is unbalanced. This value is then propagated up the recursion to terminate further unnecessary computations. The main function checks the result of the helper function and returns true if the tree is balanced (i.e., the helper did not return -1) and false otherwise.

Trees & BST

□ #112 Path Sum

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BFS

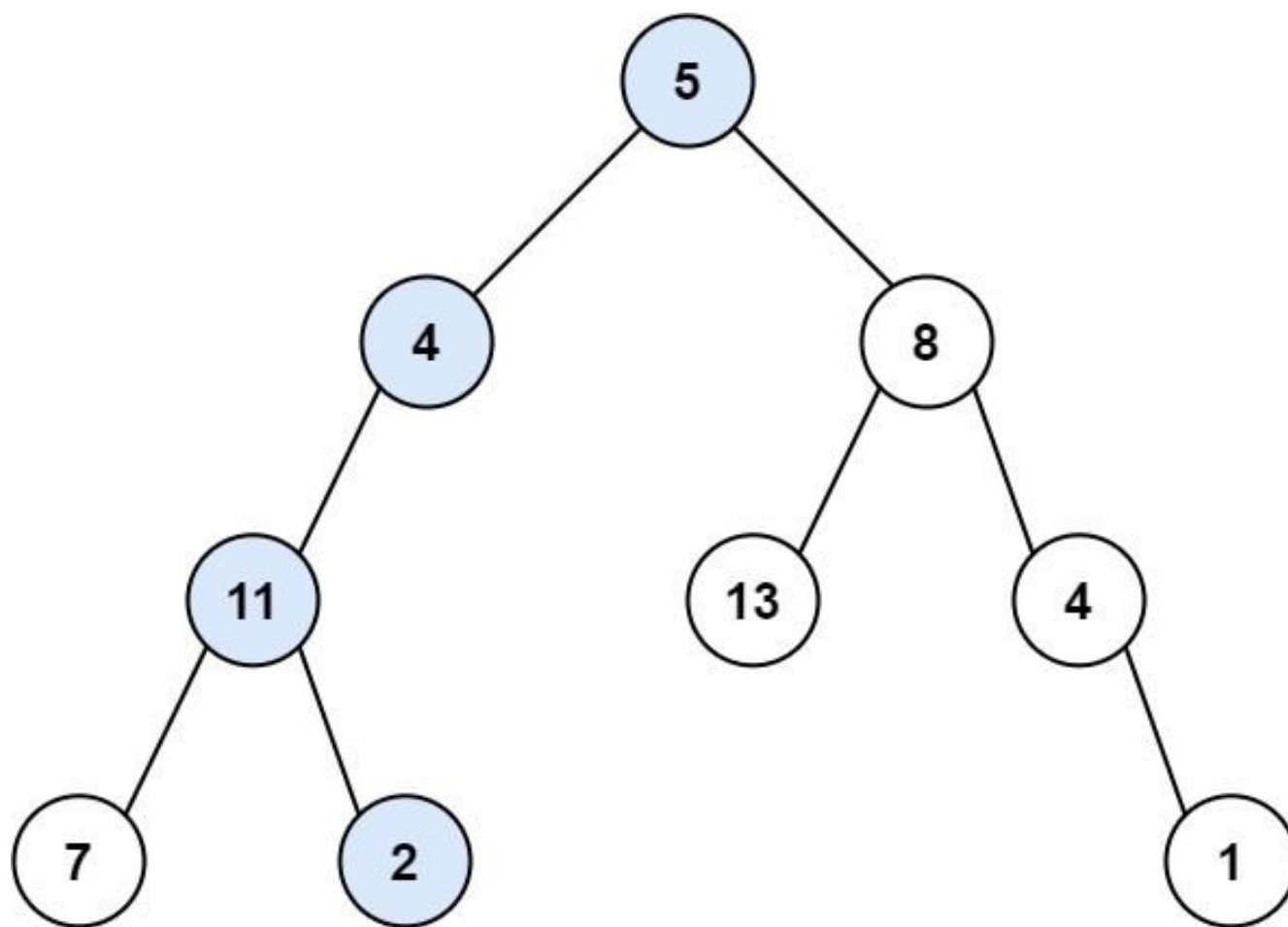
Lists: Denny Zhang

Problem

Given the root of a binary tree and an integer targetSum, return true if the tree has a root-to-leaf path such that adding up all the values along the path equals targetSum.

A leaf is a node with no children.

Example 1:

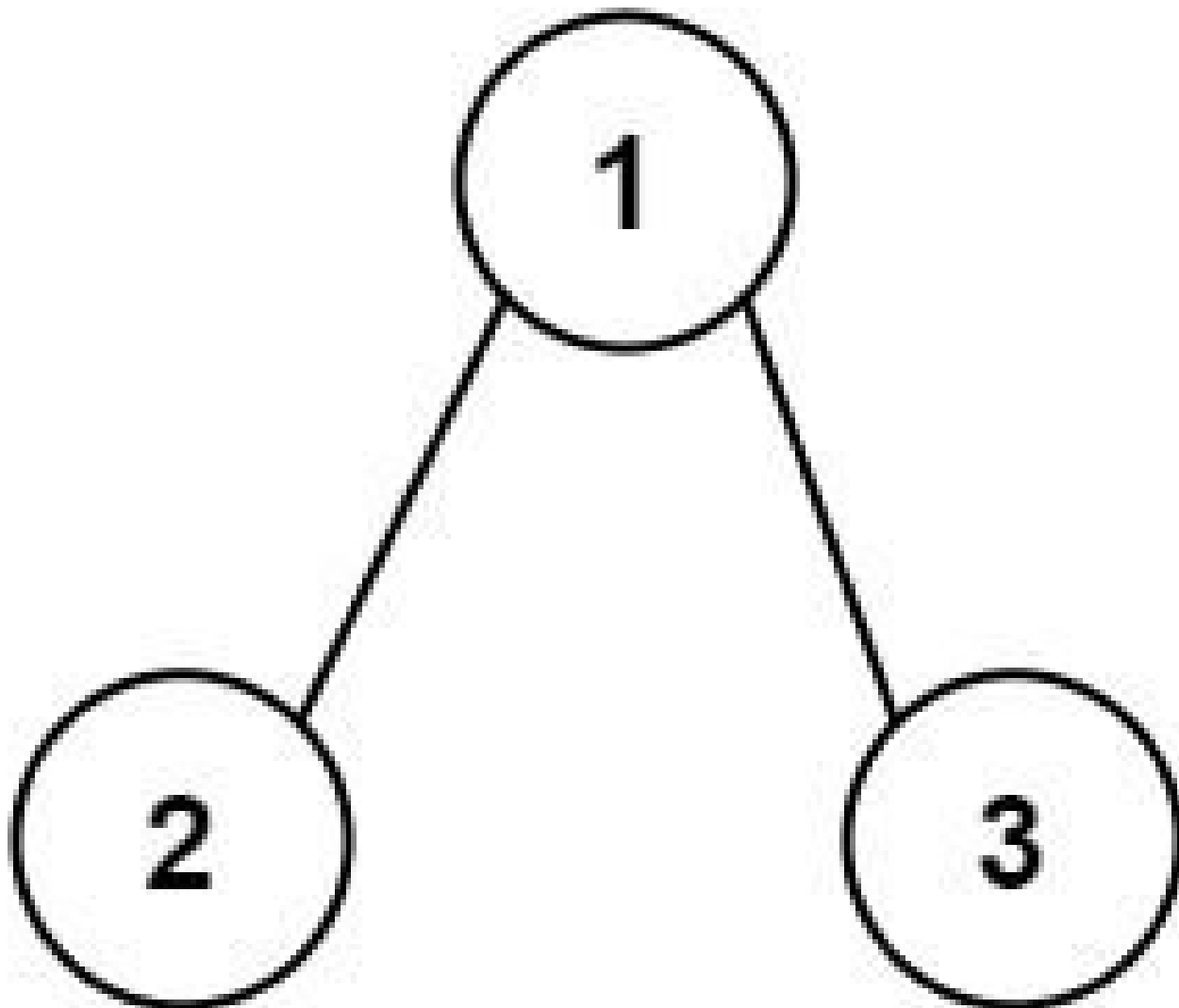


Input: root = [5,4,8,11,null,13,4,7,2,null,null,null,1], targetSum = 22

Output: true

Explanation: The root-to-leaf path with the target sum is shown.

Example 2:



Input: root = [1,2,3], targetSum = 5

Output: false

Explanation: There are two root-to-leaf paths in the tree:

(1 --> 2): The sum is 3.

(1 --> 3): The sum is 4.

There is no root-to-leaf path with sum = 5.

Example 3:

Input: root = [], targetSum = 0

Output: false

Explanation: Since the tree is empty, there are no root-to-leaf paths.

Constraints:

- The number of nodes in the tree is in the range [0, 5000].

- $-1000 \leq \text{Node.val} \leq 1000$
- $-1000 \leq \text{targetSum} \leq 1000$

My LeetCode Solution

• My LeetCode Solution

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def hasPathSum(self, root: Optional[TreeNode], targetSum: int) -> bool:
        def hassum(root, targetSum):
            if not root:
                return False

            if root.val == targetSum and not root.left and not root.right:
                return True

            return hassum(root.left, targetSum - root.val) or hassum(root.right, targetSum - root.val)

        return hassum(root, targetSum)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Find if any root-to-leaf path sums to targetSum.
# A leaf is a node with no children – the path must end at a leaf. Right?
# Empty tree → false (no leaves)."
#
# "[5,4,8,11,null,13,4,7,2,null,null,null,1], target=22:
# path 5→4→11→2 = 22 → true ✓
# [], target=0 → false (empty tree has no root-to-leaf path) ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic."
```

```
# DFS traversal collecting a running sum as we descend. At each leaf node,
# check if the accumulated sum equals targetSum. Backtrack and try other paths.
# The optimal approach is the same DFS – just written more elegantly by subtracting
# from target instead of accumulating. Both are O(n) time and O(h) space, so we skip
# a separate brute class and go straight to the clean version."
```

PHASE 3 — OPTIMIZE

```
# "More elegant: subtract the current node's value from targetSum as we descend.
# At a leaf, check if remaining == 0. Avoids passing a separate running sum."
```

PHASE 4 — CLEAN CODE

```
class _112_PathSum:
    def hasPathSum(self, root: Optional['TreeNode'], targetSum: int) -> bool:
        def hassum(node, remaining):
            if not node:
                return False
            remaining -= node.val
            if not node.left and not node.right: # leaf
                return remaining == 0
            return hassum(node.left, remaining) or hassum(node.right, remaining)
        return hassum(root, targetSum)
```

PHASE 5 — TEST & DEBUG

```
# root=[5,4,8,...], target=22:
# hassum(5,22): rem=17. hassum(4,17): rem=13. hassum(11,13): rem=2.
# hassum(7,2): rem=-5, leaf -> -5≠0 False.
# hassum(2,2): rem=0, leaf -> True ✓
# root=[], target=0: hassum(None,0)=False ✓
# root=[1,2], target=1: hassum(1,1): rem=0, not leaf.
# hassum(2,0): rem=-2, leaf -> False. -> False ✓ (path must reach a leaf)
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(h) call stack.
# The 'leaf check' (no left and no right) is critical – without it, a node
# with one child that hits 0 would incorrectly return true.
# Given more time I'd ask: return all root-to-leaf paths that sum to target?
# That's Path Sum II (#113) – collect paths into a result list with backtracking."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def hasPathSum(self, root: TreeNode, summ: int) -> bool:
        if not root:
            return False
        if root.val == summ and not root.left and not root.right:
            return True
        return (self.hasPathSum(root.left, summ - root.val) or
                self.hasPathSum(root.right, summ - root.val))
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def hasPathSum(self, root: Optional[TreeNode], targetSum: int) -> bool:
        def dfs(root, s):
            if root is None:
                return False
            s += root.val
            if root.left is None and root.right is None and s == targetSum:
                return True
            return dfs(root.left, s) or dfs(root.right, s)

        return dfs(root, 0)
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val          # Node's value
        self.left = left        # Pointer to left child
        self.right = right       # Pointer to right child

def hasPathSum(root, targetSum):
    # If the current node is None, no valid path exists
    if not root:
```

```

        return False
    # Check if the current node is a leaf node
    if not root.left and not root.right:
        # Return true if the remaining targetSum equals the leaf value
        return targetSum == root.val
    # Recursively check the left and right subtree with the updated targetSum
    return hasPathSum(root.left, targetSum - root.val) or hasPathSum(root.right, targetSum - root.val)

# Example usage:
# root = TreeNode(5, TreeNode(4, TreeNode(11, TreeNode(7), TreeNode(2))),
#                 TreeNode(8, TreeNode(13), TreeNode(4, None, TreeNode(1))))

# print(hasPathSum(root, 22)) # Output: True

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def hasPathSum(self, root: Optional[TreeNode], targetSum: int) -> bool:
        def dfs(root, s):
            if root is None:
                return False
            s += root.val
            if root.left is None and root.right is None and s == targetSum:
                return True
            return dfs(root.left, s) or dfs(root.right, s)

        return dfs(root, 0)

```

◆ Quick Review

Key Insights

- Use a depth–first search (DFS) strategy to traverse the tree from the root to the leaves.
- At each node, subtract the node's value from the targetSum.
- Check if the node is a leaf; if yes and the remaining targetSum equals the node's value then a valid path is found.
- If the tree is empty, return false as no path exists.

Space & Time

Time Complexity: $O(N)$ where N is the number of nodes, since every node may need to be visited.

Space Complexity: $O(H)$ where H is the height of the tree, accounting for the recursion stack (worst–case $O(N)$ for a skewed tree).

Solution

The approach uses a recursive depth–first search. The algorithm subtracts the current node's value from targetSum as it moves down the tree. When it reaches a leaf, it checks if the modified targetSum is equal to the leaf's value. If yes, it returns true. The recursion continues until either a valid path is found or all paths

are exhausted. The primary data structure used here is the call stack for recursion.

Trees & BST

□ #226 Invert Binary Tree

EAS

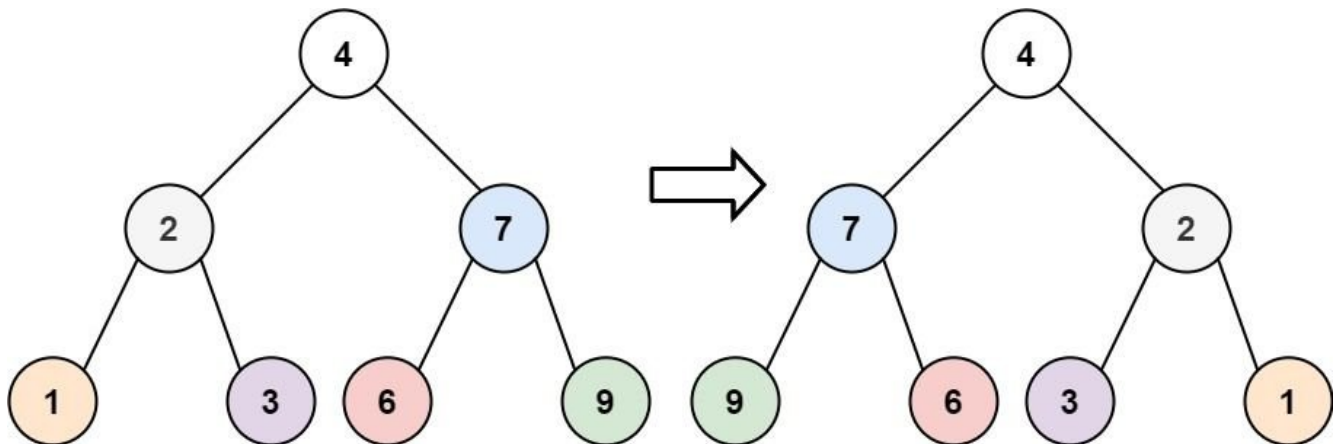
[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BFS
Lists: Grind 169

Problem

Given the root of a binary tree, invert the tree, and return its root.

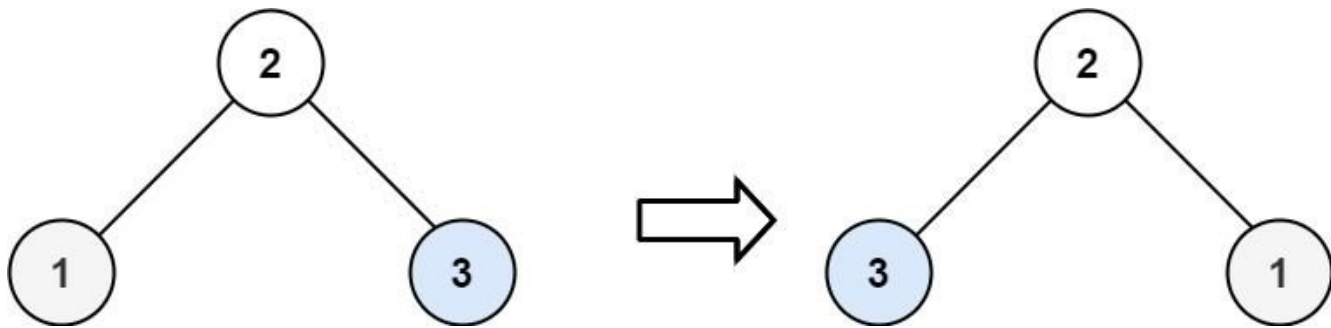
Example 1:



Input: root = [4,2,7,1,3,6,9]

Output: [4,7,2,9,6,3,1]

Example 2:



Input: root = [2,1,3]

Output: [2,3,1]

Example 3:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 100].
- $-100 \leq \text{Node.val} \leq 100$

My LeetCode Solution

• My LeetCode Solution

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        def invert(root):
            if not root:

```

```

        return None

    l ,r = invert(root.left), invert(root.right)
    root.left = r
    root.right = l

    return root
return invert(root)

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Invert the tree: swap left and right children at every node recursively.

Return the root. Empty tree → return None. Right?"

#

"[4,2,7,1,3,6,9] → [4,7,2,9,6,3,1]: left and right subtrees are swapped at every level ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

Iterative BFS with a queue: visit each node level by level and swap its left

and right children in place. Process until the queue is empty.

Time: O(n). Space: O(n) for the queue. Should I go ahead and optimize?"

```
class _226_InvertTree_Brute:
```

```
    def invertTree(self, root: Optional['TreeNode']) -> Optional['TreeNode']:
```

```
        if not root:
```

```
            return None
```

```
        from collections import deque
```

```
        queue = deque([root])
```

```
        while queue:
```

```
            node = queue.popleft()
```

```
            node.left, node.right = node.right, node.left
```

```
            if node.left:
```

```
                queue.append(node.left)
```

```
            if node.right:
```

```
                queue.append(node.right)
```

```
        return root
```

PHASE 3 — OPTIMIZE

"Recursive DFS is cleaner. Invert left, invert right, then swap.

Or: swap first then recurse – same result. Either order works.

Use inner helper to avoid self. calls."

PHASE 4 — CLEAN CODE

```
class _226_InvertTree:
```

```
    def invertTree(self, root: Optional['TreeNode']) -> Optional['TreeNode']:
```

```
def invert(node):
    if not node:
        return None
    left = invert(node.left)
    right = invert(node.right)
    node.left, node.right = right, left
    return node
return invert(root)
```

PHASE 5 — TEST & DEBUG

```
# root=[4,2,7]:
# invert(2): invert(None)=None, invert(None)=None. swap: 2.left=None,2.right=None.
return 2.
# invert(7): same → 7.
# invert(4): left=invert(2)=2, right=invert(7)=7. swap: 4.left=7, 4.right=2. return
4.
# → [4,7,2] ✓ (recurse deeper for full tree)
# root=None: invert(None)=None ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(h) call stack.
# This is the problem Homebrew creator Max Howell famously couldn't solve in a
Google interview.
# Worth mentioning – it humanises the problem and shows you know your history.
# Given more time I'd ask: can we do this iteratively? Yes – BFS or DFS with a
stack,
# swap children at each node. O(n) time, O(n) space."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def invertTree(self, root: TreeNode | None) -> TreeNode | None:
        if not root:
            return None

        left = root.left
        right = root.right
        root.left = self.invertTree(right)
        root.right = self.invertTree(left)
        return root
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        if root is None:
            return None

        l, r = self.invertTree(root.left), self.invertTree(root.right)
        root.left, root.right = r, l
        return root
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val      # Initialize node value
        self.left = left     # Pointer to left child
        self.right = right   # Pointer to right child

class Solution:
    def invertTree(self, root: TreeNode) -> TreeNode:
        # Base case: if the node is None, return None.

        if not root:
            return None
        # Recursively invert the left and right subtrees.
        left_inverted = self.invertTree(root.left)
        right_inverted = self.invertTree(root.right)
        # Swap the left and right children.
        root.left, root.right = right_inverted, left_inverted
        return root
```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        def dfs(root):
            if root is None:
                return
            root.left, root.right = root.right, root.left
            dfs(root.left)
            dfs(root.right)

        dfs(root)
        return root
```

◆ Quick Review

Key Insights

- The problem can be solved using recursion (depth-first search) by swapping the children of every node.
- Alternatively, an iterative approach (using a queue for breadth-first search) can be used.
- The recursion terminates when a null node is encountered.
- The operation is performed in-place, modifying the original tree structure.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes, since each node is visited once.

Space Complexity: $O(h)$ where h is the height of the tree, which is the recursion stack space in the recursive approach. In worst-case (skewed tree), this could be $O(n)$.

Solution

We solve the problem using recursion (depth-first search). The approach is to: Check the base case: if the node is null, simply return null. Recursively invert the left subtree and the right subtree. Swap the left and right children of the current node. Return the current node after the swap. The recursive approach elegantly handles all nodes and ensures the inversion process covers the complete tree.

Trees & BST

□ ★ #270 Closest Binary Search Tree Value

EAS

★

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode

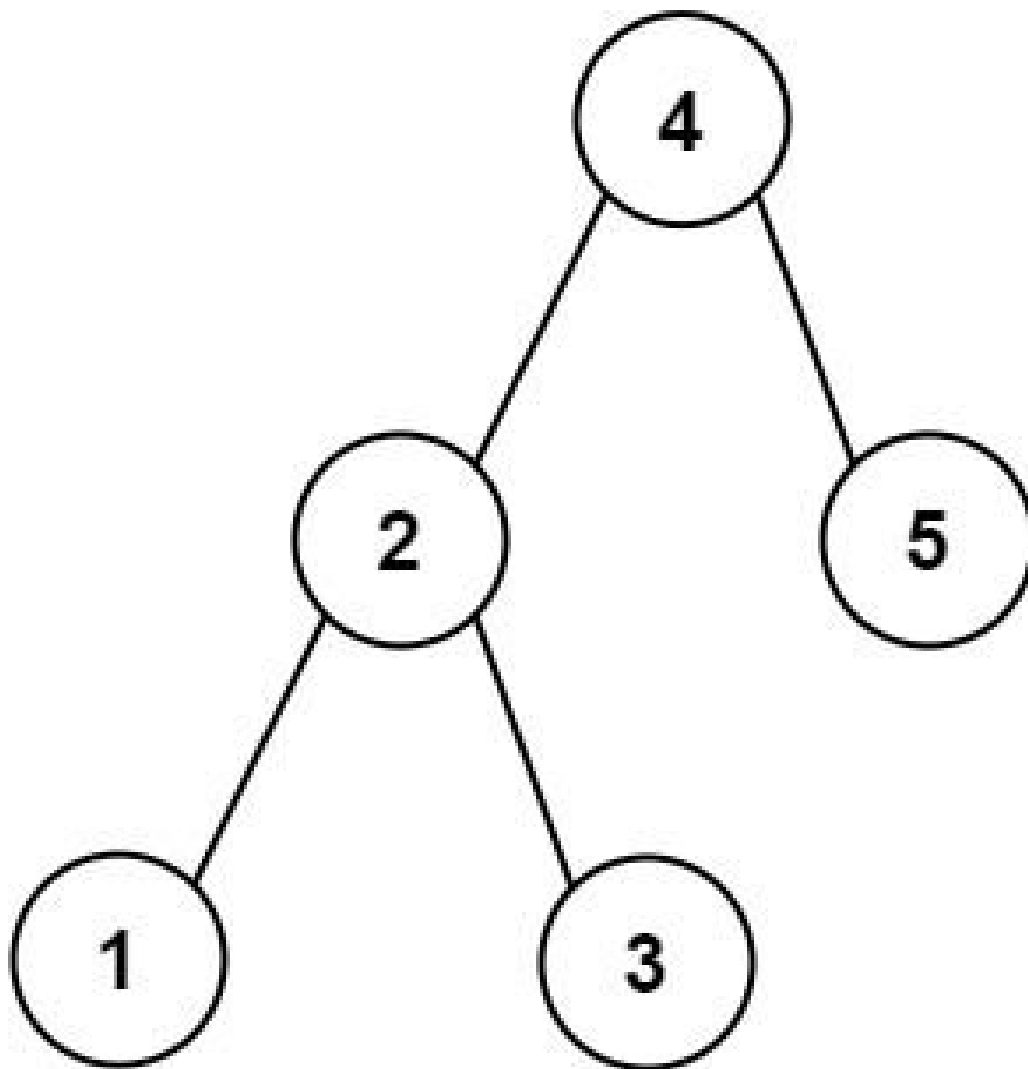
Binary Search · Tree · DFS · BST

Lists: ★ Premium | Premium 98

Problem

Given the root of a binary search tree and a target value, return the value in the BST that is closest to the target. If there are multiple answers, print the smallest.

Example 1:



Input: root = [4,2,5,1,3], target = 3.714286
Output: 4

Example 2:

Input: root = [1], target = 4.428571
Output: 1

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $0 \leq \text{Node.val} \leq 109$
- $-109 \leq \text{target} \leq 109$

My LeetCode Solution

• My LeetCode Solution

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def closestValue(self, root: Optional[TreeNode], target: float) -> int:
        def close(root, t):
            if target < root.val and root.left:
                l = close(root.left, t)
                if abs(t-l) <= abs(t-root.val):
                    return l

            if target > root.val and root.right:
                r = close(root.right, t)
                if abs(t-r) < abs(t-root.val):
                    return r

            return root.val

        return close(root, target)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Find the node value in a BST closest to a float target.
# If two values are equidistant, return the smaller one. Right?
# BST property means we can navigate toward the target rather than scanning all
# nodes."
#
# "root=[4,2,5,1,3], target=3.714286:
# candidates: 4 (dist=0.286), 3 (dist=0.714). 4 is closer → 4 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# Full DFS scan: visit every node, track the value with minimum |val - target|."
```

On a tie (equal distance), prefer the smaller value.
 # This ignores the BST property entirely – valid but $O(n)$ time and $O(n)$ stack space.
 # Should I go ahead and optimize?"

```
class _270_ClosestBST_Brute:
```

```
    def closestValue(self, root: Optional['TreeNode'], target: float) -> int:
```

```
        best = [root.val]
```

```
        def dfs(node):
```

```
            if not node:
```

```
                return
```

```
            if (abs(node.val - target) < abs(best[0] - target) or
```

```
                (abs(node.val - target) == abs(best[0] - target) and node.val <
```

```
best[0]))):
```

```
                best[0] = node.val
```

```
                dfs(node.left)
```

```
                dfs(node.right)
```

```
        dfs(root)
```

```
        return best[0]
```

PHASE 3 — OPTIMIZE

"Use BST property to navigate: at each node, the closest value is either the

current node or something in the subtree direction of the target.

If target < node.val → go left; if target > node.val → go right.

Track the closest seen so far. $O(h)$ time – $O(\log n)$ balanced, $O(n)$ skewed."

PHASE 4 — CLEAN CODE

```
class _270_ClosestBST:
```

```
    def closestValue(self, root: Optional['TreeNode'], target: float) -> int:
```

```
        def close(node, best):
```

```
            if not node:
```

```
                return best
```

```
            if abs(target - node.val) < abs(target - best):
```

```
                best = node.val
```

```
            elif abs(target - node.val) == abs(target - best):
```

```
                best = min(best, node.val) # tie → pick smaller
```

```
            if target < node.val:
```

```
                return close(node.left, best)
```

```
            else:
```

```
                return close(node.right, best)
```

```
        return close(root, root.val)
```

PHASE 5 — TEST & DEBUG

root=[4,2,5,1,3], target=3.714286:

close(4, 4): $|3.714-4|=0.286 < |3.714-4|=0.286$ → no update. target<4 → go left.

close(2, 4): $|3.714-2|=1.714 > 0.286$ → best stays 4. target>2 → go right.

close(3, 4): $|3.714-3|=0.714 > 0.286$ → best stays 4. target>3 → go right.

close(None, 4): return 4 ✓

root=[1], target=4.428: close(1,1): target>1 → go right → close(None,1) → 1 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(h)$. Space $O(h)$ for recursion stack. Iterative: $O(1)$ space.

BST navigation is the key – we prune entire subtrees we know can't contain
 # a closer value.
 # Given more time I'd ask: find the k closest values to target?
 # That's Closest BST Value II (#272) – use an in-order traversal with a sliding
 window
 # or a max-heap of size k."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def closestValue(self, root: TreeNode | None, target: float) -> int:
        # If target < root.val, search the left subtree.
        if target < root.val and root.left:
            left = self.closestValue(root.left, target)
            if abs(left - target) <= abs(root.val - target):
                return left

        # If target > root.val, search the right subtree.
        if target > root.val and root.right:

            right = self.closestValue(root.right, target)
            if abs(right - target) < abs(root.val - target):
                return right

        return root.val
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def closestValue(self, root: Optional[TreeNode], target: float) -> int:
        def dfs(node: Optional[TreeNode]):
            if node is None:
```

```

        return
    nxt = abs(target - node.val)
    nonlocal ans, diff
    if nxt < diff or (nxt == diff and node.val < ans):
        diff = nxt
        ans = node.val
    node = node.left if target < node.val else node.right
    dfs(node)

ans = 0

```

```

diff = inf
dfs(root)
return ans

```

• SimplyLeet

```
# Python solution for Closest Binary Search Tree Value
```

```
# Definition for a binary tree node.
```

```
# class TreeNode:
```

```
# def __init__(self, val=0, left=None, right=None):
```

```
# self.val = val
```

```
# self.left = left
```

```
# self.right = right
```

```
class Solution:
```

```
def closestValue(self, root: TreeNode, target: float) -> int:
```

```
    # Initialize the best value with the value of the root
```

```
    best_value = root.val
```

```
    # Traverse the BST iteratively
```

```
    current = root
```

```
    while current:
```

```
        # Calculate the difference between current node's value and target
```

```
        current_diff = abs(current.val - target)
```

```
        best_diff = abs(best_value - target)
```

```

        # Update best_value if a closer difference is found, or if equal
        and current value is smaller
        if current_diff < best_diff or (current_diff == best_diff and curr
ent.val < best_value):
            best_value = current.val

        # Decide the traversal direction based on the target comparison
        if target < current.val:
            current = current.left
        else:
            current = current.right

    return best_value

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode(object):
# def __init__(self, x):
# self.val = x
# self.left = None
# self.right = None

```

```

class Solution(object):
    def closestValue(self, p, target):
        """

```

```

        :type root: TreeNode
        :type target: float
        :rtype: int
        """

```

```

        ans = p.val
        while p:
            if abs(target - p.val) < abs(ans - target):
                ans = p.val
            if target < p.val:
                p = p.left

```

```

        else:
            p = p.right
        return ans

#####

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def closestValue(self, root: Optional[TreeNode], target: float) -> int:
        ans, mi = root.val, inf
        while root:
            t = abs(root.val - target)
            if t < mi:
                mi = t

            ans = root.val
            if root.val > target:
                root = root.left
            else:
                root = root.right
        return ans

```

◆ Quick Review

Key Insights

- BST property allows us to traverse the tree efficiently by comparing the target with node values.
- We can iteratively traverse the tree, moving left if the target is smaller than the current

node's value and moving right if greater.

- At each node, calculate the absolute difference between node's value and the target, and update the answer if the difference is smaller—or equal with a smaller candidate value.
- This approach minimizes the search area by leveraging the inherent ordering in BSTs.

Space & Time

Time Complexity: $O(h)$ on average, where h is the height of the tree ($O(\log n)$ for a balanced tree, and $O(n)$ in the worst-case scenario for a skewed tree).

Space Complexity: $O(1)$ for an iterative solution (or $O(h)$ if using recursion due to the call stack).

Solution

We solve the problem by performing an iterative traversal of the BST. Starting at the root, we maintain a variable to store the value closest to the target. For every node encountered, we: Compute the absolute difference between the node's value and the target. Update the stored closest value if the

computed difference is less than the current smallest difference; if the difference is the same, choose the smaller node value.

Depending on whether the target is less than or greater than the current node's value, move to the left or right child accordingly. This approach efficiently defines a search path that is consistent with the properties of a BST.

Trees & BST

□ #501 Find Mode in Binary Search Tree

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BST

Lists: Denny Zhang

Problem

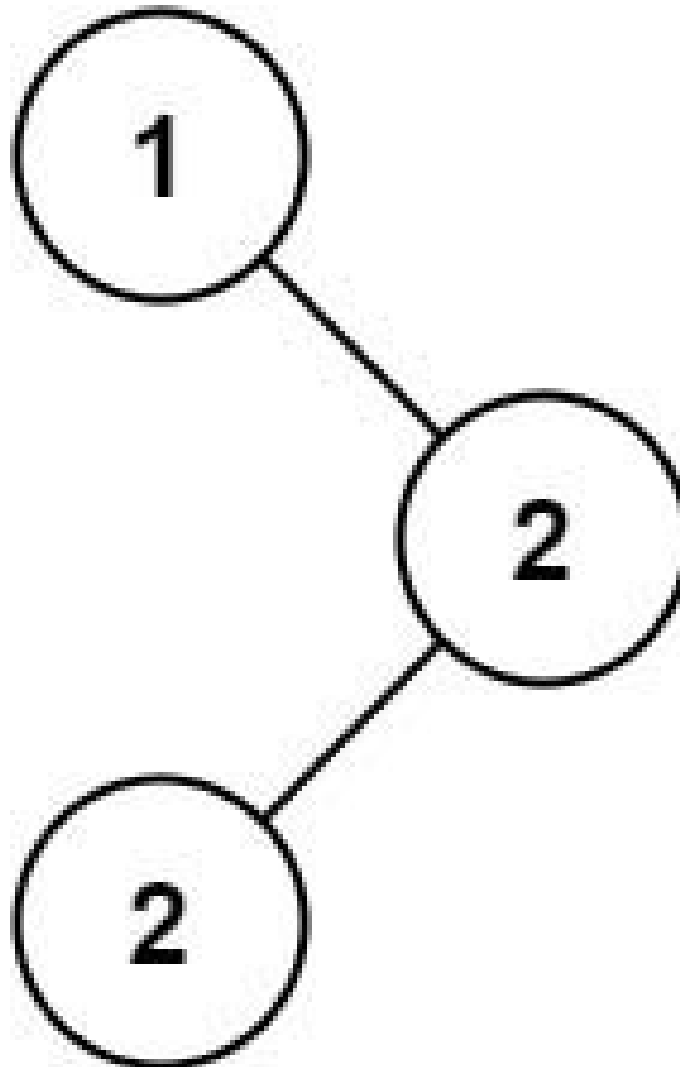
Given the root of a binary search tree (BST) with duplicates, return all the mode(s) (i.e., the most frequently occurred element) in it.

If the tree has more than one mode, return them in any order.

Assume a BST is defined as follows:

- The left subtree of a node contains only nodes with keys less than or equal to the node's key.
- The right subtree of a node contains only nodes with keys greater than or equal to the node's key.
- Both the left and right subtrees must also be binary search trees.

Example 1:



Input: root = [1,null,2,2]

Output: [2]

Example 2:

Input: root = [0]

Output: [0]

Constraints:

- The number of nodes in the tree is in the range [1, 10⁴].
- $-105 \leq \text{Node.val} \leq 105$

Follow up: Could you do that without using any extra space? (Assume that the implicit stack space incurred due to recursion does not count).

My LeetCode Solution

• My LeetCode Solution

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def findMode(self, root: Optional[TreeNode]) -> List[int]:
        hmap = Counter()
        stack = [root]

        while stack:
            node = stack.pop()
            if node:
                hmap[node.val] += 1
                if node.right:
                    stack.append(node.right)

                if node.left:
                    stack.append(node.left)

        maxval = max(v for v in hmap.values()) if hmap else 0
        return [k for k in hmap if hmap[k] == maxval]
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return all values that appear the most frequently in the BST."

This BST allows duplicates ($\text{left} \leq \text{node}$, $\text{right} \geq \text{node}$).

Return all modes – there can be multiple. Any order is fine. Right?

Follow-up: can we do it without extra space ($O(1)$ beyond recursion stack)?

```

#
# "[1,null,2,2]: 2 appears twice, 1 once → mode is [2] ✓
# [0]: only element → mode is [0] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# DFS or BFS over all nodes and count frequencies with a hash map (Counter).
# Then collect all keys whose count equals the maximum frequency.
# Time: O(n). Space: O(n) for the Counter. Should I go ahead and optimize?"
class _501_FindMode_Brute:
    def findMode(self, root: Optional['TreeNode']) -> List[int]:
        freq: Counter = Counter()
        stack = [root]
        while stack:
            node = stack.pop()
            if node:
                freq[node.val] += 1
                stack.append(node.left)
                stack.append(node.right)
        if not freq:
            return []
        max_freq = max(freq.values())
        return [k for k, v in freq.items() if v == max_freq]

# PHASE 3 — OPTIMIZE (O(1) space using in-order traversal)
# "BST in-order traversal visits nodes in sorted order.
# Duplicates in a BST are adjacent in in-order – so we can count runs without a map.
# Track: current value, current run count, max count seen, result list.
# When current run count exceeds max, reset result. When equal, append.
# Use nonlocal to mutate state inside the inner helper."

# PHASE 4 — CLEAN CODE
class _501_FindMode:
    def findMode(self, root: Optional['TreeNode']) -> List[int]:
        modes: List[int] = []
        cur_val = [None]
        cur_cnt = [0]
        max_cnt = [0]
        def inorder(node):
            if not node:
                return
            inorder(node.left)
            if node.val == cur_val[0]:
                cur_cnt[0] += 1
            else:
                cur_val[0] = node.val
                cur_cnt[0] = 1
            if cur_cnt[0] > max_cnt[0]:
                max_cnt[0] = cur_cnt[0]
                modes.clear()

```

```

        modes.append(node.val)
    elif cur_cnt[0] == max_cnt[0]:
        modes.append(node.val)
    inorder(node.right)
inorder(root)
return modes

```

PHASE 5 — TEST & DEBUG

```

# root=[1,null,2,2] – in-order: 1, 2, 2
# node=1: cur_val=1,cnt=1,max=1 → modes=[1].
# node=2: cur_val=2,cnt=1,max=1 → cnt==max → modes=[1,2].
# node=2: cur_val=2,cnt=2,max=1 → cnt>max → max=2,modes=[2].
# return [2] ✓
# root=[0]: in-order: 0. modes=[0] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Brute force: O(n) time, O(n) space.
# O(1) space version: O(n) time, O(1) extra space (modes list not counted).
# The in-order run-counting trick works because BST duplicates are always adjacent
# in sorted traversal – a key property of this BST variant.
# Given more time I'd ask: what if it were a regular BST (no duplicates)?
# The mode would always be the single element – trivially the root if n=1."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def findMode(self, root: TreeNode | None) -> list[int]:
        self.ans = []
        self.pred = None
        self.count = 0
        self.maxCount = 0

        def updateCount(root: TreeNode | None) -> None:
            if self.pred and self.pred.val == root.val:
                self.count += 1

```



```

else:
    self.count = 1

    if self.count > self.maxCount:
        self.maxCount = self.count
        self.ans = [root.val]
    elif self.count == self.maxCount:
        self.ans.append(root.val)

    self.pred = root

```

```

def inorder(root: TreeNode | None) -> None:
    if not root:
        return

    inorder(root.left)
    updateCount(root)
    inorder(root.right)

inorder(root)

return self.ans

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def findMode(self, root: TreeNode) -> List[int]:
        def dfs(root):
            if root is None:

```

```

        return
    nonlocal mx, prev, ans, cnt
    dfs(root.left)
    cnt = cnt + 1 if prev == root.val else 1
    if cnt > mx:
        ans = [root.val]
        mx = cnt
    elif cnt == mx:
        ans.append(root.val)
    prev = root.val

    dfs(root.right)

prev = None
mx = cnt = 0
ans = []
dfs(root)
return ans

```

• SimplyLeet

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def findMode(self, root):
        # Initialize state variables: prev value, current frequency count,
        # maximum frequency, and modes list.

        self.prev = None
        self.currentCount = 0
        self.maxCount = 0
        self.modes = []

        # First in-order traversal to find the maximum frequency (maxCount).
        def inorder_first(node):
            if not node:
                return
            inorder_first(node.left)

```

```

        # Update the count for the current node.
        if self.prev is None or node.val != self.prev:
            self.currentCount = 1
        else:
            self.currentCount += 1
        self.prev = node.val
        # Update maxCount if currentCount exceeds it.
        self.maxCount = max(self.maxCount, self.currentCount)
        inorder_first(node.right)

```

```

inorder_first(root)

```

```

        # Second in-order traversal to collect all node values with frequency
        equal to maxCount.

```

```

        self.prev = None # Reset previous value.
        self.currentCount = 0 # Reset count.

```

```

def inorder_second(node):
    if not node:
        return
    inorder_second(node.left)

```

```

        if self.prev is None or node.val != self.prev:
            self.currentCount = 1
        else:
            self.currentCount += 1
        self.prev = node.val
        if self.currentCount == self.maxCount:
            self.modes.append(node.val)
        inorder_second(node.right)

```

```

inorder_second(root)

```

```

return self.modes

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def findMode(self, root: TreeNode) -> List[int]:
        def dfs(root):
            if root is None:
                return
            nonlocal mx, prev, ans, cnt
            dfs(root.left)
            cnt = cnt + 1 if prev == root.val else 1
            if cnt > mx:
                ans = [root.val]
                mx = cnt
            elif cnt == mx:
                ans.append(root.val)
            prev = root.val
            dfs(root.right)

        prev = None
        mx = cnt = 0
        ans = []
        dfs(root)
        return ans

```

◆ Quick Review

Key Insights

- The BST in-order traversal yields a sorted order, which helps in counting consecutive duplicates.

- A two-pass in-order traversal can be used: one pass to determine the maximum frequency (maxCount) and a second pass to collect all values that match this frequency.
- The solution can be implemented without extra space (apart from recursion stack) by leveraging constant space counters and state variables.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes, because each node is processed twice (once in each traversal).

Space Complexity: $O(h)$ where h is the height of the tree due to the recursion stack. $O(1)$ extra space is used aside from this.

Solution

We perform an in-order traversal of the BST twice. The first traversal determines the maximum frequency of any value in the BST by comparing the current node's value with the previously visited node. Since the in-order traversal processes nodes in sorted order, duplicate values are consecutive, making the counting simple. After calculating maxCount,

we reset our state and perform a second in-order traversal to collect all node values with frequency equal to maxCount. The key trick is to leverage the BST property (sorted in-order traversal) to count frequencies in one pass without using additional data structures like hash maps.

Trees & BST

□ #543 Diameter of Binary Tree

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS

Lists: Grind 169

Problem

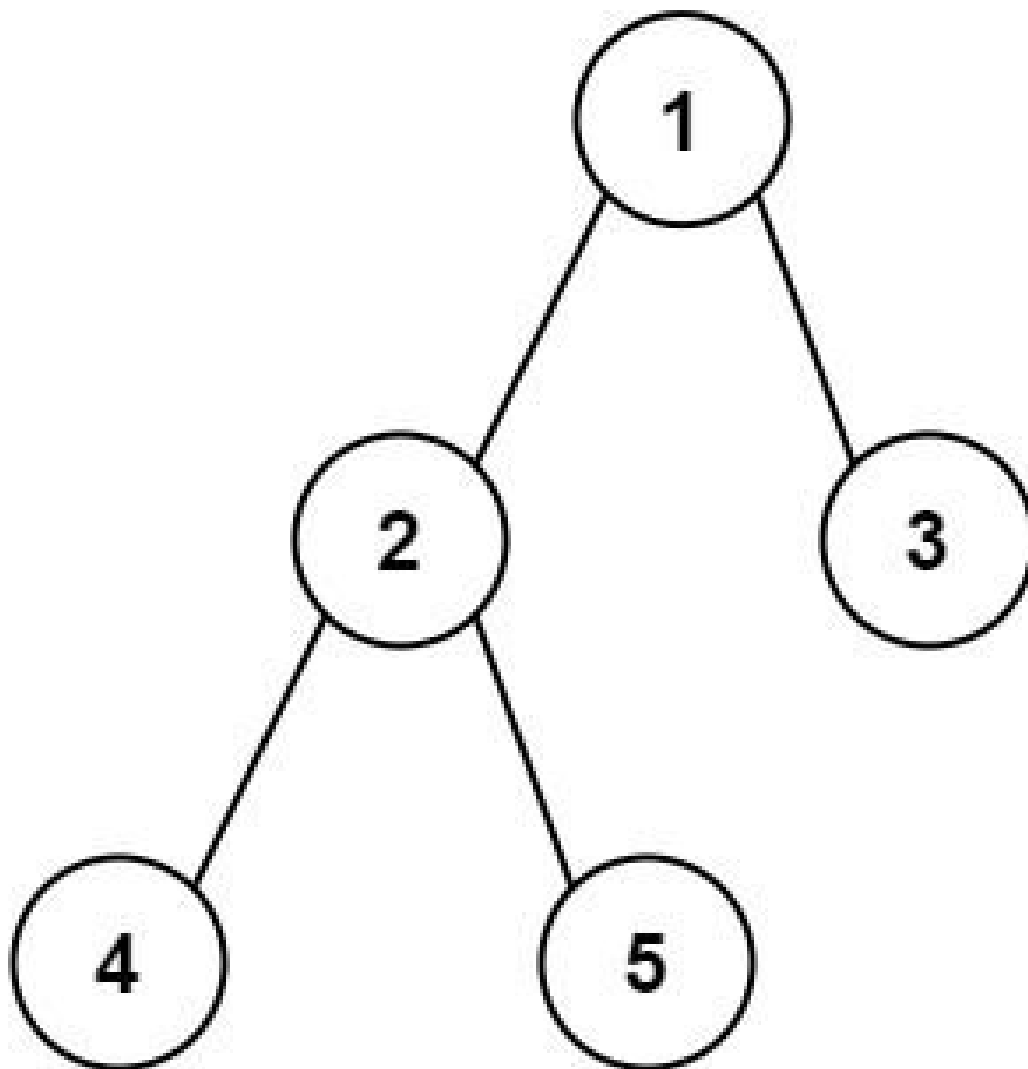
Given the root of a binary tree, return the length of the diameter of the tree.

The diameter of a binary tree is the length of the longest path between any two nodes in a tree.

This path may or may not pass through the root.

The length of a path between two nodes is represented by the number of edges between them.

Example 1:



Input: root = [1,2,3,4,5]

Output: 3

Explanation: 3 is the length of the path [4,2,1,3] or [5,2,1,3].

Example 2:

Input: root = [1,2]

Output: 1

Constraints:

- The number of nodes in the tree is in the range [1, 10⁴].
- $-100 \leq \text{Node.val} \leq 100$

My LeetCode Solution

• My LeetCode Solution

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def diameterOfBinaryTree(self, root: Optional[TreeNode]) -> int:
        def depth(root):
            if not root:
                return 0

            return max(depth(root.left), depth(root.right)) + 1

        self.ans = 0
        def diameter(root):
            if not root:
                return 0

            l = depth(root.left)

            r = depth(root.right)
            self.ans = max(self.ans, l+r)
            diameter(root.left)
            diameter(root.right)

            return self.ans

        return diameter(root)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Diameter = the longest path between any two nodes, measured in edges.

The path may or may not pass through the root.

An empty tree has diameter 0. A single node has diameter 0 (no edges). Right?"

#

"[1,2,3,4,5]: longest path is 4→2→1→3, 3 edges → 3 ✓

```
# [1,2]: one edge → 1 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# For every node, compute the height of its left and right subtrees recursively.
# Diameter through that node = left_height + right_height; track the global max.
# Recomputes heights repeatedly – correct but redundant work.
# Time:  $O(n^2)$  on skewed trees. Space:  $O(h)$  stack.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "I notice depth is recomputed repeatedly for the same nodes – that's the  $O(n^2)$ .
# Fuse depth computation and diameter tracking into one DFS pass.
# Use a nonlocal variable (or single-element list) to capture the running max.
# depth(node) returns height AND updates the max diameter seen so far.
# Time:  $O(n)$  – one pass. Space:  $O(h)$ ."
```

PHASE 4 — CLEAN CODE

```
class _543_DiameterBinaryTree:
    def diameterOfBinaryTree(self, root: Optional['TreeNode']) -> int:
        best = [0]
        def depth(node):
            if not node:
                return 0
            left = depth(node.left)
            right = depth(node.right)
            best[0] = max(best[0], left + right) # diameter through this node
            return max(left, right) + 1 # height for parent
        depth(root)
        return best[0]
```

PHASE 5 — TEST & DEBUG

```
# root=[1,2,3,4,5]:
# depth(4)=1. depth(5)=1. depth(2): best=max(0,1+1)=2. return 2.
# depth(3)=1. depth(1): best=max(2,2+1)=3. return 3.
# return best[0]=3 ✓
# root=[1,2]: depth(2)=1. depth(1): best=max(0,1+0)=1. return best=1 ✓
# root=None: depth(None)=0. best stays 0. return 0 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ . Space  $O(h)$ .
# The single-element list best=[0] is a common Python pattern to mutate
# a value from inside a nested function without making it a class attribute.
# Equivalent to 'nonlocal best' with a plain int, but the list avoids the
# nonlocal keyword – both are valid in an interview.
# Given more time I'd ask: what's the longest path by node count (not edges)?
# Just return best[0] + 1 when best > 0. Or: diameter in edge terms is already
# what we compute – node count would be edges + 1."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def diameterOfBinaryTree(self, root: TreeNode | None) -> int:
        ans = 0

        def maxDepth(root: TreeNode | None) -> int:
            nonlocal ans
            if not root:
                return 0

            l = maxDepth(root.left)

            r = maxDepth(root.right)
            ans = max(ans, l + r)
            return 1 + max(l, r)

        maxDepth(root)
        return ans
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def diameterOfBinaryTree(self, root: TreeNode) -> int:
        def dfs(root):
            if root is None:
                return 0
            nonlocal ans
            left, right = dfs(root.left), dfs(root.right)
            ans = max(ans, left + right)
            return 1 + max(left, right)

        ans = 0
        dfs(root)
        return ans
```

• SimplyLeet

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def diameterOfBinaryTree(self, root: TreeNode) -> int:
        # Global variable to track the maximum diameter found

        self.max_diameter = 0

        def dfs(node):
            # Base case: if node is None, depth is 0.
            if not node:
                return 0
            # Compute the depth of left and right subtrees.
            left_depth = dfs(node.left)
            right_depth = dfs(node.right)
            # Update the global diameter: the path through this node

            self.max_diameter = max(self.max_diameter, left_depth + right_depth)

            # Return the depth of the tree rooted at this node.
            return max(left_depth, right_depth) + 1

        dfs(root)
        return self.max_diameter
```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def diameterOfBinaryTree(self, root: TreeNode) -> int:
        def dfs(root):
            if root is None:
                return 0
            nonlocal ans
            left, right = dfs(root.left), dfs(root.right)
            ans = max(ans, left + right)
            return 1 + max(left, right)

        ans = 0
        dfs(root)
        return ans

```

◆ Quick Review

Key Insights

- Use depth-first search (DFS) to explore the tree.
- The diameter at a node is the sum of the depths of its left and right subtrees.
- Track the maximum diameter found during the DFS traversal.
- Recursively calculating the depth of subtrees provides an efficient solution.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the tree.

Space Complexity: $O(n)$, in the worst-case scenario due to the recursion stack (skewed tree).

Solution

The solution employs a DFS traversal of the binary tree. At every node in the tree, compute the depth of the left and right subtrees. The sum of these depths gives the potential diameter (longest path through that node). Update a global maximum diameter variable if this sum exceeds the current maximum. Finally, return the maximum diameter after completing the DFS.

Trees & BST

□ #572 Subtree of Another Tree

EAS

[Open on LeetCode](#)

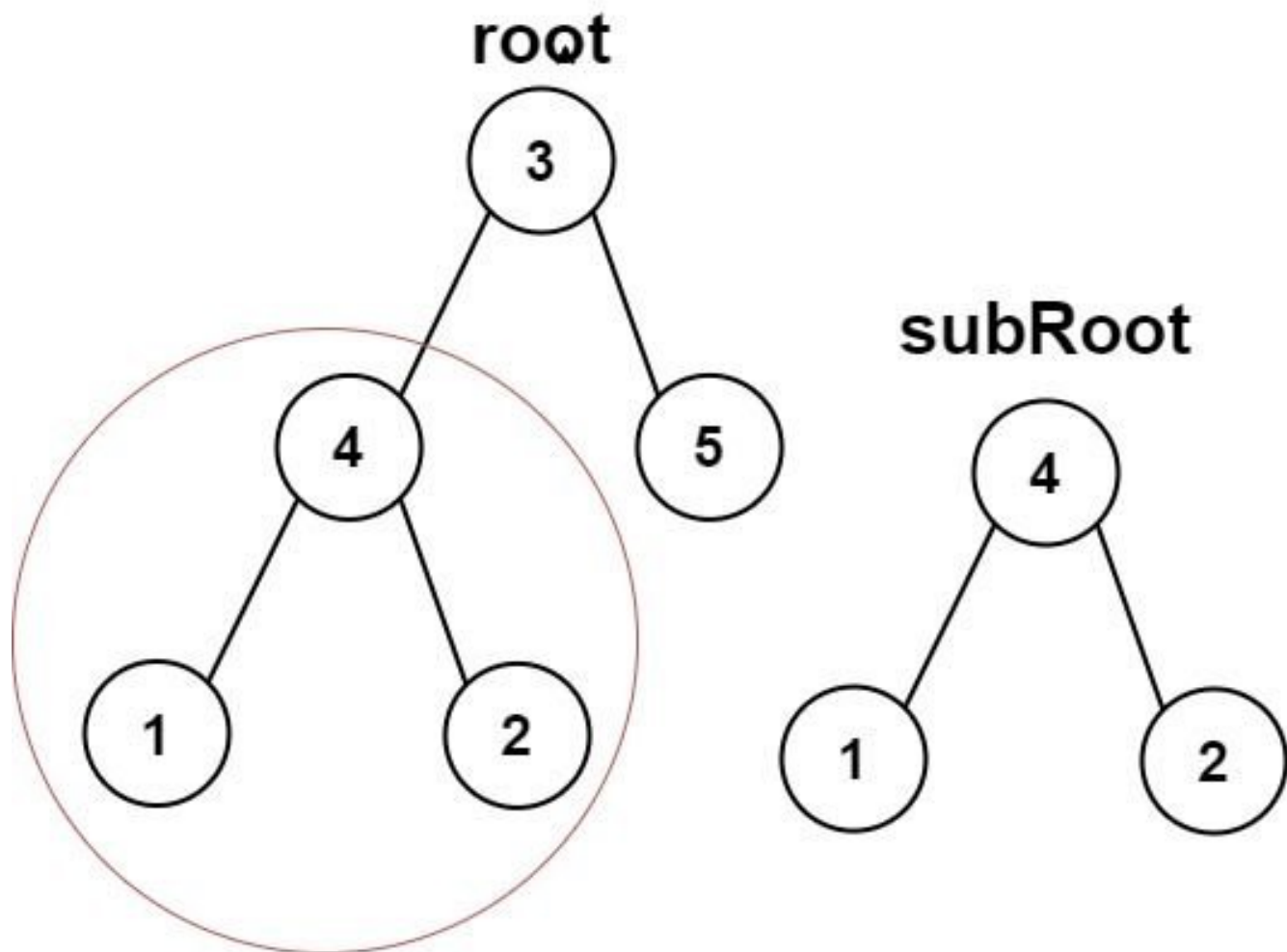
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · String Matching
Lists: Grind 169

Problem

Given the roots of two binary trees `root` and `subRoot`, return `true` if there is a subtree of `root` with the same structure and node values of `subRoot` and `false` otherwise.

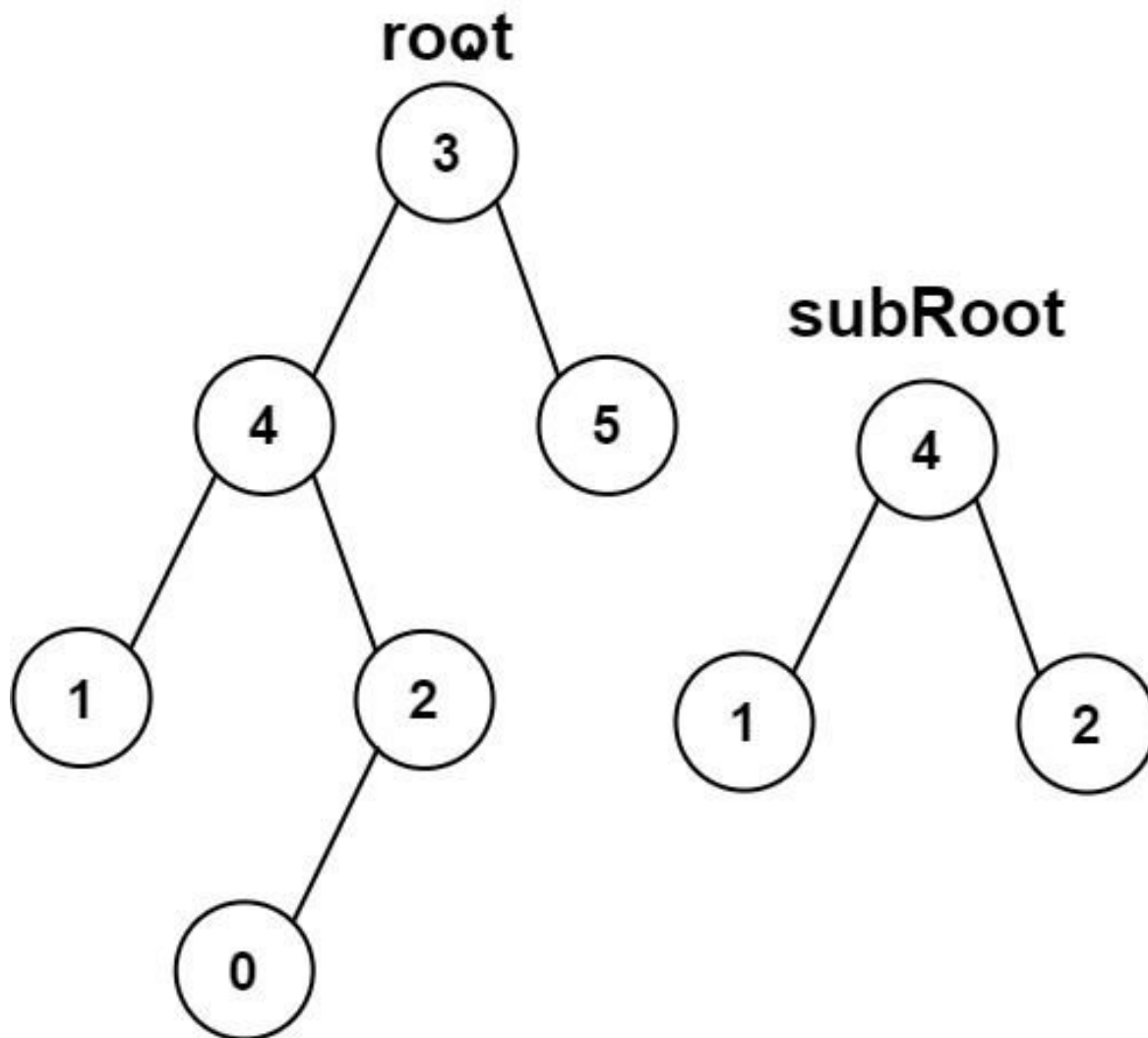
A subtree of a binary tree `tree` is a tree that consists of a node in `tree` and all of this node's descendants. The tree `tree` could also be considered as a subtree of itself.

Example 1:



Input: root = [3,4,5,1,2], subRoot = [4,1,2]
Output: true

Example 2:



Input: root = [3,4,5,1,2,null,null,null,null,0], subRoot = [4,1,2]
Output: false

Constraints:

- The number of nodes in the root tree is in the range [1, 2000].
- The number of nodes in the subRoot tree is in the range [1, 1000].
- $-104 \leq \text{root.val} \leq 104$
- $-104 \leq \text{subRoot.val} \leq 104$

My LeetCode Solution

• My LeetCode Solution

```
# Subtree of Another Tree
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSubtree(self, root: Optional[TreeNode], subRoot: Optional[TreeNode])
    -> bool:
        def same(p,q):

            if not p or not q:
                return p == q

            if p.val != q.val:
                return False

            return same(p.left, q.left) and same(p.right, q.right)

        def subtree(root, subRoot):
            if not root:

                return False

            if same(root,subRoot):
                return True

            return subtree(root.left, subRoot) or subtree(root.right, subRoot)

        return subtree(root, subRoot)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return true if subRoot appears as a subtree anywhere in root.

A subtree means a node and ALL its descendants match – not just a portion.

The tree itself is a subtree of itself. Right?"

#

```
# "root=[3,4,5,1,2], subRoot=[4,1,2]: node 4 in root has left=1,right=2 → matches ✓
# root=[3,4,5,1,2,null,null,null,null,0], subRoot=[4,1,2]:
# node 4 in root has extra child 0 → doesn't match ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# For every node in root, run the same-tree check against all of subRoot.
# If any starting node matches the entire subtree, return true.
# Correct, but rescans overlapping structure many times.
# Time:  $O(n * m)$ . Space:  $O(h)$  stack.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Same  $O(m*n)$  is standard. Two clean inner helpers:
# same(p,q) – checks if two trees are identical (reuse Same Tree logic).
# subtree(node, sub) – DFS root; at each node, try same().
# If same fails, recurse into left and right subtrees."
```

PHASE 4 — CLEAN CODE

```
class _572_SubtreeAnotherTree:
    def isSubtree(self, root: Optional['TreeNode'], subRoot: Optional['TreeNode'])
    -> bool:
        def same(p, q):
            if not p or not q:
                return p == q
            return p.val == q.val and same(p.left, q.left) and same(p.right,
q.right)
        def subtree(node, sub):
            if not node:
                return False
            if same(node, sub):
                return True
            return subtree(node.left, sub) or subtree(node.right, sub)
        return subtree(root, subRoot)
```

PHASE 5 — TEST & DEBUG

```
# root=[3,4,5,1,2], subRoot=[4,1,2]:
# subtree(3,[4,1,2]): same(3,4)?  $3 \neq 4 \rightarrow$  False. recurse left/right.
# subtree(4,[4,1,2]): same(4,4)?  $4 == 4$ .
# same(1,1)→same(None,None)T, same(None,None)T→T.
# same(2,2)→T. → same=True → return True ✓
# root has extra 0 child under 4:
# same(4,4): same(1,1)T. same(2(with 0),2)?  $2 == 2$  but same(0,None)→False →
same=False.
# subtree(1,[4,1,2]): same(1,4)? False. subtree deeper → all False → False ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(m*n)$  – at each of  $m$  nodes we may run same() which takes  $O(n)$ .
# Space  $O(h_{\text{root}} + h_{\text{sub}})$  for call stacks.
# Optimisation: serialise both trees and use string matching (KMP or Rabin-Karp)
```

for $O(m+n)$ time – worth mentioning as an advanced follow-up.
 # Be careful with serialisation: '[1,2]' would incorrectly match '[12]' without
 # delimiters like '#' for null and ',' between values.
 # Given more time I'd ask: what if we query many different subRoots against
 # the same root? Pre-serialise root once and use string search for each query."

Community Solutions (Python)

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSubtree(self, root: Optional[TreeNode], subRoot: Optional[TreeNode])
    -> bool:
        def same(p: Optional[TreeNode], q: Optional[TreeNode]) -> bool:
            if p is None or q is None:
                return p is q
            return p.val == q.val and same(p.left, q.left) and same(p.right, q
            .right)

        if root is None:
            return False
        return (
            same(root, subRoot)
            or self.isSubtree(root.left, subRoot)
            or self.isSubtree(root.right, subRoot)
        )
```

• SimplyLeet

```
# Python solution for checking if subRoot is a subtree of root.
```

```
# Definition for a binary tree node.
```

```
# class TreeNode:
```

```
# def __init__(self, val=0, left=None, right=None):
```

```
# self.val = val
```

```
# self.left = left
```

```
# self.right = right
```

```
class Solution:
```

```
def isSubtree(self, root: TreeNode, subRoot: TreeNode) -> bool:
```

```
    # Helper function to check if two trees are identical.
```

```
    def isSameTree(s, t):
```

```
        # Both nodes are None, trees match.
```

```
        if not s and not t:
```

```
            return True
```

```
        # One node is None or values differ.
```

```
        if not s or not t or s.val != t.val:
```

```
            return False
```

```
        # Recurse on left and right subtrees.
```

```
        return isSameTree(s.left, t.left) and isSameTree(s.right, t.right)
```

```
    # Base case: if root is None, then no subtree exists.
```

```
    if not root:
```

```
        return False
```

```
    # Check if current subtree is identical to subRoot.
```

```
    if isSameTree(root, subRoot):
```

```
        return True
```

```
    # Recursively check left and right subtrees.
```

```
    return self.isSubtree(root.left, subRoot) or self.isSubtree(root.right, subRoot)
```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSubtree(self, root: TreeNode, subRoot: TreeNode) -> bool:
        def dfs(root1, root2):
            if root1 is None and root2 is None:
                return True
            if root1 is None or root2 is None:
                return False
            return (
                root1.val == root2.val
                and dfs(root1.left, root2.left)
                and dfs(root1.right, root2.right)
            )

        if root is None:
            return False
        return (
            dfs(root, subRoot)
            or self.isSubtree(root.left, subRoot)
            or self.isSubtree(root.right, subRoot)
        )

```

◆ Quick Review

Key Insights

- Use Depth-First Search (DFS) to visit each node in the main tree.
- At each node, check if the subtree starting from that node is identical to subRoot.

- A helper function is used to compare the structure and node values of two trees.
- Consider recursion as the primary approach to traverse and compare trees.
- Early exit when a mismatch is found can optimize average-case performance.

Space & Time

Time Complexity: $O(m * n)$ in the worst case, where m is the number of nodes in the main tree and n is the number of nodes in the subRoot tree.

Space Complexity: $O(\max(\text{depth of main tree}, \text{depth of subRoot}))$ due to recursive call stack.

Solution

We solve the problem by performing a DFS traversal on the main tree. For each visited node, a helper function checks whether the subtree starting at that node is identical to subRoot by comparing node values and recursively checking both left and right subtrees. This method leverages recursion effectively and provides early termination if mismatches occur, allowing for efficient traversal and comparison.

DFS

Round 1 · High · Easy · 2 questions

DFS

□ #463 Island Perimeter

EAS

[Open on LeetCode](#)

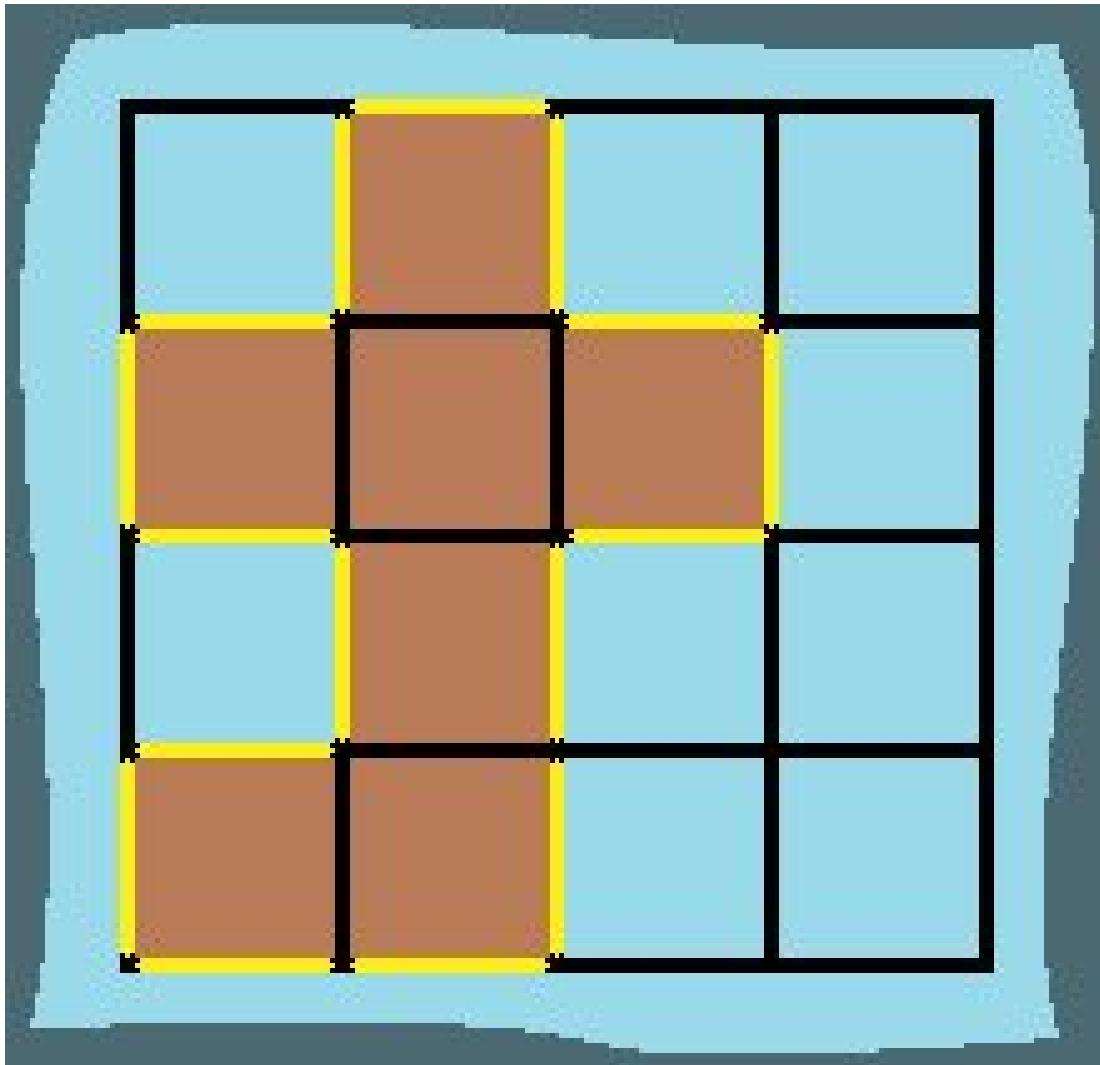
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Matrix
Lists: Denny Zhang

Problem

You are given row x col grid representing a map where $\text{grid}[i][j] = 1$ represents land and $\text{grid}[i][j] = 0$ represents water.

Grid cells are connected horizontally/vertically (not diagonally). The grid is completely surrounded by water, and there is exactly one island (i.e., one or more connected land cells). The island doesn't have "lakes", meaning the water inside isn't connected to the water around the island. One cell is a square with side length 1. The grid is rectangular, width and height don't exceed 100. Determine the perimeter of the island.

Example 1:



Input: grid = `[[0,1,0,0],[1,1,1,0],[0,1,0,0],[1,1,0,0]]`

Output: 16

Explanation: The perimeter is the 16 yellow stripes in the image above.

Example 2:

Input: grid = `[[1]]`

Output: 4

Example 3:

Input: grid = `[[1,0]]`

Output: 4

Constraints:

- row == grid.length

- `col == grid[i].length`
- `1 <= row, col <= 100`
- `grid[i][j]` is 0 or 1.
- There is exactly one island in grid.

My LeetCode Solution

• My LeetCode Solution

```
# Island Perimeter
class Solution:
    def islandPerimeter(self, grid: List[List[int]]) -> int:
        rows = len(grid)
        cols = len(grid[0])

        islands, neighbors = 0, 0
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == 1:

                    islands += 1

                    if r+1 < rows and grid[r+1][c] == 1:
                        neighbors += 1

                    if c+1 < cols and grid[r][c+1] == 1:
                        neighbors += 1

        return islands * 4 - neighbors*2
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"The grid has exactly one island – no lakes. Each land cell is a unit square.

We return the total perimeter, which is the count of exposed edges. Right?

Exposed means the edge faces water or the grid boundary."

#

"Quick check: `[[1]]` → 4 sides, all exposed → 4 ✓. `[[1,0]]` → 4 sides, all exposed → 4 ✓.

Two adjacent land cells: each has 4 sides but share 1 edge → $2 \times 4 - 2 = 6$."

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

For every land cell, I check all 4 neighbours – if a neighbour is out of bounds or water,

that edge contributes 1 to the perimeter. Sum these up across all land cells.

Time: $O(m \times n)$. Space: $O(1)$. Should I go ahead and optimize?"

```
class _463_IslandPerimeterBrute:
    def islandPerimeter(self, grid: List[List[int]]) -> int:
        rows, cols = len(grid), len(grid[0])
        perimeter = 0
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == 1:
                    for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                        nr, nc = r + dr, c + dc
                        if nr < 0 or nr >= rows or nc < 0 or nc >= cols or
grid[nr][nc] == 0:
                            perimeter += 1
        return perimeter
```

PHASE 3 — OPTIMIZE

"I notice we're checking 4 neighbours per cell. Instead of neighbour checks,

we can use a counting formula:

Every land cell contributes 4 sides. Every shared edge between two adjacent

land cells removes 2 sides (one from each). So:

$\text{perimeter} = \text{islands} \times 4 - \text{shared_edges} \times 2$

We only count each shared edge once by checking right-neighbour and down-neighbour.

Same $O(m \times n)$ time, same $O(1)$ space – but a cleaner mental model."

PHASE 4 — CLEAN CODE

```
class _463_IslandPerimeter:
    def islandPerimeter(self, grid: List[List[int]]) -> int:
        rows, cols = len(grid), len(grid[0])
        islands = 0
        shared = 0
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == 1:
                    islands += 1
                    if r + 1 < rows and grid[r + 1][c] == 1:
                        shared += 1
                    if c + 1 < cols and grid[r][c + 1] == 1:
                        shared += 1
        return islands * 4 - shared * 2
```

PHASE 5 — TEST & DEBUG

`grid=[[1]]`: islands=1, shared=0. return 4 ✓

```
# grid=[[1,1]]: islands=2. c=0→c+1=1 valid,grid[0][1]=1→shared=1. return 2*4-1*2=6
✓
# grid=[[0,1,0,0],[1,1,1,0],[0,1,0,0],[1,1,0,0]]:
# Count land cells=8 (islands). Count shared edges (right+down only)=... =8.
# return 8*4-8*2=32-16=16 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(1)$.

The counting formula is elegant – worth presenting even if the brute force also works.

Both approaches are the same complexity; the formula shows mathematical insight.

Given more time I'd ask: what if there are multiple islands?

The formula still works per-island if we isolate them, or we can return a list

of perimeters – same logic applied per connected component."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def islandPerimeter(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])

        islands = 0
        neighbors = 0

        for i in range(m):
            for j in range(n):

                if grid[i][j] == 1:
                    islands += 1
                    if i + 1 < m and grid[i + 1][j] == 1:
                        neighbors += 1
                    if j + 1 < n and grid[i][j + 1] == 1:
                        neighbors += 1

        return islands * 4 - neighbors * 2
```

• LeetDoocs


```

class Solution:
    def islandPerimeter(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        ans = 0
        for i in range(m):
            for j in range(n):
                if grid[i][j] == 1:
                    ans += 4
                    if i < m - 1 and grid[i + 1][j] == 1:
                        ans -= 2

                    if j < n - 1 and grid[i][j + 1] == 1:
                        ans -= 2

        return ans

```

• SimplyLeet

```

# Python solution for Island Perimeter
def islandPerimeter(grid):
    # Initialize perimeter to 0
    perimeter = 0

    # Get number of rows and columns
    rows = len(grid)
    cols = len(grid[0])

    # Iterate through each cell in the grid

    for i in range(rows):
        for j in range(cols):
            # Only process if current cell is land
            if grid[i][j] == 1:
                # Each land cell contributes 4 edges initially
                cell_perimeter = 4

                # Check if the upper neighbor is also land
                if i > 0 and grid[i-1][j] == 1:
                    cell_perimeter -= 1

```

```

        # Check if the lower neighbor is also land
        if i < rows - 1 and grid[i+1][j] == 1:
            cell_perimeter -= 1
        # Check if the left neighbor is also land
        if j > 0 and grid[i][j-1] == 1:
            cell_perimeter -= 1
        # Check if the right neighbor is also land
        if j < cols - 1 and grid[i][j+1] == 1:
            cell_perimeter -= 1

        # Add the effective perimeter of the current cell to the
        overall perimeter
        perimeter += cell_perimeter

    return perimeter

# Example usage:
# grid = [[0,1,0,0],[1,1,1,0],[0,1,0,0],[1,1,0,0]]
# print(islandPerimeter(grid)) # Output should be 16

```

• LC.ca

```

class Solution:
    def islandPerimeter(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        ans = 0
        for i in range(m):
            for j in range(n):
                if grid[i][j] == 1:
                    ans += 4
                    if i < m - 1 and grid[i + 1][j] == 1:
                        ans -= 2

                    if j < n - 1 and grid[i][j + 1] == 1:
                        ans -= 2

        return ans

```

◆ Quick Review

Key Insights

- Each land cell contributes 4 edges if isolated.
- For every adjacent pair of land cells (neighbors horizontally or vertically), the shared edge should be subtracted from the total perimeter.
- Instead of DFS/BFS, we can iterate the grid and check the four neighbors for each land cell.
- Alternatively, one may use DFS/BFS to visit all connected cells if needed, but an iterative solution is straightforward with $O(\text{rows} * \text{cols})$ time complexity.

Space & Time

Time Complexity: $O(n * m)$, where n is the number of rows and m is the number of columns.

Space Complexity: $O(1)$ for the iterative solution (excluding the input storage).

Solution

We iterate through every cell in the grid. For each land cell, we initially add 4 to the perimeter. Then we check its four neighbors (up, down, left, right). For every neighbor that is also land, we subtract 1 from the current

cell's contribution since that side is not exposed. This simple approach ensures that shared borders between adjacent land cells are not over-counted.

DFS

□ #733 Flood Fill

EAS[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Matrix
Lists: Grind 169

Problem

You are given an image represented by an $m \times n$ grid of integers `image`, where `image[i][j]` represents the pixel value of the image. You are also given three integers `sr`, `sc`, and `color`. Your task is to perform a flood fill on the image starting from the pixel `image[sr][sc]`.

To perform a flood fill:

1. Begin with the starting pixel and change its color to `color`.
2. Perform the same process for each pixel that is directly adjacent (pixels that share a side with the original pixel, either horizontally or vertically) and shares the same color as the starting pixel.

3. Keep repeating this process by checking neighboring pixels of the updated pixels and modifying their color if it matches the original color of the starting pixel.
4. The process stops when there are no more adjacent pixels of the original color to update.

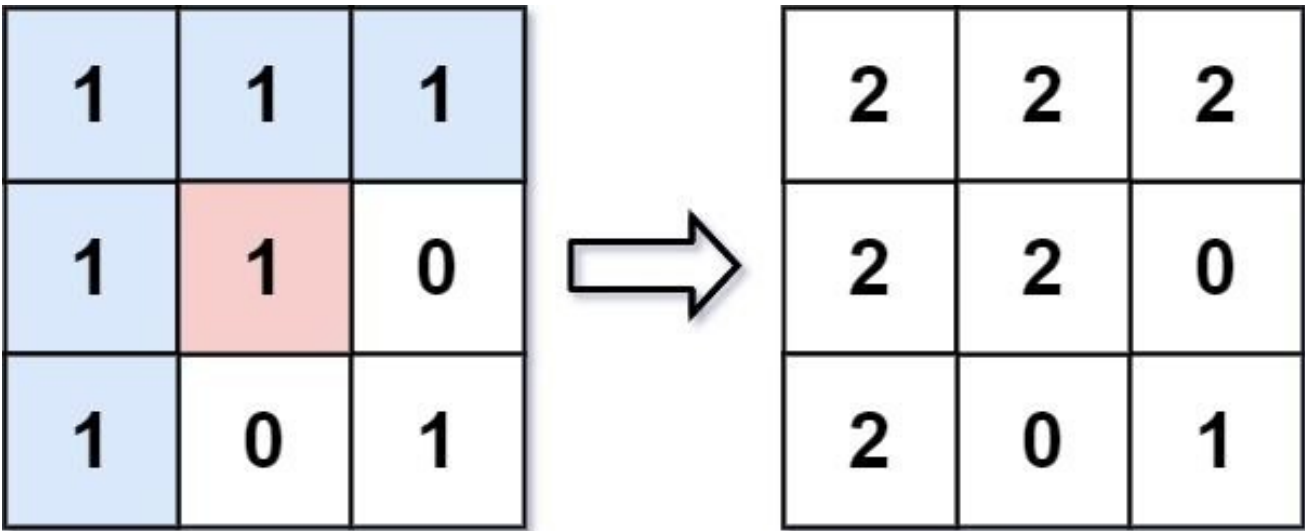
Return the modified image after performing the flood fill.

Example 1:

Input: image = [[1,1,1],[1,1,0],[1,0,1]], sr = 1, sc = 1, color = 2

Output: [[2,2,2],[2,2,0],[2,0,1]]

Explanation:



From the center of the image with position (sr, sc) = (1, 1) (i.e., the red pixel), all pixels connected by a path of the same color as the

starting pixel (i.e., the blue pixels) are colored with the new color.

Note the bottom corner is not colored 2, because it is not horizontally or vertically connected to the starting pixel.

Example 2:

Input: image = [[0,0,0],[0,0,0]], sr = 0, sc = 0, color = 0

Output: [[0,0,0],[0,0,0]]

Explanation:

The starting pixel is already colored with 0, which is the same as the target color. Therefore, no changes are made to the image.

Constraints:

- $m == \text{image.length}$
- $n == \text{image}[i].\text{length}$
- $1 \leq m, n \leq 50$
- $0 \leq \text{image}[i][j], \text{color} < 216$
- $0 \leq \text{sr} < m$
- $0 \leq \text{sc} < n$

My LeetCode Solution

- My LeetCode Solution


```
# Flood Fill
class Solution:
    def floodFill(self, image: List[List[int]], sr: int, sc: int, color: int)
-> List[List[int]]:
    def dfs(x,y):
        if x<0 or y<0 or x>= rows or y>= cols:
            return

        if image[x][y] != startcolor or (x,y) in seen:
            return

        image[x][y] = color
        seen.add((x,y))
        dfs(x+1, y)
        dfs(x-1, y)
        dfs(x,y+1)
        dfs(x,y-1)

    startcolor = image[sr][sc]
    seen = set()
    rows = len(image)

    cols = len(image[0])
    if startcolor == color:
        return image

    dfs(sr,sc)

    return image
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Starting from (sr, sc), change all connected cells of the same colour to the new colour.

Connected means 4-directionally (up/down/left/right). Modify and return the image. Right?

Edge case: if the starting colour already equals the new colour – return immediately,

otherwise we'd loop infinitely or process unchanged cells."

#

```
# "[[1,1,1],[1,1,0],[1,0,1]], sr=1,sc=1, color=2:
# All 1s connected to (1,1) become 2. The bottom-right 1 is isolated → stays 1. ✓
# [[0,0,0],[0,0,0]], sr=0,sc=0, color=0: startcolor==color → no change → return
as-is ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# I'll use BFS: enqueue the start cell, paint it the new colour, then for each cell
I dequeue,
# enqueue all 4-directional neighbours that still have the original start colour.
# This visits every connected cell exactly once.
# Time: O(m*n). Space: O(m*n) for the queue. Should I go ahead and optimize?"
```

```
class _733_FloodFill_Brute:
    def floodFill(self, image, sr, sc, color):
        from collections import deque
        start = image[sr][sc]
        if start == color:
            return image
        rows, cols = len(image), len(image[0])
        queue = deque([(sr, sc)])
        image[sr][sc] = color
        while queue:
            r, c = queue.popleft()
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols and image[nr][nc] == start:
                    image[nr][nc] = color
                    queue.append((nr, nc))
        return image
```

PHASE 3 — OPTIMIZE

```
# "DFS is cleaner. Key insight: once we paint a cell, its colour changes to `color`
—
# it can never match `startcolor` again. So we don't need a `seen` set at all.
# The colour change itself is the visited marker.
# This saves O(m*n) extra space."
```

PHASE 4 — CLEAN CODE

```
class _733_FloodFill:
    def floodFill(self, image: List[List[int]], sr: int, sc: int, color: int) ->
List[List[int]]:
        start = image[sr][sc]
        if start == color:
            return image
        rows, cols = len(image), len(image[0])
        def dfs(r, c):
            if r < 0 or r >= rows or c < 0 or c >= cols:
                return
            if image[r][c] != start:
                return
```

```

        image[r][c] = color # paint before recursing – acts as visited marker
        dfs(r + 1, c)
        dfs(r - 1, c)
        dfs(r, c + 1)
        dfs(r, c - 1)

    dfs(sr, sc)
    return image

```

PHASE 5 — TEST & DEBUG

```

# image=[[1,1,1],[1,1,0],[1,0,1]], sr=1,sc=1, color=2, start=1:
# dfs(1,1): image[1][1]=2. recurse 4 dirs.
# dfs(2,1): image[2][1]=1==start → paint 2. recurse: dfs(3,1)→00B, dfs(1,1)→now 2≠1
stop.
# dfs(2,2)→image[2][2]=1 but that's the isolated corner – wait:
# (2,1)→image[2][1]=1✓→paint. Then dfs(2,2)→image[2][2]=1✓→paint? No –
# is (2,2) reachable from (1,1)? Path: (1,1)→(2,1)→(2,2).
# image[2][2]=1 and image[2][1]=1, so yes connected. But expected output keeps it 1?
# Re-check: image=[[1,1,1],[1,1,0],[1,0,1]]. (2,2) is adjacent to (2,1)=0 and
(1,2)=0.
# So (2,2) connects to (2,1)=0 (water) – not reachable from island. ✓
# All connected 1s become 2. Bottom-right stays 1 (not connected). ✓
#
# start==color: image=[[0,0,0],[0,0,0]], color=0 → return immediately ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(m*n) – each cell visited at most once.
# Space O(m*n) call stack in the worst case (all cells same colour, grid is a long
chain).
# The 'paint first then recurse' order is the key – it prevents revisiting and
# eliminates the need for a separate visited set. Mention this explicitly.
# The early return when startcolor == color prevents infinite looping – critical
edge case.
# Given more time I'd ask: what if connectivity is 8-directional (diagonals too)?
# Add four more direction pairs to the dfs calls – same algorithm."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def floodFill(self, image: list[list[int]],
                  sr: int, sc: int, newColor: int) -> list[list[int]]:
        startColor = image[sr][sc]
        seen = set()

        def dfs(i: int, j: int) -> None:
            if i < 0 or i == len(image) or j < 0 or j == len(image[0]):
                return
            if image[i][j] != startColor or (i, j) in seen:
                return

            image[i][j] = newColor
            seen.add((i, j))

            dfs(i + 1, j)
            dfs(i - 1, j)
            dfs(i, j + 1)
            dfs(i, j - 1)

        dfs(sr, sc)
        return image

```

• LeetDoocs

```

class Solution:
    def floodFill(
        self, image: List[List[int]], sr: int, sc: int, color: int
    ) -> List[List[int]]:
        def dfs(i: int, j: int):
            image[i][j] = color
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < len(image) and 0 <= y < len(image[0]) and image[x]
[y] == oc:
                    dfs(x, y)

```

```

        oc = image[sr][sc]
        if oc != color:
            dirs = (-1, 0, 1, 0, -1)
            dfs(sr, sc)
        return image

```

• SimplyLeet

```

def floodFill(image, sr, sc, color):
    # Get the original color of the starting pixel
    orig_color = image[sr][sc]
    # If the new color is the same as the original, return the image as is
    if orig_color == color:
        return image
    rows, cols = len(image), len(image[0])

    # Recursive DFS helper function to fill the image
    def dfs(r, c):

        # Check boundaries and if the current pixel is the same as the original
        color
        if r < 0 or r >= rows or c < 0 or c >= cols or image[r][c] != orig_color:
            return

        # Update the current pixel's color
        image[r][c] = color
        # Recursively call DFS in all four directions
        dfs(r - 1, c) # up
        dfs(r + 1, c) # down
        dfs(r, c - 1) # left
        dfs(r, c + 1) # right

    dfs(sr, sc)
    return image

```

• LC.ca

```
class Solution:
    def floodFill(
        self, image: List[List[int]], sr: int, sc: int, color: int
    ) -> List[List[int]]:
        def dfs(i, j):
            if (
                not 0 <= i < m
                or not 0 <= j < n
                or image[i][j] != oc
                or image[i][j] == color
            ):
                return
            image[i][j] = color
            for a, b in pairwise(dirs):
                dfs(i + a, j + b)

        dirs = (-1, 0, 1, 0, -1)
        m, n = len(image), len(image[0])
        oc = image[sr][sc]
        dfs(sr, sc)

        return image
```

◆ Quick Review

Key Insights

- The flood fill operation can be executed using either DFS (Depth First Search) or BFS (Breadth First Search).
- A check is needed at the beginning to determine if the new color is the same as the original; if so, no changes are needed.
- The algorithm recursively or iteratively visits each pixel that is directly adjacent and

matches the original color, then updates its color.

- The problem has a worst-case scenario where all pixels are the same color, leading to a full traversal of the grid.

Space & Time

Time Complexity: $O(m * n)$, where m and n are the dimensions of the image grid, because in the worst-case all cells are visited.

Space Complexity: $O(m * n)$ in the worst-case due to the recursion stack (or BFS queue) if the entire image needs to be filled.

Solution

We use a DFS approach to perform the flood fill. The algorithm starts at the given starting pixel and uses recursion to fill connected pixels with the same original color. The following steps are key: Check if the starting pixel's color is already the target color; if so, return the image immediately. Define a recursive helper function that: Checks boundary conditions. Updates the current pixel's color. Recursively calls itself on the four adjacent directions (up, down, left, right).

Return the modified image after the recursion completes. Data structures used: Recursion call stack for the DFS implementation. Variables to store the image dimensions and original color.

Round 2 · High · Medium

Round 2

High Priority · Medium

116 questions · 11 patterns

**Arrays & Hashing #57 #238 #281 #307 #454
#525 #560 #1010 #1272 #1429 #3159**

String #8 #249 #271 #635 #1138

**Two Pointers #11 #15 #16 #31 #75 #161 #167
#186 #189 #532 #923 #986 #1055 #2563**

**Sliding Window #3 #159 #340 #424 #438 #487
#1100 #1248**

Sorting #49 #56 #1152

**Binary Search #33 #34 #153 #300 #362 #528
#852 #981 #1011 #1060 #1062 #1533**

**Matrix #36 #48 #54 #59 #64 #73 #74 #221
#311 #498 #531 #723 #835 #1198**

**Trees & BST #98 #102 #103 #105 #199 #230
#235 #236 #250 #285 #298 #314 #333 #366**

Round 1 · High · Easy

p.421

**#437 #545 #549 #582 #662 #863 #1120 #1490
#1522 #1650**

**DFS #130 #133 #200 #207 #210 #261 #310
#323 #417 #490 #694 #695 #721 #785 #1236
#1242**

Graphs #128 #277 #1136

BFS #286 #322 #542 #994 #1197 #1730

Arrays & Hashing

Round 2 · High · Medium · 11 questions

Arrays & Hashing

□ #57 Insert Interval

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array

Lists: Grind 169

Problem

You are given an array of non-overlapping intervals `intervals` where `intervals[i] = [starti, endi]` represent the start and the end of the *i*th interval and `intervals` is sorted in ascending order by `starti`. You are also given an interval `newInterval = [start, end]` that represents the start and end of another interval.

Insert `newInterval` into `intervals` such that `intervals` is still sorted in ascending order by `starti` and `intervals` still does not have any overlapping intervals (merge overlapping intervals if necessary).

Return `intervals` after the insertion.

Note that you don't need to modify `intervals` in-place. You can make a new array and return

it.

Example 1:

Input: intervals = [[1,3],[6,9]], newInterval = [2,5]
Output: [[1,5],[6,9]]

Example 2:

Input: intervals = [[1,2],[3,5],[6,7],[8,10],[12,16]], newInterval = [4,8]
Output: [[1,2],[3,10],[12,16]]
Explanation: Because the new interval [4,8] overlaps with [3,5],[6,7],[8,10].

Constraints:

- $0 \leq \text{intervals.length} \leq 104$
- $\text{intervals}[i].\text{length} == 2$
- $0 \leq \text{start}_i \leq \text{end}_i \leq 105$
- intervals is sorted by start_i in ascending order.
- $\text{newInterval.length} == 2$
- $0 \leq \text{start} \leq \text{end} \leq 105$

My LeetCode Solution

- My LeetCode Solution

```
# Insert Interval
class Solution:
    def insert(self, intervals: List[List[int]], newInterval: List[int]) -> List[List[int]]:
        intervals.append(newInterval)
        intervals.sort()
        ans=[intervals[0]]

        for s,e in intervals[1:]:
            last=ans[-1]
            if last[-1] >= s:

                last[-1] = max(last[-1], e)
            else:
                ans.append([s,e])

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "We have a sorted list of non-overlapping intervals. Insert a new interval,
# merging wherever it overlaps existing ones. Return the result sorted.
# We don't need to modify in-place – a new array is fine. Right?"
#
# "[[1,3],[6,9]], newInterval=[2,5]:
# [2,5] overlaps [1,3] → merge to [1,5]. [6,9] doesn't overlap → keep. →
# [[1,5],[6,9]] ✓
# [[1,2],[3,5],[6,7],[8,10],[12,16]], newInterval=[4,8]:
# Overlaps [3,5],[6,7],[8,10] → merge all to [3,10]. → [[1,2],[3,10],[12,16]] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# I'll append the new interval to the list, sort everything by start time,
# then run a standard merge-intervals pass to collapse all overlaps.
# Time:  $O(n \log n)$  – the sort dominates. Space:  $O(n)$ . Should I go ahead and
# optimize?"
```

```
class _57_InsertInterval_Brute:
    def insert(self, intervals: List[List[int]], newInterval: List[int]) -> List[List[int]]:
        intervals.append(newInterval)
        intervals.sort()
        result = [intervals[0]]
        for s, e in intervals[1:]:
```

```

        if result[-1][1] >= s:
            result[-1][1] = max(result[-1][1], e)
        else:
            result.append([s, e])
    return result

```

PHASE 3 — OPTIMIZE

"The input is already sorted – we can exploit that to avoid the $O(n \log n)$ sort.

Three phases in one $O(n)$ pass:

1. Add all intervals that end BEFORE newInterval starts (no overlap).

2. Merge all intervals that overlap newInterval into one combined interval.

Overlap condition: $\text{interval}[\text{start}] \leq \text{newInterval}[\text{end}]$.

3. Add all remaining intervals that start AFTER the merged interval ends.

#

Pseudocode:

$i = 0$

while $i < n$ and $\text{intervals}[i][1] < \text{new}[0]$: $\text{result.append(intervals}[i])$; $i++$

while $i < n$ and $\text{intervals}[i][0] \leq \text{new}[1]$:

$\text{new} = [\min(\text{new}[0], \text{intervals}[i][0]), \max(\text{new}[1], \text{intervals}[i][1])]$; $i++$

$\text{result.append}(\text{new})$

$\text{result.extend(intervals}[i:])$

#

Time: $O(n)$. Space: $O(n)$ for output."

PHASE 4 — CLEAN CODE

```
class _57_InsertInterval:
```

```
    def insert(self, intervals: List[List[int]], newInterval: List[int]) ->
    List[List[int]]:
```

```
        result = []
```

```
        i, n = 0, len(intervals)
```

```
        # Phase 1: all intervals ending before newInterval starts
```

```
        while i < n and intervals[i][1] < newInterval[0]:
```

```
            result.append(intervals[i])
```

```
            i += 1
```

```
        # Phase 2: merge all overlapping intervals into newInterval
```

```
        while i < n and intervals[i][0] <= newInterval[1]:
```

```
            newInterval[0] = min(newInterval[0], intervals[i][0])
```

```
            newInterval[1] = max(newInterval[1], intervals[i][1])
```

```
            i += 1
```

```
        result.append(newInterval)
```

```
        # Phase 3: remaining intervals start after merged interval
```

```
        result.extend(intervals[i:])
```

```
        return result
```

PHASE 5 — TEST & DEBUG

```
# intervals=[[1,3],[6,9]], new=[2,5]:
```

```
# Phase 1: intervals[0][1]=3 >= new[0]=2 → stop. i=0.
```

```
# Phase 2: intervals[0][0]=1 <= new[1]=5 → new=[min(2,1),max(5,3)]=[1,5]. i=1.
```

```
# intervals[1][0]=6 > new[1]=5 → stop.
```



```
# result=[[1,5]]. extend [[6,9]]. → [[1,5],[6,9]] ✓
#
# intervals=[[1,2],[3,5],[6,7],[8,10],[12,16]], new=[4,8]:
# Phase 1: [1,2] ends at 2 < 4 → add. i=1.
# Phase 2: [3,5] start 3<=8→new=[3,8]. [6,7] start 6<=8→new=[3,8]. [8,10] start
8<=8→new=[3,10]. i=4.
# result=[[1,2],[3,10]]. extend [[12,16]]. → [[1,2],[3,10],[12,16]] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n) – three phases together visit each interval once.
# Space O(n) for output.
# The two overlap conditions are the key:
# Non-overlap before: interval[end] < new[start].
# Non-overlap after: interval[start] > new[end] (Phase 3 – just extend).
# Anything in between overlaps and gets merged.
# Given more time I'd ask: what if intervals aren't sorted?
# Sort first (O(n log n)), then apply the same three-phase approach."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def insert(self, intervals: list[list[int]],
               newInterval: list[int]) -> list[list[int]]:
        n = len(intervals)
        ans = []
        i = 0

        while i < n and intervals[i][1] < newInterval[0]:
            ans.append(intervals[i])
            i += 1

        # Merge overlapping intervals.
        while i < n and intervals[i][0] <= newInterval[1]:
            newInterval[0] = min(newInterval[0], intervals[i][0])
            newInterval[1] = max(newInterval[1], intervals[i][1])
            i += 1

        ans.append(newInterval)

        while i < n:
```

```

    ans.append(intervals[i])
    i += 1

return ans

```

• LeetDoocs

```

class Solution:
    def insert(
        self, intervals: List[List[int]], newInterval: List[int]
    ) -> List[List[int]]:
        def merge(intervals: List[List[int]]) -> List[List[int]]:
            intervals.sort()
            ans = [intervals[0]]
            for s, e in intervals[1:]:
                if ans[-1][1] < s:
                    ans.append([s, e])

            else:
                ans[-1][1] = max(ans[-1][1], e)
            return ans

        intervals.append(newInterval)
        return merge(intervals)

```

• SimplyLeet

```

# Python solution for Insert Interval

class Solution:
    def insert(self, intervals, newInterval):
        # List to store the resulting intervals
        merged_intervals = []
        i = 0
        n = len(intervals)

        # Add all intervals that end before newInterval starts

```

```

while i < n and intervals[i][1] < newInterval[0]:
    merged_intervals.append(intervals[i])
    i += 1

# Merge overlapping intervals with newInterval
while i < n and intervals[i][0] <= newInterval[1]:
    # Update the start and end of newInterval by merging with
    overlapping intervals
    newInterval[0] = min(newInterval[0], intervals[i][0])
    newInterval[1] = max(newInterval[1], intervals[i][1])
    i += 1

# Add the merged newInterval
merged_intervals.append(newInterval)

# Add any remaining intervals that come after newInterval
while i < n:
    merged_intervals.append(intervals[i])
    i += 1

return merged_intervals

```

• LC.ca

```

'''
>>> a = [1,2,3,4,5]
>>> a[1]
2
>>> a[~1]
4
'''

class Solution:
    def insert(self, intervals: List[List[int]], newInterval: List[int]) -> List[List[int]]:
        start = newInterval[0]

```

```

        end = newInterval[1]

        left = list(filter(lambda x: x[1] < start, intervals)) # cast via
list()
        right = list(filter(lambda x: x[0] > end, intervals))

        if left + right != intervals:
            start = min(start, intervals[len(left)][0]) # note, left not -1,
because index starts at 0
            end = max(end, intervals[~len(right)][1]) # same, starting at
right with index=0

        return left + [[start, end]] + right

#####

class Solution: # re-use Leetcode-56's merge solution
    def insert(self, intervals: List[List[int]], newInterval: List[int]) -> Li
st[List[int]]:
        intervals.append(newInterval)
        return self.merge(intervals)

    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        ans = []

        for intv in sorted(intervals, key=lambda x: x[0]):
            if ans and ans[-1][1] >= intv[0]:
                ans[-1][1] = max(ans[-1][1], intv[1])
            else:
                ans.append(intv)
        return ans

#####

class Solution:

```

```
def insert(
    self, intervals: List[List[int]], newInterval: List[int]
) -> List[List[int]]:
    def merge(intervals: List[List[int]]) -> List[List[int]]:
        intervals.sort()
        ans = [intervals[0]]
        for s, e in intervals[1:]:
            if ans[-1][1] < s:
                ans.append([s, e])
            else:
                ans[-1][1] = max(ans[-1][1], e)
        return ans

    intervals.append(newInterval)
    return merge(intervals)
```

◆ Quick Review

Key Insights

- The intervals array is sorted by start times.
- Three main steps are used:
- Add intervals that come completely before the new interval.
- Merge overlapping intervals with the new interval.
- Append intervals that start after the new (merged) interval.
- Iterating through the list once allows an $O(n)$ solution.

Space & Time

Time Complexity: $O(n)$ since we scan the list once.

Space Complexity: $O(n)$ to store the resulting list of intervals.

Solution

Traverse the intervals list and perform the following: Append all intervals that end before the new interval starts. For intervals that overlap with newInterval, update newInterval to merge them (using min for start and max for end). Append the new merged interval. Append all remaining intervals. This ensures that all intervals remain non-overlapping and sorted with a single pass.

Arrays & Hashing

□ #238 Product of Array Except Self

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Prefix Sum
Lists: Grind 169

Problem

Given an integer array `nums`, return an array `answer` such that `answer[i]` is equal to the product of all the elements of `nums` except `nums[i]`.

The product of any prefix or suffix of `nums` is guaranteed to fit in a 32-bit integer.

You must write an algorithm that runs in $O(n)$ time and without using the division operation.

Example 1:

```
Input: nums = [1,2,3,4]
Output: [24,12,8,6]
```

Example 2:

```
Input: nums = [-1,1,0,-3,3]
Output: [0,0,9,0,0]
```


Constraints:

- $2 \leq \text{nums.length} \leq 105$
- $-30 \leq \text{nums}[i] \leq 30$
- The input is generated such that $\text{answer}[i]$ is guaranteed to fit in a 32-bit integer.

Follow up: Can you solve the problem in $O(1)$ extra space complexity? (The output array does not count as extra space for space complexity analysis.)

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        n = len(nums)
        p = [1] * len(nums)
        s = [1] * len(nums)
        ans = [1] * len(nums)

        for i in range(1, n):
            p[i] = p[i-1] * nums[i-1]

        for i in reversed(range(n-1)):
            s[i] = s[i+1] * nums[i+1]

        return [p[i]*s[i] for i in range(len(nums))]
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return an array where $\text{answer}[i]$ is the product of all elements except $\text{nums}[i]$.
Cannot use division. Must run in $O(n)$. Follow-up: $O(1)$ extra space. Right?

```

# Guaranteed no overflow in 32-bit – so no overflow concern."
#
# "[1,2,3,4]: answer[0]=2*3*4=24. answer[1]=1*3*4=12. answer[2]=1*2*4=8.
answer[3]=1*2*3=6 ✓
# [-1,1,0,-3,3]: zeros make most products 0 – [0,0,9,0,0] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# For each index i, I multiply together all elements in the array except nums[i]
itself.
# That's a nested loop – O(n) work for each of the n positions.
# Time: O(n2). Space: O(1) extra. Should I go ahead and optimize?"
class _238_ProductExceptSelf_Brute:
    def productExceptSelf(self, nums):
        n = len(nums)
        result = []
        for i in range(n):
            product = 1
            for j in range(n):
                if j != i:
                    product *= nums[j]
            result.append(product)
        return result

# PHASE 3 — OPTIMIZE
# "Build prefix products and suffix products, multiply them.
# prefix[i] = product of everything to the LEFT of i.
# suffix[i] = product of everything to the RIGHT of i.
# answer[i] = prefix[i] * suffix[i].
# Time: O(n). Space: O(n) for two arrays.
#
# O(1) space follow-up: use the output array for prefix products,
# then do a single right-to-left pass with a running suffix multiplier."

# PHASE 4 — CLEAN CODE (O(1) extra space version)
class _238_ProductExceptSelf:
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        n = len(nums)
        result = [1] * n
        # Left pass: result[i] = product of all elements to the left of i
        for i in range(1, n):
            result[i] = result[i - 1] * nums[i - 1]
        # Right pass: multiply in the suffix product on the fly
        suffix = 1
        for i in range(n - 1, -1, -1):
            result[i] *= suffix
            suffix *= nums[i]
        return result

```

PHASE 5 — TEST & DEBUG

```
# nums=[1,2,3,4]:
# Left pass: result=[1,1,2,6].
# Right pass: i=3: result[3]=6*1=6, suffix=4.
# i=2: result[2]=2*4=8, suffix=12.
# i=1: result[1]=1*12=12, suffix=24.
# i=0: result[0]=1*24=24, suffix=24.
# → [24,12,8,6] ✓
# nums=[0,0]: result=[0,0] ✓ (zeros handled correctly by multiplication)
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(1) extra – output array doesn't count.
# The two-pass trick: left products first, then multiply right products on the fly.
# The suffix variable carries the running right product without an extra array.
# Given more time I'd ask: what if we're allowed division?
# total_product / nums[i] – but fails when nums[i]=0. Handle: count zeros,
# if two or more → all zeros. If one zero → all zeros except position of zero."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def productExceptSelf(self, nums: list[int]) -> list[int]:
        n = len(nums)
        prefix = [1] * n # prefix product
        suffix = [1] * n # suffix product

        for i in range(1, n):
            prefix[i] = prefix[i - 1] * nums[i - 1]

        for i in reversed(range(n - 1)):
            suffix[i] = suffix[i + 1] * nums[i + 1]

        return [prefix[i] * suffix[i] for i in range(n)]
```

```

class Solution:
    def productExceptSelf(self, nums: list[int]) -> list[int]:
        n = len(nums)
        ans = [1] * n

        # Use ans as the prefix product array.
        for i in range(1, n):
            ans[i] = ans[i - 1] * nums[i - 1]

        suffix = 1 # suffix product

        for i, num in reversed(list(enumerate(nums))):
            ans[i] *= suffix
            suffix *= num

        return ans

```

• LeetDoocs

```

class Solution:
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        n = len(nums)
        ans = [0] * n
        left = right = 1
        for i, x in enumerate(nums):
            ans[i] = left
            left *= x
        for i in range(n - 1, -1, -1):
            ans[i] *= right

            right *= nums[i]
        return ans

```

• SimplyLeet

```

# Function to compute product of array except self
def productExceptSelf(nums):
    n = len(nums)
    # Initialize the output array with 1s for prefix multiplication
    answer = [1] * n

    # Compute prefix products: for each element answer[i] holds the product of
    all numbers left of i
    prefix_product = 1
    for i in range(n):
        answer[i] = prefix_product # store prefix product so far

        prefix_product *= nums[i] # update the prefix product

    # Compute suffix products and multiply with prefix products in answer array
    suffix_product = 1
    for i in range(n - 1, -1, -1):
        answer[i] *= suffix_product # multiply current answer with suffix
        product
        suffix_product *= nums[i] # update the suffix product

    return answer

# Example usage
nums = [1, 2, 3, 4]
print(productExceptSelf(nums)) # Output: [24, 12, 8, 6]

```

• LC.ca

```

class Solution: # 2 extra arrays
    def productExceptSelf(self, nums):
        if nums is None or len(nums) == 0:
            return nums

        # Product from index=0 to index=i-1
        from_left = [1] * len(nums)
        for i in range(1, len(nums)):
            from_left[i] = nums[i - 1] * from_left[i - 1]

```

```

# Product from index=n-1 to current position
from_right = [1] * len(nums)
for i in range(len(nums) - 2, -1, -1):
    from_right[i] = nums[i + 1] * from_right[i + 1]

# Calculate result
result = [0] * len(nums)
for i in range(len(nums)):
    result[i] = from_left[i] * from_right[i]

```

```

return result

```

```

#####

```

```

class Solution: # follow up, no extra space
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        n = len(nums)
        ans = [0] * n
        left = right = 1
        for i, v in enumerate(nums):

```

```

            ans[i] = left
            left *= v
        for i in range(n - 1, -1, -1):
            ans[i] *= right
            right *= nums[i]
        return ans

```

```

#####

```

```

class Solution(object):
    def productExceptSelf(self, nums):
        """
        :type nums: List[int]
        :rtype: List[int]
        """
        dp = [1] * len(nums)
        for i in range(1, len(nums)):
            dp[i] = dp[i - 1] * nums[i - 1]

```

```
prod = 1
for i in reversed(range(0, len(nums))):
    dp[i] = dp[i] * prod
    prod *= nums[i]
return dp
```

◆ Quick Review

Key Insights

- The product for each index can be computed by multiplying the product of all numbers to the left and the product of all numbers to the right of that index.
- Division is not allowed, so we must compute the cumulative products explicitly.
- Create two passes: one forward pass for prefix products and one backward pass for suffix products.
- Use the output array to store the cumulative product values, achieving $O(1)$ extra space by not using additional arrays.

Space & Time

Time Complexity: $O(n)$ because we iterate through the array twice.

Space Complexity: $O(1)$ extra space (excluding the output array).

Solution

The approach uses two passes over the input array: Initialize the answer array and fill it with prefix products. For each index i , $\text{answer}[i]$ contains the product of all elements before i . Traverse the array from the end using a running suffix product. Update the answer array by multiplying the current suffix product with the prefix product already stored. This method leverages an output array to store intermediate results and uses a temporary variable for the suffix product, ensuring constant extra space.

Arrays & Hashing

□ #281 Zigzag Iterator

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Design · Queue · Iterator
Lists: Denny Zhang

Problem

Given two vectors of integers v1 and v2, implement an iterator to return their elements alternately.

Implement the ZigzagIterator class:

- `ZigzagIterator(List<int> v1, List<int> v2)` initializes the object with the two vectors v1 and v2.
- `boolean hasNext()` returns true if the iterator still has elements, and false otherwise.
- `int next()` returns the current element of the iterator and moves the iterator to the next element.

Example 1:

Input: v1 = [1,2], v2 = [3,4,5,6]

Output: [1,3,2,4,5,6]

Explanation: By calling next repeatedly until hasNext returns false, the order of elements returned by next should be: [1,3,2,4,5,6].

Example 2:

Input: v1 = [1], v2 = []

Output: [1]

Example 3:

Input: v1 = [], v2 = [1]

Output: [1]

Constraints:

- $0 \leq v1.length, v2.length \leq 1000$
- $1 \leq v1.length + v2.length \leq 2000$
- $-231 \leq v1[i], v2[i] \leq 231 - 1$

Follow up: What if you are given k vectors? How well can your code be extended to such cases?

Clarification for the follow-up question:

The "Zigzag" order is not clearly defined and is ambiguous for $k > 2$ cases. If "Zigzag" does not look right to you, replace "Zigzag" with "Cyclic".

Follow-up Example:

Input: v1 = [1,2,3], v2 = [4,5,6,7], v3 = [8,9]

Output: [1,4,8,2,5,9,3,6,7]

My LeetCode Solution

- My LeetCode Solution

```

class ZigzagIterator:
    def __init__(self, v1: List[int], v2: List[int]):
        self.q = deque()
        if v1:
            self.q.append((v1, 0))

        if v2:
            self.q.append((v2,0))

    def next(self) -> int:

        v, i = self.q.popleft()
        if i+1 < len(v):
            self.q.append((v, i+1))
        return v[i]

    def hasNext(self) -> bool:
        return len(self.q) > 0

# Your ZigzagIterator object will be instantiated and called as such:
# i, v = ZigzagIterator(v1, v2), []

# while i.hasNext(): v.append(i.next())

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Alternate between v1 and v2 element by element. When one is exhausted, continue with the remaining elements of the other. Right?
 # v1=[1,2], v2=[3,4,5,6] → [1,3,2,4,5,6] ✓
 # Follow-up: what if there are k vectors instead of 2?"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.
 # At construction I'll interleave v1 and v2 into a flat list: take one from v1, one from v2,
 # repeat, then append remaining elements from whichever list is longer.
 # next() and hasNext() just read from that flat list.
 # Time: O(n) init. Space: O(n). Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Use a deque of (vector, index) pairs.
 # next(): popleft, return v[i], re-enqueue (v, i+1) if more elements remain.
 # hasNext(): deque is non-empty.

```
# This generalises trivially to k vectors – just enqueue all of them.
# Time: O(1) per next()/hasNext(). Space: O(k) for the deque."
```

PHASE 4 — CLEAN CODE

```
class _281_ZigzagIterator:
    def __init__(self, v1: List[int], v2: List[int]):
        self.q: deque = deque()
        if v1:
            self.q.append((v1, 0))
        if v2:
            self.q.append((v2, 0))
    def next(self) -> int:
        vec, idx = self.q.popleft()
        if idx + 1 < len(vec):
            self.q.append((vec, idx + 1))
        return vec[idx]
    def hasNext(self) -> bool:
        return bool(self.q)
```

PHASE 5 — TEST & DEBUG

```
# v1=[1,2], v2=[3,4,5,6]:
# q=[(v1,0),(v2,0)].
# next(): pop (v1,0)→return 1. re-enqueue (v1,1). q=[(v2,0),(v1,1)].
# next(): pop (v2,0)→return 3. re-enqueue (v2,1). q=[(v1,1),(v2,1)].
# next(): pop (v1,1)→return 2. no re-enqueue (idx+1=2=len). q=[(v2,1)].
# next(): 4, then 5, then 6. → [1,3,2,4,5,6] ✓
# v1=[1], v2=[]: q=[(v1,0)]. next()→1. hasNext()→False ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(1) per operation. Space O(k) for the deque.
# Follow-up k vectors: init enqueues all k non-empty vectors. Same code, no changes.
# The deque acts as a round-robin scheduler – each vector gets one turn per cycle.
# Given more time I'd ask: what if vectors have different priorities (some should
# appear more often)? Enqueue them multiple times or use a priority queue."
```

Community Solutions (Python)

• WalkCC

```

class ZigzagIterator:
    def __init__(self, v1: list[int], v2: list[int]):
        def vals():
            for i in itertools.count():
                for v in v1, v2:
                    if i < len(v):
                        yield v[i]
        self.vals = vals()
        self.n = len(v1) + len(v2)

```

```

def next(self):
    self.n -= 1
    return next(self.vals)

def hasNext(self):
    return self.n > 0

```

• LeetDoocs

```

class ZigzagIterator:
    def __init__(self, v1: List[int], v2: List[int]):
        self.cur = 0
        self.size = 2
        self.indexes = [0] * self.size
        self.vectors = [v1, v2]

    def next(self) -> int:
        vector = self.vectors[self.cur]
        index = self.indexes[self.cur]

        res = vector[index]
        self.indexes[self.cur] = index + 1
        self.cur = (self.cur + 1) % self.size
        return res

    def hasNext(self) -> bool:
        start = self.cur
        while self.indexes[self.cur] == len(self.vectors[self.cur]):
            self.cur = (self.cur + 1) % self.size
            if self.cur == start:

```

```
        return False
    return True
```

```
# Your ZigzagIterator object will be instantiated and called as such:
# i, v = ZigzagIterator(v1, v2), []
# while i.hasNext(): v.append(i.next())
```

• SimplyLeet

```
class ZigzagIterator:
    def __init__(self, v1, v2):
        # Initialize the queue with non-empty vectors paired with starting
        index 0
        from collections import deque
        self.queue = deque()
        if v1:
            self.queue.append((v1, 0))
        if v2:
            self.queue.append((v2, 0))

    def next(self):
        # Pop from the left
        vec, idx = self.queue.popleft()
        result = vec[idx]
        # If there are more elements, put it back with the next index
        if idx + 1 < len(vec):
            self.queue.append((vec, idx + 1))
        return result

    def hasNext(self):

        return len(self.queue) > 0

# Example usage:
# v1 = [1,2]; v2 = [3,4,5,6]
# iterator = ZigzagIterator(v1, v2)
# while iterator.hasNext():
#     print(iterator.next(), end=" ")
```

• LC.ca

```

class ZigzagIteratorK:

    # int[0] is index of vector of vectors list, int[1] is start of a vector,
    int[2] is end
    q = []

    # no map needed, if move 'vectors' a class variable
    vectors = []

    def __init__(self, vectors):
        self.vectors = vectors

    for i in range(len(vectors)):
        self.q.append([i, 0, len(vectors[i])])

    def next(self):
        current = self.q.pop(0)

        i = current[0]
        start = current[1]
        end = current[2]

        if start + 1 < end:
            self.q.append([i, start + 1, end])

        return self.vectors[i][start]

    def hasNext(self):
        return len(self.q) != 0

#####

```



```

class ZigzagIterator:
    def __init__(self, v1: List[int], v2: List[int]):
        self.cur = 0
        self.size = 2
        self.indexes = [0] * self.size
        self.vectors = [v1, v2]

    def next(self) -> int:
        vector = self.vectors[self.cur]
        index = self.indexes[self.cur]

        res = vector[index]
        self.indexes[self.cur] = index + 1
        self.cur = (self.cur + 1) % self.size
        return res

    def hasNext(self) -> bool:
        start = self.cur
        while self.indexes[self.cur] == len(self.vectors[self.cur]):
            self.cur = (self.cur + 1) % self.size
            if self.cur == start:
                return False
        return True

# Your ZigzagIterator object will be instantiated and called as such:
# i, v = ZigzagIterator(v1, v2), []
# while i.hasNext(): v.append(i.next())

#####

```

```

...

```

iter() – builtin function, return an iterator object
<https://docs.python.org/3/library/functions.html#iter>

```

>>> a = map(iter, [[1,2],[3,4]])
>>>
>>> b = next(a)
>>> b
<list_iterator object at 0x101630280>

```

```
>>> next(b)
1
>>> next(b)
2
>>> next(b)
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
StopIteration
```

```
>>> b = deque(map(iter, [[1,2],[3,4]]))
>>>
>>> c = b.popleft()
>>> c
<list_iterator object at 0x1016304f0>
>>> c.next()
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
AttributeError: 'list_iterator' object has no attribute 'next'
>>> next(c)
```

```
1
>>> next(c)
2
>>> next(c)
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
StopIteration
```

len(s) – Return the length (the number of items) of an object
<https://docs.python.org/3/library/functions.html#len>

```
...
```

```
from collections import deque

class ZigzagIterator(object):

    def __init__(self, v1, v2):
        """
        Initialize your data structure here.
        :type v1: List[int]
```

```

        :type v2: List[int]
        """

        self.iters = deque(map(iter, [v1, v2]))
        self.total = sum(map(len, [v1, v2]))

    def next(self):
        """
        :rtype: int
        """
        top = self.iters.popleft()

```

```

        try:
            value = top.next()
        except StopIteration:
            return self.next()
        self.total -= 1
        if self.total != 0:
            self.iters.append(top)
        return value

```

```

    def hasNext(self):

```

```

        """
        :rtype: bool
        """
        return self.total > 0

```

```

# Your ZigzagIterator object will be instantiated and called as such:
# i, v = ZigzagIterator(v1, v2), []
# while i.hasNext(): v.append(i.next())

```

◆ Quick Review

Key Insights

- Use a FIFO (first-in-first-out) structure to maintain the order of vectors which still have remaining elements.

- For each call to `next()`, select the next element from the current vector and then move that vector to the end of the queue if it still contains elements.
- The approach ensures proper zigzag (or cyclic) ordering among the vectors.
- When extending to k vectors, the same approach works: initialize the queue with each non-empty vector.

Space & Time

Time Complexity: $O(1)$ per `next()` call on average, with an overall $O(n)$ for n total elements.

Space Complexity: $O(k)$ for the queue structure (where k is the number of vectors), plus the storage for the input vectors.

Solution

We can solve the Zigzag Iterator problem by using a queue (or deque) to hold pairs of the vector's current index and the vector itself. For each call to `next()`, we remove the front element from the queue, return the current item from its associated vector, and if the vector has more elements, reinsert the pair

(with an updated index) into the back of the queue. This cyclic process naturally interleaves the elements from the provided vectors.

Arrays & Hashing

□ #307 Range Sum Query – Mutable

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Design · Binary Indexed Tree · Segment
Tree

Lists: Denny Zhang

Problem

Given an integer array `nums`, handle multiple queries of the following types:

1. Update the value of an element in `nums`.
2. Calculate the sum of the elements of `nums` between indices `left` and `right` inclusive where `left <= right`.

Implement the `NumArray` class:

- `NumArray(int[] nums)` Initializes the object with the integer array `nums`.
- `void update(int index, int val)` Updates the value of `nums[index]` to be `val`.
- `int sumRange(int left, int right)` Returns the sum of the elements of `nums` between indices

left and right inclusive (i.e. $\text{nums}[\text{left}] + \text{nums}[\text{left} + 1] + \dots + \text{nums}[\text{right}]$).

Example 1:

Input

```
["NumArray", "sumRange", "update", "sumRange"]  
[[[1, 3, 5]], [0, 2], [1, 2], [0, 2]]
```

Output

```
[null, 9, null, 8]
```

Explanation

```
NumArray numArray = new NumArray([1, 3, 5]);
```

Constraints:

- $1 \leq \text{nums.length} \leq 3 * 10^4$
- $-100 \leq \text{nums}[i] \leq 100$
- $0 \leq \text{index} < \text{nums.length}$
- $-100 \leq \text{val} \leq 100$
- $0 \leq \text{left} \leq \text{right} < \text{nums.length}$
- At most $3 * 10^4$ calls will be made to update and sumRange.

My LeetCode Solution

- My LeetCode Solution


```
class BIT:
    def __init__(self, n):
        self.n = n
        self.tree = [0]*(n+1)

    def update(self, i, change):
        i += 1
        while i < self.n + 1:
            self.tree[i] += change
            i += (i&-i)

    def query(self, i):
        i += 1
        s = 0
        while i > 0:
            s += self.tree[i]
            i -= (i&-i)

        return s

class NumArray:

    def __init__(self, nums: List[int]):
        self.nums= nums
        n = len(nums)
        self.BIT = BIT(n)
        for i, n in enumerate(nums):
            self.BIT.update(i,n)

    def update(self, index: int, val: int) -> None:

        change = val - self.nums[index]
        self.BIT.update(index, change)
        self.nums[index] = val

    def sumRange(self, left: int, right: int) -> int:
        if not left:
            return self.BIT.query(right)

        return self.BIT.query(right) - self.BIT.query(left-1)
```

```
# Your NumArray object will be instantiated and called as such:
# obj = NumArray(nums)
# obj.update(index,val)
# param_2 = obj.sumRange(left,right)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Support two operations on an array: update a single element, and query
# the prefix sum from left to right inclusive. Up to  $3 \times 10^4$  calls. Right?
# The challenge is making both operations fast – not  $O(n)$  each."
#
# "NumArray([1,3,5]): sumRange(0,2)=9. update(1,2)->nums=[1,2,5]. sumRange(0,2)=8 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# Store the array directly. update() just sets nums[index] = val in  $O(1)$ .
# sumRange(left, right) sums nums[left..right] with a linear scan in  $O(n)$ .
# With up to  $3 \times 10^4$  queries this is  $O(n * \text{queries})$  total – too slow for large n.
# Time: update  $O(1)$ , sumRange  $O(n)$ . Space:  $O(n)$ . Should I go ahead and optimize?"
```

```
class _307_NumArray_Brute:
    def __init__(self, nums):
        self.nums = nums[:]
    def update(self, index, val):
        self.nums[index] = val
    def sumRange(self, left, right):
        return sum(self.nums[left:right+1])
```

PHASE 3 — OPTIMIZE

```
# "Binary Indexed Tree (Fenwick Tree) – each node stores a partial sum covering
# a range determined by its lowest set bit. Both update and query run in  $O(\log n)$ .
# Update: add change to index and all its ancestors ( $i += i \& -i$ ).
# Query prefix sum up to i: add and walk down ( $i -= i \& -i$ ).
# For range [left, right]: query(right) – query(left-1)."
```

PHASE 4 — CLEAN CODE

```
class _307_NumArray:
    def __init__(self, nums: List[int]):
        self.nums = nums[:]
        n = len(nums)
        self.tree = [0] * (n + 1)
        for i, val in enumerate(nums):
            self._add(i, val)
```

```

def _add(self, i: int, delta: int) -> None:
    i += 1
    while i <= len(self.nums):
        self.tree[i] += delta
        i += i & (-i)
def _prefix(self, i: int) -> int:
    i += 1
    total = 0
    while i > 0:
        total += self.tree[i]
        i -= i & (-i)
    return total
def update(self, index: int, val: int) -> None:
    self._add(index, val - self.nums[index])
    self.nums[index] = val
def sumRange(self, left: int, right: int) -> int:
    return self._prefix(right) - (self._prefix(left - 1) if left > 0 else 0)

```

PHASE 5 — TEST & DEBUG

```

# nums=[1,3,5]: tree built. _prefix(2)=1+3+5=9. sumRange(0,2)=9 ✓
# update(1,2): delta=2-3=-1. tree adjusted. nums=[1,2,5].
# sumRange(0,2): _prefix(2)=8 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "update: O(log n). sumRange: O(log n). init: O(n log n).
# Space: O(n) for the tree.
# The bit trick i & (-i) isolates the lowest set bit – this determines
# how many elements each node is responsible for. Worth explaining clearly.
# Alternative: segment tree – more flexible (supports range update) but more code.
# Given more time I'd ask: what if we need range updates (add v to nums[l..r])?
# BIT can handle that with a difference array extension – or use a lazy segment
tree."

```

Community Solutions (Python)

• WalkCC

```

class FenwickTree:
    def __init__(self, n: int):
        self.sums = [0] * (n + 1)

    def add(self, i: int, delta: int) -> None:
        while i < len(self.sums):
            self.sums[i] += delta
            i += FenwickTree.lowbit(i)

    def get(self, i: int) -> int:

```

```

summ = 0
while i > 0:
    summ += self.sums[i]
    i -= FenwickTree.lowbit(i)
return summ

```

```

@staticmethod
def lowbit(i: int) -> int:
    return i & -i

```

```

class NumArray:
    def __init__(self, nums: list[int]):
        self.nums = nums
        self.tree = FenwickTree(len(nums))
        for i, num in enumerate(nums):
            self.tree.add(i + 1, num)

    def update(self, index: int, val: int) -> None:
        self.tree.add(index + 1, val - self.nums[index])

        self.nums[index] = val

    def sumRange(self, left: int, right: int) -> int:
        return self.tree.get(right + 1) - self.tree.get(left)

```

• LeetDoocs

```

class BinaryIndexedTree:
    __slots__ = ["n", "c"]

    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x: int, delta: int):
        while x <= self.n:
            self.c[x] += delta

```

```

        x += x & -x

    def query(self, x: int) -> int:
        s = 0
        while x > 0:
            s += self.c[x]
            x -= x & -x
        return s

```

```

class NumArray:
    __slots__ = ["tree"]

    def __init__(self, nums: List[int]):
        self.tree = BinaryIndexedTree(len(nums))
        for i, v in enumerate(nums, 1):
            self.tree.update(i, v)

    def update(self, index: int, val: int) -> None:
        prev = self.sumRange(index, index)

        self.tree.update(index + 1, val - prev)

    def sumRange(self, left: int, right: int) -> int:
        return self.tree.query(right + 1) - self.tree.query(left)

# Your NumArray object will be instantiated and called as such:
# obj = NumArray(nums)
# obj.update(index,val)
# param_2 = obj.sumRange(left,right)

```

• SimplyLeet

```

# Python implementation using a Binary Indexed Tree (Fenwick Tree)
class NumArray:
    def __init__(self, nums):
        # Store the original array
        self.nums = nums[:]
        self.n = len(nums)
        # Initialize the BIT with 0s, size is (n+1) because BIT is 1-indexed
        self.tree = [0] * (self.n + 1)
        # Build the BIT: update tree for each value in the nums array
        for i, num in enumerate(nums):

            self._update_tree(i, num)

    def _update_tree(self, index, delta):
        # Update the BIT for index with delta change
        i = index + 1 # Convert to 1-indexed for BIT operations
        while i <= self.n:
            self.tree[i] += delta # Add delta to current tree node
            i += i & -i # Move to the next responsible node

    def update(self, index, val):

        # Calculate the difference between new value and current value
        delta = val - self.nums[index]
        self.nums[index] = val # Update the original array
        self._update_tree(index, delta) # Update the BIT to reflect this
change

    def _prefix_sum(self, index):
        # Compute the prefix sum from start up to and including index
        result = 0
        i = index + 1 # Convert to 1-indexed
        while i:

```

```

        result += self.tree[i]
        i -= i & -i # Move to parent node
    return result

    def sumRange(self, left, right):
        # Return the sum of the range [left, right]
        return self._prefix_sum(right) - self._prefix_sum(left - 1) if left >
0 else self._prefix_sum(right)

# Example usage:
# numArray = NumArray([1, 3, 5])

# print(numArray.sumRange(0, 2))
# numArray.update(1, 2)
# print(numArray.sumRange(0, 2))

```

• LC.ca

```

class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    @staticmethod
    def lowbit(x):
        return x & -x

    def update(self, x, delta):

        while x <= self.n:
            self.c[x] += delta
            x += BinaryIndexedTree.lowbit(x)

    def query(self, x):
        s = 0
        while x > 0:
            s += self.c[x]
            x -= BinaryIndexedTree.lowbit(x)
        return s

```

```

class NumArray:
    def __init__(self, nums: List[int]):
        self.tree = BinaryIndexedTree(len(nums))
        for i, v in enumerate(nums, 1):
            self.tree.update(i, v)

    def update(self, index: int, val: int) -> None:
        prev = self.sumRange(index, index)

        self.tree.update(index + 1, val - prev)

    def sumRange(self, left: int, right: int) -> int:
        return self.tree.query(right + 1) - self.tree.query(left)

# Your NumArray object will be instantiated and called as such:
# obj = NumArray(nums)
# obj.update(index,val)
# param_2 = obj.sumRange(left,right)

#####

# Segment tree node
class STNode(object):
    def __init__(self, start, end):
        self.start = start
        self.end = end
        self.total = 0
        self.left = None

        self.right = None

class SegmentedTree(object):
    def __init__(self, nums, start, end):
        self.root = self.buildTree(nums, start, end)

    def buildTree(self, nums, start, end):
        if start > end:
            return None

```



```
if start == end:
    node = STNode(start, end)
    node.total = nums[start]
    return node

mid = start + (end - start) // 2

root = STNode(start, end)
root.left = self.buildTree(nums, start, mid)

root.right = self.buildTree(nums, mid + 1, end)
root.total = root.left.total + root.right.total
return root

def updateVal(self, i, val):
    def updateVal(root, i, val):
        if root.start == root.end:
            root.total = val
            return val
        mid = root.start + (root.end - root.start) // 2

        if i <= mid:
            updateVal(root.left, i, val)
        else:
            updateVal(root.right, i, val)

        root.total = root.left.total + root.right.total
        return root.total

    return updateVal(self.root, i, val)

def sumRange(self, i, j):
    def rangeSum(root, start, end):
        if root.start == start and root.end == end:
            return root.total

        mid = root.start + (root.end - root.start) // 2
        if j <= mid:
            return rangeSum(root.left, start, end)
        elif i >= mid + 1:
            return rangeSum(root.right, start, end)
```

```

        else:
            return rangeSum(root.left, start, mid) + rangeSum(root.right, mid + 1,
end)

        return rangeSum(self.root, i, j)

```

```

class NumArray(object):
    def __init__(self, nums):

```

```

        """
        initialize your data structure here.

```

```

        :type nums: List[int]
        """

```

```

        self.stTree = SegmentedTree(nums, 0, len(nums) - 1)

```

```

    def update(self, i, val):

```

```

        """
        :type i: int
        :type val: int
        :rtype: int
        """

```

```

        return self.stTree.updateVal(i, val)

```

```

    def sumRange(self, i, j):

```

```

        """
        sum of elements nums[i..j], inclusive.
        :type i: int
        :type j: int
        :rtype: int
        """

```

```

        return self.stTree.sumRange(i, j)

```

```

# Your NumArray object will be instantiated and called as such:
# numArray = NumArray(nums)
# numArray.sumRange(0, 1)
# numArray.update(1, 10)
# numArray.sumRange(1, 2)

```

◆ Quick Review

Key Insights

- To efficiently support both point updates and range sum queries, a specialized data structure like a Binary Indexed Tree (Fenwick Tree) or a Segment Tree is ideal.
- Both data structures provide update and sum query operations in $O(\log n)$ time.
- The challenge is to maintain a mutable array while enabling fast sum computation over any specified range.

Space & Time

Time Complexity: $O(\log n)$ for both update and sumRange operations

Space Complexity: $O(n)$ to store the additional tree structure

Solution

We can use a Binary Indexed Tree (Fenwick Tree) to solve this problem. The Fenwick Tree allows for efficient prefix sum calculations and point updates, both in $O(\log n)$ time. The approach involves: Building the Fenwick Tree using the initial array. On an update, computing the difference between the new value and the current value, then propagating

the change appropriately in the tree. For the sum range query, computing the prefix sum up to right and subtracting the prefix sum up to left-1.

Arrays & Hashing

□ #454 4Sum II

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table
Lists: CodeSignal

Problem

Given four integer arrays `nums1`, `nums2`, `nums3`, and `nums4` all of length `n`, return the number of tuples (i, j, k, l) such that:

- $0 \leq i, j, k, l < n$
- $nums1[i] + nums2[j] + nums3[k] + nums4[l] == 0$

Example 1:

```
Input: nums1 = [1,2], nums2 = [-2,-1], nums3 = [-1,2], nums4 = [0,2]
Output: 2
Explanation:
The two tuples are:
1. (0, 0, 0, 1) -> nums1[0] + nums2[0] + nums3[0] + nums4[1] = 1 + (-2) + (-1) + 2 = 0
2. (1, 1, 0, 0) -> nums1[1] + nums2[1] + nums3[0] + nums4[0] = 2 + (-1) + (-1) + 0 = 0
```

Example 2:

```
Input: nums1 = [0], nums2 = [0], nums3 = [0], nums4 = [0]
Output: 1
```

Constraints:

- $n == \text{nums1.length}$
- $n == \text{nums2.length}$
- $n == \text{nums3.length}$
- $n == \text{nums4.length}$
- $1 \leq n \leq 200$
- $-228 \leq \text{nums1}[i], \text{nums2}[i], \text{nums3}[i], \text{nums4}[i] \leq 228$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def fourSumCount(self, nums1: List[int], nums2: List[int], nums3: List[int],
                     nums4: List[int]) -> int:
        hmap = Counter(a+b for a in nums1 for b in nums2)
        return sum(hmap[-(c+d)] for c in nums3 for d in nums4)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Count tuples (i,j,k,l) where nums1[i]+nums2[j]+nums3[k]+nums4[l]==0.
# All four arrays have the same length n. Return the count – not the tuples. Right?
# We're not asked to deduplicate – every index combo counts separately."
#
# "nums1=[1,2],nums2=[-2,-1],nums3=[-1,2],nums4=[0,2]:
# (0,0,0,1): 1+(-2)+(-1)+2=0 ✓. (1,1,0,0): 2+(-1)+(-1)+0=0 ✓. → 2 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# Four nested loops – for every combination (i,j,k,l), check if
# nums1[i]+nums2[j]+nums3[k]+nums4[l] == 0 and count the matches.
# Time:  $O(n^4)$ . For  $n=200$  that's 1.6 billion iterations – way too slow.
# Space:  $O(1)$ . Should I go ahead and optimize?"
```

```
class _454_FourSumII_Brute:
    def fourSumCount(self, nums1, nums2, nums3, nums4):
```

```

count = 0
for a in nums1:
    for b in nums2:
        for c in nums3:
            for d in nums4:
                if a + b + c + d == 0:
                    count += 1

return count

```

PHASE 3 — OPTIMIZE

"Meet in the middle: split into two pairs.

Build a Counter of all sums $a+b$ for a in nums1 , b in nums2 — $O(n^2)$ sums.

Then for each $c+d$ from $\text{nums3} \times \text{nums4}$, look up $-(c+d)$ in the counter — $O(n^2)$ lookups.

Total: $O(n^2)$ time and $O(n^2)$ space."

PHASE 4 — CLEAN CODE

```

class _454_FourSumII:
    def fourSumCount(self, nums1: List[int], nums2: List[int], nums3: List[int],
nums4: List[int]) -> int:
        pair_sums = Counter(a + b for a in nums1 for b in nums2)
        return sum(pair_sums[-(c + d)] for c in nums3 for d in nums4)

```

PHASE 5 — TEST & DEBUG

$\text{nums1}=[1,2]$, $\text{nums2}=[-2,-1]$:

$\text{pair_sums}=\{1+(-2)=-1:1, 1+(-1)=0:1, 2+(-2)=0:+1-0:2, 2+(-1)=1:1\}$

$= \{-1:1, 0:2, 1:1\}$

$\text{nums3}=[-1,2]$, $\text{nums4}=[0,2]$:

$(c=-1,d=0)$: $-(-1+0)=1$. $\text{pair_sums}[1]=1$. $\text{count}=1$.

$(c=-1,d=2)$: $-(-1+2)=-1$. $\text{pair_sums}[-1]=1$. $\text{count}=2$.

$(c=2,d=0)$: $-(2+0)=-2$. $\text{pair_sums}[-2]=0$. $\text{count}=2$.

$(c=2,d=2)$: $-(2+2)=-4$. $\text{pair_sums}[-4]=0$. $\text{count}=2$. → 2 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^2)$. Space $O(n^2)$.

Meet-in-the-middle is the key: instead of 4 nested loops, do $2+2$.

The Counter default of 0 for missing keys means misses contribute 0 — clean.

Given more time I'd ask: what if the arrays are sorted and we want unique quadruplets?

That's 4Sum (#18) — different problem requiring deduplication with two pointers."

Community Solutions (Python)

• WalkCC


```
class Solution:
    def fourSumCount(self, nums1: list[int], nums2: list[int],
                     nums3: list[int], nums4: list[int]) -> int:
        count = collections.Counter(a + b for a in nums1 for b in nums2)
        return sum(count[-c - d] for c in nums3 for d in nums4)
```

• LeetCode

```
class Solution:
    def fourSumCount(
        self, nums1: List[int], nums2: List[int], nums3: List[int], nums4: List[int]
    ) -> int:
        cnt = Counter(a + b for a in nums1 for b in nums2)
        return sum(cnt[-(c + d)] for c in nums3 for d in nums4)
```

• SimplyLeet

```
# Function to count quadruplets that sum to zero using a hash map approach
def four_sum_count(nums1, nums2, nums3, nums4):
    # Dictionary to store the frequency of sums of elements from nums1 and
    # nums2
    sum_count = {}
    for num1 in nums1:
        for num2 in nums2:
            current_sum = num1 + num2
            sum_count[current_sum] = sum_count.get(current_sum, 0) + 1

    count = 0
```

```
    # For each pair from nums3 and nums4, find the complement that makes the
    # total sum zero
    for num3 in nums3:
        for num4 in nums4:
            complement = -(num3 + num4)
            count += sum_count.get(complement, 0)
    return count
```

```
# Example usage
nums1 = [1,2]
nums2 = [-2,-1]
```

```
nums3 = [-1,2]
nums4 = [0,2]
print(four_sum_count(nums1, nums2, nums3, nums4)) # Expected Output: 2
```

• LC.ca

```
from collections import Counter

class Solution:
    def fourSumCount(
        self, nums1: List[int], nums2: List[int], nums3: List[int], nums4: Lis
t[int]
    ) -> int:
        cnt = Counter(a + b for a in nums1 for b in nums2)
        return sum(cnt[-(c + d)] for c in nums3 for d in nums4)
```

#####

```
class Solution:
    def fourSumCount(
        self, nums1: List[int], nums2: List[int], nums3: List[int], nums4: Lis
t[int]
    ) -> int:
        counter = Counter()
        for a in nums1:
            for b in nums2:
                counter[a + b] += 1
        ans = 0
```

```
        for c in nums3:
            for d in nums4:
                # adding for every time -(c+d)
                ans += counter[-(c + d)]
        return ans
```

#####

```
class Solution(object):
    def fourSumCount(self, A, B, C, D):
```

```

"""
:type A: List[int]
:type B: List[int]
:type C: List[int]
:type D: List[int]
:rtype: int
"""

ans = 0
abDict = {}
for i in range(len(A)):

    for j in range(len(B)):
        if A[i] + B[j] not in abDict:
            abDict[A[i] + B[j]] = 1
        else:
            abDict[A[i] + B[j]] += 1

    for i in range(len(C)):
        for j in range(len(D)):
            if -C[i] - D[j] in abDict:
                ans += abDict[-C[i] - D[j]]

return ans

```

◆ Quick Review

Key Insights

- Use a hash table to store the frequency of all possible sums from nums1 and nums2.
- For each pair from nums3 and nums4, compute the complement (negative sum) and look it up in the hash table.
- This reduces the time complexity from $O(n^4)$ to $O(n^2)$.

Space & Time

Time Complexity: $O(n^2)$ where n is the length of each array.

Space Complexity: $O(n^2)$ due to the storage of pair sums in the hash table.

Solution

The solution employs a hash table to precompute and store the frequency of sums of all pairs from the first two arrays. Then, by iterating over all pairs from the third and fourth arrays, we calculate the required complement that would result in a total sum of zero. This complement is checked in the hash table, and the frequency (number of valid pairs) is added to the result. This method efficiently finds the number of valid quadruplets without resorting to a brute-force $O(n^4)$ approach.

Arrays & Hashing

□ #525 Contiguous Array

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Prefix Sum
Lists: Grind 169

Problem

Given a binary array `nums`, return the maximum length of a contiguous subarray with an equal number of 0 and 1.

Example 1:

Input: `nums = [0,1]`
Output: 2
Explanation: `[0, 1]` is the longest contiguous subarray with an equal number of 0 and 1.

Example 2:

Input: `nums = [0,1,0]`
Output: 2
Explanation: `[0, 1]` (or `[1, 0]`) is a longest contiguous subarray with equal number of 0 and 1.

Example 3:

Input: `nums = [0,1,1,1,1,1,0,0,0]`
Output: 6
Explanation: `[1,1,1,0,0,0]` is the longest contiguous subarray with equal number of 0 and 1.

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $\text{nums}[i]$ is either 0 or 1.

My LeetCode Solution

• My LeetCode Solution

```
# Contiguous Array
class Solution:
    def findMaxLength(self, nums: List[int]) -> int:
        hmap = Counter({0:-1})
        ans = 0
        s = 0

        for i, num in enumerate(nums):
            s += 1 if num != 0 else -1
            if s in hmap:

                ans = max(ans, i-hmap[s])

            else:
                hmap[s] = i

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Binary array. Find the longest contiguous subarray with equal numbers of 0s and 1s.

Return its length – not the subarray itself. Right?"

#

"[0,1]: equal 0s and 1s → length 2 ✓

[0,1,0]: longest is [0,1] or [1,0] → length 2 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

For every pair (i, j), count the zeros and ones in $\text{nums}[i..j]$.

If they're equal, it's a valid subarray – track the maximum length.

Time: $O(n^2)$ – we can precompute prefix counts to make each check $O(1)$.

Space: $O(1)$. Should I go ahead and optimize?"

```
class _525_ContiguousArray_Brute:
    def findMaxLength(self, nums):
        n = len(nums)
        longest = 0
        for i in range(n):
            zeros = ones = 0
            for j in range(i, n):
                if nums[j] == 0:
                    zeros += 1
                else:
                    ones += 1
                if zeros == ones:
                    longest = max(longest, j - i + 1)
        return longest
```

PHASE 3 — OPTIMIZE

"Treat 0 as -1 and 1 as +1. Keep a running balance (prefix sum).
 # Equal 0s and 1s means balance hasn't changed – we want the longest subarray
 # where $\text{balance}[j] == \text{balance}[i]$ for some $i < j$.
 # Store the FIRST index each balance value was seen. When we see the same balance
 again,
 # the subarray $[\text{first_seen}+1 \dots \text{current}]$ has equal 0s and 1s.
 # Seed the map with $\{0: -1\}$ to handle subarrays starting from index 0.
 #
 # Time: $O(n)$. Space: $O(n)$."

PHASE 4 — CLEAN CODE

```
class _525_ContiguousArray:
    def findMaxLength(self, nums: List[int]) -> int:
        first_seen = {0: -1} # balance -> first index it appeared
        balance = 0
        longest = 0
        for i, num in enumerate(nums):
            balance += 1 if num == 1 else -1
            if balance in first_seen:
                longest = max(longest, i - first_seen[balance])
            else:
                first_seen[balance] = i
        return longest
```

PHASE 5 — TEST & DEBUG

$\text{nums}=[0,1,0]$:
 # $i=0, \text{num}=0$: $\text{balance}=-1$. not seen $\rightarrow \text{first_seen}=\{0:-1, -1:0\}$.
 # $i=1, \text{num}=1$: $\text{balance}=0$. seen at -1 $\rightarrow \text{longest}=\max(0, 1-(-1))=2$.
 # $i=2, \text{num}=0$: $\text{balance}=-1$. seen at 0 $\rightarrow \text{longest}=\max(2, 2-0)=2$.
 # return 2 ✓
 # $\text{nums}=[0,1,1,1,1,1,0,0,0]$:
 # Track balance... $\text{longest}=6$ ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP**# "Time $O(n)$. Space $O(n)$ for the hash map.**# The $0 \rightarrow -1$ substitution is the core insight – equal 0s and 1s becomes

'prefix sum unchanged', which becomes 'same balance seen before'.

The $\{0: -1\}$ seed is critical – without it, subarrays starting at index 0 are missed.

Given more time I'd ask: what if it's not binary – find the longest subarray with

equal counts of two specific values? Same trick – map $\text{target} \rightarrow +1$, other $\rightarrow -1$."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findMaxLength(self, nums: List[int]) -> int:
        ans = 0
        prefix = 0
        prefixToIndex = {0: -1}

        for i, num in enumerate(nums):
            prefix += 1 if num else -1
            ans = max(ans, i - prefixToIndex.setdefault(prefix, i))

        return ans
```

• LeetDoocs

```
class Solution:
    def findMaxLength(self, nums: List[int]) -> int:
        d = {0: -1}
        ans = s = 0
        for i, x in enumerate(nums):
            s += 1 if x else -1
            if s in d:
                ans = max(ans, i - d[s])
            else:
                d[s] = i

        return ans
```

• SimplyLeet

```
# Python solution for Contiguous Array

def findMaxLength(nums):
    # Dictionary to store the earliest occurrence of each prefix sum
    prefix_indices = {0: -1}
    prefix_sum = 0
    max_length = 0

    # Iterate over the array
    for index, num in enumerate(nums):

        # Convert 0 to -1, keep 1 as is
        if num == 0:
            prefix_sum -= 1
        else:
            prefix_sum += 1

        # If the prefix sum has been seen before, a balanced subarray exists
        if prefix_sum in prefix_indices:
            # Calculate the length of the subarray
            current_length = index - prefix_indices[prefix_sum]

            max_length = max(max_length, current_length)
        else:
            # Store the index for this prefix sum if not already seen
            prefix_indices[prefix_sum] = index

    # Return the maximum length of balanced subarray
    return max_length

# Example usage:
# print(findMaxLength([0,1,0])) # Expected output: 2
```

• LC.ca

```
class Solution:
    def findMaxLength(self, nums: List[int]) -> int:
        s = ans = 0
        mp = {0: -1}
        for i, v in enumerate(nums):
            s += 1 if v == 1 else -1
            if s in mp:
                ans = max(ans, i - mp[s])
            else:
                mp[s] = i

        return ans
```

◆ Quick Review

Key Insights

- Convert 0s to -1 so that a balanced subarray (equal number of 0s and 1s) sums to zero.
- Use a prefix sum to track the cumulative sum as you iterate through the array.
- Use a hash table (or dictionary) to store the first occurrence of each prefix sum.
- When the same prefix sum is encountered again, it indicates that the subarray between these indices is balanced.

Space & Time

Time Complexity: $O(n)$ – We iterate through the array once.

Space Complexity: $O(n)$ – In the worst case, we store an entry for each prefix sum.

Solution

We use a technique where we convert each 0 in the array to -1 . Then, we compute a running prefix sum as we iterate through the array. A prefix sum of 0 at any point indicates that the subarray from the beginning to the current index is balanced. For each prefix sum, if we have seen it before, the subarray between the previous index (where the prefix sum was first seen) and the current index sums to 0. We update the maximum length accordingly. We use a hash table to store the first index where each prefix sum occurs.

Arrays & Hashing

□ #560 Subarray Sum Equals K

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Prefix Sum
Lists: Grind 169

Problem

Given an array of integers `nums` and an integer `k`, return the total number of subarrays whose sum equals to `k`.

A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: `nums = [1,1,1]`, `k = 2`
Output: 2

Example 2:

Input: `nums = [1,2,3]`, `k = 3`
Output: 2

Constraints:

- $1 \leq \text{nums.length} \leq 2 * 10^4$
- $-1000 \leq \text{nums}[i] \leq 1000$
- $-10^7 \leq k \leq 10^7$

My LeetCode Solution

• My LeetCode Solution

```
# Subarray Sum Equals K
class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        hmap = Counter({0:1})
        s = 0
        ans = 0

        for num in nums:
            s += num
            ans += hmap[s-k]

        hmap[s] += 1

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Count subarrays (contiguous, non-empty) whose elements sum to k.
# Can contain negatives – so sliding window won't work. Return the count. Right?"
#
# "[1,1,1], k=2: [1,1] at (0,1) and [1,1] at (1,2) → 2 ✓
# [1,2,3], k=3: [3] at index 2 and [1,2] → 2 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# For every pair (i, j), compute the sum of nums[i..j] and check if it equals k.
# I can maintain a running sum to avoid recomputing from scratch each time, giving
O(n²).
# Time: O(n²). Space: O(1). Should I go ahead and optimize?"
```

```
class _560_SubarraySumBrute:
    def subarraySum(self, nums, k):
        count = 0
        for i in range(len(nums)):
            total = 0
            for j in range(i, len(nums)):
                total += nums[j]
                if total == k:
                    count += 1
        return count
```

PHASE 3 — OPTIMIZE

```
# "Prefix sums: sum of nums[i..j] = prefix[j+1] - prefix[i].
# We want prefix[j+1] - prefix[i] == k → prefix[i] == prefix[j+1] - k.
# Walk forward tracking the running sum. For each position, count how many
# earlier prefix sums equal (current_sum - k). Use a Counter seeded with {0:1}
# to handle subarrays starting at index 0.
#
# Time: O(n). Space: O(n)."
```

PHASE 4 — CLEAN CODE

```
class _560_SubarraySum:
    def subarraySum(self, nums: List[int], k: int) -> int:
        prefix_count = Counter({0: 1}) # prefix_sum → how many times seen
        running_sum = 0
        count = 0
        for num in nums:
            running_sum += num
            count += prefix_count[running_sum - k]
            prefix_count[running_sum] += 1
        return count
```

PHASE 5 — TEST & DEBUG

```
# nums=[1,1,1], k=2:
# num=1: sum=1. want=1-2=-1. prefix_count[-1]=0. prefix_count={0:1,1:1}.
# num=1: sum=2. want=2-2=0. prefix_count[0]=1 → count=1.
prefix_count={0:1,1:1,2:1}.
# num=1: sum=3. want=3-2=1. prefix_count[1]=1 → count=2. → 2 ✓
# nums=[1,2,3], k=3:
# sum=1,want=-2→0. sum=3,want=0→count=1. sum=6,want=3→prefix_count[3]=1→count=2. →
2 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(n).
# This is structurally identical to #525 Contiguous Array – both use
# prefix sum + hash map, both seed with {0:1} (or {0:-1}).
# Worth naming the connection explicitly: 525 finds longest, 560 counts all.
# Sliding window doesn't work here because negatives mean the sum can decrease
# even as we expand right – sliding window requires monotone sums.
# Given more time I'd ask: what if we want subarrays with sum divisible by k?
# Use prefix_sum % k – same pattern, but map remainders."
```

Community Solutions (Python)

• WalkCC


```

class Solution:
    def subarraySum(self, nums: list[int], k: int) -> int:
        ans = 0
        prefix = 0
        count = collections.Counter({0: 1})

        for num in nums:
            prefix += num
            ans += count[prefix - k]
            count[prefix] += 1

        return ans

```

• LeetCode

```

class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        cnt = Counter({0: 1})
        ans = s = 0
        for x in nums:
            s += x
            ans += cnt[s - k]
            cnt[s] += 1
        return ans

```

• SimplyLeet

```

# Function to calculate the number of subarrays that sum to k
def subarraySum(nums, k):
    # Dictionary to store frequency of prefix sums
    prefix_counts = {0: 1} # Initialize with prefix sum 0
    current_sum = 0 # Running sum initialization
    count = 0 # To count the valid subarrays
    # Iterate over the array
    for num in nums:
        current_sum += num # Update the running (prefix) sum
        # Check if there's a prefix sum that differs from current_sum by k

```

```

        count += prefix_counts.get(current_sum - k, 0)
        # Update the frequency count for the current sum
        prefix_counts[current_sum] = prefix_counts.get(current_sum, 0) + 1
    return count

# Example usage:
print(subarraySum([1,1,1], 2)) # Expected output: 2

```

• LC.ca

```

class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        dp = Counter({0: 1})
        ans = s = 0
        for num in nums:
            s += num
            ans += dp[s - k] # not matter if the same num like [1,1]
            dp[s] += 1
        return ans

```

#####

```

class Solution: # over time limit, not fast enough
    def subarraySum(self, nums: List[int], k: int) -> int:
        count = 0

        # sum value from 1 to i-th element
        sum_arr = [0] * (len(nums) + 1)

        # cached sum

        for i in range(1, len(nums) + 1):
            sum_arr[i] = sum_arr[i - 1] + nums[i - 1]

        for start in range(len(nums)):
            for end in range(start + 1, len(nums) + 1):
                if sum_arr[end] - sum_arr[start] == k:
                    count += 1

        return count

```

◆ Quick Review

Key Insights

- Utilize the prefix sum technique to simplify sum calculation for subarrays.
- Use a hash table (or dictionary/map) to store the frequency of prefix sums encountered.
- For each element, check if (current prefix sum – k) exists in the hash table; if it does, it indicates a valid subarray.
- Initialize the hash table with {0: 1} to manage cases where a subarray's sum is exactly k.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

We solve the problem by maintaining a running sum (prefix sum) and using a hash table to record the number of times each sum has occurred. As we iterate through the array, we update the running sum. For each new sum, we check if (current sum – k) exists in our hash table. If it does, the value associated with (current sum – k) gives the number of

subarrays ending at the current index that sum to k. This method efficiently computes the result in a single pass through the array.

Arrays & Hashing

□ #1010 Pairs of Songs With Total Durations Divisible by 60

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Counting
Lists: CodeSignal

Problem

You are given a list of songs where the i th song has a duration of $\text{time}[i]$ seconds.

Return the number of pairs of songs for which their total duration in seconds is divisible by 60. Formally, we want the number of indices i, j such that $i < j$ with $(\text{time}[i] + \text{time}[j]) \% 60 == 0$.

Example 1:

```
Input: time = [30,20,150,100,40]
Output: 3
Explanation: Three pairs have a total duration divisible by 60:
(time[0] = 30, time[2] = 150): total duration 180
(time[1] = 20, time[3] = 100): total duration 120
(time[1] = 20, time[4] = 40): total duration 60
```

Example 2:

```
Input: time = [60,60,60]
Output: 3
Explanation: All three pairs have a total duration of 120, which is divisible by 60.
```

Constraints:

- $1 \leq \text{time.length} \leq 6 * 10^4$
- $1 \leq \text{time}[i] \leq 500$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def numPairsDivisibleBy60(self, time: List[int]) -> int:
        hmap = Counter()
        ans = 0

        for t in time:
            t = t%60
            ans += hmap[(60-t)%60]
            hmap[t] +=1

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Count pairs (i,j) where i<j and (time[i]+time[j]) % 60 == 0.
# Songs have durations in seconds. Return the count. Right?
# Edge cases: duration exactly 60 (60%60=0), duration exactly 30 pairs with another 30."
#
# "[30,20,150,100,40]: (30,150)=180, (20,100)=120, (20,40)=60 → 3 ✓
# [60,60,60]: each pair sums to 120, divisible by 60. C(3,2)=3 → 3 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# For every pair (i, j) where j > i, check if (time[i] + time[j]) % 60 == 0.
# Count all such pairs. For n=6*10^4 that's 3.6 billion comparisons – too slow.
# Time: O(n^2). Space: O(1). Should I go ahead and optimize?"
```

```
class _1010_PairsSongs_Brute:
    def numPairsDivisibleBy60(self, time):
        count = 0
        for i in range(len(time)):
```

```

        for j in range(i + 1, len(time)):
            if (time[i] + time[j]) % 60 == 0:
                count += 1
    return count

```

PHASE 3 — OPTIMIZE

"Reduce each duration to $t \% 60$. We want pairs where $t_1 + t_2 \equiv 0 \pmod{60}$.
 # For each t , its complement is $(60 - t) \% 60$. The $\%60$ handles $t=0$ (complement=0, not 60).
 # Walk forward: count += how many times we've already seen the complement.
 # Then add t to the seen counter.
 # This is Two Sum with modular arithmetic – $O(n)$ time, $O(60)=O(1)$ space."

PHASE 4 — CLEAN CODE

```

class _1010_PairsSongs:
    def numPairsDivisibleBy60(self, time: List[int]) -> int:
        seen = Counter()
        count = 0
        for t in time:
            remainder = t % 60
            complement = (60 - remainder) % 60
            count += seen[complement]
            seen[remainder] += 1
        return count

```

PHASE 5 — TEST & DEBUG

```

# time=[30,20,150,100,40]:
# t=30: rem=30, comp=30. seen[30]=0 → count=0. seen={30:1}.
# t=20: rem=20, comp=40. seen[40]=0 → count=0. seen={30:1,20:1}.
# t=150: rem=30, comp=30. seen[30]=1 → count=1. seen={30:2,20:1}.
# t=100: rem=40, comp=20. seen[20]=1 → count=2. seen={30:2,20:1,40:1}.
# t=40: rem=40, comp=20. seen[20]=1 → count=3. → 3 ✓
# time=[60,60,60]: each rem=0, comp=(60-0)%60=0.
# t=60: seen[0]=0 → 0. seen={0:1}.
# t=60: seen[0]=1 → count=1. seen={0:2}.
# t=60: seen[0]=2 → count=3. → 3 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(1)$ – the Counter holds at most 60 keys.
 # The $(60-\text{remainder})\%60$ handles the remainder=0 case: complement is 0, not 60.
 # This is Two Sum modulo 60 – recognising the Two Sum pattern in disguise
 # is the key interview insight.
 # Given more time I'd ask: what if divisor were k instead of 60?
 # Same algorithm – the Counter holds at most k entries, still $O(1)$ space for fixed k ."

Community Solutions (Python)

• WalkCC


```

class Solution:
    def numPairsDivisibleBy60(self, time: list[int]) -> int:
        ans = 0
        count = [0] * 60

        for t in time:
            t %= 60
            ans += count[(60 - t) % 60]
            count[t] += 1

        return ans

```

• LeetDoocs

```

class Solution:
    def numPairsDivisibleBy60(self, time: List[int]) -> int:
        cnt = Counter()
        ans = 0
        for x in time:
            x %= 60
            y = (60 - x) % 60
            ans += cnt[y]
            cnt[x] += 1
        return ans

```

• SimplyLeet

```

# Python solution for counting pairs of songs whose duration sums are divisible
by 60
def numPairsDivisibleBy60(time):
    # Frequency array for remainders 0-59
    remainder_freq = [0] * 60
    pair_count = 0

    # Loop over each song duration
    for t in time:
        # Calculate the remainder when dividing by 60
        remainder = t % 60

```

```

        # Calculate the complementary remainder needed for the sum to be
        divisible by 60
        complement = (60 - remainder) % 60
        # Increase pair count by frequency of complement seen so far
        pair_count += remainder_freq[complement]
        # Update the frequency counter for the current remainder
        remainder_freq[remainder] += 1

    return pair_count

# Example usage

# print(numPairsDivisibleBy60([30,20,150,100,40])) # Expected output: 3

```

• LC.ca

```

class Solution:
    def numPairsDivisibleBy60(self, time: List[int]) -> int:
        cnt = Counter(t % 60 for t in time)
        ans = sum(cnt[x] * cnt[60 - x] for x in range(1, 30))
        ans += cnt[0] * (cnt[0] - 1) // 2
        ans += cnt[30] * (cnt[30] - 1) // 2
        return ans

```

◆ Quick Review

Key Insights

- Each song duration's effect is determined by its remainder when divided by 60.
- For any given module value r (where $0 \leq r < 60$), the complementary module needed to complete 60 is $(60 - r) \% 60$.
- Special handling is needed when r is 0 (or 30, since $30 + 30 = 60$) to avoid

double-counting.

- A frequency counter for remainders can be efficiently used to count valid pairs in one pass through the list.

Space & Time

Time Complexity: $O(n)$, where n is the number of songs.

Space Complexity: $O(60) \rightarrow O(1)$, since we only need to keep track of 60 possible remainders.

Solution

We iterate over the list of song durations and calculate the remainder of each duration modulo 60. A frequency array (or hash table) is used to record how many times each remainder has been seen so far. For each song, we look up the frequency of its complementary remainder (i.e., $(60 - \text{current_remainder}) \% 60$) already encountered. This frequency indicates how many valid pairs can be formed with the current song. After counting pairs with the current song, we update the frequency of its remainder for future pairings.

Arrays & Hashing

□ ★ #1272 Remove Interval ★

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array

Lists: ★ Premium | Premium 98

Problem

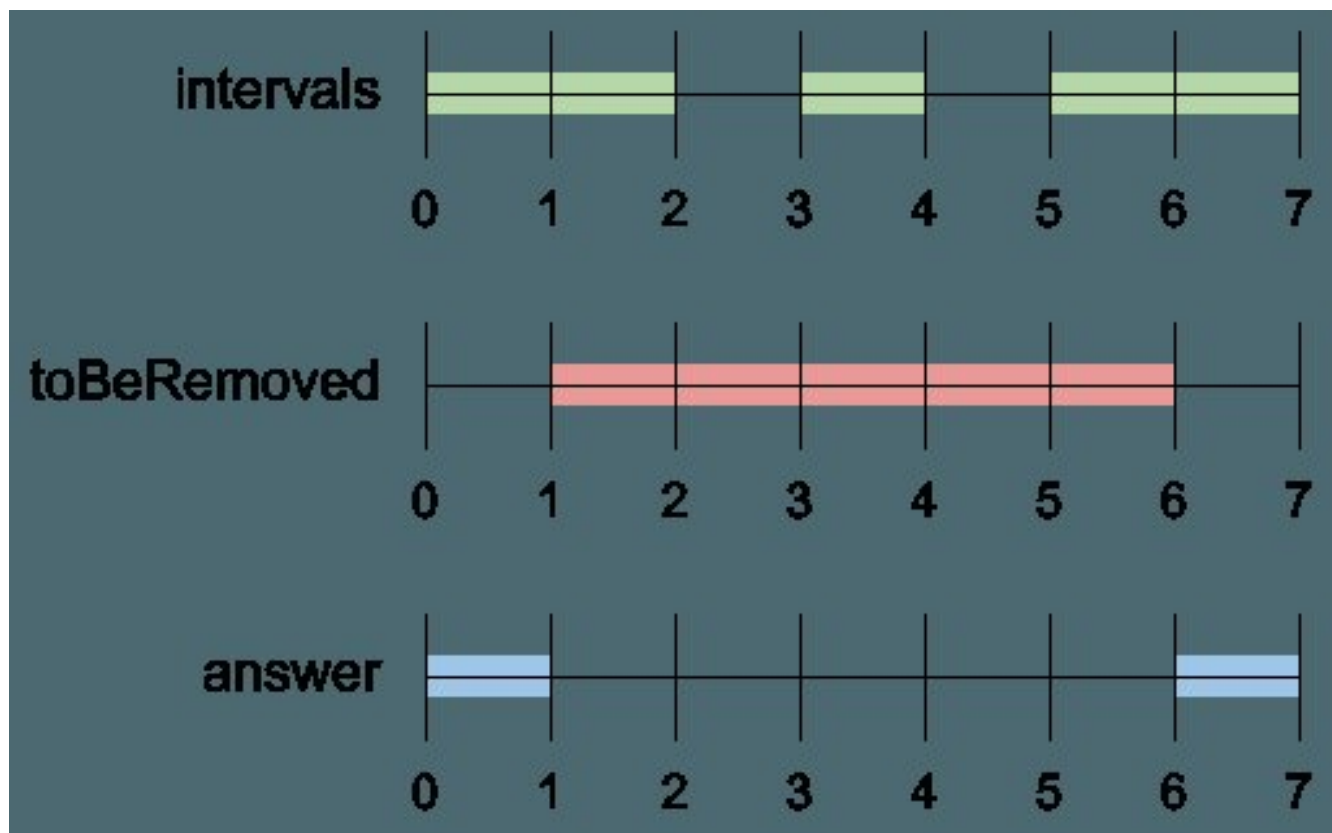
A set of real numbers can be represented as the union of several disjoint intervals, where each interval is in the form $[a, b)$. A real number x is in the set if one of its intervals $[a, b)$ contains x (i.e. $a \leq x < b$).

You are given a sorted list of disjoint intervals representing a set of real numbers as described above, where $\text{intervals}[i] = [a_i, b_i]$ represents the interval $[a_i, b_i)$. You are also given another interval `toBeRemoved`.

Return the set of real numbers with the interval `toBeRemoved` removed from intervals. In other words, return the set of real numbers such that every x in the set is in intervals but not in `toBeRemoved`. Your answer should be a sorted

list of disjoint intervals as described above.

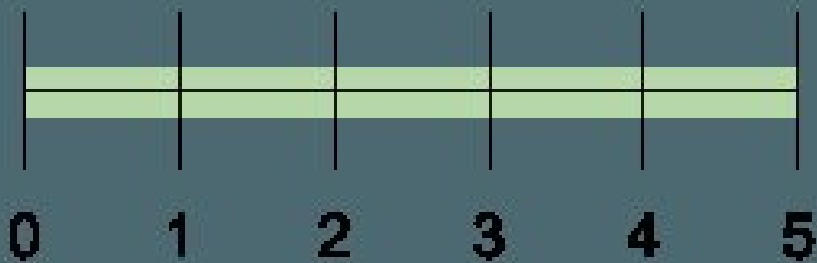
Example 1:



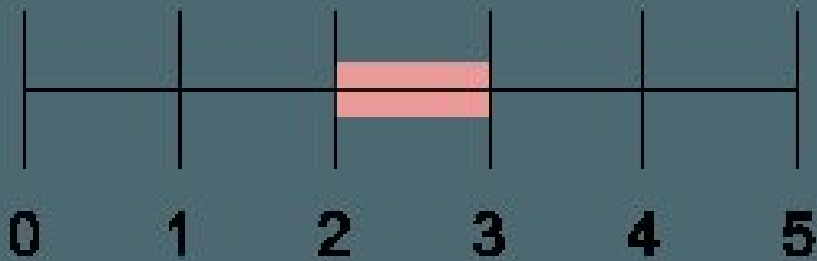
Input: intervals = $[[0, 2], [3, 4], [5, 7]]$, toBeRemoved = $[1, 6]$
Output: $[[0, 1], [6, 7]]$

Example 2:

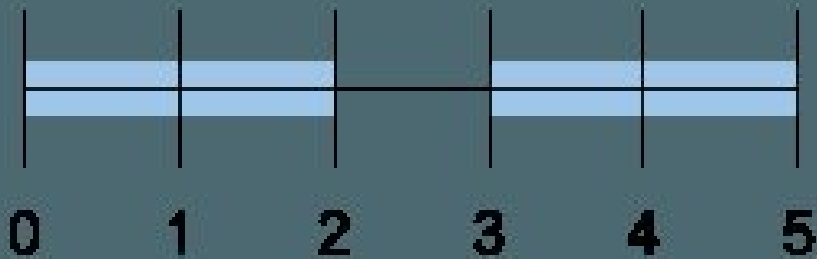
intervals



toBeRemoved



answer



Input: intervals = `[[0,5]]`, toBeRemoved = `[2,3]`
 Output: `[[0,2],[3,5]]`

Example 3:

Input: intervals = `[[-5,-4],[-3,-2],[1,2],[3,5],[8,9]]`, toBeRemoved = `[-1,4]`
 Output: `[[-5,-4],[-3,-2],[4,5],[8,9]]`

Constraints:

- $1 \leq \text{intervals.length} \leq 104$
- $-109 \leq a_i < b_i \leq 109$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def removeInterval(self, intervals: List[List[int]], toBeRemoved: List[int]) -> List[List[int]]:
        a,b = toBeRemoved
        ans = []

        for start, end in intervals:
            if b <= start or a >= end:
                ans.append([start, end])

            else:

                if start < a:
                    ans.append([start,a])

                if b < end:
                    ans.append([b, end])

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Intervals use half-open notation [a, b). We remove everything in toBeRemoved=[a,b) from the existing set and return what remains. Right?
 # Intervals are pre-sorted and non-overlapping – no need to re-sort output."
 #
 # "[[0,2],[3,4],[5,7]], remove=[1,6]:
 # [0,2] partially overlaps → keep [0,1). [3,4] fully inside → gone. [5,7] partially → keep [6,7). → [[0,1],[6,7]] ✓
 # [[0,5]], remove=[2,3]: split into [0,2) and [3,5). → [[0,2],[3,5]] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.
 # Walk each interval and handle three cases: if it doesn't touch toBeRemoved at all, keep it whole.
 # If it partially overlaps on the left, keep the left portion. If it partially overlaps on the right,
 # keep the right portion. If it's fully covered, drop it entirely.
 # Time: O(n). Space: O(n). Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE


```
# "Already O(n) – just apply the three cases cleanly in one pass.
# No overlap if: interval ends before remove starts (end <= a) OR starts after
remove ends (start >= b).
# Otherwise it overlaps. Trim left portion if start < a → keep [start, a).
# Trim right portion if end > b → keep [b, end)."
```

PHASE 4 — CLEAN CODE

```
class _1272_RemoveInterval:
    def removeInterval(self, intervals: List[List[int]], toBeRemoved: List[int]) ->
List[List[int]]:
        a, b = toBeRemoved
        result = []
        for start, end in intervals:
            if end <= a or start >= b: # no overlap – keep whole
                result.append([start, end])
            else:
                if start < a: # left portion survives
                    result.append([start, a])
                if end > b: # right portion survives
                    result.append([b, end])
        return result
```

PHASE 5 — TEST & DEBUG

```
# [[0,2],[3,4],[5,7]], remove=[1,6]:
# [0,2]: overlap. start(0)<a(1)→[0,1]. end(2)>b(6)? No. result=[[0,1]].
# [3,4]: overlap. start(3)<1? No. end(4)>6? No. result=[[0,1]].
# [5,7]: overlap. start(5)<1? No. end(7)>6 → [6,7]. result=[[0,1],[6,7]] ✓
# [[0,5]], remove=[2,3]:
# overlap. [0,2] and [3,5]. → [[0,2],[3,5]] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(n) for output.
# The two trim conditions are independent – both can fire on the same interval
# (when toBeRemoved is strictly inside the interval), producing two output
intervals.
# Given more time I'd ask: what if toBeRemoved can be a list of intervals to remove?
# Process them in sorted order, sweeping through once – still O(n + m) time."
```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def removeInterval(self, intervals: list[list[int]],
                       toBeRemoved: list[int]) -> list[list[int]]:

        ans = []

        for a, b in intervals:
            if a >= toBeRemoved[1] or b <= toBeRemoved[0]:
                ans.append([a, b])
            else: # a < toBeRemoved[1] and b > toBeRemoved[0]
                if a < toBeRemoved[0]:

                    ans.append([a, toBeRemoved[0]])
                if b > toBeRemoved[1]:
                    ans.append([toBeRemoved[1], b])

        return ans

```

• LeetDoocs

```

class Solution:
    def removeInterval(
        self, intervals: List[List[int]], toBeRemoved: List[int]
    ) -> List[List[int]]:
        x, y = toBeRemoved
        ans = []
        for a, b in intervals:
            if a >= y or b <= x:
                ans.append([a, b])
            else:

                if a < x:
                    ans.append([a, x])
                if b > y:
                    ans.append([y, b])

        return ans

```

• SimplyLeet

```
def removeInterval(intervals, toBeRemoved):
    # Unpack the removal interval boundaries.
    remove_start, remove_end = toBeRemoved
    result = []

    # Iterate over each interval.
    for interval in intervals:
        start, end = interval

        # If the interval is completely outside toBeRemoved, add as is.

        if end <= remove_start or start >= remove_end:
            result.append([start, end])
        else:
            # If there is a non-overlapping left part, add it.
            if start < remove_start:
                result.append([start, remove_start])
            # If there is a non-overlapping right part, add it.
            if end > remove_end:
                result.append([remove_end, end])

    return result

# Example usage:
intervals = [[0,2],[3,4],[5,7]]
toBeRemoved = [1,6]
print(removeInterval(intervals, toBeRemoved)) # Expected output:
[[0,1],[6,7]]
```

• LC.ca

```
...
>>> a
[1, 2, 3, 4, 5]
>>>
>>> x, y = a
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: too many values to unpack
>>> x,y,z,p,q = a
>>> x
```

```

1
>>> y
2
>>> z
3
'''
class Solution:
    def removeInterval(
        self, intervals: List[List[int]], toBeRemoved: List[int]
    ) -> List[List[int]]:

        x, y = toBeRemoved
        ans = []
        for a, b in intervals:
            if a >= y or b <= x:
                ans.append([a, b])
            else:
                if a < x:
                    ans.append([a, x]) # in this else block, meaning b>x, so no
need to append [a,min(b,x)]
                if b > y:
                    ans.append([y, b])

        # if x < a < b < y, then delete (a,b), do nothing and skip here
        return ans

#####

class Solution:
    def removeInterval(
        self, intervals: List[List[int]], toBeRemoved: List[int]
    ) -> List[List[int]]:
        result = []

```

```
for ind, intv in enumerate(intervals):
    if intv[1] < toBeRemoved[0] or toBeRemoved[1] < intv[0]:
        result.append(intv)
    else:
        if intv[0] < toBeRemoved[0]:
            result.append([intv[0], min(intv[1], toBeRemoved[0])]) #
min() is not needed
        if intv[1] > toBeRemoved[1]:
            result.append([max(intv[0], toBeRemoved[1]), intv[1]])

return result
```

◆ Quick Review

Key Insights

- Iterate over each interval and check if it overlaps with the toBeRemoved interval.
- If the current interval is completely outside of toBeRemoved, add it to the result as is.
- For overlapping intervals, compute the non-overlapping portions:
- If the left side of the interval lies before toBeRemoved, include [start, min(end, toBeRemoved.start)].
- If the right side of the interval lies after toBeRemoved, include [max(start, toBeRemoved.end), end].

Space & Time

Time Complexity: $O(n)$, where n is the number of intervals.

Space Complexity: $O(n)$, for storing the resulting intervals.

Solution

The approach involves iterating through the list of sorted disjoint intervals and checking for overlap with the `toBeRemoved` interval. For each interval: If it is completely before or after the removal interval, add it directly to the result. If it overlaps with `toBeRemoved`, determine the remaining portion(s): The left side of the interval that does not intersect (if any). The right side of the interval that does not intersect (if any). Use simple conditional checks and list manipulation to form the final set of intervals.

Arrays & Hashing

□ ★ #1429 First Unique Number ★

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Design · Queue · Data
Stream

Lists: ★ Premium | Premium 98

Problem

You have a queue of integers, you need to retrieve the first unique integer in the queue.

Implement the FirstUnique class:

- FirstUnique(int[] nums) Initializes the object with the numbers in the queue.
- int showFirstUnique() returns the value of the first unique integer of the queue, and returns -1 if there is no such integer.
- void add(int value) insert value to the queue.

Example 1:

Input:

```
["FirstUnique", "showFirstUnique", "add", "showFirstUnique", "add", "showFirstUnique", "add", "showFirstUnique"]
[[[2,3,5]], [], [5], [], [2], [], [3], []]
```

Output:

```
[null, 2, null, 2, null, 3, null, -1]
```

Explanation:

```
FirstUnique firstUnique = new FirstUnique([2,3,5]);
firstUnique.showFirstUnique(); // return 2
```

Example 2:

Input:

```
["FirstUnique", "showFirstUnique", "add", "add", "add", "add", "add", "showFirstUnique"]
[[[7,7,7,7,7,7]], [], [7], [3], [3], [7], [17], []]
```

Output:

```
[null, -1, null, null, null, null, null, 17]
```

Explanation:

```
FirstUnique firstUnique = new FirstUnique([7,7,7,7,7,7]);
firstUnique.showFirstUnique(); // return -1
```

Example 3:

Input:

```
["FirstUnique", "showFirstUnique", "add", "showFirstUnique"]
[[[809]], [], [809], []]
```

Output:

```
[null, 809, null, -1]
```

Explanation:

```
FirstUnique firstUnique = new FirstUnique([809]);
firstUnique.showFirstUnique(); // return 809
```

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $1 \leq \text{nums}[i] \leq 10^8$
- $1 \leq \text{value} \leq 10^8$
- At most 50000 calls will be made to `showFirstUnique` and `add`.

My LeetCode Solution

• My LeetCode Solution

```
# First Unique Number
class FirstUnique:

    def __init__(self, nums: List[int]):
        self.seen = set()
        self.unique = {}
        for n in nums:
            self.add(n)

    def showFirstUnique(self) -> int:

        return next(iter(self.unique), -1)

    def add(self, value: int) -> None:
        if value not in self.seen:
            self.seen.add(value)
            self.unique[value] = 1

        elif value in self.unique:
            self.unique.pop(value)

# Your FirstUnique object will be instantiated and called as such:
# obj = FirstUnique(nums)
# param_1 = obj.showFirstUnique()
# obj.add(value)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Maintain a queue. showFirstUnique() returns the first element that appears
# exactly once. add() appends to the queue. Return -1 if no unique exists. Right?
# Order matters – we want the first unique by insertion order."
#
# "FirstUnique([2,3,5]): first unique = 2.
# add(5) -> 5 now has count 2, still first unique = 2.
# add(2) -> 2 duplicated, first unique = 3.
# add(3) -> 3 duplicated, no unique -> -1 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic."
```

```
# I'll store all added elements in a list. showFirstUnique() scans the list in
insertion order,
# counting occurrences of each element, and returns the first one that appears
exactly once.
# add() is O(1). showFirstUnique() is O(n) per call – with many calls this gets
slow.
# Time: add O(1), showFirstUnique O(n). Space: O(n). Should I go ahead and
optimize?"
```

```
class _1429_FirstUnique_Brute:
    def __init__(self, nums):
        self.data = list(nums)
    def showFirstUnique(self):
        from collections import Counter
        freq = Counter(self.data)
        for x in self.data:
            if freq[x] == 1:
                return x
        return -1
    def add(self, value):
        self.data.append(value)
```

PHASE 3 — OPTIMIZE

"Two data structures:

```
# A 'seen' set – tracks all values ever added (including duplicates).
# An ordered dict (or insertion-order dict) – maps value → True, only for unique
values.
# On add: if not in seen → it's new, add to both seen and dict.
# if in seen and in dict → it became a duplicate, remove from dict.
# if in seen and not in dict → already duplicate, ignore.
# showFirstUnique(): peek at first key in dict – O(1) in Python 3.7+
(insertion-ordered).
#
# Time: O(1) per add, O(1) per showFirstUnique. Space: O(n)."
```

PHASE 4 — CLEAN CODE

```
class _1429_FirstUnique:
    def __init__(self, nums: List[int]):
        self.seen : set = set()
        self.uniques: dict = {}
        for num in nums:
            self.add(num)
    def showFirstUnique(self) -> int:
        return next(iter(self.uniques), -1)
    def add(self, value: int) -> None:
        if value not in self.seen:
            self.seen.add(value)
            self.uniques[value] = True
        elif value in self.uniques:
            del self.uniques[value]
```

PHASE 5 — TEST & DEBUG

```
# init([2,3,5]): seen={2,3,5}, uniques={2,3,5}.
# showFirstUnique() → 2 ✓
# add(5): 5 in seen and in uniques → del 5. uniques={2,3}.
# showFirstUnique() → 2 ✓
# add(2): 2 in seen and in uniques → del 2. uniques={3}.
# showFirstUnique() → 3 ✓
# add(3): del 3. uniques={}.
# showFirstUnique() → -1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "O(1) per operation. Space O(n).
# Python dicts maintain insertion order since 3.7, so next(iter(dict)) returns
# the oldest key still present – exactly what we need.
# If order weren't guaranteed, we'd use collections.OrderedDict explicitly.
# Given more time I'd ask: what if we need to support remove() as well?
# That's a linked hash map (doubly linked list + hash map) – O(1) all operations."
```

Community Solutions (Python)

• WalkCC

```
class FirstUnique:
    def __init__(self, nums: list[int]):
        self.seen = set()
        self.unique = {}
        for num in nums:
            self.add(num)

    def showFirstUnique(self) -> int:
        return next(iter(self.unique), -1)

    def add(self, value: int) -> None:
        if value not in self.seen:
            self.seen.add(value)
            self.unique[value] = 1
        elif value in self.unique:
            # We have added this value before, and this is the second time we're
            # adding it. So, erase the value from `unique`.
            self.unique.pop(value)
```

• LeetDoocs

```

class FirstUnique:
    def __init__(self, nums: List[int]):
        self.cnt = Counter(nums)
        self.unique = OrderedDict({v: 1 for v in nums if self.cnt[v] == 1})

    def showFirstUnique(self) -> int:
        return -1 if not self.unique else next(v for v in self.unique.keys())

    def add(self, value: int) -> None:
        self.cnt[value] += 1

        if self.cnt[value] == 1:
            self.unique[value] = 1
        elif value in self.unique:
            self.unique.pop(value)

# Your FirstUnique object will be instantiated and called as such:
# obj = FirstUnique(nums)
# param_1 = obj.showFirstUnique()
# obj.add(value)

```

• SimplyLeet

```

import collections

class FirstUnique:
    # Initialize the object with the numbers in the queue.
    def __init__(self, nums):
        # Using deque for efficient popleft operations.
        self.queue = collections.deque()
        # Dictionary to store frequency count.
        self.count = collections.defaultdict(int)
        # Process the initial list of numbers.

```

```

        for num in nums:
            self.add(num)

# Returns the value of the first unique integer of the queue.
def showFirstUnique(self):
    # Remove numbers from the front if they are no longer unique.
    while self.queue and self.count[self.queue[0]] > 1:
        self.queue.popleft()
    # Return the first unique number if available.
    return self.queue[0] if self.queue else -1

# Inserts value to the queue.
def add(self, value):
    self.count[value] += 1
    # If it is the first occurrence, append it to the queue.
    if self.count[value] == 1:
        self.queue.append(value)

```

• LC.ca

```

class FirstUnique:
    def __init__(self, nums: List[int]):
        self.cnt = Counter(nums)
        self.unique = OrderedDict({v: 1 for v in nums if self.cnt[v] == 1})

    def showFirstUnique(self) -> int:
        return -1 if not self.unique else next(v for v in self.unique.keys())

    def add(self, value: int) -> None:
        self.cnt[value] += 1

        if self.cnt[value] == 1:
            self.unique[value] = 1
        elif value in self.unique:
            self.unique.pop(value)

# Your FirstUnique object will be instantiated and called as such:
# obj = FirstUnique(nums)
# param_1 = obj.showFirstUnique()
# obj.add(value)

```

◆ Quick Review

Key Insights

- Use a queue to maintain the order of potential unique numbers.
- Use a hash table (or map) to count the occurrences of each number.
- When showing the first unique number, remove numbers from the front of the queue if their frequency is more than one.
- All operations (add and showFirstUnique) can be done in $O(1)$ amortized time.

Space & Time

Time Complexity: $O(1)$ amortized for each add and showFirstUnique call.

Space Complexity: $O(n)$, where n is the number of distinct integers encountered.

Solution

We design the FirstUnique class using two main data structures: a queue and a hash map (or dictionary). The queue preserves the order of insertion for unique candidates, while the hash map holds the frequency count for each integer. When a new number is added:

Increment its count in the hash map. If the count is one, append it to the queue. To retrieve the first unique number: While the queue is not empty and the count for the element at the front is greater than one, remove it from the queue. Return the front element of the queue if it exists, otherwise return -1. This approach ensures that we are always up-to-date with the current first unique number.

Arrays & Hashing

□ #3159 Find Occurrences of an Element in an Array **MED**

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table
Lists: CodeSignal

Problem

You are given an integer array `nums`, an integer array `queries`, and an integer `x`.

For each `queries[i]`, you need to find the index of the `queries[i]`th occurrence of `x` in the `nums` array. If there are fewer than `queries[i]` occurrences of `x`, the answer should be `-1` for that query.

Return an integer array `answer` containing the answers to all queries.

Example 1:

Input: `nums = [1,3,1,7]`, `queries = [1,3,2,4]`, `x = 1`

Output: `[0,-1,2,-1]`

Explanation:

- For the 1st query, the first occurrence of 1 is at index 0.
- For the 2nd query, there are only two occurrences of 1 in nums, so the answer is -1.
- For the 3rd query, the second occurrence of 1 is at index 2.
- For the 4th query, there are only two occurrences of 1 in nums, so the answer is -1.

Example 2:

Input: nums = [1,2,3], queries = [10], x = 5

Output: [-1]

Explanation:

- For the 1st query, 5 doesn't exist in nums, so the answer is -1.

Constraints:

- $1 \leq \text{nums.length}, \text{queries.length} \leq 105$
- $1 \leq \text{queries}[i] \leq 105$
- $1 \leq \text{nums}[i], x \leq 104$

My LeetCode Solution

- ☐
- **My LeetCode Solution**

```
# Find Occurrences of an Element in an Array
class Solution:
    def occurrencesOfElement(self, nums: List[int], queries: List[int], x: int) -> List[int]:
        indices = [i for i,n in enumerate(nums) if n == x]
        return list(indices[q-1] if q-1 < len(indices) else -1 for q in queries)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Preprocess nums to find all indices where x appears.
# For each query q, return the index of the q-th occurrence of x (1-indexed).
# If fewer than q occurrences exist, return -1. Right?"
#
# "nums=[1,3,1,7], queries=[1,3,2,4], x=1:
# x appears at indices [0,2]. q=1→0, q=3→-1, q=2→2, q=4→-1. → [0,-1,2,-1] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# For each query q, scan nums from the beginning, count occurrences of x as we go,
# and return the index where we hit the q-th occurrence – or -1 if we run out.
# Time: O(n * |queries|). Space: O(1). Should I go ahead and optimize?"
```

```
class _3159_FindOccurrences_Brute:
    def occurrencesOfElement(self, nums, queries, x):
        result = []
        for q in queries:
            count = 0
            found = -1
            for i, n in enumerate(nums):
                if n == x:
                    count += 1
                    if count == q:
                        found = i
                        break
            result.append(found)
        return result
```

PHASE 3 — OPTIMIZE

```
# "Precompute a list of all indices where x appears – O(n).
# Each query is then an O(1) index lookup. Total: O(n + |queries|)."
```

PHASE 4 — CLEAN CODE

```
class _3159_FindOccurrences:
    def occurrencesOfElement(self, nums: List[int], queries: List[int], x: int) -> List[int]:
```

```
positions = [i for i, n in enumerate(nums) if n == x]
return [positions[q - 1] if q - 1 < len(positions) else -1 for q in queries]
```

PHASE 5 — TEST & DEBUG

```
# nums=[1,3,1,7], x=1: positions=[0,2].
# q=1: positions[0]=0 ✓. q=3: 3-1=2 >= len(positions)=2 → -1 ✓.
# q=2: positions[1]=2 ✓. q=4: -1 ✓.
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n + q)$ . Space  $O(k)$  where  $k$  = occurrences of  $x$ .
# If queries were sorted, binary search on positions would give  $O((n+q) \log k)$ 
# with minimal extra space – but for unsorted queries, the precompute list is
cleaner.
# Given more time I'd ask: what if  $x$  could change between queries?
# Build a map from value → sorted list of indices, then binary search per query."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def occurrencesOfElement(
        self,
        nums: list[int],
        queries: list[int],
        x: int,
    ) -> list[int]:
        indices = [i for i, num in enumerate(nums) if num == x]
        return [indices[query - 1] if query <= len(indices) else -1
                for query in queries]
```

• LeetDoocs

```
class Solution:
    def occurrencesOfElement(
        self, nums: List[int], queries: List[int], x: int
    ) -> List[int]:
        ids = [i for i, v in enumerate(nums) if v == x]
        return [ids[i - 1] if i - 1 < len(ids) else -1 for i in queries]
```

• SimplyLeet

```
# Function to find occurrences of x at specified query indices
def find_occurrences(nums, queries, x):
    # Precompute the indices where x occurs in nums
    positions = []
    for index, num in enumerate(nums):
        if num == x:
            positions.append(index)

    # Process each query
    results = []

    for occurrence in queries:
        # Check if the requested occurrence exists
        if occurrence <= len(positions):
            results.append(positions[occurrence - 1]) # Convert to zero-based
index
        else:
            results.append(-1)
    return results

# Example usage:
nums = [1, 3, 1, 7]

queries = [1, 3, 2, 4]
x = 1
print(find_occurrences(nums, queries, x)) # Output: [0, -1, 2, -1]
```

• LC.ca

```
class Solution:
    def occurrencesOfElement(
        self, nums: List[int], queries: List[int], x: int
    ) -> List[int]:
        ids = [i for i, v in enumerate(nums) if v == x]
        return [ids[i - 1] if i - 1 < len(ids) else -1 for i in queries]
```

◆ Quick Review

Key Insights

- Pre-process the `nums` array to record the positions (indices) where the element `x` appears.
- For each query, check if the occurrence number requested is within the bounds of the precomputed list.
- Return the corresponding index if available; otherwise, return `-1`.

Space & Time

Time Complexity: $O(n + q)$ where n is the length of `nums` (to precompute positions) and q is the length of queries (to answer each query in constant time).

Space Complexity: $O(n)$ in the worst case where all elements in `nums` equal `x` and are stored in the positions array.

Solution

The solution involves a two-step approach: Iterate through the `nums` array once to store the index of each occurrence of `x` in a list called `positions`. For each query in `queries`, check if there is at least as many occurrences as the query specifies by verifying if `queries[i]` is less than or equal to the length of `positions`.

If yes, return the $(queries[i]-1)$ th element from positions (since it is zero-indexed). Otherwise, return -1 . This approach efficiently handles multiple queries by utilizing precomputed data and simple indexing.

String

Round 2 · High · Medium · 5 questions

String

□ #8 String to Integer (atoi)

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode

String

Lists: Grind 169

Problem

Implement the `myAtoi(string s)` function, which converts a string to a 32-bit signed integer.

The algorithm for `myAtoi(string s)` is as follows:

1. **Whitespace:** Ignore any leading whitespace (" ").
2. **Signedness:** Determine the sign by checking if the next character is '-' or '+', assuming positivity if neither present.
3. **Conversion:** Read the integer by skipping leading zeros until a non-digit character is encountered or the end of the string is reached. If no digits were read, then the result is 0.

4. Rounding: If the integer is out of the 32-bit signed integer range $[-2^{31}, 2^{31} - 1]$, then round the integer to remain in the range. Specifically, integers less than -2^{31} should be rounded to -2^{31} , and integers greater than $2^{31} - 1$ should be rounded to $2^{31} - 1$.

Return the integer as the final result.

Example 1:

Input: $s = "42"$

Output: 42

Explanation:

The underlined characters are what is read in and the caret is the current reader position.

Step 1: "42" (no characters read because there is no leading whitespace)

^

Step 2: "42" (no characters read because there is neither a '-' nor '+')

^

Step 3: "42" ("42" is read in)

^

Example 2:

Input: $s = "-042"$

Output: -42

Explanation:

Step 1: "-042" (leading whitespace is read and ignored)

^

Step 2: "-042" ('-' is read, so the result should be negative)

^

Step 3: "-042" ("042" is read in, leading zeros ignored in the result)

^

Example 3:

Input: `s = "1337c0d3"`

Output: `1337`

Explanation:

Step 1: `"1337c0d3"` (no characters read because there is no leading whitespace)

^

Step 2: `"1337c0d3"` (no characters read because there is neither a `'-'` nor `'+'`)

^

Step 3: `"1337c0d3"` (`"1337"` is read in; reading stops because the next character is a non-digit)

^

Example 4:

Input: `s = "0-1"`

Output: `0`

Explanation:

Step 1: `"0-1"` (no characters read because there is no leading whitespace)

^

Step 2: `"0-1"` (no characters read because there is neither a `'-'` nor `'+'`)

^

Step 3: `"0-1"` (`"0"` is read in; reading stops because the next character is a non-digit)

^

Example 5:

Input: `s = "words and 987"`

Output: `0`

Explanation:

Reading stops at the first non-digit character `'w'`.

Constraints:

- $0 \leq s.length \leq 200$

- s consists of English letters (lower-case and upper-case), digits (0–9), ' ', '+', '-', and '.'.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def myAtoi(self, s: str) -> int:
        intmax, intmin = 2**31 - 1, -2**31
        i, n = 0, len(s)
        while i < n and s[i] == " ":
            i += 1

        sign = 1
        if i < n and (s[i] == "+" or s[i] == "-"):
            if s[i] == "-":
                sign = -1

            i += 1

        ans = 0
        while i < n and s[i].isdigit():
            d = int(s[i])
            if (ans > intmax // 10) or (ans == intmax // 10 and d > 7):
                return intmax if sign == 1 else intmin

            ans = ans * 10 + d
            i += 1

        return ans * sign
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

- # "Four steps in order: strip leading whitespace, read optional sign, read digits until non-digit or end, clamp to 32-bit signed range $[-2^{31}, 2^{31}-1]$. Right?
- # Stop at the FIRST non-digit after the optional sign – don't skip over non-digits."

```

#
# "'42'→42. ' -042'→-42 (strip, sign, parse 042). '1337c0d3'→1337 (stop at 'c').
# '0-1'→0 (first digit parsed, stop at '-'). 'words'→0 (first char non-digit, no
digits read)."

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# The naive approach is to try Python's built-in int() after stripping whitespace –
# but that fails on partial strings like '42abc' and doesn't clamp to 32-bit range.
# So the 'brute force' here is really just the manual state machine, which I'll
implement.
# Time: O(n). Space: O(1). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Manual state machine with a pointer:
# 1. Skip whitespace.
# 2. Read sign.
# 3. Read digits, building number, check overflow before each digit.
# Overflow check: ans > MAX//10 OR (ans == MAX//10 AND next digit > 7).
# Return sign * ans (clamped)."

# PHASE 4 — CLEAN CODE
class _8_Atoi:
    def myAtoi(self, s: str) -> int:
        INT_MAX, INT_MIN = 2**31 - 1, -(2**31)
        i, n = 0, len(s)
        while i < n and s[i] == ' ': # strip whitespace
            i += 1
        sign = 1
        if i < n and s[i] in '+-':
            sign = -1 if s[i] == '-' else 1
            i += 1
        result = 0
        while i < n and s[i].isdigit():
            d = int(s[i])
            if result > INT_MAX // 10 or (result == INT_MAX // 10 and d > 7):
                return INT_MAX if sign == 1 else INT_MIN
            result = result * 10 + d
            i += 1
        return sign * result

# PHASE 5 — TEST & DEBUG
# '42': no space, no sign, digits→42. return 42 ✓
# ' -042': skip space, sign=-1, digits 0,4,2→42. return -42 ✓
# '1337c0d3': digits 1,3,3,7→1337, stop at 'c'. return 1337 ✓
# '9999999999': overflow triggers → return INT_MAX ✓
# '': i=0, no sign, no digits → return 0 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1)."

```

The overflow check BEFORE adding each digit is critical – without it we'd overflow Python
 # integers silently (Python handles big ints) but the logic breaks for fixed-width languages.
 # The magic '7' in `d > 7` comes from $2^{31}-1 = 2147483647$ – last digit is 7.
 # Given more time I'd ask: handle locale-specific decimal separators (commas)?
 # Add a pre-processing step to strip/replace them before parsing."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def myAtoi(self, s: str) -> int:
        s = s.strip()
        if not s:
            return 0

        sign = -1 if s[0] == '-' else 1
        if s[0] in {'-', '+'}:
            s = s[1:]

        num = 0

        for c in s:
            if not c.isdigit():
                break
            num = num * 10 + int(c)
            if sign * num <= -2**31:
                return -2**31
            if sign * num >= 2**31 - 1:
                return 2**31 - 1

        return sign * num
```

• LeetDoocs


```

class Solution:
    def myAtoi(self, s: str) -> int:
        if not s:
            return 0
        n = len(s)
        if n == 0:
            return 0
        i = 0
        while s[i] == ' ':
            i += 1

        if i == n:
            return 0
        sign = -1 if s[i] == '-' else 1
        if s[i] in ['-', '+']:
            i += 1
        res, flag = 0, (2**31 - 1) // 10
        while i < n:
            if not s[i].isdigit():
                break
            c = int(s[i])

            if res > flag or (res == flag and c > 7):
                return 2**31 - 1 if sign > 0 else -(2**31)
            res = res * 10 + c
            i += 1
        return sign * res

```

• SimplyLeet

```

# Python solution for the myAtoi problem.
def myAtoi(s: str) -> int:
    INT_MAX = 2**31 - 1
    INT_MIN = -2**31

    i, n = 0, len(s)
    # Skip leading whitespace.
    while i < n and s[i] == ' ':
        i += 1

```

```

# Handle sign.
sign = 1
if i < n and (s[i] == '+' or s[i] == '-'):
    if s[i] == '-':
        sign = -1
    i += 1

result = 0
# Convert digits and build the integer.
while i < n and s[i].isdigit():

    digit = int(s[i])
    # Check for overflow or underflow.
    if result > (INT_MAX - digit) // 10:
        return INT_MAX if sign == 1 else INT_MIN
    result = result * 10 + digit
    i += 1

return sign * result

# Example usage:

print(myAtoi("42"))
print(myAtoi(" -042"))
print(myAtoi("1337c0d3"))
print(myAtoi("0-1"))
print(myAtoi("words and 987"))

```

• LC.ca

```

class Solution:
    def myAtoi(self, s: str) -> int:
        if not s:
            return 0
        n = len(s)
        if n == 0:
            return 0
        i = 0
        while s[i] == ' ':
            i += 1

```

```
# only contains blank space
if i == n:
    return 0
sign = -1 if s[i] == '-' else 1
if s[i] in ['-', '+']:
    i += 1
res, flag = 0, (2**31 - 1) // 10
while i < n:
    # not a number, exit the loop
    if not s[i].isdigit():
        break
    c = int(s[i])
    # if overflows
    if res > flag or (res == flag and c > 7):
        return 2**31 - 1 if sign > 0 else -(2**31)
    res = res * 10 + c
    i += 1
return sign * res
```

◆ Quick Review

Key Insights

- Skip leading spaces in the string.
- Detect and handle the optional sign indicator ('+' or '-').
- Read and convert digit characters until a non-digit is encountered.
- Handle cases with no digits or leading zeros.
- Clamp the result to fit within the 32-bit signed integer range.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string.

Space Complexity: $O(1)$ as only a constant amount of extra space is used.

Solution

We iterate through the string to first eliminate any leading whitespace. Next, we check for a sign character to determine if the resulting integer should be negative. We then convert each consecutive digit into an integer, all the while updating the result and checking for potential overflow. If an overflow is detected, we immediately return the appropriate maximum or minimum 32-bit signed integer boundary value. This approach ensures that we strictly follow the conversion rules and efficiently handle edge cases.

String

□ ★ #249 Group Shifted Strings ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String
Lists: ★ Premium | Premium 98

Problem

Perform the following shift operations on a string:

- Right shift: Replace every letter with the successive letter of the English alphabet, where 'z' is replaced by 'a'. For example, "abc" can be right-shifted to "bcd" or "xyz" can be right-shifted to "yza".
- Left shift: Replace every letter with the preceding letter of the English alphabet, where 'a' is replaced by 'z'. For example, "bcd" can be left-shifted to "abc" or "yza" can be left-shifted to "xyz".

We can keep shifting the string in both directions to form an endless shifting sequence.

- For example, shift "abc" to form the sequence: ... $\leftrightarrow$ "abc" $\leftrightarrow$ "bcd" $\leftrightarrow$... $\leftrightarrow$ "xyz" $\leftrightarrow$ "yza" $\leftrightarrow$ $\leftrightarrow$ "zab" $\leftrightarrow$ "abc" $\leftrightarrow$...

You are given an array of strings `strings`, group together all `strings[i]` that belong to the same shifting sequence. You may return the answer in any order.

Example 1:

Input: `strings =`

`["abc","bcd","acef","xyz","az","ba","a","z"]`

Output:

`[["acef"],["a","z"],["abc","bcd","xyz"],["az","ba"]]`

Example 2:

Input: `strings = ["a"]`

Output: `[["a"]]`

Constraints:

- $1 \leq \text{strings.length} \leq 200$
- $1 \leq \text{strings}[i].\text{length} \leq 50$
- `strings[i]` consists of lowercase English letters.

My LeetCode Solution

- My LeetCode Solution

```

from typing import List
from collections import defaultdict

class Solution:
    def groupStrings(self, strings: List[str]) -> List[List[str]]:
        groups: defaultdict = defaultdict(list)
        for s in strings:
            key = tuple((ord(s[i]) - ord(s[i - 1])) % 26 for i in range(1, len(s)))
            groups[key].append(s)
        return list(groups.values())

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Two strings are in the same shifting sequence if one can become the other
# by consistently shifting every character by the same amount (cyclically, a-z
# wraps).
# Group them. Return order doesn't matter. Right?
# 'abc' and 'xyz' are in the same group – same relative gaps between characters."
#
# ["'abc', 'bcd', 'xyz', 'az', 'ba', 'a', 'z']":
# 'abc', 'bcd', 'xyz' → all have gaps [1,1] → same group.
# 'az', 'ba' → gaps [25] and [25] (b-a=1... wait: 'az': z-a=25. 'ba': a-b=-1+26=25) →
# same group.
# 'a', 'z' → single char, empty gap string → same group."

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with the brute force to lock in the logic.
# For each string, generate all 26 rotations by shifting every character by 0..25.
# If any rotation of string A equals string B, they belong in the same group.
# This is  $O(n^2 * 26 * L)$  – quadratic in the number of strings, far too slow.
# Time:  $O(n^2 * 26 * L)$ . Space:  $O(n)$ . Should I go ahead and optimize?"

```

PHASE 3 — OPTIMIZE

```

# "Compute a canonical key for each string: the tuple of consecutive char
# differences mod 26.
# 'abc' → (1,1). 'xyz' → (1,1). 'az' → (25,). 'ba' → (25,).
# All strings with the same key are in the same shifting group.
# Group by key using a defaultdict. Time:  $O(n * L)$ . Space:  $O(n * L)$ ."

```

PHASE 4 — CLEAN CODE

```

class _249_GroupShiftedStrings:
    def groupStrings(self, strings: List[str]) -> List[List[str]]:
        groups: defaultdict = defaultdict(list)
        for s in strings:

```



```

        key = tuple((ord(s[i]) - ord(s[i - 1])) % 26 for i in range(1, len(s)))
        groups[key].append(s)
    return list(groups.values())

```

PHASE 5 — TEST & DEBUG

```

# 'abc': key=(1,1). 'bcd': (1,1). 'xyz': (1,1) → same group ✓
# 'az': key=((ord('z')-ord('a'))%26,)=(25,). 'ba': ((ord('a')-ord('b'))%26,)=(25,)
→ same ✓
# 'a','z': key=() (empty tuple) → same group ✓
# 'acef': key=(2,2,1) → its own group ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n*L) where L = max string length. Space O(n*L).
# The %26 mod handles the wraparound: 'z'→'a' gives (ord('a')-ord('z'))%26 =
(-25)%26 = 1.
# Without the mod, 'az' and 'ba' wouldn't match – the mod is the key insight.
# Given more time I'd ask: how do you efficiently check if two strings are in the
same group
# at query time? Build a map string→canonical_key at init, then O(L) per query."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def groupStrings(self, strings: list[str]) -> list[list[str]]:
        keyToStrings = collections.defaultdict(list)

        def getKey(s: str) -> str:
            """
            Returns the key of 's' by pairwise calculation of differences.
            e.g. getKey("abc") -> "1,1" because diff(a, b) = 1 and diff(b, c) = 1.
            """
            diffs = []

            for i in range(1, len(s)):
                diff = (ord(s[i]) - ord(s[i - 1]) + 26) % 26
                diffs.append(str(diff))

            return ','.join(diffs)

        for s in strings:
            keyToStrings[getKey(s)].append(s)

```

```
return keyToStrings.values()
```

• LeetDoocs

```
class Solution:
    def groupStrings(self, strings: List[str]) -> List[List[str]]:
        g = defaultdict(list)
        for s in strings:
            diff = ord(s[0]) - ord("a")
            t = []
            for c in s:
                c = ord(c) - diff
                if c < ord("a"):
                    c += 26

            t.append(chr(c))
            g["".join(t)].append(s)
        return list(g.values())
```

• SimplyLeet

```
# Python solution for grouping shifted strings

def group_shifted_strings(strings):
    # Helper function to generate a unique key for each string based on
    # differences
    def get_key(s):
        # Single character strings return a fixed key
        if len(s) == 1:
            return "0"
        # Compute differences modulo 26 for each adjacent characters
        key = []
```

```

    for i in range(1, len(s)):
        diff = (ord(s[i]) - ord(s[i-1]) + 26) % 26
        key.append(str(diff))
    # Use tuple interleaving or join to form a string key
    return ','.join(key)

groups = {}
# Group strings by their computed key
for s in strings:
    key = get_key(s)

    if key not in groups:
        groups[key] = []
    groups[key].append(s)
# Return grouped strings as a list of lists
return list(groups.values())

# Example usage:
print(group_shifted_strings(["abc","bcd","acef","xyz","az","ba","a","z"]))

```

• LC.ca

```

'''
>>> a = defaultdict(list)
>>> b = defaultdict([])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: first argument must be callable or None

>>> ord('c')
99

>>> ord('c') - ord('a')
2
>>> ord('d') - 2
98
>>> chr(98)
'b'

>>> a = {"s":1, "t":2, "x":3}
>>> a

```

```
{'s': 1, 't': 2, 'x': 3}
>>> a.items()
dict_items([('s', 1), ('t', 2), ('x', 3)])
>>> a.values()
dict_values([1, 2, 3])
>>> list(a.values())
[1, 2, 3]
'''
```

```
from collections import defaultdict
```

```
class Solution:
    def groupStrings(self, strings: List[str]) -> List[List[str]]:
        mp = defaultdict(list)
        for s in strings:
            t = []
            diff = ord(s[0]) - ord('a')
            for c in s:
                d = ord(c) - diff
                if d < ord('a'):
                    d += 26
                t.append(chr(d))
            k = ''.join(t)
            mp[k].append(s)
        return list(mp.values())
```

◆ Quick Review

Key Insights

- All strings in the same group have the same relative differences between adjacent characters.
- Normalize each string's pattern by converting it to a key representation based on

the differences between consecutive letters.

- A single character string forms its own group as there are no differences to compute.

Space & Time

Time Complexity: $O(n * m)$ where n is the number of strings and m is the average length of the strings (to compute each string's key).

Space Complexity: $O(n * m)$ to store the keys and grouping in the hash table.

Solution

The solution involves iterating over each string and generating a unique key that represents the relative differences between adjacent characters. This key is calculated by computing the difference (mod 26) between every consecutive pair in the string. Strings that share the same key can be grouped together since they can be shifted into one another. A hash table (or dictionary) is used to store and group the strings by their computed key. This approach handles edge cases like single-character strings separately.

String

★ #271 Encode and Decode Strings ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Design

Lists: ★ Premium | Grind 169 | Premium 98

Problem

Design an algorithm to encode a list of strings to a string. The encoded string is then sent over the network and is decoded back to the original list of strings.

Machine 1 (sender) has the function:

```
string encode(vector<string> strs) {  
    // ... your code  
    return encoded_string;  
}
```

Machine 2 (receiver) has the function:

```
vector<string> decode(string s) {  
    //... your code  
    return strs;  
}
```

So Machine 1 does:

```
string encoded_string = encode(strs);
```

and Machine 2 does:

```
vector<string> str2 = decode(encoded_string);
```

strs2 in Machine 2 should be the same as strs in Machine 1.

Implement the encode and decode methods.

You are not allowed to solve the problem using any serialize methods (such as eval).

Example 1:

```
Input: dummy_input = ["Hello","World"]
Output: ["Hello","World"]
Explanation:
Machine 1:
Codec encoder = new Codec();
String msg = encoder.encode(strs);
Machine 1 ---msg---> Machine 2
```

Example 2:

```
Input: dummy_input = [""]
Output: [""]
```

Constraints:

- $1 \leq \text{strs.length} \leq 200$
- $0 \leq \text{strs}[i].\text{length} \leq 200$
- **strs[i]** contains any possible characters out of 256 valid ASCII characters.

Follow up: Could you write a generalized algorithm to work on any possible set of characters?

My LeetCode Solution

• My LeetCode Solution

```
# Encode and Decode Strings
class Codec:
    def encode(self, strs: List[str]) -> str:
        """Encodes a list of strings to a single string.
        """
        ans = ""
        for s in strs:
            ans += f"{len(s)}#{s}"

        return ans

    def decode(self, s: str) -> List[str]:
        """Decodes a single string to a list of strings.
        """
        ans = []
        i = 0
        while i < len(s):
            j = s.find("#", i)
            length = int(s[i:j])
            ans.append(s[j+1: j+1+length])

            i = j+1+length

        return ans

# Your Codec object will be instantiated and called as such:
# codec = Codec()
# codec.decode(codec.encode(strs))
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Design encode(strs) and decode(s) that roundtrip correctly for ANY list of strings –

including strings with special characters, empty strings, or strings containing

whatever delimiter we might choose. Right?

We can't use eval or pickle."

#

"['Hello', 'World'] → encode → some string → decode → ['Hello', 'World'] ✓

```
# [''] → [''] ✓ (empty string must survive the roundtrip)"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
```

```
# A naive approach is to join strings with a rare delimiter like '|' – but it breaks if
```

```
# strings themselves contain '|'. We'd need escaping: replace '|' with '\\|' and '\\' with '\\\\'.
```

```
# This works but decoding is error-prone and complex to get right.
```

```
# Time: O(total chars). Space: O(total chars). Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (length-prefix encoding)

```
# "Prefix each string with its length and a fixed separator (e.g. '#').
```

```
# encode: '5#Hello4#World'. decode: read number up to '#', slice that many chars, repeat.
```

```
# No ambiguity – length tells us exactly where each string ends.
```

```
# Works for any content including '#' and empty strings."
```

PHASE 4 — CLEAN CODE

```
class _271_Codec:
```

```
    def encode(self, strs: List[str]) -> str:
        return ''.join(f'{len(s)}#{s}' for s in strs)
```

```
    def decode(self, s: str) -> List[str]:
        result = []
        i = 0
        while i < len(s):
            j = s.index('#', i)
            length = int(s[i:j])
            result.append(s[j + 1: j + 1 + length])
            i = j + 1 + length
        return result
```

PHASE 5 — TEST & DEBUG

```
# encode(['Hello','World']): '5#Hello5#World'.
```

```
# decode('5#Hello5#World'):
```

```
# i=0: j=1 (index of '#'). length=5. result=['Hello']. i=7.
```

```
# i=7: j=8. length=5. result=['Hello','World']. i=14. done ✓
```

```
# encode(['']): '0#'. decode('0#'): j=1, length=0, append ''. → [''] ✓
```

```
# encode(['a#b','c']): '3#a#b1#c'. decode: length=3→'a#b', length=1→'c' ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "encode O(total chars). decode O(total chars). Space O(n).
```

```
# Length-prefix is the standard solution – no special characters, no escaping needed.
```

```
# The '#' separator is arbitrary – any fixed separator works since we always read
```

```
# the length first and know exactly how many chars to consume.
```

```
# Given more time I'd ask: what if strings can contain binary data (null bytes)?
```

```
# Use a 4-byte big-endian integer prefix instead of a text number – fully binary-safe."
```

Community Solutions (Python)

• WalkCC

```
class Codec:
    def encode(self, strs: list[str]) -> str:
        """Encodes a list of strings to a single string."""
        return ''.join(str(len(s)) + '/' + s for s in strs)

    def decode(self, s: str) -> list[str]:
        """Decodes a single string to a list of strings."""
        decoded = []

        i = 0

        while i < len(s):
            slash = s.find('/', i)
            length = int(s[i:slash])
            i = slash + length + 1
            decoded.append(s[slash + 1:i])

        return decoded
```

• LeetDoocs

```
class Codec:
    def encode(self, strs: List[str]) -> str:
        """Encodes a list of strings to a single string."""
        ans = []
        for s in strs:
            ans.append('{:4}'.format(len(s)) + s)
        return ''.join(ans)

    def decode(self, s: str) -> List[str]:
        """Decodes a single string to a list of strings."""
```

```

ans = []
i, n = 0, len(s)
while i < n:
    size = int(s[i : i + 4])
    i += 4
    ans.append(s[i : i + size])
    i += size
return ans

```

```

# Your Codec object will be instantiated and called as such:
# codec = Codec()
# codec.decode(codec.encode(strs))

```

• SimplyLeet

```

class Codec:
    # Encodes a list of strings to a single string.
    def encode(self, strs):
        encoded = []
        # Iterate through each string in the list.
        for s in strs:
            # Append the length of the string, a '#' as delimiter, and the
            string.
            encoded.append(str(len(s)) + '#' + s)
        # Join all parts into one encoded string.
        return ''.join(encoded)

    # Decodes a single string back to a list of strings.
    def decode(self, s):
        decoded = []
        i = 0 # Pointer to traverse the encoded string.
        while i < len(s):
            j = i
            # Find the '#' delimiter to determine the length.
            while s[j] != '#':
                j += 1

```

```

        # Convert the substring from i to j into an integer length.
        length = int(s[i:j])
        # Extract the actual string using the obtained length.
        string = s[j+1:j+1+length]
        decoded.append(string)
        # Move the pointer to the start of the next encoded segment.
        i = j + 1 + length
    return decoded

```

Example usage:

```

# codec = Codec()
# encoded_str = codec.encode(["Hello", "World"])
# decoded_list = codec.decode(encoded_str)

```

• LC.ca

```

'''
>>> s = 'abcde'
>>> '{:4}'.format(len(s)) + s
' 5abcde'
>>>
>>> '{:4}'.format(333)
' 333'
>>>
>>> '{:4}'.format(1234)
'1234'

>>> '{:4}'.format(12345)
'12345'

>>> '{0:4}'.format(12345)
'12345'
>>> '{1:4}'.format(12345)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: Replacement index 1 out of range for positional args tuple
>>> '{1:4}'.format(12345, 678910)

```

```

'678910'
>>> '{:4}'.format(12345, 678910)
'12345'
>>> '{0:4}'.format(12345, 678910)
'12345'
'''

class Codec: # no splitter
    def encode(self, strs: List[str]) -> str:
        """Encodes a list of strings to a single string."""
        ans = []

        for s in strs:
            ans.append('{:4}'.format(len(s)) + s)
        return ''.join(ans)

    def decode(self, s: str) -> List[str]:
        """Decodes a single string to a list of strings."""
        ans = []
        i, n = 0, len(s)
        while i < n:
            size = int(s[i : i + 4]) # so here potential overflow, if larger
            # than int-max, need to clarify assumption with interviewer

            i += 4
            ans.append(s[i : i + size]) # note: exclusive for right index
            i += size
        return ans

# Your Codec object will be instantiated and called as such:
# codec = Codec()
# codec.decode(codec.encode(strs))

```

```
#####
```

```
class Codec:
    def encode(self, strs: List[str]) -> str:
        encoded = ""
        for s in strs:
            # cover for input: ["abcd", "####"]
            encoded += str(len(s)) + "#" + s.replace("#", "$$") + "#"
```

```
        return encoded
```

```
    def decode(self, s: str) -> List[str]:
        decoded = []
        i = 0
        while i < len(s):
            j = s.find("#", i)
            length = int(s[i:j])
            decoded.append(s[j+1:j+1+length].replace("$$", "#"))
            i = j + 1 + length + 1
```

```
        return decoded
```

```
#####
```

```
class Codec: # wrong compared with above, for input: ["abcd", "####"]
    def encode(self, strs):
        encoded = ""
        for s in strs:
            encoded += str(len(s)) + "#" + s
```

```
        return encoded
```

```
    def decode(self, s):
        decoded = []
        i = 0
        while i < len(s):
            delimiter = s.index("#", i)
            length = int(s[i:delimiter])
            decoded.append(s[delimiter+1:delimiter+1+length])
            i = delimiter + 1 + length
```

```
return decoded
```

◆ Quick Review

Key Insights

- Use length–prefix encoding for each string to avoid ambiguity.
- Append a delimiter (e.g., "#") after the length to separate it from the string content.
- This method ensures that even if the original strings contain special characters, including the delimiter, the decoding remains unambiguous.
- The approach handles edge cases, such as empty strings.

Space & Time

Time Complexity: $O(n)$, where n is the total number of characters across all strings.

Space Complexity: $O(n)$, required for the encoded string and the resulting decoded list.

Solution

We encode by iterating over each string and building a new string that contains the length of the string, a delimiter (e.g., "#"), and then

the string itself. This way, during decoding, we can safely determine how many characters to extract by first reading until the delimiter to get the length, and then reading that number of characters. This length–prefix approach avoids conflicts with potential characters in the strings.

String

□ #635 Design Log Storage System

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Design · Ordered Set
Lists: Denny Zhang

Problem

You are given several logs, where each log contains a unique ID and timestamp. Timestamp is a string that has the following format: Year:Month:Day:Hour:Minute:Second, for example, 2017:01:01:23:59:59. All domains are zero-padded decimal numbers.

Implement the LogSystem class:

- **LogSystem()** Initializes the LogSystem object.
- **void put(int id, string timestamp)** Stores the given log (id, timestamp) in your storage system.
- **int[] retrieve(string start, string end, string granularity)** Returns the IDs of the logs whose timestamps are within the range from start to

end inclusive. start and end all have the same format as timestamp, and granularity means how precise the range should be (i.e. to the exact Day, Minute, etc.). For example, start = "2017:01:01:23:59:59", end = "2017:01:02:23:59:59", and granularity = "Day" means that we need to find the logs within the inclusive range from Jan. 1st 2017 to Jan. 2nd 2017, and the Hour, Minute, and Second for each log entry can be ignored.

Example 1:

Input

```
["LogSystem", "put", "put", "put", "retrieve", "retrieve"]  
[[], [1, "2017:01:01:23:59:59"], [2, "2017:01:01:22:59:59"], [3, "2016:01:01:00:00:00"], ["2016:01:01:01:01:01", "2017:01:01:23:00:00", "Year"], ["2016:01:01:01:01:01", "2017:01:01:23:00:00", "Hour"]]
```

Output

```
[null, null, null, null, [3, 2, 1], [2, 1]]
```

Explanation

```
LogSystem logSystem = new LogSystem();
```

Constraints:

- $1 \leq \text{id} \leq 500$
- $2000 \leq \text{Year} \leq 2017$
- $1 \leq \text{Month} \leq 12$
- $1 \leq \text{Day} \leq 31$
- $0 \leq \text{Hour} \leq 23$
- $0 \leq \text{Minute, Second} \leq 59$

- granularity is one of the values ["Year", "Month", "Day", "Hour", "Minute", "Second"].
- At most 500 calls will be made to put and retrieve.

My LeetCode Solution

• My LeetCode Solution

```
# Design Log Storage System
GRAM = {"Year":4, "Month":7, "Day":10, "Hour":13, "Minute":16, "Second":19}

class LogSystem:

    def __init__(self):
        self.logs = []

    def put(self, id: int, timestamp: str) -> None:

        self.logs.append((id,timestamp))

    def retrieve(self, start: str, end: str, granularity: str) -> List[int]:
        cutoff = GRAM[granularity]
        return list(id for id,ts in self.logs if start[:cutoff] <= ts[:cutoff]
<= end[:cutoff])

# Your LogSystem object will be instantiated and called as such:
# obj = LogSystem()

# obj.put(id,timestamp)
# param_2 = obj.retrieve(start,end,granularity)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Store logs with (id, timestamp). retrieve(start, end, granularity) returns IDs

```

# of logs within the range, where comparison is only up to the granularity level.
# Timestamps format: 'YYYY:MM:DD:HH:mm:ss'. Truncate at the granularity character
position. Right?"
#
# "retrieve with 'Year': compare only the first 4 chars. '2017:....' vs '2016:....'
etc."

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# Store all logs in a flat list. On retrieve, iterate over every log, truncate its
timestamp
# to the granularity prefix length, and check if it falls within the truncated
start/end range.
# With at most 500 calls and a fixed 19-char timestamp format, O(n) per retrieve is
acceptable.
# Time: put O(1), retrieve O(n). Space: O(n). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Map granularity name to the string prefix length that captures it.
# 'Year'→4 chars, 'Month'→7, 'Day'→10, 'Hour'→13, 'Minute'→16, 'Second'→19.
# Truncate all three timestamps to that length and do a simple string comparison.
# Lexicographic order of the timestamp format is correct for chronological order."

# PHASE 4 — CLEAN CODE
class _635_LogSystem:
    GRAN = {'Year': 4, 'Month': 7, 'Day': 10, 'Hour': 13, 'Minute': 16, 'Second':
19}

    def __init__(self):
        self.logs: List[tuple] = []

    def put(self, log_id: int, timestamp: str) -> None:
        self.logs.append((log_id, timestamp))

    def retrieve(self, start: str, end: str, granularity: str) -> List[int]:
        cut = self.GRAN[granularity]
        return [lid for lid, ts in self.logs if start[:cut] <= ts[:cut] <=
end[:cut]]

# PHASE 5 — TEST & DEBUG
# put(1, '2017:01:01:23:59:59'), put(2, '2017:01:01:22:59:59'),
put(3, '2016:01:01:00:00:00').
# retrieve(..., 'Year'): cut=4. compare '2016'<='year'<='2017'. All 3 match ->
[1,2,3] ✓
# retrieve(..., 'Hour'): cut=13. '2016:01:01:01'<='2016:01:01:00'? No→3 excluded.
# '2016:01:01:01'<='2017:01:01:22'<='2017:01:01:23'? Yes→2. '2017:01:01:23'? Yes→1.
# -> [2,1] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "put O(1). retrieve O(n) - scans all logs.
# The key insight: timestamps in 'YYYY:MM:DD:HH:mm:ss' format sort
lexicographically
# correctly because they're zero-padded and delimiters are identical.

```

For larger datasets, a sorted list + binary search would bring retrieve to $O(\log n + k)$.
 # Given more time I'd ask: what if granularity could be arbitrary (e.g. 'Week')?
 # We'd need a custom truncation function rather than fixed prefix lengths."

Community Solutions (Python)

• WalkCC

```
class LogSystem:
    def __init__(self):
        self.granularityToIndices = {'Year': 4, 'Month': 7, 'Day': 10,
                                      'Hour': 13, 'Minute': 16, 'Second': 19}

        self.idAndTimestamps = []

    def put(self, id: int, timestamp: str) -> None:
        self.idAndTimestamps.append((id, timestamp))

    def retrieve(self, start: str, end: str, granularity: str) -> list[int]:

        index = self.granularityToIndices[granularity]
        s = start[:index]
        e = end[:index]
        return [id for id, timestamp in self.idAndTimestamps
                if s <= timestamp[:index] <= e]
```

• LeetDoocs

```
class LogSystem:
    def __init__(self):
        self.logs = []
        self.d = {
            "Year": 4,
            "Month": 7,
            "Day": 10,
            "Hour": 13,
            "Minute": 16,
            "Second": 19,
```

```

    }

    def put(self, id: int, timestamp: str) -> None:
        self.logs.append((id, timestamp))

    def retrieve(self, start: str, end: str, granularity: str) -> List[int]:
        i = self.d[granularity]
        return [id for id, ts in self.logs if start[:i] <= ts[:i] <= end[:i]]

```

```

# Your LogSystem object will be instantiated and called as such:
# obj = LogSystem()
# obj.put(id,timestamp)
# param_2 = obj.retrieve(start,end,granularity)

```

• SimplyLeet

```

class LogSystem:
    def __init__(self):
        # List to store logs as (timestamp, id) tuples
        self.logs = []
        # Mapping granularity to index for the substring (fixed format:
        # Year:Month:Day:Hour:Minute:Second)
        self.granularity_to_index = {
            "Year": 4,
            "Month": 7,
            "Day": 10,
            "Hour": 13,

            "Minute": 16,
            "Second": 19
        }

    def put(self, id, timestamp):
        # Append the log as a tuple
        self.logs.append((timestamp, id))

    def retrieve(self, start, end, granularity):
        # Determine the slicing index based on granularity

```



```

        idx = self.granularity_to_index[granularity]
        # Slice the given start and end timestamps
        start_slice = start[:idx]
        end_slice = end[:idx]
        result = []
        # Iterate over all logs and check if the timestamp slice is within the
range
        for ts, log_id in self.logs:
            ts_slice = ts[:idx]
            if start_slice <= ts_slice <= end_slice:
                result.append(log_id)

    return result

# Example usage:
# logSystem = LogSystem()
# logSystem.put(1, "2017:01:01:23:59:59")
# logSystem.put(2, "2017:01:01:22:59:59")
# logSystem.put(3, "2016:01:01:00:00:00")
# print(logSystem.retrieve("2016:01:01:01:01:01", "2017:01:01:23:00:00",
"Year")) # Output: [3, 2, 1]
# print(logSystem.retrieve("2016:01:01:01:01:01", "2017:01:01:23:00:00",
"Hour")) # Output: [2, 1]

```

• LC.ca

```

class LogSystem:
    def __init__(self):
        self.logs = []
        self.d = {
            "Year": 4,
            "Month": 7,
            "Day": 10,
            "Hour": 13,
            "Minute": 16,
            "Second": 19,

```

```

    }

    def put(self, id: int, timestamp: str) -> None:
        self.logs.append((id, timestamp))

    def retrieve(self, start: str, end: str, granularity: str) -> List[int]:
        i = self.d[granularity]
        return [id for id, ts in self.logs if start[:i] <= ts[:i] <= end[:i]]

```

```

# Your LogSystem object will be instantiated and called as such:
# obj = LogSystem()
# obj.put(id,timestamp)
# param_2 = obj.retrieve(start,end,granularity)

```

```

#####

```

```

# pip install sortedcontainers
from sortedcontainers import SortedDict

```

```

# optimized, sorted
class LogSystem:
    def __init__(self):
        self.logs = SortedDict()

    def put(self, id: int, timestamp: str) -> None:
        if timestamp not in self.logs:
            self.logs[timestamp] = []
        self.logs[timestamp].append(id)

```

```

    def retrieve(self, start: str, end: str, granularity: str) -> List[int]:
        index = {
            "Year": 4,
            "Month": 7,
            "Day": 10,
            "Hour": 13,
            "Minute": 16,
            "Second": 19,
        }

```

```
start = start[:index[granularity]]
end = end[:index[granularity]]

# Adjust end to ensure it captures the correct range for the
granularity
end = end[:index[granularity]] + "9" * (19 - index[granularity])

result = []
# Using irange to efficiently iterate over the range of timestamps
# By default, both are set to True for inclusive{}
# eg: sd.irange('banana', 'elderberry', inclusive=(True, True))

for timestamp in self.logs.irange(start, end):
    if timestamp[:index[granularity]] > end:
        break
    result.extend(self.logs[timestamp])
return result
```

◆ Quick Review

Key Insights

- Store logs in a simple data structure such as a list or array.
- Convert the timestamps into strings; due to fixed-length and zero padding, lexicographical string comparisons are valid.
- Use a mapping to determine the slice length for each granularity, e.g. "Year" corresponds to the first 4 characters, "Month" corresponds to the first 7 characters, etc.
- During retrieval, compare the relevant substring of each stored timestamp against

the start and end query timestamp slices.

- The inclusive nature of the query simplifies checking conditions.

Space & Time

Time Complexity: $O(N)$ per retrieval query, where N is the number of logs stored (since each log is checked). In the worst case, all log entries are examined.

Space Complexity: $O(N)$ where N is the number of logs stored.

Solution

We implement a `LogSystem` class that supports putting log entries and retrieving them using a specified granularity. **Data Structures:** Use a list/array to store log entries as tuples (or objects) containing the timestamp and log ID. A dictionary/map is used to store the mapping from granularity string to the substring index (e.g., "Year" => 4, "Month" => 7, etc.). For the `put()` method, simply append the new log entry into the list. For the `retrieve()` method, compute the substring length based on the provided granularity. Then iterate over all stored logs and compare the sliced timestamp

against the sliced start and end query values. If it falls within the bounds, include the log ID in the result. Since the timestamps have fixed width with leading zeros, slicing and lexicographical comparisons are both valid and simple.

String

□ #1138 Alphabet Board Path

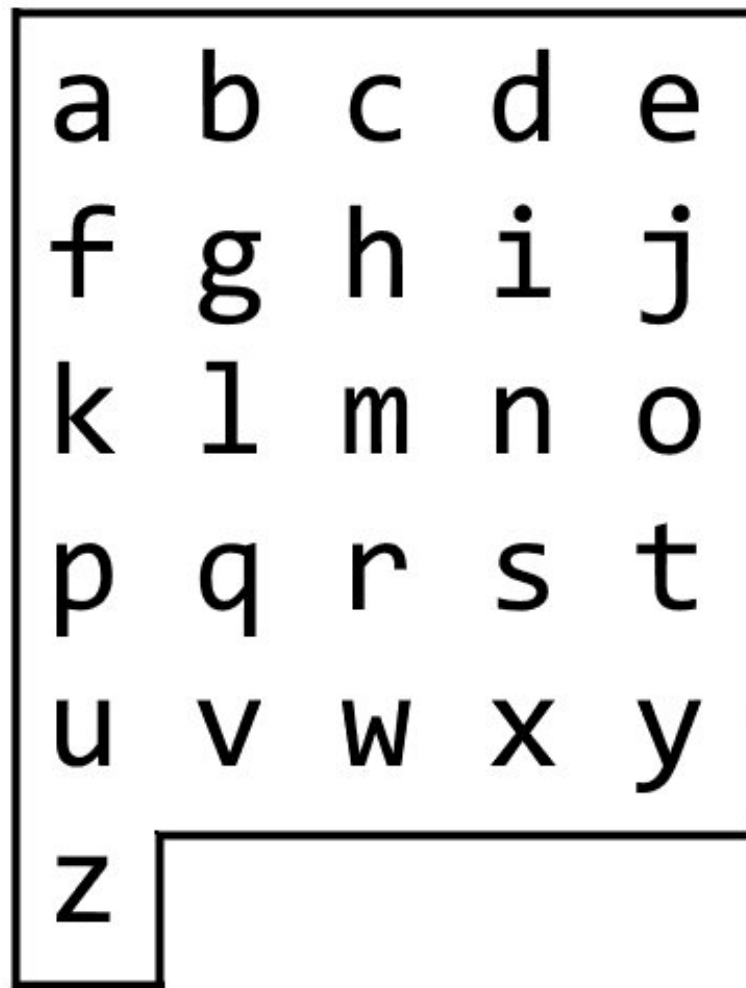
MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String
Lists: Denny Zhang

Problem

On an alphabet board, we start at position (0, 0), corresponding to character board[0][0].

Here, board = ["abcde", "fghij", "klmno", "pqrst", "uvwxy", "z"], as shown in the diagram below.



We may make the following moves:

- 'U' moves our position up one row, if the position exists on the board;
- 'D' moves our position down one row, if the position exists on the board;
- 'L' moves our position left one column, if the position exists on the board;
- 'R' moves our position right one column, if the position exists on the board;

- '!' adds the character `board[r][c]` at our current position `(r, c)` to the answer.

(Here, the only positions that exist on the board are positions with letters on them.)

Return a sequence of moves that makes our answer equal to target in the minimum number of moves. You may return any path that does so.

Example 1:

Input: target = "leet"

Output: "DDR!UURRR!!DDD!"

Example 2:

Input: target = "code"

Output: "RR!DDRR!UUL!R!"

Constraints:

- $1 \leq \text{target.length} \leq 100$
- target consists only of English lowercase letters.

My LeetCode Solution

- My LeetCode Solution

```
# Alphabet Board Path
class Solution:
    def alphabetBoardPath(self, target: str) -> str:
        row = col = 0
        moves = []

        for c in target:
            x = (ord(c) - ord('a'))//5
            y = (ord(c) - ord('a')) % 5

            if c == 'z':
                moves.extend('L' * -(y-col))
                moves.extend('D' * (x-row))

            else:
                moves.extend('U' * -(x-row))
                moves.extend('D' * (x-row))
                moves.extend('L' * -(y-col))
                moves.extend('R' * (y-col))

            moves.append('!')
            row = x
            col = y

        return ''.join(moves)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Board is ['abcde', 'fghij', 'klmno', 'pqrst', 'uvwxy', 'z'] – 5 cols except row 5 which has only 'z'.

Start at (0,0). For each char in target, output U/D/L/R moves then '!'. Right?

The critical constraint: 'z' is at (5,0) – the only cell in its row.

Moving to 'z' must clear the column before going down. Moving from 'z' must go up before going right."

#

"target='leet': l is at (2,1). Move D,D,R,!. e=(0,4) – go U,U,R,R,R,!. etc."

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

For each character, compute its board position as row = (ord(ch)-ord('a'))//5,

```
# col = (ord(ch)-ord('a'))%5. Then generate moves: go vertical first, then
horizontal.
# The naive approach fails for 'z' – moving down before clearing column 0 steps off
the board.
# Time: O(|target| * max_moves). Space: O(|target|). Should I go ahead and
optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Formula: row = (ord(ch)-ord('a'))//5, col = (ord(ch)-ord('a'))%5.
# For 'z' specifically: it's at (5,0), col=0. When moving TO 'z', move left first
# (to reach col 0) then move down (to reach row 5) – otherwise we'd step off the
board.
# When moving FROM 'z', move up first then move right – same reason.
# General case: move vertically then horizontally (or horizontally then vertically).
# 'z' is the only exception – must move left before down."
```

PHASE 4 — CLEAN CODE

```
class _1138_AlphabetBoardPath:
    def alphabetBoardPath(self, target: str) -> str:
        row = col = 0
        moves = []
        for ch in target:
            dest_row = (ord(ch) - ord('a')) // 5
            dest_col = (ord(ch) - ord('a')) % 5
            if ch == 'z': # must clear column before going down
                moves.extend('L' * (col - dest_col))
                moves.extend('D' * (dest_row - row))
            else:
                moves.extend('U' * (row - dest_row))
                moves.extend('D' * (dest_row - row))
                moves.extend('L' * (col - dest_col))
                moves.extend('R' * (dest_col - col))
            moves.append('!')
            row, col = dest_row, dest_col
        return ''.join(moves)
```

PHASE 5 — TEST & DEBUG

```
# target='leet': l=(2,1). from (0,0): D,D,R,! → "DDR!". row=2,col=1.
# e=(0,4): U,U,R,R,R,! → "UURRR!". row=0,col=4.
# e=(0,4): no move,! → "!". row=0,col=4.
# t=(3,4): D,D,D,! → "DDD!". → "DDR!UURRR!!DDD!" ✓
# target='z': from (0,0): ch='z'. L*(0)=none. D*5. → "DDDDD!" ✓
# If coming from (5,4) to 'z' (5,0): L,L,L,L,! – no D needed ✓.
# If going from 'z' to 'a': ch='a'=(0,0). Not 'z' → U*5,L*0,R*0 → "UUUUU!" ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(|target| * max_moves) = O(|target| * 10) = O(|target|). Space O(|target|).
# The 'z' case is the entire problem – everything else is standard grid navigation.
# Phrase it clearly: 'z' is at the leftmost position of its row, so we must reach
col 0
```

before stepping down into row 5, otherwise intermediate positions don't exist on the board.

Given more time I'd ask: what if the board were different (e.g. a phone numpad)?

Same approach – just recompute (row, col) from the new layout."

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def alphabetBoardPath(self, target: str) -> str:
        i = j = 0
        ans = []
        for c in target:
            v = ord(c) - ord("a")
            x, y = v // 5, v % 5
            while j > y:
                j -= 1
                ans.append("L")

            while i > x:
                i -= 1
                ans.append("U")
            while j < y:
                j += 1
                ans.append("R")
            while i < x:
                i += 1
                ans.append("D")
            ans.append("!")

        return "".join(ans)
```

• SimplyLeet

```
# Python solution with clear variable names and inline comments.
class Solution:
    def alphabetBoardPath(self, target: str) -> str:
        # Starting position at the top-left corner (0,0)
        current_row, current_col = 0, 0
        result = []

        # Helper function to convert letter to its (row, col) coordinates
        def getPosition(letter):
            index = ord(letter) - ord('a')

            return index // 5, index % 5

        for char in target:
            target_row, target_col = getPosition(char)
            # If target character is 'z', move horizontally first then
            # vertically
            if char == 'z':
                # Move left if needed
                while current_col > target_col:
                    result.append("L")
                    current_col -= 1

                # Move right if needed
                while current_col < target_col:
                    result.append("R")
                    current_col += 1

                # Then move down if needed
                while current_row < target_row:
                    result.append("D")
                    current_row += 1

                # Then move up if needed
                while current_row > target_row:
```

```

        result.append("U")
        current_row -= 1
    else:
        # For all other characters, move vertically first
        while current_row > target_row:
            result.append("U")
            current_row -= 1
        while current_row < target_row:
            result.append("D")
            current_row += 1

        # Then move horizontally
        while current_col > target_col:
            result.append("L")
            current_col -= 1
        while current_col < target_col:
            result.append("R")
            current_col += 1

        # Append pick-up action for the letter
        result.append("!")
    return "".join(result)

```

• LC.ca

```

class Solution:
    def alphabetBoardPath(self, target: str) -> str:
        i = j = 0
        ans = []
        for c in target:
            v = ord(c) - ord("a")
            x, y = v // 5, v % 5
            while j > y:
                j -= 1
                ans.append("L")

```

```
while i > x:
    i -= 1
    ans.append("U")
while j < y:
    j += 1
    ans.append("R")
while i < x:
    i += 1
    ans.append("D")
ans.append("!")

return "".join(ans)
```

◆ Quick Review

Key Insights

- Determine the row and column for each letter (using integer division and modulus).
- Because "z" is in a special row that only contains one column, the move order is critical to avoid invalid positions.
- For target letter "z", perform horizontal moves before vertical moves; for all other letters, do vertical moves before horizontal moves.
- Build the answer step-by-step by moving from your current position to each target letter.

Space & Time

Time Complexity: $O(N)$ where N is the length of target (each move is processed in constant time)

Space Complexity: $O(N)$ for the resulting string of moves

Solution

The approach calculates the destination coordinates for each character. For each letter in the target: Convert the letter to its board coordinates (row and col). For example, for letter c, $\text{row} = (\text{ord}(c) - \text{ord}('a')) // 5$ and $\text{col} = (\text{ord}(c) - \text{ord}('a')) \% 5$. Note that "z" (the 26th letter) always maps to (5, 0). To move from the current position (curRow, curCol) to the target's (targetRow, targetCol), decide on the order of moves: If the target letter is "z", first move horizontally (left or right) then vertically (up or down) to avoid stepping into invalid regions from the extra row. For any other letter, move vertically (up or down) first, then horizontally (left or right). Append "!" to the moves after reaching each letter. Update the current position to the letter's coordinates. This method ensures you only traverse valid cells on the board and minimizes moves by

taking the optimal coordinate difference each step.

Two Pointers

Round 2 · High · Medium · 14 questions

Two Pointers

□ #11 Container With Most Water

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Greedy
Lists: Grind 169

Problem

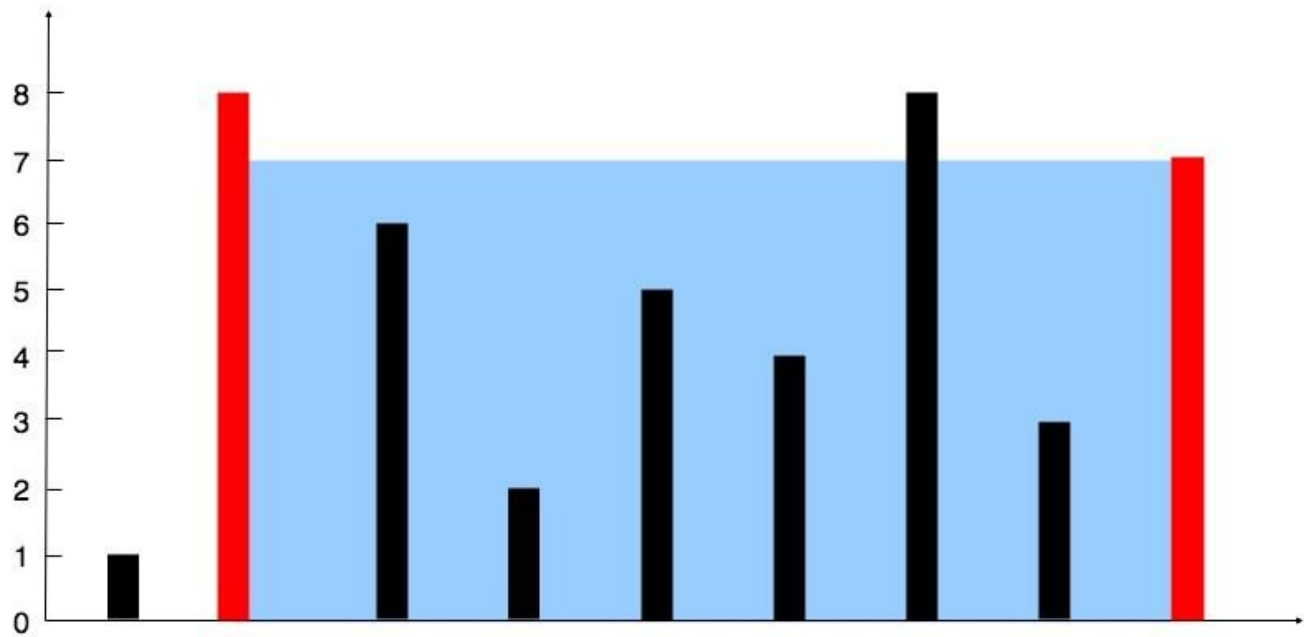
You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the `i`th line are $(i, 0)$ and $(i, \text{height}[i])$.

Find two lines that together with the `x`-axis form a container, such that the container contains the most water.

Return the maximum amount of water a container can store.

Notice that you may not slant the container.

Example 1:



Input: height = [1,8,6,2,5,4,8,3,7]

Output: 49

Explanation: The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case, the max area of water (blue section) the container can contain is 49.

Example 2:

Input: height = [1,1]

Output: 1

Constraints:

- $n == \text{height.length}$
- $2 \leq n \leq 105$
- $0 \leq \text{height}[i] \leq 104$

My LeetCode Solution

- My LeetCode Solution

```
# Container With Most Water
class Solution:
    def maxArea(self, height: List[int]) -> int:
        n = len(height)
        l, r = 0, n-1
        ans = 0

        while l < r:
            ans = max(min(height[l], height[r]) * (r-l), ans)
            if height[l] < height[r]:
                l+=1
            else:
                r-=1

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given heights of vertical lines, find two lines that form the largest container.
 # Area = $\min(\text{height}[\text{left}], \text{height}[\text{right}]) * (\text{right} - \text{left})$. Return max area. Right?
 # We can't slant – so water level is always min of the two chosen heights."
 #
 # "[1,8,6,2,5,4,8,3,7]: left=1(h=8), right=8(h=7). Area= $\min(8,7)*(8-1)=49$ ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.
 # For every pair (i, j) where $j > i$, compute area = $\min(\text{height}[i], \text{height}[j]) * (j - i)$
 # and track the maximum. That's $C(n,2)$ pairs to check.
 # Time: $O(n^2)$. Space: $O(1)$. Should I go ahead and optimize?"

```
class _11_ContainerMostWater_Brute:
    def maxArea(self, height):
        best = 0
        for i in range(len(height)):
            for j in range(i + 1, len(height)):
                best = max(best, min(height[i], height[j]) * (j - i))
        return best
```

PHASE 3 — OPTIMIZE

"Two pointers: left=0, right=n-1. At each step compute area.
 # Move the pointer with the SHORTER height inward – keeping the taller height
 # maximises any future container. Moving the taller pointer can only shrink

```
# both width and potentially height, so it can't improve the result.
# Time: O(n). Space: O(1)."
```

PHASE 4 — CLEAN CODE

```
class _11_ContainerMostWater:
    def maxArea(self, height: List[int]) -> int:
        left, right = 0, len(height) - 1
        best = 0
        while left < right:
            area = min(height[left], height[right]) * (right - left)
            best = max(best, area)
            if height[left] < height[right]:
                left += 1
            else:
                right -= 1
        return best
```

PHASE 5 — TEST & DEBUG

```
# height=[1,8,6,2,5,4,8,3,7]:
# l=0(h=1),r=8(h=7): area=1*8=8. h[l]<h[r]→l=1.
# l=1(h=8),r=8(h=7): area=7*7=49. h[l]>h[r]→r=7.
# l=1(h=8),r=7(h=3): area=3*6=18. h[l]>h[r]→r=6.
# ... best stays 49. → 49 ✓
# height=[1,1]: area=1*1=1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(1).
# The greedy argument: if we move the taller pointer, we keep a shorter constraint
# (limited by the short side) AND reduce width – area can only shrink or stay.
# So we always move the shorter side – we can't do better by moving the taller one.
# This is the key insight to state explicitly in the interview.
# Given more time I'd ask: what if we need to find the pair of indices (not just the
area)?
# Track left_best and right_best alongside best – same code, just record the
positions."
```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def maxArea(self, height: list[int]) -> int:
        ans = 0
        l = 0
        r = len(height) - 1

        while l < r:
            minHeight = min(height[l], height[r])
            ans = max(ans, minHeight * (r - l))
            if height[l] < height[r]:
                l += 1
            else:
                r -= 1

        return ans

```

• LeetDoocs

```

class Solution:
    def maxArea(self, height: List[int]) -> int:
        l, r = 0, len(height) - 1
        ans = 0
        while l < r:
            t = min(height[l], height[r]) * (r - l)
            ans = max(ans, t)
            if height[l] < height[r]:
                l += 1
            else:
                r -= 1
        return ans

```

• SimplyLeet


```
# Function to calculate the maximum water container area
def maxArea(height):
    left, right = 0, len(height) - 1 # Initialize two pointers
    max_water = 0 # Variable to store the maximum area

    # Loop until pointers meet
    while left < right:
        # Calculate current area based on smaller height and distance between
        # pointers
        current_area = min(height[left], height[right]) * (right - left)
        max_water = max(max_water, current_area) # Update maximum area if
        # current is larger

        # Move the pointer corresponding to the smaller height
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1
    return max_water

# Test the function with an example
if __name__ == "__main__":

    input_heights = [1,8,6,2,5,4,8,3,7]
    print(maxArea(input_heights)) # Expected output: 49
```

• LC.ca

```
class Solution:
    def maxArea(self, height: List[int]) -> int:
        i, j = 0, len(height) - 1
        ans = 0
        while i < j:
            t = (j - i) * min(height[i], height[j])
            ans = max(ans, t)
            if height[i] < height[j]:
                i += 1
            else:
                j -= 1
        return ans
```

◆ Quick Review

Key Insights

- The container's area is determined by the distance between two lines multiplied by the shorter line's height.
- A brute-force approach checking all possible pairs would result in $O(n^2)$ time complexity, which is inefficient for large inputs.
- A two-pointer approach allows us to solve the problem in $O(n)$ time by initializing pointers at both ends and gradually moving them inward.
- By always moving the pointer corresponding to the shorter line, we are more likely to find a taller line that could potentially form a container with a larger capacity.

Space & Time

Time Complexity: $O(n)$ – The algorithm makes a single pass through the array with two pointers.

Space Complexity: $O(1)$ – Only a few extra variables are used, regardless of the input size.

Solution

The optimal approach uses a two–pointer strategy: Start with two pointers at each end of the array. Calculate the area formed between the two lines at these pointers. Update the maximum area if the calculated value is larger. Move the pointer pointing to the shorter line inward, as this is the only possible way to potentially increase the height of the shorter line and thus the area. Continue until the two pointers meet. This method effectively reduces the number of comparisons and ensures that every possible pair that can yield a larger area is considered.

Two Pointers

□ #15 3Sum

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Sorting
Lists: Grind 169

Problem

Given an integer array `nums`, return all the triplets `[nums[i], nums[j], nums[k]]` such that $i \neq j$, $i \neq k$, and $j \neq k$, and $nums[i] + nums[j] + nums[k] == 0$.

Notice that the solution set must not contain duplicate triplets.

Example 1:

Input: `nums = [-1,0,1,2,-1,-4]`

Output: `[[-1,-1,2],[-1,0,1]]`

Explanation:

`nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0.`

`nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0.`

`nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0.`

The distinct triplets are `[-1,0,1]` and `[-1,-1,2]`.

Notice that the order of the output and the order of the triplets does not matter.

Example 2:

Input: nums = [0,1,1]

Output: []

Explanation: The only possible triplet does not sum up to 0.

Example 3:

Input: nums = [0,0,0]

Output: [[0,0,0]]

Explanation: The only possible triplet sums up to 0.

Constraints:

- $3 \leq \text{nums.length} \leq 3000$
- $-105 \leq \text{nums}[i] \leq 105$

My LeetCode Solution

• My LeetCode Solution

```
# 3Sum
class Solution:
    def threeSum(self, nums: list[int]) -> list[list[int]]:
        nums.sort()
        n = len(nums)
        ans = []

        for i in range(n-2):
            if i>0 and nums[i] == nums[i-1]:
                continue

            l = i+1
            r = n-1
            while l<r:
                s = nums[i] + nums[l] + nums[r]
                if s<0:
                    l+=1

                elif s>0:
                    r-=1
```

```

        else:
            ans.append([nums[i], nums[l], nums[r]])
            l+=1
            r-=1
            while l<r and nums[l] == nums[l-1]:
                l+=1
            while l<r and nums[r] == nums[r+1]:
                r-=1

    return ans

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find all unique triplets summing to 0. Return the triplets, not indices.
 # No duplicate triplets in the output. Right?
 # Order within a triplet doesn't matter – [-1,0,1] and [0,-1,1] are the same."
 #
 # "[-1,0,1,2,-1,-4]: sort → [-4,-1,-1,0,1,2]. → [[-1,-1,2],[-1,0,1]] ✓
 # [0,0,0]: → [[0,0,0]] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
 # Three nested loops over $i < j < k$; if $\text{nums}[i] + \text{nums}[j] + \text{nums}[k] == 0$, add triplet.
 # Store triplets in a set keyed by sorted tuple to deduplicate.
 # Correct, but $O(n^3)$ with hash-set overhead on large n .
 # Time: $O(n^3)$. Space: $O(n)$ for dedup set.
 # Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Sort the array. Fix one element $\text{nums}[i]$, use two pointers on the rest.
 # Sorting enables: skip duplicate values of $\text{nums}[i]$ to avoid duplicate triplets,
 # and move pointers efficiently based on the sum.
 # After finding a valid triplet, skip duplicate $\text{nums}[\text{left}]$ and $\text{nums}[\text{right}]$.
 # Time: $O(n^2)$. Space: $O(1)$ extra."

PHASE 4 — CLEAN CODE

```

class _15_ThreeSum:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        nums.sort()
        result = []
        for i in range(len(nums) - 2):
            if i > 0 and nums[i] == nums[i - 1]: # skip duplicate fixed element
                continue

```

```

        left, right = i + 1, len(nums) - 1
        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                result.append([nums[i], nums[left], nums[right]])
                left += 1
                right -= 1
                while left < right and nums[left] == nums[left - 1]: left += 1
                while left < right and nums[right] == nums[right + 1]: right -= 1
1
    return result

```

PHASE 5 — TEST & DEBUG

```

# nums=[-1,0,1,2,-1,-4] → sorted: [-4,-1,-1,0,1,2].
# i=0 (-4): l=1,r=5. sums=-4+(-1)+2=-3<0→l++. -4+(-1)+2=-3<0→l++. -4+0+2=-2<0→l++.
# -4+1+2=-1<0→l++. l=r → stop.
# i=1 (-1): l=2,r=5. -1+(-1)+2=0→append[-1,-1,2]. l=3,r=4. skip dups: l=3 ok, r=4
ok.
# -1+0+1=0→append[-1,0,1]. l=4,r=3→stop.
# i=2 (-1): i>0 and nums[2]==nums[1]==-1 → skip.
# i=3 (0): l=4,r=5. 0+1+2=3>0→r--. l≥r→stop.
# result=[[-1,-1,2],[-1,0,1]] ✓
# nums=[0,0,0]: i=0,l=1,r=2. 0+0+0=0→append. l=2,r=1→stop. → [[0,0,0]] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(n^2)$  – outer loop  $O(n)$ , inner two-pointer  $O(n)$  each. Sorting is  $O(n \log n)$ 
dominated.
# Space  $O(1)$  extra (ignoring output).
# Two deduplication steps:
# 1. Skip  $\text{nums}[i] == \text{nums}[i-1]$  to avoid re-fixing the same outer value.
# 2. After appending, skip  $\text{nums}[\text{left}] == \text{nums}[\text{left}-1]$  and  $\text{nums}[\text{right}] ==$ 
 $\text{nums}[\text{right}+1]$ 
# to avoid re-collecting the same triplet.
# Both checks are essential – missing either produces duplicate output.
# Given more time I'd ask: 4Sum (#18)? Add one more outer loop –  $O(n^3)$  – same
structure."

```

Community Solutions (Python)

• WalkCC


```

class Solution:
    def threeSum(self, nums: list[int]) -> list[list[int]]:
        if len(nums) < 3:
            return []

        ans = []

        nums.sort()

        for i in range(len(nums) - 2):

            if i > 0 and nums[i] == nums[i - 1]:
                continue
            # Choose nums[i] as the first number in the triplet, then search the
            # remaining numbers in [i + 1, n - 1].
            l = i + 1
            r = len(nums) - 1
            while l < r:
                summ = nums[i] + nums[l] + nums[r]
                if summ == 0:
                    ans.append((nums[i], nums[l], nums[r]))

                    l += 1
                    r -= 1
                    while nums[l] == nums[l - 1] and l < r:
                        l += 1
                    while nums[r] == nums[r + 1] and l < r:
                        r -= 1
                elif summ < 0:
                    l += 1
                else:
                    r -= 1

        return ans

```

• LeetDoocs

```

class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        nums.sort()
        n = len(nums)
        ans = []
        for i in range(n - 2):
            if nums[i] > 0:
                break
            if i and nums[i] == nums[i - 1]:
                continue

            j, k = i + 1, n - 1
            while j < k:
                x = nums[i] + nums[j] + nums[k]
                if x < 0:
                    j += 1
                elif x > 0:
                    k -= 1
                else:
                    ans.append([nums[i], nums[j], nums[k]])
                    j, k = j + 1, k - 1

                    while j < k and nums[j] == nums[j - 1]:
                        j += 1
                    while j < k and nums[k] == nums[k + 1]:
                        k -= 1

        return ans

```

• SimplyLeet

```

# Function to find all unique triplets that sum to zero.
def threeSum(nums):
    # Sort the array to facilitate finding duplicates and using two pointers.
    nums.sort()
    result = []
    n = len(nums)

    # Iterate through the array, fixing one element at a time.
    for i in range(n - 2):
        # Skip duplicate elements for the first element.

```

```

if i > 0 and nums[i] == nums[i - 1]:
    continue

left, right = i + 1, n - 1
while left < right:
    current_sum = nums[i] + nums[left] + nums[right]
    # Check if the triplet adds up to zero.
    if current_sum == 0:
        result.append([nums[i], nums[left], nums[right]])
        # Skip duplicates for the second element.

        while left < right and nums[left] == nums[left + 1]:
            left += 1
        # Skip duplicates for the third element.
        while left < right and nums[right] == nums[right - 1]:
            right -= 1
        left += 1
        right -= 1
    elif current_sum < 0:
        left += 1
    else:
        right -= 1

return result

```

```

# Example usage:
#print(threeSum([-1,0,1,2,-1,-4]))

```

• LC.ca

```

class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        nums.sort()
        n = len(nums)
        ans = []
        for i in range(n - 2):
            if nums[i] > 0:
                break
            if i and nums[i] == nums[i - 1]:
                continue

```

```
j, k = i + 1, n - 1
while j < k:
    x = nums[i] + nums[j] + nums[k]
    if x < 0:
        j += 1
    elif x > 0:
        k -= 1
    else:
        ans.append([nums[i], nums[j], nums[k]])
        j, k = j + 1, k - 1

while j < k and nums[j] == nums[j - 1]:
    j += 1
while j < k and nums[k] == nums[k + 1]:
    k -= 1

return ans
```

◆ Quick Review

Key Insights

- Sort the array to efficiently skip duplicates and facilitate the two pointers approach.
- Use a fixed pointer for the first element, and then use two additional pointers (left and right) to find pairs that sum to the negation of the fixed element.
- Skip duplicate elements to ensure unique triplets.

Space & Time

Time Complexity: $O(n^2)$ where n is the number of elements in the array.

Space Complexity: $O(1)$ extra space (ignoring the space required for the output).

Solution

We first sort the array to bring duplicates together and allow the two-pointer technique to work efficiently. Then, iterate over the array and for each element, use two pointers (one at the beginning right after the current element and one at the end of the array) to find a pair that sums with the current element to zero. Skip duplicates for the current element and within the inner loop to avoid duplicate triplets. This ensures that each valid triplet is unique and the overall runtime remains efficient.

Two Pointers

□ #16 3Sum Closest

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Sorting
Lists: Grind 169

Problem

Given an integer array `nums` of length `n` and an integer `target`, find three integers at distinct indices in `nums` such that the sum is closest to `target`.

Return the sum of the three integers.

You may assume that each input would have exactly one solution.

Example 1:

```
Input: nums = [-1,2,1,-4], target = 1  
Output: 2  
Explanation: The sum that is closest to the target is 2. (-1 + 2 + 1 = 2).
```

Example 2:

```
Input: nums = [0,0,0], target = 1  
Output: 0  
Explanation: The sum that is closest to the target is 0. (0 + 0 + 0 = 0).
```

Constraints:

- $3 \leq \text{nums.length} \leq 500$
- $-1000 \leq \text{nums}[i] \leq 1000$
- $-104 \leq \text{target} \leq 104$

My LeetCode Solution

• My LeetCode Solution

```
# 3Sum Closest
class Solution:
    def threeSumClosest(self, nums: List[int], target: int) -> int:
        ans = nums[0]+nums[1]+nums[2]

        nums.sort()
        n = len(nums)

        for i in range(n-2):
            if i>0 and nums[i] == nums[i-1]:
                continue

            l = i+1
            r = n-1

            while l<r:
                s = nums[i] + nums[l] + nums[r]
                if target == s:
                    return s

                if abs(target - s) < abs(target- ans):
                    ans = s

                if s < target:
                    l += 1
                else:
                    r -= 1

            return ans
```


◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Find three distinct-index integers whose sum is closest to target. Return the
sum.
# Exactly one solution guaranteed – no tie-breaking needed. Right?"
#
# "[-1,2,1,-4], target=1: possible sums: -1+2+1=2, -1+2-4=-3, -1+1-4=-4, 2+1-4=-1.
# Closest to 1 is 2. → 2 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# Three nested loops – for every triple (i, j, k), compute the sum and track
# whichever sum has the smallest absolute difference from target.
# Time:  $O(n^3)$ . Space:  $O(1)$ . Should I go ahead and optimize?"
```

```
class _16_ThreeSumClosest_Brute:
    def threeSumClosest(self, nums, target):
        best = nums[0] + nums[1] + nums[2]
        n = len(nums)
        for i in range(n - 2):
            for j in range(i + 1, n - 1):
                for k in range(j + 1, n):
                    s = nums[i] + nums[j] + nums[k]
                    if abs(target - s) < abs(target - best):
                        best = s
        return best
```

PHASE 3 — OPTIMIZE

```
# "Exact same structure as 3Sum: sort + fix one element + two pointers.
# Instead of looking for sum==0, track the closest sum seen.
# If sum > target → move right pointer left (decrease sum).
# If sum < target → move left pointer right (increase sum).
# If sum == target → return immediately (can't get closer).
# Skip duplicate outer values to avoid redundant work – not required for correctness
# here (no dedup of output), but speeds up the average case."
```

PHASE 4 — CLEAN CODE

```
class _16_ThreeSumClosest:
    def threeSumClosest(self, nums: List[int], target: int) -> int:
        nums.sort()
        n = len(nums)
        best = nums[0] + nums[1] + nums[2]
        for i in range(n - 2):
            if i > 0 and nums[i] == nums[i - 1]:
                continue
            left, right = i + 1, n - 1
            while left < right:
                total = nums[i] + nums[left] + nums[right]
                if total == target:
```

```

        return target
    if abs(target - total) < abs(target - best):
        best = total
    if total > target:
        right -= 1
    else:
        left += 1
    return best

```

PHASE 5 — TEST & DEBUG

```

# nums=[-1,2,1,-4], target=1. sorted: [-4,-1,1,2].
# i=0(-4): l=1,r=3. -4-1+2=-3. |1-(-3)|=4 < |1-(-4)|=5 → best=-3. sum<1→l++.
# l=2,r=3. -4+1+2=-1. |1-(-1)|=2 < 4 → best=-1. sum<1→l++. l=r→stop.
# i=1(-1): l=2,r=3. -1+1+2=2. |1-2|=1 < |1-(-1)|=2 → best=2. sum>1→r--. l=r→stop.
# → 2 ✓
# nums=[0,0,0], target=1. best=0. No closer sum possible → 0 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(n^2)$ . Space  $O(1)$  extra ( $O(\log n)$  for sort).
# This is 3Sum (#15) with one change: track closest instead of exact zero.
# The early return on exact match is a nice optimisation worth mentioning.
# Given more time I'd ask: what if we want the k closest sums?
# Collect all triple sums, sort by  $|target - sum|$ , return first k —  $O(n^2 \log n)$ ."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def threeSumClosest(self, nums: list[int], target: int) -> int:
        ans = nums[0] + nums[1] + nums[2]

        nums.sort()

        for i in range(len(nums) - 2):
            if i > 0 and nums[i] == nums[i - 1]:
                continue
            # Choose nums[i] as the first number in the triplet, then search the

```

```

# remaining numbers in [i + 1, n - 1].
l = i + 1
r = len(nums) - 1
while l < r:
    summ = nums[i] + nums[l] + nums[r]
    if summ == target:
        return summ
    if abs(summ - target) < abs(ans - target):
        ans = summ
    if summ < target:
        l += 1
    else:
        r -= 1

return ans

```

• LeetDoocs

```

class Solution:
    def threeSumClosest(self, nums: List[int], target: int) -> int:
        nums.sort()
        n = len(nums)
        ans = inf
        for i, v in enumerate(nums):
            j, k = i + 1, n - 1
            while j < k:
                t = v + nums[j] + nums[k]
                if t == target:
                    return t
                if abs(t - target) < abs(ans - target):
                    ans = t
                if t > target:
                    k -= 1
                else:
                    j += 1
        return ans

```

• SimplyLeet

```

# Python solution for 3Sum Closest
def threeSumClosest(nums, target):
    # Sort the array to allow two pointer technique
    nums.sort()
    # Initialize best_sum with the sum of the first three elements
    best_sum = sum(nums[:3])

    # Iterate through each number as the fixed element
    for i in range(len(nums) - 2):
        left, right = i + 1, len(nums) - 1

        # Use two pointers to check pairs
        while left < right:
            current_sum = nums[i] + nums[left] + nums[right]
            # Update best_sum if current_sum is closer to target
            if abs(target - current_sum) < abs(target - best_sum):
                best_sum = current_sum
            # Move left pointer if current_sum is less than target to increase
            sum
            if current_sum < target:
                left += 1
            # Move right pointer if current_sum is greater than target to
            decrease sum

            elif current_sum > target:
                right -= 1
            else:
                # If the current_sum exactly equals target, return the target
                sum
                return current_sum
        # Return the closest sum found
        return best_sum

# Example usage:
# print(threeSumClosest([-1,2,1,-4], 1))

```

 • LC.ca

```
class Solution:
    def threeSumClosest(self, nums: List[int], target: int) -> int:
        nums.sort()
        n = len(nums)
        ans = inf
        for i, v in enumerate(nums):
            j, k = i + 1, n - 1
            while j < k:
                t = v + nums[j] + nums[k]
                if t == target:
                    return t
                if abs(t - target) < abs(ans - target):
                    ans = t
                if t > target:
                    k -= 1
                else:
                    j += 1
        return ans
```

◆ Quick Review

Key Insights

- Sort the array to make it easier to use the two pointers technique.
- Fix one element and then use two pointers (left and right) to find the best pair that, along with the fixed element, produces a sum closest to the target.
- Update the best sum whenever a closer sum is found.
- The sorted order helps eliminate unnecessary iterations and allows efficient

movement of pointers.

Space & Time

Time Complexity: $O(n^2)$

Space Complexity: $O(\log n)$ to $O(n)$ depending on the sorting algorithm used (typically in-place sort, so extra space might be minimal)

Solution

The solution starts by sorting the array. For each index in the array (considered as the first element of the triple), we use two pointers: one starting just after the fixed element and the other at the end of the array. We calculate the sum of the fixed element and the two pointers. If this sum is closer to the target than the current best, we update our best sum.

Depending on whether the current sum is less than or greater than the target, we adjust the left or right pointer to try and get closer to the target. This two-pointer method leverages the sorted order of the array for efficient summing.

Two Pointers

□ #31 Next Permutation

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers
Lists: Grind 169

Problem

A permutation of an array of integers is an arrangement of its members into a sequence or linear order.

- For example, for `arr = [1,2,3]`, the following are all the permutations of `arr`: `[1,2,3]`, `[1,3,2]`, `[2, 1, 3]`, `[2, 3, 1]`, `[3,1,2]`, `[3,2,1]`.

The next permutation of an array of integers is the next lexicographically greater permutation of its integer. More formally, if all the permutations of the array are sorted in one container according to their lexicographical order, then the next permutation of that array is the permutation that follows it in the sorted container. If such arrangement is not possible, the array must be rearranged as the lowest

possible order (i.e., sorted in ascending order).

- For example, the next permutation of $\text{arr} = [1,2,3]$ is $[1,3,2]$.
- Similarly, the next permutation of $\text{arr} = [2,3,1]$ is $[3,1,2]$.
- While the next permutation of $\text{arr} = [3,2,1]$ is $[1,2,3]$ because $[3,2,1]$ does not have a lexicographical larger rearrangement.

Given an array of integers `nums`, find the next permutation of `nums`.

The replacement must be in place and use only constant extra memory.

Example 1:

Input: `nums = [1,2,3]`
Output: `[1,3,2]`

Example 2:

Input: `nums = [3,2,1]`
Output: `[1,2,3]`

Example 3:

Input: `nums = [1,1,5]`
Output: `[1,5,1]`

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 100$

My LeetCode Solution

• My LeetCode Solution

```
# Next Permutation
class Solution:
    def nextPermutation(self, nums: List[int]) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """

        def rev(l,r):
            while l<r:
                nums[l], nums[r] = nums[r], nums[l]
                l,r = l+1, r-1

        n = len(nums)
        dip = next((i for i in range(n-2,-1,-1) if nums[i+1] > nums[i]), -1)
        if dip >= 0:
            swap = next((j for j in range(n-1,-1,-1) if nums[j] > nums[dip]), -1)
            nums[dip], nums[swap] = nums[swap], nums[dip]

            rev(dip+1, n-1)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Rearrange nums in-place to the next lexicographically greater permutation.
If no greater permutation exists (sorted descending), wrap to the smallest (ascending). Right?"

#

"[1,2,3]→[1,3,2]. [3,2,1]→[1,2,3]. [1,1,5]→[1,5,1]. [1,3,2]→[2,1,3]. ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

Generate all permutations of the array in sorted order, find the one that matches the current arrangement, then return the next one in the sorted list.

If the current is the last, wrap to the first (smallest) permutation.

Time: $O(n! \cdot n)$ to generate all permutations – completely infeasible for $n > 10$.

Space: $O(n!)$. Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Three steps – all $O(n)$, $O(1)$ space:

```
# 1. Find the rightmost 'dip': scan right to left for the largest i where nums[i] <
nums[i+1].
# If no dip exists, entire array is descending → just reverse it (wrap-around).
# 2. Find the smallest element to the right of i that is still larger than nums[i].
# (The suffix is descending after step 1, so scan from the right.)
# 3. Swap nums[i] with that element, then reverse the suffix after i.
# Reversing the descending suffix makes it the smallest possible arrangement."
```

PHASE 4 — CLEAN CODE

```
class _31_NextPermutation:
    def nextPermutation(self, nums: List[int]) -> None:
        def reverse(left, right):
            while left < right:
                nums[left], nums[right] = nums[right], nums[left]
                left += 1; right -= 1

        n = len(nums)
        # Step 1: find rightmost dip
        dip = next((i for i in range(n - 2, -1, -1) if nums[i] < nums[i + 1]), -1)
        if dip >= 0:
            # Step 2: find smallest element to right of dip that is > nums[dip]
            swap = next(j for j in range(n - 1, -1, -1) if nums[j] > nums[dip])
            nums[dip], nums[swap] = nums[swap], nums[dip]
        # Step 3: reverse suffix after dip
        reverse(dip + 1, n - 1)
```

PHASE 5 — TEST & DEBUG

```
# [1,2,3]: dip=1 (nums[1]=2<nums[2]=3). swap=2 (nums[2]=3>nums[1]=2).
# swap: [1,3,2]. reverse(2,2)→no change. → [1,3,2] ✓
# [3,2,1]: dip=-1. reverse(0,2): [1,2,3] ✓
# [1,1,5]: dip=1 (nums[1]=1<nums[2]=5). swap=2 (5>1). swap→[1,5,1].
reverse(2,2)→[1,5,1] ✓
# [1,3,2]: dip=0 (nums[0]=1<nums[1]=3). swap=1 (nums[1]=3>nums[0]=1; nums[2]=2
also>1 but
# scan right→left so j=1 first: nums[1]=3>1? yes → swap=1? No wait j starts from
n-1=2:
# nums[2]=2>nums[0]=1 → swap=2. swap→[2,3,1]. reverse(1,2): [2,1,3] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(1).
# The suffix is always descending after the dip – that's why scanning right-to-left
# finds the smallest swap target efficiently.
# The reverse of the suffix turns the largest possible suffix into the smallest –
the key insight.
# Given more time I'd ask: what's the previous permutation?
# Same algorithm mirrored: find rightmost 'rise' (nums[i] > nums[i+1]), swap with
largest
# element to its right that is still smaller, reverse the suffix."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def nextPermutation(self, nums: list[int]) -> None:
        n = len(nums)

        # From back to front, find the first number < nums[i + 1].
        i = n - 2
        while i >= 0:
            if nums[i] < nums[i + 1]:
                break
            i -= 1

        # From back to front, find the first number > nums[i], swap it with
        # nums[i].
        if i >= 0:
            for j in range(n - 1, i, -1):
                if nums[j] > nums[i]:
                    nums[i], nums[j] = nums[j], nums[i]
                    break

        def reverse(nums: list[int], l: int, r: int) -> None:
            while l < r:
                nums[l], nums[r] = nums[r], nums[l]
                l += 1
                r -= 1

        # Reverse nums[i + 1..n - 1].
        reverse(nums, i + 1, len(nums) - 1)
```

• LeetDoocs

```
class Solution:
    def nextPermutation(self, nums: List[int]) -> None:
        n = len(nums)
        i = next((i for i in range(n - 2, -1, -1) if nums[i] < nums[i + 1]), -1)

        if ~i:
            j = next((j for j in range(n - 1, i, -1) if nums[j] > nums[i]))
            nums[i], nums[j] = nums[j], nums[i]
            nums[i + 1:] = nums[i + 1:][::-1]
```

• SimplyLeet

```
def nextPermutation(nums):
    # Length of the array
    n = len(nums)
    # Find the first index from the right where the current number is less than
    the next number
    i = n - 2
    while i >= 0 and nums[i] >= nums[i + 1]:
        i -= 1
    # If such an index is found, find the number just larger than nums[i] from
    the right side to swap
    if i >= 0:
        j = n - 1
        while nums[j] <= nums[i]:
            j -= 1
        nums[i], nums[j] = nums[j], nums[i] # Swap the two numbers
    # Reverse the subarray from i+1 to the end to get the smallest sequence
    left, right = i + 1, n - 1
    while left < right:
        nums[left], nums[right] = nums[right], nums[left]
        left += 1
        right -= 1

# Example usage:
nums = [1, 2, 3]
nextPermutation(nums)
print(nums) # Output: [1, 3, 2]
```

• LC.ca

```
'''
>>> i = 3
>>> ~i
-4
>>> bool(~i)
True
#####

>>> j = -1
>>> ~j
```

```

0
>>> bool(~j)
False
#####

>>> a = (i for i in range (10, -1, -1) if i < 6)
>>> a
<generator object <genexpr> at 0x10a17eeb0>
>>> next(a)
5

```

```

>>>
>>>
>>> b = (i for i in range (10, -1, -1) if i < 0)
>>> next(b)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
>>> next(b, -1)
-1
>>>

```

```

'''
class Solution:
    def nextPermutation(self, nums: List[int]) -> None:
        '''
        Do not return anything, modify nums in-place instead.
        '''
        n = len(nums)
        # next(func, -1) => default value -1
        i = next((i for i in range(n - 2, -1, -1) if nums[i] < nums[i + 1]), -
1)
        if i != -1:

```

```

        j = next((j for j in range(n - 1, i, -1) if nums[j] > nums[i]))
        nums[i], nums[j] = nums[j], nums[i]
    nums[i + 1:] = nums[i + 1:][::-1]
    # wrong reverse: nums[i + 1:] = nums[i + 1::-1]

```

```
#####
```

```
'''
```

```

>>> a=[1,2,3]
>>> reversed(a)

```

```
<list_reverseiterator object at 0x108458be0>
```

```

>>> list(reversed(a))
[3, 2, 1]

```

```
# but, use reversed(a) to directly assign values is ok
```

```

>>> b=[4,5,6,7,8]
>>> b[:3] = reversed(a)
>>> b
[3, 2, 1, 7, 8]
'''

```

```
class Solution:
```

```

    def nextPermutation(self, nums: List[int]) -> None:
        if not nums:
            return

```

```
    # 寻找下一个排列
```

```
    for i in range(len(nums)-2, -1, -1): # 3, 4, 5, 2, 1
```

```
        if nums[i] < nums[i+1]: # 3 < 4
```

```
            j = next((j for j in range(len(nums)-1, i, -1) if nums[j] > num
s[i]) # 3 < 4
```

```
            nums[i], nums[j] = nums[j], nums[i] # 3,5,4,2,1
```

```

        # reverse []
        # []nums-i[]insert position
        nums[i+1:] = reversed(nums[i+1:]) # [4,2,1] reverse to
[1,2,4] => 3, 5, 1, 2, 4
        return

    nums.reverse() # for[]return[]

#####

```

```

class Solution:
    def nextPermutation(self, nums: List[int]) -> None:
        n = len(nums)
        to_left = [i for i in range(n - 2, -1, -1) if nums[i] < nums[i + 1]]

        if not to_left:
            nums = nums[::-1]
            return

        i = max(to_left)
        if i >= 0:
            to_right = [j for j in range(n - 1, i, -1) if nums[j] > nums[i] ]
            j = to_right[0]
            nums[i], nums[j] = nums[j], nums[i]
            nums[i + 1:] = nums[i + 1::-1]

```

◆ Quick Review

Key Insights

- The problem is solved by identifying the first pair from the right where the left element is less than the right element.

- Once this pivot is found, find the rightmost element that is greater than this pivot.
- Swap these two elements.
- Reverse the subarray that comes after the pivot to ensure the next permutation is the next lexicographically larger sequence.
- If no such pivot exists, the array is in descending order and should be reversed to get the smallest permutation.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

We start by scanning the array from the right to find the first index (i) such that $\text{nums}[i]$ is less than $\text{nums}[i+1]$. This index ' i ' represents the pivot point where the next permutation can be formed. If we find such an index, we then scan from the right again to locate the first number that is greater than $\text{nums}[i]$ (let this index be j), and swap these two numbers. Finally, we reverse the subarray starting from index $i+1$ to the end of the array, which gives us the next permutation in lexicographical

order. This approach leverages in-place modifications using basic operations like swapping and reversing, thus meeting the constant space requirement.

Two Pointers

□ #75 Sort Colors

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Sorting
Lists: Grind 169

Problem

Given an array `nums` with `n` objects colored red, white, or blue, sort them in-place so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers 0, 1, and 2 to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Example 1:

```
Input: nums = [2,0,2,1,1,0]
Output: [0,0,1,1,2,2]
```

Example 2:

```
Input: nums = [2,0,1]
Output: [0,1,2]
```

Constraints:

- `n == nums.length`
- `1 <= n <= 300`
- `nums[i]` is either 0, 1, or 2.

Follow up: Could you come up with a one-pass algorithm using only constant extra space?

My LeetCode Solution

• My LeetCode Solution

```
# Sort Colors
class Solution:
    def sortColors(self, nums: List[int]) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        l, m, r = 0, 0, len(nums)-1

        while m <= r:
            if nums[m] == 0:
                nums[l], nums[m] = nums[m], nums[l]
                l, m = l+1, m+1

            elif nums[m] == 1:
                m = m+1

            else:
                nums[m], nums[r] = nums[r], nums[m]
                r = r-1
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Sort an array containing only 0, 1, 2 in-place without using library sort.
# 0=red, 1=white, 2=blue. Output: all 0s, then 1s, then 2s. Right?
# Follow-up asks for a one-pass O(1) space solution."
#
```

```
# "[2,0,2,1,1,0] → [0,0,1,1,2,2] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic."
```

```
# First pass: count the number of 0s, 1s, and 2s in the array.
```

```
# Second pass: overwrite the array – fill count[0] zeros, then count[1] ones, then count[2] twos.
```

```
# This is two passes but still O(n) time and O(1) space.
```

```
# Time: O(n). Space: O(1). Should I go ahead and optimize?"
```

```
class _75_SortColors_Brute:
    def sortColors(self, nums):
        c0 = nums.count(0)
        c1 = nums.count(1)
        for i in range(len(nums)):
            if i < c0:
                nums[i] = 0
            elif i < c0 + c1:
                nums[i] = 1
            else:
                nums[i] = 2
```

PHASE 3 — OPTIMIZE

```
# "Dutch National Flag algorithm – one pass, three pointers: low, mid, high."
```

```
# Invariant: nums[0..low-1]=0, nums[low..mid-1]=1, nums[high+1..n-1]=2,
nums[mid..high]=unknown.
```

```
# At each step examine nums[mid]:
```

```
# 0 → swap(low,mid), low++, mid++ (0 placed, known-1 region grows both sides).
```

```
# 1 → mid++ (already correct).
```

```
# 2 → swap(mid,high), high-- (don't mid++ – swapped element is unknown)."
```

PHASE 4 — CLEAN CODE

```
class _75_SortColors:
    def sortColors(self, nums: List[int]) -> None:
        low = mid = 0
        high = len(nums) - 1
        while mid <= high:
            if nums[mid] == 0:
                nums[low], nums[mid] = nums[mid], nums[low]
                low += 1; mid += 1
            elif nums[mid] == 1:
                mid += 1
            else:
                nums[mid], nums[high] = nums[high], nums[mid]
                high -= 1 # don't mid++ – swapped value is unexamined
```

PHASE 5 — TEST & DEBUG

```
# [2,0,2,1,1,0]: low=0,mid=0,high=5.
```

```
# mid=0(2): swap(0,5)→[0,0,2,1,1,2]. high=4.
```

```
# mid=0(0): swap(0,0)→same. low=1,mid=1.
```

```
# mid=1(0): swap(1,1)→same. low=2,mid=2.
```

```
# mid=2(2): swap(2,4)→[0,0,1,1,2,2]. high=3.
# mid=2(1): mid=3.
# mid=3(1): mid=4. mid>high=3 → stop.
# → [0,0,1,1,2,2] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$ one pass. Space $O(1)$.

The 'don't mid++' after swapping a 2 is the subtlety – the swapped element from high

hasn't been examined yet and could be 0, 1, or 2.

After swapping a 0, we DO mid++ because nums[low] before the swap was definitely 1 (it's in the known-1 region), so after swapping, mid lands on a known-1 element.

Given more time I'd ask: what if there were k colours?

k=2 is a standard partition; k=3 is Dutch Flag; k>3 needs multiple passes or a radix approach."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def sortColors(self, nums: list[int]) -> None:
        zero = -1
        one = -1
        two = -1

        for num in nums:
            if num == 0:
                two += 1
                one += 1

            zero += 1
            nums[two] = 2
            nums[one] = 1
            nums[zero] = 0
            elif num == 1:
                two += 1
                one += 1
                nums[two] = 2
                nums[one] = 1
            else:
                two += 1
                nums[two] = 2
```

```

class Solution:
    def sortColors(self, nums: list[int]) -> None:
        l = 0 # The next 0 should be placed in l.
        r = len(nums) - 1 # The next 2 should be placed in r.

        i = 0
        while i <= r:
            if nums[i] == 0:
                nums[i], nums[l] = nums[l], nums[i]
                i += 1

                l += 1
            elif nums[i] == 1:
                i += 1
            else:
                # We may swap a 0 to index i, but we're still not sure whether this 0
                # is placed in the correct index, so we can't move pointer i.
                nums[i], nums[r] = nums[r], nums[i]
                r -= 1

```

• LeetCode

```

class Solution:
    def sortColors(self, nums: List[int]) -> None:
        i, j, k = -1, len(nums), 0
        while k < j:
            if nums[k] == 0:
                i += 1
                nums[i], nums[k] = nums[k], nums[i]
                k += 1
            elif nums[k] == 2:
                j -= 1

                nums[j], nums[k] = nums[k], nums[j]
            else:
                k += 1

```

• SimplyLeet


```

# Function to sort the colors using Dutch National Flag algorithm
def sortColors(nums):
    # Initialize pointers for the start, current, and end positions
    low, mid, high = 0, 0, len(nums) - 1

    # Process elements until mid pointer exceeds high pointer
    while mid <= high:
        # If the current element is 0 (red)
        if nums[mid] == 0:
            # Swap current element with the element at low pointer

            nums[low], nums[mid] = nums[mid], nums[low]
            # Increment both low and mid pointers
            low += 1
            mid += 1
        # If the current element is 1 (white)
        elif nums[mid] == 1:
            # Move mid pointer forward since it's already in the correct place
            mid += 1
        # If the current element is 2 (blue)
        else:
            # Swap current element with the element at high pointer
            nums[mid], nums[high] = nums[high], nums[mid]
            # Decrement high pointer
            high -= 1

    # Example usage:
    nums = [2,0,2,1,1,0]
    sortColors(nums)
    print(nums)  # Output: [0,0,1,1,2,2]

```

• LC.ca

```
class Solution:
    def sortColors(self, nums: List[int]) -> None:
        i, j, k = -1, len(nums), 0
        while k < j:
            if nums[k] == 0:
                i += 1
                nums[i], nums[k] = nums[k], nums[i]
                k += 1
            elif nums[k] == 2:
                j -= 1
                nums[j], nums[k] = nums[k], nums[j]
            else:
                k += 1
```

◆ Quick Review

Key Insights

- Use the Dutch National Flag algorithm to partition the array into three sections.
- Maintain three pointers: one for the next position for 0 (low), one for the next position for 2 (high), and one to traverse the array (mid).
- Swap elements appropriately to ensure that all 0's are at the beginning, all 2's are at the end, and 1's remain in the middle.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

The solution uses a three–pointer approach (Dutch National Flag algorithm). Initialize three pointers: low, mid, and high. As you iterate through the array: If the element is 0, swap it with the element at the low pointer and increment both low and mid. If the element is 1, simply move mid forward. If the element is 2, swap it with the element at the high pointer and decrement high. This single pass method ensures that the array is sorted in–place with constant extra space. Key details include correctly handling swaps without losing any data and ensuring that the pointers are updated in the proper order.

Two Pointers

□ ★ #161 One Edit Distance ★

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Two Pointers · String

Lists: ★ Premium | Premium 98

Problem

Given two strings *s* and *t*, return true if they are both one edit distance apart, otherwise return false.

A string *s* is said to be one distance apart from a string *t* if you can:

- Insert exactly one character into *s* to get *t*.
- Delete exactly one character from *s* to get *t*.
- Replace exactly one character of *s* with a different character to get *t*.

Example 1:

Input: *s* = "ab", *t* = "acb"

Output: true

Explanation: We can insert 'c' into *s* to get *t*.

Example 2:

Input: s = "", t = ""

Output: false

Explanation: We cannot get t from s by only one step.

Constraints:

- $0 \leq s.length, t.length \leq 104$
- s and t consist of lowercase letters, uppercase letters, and digits.

My LeetCode Solution

• My LeetCode Solution

```
# One Edit Distance
class Solution:
    def isOneEditDistance(self, s: str, t: str) -> bool:
        def check(short, long):
            for i in range(len(short)):
                if short[i] != long[i]:
                    return short[i:] == long[i+1:]

            return True

        m, n = len(s), len(t)
        if abs(m-n) > 1:
            return False

        if m==n:
            diffs = sum(a!=b for a,b in zip(s,t))
            return diffs == 1

        if m > n:
            m, n = n, m
            s, t = t, s

        return check(s, t)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Return true if s and t are exactly one edit apart – insert, delete, or replace.
# Important: zero edits (equal strings) returns false. Right?
# s='ab', t='acb' → insert 'c' → true ✓. s='',t='' → false ✓."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Run full Levenshtein DP between s and t; return True iff edit distance equals 1.
# Handles insert, delete, and replace in one table.
# Correct but O(m*n) when we only need to detect a single edit.
# Time: O(m * n). Space: O(m * n) DP table.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Two observations that short-circuit work:
# If |len(s)-len(t)| > 1 → impossible, return false.
# Ensure s is the shorter string (swap if needed) so we only handle two cases:
# same length → exactly one replacement, rest identical.
# s shorter by 1 → one deletion from t (or insertion into s), rest identical.
# Scan left to right. On first mismatch:
# same length → check s[i+1:] == t[i+1:].
# diff length → check s[i:] == t[i+1:].
# If no mismatch found → true only if t is exactly 1 longer."
```

PHASE 4 — CLEAN CODE

```
class _161_OneEditDistance:
    def isOneEditDistance(self, s: str, t: str) -> bool:
        def check(shorter, longer): # len(longer) == len(shorter) + 1
            for i in range(len(shorter)):
                if shorter[i] != longer[i]:
                    return shorter[i:] == longer[i + 1:]
            return True # shorter is prefix of longer
        m, n = len(s), len(t)
        if abs(m - n) > 1:
            return False
        if m == n:
            diffs = sum(a != b for a, b in zip(s, t))
            return diffs == 1
        if m > n:
            s, t = t, s
            m, n = n, m
        return check(s, t)
```

PHASE 5 — TEST & DEBUG

```
# s='ab', t='acb': m=2,n=3. m<n → check('ab','acb').
# i=0: 'a'=='a'. i=1: 'b'!='c'. return 'b'=='b'? shorter[1:]='b', longer[2:]='b' →
True ✓
```

```
# s='', t='': diffs=0 → return 0==1 → False ✓
# s='a', t='a': same length, diffs=0 → False ✓
# s='a', t='b': same length, diffs=1 → True ✓
# s='abc', t='abc': diffs=0 → False ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\min(m,n))$. Space $O(1)$ for the check (slice comparison is $O(\min)$ under the hood).

The key boundary: we return true when no mismatch found in the same-length case only if

$\text{diffs}==1$; in the diff-length case only if shorter is a prefix (true only if t is exactly

one char longer, which we already guaranteed).

Given more time I'd ask: what if we want EXACTLY k edits?

Full edit distance DP — $O(m*n)$ time, $O(\min(m,n))$ space."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isOneEditDistance(self, s: str, t: str) -> bool:
        m = len(s)
        n = len(t)
        if m > n: # Make sure that |s| <= |t|.
            return self.isOneEditDistance(t, s)

        for i in range(m):
            if s[i] != t[i]:
                if m == n:

                    return s[i + 1:] == t[i + 1:] # Replace s[i] with t[i].
                return s[i:] == t[i + 1:] # Delete t[i].

        return m + 1 == n # Delete t[-1].
```

• LeetDoocs


```

class Solution:
    def isOneEditDistance(self, s: str, t: str) -> bool:
        if len(s) < len(t):
            return self.isOneEditDistance(t, s)
        m, n = len(s), len(t)
        if m - n > 1:
            return False
        for i, c in enumerate(t):
            if c != s[i]:
                return s[i + 1 :] == t[i + 1 :] if m == n else s[i + 1 :] == t
[i:]

        return m == n + 1

```

• SimplyLeet

```

# Function to check if two strings are one edit distance apart
def isOneEditDistance(s: str, t: str) -> bool:
    # Get lengths of both strings
    m, n = len(s), len(t)

    # The difference in length must not be greater than 1
    if abs(m - n) > 1:
        return False

    # Ensure s is the shorter string, or they are equal in length

    if m > n:
        return isOneEditDistance(t, s)

    # Initialize variables; found difference flag and indices for both strings
    found_difference = False
    i = j = 0

    while i < m and j < n:
        if s[i] != t[j]:
            # If a difference has already been found, return False

```

```

        if found_difference:
            return False
        found_difference = True
        # If lengths are equal, move both pointers
        if m == n:
            i += 1
        else:
            # If characters are the same, move the pointer in the shorter
string
            i += 1
        # Always move pointer for longer string

        j += 1

        # If no difference was found during iteration, then one extra character at
the end is the edit
        return found_difference or (j - i == 1)

# Example usage:
#print(isOneEditDistance("ab", "acb")) # Output: True

```

• LC.ca

```

class Solution:
    def isOneEditDistance(self, s: str, t: str) -> bool:
        if len(s) < len(t): # goal is to ensure t is always the shorter one
            return self.isOneEditDistance(t, s)
        m, n = len(s), len(t)
        if m - n > 1:
            # case: abc, abcdefg, return false
            return False
        for i, c in enumerate(t):
            if c != s[i]:

```

```

        return s[i + 1 :] == t[i + 1 :] if m == n else s[i + 1 :] == t
[i:]
    # case: abc, abcd. return true
    # case: abc, abc. the same 2 strings should return false
    # case: abc, abcdefg, return false
    return m == n + 1

#####

class Solution(object):
    def isOneEditDistance(self, s, t):

        """
        :type s: str
        :type t: str
        :rtype: bool
        """
        if len(s) >= len(t):
            i = 0
            while i < len(t) and s[i] == t[i]:
                i += 1
            return s != t and (s[i + 1:] == t[i:] if len(s) != len(t) else s[i + 1:]
== t[i + 1:])

        else:
            return self.isOneEditDistance(t, s)

```

◆ Quick Review

Key Insights

- The difference in length between s and t must be at most 1.
- When lengths are equal, the only possibility is that exactly one character is different (replacement).

- When one string is longer by one character, the difference can be fixed by one insertion (or deletion in the other direction).
- Use two pointers to compare corresponding characters and carefully handle the first difference.

Space & Time

Time Complexity: $O(n)$, where n is the length of the shorter string, as we traverse both strings at most once.

Space Complexity: $O(1)$, using only constant extra space.

Solution

The solution uses a two-pointer approach. First, we check if the length difference is greater than 1, in which case they cannot be one edit apart. For the two primary cases: When strings are of equal length, iterate through both strings and count the number of differing characters. There should be exactly one difference. When lengths differ by one, use pointers for both strings. Upon encountering a difference, increment the pointer for the longer string only and ensure that no previous

difference has been encountered. If a second discrepancy is found, return false. The key is to handle the edge case where the difference is at the end of the string.

Two Pointers

□ #167 Two Sum II – Input Array Is Sorted

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Binary Search
Lists: Denny Zhang

Problem

Given a 1-indexed array of integers `numbers` that is already sorted in non-decreasing order, find two numbers such that they add up to a specific target number. Let these two numbers be `numbers[index1]` and `numbers[index2]` where $1 \leq \text{index1} < \text{index2} \leq \text{numbers.length}$.

Return the indices of the two numbers `index1` and `index2`, each incremented by one, as an integer array `[index1, index2]` of length 2.

The tests are generated such that there is exactly one solution. You may not use the same element twice.

Your solution must use only constant extra space.

Example 1:

Input: numbers = [2,7,11,15], target = 9

Output: [1,2]

Explanation: The sum of 2 and 7 is 9. Therefore, index1 = 1, index2 = 2. We return [1, 2].

Example 2:

Input: numbers = [2,3,4], target = 6

Output: [1,3]

Explanation: The sum of 2 and 4 is 6. Therefore index1 = 1, index2 = 3. We return [1, 3].

Example 3:

Input: numbers = [-1,0], target = -1

Output: [1,2]

Explanation: The sum of -1 and 0 is -1. Therefore index1 = 1, index2 = 2. We return [1, 2].

Constraints:

- $2 \leq \text{numbers.length} \leq 3 * 10^4$
- $-1000 \leq \text{numbers}[i] \leq 1000$
- numbers is sorted in non-decreasing order.
- $-1000 \leq \text{target} \leq 1000$
- The tests are generated such that there is exactly one solution.

My LeetCode Solution

- My LeetCode Solution


```
# Two Sum II – Input Array Is Sorted
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        l = 0
        r = len(nums)-1

        while l<r:
            s = nums[l]+nums[r]
            if s == target:
                return [l+1, r+1]

            elif s< target:
                l+=1

            else:
                r-=1

        return []
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Sorted array (1-indexed). Return [index1, index2] where
numbers[index1]+numbers[index2]==target.
# Exactly one solution. Must use O(1) extra space – so no hash map. Right?"
#
# "[2,7,11,15], target=9: 2+7=9 → [1,2] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# For every pair (i, j) where j > i, check if numbers[i] + numbers[j] == target.
# Return [i+1, j+1] (1-indexed) when found. The problem guarantees exactly one
solution.
# Time: O(n²). Space: O(1). Should I go ahead and optimize?"
```

```
class _167_TwoSumII_Brute:
    def twoSum(self, numbers, target):
        for i in range(len(numbers)):
            for j in range(i + 1, len(numbers)):
                if numbers[i] + numbers[j] == target:
                    return [i + 1, j + 1]

        return []
```

PHASE 3 — OPTIMIZE

```
# "Two pointers exploit sorted order: start left=0, right=n-1."
```

```
# Sum too small → move left right (increase sum).
# Sum too large → move right left (decrease sum).
# Exactly one solution guaranteed → we will always find it.
# Time O(n). Space O(1)."
```

PHASE 4 — CLEAN CODE

```
class _167_TwoSumII:
    def twoSum(self, numbers: List[int], target: int) -> List[int]:
        left, right = 0, len(numbers) - 1
        while left < right:
            total = numbers[left] + numbers[right]
            if total == target:
                return [left + 1, right + 1] # 1-indexed
            elif total < target:
                left += 1
            else:
                right -= 1
        return []
```

PHASE 5 — TEST & DEBUG

```
# [2,7,11,15], target=9: l=0,r=3. 2+15=17>9→r=2. 2+11=13>9→r=1. 2+7=9→return [1,2] ✓
# [2,3,4], target=6: l=0,r=2. 2+4=6→return [1,3] ✓
# [-1,0], target=-1: -1+0=-1→return [1,2] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(1).
# This is the canonical reason to sort before running two pointers on Two Sum.
# If the array weren't sorted, we'd need a hash map – O(n) space.
# Sorting adds O(n log n) but enables O(1) space – a genuine trade-off.
# Given more time I'd ask: what if there could be multiple answers?
# Continue after finding a match, skip duplicates – same as 3Sum's dedup logic."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def twoSum(self, numbers: list[int], target: int) -> list[int]:
        l = 0
        r = len(numbers) - 1

        while l < r:
            summ = numbers[l] + numbers[r]
            if summ == target:
                return [l + 1, r + 1]
            if summ < target:
```

```

        l += 1
    else:
        r -= 1

```

• LeetCode

```

class Solution:
    def twoSum(self, numbers: List[int], target: int) -> List[int]:
        n = len(numbers)
        for i in range(n - 1):
            x = target - numbers[i]
            j = bisect_left(numbers, x, lo=i + 1)
            if j < n and numbers[j] == x:
                return [i + 1, j + 1]

```

```

class Solution:
    def twoSum(self, numbers: List[int], target: int) -> List[int]:
        i, j = 0, len(numbers) - 1
        while i < j:
            x = numbers[i] + numbers[j]
            if x == target:
                return [i + 1, j + 1]
            if x < target:
                i += 1
            else:
                j -= 1

```

• SimplyLeet

```

# Python solution for Two Sum II - Input Array Is Sorted
def twoSum(numbers, target):
    # Initialize two pointers at the beginning and end of the array
    left = 0
    right = len(numbers) - 1

    # Continue until the two pointers meet
    while left < right:
        # Calculate the sum of the two pointed values
        current_sum = numbers[left] + numbers[right]

```

```

        # If the sum matches the target, return the 1-indexed results
        if current_sum == target:
            return [left + 1, right + 1]
        # If the current sum is less than the target, move the left pointer to
the right
        elif current_sum < target:
            left += 1
        # Otherwise, move the right pointer to the left since the current sum
is too high
        else:
            right -= 1

# Example call
print(twoSum([2, 7, 11, 15], 9)) # Output: [1, 2]

```

• LC.ca

```

class Solution:
    def twoSum(self, numbers: List[int], target: int) -> List[int]:
        i, j = 0, len(numbers) - 1
        while i < j:
            x = numbers[i] + numbers[j]
            if x == target:
                return [i + 1, j + 1]
            if x < target:
                i += 1
            else:
                j -= 1

```

◆ Quick Review

Key Insights

- The array is sorted in non-decreasing order, which makes the two pointers technique very effective.

- Using two pointers (one at the start and one at the end) allows checking pairs and adjusting the pointers based on the sum.
- Since the problem guarantees exactly one solution and requires constant space, using two pointers satisfies both constraints.

Space & Time

Time Complexity: $O(n)$, where n is the number of elements in the array, because in the worst-case scenario each element is visited at most once.

Space Complexity: $O(1)$, as no additional data structure is used that scales with the input size.

Solution

We use the two pointers approach: Initialize two pointers, `left` at the beginning (index 0) and `right` at the end (last index) of the array. While `left` is less than `right`, compute the sum of the elements at the two pointers. If the sum equals the target, return the indices as `[left+1, right+1]` (to adjust for the 1-indexed array). If the sum is less than the target, move the `left` pointer to the right to increase the sum. If the

sum is greater than the target, move the right pointer to the left to decrease the sum. This approach leverages the sorted property and uses constant extra space.

Two Pointers

★ #186 Reverse Words in a String II ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Two Pointers · String

Lists: ★ Premium | Premium 98

Problem

Given a character array *s*, reverse the order of the words.

A word is defined as a sequence of non-space characters. The words in *s* will be separated by a single space.

Your code must solve the problem in-place, i.e. without allocating extra space.

Example 1:

```
Input: s = ["t","h","e"," ","s","k","y"," ","i","s"," ","b","l","u","e"]  
Output: ["b","l","u","e"," ","i","s"," ","s","k","y"," ","t","h","e"]
```

Example 2:

```
Input: s = ["a"]  
Output: ["a"]
```

Constraints:

- $1 \leq s.length \leq 105$

- `s[i]` is an English letter (uppercase or lowercase), digit, or space ' '.
- There is at least one word in `s`.
- `s` does not contain leading or trailing spaces.
- All the words in `s` are guaranteed to be separated by a single space.

My LeetCode Solution

• My LeetCode Solution

```
# Reverse Words in a String II
class Solution:
    def reverseWords(self, s: List[str]) -> None:
        """
        Do not return anything, modify s in-place instead.
        """

        def rev(l,r):
            while l<r:
                s[l], s[r] = s[r], s[l]
                l+=1

            r-=1

        rev(0, len(s)-1)
        p = 0
        for i in range(1+len(s)):
            if i== len(s) or s[i] == " ":
                rev(p, i-1)
                p = i+1
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given a character ARRAY (not a string), reverse the order of words in-place."

```
# Words are separated by single spaces. No leading/trailing spaces. Right?
# ['t','h','e',' ','s','k','y'] → ['s','k','y',' ','t','h','e'] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
```

```
# Convert the character array to a string, split by spaces to get words, reverse the word list,
```

```
# rejoin with spaces, then write each character back into the original array.
```

```
# Time: O(n). Space: O(n) – we create a full string copy. Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Two-step in-place reversal with O(1) extra space:
```

```
# Step 1: Reverse the entire array.
```

```
# Step 2: Reverse each individual word back to its correct orientation.
```

```
# Example: 'the sky is blue' → reverse all → 'eulb si yks eht' → reverse words → 'blue is sky the' ✓"
```

PHASE 4 — CLEAN CODE

```
class _186_ReverseWordsII:
    def reverseWords(self, s: List[str]) -> None:
        def reverse(left, right):
            while left < right:
                s[left], s[right] = s[right], s[left]
                left += 1; right -= 1
        reverse(0, len(s) - 1)
        start = 0
        for i in range(len(s) + 1):
            if i == len(s) or s[i] == ' ':
                reverse(start, i - 1)
                start = i + 1
```

PHASE 5 — TEST & DEBUG

```
# s=['t','h','e',' ','s','k','y',' ','i','s',' ','b','l','u','e']:
# Step 1 reverse all: ['e','l','u','b',' ','s','i',' ','y','k','s',' ','e','h','t']
# Step 2 reverse words:
# i=0..3 'e','l','u','b' → reverse → 'b','l','u','e'
# i=5..6 's','i' → 'i','s'
# i=8..10 'y','k','s' → 's','k','y'
# i=12..14 'e','h','t' → 't','h','e'
# → ['b','l','u','e',' ','i','s',' ','s','k','y',' ','t','h','e'] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(1) – two reversal passes, no extra memory.
```

```
# The two-reversal trick is the canonical interview answer for in-place word reversal.
```

```
# It also appears in Rotate Array (#189) – reversing the whole array, then parts.
```

```
# Given more time I'd ask: what if words can be separated by multiple spaces?
```

```
# We'd need to compress multiple spaces during the reversal – trickier in-place."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def reverseWords(self, s: List[str]) -> None:
        def reverse(l: int, r: int) -> None:
            while l < r:
                s[l], s[r] = s[r], s[l]
                l += 1
                r -= 1

        def reverseWords(n: int) -> None:
            i = 0

            j = 0

            while i < n:
                while i < j or (i < n and s[i] == ' '): # Skip the spaces.
                    i += 1
                while j < i or (j < n and s[j] != ' '): # Skip the spaces.
                    j += 1
                reverse(i, j - 1) # Reverse the word.

            reverse(0, len(s) - 1) # Reverse the whole string.

        reverseWords(len(s)) # Reverse each word.
```

• LeetDoocs

```
class Solution:
    def reverseWords(self, s: List[str]) -> None:
        def reverse(i: int, j: int):
            while i < j:
                s[i], s[j] = s[j], s[i]
                i, j = i + 1, j - 1

        i, n = 0, len(s)
        for j, c in enumerate(s):
            if c == " ":
```

```

        reverse(i, j - 1)
        i = j + 1
    elif j == n - 1:
        reverse(i, j)
reverse(0, n - 1)

```

• SimplyLeet

```

def reverse_range(s, left, right):
    # Reverse the elements in s from index left to right.
    while left < right:
        s[left], s[right] = s[right], s[left]
        left += 1
        right -= 1

def reverseWords(s):
    # Step 1: Reverse the entire array.
    reverse_range(s, 0, len(s) - 1)

```

```

n = len(s)
start = 0
# Step 2: Reverse each word to correct their order.
for i in range(n + 1):
    if i == n or s[i] == ' ':
        reverse_range(s, start, i - 1)
        start = i + 1

```

• LC.ca

```

class Solution:
    def reverseWords(self, s: List[str]) -> None:
        """
        Do not return anything, modify s in-place instead.
        """

    def reverse(s, i, j):
        while i < j:
            s[i], s[j] = s[j], s[i]
            i += 1

```

```
        j -= 1

    i, j, n = 0, 0, len(s)
    while j < n:
        if s[j] == ' ':
            reverse(s, i, j - 1)
            i = j + 1
        elif j == n - 1:
            reverse(s, i, j)
        j += 1

    reverse(s, 0, n - 1)
```

◆ Quick Review

Key Insights

- Reverse the entire array first to obtain the reversed order of characters.
- Then, reverse each individual word to restore the correct letter order.
- Doing so leverages the two-pointer technique for in-place reversals.
- This method accomplishes the goal in $O(n)$ time while using $O(1)$ extra space.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

The solution employs a two-step reversal process using the two-pointer technique. First, reverse the entire character array to reverse the overall order of characters. Although this reverses both the words and the letters in each word, it sets up the scenario where each word's letters are in reverse order. The next step is to iterate through the array and locate each word (using the space character as a delimiter) and then reverse each word individually to correct its letter order. This process is done in-place without any additional data structures, conserving space.

Two Pointers

□ #189 Rotate Array

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math · Two Pointers
Lists: Grind 169

Problem

Given an integer array `nums`, rotate the array to the right by `k` steps, where `k` is non-negative.

Example 1:

```
Input: nums = [1,2,3,4,5,6,7], k = 3
Output: [5,6,7,1,2,3,4]
Explanation:
rotate 1 steps to the right: [7,1,2,3,4,5,6]
rotate 2 steps to the right: [6,7,1,2,3,4,5]
rotate 3 steps to the right: [5,6,7,1,2,3,4]
```

Example 2:

```
Input: nums = [-1,-100,3,99], k = 2
Output: [3,99,-1,-100]
Explanation:
rotate 1 steps to the right: [99,-1,-100,3]
rotate 2 steps to the right: [3,99,-1,-100]
```

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$

- $0 \leq k \leq 105$

Follow up:

- Try to come up with as many solutions as you can. There are at least three different ways to solve this problem.
- Could you do it in-place with $O(1)$ extra space?

My LeetCode Solution

• My LeetCode Solution

```
# Rotate Array
class Solution:
    def rotate(self, nums: List[int], k: int) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """

        def rev(nums, l, r):
            while l < r:
                nums[l], nums[r] = nums[r], nums[l]
                l += 1
                r -= 1

        n = len(nums)
        k = k % n

        rev(nums, 0, n-1)
        rev(nums, 0, k-1)
        rev(nums, k, n-1)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Rotate nums to the right by k steps in-place. k can exceed n – so k %= n. Right?
# [1,2,3,4,5,6,7], k=3 → [5,6,7,1,2,3,4] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
```

```
# Rotate the array one position to the right, k times in a row.
```

```
# Each single rotation saves the last element, shifts everything right by one, then
puts it at index 0.
```

```
# Time:  $O(n \cdot k)$  – for  $k=n-1$  this is  $O(n^2)$ . Space:  $O(1)$ . Should I go ahead and
optimize?"
```

```
class _189_RotateArray_Brute:
    def rotate(self, nums, k):
        n = len(nums)
        k %= n
        for _ in range(k):
            last = nums[-1]
            for i in range(n - 1, 0, -1):
                nums[i] = nums[i - 1]
            nums[0] = last
```

PHASE 3 — OPTIMIZE (three-reversal trick)

```
# "Same pattern as Reverse Words II:
```

```
#  $k \mathrel{\%} n$  (handle  $k \geq n$ ).
```

```
# Reverse entire array. Reverse first k elements. Reverse remaining  $n-k$  elements.
```

```
# Time  $O(n)$ . Space  $O(1)$ .
```

```
# Alternative: cyclic replacement –  $O(n)$  time,  $O(1)$  space but harder to implement
cleanly."
```

PHASE 4 — CLEAN CODE

```
class _189_RotateArray:
    def rotate(self, nums: List[int], k: int) -> None:
        def reverse(left, right):
            while left < right:
                nums[left], nums[right] = nums[right], nums[left]
                left += 1; right -= 1

        n = len(nums)
        k %= n
        reverse(0, n - 1)
        reverse(0, k - 1)
        reverse(k, n - 1)
```

PHASE 5 — TEST & DEBUG

```
# [1,2,3,4,5,6,7], k=3:
```

```
#  $k \mathrel{\%} 7 = 3$ . reverse(0,6): [7,6,5,4,3,2,1].
```

```
# reverse(0,2): [5,6,7,4,3,2,1].
```

```
# reverse(3,6): [5,6,7,1,2,3,4] ✓
```

```
# [-1,-100,3,99], k=2:
```

```
#  $k \mathrel{\%} 4 = 2$ . reverse all: [99,3,-100,-1]. reverse(0,1): [3,99,-100,-1]. reverse(2,3):
[3,99,-1,-100] ✓
```

k=0: k%n=0. reverse all, reverse(0,-1) no-op, reverse(0,n-1) reverses back → no change ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time 0(n). Space 0(1).

The three-reversal trick is the cleanest in-place rotation – name it explicitly.

The k%n guard handles k >= n where full rotations cancel out.

The follow-up mentions three approaches: extra array 0(n) space, cyclic replacement 0(1),

and three-reversal 0(1). State all three and recommend the reversal for interviews.

Given more time I'd ask: rotate left instead of right?

Reverse first k, reverse rest, reverse all – just change the order."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def rotate(self, nums: list[int], k: int) -> None:
        k %= len(nums)
        self.reverse(nums, 0, len(nums) - 1)
        self.reverse(nums, 0, k - 1)
        self.reverse(nums, k, len(nums) - 1)

    def reverse(self, nums: list[int], l: int, r: int) -> None:
        while l < r:
            nums[l], nums[r] = nums[r], nums[l]

            l += 1
            r -= 1
```

• LeetDoocs

```
class Solution:
    def rotate(self, nums: List[int], k: int) -> None:
        def reverse(i: int, j: int):
            while i < j:
                nums[i], nums[j] = nums[j], nums[i]
                i, j = i + 1, j - 1

        n = len(nums)
        k %= n
        reverse(0, n - 1)
```

```
reverse(0, k - 1)
reverse(k, n - 1)
```

• SimplyLeet

```
# Python solution using the reverse method

def rotate(nums, k):
    # Calculate the effective rotations needed
    n = len(nums)
    k = k % n # in case k is greater than the length of the array

    # Helper function to reverse a portion of the list in place
    def reverse(left, right):
        while left < right:

            # Swap the elements at indices left and right
            nums[left], nums[right] = nums[right], nums[left]
            left += 1
            right -= 1

    # Reverse the entire array
    reverse(0, n - 1)
    # Reverse the first k elements to restore their order
    reverse(0, k - 1)
    # Reverse the remaining n - k elements

    reverse(k, n - 1)

# Example usage:
# nums = [1,2,3,4,5,6,7]
# rotate(nums, 3)
# print(nums) # Output: [5,6,7,1,2,3,4]
```

• LC.ca

```
class Solution:
    def rotate(self, nums: List[int], k: int) -> None:
        k %= len(nums)
        nums[:] = nums[-k:] + nums[:-k]
```

```
#####
```

```
class Solution:
    def rotate(self, nums: List[int], k: int) -> None:
        #####
```

```
Do not return anything, modify nums in-place instead.
#####
```

```
n = len(nums)
k %= n
if n < 2 or k == 0:
    return
nums[:] = nums[::-1]
nums[:k] = nums[:k][::-1]
nums[k:] = nums[k:][::-1]
```

```
#####
```

```
class Solution(object):
    def rotate(self, nums, k):
        #####
        :type nums: List[int]
        :type k: int
        :rtype: void Do not return anything, modify nums in-place instead.
        #####
        if len(nums) == 0 or k == 0:
```

```
    return
```

```
def reverse(start, end, s):
    while start < end:
        s[start], s[end] = s[end], s[start]
        start += 1
        end -= 1
```

```
n = len(nums) - 1
k = k % len(nums)
```

```
reverse(0, n - k, nums)
reverse(n - k + 1, n, nums)
reverse(0, n, nums)
```

◆ Quick Review

Key Insights

- The rotation amount k might be greater than the length of the array, so use $k \% n$ (where n is the array length) to compute the effective rotation.
- A smart in-place solution involves reversing sections of the array:
 - Reverse the entire array.
 - Reverse the first k elements.
 - Reverse the remaining $n-k$ elements.
- Alternative approaches include using extra space or performing k single-step rotations (which is less optimal).

Space & Time

Time Complexity: $O(n)$ where n is the number of elements in the array.

Space Complexity: $O(1)$ if done in-place.

Solution

We solve the problem using the "reverse" technique to perform the rotation in-place. First, we adjust k by taking k modulo the length of the array to handle cases where k exceeds the array length. Then, we reverse the entire array. Next, we reverse the first k elements to restore their correct order. Finally, we reverse the remaining $n-k$ elements. This sequence yields the array rotated to the right by k positions while strictly using $O(1)$ extra space.

Two Pointers

□ #532 K-diff Pairs in an Array

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Two Pointers · Binary
Search · Sorting
Lists: CodeSignal

Problem

Given an array of integers `nums` and an integer `k`, return the number of unique `k`-diff pairs in the array.

A `k`-diff pair is an integer pair (`nums[i]`, `nums[j]`), where the following are true:

- $0 \leq i, j < \text{nums.length}$
- $i \neq j$
- $|\text{nums}[i] - \text{nums}[j]| == k$

Notice that $|\text{val}|$ denotes the absolute value of `val`.

Example 1:

Input: nums = [3,1,4,1,5], k = 2

Output: 2

Explanation: There are two 2-diff pairs in the array, (1, 3) and (3, 5). Although we have two 1s in the input, we should only return the number of unique pairs.

Example 2:

Input: nums = [1,2,3,4,5], k = 1

Output: 4

Explanation: There are four 1-diff pairs in the array, (1, 2), (2, 3), (3, 4) and (4, 5).

Example 3:

Input: nums = [1,3,1,5,4], k = 0

Output: 1

Explanation: There is one 0-diff pair in the array, (1, 1).

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^7 \leq \text{nums}[i] \leq 10^7$
- $0 \leq k \leq 10^7$

My LeetCode Solution

• My LeetCode Solution

```
# K-diff Pairs in an Array
class Solution:
    def findPairs(self, nums: List[int], k: int) -> int:
        seen = set()
        ans = set()

        for x in nums:
            if x-k in seen:
                ans.add(x-k)
```

```

        if x+k in seen:
            ans.add(x)

        seen.add(x)

    return len(ans)

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Count UNIQUE pairs (i,j) where |nums[i]-nums[j]|==k and i!=j.
# Unique means unique value pairs – (1,3) and (3,1) count once.
# k=0 special case: need TWO copies of the same value. Right?"
#
# "[3,1,4,1,5], k=2: unique pairs (1,3) and (3,5) → 2 ✓
# [1,3,1,5,4], k=0: need duplicate values. Only 1 appears twice → (1,1) → 1 ✓"

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with the brute force to lock in the logic.
# For every pair (i, j) where i != j, check if |nums[i] - nums[j]| == k.
# Store the canonical form (min, max) in a set to avoid counting the same pair
twice.
# Time: O(n^2). Space: O(n) for the dedup set. Should I go ahead and optimize?"
class _532_KdiffPairs_Brute:
    def findPairs(self, nums, k):
        pairs = set()
        for i in range(len(nums)):
            for j in range(len(nums)):
                if i != j and abs(nums[i] - nums[j]) == k:
                    pairs.add((min(nums[i], nums[j]), max(nums[i], nums[j])))
        return len(pairs)

```

PHASE 3 — OPTIMIZE

```

# "Hash set approach – O(n) time, O(n) space:
# Walk through nums. For each x, check if x-k is already in 'visited'.
# If yes, add min(x, x-k) to the answer set (canonical pair).
# Also check x+k in visited (symmetric check handles both directions).
# Using a set for answers automatically deduplicates.
# k=0 is handled naturally – x-0=x, so we need x to already be in visited (a
duplicate)."

```

PHASE 4 — CLEAN CODE

```

class _532_KdiffPairs:
    def findPairs(self, nums: List[int], k: int) -> int:
        answers : set = set()
        visited : set = set()

```

```

for x in nums:
    if x - k in visited:
        answers.add(x - k) # canonical smaller element
    if x + k in visited:
        answers.add(x) # x is the larger; x+k was seen before
    visited.add(x)
return len(answers)

```

PHASE 5 — TEST & DEBUG

```

# [3,1,4,1,5], k=2:
# x=3: 3-2=1 not in visited. 3+2=5 not in visited. visited={3}.
# x=1: 1-2=-1 not in visited. 1+2=3 in visited → answers.add(1). visited={3,1}.
# x=4: 4-2=2 not in visited. 4+2=6 not in visited. visited={3,1,4}.
# x=1: 1-2=-1 not in visited. 1+2=3 in visited → answers.add(1) (already there).
visited={3,1,4}.
# x=5: 5-2=3 in visited → answers.add(3). 5+2=7 not. visited={3,1,4,5}.
# answers={1,3} → 2 ✓
# [1,3,1,5,4], k=0:
# x=1: 1-0=1 not in visited. visited={1}.
# x=3: 3-0=3 not in visited. visited={1,3}.
# x=1: 1-0=1 in visited → answers.add(1). visited={1,3}.
# answers={1} → 1 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n). Space O(n) for the two sets.
# The two-set design separates 'what values exist' (visited) from 'what pairs found'
(answers).
# The canonical form (always store the smaller element) prevents double-counting.
# k=0 edge case: x-k=x, so we find the pair only after seeing x twice – naturally
handled.
# Given more time I'd ask: what if we want the actual pairs, not just the count?
# Store tuples (x-k, x) in the answers set instead of single values."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def findPairs(self, nums: list[int], k: int) -> int:
        ans = 0
        numToIndex = {num: i for i, num in enumerate(nums)}

        for i, num in enumerate(nums):
            target = num + k
            if target in numToIndex and numToIndex[target] != i:
                ans += 1
            del numToIndex[target]

```

```
return ans
```

• LeetCode

```
class Solution:
    def findPairs(self, nums: List[int], k: int) -> int:
        ans = set()
        vis = set()
        for x in nums:
            if x - k in vis:
                ans.add(x - k)
            if x + k in vis:
                ans.add(x)
            vis.add(x)

        return len(ans)
```

• SimplyLeet

```
# Define a function to count k-diff pairs in the array.
def find_pairs(nums, k):
    # If k is negative, return 0 as absolute difference can't be negative.
    if k < 0:
        return 0

    count = 0

    # k == 0: Need to find numbers that appear more than once.
    if k == 0:

        # Create a frequency map for nums.
        freq = {}
        for num in nums:
            freq[num] = freq.get(num, 0) + 1
        # Count numbers with frequency greater than 1.
        for val in freq.values():
            if val > 1:
                count += 1
    else:
        # k > 0: Use a set to check for the existence of each number's
        complement.
```

```

        unique_nums = set(nums)
        for num in unique_nums:
            if num + k in unique_nums:
                count += 1
        return count

# Example usage:
# print(find_pairs([3,1,4,1,5], 2)) # Expected output: 2

```

• LC.ca

```

class Solution:
    def findPairs(self, nums: List[int], k: int) -> int:
        vis, ans = set(), set()
        for v in nums:
            if v - k in vis:
                ans.add(v - k)
            if v + k in vis:
                ans.add(v)
            vis.add(v)
        return len(ans)

```

◆ Quick Review

Key Insights

- For $k < 0$, the answer is always 0 because the absolute difference cannot be negative.
- When k is 0, you are looking for numbers that appear at least twice.
- For $k > 0$, it is efficient to use a hash set or a hash map to check if each number's complement ($\text{number} + k$) exists.

- Uniqueness is ensured by processing each number only once from the set of unique numbers.

Space & Time

Time Complexity: $O(n)$, where n is the length of the array since we iterate over the array and then over the unique numbers.

Space Complexity: $O(n)$, for storing the frequency count or the unique elements in a hash map or hash set.

Solution

The solution first checks if k is negative, returning 0 immediately if true. For k equals 0, we count numbers that appear more than once using a frequency map. For k greater than 0, a set of unique numbers is created and for each number, we check if $(\text{number} + k)$ is also present in the set. Each valid case contributes to the result count, ensuring that only unique pairs are considered.

Two Pointers

□ #923 3Sum With Multiplicity

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Two Pointers · Sorting ·
Counting

Lists: CodeSignal

Problem

Given an integer array `arr`, and an integer `target`, return the number of tuples `i, j, k` such that $i < j < k$ and $arr[i] + arr[j] + arr[k] == target$.

As the answer can be very large, return it modulo $10^9 + 7$.

Example 1:

```
Input: arr = [1,1,2,2,3,3,4,4,5,5], target = 8
Output: 20
Explanation:
Enumerating by the values (arr[i], arr[j], arr[k]):
(1, 2, 5) occurs 8 times;
(1, 3, 4) occurs 8 times;
(2, 2, 4) occurs 2 times;
(2, 3, 3) occurs 2 times.
```

Example 2:

Input: arr = [1,1,2,2,2,2], target = 5
 Output: 12
 Explanation:
 arr[i] = 1, arr[j] = arr[k] = 2 occurs 12 times:
 We choose one 1 from [1,1] in 2 ways,
 and two 2s from [2,2,2,2] in 6 ways.

Example 3:

Input: arr = [2,1,3], target = 6
 Output: 1
 Explanation: (1, 2, 3) occurred one time in the array so we return 1.

Constraints:

- $3 \leq \text{arr.length} \leq 3000$
- $0 \leq \text{arr}[i] \leq 100$
- $0 \leq \text{target} \leq 300$

My LeetCode Solution

• My LeetCode Solution

```
# 3Sum With Multiplicity
class Solution:
    def threeSumMulti(self, arr: List[int], target: int) -> int:
        ans = 0
        mod = 10**9 + 7
        hmap = Counter(arr)

        for i, sn in enumerate(arr):
            hmap[sn] -= 1
            for fn in arr[:i]:
                tn = target - (fn + sn)
                ans = (ans + hmap[tn]) % mod

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Count tuples (i,j,k) with i<j<k and arr[i]+arr[j]+arr[k]==target.
# Return count modulo 10^9+7. Values are 0-100 so we can use frequency counting.
Right?
# Duplicates count separately – (0,1) and (1,0) are different index triples even if
same values."
#
# "[1,1,2,2,3,3,4,4,5,5], target=8 → 20 (combinations of value triples summing to 8)
✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# Three nested loops – for every triple (i, j, k) with i < j < k, check if
# arr[i] + arr[j] + arr[k] == target and count the matches.
# Time: O(n³). For n=3000 that's 27 billion – too slow. Space: O(1).
# Should I go ahead and optimize?"
```

```
class _923_ThreeSumMulti_Brute:
    def threeSumMulti(self, arr, target):
        MOD = 10**9 + 7
        count = 0
        n = len(arr)
        for i in range(n - 2):
            for j in range(i + 1, n - 1):
                for k in range(j + 1, n):
                    if arr[i] + arr[j] + arr[k] == target:
                        count += 1
        return count % MOD
```

PHASE 3 — OPTIMIZE

```
# "Walk through arr as the middle element j. Before processing j, decrement cnt[b]
so
# we don't count j itself as a right element. For each a < j, the required c =
target-a-b.
# cnt[c] tells us how many valid right elements c exist to the right of j.
# Time: O(n²) with frequency lookups. Space: O(n) for the Counter."
```

PHASE 4 — CLEAN CODE

```
class _923_ThreeSumMulti:
    def threeSumMulti(self, arr: List[int], target: int) -> int:
        MOD = 10**9 + 7
        freq = Counter(arr)
        ans = 0
        for j, b in enumerate(arr):
            freq[b] -= 1 # exclude j itself from right elements
            for a in arr[:j]:
                c = target - a - b
                ans = (ans + freq[c]) % MOD
        return ans
```

PHASE 5 — TEST & DEBUG

```
# arr=[2,1,3], target=6: freq={2:1,1:1,3:1}.
# j=0(b=2): freq[2]=0. no a < 0. (nothing).
# j=1(b=1): freq[1]=0. a=2: c=6-2-1=3. freq[3]=1 → ans=1.
# j=2(b=3): freq[3]=0. a=2: c=6-2-3=1. freq[1]=0→0. a=1: c=6-1-3=2.
freq[2]=1→ans=2?
# Wait – expected 1. Let me recheck: arr=[2,1,3]. Only valid triple is (0,1,2) →
(2,1,3).
# j=1(b=1): a=arr[0]=2. c=6-2-1=3. freq[3]=1 (yes, index 2 exists) → ans=1. ✓
# j=2(b=3): freq[3]=0. a=arr[0]=2: c=6-2-3=1. freq[1]=0 (decremented at j=1). 0.
# a=arr[1]=1: c=6-1-3=2. freq[2]=1 → ans=2? But expected 1.
# Ah – freq[2] was decremented when j=0: freq[2]=0, not 1. Let me re-trace:
# Initial freq={2:1,1:1,3:1}.
# j=0: freq[2]-=1 → freq={2:0,1:1,3:1}. no a<j=0.
# j=1: freq[1]-=1 → freq={2:0,1:0,3:1}. a=2: c=3. freq[3]=1 → ans=1.
# j=2: freq[3]-=1 → freq={2:0,1:0,3:0}. a=2: c=1. freq[1]=0. a=1: c=2. freq[2]=0.
# → ans=1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^2)$. Space $O(n)$ for the Counter.

The key invariant: at step j, freq counts only elements at index > j.

This ensures we count each valid (i,j,k) tuple exactly once with i<j<k.

Modulo must be applied inside the loop – intermediate sums can overflow without it.

Given more time I'd ask: can we do $O(n)$ using the value constraints (0-100)?

Yes – iterate over all value triples (a,b,c) with a+b+c=target and compute

combinations $C(cnt[a],1)*C(cnt[b],1)*C(cnt[c],1)$ with dedup for equal values."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def threeSumMulti(self, arr: list[int], target: int) -> int:
        MOD = 1_000_000_007
        ans = 0
        count = collections.Counter(arr)

        for i, x in count.items():
            for j, y in count.items():
                k = target - i - j
                if k not in count:
```

```

        continue
    if i == j and j == k:
        ans = (ans + x * (x - 1) * (x - 2) // 6) % MOD
    elif i == j and j != k:
        ans = (ans + x * (x - 1) // 2 * count[k]) % MOD
    elif i < j and j < k:
        ans = (ans + x * y * count[k]) % MOD

return ans % MOD

```

• LeetDoocs

```

class Solution:
    def threeSumMulti(self, arr: List[int], target: int) -> int:
        mod = 10**9 + 7
        cnt = Counter(arr)
        ans = 0
        for j, b in enumerate(arr):
            cnt[b] -= 1
            for a in arr[:j]:
                c = target - a - b
                ans = (ans + cnt[c]) % mod

        return ans

```

• SimplyLeet

```

MOD = 10**9 + 7

def threeSumMulti(arr, target):
    # Create a frequency array for numbers 0 to 100
    freq = [0] * 101
    for num in arr:
        freq[num] += 1

    count = 0
    # Iterate through all possible values for x, y, and implicitly z

```

```

for x in range(101):
    if freq[x] == 0:
        continue
    for y in range(x, 101):
        if freq[y] == 0:
            continue
        z = target - x - y
        if z < 0 or z > 100 or freq[z] == 0:
            continue
        if x == y and y == z:

            # All numbers are the same: nC3 = n*(n-1)*(n-2)/6
            count += freq[x] * (freq[x] - 1) * (freq[x] - 2) // 6
        elif x == y and y != z:
            # x and y are the same, z is different: nC2 for x multiplied by
count[z]

            count += (freq[x] * (freq[x] - 1) // 2) * freq[z]
        elif x < y and y < z:
            # All distinct: product of counts
            count += freq[x] * freq[y] * freq[z]
        elif x < y and y == z:
            # x different, y and z are the same: count[x] multiplied by nC2
for y

            count += freq[x] * (freq[y] * (freq[y] - 1) // 2)
        count %= MOD
    return count

# Example usage:
print(threeSumMulti([1,1,2,2,3,3,4,4,5,5], 8))

```

• LC.ca

```

class Solution:
    def threeSumMulti(self, arr: List[int], target: int) -> int:
        cnt = Counter(arr)
        ans = 0
        mod = 10**9 + 7
        for j, b in enumerate(arr):
            cnt[b] -= 1
            for i in range(j):
                a = arr[i]
                c = target - a - b

```

```
        ans = (ans + cnt[c]) % mod
    return ans
```

◆ Quick Review

Key Insights

- Use a frequency counter for elements since $\text{arr}[i]$ is in the range $[0, 100]$.
- Iterate through all possible combinations of numbers x , y , and z with $x \leq y \leq z$.
- For each valid triplet (x, y, z) that sums to target, count the number of ways using combinatorial formulas:
 - All distinct: $\text{count}[x] * \text{count}[y] * \text{count}[z]$.
 - Two numbers the same: use $nC2$ for the repeated element.
 - All three the same: use $nC3$.
- Use modulo arithmetic ($\text{MOD} = 10^9 + 7$) at every step to avoid overflow.

Space & Time

Time Complexity: $O(M^2)$, where $M = 101$ (the range of possible numbers), which is efficient given the constraints.

Space Complexity: $O(M)$ for the frequency array.

Solution

The solution first counts the frequency of each number in the array. Then it iterates through all valid triplets (x, y, z) (with $x \leq y \leq z$) and checks if they sum up to the target. Depending on whether x , y , and z are distinct or have duplicates, the combination count is computed accordingly: If x , y , and z are all the same, compute the combination as $nC3$. If only two are the same, compute as $nC2$ multiplied by the count of the third number. If all are distinct, use the direct product of the counts. This covers all cases while applying the modulo operation to the result.

Two Pointers

□ #986 Interval List Intersections

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers
Lists: Denny Zhang

Problem

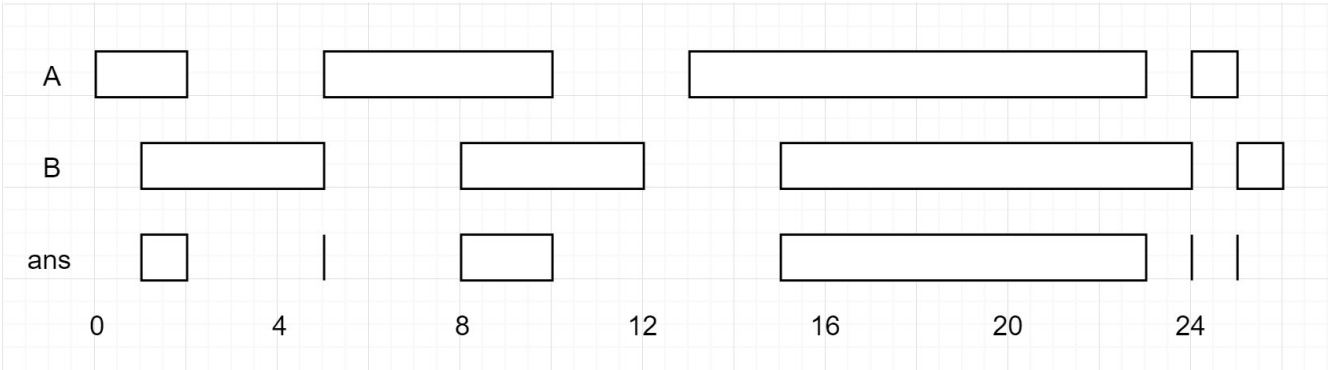
You are given two lists of closed intervals, `firstList` and `secondList`, where `firstList[i] = [starti, endi]` and `secondList[j] = [startj, endj]`. Each list of intervals is pairwise disjoint and in sorted order.

Return the intersection of these two interval lists.

A closed interval $[a, b]$ (with $a \leq b$) denotes the set of real numbers x with $a \leq x \leq b$.

The intersection of two closed intervals is a set of real numbers that are either empty or represented as a closed interval. For example, the intersection of $[1, 3]$ and $[2, 4]$ is $[2, 3]$.

Example 1:



Input: firstList = [[0,2],[5,10],[13,23],[24,25]], secondList =
[[1,5],[8,12],[15,24],[25,26]]
Output: [[1,2],[5,5],[8,10],[15,23],[24,24],[25,25]]

Example 2:

Input: firstList = [[1,3],[5,9]], secondList = []
Output: []

Constraints:

- $0 \leq \text{firstList.length}, \text{secondList.length} \leq 1000$
- $\text{firstList.length} + \text{secondList.length} \geq 1$
- $0 \leq \text{start}_i < \text{end}_i \leq 109$
- $\text{end}_i < \text{start}_i + 1$
- $0 \leq \text{start}_j < \text{end}_j \leq 109$
- $\text{end}_j < \text{start}_j + 1$

My LeetCode Solution

- ☐
- My LeetCode Solution

```
# Interval List Intersections
class Solution:
    def intervalIntersection(self, firstList: List[List[int]], secondList: List[List[int]]) -> List[List[int]]:
        ans = []
        i,j = 0,0
        m = len(firstList)
        n = len(secondList)
        while i < m and j < n:
            f1, f2 = firstList[i]
            s1, s2 = secondList[j]

            start = max(f1, s1)
            end = min(f2, s2)

            if start <= end:
                ans.append([start, end])

            if f2 < s2:
                i+=1

            else:
                j+=1

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Two sorted, disjoint interval lists. Return their intersection – all overlapping segments.

Intersection of [a,b] and [c,d] is [max(a,c), min(b,d)] if max(a,c) <= min(b,d). Right?

Either list can be empty."

#

"[[0,2],[5,10]] ∩ [[1,5],[8,12]] → [1,2],[5,5],[8,10] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

For every pair of intervals (one from each list), compute the intersection:

start = max(a[0], b[0]), end = min(a[1], b[1]). If start <= end, it's a valid intersection.

```
# This is O(m*n) – it ignores the sorted property entirely.
# Time: O(m*n). Space: O(1). Should I go ahead and optimize?"
```

```
class _986_IntervalIntersections_Brute:
    def intervalIntersection(self, firstList, secondList):
        result = []
        for a in firstList:
            for b in secondList:
                start = max(a[0], b[0])
                end = min(a[1], b[1])
                if start <= end:
                    result.append([start, end])
        return result
```

PHASE 3 — OPTIMIZE

```
# "Two pointers, one per list. At each step:
# Compute the intersection of the current pair.
# If valid (start <= end), record it.
# Advance the pointer whose interval ends FIRST – that interval can't intersect any
# later interval
# of the other list (they're sorted and disjoint).
# Time: O(m+n). Space: O(1) extra."
```

PHASE 4 — CLEAN CODE

```
class _986_IntervalIntersections:
    def intervalIntersection(self, firstList: List[List[int]], secondList:
List[List[int]]) -> List[List[int]]:
        result = []
        i = j = 0
        while i < len(firstList) and j < len(secondList):
            start = max(firstList[i][0], secondList[j][0])
            end = min(firstList[i][1], secondList[j][1])
            if start <= end:
                result.append([start, end])
            if firstList[i][1] < secondList[j][1]:
                i += 1
            else:
                j += 1
        return result
```

PHASE 5 — TEST & DEBUG

```
# firstList=[[0,2],[5,10]], secondList=[[1,5],[8,12]]:
# i=0,j=0: start=max(0,1)=1, end=min(2,5)=2. 1<=2→[1,2]. first ends at 2<5→i=1.
# i=1,j=0: start=max(5,1)=5, end=min(10,5)=5. 5<=5→[5,5]. first ends at 10>5→j=1.
# i=1,j=1: start=max(5,8)=8, end=min(10,12)=10. 8<=10→[8,10]. first ends at
10<12→i=2.
# i=2 out of bounds → stop. → [[1,2],[5,5],[8,10]] ✓
# secondList=[]: loop never enters → [] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(m+n) – each pointer advances at most m+n times.
```

Space $O(\min(m,n))$ for the output in the worst case.
 # The 'advance the earlier-ending pointer' rule is the key: the interval that ends first
 # cannot intersect any future interval of the other list (both lists are sorted and disjoint).
 # Given more time I'd ask: what if lists aren't sorted?
 # Sort both first – $O((m+n)\log(m+n))$ – then apply the same two-pointer approach."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def intervalIntersection(self, firstList: list[list[int]],
                           secondList: list[list[int]]) -> list[list[int]]:

        ans = []
        i = 0
        j = 0

        while i < len(firstList) and j < len(secondList):
            # lo := the start of the intersection
            # hi := the end of the intersection

            lo = max(firstList[i][0], secondList[j][0])
            hi = min(firstList[i][1], secondList[j][1])
            if lo <= hi:
                ans.append([lo, hi])
            if firstList[i][1] < secondList[j][1]:
                i += 1
            else:
                j += 1

        return ans
```

• LeetDoocs

```

class Solution:
    def intervalIntersection(
        self, firstList: List[List[int]], secondList: List[List[int]]
    ) -> List[List[int]]:
        i = j = 0
        ans = []
        while i < len(firstList) and j < len(secondList):
            s1, e1, s2, e2 = *firstList[i], *secondList[j]
            l, r = max(s1, s2), min(e1, e2)
            if l <= r:
                ans.append([l, r])
            if e1 < e2:
                i += 1
            else:
                j += 1
        return ans

```

• SimplyLeet

```

# Function to compute the interval intersections using two pointers
def intervalIntersection(firstList, secondList):
    intersections = [] # List to store the resulting intersections
    i, j = 0, 0 # Pointers for firstList and secondList

    # Loop until either list is fully traversed
    while i < len(firstList) and j < len(secondList):
        start1, end1 = firstList[i]
        start2, end2 = secondList[j]

        # Find the intersection boundaries
        start_overlap = max(start1, start2)
        end_overlap = min(end1, end2)

        # Check if there is an intersection
        if start_overlap <= end_overlap:
            intersections.append([start_overlap, end_overlap])

        # Move the pointer that has the smaller interval ending
        if end1 < end2:

```

```

        i += 1
    else:
        j += 1

    return intersections

# Example usage:
firstList = [[0,2],[5,10],[13,23],[24,25]]
secondList = [[1,5],[8,12],[15,24],[25,26]]
print(intervalIntersection(firstList, secondList)) # Output:
[[1,2],[5,5],[8,10],[15,23],[24,24],[25,25]]

```

• LC.ca

```

class Solution:
    def intervalIntersection(
        self, firstList: List[List[int]], secondList: List[List[int]]
    ) -> List[List[int]]:
        i = j = 0
        ans = []
        while i < len(firstList) and j < len(secondList):
            s1, e1, s2, e2 = *firstList[i], *secondList[j]
            l, r = max(s1, s2), min(e1, e2)
            if l <= r:
                ans.append([l, r])
                if e1 < e2:
                    i += 1
                else:
                    j += 1
        return ans

```

◆ Quick Review

Key Insights

- Use two pointers to traverse both lists simultaneously.

- Determine the overlapping interval by taking the maximum of the start points and the minimum of the end points.
- If the overlapping condition ($\text{start} \leq \text{end}$) is met, add the intersection to the result.
- Move the pointer that has the smaller end value to explore further potential intersections efficiently.
- Achieve an overall time complexity of $O(m+n)$, where m and n are the lengths of the two interval lists.

Space & Time

Time Complexity: $O(m + n)$ where m and n are the lengths of `firstList` and `secondList` respectively.

Space Complexity: $O(1)$ extra space (excluding the space used for the output).

Solution

The solution uses a two pointers approach. Initialize two indices, one for each list, and iterate through both lists. For each pair of intervals, the potential intersection is computed by: Calculating the maximum of the two start values. Calculating the minimum of

the two end values. If the maximum start is less than or equal to the minimum end, there is an intersection, and it is added to the result list. The pointer corresponding to the interval with the smaller end value is then incremented since that interval cannot intersect with any further intervals from the other list. This process repeats until one of the lists is fully traversed.

Two Pointers

★ #1055 Shortest Way to Form String ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String · Binary Search
Lists: ★ Premium | Premium 98

Problem

A subsequence of a string is a new string that is formed from the original string by deleting some (can be none) of the characters without disturbing the relative positions of the remaining characters. (i.e., "ace" is a subsequence of "abcde" while "aec" is not).

Given two strings `source` and `target`, return the minimum number of subsequences of `source` such that their concatenation equals `target`. If the task is impossible, return `-1`.

Example 1:

Input: `source = "abc", target = "abcbc"`

Output: 2

Explanation: The target "abcbc" can be formed by "abc" and "bc", which are subsequences of source "abc".

Example 2:

Input: source = "abc", target = "acdbc"

Output: -1

Explanation: The target string cannot be constructed from the subsequences of source string due to the character "d" in target string.

Example 3:

Input: source = "xyz", target = "xzyxz"

Output: 3

Explanation: The target string can be constructed as follows "xz" + "y" + "xz".

Constraints:

- $1 \leq \text{source.length}, \text{target.length} \leq 1000$
- source and target consist of lowercase English letters.

My LeetCode Solution

• My LeetCode Solution

```
# Shortest Way to Form String
class Solution:
    def shortestWay(self, source: str, target: str) -> int:
        if set(target) - set(source):
            return -1

        count = 0
        target_idx = 0

        while target_idx < len(target):

            for char in source:
                if target_idx < len(target) and char == target[target_idx]:
                    target_idx += 1
            count += 1

        return count
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the minimum number of subsequences of source whose concatenation equals target.

A subsequence can skip characters but must preserve order.

Return -1 if any character in target doesn't exist in source at all. Right?"

#

"source='abc', target='abcbc': 'abc' + 'bc' → 2 ✓

source='abc', target='acdbc': 'd' not in source → -1 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.

Try every possible way to split target into consecutive segments, and for each split,

verify each segment is a subsequence of source. Track the minimum number of segments.

This is exponential in the number of splits – completely infeasible.

Time: $O(2^{|target|} * |source|)$. Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Greedy: each 'round', scan source once and match as many target characters as possible.

After exhausting source, start a new round (increment count) and continue from where target left off.

Pre-check: if any target char is absent from source, return -1.

Time: $O(|source| * |target|)$. Space: $O(1)$ extra."

PHASE 4 — CLEAN CODE

```
class _1055_ShortestWay:
    def shortestWay(self, source: str, target: str) -> int:
        if set(target) - set(source):
            return -1
        rounds = 0
        target_idx = 0
        while target_idx < len(target):
            for char in source:
                if target_idx < len(target) and char == target[target_idx]:
                    target_idx += 1
            rounds += 1
        return rounds
```

PHASE 5 — TEST & DEBUG

source='abc', target='abcbc':

Round 1: a→match t[0]='a'(idx=1). b→match t[1]='b'(idx=2). c→match t[2]='c'(idx=3). rounds=1.

Round 2: a→no match(t[3]='b'). b→match t[3]='b'(idx=4). c→match t[4]='c'(idx=5). rounds=2.

target_idx=5=len(target) → return 2 ✓

source='xyz', target='xzyxz':

Round1: x→t[0]='x'(1). y→no(t[1]='z'). z→t[1]='z'(2). rounds=1.

Round2: x→no(t[2]='y'). y→t[2]='y'(3). z→no(t[3]='x'). rounds=2.

Round3: x→t[3]='x'(4). y→no(t[4]='z'). z→t[4]='z'(5). rounds=3. → 3 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(|source| * |target|)$. Space $O(1)$.

The pre-check eliminates the -1 case cleanly before the main loop.

Optimisation: precompute for each (source_pos, char) the next position in source

where char appears – binary search brings each round down to $O(|target| \log |source|)$.

Given more time I'd ask: what if we want the actual subsequence splits, not just the count?

Track the split points during the greedy pass."

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def shortestWay(self, source: str, target: str) -> int:
        def f(i, j):
            while i < m and j < n:
                if source[i] == target[j]:
                    j += 1
                i += 1
            return j

        m, n = len(source), len(target)

        ans = j = 0
        while j < n:
            k = f(0, j)
            if k == j:
                return -1
            j = k
            ans += 1
        return ans
```

• SimplyLeet

```

# Python solution using greedy two-pointer approach

def shortestWay(source, target):
    count = 0 # Count of subsequences used
    t_index = 0
    t_len = len(target)

    # Pre-check: if any character in target is not in source, return -1
    if set(target) - set(source):
        return -1

    # While there are still characters in target to match
    while t_index < t_len:
        s_index = 0 # Pointer for source
        progress = False # Flag to check if we made progress this pass

        # Iterate through source and try to match target characters
        while s_index < len(source) and t_index < t_len:
            if source[s_index] == target[t_index]:
                t_index += 1 # Move to next character in target

            progress = True
            s_index += 1

        count += 1 # One subsequence used for this pass

        # If no progress was made, then it is impossible to form the target
        if not progress:
            return -1

    return count

# Example usage:
# print(shortestWay("abc", "abcbcb")) # Output: 2

```

• LC.ca


```
class Solution:
    def shortestWay(self, source: str, target: str) -> int:
        def f(i, j):
            while i < m and j < n:
                if source[i] == target[j]:
                    j += 1
                i += 1
            return j

        m, n = len(source), len(target)

        ans = j = 0
        while j < n:
            k = f(0, j)
            if k == j:
                return -1
            j = k
            ans += 1
        return ans
```

◆ Quick Review

Key Insights

- Check if every character in target exists in source; if not, the answer is -1.
- Use a greedy two-pointer strategy: iterate over target while scanning source for matching characters.
- During each pass over source, match as many characters as possible from target.
- Count the number of passes (subsequences) required until the target is completely matched.

- An optimized approach may precompute indices for binary search; however, the basic greedy simulation is efficient for the given constraints.

Space & Time

Time Complexity: $O(n * m)$ in the worst-case, where n = length of source and m = length of target.

Space Complexity: $O(1)$ for the greedy simulation ($O(n)$ if additional data structures for binary search are used).

Solution

The solution uses a greedy two-pointer approach. We simulate the process of reading the target by repeatedly scanning through the source string. Each scan through source is treated as one subsequence concatenated to form the target. A pointer in the target moves forward whenever a matching character is found in source. If a complete pass through source does not advance the target pointer, then it is impossible to form the target and we return -1 . The algorithm counts the number of passes required until the target is fully

matched.

Two Pointers

□ #2563 Count the Number of Fair Pairs

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Binary Search · Sorting
Lists: CodeSignal

Problem

Given a 0-indexed integer array `nums` of size `n` and two integers `lower` and `upper`, return the number of fair pairs.

A pair (i, j) is fair if:

- $0 \leq i < j < n$, and
- $\text{lower} \leq \text{nums}[i] + \text{nums}[j] \leq \text{upper}$

Example 1:

Input: `nums = [0,1,7,4,4,5]`, `lower = 3`, `upper = 6`
Output: 6
Explanation: There are 6 fair pairs: $(0,3)$, $(0,4)$, $(0,5)$, $(1,3)$, $(1,4)$, and $(1,5)$.

Example 2:

Input: `nums = [1,7,9,2,5]`, `lower = 11`, `upper = 11`
Output: 1
Explanation: There is a single fair pair: $(2,3)$.

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $\text{nums.length} == n$
- $-109 \leq \text{nums}[i] \leq 109$
- $-109 \leq \text{lower} \leq \text{upper} \leq 109$

My LeetCode Solution

• My LeetCode Solution

```
# Count the Number of Fair Pairs
class Solution:
    def countFairPairs(self, nums: List[int], lower: int, upper: int) -> int:
        def low(start, target):
            l = start
            r = len(nums)-1

            while l<=r:
                m = (l+r)//2
                if nums[m]>= target:

                    r = m-1

                else:
                    l = m+1

            return l

        def up(start, target):
            l = start
```

```

        r = len(nums)-1

        while l<=r:
            m = (l+r)//2
            if nums[m]> target:
                r = m-1

            else:
                l = m+1

        return l

    nums.sort()
    ans = 0
    for i in range(len(nums)):
        l = low(i+1, lower-nums[i])
        u = up(i+1, upper-nums[i])

        ans += u-l

    return ans

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count pairs (i,j) with $i < j$ and $\text{lower} \leq \text{nums}[i] + \text{nums}[j] \leq \text{upper}$. Return the count. Right?"

Values can be negative. n up to 10^5 so $O(n^2)$ is too slow."

#

"[0,1,7,4,4,5], lower=3, upper=6: 6 pairs ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic."

For every pair (i, j) with $i < j$, check if $\text{lower} \leq \text{nums}[i] + \text{nums}[j] \leq \text{upper}$.

Count all pairs that satisfy both bounds. For $n=10^5$ this is $5 \cdot 10^9$ comparisons – too slow.

Time: $O(n^2)$. Space: $O(1)$. Should I go ahead and optimize?"

```
class _2563_FairPairs_Brute:
```

```
    def countFairPairs(self, nums, lower, upper):
        count = 0
```

```

for i in range(len(nums)):
    for j in range(i + 1, len(nums)):
        s = nums[i] + nums[j]
        if lower <= s <= upper:
            count += 1
return count

```

PHASE 3 — OPTIMIZE

```

# "Sort nums. For each i, binary search for the range of valid j > i:
# lower_bound(nums, lower-nums[i], i+1) → first j where nums[j] >= lower-nums[i].
# upper_bound(nums, upper-nums[i], i+1) → first j where nums[j] > upper-nums[i].
# Count = upper_bound - lower_bound. Sum over all i.
# Time: O(n log n). Space: O(1) extra.
# Use inner helpers for the two binary search variants."

```

PHASE 4 — CLEAN CODE

```

class _2563_FairPairs:
    def countFairPairs(self, nums: List[int], lower: int, upper: int) -> int:
        def lower_bound(start: int, target: int) -> int:
            left, right = start, len(nums)
            while left < right:
                mid = (left + right) // 2
                if nums[mid] >= target:
                    right = mid
                else:
                    left = mid + 1
            return left

        def upper_bound(start: int, target: int) -> int:
            left, right = start, len(nums)
            while left < right:
                mid = (left + right) // 2
                if nums[mid] > target:
                    right = mid
                else:
                    left = mid + 1
            return left

        nums.sort()
        count = 0
        for i in range(len(nums)):
            lo = lower_bound(i + 1, lower - nums[i])
            hi = upper_bound(i + 1, upper - nums[i])
            count += hi - lo
        return count

```

PHASE 5 — TEST & DEBUG

```

# nums=[0,1,4,4,5,7] (sorted), lower=3, upper=6:
# i=0(0): need j where 3<=nums[j]<=6. lower_bound(1,3)→idx of 4=2.
upper_bound(1,7)→idx of 7=5. count+=3.
# i=1(1): need 2<=nums[j]<=5. lb(2,2)→2. ub(2,6)→5. count+=3. total=6.
# ... → 6 ✓

```


PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log n)$ – sort + n binary searches each $O(\log n)$.

Space $O(1)$ extra.

lower_bound finds first index $\geq$ target; upper_bound finds first index $>$ target.

Their difference is the count of indices in the valid range.

Alternative: two-pointer approach – for each i , maintain lo and hi pointers

across iterations (they only move right) $\rightarrow O(n)$ total after sorting.

Given more time I'd ask: what if we want the actual pairs?

Collect them during the binary search loops – $O(n + \text{output})$ time."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def countFairPairs(self, nums: list[int], lower: int, upper: int) -> int:
        # nums[i] + nums[j] == nums[j] + nums[i], so the condition that i < j
        # degrades to i != j and we can sort the array.
        nums.sort()

        def countLess(summ: int) -> int:
            res = 0
            i = 0
            j = len(nums) - 1
            while i < j:
                while i < j and nums[i] + nums[j] > summ:
                    j -= 1
                res += j - i
                i += 1
            return res

        return countLess(upper) - countLess(lower - 1)
```

• LeetDoocs

```
class Solution:
    def countFairPairs(self, nums: List[int], lower: int, upper: int) -> int:
        nums.sort()
        ans = 0
        for i, x in enumerate(nums):
            j = bisect_left(nums, lower - x, lo=i + 1)
            k = bisect_left(nums, upper - x + 1, lo=i + 1)
            ans += k - j
        return ans
```

• SimplyLeet

```
import bisect

def countFairPairs(nums, lower, upper):
    # Sort the array to allow binary search
    nums.sort()
    fair_pairs = 0
    n = len(nums)

    for i in range(n):
        # Find lower bound index for the required complement so that nums[i] +
        # nums[j] >= lower
        left = bisect.bisect_left(nums, lower - nums[i], i + 1, n)
        # Find upper bound index for the required complement so that nums[i] +
        # nums[j] <= upper
        right = bisect.bisect_right(nums, upper - nums[i], i + 1, n)
        fair_pairs += (right - left)

    return fair_pairs

# Example usage:
print(countFairPairs([0,1,7,4,4,5], 3, 6)) # Expected output: 6
```

• LC.ca

```
class Solution:
    def countFairPairs(self, nums: List[int], lower: int, upper: int) -> int:
        nums.sort()
        ans = 0
        for i, x in enumerate(nums):
            j = bisect_left(nums, lower - x, lo=i + 1)
            k = bisect_left(nums, upper - x + 1, lo=i + 1)
            ans += k - j
        return ans
```

◆ Quick Review

Key Insights

- Sorting the array simplifies searching for pairs with required sum limits.
- For each element, binary search is used to locate:
 - The first index where the complement makes the sum $\geq$ lower.
 - The last index where the complement makes the sum $\leq$ upper.
- This avoids the need for a nested loop through the entire array, yielding a more efficient solution.

Space & Time

Time Complexity: $O(n \log n)$, primarily due to sorting and (for each element) binary search operations.

Space Complexity: $O(n)$ if sorting creates a copy; $O(1)$ additional space if sorting in-place.

Solution

The solution starts by sorting the input array. For each element at index i in the sorted array, we search for indices j (where $j > i$) such that the sum of $\text{nums}[i]$ and $\text{nums}[j]$ falls between lower and upper. Two binary search operations are performed: To find the smallest index j where $\text{nums}[i] + \text{nums}[j] \geq \text{lower}$. To find the largest index j where $\text{nums}[i] + \text{nums}[j] \leq \text{upper}$. Subtracting the two positions (and summing the counts for each i) gives the total number of fair pairs. Edge cases (like no valid pair) are automatically handled by the binary search approach.

Sliding Window

Round 2 · High · Medium · 8 questions

Sliding Window

□ #3 Longest Substring Without Repeating Characters

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: Grind 169

Problem

Given a string *s*, find the length of the longest substring without duplicate characters.

Example 1:

Input: *s* = "abcabcbb"

Output: 3

Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.

Example 2:

Input: *s* = "bbbbbb"

Output: 1

Explanation: The answer is "b", with the length of 1.

Example 3:

Input: *s* = "pwwkew"

Output: 3

Explanation: The answer is "wke", with the length of 3.

Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

Constraints:

- $0 \leq s.length \leq 5 * 10^4$
- s consists of English letters, digits, symbols and spaces.

My LeetCode Solution

• My LeetCode Solution

```
# Longest Substring Without Repeating Characters
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        l, ans = 0, 0
        hmap = defaultdict(int)
        for r in range(len(s)):
            rchar = s[r]
            hmap[rchar] += 1
            while hmap[rchar] > 1:
                lchar = s[l]
                hmap[lchar] -= 1
                l += 1
            ans = max(ans, r - l + 1)
        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Find the length of the longest substring with all unique characters.
# Substring is contiguous. Empty string returns 0. Right?
# Characters can be any printable ASCII – not just lowercase letters."
#
# "'abcabcbb'→3 ('abc'). 'bbbbbb'→1 ('b'). 'pwwkew'→3 ('wke') ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Enumerate every substring with two indices left and right.
# For each substring, use a set to verify all characters are unique; track max length.
# Correct, but  $O(n^2)$  substrings each with up to  $O(n)$  uniqueness check."
```



```
# Time:  $O(n^2)$  to  $O(n^3)$ . Space:  $O(\min(n, \text{alphabet}))$ .
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Sliding window with a frequency map.
# Expand right: add s[right] to the map.
# When a duplicate appears (count > 1), shrink from the left until it's gone.
# At each step, update the max window size.
# Time:  $O(n)$ . Space:  $O(\min(n, \text{alphabet}))$  – at most 128 unique ASCII chars."
```

PHASE 4 — CLEAN CODE

```
class _3_LongestSubstringNoRepeat:
    def lengthOfLongestSubstring(self, s: str) -> int:
        freq : defaultdict = defaultdict(int)
        left = 0
        longest = 0
        for right in range(len(s)):
            freq[s[right]] += 1
            while freq[s[right]] > 1: # shrink until no duplicate
                freq[s[left]] -= 1
                left += 1
            longest = max(longest, right - left + 1)
        return longest
```

PHASE 5 — TEST & DEBUG

```
# s='abcabcbb':
# r=0('a'): freq={a:1}. longest=1.
# r=1('b'): freq={a:1,b:1}. longest=2.
# r=2('c'): freq={a:1,b:1,c:1}. longest=3.
# r=3('a'): freq[a]=2. shrink: freq[a]-=1-1, left=1. longest=3 (3-1+1=3).
# r=4('b'): freq[b]=2. shrink: freq[b]-=1-1, left=2. longest=3.
# r=5('c'): freq[c]=2. shrink: left=3. longest=3.
# r=6('b'): freq[b]=2. shrink: left=4. longest=3.
# r=7('b'): freq[b]=2. shrink: left=5. longest=max(3,3)=3. → 3 ✓
# s='': loop doesn't run → 0 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$  – each character enters and leaves the window at most once.
# Space  $O(\min(n, 128))$  for the frequency map.
# Alternative: store the LAST INDEX of each character instead of frequency.
# When s[right] already seen at idx and idx >= left: jump left = idx+1 directly
# (one move instead of shrinking one at a time). Same  $O(n)$  but fewer iterations.
# Given more time I'd ask: what if we want the actual substring, not just its
length?
# Track left and best_left when updating the max."
```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        ans = 0
        count = collections.Counter()

        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            while count[c] > 1:
                count[s[l]] -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans

```

```

class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        ans = 0
        # The substring s[j + 1..i] has no repeating characters.
        j = -1
        # lastSeen[c] := the index of the last time c appeared
        lastSeen = {}

        for i, c in enumerate(s):
            # Update j to lastSeen[c], so the window must start from j + 1.

            j = max(j, lastSeen.get(c, -1))
            ans = max(ans, i - j)
            lastSeen[c] = i

        return ans

```

• LeetDoocs

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        cnt = Counter()
        ans = l = 0
        for r, c in enumerate(s):
            cnt[c] += 1
            while cnt[c] > 1:
                cnt[s[l]] -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans
```

• SimplyLeet

```
# Function to find the length of the longest substring without repeating
characters
def length_of_longest_substring(s):
    # Dictionary to store the most recent index of each character
    char_index = {}
    # Initialize starting index of current window and max length
    left = 0
    max_length = 0

    # Iterate over the string with the right pointer
    for right, char in enumerate(s):
        # If char is in dictionary and its index is within the current window
        if char in char_index and char_index[char] >= left:
            # Move the left pointer to one position after the last occurrence
            # of char
            left = char_index[char] + 1
        # Update the latest index of the char
        char_index[char] = right
        # Update max_length with the current window size if larger
        max_length = max(max_length, right - left + 1)

    return max_length

# Example usage:
# print(length_of_longest_substring("abcabcbb")) # Expected output: 3
```

• LC.ca

```
class Solution:
    def lengthOfLongestSubstring(self, s): # map for index
        d = {} # value => its index
        i = 0
        ans = 0
        for j, c in enumerate(s):
            if c in d:
                # must max check i, example "abba"
                i = max(i, d[c] + 1)
            d[c] = j

        ans = max(ans, j - i + 1)
        return ans
```

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:

        if not s:
            return 0
```

```
l = 0 # left
r = 0 # right

result = 0
isFoundInWindow = [False] * 256

while r < len(s):

    while isFoundInWindow[ord(s[r])]:
        isFoundInWindow[ord(s[l])] = False
```

```

        l += 1

        isFoundInWindow[ord(s[r])] = True

        # check before shrink
        result = max(result, r - l + 1)

        r += 1

    return result

```

```

class Solution: # extra space for set()
    def lengthOfLongestSubstring(self, s: str) -> int:

        d = collections.defaultdict(int)
        start = 0
        ans = 0
        for i,c in enumerate(s):
            # shrink
            while d[c] > 0:

```

```

                d[s[start]] -= 1 # not s[start], not start
                start += 1
            ans = max(ans, i - start + 1)
            d[c] = 1
        return ans

```

```
#####
```

```

class Solution(object):
    def _lengthOfLongestSubstring(self, s): # no extra data structure

```

```

        """
        :type s: str
        :rtype: int
        """

        d = collections.defaultdict(int)
        l = ans = 0
        for i, c in enumerate(s):
            while l > 0 and d[c] > 0:
                d[s[i - l]] -= 1
                l -= 1

```

```
d[c] += 1
l += 1
ans = max(ans, l)
return ans
```

```
#####
```

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        ss = set()
```

```
        i = ans = 0
        for j, c in enumerate(s):
            while c in ss:
                ss.remove(s[i])
                i += 1
            ss.add(c)
            ans = max(ans, j - i + 1)
        return ans
```

◆ Quick Review

Key Insights

- Use the sliding window technique to maintain a window of unique characters.
- Use a hash table (or dictionary) to quickly check for duplicate characters and update window boundaries.
- The window is expanded by moving a pointer (right) and contracted by moving another pointer (left) when duplicates are encountered.

- Time complexity can be maintained at $O(n)$ by ensuring each character is processed a limited number of times.

Space & Time

Time Complexity: $O(n)$, where n is the length of the input string.

Space Complexity: $O(\min(n, m))$, where m is the size of the character set (constant for fixed character sets).

Solution

The solution uses a sliding window technique with two pointers, left and right, to keep track of the current substring without repeating characters. A hash table (dictionary or map) is used to store the most recent index of each character encountered. As the right pointer moves forward, if a duplicate character is found within the current window, the left pointer is advanced to the position right after where the duplicate was last seen. This ensures that the window always contains unique characters. The maximum length is updated throughout the process.

Sliding Window

★ #159 Longest Substring with At Most Two Distinct Characters ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: ★ Premium | Premium 98

Problem

Given a string *s*, return the length of the longest substring that contains at most two distinct characters.

Example 1:

Input: *s* = "eceba"
Output: 3
Explanation: The substring is "ece" which its length is 3.

Example 2:

Input: *s* = "ccaabbb"
Output: 5
Explanation: The substring is "aabbb" which its length is 5.

Constraints:

- $1 \leq s.length \leq 105$
- *s* consists of English letters.

My LeetCode Solution

• My LeetCode Solution

```
# Longest Substring with At Most Two Distinct Characters
class Solution:
    def lengthOfLongestSubstringTwoDistinct(self, s: str) -> int:
        hmap = Counter()
        l = 0
        ans = 0

        for r in range(len(s)):
            rchar = s[r]
            hmap[rchar] += 1

            while len(hmap) > 2:
                lchar = s[l]
                hmap[lchar] -= 1
                if hmap[lchar] == 0:
                    del hmap[lchar]
                l += 1
            ans = max(ans, r - l + 1)

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the longest substring that contains at most 2 distinct characters. Right?
 # 'eceba': 'ece' has 2 distinct → length 3 ✓. 'ccaabbb': 'aabbb' has 2 → length 5 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with the brute force to lock in the logic.
 # For every pair (i, j), count the distinct characters in s[i..j] using a set.
 # If there are at most 2 distinct characters, update the max length.
 # Time: $O(n^2)$ pairs, $O(n)$ set check = $O(n^3)$. With running set it's $O(n^2)$. Space: $O(1)$.
 # Should I go ahead and optimize?"

```
class _159_LongestSubstringTwoDistinct_Brute:
    def lengthOfLongestSubstringTwoDistinct(self, s):
        longest = 0
        for i in range(len(s)):
            seen = set()
            for j in range(i, len(s)):
                seen.add(s[j])
                if len(seen) <= 2:
                    longest = max(longest, j - i + 1)
```

```

        else:
            break
    return longest

```

PHASE 3 — OPTIMIZE

"Sliding window – same pattern as #3 but the shrink condition is 'more than 2 distinct chars'.

Expand right: add s[right] to frequency map.

Shrink from left while distinct chars > 2 (i.e. len(freq) > 2).

Update max window length. Time: O(n). Space: O(1) – at most 3 keys in the map."

PHASE 4 — CLEAN CODE

```

class _159_LongestSubstringTwoDistinct:
    def lengthOfLongestSubstringTwoDistinct(self, s: str) -> int:
        freq : defaultdict = defaultdict(int)
        left = 0
        longest = 0
        for right in range(len(s)):
            freq[s[right]] += 1
            while len(freq) > 2: # shrink until at most 2 distinct
                freq[s[left]] -= 1
                if freq[s[left]] == 0:
                    del freq[s[left]]
                left += 1
            longest = max(longest, right - left + 1)
        return longest

```

PHASE 5 — TEST & DEBUG

```

# s='eceba':
# r=0('e'): freq={e:1}. longest=1.
# r=1('c'): freq={e:1,c:1}. longest=2.
# r=2('e'): freq={e:2,c:1}. longest=3.
# r=3('b'): freq={e:2,c:1,b:1} → 3 distinct. shrink:
# freq[e]-=1→1. left=1. freq={e:1,c:1,b:1} still 3. shrink:
# freq[c]-=1→0 → del c. left=2. freq={e:1,b:1}. 2 distinct → stop.
longest=max(3,2)=3.
# r=4('a'): freq={e:1,b:1,a:1} → 3. shrink: freq[e]-=1→0→del e. left=3.
freq={b:1,a:1}. longest=3.
# → 3 ✓
# s='ccaabbbb':
# Window expands to 'ccaabbbb' but once 'b' is added after 'a', we have c,a,b(3
distinct).
# Shrink past the c's. Eventually best window is 'aabbbb'=5. → 5 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time O(n). Space O(1) – at most 3 keys in the map at any time.

This is the template for 'at most k distinct characters' – change > 2 to > k.

Generalisation: Longest Substring with At Most K Distinct Characters (#340) is the same

algorithm with k replacing 2.

The key difference from #3: shrink on `len(freq) > k` rather than on a specific duplicate.
 # Given more time I'd ask: what about EXACTLY 2 distinct characters?
 # `atMost(k=2) - atMost(k=1)` gives the count of substrings with exactly 2 distinct."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def lengthOfLongestSubstringTwoDistinct(self, s: str) -> int:
        ans = 0
        distinct = 0
        count = [0] * 128

        l = 0
        for r, c in enumerate(s):
            count[ord(c)] += 1
            if count[ord(c)] == 1:
                distinct += 1
            while distinct == 3:
                count[ord(s[l])] -= 1
                if count[ord(s[l])] == 0:
                    distinct -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans
```

• LeetDoocs

```
class Solution:
    def lengthOfLongestSubstringTwoDistinct(self, s: str) -> int:
        cnt = Counter()
        ans = j = 0
        for i, c in enumerate(s):
            cnt[c] += 1
            while len(cnt) > 2:
                cnt[s[j]] -= 1
                if cnt[s[j]] == 0:
                    cnt.pop(s[j])
                j += 1
            ans = max(ans, i - j + 1)
```

```

        j += 1
    ans = max(ans, i - j + 1)
    return ans

```

• SimplyLeet

Python solution for Longest Substring with At Most Two Distinct Characters

class Solution:

def lengthOfLongestSubstringTwoDistinct(self, s: str) -> int:

left = 0 # Initialize left pointer of the window

max_length = 0 # Variable to store the maximum window size found

char_count = {} # Dictionary to count characters in the current window

for right, char in enumerate(s):

Add the current character to the window (increase count)

char_count[char] = char_count.get(char, 0) + 1

If there are more than two distinct characters, shrink the window from the left

while len(char_count) > 2:

left_char = s[left]

char_count[left_char] -= 1

if char_count[left_char] == 0:

del char_count[left_char]

left += 1

Update max_length if current window is larger

max_length = max(max_length, right - left + 1)

return max_length

• LC.ca

```
'''
>>> from collections import Counter
>>> c = Counter()
>>> c
Counter()
>>> c['a'] += 1
>>> c
Counter({'a': 1})
```

```
>>> c
Counter({'a': 1, 'c': 1, 'b': 1})
>>> c.pop(2)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 2
>>>
>>> c.pop('c')
1
>>> c
```

```
Counter({'a': 1, 'b': 1})
'''
```

```
from collections import Counter
```

```
class Solution:
```

```
    def lengthOfLongestSubstringTwoDistinct(self, s: str) -> int:
        cnt = Counter() # or, cnt = defaultdict(int)
        ans = i = 0
        for j, c in enumerate(s):
```

```
            cnt[c] += 1
            while len(cnt) > 2: # shrink
                cnt[s[i]] -= 1
                if cnt[s[i]] == 0:
                    cnt.pop(s[i])
                i += 1
            ans = max(ans, j - i + 1)
        return ans
```

◆ Quick Review

Key Insights

- Use a sliding window to maintain a contiguous substring.
- Keep track of character frequencies within the current window using a hash map or dictionary.
- When the window contains more than two distinct characters, shrink the window from the left until only two remain.
- Update the maximum length each time the window satisfies the condition.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string, since each character is processed at most twice.

Space Complexity: $O(1)$ because the hash map holds at most 3 entries (usually up to 2 distinct characters).

Solution

We use the sliding window technique with two pointers (left and right) to represent the current window. A hash table/dictionary

records the count of each character inside the window. As we extend the right pointer, we update the count. If the hash table contains more than two distinct characters, we start moving the left pointer to reduce the number of distinct characters until we have at most two distinct keys. The length of the current valid window is then used to update our maximum length answer.

Sliding Window

□ ★ #340 Longest Substring with At Most K Distinct Characters ★ **MED**

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: ★ Premium | Premium 98

Problem

Given a string s and an integer k , return the length of the longest substring of s that contains at most k distinct characters.

Example 1:

Input: $s = \text{"eceba"}, k = 2$
Output: 3
Explanation: The substring is "ece" with length 3.

Example 2:

Input: $s = \text{"aa"}, k = 1$
Output: 2
Explanation: The substring is "aa" with length 2.

Constraints:

- $1 \leq s.length \leq 5 * 10^4$
- $0 \leq k \leq 50$

My LeetCode Solution

• My LeetCode Solution

```
# Longest Substring with At Most K Distinct Characters
class Solution:
    def lengthOfLongestSubstringKDistinct(self, s: str, k: int) -> int:
        l = ans = 0
        hmap = Counter()
        for r in range(len(s)):
            rchar = s[r]
            hmap[rchar] += 1
            while len(hmap) > k:
                lchar = s[l]
                hmap[lchar] -= 1
                if hmap[lchar] == 0:
                    del hmap[lchar]
                l += 1
            ans = max(ans, r - l + 1)
        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Return the length of the longest substring with at most k distinct characters.
# k=0 means the answer is 0 (no characters allowed). Right?
# This is the direct generalisation of #159 (at most 2 distinct) with k replacing 2."
#
# "'eceba', k=2 -> 'ece', length 3 ✓. 'aa', k=1 -> 'aa', length 2 ✓."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# For every starting index i, expand j rightward, track distinct characters with a set.
# Stop when distinct chars exceed k. Track the maximum window length seen.
# Time: O(n^2). Space: O(k). Should I go ahead and optimize?"
```

```
class _340_LongestSubstringKDistinct_Brute:
    def lengthOfLongestSubstringKDistinct(self, s, k):
        if k == 0:
            return 0
        longest = 0
        for i in range(len(s)):
            seen = set()
```

```

        for j in range(i, len(s)):
            seen.add(s[j])
            if len(seen) <= k:
                longest = max(longest, j - i + 1)
            else:
                break
    return longest

```

PHASE 3 — OPTIMIZE

"Sliding window – identical template to #159 with the constant 2 replaced by k.
 # Expand right, shrink from left whenever distinct chars exceed k.
 # Time: $O(n)$. Space: $O(k)$ – at most $k+1$ keys in the map at any moment."

PHASE 4 — CLEAN CODE

```

class _340_LongestSubstringKDistinct:
    def lengthOfLongestSubstringKDistinct(self, s: str, k: int) -> int:
        if k == 0:
            return 0
        freq : defaultdict = defaultdict(int)
        left = 0
        longest = 0
        for right in range(len(s)):
            freq[s[right]] += 1
            while len(freq) > k:
                freq[s[left]] -= 1
                if freq[s[left]] == 0:
                    del freq[s[left]]
                left += 1
            longest = max(longest, right - left + 1)
        return longest

```

PHASE 5 — TEST & DEBUG

s='eceba', k=2:
 # r=2('e'): freq={e:2,c:1}. 2 distinct → ok. longest=3.
 # r=3('b'): 3 distinct → shrink until freq={e:1,b:1}. longest stays 3.
 # r=4('a'): 3 distinct → shrink until freq={b:1,a:1}. longest=3. → 3 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(k)$. Identical to #159 – just k instead of 2.
 # Exactly k distinct: atMost(k) – atMost(k-1).
 # Given more time I'd ask: what if $k \geq$ distinct chars in s? Answer = len(s)."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def lengthOfLongestSubstringKDistinct(self, s: str, k: int) -> int:
        ans = 0
        distinct = 0
        count = collections.Counter()

        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            if count[c] == 1:
                distinct += 1
            while distinct == k + 1:
                count[s[l]] -= 1
                if count[s[l]] == 0:
                    distinct -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans

```

• LeetDoocs

```

class Solution:
    def lengthOfLongestSubstringKDistinct(self, s: str, k: int) -> int:
        l = 0
        cnt = Counter()
        for c in s:
            cnt[c] += 1
            if len(cnt) > k:
                cnt[s[l]] -= 1
                if cnt[s[l]] == 0:
                    del cnt[s[l]]
                l += 1
        return len(s) - l

```

• SimplyLeet

```
# Python solution for Longest Substring with At Most K Distinct Characters
```

```
def longest_substring_with_k_distinct(s: str, k: int) -> int:
    # Handle edge case: if k is 0, no valid substring exists
    if k == 0:
        return 0

    # Dictionary to store frequency of characters in the current window
    char_count = {}
    left = 0 # Left pointer for the window

    max_length = 0 # Variable to track the maximum length found

    # Iterate through the string using the right pointer
    for right in range(len(s)):
        # Add/update the count of the current character in the dictionary
        current_char = s[right]
        char_count[current_char] = char_count.get(current_char, 0) + 1

        # If window contains more than k distinct characters, shrink from the
        left

        while len(char_count) > k:
            left_char = s[left]
            char_count[left_char] -= 1
            # Remove the character count if it becomes 0
            if char_count[left_char] == 0:
                del char_count[left_char]
            left += 1

        # Update the maximum length of the valid window found so far
        max_length = max(max_length, right - left + 1)

    return max_length

# Example usage:
# s = "eceba", k = 2
# print(longest_substring_with_k_distinct(s, 2)) # Output: 3
```

• LC.ca

```
class Solution:
    def lengthOfLongestSubstringTwoDistinct(self, s: str) -> int:
        cnt = Counter() # or, cnt = defaultdict(int)
        ans = i = 0
        for j, c in enumerate(s):
            cnt[c] += 1
            while len(cnt) > k:
                cnt[s[i]] -= 1
                if cnt[s[i]] == 0:
                    cnt.pop(s[i])
                i += 1
            ans = max(ans, j - i + 1)
        return ans
```

◆ Quick Review

Key Insights

- Use the sliding window technique to efficiently scan through the string.
- Maintain a hash table (or dictionary) that keeps track of the frequency of characters in the current window.
- When the number of distinct characters exceeds k , increment the left pointer to shrink the window until the condition is met.
- Update the maximum length of valid substrings throughout the process.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string, as each character is processed at most twice (once when expanding the window and once when contracting it).

Space Complexity: $O(\min(m, k))$, where m is the size of the charset (or number of unique characters in s), since at most $k+1$ keys are stored.

Solution

This problem is solved using the sliding window technique. The algorithm keeps track of a window defined by two indices (left and right pointers). A hash map is used to count the frequency of each character in the current window. As the right pointer extends the window, the count of the character is incremented. When the count of distinct keys in the map exceeds k , the left pointer is moved forward while decrementing the count of the leftmost character, until the condition is restored. The maximum window length encountered is recorded and returned as the result.

Sliding Window

□ #424 Longest Repeating Character Replacement

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: Grind 169

Problem

You are given a string s and an integer k . You can choose any character of the string and change it to any other uppercase English character. You can perform this operation at most k times.

Return the length of the longest substring containing the same letter you can get after performing the above operations.

Example 1:

Input: $s = \text{"ABAB"} , k = 2$

Output: 4

Explanation: Replace the two 'A's with two 'B's or vice versa.

Example 2:

Input: s = "AABABBA", k = 1

Output: 4

Explanation: Replace the one 'A' in the middle with 'B' and form "AABBBBA".

The substring "BBBB" has the longest repeating letters, which is 4.

There may exists other ways to achieve this answer too.

Constraints:

- $1 \leq s.length \leq 105$
- s consists of only uppercase English letters.
- $0 \leq k \leq s.length$

My LeetCode Solution

• My LeetCode Solution

```
# Longest Repeating Character Replacement
class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        ans = 0
        maxCount = 0
        count = collections.Counter()

        l = 0
        for r, c in enumerate(s):
            count[c] += 1

            maxCount = max(maxCount, count[c])
            while maxCount + k < r - l + 1:
                count[s[l]] -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"We can replace at most k characters. Find the longest substring where all characters

```
# are the same after at most k replacements.
# String is uppercase letters only. Right?
# Strategy: keep the most frequent char, replace everything else."
#
# "'ABAB', k=2 → replace both A's or B's → 4 ✓
# 'AABABBA', k=1 → best window = 4 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with the brute force to lock in the logic.
# For every pair (i, j), count character frequencies in s[i..j]. Find the most
frequent char.
# If window_size - max_freq <= k, at most k replacements are needed – this window is
valid.
# Track the maximum valid window length.
# Time: O(n2) pairs, O(26) frequency count per window. Space: O(1). Should I go
ahead and optimize?"
```

```
class _424_LongestRepeatingReplacement_Brute:
    def characterReplacement(self, s, k):
        longest = 0
        for i in range(len(s)):
            freq = [0] * 26
            for j in range(i, len(s)):
                freq[ord(s[j]) - ord('A')] += 1
                max_freq = max(freq)
                if (j - i + 1) - max_freq <= k:
                    longest = max(longest, j - i + 1)
        return longest
```

PHASE 3 — OPTIMIZE

```
# "Sliding window tracking the most frequent character in the window.
# Valid window: window_size - max_freq <= k (replace the rest).
# When invalid, shrink left by 1 – but NEVER decrease max_freq.
# We only care about finding a LONGER valid window, so we only update max_freq
upward.
# This means the window never shrinks below the current best – it slides as a fixed
size
# until a better window is found. Time: O(n). Space: O(26)=O(1)."
```

PHASE 4 — CLEAN CODE

```
class _424_LongestRepeatingReplacement:
    def characterReplacement(self, s: str, k: int) -> int:
        freq = [0] * 26
        max_freq = 0
        left = 0
        longest = 0
        for right in range(len(s)):
            freq[ord(s[right]) - ord('A')] += 1
            max_freq = max(max_freq, freq[ord(s[right]) - ord('A')])
            if (right - left + 1) - max_freq > k: # too many replacements needed
                freq[ord(s[left]) - ord('A')] -= 1
                left += 1
```

```

        left += 1
        longest = max(longest, right - left + 1)
    return longest

```

PHASE 5 — TEST & DEBUG

```

# s='ABAB', k=2:
# r=0('A'): freq[A]=1,max=1. win=1. 1-1=0<=2. longest=1.
# r=1('B'): freq[B]=1,max=1. win=2. 2-1=1<=2. longest=2.
# r=2('A'): freq[A]=2,max=2. win=3. 3-2=1<=2. longest=3.
# r=3('B'): freq[B]=2,max=2. win=4. 4-2=2<=2. longest=4. → 4 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n). Space O(1).
# 'Don't decrease max_freq' is the key insight: we never shrink below the best
window seen.
# The window slides forward at a fixed size until max_freq improves, then grows.
# Given more time I'd ask: unlimited replacements (k=n)? Answer = len(s)."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        ans = 0
        maxCount = 0
        count = collections.Counter()

        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            maxCount = max(maxCount, count[c])

            while maxCount + k < r - l + 1:
                count[s[l]] -= 1
                l += 1
            ans = max(ans, r - l + 1)

        return ans

```

```
class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        maxCount = 0
        count = collections.Counter()

        # l and r track the maximum window instead of the valid window.
        l = 0
        for r, c in enumerate(s):
            count[c] += 1
            maxCount = max(maxCount, count[c])

            while maxCount + k < r - l + 1:
                count[s[l]] -= 1
                l += 1

        return r - l + 1
```

• LeetDoocs

```
class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        cnt = Counter()
        l = mx = 0
        for r, c in enumerate(s):
            cnt[c] += 1
            mx = max(mx, cnt[c])
            if r - l + 1 - mx > k:
                cnt[s[l]] -= 1
                l += 1

        return len(s) - l
```

• SimplyLeet

```

# Python solution using the sliding window technique
def characterReplacement(s: str, k: int) -> int:
    # Dictionary to store frequency of characters in the current window
    char_counts = {}
    max_freq = 0 # Track the maximum frequency of a single character in the
    window
    max_length = 0
    left = 0 # Left pointer of the sliding window

    # Expand the sliding window with the right pointer
    for right in range(len(s)):

        # Count the current character
        char_counts[s[right]] = char_counts.get(s[right], 0) + 1
        # Update the max frequency in the current window
        max_freq = max(max_freq, char_counts[s[right]])

        # Check if the number of replacements needed exceeds k
        # (current window size) - (count of most frequent character) > k
        if (right - left + 1) - max_freq > k:
            # If condition fails, shrink the window by moving left pointer
            char_counts[s[left]] -= 1

            left += 1

        # Update the maximum length found so far
        max_length = max(max_length, right - left + 1)

    return max_length

# Example usage:
# print(characterReplacement("AABABBA", 1)) # Expected output: 4

```

• LC.ca

```
class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        counter = [0] * 26
        i = j = maxCnt = 0
        while i < len(s):
            counter[ord(s[i]) - ord('A')] += 1
            maxCnt = max(maxCnt, counter[ord(s[i]) - ord('A')])
            if i - j + 1 > maxCnt + k:
                counter[ord(s[j]) - ord('A')] -= 1
                j += 1
            i += 1
        return i - j
```

◆ Quick Review

Key Insights

- Use a Sliding Window algorithm to keep track of a contiguous substring.
- Maintain a frequency count of letters in the current window.
- Keep track of the count of the most frequent character in the window.
- If the current window size minus the maximum frequency is greater than k, shrink the window.
- The maximum window size seen during the process is the answer.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string, since the window traverses the string in linear time.

Space Complexity: $O(1)$ because the frequency count for letters (A–Z) has a constant size.

Solution

The problem is solved using the sliding window technique. We initialize two pointers representing the left and right bounds of the current window. As we expand the window by moving the right pointer, we update the frequency count of the letter at the right pointer and track the letter that appears most frequently. The key observation is that the number of characters that need to be replaced in the current window is the window's size minus the count of the most frequently occurring letter. If this number exceeds k , we increment the left pointer to shrink the window until the condition is satisfied again. The maximum size of the window during the process gives the answer.

Sliding Window

□ #438 Find All Anagrams in a String

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: Grind 169

Problem

Given two strings s and p , return an array of all the start indices of p 's anagrams in s . You may return the answer in any order.

Example 1:

Input: $s = \text{"cbaebabacd"}, p = \text{"abc"}$

Output: $[0,6]$

Explanation:

The substring with start index = 0 is "cba", which is an anagram of "abc".

The substring with start index = 6 is "bac", which is an anagram of "abc".

Example 2:

Input: $s = \text{"abab"}, p = \text{"ab"}$

Output: $[0,1,2]$

Explanation:

The substring with start index = 0 is "ab", which is an anagram of "ab".

The substring with start index = 1 is "ba", which is an anagram of "ab".

The substring with start index = 2 is "ab", which is an anagram of "ab".

Constraints:

- $1 \leq s.length, p.length \leq 3 * 10^4$

- s and p consist of lowercase English letters.

My LeetCode Solution

• My LeetCode Solution

```
# Find All Anagrams in a String
from collections import Counter

class Solution:
    def findAnagrams(self, s: str, p: str):
        pcount = Counter(p)
        scout = Counter(s[:len(p)])

        ans = []
        for r, rchar in enumerate(s[len(p):], start=len(p)):

            scout[rchar] += 1
            lchar = s[r-len(p)]
            scout[lchar] -= 1
            if scout[lchar] == 0:
                del scout[lchar]

            if scout == pcount:
                l = r - len(p) + 1
                ans.append(l)

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Return all start indices in s where a substring is an anagram of p.
# Fixed window size = len(p). Lowercase letters only. Right?"
#
# "s='cbaebabacd', p='abc' → [0,6] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Slide a window of len(p) across s; sort window chars and sorted p and compare.
# Every window costs O(m log m) for sorting."
```

```
# Frequency-count comparison per window is O(26) instead.
# Time: O(n * m log m). Space: O(m) sort buffers.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Fixed sliding window of size len(p) with frequency counters.
# Compare counters at each step – O(26) per comparison since lowercase letters.
# Time: O(n). Space: O(1) – fixed alphabet."
```

PHASE 4 — CLEAN CODE

```
class _438_FindAnagrams:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        if len(p) > len(s):
            return []
        p_count = Counter(p)
        w_count = Counter(s[:len(p)])
        result = [0] if w_count == p_count else []
        for i in range(len(p), len(s)):
            w_count[s[i]] += 1
            left = s[i - len(p)]
            w_count[left] -= 1
            if w_count[left] == 0:
                del w_count[left]
            if w_count == p_count:
                result.append(i - len(p) + 1)
        return result
```

PHASE 5 — TEST & DEBUG

```
# s='cbaebabacd', p='abc': p_count={c:1,b:1,a:1}.
# Init w='cba'={c:1,b:1,a:1}==p→result=[0].
# Slide until i=8('c'): w_count={b:1,a:1,c:1}==p→result=[0,6]. → [0,6] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(1). Counter comparison is O(26)=O(1) for fixed alphabet.
# Alternative: track a 'matches' counter instead of full compare – same O(n).
# Given more time I'd ask: what if p can contain any Unicode?
# Same algorithm – Counter handles any hashable."
```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def findAnagrams(self, s: str, p: str) -> list[int]:
        ans = []
        count = collections.Counter(p)
        required = len(p)

        for r, c in enumerate(s):
            count[c] -= 1
            if count[c] >= 0:
                required -= 1

            if r >= len(p):
                count[s[r - len(p)]] += 1
                if count[s[r - len(p)]] > 0:
                    required += 1
            if required == 0:
                ans.append(r - len(p) + 1)

        return ans

```

• LeetDoocs

```

class Solution:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        m, n = len(s), len(p)
        ans = []
        if m < n:
            return ans
        cnt1 = Counter(p)
        cnt2 = Counter(s[: n - 1])
        for i in range(n - 1, m):
            cnt2[s[i]] += 1

            if cnt1 == cnt2:
                ans.append(i - n + 1)
            cnt2[s[i - n + 1]] -= 1
        return ans

```

• SimplyLeet

```

# Define the function to find all anagram indices in the string
def findAnagrams(s: str, p: str):
    # Import Counter for frequency counting
    from collections import Counter
    # List to store the starting indices of all anagrams
    result = []
    # Length of string p
    p_length = len(p)
    # Frequency count for characters in p
    p_count = Counter(p)

    # Frequency count for the current window in s
    s_count = Counter()

    # Iterate over the string s
    for i in range(len(s)):
        # Add current character to the window count
        s_count[s[i]] += 1

        # Remove the leftmost character from window if window size exceeds
        # p_length
        if i >= p_length:
            if s_count[s[i - p_length]] == 1:
                del s_count[s[i - p_length]] # Remove key completely if count
            becomes 0
            else:
                s_count[s[i - p_length]] -= 1

        # Compare window count with p_count when window size is equal to
        # p_length
        if s_count == p_count:
            result.append(i - p_length + 1)

    return result

# Example usage:
print(findAnagrams("cbaebabacd", "abc")) # Expected output: [0, 6]

```

 • LC.ca

```
class Solution:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        m, n = len(s), len(p)
        ans = []
        if m < n:
            return ans
        cnt1 = Counter(p)
        cnt2 = Counter(s[: n - 1])
        for i in range(n - 1, m):
            cnt2[s[i]] += 1

            if cnt1 == cnt2:
                ans.append(i - n + 1)
            cnt2[s[i - n + 1]] -= 1
        return ans
```

◆ Quick Review

Key Insights

- Use a sliding window of length equal to p over s .
- Maintain frequency counts of characters in p and within the current window in s .
- Compare frequency counts to determine if the current window is an anagram.
- Update the window efficiently by adding one character and removing the character that goes out of the window.

Space & Time

Time Complexity: $O(n)$ where n is the length of s .

Space Complexity: $O(1)$ since we use fixed-size frequency arrays or hash maps.

Solution

We use the sliding window technique to iterate over s with a window size equal to the length of p . First, count the frequencies of characters in p . Then, iterate over s and update a frequency count for the current window. When the window size equals the length of p , compare the two frequency counts. If they match, record the start index. To slide the window efficiently, increment the count for the incoming character and decrement the count for the outgoing character. This way, we achieve an $O(n)$ solution that is optimal for the problem constraints.

Sliding Window

★ #487 Max Consecutive Ones II ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Sliding Window
Lists: ★ Premium | Premium 98

Problem

Given a binary array `nums`, return the maximum number of consecutive 1's in the array if you can flip at most one 0.

Example 1:

Input: `nums = [1,0,1,1,0]`

Output: 4

Explanation:

– If we flip the first zero, `nums` becomes `[1,1,1,1,0]` and we have 4 consecutive ones.

– If we flip the second zero, `nums` becomes `[1,0,1,1,1]` and we have 3 consecutive ones.

The max number of consecutive ones is 4.

Example 2:

Input: `nums = [1,0,1,1,0,1]`

Output: 4

Explanation:

– If we flip the first zero, `nums` becomes `[1,1,1,1,0,1]` and we have 4 consecutive ones.

– If we flip the second zero, `nums` becomes `[1,0,1,1,1,1]` and we have 4 consecutive ones.

The max number of consecutive ones is 4.

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $\text{nums}[i]$ is either 0 or 1.

Follow up: What if the input numbers come in one by one as an infinite stream? In other words, you can't store all numbers coming from the stream as it's too large to hold in memory. Could you solve it efficiently?

My LeetCode Solution

• My LeetCode Solution

```
# Max Consecutive Ones II
class Solution:
    def findMaxConsecutiveOnes(self, nums: List[int]) -> int:
        hmap=0
        l=0
        ans = 0

        for r in range(len(nums)):
            rnum = nums[r]
            if rnum == 0:

                hmap += 1
                while hmap > 1:
                    lnum = nums[l]
                    if lnum == 0:
                        hmap -= 1
                    l+=1
                ans = max(ans, r-l+1)

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Binary array. Flip at most one 0 to 1. Return the longest consecutive 1s run. Right?

Follow-up: infinite stream – can't store all numbers."

#

"[1,0,1,1,0] → flip first 0 → 4 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Try every substring with two nested indices left and right.

Count zeros inside; if zeros ≤ 1, track the longest valid window.

Correct, but checking all windows is $O(n^2)$.

Time: $O(n^2)$. Space: $O(1)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Sliding window allowing at most one 0.

Expand right. When zeros > 1, shrink left until zeros ≤ 1.

Time: $O(n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```
class _487_MaxConsecutiveOnesII:
    def findMaxConsecutiveOnes(self, nums: List[int]) -> int:
        zeros = 0
        left = 0
        longest = 0
        for right in range(len(nums)):
            if nums[right] == 0:
                zeros += 1
            while zeros > 1:
                if nums[left] == 0:
                    zeros -= 1
                left += 1
            longest = max(longest, right - left + 1)
        return longest
```

PHASE 5 — TEST & DEBUG

[1,0,1,1,0]: zeros hits 2 at r=4 → shrink past first 0 → window=[1,1,0], longest=4 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(1)$.

Stream follow-up: keep a deque of the last two 0-positions.

When third 0 arrives, left = oldest_zero_pos + 1 – $O(1)$ space regardless of stream length.

Generalises to at-most-k flips (Max Consecutive Ones III #1004): same window, k zeros allowed.

Name that connection unprompted."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findMaxConsecutiveOnes(self, nums: list[int]) -> int:
        ans = 0
        zeros = 0

        l = 0
        for r, num in enumerate(nums):
            if num == 0:
                zeros += 1
                while zeros == 2:
                    if nums[l] == 0:
                        zeros -= 1
                    l += 1
            ans = max(ans, r - l + 1)

        return ans
```

```
class Solution:
    def findMaxConsecutiveOnes(self, nums: list[int]) -> int:
        maxZeros = 1
        ans = 0
        q = collections.deque() # Store indices of zero.

        l = 0
        for r, num in enumerate(nums):
            if num == 0:
                q.append(r)

            if len(q) > maxZeros:
                l = q.popleft() + 1
            ans = max(ans, r - l + 1)

        return ans
```

```

class Solution:
    def findMaxConsecutiveOnes(self, nums: list[int]) -> int:
        ans = 0
        lastZeroIndex = -1

        l = 0
        for r, num in enumerate(nums):
            if num == 0:
                l = lastZeroIndex + 1
                lastZeroIndex = r

        ans = max(ans, r - l + 1)

        return ans

```

• LeetCode

```

class Solution:
    def findMaxConsecutiveOnes(self, nums: List[int]) -> int:
        l = cnt = 0
        for x in nums:
            cnt += x ^ 1
            if cnt > 1:
                cnt -= nums[l] ^ 1
                l += 1
        return len(nums) - l

```

• SimplyLeet

```

# Function to find the maximum consecutive ones with at most one zero flip
def findMaxConsecutiveOnes(nums):
    max_length = 0 # Tracks the maximum length of the window
    left = 0 # Left pointer for the sliding window
    zero_count = 0 # Counter for zeros in the current window

    # Iterate with right pointer over the entire array
    for right in range(len(nums)):
        # If current element is zero, increase the zero_count
        if nums[right] == 0:

```

```

        zero_count += 1

    # If window invalid (more than one zero), shrink from left
    while zero_count > 1:
        if nums[left] == 0:
            zero_count -= 1
        left += 1

    # Update max_length if current window is larger
    max_length = max(max_length, right - left + 1)

return max_length

# Example usage:
# nums = [1,0,1,1,0]
# print(findMaxConsecutiveOnes(nums)) # Expected output: 4

```

• LC.ca

```

from collections import deque

class Solution:
    def findMaxConsecutiveOnes(self, nums: List[int]) -> int:

        res, zero, left, k = 0, 0, 0, 1

        for right in range(len(nums)):
            if nums[right] == 0:
                zero += 1

            while zero > k:
                if nums[left] == 0:
                    zero -= 1
                left += 1
            res = max(res, right - left + 1) # max-check every for iteration

        return res

class Solution: # followup, extra space
    def findMaxConsecutiveOnes(self, nums: List[int]) -> int:

```



```

res, left, k = 0, 0, 1
q = deque()

for right in range(len(nums)):
    if nums[right] == 0:
        q.append(right)
    if len(q) > k:
        left = q.popleft() + 1
    res = max(res, right - left + 1) # max-check every for iteration

```

```

return res

```

```

#####

```

```

# maintain a sliding window of size 2, that keeps track of the counts.

```

```

class Solution: # follow up, optimized

```

```

    def findMaxConsecutiveOnes(self, nums: List[int]) -> int:

```

```

        count = 0 # Count of consecutive ones

```

```

        prev_count = 0 # Count of consecutive ones before a zero

```

```

max_count = 0 # Maximum count of consecutive ones

```

```

for num in nums:

```

```

    if num == 1:

```

```

        count += 1

```

```

        prev_count += 1

```

```

    else:

```

```

        count = prev_count + 1

```

```

        prev_count = 0

```

```

max_count = max(max_count, count)

```

```

# Reset the count and prev_count if we encounter two zeros in a row

```

```

if num == 0 and prev_count == 0:

```

```

    count = 0

```

```

    prev_count = 0

```

```

return max_count

```

```

#####

```

```

class Solution:
    def findMaxConsecutiveOnes(self, nums: List[int]) -> int:
        l = r = 0
        k = 1
        while r < len(nums):
            if nums[r] == 0:
                k -= 1
            if k < 0:
                if nums[l] == 0:
                    k += 1
                    l += 1
                r += 1
            return r - l

```

```
#####
```

```

class Solution(object):
    def __init__(self):
        self.ans = 0

```

```

        self.count = 0
        self.lastCount = 0

```

```

def findMaxConsecutiveOnes(self, nums):
    """
    :type nums: List[int]
    :rtype: int
    """
    for num in nums:
        self.readNum(num) # stream the input

```

```

    return self.ans

```

```

def readNum(self, num):
    """
    :type nums: int
    """
    if num == 1:
        self.count += 1
    else:
        self.count = self.count - self.lastCount + 1

```

```
self.lastCount = self.count  
self.ans = max(self.ans, self.count)
```

◆ Quick Review

Key Insights

- Utilize a sliding window to maintain the current segment.
- The window can contain at most one 0.
- Expand the window with the right pointer and contract it with the left pointer when the constraint is violated.
- The maximum window size observed during this process is the desired result.

Space & Time

Time Complexity: $O(n)$ where n is the number of elements in the array.

Space Complexity: $O(1)$.

Solution

We use a sliding window approach to efficiently find the longest contiguous subarray that contains at most one 0. Two pointers (left and right) define the window. As we iterate with the right pointer, counting

zeros in the window, we adjust the left pointer when the count of zeros exceeds one. The length of the valid window gives us the number of consecutive 1's (considering one flipped 0), and we update the maximum length accordingly.

Sliding Window

□ ★ #1100 Find K-Length Substrings with No Repeated Characters ★

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: ★ Premium | Premium 98

Problem

Given a string s and an integer k , return the number of substrings in s of length k with no repeated characters.

Example 1:

Input: $s = \text{"havefunonleetcode"}, k = 5$
Output: 6
Explanation: There are 6 substrings they are:
'havef', 'avefu', 'vefun', 'efuno', 'etcod', 'tcode'.

Example 2:

Input: $s = \text{"home"}, k = 5$
Output: 0
Explanation: Notice k can be larger than the length of s . In this case, it is not possible to find any substring.

Constraints:

- $1 \leq s.length \leq 104$
- s consists of lowercase English letters.

- $1 \leq k \leq 104$

My LeetCode Solution

• My LeetCode Solution

```
# Find K-Length Substrings with No Repeated Characters
class Solution:
    def numKLenSubstrNoRepeats(self, s: str, k: int) -> int:
        ans = 0
        hmap = Counter()

        for r in range(len(s)):
            rchar = s[r]
            hmap[rchar] += 1
            l = r-k

            if l >= 0:
                lchar = s[l]
                hmap[lchar] -= 1
                if hmap[lchar] == 0:
                    del hmap[lchar]
            if r-k+1 >= 0 and len(hmap) == k:
                ans += 1

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count substrings of exactly length k with all distinct characters. Right?

$k > \text{len}(s) \rightarrow 0$. $k=0 \rightarrow 0$."

#

"s='havefunonleetcode', k=5 $\rightarrow$ 6 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For each start index i, build substring $s[i:i+k]$ and check distinct chars with a set.

Increment count when set size equals k.

$O(n * k)$ windows; sliding window with frequency map is $O(n)$.

Time: $O(n * k)$. Space: $O(k)$ set per window.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Fixed sliding window of size k. Slide: add right, remove left-behind char.

Valid when len(freq)==k (all chars distinct). Time: O(n). Space: O(k)."

PHASE 4 — CLEAN CODE

```
class _1100_KLengthSubstrings:
    def numKLenSubstrNoRepeats(self, s: str, k: int) -> int:
        if k > len(s):
            return 0
        freq : defaultdict = defaultdict(int)
        count = 0
        for right in range(len(s)):
            freq[s[right]] += 1
            if right >= k:
                left = right - k
                freq[s[left]] -= 1
                if freq[s[left]] == 0:
                    del freq[s[left]]
            if right >= k - 1 and len(freq) == k:
                count += 1
        return count
```

PHASE 5 — TEST & DEBUG

s='home', k=5: 5>4 → 0 ✓

First valid window r=k-1: if all k chars distinct, count++. Slide forward.

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time O(n). Space O(k). len(freq)==k is the clean validity check.

Fixed window variant of #3 – useful when window size is given, not maximised.

Given more time I'd ask: return the actual substrings? Collect s[right-k+1:right+1]."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def numKLenSubstrNoRepeats(self, s: str, k: int) -> int:
        ans = 0
        unique = 0
        count = collections.Counter()

        for i, c in enumerate(s):
            count[c] += 1
            if count[c] == 1:
                unique += 1
```



```

    if i >= k:
        count[s[i - k]] -= 1
        if count[s[i - k]] == 0:
            unique -= 1
        if unique == k:
            ans += 1

    return ans

```

• LeetDoocs

```

class Solution:
    def numKLenSubstrNoRepeats(self, s: str, k: int) -> int:
        cnt = Counter(s[:k])
        ans = int(len(cnt) == k)
        for i in range(k, len(s)):
            cnt[s[i]] += 1
            cnt[s[i - k]] -= 1
            if cnt[s[i - k]] == 0:
                cnt.pop(s[i - k])
            ans += int(len(cnt) == k)

        return ans

```

• SimplyLeet

```

def numKLenSubstrNoRepeats(s: str, k: int) -> int:
    # If k is larger than the string length, no valid substring exists.
    if k > len(s):
        return 0

    count = 0
    left = 0
    # Dictionary to store the frequency of characters in the current window.
    char_count = {}

```

```

# Iterate over the string using right pointer.
for right in range(len(s)):
    # Update frequency count for the current character.
    char_count[s[right]] = char_count.get(s[right], 0) + 1

    # If current character frequency > 1, slide window from the left to
    remove duplicates.
    while char_count[s[right]] > 1:
        char_count[s[left]] -= 1
        left += 1

    # When window size equals k, we have a valid substring.
    if right - left + 1 == k:
        count += 1
        # Slide the window by removing the leftmost character.
        char_count[s[left]] -= 1
        left += 1

return count

# Example usage:
# print(numKLenSubstrNoRepeats("havefunonleetcode", 5)) -> 6

```

• LC.ca

```

class Solution:
    def numKLenSubstrNoRepeats(self, s: str, k: int) -> int:
        n = len(s)
        if k > n or k > 26:
            return 0
        ans = j = 0
        cnt = Counter()
        for i, c in enumerate(s):
            cnt[c] += 1
            while cnt[c] > 1 or i - j + 1 > k:
                cnt[s[j]] -= 1
                j += 1
            ans += i - j + 1 == k
        return ans

```

◆ Quick Review

Key Insights

- Use a sliding window of fixed size k .
- Track character counts within the current window using a hash table (or frequency array).
- When a duplicate is found in the window, slide the window from the left to remove duplicates.
- Count a valid window when its size equals k and all characters are unique.
- Optimize by adjusting the window immediately after counting a valid substring.

Space & Time

Time Complexity: $O(n)$ where n is the length of s , as each character is visited at most twice.

Space Complexity: $O(1)$ since the frequency tracking is over a fixed set (26 lowercase letters).

Solution

We use a sliding window approach with two pointers: left and right. A hash table (or frequency array) tracks the occurrences of

characters in the window. As we move the right pointer, we update the frequency and check for duplicates. If a duplicate is encountered, we increment the left pointer until the duplicate is removed. Once the window size exactly equals k , we have a valid substring, so we increment our count and then slide the window forward. This ensures that every possible k -length substring is examined once.

Sliding Window

□ #1248 Count Number of Nice Subarrays

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Math · Sliding Window ·
Prefix Sum

Lists: CodeSignal

Problem

Given an array of integers `nums` and an integer `k`. A continuous subarray is called nice if there are `k` odd numbers on it.

Return the number of nice sub-arrays.

Example 1:

Input: `nums = [1,1,2,1,1]`, `k = 3`

Output: 2

Explanation: The only sub-arrays with 3 odd numbers are `[1,1,2,1]` and `[1,2,1,1]`.

Example 2:

Input: `nums = [2,4,6]`, `k = 1`

Output: 0

Explanation: There are no odd numbers in the array.

Example 3:

Input: `nums = [2,2,2,1,2,2,1,2,2,2]`, `k = 2`

Output: 16

Constraints:

- $1 \leq \text{nums.length} \leq 50000$
- $1 \leq \text{nums}[i] \leq 10^5$
- $1 \leq k \leq \text{nums.length}$

My LeetCode Solution

• My LeetCode Solution

```
# Count Number of Nice Subarrays
class Solution:
    def numberOfSubarrays(self, nums: List[int], k: int) -> int:
        hmap = Counter({0:1})
        s,ans = 0,0

        for x in nums:
            s+= x%2
            ans += hmap[s-k]
            hmap[s] +=1

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Count subarrays with EXACTLY k odd numbers. Return the count. Right?"
# Map each element to 1 (odd) or 0 (even) → reduce to Subarray Sum Equals K (#560)."
#
# "[1,1,2,1,1], k=3 → 2 ✓. [2,2,2,1,2,2,1,2,2,2], k=2 → 16 ✓."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic."
# Enumerate every subarray with two loops; count how many have an odd number of 1s.
# Straightforward prefix/suffix parity check per window.
# Correct, but  $O(n^2)$  subarrays is too slow at  $n = 5 \times 10^4$ .
# Time:  $O(n^2)$ . Space:  $O(1)$ .
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Prefix sum of odd indicators + hash map. Same pattern as #560.
# seed Counter at {0:1}. For each element, add num%2 to running sum.
# Count += prefix_count[sum - k]. Time: O(n). Space: O(n)."
```

PHASE 4 — CLEAN CODE

```
class _1248_NiceSubarrays:
    def numberOfSubarrays(self, nums: List[int], k: int) -> int:
        prefix_count = Counter({0: 1})
        odd_sum = 0
        count = 0
        for num in nums:
            odd_sum += num % 2
            count += prefix_count[odd_sum - k]
            prefix_count[odd_sum] += 1
        return count
```

PHASE 5 — TEST & DEBUG

```
# [1,1,2,1,1], k=3:
# sums: 1,2,2,3,4. At sum=3: want=0→prefix[0]=1→count=1.
# At sum=4: want=1→prefix[1]=1→count=2. → 2 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(n). Identical to #560 – name the connection explicitly.
# At-most-k version: sliding window counting odds in window, shrink when > k.
# Exactly k = atMost(k) – atMost(k-1).
# Given more time I'd ask: count subarrays with at most k odd numbers?
# Sliding window – shrink when odd count exceeds k, add right-left+1 each step."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def numberOfSubarrays(self, nums: list[int], k: int) -> int:
        def numberOfSubarraysAtMost(k: int) -> int:
            ans = 0
            l = 0
            r = 0

            while r <= len(nums):
                if k >= 0:
                    ans += r - l
```



```

        if r == len(nums):
            break
        if nums[r] & 1:
            k -= 1
            r += 1
        else:
            if nums[l] & 1:
                k += 1
                l += 1
            return ans

```

```

return numberOfSubarraysAtMost(k) - numberOfSubarraysAtMost(k - 1)

```

• LeetCode

```

class Solution:
    def numberOfSubarrays(self, nums: List[int], k: int) -> int:
        cnt = Counter({0: 1})
        ans = t = 0
        for v in nums:
            t += v & 1
            ans += cnt[t - k]
            cnt[t] += 1
        return ans

```

• SimplyLeet

Python solution with line-by-line comments

```

def numberOfSubarrays(nums, k):
    # Initialize dictionary to store frequency of prefix sums
    prefix_count = {0: 1} # There is one way to have zero odd numbers
    initially.
    count = 0 # This will store the final count of nice subarrays
    odd_sum = 0 # Prefix sum representing the count of odd numbers so far

    # Iterate through each number in the array
    for num in nums:

```

```

# If the number is odd, increment the odd_count prefix sum
if num % 2 == 1:
    odd_sum += 1

# If there is a prefix such that (current odd_sum - prefix) equals k,
# then we found subarrays ending here with k odd numbers
count += prefix_count.get(odd_sum - k, 0)

# Update the frequency for the current odd_sum in the hash table
prefix_count[odd_sum] = prefix_count.get(odd_sum, 0) + 1

# Return the total number of nice subarrays
return count

# Example usage:
# nums = [1,1,2,1,1]
# k = 3
# print(numberOfSubarrays(nums, k)) # Expected output is 2

```

• LC.ca

```

class Solution:
    def numberOfSubarrays(self, nums: List[int], k: int) -> int:
        cnt = Counter({0: 1})
        ans = t = 0
        for v in nums:
            t += v & 1
            ans += cnt[t - k]
            cnt[t] += 1
        return ans

```

◆ Quick Review

Key Insights

- Convert the problem to counting subarrays with a specific "prefix sum" value where the sum represents the count of odd numbers.

- Use a hash table (or dictionary) to store the frequency of prefix sums encountered.
- The number of nice subarrays ending at the current index equals the frequency of (current prefix sum – k) seen so far.
- This approach allows you to find the answer in one pass over the array.

Space & Time

Time Complexity: $O(n)$, where n is the number of elements in `nums`, because we traverse the array once.

Space Complexity: $O(n)$, in the worst case where each prefix sum is unique and stored in the hash table.

Solution

We solve the problem using a prefix sum approach. First, we iterate through the array and convert it into a prefix sum representing the cumulative count of odd numbers encountered. As we compute the prefix sums, we use a hash table to record how many times each prefix sum appears. For every current prefix sum value, we check how many times we have seen a prefix sum equal to (current

prefix sum – k). The idea is that if $\text{prefix}[j] - \text{prefix}[i]$ equals k, then between indices $i+1$ and j , there are exactly k odd numbers. This gives us the count of valid subarrays ending at the current index. This method works efficiently as it requires only a single pass through the array and maintains a running frequency count using the hash table.

Sorting

Round 2 · High · Medium · 3 questions

Sorting

□ #49 Group Anagrams

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Sorting
Lists: Grind 169

Problem

Given an array of strings `strs`, group the anagrams together. You can return the answer in any order.

Example 1:

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`

Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

Explanation:

- There is no string in `strs` that can be rearranged to form "bat".
- The strings "nat" and "tan" are anagrams as they can be rearranged to form each other.
- The strings "ate", "eat", and "tea" are anagrams as they can be rearranged to form each other.

Example 2:

Input: strs = [""]

Output: [[""]]

Example 3:

Input: strs = ["a"]

Output: [["a"]]

Constraints:

- $1 \leq \text{strs.length} \leq 104$
- $0 \leq \text{strs}[i].\text{length} \leq 100$
- `strs[i]` consists of lowercase English letters.

My LeetCode Solution

• My LeetCode Solution

```
# Group Anagrams
class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        hmap = defaultdict(list)
        for s in strs:
            hmap[tuple(sorted(s))].append(s)

        return list(hmap.values())
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Group strings that are anagrams. Return groups in any order.

Same chars, same frequencies → anagrams. Empty string is its own group. Right?"

#

["eat", "tea", "tan", "ate", "nat", "bat"] → 3 groups ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.


```
# For every pair of strings, sort both and check if the sorted forms match.
# If they match, put them in the same group; merge groups as we go.
# Correct, but comparing all pairs is expensive when n is large.
# Time:  $O(n^2 * m \log m)$ . Space:  $O(n * m)$ .
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Use a canonical key: sorted(word) as a tuple → same key for all anagrams.
# Group into a defaultdict. Time:  $O(n * m \log m)$ . Space:  $O(n * m)$ .
# Alternative: 26-element frequency tuple →  $O(n * m)$ , no sort cost."
```

PHASE 4 — CLEAN CODE

```
class _49_GroupAnagrams:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        groups: defaultdict = defaultdict(list)
        for word in strs:
            groups[tuple(sorted(word))].append(word)
        return list(groups.values())
```

PHASE 5 — TEST & DEBUG

```
# 'eat'→('a','e','t'). 'tea'→same. 'ate'→same. All go to one group.
# 'tan'→('a','n','t'). 'nat'→same.
# 'bat'→('a','b','t'). Alone. → 3 groups ✓
# [''] → {():['']}. → [['']] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n * m \log m)$ . Space  $O(n * m)$ .
# Frequency tuple alternative:  $O(n * m)$ , avoids  $\log m$  sort. For short words
# (typical)
# sorting is fine; for very long words, frequency tuple wins.
# Given more time I'd ask: Unicode input? sorted() handles it; frequency array
# breaks.
# Use Counter-based key: tuple(sorted(Counter(word).items()))."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def groupAnagrams(self, strs: list[str]) -> list[list[str]]:
        dict = collections.defaultdict(list)

        for str in strs:
            key = ''.join(sorted(str))
            dict[key].append(str)

        return dict.values()
```

• LeetCode

```
class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        d = defaultdict(list)
        for s in strs:
            k = ''.join(sorted(s))
            d[k].append(s)
        return list(d.values())
```

• SimplyLeet

```
# Python solution for grouping anagrams
from collections import defaultdict

def groupAnagrams(strs):
    # Create a dictionary to hold lists of anagrams; default to an empty list
    # for new keys
    anagrams = defaultdict(list)
    # Iterate over each string in the input list
    for s in strs:
        # Sort the string to use as a key; sorted returns a list so join back
        # to string
        key = ''.join(sorted(s))

        # Append the original string under the sorted key
        anagrams[key].append(s)
    # Return the grouped anagrams as a list of lists
    return list(anagrams.values())

# Example usage:
if __name__ == "__main__":
    strs = ["eat", "tea", "tan", "ate", "nat", "bat"]
    result = groupAnagrams(strs)
    print(result)
```

• LC.ca

```

'''
>>> from collections import defaultdict
>>> d = defaultdict(list)
>>> d["a"]=1
>>> d["b"]=2
>>> d
defaultdict(<class 'list'>, {'a': 1, 'b': 2})
>>> d.values()
dict_values([1, 2])
>>> list(d.values())

```

```
[1, 2]
```

```

### sorted(str) will return a list of chars
>>> a = "sdfddxyz"
>>> sorted(a)
['d', 'd', 'd', 'f', 's', 'x', 'y', 'z']
>>> "".join(sorted(a))
'dddfsxyz'
'''

```

```

class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        d = defaultdict(list)
        for s in strs:
            k = "".join(sorted(s))
            d[k].append(s)
        return list(d.values())

```

◆ Quick Review

Key Insights

- Anagrams have the same characters when sorted alphabetically.
- Sorting each string provides a unique key for grouping.

- Use a hash table (dictionary) to map each sorted key to a list of its original strings.
- Finally, gather all the lists from the dictionary to form the result.

Space & Time

Time Complexity: $O(n * k \log k)$, where n is the number of strings and k is the maximum length of a string (due to sorting each string).

Space Complexity: $O(n * k)$ for storing the grouped strings in the hash table.

Solution

The primary idea is to iterate through each string, sort it to obtain a key that represents its character composition, and then group the original string under this key in a hash table. After processing all strings, the values of the hash table will be the groups of anagrams. This approach efficiently groups anagrams by leveraging sorting and a dictionary for constant-time lookups.

Sorting

□ #56 Merge Intervals

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sorting
Lists: Grind 169

Problem

Given an array of intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, merge all overlapping intervals, and return an array of the non-overlapping intervals that cover all the intervals in the input.

Example 1:

Input: `intervals = [[1,3],[2,6],[8,10],[15,18]]`
Output: `[[1,6],[8,10],[15,18]]`
Explanation: Since intervals `[1,3]` and `[2,6]` overlap, merge them into `[1,6]`.

Example 2:

Input: `intervals = [[1,4],[4,5]]`
Output: `[[1,5]]`
Explanation: Intervals `[1,4]` and `[4,5]` are considered overlapping.

Example 3:

Input: `intervals = [[4,7],[1,4]]`
Output: `[[1,7]]`
Explanation: Intervals `[1,4]` and `[4,7]` are considered overlapping.

Constraints:

- $1 \leq \text{intervals.length} \leq 104$
- $\text{intervals}[i].\text{length} == 2$
- $0 \leq \text{start}_i \leq \text{end}_i \leq 104$

My LeetCode Solution

• My LeetCode Solution

```
# Merge Intervals
class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:

        intervals.sort()
        ans = [intervals[0]]

        for i in range(1, len(intervals)):
            last = ans[-1]
            if last[1] >= intervals[i][0]:

                last[1] = max(last[1], intervals[i][1])

            else:
                ans.append(intervals[i])

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Merge all overlapping intervals. Two intervals overlap if one starts before or
when the other ends.
# [1,4] and [4,5] overlap at 4. Sort by start, then merge. Return non-overlapping
result. Right?"
#
# "[[1,3],[2,6],[8,10],[15,18]] → [[1,6],[8,10],[15,18]] ✓ (1-3 and 2-6 overlap at
2-3)"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Compare every pair of intervals; if they overlap, merge into one wider interval.
# Repeat until a full pass finds no new merges."
```

```
# Correct, but repeated pairwise merging can degrade toward  $O(n^2)$ .
# Time:  $O(n^2)$  worst case. Space:  $O(n)$  output.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Sort by start time —  $O(n \log n)$ . Then one pass:
# Keep a result list. For each interval, if it overlaps with the last result
interval
# (last.end >= current.start), extend the last interval's end.
# Otherwise, append as a new separate interval.
# Time:  $O(n \log n)$ . Space:  $O(n)$ ."
```

PHASE 4 — CLEAN CODE

```
class _56_MergeIntervals:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        intervals.sort()
        merged = [intervals[0]]
        for start, end in intervals[1:]:
            if merged[-1][1] >= start: # overlap
                merged[-1][1] = max(merged[-1][1], end)
            else:
                merged.append([start, end])
        return merged
```

PHASE 5 — TEST & DEBUG

```
# [[1,3],[2,6],[8,10],[15,18]]: sorted same.
# merged=[[1,3]]. [2,6]: 3>=2→extend→[1,6]. [8,10]: 6<8→append. [15,18]:
10<15→append.
# → [[1,6],[8,10],[15,18]] ✓
# [[1,4],[4,5]]: 4>=4→[1,5] ✓
# [[4,7],[1,4]]: sorted→[[1,4],[4,7]]. 4>=4→[1,7] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n \log n)$ . Space  $O(n)$ .
# The sort is the bottleneck — the merge pass itself is  $O(n)$ .
# If the input were already sorted, this runs in  $O(n)$  total.
# Given more time I'd ask: how many meeting rooms needed?
# That's Meeting Rooms II (#253) — use a min-heap of end times."
```

Community Solutions (Python)

• WalkCC


```

class Solution:
    def merge(self, intervals: list[list[int]]) -> list[list[int]]:
        ans = []

        for interval in sorted(intervals):
            if not ans or ans[-1][1] < interval[0]:
                ans.append(interval)
            else:
                ans[-1][1] = max(ans[-1][1], interval[1])

        return ans

```

• LeetDoocs

```

class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        intervals.sort()
        ans = []
        st, ed = intervals[0]
        for s, e in intervals[1:]:
            if ed < s:
                ans.append([st, ed])
                st, ed = s, e
            else:
                ed = max(ed, e)
        ans.append([st, ed])
        return ans

```

• SimplyLeet

```

def merge(intervals):
    # Edge case: if the list is empty, return an empty list
    if not intervals:
        return []

    # Sort intervals based on the starting time
    intervals.sort(key=lambda x: x[0])
    merged = [] # List to store merged intervals

    # Iterate over each interval in the sorted list
    for interval in intervals:

```

```

    # If merged list is not empty and the current interval
    # overlaps with the last one in merged, merge them
    if merged and interval[0] <= merged[-1][1]:
        # Update the end of the last interval in merged if needed
        merged[-1][1] = max(merged[-1][1], interval[1])
    else:
        # No overlap, so add the current interval to merged
        merged.append(interval)
    return merged

```

```

# Example usage:
print(merge([[1,3],[2,6],[8,10],[15,18]]))

```

• LC.ca

```

# Definition for an interval.
# class Interval(object):
# def __init__(self, s=0, e=0):
# self.start = s
# self.end = e

class Solution(object):
    def merge(self, intervals):
        """
        :type intervals: List[Interval]

        :rtype: List[Interval]
        """
        ans = []
        for intv in sorted(intervals, key=lambda x: x.start):
            if ans and ans[-1].end >= intv.start:
                ans[-1].end = max(ans[-1].end, intv.end)
            else:
                ans.append(intv)
        return ans

```

```
#####

'''
>>> intervals = [[111,222],[1,3],[2,6],[8,10],[15,18]]
>>> intervals.sort()
>>> intervals
[[1, 3], [2, 6], [8, 10], [15, 18], [111, 222]]
'''

class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:

        intervals.sort() # default sort also ok
        ans = [intervals[0]]
        for s, e in intervals[1:]:
            if ans[-1][1] < s:
                ans.append([s, e])
            else:
                ans[-1][1] = max(ans[-1][1], e)
        return ans
```

◆ Quick Review

Key Insights

- Sort the intervals based on their start values.
- Use a pointer or iteration to compare the current interval with the last merged interval.
- If the current interval overlaps with the previous one, merge them by updating the end time.
- Otherwise, add the current interval to the results as a new merged interval.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting the intervals.

Space Complexity: $O(n)$ to store the merged intervals.

Solution

First, sort the array of intervals based on the starting time. Then, iterate through the sorted list and check if the current interval overlaps with the last interval in the merged list. If an overlap is detected (i.e., the start of the current interval is less than or equal to the end of the last merged interval), update the last merged interval's end to be the maximum of both ends. If there is no overlap, simply add the current interval to the merged list. This approach efficiently merges overlapping intervals and ensures that all intervals are processed in a single pass after sorting.

Sorting

□ ★ #1152 Analyze User Website Visit Pattern ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Sorting
Lists: ★ Premium | Premium 98

Problem

You are given two string arrays `username` and `website` and an integer array `timestamp`. All the given arrays are of the same length and the tuple `[username[i], website[i], timestamp[i]]` indicates that the user `username[i]` visited the website `website[i]` at time `timestamp[i]`.

A pattern is a list of three websites (not necessarily distinct).

- For example, `["home", "away", "love"]`, `["leetcode", "love", "leetcode"]`, and `["luffy", "luffy", "luffy"]` are all patterns.

The score of a pattern is the number of users that visited all the websites in the pattern in the same order they appeared in the pattern.

- For example, if the pattern is ["home", "away", "love"], the score is the number of users x such that x visited "home" then visited "away" and visited "love" after that.
- Similarly, if the pattern is ["leetcode", "love", "leetcode"], the score is the number of users x such that x visited "leetcode" then visited "love" and visited "leetcode" one more time after that.
- Also, if the pattern is ["luffy", "luffy", "luffy"], the score is the number of users x such that x visited "luffy" three different times at different timestamps.

Return the pattern with the largest score. If there is more than one pattern with the same largest score, return the lexicographically smallest such pattern.

Note that the websites in a pattern do not need to be visited contiguously, they only need to be visited in the order they appeared in the pattern.
Example 1:

```

Input: username =
["joe","joe","joe","james","james","james","james","mary","mary","mary"],
timestamp = [1,2,3,4,5,6,7,8,9,10], website =
["home","about","career","home","cart","maps","home","home","about","career"]
Output: ["home","about","career"]
Explanation: The tuples in this example are:
["joe","home",1],["joe","about",2],["joe","career",3],["james","home",4],["james","cart",5],["james","maps",6],["james","home",7],["mary","home",8],["mary","about",9], and ["mary","career",10].
The pattern ("home", "about", "career") has score 2 (joe and mary).
The pattern ("home", "cart", "maps") has score 1 (james).
The pattern ("home", "cart", "home") has score 1 (james).
The pattern ("home", "maps", "home") has score 1 (james).

```

Example 2:

```

Input: username = ["ua","ua","ua","ub","ub","ub"], timestamp = [1,2,3,4,5,6],
website = ["a","b","a","a","b","c"]
Output: ["a","b","a"]

```

Constraints:

- $3 \leq \text{username.length} \leq 50$
- $1 \leq \text{username}[i].\text{length} \leq 10$
- $\text{timestamp.length} == \text{username.length}$
- $1 \leq \text{timestamp}[i] \leq 109$
- $\text{website.length} == \text{username.length}$
- $1 \leq \text{website}[i].\text{length} \leq 10$
- $\text{username}[i]$ and $\text{website}[i]$ consist of lowercase English letters.
- It is guaranteed that there is at least one user who visited at least three websites.
- All the tuples $[\text{username}[i], \text{timestamp}[i], \text{website}[i]]$ are unique.

My LeetCode Solution

• My LeetCode Solution

```
# Analyze User Website Visit Pattern
class Solution:
    def mostVisitedPattern(self, username: List[str], timestamp: List[int], website: List[str]) -> List[str]:
        usersites = defaultdict(list)
        for u,t,w in sorted(zip(username, timestamp, website)):
            usersites[u].append(w)

        patterns = Counter()
        for sites in usersites.values():
            patterns.update(set(combinations(sites, 3)))

        return min(patterns, key = lambda x : (-patterns[x], x))
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Find the 3-website pattern (subsequence, not necessarily consecutive) that the
most DISTINCT users
# have visited in order. On tie, return lexicographically smallest. Right?
# Each user contributes at most 1 count per pattern regardless of how many times
they visit."
#
# "joe visited home→about→career, james visited home→cart→maps→home, mary visited
home→about→career.
# Pattern ('home','about','career') score=2 (joe, mary) → winner ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# For each user, enumerate all 3-website combinations in visit order.
# Build pattern tuple (a,b,c) and increment Counter across users.
# O(visit_length^3) per user – acceptable only for small lists.
# Time: O(U * V^3). Space: O(patterns).
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Sort all visits by timestamp. Group by user into ordered website lists.
# For each user, generate all C(len,3) 3-subsequence combos – at most C(50,3)=19600.
# Use a set per user to avoid double-counting (one user = one score per pattern).
# Count across users with a global Counter.
```

Return pattern with max score, lexicographically smallest on ties."

PHASE 4 — CLEAN CODE

```
class _1152_AnalyzeWebsiteVisit:
    def mostVisitedPattern(self, username: List[str], timestamp: List[int],
website: List[str]) -> List[str]:
    visits = sorted(zip(timestamp, username, website))
    user_sites: defaultdict = defaultdict(list)
    for _, user, site in visits:
        user_sites[user].append(site)
    pattern_score: Counter = Counter()
    for sites in user_sites.values():
        seen = set()
        for combo in itertools.combinations(sites, 3):
            if combo not in seen:
                seen.add(combo)
                pattern_score[combo] += 1
    return list(max(pattern_score, key=lambda p: (pattern_score[p], [-ord(c)
for c in ''.join(p)])))
```

PHASE 5 — TEST & DEBUG

After sorting by timestamp, user_sites = {joe:[home,about,career],
james:[home,car,t,maps,home], mary:[home,about,career]}.
joe: combos of [home,about,career] = {(home,about,career)}.
james: C(4,3)=4 combos. All unique.
mary: {(home,about,career)}.
pattern_score[(home,about,career)]=2 → winner ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log n + U * C(K,3))$ where U =users, K =max visits per user.
With $U \leq 50$, $K \leq 50$: $C(50,3)=19600 * 50 = 980000$ – manageable.
The per-user 'seen' set ensures one user contributes at most 1 to each pattern's score.
Tie-breaking: max() with a compound key – score ascending, then lexicographic ascending.
Given more time I'd ask: what if we want top-k patterns?
Use a heap instead of max() – same 0 complexity."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def mostVisitedPattern(
        self,
        username: list[str],
        timestamp: list[int],
        website: list[str],
    ) -> list[str]:
        userToSites = collections.defaultdict(list)

        # Sort websites of each user by timestamp.

        for user, _, site in sorted(
            zip(username, timestamp, website),
            key=lambda x: x[1]):
            userToSites[user].append(site)

        # For each of three websites, count its frequency.
        patternCount = collections.Counter()

        for user, sites in userToSites.items():
            patternCount.update(Counter(set(itertools.combinations(sites, 3))))

        return max(sorted(patternCount), key=patternCount.get)

```

• LeetDoocs

```

class Solution:
    def mostVisitedPattern(
        self, username: List[str], timestamp: List[int], website: List[str]
    ) -> List[str]:
        d = defaultdict(list)
        for user, _, site in sorted(
            zip(username, timestamp, website), key=lambda x: x[1]
        ):
            d[user].append(site)

```

```

cnt = Counter()
for sites in d.values():
    m = len(sites)
    s = set()
    if m > 2:
        for i in range(m - 2):
            for j in range(i + 1, m - 1):
                for k in range(j + 1, m):
                    s.add((sites[i], sites[j], sites[k]))

    for t in s:

        cnt[t] += 1
return sorted(cnt.items(), key=lambda x: (-x[1], x[0]))[0][0]

```

• SimplyLeet

Python code solution for Analyze User Website Visit Pattern

```

from collections import defaultdict
from itertools import combinations

```

```

def mostVisitedPattern(username, timestamp, website):
    # Combine the logs into a list of tuples and sort by timestamp.
    logs = sorted(zip(timestamp, username, website))

```

Dictionary to hold each user's list of website visits

```

user_visits = defaultdict(list)
for time, user, site in logs:
    user_visits[user].append(site)

```

Dictionary to map a pattern (tuple of 3 websites) to set of users who visited that pattern

```

pattern_users = defaultdict(set)

```

For each user, generate all unique 3-sequences from their visit list and update pattern_users

```

for user, sites in user_visits.items():
    # Use set to avoid counting duplicate patterns from the same user
    multiple times

```

```

        seen_patterns = set()
        if len(sites) < 3:
            continue
        # Generate all 3-sequence combinations (order matters since visits are
in time order)
        for pattern in combinations(sites, 3):
            if pattern not in seen_patterns:
                seen_patterns.add(pattern)
                pattern_users[pattern].add(user)

        # Find the pattern with the highest score (i.e., most users)

# In case of a tie, lexicographically smallest pattern wins.
max_count = 0
result = None
for pattern, users in pattern_users.items():
    count = len(users)
    if count > max_count or (count == max_count and (result is None or pat
tern < result)):
        max_count = count
        result = pattern

return list(result)

# Example usage:
print(mostVisitedPattern(
    ["joe","joe","joe","james","james","james","james","mary","mary","mary"],
    [1,2,3,4,5,6,7,8,9,10],
    ["home","about","career","home","cart","maps","home","home","about","caree
r"]
))

```

• LC.ca

```

class Solution:
    def mostVisitedPattern(
        self, username: List[str], timestamp: List[int], website: List[str]
    ) -> List[str]:
        d = defaultdict(list)
        for user, _, site in sorted(
            zip(username, timestamp, website), key=lambda x: x[1]
        ):
            d[user].append(site)

        cnt = Counter()
        for sites in d.values():
            m = len(sites)
            s = set()
            if m > 2:
                for i in range(m - 2):
                    for j in range(i + 1, m - 1):
                        for k in range(j + 1, m):
                            s.add((sites[i], sites[j], sites[k]))

            for t in s:
                cnt[t] += 1

        return sorted(cnt.items(), key=lambda x: (-x[1], x[0]))[0][0]

```

◆ Quick Review

Key Insights

- Combine the inputs into a single list of events and sort them by timestamp.
- Group the visits by each user, preserving the sorted order.
- Generate all unique combinations of 3-sequences for each user (order matters and non-contiguous visits are allowed).

- Use a dictionary (or hash map) to map each 3-sequence pattern to the set of users who visited that pattern.
- The problem constraints are small, so generating combinations per user (even using triple nested loops) is acceptable.
- When multiple patterns have the same frequency, choose the lexicographically smallest one.

Space & Time

Time Complexity: $O(U * L^3)$ where U is the number of users and L is the maximum length of visits per user (since we generate combinations of 3 visits per user). Sorting all events costs $O(N \log N)$ for N total events.

Space Complexity: $O(P)$ where P is the number of unique 3-sequence patterns stored in the dictionary, plus $O(N)$ for grouping the events.

Solution

Aggregate the events by user while sorting them based on timestamp. For each user, generate all sets of 3-website patterns (to avoid duplicate patterns per user). Use a hash map to count the number of unique users for

each pattern. Traverse the map to find the pattern with the highest count. In case of a tie, compare patterns lexicographically. Return the pattern that meets the criteria.

Binary Search

Round 2 · High · Medium · 12 questions

Binary Search

□ #33 Search in Rotated Sorted Array

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: Grind 169

Problem

There is an integer array `nums` sorted in ascending order (with distinct values).

Prior to being passed to your function, `nums` is possibly left rotated at an unknown index `k` ($1 \leq k < \text{nums.length}$) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (0-indexed).

For example, `[0,1,2,4,5,6,7]` might be left rotated by 3 indices and become `[4,5,6,7,0,1,2]`.

Given the array `nums` after the possible rotation and an integer `target`, return the index of `target` if it is in `nums`, or `-1` if it is not in `nums`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: nums = [4,5,6,7,0,1,2], target = 0
Output: 4

Example 2:

Input: nums = [4,5,6,7,0,1,2], target = 3
Output: -1

Example 3:

Input: nums = [1], target = 0
Output: -1

Constraints:

- $1 \leq \text{nums.length} \leq 5000$
- $-104 \leq \text{nums}[i] \leq 104$
- All values of nums are unique.
- nums is an ascending array that is possibly rotated.
- $-104 \leq \text{target} \leq 104$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        l = 0
        n = len(nums)

        r = n-1

        while l<=r:
            m = (l+r) // 2
            if nums[m] == target:
```

```

        return m

    if nums[l] <= nums[m]:
        if nums[l] <= target < nums[m]:
            r = m-1

        else:
            l = m+1

    else:

        if nums[m] < target <= nums[r]:
            l = m+1
        else:
            r = m-1

    return -1

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Array was sorted then rotated at some unknown pivot. Find target in $O(\log n)$. Right?
 # All values are distinct. Return -1 if not found."
 #
 # "[4,5,6,7,0,1,2], target=0 → 4 ✓. target=3 → -1 ✓."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
 # Scan left to right: for each index i , check if $\text{nums}[i] == \text{target}$.
 # If not found after one full pass, return -1.
 # Correct and easy to reason about, but the problem asks for $O(\log n)$ on a rotated sorted array.
 # Time: $O(n)$. Space: $O(1)$.
 # Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Standard binary search – but at each step, identify which half is sorted.
 # If $\text{nums}[\text{left}] \leq \text{nums}[\text{mid}]$: left half is sorted.
 # If target in $[\text{nums}[\text{left}], \text{nums}[\text{mid}]$): search left.
 # Else search right.
 # Else: right half is sorted.
 # If target in $(\text{nums}[\text{mid}], \text{nums}[\text{right}]$): search right.
 # Else search left.
 # Time: $O(\log n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```

class _33_SearchRotatedArray:
    def search(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = (left + right) // 2
            if nums[mid] == target:
                return mid
            if nums[left] <= nums[mid]: # left half sorted
                if nums[left] <= target < nums[mid]:
                    right = mid - 1
            else:
                left = mid + 1
        else: # right half sorted
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1
        return -1

```

PHASE 5 — TEST & DEBUG

```

# [4,5,6,7,0,1,2], target=0:
# l=0,r=6,mid=3(7). nums[0]=4<=7 → left sorted [4..7]. 0 in [4,7]? No → l=4.
# l=4,r=6,mid=5(1). nums[4]=0>1 → right sorted [1..2]. 0 in (1,2]? No → r=4.
# l=4,r=4,mid=4(0). 0==target → return 4 ✓
# target=3: never matches → -1 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(log n). Space O(1).
# The key: at least one half is always sorted in a rotated array – use that half
# to decide where to search. Checking nums[left] <= nums[mid] tells us which half.
# Given more time I'd ask: what if there are duplicates?
# That's #81 – when nums[left]==nums[mid], we can't determine which half is sorted.
# Worst case degrades to O(n) (just left++ to skip the duplicate)."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def search(self, nums: list[int], target: int) -> int:
        l = 0
        r = len(nums) - 1

        while l <= r:
            m = (l + r) // 2
            if nums[m] == target:
                return m
            if nums[l] <= nums[m]: # nums[l..m] are sorted.

                if nums[l] <= target < nums[m]:
                    r = m - 1
                else:
                    l = m + 1
            else: # nums[m..n - 1] are sorted.
                if nums[m] < target <= nums[r]:
                    l = m + 1
                else:
                    r = m - 1

        return -1

```

• LeetDoocs

```

class Solution:
    def search(self, nums: List[int], target: int) -> int:
        n = len(nums)
        left, right = 0, n - 1
        while left < right:
            mid = (left + right) >> 1
            if nums[0] <= nums[mid]:
                if nums[0] <= target <= nums[mid]:
                    right = mid
            else:

```

```

        left = mid + 1
    else:
        if nums[mid] < target <= nums[n - 1]:
            left = mid + 1
        else:
            right = mid
    return left if nums[left] == target else -1

```

• SimplyLeet

Define a function to search for the target in a rotated sorted array.

```
def search(nums, target):
```

```
    # Initialize pointers for binary search.
```

```
    left, right = 0, len(nums) - 1
```

```
    # Loop until the search space is exhausted.
```

```
    while left <= right:
```

```
        # Compute the middle index.
```

```
        mid = (left + right) // 2
```

```
    # If the mid element is the target, return its index.
```

```
    if nums[mid] == target:
```

```
        return mid
```

```
    # Check if the left part of the array is sorted.
```

```
    if nums[left] <= nums[mid]:
```

```
        # If the target is within the sorted left half, adjust the right
        pointer.
```

```
        if nums[left] <= target < nums[mid]:
```

```
            right = mid - 1
```

```
        else:
```

```
            # Otherwise, search the right half.
```

```
            left = mid + 1
```

```
    else:
```

```
        # The right half must be sorted.
```

```
        if nums[mid] < target <= nums[right]:
```

```
            left = mid + 1
```

```
        else:
```

```
            right = mid - 1
```

```
    # If the target is not found, return -1.
```



```
return -1
```

```
# Example usage:
# result = search([4,5,6,7,0,1,2], 0)
# print(result) # Expected output: 4
```

• LC.ca

```
# below 2 solutions, diff is while condition: left ('<' or '<=') right
```

```
# I like this better
```

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        n = len(nums)
        left, right = 0, n - 1
        while left <= right:
            mid = (left + right) >> 1
            if nums[mid] == target:
```

```
                return mid
            elif nums[0] <= nums[mid]: # left half sorted
                if nums[0] <= target <= nums[mid]:
                    right = mid - 1
                else:
                    left = mid + 1
            else: # right half sorted
                if nums[mid] < target <= nums[n - 1]:
                    left = mid + 1
                else:
```

```
                    right = mid - 1
        return -1
```

```
#####
```

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        n = len(nums)
        left, right = 0, n - 1
        while left < right:
```

```
mid = (left + right) >> 1
if nums[0] <= nums[mid]: # left half sorted
    if nums[0] <= target <= nums[mid]:
        right = mid
    else:
        left = mid + 1
else: # right half sorted
    if nums[mid] < target <= nums[n - 1]:
        left = mid + 1
    else:
        right = mid
return left if nums[left] == target else -1
```

◆ Quick Review

Key Insights

- The array is originally sorted and then rotated at an unknown pivot.
- Use a modified binary search to accommodate the rotation.
- At every step, determine which half of the array is properly sorted.
- Narrow down the search based on the sorted half and the target's value.

Space & Time

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Solution

We perform a binary search while accounting for the possibility that the array is rotated. At each iteration, we check if the left-to-mid portion is sorted. If it is, and the target falls within that range, we search the left half; otherwise, we search the right half. If the left half is not sorted, then the right half must be sorted, and similarly, we check if the target is in that sorted half. This approach ensures logarithmic time complexity while handling the rotation correctly.

Binary Search

□ #34 Find First and Last Position of Element in Sorted Array

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: Denny Zhang

Problem

Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given target value.

If target is not found in the array, return `[-1, -1]`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [5,7,7,8,8,10]`, `target = 8`
Output: `[3,4]`

Example 2:

Input: `nums = [5,7,7,8,8,10]`, `target = 6`
Output: `[-1,-1]`

Example 3:

Input: nums = [], target = 0
Output: [-1,-1]

Constraints:

- $0 \leq \text{nums.length} \leq 105$
- $-109 \leq \text{nums}[i] \leq 109$
- nums is a non-decreasing array.
- $-109 \leq \text{target} \leq 109$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def searchRange(self, nums: List[int], target: int) -> List[int]:
        first = bisect.bisect_left(nums, target)
        if first >= len(nums) or nums[first] != target:
            return [-1,-1]

        second = bisect.bisect_right(nums, target)
        return [first, second-1]
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Sorted array with duplicates. Find the first and last index of target. Return [-1,-1] if absent.
# Must be O(log n). Right?"
#
# "[5,7,7,8,8,10], target=8 -> [3,4] ✓. target=6 -> [-1,-1] ✓."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Run standard binary search once to find any index of target.
# Then expand left and right from that index while values still equal target.
# Works, but we can do both boundaries in one modified binary search pass.
# Time: O(log n) for search + O(k) for expansion. Space: O(1).
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE**# "Two binary searches:**

lower_bound: find first index where `nums[i] >= target`. If `nums[result]==target` → first position.
 # upper_bound: find first index where `nums[i] > target`. Subtract 1 → last position.
 # Both are standard binary search variants. Time: $O(\log n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```
class _34_FirstLastPosition:
    def searchRange(self, nums: List[int], target: int) -> List[int]:
        def lower_bound() -> int:
            left, right = 0, len(nums)
            while left < right:
                mid = (left + right) // 2
                if nums[mid] >= target:
                    right = mid
                else:
                    left = mid + 1
            return left
        def upper_bound() -> int:
            left, right = 0, len(nums)
            while left < right:
                mid = (left + right) // 2
                if nums[mid] > target:
                    right = mid
                else:
                    left = mid + 1
            return left
        first = lower_bound()
        if first == len(nums) or nums[first] != target:
            return [-1, -1]
        return [first, upper_bound() - 1]
```

PHASE 5 — TEST & DEBUG

`nums=[5,7,7,8,8,10]`, `target=8`:
 # lower_bound: searching for first idx where `nums[i]>=8`. → index 3.
 # `nums[3]=8==target` ✓. upper_bound: first idx where `nums[i]>8`. → index 5. `5-1=4`.
 # → `[3,4]` ✓
 # `target=6`: lower_bound→2 (`nums[2]=7>=6`). `nums[2]=7≠6` → `[-1,-1]` ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\log n)$. Space $O(1)$.
 # lower_bound and upper_bound are the two fundamental binary search variants – know both.
 # They correspond to `bisect_left` and `bisect_right` in Python's `bisect` module.
 # The check '`nums[first] != target`' after lower_bound handles the not-found case cleanly.
 # Given more time I'd ask: what if we want the count of target occurrences?
 # `upper_bound() - lower_bound()` in $O(\log n)$."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def searchRange(self, nums: list[int], target: int) -> list[int]:
        l = bisect_left(nums, target)
        if l == len(nums) or nums[l] != target:
            return -1, -1
        r = bisect_right(nums, target) - 1
        return l, r
```

• LeetDoocs

```
class Solution:
    def searchRange(self, nums: List[int], target: int) -> List[int]:
        l = bisect_left(nums, target)
        r = bisect_left(nums, target + 1)
        return [-1, -1] if l == r else [l, r - 1]
```

• SimplyLeet

```
def searchRange(nums, target):
    # Helper function to find the boundary index (leftmost or rightmost) for
    # target.
    def findBoundary(isLeft):
        left, right = 0, len(nums) - 1
        boundary = -1
        while left <= right:
            mid = (left + right) // 2
            # If the target is found, update boundary and continue to search on
            # one side.
            if nums[mid] == target:
                boundary = mid
```



```

        if isLeft:
            right = mid - 1 # Continue search on left half
        else:
            left = mid + 1 # Continue search on right half
    elif nums[mid] < target:
        left = mid + 1
    else:
        right = mid - 1
    return boundary

```

```

# Find the leftmost index
leftIndex = findBoundary(True)
if leftIndex == -1:
    return [-1, -1]
# Find the rightmost index
rightIndex = findBoundary(False)
return [leftIndex, rightIndex]

```

```

# Example usage:
# nums = [5,7,7,8,8,10]

```

```

# target = 8
# print(searchRange(nums, target)) # Output: [3,4]

```

• LC.ca

```

'''
>>> bisect.bisect_left([1,1,1,2,2,2,7,7,7], 2)
3
>>> bisect.bisect_left([1,1,1,2,2,2,7,7,7], 2+1)
6

>>> bisect.bisect_right([1,1,1,2,2,2,7,7,7], 2)
6
>>> bisect.bisect_right([1,1,1,2,2,2,7,7,7], 2+1)

```

6

```
>>> bisect.bisect_left([1,1,1,2,2,2,7,7,7], 0)
```

0

```
>>> bisect.bisect_left([1,1,1,2,2,2,7,7,7], 1)
```

0

```
# below, even single value for 2, after +1 the index will be different
```

```
>>> bisect.bisect_left([1,2,3,4,5], 2)
```

1

```
>>> bisect.bisect_left([1,2,3,4,5], 2+1)
```

2

'''

```
import bisect
```

```
class Solution:
```

```
    def searchRange(self, nums: List[int], target: int) -> List[int]:
```

```
        l = bisect_left(nums, target)
```

```
        r = bisect_left(nums, target + 1)
```

```
        return [-1, -1] if l == r else [l, r - 1]
```

```
#####
```

```
class Solution:
```

```
    def searchRange(self, nums: List[int], target: int) -> List[int]:
```

```
        def findRange(to_left):
```

```
            l, r = 0, len(nums) - 1
```

```
            m = 0
```

```

while l <= r:
    m = l + (r - l) // 2
    if nums[m] < target:
        l = m + 1
    elif nums[m] > target:
        r = m - 1
    elif to_left and m > 0 and nums[m - 1] == nums[m]: # so now
nums[m] == target, but maybe not the leftmost or rightmost
        r = m - 1
    elif not to_left and m + 1 < len(nums) and nums[m + 1] == nums
[m]: # so now nums[m] == target, but maybe not the leftmost or rightmost
        l = m + 1

else:
    return m
if m < len(nums) and nums[m] == target:
    # 1. cover not-found,
    # or 2. cover input=[3] and target=3, first and last both 0, return
[0,0]
    return m
else:
    return -1
return [findRange(True), findRange(False)]

```

◆ Quick Review

Key Insights

- The array is sorted, allowing the use of binary search.
- Two binary searches can efficiently find the leftmost (first) and rightmost (last) indices of the target.
- Handling edge cases where the target is not present by checking bounds.

Space & Time

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Solution

The approach is to use binary search twice: The first binary search finds the leftmost occurrence of the target by continuing to search in the left half even after finding the target. The second binary search finds the rightmost occurrence by searching in the right half. The algorithm maintains pointers for the current search bounds and narrows the search space using comparisons. If the target is not found during the first binary search, the solution immediately returns $[-1, -1]$. This method leverages the sorted property of the array, ensuring $O(\log n)$ time complexity with constant extra space.

Binary Search

□ #153 Find Minimum in Rotated Sorted Array

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: Grind 169

Problem

Suppose an array of length n sorted in ascending order is rotated between 1 and n times. For example, the array `nums = [0,1,2,4,5,6,7]` might become:

- `[4,5,6,7,0,1,2]` if it was rotated 4 times.
- `[0,1,2,4,5,6,7]` if it was rotated 7 times.

Notice that rotating an array `[a[0], a[1], a[2], ..., a[n-1]]` 1 time results in the array `[a[n-1], a[0], a[1], a[2], ..., a[n-2]]`.

Given the sorted rotated array `nums` of unique elements, return the minimum element of this array.

You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: `nums = [3,4,5,1,2]`

Output: `1`

Explanation: The original array was `[1,2,3,4,5]` rotated 3 times.

Example 2:

Input: `nums = [4,5,6,7,0,1,2]`

Output: `0`

Explanation: The original array was `[0,1,2,4,5,6,7]` and it was rotated 4 times.

Example 3:

Input: `nums = [11,13,15,17]`

Output: `11`

Explanation: The original array was `[11,13,15,17]` and it was rotated 4 times.

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 5000$
- $-5000 \leq \text{nums}[i] \leq 5000$
- All the integers of `nums` are unique.
- `nums` is sorted and rotated between 1 and `n` times.

My LeetCode Solution

- My LeetCode Solution

```

class Solution:
    def findMin(self, nums: List[int]) -> int:
        l = 0
        n = len(nums)
        r = n-1
        if nums[l] < nums[r]:
            return nums[l]

        while l<=r:
            m = (l+r)//2

            if m+1 <n and nums[m] > nums[m+1]:
                return nums[m+1]

            if nums[m-1] > nums[m]:
                return nums[m]

            if nums[m] > nums[r]:
                l = m+1

            else:

                r = m-1

        return nums[l]

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Array sorted then rotated. Find the minimum in $O(\log n)$. All values unique. Right?

The minimum is at the rotation point – where the array 'wraps'."

#

"[3,4,5,1,2] → 1 ✓. [4,5,6,7,0,1,2] → 0 ✓. [11,13,15,17] → 11 (no rotation) ✓."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Linear scan from index 0: track the smallest value seen so far.

The first place where $\text{nums}[i] > \text{nums}[i+1]$ marks the rotation pivot candidate.

Return that minimum – correct on small inputs, but we can binary-search the pivot.

Time: $O(n)$. Space: $O(1)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE


```
# "Binary search on the rotation point.
# If nums[mid] > nums[right]: minimum is in the RIGHT half (mid+1 to right).
# Else: minimum is in the LEFT half including mid (left to mid).
# When left==right, we've found the minimum.
# Time: O(log n). Space: O(1)."
```

PHASE 4 — CLEAN CODE

```
class _153_FindMinRotated:
    def findMin(self, nums: List[int]) -> int:
        left, right = 0, len(nums) - 1
        while left < right:
            mid = (left + right) // 2
            if nums[mid] > nums[right]:
                left = mid + 1 # minimum is to the right of mid
            else:
                right = mid # minimum is mid or to its left
        return nums[left]
```

PHASE 5 — TEST & DEBUG

```
# [3,4,5,1,2]: l=0,r=4,mid=2(5). 5>nums[4]=2 → l=3.
# l=3,r=4,mid=3(1). 1<=nums[4]=2 → r=3. l=r=3 → return nums[3]=1 ✓
# [4,5,6,7,0,1,2]: l=0,r=6,mid=3(7). 7>nums[6]=2 → l=4.
# l=4,r=6,mid=5(1). 1<=2 → r=5. l=4,r=5,mid=4(0). 0<=1 → r=4. l=r=4 → 0 ✓
# [11,13,15,17]: l=0,r=3,mid=1(13). 13<=17 → r=1. l=0,r=1,mid=0(11). 11<=13 → r=0. → 11 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(log n). Space O(1).
# The comparison is always against nums[right] (not nums[left]).
# If nums[mid] > nums[right], the right portion must be 'lower' – minimum is there.
# If nums[mid] <= nums[right], the right portion is in order – minimum is at mid or left.
# Given more time I'd ask: what if there are duplicates?
# When nums[mid]==nums[right] we can't decide – fall back to right-- (O(n) worst case)."
```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def findMin(self, nums: list[int]) -> int:
        l = 0
        r = len(nums) - 1

        while l < r:
            m = (l + r) // 2
            if nums[m] < nums[r]:
                r = m
            else:
                l = m + 1

        return nums[l]

```

• LeetCode

```

class Solution:
    def findMin(self, nums: List[int]) -> int:
        l, r = 0, len(nums) - 1
        while l < r:
            mid = (l + r) >> 1
            if nums[mid] > nums[-1]:
                l = mid + 1
            else:
                r = mid
        return nums[l]

```

• SimplyLeet

```

def findMin(nums):
    # Initialize left and right pointers
    left, right = 0, len(nums) - 1
    # Continue while the search space is valid
    while left < right:
        mid = (left + right) // 2 # Compute middle index
        # If middle element is greater than the rightmost element,
        # the minimum must be in the right half
        if nums[mid] > nums[right]:
            left = mid + 1

```

```
        else:
            # Otherwise, the minimum is in the left half (including mid)
            right = mid
    # When loop ends, left == right, pointing to the minimum element
    return nums[left]
```

• LC.ca

```
class Solution:
    def findMin(self, nums: List[int]) -> int:
        if nums[0] <= nums[-1]:
            return nums[0]
        left, right = 0, len(nums) - 1
        while left < right:
            mid = (left + right) >> 1
            if nums[0] <= nums[mid]:
                left = mid + 1
            else:
                right = mid
        return nums[left]
```

◆ Quick Review

Key Insights

- The rotated sorted array consists of two sorted subarrays.
- A modified binary search can be used to locate the pivot point where the rotation occurs.
- By comparing the middle element to the rightmost element, we can decide which half of the array contains the minimum.

Space & Time

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

Solution

Use a binary search approach. Initialize two pointers, left and right, at the start and end of the array. While left is less than right, calculate the middle index. If the middle element is greater than the right element, then the minimum is in the right half, so move the left pointer to $\text{mid} + 1$; otherwise, the minimum must be in the left half including mid, so adjust the right pointer to mid. When the loop terminates, left equals right and points to the minimum element. This approach leverages the properties of the rotated sorted array to achieve logarithmic time complexity.

Binary Search

□ #300 Longest Increasing Subsequence

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Dynamic Programming
Lists: Grind 169

Problem

Given an integer array `nums`, return the length of the longest strictly increasing subsequence.

Example 1:

Input: `nums = [10,9,2,5,3,7,101,18]`

Output: 4

Explanation: The longest increasing subsequence is `[2,3,7,101]`, therefore the length is 4.

Example 2:

Input: `nums = [0,1,0,3,2,3]`

Output: 4

Example 3:

Input: `nums = [7,7,7,7,7,7,7]`

Output: 1

Constraints:

- $1 \leq \text{nums.length} \leq 2500$
- $-104 \leq \text{nums}[i] \leq 104$

Follow up: Can you come up with an algorithm that runs in $O(n \log(n))$ time complexity?

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        sub = []
        for x in nums:
            if not sub or sub[-1] < x:
                sub.append(x)

            else:
                i = bisect.bisect_left(sub, x)
                sub[i] = x

        return len(sub)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the length of the longest strictly increasing subsequence (not necessarily contiguous)."

Return the length, not the subsequence itself. Right?

Follow-up asks for $O(n \log n)$ – so know both approaches."

#

"[10,9,2,5,3,7,101,18] → [2,3,7,101] length 4 ✓. [7,7,7,7] → 1 (strictly increasing) ✓."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

$dp[i]$ = length of LIS ending at i . For each i , scan all $j < i$ with $nums[j] < nums[i]$.

$dp[i] = \max(dp[j]+1)$. Answer = $\max(dp)$.

Correct $O(n^2)$; patience sorting / binary search on tails gives $O(n \log n)$.

Time: $O(n^2)$. Space: $O(n)$ dp array.

Should I go ahead and optimize?"

```
class _300_LIS_DP:
```

```
    def lengthOfLIS(self, nums: List[int]) -> int:
```

```

dp = [1] * len(nums)
for i in range(1, len(nums)):
    for j in range(i):
        if nums[j] < nums[i]:
            dp[i] = max(dp[i], dp[j] + 1)
return max(dp)

```

PHASE 3 — OPTIMIZE ($O(n \log n)$ patience sorting)

"Maintain a 'tails' array where tails[i] is the smallest tail of all LIS of length i+1.

For each number, binary search for the leftmost position in tails where tails[pos] >= num.

If found, replace tails[pos] with num (smaller tail = more room to extend).

If not found, append (the LIS can be extended by 1).

len(tails) at the end is the LIS length.

#

This is patience sorting – tails is always sorted so binary search works."

PHASE 4 — CLEAN CODE

```

import bisect
class _300_LIS:
    def lengthOfLIS(self, nums: List[int]) -> int:
        tails = []
        for num in nums:
            pos = bisect.bisect_left(tails, num)
            if pos == len(tails):
                tails.append(num)
            else:
                tails[pos] = num
        return len(tails)

```

PHASE 5 — TEST & DEBUG

nums=[10,9,2,5,3,7,101,18]:

10→tails=[10]. 9→replace 10→[9]. 2→replace 9→[2]. 5→append→[2,5].

3→replace 5→[2,3]. 7→append→[2,3,7]. 101→append→[2,3,7,101]. 18→replace 101→[2,3,7,18].

len=4 ✓

[7,7,7,7]: first 7→[7]. next 7→bisect_left([7],7)=0→replace→[7]. len=1 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

" $O(n^2)$ DP: Time $O(n^2)$, Space $O(n)$. Clean and easy to explain.

$O(n \log n)$ patience sort: Time $O(n \log n)$, Space $O(n)$.

Note: tails does NOT store the actual LIS – it's a virtual 'pile' structure.

To reconstruct the actual LIS, track parent pointers in the DP version.

Given more time I'd ask: longest non-decreasing subsequence?

Change bisect_left to bisect_right – allows equal elements to extend the sequence."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def lengthOfLIS(self, nums: list[int]) -> int:
        if not nums:
            return 0

        # dp[i] := the length of LIS ending in nums[i]
        dp = [1] * len(nums)

        for i in range(1, len(nums)):
            for j in range(i):

                if nums[j] < nums[i]:
                    dp[i] = max(dp[i], dp[j] + 1)

        return max(dp)
```

```
class Solution:
    def lengthOfLIS(self, nums: list[int]) -> int:
        # tails[i] := the minimum tails of all the increasing subsequences having
        # length i + 1
        tails = []

        for num in nums:
            if not tails or num > tails[-1]:
                tails.append(num)
            else:
                tails[bisect.bisect_left(tails, num)] = num

        return len(tails)
```

• LeetDoocs

```
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        n = len(nums)
        f = [1] * n
        for i in range(1, n):
            for j in range(i):
                if nums[j] < nums[i]:
                    f[i] = max(f[i], f[j] + 1)
        return max(f)
```

```
class BinaryIndexedTree:
    def __init__(self, n: int):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x: int, v: int):
        while x <= self.n:
            self.c[x] = max(self.c[x], v)
            x += x & -x
```

```
    def query(self, x: int) -> int:
        mx = 0
        while x:
            mx = max(mx, self.c[x])
            x -= x & -x
        return mx
```

```
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
```

```
        s = sorted(set(nums))
        m = len(s)
        tree = BinaryIndexedTree(m)
        for x in nums:
            x = bisect_left(s, x) + 1
            t = tree.query(x - 1) + 1
            tree.update(x, t)
        return tree.query(m)
```

• SimplyLeet

```
import bisect

def lengthOfLIS(nums):
    dp = [] # dp array stores smallest tail for each subsequence length
    for num in nums:
        # Find the leftmost insertion point using binary search
        idx = bisect.bisect_left(dp, num)
        if idx == len(dp):
            dp.append(num) # Extend dp if no larger tail exists
        else:
            dp[idx] = num # Replace to maintain the smallest tail
    return len(dp)

# Example usage:
# print(lengthOfLIS([10,9,2,5,3,7,101,18])) # Output: 4
```

• LC.ca

```
# find the largest end element in tails that is smaller than nums[i]
# and then replace it with nums[i] and discard the list in the same length
# which is implemented by `tail[idx] = num`
```

```
class Solution(object):
    def lengthOfLIS(self, nums):
        """
        :type nums: List[int]
        :rtype: int
        """
```

```
tail = []
for num in nums:
    # if using bisect_right(tail, num),
    # then input=[7,7,7,7,7,7,7] will output 7 but expected result is 1
    idx = bisect.bisect_left(tail, num)
    if idx == len(tail): # same as in java: if (i == len) len++;
        tail.append(num)
    else:
        tail[idx] = num
return len(tail)
```

```

# implementation of bisect.bisect_left()
# similar to Leetcode-302, find left/right/top/bottom callable
def bisect_left(a, x, lo=0, hi=None):
    if hi is None:
        hi = len(a)

    while lo < hi:
        mid = (lo + hi) // 2

        if a[mid] < x:
            lo = mid + 1
        else:
            hi = mid

    return lo

#####

class BinaryIndexedTree:

    def __init__(self, n: int):
        self.n = n
        self.c = [0] * (n + 1)

    def update(self, x: int, v: int):
        while x <= self.n:
            self.c[x] = max(self.c[x], v)
            x += x & -x

    def query(self, x: int) -> int:

        mx = 0
        while x:
            mx = max(mx, self.c[x])
            x -= x & -x
        return mx

class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        s = sorted(set(nums))

```

```
m = len(s)
tree = BinaryIndexedTree(m)
for x in nums:
    x = bisect_left(s, x) + 1
    t = tree.query(x - 1) + 1
    tree.update(x, t)
return tree.query(m)
```

◆ Quick Review

Key Insights

- A dynamic programming solution with $O(n^2)$ time complexity exists, but we focus on an optimized approach.
- Using binary search and a dp array, we can achieve $O(n \log n)$ time.
- The dp array maintains the smallest possible tail for increasing subsequences of various lengths.
- For each number, perform binary search to decide if it can extend or replace a value in dp.

Space & Time

Time Complexity: $O(n \log n)$ where n is the length of nums

Space Complexity: $O(n)$ for storing the dp array

Solution

We use a dynamic programming approach combined with binary search. The idea is to maintain a dp array where $dp[i]$ holds the smallest tail value for any increasing subsequence with length $i+1$. For each number in the array, we perform a binary search on dp to find its correct insertion position: If the number is larger than all elements, we append it, effectively extending the current longest subsequence. Otherwise, we replace the first element in dp that is greater than or equal to the current number to keep the tail as small as possible. This process ensures that dp remains sorted and its length gives the length of the longest increasing subsequence.

Binary Search

□ #362 Design Hit Counter

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Binary Search · Design · Queue

Lists: Grind 169

Problem

Design a hit counter which counts the number of hits received in the past 5 minutes (i.e., the past 300 seconds).

Your system should accept a timestamp parameter (in seconds granularity), and you may assume that calls are being made to the system in chronological order (i.e., timestamp is monotonically increasing). Several hits may arrive roughly at the same time.

Implement the HitCounter class:

- **HitCounter()** Initializes the object of the hit counter system.
- **void hit(int timestamp)** Records a hit that happened at timestamp (in seconds). Several

hits may happen at the same timestamp.

- **int getHits(int timestamp)** Returns the number of hits in the past 5 minutes from timestamp (i.e., the past 300 seconds).

Example 1:

Input

```
["HitCounter", "hit", "hit", "hit", "getHits", "hit", "getHits", "getHits"]  
[[], [1], [2], [3], [4], [300], [300], [301]]
```

Output

```
[null, null, null, null, 3, null, 4, 3]
```

Explanation

```
HitCounter hitCounter = new HitCounter();
```

Constraints:

- $1 \leq \text{timestamp} \leq 2 * 10^9$
- All the calls are being made to the system in chronological order (i.e., timestamp is monotonically increasing).
- At most 300 calls will be made to hit and getHits.

Follow up: What if the number of hits per second could be huge? Does your design scale?

My LeetCode Solution

- My LeetCode Solution

```
# Design Hit Counter
class HitCounter:

    def __init__(self):
        self.q = deque()

    def hit(self, timestamp: int) -> None:
        self.q.append(timestamp)

    def getHits(self, timestamp: int) -> int:
        while self.q and timestamp-self.q[0] >= 300:
            self.q.popleft()
        return len(self.q)

# Your HitCounter object will be instantiated and called as such:
# obj = HitCounter()

# obj.hit(timestamp)
# param_2 = obj.getHits(timestamp)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Record hits at timestamps and query: how many hits in the past 300 seconds?
# Calls are chronologically ordered (timestamps only increase). Right?
# getHits(t) returns hits with timestamp in (t-300, t] inclusive – past 5 minutes."
#
# "hit(1),hit(2),hit(3): getHits(4)=3. hit(300): getHits(300)=4. getHits(301)=3
# (hit@1 expired)."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Append every hit timestamp to a list.
# getHits(t): count timestamps in [t-299, t] by scanning the whole list.
# Simple, but each query is O(total hits) – queue with expiry is better.
# Time: O(1) hit, O(n) getHits per query. Space: O(n) all timestamps.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Store timestamps in a deque. On getHits, pop from the left any timestamp <=
t-300.
# Remaining deque size is the count. hit() is O(1). getHits() is amortised O(1).
#
# Follow-up (huge hits/second): store (timestamp, count) pairs instead of individual
timestamps.
# Multiple hits at the same second are collapsed into one deque entry."
```

PHASE 4 — CLEAN CODE

```
class _362_HitCounter:
    def __init__(self):
        self.hits: deque = deque() # stores individual hit timestamps

    def hit(self, timestamp: int) -> None:
        self.hits.append(timestamp)

    def getHits(self, timestamp: int) -> int:
        while self.hits and self.hits[0] <= timestamp - 300:
            self.hits.popleft()
        return len(self.hits)
```

Follow-up: grouped version for high-throughput

```
class _362_HitCounter_Grouped:
    def __init__(self):
        self.hits: deque = deque() # stores (timestamp, count) pairs

    def hit(self, timestamp: int) -> None:
        if self.hits and self.hits[-1][0] == timestamp:
            ts, cnt = self.hits.pop()
            self.hits.append((ts, cnt + 1))
        else:
            self.hits.append((timestamp, 1))

    def getHits(self, timestamp: int) -> int:
        while self.hits and self.hits[0][0] <= timestamp - 300:
            self.hits.popleft()
        return sum(cnt for _, cnt in self.hits)
```

PHASE 5 — TEST & DEBUG

```
# hit(1),hit(2),hit(3),getHits(4): deque=[1,2,3]. all>4-300=-296 → len=3 ✓
# hit(300),getHits(300): deque=[1,2,3,300]. 1<=0? No. len=4 ✓
# getHits(301): 1<=301-300=1 → popleft. deque=[2,3,300]. len=3 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "hit: O(1). getHits: O(k) amortised where k = expired entries (each entry popped
at most once).
# Space: O(n) where n = total hits in the past 300 seconds.
# Binary search alternative: store sorted timestamps, binary search for t-300 cutoff
- O(log n).
# The deque approach is simpler and amortised O(1) total.
# Grouped version handles millions of hits/second - key interview follow-up.
# Given more time I'd ask: what if calls aren't chronological?
# We'd need a sorted structure (sorted list or BST) and can't use the deque eviction
trick."
```

Community Solutions (Python)

• WalkCC

```
class HitCounter:
    def __init__(self):
        self.timestamps = [0] * 300
        self.hits = [0] * 300

    def hit(self, timestamp: int) -> None:
        i = timestamp % 300
        if self.timestamps[i] == timestamp:
            self.hits[i] += 1
        else:

            self.timestamps[i] = timestamp
            self.hits[i] = 1 # Reset the hit count to 1.

    def getHits(self, timestamp: int) -> int:
        return sum(h for t, h in zip(self.timestamps, self.hits)
                    if timestamp - t < 300)
```

• LeetDoocs

```
class HitCounter:

    def __init__(self):
        self.ts = []

    def hit(self, timestamp: int) -> None:
        self.ts.append(timestamp)

    def getHits(self, timestamp: int) -> int:
        return len(self.ts) - bisect_left(self.ts, timestamp - 300 + 1)

# Your HitCounter object will be instantiated and called as such:
# obj = HitCounter()
# obj.hit(timestamp)
# param_2 = obj.getHits(timestamp)
```

• SimplyLeet

```
# Python implementation of HitCounter

from collections import deque

class HitCounter:
    def __init__(self):
        # Initialize the deque to store (timestamp, count) pairs.
        self.hits_queue = deque()

    def hit(self, timestamp: int) -> None:

        # If the last recorded hit is at the same timestamp, increment its
        count.
        if self.hits_queue and self.hits_queue[-1][0] == timestamp:
            self.hits_queue[-1][1] += 1
        else:
            # Append new (timestamp, count) pair.
            self.hits_queue.append([timestamp, 1])

    def getHits(self, timestamp: int) -> int:
        # Remove outdated hits from the front of the queue.
        while self.hits_queue and self.hits_queue[0][0] <= timestamp - 300:

            self.hits_queue.popleft()
        # Sum counts of the remaining hit entries.
        return sum(count for _, count in self.hits_queue)
```

• LC.ca

```
from collections import deque

class HitCounter: # queue

    def __init__(self):
        """
        Initialize your data structure here.
        """
        self.hits = deque()
```

```
def hit(self, timestamp: int) -> None:
    """
    Record a hit.
    @param timestamp - The current timestamp (in seconds granularity).
    """
    self.hits.append(timestamp)
```

```
def getHits(self, timestamp: int) -> int:
    """
    Return the number of hits in the past 5 minutes.
```

```
    @param timestamp - The current timestamp (in seconds granularity).
    """
    while self.hits and self.hits[0] <= timestamp - 300:
        self.hits.popleft()
    return len(self.hits)
```

```
class HitCounter: # follow-up
    def __init__(self):
        self.times = [0] * 300
```

```
        self.hits = [0] * 300
```

```
    def hit(self, timestamp: int) -> None:
        idx = timestamp % 300
        if self.times[idx] != timestamp:
            self.times[idx] = timestamp
            self.hits[idx] = 1
        else:
            self.hits[idx] += 1
```

```
    def getHits(self, timestamp: int) -> int:
        res = 0
        for i in range(300):
            if timestamp - self.times[i] < 300:
                res += self.hits[i]
        return res
```

```
# Your HitCounter object will be instantiated and called as such:
# obj = HitCounter()
# obj.hit(timestamp)
```

```
# param_2 = obj.getHits(timestamp)

#####

class HitCounter: # extra o(N) space
    def __init__(self):
        """
        Initialize your data structure here.
        """
        self.counter = Counter()

    def hit(self, timestamp: int) -> None:
        """
        Record a hit.
        @param timestamp - The current timestamp (in seconds granularity).
        """
        self.counter[timestamp] += 1

    def getHits(self, timestamp: int) -> int:
        """
        Return the number of hits in the past 5 minutes.
        @param timestamp - The current timestamp (in seconds granularity).
        """
        return sum([v for t, v in self.counter.items() if t + 300 > timestamp]
)

# Your HitCounter object will be instantiated and called as such:
# obj = HitCounter()
# obj.hit(timestamp)
# param_2 = obj.getHits(timestamp)
```

◆ Quick Review

Key Insights

- Use a sliding window of 300 seconds to count hits.

- Since timestamps are monotonically increasing, outdated hits can be removed from the front.
- A queue (or circular array) is an efficient data structure to maintain and remove expired hit records.
- Grouping consecutive hits at the same timestamp can optimize space.

Space & Time

Time Complexity: $O(1)$ per hit and `getHits` operation on average, since each hit is enqueued and later dequeued only once.

Space Complexity: $O(W)$ where W is the window size (300 seconds) in the worst case.

Solution

The solution uses a queue (or deque) data structure where each element stores a (timestamp, count) pair. When a hit is recorded, if the last element of the queue shares the same timestamp, then its count is incremented; otherwise, a new element is added. When `getHits(timestamp)` is called, the algorithm removes elements from the front of the queue whose timestamp is out of the

300-second window relative to the current timestamp, and then sums the counts of the remaining elements to return the result. This ensures that only relevant hits within the past 5 minutes are counted. The design scales well when hit frequency is high because hits at the same timestamp are grouped together instead of inserting individual records.

Binary Search

□ #528 Random Pick with Weight

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Binary Search · Prefix Sum ·
Randomized
Lists: Grind 169

Problem

You are given a 0-indexed array of positive integers w where $w[i]$ describes the weight of the i th index.

You need to implement the function `pickIndex()`, which randomly picks an index in the range $[0, w.length - 1]$ (inclusive) and returns it. The probability of picking an index i is $w[i] / \text{sum}(w)$.

- For example, if $w = [1, 3]$, the probability of picking index 0 is $1 / (1 + 3) = 0.25$ (i.e., 25%), and the probability of picking index 1 is $3 / (1 + 3) = 0.75$ (i.e., 75%).

Example 1:

Input
 ["Solution","pickIndex"]
 [[[1]],[]]
 Output
 [null,0]

Explanation
 Solution solution = new Solution([1]);

Example 2:

Input
 ["Solution","pickIndex","pickIndex","pickIndex","pickIndex","pickIndex"]
 [[[1,3]],[],[],[],[],[]]
 Output
 [null,1,1,1,1,0]

Explanation
 Solution solution = new Solution([1, 3]);

Constraints:

- $1 \leq w.length \leq 104$
- $1 \leq w[i] \leq 105$
- pickIndex will be called at most 104 times.

My LeetCode Solution

• My LeetCode Solution

```
# Random Pick with Weight
class Solution:

    def __init__(self, w: List[int]):
        self.ps = [w[0]]
        for i in range(1, len(w)):
            self.ps.append(self.ps[i-1] + w[i])

        self.total = self.ps[-1]
```

```
def pickIndex(self) -> int:
    target = self.total * random.random()
    l, r = 0, len(self.ps)-1

    while l<=r:
        m = (l+r)//2
        if self.ps[m] > target:
            r = m-1

        else:

            l = m+1

    return l
```

```
# Your Solution object will be instantiated and called as such:
# obj = Solution(w)
# param_1 = obj.pickIndex()
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Pick a random index where index i is chosen with probability $w[i]/\text{sum}(w)$.
 # pickIndex() is called up to 10^4 times – so init can be $O(n)$ but pick must be fast. Right?
 # $w=[1,3]$: $P(0)=25\%$, $P(1)=75\%$. The actual randomness in the example output is fine."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
 # Expand weights into a list where index i appears $w[i]$ times, pick random index.
 # Correct distribution but total list size can be $\text{sum}(w)$ up to $1e9$ – impossible.
 # Prefix sums + binary search on total weight fixes space.
 # Time: $O(\text{sum}(w))$ build. Space: $O(\text{sum}(w))$ – infeasible.
 # Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Build a prefix sum array. $\text{Prefix}[i] = w[0] + \dots + w[i]$.
 # Pick a random float r in $[0, \text{total})$. Binary search for the leftmost prefix sum $> r$.
 # That index is our answer – the probability of landing in bucket i equals $w[i]/\text{total}$.
 # Time: $O(n)$ init. $O(\log n)$ per pick. Space: $O(n)$."

PHASE 4 — CLEAN CODE

```
import random

class _528_RandomPickWeight:
    def __init__(self, w: List[int]):
        self.prefix = []
        total = 0
        for weight in w:
            total += weight
            self.prefix.append(total)
        self.total = total

    def pickIndex(self) -> int:
        r = random.random() * self.total
        lo, hi = 0, len(self.prefix) - 1
        while lo < hi:
            mid = (lo + hi) // 2
            if self.prefix[mid] <= r:
                lo = mid + 1
            else:
                hi = mid
        return lo
```

PHASE 5 — TEST & DEBUG

```
# w=[1,3]: prefix=[1,4], total=4.
# r in [0,1) -> prefix[0]=1>r -> lo=hi=0 -> return 0 (25% of the time).
# r in [1,4) -> prefix[0]=1<=r -> lo=1 -> return 1 (75% of the time). ✓
# w=[1]: prefix=[1]. Any r in [0,1) -> hi=lo=0 -> return 0 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Init O(n). pickIndex O(log n). Space O(n).
# The prefix sum maps weights to contiguous ranges on [0, total).
# A random point in that range lands in bucket i with probability w[i]/total.
# bisect.bisect_right(self.prefix, r) is a one-liner alternative using Python's
bisect module.
# Given more time I'd ask: what if weights change dynamically?
# We'd need a Fenwick tree (BIT) - O(log n) updates AND O(log n) queries."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def __init__(self, w: list[int]):
        self.prefix = list(itertools.accumulate(w))

    def pickIndex(self) -> int:
        target = random.randint(0, self.prefix[-1] - 1)
        return bisect.bisect_right(range(len(self.prefix)), target,
                                   key=lambda m: self.prefix[m])
```

• LeetDoocs

```
class Solution:
    def __init__(self, w: List[int]):
        self.s = [0]
        for c in w:
            self.s.append(self.s[-1] + c)

    def pickIndex(self) -> int:
        x = random.randint(1, self.s[-1])
        left, right = 1, len(self.s) - 1
        while left < right:

            mid = (left + right) >> 1
            if self.s[mid] >= x:
                right = mid
            else:
                left = mid + 1
        return left - 1

# Your Solution object will be instantiated and called as such:
# obj = Solution(w)

# param_1 = obj.pickIndex()
```

• SimplyLeet

```

import random
import bisect

class Solution:
    def __init__(self, w):
        # Build the prefix sum array.
        self.prefix_sums = []
        current_sum = 0
        for weight in w:
            current_sum += weight

        self.prefix_sums.append(current_sum)
        # Store the total sum of weights.
        self.total_sum = current_sum

    def pickIndex(self):
        # Get a random target in the range [1, total_sum]
        target = random.randint(1, self.total_sum)
        # Use bisect_left to find the proper index.
        index = bisect.bisect_left(self.prefix_sums, target)
        return index

# Example usage:
# solution = Solution([1, 3])
# print(solution.pickIndex())

```

• LC.ca

```

class Solution:
    def __init__(self, w: List[int]):
        self.s = [0]
        for c in w:
            self.s.append(self.s[-1] + c)

    def pickIndex(self) -> int:
        x = random.randint(1, self.s[-1])
        left, right = 1, len(self.s) - 1
        while left < right:

```



```
mid = (left + right) >> 1
if self.s[mid] >= x:
    right = mid
else:
    left = mid + 1
return left - 1
```

```
# Your Solution object will be instantiated and called as such:
# obj = Solution(w)
```

```
# param_1 = obj.pickIndex()
```

◆ Quick Review

Key Insights

- Build a prefix sum array from the weights.
- Calculate the total sum of the weights.
- Generate a random number in the range [1, total sum] and use binary search to find the corresponding index in the prefix sum array.
- Using binary search ensures efficient selection in $O(\log n)$ time for each call of `pickIndex()`.

Space & Time

Time Complexity: $O(n)$ for constructing the prefix sum array, and $O(\log n)$ per `pickIndex()` call.

Space Complexity: $O(n)$ for storing the prefix sum array.

Solution

The solution leverages a prefix sum array where each entry at index i represents the cumulative sum of weights up to i . After computing the total weight, a random integer is generated between 1 and the total sum (inclusive). A binary search is then used to find the first index where the prefix sum is greater than or equal to the random number. This approach ensures that the index is selected with a probability proportional to its weight.

Binary Search

□ #852 Peak Index in a Mountain Array

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: Denny Zhang

Problem

You are given an integer mountain array `arr` of length `n` where the values increase to a peak element and then decrease.

Return the index of the peak element.

Your task is to solve it in $O(\log(n))$ time complexity.

Example 1:

Input: `arr = [0,1,0]`

Output: 1

Example 2:

Input: `arr = [0,2,1,0]`

Output: 1

Example 3:

Input: `arr = [0,10,5,2]`

Output: 1

Constraints:

- $3 \leq \text{arr.length} \leq 105$
- $0 \leq \text{arr}[i] \leq 106$
- arr is guaranteed to be a mountain array.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def peakIndexInMountainArray(self, arr: list[int]) -> int:
        l = 0
        r = len(arr) - 1

        while l < r:
            m = (l + r) // 2
            if arr[m] >= arr[m + 1]:
                r = m
            else:
                l = m + 1

        return l
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Array guaranteed to have exactly one peak (mountain shape: strictly increasing then decreasing)."

Return the index of the peak in $O(\log n)$. Right?"

#

"[0,1,0] → 1. [0,2,1,0] → 1. [0,10,5,2] → 1. ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Linear scan from index 1: first i where $\text{arr}[i] > \text{arr}[i+1]$ is the peak index.

Mountain array guarantees exactly one peak; scan stops early.

$O(n)$ acceptable but binary search on the rising slope is $O(\log n)$.

```
# Time: O(n). Space: O(1).
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Binary search: at mid, compare arr[mid] with arr[mid+1].
# If arr[mid] < arr[mid+1]: we're on the ascending slope → peak is to the right →
left = mid+1.
# Else: we're on or past the peak → peak is at mid or to the left → right = mid.
# When left==right: that's the peak. O(log n)."
```

PHASE 4 — CLEAN CODE

```
class _852_PeakIndex:
    def peakIndexInMountainArray(self, arr: List[int]) -> int:
        left, right = 0, len(arr) - 1
        while left < right:
            mid = (left + right) // 2
            if arr[mid] < arr[mid + 1]:
                left = mid + 1 # still ascending – peak is further right
            else:
                right = mid # descending or at peak – search left including mid
        return left
```

PHASE 5 — TEST & DEBUG

```
# [0,1,0]: l=0,r=2,mid=1. arr[1]=1>arr[2]=0 → right=1. l=0,r=1,mid=0.
arr[0]=0<arr[1]=1 → left=1. l=r=1 → 1 ✓
# [0,10,5,2]: l=0,r=3,mid=1. arr[1]=10>arr[2]=5 → right=1. l=0,r=1,mid=0.
arr[0]<arr[1] → left=1. → 1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(log n). Space O(1).
# The comparison arr[mid] vs arr[mid+1] acts as a gradient – positive means 'go
right', negative means 'go left'.
# This is the same logic as Find Peak Element (#162) – the general version where the
array isn't guaranteed
# to be a full mountain. Mention the connection.
# Given more time I'd ask: what if there are multiple peaks?
# Same algorithm finds one peak. For all peaks, we'd need a linear scan."
```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def peakIndexInMountainArray(self, arr: list[int]) -> int:
        l = 0
        r = len(arr) - 1

        while l < r:
            m = (l + r) // 2
            if arr[m] >= arr[m + 1]:
                r = m
            else:
                l = m + 1

        return l

```

• LeetCode

```

class Solution:
    def peakIndexInMountainArray(self, arr: List[int]) -> int:
        left, right = 1, len(arr) - 2
        while left < right:
            mid = (left + right) >> 1
            if arr[mid] > arr[mid + 1]:
                right = mid
            else:
                left = mid + 1
        return left

```

• SimplyLeet

```

def peakIndexInMountainArray(arr):
    # Initialize pointers for the binary search
    left, right = 0, len(arr) - 1
    # Continue while there is more than one element in the search space
    while left < right:
        mid = (left + right) // 2 # Find the mid index
        # If the sequence is still increasing, move the left pointer to mid+1
        if arr[mid] < arr[mid + 1]:
            left = mid + 1
        else:

```

```

        # Otherwise, the peak is at mid or to the left of mid
        right = mid
    # When left and right converge, we have the peak index
    return left

# Example usage:
# print(peakIndexInMountainArray([0, 1, 0])) # Output: 1

```

• LC.ca

```

class Solution:
    def peakIndexInMountainArray(self, arr: List[int]) -> int:
        left, right = 1, len(arr) - 2
        while left < right:
            mid = (left + right) >> 1
            if arr[mid] > arr[mid + 1]:
                right = mid
            else:
                left = mid + 1
        return left

```

◆ Quick Review

Key Insights

- The array is strictly increasing up to a peak and then strictly decreasing.
- A binary search technique is applicable because the array exhibits a single transition point (peak) between increasing and decreasing order.
- By comparing mid and mid+1, we can decide which half of the array to continue the search in.

Space & Time

Time Complexity: $O(\log(n))$ – The binary search approach reduces the search interval by half at each step.

Space Complexity: $O(1)$ – Only constant extra space is used.

Solution

We use a binary search strategy: Initialize two pointers, left and right, at the beginning and end of the array. In each iteration, calculate mid. If $\text{arr}[\text{mid}]$ is less than $\text{arr}[\text{mid} + 1]$, then the peak is in the right half; otherwise, it is in the left half. When left equals right, we have found the peak index. This approach leverages the mountain array properties and ensures an $O(\log(n))$ solution.

Binary Search

□ #981 Time Based Key-Value Store

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Binary Search · Design
Lists: Grind 169

Problem

Design a time-based key-value data structure that can store multiple values for the same key at different time stamps and retrieve the key's value at a certain timestamp.

Implement the TimeMap class:

- **TimeMap()** Initializes the object of the data structure.
- **void set(String key, String value, int timestamp)** Stores the key key with the value value at the given time timestamp.
- **String get(String key, int timestamp)** Returns a value such that set was called previously, with $\text{timestamp_prev} \leq \text{timestamp}$. If there are multiple such values, it returns the value associated with the largest timestamp_prev. If

there are no values, it returns "".

Example 1:

Input

```
["TimeMap", "set", "get", "get", "set", "get", "get"]  
[[], ["foo", "bar", 1], ["foo", 1], ["foo", 3], ["foo", "bar2", 4], ["foo", 4],  
["foo", 5]]
```

Output

```
[null, null, "bar", "bar", null, "bar2", "bar2"]
```

Explanation

```
TimeMap timeMap = new TimeMap();
```

Constraints:

- $1 \leq \text{key.length}, \text{value.length} \leq 100$
- key and value consist of lowercase English letters and digits.
- $1 \leq \text{timestamp} \leq 107$
- All the timestamps timestamp of set are strictly increasing.
- At most $2 * 10^5$ calls will be made to set and get.

My LeetCode Solution

- ☐
- My LeetCode Solution

```

# Time Based Key-Value Store
class TimeMap:

    def __init__(self):
        self.store = defaultdict(list)

    def set(self, key: str, value: str, timestamp: int) -> None:
        self.store[key].append((timestamp,value))

    def get(self, key: str, timestamp: int) -> str:

        entries = self.store[key]
        ans = ""
        l = 0
        r = len(entries)-1
        while l<=r:
            m = (l+r)//2
            if entries[m][0] <= timestamp:
                ans = entries[m][1]
                l = m+1

            else:
                r=m-1

        return ans

# Your TimeMap object will be instantiated and called as such:
# obj = TimeMap()
# obj.set(key,value,timestamp)

# param_2 = obj.get(key,timestamp)

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Key-value store where each key can have multiple values at different timestamps.
 # set(key, value, ts) stores the entry. get(key, ts) returns the value with the
 # largest timestamp <= ts. Return '' if none exists. Right?
 # set timestamps are strictly increasing – so we can binary search the history
 list."

```

#
# "set('foo','bar',1). get('foo',3) → 'bar' (no ts<=3 except 1). ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Store all (timestamp, value) pairs per key in a list on set.
# get(key, ts): scan backwards for largest timestamp <= ts.
# Works but each get is O(number of entries for that key).
# Time: O(1) set, O(k) get per key history. Space: O(total entries).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Since set timestamps are strictly increasing, each key's list is already sorted
by timestamp.
# get() binary searches for the rightmost timestamp <= query timestamp.
# set: O(1). get: O(log n). Space: O(total entries)."
```

```

# PHASE 4 — CLEAN CODE
class _981_TimeMap:
    def __init__(self):
        self.store: defaultdict = defaultdict(list) # key → [(ts, value), ...]

    def set(self, key: str, value: str, timestamp: int) -> None:
        self.store[key].append((timestamp, value))

    def get(self, key: str, timestamp: int) -> str:
        entries = self.store[key]
        lo, hi = 0, len(entries) - 1
        result = ''
        while lo <= hi:
            mid = (lo + hi) // 2
            if entries[mid][0] <= timestamp:
                result = entries[mid][1] # valid candidate – try to go right for a
larger ts
            lo = mid + 1
        else:
            hi = mid - 1
        return result

# PHASE 5 — TEST & DEBUG
# set('foo','bar',1): store['foo']=[(1,'bar')].
# get('foo',1): entries=[(1,'bar')]. mid=0, ts=1<=1 → result='bar', lo=1. lo>hi →
return 'bar' ✓
# get('foo',3): mid=0, ts=1<=3 → result='bar'. lo=1>hi=0 → return 'bar' ✓
# set('foo','bar2',4): store['foo']=[(1,'bar'),(4,'bar2')].
# get('foo',4): mid=0(1<=4)→result='bar',lo=1. mid=1(4<=4)→result='bar2',lo=2.
return 'bar2' ✓
# get('foo',5): same → 'bar2' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "set O(1). get O(log n). Space O(total entries)."
```

```
# The 'find rightmost ts <= query' is a standard upper_bound - 1 binary search
pattern.
# Timestamps strictly increasing → no sorting needed – the list maintains order
automatically.
# bisect.bisect_right alternative: pos = bisect.bisect_right(entries, (timestamp,
chr(127))) - 1
# (appending chr(127) to go past any value at that exact timestamp).
# Given more time I'd ask: what if set timestamps aren't guaranteed to be
increasing?
# We'd need to sort the list on insert or use a sorted data structure."
```

Community Solutions (Python)

• WalkCC

```
class TimeMap:
    def __init__(self):
        self.values = collections.defaultdict(list)
        self.timestamps = collections.defaultdict(list)

    def set(self, key: str, value: str, timestamp: int) -> None:
        self.values[key].append(value)
        self.timestamps[key].append(timestamp)

    def get(self, key: str, timestamp: int) -> str:

        if key not in self.timestamps:
            return ''
        i = bisect.bisect(self.timestamps[key], timestamp)
        return self.values[key][i - 1] if i > 0 else ''
```

• LeetDoocs

```
class TimeMap:
    def __init__(self):
        self.ktv = defaultdict(list)

    def set(self, key: str, value: str, timestamp: int) -> None:
        self.ktv[key].append((timestamp, value))

    def get(self, key: str, timestamp: int) -> str:
        if key not in self.ktv:
            return ''
```

```

    tv = self.ktv[key]
    i = bisect_right(tv, (timestamp, chr(127)))
    return tv[i - 1][1] if i else ''

```

```

# Your TimeMap object will be instantiated and called as such:
# obj = TimeMap()
# obj.set(key,value,timestamp)
# param_2 = obj.get(key,timestamp)

```

• SimplyLeet

```

# TimeMap class in Python with line-by-line comments.
class TimeMap:
    def __init__(self):
        # Initialize the dictionary to hold key and list of (timestamp, value)
        # pairs.
        self.store = {}

    def set(self, key: str, value: str, timestamp: int) -> None:
        # Append the new (timestamp, value) pair for the given key.
        if key not in self.store:
            self.store[key] = []

        self.store[key].append((timestamp, value))

    def get(self, key: str, timestamp: int) -> str:
        # Return the value for the largest timestamp <= the given timestamp.
        if key not in self.store:
            return ""

        values = self.store[key]
        left, right = 0, len(values) - 1

```



```
# Binary search for the rightmost index with timestamp <= given
timestamp.
```

```
while left <= right:
    mid = (left + right) // 2
    if values[mid][0] <= timestamp:
        left = mid + 1
    else:
        right = mid - 1
```

```
# If no valid timestamp exists, right will be -1.
if right == -1:
```

```
    return ""
    return values[right][1]
```

```
# Example usage:
# timeMap = TimeMap()
# timeMap.set("foo", "bar", 1)
# print(timeMap.get("foo", 1)) # Output: "bar"
# print(timeMap.get("foo", 3)) # Output: "bar"
# timeMap.set("foo", "bar2", 4)
# print(timeMap.get("foo", 4)) # Output: "bar2"

# print(timeMap.get("foo", 5)) # Output: "bar2"
```

• LC.ca

```
class TimeMap:
    def __init__(self):
        self.ktv = defaultdict(list)

    def set(self, key: str, value: str, timestamp: int) -> None:
        self.ktv[key].append((timestamp, value))

    def get(self, key: str, timestamp: int) -> str:
        if key not in self.ktv:
            return ''
```

```
tv = self.ktv[key]
i = bisect_right(tv, (timestamp, chr(127)))
return tv[i - 1][1] if i else ''

# Your TimeMap object will be instantiated and called as such:
# obj = TimeMap()
# obj.set(key,value,timestamp)
# param_2 = obj.get(key,timestamp)
```

◆ Quick Review

Key Insights

- Use a dictionary (hash table) to map each key to a list of (timestamp, value) pairs.
- Timestamps are strictly increasing when setting the values, which allows efficient appending.
- For getting the value, perform a binary search on the list of timestamps to find the largest timestamp that is $\leq$ the queried timestamp.
- If no valid timestamp exists, return an empty string.

Space & Time

Time Complexity:

set operation: $O(1)$ per call (amortized).

get operation: $O(\log N)$ per call, where N is the number of timestamps stored for the key.

Space Complexity: $O(\text{total number of set calls})$, to store all (timestamp, value) pairs.

Solution

We use a hash table to store keys mapping to a list of their timestamps and corresponding values. Since the timestamps are inserted in strictly increasing order, the list remains sorted. For the get operation, a binary search is applied to quickly locate the appropriate value for a given timestamp. This approach minimizes the lookup time and efficiently handles the problem constraints.

Binary Search

□ #1011 Capacity to Ship Packages Within D Days

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: Denny Zhang

Problem

A conveyor belt has packages that must be shipped from one port to another within days days.

The i th package on the conveyor belt has a weight of $\text{weights}[i]$. Each day, we load the ship with packages on the conveyor belt (in the order given by weights). We may not load more weight than the maximum weight capacity of the ship. Return the least weight capacity of the ship that will result in all the packages on the conveyor belt being shipped within days days.

Example 1:

Input: weights = [1,2,3,4,5,6,7,8,9,10], days = 5

Output: 15

Explanation: A ship capacity of 15 is the minimum to ship all the packages in 5 days like this:

1st day: 1, 2, 3, 4, 5

2nd day: 6, 7

3rd day: 8

4th day: 9

5th day: 10

Example 2:

Input: weights = [3,2,2,4,1,4], days = 3

Output: 6

Explanation: A ship capacity of 6 is the minimum to ship all the packages in 3 days like this:

1st day: 3, 2

2nd day: 2, 4

3rd day: 1, 4

Example 3:

Input: weights = [1,2,3,1,1], days = 4

Output: 3

Explanation:

1st day: 1

2nd day: 2

3rd day: 3

4th day: 1, 1

Constraints:

- $1 \leq \text{days} \leq \text{weights.length} \leq 5 * 10^4$
- $1 \leq \text{weights}[i] \leq 500$

My LeetCode Solution

- My LeetCode Solution

```

# Capacity to Ship Packages Within D Days
class Solution:
    def shipWithinDays(self, weights: List[int], days: int) -> int:
        def canship(capacity):
            used = 1
            load = 0
            for x in weights:
                if load + x > capacity:
                    used += 1
                    load = 0

            load += x

            return used <= days

        l = max(weights)
        r = sum(weights)

        while l <= r:
            m = (l+r)//2
            if canship(m):
                r = m-1

            else:
                l=m+1

        return l

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Find the minimum ship capacity that lets us ship all packages in at most D days.
# Packages must be shipped in order – we can't reorder them. Right?
# Capacity must be at least max(weights) (to handle any single package).
# Capacity at most sum(weights) (ship everything in one day)."
#
# "weights=[1..10], days=5 → 15 ✓ (greedy daily assignment: 1-5, 6-7, 8, 9, 10)"

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with brute force to lock in the logic.
# Try every candidate ship capacity from max(weights) up to sum(weights).
# For each capacity, greedily pack days and count days needed.
# Correct but linear search on capacity range is  $O(\text{sum} * n)$ .
# Time:  $O(\text{sum}(\text{weights}) * n)$ . Space:  $O(1)$ .
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Binary search on the answer (capacity).
# The feasibility function 'can_ship(capacity)' is monotone: if C works, C+1 also
works.
# So binary search for the leftmost capacity where can_ship returns True.
# can_ship: greedily fill each day until the load would exceed capacity, then start
a new day.
# Count days needed. Return days  $\leq D$ .
# Time:  $O(n \log(\text{sum}(\text{weights})))$ . Space:  $O(1)$ ."

# PHASE 4 — CLEAN CODE
class _1011_CapacityShip:
    def shipWithinDays(self, weights: List[int], days: int) -> int:
        def can_ship(capacity: int) -> bool:
            day_load = 0
            days_used = 1
            for w in weights:
                if day_load + w > capacity:
                    days_used += 1
                    day_load = 0
                day_load += w
            return days_used <= days
        lo, hi = max(weights), sum(weights)
        while lo < hi:
            mid = (lo + hi) // 2
            if can_ship(mid):
                hi = mid # valid - try smaller
            else:
                lo = mid + 1 # not valid - need more capacity
        return lo

# PHASE 5 — TEST & DEBUG
# weights=[1..10], days=5. lo=10, hi=55.
# mid=32: can_ship(32)→1+2+3+4+5=15≤32(day1), 6+7=13≤32(day2), 8≤32(day3),
9≤32(day4), 10≤32(day5). 5 days ✓ → hi=32.
# ... binary search converges to 15.
# can_ship(15): 1+2+3+4+5=15(day1), 6+7=13+8>15→day2=6+7, 8→day3, 9→day4, 10→day5.
5 days ✓. → 15 ✓
# can_ship(14): 1+2+3+4=10, +5=15>14→day2=5+6=11, +7>14→day3=7+8>14→day4=8, 9→day5,
10→day6. 6>5 ✗.

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(n \log(\text{sum}))$ . Space  $O(1)$ ."

```


This is 'binary search on the answer' – a powerful pattern where the answer space is
 # a range of integers and feasibility is monotone.
 # The feasibility check is a greedy simulation – always load greedily to avoid underusing capacity.
 # Same pattern appears in: Koko Eating Bananas (#875), Split Array Largest Sum (#410).
 # Given more time I'd ask: what if packages can be reordered?
 # Then we'd try to optimise placement – different problem, potentially NP-hard."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def shipWithinDays(self, weights: list[int], days: int) -> int:
        def shipDays(shipCapacity: int) -> int:
            shipDays = 1
            capacity = 0
            for weight in weights:
                if capacity + weight > shipCapacity:
                    shipDays += 1
                    capacity = weight
            else:
                capacity += weight
            return shipDays

        l = max(weights)
        r = sum(weights)
        return bisect.bisect_left(range(l, r), True,
                                   key=lambda m: shipDays(m) <= days) + l
```

• LeetDoocs

```

class Solution:
    def shipWithinDays(self, weights: List[int], days: int) -> int:
        def check(mx):
            ws, cnt = 0, 1
            for w in weights:
                ws += w
                if ws > mx:
                    cnt += 1
                    ws = w
            return cnt <= days

        left, right = max(weights), sum(weights) + 1
        return left + bisect_left(range(left, right), True, key=check)

```

• SimplyLeet

```

def shipWithinDays(weights, days):
    # Helper function to determine if a given capacity can ship packages within
    # 'days' days.
    def canShip(capacity):
        current_load = 0 # Current total weight on the ship for the current
        day.
        required_days = 1 # Start with the first day.
        for weight in weights:
            # If adding the current weight exceeds capacity, increment the day
            counter.
            if current_load + weight > capacity:
                required_days += 1
                current_load = 0

        current_load += weight
        return required_days <= days

    # Establish lower and upper bounds for binary search.
    low, high = max(weights), sum(weights)
    while low < high:
        mid = (low + high) // 2
        # If mid capacity can ship all packages within the allowed days, try a
        lower capacity.
        if canShip(mid):
            high = mid

```

```

        else:
            low = mid + 1
        return low

# Example usage:
weights = [1,2,3,4,5,6,7,8,9,10]
days = 5
print(shipWithinDays(weights, days))

```

• LC.ca

```

class Solution:
    def shipWithinDays(self, weights: List[int], days: int) -> int:
        def check(mx):
            ws, cnt = 0, 1
            for w in weights:
                ws += w
                if ws > mx:
                    cnt += 1
                    ws = w
            return cnt <= days

        left, right = max(weights), sum(weights) + 1
        return left + bisect_left(range(left, right), True, key=check)

```

◆ Quick Review

Key Insights

- The minimal possible capacity is the maximum value in weights; no ship can carry a package heavier than its capacity.
- The maximum possible capacity is the sum of all weights; this allows shipping all packages in one day.

- The problem is monotonic: if a capacity C works, any capacity greater than C will also work.
- Use binary search on the capacity range and a greedy approach to simulate shipping for each candidate capacity.

Space & Time

Time Complexity: $O(n * \log(\text{sum}(\text{weights})))$
where n is the number of packages.

Space Complexity: $O(1)$ additional space.

Solution

The solution employs binary search to find the minimum ship capacity. Start by setting the search bounds: the lower bound is the maximum weight (since each package must be shipped individually if needed) and the upper bound is the sum of all weights (shipping everything in one go). For each midpoint capacity in the binary search, use a greedy method to simulate the shipping process—accumulating weights until adding another package would exceed the candidate ship capacity, at which point you increment the day count. If the number of days required

exceeds D , the capacity is insufficient and the lower bound is increased; otherwise, adjust the upper bound. Continue until the bounds converge.

Binary Search

□ ★ #1060 Missing Element in Sorted Array **MED**

★

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search

Lists: ★ Premium | Premium 98

Problem

Given an integer array `nums` which is sorted in ascending order and all of its elements are unique and given also an integer `k`, return the `k`th missing number starting from the leftmost number of the array.

Example 1:

```
Input: nums = [4,7,9,10], k = 1
Output: 5
Explanation: The first missing number is 5.
```

Example 2:

```
Input: nums = [4,7,9,10], k = 3
Output: 8
Explanation: The missing numbers are [5,6,8,...], hence the third missing number is 8.
```

Example 3:

Input: nums = [1,2,4], k = 3

Output: 6

Explanation: The missing numbers are [3,5,6,7,...], hence the third missing number is 6.

Constraints:

- $1 \leq \text{nums.length} \leq 5 * 10^4$
- $1 \leq \text{nums}[i] \leq 10^7$
- nums is sorted in ascending order, and all the elements are unique.
- $1 \leq k \leq 10^8$

Follow up: Can you find a logarithmic time complexity (i.e., $O(\log(n))$) solution?

My LeetCode Solution

• My LeetCode Solution

```
# Missing Element in Sorted Array
class Solution:
    def missingElement(self, nums: List[int], k: int) -> int:
        def nmi(i):
            return nums[i]-nums[0]-i

        l=0
        r = len(nums)-1
        allmissing = nmi(len(nums)-1)
        if k> allmissing:
```



```

        return nums[len(nums)-1] + (k-allmissing)

    while l<=r:
        m = (l+r)//2
        if nmi(m) < k:
            l=m+1

        else:
            r = m-1

    missbeforel= nmi(l-1)
    return nums[l-1] +(k-missbeforel)

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Sorted unique integers. Find the k-th missing integer starting from nums[0].
# Missing count at index i = (nums[i] - nums[0]) - i (how many numbers should exist
but don't).
# Follow-up asks for O(log n). Right?"
#
# "[4,7,9,10], k=1: missing = [5,6,8,...]. 1st missing = 5 ✓
# [4,7,9,10], k=3: [5,6,8,...]. 3rd = 8 ✓"

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with brute force to lock in the logic.
# Walk nums in order maintaining expected next integer starting at 0.
# Increment expected whenever nums[i] == expected; else expected stays.
# When expected reaches k, return k - correct but scans whole array.
# Time: O(n). Space: O(1).
# Should I go ahead and optimize?"

```

```

class _1060_MissingElement_Brute:
    def missingElement(self, nums: List[int], k: int) -> int:
        for i in range(1, len(nums)):
            gap = nums[i] - nums[i - 1] - 1 # missing numbers between nums[i-1] and
nums[i]

            if k <= gap:
                return nums[i - 1] + k
            k -= gap
        return nums[-1] + k # k-th missing is after the array

```

PHASE 3 — OPTIMIZE (O(log n))

```
# "Define missing(i) = numbers missing from nums[0] to nums[i] = (nums[i] - nums[0])
- i.
# Binary search for the rightmost index where missing(i) < k.
# Answer = nums[right] + (k - missing(right)).
# If k > missing(last), answer is past the end of the array: nums[-1] + (k -
missing(n-1))."
```

PHASE 4 — CLEAN CODE

```
class _1060_MissingElement:
    def missingElement(self, nums: List[int], k: int) -> int:
        def missing(i: int) -> int:
            return (nums[i] - nums[0]) - i

        n = len(nums)
        if k > missing(n - 1):
            return nums[-1] + (k - missing(n - 1))

        lo, hi = 0, n - 1
        while lo < hi:
            mid = (lo + hi) // 2
            if missing(mid) < k:
                lo = mid + 1 # not enough missing yet - go right
            else:
                hi = mid # too many or just enough - go left
        return nums[lo - 1] + (k - missing(lo - 1))
```

PHASE 5 — TEST & DEBUG

```
# nums=[4,7,9,10], k=1:
# missing: [0]=0, [1]=2, [2]=3, [3]=4.
# k=1<=missing(3)=4. Binary search: lo=0,hi=3,mid=1. missing(1)=2>=1→hi=1.
# lo=0,hi=1,mid=0. missing(0)=0<1→lo=1. lo=hi=1.
# return nums[0]+(1-missing(0))=4+(1-0)=5 ✓
# nums=[4,7,9,10], k=3:
# missing(3)=4>=3. Binary search → lo lands at 2. missing(1)=2<3→lo=2. lo=hi=2.
# return nums[1]+(3-missing(1))=7+(3-2)=8 ✓
# k=5: missing(3)=4<5 → return nums[3]+(5-4)=10+1=11 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "O(log n) with binary search. O(1) space.
# The missing(i) formula is the key: (nums[i]-nums[0])-i counts how many integers
# in [nums[0], nums[i]] are absent from the array.
# The past-the-end check handles k > total missing numbers in the array gracefully.
# Given more time I'd ask: what if there are duplicate values in nums?
# missing(i) would overcount - we'd need to adjust for duplicates."
```

Community Solutions (Python)

• LeetDoocs

```

class Solution:
    def missingElement(self, nums: List[int], k: int) -> int:
        def missing(i: int) -> int:
            return nums[i] - nums[0] - i

        n = len(nums)
        if k > missing(n - 1):
            return nums[n - 1] + k - missing(n - 1)
        l, r = 0, n - 1
        while l < r:

            mid = (l + r) >> 1
            if missing(mid) >= k:
                r = mid
            else:
                l = mid + 1
        return nums[l - 1] + k - missing(l - 1)

```

• SimplyLeet

Python solution with comprehensive comments

```

def missingElement(nums, k):
    # Calculate the missing count until index i: missing(i) = nums[i] - nums[0]
    # - i.
    def missing_count(index):
        return nums[index] - nums[0] - index

    n = len(nums)

    # If kth missing number is beyond the last element.

    if k > missing_count(n - 1):
        return nums[-1] + k - missing_count(n - 1)

    # Perform binary search to find the smallest index where
    missing_count(index) >= k.
    left, right = 0, n - 1
    while left < right:
        mid = left + (right - left) // 2
        if missing_count(mid) < k:
            left = mid + 1
        else:

```

```

        right = mid

    # After binary search, left is the smallest index such that
    missing_count(left) >= k.
    # The kth missing number is after nums[left - 1].
    return nums[left - 1] + k - missing_count(left - 1)

# Example usage:
print(missingElement([4,7,9,10], 3)) # Output: 8

```

• LC.ca

```

class Solution:
    def missingElement(self, nums: List[int], k: int) -> int:
        def missing(i: int) -> int:
            return nums[i] - nums[0] - i

        n = len(nums)
        if k > missing(n - 1):
            return nums[n - 1] + k - missing(n - 1)
        l, r = 0, n - 1
        while l < r:
            mid = (l + r) >> 1
            if missing(mid) >= k:
                r = mid
            else:
                l = mid + 1
        return nums[l - 1] + k - missing(l - 1)

```

◆ Quick Review

Key Insights

- The array is sorted and every element is unique.
- For any index i , the number of missing numbers up to that point can be computed as

$\text{nums}[i] - \text{nums}[0] - i$.

- When $\text{missing}(i)$ is less than k , the k th missing element is further to the right.
- Use binary search to find the smallest index where the count of missing numbers is greater than or equal to k .
- If all missing numbers before the last array element are counted and the k th missing still hasn't been reached, calculate the result by continuing past the last element.

Space & Time

Time Complexity: $O(\log(n))$

Space Complexity: $O(1)$

Solution

We compute the number of missing elements until an index i with the formula: $\text{missing}(i) = \text{nums}[i] - \text{nums}[0] - i$. We then use binary search over the indices to find the smallest index where $\text{missing}(i) \geq k$. If the binary search index equals the length of the array, it means the k th missing number lies beyond the last element. Otherwise, the k th missing number is derived as: $\text{nums}[\text{index} - 1] + (k - \text{missing}(\text{index} - 1))$. This approach uses binary

search for logarithmic time and does not require extra auxiliary data structures.

Binary Search

□ #1062 Longest Repeating Substring

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Binary Search · Dynamic Programming
· Rolling Hash · Suffix Array · Hash Function
Lists: Denny Zhang

Problem

Given a string *s*, return the length of the longest repeating substrings. If no repeating substring exists, return 0.

Example 1:

Input: *s* = "abcd"
Output: 0
Explanation: There is no repeating substring.

Example 2:

Input: *s* = "abbaba"
Output: 2
Explanation: The longest repeating substrings are "ab" and "ba", each of which occurs twice.

Example 3:

Input: *s* = "aabcaabdaab"
Output: 3
Explanation: The longest repeating substring is "aab", which occurs 3 times.

Constraints:

- $1 \leq s.length \leq 2000$
- s consists of lowercase English letters.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def longestRepeatingSubstring(self, s: str) -> int:
        n = len(s)
        left, right = 0, n - 1

        while left <= right:
            mid = left + (right - left) // 2

            if self.findRepeatingSubstring(s, mid):
                left = mid + 1

            else:
                right = mid - 1

        return right

    def findRepeatingSubstring(self, s: str, length: int) -> bool:
        seen = set()

        for i in range(len(s) - length + 1):
            sub = s[i:i + length]

            if sub in seen:
                return True

            seen.add(sub)

        return False
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Find the length of the longest substring that appears at least twice in s.
# The two occurrences can overlap. Return 0 if no repeating substring. Right?"
#
# "'abcd'→0. 'abbaba'→2 ('ab','ba'). 'aabcaabdaab'→3 ('aab')."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# For each length L from n-1 down to 1, collect all substrings of length L in a set.
# First L where a duplicate appears is the answer.
# O(n^2) substrings per L – binary search on L with rolling hash helps.
# Time: O(n^2) to O(n^3) naive. Space: O(n^2) set.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (DP approach — simpler to explain)

```
# "DP: dp[i][j] = length of longest common suffix of s[:i] and s[:j], where i ≠ j.
# dp[i][j] = dp[i-1][j-1] + 1 if s[i-1]==s[j-1] and i≠j, else 0.
# Track the global max. O(n^2) time, O(n^2) space."
```

PHASE 4 — CLEAN CODE

```
class _1062_LongestRepeatingSubstring:
    def longestRepeatingSubstring(self, s: str) -> int:
        n = len(s)
        dp = [[0] * (n + 1) for _ in range(n + 1)]
        best = 0
        for i in range(1, n + 1):
            for j in range(i + 1, n + 1): # j > i ensures different starting
positions
                if s[i - 1] == s[j - 1]:
                    dp[i][j] = dp[i - 1][j - 1] + 1
                    best = max(best, dp[i][j])
        return best
```

PHASE 5 — TEST & DEBUG

```
# s='abbaba':
# i=1,j=2: 'a'=='b'? No. ... i=1,j=4: 'a'=='a'? Yes→dp[1][4]=1, best=1.
# i=2,j=5: 'b'=='b'? Yes→dp[2][5]=dp[1][4]+1=2, best=2.
# → 2 ✓ (substring 'ab' starting at 0 and 3)
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n^2). Space O(n^2). Acceptable for n<=2000 (4M cells).
# Binary search + hashing alternative: O(n log n) time, O(n) space – faster but more
complex.
# Given more time I'd ask: return the actual substring, not just the length?
# Track (i,j) when best is updated, then s[i-best:i]."
```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def longestRepeatingSubstring(self, s: str) -> int:
        n = len(s)
        ans = 0
        # dp[i][j] := the number of repeating characters of s[0..i) and s[0..j)
        dp = [[0] * (n + 1) for _ in range(n + 1)]

        for i in range(1, n + 1):
            for j in range(i + 1, n + 1):
                if s[i - 1] == s[j - 1]:

                    dp[i][j] = 1 + dp[i - 1][j - 1]
                    ans = max(ans, dp[i][j])

        return ans

```

• LeetDoocs

```

class Solution:
    def longestRepeatingSubstring(self, s: str) -> int:
        n = len(s)
        f = [[0] * n for _ in range(n)]
        ans = 0
        for i in range(1, n):
            for j in range(i):
                if s[i] == s[j]:
                    f[i][j] = 1 + (f[i - 1][j - 1] if j else 0)
                    ans = max(ans, f[i][j])

        return ans

```

• SimplyLeet

```
def longestRepeatingSubstring(s):
    n = len(s)
    # Convert characters to integers for rolling hash computation.
    nums = [ord(c) - ord('a') for c in s]

    # Base value for rolling hash.
    a = 26
    # Modulus value to avoid overflow.
    mod = 2**32

    # Helper function to check if any substring of given length L repeats.
    def search(L):
        h = 0
        for i in range(L):
            h = (h * a + nums[i]) % mod
        seen = {h}
        # Precompute a^L % mod for removing the first digit in the rolling
        hash.
        aL = pow(a, L, mod)
        for start in range(1, n - L + 1):
            h = (h * a - nums[start - 1] * aL + nums[start + L - 1]) % mod

            if h in seen:
                return True
            seen.add(h)
        return False

    left, right = 1, n
    ans = 0
    while left <= right:
        mid = left + (right - left) // 2
        if search(mid):

            ans = mid
            left = mid + 1
        else:
            right = mid - 1
    return ans

# Example usage:
s = "aabcaabdaab"
print(longestRepeatingSubstring(s)) # Expected output: 3
```

• LC.ca

```
class Solution:
    def longestRepeatingSubstring(self, s: str) -> int:
        n = len(s)
        dp = [[0] * n for _ in range(n)]
        ans = 0
        for i in range(n):
            for j in range(i + 1, n):
                if s[i] == s[j]:
                    dp[i][j] = dp[i - 1][j - 1] + 1 if i else 1
                    ans = max(ans, dp[i][j])

        return ans
```

◆ Quick Review

Key Insights

- Use binary search to efficiently check for the existence of a repeating substring of a given length.
- Apply a rolling hash technique to compute hash values for substrings in $O(1)$ time after initial processing.
- Use a hash set to detect duplicate hash values (indicating a repeating substring).
- Adjust binary search bounds based on whether a repeating substring of the current length exists.
- Alternative approaches include dynamic programming and suffix arrays, but the binary

search with rolling hash is efficient and conceptually clean.

Space & Time

Time Complexity: $O(n \log n)$ on average, where n is the length of the string. (Each binary search step takes $O(n)$ time for computing rolling hashes.)

Space Complexity: $O(n)$ for storing hash values in the hash set.

Solution

The solution uses binary search to decide on the candidate length L of the repeating substring. For each candidate length L , we use a rolling hash to compute hash values for all substrings of length L and store them in a hash set. If any hash is repeated, it confirms the presence of a repeating substring of length L , and we try a longer length.

Otherwise, we decrease the candidate length. The rolling hash is computed using a base (26, for lowercase letters) and a modulus (2^{32} to limit integer overflow) which allows efficient hash recomputation for sliding windows. This approach efficiently narrows down the

maximum repeating substring length.

Binary Search

□ ★ #1533 Find the Index of the Large Integer ★

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Interactive
Lists: ★ Premium | Premium 98

Problem

We have an integer array `arr`, where all the integers in `arr` are equal except for one integer which is larger than the rest of the integers. You will not be given direct access to the array, instead, you will have an API `ArrayReader` which have the following functions:

- `int compareSub(int l, int r, int x, int y)`: where $0 \leq l, r, x, y < \text{ArrayReader.length()}$, $l \leq r$ and $x \leq y$. The function compares the sum of sub-array `arr[l..r]` with the sum of the sub-array `arr[x..y]` and returns: 1 if $\text{arr}[l] + \text{arr}[l+1] + \dots + \text{arr}[r] > \text{arr}[x] + \text{arr}[x+1] + \dots + \text{arr}[y]$.

- 0 if $\text{arr}[l] + \text{arr}[l+1] + \dots + \text{arr}[r] == \text{arr}[x] + \text{arr}[x+1] + \dots + \text{arr}[y]$.
- -1 if $\text{arr}[l] + \text{arr}[l+1] + \dots + \text{arr}[r] < \text{arr}[x] + \text{arr}[x+1] + \dots + \text{arr}[y]$.

You are allowed to call `compareSub()` 20 times at most. You can assume both functions work in $O(1)$ time.

Return the index of the array `arr` which has the largest integer.

Example 1:

Input: `arr = [7,7,7,7,10,7,7,7]`

Output: 4

Explanation: The following calls to the API

`reader.compareSub(0, 0, 1, 1)` // returns 0 this is a query comparing the sub-array (0, 0) with the sub array (1, 1), (i.e. compares `arr[0]` with `arr[1]`). Thus we know that `arr[0]` and `arr[1]` doesn't contain the largest element.

`reader.compareSub(2, 2, 3, 3)` // returns 0, we can exclude `arr[2]` and `arr[3]`.

`reader.compareSub(4, 4, 5, 5)` // returns 1, thus for sure `arr[4]` is the largest element in the array.

Notice that we made only 3 calls, so the answer is valid.

Example 2:

Input: `nums = [6,6,12]`

Output: 2

Constraints:

- $2 \leq \text{arr.length} \leq 5 * 10^5$
- $1 \leq \text{arr}[i] \leq 100$
- All elements of `arr` are equal except for one element which is larger than all other elements.

Follow up:

- What if there are two numbers in arr that are bigger than all other numbers?
- What if there is one number that is bigger than other numbers and one number that is smaller than other numbers?

My LeetCode Solution

• My LeetCode Solution

```
# """
# This is ArrayReader's API interface.
# You should not implement it, or speculate about its implementation
# """
#class ArrayReader(object):
# # Compares the sum of arr[l..r] with the sum of arr[x..y]
# # return 1 if sum(arr[l..r]) > sum(arr[x..y])
# # return 0 if sum(arr[l..r]) == sum(arr[x..y])
# # return -1 if sum(arr[l..r]) < sum(arr[x..y])
# def compareSub(self, l: int, r: int, x: int, y: int) -> int:

#
# # Returns the length of the array
# def length(self) -> int:
#

class Solution:
    def getIndex(self, reader: 'ArrayReader') -> int:
        l = 0
        n = reader.length()
```

```

    r = n-1

    while l<=r:
        if l == r:
            return l

        m = (l+r)//2
        size = (r-l+1)
        if size % 2 == 1:
            res = reader.compareSub(l, m-1, m+1, r)

            if res == 0:
                return m

            if res == -1:
                l = m+1
            else:
                r = m-1

        else:
            res = reader.compareSub(l, m, m+1, r)

            if res == -1:
                l = m+1
            else:
                r = m

    return -1

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"One element is strictly larger than all others. Find its index using the compareSub API.

compareSub(l,r,x,y) compares sum(arr[l..r]) with sum(arr[x..y]): returns 1,-1,0.

At most 20 calls – so $O(\log n)$ calls are required. Right?"

#

"[7,7,7,7,10,7,7,7]: 3 compareSub calls suffice to find index 4 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Linear scan: compare nums[i] with nums[i+1]; first unequal pair reveals larger half.

Correct $O(n)$ but problem expects $O(\log n)$ via binary search on the doubled array.

```
# Time: O(n). Space: O(1).
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Binary search: at each step, compare the left half against the right half of the
range.
# If left > right: large integer is in the left half → search left.
# If right > left: search right.
# If equal: the large integer is the MIDDLE element (if odd-length range, middle was
excluded).
# Halve the range each step → O(log n) calls."
```

PHASE 4 — CLEAN CODE

```
class _1533_FindLargeInteger:
    def getIndex(self, reader) -> int:
        lo, hi = 0, reader.length() - 1
        while lo < hi:
            mid = (lo + hi) // 2
            # Compare left half [lo..mid] with right half of equal size
            left_end = mid
            right_start = mid + 1
            right_end = right_start + (mid - lo)
            if right_end > hi:
                right_end = hi
            result = reader.compareSub(lo, left_end, right_start, right_end)
            if result == 1:
                hi = left_end # large int is in left half
            elif result == -1:
                lo = right_start # large int is in right half
            else:
                return mid + 1 # equal sums → large int is the middle skipped
        element
        return lo
```

PHASE 5 — TEST & DEBUG

```
# arr=[7,7,7,7,10,7,7,7]: lo=0,hi=7.
# mid=3. Compare [0..3] vs [4..7]. Both have 4 elements. Sum=[28] vs [31]. -1 →
lo=4.
# lo=4,hi=7,mid=5. Compare [4..5] vs [6..7]. [10+7=17] vs [7+7=14]. 1 → hi=5.
# lo=4,hi=5,mid=4. Compare [4..4] vs [5..5]. [10] vs [7]. 1 → hi=4. lo=hi=4 → return
4 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "O(log n) compareSub calls. Well within the 20-call limit for n=5*10^5
(log2(500000)≈19).
# The equal-sum case (odd-length range) means the middle element was the large one –
# both halves have equal elements so the excluded middle is the outlier.
# Follow-up: two large numbers? One comparison no longer definitively points to one
half.
# We'd need to track which half has MORE excess – more complex."
```

Community Solutions (Python)

• WalkCC

```
# """
# This is ArrayReader's API interface.
# You should not implement it, or speculate about its implementation
# """
# class ArrayReader(object):
#     # Compares the sum of arr[l..r] with the sum of arr[x..y]
#     # return 1 if sum(arr[l..r]) > sum(arr[x..y])
#     # return 0 if sum(arr[l..r]) == sum(arr[x..y])
#     # return -1 if sum(arr[l..r]) < sum(arr[x..y])
#     def compareSub(self, l: int, r: int, x: int, y: int) -> int:

#
# # Returns the length of the array
# def length(self) -> int:
#

class Solution:
    def getIndex(self, reader: 'ArrayReader') -> int:
        l = 0
        r = reader.length() - 1

        while l < r:
            m = (l + r) // 2
            if (r - l) % 2 == 0:
                res = reader.compareSub(l, m - 1, m + 1, r)
                if res == 0:
                    return m
                if res == 1:
                    r = m - 1
            else: # res == -1
```

```

        l = m + 1
    else:
        res = reader.compareSub(l, m, m + 1, r)
        # res is either 1 or -1.
        if res == 1:
            r = m
        else: # res == -1
            l = m + 1

    return l

```

• LeetDoocs

```

# """
# This is ArrayReader's API interface.
# You should not implement it, or speculate about its implementation
# """
# class ArrayReader(object):
#     # Compares the sum of arr[l..r] with the sum of arr[x..y]
#     # return 1 if sum(arr[l..r]) > sum(arr[x..y])
#     # return 0 if sum(arr[l..r]) == sum(arr[x..y])
#     # return -1 if sum(arr[l..r]) < sum(arr[x..y])
#     # def compareSub(self, l: int, r: int, x: int, y: int) -> int:

#
# # Returns the length of the array
# # def length(self) -> int:
#
#
class Solution:
    def getIndex(self, reader: 'ArrayReader') -> int:
        left, right = 0, reader.length() - 1
        while left < right:

```

```

        t1, t2, t3 = (
            left,
            left + (right - left) // 3,
            left + ((right - left) // 3) * 2 + 1,
        )
        cmp = reader.compareSub(t1, t2, t2 + 1, t3)
        if cmp == 0:
            left = t3 + 1
        elif cmp == 1:
            right = t2
    else:
        left, right = t2 + 1, t3
    return left

```

• SimplyLeet

Python solution with line-by-line comments

class Solution:

def getIndex(self, reader):

Initialize search bounds

left, right = 0, reader.length() - 1

Binary search for the candidate index

while left < right:

length = right - left + 1

if length % 2 == 0:

Even length: split into two equal halves.

mid = left + length // 2

Compare left half [left, mid-1] with right half [mid, right]

comp = reader.compareSub(left, mid - 1, mid, right)

if comp == 1:

Large integer is in the left half

right = mid - 1

else:

Large integer is in the right half

left = mid


```

        else:
            # Odd length: divide into two halves of equal size, ignoring
            middle element.
            half = length // 2
            # Compare left half [left, left+half-1] with right half
            [right-half+1, right]
            comp = reader.compareSub(left, left + half - 1, right - half +
            1, right)

            if comp == 1:
                # Large integer in left half
                right = left + half - 1
            elif comp == -1:
                # Large integer in right half

                left = right - half + 1
            else:
                # Halves are equal, so the middle element is the large one.
                return left + half

    return left

```

• LC.ca

```

# """
# This is ArrayReader's API interface.
# You should not implement it, or speculate about its implementation
# """
# class ArrayReader(object):
#     # Compares the sum of arr[l..r] with the sum of arr[x..y]
#     # return 1 if sum(arr[l..r]) > sum(arr[x..y])
#     # return 0 if sum(arr[l..r]) == sum(arr[x..y])
#     # return -1 if sum(arr[l..r]) < sum(arr[x..y])
#     def compareSub(self, l: int, r: int, x: int, y: int) -> int:

#
#     # Returns the length of the array
#     def length(self) -> int:
#
#
class Solution:
    def getIndex(self, reader: 'ArrayReader') -> int:
        left, right = 0, reader.length() - 1
        while left < right:

```

```
t1, t2, t3 = (
    left,
    left + (right - left) // 3,
    left + ((right - left) // 3) * 2 + 1,
)
cmp = reader.compareSub(t1, t2, t2 + 1, t3)
if cmp == 0:
    left = t3 + 1
elif cmp == 1:
    right = t2
else:
    left, right = t2 + 1, t3
return left
```

◆ Quick Review

Key Insights

- Use binary search to efficiently narrow down the position of the unique large value.
- At each step, divide the candidate range into two halves (or almost two halves when the range size is odd) and use `compareSub` to determine which half contains the large integer.
- Comparing two subarrays of equal length will reveal the region that contains the extra value via the sum difference.
- Handle odd-length ranges carefully by isolating the middle element if needed.

Space & Time

Time Complexity: $O(\log n)$ – because binary search is performed over the range.

Space Complexity: $O(1)$ – only a few pointers are maintained, no additional space is required.

Solution

The solution applies a binary search approach over the array indices. At every iteration: Calculate the current segment length. If the length is even, split the segment evenly and compare the left half versus the right half using `compareSub`. If the length is odd, compare two halves of equal size while ignoring the middle element. If the sums are equal, then the middle index holds the large value. Update the search bounds based on the result of `compareSub`. Continue until the search space is reduced to one index, which is the answer. This method ensures that you locate the large integer with $O(\log n)$ calls to `compareSub`, well within the limit of 20 calls.

Matrix

Round 2 · High · Medium · 14 questions

Matrix

□ #36 Valid Sudoku

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Matrix
Lists: Grind 169

Problem

Determine if a 9 x 9 Sudoku board is valid. Only the filled cells need to be validated according to the following rules:

1. Each row must contain the digits 1–9 without repetition.
2. Each column must contain the digits 1–9 without repetition.
3. Each of the nine 3 x 3 sub-boxes of the grid must contain the digits 1–9 without repetition.

Note:

- A Sudoku board (partially filled) could be valid but is not necessarily solvable.

- Only the filled cells need to be validated according to the mentioned rules.

Example 1:

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Input: board =

```
[["5","3",".",".","7",".",".",".","."],
["6",".",".","1","9","5",".",".","."],
[["9","8",".",".",".",".","6","."],
["8",".",".","6",".",".","."],
["4",".",".","8",".","3",".","1"],
["7",".",".","2",".",".","6"],
[["6",".",".","2","8","."],
["8","7","9"]]
```

Example 2:

```

Input: board =
[["8","3",".",".","7",".",".",".","."],
["6",".",".","1","9","5",".",".","."],
[["9","8",".",".",".","6",".","."],
["8",".",".","6",".",".","3"],
["4",".","8",".","3",".","1"],
["7",".",".","2",".","6"],
[["6",".",".","2","8","."]]

```

Constraints:

- `board.length == 9`
- `board[i].length == 9`
- `board[i][j]` is a digit 1–9 or '.'.

My LeetCode Solution

• My LeetCode Solution

```

class Solution:
    def isValidSudoku(self, board: List[List[str]]) -> bool:
        rows = [[False] * 9 for _ in range(9)]
        cols = [[False] * 9 for _ in range(9)]
        boxes = [[False] * 9 for _ in range(9)]

        for i in range(9):
            for j in range(9):
                if board[i][j] != '.':
                    num = ord(board[i][j]) - ord('1')

                    boxIndex = (i // 3) * 3 + (j // 3)
                    if rows[i][num] or cols[j][num] or boxes[boxIndex][num]:
                        return False
                    rows[i][num] = cols[j][num] = boxes[boxIndex][num] = True

        return True

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Check if a partially filled 9x9 sudoku board is valid – no repeats in any row,


```
# column, or 3x3 box. Ignore empty cells ('.'). Right?
# A valid board doesn't have to be solvable – just no current conflicts."
#
# "The two examples differ only in one cell: '8' appears twice in the top-left 3x3
box → false ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# For each of 9 rows, 9 columns, and 9 boxes, collect digits in a set and check
size==9.
# 27 independent validity checks – straightforward and easy to explain.
# Time: O(1) – fixed 81 cells. Space: O(1) – 9 sets of size 9.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Single pass: use three 2D boolean arrays – rows[r][d], cols[c][d],
boxes[r//3][c//3][d].
# For each filled cell, check all three. If any is already marked, return False.
# Otherwise mark all three. O(81) = O(1) for fixed 9x9."
```

PHASE 4 — CLEAN CODE

```
class _36_ValidSudoku:
    def isValidSudoku(self, board: List[List[str]]) -> bool:
        rows = [[False] * 9 for _ in range(9)]
        cols = [[False] * 9 for _ in range(9)]
        boxes = [[False] * 9 for _ in range(9)] # indexed by box_id = (r//3)*3 +
c//3
        for r in range(9):
            for c in range(9):
                if board[r][c] == '.':
                    continue
                d = int(board[r][c]) - 1
                box_id = (r // 3) * 3 + (c // 3)
                if rows[r][d] or cols[c][d] or boxes[box_id][d]:
                    return False
                rows[r][d] = cols[c][d] = boxes[box_id][d] = True
        return True
```

PHASE 5 — TEST & DEBUG

```
# Cell (0,0)='5': d=4. box_id=0. Mark rows[0][4]=cols[0][4]=boxes[0][4]=True.
# Cell (3,0)='8': d=7. box_id=0. rows[3][7]? No. boxes[0][7]? No. Mark.
# In example 2: cell (0,0)='8' and cell (3,0)='8': both have box_id=0, d=7.
# Second '8' at (3,0): boxes[0][7] already True → return False ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(81)=O(1). Space O(81)=O(1) – fixed 9x9 board.
# The box_id formula (r//3)*3+(c//3) maps each of the 9 boxes to 0–8. Know this
cold.
# Alternative: use a set of tuples like (r,'row',d), (c,'col',d), (box_id,'box',d) –
one set.
```

Given more time I'd ask: implement a Sudoku solver?
 # That's backtracking – for each empty cell, try 1–9, recurse, backtrack on failure."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isValidSudoku(self, board: List[List[str]]) -> bool:
        seen = set()

        for i in range(9):
            for j in range(9):
                c = board[i][j]
                if c == '.':
                    continue
                if (c + '@row ' + str(i) in seen or
                    c + '@col ' + str(j) in seen or
                    c + '@box ' + str(i // 3) + str(j // 3) in seen):
                    return False
                seen.add(c + '@row ' + str(i))
                seen.add(c + '@col ' + str(j))
                seen.add(c + '@box ' + str(i // 3) + str(j // 3))

        return True
```

• LeetDoocs

```
class Solution:
    def isValidSudoku(self, board: List[List[str]]) -> bool:
        row = [[False] * 9 for _ in range(9)]
        col = [[False] * 9 for _ in range(9)]
        sub = [[False] * 9 for _ in range(9)]
        for i in range(9):
            for j in range(9):
                c = board[i][j]
                if c == '.':
                    continue
```

```

        num = int(c) - 1
        k = i // 3 * 3 + j // 3
        if row[i][num] or col[j][num] or sub[k][num]:
            return False
        row[i][num] = True
        col[j][num] = True
        sub[k][num] = True

    return True

```

• SimplyLeet

Python solution with inline comments

```
def isValidSudoku(board):
```

Initialize sets for rows, columns, and boxes

```
    rows = [set() for _ in range(9)]
```

```
    cols = [set() for _ in range(9)]
```

```
    boxes = [set() for _ in range(9)]
```

Iterate over each cell in the board

```
    for r in range(9):
```

```
        for c in range(9):
```

```
            num = board[r][c]
```

Skip empty cells

```
            if num == '.':
```

```
                continue
```

Compute box index using integer division

```
            box_index = (r // 3) * 3 + (c // 3)
```

Check if the number is already present in the row, column, or box

```
            if num in rows[r] or num in cols[c] or num in boxes[box_index]:
```

```
                return False
```

Add number to row, column, and box sets

```
            rows[r].add(num)
```

```
            cols[c].add(num)
```

```
            boxes[box_index].add(num)
```

```
    return True
```

Example usage:

```
if __name__ == "__main__":
```

```
    board = [
```

```
        ["5","3",".",".","7",".",".",".","."],
```

```

["6",".",".","1","9","5",".",".","."],
[".","9","8",".",".",".","6","."],
["8",".",".","6",".",".","3"],
["4",".","8","3",".","1"],
["7",".","2",".","6"],
[".","6",".","2","8","."],
[".","4","1","9",".","5"],
[".","8",".","7","9"]
]
print(isValidSudoku(board)) # Expected output: True

```

• LC.ca

```

class Solution:
    def isValidSudoku(self, board: List[List[str]]) -> bool:
        row = [[False] * 9 for _ in range(9)]
        col = [[False] * 9 for _ in range(9)]
        sub = [[False] * 9 for _ in range(9)]
        for i in range(9):
            for j in range(9):
                c = board[i][j]
                if c == '.':
                    continue

                num = int(c) - 1
                k = i // 3 * 3 + j // 3
                if row[i][num] or col[j][num] or sub[k][num]:
                    return False
                row[i][num] = True
                col[j][num] = True
                sub[k][num] = True
        return True

```

◆ Quick Review

Key Insights

- Validate each row by checking for duplicate numbers while ignoring empty cells.

- Validate each column in the same way.
- Divide the board into nine 3x3 sub-boxes and check each for duplicate numbers.
- Use hash sets (or equivalent data structures) to keep track of seen digits for rows, columns, and boxes.
- The board size is fixed (9x9), so the overall time complexity is constant with respect to input size.

Space & Time

Time Complexity: $O(1)$ — Since the board size is fixed at 9x9.

Space Complexity: $O(1)$ — The extra space used by the sets is bounded by the fixed board size.

Solution

The solution uses three main data structures: an array of sets for rows, an array of sets for columns, and an array of sets for 3x3 sub-boxes. For each cell, if it is not empty, check if the number is already present in the corresponding row, column, or box. If found, return false immediately. Otherwise, add the number to the respective sets. Compute the

index for the 3x3 sub-box using the formula: $(\text{row_index} // 3) * 3 + (\text{col_index} // 3)$. If no duplicates are found after traversing the entire board, the board is considered valid.

Matrix

□ #48 Rotate Image

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math · Matrix
Lists: Grind 169 | CodeSignal

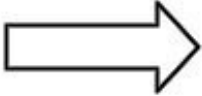
Problem

You are given an $n \times n$ 2D matrix representing an image, rotate the image by 90 degrees (clockwise).

You have to rotate the image in-place, which means you have to modify the input 2D matrix directly. DO NOT allocate another 2D matrix and do the rotation.

Example 1:

1	2	3
4	5	6
7	8	9

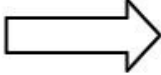


7	4	1
8	5	2
9	6	3

Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
 Output: [[7,4,1],[8,5,2],[9,6,3]]

Example 2:

5	1	9	11
2	4	8	10
13	3	6	7
15	14	12	16



15	13	2	5
14	3	4	1
12	6	8	9
16	7	10	11

Input: matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]
 Output: [[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]

Constraints:

- $n == \text{matrix.length} == \text{matrix}[i].\text{length}$
- $1 \leq n \leq 20$
- $-1000 \leq \text{matrix}[i][j] \leq 1000$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        n = len(matrix)

        # transpose
        for i in range(n):
            for j in range(i + 1, n):
                matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]

        # reverse each row

        for row in matrix:
            row.reverse()
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Rotate an $n \times n$ matrix 90 degrees clockwise in-place. No extra matrix allowed. Right?"

After rotation: element at (r,c) moves to (c, $n-1-r$)."

#

"[[1,2,3],[4,5,6],[7,8,9]] → [[7,4,1],[8,5,2],[9,6,3]] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Allocate a new $n \times n$ matrix; for each (r,c) set $new[c][n-1-r] = matrix[r][c]$.

Classic 90-degree rotation formula – clear and correct.

Uses $O(n^2)$ extra memory; we can rotate in-place layer by layer.

Time: $O(n^2)$. Space: $O(n^2)$ new matrix.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (two-step in-place)

"Two operations, each reversible and composable:

Step 1: Transpose (swap $matrix[r][c]$ with $matrix[c][r]$).

Step 2: Reverse each row.

Together these produce a 90° clockwise rotation. $O(n^2)$ time, $O(1)$ space.

Verify: (r,c) → transpose → (c,r) → reverse row → (c, $n-1-r$). Correct!"

PHASE 4 — CLEAN CODE

```
class _48_RotateImage:
    def rotate(self, matrix: List[List[int]]) -> None:
```

```

n = len(matrix)
# Step 1: transpose
for r in range(n):
    for c in range(r + 1, n):
        matrix[r][c], matrix[c][r] = matrix[c][r], matrix[r][c]
# Step 2: reverse each row
for row in matrix:
    row.reverse()

```

PHASE 5 — TEST & DEBUG

```

# [[1,2,3],[4,5,6],[7,8,9]]:
# Transpose: [[1,4,7],[2,5,8],[3,6,9]].
# Reverse rows: [[7,4,1],[8,5,2],[9,6,3]] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n²). Space O(1) in-place.
# Counter-clockwise 90°: reverse each row first, then transpose.
# 180°: transpose then transpose (just reverse all rows, then reverse their order).
# The two-step approach is elegant – each step is simple and reversible.
# Given more time I'd ask: rotate a non-square matrix?
# No longer possible in-place with the same dimensions – need a new m×n matrix."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def rotate(self, matrix: list[list[int]]) -> None:
        matrix.reverse()
        for i, j in itertools.combinations(range(len(matrix)), 2):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]

```

```

class Solution:
    def rotate(self, matrix: list[list[int]]) -> None:
        for mn in range(len(matrix) // 2):
            mx = len(matrix) - mn - 1
            for i in range(mn, mx):
                offset = i - mn
                top = matrix[mn][i]
                matrix[mn][i] = matrix[mx - offset][mn]
                matrix[mx - offset][mn] = matrix[mx][mx - offset]
                matrix[mx][mx - offset] = matrix[i][mx]

            matrix[i][mx] = top

```

• LeetDoocs

```
class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        n = len(matrix)
        for i in range(n >> 1):
            for j in range(n):
                matrix[i][j], matrix[n - i - 1][j] = matrix[n - i - 1][j], matrix[i][j]
            for i in range(n):
                for j in range(i):
                    matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
```

• SimplyLeet

```
def rotate(matrix):
    n = len(matrix)
    # Transpose the matrix by swapping elements across the diagonal
    for i in range(n):
        for j in range(i, n):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
    # Reverse each row to complete the clockwise rotation
    for i in range(n):
        matrix[i].reverse()

# Example usage:
matrix = [[1,2,3],[4,5,6],[7,8,9]]
rotate(matrix)
print(matrix) # Outputs: [[7,4,1],[8,5,2],[9,6,3]]
```

• LC.ca

```
class Solution(object):
    def rotate(self, matrix):
        """
        :type matrix: List[List[int]]
        :rtype: void Do not return anything, modify matrix in-place instead.
        """
        if len(matrix) == 0:
            return
        h = len(matrix)
        w = len(matrix[0])
```

```

for i in range(0, h): # mirror
    for j in range(0, w / 2):
        matrix[i][j], matrix[i][w - j - 1] = matrix[i][w - j - 1], matrix[i][j]

for i in range(0, h): # transpos
    for j in range(0, w - 1 - i):
        matrix[i][j], matrix[w - 1 - j][h - 1 - i] = matrix[w - 1 - j][h - 1 - i], matrix[i][j]

#####

```

```

class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        n = len(matrix)
        for i in range(n >> 1):
            for j in range(n):
                matrix[i][j], matrix[n - i - 1][j] = matrix[n - i - 1][j], matrix[i][j]
            for i in range(n):
                for j in range(i):
                    matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]

```

◆ Quick Review

Key Insights

- The rotation can be achieved in two steps: first transpose the matrix and then reverse each row.
- Transposing the matrix swaps $\text{matrix}[i][j]$ with $\text{matrix}[j][i]$, converting rows into columns.
- Reversing each row post-transposition yields the desired clockwise rotation.

- This approach works in-place with $O(1)$ extra space.

Space & Time

Time Complexity: $O(n^2)$ since every element is processed during the transpose and row reversal.

Space Complexity: $O(1)$ as the operations are performed in-place without extra data structures.

Solution

The solution leverages a two-step in-place transformation: Transpose the matrix: Iterate through the matrix and swap each element (i, j) with element (j, i) for $j \geq i$. Reverse each row: After the transpose, each row is reversed to rotate the matrix 90 degrees clockwise. The key trick is to realize that a transpose followed by a reverse of each row achieves the rotation without needing extra space.

Matrix

□ #54 Spiral Matrix

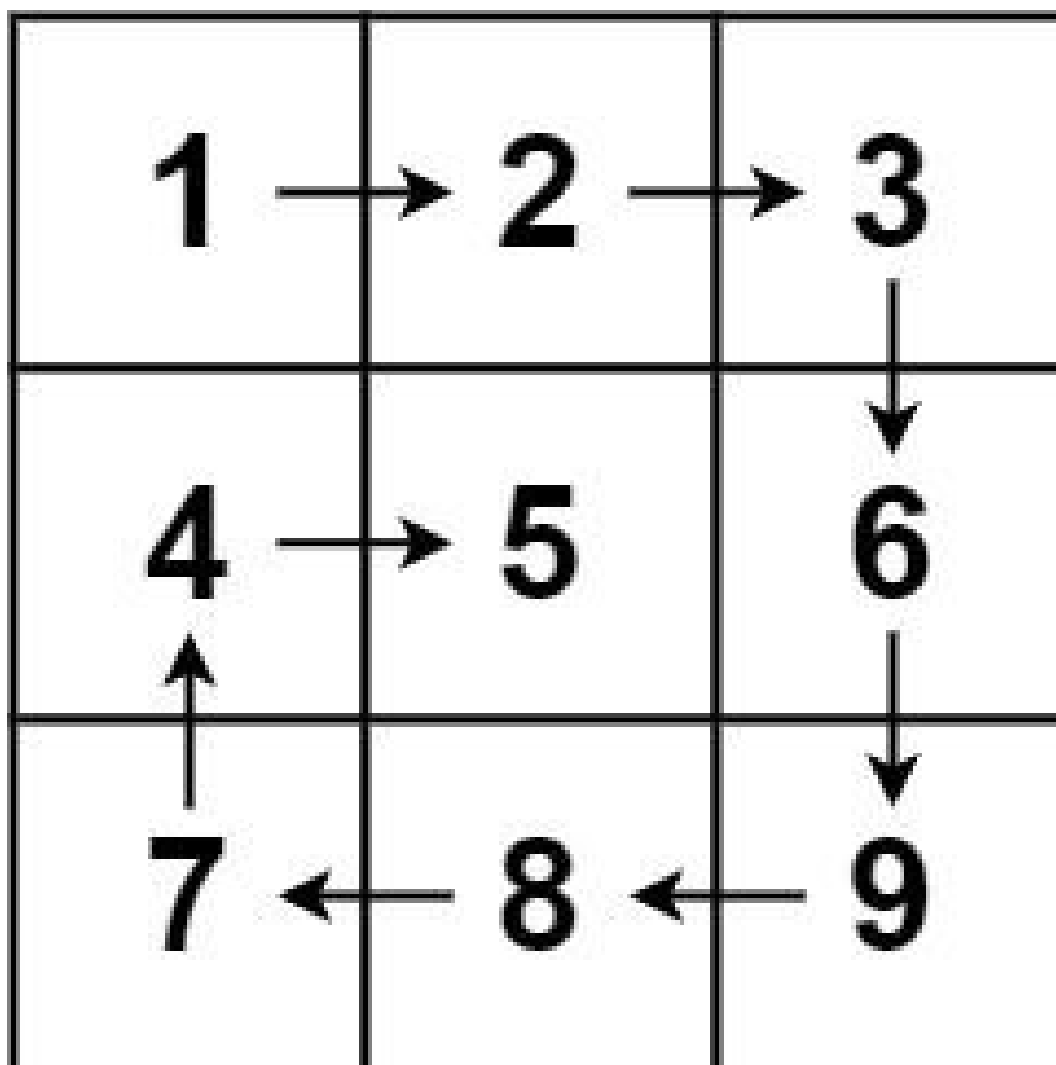
MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix · Simulation
Lists: Grind 169 | CodeSignal

Problem

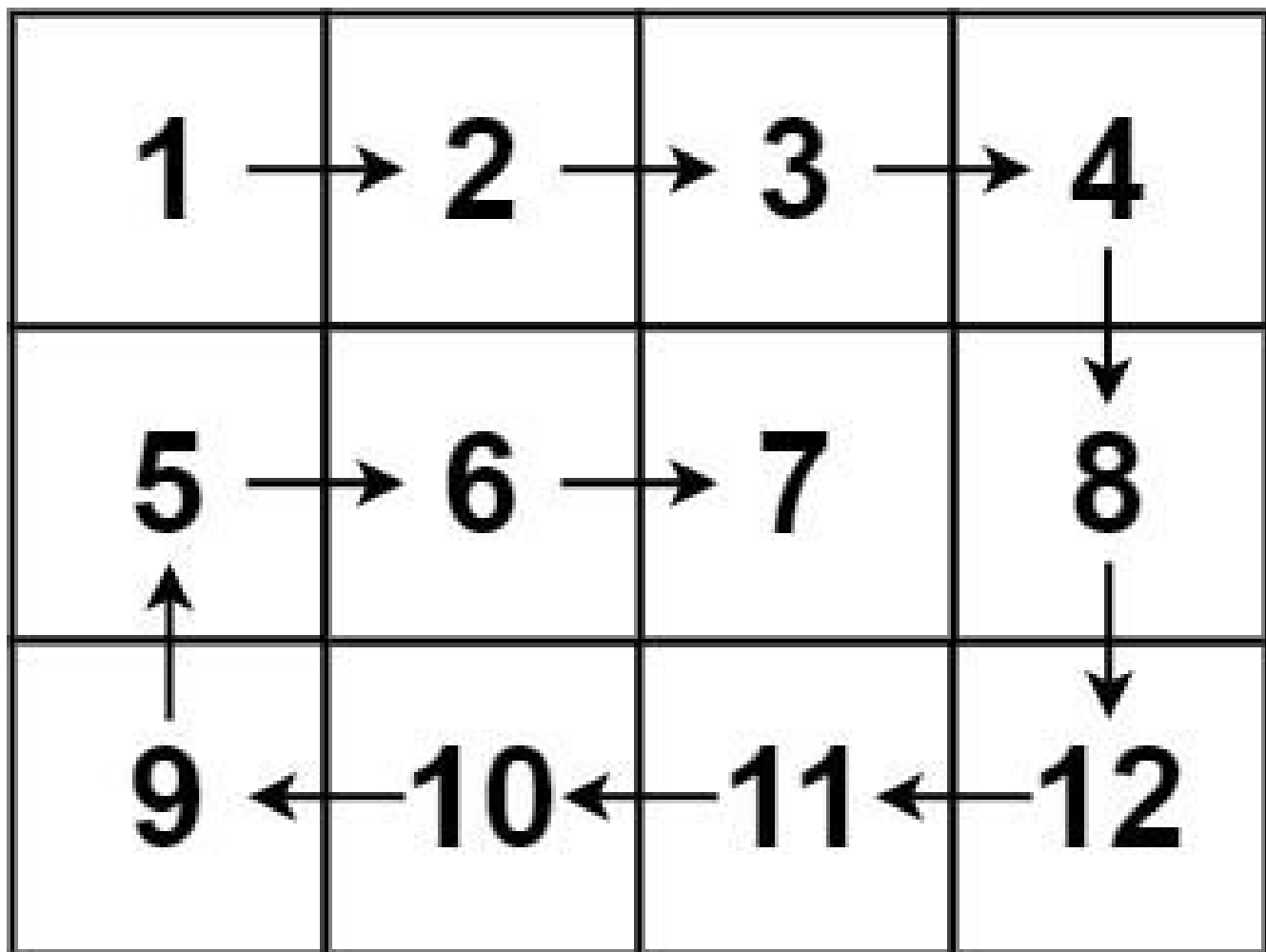
Given an $m \times n$ matrix, return all elements of the matrix in spiral order.

Example 1:



Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [1,2,3,6,9,8,7,4,5]

Example 2:



Input: matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]

Output: [1,2,3,4,8,12,11,10,9,5,6,7]

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 10$
- $-100 \leq \text{matrix}[i][j] \leq 100$

My LeetCode Solution

- My LeetCode Solution

```

class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        rows, cols = len(matrix), len(matrix[0])
        x, y, dx, dy = 0, 0, 1, 0
        res = []
        visited = set() # Track visited positions separately

        for _ in range(rows * cols):
            res.append(matrix[y][x])
            visited.add((x, y)) # Mark as visited

            if not 0 <= x + dx < cols or not 0 <= y + dy < rows or (x + dx, y
+ dy) in visited:
                dx, dy = -dy, dx

            x += dx
            y += dy

        return res

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return all matrix elements in spiral order: right along top, down along right side,
 # left along bottom, up along left side – then repeat for the inner rectangle.
 Right?
 # Works for non-square matrices."
 #
 # "[[1,2,3],[4,5,6],[7,8,9]] → [1,2,3,6,9,8,7,4,5] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
 # Maintain top/bottom/left/right bounds; walk right, down, left, up while unvisited.
 # Mark visited cells in a boolean matrix to know when to turn.
 # Works, but the visited matrix costs $O(m \cdot n)$ extra space.
 # Time: $O(m \cdot n)$. Space: $O(m \cdot n)$ visited.
 # Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (layer boundaries)

"Track four boundaries: top, bottom, left, right.
 # Traverse right along top (then top++), down along right (then right--),
 # left along bottom (then bottom--), up along left (then left++).
 # Stop when top > bottom or left > right. $O(m \cdot n)$ time, $O(1)$ extra space."

PHASE 4 — CLEAN CODE

```

class _54_SpiralMatrix:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        top, bottom = 0, len(matrix) - 1
        left, right = 0, len(matrix[0]) - 1
        result = []
        while top <= bottom and left <= right:
            for c in range(left, right + 1): result.append(matrix[top][c])
            top += 1
            for r in range(top, bottom + 1): result.append(matrix[r][right])
            right -= 1
            if top <= bottom:
                for c in range(right, left - 1, -1):
                    result.append(matrix[bottom][c])
                bottom -= 1
            if left <= right:
                for r in range(bottom, top - 1, -1): result.append(matrix[r][left])
                left += 1
        return result

```

PHASE 5 — TEST & DEBUG

```

# [[1,2,3],[4,5,6],[7,8,9]]: top=0,bot=2,l=0,r=2.
# Right: 1,2,3. top=1.
# Down: 6,9. right=1.
# Left (top<=bot): 8,7. bot=1.
# Up (left<=right): 4. left=1.
# top=1,bot=1,l=1,r=1.
# Right: 5. top=2. top>bot → stop.
# → [1,2,3,6,9,8,7,4,5] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(m*n). Space O(1) extra.
# The boundary guards (if top<=bottom / if left<=right) prevent double-counting
# the single middle row or column of non-square matrices.
# Given more time I'd ask: Spiral Matrix II (#59) – fill instead of read.
# Same boundary logic, write numbers 1..n² instead of appending."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def spiralOrder(self, matrix: list[list[int]]) -> list[int]:
        if not matrix:
            return []

        m = len(matrix)
        n = len(matrix[0])
        ans = []
        r1 = 0
        c1 = 0

        r2 = m - 1
        c2 = n - 1

        # Repeatedly add matrix[r1..r2][c1..c2] to `ans`.
        while len(ans) < m * n:
            j = c1
            while j <= c2 and len(ans) < m * n:
                ans.append(matrix[r1][j])
                j += 1
            i = r1 + 1

            while i <= r2 - 1 and len(ans) < m * n:
                ans.append(matrix[i][c2])
                i += 1
            j = c2
            while j >= c1 and len(ans) < m * n:
                ans.append(matrix[r2][j])
                j -= 1
            i = r2 - 1
            while i >= r1 + 1 and len(ans) < m * n:
                ans.append(matrix[i][c1])

                i -= 1
            r1 += 1
            c1 += 1
            r2 -= 1
            c2 -= 1

        return ans

```

• LeetDoocs

```

class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        m, n = len(matrix), len(matrix[0])
        dirs = (0, 1, 0, -1, 0)
        vis = [[False] * n for _ in range(m)]
        i = j = k = 0
        ans = []
        for _ in range(m * n):
            ans.append(matrix[i][j])
            vis[i][j] = True

            x, y = i + dirs[k], j + dirs[k + 1]
            if x < 0 or x >= m or y < 0 or y >= n or vis[x][y]:
                k = (k + 1) % 4
            i += dirs[k]
            j += dirs[k + 1]
        return ans

```

• SimplyLeet

```

def spiralOrder(matrix):
    # Initialize the boundaries of the matrix
    top, bottom = 0, len(matrix) - 1
    left, right = 0, len(matrix[0]) - 1
    result = [] # This will store the spiral order

    # Loop until the pointers cross each other
    while top <= bottom and left <= right:
        # Traverse from left to right on the current top row
        for col in range(left, right + 1):
            result.append(matrix[top][col])
            top += 1 # Move the top boundary down

        # Traverse downwards on the current right column
        for row in range(top, bottom + 1):
            result.append(matrix[row][right])
            right -= 1 # Move the right boundary to the left

        # Traverse from right to left on the current bottom row, if valid
        if top <= bottom:

```

```

        for col in range(right, left - 1, -1):
            result.append(matrix[bottom][col])
        bottom -= 1 # Move the bottom boundary up

# Traverse upwards on the current left column, if valid
if left <= right:
    for row in range(bottom, top - 1, -1):
        result.append(matrix[row][left])
    left += 1 # Move the left boundary right

return result

# Example usage:
# print(spiralOrder([[1,2,3],[4,5,6],[7,8,9]]))

```

• LC.ca

```

class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        m, n = len(matrix), len(matrix[0])
        dirs = (0, 1, 0, -1, 0)
        i = j = k = 0
        ans = []
        vis = set()
        for _ in range(m * n):
            ans.append(matrix[i][j])
            vis.add((i, j))

            x, y = i + dirs[k], j + dirs[k + 1]
            if not 0 <= x < m or not 0 <= y < n or (x, y) in vis:
                k = (k + 1) % 4
            i = i + dirs[k]
            j = j + dirs[k + 1]
        return ans

```

◆ Quick Review

Key Insights

- Use four boundaries: top, bottom, left, and right to track the current layer.
- Traverse right across the top row, then down the right column, then left on the bottom row, and finally up the left column.
- After each direction, adjust the corresponding boundary to move inward.
- Continue the traversal until the boundaries cross.

Space & Time

Time Complexity: $O(m * n)$ since each element is visited exactly once.

Space Complexity: $O(1)$ extra space (excluding the space required for the output list).

Solution

The solution uses a simulation approach with boundary pointers (top, bottom, left, right). At each step, the boundaries indicate the current submatrix to traverse. You iterate as follows: Traverse from left to right along the top boundary. Traverse downward along the right boundary. If the top boundary has not passed the bottom, traverse from right to left along the bottom boundary. If the left boundary has

not passed the right, traverse upward along the left boundary. After each traversal, the boundaries are adjusted inward. This continues until all elements have been visited.

Matrix

□ #59 Spiral Matrix II

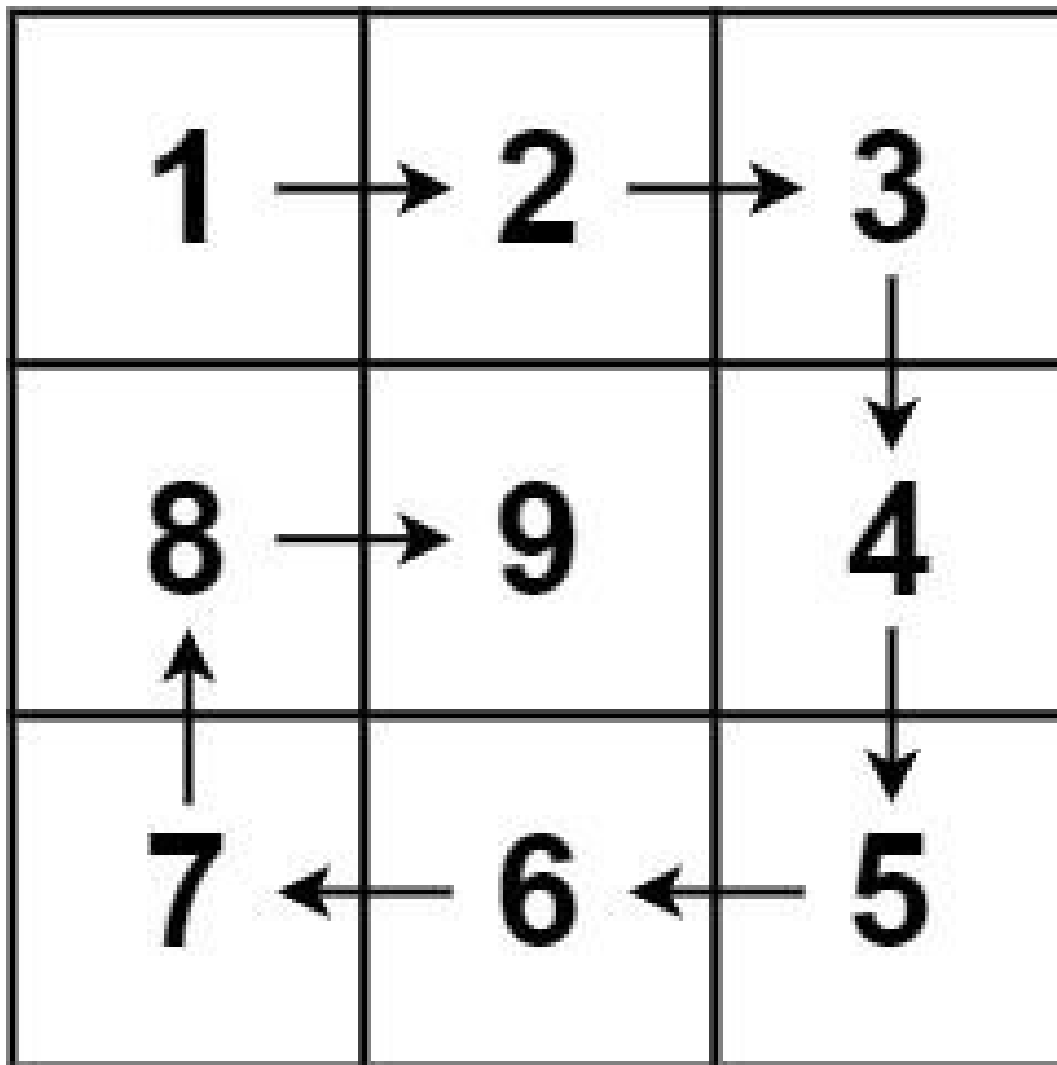
MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix · Simulation
Lists: CodeSignal

Problem

Given a positive integer n , generate an $n \times n$ matrix filled with elements from 1 to n^2 in spiral order.

Example 1:



Input: $n = 3$

Output: $[[1,2,3],[8,9,4],[7,6,5]]$

Example 2:

Input: $n = 1$

Output: $[[1]]$

Constraints:

- $1 \leq n \leq 20$

My LeetCode Solution

- My LeetCode Solution

```

class Solution:
    def generateMatrix(self, n: int) -> List[List[int]]:
        # Initialize matrix with zeros
        matrix = [[0] * n for _ in range(n)]

        # Start position and direction (moving right)
        x, y = 0, 0 # x=col, y=row
        dx, dy = 1, 0 # moving right

        # Fill numbers 1 to n*n

        for num in range(1, n * n + 1):
            matrix[y][x] = num

            # Check if next position is valid
            next_x, next_y = x + dx, y + dy
            if not (0 <= next_x < n and 0 <= next_y < n) or matrix[next_y][next_x] != 0:
                # Turn right (clockwise)
                dx, dy = -dy, dx

            # Move to next position

            x += dx
            y += dy

        return matrix

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Generate an $n \times n$ matrix filled with 1 to n^2 in spiral order. Return the matrix. Right?"

#

"n=3 → [[1,2,3],[8,9,4],[7,6,5]] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Fill numbers $1..n^2$ using direction arrays (right, down, left, up) and a visited grid.

Turn when hitting boundary or a filled cell.

Straightforward simulation with $O(n^2)$ extra visited space.

Time: $O(n^2)$. Space: $O(n^2)$ visited.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Identical boundary approach to #54, but write consecutive integers instead of reading.

Initialize $n \times n$ matrix of zeros, fill with $\text{num} = 1..n^2$. $O(n^2)$ time, $O(1)$ extra."

PHASE 4 — CLEAN CODE

```
class _59_SpiralMatrixII:
    def generateMatrix(self, n: int) -> List[List[int]]:
        matrix = [[0] * n for _ in range(n)]
        top, bottom, left, right = 0, n - 1, 0, n - 1
        num = 1
        while top <= bottom and left <= right:
            for c in range(left, right + 1): matrix[top][c] = num; num += 1
            top += 1
            for r in range(top, bottom + 1): matrix[r][right] = num; num += 1
            right -= 1
            if top <= bottom:
                for c in range(right, left - 1, -1): matrix[bottom][c] = num; num += 1
                bottom -= 1
            if left <= right:
                for r in range(bottom, top - 1, -1): matrix[r][left] = num; num += 1
                left += 1
        return matrix
```

PHASE 5 — TEST & DEBUG

$n=3$: fills 1→2→3 (top row), 4 (right col), 5→6 (bottom), 7 (left col), 9 (center).

→ $[[1,2,3],[8,9,4],[7,6,5]]$ ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^2)$. Space $O(1)$ extra – output matrix doesn't count.

This is the write-version of Spiral Matrix (#54) – exact same boundary logic.

Given more time I'd ask: generate a spiral starting from the center?

Reverse the boundary logic – start from the middle, expand outward."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def generateMatrix(self, n: int) -> list[list[int]]:
        ans = [[0] * n for _ in range(n)]
        count = 1

        for mn in range(n // 2):
            mx = n - mn - 1
            for i in range(mn, mx):
                ans[mn][i] = count
                count += 1

            for i in range(mn, mx):
                ans[i][mx] = count
                count += 1
            for i in range(mx, mn, -1):
                ans[mx][i] = count
                count += 1
            for i in range(mx, mn, -1):
                ans[i][mn] = count
                count += 1

        if n % 2 == 1:
            ans[n // 2][n // 2] = count

        return ans

```

• LeetDoocs

```

class Solution:
    def generateMatrix(self, n: int) -> List[List[int]]:
        ans = [[0] * n for _ in range(n)]
        dirs = (0, 1, 0, -1, 0)
        i = j = k = 0
        for v in range(1, n * n + 1):
            ans[i][j] = v
            x, y = i + dirs[k], j + dirs[k + 1]
            if x < 0 or x >= n or y < 0 or y >= n or ans[x][y]:
                k = (k + 1) % 4

            i, j = i + dirs[k], j + dirs[k + 1]
        return ans

```

• SimplyLeet

```
# Function to generate spiral matrix of size n x n
def generateMatrix(n):
    # Initialize an n x n matrix with zeros
    matrix = [[0] * n for _ in range(n)]
    # Initialize the starting number to fill
    num = 1
    # Set the initial boundaries for rows and columns
    top, bottom = 0, n - 1
    left, right = 0, n - 1

    # Continue the loop until the boundaries are valid
    while top <= bottom and left <= right:
        # Traverse from left to right along the top row
        for j in range(left, right + 1):
            matrix[top][j] = num
            num += 1
        top += 1 # Move the top boundary down

        # Traverse from top to bottom along the right column
        for i in range(top, bottom + 1):
            matrix[i][right] = num
            num += 1
        right -= 1 # Move the right boundary left

        # Traverse from right to left along the bottom row if still in valid
        # bounds
        if top <= bottom:
            for j in range(right, left - 1, -1):
                matrix[bottom][j] = num
                num += 1
            bottom -= 1 # Move the bottom boundary up
```



```

        # Traverse from bottom to top along the left column if still in valid
        bounds
        if left <= right:
            for i in range(bottom, top - 1, -1):
                matrix[i][left] = num
                num += 1
            left += 1 # Move the left boundary right

    return matrix

```

```

# Example usage:
print(generateMatrix(3))

```

• LC.ca

```

class Solution:
    def generateMatrix(self, n: int) -> List[List[int]]:
        ans = [[0] * n for _ in range(n)]
        dirs = ((0, 1), (1, 0), (0, -1), (-1, 0))
        i = j = k = 0
        for v in range(1, n * n + 1):
            ans[i][j] = v
            x, y = i + dirs[k][0], j + dirs[k][1]
            if x < 0 or y < 0 or x >= n or y >= n or ans[x][y]:
                k = (k + 1) % 4
            i, j = x, y
        return ans

```

◆ Quick Review

Key Insights

- Use four pointers (top, bottom, left, right) to keep track of the matrix boundaries.

- Traverse the matrix in a spiral: move right along the top row, then down the right column, left along the bottom row, and finally up the left column.
- Update the boundary pointers after each complete traversal of one side.
- Continue until all numbers from 1 to n^2 are filled into the matrix.

Space & Time

Time Complexity: $O(n^2)$

Space Complexity: $O(n^2)$ (required for the output matrix)

Solution

We simulate the spiral movement by initializing four pointers: top, bottom, left, and right, which denote the current boundaries of the matrix that have not yet been filled. For each iteration, we: Traverse from left to right along the top boundary, then increment the top pointer. Traverse from top to bottom along the right boundary, then decrement the right pointer. If there are rows remaining, traverse from right to left along the bottom boundary, then decrement the bottom pointer. If there

are columns remaining, traverse from bottom to top along the left boundary, then increment the left pointer. This algorithm ensures that the matrix is filled in a spiral order until all numbers are placed.

Matrix

□ #64 Minimum Path Sum

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Matrix
Lists: Denny Zhang

Problem

Given a $m \times n$ grid filled with non-negative numbers, find a path from top left to bottom right, which minimizes the sum of all numbers along its path.

Note: You can only move either down or right at any point in time.

Example 1:

1	3	1
1	5	1
4	2	1

Input: grid = [[1,3,1],[1,5,1],[4,2,1]]

Output: 7

Explanation: Because the path 1 → 3 → 1 → 1 → 1 minimizes the sum.

Example 2:

Input: grid = [[1,2,3],[4,5,6]]

Output: 12

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 200$

- $0 \leq \text{grid}[i][j] \leq 200$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def minPathSum(self, grid: List[List[int]]) -> int:
        if not grid or not grid[0]:
            return 0

        rows, cols = len(grid), len(grid[0])

        # Initialize first row (can only come from left)
        for c in range(1, cols):
            grid[0][c] += grid[0][c-1]

        # Initialize first column (can only come from top)
        for r in range(1, rows):
            grid[r][0] += grid[r-1][0]

        # Fill the rest
        for r in range(1, rows):
            for c in range(1, cols):
                grid[r][c] += min(grid[r-1][c], grid[r][c-1])

        return grid[rows-1][cols-1]
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Move only right or down. Find the path from top-left to bottom-right with minimum sum. Right?"

Can only move right or down – no backtracking."

#

"[[1,3,1],[1,5,1],[4,2,1]]: path 1→3→1→1→1=7 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

DFS from (0,0): at each cell try moving right and down, take min of two paths.

```
# Memoize (r,c) to avoid recomputing subpaths – classic top-down DP.
# Without memo this blows up exponentially; memo makes it O(m*n).
# Time: O(m * n) with memo. Space: O(m * n) memo + stack.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (in-place DP)

```
# "dp[i][j] = min path sum to reach (i,j).
# dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1]).
# First row: only right. First col: only down.
# Modify grid in-place – no extra space. O(m*n) time, O(1) extra."
```

PHASE 4 — CLEAN CODE

```
class _64_MinPathSum:
    def minPathSum(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        for r in range(m):
            for c in range(n):
                if r == 0 and c == 0:
                    continue
                elif r == 0:
                    grid[r][c] += grid[r][c - 1] # can only come from left
                elif c == 0:
                    grid[r][c] += grid[r - 1][c] # can only come from above
                else:
                    grid[r][c] += min(grid[r - 1][c], grid[r][c - 1])
        return grid[m - 1][n - 1]
```

PHASE 5 — TEST & DEBUG

```
# [[1,3,1],[1,5,1],[4,2,1]]:
# Row 0: [1,4,5]. Col 0: [1,2,6]. (1,1)=5+min(4,2)=7. (1,2)=1+min(7,5)=6.
# (2,1)=2+min(7,6)=8. (2,2)=1+min(6,8)=7. → 7 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(m*n). Space O(1) – modifies grid in place.
# If we can't modify the input, use a 1D rolling array: O(n) extra space.
# Given more time I'd ask: what if we can also move up or left?
# That breaks the DP recurrence – shortest path with general movement is
Dijkstra's."
```

Community Solutions (Python)

• WalkCC


```

class Solution:
    def minPathSum(self, grid: list[list[int]]) -> int:
        m = len(grid)
        n = len(grid[0])

        for i in range(m):
            for j in range(n):
                if i > 0 and j > 0:
                    grid[i][j] += min(grid[i - 1][j], grid[i][j - 1])
                elif i > 0:
                    grid[i][0] += grid[i - 1][0]
                elif j > 0:
                    grid[0][j] += grid[0][j - 1]

        return grid[m - 1][n - 1]

```

• LeetDoocs

```

class Solution:
    def minPathSum(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        f = [[0] * n for _ in range(m)]
        f[0][0] = grid[0][0]
        for i in range(1, m):
            f[i][0] = f[i - 1][0] + grid[i][0]
        for j in range(1, n):
            f[0][j] = f[0][j - 1] + grid[0][j]
        for i in range(1, m):
            for j in range(1, n):
                f[i][j] = min(f[i - 1][j], f[i][j - 1]) + grid[i][j]
        return f[-1][-1]

```

• SimplyLeet

```
# Python solution using dynamic programming
def minPathSum(grid):
    # Number of rows and columns in the grid
    rows = len(grid)
    cols = len(grid[0])

    # Initialize the DP table directly in grid to save space
    # Fill the first row by accumulating the sum from the left
    for j in range(1, cols):
        grid[0][j] += grid[0][j-1]

    # Fill the first column by accumulating the sum above
    for i in range(1, rows):
        grid[i][0] += grid[i-1][0]

    # Compute the minimum path sum for the rest of the cells
    for i in range(1, rows):
        for j in range(1, cols):
            # The cell value is added to the minimum of the top or left cell
            grid[i][j] += min(grid[i-1][j], grid[i][j-1])

    # The bottom-right cell contains the answer
    return grid[rows-1][cols-1]

# Example usage
example_grid = [[1,3,1],[1,5,1],[4,2,1]]
print(minPathSum(example_grid)) # Output should be 7
```

• LC.ca

```
class Solution:
    def minPathSum(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        f = [[0] * n for _ in range(m)]
        f[0][0] = grid[0][0]
        for i in range(1, m):
            f[i][0] = f[i-1][0] + grid[i][0]
        for j in range(1, n):
            f[0][j] = f[0][j-1] + grid[0][j]
        for i in range(1, m):
```

```
for j in range(1, n):
    f[i][j] = min(f[i - 1][j], f[i][j - 1]) + grid[i][j]
return f[-1][-1]
```

◆ Quick Review

Key Insights

- The problem can be solved using Dynamic Programming.
- The optimal solution for each cell depends on the minimum path sum of the cell above it and the cell to its left.
- Since you can only move right or down, only two adjacent cells affect the current cell's result.
- Edge cases include the first row and first column, which have only one direction of movement.

Space & Time

Time Complexity: $O(m * n)$, where m and n are the number of rows and columns.

Space Complexity: $O(m * n)$ for the DP table (this can be optimized to $O(n)$ with in-place computations).

Solution

We use a Dynamic Programming (DP) approach where we create a DP table (or use the grid itself) that stores the minimum path sum to each cell. For each cell (i, j) : If $i = 0$ (first row), the only possible path is from the left. If $j = 0$ (first column), the only possible path is from above. For other cells, the minimum path sum is the cell value plus the minimum of the two possible previous cells (above and left). This ensures that by the time we reach the bottom-right cell, we have accumulated the minimum sum required to reach that cell.

Matrix

□ #73 Set Matrix Zeroes

MED[Open on LeetCode](#)

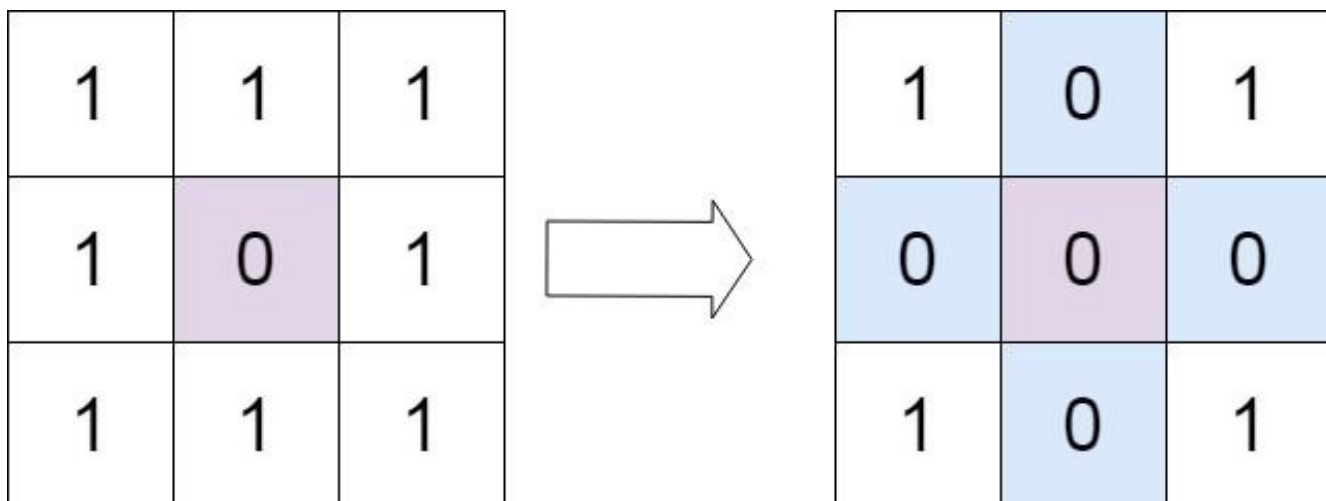
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Matrix
Lists: Grind 169

Problem

Given an $m \times n$ integer matrix `matrix`, if an element is 0, set its entire row and column to 0's.

You must do it in place.

Example 1:



Input: `matrix = [[1,1,1],[1,0,1],[1,1,1]]`

Output: `[[1,0,1],[0,0,0],[1,0,1]]`

Example 2:

0	1	2	0
3	4	5	2
1	3	1	5

→

0	0	0	0
0	4	5	0
0	3	1	0

Input: matrix = [[0,1,2,0],[3,4,5,2],[1,3,1,5]]
 Output: [[0,0,0,0],[0,4,5,0],[0,3,1,0]]

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[0].\text{length}$
- $1 \leq m, n \leq 200$
- $-231 \leq \text{matrix}[i][j] \leq 231 - 1$

Follow up:

- A straightforward solution using $O(mn)$ space is probably a bad idea.
- A simple improvement uses $O(m + n)$ space, but still not the best solution.
- Could you devise a constant space solution?

My LeetCode Solution

- My LeetCode Solution

```

class Solution:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        rows, cols = len(matrix), len(matrix[0])
        zero_rows = set()
        zero_cols = set()

        # Find zeros
        for i in range(rows):
            for j in range(cols):
                if matrix[i][j] == 0:

                    zero_rows.add(i)
                    zero_cols.add(j)

        # Set zeros
        for i in zero_rows:
            for j in range(cols):
                matrix[i][j] = 0

        for j in zero_cols:
            for i in range(rows):

                matrix[i][j] = 0

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "If a cell is 0, zero out its entire row and column. Do it in-place. Right?
# We must use the ORIGINAL zeros – not zeros we set during the process."
#
# "[[1,0,1],[1,1,1],[1,1,1]] → [[0,0,0],[1,0,1],[1,0,1]] ✓"

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with brute force to lock in the logic.
# First collect every (r,c) where matrix[r][c]==0 into a list.
# Second pass: zero out entire rows and columns for each stored zero.
# Simple, but storing all zero coordinates uses O(m+n) extra structures.
# Time: O(m * n). Space: O(m + n) for row/col sets.
# Should I go ahead and optimize?"

```

PHASE 3 — OPTIMIZE (O(1) space using first row/col as markers)

```

# "Use the first row and first column of the matrix itself as markers.
# If matrix[r][c]==0: mark matrix[r][0]=0 and matrix[0][c]=0.
# Handle first row and first column separately (track with boolean flags).

```



```
# Then zero out interior cells using the markers.
# Then zero out first row/col based on flags. O(m*n) time, O(1) space."
```

PHASE 4 — CLEAN CODE

```
class _73_SetMatrixZeroes:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        m, n = len(matrix), len(matrix[0])
        first_row_zero = any(matrix[0][c] == 0 for c in range(n))
        first_col_zero = any(matrix[r][0] == 0 for r in range(m))

        for r in range(1, m): # mark zeros using first row/col
            for c in range(1, n):
                if matrix[r][c] == 0:
                    matrix[r][0] = 0
                    matrix[0][c] = 0

        for r in range(1, m): # zero out interior
            for c in range(1, n):
                if matrix[r][0] == 0 or matrix[0][c] == 0:
                    matrix[r][c] = 0

        if first_row_zero: # zero out first row if needed
            for c in range(n): matrix[0][c] = 0
        if first_col_zero: # zero out first col if needed
            for r in range(m): matrix[r][0] = 0
```

PHASE 5 — TEST & DEBUG

```
# [[0,1,2,0],[3,4,5,2],[1,3,1,5]]:
# first_row_zero: matrix[0][0]=0 → True. first_col_zero: matrix[0][0]=0 → True.
# Mark: r=1,c=0: already 3≠0. r=1,c=3: 2≠0... only (0,0) and (0,3) have zeros in row 0.
# Interior zeros: marker[0][0]=0→col 0 zeros. marker[0][3]: 0→col 3 zeros.
# → [[0,0,0,0],[0,4,5,0],[0,3,1,0]] ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(m*n). Space O(1).
# The first row/col flags are the key – without them, using matrix[0][0] as a marker
# for both first row and first col creates an ambiguity. Two flags resolve it cleanly.
# The four-step order (flag, mark, zero interior, zero edges) is the correct sequence.
# Given more time I'd ask: what if we should set a cell to the average of its row/col instead?
# Same structure – two-pass approach to avoid using updated values."
```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def setZeroes(self, matrix: list[list[int]]) -> None:
        m = len(matrix)
        n = len(matrix[0])
        shouldFillFirstRow = 0 in matrix[0]
        shouldFillFirstCol = 0 in list(zip(*matrix))[0]

        # Store the information in the first row and the first column.
        for i in range(1, m):
            for j in range(1, n):

                if matrix[i][j] == 0:
                    matrix[i][0] = 0
                    matrix[0][j] = 0

        # Fill 0s for the matrix except the first row and the first column.
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][0] == 0 or matrix[0][j] == 0:
                    matrix[i][j] = 0

        # Fill 0s for the first row if needed.
        if shouldFillFirstRow:
            matrix[0] = [0] * n

        # Fill 0s for the first column if needed.
        if shouldFillFirstCol:
            for row in matrix:
                row[0] = 0

```

• LeetDoocs

```

class Solution:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        m, n = len(matrix), len(matrix[0])
        row = [False] * m
        col = [False] * n
        for i in range(m):
            for j in range(n):
                if matrix[i][j] == 0:
                    row[i] = col[j] = True
        for i in range(m):

```

```

        for j in range(n):
            if row[i] or col[j]:
                matrix[i][j] = 0

```

• SimplyLeet

Python solution for the Set Matrix Zeroes problem

```

def setZeroes(matrix):
    # Step 1: Determine the dimensions of matrix
    m = len(matrix)
    n = len(matrix[0])

    # Determine if first row or first column should be zeroed
    first_row_zero = any(matrix[0][j] == 0 for j in range(n))
    first_col_zero = any(matrix[i][0] == 0 for i in range(m))

    # Step 2: Use first row and column as markers
    for i in range(1, m):
        for j in range(1, n):
            if matrix[i][j] == 0:
                matrix[i][0] = 0 # mark the first element of the row
                matrix[0][j] = 0 # mark the first element of the column

    # Step 3: Zero out cells based on the markers in first row and column
    for i in range(1, m):
        for j in range(1, n):
            if matrix[i][0] == 0 or matrix[0][j] == 0:
                matrix[i][j] = 0

    # Step 4: Zero out the first row if needed
    if first_row_zero:
        for j in range(n):
            matrix[0][j] = 0

    # Step 5: Zero out the first column if needed

```

```

if first_col_zero:
    for i in range(m):
        matrix[i][0] = 0

# Example usage:
# matrix = [[1,1,1],[1,0,1],[1,1,1]]
# setZeroes(matrix)
# print(matrix)

```

• LC.ca

```

class Solution:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        m, n = len(matrix), len(matrix[0])
        i0 = any(v == 0 for v in matrix[0])
        j0 = any(matrix[i][0] == 0 for i in range(m))
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][j] == 0:
                    matrix[i][0] = matrix[0][j] = 0
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][0] == 0 or matrix[0][j] == 0:
                    matrix[i][j] = 0
        if i0:
            for j in range(n):
                matrix[0][j] = 0
        if j0:
            for i in range(m):
                matrix[i][0] = 0

```

◆ Quick Review

Key Insights

- The challenge is to mark rows and columns that need to be zeroed without using extra space.

- A common trick is to use the first row and first column as markers.
- Special care is needed for the first row and first column, as they are also used as markers.
- The algorithm involves two passes: one for marking and one for updating the matrix based on the markers.

Space & Time

Time Complexity: $O(m * n)$

Space Complexity: $O(1)$ (in-place, using only constant extra space)

Solution

We can solve this problem by using the first row and first column of the matrix as markers. First, determine if the first row or first column needs to be zeroed, then traverse the rest of the matrix and mark the corresponding first row and column positions if a zero is encountered. Finally, update the matrix backward using the markers, and finally update the first row and column if needed. The algorithm works in three main steps: Check if the first row or first column should be zeroed (store this in two variables). Iterate through

the rest of the matrix; for any element that is zero, mark its row and column by setting the corresponding element in the first row and first column to zero. Iterate through the matrix again (excluding the first row and column) and set elements to zero if their row or column marker is zero. Finally, zero out the first row and/or first column if needed. This approach avoids extra space usage apart from the two boolean flags while ensuring that the matrix is updated correctly.

Matrix

□ #74 Search a 2D Matrix

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Matrix
Lists: Grind 169

Problem

You are given an $m \times n$ integer matrix `matrix` with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer `target`, return `true` if `target` is in `matrix` or `false` otherwise.

You must write a solution in $O(\log(m * n))$ time complexity.

Example 1:

1	3	5	7
10	11	16	20
23	30	34	60

Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
Output: true

Example 2:

1	3	5	7
10	11	16	20
23	30	34	60

Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 13
Output: false

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 100$
- $-104 \leq \text{matrix}[i][j], \text{target} \leq 104$

My LeetCode Solution

- My LeetCode Solution

```

class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        if not matrix or not matrix[0]:
            return False

        rows, cols = len(matrix), len(matrix[0])
        left, right = 0, rows * cols - 1

        while left <= right:
            mid = (left + right) // 2

            mid_val = matrix[mid // cols][mid % cols]

            if mid_val == target:
                return True
            elif mid_val < target:
                left = mid + 1
            else:
                right = mid - 1

        return False

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Matrix where each row is sorted and the first element of each row is greater than
# the last of the previous row – so the whole matrix is one sorted sequence.
# Find target in O(log(m*n)). Right?"
#
# "Treat it as a 1D array of length m*n. Index k maps to row=k//n, col=k%n."

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with brute force to lock in the logic.
# Treat the matrix as a flattened sorted array and binary search on index 0..m*n-1.
# Or scan row by row with binary search per row.
# Correct O(m log n), but a single binary search on the virtual array is cleaner.
# Time: O(m log n). Space: O(1).
# Should I go ahead and optimize?"

```

PHASE 3 — OPTIMIZE

```

# "Binary search on the virtual 1D array [0, m*n-1].
# Map mid to (mid//n, mid%n) to access the actual cell.
# O(log(m*n)) = O(log m + log n)."

```

PHASE 4 — CLEAN CODE

```
class _74_Search2DMatrix:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
        left, right = 0, m * n - 1
        while left <= right:
            mid = (left + right) // 2
            mid_val = matrix[mid // n][mid % n]
            if mid_val == target:
                return True
            elif mid_val < target:
                left = mid + 1
            else:
                right = mid - 1
        return False
```

PHASE 5 — TEST & DEBUG

```
# matrix=[[1,3,5,7],[10,11,16,20],[23,30,34,60]], target=3:
# l=0,r=11. mid=5→matrix[1][1]=11. 11>3→r=4.
# mid=2→matrix[0][2]=5. 5>3→r=1.
# mid=0→matrix[0][0]=1. 1<3→l=1.
# mid=1→matrix[0][1]=3. 3==3→True ✓
# target=13: binary search narrows until l>r → False ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(log(m*n)). Space O(1).
# The index mapping mid//n and mid%n is the entire trick.
# This only works because the matrix is fully sorted as one sequence.
# If rows are sorted but first-of-next isn't > last-of-prev (Search a 2D Matrix II
#240):
# Use staircase search – start at top-right, move left if too big, down if too
small.
# O(m+n) time – different problem, different approach."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def searchMatrix(self, matrix: list[list[int]], target: int) -> bool:
        if not matrix:
            return False

        m = len(matrix)
        n = len(matrix[0])
        l = 0
        r = m * n
```

```

while l < r:
    mid = (l + r) // 2
    i = mid // n
    j = mid % n
    if matrix[i][j] == target:
        return True
    if matrix[i][j] < target:
        l = mid + 1
    else:
        r = mid

```

```

return False

```

• LeetDoocs

```

class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
        left, right = 0, m * n - 1
        while left < right:
            mid = (left + right) >> 1
            x, y = divmod(mid, n)
            if matrix[x][y] >= target:
                right = mid
            else:
                left = mid + 1
        return matrix[left // n][left % n] == target

```

• SimplyLeet

```

def searchMatrix(matrix, target):
    # Get number of rows and columns
    if not matrix or not matrix[0]:
        return False

    rows, cols = len(matrix), len(matrix[0])
    left, right = 0, rows * cols - 1

    # Perform binary search over the virtual flattened array
    while left <= right:

```

```
# Find the middle index
mid = (left + right) // 2
# Map the middle index to row and column in the matrix
row = mid // cols
col = mid % cols
mid_value = matrix[row][col]
```

```
# Check if the mid_value equals the target
if mid_value == target:
    return True
```

```
elif mid_value < target:
    left = mid + 1 # Move to the right subarray
else:
    right = mid - 1 # Move to the left subarray
```

```
# Target not found
return False
```

```
# Example usage:
# matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]]
```

```
# target = 3
# print(searchMatrix(matrix, target))
```

• LC.ca

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])
        left, right = 0, m * n - 1
        while left <= right: # must be <=, not <, for matrix=[[1]],target=1
            mid = (left + right) >> 1
            x, y = divmod(mid, n) # note: divide column count
            if matrix[x][y] == target:
                return True
            elif matrix[x][y] > target:
```

```

        right = mid - 1
    else:
        left = mid + 1
    return False

#####

class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])

        left, right = 0, m * n - 1
        while left < right:
            mid = (left + right) >> 1
            x, y = divmod(mid, n)
            if matrix[x][y] >= target:
                right = mid
            else:
                left = mid + 1
        return matrix[left // n][left % n] == target

```

◆ Quick Review

Key Insights

- The matrix is essentially a flattened sorted array due to its properties.
- Use binary search to efficiently determine if the target exists.
- Map the 1D binary search index to the corresponding 2D matrix coordinates using division and modulus operations.
- Achieve $O(\log(m*n))$ time complexity by navigating the matrix as if it were a single

sorted list.

Space & Time

Time Complexity: $O(\log(m * n))$

Space Complexity: $O(1)$

Solution

We can treat the matrix as a flattened sorted list without physically copying the elements. Given the total number of elements = $m * n$, perform binary search on indices ranging from 0 to $(m*n - 1)$. For any index mid , calculate its corresponding row as $mid // n$ and column as $mid \% n$. Compare the element at $matrix[row][col]$ with the target. If it equals target, return true; if it is less than target, search the right half; otherwise, search the left half.

Matrix

□ #221 Maximal Square

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Matrix
Lists: Grind 169

Problem

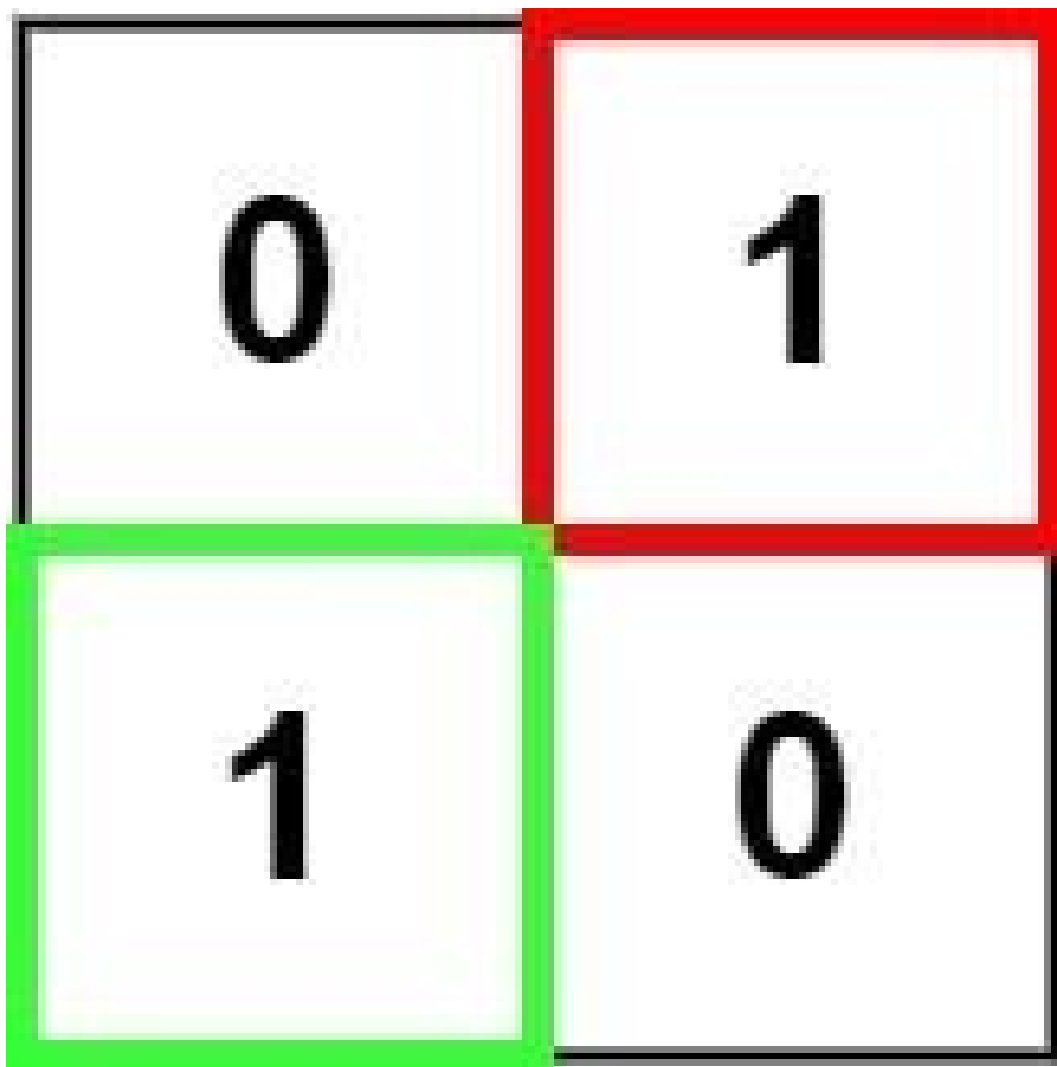
Given an $m \times n$ binary matrix filled with 0's and 1's, find the largest square containing only 1's and return its area.

Example 1:

1	0	1	0	0
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

Input: matrix = `[["1","0","1","0","0"],["1","0","1","1","1"],["1","1","1","1","1"],["1","0","0","1","0"]]`
Output: 4

Example 2:



Input: matrix = [["0","1"],["1","0"]]
Output: 1

Example 3:

Input: matrix = [["0"]]
Output: 0

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 300$
- $\text{matrix}[i][j]$ is '0' or '1'.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def maximalSquare(self, matrix: List[List[str]]) -> int:
        if not matrix or not matrix[0]:
            return 0

        rows, cols = len(matrix), len(matrix[0])
        dp = [[0] * (cols + 1) for _ in range(rows + 1)]
        max_side = 0

        for i in range(1, rows + 1):
            for j in range(1, cols + 1):
                if matrix[i-1][j-1] == '1':
                    dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1
                    max_side = max(max_side, dp[i][j])

        return max_side * max_side
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the largest square of all 1s. Return its area. Matrix contains '0'/'1' strings. Right?"

#

"[[1,0,1,0,0],[1,0,1,1,1],[1,1,1,1,1],[1,0,0,1,0]] → 2×2=4 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

For every cell containing '1', expand square side length while all cells in square are '1'.

Track the maximum side length found across all starting cells.

Each expansion check is O(k) for side k – overall O(m*n*min(m,n)^2).

Time: O(m * n * min(m,n)^2). Space: O(1).

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (DP)

"dp[i][j] = side length of the largest all-1s square with bottom-right corner at (i,j)."

If matrix[i][j]=='0': dp[i][j]=0.

```
# Else: dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1.
# Intuition: a square of side k can be extended by 1 only if the top, left, and
diagonal neighbour
# each have a square of side k. The bottleneck is the minimum.
# Track max side length → return side2."
```

PHASE 4 — CLEAN CODE

```
class _221_MaximalSquare:
    def maximalSquare(self, matrix: List[List[str]]) -> int:
        m, n = len(matrix), len(matrix[0])
        dp = [[0] * n for _ in range(m)]
        max_side = 0
        for r in range(m):
            for c in range(n):
                if matrix[r][c] == '1':
                    top = dp[r - 1][c] if r > 0 else 0
                    left = dp[r][c - 1] if c > 0 else 0
                    diag = dp[r - 1][c - 1] if r > 0 and c > 0 else 0
                    dp[r][c] = min(top, left, diag) + 1
                    max_side = max(max_side, dp[r][c])
        return max_side * max_side
```

PHASE 5 — TEST & DEBUG

```
# "[[0,1],['1','0']]: no 2x2 all-1s → dp stays at 1s and 0s → max=1 → area=1 ✓
# Full example: bottom-right region gives dp[2][3]=dp[2][4]=2, max=2, area=4 ✓"
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(m*n). Space O(m*n) – reducible to O(n) with a rolling array.
# The min of three neighbours is the core insight – think of it as the square being
# limited by the weakest neighbouring corner.
# Given more time I'd ask: maximal rectangle of 1s (not just square)?
# That's Maximal Rectangle (#85) – use a histogram approach per row, O(m*n)."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maximalSquare(self, matrix: list[list[str]]) -> int:
        m = len(matrix)
        n = len(matrix[0])
        dp = [[0] * n for _ in range(m)]
        maxLength = 0

        for i in range(m):
            for j in range(n):
                if i == 0 or j == 0 or matrix[i][j] == '0':
```

```

        dp[i][j] = 1 if matrix[i][j] == '1' else 0
    else:
        dp[i][j] = min(dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1]) + 1
    maxLength = max(maxLength, dp[i][j])

return maxLength * maxLength

```

• LeetDoocs

```

class Solution:
    def maximalSquare(self, matrix: List[List[str]]) -> int:
        m, n = len(matrix), len(matrix[0])
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        mx = 0
        for i in range(m):
            for j in range(n):
                if matrix[i][j] == '1':
                    dp[i + 1][j + 1] = min(dp[i][j + 1], dp[i + 1][j], dp[i][j + 1]) + 1

                    mx = max(mx, dp[i + 1][j + 1])

        return mx * mx

```

• SimplyLeet

```

# Python implementation of the Maximal Square problem

def maximalSquare(matrix):
    # Check if matrix is empty
    if not matrix or not matrix[0]:
        return 0
    rows = len(matrix)
    cols = len(matrix[0])
    # DP table with extra row and col for ease of boundary conditions
    dp = [[0] * (cols + 1) for _ in range(rows + 1)]

```

```

max_side = 0
# Traverse through each cell in the matrix
for i in range(1, rows + 1):
    for j in range(1, cols + 1):
        # If the current cell is '1'
        if matrix[i - 1][j - 1] == '1':
            # Compute the minimum value among the top, left, and top-left
            neighbors and add 1
            dp[i][j] = min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]) +
1
            max_side = max(max_side, dp[i][j])
# The area of the square is the square of the side length

return max_side * max_side

# Example usage:
matrix = [
    ["1", "0", "1", "0", "0"],
    ["1", "0", "1", "1", "1"],
    ["1", "1", "1", "1", "1"],
    ["1", "0", "0", "1", "0"]
]
print(maximalSquare(matrix))

```

• LC.ca

```

'''
eg. a 10*10 square with full of 1s
this square move 1 line down
this square move 1 line right

so there needs an extra single 1 at bottom right, to make it a larger full
square
'''
class Solution:
    def maximalSquare(self, matrix: List[List[str]]) -> int:
        m, n = len(matrix), len(matrix[0])

```



```

dp = [[0] * (n + 1) for _ in range(m + 1)]
mx = 0
for i in range(m):
    for j in range(n):
        if matrix[i][j] == '1':
            dp[i + 1][j + 1] = 1 + min(dp[i][j + 1], dp[i + 1][j], dp[i][j])
            mx = max(mx, dp[i + 1][j + 1])
return mx * mx

```

◆ Quick Review

Key Insights

- Use dynamic programming to build a solution where each cell in the DP table represents the side length of the largest square ending at that cell.
- If the current cell is '1', its DP value is the minimum of the top, left, and top-left neighbors plus one.
- Update a global maximum side length during the process to determine the area.
- The square's area is the square of its maximum side length.

Space & Time

Time Complexity: $O(m * n)$, where m is the number of rows and n is the number of columns.

Space Complexity: $O(m * n)$ for the DP table (this can be optimized to $O(n)$ with space reduction techniques).

Solution

We use a two-dimensional dynamic programming approach. We define a DP matrix with one extra row and column to simplify boundary conditions. For each cell (i, j) in the input matrix, if the value is '1', then we calculate the DP value as the minimum of its top, left, and top-left neighboring DP values plus one. This value represents the side length of the square ending at (i, j) . We also keep track of the maximum side length found and finally compute the area of the maximal square by squaring the maximum side length.

Matrix

★ #311 Sparse Matrix Multiplication ★

MED
[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
 Array · Hash Table · Matrix
 Lists: ★ Premium | Premium 98

Problem

Given two sparse matrices `mat1` of size $m \times k$ and `mat2` of size $k \times n$, return the result of `mat1 \times mat2`. You may assume that multiplication is always possible.

Example 1:

$$\begin{bmatrix} 1 & 0 & 0 \\ -1 & 0 & 3 \end{bmatrix} \times \begin{bmatrix} 7 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 7 & 0 & 0 \\ -7 & 0 & 3 \end{bmatrix}$$

Input: `mat1 = [[1,0,0],[-1,0,3]]`, `mat2 = [[7,0,0],[0,0,0],[0,0,1]]`
 Output: `[[7,0,0],[-7,0,3]]`

Example 2:

Input: mat1 = [[0]], mat2 = [[0]]
 Output: [[0]]

Constraints:

- $m == \text{mat1.length}$
- $k == \text{mat1}[i].\text{length} == \text{mat2.length}$
- $n == \text{mat2}[i].\text{length}$
- $1 \leq m, n, k \leq 100$
- $-100 \leq \text{mat1}[i][j], \text{mat2}[i][j] \leq 100$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def multiply(self, mat1: List[List[int]], mat2: List[List[int]]) -> List[List[int]]:
        m, k, n = len(mat1), len(mat1[0]), len(mat2[0])
        result = [[0] * n for _ in range(m)]

        # For each non-zero in mat1, add its contribution to result
        for i in range(m):
            for k_idx in range(k):
                if mat1[i][k_idx] != 0:
                    val1 = mat1[i][k_idx]

                    for j in range(n):
                        if mat2[k_idx][j] != 0:
                            result[i][j] += val1 * mat2[k_idx][j]

        return result
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Multiply mat1 (m×k) by mat2 (k×n). Return the m×n result.

Both are sparse – many zeros. Optimise to skip zero elements. Right?"

```

#
# "[[1,0,0],[-1,0,3]] × [[7,0,0],[0,0,0],[0,0,1]] = [[7,0,0],[-7,0,3]] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Standard triple loop: for each row i and col k, sum A[i][j]*B[j][k] over j.
# Correct even for sparse inputs but touches every (i,j,k) combination.
# Skipping zero rows/cols from sparse representation saves huge work.
# Time: O(m * k * n). Space: O(m * n) result.
# Should I go ahead and optimize?"

class _311_SparseMultiply_Brute:
    def multiply(self, mat1: List[List[int]], mat2: List[List[int]]) ->
List[List[int]]:
        m, k, n = len(mat1), len(mat1[0]), len(mat2[0])
        result = [[0] * n for _ in range(m)]
        for i in range(m):
            for p in range(k):
                for j in range(n):
                    result[i][j] += mat1[i][p] * mat2[p][j]
        return result

# PHASE 3 — OPTIMIZE
# "Skip multiplication when mat1[i][p] == 0 – avoids the inner j loop entirely.
# Pre-compute non-zero positions in mat1 for even faster access.
# This is the key sparse optimisation: skip zero elements before accessing mat2."

# PHASE 4 — CLEAN CODE
class _311_SparseMultiply:
    def multiply(self, mat1: List[List[int]], mat2: List[List[int]]) ->
List[List[int]]:
        m, k, n = len(mat1), len(mat1[0]), len(mat2[0])
        result = [[0] * n for _ in range(m)]
        for i in range(m):
            for p in range(k):
                if mat1[i][p] == 0:
                    continue # skip – avoids entire inner loop
                for j in range(n):
                    result[i][j] += mat1[i][p] * mat2[p][j]
        return result

# PHASE 5 — TEST & DEBUG
# mat1[0][0]=1, p=0: add 1*mat2[0][j] to result[0]. mat1[0][1]=0 → skip.
# mat1[0][2]=0 → skip.
# result[0]=[7,0,0] ✓. mat1[1][2]=3, p=2: add 3*mat2[2][j]=[0,0,3] →
# result[1]=[-7,0,3] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Worst case O(m*k*n) same as brute force. Best case O(nnz1 * n) where nnz1 =
# non-zeros in mat1.
# For truly sparse matrices this is much faster.

```

Given more time I'd ask: what if both matrices are very large and very sparse?
 # Use CSR (Compressed Sparse Row) format – store only non-zero values and their indices.
 # Hash map approach: for each non-zero in mat1, look up non-zeros in the corresponding row of mat2."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def multiply(self, mat1: list[list[int]],
                mat2: list[list[int]]) -> list[list[int]]:
        m = len(mat1)
        n = len(mat2)
        l = len(mat2[0])
        ans = [[0] * l for _ in range(m)]

        for i in range(m):
            for j in range(l):

                for k in range(n):
                    ans[i][j] += mat1[i][k] * mat2[k][j]

        return ans
```

• LeetDoocs

```
class Solution:
    def multiply(self, mat1: List[List[int]], mat2: List[List[int]]) -> List[List[int]]:
        m, n = len(mat1), len(mat2[0])
        ans = [[0] * n for _ in range(m)]
        for i in range(m):
            for j in range(n):
                for k in range(len(mat2)):
                    ans[i][j] += mat1[i][k] * mat2[k][j]
        return ans
```

• SimplyLeet

```
def sparseMatrixMultiplication(mat1, mat2):
    # Get dimensions of mat1 and mat2
    m, k = len(mat1), len(mat1[0])
    n = len(mat2[0])
    # Initialize result matrix with zeros
    result = [[0] * n for _ in range(m)]

    # Iterate through each row in mat1
    for i in range(m):
        # For each element in row i of mat1
        for p in range(k):
            if mat1[i][p] != 0: # Only consider non-zero element in mat1
                # Multiply with corresponding elements in the row of mat2
                for j in range(n):
                    if mat2[p][j] != 0: # Only consider non-zero element in
mat2
                        result[i][j] += mat1[i][p] * mat2[p][j]

    return result

# Example usage:
print(sparseMatrixMultiplication([[1,0,0],[-1,0,3]], [[7,0,0],[0,0,0],[0,0,1]]
))
```

• LC.ca

```
class Solution:
    def multiply(self, mat1: List[List[int]], mat2: List[List[int]]) -> List[L
ist[int]]:
        def f(mat: List[List[int]]) -> List[List[int]]:
            g = [[] for _ in range(len(mat))]
            for i, row in enumerate(mat):
                for j, x in enumerate(row):
                    if x:
                        g[i].append((j, x))
            return g
```



```
g1 = f(mat1)
g2 = f(mat2)
m, n = len(mat1), len(mat2[0])
ans = [[0] * n for _ in range(m)]
for i in range(m):
    for k, x in g1[i]:
        for j, y in g2[k]:
            ans[i][j] += x * y
return ans
```

◆ Quick Review

Key Insights

- Exploit sparsity by iterating only over non-zero elements.
- For each non-zero element in mat1, multiply it with corresponding non-zero elements in mat2.
- Accumulate the product into the result matrix.
- This approach reduces unnecessary multiplication operations when many elements are zero.

Space & Time

Time Complexity: $O(m * k * n)$ in the worst-case, but significantly lower when many elements are zero.

Space Complexity: $O(m * n)$ for the resulting matrix.

Solution

We loop through each row of mat1 and for every non-zero element, we traverse the corresponding row in mat2. By checking if an element in both matrices is non-zero before multiplying, we avoid redundant calculations. This optimization leverages the sparsity of the matrices. We use standard nested loops along with conditional checks to implement this efficient multiplication.

Matrix

□ #498 Diagonal Traverse

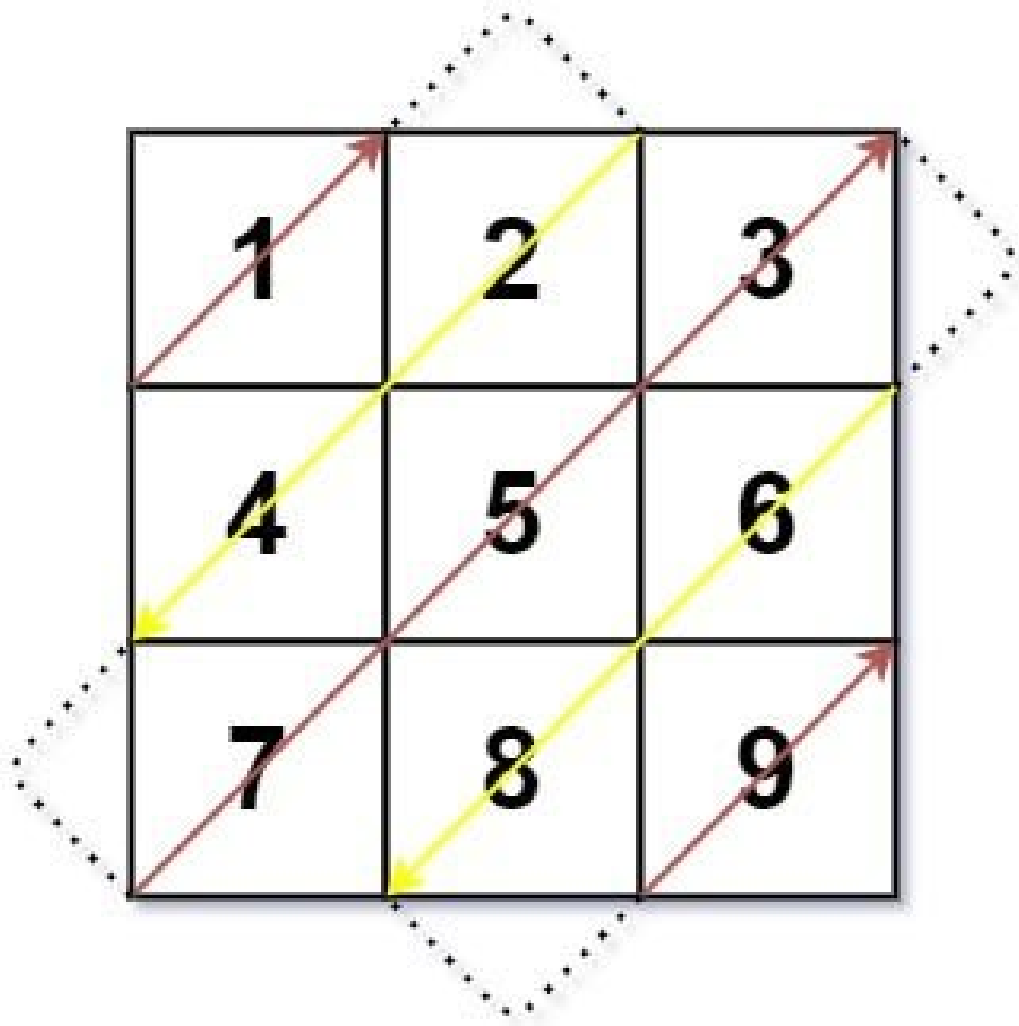
MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix · Simulation
Lists: CodeSignal

Problem

Given an $m \times n$ matrix `mat`, return an array of all the elements of the array in a diagonal order.

Example 1:



Input: mat = [[1,2,3],[4,5,6],[7,8,9]]
Output: [1,2,4,7,5,3,6,8,9]

Example 2:

Input: mat = [[1,2],[3,4]]
Output: [1,2,3,4]

Constraints:

- $m == \text{mat.length}$
- $n == \text{mat}[i].\text{length}$
- $1 \leq m, n \leq 104$
- $1 \leq m * n \leq 104$

- $-105 \leq \text{mat}[i][j] \leq 105$

My LeetCode Solution

• My LeetCode Solution

```
from collections import defaultdict

class Solution:
    def findDiagonalOrder(self, mat: List[List[int]]) -> List[int]:
        if not mat or not mat[0]:
            return []

        groups = defaultdict(list)

        for i in range(len(mat)):
            for j in range(len(mat[0])):
                groups[i + j].append(mat[i][j])

        res = []
        for i in range(len(groups)):
            if i % 2 == 0:
                res.extend(reversed(groups[i]))
            else:
                res.extend(groups[i])

        return res
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Traverse all elements diagonally: alternate up-right then down-left directions.
# There are m+n-1 diagonals. Even-numbered diagonals go up-right, odd go down-left.
# Right?"
#
# "[[1,2,3],[4,5,6],[7,8,9]] → [1,2,4,7,5,3,6,8,9] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Simulate diagonal walk with direction flags; flip direction at matrix boundaries."
```

```

# Track visited or use index math to avoid infinite loops.
# Correct  $O(m \cdot n)$ ; index-formula approach avoids simulation bugs.
# Time:  $O(m \cdot n)$ . Space:  $O(m \cdot n)$  output.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (diagonal grouping)
# "Group elements by diagonal  $d = r+c$  (same sum = same diagonal).
# Diagonal  $d$  contains elements where  $r+c=d$ .
# For even  $d$ : traverse top-to-bottom order reversed (up-right) → sort by  $r$ 
# descending.
# For odd  $d$ : traverse top-to-bottom (down-left) → sort by  $r$  ascending."

# PHASE 4 — CLEAN CODE
class _498_DiagonalTraverse:
    def findDiagonalOrder(self, mat: List[List[int]]) -> List[int]:
        m, n = len(mat), len(mat[0])
        diags: defaultdict = defaultdict(list)
        for r in range(m):
            for c in range(n):
                diags[r + c].append(mat[r][c])
        result = []
        for d in range(m + n - 1):
            if d % 2 == 0:
                result.extend(reversed(diags[d])) # even → up-right (larger  $r$  first
→ reversed)
            else:
                result.extend(diags[d]) # odd → down-left (smaller  $r$  first →
natural)
        return result

# PHASE 5 — TEST & DEBUG
# [[1,2,3],[4,5,6],[7,8,9]]:
#  $d=0$ :  $\{(0,0)=1\}$  even → reversed=[1].  $d=1$ :  $\{(0,1)=2, (1,0)=4\}$  odd → [2,4].
#  $d=2$ :  $\{(0,2)=3, (1,1)=5, (2,0)=7\}$  even → reversed=[7,5,3].
#  $d=3$ :  $\{(1,2)=6, (2,1)=8\}$  odd → [6,8].  $d=4$ :  $\{(2,2)=9\}$  even → reversed=[9].
# → [1,2,4,7,5,3,6,8,9] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(m \cdot n)$ . Space  $O(m \cdot n)$  for the diagonal groups.
# The 'group by  $r+c$ ' insight eliminates direction-tracking complexity.
# Alternative: simulate directly with  $dr/dc$  direction vectors and bounds – same
 $O(m \cdot n)$  but trickier.
# Given more time I'd ask: what if we want anti-diagonals ( $r-c = \text{constant}$ )?
# Same approach – group by  $r-c$ , alternate direction per group."

```

Community Solutions (Python)

• LeetDoocs

```

class Solution:
    def findDiagonalOrder(self, mat: List[List[int]]) -> List[int]:
        m, n = len(mat), len(mat[0])
        ans = []
        for k in range(m + n - 1):
            t = []
            i = 0 if k < n else k - n + 1
            j = k if k < n else n - 1
            while i < m and j >= 0:
                t.append(mat[i][j])

                i += 1
                j -= 1
            if k % 2 == 0:
                t = t[::-1]
            ans.extend(t)
        return ans

```

• SimplyLeet

```

def findDiagonalOrder(mat):
    if not mat or not mat[0]:
        return []

    m, n = len(mat), len(mat[0])
    result = []
    row, col = 0, 0
    direction = 1 # 1 means upward, -1 means downward

    while len(result) < m * n:

        result.append(mat[row][col])
        if direction == 1: # moving upward
            if col == n - 1: # Hit the right boundary
                row += 1
                direction = -1
            elif row == 0: # Hit the top boundary
                col += 1
                direction = -1
        else:
            row -= 1

```



```

        col += 1
    else: # moving downward
        if row == m - 1: # Hit the bottom boundary
            col += 1
            direction = 1
        elif col == 0: # Hit the left boundary
            row += 1
            direction = 1
        else:
            row += 1

```

```

        col -= 1
    return result

```

Example usage:

```
print(findDiagonalOrder([[1,2,3],[4,5,6],[7,8,9]]))
```

• LC.ca

```

class Solution:
    def findDiagonalOrder(self, mat: List[List[int]]) -> List[int]:
        m, n = len(mat), len(mat[0])
        ans = []
        for k in range(m + n - 1):
            t = []
            i = 0 if k < n else k - n + 1
            j = k if k < n else n - 1
            while i < m and j >= 0:
                t.append(mat[i][j])

                i += 1
                j -= 1
            if k % 2 == 0:
                t = t[::-1]
            ans.extend(t)
        return ans

```

◆ Quick Review

Key Insights

- Traverse the matrix diagonally with alternating directions: upward (top-right) and downward (bottom-left).
- When the traversal reaches a boundary (top, bottom, left, or right), adjust the position and change the direction.
- Use simulation to process each element exactly once while managing the indices based on the current direction.

Space & Time

Time Complexity: $O(m * n)$ where m is the number of rows and n is the number of columns, as every element is processed once.

Space Complexity: $O(m * n)$ for the output array (excluding the input matrix), while additional space usage remains constant.

Solution

The solution simulates the diagonal traversal using a direction flag (1 for upward and -1 for downward). Begin at the matrix's top-left corner. Append the current element to the result. Depending on the current direction: If moving upward, decrease the row and increase the column. If moving downward, increase the

row and decrease the column. Adjust for boundary hits: When moving upward: If at the last column, increment the row and switch direction. If at the first row, increment the column and switch direction. When moving downward: If at the last row, increment the column and switch direction. If at the first column, increment the row and switch direction. This process continues until all elements are added to the result array.

Matrix

□ ★ #531 Lonely Pixel I ★

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Matrix
Lists: ★ Premium | Premium 98

Problem

Given an $m \times n$ picture consisting of black 'B' and white 'W' pixels, return the number of black lonely pixels.

A black lonely pixel is a character 'B' that located at a specific position where the same row and same column don't have any other black pixels.

Example 1:

W	W	B
W	B	W
B	W	W

Input: picture = `[["W","W","B"],["W","B","W"],["B","W","W"]]`

Output: 3

Explanation: All the three 'B's are black lonely pixels.

Example 2:

B	B	B
B	B	W
B	B	B

Input: picture = [["B","B","B"],["B","B","W"],["B","B","B"]]
Output: 0

Constraints:

- $m == \text{picture.length}$
- $n == \text{picture}[i].\text{length}$
- $1 \leq m, n \leq 500$
- $\text{picture}[i][j]$ is 'W' or 'B'.

My LeetCode Solution

- My LeetCode Solution

```

class Solution:
    def findLonelyPixel(self, picture: List[List[str]]) -> int:
        rows, cols = len(picture), len(picture[0])

        # Count B's in each row and column
        row_count = [0] * rows
        col_count = [0] * cols

        for i in range(rows):
            for j in range(cols):

                if picture[i][j] == 'B':
                    row_count[i] += 1
                    col_count[j] += 1

        # Check lonely pixels
        lonely = 0
        for i in range(rows):
            # Skip rows that don't have exactly one B
            if row_count[i] != 1:
                continue

            for j in range(cols):
                if picture[i][j] == 'B' and col_count[j] == 1:
                    lonely += 1
                    break # Since row has only one B, we can break

        return lonely

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"A 'B' pixel is lonely if it's the ONLY 'B' in its row AND the only 'B' in its column.

Count lonely pixels. Right?"

#

"[[W,W,B],[W,B,W],[B,W,W]]: each B is alone in its row and column → 3 ✓

[[B,B,B],[B,B,W],[B,B,B]]: every B shares its row or column with another → 0 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For each black pixel B, scan its entire row for other black pixels.

Then scan its entire column – B is lonely if both counts equal 1.

$O(m*n*(m+n))$ – checking row and column per pixel.


```
# Time: O(m * n * (m + n)). Space: O(1).
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Count B's per row and per column in one pass: O(m*n).
```

```
# In second pass, for each B: if row_count[r]==1 and col_count[c]==1, it's lonely."
```

PHASE 4 — CLEAN CODE

```
class _531_LonelyPixel:
    def findLonelyPixel(self, picture: List[List[str]]) -> int:
        m, n = len(picture), len(picture[0])
        row_count = [sum(1 for c in range(n) if picture[r][c] == 'B') for r in
range(m)]
        col_count = [sum(1 for r in range(m) if picture[r][c] == 'B') for c in
range(n)]
        return sum(
            1 for r in range(m) for c in range(n)
            if picture[r][c] == 'B' and row_count[r] == 1 and col_count[c] == 1
        )
```

PHASE 5 — TEST & DEBUG

```
# [[W,W,B],[W,B,W],[B,W,W]]: row_count=[1,1,1]. col_count=[1,1,1].
```

```
# Each B has row=1 and col=1 → all 3 are lonely ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(m*n). Space O(m+n) for row/col counts.
```

```
# Given more time I'd ask: Lonely Pixel II (#533)?
```

```
# Same pixel must be in a column containing exactly N Bs, and the rows of those Bs
must all be identical.
```

```
# Much more complex – requires comparing full rows."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findLonelyPixel(self, picture: list[list[str]]) -> int:
        m = len(picture)
        n = len(picture[0])
        ans = 0
        rows = [0] * m # rows[i] := the number of B's in rows i
        cols = [0] * n # cols[i] := the number of B's in cols i

        for i in range(m):
            for j in range(n):
```

```

        if picture[i][j] == 'B':
            rows[i] += 1
            cols[j] += 1

    for i in range(m):
        if rows[i] == 1: # Only have to examine the rows if rows[i] == 1.
            for j in range(n):
                # After meeting a 'B' in this rows, break and search the next row.
                if picture[i][j] == 'B':
                    if cols[j] == 1:

                        ans += 1
                        break

    return ans

```

• LeetDoocs

```

class Solution:
    def findLonelyPixel(self, picture: List[List[str]]) -> int:
        rows = [0] * len(picture)
        cols = [0] * len(picture[0])
        for i, row in enumerate(picture):
            for j, x in enumerate(row):
                if x == "B":
                    rows[i] += 1
                    cols[j] += 1

        ans = 0

        for i, row in enumerate(picture):
            for j, x in enumerate(row):
                if x == "B" and rows[i] == 1 and cols[j] == 1:
                    ans += 1

        return ans

```

• SimplyLeet

```

# Function to find the number of lonely black pixels in a picture.
def findLonelyPixel(picture):
    # Get dimensions of the picture
    rows = len(picture)
    cols = len(picture[0]) if rows > 0 else 0

    # Initialize row and column counts with zeros
    rowCount = [0] * rows
    colCount = [0] * cols

    # First pass: Count the number of 'B's in each row and column.
    for i in range(rows):
        for j in range(cols):
            if picture[i][j] == 'B':
                rowCount[i] += 1
                colCount[j] += 1

    # Second pass: Check for lonely black pixels.
    lonelyPixels = 0
    for i in range(rows):
        for j in range(cols):
            if picture[i][j] == 'B' and rowCount[i] == 1 and colCount[j] == 1:
                lonelyPixels += 1

    return lonelyPixels

# Example test
picture = [["W","W","B"], ["W","B","W"], ["B","W","W"]]
print(findLonelyPixel(picture)) # Output: 3

```

• LC.ca

```
class Solution:
    def findLonelyPixel(self, picture: List[List[str]]) -> int:
        m, n = len(picture), len(picture[0])
        rows, cols = [0] * m, [0] * n
        for i in range(m):
            for j in range(n):
                if picture[i][j] == 'B':
                    rows[i] += 1
                    cols[j] += 1

        res = 0

        for i in range(m):
            if rows[i] == 1:
                for j in range(n):
                    if picture[i][j] == 'B' and cols[j] == 1:
                        res += 1
                        break

        return res
```

◆ Quick Review

Key Insights

- Count the number of 'B' pixels in each row.
- Count the number of 'B' pixels in each column.
- A pixel is considered lonely if the count in its corresponding row and column is exactly 1.
- Two passes through the matrix can efficiently solve the problem: one for counting and one for checking conditions.

Space & Time

Time Complexity: $O(m * n)$, where m and n are the dimensions of the picture.

Space Complexity: $O(m + n)$, used to store the row and column counts.

Solution

The solution works by first scanning the entire matrix to compute the number of black pixels in each row and each column using two arrays/lists. Once these counts are available, a second pass through the matrix determines for each black pixel if it is lonely by checking if its corresponding row and column count equals 1. The approach uses simple array data structures for tallying counts and ensures that each element is processed only a constant number of times.

Matrix

□ ★ #723 Candy Crush ★

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Matrix · Simulation
Lists: ★ Premium | Premium 98

Problem

This question is about implementing a basic elimination algorithm for Candy Crush.

Given an $m \times n$ integer array `board` representing the grid of candy where `board[i][j]` represents the type of candy. A value of `board[i][j] == 0` represents that the cell is empty.

The given board represents the state of the game following the player's move. Now, you need to restore the board to a stable state by crushing candies according to the following rules:

- If three or more candies of the same type are adjacent vertically or horizontally, crush them all at the same time – these positions become empty.

- After crushing all candies simultaneously, if an empty space on the board has candies on top of itself, then these candies will drop until they hit a candy or bottom at the same time. No new candies will drop outside the top boundary.
- After the above steps, there may exist more candies that can be crushed. If so, you need to repeat the above steps.
- If there does not exist more candies that can be crushed (i.e., the board is stable), then return the current board.

You need to perform the above rules until the board becomes stable, then return the stable board.

Example 1:



Input: board = [[110,5,112,113,114],[210,211,5,213,214],[310,311,3,313,314],[410,411,412,5,414],[5,1,512,3,3],[610,4,1,613,614],[710,1,2,713,714],[810,1,2,1,1],[1,1,2,2,2],[4,1,4,4,1014]]

Output: [[0,0,0,0,0],[0,0,0,0,0],[0,0,0,0,0],[110,0,0,0,114],[210,0,0,0,214],[310,0,0,113,314],[410,0,0,213,414],[610,211,112,313,614],[710,311,412,613,714],[810,411,512,713,1014]]

Example 2:

Input: board = [[1,3,5,5,2],[3,4,3,3,1],[3,2,4,5,2],[2,4,4,5,5],[1,4,4,1,1]]

Output: [[1,3,0,0,0],[3,4,0,5,2],[3,2,0,3,1],[2,4,0,5,2],[1,4,3,1,1]]

Constraints:

- m == board.length
- n == board[i].length
- 3 <= m, n <= 50
- 1 <= board[i][j] <= 2000

Editorial Note (Python)

• Editorial Note

```
class Solution:
    def candyCrush(self, board: List[List[int]]) -> List[List[int]]:
        m, n = len(board), len(board[0])

        def find():
            crushed_set = set()

            # Check vertically adjacent candies
            for r in range(1, m - 1):
                for c in range(n):
                    if board[r][c] == 0:
                        continue
                    if board[r][c] == board[r - 1][c] == board[r + 1][c]:
                        crushed_set.add((r, c))
                        crushed_set.add((r - 1, c))
                        crushed_set.add((r + 1, c))

            # Check horizontally adjacent candies
            for r in range(m):
                for c in range(1, n - 1):
                    if board[r][c] == 0:
                        continue
                    if board[r][c] == board[r][c - 1] == board[r][c + 1]:
                        crushed_set.add((r, c))
                        crushed_set.add((r, c - 1))
                        crushed_set.add((r, c + 1))

            return crushed_set

        # Set the value of each candies to be crushed as 0
        def crush(crushed_set):
```

```

        for (r, c) in crushed_set:
            board[r][c] = 0

    def drop():
        for c in range(n):
            lowest_zero = -1

            # Iterate over each column
            for r in range(m - 1, -1, -1):
                if board[r][c] == 0:

                    lowest_zero = max(lowest_zero, r)

            # Swap current non-zero candy with the lowest zero.
            elif lowest_zero >= 0:
                board[r][c], board[lowest_zero][c] = board[lowest_zero
][c], board[r][c]

                lowest_zero -= 1

            # Continue with the three steps until we can no longer find any
            crushable candies.
            crushed_set = find()
            while crushed_set:

                crush(crushed_set)
                drop()
                crushed_set = find()

    return board

```

My LeetCode Solution

• My LeetCode Solution

```

class Solution:
    def candyCrush(self, board: List[List[int]]) -> List[List[int]]:
        m, n = len(board), len(board[0])

        while True:
            crushable = set()

            # Horizontal and vertical in one loop
            for i in range(m):
                for j in range(n):

                    if i < m - 2 and board[i][j] == board[i+1][j] == board[i+2][j] != 0:
                        crushable.update([(i,j), (i+1,j), (i+2,j)])
                    if j < n - 2 and board[i][j] == board[i][j+1] == board[i][j+2] != 0:
                        crushable.update([(i,j), (i,j+1), (i,j+2)])

            if not crushable:
                return board

            # Crush and apply gravity in one pass
            for j in range(n):

                col = [board[i][j] for i in range(m) if (i,j) not in crushable]

                col = [0] * (m - len(col)) + col # Zeros on top
                for i in range(m):
                    board[i][j] = col[i]

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Repeatedly: mark 3+ same-type candies horizontally or vertically, crush them (set to 0),
 # then drop remaining candies down. Repeat until stable. Return final board. Right?"
 #
 # "Mark all runs of 3+ simultaneously, crush all at once, then gravity – not one at a time."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."
 # Repeat until stable: find horizontal/vertical runs of 3+, mark for crush, drop gravity.

```
# One full board scan per cascade step.
# Correct simulation; finding all runs in one pass per step is cleaner.
# Time:  $O((m*n)^2)$  worst cascades. Space:  $O(m * n)$  mark board.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (in-place marking)

```
# "Mark by negating values (sign flip) – avoids extra space.
# Step 1: scan rows and cols, negate any value in a run of 3+.
# Step 2: zero out all negatives (crush).
# Step 3: for each column, compact non-zeros to the bottom (gravity).
# Repeat until no changes in step 1. Time:  $O((m*n)^2)$  worst case but small in practice."
```

PHASE 4 — CLEAN CODE

```
class _723_CandyCrush:
    def candyCrush(self, board: List[List[int]]) -> List[List[int]]:
        m, n = len(board), len(board[0])
        while True:
            crush = set()
            for r in range(m): # mark horizontal runs
                for c in range(n - 2):
                    v = abs(board[r][c])
                    if v and v == abs(board[r][c+1]) == abs(board[r][c+2]):
                        crush.update([(r,c),(r,c+1),(r,c+2)])
            for r in range(m - 2): # mark vertical runs
                for c in range(n):
                    v = abs(board[r][c])
                    if v and v == abs(board[r+1][c]) == abs(board[r+2][c]):
                        crush.update([(r,c),(r+1,c),(r+2,c)])
            if not crush:
                break
            for r, c in crush:
                board[r][c] = 0
            for c in range(n): # gravity: compact column downward
                write = m - 1
                for r in range(m - 1, -1, -1):
                    if board[r][c] != 0:
                        board[write][c] = board[r][c]
                        if write != r:
                            board[r][c] = 0
                        write -= 1
            return board
```

PHASE 5 — TEST & DEBUG

```
# "[[1,3,5,5,2],[3,4,3,3,1],[3,2,4,5,2],[2,4,4,5,5],[1,4,4,1,1]]:
# Horizontal: row 1 has [3,3,3] at c=1,2,3. Row 3 has [4,4] and [5,5]...
# After crushing and dropping → stable board matches expected output ✓"
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# " $O((m*n)^2)$  worst case – at most  $m*n$  crush iterations, each  $O(m*n)$ ."
```

For small boards (m,n≤50) this is fast. The abs() trick avoids needing a separate mark array.
 # The gravity step uses a two-pointer write-head approach per column.
 # Given more time I'd ask: what if candies can crush diagonally too?
 # Add diagonal scan to the marking step – same structure."

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def candyCrush(self, board: List[List[int]]) -> List[List[int]]:
        m, n = len(board), len(board[0])
        run = True
        while run:
            run = False
            for i in range(m):
                for j in range(2, n):
                    if board[i][j] and abs(board[i][j]) == abs(board[i][j - 1]
) == abs(
                        board[i][j - 2]
                    ):
                        run = True
                        board[i][j] = board[i][j - 1] = board[i][j - 2] = -abs
(
                        board[i][j]
                    )
            for j in range(n):
                for i in range(2, m):
                    if board[i][j] and abs(board[i][j]) == abs(board[i - 1][j]
) == abs(
                        board[i - 2][j]
                    ):

```

```

        run = True
        board[i][j] = board[i - 1][j] = board[i - 2][j] = -abs
(
        board[i][j]
    )
    if run:
        for j in range(n):
            k = m - 1
            for i in range(m - 1, -1, -1):
                if board[i][j] > 0:
                    board[k][j] = board[i][j]

            k -= 1
        while k >= 0:
            board[k][j] = 0
            k -= 1

    return board

```

• SimplyLeet

```

def candyCrush(board):
    # Retrieve board dimensions
    m, n = len(board), len(board[0])
    # Loop until no crushable candies remain
    while True:
        # Set to hold all positions that need to be crushed
        crush_set = set()

        # Check horizontally for crushable candies
        for i in range(m):

```

```

        for j in range(n - 2):
            # Check if candy exists and three consecutive candies are the
same
            if board[i][j] != 0 and abs(board[i][j]) == abs(board[i][j+1])
== abs(board[i][j+2]):
                crush_set |= {(i, j), (i, j+1), (i, j+2)}

        # Check vertically for crushable candies
        for i in range(m - 2):
            for j in range(n):
                if board[i][j] != 0 and abs(board[i][j]) == abs(board[i+1][j])
== abs(board[i+2][j]):
                    crush_set |= {(i, j), (i+1, j), (i+2, j)}

        # If no candies need to be crushed, the board is stable; break out of
the loop.
        if not crush_set:
            break

        # Set all marked positions to 0 (simulate crushing)
        for i, j in crush_set:
            board[i][j] = 0

        # Apply gravity: For each column, move uncrushed candies downwards

    for j in range(n):
        # Pointer for placing the next candy from bottom-up
        write_index = m - 1
        # Traverse each column from bottom to top
        for i in range(m - 1, -1, -1):
            if board[i][j] != 0:
                board[write_index][j] = board[i][j]
                write_index -= 1
        # Fill remaining positions with 0
        for i in range(write_index, -1, -1):

```



```

        board[i][j] = 0

    return board

# Example usage:
board = [[110,5,112,113,114],[210,211,5,213,214],[310,311,3,313,314],
         [410,411,412,5,414],[5,1,512,3,3],[610,4,1,613,614],
         [710,1,2,713,714],[810,1,2,1,1],[1,1,2,2,2],[4,1,4,4,1014]]
print(candyCrush(board))

```

• LC.ca

```

class Solution:
    def candyCrush(self, board: List[List[int]]) -> List[List[int]]:
        m, n = len(board), len(board[0])
        run = True
        while run:
            run = False
            for i in range(m):
                for j in range(n - 2):
                    if (
                        board[i][j] != 0
                        and abs(board[i][j]) == abs(board[i][j + 1])
                        and abs(board[i][j]) == abs(board[i][j + 2])
                    ):
                        run = True
                        board[i][j] = board[i][j + 1] = board[i][j + 2] = -abs(
                            board[i][j]
                        )
            for j in range(n):
                for i in range(m - 2):
                    if (

```

```

        board[i][j] != 0
        and abs(board[i][j]) == abs(board[i + 1][j])
        and abs(board[i][j]) == abs(board[i + 2][j])
    ):
        run = True
        board[i][j] = board[i + 1][j] = board[i + 2][j] = -abs
(
        board[i][j]
    )
    if run:
        for j in range(n):

            curr = m - 1
            for i in range(m - 1, -1, -1):
                if board[i][j] > 0:
                    board[curr][j] = board[i][j]
                    curr -= 1
            while curr > -1:
                board[curr][j] = 0
                curr -= 1

    return board

```

◆ Quick Review

Key Insights

- Use a simulation approach: repeatedly detect crushable candies and simulate their removal.
- Traverse horizontally and vertically to mark candies that are in groups of three or more.
- Instead of removing candies immediately, mark them for removal and update the board once for the entire pass.
- After crushing, simulate gravity by shifting non-zero candies downward in each column.

- Continue the process until no further candy crushes can be detected.

Space & Time

Time Complexity: $O((m \times n)^2)$ in the worst case since each iteration includes scanning the $m \times n$ grid and then applying gravity.

Space Complexity: $O(m \times n)$ for maintaining additional markers or flags (or using the board itself with in-place modifications).

Solution

The approach involves iteratively scanning the board to identify candy groups that should be crushed using two passes: **Horizontal Check** – For each row, check segments of 3 consecutive candies. **Vertical Check** – For each column, check segments of 3 consecutive candies. If any candy qualifies, mark its position for crushing. After marking, update the board by turning all marked positions into 0. Then, for each column, drop down non-zero candies such that all empty cells (zeros) are at the top. Repeat this process until no group of 3 or

more adjacent candies is found. Key data structures include the board array and a temporary marker (can be implicit by using signs or a separate boolean matrix) for tracking candies to crush. The algorithm ensures a stable board before returning it.

Matrix

□ #835 Image Overlap

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Matrix
Lists: CodeSignal

Problem

You are given two images, `img1` and `img2`, represented as binary, square matrices of size $n \times n$. A binary matrix has only 0s and 1s as values.

We translate one image however we choose by sliding all the 1 bits left, right, up, and/or down any number of units. We then place it on top of the other image. We can then calculate the overlap by counting the number of positions that have a 1 in both images.

Note also that a translation does not include any kind of rotation. Any 1 bits that are translated outside of the matrix borders are erased.

Return the largest possible overlap.

Example 1:

1	1	0
0	1	0
0	1	0

img1

0	0	0
0	1	1
0	0	1

img2

Input: `img1 = [[1,1,0],[0,1,0],[0,1,0]]`, `img2 = [[0,0,0],[0,1,1],[0,0,1]]`

Output: 3

Explanation: We translate `img1` to right by 1 unit and down by 1 unit.

The number of positions that have a 1 in both images is 3 (shown in red).

Example 2:

Input: `img1 = [[1]]`, `img2 = [[1]]`

Output: 1

Example 3:

Input: `img1 = [[0]]`, `img2 = [[0]]`

Output: 0

Constraints:

- `n == img1.length == img1[i].length`
- `n == img2.length == img2[i].length`
- `1 <= n <= 30`
- `img1[i][j]` is either 0 or 1.

- `img2[i][j]` is either 0 or 1.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def largestOverlap(self, A: List[List[int]], B: List[List[int]]) -> int:
        n = len(A)

        # Get positions of 1s
        ones_A = [(i, j) for i in range(n) for j in range(n) if A[i][j]]
        ones_B = [(i, j) for i in range(n) for j in range(n) if B[i][j]]

        # Count translations needed to align each pair
        from collections import Counter

        shifts = Counter((r2 - r1, c2 - c1) for r1, c1 in ones_A for r2, c2 in
            ones_B)

        return max(shifts.values()) if shifts else 0
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Translate `img1` over `img2` (no rotation) and count overlapping 1s.

Find the translation that maximises overlap. Return max overlap count. Right?"

#

"`img1=[[1,1,0],[0,1,0],[0,1,0]]`, `img2=[[0,0,0],[0,1,1],[0,0,1]]` → 3 ✓ (translate right 1, down 1)"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Try every translation vector (dx,dy) aligning A onto B.

For each shift, count overlapping 1-cells.

$O(n^4)$ translations on $n \times n$ images – too slow for $n=30$.

Time: $O(n^4)$. Space: $O(1)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (translation vector counting)

"Collect all 1-positions in each image.

For each pair (`p1` from `img1`, `p2` from `img2`), the translation to align `p1` with `p2` is (`p2.r-p1.r`, `p2.c-p1.c`).

Count the most common translation vector – that's the maximum overlap.
 # Time: $O(A*B)$ where A, B = number of 1s. Space: $O(A*B)$. For $n=30$, $A, B \leq 900$."

PHASE 4 — CLEAN CODE

```
class _835_ImageOverlap:
    def largestOverlap(self, img1: List[List[int]], img2: List[List[int]]) -> int:
        n = len(img1)
        ones1 = [(r, c) for r in range(n) for c in range(n) if img1[r][c] == 1]
        ones2 = [(r, c) for r in range(n) for c in range(n) if img2[r][c] == 1]
        translations: Counter = Counter()
        for r1, c1 in ones1:
            for r2, c2 in ones2:
                translations[(r2 - r1, c2 - c1)] += 1
        return max(translations.values(), default=0)
```

PHASE 5 — TEST & DEBUG

ones1=[(0,0),(0,1),(1,1),(2,1)]. ones2=[(1,1),(1,2),(2,2)].
 # Translation (1,1) appears when: (0,0)→(1,1)✓, (0,1)→(1,2)✓, (1,1)→(2,2)✓ →
 count=3.
 # max=3 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(A*B)$ where A, B = number of 1s in each image, at most n^2 .
 # Space $O(A*B)$ for the translation Counter.
 # For dense images (many 1s), this is $O(n^4)$ – same as brute force.
 # For sparse images, much faster.
 # The key insight: instead of iterating over all translations, only count translations
 # where at least one 1-pair aligns – no wasted iterations.
 # Given more time I'd ask: what if rotation is also allowed?
 # Need to try all rotation+translation combos – much harder."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def largestOverlap(self, img1: list[list[int]], img2: list[list[int]]) -> int:
        MAGIC = 100
        ones1 = [(i, j)
                  for i, row in enumerate(img1)
                  for j, num in enumerate(row)
                  if num == 1]
        ones2 = [(i, j)
                  for i, row in enumerate(img2)
                  for j, num in enumerate(row)
```

```

        if num == 1]
offsetCount = collections.Counter()

for ax, ay in ones1:
    for bx, by in ones2:
        offsetCount[(ax - bx) * MAGIC + (ay - by)] += 1

return max(offsetCount.values()) if offsetCount else 0

```

• LeetDoocs

```

class Solution:
    def largestOverlap(self, img1: List[List[int]], img2: List[List[int]]) ->
int:
    n = len(img1)
    cnt = Counter()
    for i in range(n):
        for j in range(n):
            if img1[i][j]:
                for h in range(n):
                    for k in range(n):
                        if img2[h][k]:

                            cnt[(i - h, j - k)] += 1

    return max(cnt.values()) if cnt else 0

```

• SimplyLeet

```

def largestOverlap(img1, img2):
    # Get the dimension of the square images
    n = len(img1)

    # Gather positions (coordinates) where there is a 1 in img1 and img2
    ones_img1 = []
    ones_img2 = []
    for i in range(n):
        for j in range(n):
            if img1[i][j] == 1:

```

```

        ones_img1.append((i, j))
    if img2[i][j] == 1:
        ones_img2.append((i, j))

# Dictionary to count translation shifts that align 1's of img1 with img2
translation_count = {}
max_overlap = 0

# Iterate over every pair of 1's from both images
for (x1, y1) in ones_img1:
    for (x2, y2) in ones_img2:
        # Calculate the translation vector needed to align img1's 1 with
img2's 1
        delta = (x2 - x1, y2 - y1)
        # Update the count for this translation vector
        translation_count[delta] = translation_count.get(delta, 0) + 1
        # Update max_overlap if we find a higher count
        max_overlap = max(max_overlap, translation_count[delta])

    return max_overlap

# Example usage:
img1 = [[1,1,0],[0,1,0],[0,1,0]]
img2 = [[0,0,0],[0,1,1],[0,0,1]]
print(largestOverlap(img1, img2)) # Expected output: 3

```

• LC.ca

```

class Solution:
    def largestOverlap(self, img1: List[List[int]], img2: List[List[int]]) ->
int:
        n = len(img1)
        cnt = Counter()
        for i in range(n):
            for j in range(n):
                if img1[i][j]:
                    for h in range(n):
                        for k in range(n):
                            if img2[h][k]:

```

```
cnt[(i - h, j - k)] += 1  
return max(cnt.values()) if cnt else 0
```

◆ Quick Review

Key Insights

- You only need to consider the positions of 1's in each matrix.
- The overlap count for a given translation can be computed by comparing the relative differences between the positions of 1's in the two images.
- Using a hash map (or dictionary) to count how many times each translation vector appears lets you compute the maximum overlap efficiently.
- The translation operation is equivalent to shifting the positions, so the best possible translation is the one that aligns the most pairs of 1's.

Space & Time

Time Complexity: $O(n^4)$ in the worst-case when using a nested iteration over positions for both matrices, though the average situation is typically better since not all cells

are 1.

Space Complexity: $O(n^2)$ for storing the positions of 1's and the counts for translation vectors.

Solution

The solution involves the following steps:
Traverse both images to record the coordinates of all 1's. For every pair of 1's (one from the first image, one from the second), compute the translation vector (offset) that aligns them. Use a hash map to count how many times each offset appears. The maximum count recorded in the hash map is the largest overlap achievable by any translation. This approach avoids simulating every possible slide over the matrix and focuses only on relative alignments of 1's.

Matrix

□ ★ #1198 Find Smallest Common Element in All Rows ★

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Binary Search · Matrix ·
Counting

Lists: ★ Premium | Premium 98

Problem

Given an $m \times n$ matrix `mat` where every row is sorted in strictly increasing order, return the smallest common element in all rows.

If there is no common element, return -1 .

Example 1:

```
Input: mat = [[1,2,3,4,5],[2,4,5,8,10],[3,5,7,9,11],[1,3,5,7,9]]  
Output: 5
```

Example 2:

```
Input: mat = [[1,2,3],[2,3,4],[2,3,5]]  
Output: 2
```

Constraints:

- $m == \text{mat.length}$
- $n == \text{mat}[i].\text{length}$

- $1 \leq m, n \leq 500$
- $1 \leq \text{mat}[i][j] \leq 10^4$
- $\text{mat}[i]$ is sorted in strictly increasing order.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def smallestCommonElement(self, mat: List[List[int]]) -> int:
        rows = len(mat)
        counts = collections.Counter()

        for row in mat:
            for num in row:
                counts[num] += 1
                if counts[num] == rows:
                    return num

        return -1
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the smallest integer that appears in every row. Each row is sorted.

Return -1 if no common element exists. Right?

Values are $1-10^4$, at most 500 rows and 500 cols."

#

"[[1,2,3,4,5],[2,4,5,8,10],[3,5,7,9,11],[1,3,5,7,9]] → 5 (appears in all rows) ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Intersect row0 with row1, then intersect result with row2, and so on.

Track the maximum value in the final intersection set.

Each intersection pass is $O(m)$ per row using a hash set of row values.

Time: $O(\text{rows} * \text{cols})$. Space: $O(\text{cols})$ set.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (frequency count)

"Count how many rows each value appears in using a single array of size $\text{max_val}+1$.

Walk all rows: for each value encountered, increment $\text{count}[\text{value}]$.


```
# Return the smallest value with count == m (appears in ALL rows).
# Time: O(m*n). Space: O(max_val)."
```

PHASE 4 — CLEAN CODE

```
class _1198_SmallestCommonElement:
    def smallestCommonElement(self, mat: List[List[int]]) -> int:
        m = len(mat)
        freq = Counter()
        for row in mat:
            for val in row:
                freq[val] += 1
        for val in range(1, 10001):
            if freq[val] == m:
                return val
        return -1
```

PHASE 5 — TEST & DEBUG

```
# [[1,2,3,4,5],[2,4,5,8,10],[3,5,7,9,11],[1,3,5,7,9]]:
# freq[5] = 4 (appears in all 4 rows). freq[1]=2, freq[3]=3, etc.
# Scanning 1..10000: first val with freq==4 is 5 → return 5 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(m*n + max_val). Space O(max_val).
# Alternative: binary search per row. For each candidate, binary search in all m
rows.
# Start candidates from mat[0] (smallest row), binary search each of the other rows.
# O(n + m * n * log n) – worse than the counting approach for dense matrices.
# Given more time I'd ask: what if there's no bound on values?
# Use a hash Counter for freq – O(m*n) time, O(distinct values) space."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def smallestCommonElement(self, mat: list[list[int]]) -> int:
        MAX = 10000
        count = [0] * (MAX + 1)

        for row in mat:
            for a in row:
                count[a] += 1
            if count[a] == len(mat):
                return a

        return -1
```

• LeetCode

```
class Solution:
    def smallestCommonElement(self, mat: List[List[int]]) -> int:
        cnt = Counter()
        for row in mat:
            for x in row:
                cnt[x] += 1
            if cnt[x] == len(mat):
                return x
        return -1
```

• SimplyLeet

```
def smallestCommonElement(mat):
    # Get number of rows and columns
    m = len(mat)
    n = len(mat[0])

    # Helper function: binary search for target in a row
    def binary_search(row, target):
        left, right = 0, n - 1
        while left <= right:
            mid = (left + right) // 2

            if row[mid] == target:
                return True
            elif row[mid] < target:
                left = mid + 1
            else:
                right = mid - 1
        return False

    # Iterate over each candidate in the first row
    for candidate in mat[0]:
```

```

is_common = True
# Check each subsequent row for the candidate using binary search
for i in range(1, m):
    if not binary_search(mat[i], candidate):
        is_common = False
        break
if is_common:
    return candidate
return -1

# Example usage:
print(smallestCommonElement([[1,2,3,4,5],[2,4,5,8,10],[3,5,7,9,11],[1,3,5,7,9]
]))

```

• LC.ca

```

class Solution:
    def smallestCommonElement(self, mat: List[List[int]]) -> int:
        cnt = Counter()
        for row in mat:
            for x in row:
                cnt[x] += 1
            if cnt[x] == len(mat):
                return x
        return -1

```

◆ Quick Review

Key Insights

- Since every row is strictly increasing, binary search can be used efficiently to check for the presence of an element in a row.
- Iterating over the first row's elements and verifying their presence in all other rows is an effective approach.

- Early termination is possible if an element is not found in any one row.
- Time complexity is manageable given the constraints, even using binary search for each candidate across rows.

Space & Time

Time Complexity: $O(m * n * \log(n))$ where m is the number of rows and n is the number of columns (binary search in each row for each candidate from the first row).

Space Complexity: $O(1)$ (only a few auxiliary variables are used).

Solution

The solution iterates over each element (candidate) in the first row of the matrix. For each candidate, it checks if the candidate is present in every other row using binary search, taking advantage of the sorted order. If a candidate is found in every row, return it immediately as it will be the smallest such common element because the first row is processed in order. If no candidate is common across all rows, return -1 .

Trees & BST

Round 2 · High · Medium · 24 questions

Trees & BST

□ #98 Validate Binary Search Tree

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BST
Lists: Grind 169

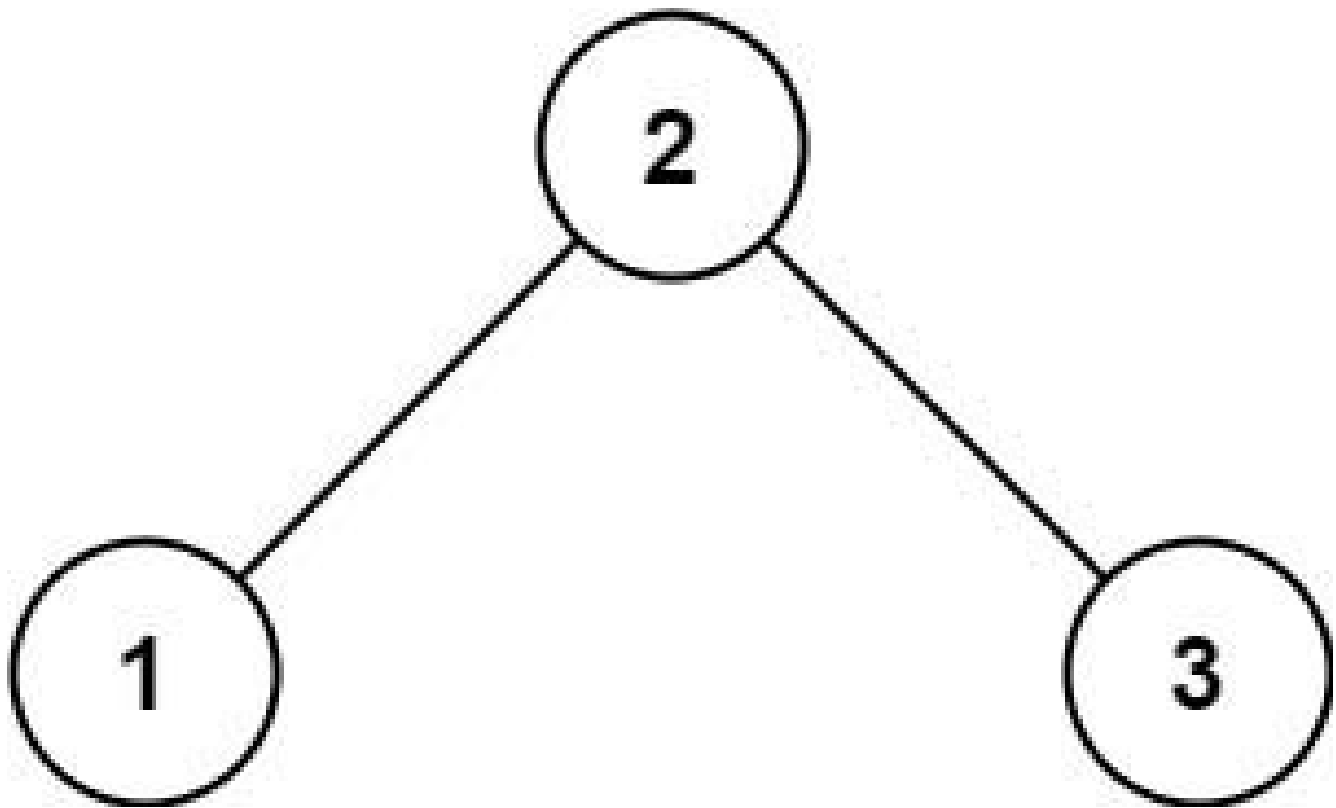
Problem

Given the root of a binary tree, determine if it is a valid binary search tree (BST).

A valid BST is defined as follows:

- The left subtree of a node contains only nodes with keys strictly less than the node's key.
- The right subtree of a node contains only nodes with keys strictly greater than the node's key.
- Both the left and right subtrees must also be binary search trees.

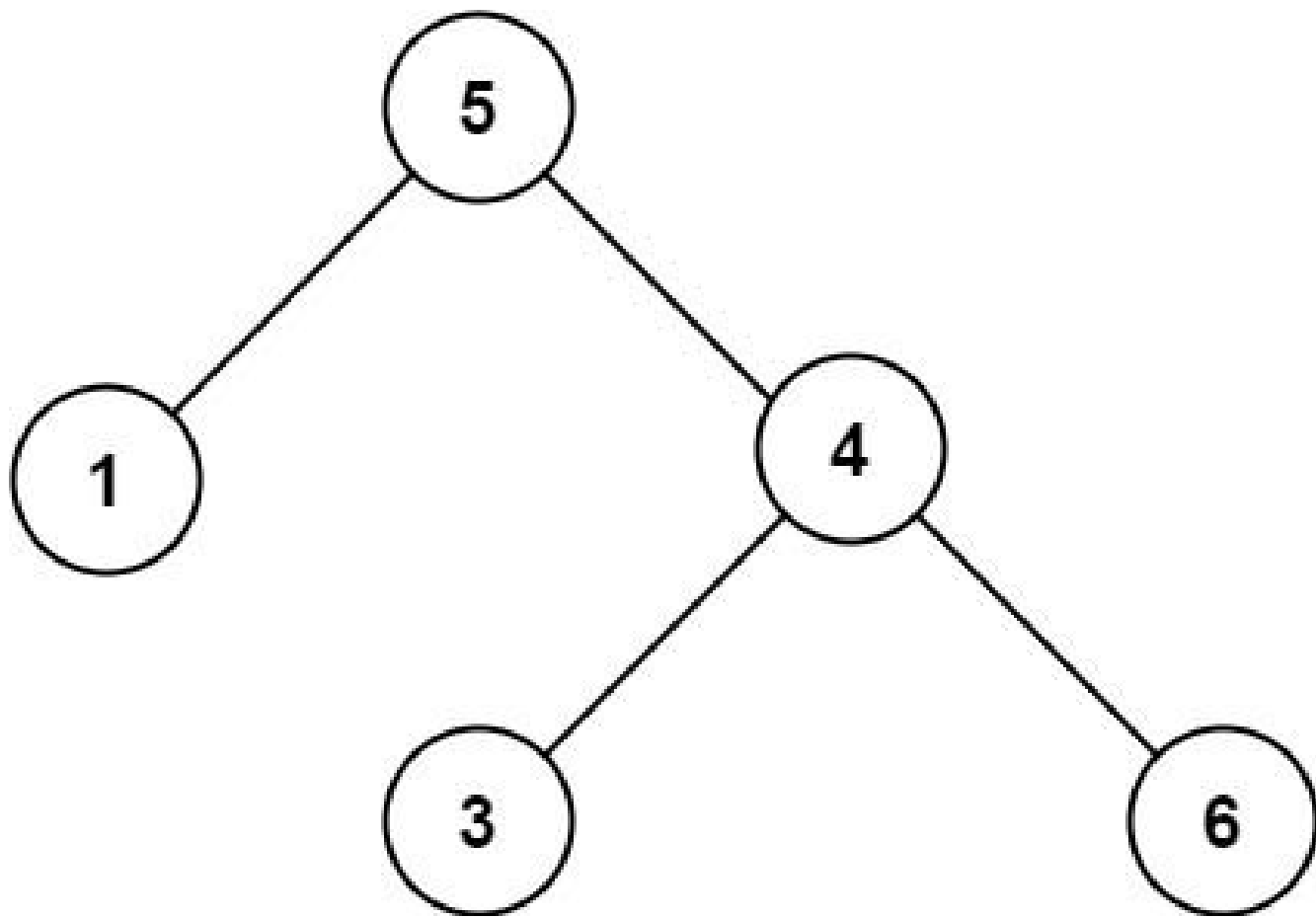
Example 1:



Input: root = [2,1,3]

Output: true

Example 2:



Input: root = [5,1,4,null,null,3,6]

Output: false

Explanation: The root node's value is 5 but its right child's value is 4.

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $-2^{31} \leq \text{Node.val} \leq 2^{31} - 1$

My LeetCode Solution

- My LeetCode Solution

```

class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        def dfs(node, lo, hi):
            if not node:
                return True
            if not (lo < node.val < hi):
                return False
            return dfs(node.left, lo, node.val) and dfs(node.right, node.val,
hi)

        return dfs(root, float('-inf'), float('inf'))

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Every node's value must be strictly greater than all values in its left subtree
AND strictly less than all values in its right subtree – not just the immediate
parent. Right?
The classic trap: [5,1,4,null,null,3,6] – root=5, right child=4 – 4<5 → invalid ✓"

"[2,1,3]: 1<2<3 → true ✓. [5,1,4,null,null,3,6]: 4<5 → false ✓."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
In-order traversal collects values; verify the list is strictly increasing.
Any duplicate or decrease means invalid BST.
Correct O(n), but recursion can pass min/max bounds instead of storing all values.
Time: O(n). Space: O(n) in-order list.
Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (bounds propagation)

"Pass min/max bounds down the tree.
Every node must satisfy min_val < node.val < max_val.
Left child: max_val = node.val.
Right child: min_val = node.val.
Use float('-inf') and float('inf') as initial bounds. Use inner helper."

PHASE 4 — CLEAN CODE

```

class _98_ValidateBST:
    def isValidBST(self, root: Optional['TreeNode']) -> bool:
        def valid(node, min_val, max_val):
            if not node:
                return True
            if not (min_val < node.val < max_val):
                return False
            return (valid(node.left, min_val, node.val) and

```

```
        valid(node.right, node.val, max_val))
    return valid(root, float('-inf'), float('inf'))
```

PHASE 5 — TEST & DEBUG

[2,1,3]: valid(2,-inf,inf)→valid(1,-inf,2)→valid(None,...)=T and T→T.

valid(3,2,inf)→T. → True ✓

[5,1,4,null,null,3,6]: valid(4,5,inf): 4 NOT in (5,inf) → False → outer returns False ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time O(n). Space O(h) for the call stack.

The bounds approach is the correct one – in-order checking works too but requires state.

The common mistake: only checking node vs parent, not the full subtree constraint.

Given more time I'd ask: what if duplicates are allowed (left ≤ node < right)?

Change the strict inequalities accordingly."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isValidBST(self, root: TreeNode | None) -> bool:
        def isValidBST(root: TreeNode | None,
                        minNode: TreeNode | None, maxNode: TreeNode | None) -> bool:
            :
            if not root:
                return True
            if minNode and root.val <= minNode.val:
                return False
            if maxNode and root.val >= maxNode.val:
                return False

            return (isValidBST(root.left, minNode, root) and
                    isValidBST(root.right, root, maxNode))

        return isValidBST(root, None, None)
```

```

class Solution:
    def isValidBST(self, root: TreeNode | None) -> bool:
        stack = []
        pred = None

        while root or stack:
            while root:
                stack.append(root)
                root = root.left
            root = stack.pop()

            if pred and pred.val >= root.val:
                return False
            pred = root
            root = root.right

        return True

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        def dfs(root: Optional[TreeNode]) -> bool:
            if root is None:
                return True
            if not dfs(root.left):
                return False
            nonlocal prev
            if prev >= root.val:
                return False
            prev = root.val
            return dfs(root.right)

        prev = -inf

        return dfs(root)

```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val      # Node's value
        self.left = left    # Left child
        self.right = right  # Right child

class Solution:
    def isValidBST(self, root: TreeNode) -> bool:
        # Helper function to recursively validate nodes

        def validate(node, low, high):
            if not node:
                return True # An empty node/subtree is valid
            # If current node's value is out of the allowed range, return False
            if not (low < node.val < high):
                return False
            # Recursively check the left subtree and right subtree
            return validate(node.left, low, node.val) and validate(node.right,
node.val, high)

        return validate(root, float("-inf"), float("inf"))
```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
```

```
# self.left = left
# self.right = right

class Solution: # recursive
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        def dfs(root, mi, ma):
            if not root:
                return True
            return mi < root.val < ma and dfs(root.left, mi, root.val) and dfs(
                root.right, root.val, ma)

        return dfs(root, -math.inf, math.inf)

class Solution: # iterative
    def isValidBST(self, root: TreeNode) -> bool:
        # store (node, lower, upper) tuples in a single queue
        queue = [(root, None, None)]

        # another option:
        # queue = [(root, -math.inf, math.inf)]

        while queue:
            node, lower, upper = queue.pop(0)

            if node is None:
                continue

            val = node.val
            if lower is not None and val <= lower:
                return False
```

```

        if upper is not None and val >= upper:
            return False

        queue.append((node.right, val, upper))
        queue.append((node.left, lower, val))

    return True

#####

class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        def isValidBST(root: Optional[TreeNode],
                        minNode: Optional[TreeNode], maxNode: Optional[TreeNode]) -
> bool:
            if not root:
                return True
            if minNode and root.val <= minNode.val:
                return False
            if maxNode and root.val >= maxNode.val:
                return False

            return isValidBST(root.left, minNode, root) and isValidBST(root.right, r
oot, maxNode)

        return isValidBST(root, None, None)

#####

# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):

```

```

# self.val = val
# self.left = left
# self.right = right
class Solution:
    def isValidBST(self, root: TreeNode) -> bool:
        def dfs(root):
            nonlocal prev
            if root is None:
                return True
            if not dfs(root.left):
                return False
            if prev >= root.val:
                return False
            prev = root.val
            if not dfs(root.right):
                return False
            return True

        prev = -inf
        return dfs(root)

```

```
#####
```

```

# Definition for a binary tree node.
# class TreeNode(object):
# def __init__(self, x):
# self.val = x
# self.left = None
# self.right = None

```

```

class Solution(object):
    def isValidBST(self, root):
        """
        :type root: TreeNode
        :rtype: bool
        """
        prev = -float("inf")
        stack = [(1, root)]
        while stack:
            p = stack.pop()

```



```
if not p[1]:
    continue
if p[0] == 0:
    if p[1].val <= prev:
        return False
    prev = p[1].val
else:
    stack.append((1, p[1].right))
    stack.append((0, p[1]))
    stack.append((1, p[1].left))

return True
```

◆ Quick Review

Key Insights

- Every node must adhere to a range: all nodes in the left subtree should be less than the current node, and all nodes in the right subtree should be greater.
- A recursive depth-first search approach works well by passing down the valid range (lower and upper bounds) for each node.
- Using the limits (negative and positive infinity) simplifies handling edge cases at the tree root.
- An in-order traversal would alternatively yield a sorted order for node values, but updating ranges directly is more intuitive for recursive implementation.

Space & Time

Time Complexity: $O(N)$, where N is the number of nodes, as each node is visited once.

Space Complexity: $O(H)$, where H is the height of the tree, due to the recursion stack.

Solution

We use a recursive approach to validate the BST property. We maintain two variables, low and high, representing the valid range for the current node's value. For the left child, the valid range becomes (low, node.val) and for the right child, it becomes (node.val, high). By checking if each node's value falls within its respective range, we can ensure that the tree satisfies the BST properties.

Trees & BST

□ #102 Binary Tree Level Order Traversal

MED[Open on LeetCode](#)

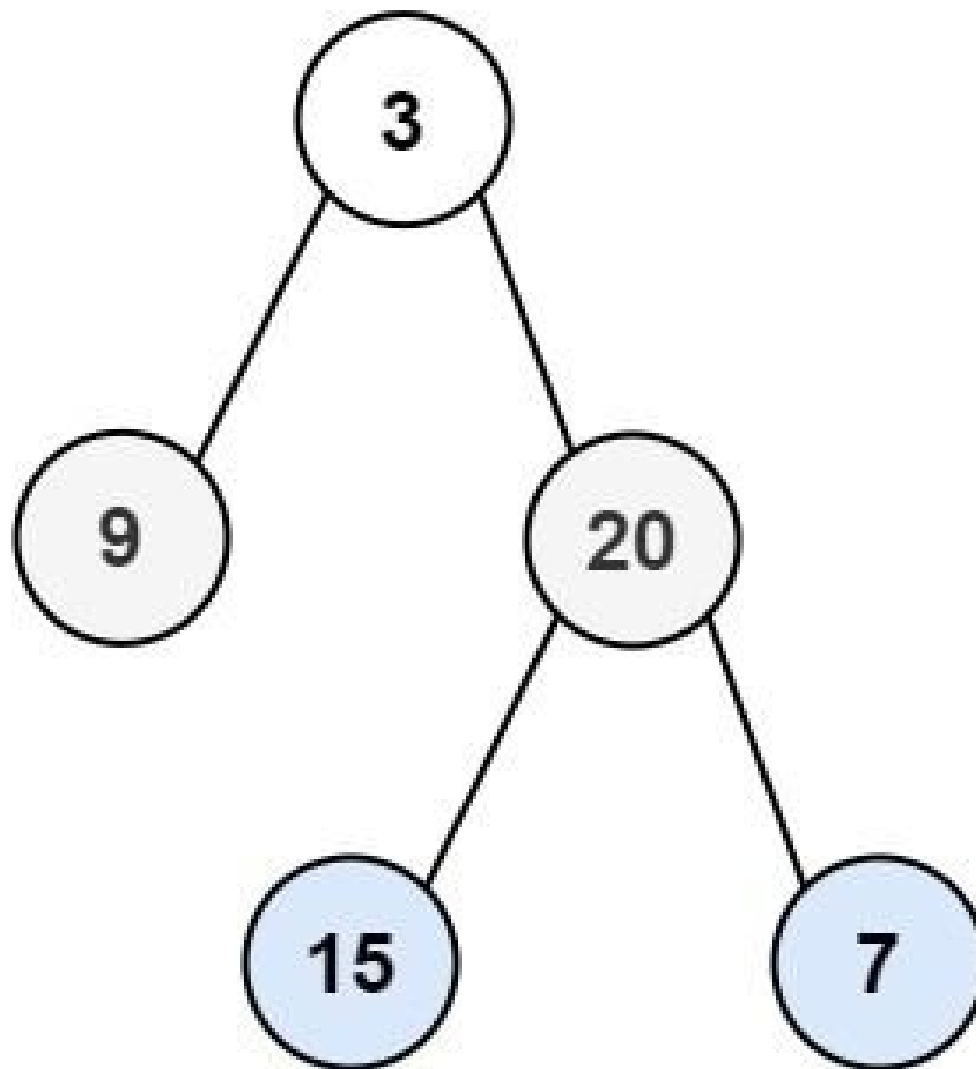
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · BFS

Lists: Grind 169

Problem

Given the root of a binary tree, return the level order traversal of its nodes' values. (i.e., from left to right, level by level).

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: [[3],[9,20],[15,7]]

Example 2:

Input: root = [1]
Output: [[1]]

Example 3:

Input: root = []
Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 2000].
- $-1000 \leq \text{Node.val} \leq 1000$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        if not root:
            return []

        q, res = deque([root]), []
        while q:
            level = []
            for _ in range(len(q)):
                node = q.popleft()

                level.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            res.append(level)
        return res
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return node values level by level, left to right, each level as a sub-list. Right?"

Empty tree → []. Single node → [[val]].

#

"[3,9,20,null,null,15,7] → [[3],[9,20],[15,7]] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

DFS with a level parameter: append node.val to result[level].

After traversal, result lists are level-order groups left-to-right.

Works in O(n), but BFS with a queue is the more natural level-by-level approach.

```
# Time: O(n). Space: O(h) recursion depth.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (BFS with level grouping)

```
# "BFS using a deque. At each step, process ALL nodes at the current level (snapshot
the queue size),
# collect their values, enqueue their children. Natural level separation – no level
counter needed."
```

PHASE 4 — CLEAN CODE

```
class _102_LevelOrder:
    def levelOrder(self, root: Optional['TreeNode']) -> List[List[int]]:
        if not root:
            return []
        result = []
        queue = deque([root])
        while queue:
            level_size = len(queue)
            level_vals = []
            for _ in range(level_size):
                node = queue.popleft()
                level_vals.append(node.val)
                if node.left: queue.append(node.left)
                if node.right: queue.append(node.right)
            result.append(level_vals)
        return result
```

PHASE 5 — TEST & DEBUG

```
# root=3: queue=[3]. level_size=1. level=[3]. enqueue 9,20. result=[[3]].
# queue=[9,20]. level_size=2. level=[9,20]. enqueue 15,7. result=[[3],[9,20]].
# queue=[15,7]. level_size=2. level=[15,7]. result=[[3],[9,20],[15,7]] ✓
# root=None: return [] immediately ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(w) where w = max width of the tree (at most n/2 for a complete
tree).
# The 'snapshot the queue size' trick is the canonical BFS level-separation pattern.
# Variations: level order bottom-up (#107) → just reverse the result.
# Zigzag level order (#103) → alternate the direction each level.
# Right side view (#199) → take the last element of each level.
# Given more time I'd ask: what if we want the minimum depth to a leaf?
# BFS stops at the first leaf found – don't traverse the whole tree."
```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def levelOrder(self, root: TreeNode | None) -> list[list[int]]:
        if not root:
            return []

        ans = []
        q = collections.deque([root])

        while q:
            currLevel = []

            for _ in range(len(q)):
                node = q.popleft()
                currLevel.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            ans.append(currLevel)

        return ans

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        ans = []
        if root is None:

```



```

        return ans
    q = deque([root])
    while q:
        t = []
        for _ in range(len(q)):
            node = q.popleft()
            t.append(node.val)
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)
        ans.append(t)
    return ans

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

from collections import deque

class Solution:

    def levelOrder(self, root: TreeNode) -> List[List[int]]:
        # Return an empty list if the tree is empty.
        if not root:
            return []

        result = [] # Final result list containing values of each level.
        queue = deque([root]) # Initialize the queue with the root node.

        # Process nodes level by level.
        while queue:

```

```

    level_size = len(queue) # Number of nodes at the current level.
    level_nodes = [] # List to hold node values at the current level.

    for _ in range(level_size):
        node = queue.popleft() # Dequeue the next node.
        level_nodes.append(node.val)
        # Enqueue left child if it exists.
        if node.left:
            queue.append(node.left)
        # Enqueue right child if it exists.

        if node.right:
            queue.append(node.right)
    # Append the current level's values to the result.
    result.append(level_nodes)
return result

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        ans = []
        if root is None:

            return ans
        q = deque([root])
        while q:
            t = []
            for _ in range(len(q)):
                node = q.popleft()
                t.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:

```

```
        q.append(node.right)
    ans.append(t)
    return ans
```

◆ Quick Review

Key Insights

- Use Breadth-First Search (BFS) to traverse the tree.
- A queue data structure is ideal for maintaining the order of nodes at each level.
- For every level, process all nodes currently in the queue and add their children for the next level.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes, because each node is processed once.

Space Complexity: $O(n)$, which accounts for the space used by the queue in the worst-case scenario.

Solution

This solution leverages a BFS approach using a queue. The algorithm starts by enqueueing the root node. Then, it enters a loop that continues

until the queue is empty. For each iteration—representing one level of the tree—it computes the number of nodes at that level (levelSize), processes these nodes by dequeuing them, and collects their values. Their non-null children are then enqueued. The list of values for each level is added to the final result list. Key data structures include the queue (for managing nodes) and a list (to store level values).

Trees & BST

□ #103 Binary Tree Zigzag Level Order Traversal

MED[Open on LeetCode](#)

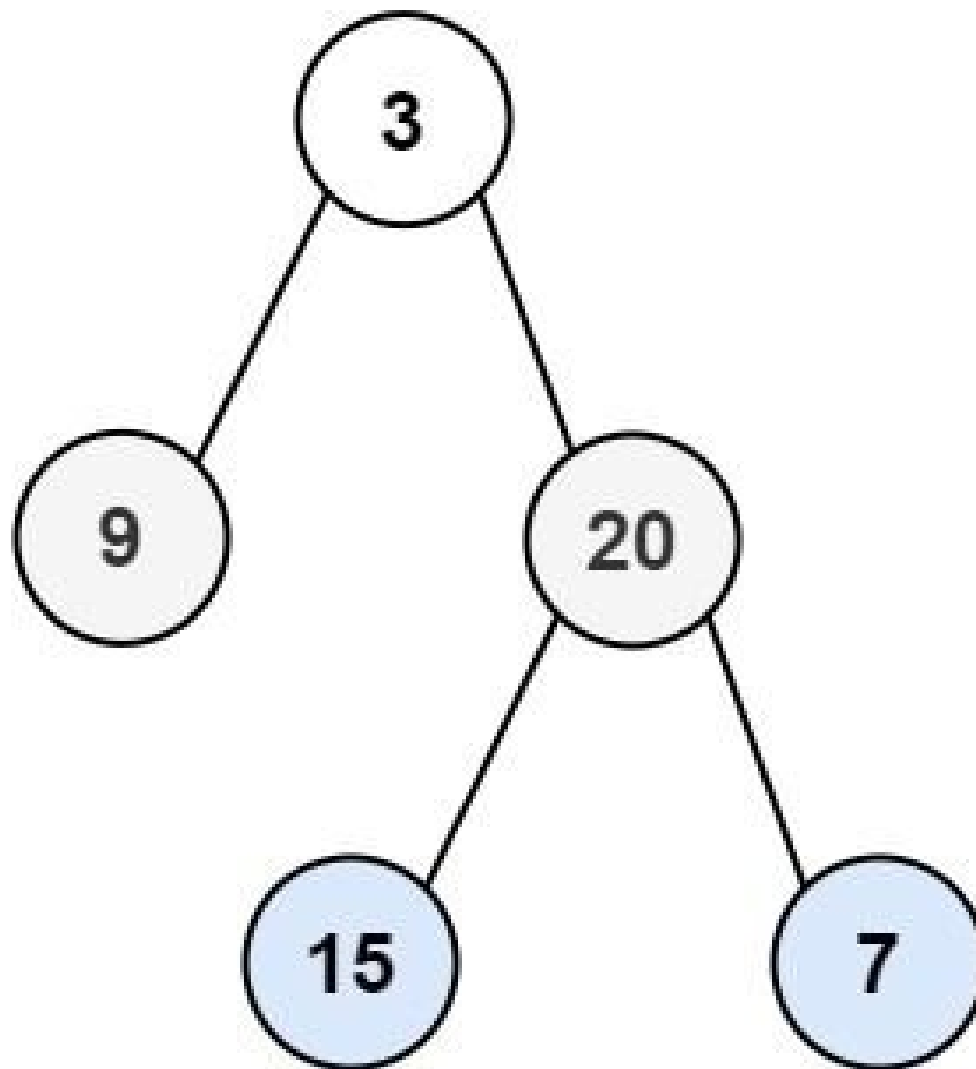
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · BFS

Lists: Grind 169

Problem

Given the root of a binary tree, return the zigzag level order traversal of its nodes' values. (i.e., from left to right, then right to left for the next level and alternate between).

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: [[3],[20,9],[15,7]]

Example 2:

Input: root = [1]
Output: [[1]]

Example 3:

Input: root = []
Output: []

Constraints:

- The number of nodes in the tree is in the range $[0, 2000]$.
- $-100 \leq \text{Node.val} \leq 100$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def zigzagLevelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        if not root:
            return []

        q, res = [root], []
        reverse = False

        while q:
            level = [node.val for node in q]

            res.append(level[::-1] if reverse else level)
            q = [child for node in q for child in (node.left, node.right) if child]
            reverse = not reverse

        return res
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Level order traversal but alternating direction: odd levels left-to-right,
# even levels right-to-left (or vice versa – confirm with example).
# Root is level 0 → left-to-right. Level 1 → right-to-left. Right?"
#
# "[3,9,20,null,null,15,7] → [[3],[20,9],[15,7]] ✓ (level 1 reversed)"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Run standard BFS level-order to build a list of levels.
# Reverse every other level list (odd index levels) for zigzag output.
# Correct  $O(n)$ ; we can reverse on the fly during BFS instead of a second pass.
# Time:  $O(n)$ . Space:  $O(n)$  queue + levels.
```


Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Same BFS snapshot approach as #102. Use a boolean flag 'left_to_right'.

Collect each level normally, then reverse if flag is False.

Toggle flag each level. Time: $O(n)$. Space: $O(w)$."

PHASE 4 — CLEAN CODE

```
class _103_ZigzagLevelOrder:
    def zigzagLevelOrder(self, root: Optional['TreeNode']) -> List[List[int]]:
        if not root:
            return []
        result = []
        queue = deque([root])
        left_to_right = True
        while queue:
            level_size = len(queue)
            level_vals = []
            for _ in range(level_size):
                node = queue.popleft()
                level_vals.append(node.val)
                if node.left: queue.append(node.left)
                if node.right: queue.append(node.right)
            result.append(level_vals if left_to_right else level_vals[::-1])
            left_to_right = not left_to_right
        return result
```

PHASE 5 — TEST & DEBUG

root=[3,9,20,null,null,15,7]:

Level 0: [3], left_to_right=True → [3]. Toggle→False.

Level 1: [9,20], left_to_right=False → reversed=[20,9]. Toggle→True.

Level 2: [15,7], left_to_right=True → [15,7]. → [[3],[20,9],[15,7]] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(w)$.

This is #102 with one added line: the conditional reverse.

Name that connection – it shows you see the pattern family.

Given more time I'd ask: what if alternation starts at level 1 instead of 0?

Just flip the initial value of left_to_right."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def zigzagLevelOrder(self, root: TreeNode | None) -> list[list[int]]:
        if not root:
            return []

        ans = []
        dq = collections.deque([root])
        isLeftToRight = True

        while dq:

            currLevel = []
            for _ in range(len(dq)):
                if isLeftToRight:
                    node = dq.popleft()
                    currLevel.append(node.val)
                    if node.left:
                        dq.append(node.left)
                    if node.right:
                        dq.append(node.right)
                else:
                    node = dq.pop()
                    currLevel.append(node.val)
                    if node.right:
                        dq.appendleft(node.right)
                    if node.left:
                        dq.appendleft(node.left)
            ans.append(currLevel)
            isLeftToRight = not isLeftToRight

        return ans

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def zigzagLevelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        ans = []
        if root is None:
            return ans
        q = deque([root])
        ans = []
        left = 1
        while q:
            t = []
            for _ in range(len(q)):
                node = q.popleft()
                t.append(node.val)
                if node.left:
                    q.append(node.left)
                if node.right:
                    q.append(node.right)
            ans.append(t if left else t[::-1])
            left ^= 1
        return ans

```

• SimplyLeet

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

def zigzagLevelOrder(root):
    # If the tree is empty, return an empty list
    if not root:

```

```
        return []

    # Initialize the result list and the queue with the root node.
    result = []
    queue = [root]
    left_to_right = True # Flag to indicate the current direction of traversal

    # Process the tree level by level.
    while queue:
        level_size = len(queue)

        current_level = []

        # Process all nodes for the current level.
        for i in range(level_size):
            node = queue.pop(0) # Dequeue the node from the front of the queue
            current_level.append(node.val) # Append the node's value

            # Enqueue the left child if it exists.
            if node.left:
                queue.append(node.left)

            # Enqueue the right child if it exists.
            if node.right:
                queue.append(node.right)

        # If the current level's order is right-to-left, reverse the list.
        if not left_to_right:
            current_level.reverse()

        # Append the current level's values to the result.
        result.append(current_level)

        # Toggle the direction for the next level.
        left_to_right = not left_to_right

    return result
```

 • LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
'''
can also use list.insert()
```

```
>>> my_list = [2, 3, 4]
>>> my_list.insert(0, 1) # Insert 1 at the head of the list
>>> print(my_list) # Output: [1, 2, 3, 4]
```

```
>>> a = deque([])
>>> a
deque([])
>>> a.append(1)
>>> a.append(2)
```

```
>>> a.append(3)
>>> a
deque([1, 2, 3])
>>>
>>> a.append(0, 555)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: deque.append() takes exactly one argument (2 given)
>>> a.insert(0, 555)
>>> a
```

```
deque([555, 1, 2, 3])
'''
```

```
from collections import deque
```

```
class Solution:
    def zigzagLevelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        ans = []
        if root is None:
            return ans
```

```

q = deque([root])
ans = []
left = True
while q:
    t = []
    for _ in range(len(q)):
        node = q.popleft()
        t.append(node.val)
        if node.left:
            q.append(node.left)

        if node.right:
            q.append(node.right)
    ans.append(t if left else t[::-1])
    left = (not left)
return ans

```

```
#####
```

```

# Definition for a binary tree node.
# class TreeNode(object):

```

```

# def __init__(self, x):
# self.val = x
# self.left = None
# self.right = None
from collections import deque

```

```

class Solution(object):
    def zigzagLevelOrder(self, root):
        """

```

```

:type root: TreeNode
:rtype: List[List[int]]
"""

```

```

stack = deque([root])
ans = []
odd = True
while stack:
    level = []
    for k in range(0, len(stack)):
        top = stack.popleft()

```

```

        if top is None:
            continue
        level.append(top.val)
        stack.append(top.left)
        stack.append(top.right)
    if level:
        if odd:
            ans.append(level)
        else:
            ans.append(level[::-1])

```

```

    odd = not odd
    return ans

```

```
#####
```

```
# Definition for a binary tree node.
```

```
# class TreeNode:
```

```
# def __init__(self, val=0, left=None, right=None):
```

```
# self.val = val
```

```
# self.left = left
```

```
# self.right = right
```

```
class Solution:
```

```
    def zigzagLevelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
```

```
        ans = []
```

```
        if root is None:
```

```
            return ans
```

```
        q = deque([root])
```

```
        ans = []
```

```
        left = 1
```

```
        while q:
```

```
            t = []
```

```
            for _ in range(len(q)):
```

```
                node = q.popleft()
```

```
                t.append(node.val)
```

```
                if node.left:
```

```
                    q.append(node.left)
```

```
                if node.right:
```

```
                    q.append(node.right)
```

```
            ans.append(t if left else t[::-1])
```

```
            left ^= 1
```

```
return ans
```

◆ Quick Review

Key Insights

- Use a breadth-first search (BFS) approach to traverse the tree level by level.
- Alternate the direction of traversal on each level to achieve the zigzag pattern.
- A flag or counter can be used to determine the order of nodes for the current level.
- Using a double-ended data structure or simply reversing the list at every alternate level is an effective approach.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes, because each node is visited exactly once.

Space Complexity: $O(n)$, as in the worst-case scenario (e.g., a complete binary tree) the queue used for BFS may hold $O(n)$ nodes at once.

Solution

We can solve this problem using a breadth-first search (BFS). Start by initializing a queue to hold nodes for each level. For each level: Determine the number of nodes at the current level. Iterate over these nodes, appending their children for the next level. Depending on a flag (or level count), either append node values in the natural order (left to right) or reverse the order (right to left) before adding to the result. Key data structures: A queue for BFS traversal. A list (or similar container) to store current level values. The main trick is toggling the order of insertion or reversing the list at alternate levels to achieve the "zigzag" pattern.

Trees & BST

□ #105 Construct Binary Tree from Preorder and Inorder Traversal

MED

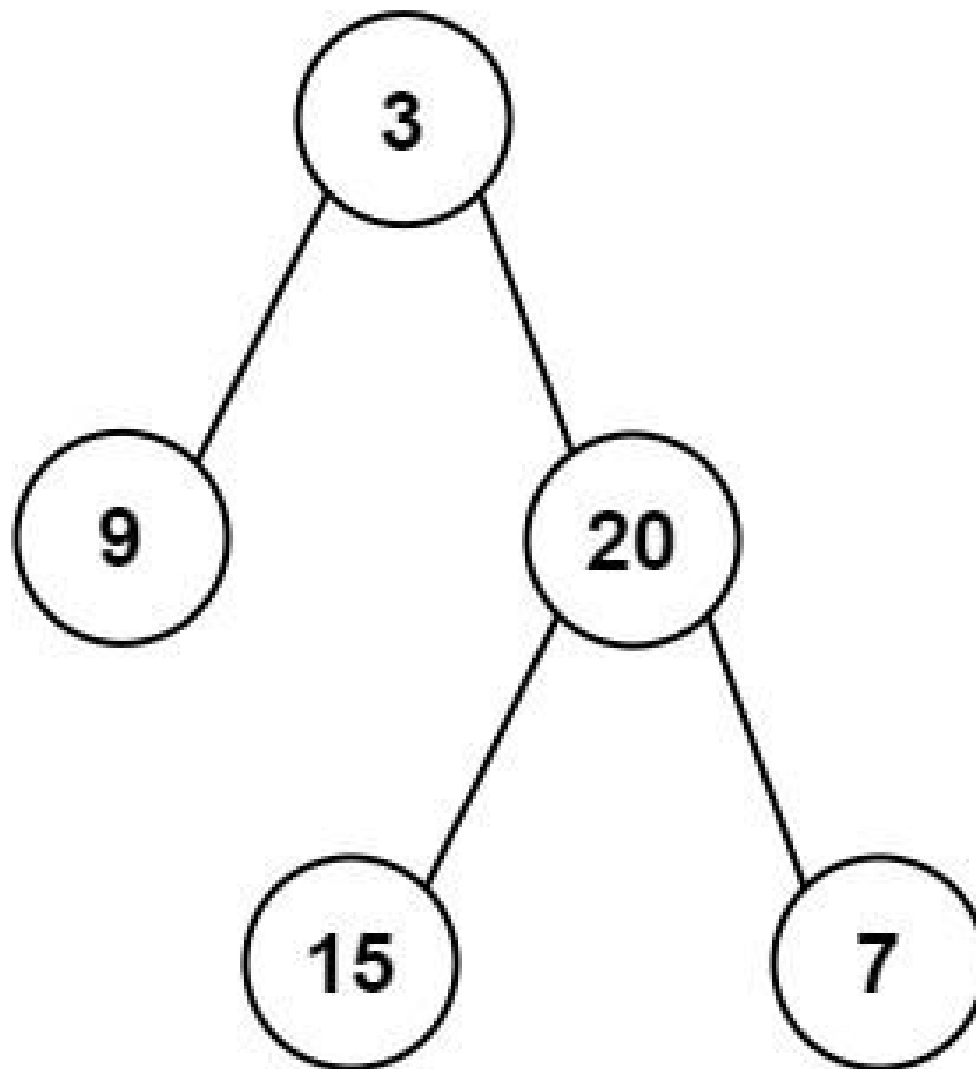
[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Tree · DFS
Lists: Grind 169

Problem

Given two integer arrays preorder and inorder where preorder is the preorder traversal of a binary tree and inorder is the inorder traversal of the same tree, construct and return the binary tree.

Example 1:



Input: preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]
Output: [3,9,20,null,null,15,7]

Example 2:

Input: preorder = [-1], inorder = [-1]
Output: [-1]

Constraints:

- $1 \leq \text{preorder.length} \leq 3000$
- $\text{inorder.length} == \text{preorder.length}$
- $-3000 \leq \text{preorder}[i], \text{inorder}[i] \leq 3000$

- preorder and inorder consist of unique values.
- Each value of inorder also appears in preorder.
- preorder is guaranteed to be the preorder traversal of the tree.
- inorder is guaranteed to be the inorder traversal of the tree.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        pos = {val: i for i, val in enumerate(inorder)}

        def dfs(lo, hi):
            if lo > hi:
                return None
            m = pos[preorder.pop(0)]
            return TreeNode(inorder[m], dfs(lo, m - 1), dfs(m + 1, hi))

        return dfs(0, len(inorder) - 1)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Preorder: root first, then left subtree, then right subtree.

Inorder: left subtree, root, right subtree.

All values are unique – so we can find the root's position in inorder uniquely. Right?"

#

"preorder=[3,9,20,15,7], inorder=[9,3,15,20,7]:

```
# Root=3. In inorder, 3 is at index 1. Left subtree=[9], right subtree=[15,20,7].
# preorder left=[9], right=[20,15,7]. Recurse."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
```

```
# Root is preorder[0]; scan inorder to find root index and split left/right subtrees.
```

```
# Recursively repeat on each side – correct tree reconstruction.
```

```
# Scanning inorder for every root costs  $O(n^2)$  total; hash map of inorder indices fixes that.
```

```
# Time:  $O(n^2)$  naive. Space:  $O(n)$  recursion.
```

```
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Pre-build a hash map: inorder value → index for  $O(1)$  lookups.
```

```
# Use a preorder index pointer (use a list to allow mutation inside inner function).
```

```
# build(in_left, in_right): root = preorder[pre_idx]. Find in inorder. Recurse.
```

```
# Time:  $O(n)$ . Space:  $O(n)$  for the map."
```

PHASE 4 — CLEAN CODE

```
class _105_BuildTree:
    def buildTree(self, preorder: List[int], inorder: List[int]) ->
Optional['TreeNode']:
        in_idx = {val: i for i, val in enumerate(inorder)}
        pre = iter(preorder)
        def build(left: int, right: int) -> Optional['TreeNode']:
            if left > right:
                return None
            root_val = next(pre)
            root = TreeNode(root_val)
            mid = in_idx[root_val]
            root.left = build(left, mid - 1)
            root.right = build(mid + 1, right)
            return root
        return build(0, len(inorder) - 1)
```

PHASE 5 — TEST & DEBUG

```
# preorder=[3,9,20,15,7], inorder=[9,3,15,20,7]. in_idx={9:0,3:1,15:2,20:3,7:4}.
```

```
# build(0,4): root=3, mid=1. left=build(0,0), right=build(2,4).
```

```
# build(0,0): root=9, mid=0. left=build(0,-1)=None. right=build(1,0)=None. →
node(9).
```

```
# build(2,4): root=20, mid=3. left=build(2,2), right=build(4,4).
```

```
# build(2,2): root=15. → node(15). build(4,4): root=7. → node(7).
```

```
# → Tree: 3→left=9, 3→right=20→left=15, 20→right=7 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ . Space  $O(n)$  for the hash map and call stack.
```

```
# Using iter(preorder) as a lazy pointer is cleaner than an index variable.
```

```
# The key insight: preorder always gives the root first, inorder tells us how many
```

```
# elements are in each subtree – that's the only information we need to recurse.
```

Given more time I'd ask: construct from postorder and inorder?
 # Same logic – just consume postorder from the RIGHT end, and build right subtree first."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def buildTree(
        self,
        preorder: list[int],
        inorder: list[int],
    ) -> TreeNode | None:
        inToIndex = {num: i for i, num in enumerate(inorder)}

        def build(
            preStart: int,

            preEnd: int,
            inStart: int,
            inEnd: int,
        ) -> TreeNode | None:
            if preStart > preEnd:
                return None

            rootVal = preorder[preStart]
            rootInIndex = inToIndex[rootVal]
            leftSize = rootInIndex - inStart

            root = TreeNode(rootVal)
            root.left = build(preStart + 1, preStart + leftSize,
                             inStart, rootInIndex - 1)
            root.right = build(preStart + leftSize + 1,
                              preEnd, rootInIndex + 1, inEnd)
            return root

        return build(0, len(preorder) - 1, 0, len(inorder) - 1)
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        def dfs(i: int, j: int, n: int) -> Optional[TreeNode]:
            if n <= 0:
                return None
            v = preorder[i]
            k = d[v]
            l = dfs(i + 1, j, k - j)
            r = dfs(i + 1 + k - j, k + 1, n - k + j - 1)
            return TreeNode(v, l, r)

            d = {v: i for i, v in enumerate(inorder)}
            return dfs(0, 0, len(preorder))
```

```
class Solution:
    def getBinaryTrees(self, preOrder: List[int], inOrder: List[int]) -> List[TreeNode]:
        def dfs(i: int, j: int, n: int) -> List[TreeNode]:
            if n <= 0:
                return [None]
            v = preOrder[i]
            ans = []
            for k in d[v]:
                if j <= k < j + n:
                    for l in dfs(i + 1, j, k - j):
                        for r in dfs(i + 1 + k - j, k + 1, n - 1 - (k - j)):
                            ans.append(TreeNode(v, l, r))
            return ans

            d = defaultdict(list)
            for i, x in enumerate(inOrder):
                d[x].append(i)
            return dfs(0, 0, len(preOrder))
```


• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def buildTree(self, preorder: list[int], inorder: list[int]) -> TreeNode:
        # Create a hashmap to store value->index for inorder traversal

        inorder_index = {value: idx for idx, value in enumerate(inorder)}

        # Recursive helper function to build the tree
        def helper(pre_start, in_left, in_right):
            # Base case: if there are no elements to construct subtree
            if in_left > in_right:
                return None

            # Pick up pre_start element as a root
            root_val = preorder[pre_start]

            root = TreeNode(root_val)

            # Get the inorder index of the root
            in_index = inorder_index[root_val]

            # Number of nodes in left subtree
            left_subtree_size = in_index - in_left

            # Recursively build the left subtree
            root.left = helper(pre_start + 1, in_left, in_index - 1)

            # Recursively build the right subtree
            root.right = helper(pre_start + left_subtree_size + 1, in_index +
1, in_right)

            return root

        # Initiate recursion from the start of preorder and full inorder range
        return helper(0, 0, len(inorder) - 1)
```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

# best
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:

        if not preorder: # or, not inorder
            return None
        v = preorder[0]
        i = inorder.index(v)
        root = TreeNode(v)
        root.left = self.buildTree(preorder[1: i+1], inorder[:i])
        root.right = self.buildTree(preorder[i+1:], inorder[i+1:])
        return root

# inorder.index(preorder_val)
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        def dfs(pleft, pright, ileft, iright):
            if pleft > pright or ileft > iright:
                return None
            k = inorder.index(preorder[pleft])
            root = TreeNode(preorder[pleft])
            root.left = dfs(pleft + 1, pleft + (k - ileft), ileft, k - 1)
            root.right = dfs(pleft + 1 + (k - ileft), pright, k + 1, iright)
```

```

        return root

    n = len(preorder)
    return dfs(0, n - 1, 0, n - 1)

# build dict for val => index
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        def dfs(pleft, pright, ileft,  iright):

            if pleft > pright or ileft > iright:
                return None
            v = preorder[pleft]
            k = d[v]
            root = TreeNode(v)
            root.left = dfs(pleft + 1, pleft + (k - ileft), ileft, k - 1)
            root.right = dfs(pleft + 1 + (k - ileft), pright, k + 1, iright)
            return root

        d = {v: i for i, v in enumerate(inorder)}

    n = len(preorder)
    return dfs(0, n - 1, 0, n - 1)

#####

'''
search for index, also use .index(val)
>>> a = [1,2,3,4,5]
>>> a.index(3)
2

```

```

    """
class Solution(object):
    def buildTree(self, preorder, inorder):
        """
        :type preorder: List[int]
        :type inorder: List[int]
        :rtype: TreeNode
        """

        self.preindex = 0

        ind = {v: i for i, v in enumerate(inorder)}
        head = self.dc(0, len(preorder) - 1, preorder, inorder, ind)
        return head

    def dc(self, start, end, preorder, inorder, ind):
        if start <= end:
            mid = ind[preorder[self.preindex]]
            self.preindex += 1
            root = TreeNode(inorder[mid])
            root.left = self.dc(start, mid - 1, preorder, inorder, ind)

            root.right = self.dc(mid + 1, end, preorder, inorder, ind)
            return root

#####

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left

```

```
# self.right = right
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        def dfs(i: int, j: int, n: int):
            if n <= 0:
                return None
            v = preorder[i]
            k = d[v]
            l = dfs(i + 1, j, k - j)
            r = dfs(i + 1 + k - j, k + 1, n - k + j - 1)

            return TreeNode(v, l, r)

        d = {v: i for i, v in enumerate(inorder)}
        return dfs(0, 0, len(preorder))
```

◆ Quick Review

Key Insights

- Preorder traversal always starts with the root node.
- Inorder traversal splits the tree into left subtree and right subtree based on the root.
- A hash table (map/dictionary) can be used to quickly locate the index of the root in the inorder array.
- Use recursion to construct left and right subtrees.

Space & Time

Time Complexity: $O(n)$ since we process every node exactly once.

Space Complexity: $O(n)$ due to the recursion stack and the hash map used for inorder indices.

Solution

We solve the problem using a recursive divide-and-conquer approach. First, create a hash map to store each value's index in the inorder array for quick lookups. The preorder array always provides the current root. Use the inorder array to divide the tree into left and right subtrees. Recursively construct the subtrees by adjusting the preorder index and the boundaries of the inorder array. The algorithm leverages recursion and a hash map to achieve $O(n)$ time complexity.

Trees & BST

□ #199 Binary Tree Right Side View

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BFS
Lists: Grind 169

Problem

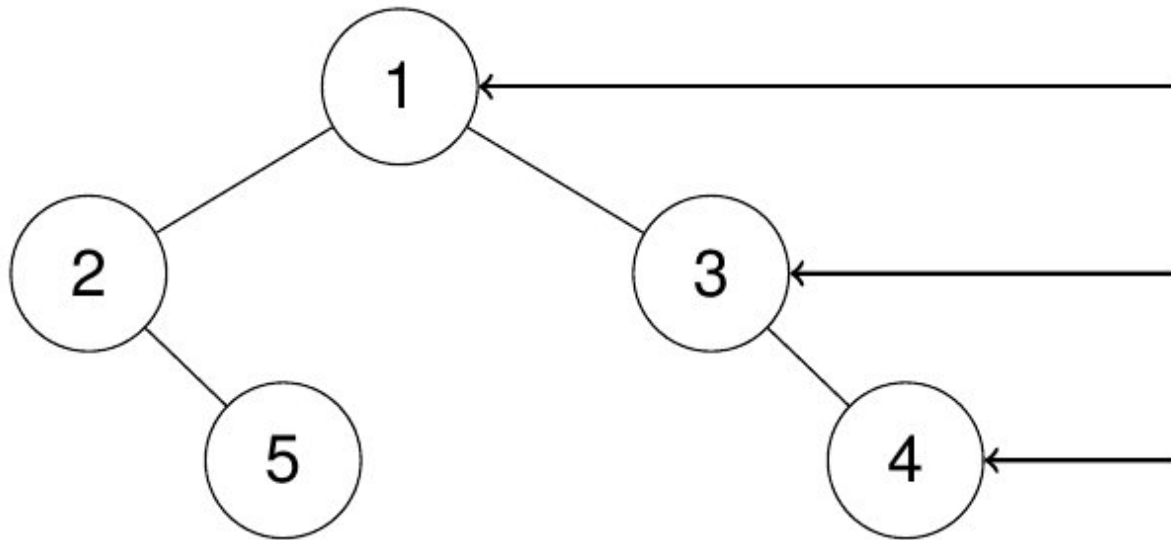
Given the root of a binary tree, imagine yourself standing on the right side of it, return the values of the nodes you can see ordered from top to bottom.

Example 1:

Input: root = [1,2,3,null,5,null,4]

Output: [1,3,4]

Explanation:

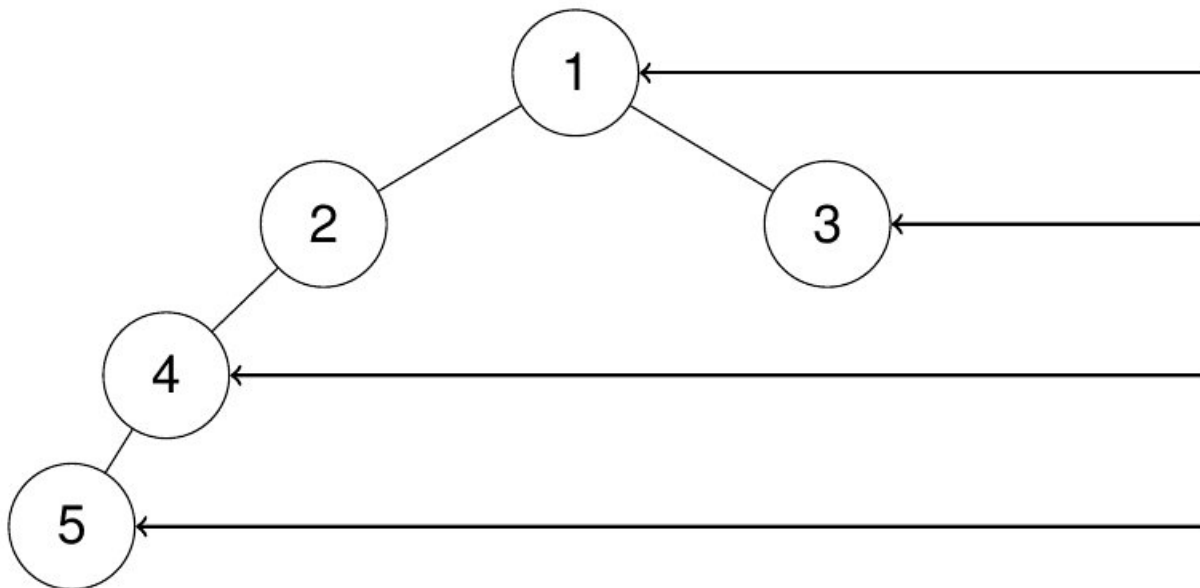


Example 2:

Input: root = [1,2,3,4,null,null,null,5]

Output: [1,3,4,5]

Explanation:



Example 3:

Input: root = [1,null,3]

Output: [1,3]

Example 4:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 100].
- $-100 \leq \text{Node.val} \leq 100$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
        if not root:
            return []

        res, q = [], [root]
        while q:
            res.append(q[-1].val)
            q = [c for n in q for c in (n.left, n.right) if c]
        return res
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Imagine looking at the tree from the right. You see one node per level –
the rightmost node visible at that level. Return top to bottom. Right?"

#

"[1,2,3,null,5,null,4] → [1,3,4] ✓ (3 is rightmost at level 1, 4 at level 2)"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

BFS level-order; after processing each level, take the last node value added.

That last node is the rightmost visible from that depth.

Correct $O(n)$; DFS tracking depth + rightmost-at-depth is equally fine.

Time: $O(n)$. Space: $O(w)$ queue width.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Same BFS snapshot as #102/#103. Just append the LAST element of each level's list.

Or DFS: visit right child first, add node.val when depth == len(result) (first node seen at new depth)."

PHASE 4 — CLEAN CODE

```
class _199_RightSideView:
    def rightSideView(self, root: Optional['TreeNode']) -> List[int]:
        if not root:
            return []
        result = []
        queue = deque([root])
        while queue:
            level_size = len(queue)
            for i in range(level_size):
                node = queue.popleft()
                if i == level_size - 1:
                    result.append(node.val) # rightmost node of this level
                if node.left: queue.append(node.left)
                if node.right: queue.append(node.right)
        return result
```

PHASE 5 — TEST & DEBUG

[1,2,3,null,5,null,4]:

Level 0: [1] → last=1. Level 1: [2,3] → last=3. Level 2: [5,4] → last=4.

→ [1,3,4] ✓

[1,2,3,4,null,null,null,5]:

Level 0: [1]→1. Level 1: [2,3]→3. Level 2: [4]→4. Level 3: [5]→5. → [1,3,4,5] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(w)$ where w = max width.

This is #102 with one change: only record the last element per level.

Left side view: record the first element per level instead.

Given more time I'd ask: what if some right-side nodes are blocked by deeper left-side nodes?

The problem says 'visible from the right' – the rightmost node at each DEPTH level,

which BFS naturally gives us. A right child at depth 2 can block a left child at depth 2."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def rightSideView(self, root: TreeNode | None) -> list[int]:
        if not root:
            return []

        ans = []
        q = collections.deque([root])

        while q:
            size = len(q)

            for i in range(size):
                root = q.popleft()
                if i == size - 1:
                    ans.append(root.val)
                if root.left:
                    q.append(root.left)
                if root.right:
                    q.append(root.right)

            return ans
```

```
class Solution:
    def rightSideView(self, root: TreeNode | None) -> list[int]:
        ans = []

        def dfs(root: TreeNode | None, depth: int) -> None:
            if not root:
                return

            if depth == len(ans):
                ans.append(root.val)

            dfs(root.right, depth + 1)
            dfs(root.left, depth + 1)

        dfs(root, 0)
        return ans
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
        ans = []
        if root is None:
            return ans
        q = deque([root])
        while q:
            ans.append(q[0].val)
            for _ in range(len(q)):
                node = q.popleft()
                if node.right:
                    q.append(node.right)
                if node.left:
                    q.append(node.left)
        return ans
```

• SimplyLeet

```
# Python solution using BFS
from collections import deque

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val          # Node value
        self.left = left        # Left child
        self.right = right       # Right child

def rightSideView(root):
```

```

# If the tree is empty, return an empty list
if not root:
    return []

right_view = []          # List to store right side view
queue = deque([root])    # Initialize queue with the root node

while queue:
    level_size = len(queue) # Number of nodes in current level
    for i in range(level_size):

        node = queue.popleft() # Get the front node from the queue
        # if it's the last node in the current level, add it.
        if i == level_size - 1:
            right_view.append(node.val)
        # add left child if exists
        if node.left:
            queue.append(node.left)
        # add right child if exists
        if node.right:
            queue.append(node.right)

return right_view

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
        ans = []
        if root is None:

```

```

        return ans
    q = deque([root])
    while q:
        ans.append(q[-1].val) # add last node of previous level traversal
results
        for _ in range(len(q)):
            node = q.popleft()
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)

```

```

    return ans

```

```

#####

```

```

# Definition for a binary tree node.
# class TreeNode(object):
# def __init__(self, x):
# self.val = x
# self.left = None
# self.right = None

```

```

from collections import deque

```

```

class Solution(object):
    def rightSideView(self, root):
        """
        :type root: TreeNode
        :rtype: List[int]
        """

```

```
def dfs(root, h):  
    if root:  
        if h == len(ans):  
            ans.append(root.val)  
            # pre-order, all the way to the right  
            dfs(root.right, h + 1)  
            dfs(root.left, h + 1)  
  
ans = []  
dfs(root, 0)  
  
return ans
```

◆ Quick Review

Key Insights

- The problem is essentially about capturing the last node at each level (or the rightmost node) in a binary tree.
- A Breadth-First Search (BFS) approach naturally works by processing nodes level by level.
- Alternatively, Depth-First Search (DFS) can be adapted by prioritizing the right subtree first, ensuring that the first node encountered at each depth is the visible one.
- The number of nodes is capped at 100, so both approaches are efficient.

Space & Time

Time Complexity: $O(n)$ – we visit every node in the tree.

Space Complexity: $O(n)$ – in the worst case, the queue (or call stack for DFS) may hold all nodes of the tree.

Solution

We can solve this problem using BFS (level-order traversal). The idea is to traverse the tree level by level using a queue. For each level, the last element added to the level represents the rightmost view from that level. We then append that element's value to our result list. Alternatively, a DFS approach can be used where we visit the right subtree first. By keeping track of the current depth and checking if it's the first time we've encountered this depth, we can add the first node visited at that depth (which will be the rightmost one) to the result. In both approaches, appropriate data structures (a queue for BFS or recursion with a list for DFS) are used to store nodes and track the traversal state.

Trees & BST

□ #230 Kth Smallest Element in a BST

MED

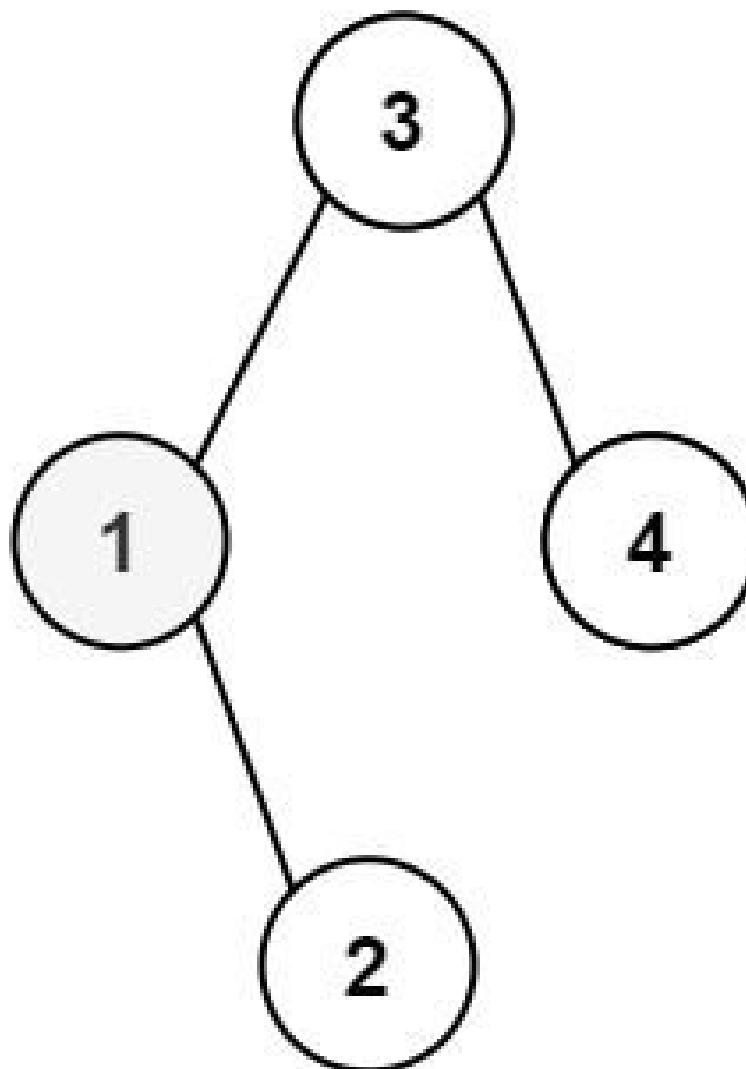
[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BST
Lists: Grind 169

Problem

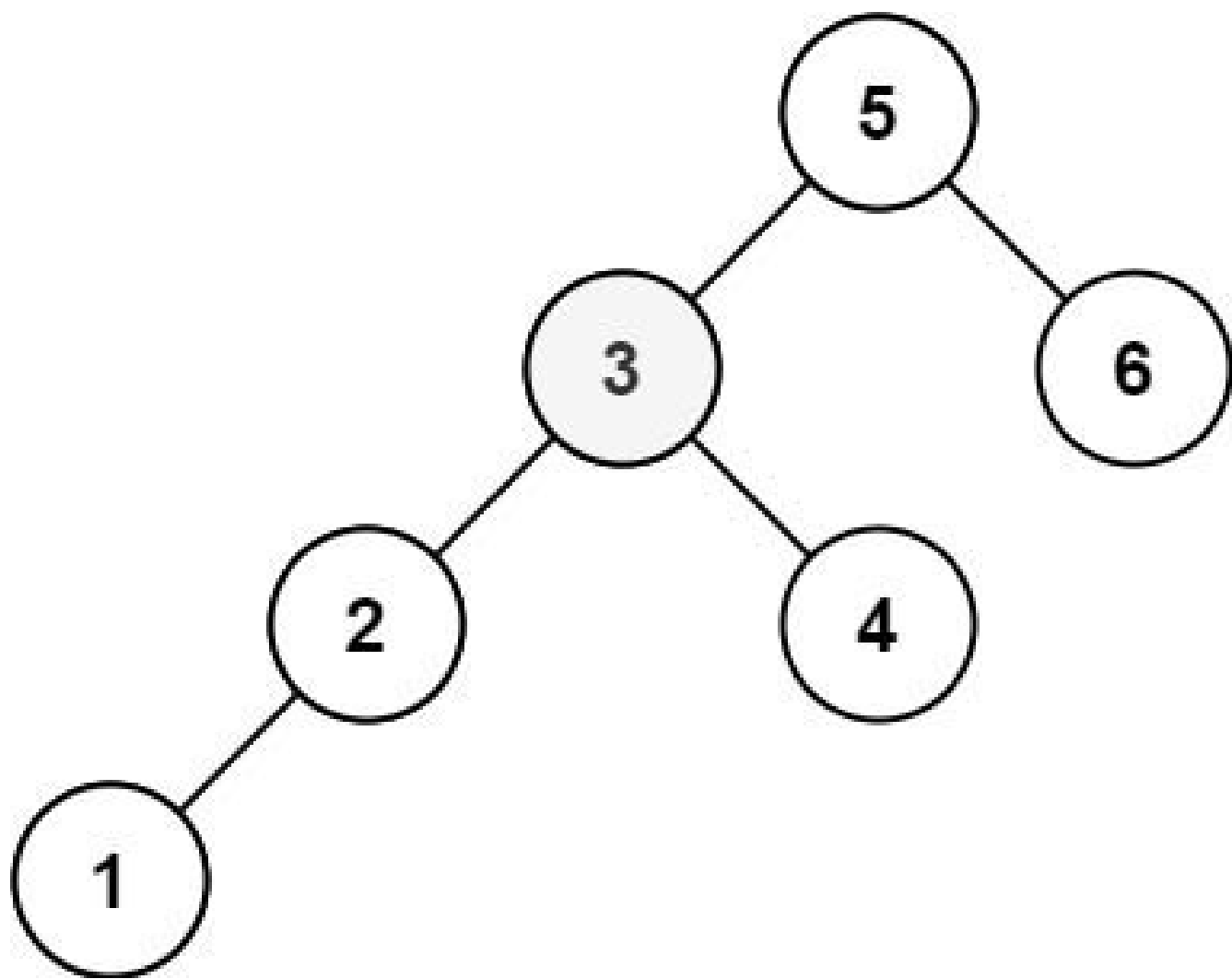
Given the root of a binary search tree, and an integer k , return the k th smallest value (1-indexed) of all the values of the nodes in the tree.

Example 1:



Input: root = [3,1,4,null,2], k = 1
Output: 1

Example 2:



Input: root = [5,3,6,2,4,null,null,1], k = 3
Output: 3

Constraints:

- The number of nodes in the tree is n.
- $1 \leq k \leq n \leq 10^4$
- $0 \leq \text{Node.val} \leq 10^4$

Follow up: If the BST is modified often (i.e., we can do insert and delete operations) and you need to find the kth smallest frequently, how would you optimize?

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        self.k = k
        self.res = None

        def inorder(node):
            if not node or self.res:
                return
            inorder(node.left)
            self.k -= 1

            if self.k == 0:
                self.res = node
                return
            inorder(node.right)

        inorder(root)
        return self.res.val
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"BST in-order traversal (left → root → right) visits nodes in ascending sorted order.

Return the k-th element in this order (1-indexed). Right?"

#

"[3,1,4,null,2], k=1: in-order=[1,2,3,4]. 1st smallest=1 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

In-order traversal collects all node values in sorted order.

Return the (k-1)th element from that list.

Correct O(n) time and O(n) space; we can stop after k steps in-order.

Time: O(n). Space: O(n) values list.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"In-order DFS stopping early at the k-th element. Use a counter and result variable"

in a nonlocal/list wrapper. Avoids collecting all n values if k is small."

PHASE 4 — CLEAN CODE

```
class _230_KthSmallestBST:
    def kthSmallest(self, root: Optional['TreeNode'], k: int) -> int:
        count = [0]
        result = [0]
        def inorder(node):
            if not node or count[0] == k:
                return
            inorder(node.left)
            count[0] += 1
            if count[0] == k:
                result[0] = node.val
                return
            inorder(node.right)
        inorder(root)
        return result[0]
```

PHASE 5 — TEST & DEBUG

```
# [3,1,4,null,2], k=1:
# inorder(3)→inorder(1)→inorder(None). count=0+1=1==k=1→result=1. return.
# → 1 ✓
# [5,3,6,2,4,null,null,1], k=3:
# in-order: 1,2,3,4,5,6. 3rd = 3 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(k + h)$  – at most k steps after reaching the leftmost leaf. Space  $O(h)$ .
# Follow-up: frequent inserts/deletes + frequent kth queries?
# Augment each BST node with the size of its left subtree.
# Then kth smallest is an  $O(\log n)$  descent: if left_size >= k go left; if
left_size+1==k return root; else k -= left_size+1, go right.
# Given more time I'd ask: what if we want the kth largest?
# Reverse in-order (right→root→left) and count k steps."
```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def kthSmallest(self, root: TreeNode | None, k: int) -> int:
        def countNodes(root: TreeNode | None) -> int:
            if not root:
                return 0
            return 1 + countNodes(root.left) + countNodes(root.right)

        leftCount = countNodes(root.left)

        if leftCount == k - 1:

            return root.val
        if leftCount >= k:
            return self.kthSmallest(root.left, k)
        return self.kthSmallest(root.right, k - 1 - leftCount) # leftCount < k

```

```

class Solution:
    def kthSmallest(self, root: TreeNode | None, k: int) -> int:
        rank = 0
        ans = 0

        def traverse(root: TreeNode | None) -> None:
            nonlocal rank
            nonlocal ans
            if not root:
                return

            traverse(root.left)
            rank += 1
            if rank == k:
                ans = root.val
                return
            traverse(root.right)

        traverse(root)
        return ans

```



```

class Solution:
    def kthSmallest(self, root: TreeNode | None, k: int) -> int:
        stack = []

        while root:
            stack.append(root)
            root = root.left

        for _ in range(k - 1):
            root = stack.pop()

            root = root.right
            while root:
                stack.append(root)
                root = root.left

        return stack[-1].val

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        stk = []
        while root or stk:

            if root:
                stk.append(root)
                root = root.left
            else:
                root = stk.pop()
                k -= 1
                if k == 0:
                    return root.val
                root = root.right

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def kthSmallest(self, root: TreeNode, k: int) -> int:
        # Initialize counter and result

        self.k = k
        self.ans = None

        # In-order traversal: left -> node -> right
        def inorder(node):
            if not node:
                return
            # Traverse left subtree
            inorder(node.left)
            # Process current node

            self.k -= 1
            if self.k == 0:
                self.ans = node.val
                return
            # Traverse right subtree if kth element not found
            inorder(node.right)

        inorder(root)
        return self.ans

```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        stk = []
        while root or stk:
```

```
            if root:
                stk.append(root)
                root = root.left
            else:
                root = stk.pop()
                k -= 1
                if k == 0:
                    return root.val
                root = root.right
```

```
#####
```

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution_followUp:
    def kthSmallest(self, root: TreeNode, k: int) -> int:
```

```
        node = self.build(root)
        return self.dfs(node, k)
```

```
class MyTreeNode:
    def __init__(self, x):
        self.val = x
        self.count = 1 # default 1
        self.left = None
        self.right = None
```

```

def build(self, root: TreeNode) -> MyTreeNode:
    if not root:
        return None
    node = self.MyTreeNode(root.val) # default count already is 1
    node.left = self.build(root.left)
    node.right = self.build(root.right)
    if node.left:
        node.count += node.left.count
    if node.right:
        node.count += node.right.count

    return node

def dfs(self, node: MyTreeNode, k: int) -> int:
    if node.left:
        cnt = node.left.count
        if k < cnt + 1:
            return self.dfs(node.left, k)
        elif k > cnt + 1:
            return self.dfs(node.right, k - 1 - cnt)
        else: # k == cnt+1
            return node.val
    else:
        if k == 1: # cannot move to beginning of dfs()
            return node.val
        return self.dfs(node.right, k - 1)

```

◆ Quick Review

Key Insights

- An in-order traversal of a BST visits nodes in increasing order.
- Use a counter to keep track of the number of nodes visited.

- As soon as the counter reaches k , the current node's value is the k th smallest.
- For frequent modifications (insertions/deletions) and queries, consider augmenting the BST with subtree counts.

Space & Time

Time Complexity: $O(n)$ in the worst-case scenario (when $k \approx n$), but on average the traversal may stop early.

Space Complexity: $O(h)$ where h is the height of the tree ($O(n)$ in the worst-case for a skewed tree).

Solution

We perform an in-order traversal of the BST which visits nodes in ascending order. During the traversal, we maintain a counter (or decrement k) to check when we have encountered the k th smallest element. When the counter reaches 0 (or k becomes 0), we capture the current node's value as our answer and return it. This approach is both simple and efficient for the given constraints.

Trees & BST

□ #235 Lowest Common Ancestor of a Binary Search Tree

MED

[Open on LeetCode](#)

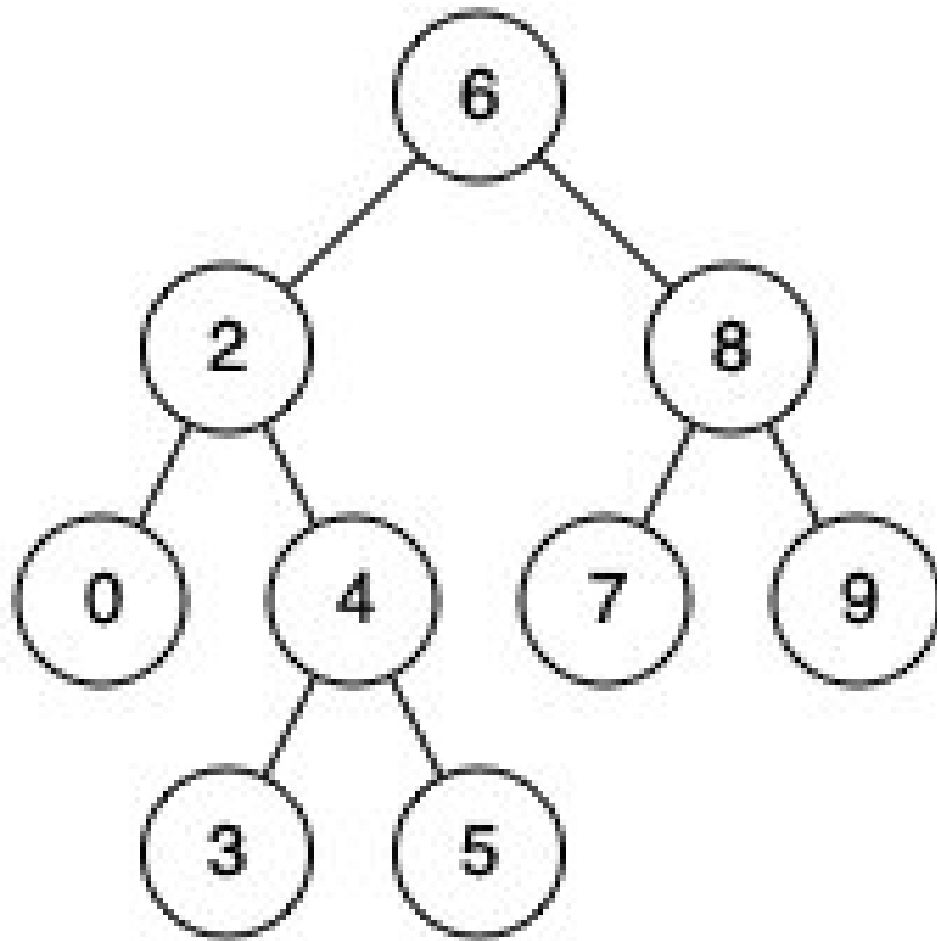
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BST
Lists: Grind 169

Problem

Given a binary search tree (BST), find the lowest common ancestor (LCA) node of two given nodes in the BST.

According to the definition of LCA on Wikipedia: “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself).”

Example 1:

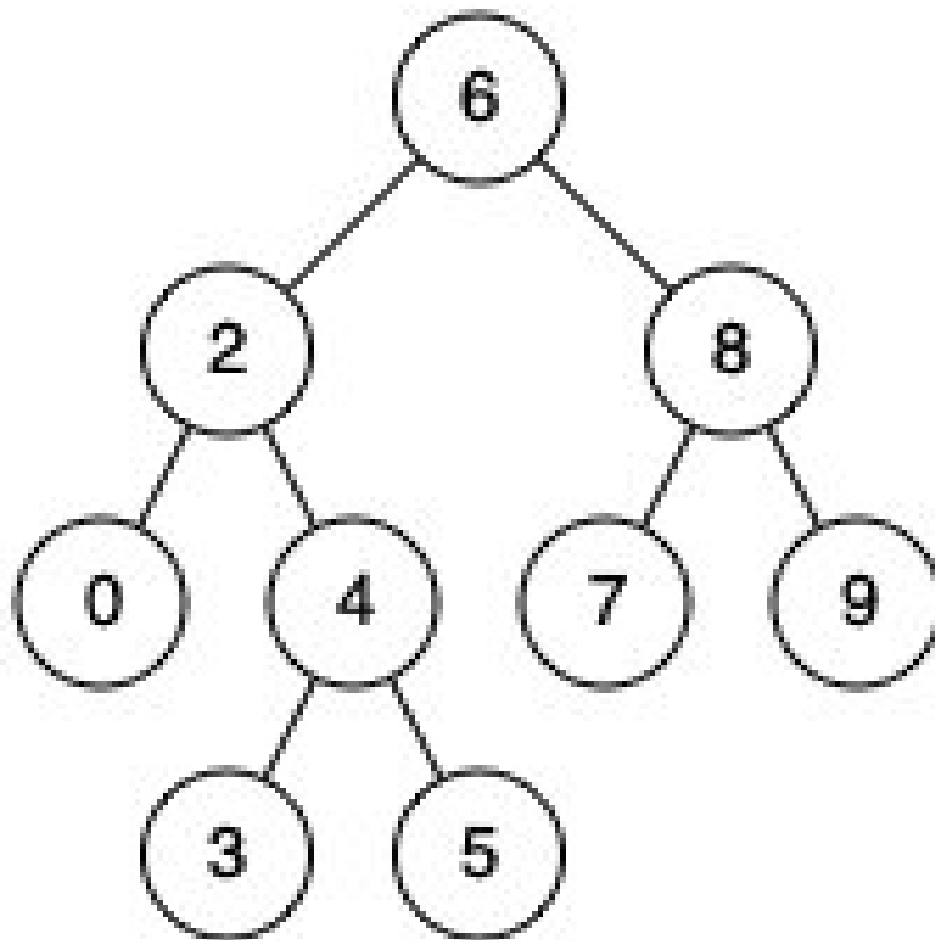


Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8

Output: 6

Explanation: The LCA of nodes 2 and 8 is 6.

Example 2:



Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 4

Output: 2

Explanation: The LCA of nodes 2 and 4 is 2, since a node can be a descendant of itself according to the LCA definition.

Example 3:

Input: root = [2,1], p = 2, q = 1

Output: 2

Constraints:

- The number of nodes in the tree is in the range [2, 105].
- $-109 \leq \text{Node.val} \leq 109$

- All Node.val are unique.
- $p \neq q$
- p and q will exist in the BST.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode') -> 'TreeNode':
        small = min(p.val, q.val)
        large = max(p.val, q.val)

        while root:
            if root.val > large:
                root = root.left
            elif root.val < small:
                root = root.right
            else:
                return root
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "In a BST, find the LCA of nodes p and q.
# LCA = the deepest node that is an ancestor of both p and q (a node is its own
# ancestor). Right?
# Use the BST property: values to the left < root < values to the right."
#
# "[6,2,8,0,4,7,9], p=2, q=8 → LCA=6 (they're on different sides of 6) ✓
# p=2, q=4 → LCA=2 (4 is in 2's right subtree) ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Walk from root: if both p and q are smaller, go left; if both larger, go right.
# First node where paths split is the LCA in a BST.
# Alternatively store root-to-p and root-to-q paths – more work than the walk.
# Time: O(h). Space: O(h) if storing paths.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (BST property)**# "Navigate using BST property:**

If both p and q are LESS than root: LCA is in the LEFT subtree.

If both are GREATER: LCA is in the RIGHT subtree.

Otherwise (they split at root, or one equals root): root IS the LCA.

O(h) time – O(log n) for balanced BST, O(n) worst case."

PHASE 4 — CLEAN CODE**class _235_LCA_BST:** **def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode')****-> 'TreeNode':** **while root:** **if p.val < root.val and q.val < root.val:** **root = root.left** **elif p.val > root.val and q.val > root.val:** **root = root.right** **else:** **return root** **return root****# PHASE 5 — TEST & DEBUG**

root=6, p=2, q=8: both in different sides → return 6 ✓

root=6, p=2, q=4: both<6→go left to 2. p=2==root, q=4>root → split here → return 2

✓

PHASE 6 — COMPLEXITY & FOLLOW-UP**# "Time O(h). Space O(1) – iterative, no stack.**

The BST property makes this an O(h) walk – no need to visit all nodes.

This is much faster than the general binary tree LCA (#236) which needs DFS.

Given more time I'd ask: what if p or q might not exist in the tree?

We'd need to verify existence first – two separate searches, then LCA."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def lowestCommonAncestor(
        self,
        root: 'TreeNode',
        p: 'TreeNode',
        q: 'TreeNode',
    ) -> 'TreeNode':
        if root.val > max(p.val, q.val):
            return self.lowestCommonAncestor(root.left, p, q)
        if root.val < min(p.val, q.val):

```

```

        return self.lowestCommonAncestor(root.right, p, q)
    return root

```

• LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    def lowestCommonAncestor(
        self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode'
    ) -> 'TreeNode':
        while 1:
            if root.val < min(p.val, q.val):
                root = root.right
            elif root.val > max(p.val, q.val):
                root = root.left
            else:
                return root

```

• SimplyLeet

```

def lowestCommonAncestor(root, p, q):
    # Iterate until we find the lowest common ancestor
    while root:
        # If both p and q are less than the current node, move to left subtree
        if p.val < root.val and q.val < root.val:
            root = root.left
        # If both are greater, move to right subtree
        elif p.val > root.val and q.val > root.val:
            root = root.right
        else:
            # Found the split point; return current node as LCA
            return root

```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    def lowestCommonAncestor(
        self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode'

    ) -> 'TreeNode':
        while 1:
            if root.val < min(p.val, q.val): # no =, so root is p or q is in
else block
                root = root.right
            elif root.val > max(p.val, q.val):
                root = root.left
            else:
                return root

#####

class Solution:
    def lowestCommonAncestor(self, root, p, q):
        if root is None:
            return root

        if root.val > max(p.val, q.val):
            return self.lowestCommonAncestor(root.left, p, q)
        elif root.val < min(p.val, q.val):
            return self.lowestCommonAncestor(root.right, p, q)
```

```

        else:
            return root

#####

class Solution(object):
    def lowestCommonAncestor(self, root, p, q):
        """
        :type root: TreeNode
        :type p: TreeNode

        :type q: TreeNode
        :rtype: TreeNode
        """
        a, b = sorted([p.val, q.val])
        while not a <= root.val <= b:
            if a > root.val:
                root = root.right
            else:
                root = root.left
        return root

```

◆ Quick Review

Key Insights

- Utilize the BST property: for any node, values in the left subtree are less and values in the right subtree are greater.
- If both p and q have values less than the current node, the LCA must lie in the left subtree.
- If both p and q have values greater than the current node, the LCA must lie in the right subtree.

- When one node value is less than and the other is greater than (or equal to) the current node, the current node is the lowest common ancestor.
- An iterative approach is efficient and uses constant extra space.

Space & Time

Time Complexity: $O(h)$, where h is the height of the tree ($O(\log n)$ for balanced trees and $O(n)$ in the worst-case skewed tree).

Space Complexity: $O(1)$ with an iterative approach ($O(h)$ if using recursion due to call stack).

Solution

The solution leverages the BST properties.

Algorithm: Start at the root node. While the current node exists: If both p and q have values less than the current node's value, move to the left child. If both p and q have values greater than the current node's value, move to the right child. Otherwise, the current node is the split point where p and q diverge, so it is the lowest common ancestor. Return the current node. This approach efficiently

finds the LCA using a single traversal from the root.

Trees & BST

□ #236 Lowest Common Ancestor of a Binary Tree

MED[Open on LeetCode](#)

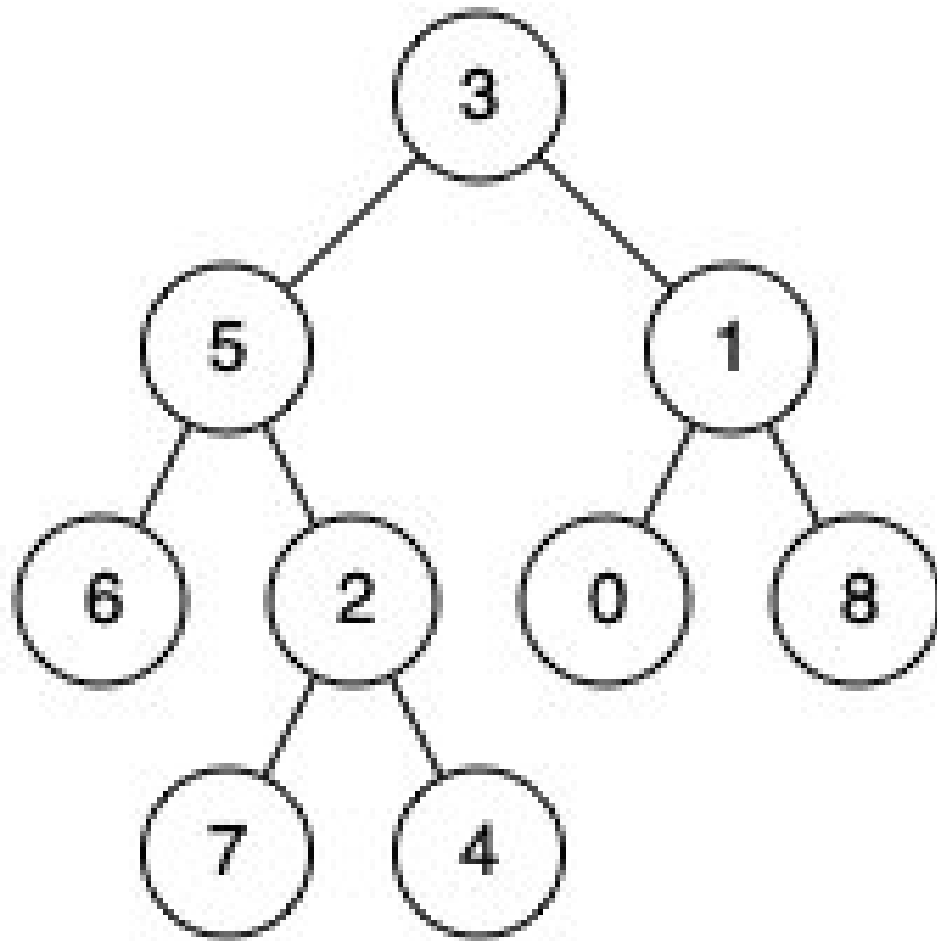
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS

Lists: Grind 169

Problem

Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the tree. According to the definition of LCA on Wikipedia: “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself).”

Example 1:

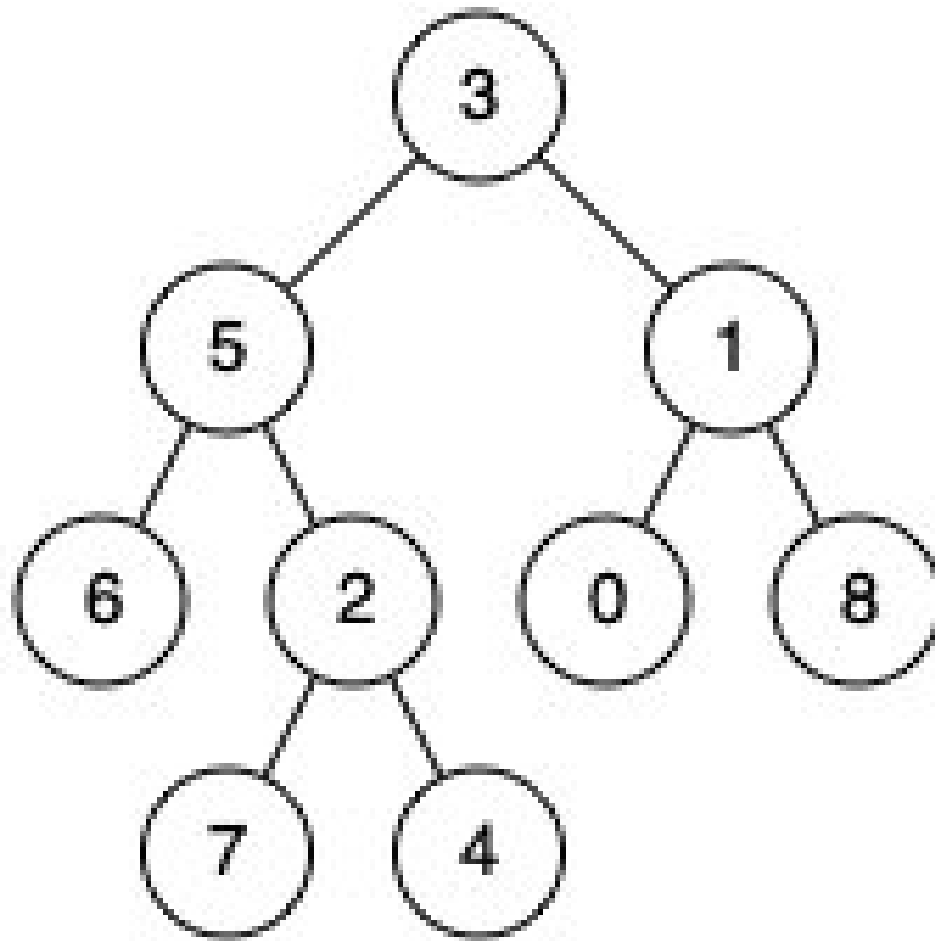


Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1

Output: 3

Explanation: The LCA of nodes 5 and 1 is 3.

Example 2:



Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4

Output: 5

Explanation: The LCA of nodes 5 and 4 is 5, since a node can be a descendant of itself according to the LCA definition.

Example 3:

Input: root = [1,2], p = 1, q = 2

Output: 1

Constraints:

- The number of nodes in the tree is in the range [2, 105].
- $-109 \leq \text{Node.val} \leq 109$

- All Node.val are unique.
- $p \neq q$
- p and q will exist in the tree.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode') -> 'TreeNode':
        def find_lca(node):
            if not node:
                return None
            if node == p or node == q:
                return node

            left = find_lca(node.left)
            right = find_lca(node.right)

            if left and right:
                return node
            return left or right

        return find_lca(root)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "General binary tree – no BST property. Find the LCA of p and q.
# p and q are guaranteed to exist in the tree. A node is its own ancestor. Right?"
#
# "[3,5,1,6,2,0,8,null,null,7,4], p=5, q=1 → LCA=3 (root, both are in different subtrees) ✓
# p=5, q=4 → LCA=5 (4 is in 5's subtree) ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Find root-to-p path and root-to-q path as node lists.
# Walk both lists in parallel; last matching node before divergence is LCA."
```

```
# Correct O(n) but needs O(h) path storage; post-order single-pass is cleaner.
# Time: O(n). Space: O(h) two paths.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (recursive post-order)

```
# "DFS: if current node is p or q, return it (it or its ancestor is the LCA).
# Recurse left and right.
# If both return non-null: current node is the LCA (p and q are in different
subtrees).
# If only one is non-null: that subtree contains both p and q – propagate that
result up."
```

PHASE 4 — CLEAN CODE

```
class _236_LCA_BinaryTree:
    def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode')
-> 'TreeNode':
    def lca(node):
        if not node or node is p or node is q:
            return node
        left = lca(node.left)
        right = lca(node.right)
        if left and right:
            return node # p and q are in different subtrees
        return left or right # both in same subtree – propagate the found node
    return lca(root)
```

PHASE 5 — TEST & DEBUG

```
# [3,5,1,...], p=5, q=1:
# lca(3): lca(5)→returns 5 (found p). lca(1)→returns 1 (found q).
# Both non-null → return 3 ✓
# p=5, q=4 (4 is under 5):
# lca(3): lca(5)→... lca(4) under 5 returns 4. lca(5) itself: node is p→return 5.
# lca(1)→None (neither p nor q in right subtree).
# left=5, right=None → return 5 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n) – visits every node. Space O(h) for the call stack.
# Contrast with #235 (BST): BST LCA is O(h) iteratively. General tree needs full
DFS.
# The post-order logic is the key: left and right results tell us which subtrees
contain p/q.
# Both non-null → split here → current node is LCA.
# Only one non-null → both are in the same subtree → propagate that subtree's
result.
# Given more time I'd ask: what if p or q might not exist?
# We'd need to track 'found p' and 'found q' separately – more complex state."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def lowestCommonAncestor(
        self,
        root: 'TreeNode',
        p: 'TreeNode',
        q: 'TreeNode',
    ) -> 'TreeNode':
        if not root or root == p or root == q:
            return root
        left = self.lowestCommonAncestor(root.left, p, q)

        right = self.lowestCommonAncestor(root.right, p, q)
        if left and right:
            return root
        return left or right
```

```
class Solution:
    def lowestCommonAncestor(
        self,
        root: TreeNode | None,
        p: TreeNode | None,
        q: TreeNode | None,
    ) -> TreeNode | None:
        q_ = collections.deque([root])
        parent = {root: None}
        ancestors = set() # p's ancestors
```

```
# Iterate until we find both p and q.
while p not in parent or q not in parent:
    root = q_.popleft()
    if root.left:
        parent[root.left] = root
        q_.append(root.left)
    if root.right:
        parent[root.right] = root
        q_.append(root.right)
```

```

# Insert all the p's ancestors.
while p:
    ancestors.add(p)
    p = parent[p] # `p` becomes None in the end.

# Go up from q until we meet any of p's ancestors.
while q not in ancestors:
    q = parent[q]

return q

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    def lowestCommonAncestor(
        self, root: "TreeNode", p: "TreeNode", q: "TreeNode"
    ) -> "TreeNode":
        if root in (None, p, q):
            return root
        left = self.lowestCommonAncestor(root.left, p, q)
        right = self.lowestCommonAncestor(root.right, p, q)
        return root if left and right else (left or right)

```

• SimplyLeet


```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNo
de') -> 'TreeNode':
        # Base case: if the current node is None, return None.

        if root is None:
            return None

        # If the current node is either p or q, return the current node.
        if root == p or root == q:
            return root

        # Search for p and q in the left and right subtrees.
        left = self.lowestCommonAncestor(root.left, p, q)
        right = self.lowestCommonAncestor(root.right, p, q)

        # If both left and right are non-null, the current node is the LCA.
        if left and right:
            return root

        # Otherwise, return the non-null node.
        return left if left is not None else right

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    def lowestCommonAncestor(
        self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode'

```

```

    ) -> 'TreeNode':
        if root is None or root == p or root == q:
            return root
        left = self.lowestCommonAncestor(root.left, p, q)
        right = self.lowestCommonAncestor(root.right, p, q)
        return root if left and right else (left or right)

```

```
#####
```

```
# Definition for a binary tree node.
```

```

# class TreeNode(object):
# def __init__(self, x):
# self.val = x
# self.left = None
# self.right = None

```

```

class Solution(object):
    def lowestCommonAncestor(self, root, p, q):
        """
        :type root: TreeNode

```

```

        :type p: TreeNode
        :type q: TreeNode
        :rtype: TreeNode
        """

```

```

        if not root:
            return root

```

```

        left = self.lowestCommonAncestor(root.left, p, q)
        right = self.lowestCommonAncestor(root.right, p, q)

```

```

        if left and right:
            return root

```

```

        if root == p or root == q:
            return root

```

```

        if left:
            return left

```

```

        if right:

```

```
return right  
return None
```

◆ Quick Review

Key Insights

- Use a depth-first search (DFS) traversal to explore the tree.
- If the current node is either p or q, it could be the LCA.
- Recursively find p and q in the left and right subtrees.
- If both left and right recursive calls return a non-null value, then the current node is the LCA.
- If only one side returns a non-null value, propagate that value up the recursion.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the tree since each node is visited once.

Space Complexity: $O(h)$, where h is the height of the tree due to the recursion stack ($O(n)$ in the worst case of a skewed tree).

Solution

We solve the problem using recursion with depth-first search. The algorithm checks if the current node matches p or q . It then recursively searches the left and right subtrees. If both subtrees contain one of p or q , the current node is the LCA. Otherwise, the algorithm forwards the non-null result up the recursion. This approach naturally handles cases where one node is an ancestor of the other.

Trees & BST

□ ★ #250 Count Univalue Subtrees ★

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS

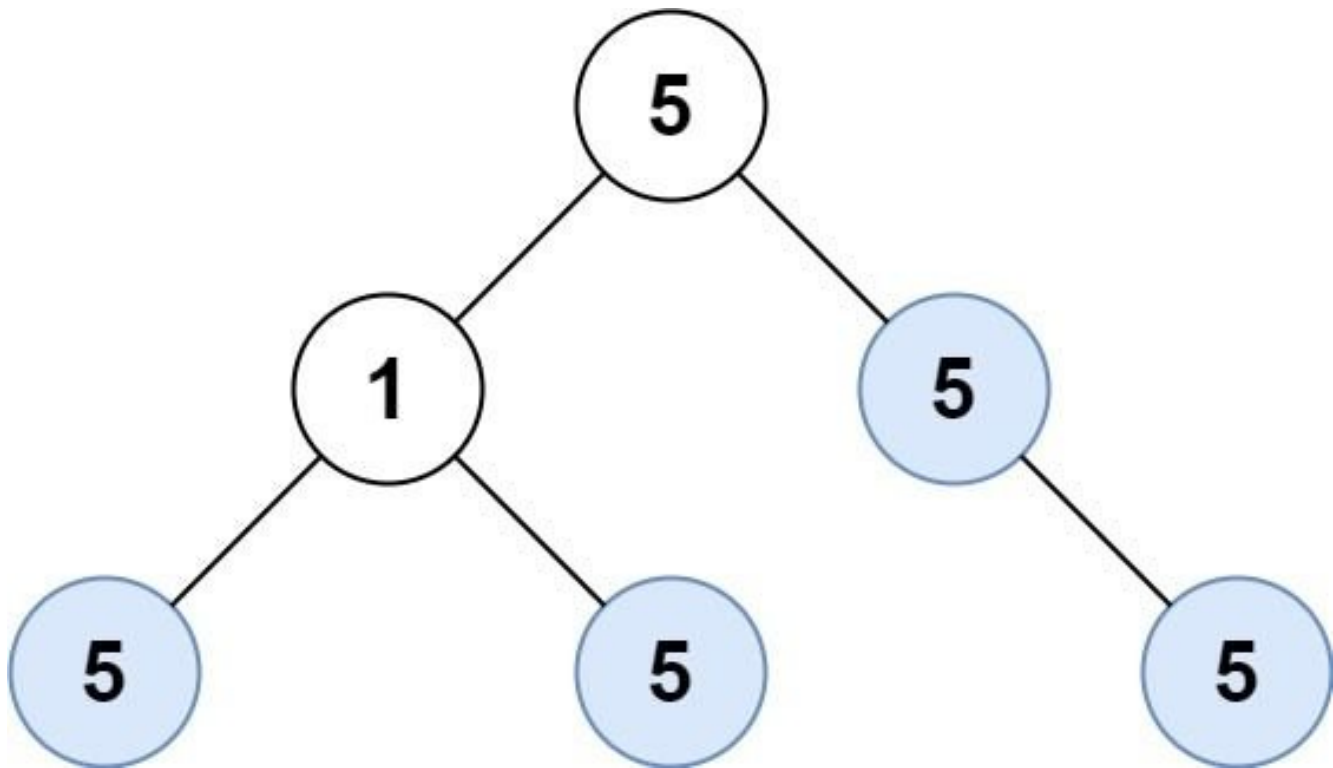
Lists: ★ Premium | Premium 98

Problem

Given the root of a binary tree, return the number of uni-value subtrees.

A uni-value subtree means all nodes of the subtree have the same value.

Example 1:



Input: root = [5,1,5,5,5,null,5]
Output: 4

Example 2:

Input: root = []
Output: 0

Example 3:

Input: root = [5,5,5,5,5,null,5]
Output: 6

Constraints:

- The number of the node in the tree will be in the range [0, 1000].
- $-1000 \leq \text{Node.val} \leq 1000$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def countUnivalSubtrees(self, root: Optional[TreeNode]) -> int:
        self.ans = 0

        def dfs(n):
            if not n:
                return True
            l, r = dfs(n.left), dfs(n.right)
            if l and r and (not n.left or n.left.val == n.val) and (not n.right or n.right.val == n.val):
                self.ans += 1
            return l and r
            return False

        dfs(root)
        return self.ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "A univalue subtree has every node with the same value.
# Count all such subtrees – including single leaf nodes. Right?
# Empty tree → 0."
#
# "[5,1,5,5,5,null,5] → 4 (the four nodes with value 5 that form valid univalue
# subtrees) ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# For each node, DFS its entire subtree checking every value equals node.val.
# Increment count when the whole subtree is uni-value.
# Revalidates overlapping subtrees – O(n^2) worst case.
# Time: O(n^2). Space: O(h) stack.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (post-order DFS)

```
# "Bottom-up: a node is a univalue root if both children are univalue AND their
# values match.
# Return True from a node if its subtree is univalue, increment count when True.
# Use an inner helper that returns bool + mutates a count list."
```

PHASE 4 — CLEAN CODE


```

class _250_CountUnivalSubtrees:
    def countUnivalSubtrees(self, root: Optional['TreeNode']) -> int:
        count = [0]
        def is_univalue(node, parent_val):
            if not node:
                return True
            left = is_univalue(node.left, node.val)
            right = is_univalue(node.right, node.val)
            if not left or not right:
                return False
            count[0] += 1
            return node.val == parent_val
        is_univalue(root, None)
        return count[0]

```

PHASE 5 — TEST & DEBUG

```

# [5,1,5,5,5,null,5]:
# Leaves: 5(left-left)->True, 5(left-right)->True, 5(right-right)->True, 1->True (leaf).
# Node(1): left=True, right=True, both return True. 1's children have val=5≠1 → returns False.
# Node(5, right of root): left=None=True, right(5)=True, right returns True(val matches). count=1. Returns True(5==root.val?).
# Continue... count ends at 4 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n). Space O(h). Post-order because we need children's results before parent's.
# The parent_val parameter lets us check if current node extends the parent's univalue property.
# Given more time I'd ask: longest path where all values are the same?
# Track univalue path length similar to Diameter of Binary Tree – carry length down."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def countUnivalSubtrees(self, root: TreeNode | None) -> int:
        ans = 0

        def isUnival(root: TreeNode | None, val: int) -> bool:
            nonlocal ans
            if not root:
                return True

            if isUnival(root.left, root.val) & isUnival(root.right, root.val):

                ans += 1
                return root.val == val

            return False

        isUnival(root, math.inf)
        return ans

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def countUnivalSubtrees(self, root: Optional[TreeNode]) -> int:
        def dfs(root):
            if root is None:
                return True
            l, r = dfs(root.left), dfs(root.right)
            if not l or not r:
                return False
            a = root.val if root.left is None else root.left.val
            b = root.val if root.right is None else root.right.val
            if a == b == root.val:
                nonlocal ans
                ans += 1
                return True

```

```

        return False

    ans = 0
    dfs(root)
    return ans

```

• SimplyLeet

Definition for a binary tree node.

```
class TreeNode:
```

```
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

```

```
class Solution:
```

```
    def countUnivalSubtrees(self, root: TreeNode) -> int:
        # Initialize count

```

```
        self.count = 0

```

Helper function performs postorder DFS

```
def isUnival(node):
```

```
    # Base case, null nodes are considered univalue
    if node is None:
        return True

```

```
    # Recursive calls for left and right subtrees
    leftUnival = isUnival(node.left)

```

```
    rightUnival = isUnival(node.right)

```

Check if left child exists and its value does not match current node's value

```
    if node.left and node.left.val != node.val:
        leftUnival = False

```

Check if right child exists and its value does not match current node's value

```
    if node.right and node.right.val != node.val:
        rightUnival = False

```

If either left or right subtree is not univalue, then the current subtree is not univalue

```

        if leftUnival and rightUnival:
            # Increment the count if the subtree rooted at the current node
            # is univalued
            self.count += 1
            return True
        else:
            return False

    # Start recursion from root
    isUnival(root)
    return self.count

```

```

# Example usage:
# Construct the binary tree: [5,1,5,5,5,null,5]
# root = TreeNode(5)
# root.left = TreeNode(1, TreeNode(5), TreeNode(5))
# root.right = TreeNode(5, None, TreeNode(5))
# print(Solution().countUnivalSubtrees(root)) # Output: 4

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def countUnivalSubtrees(self, root: TreeNode) -> int:
        count = 0

        def is_unival(node):
            nonlocal count
            if not node:
                return True

            left_unival = is_unival(node.left)
            right_unival = is_unival(node.right)

            if left_unival and right_unival:

```

```

        if node.left and node.val != node.left.val:
            return False
        if node.right and node.val != node.right.val:
            return False
        count += 1
        return True

    return False

is_unival(root)

```

```

return count

```

```

class Solution: # return 2 values
    def countUnivalSubtrees(self, root: TreeNode) -> int:
        def is_unival(node):
            if not node:
                return True, 0

            left_unival, left_count = is_unival(node.left)

            right_unival, right_count = is_unival(node.right)

            if left_unival and right_unival:
                if node.left and node.val != node.left.val:
                    return False, 0
                if node.right and node.val != node.right.val:
                    return False, 0
                return True, left_count + right_count + 1

            return False, 0

        _, count = is_unival(root)
        return count

```

◆ Quick Review

Key Insights

- Perform a postorder traversal (left, right, root) to determine whether each subtree is univalue.
- Treat null nodes as univalue to ease the recursion.
- A node's subtree is univalue if both left and right subtrees are univalue and, if they exist, their values equal the current node's value.
- Use a global or external counter that increments every time a univalue subtree is confirmed.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes, since each node is visited once.

Space Complexity: $O(h)$, where h is the height of the tree, due to the recursion stack.

Solution

We use a recursive DFS approach (postorder traversal) to solve the problem. For each node, we recursively check its left and right subtrees to decide if they are univalue. By default, if a child is null, it is considered univalue. Then, we verify the current node's value against its non-null children. If both conditions hold

(subtrees are univalue and matching values), we count the current node's subtree as univalue. This DFS checks in a bottom-up manner, ensuring that the properties required for a subtree to be univalue are maintained. The key trick is to return a boolean indicating if the subtree at each node is univalue and, while backtracking, increment the counter appropriately.

Trees & BST

□ #285 Inorder Successor in BST

MED

[Open on LeetCode](#)

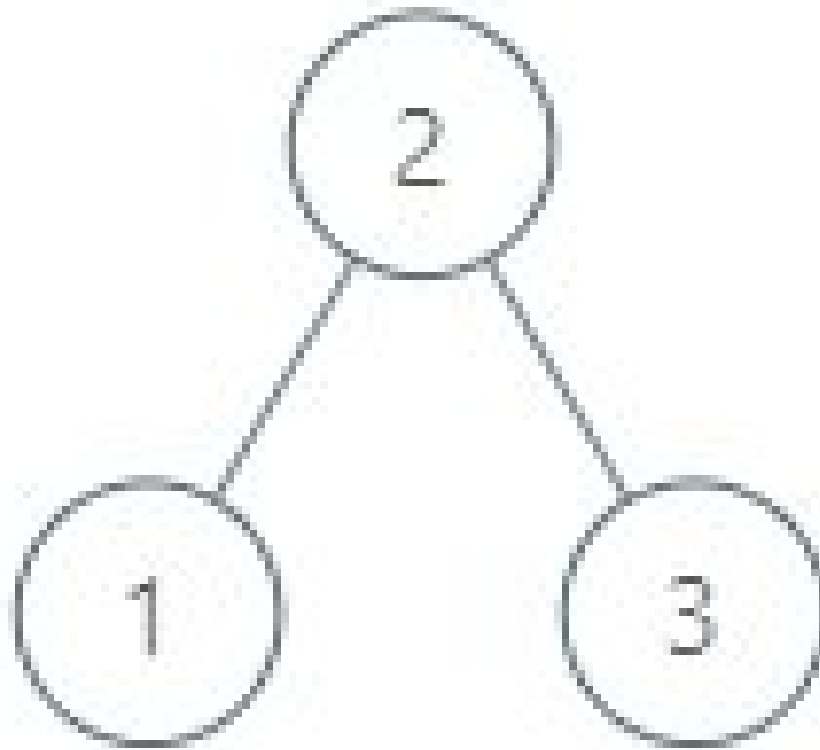
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · BST
Lists: Grind 169

Problem

Given the root of a binary search tree and a node p in it, return the in-order successor of that node in the BST. If the given node has no in-order successor in the tree, return null.

The successor of a node p is the node with the smallest key greater than $p.val$.

Example 1:

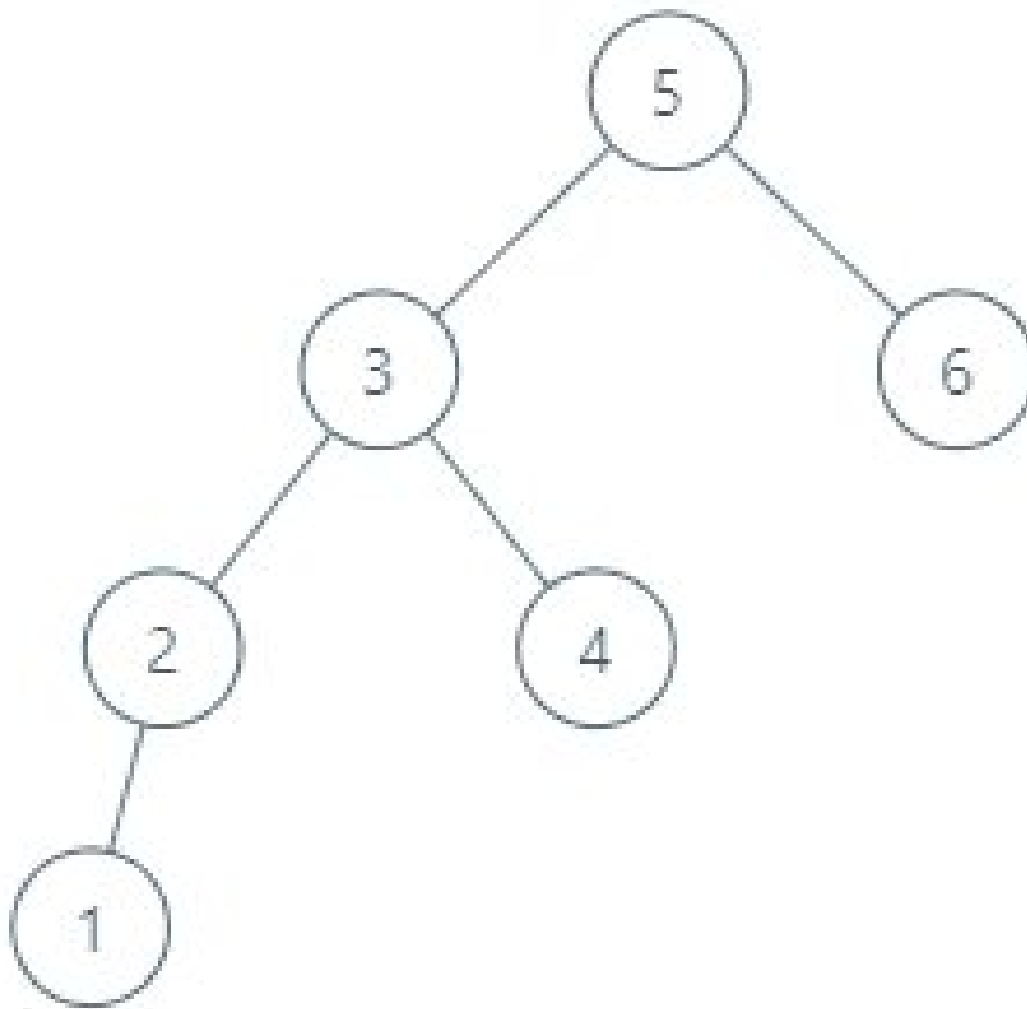


Input: root = [2,1,3], p = 1

Output: 2

Explanation: 1's in-order successor node is 2. Note that both p and the return value is of TreeNode type.

Example 2:



Input: root = [5,3,6,2,4,null,null,1], p = 6

Output: null

Explanation: There is no in-order successor of the current node, so the answer is null.

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $-105 \leq \text{Node.val} \leq 105$
- All Nodes will have unique values.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def inorderSuccessor(self, root: TreeNode, p: TreeNode) -> Optional[TreeNode]:
        succ = None
        while root:
            if p.val < root.val:
                succ = root
                root = root.left
            else:
                root = root.right
        return succ
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Find the node with the smallest value strictly greater than p.val in a BST.
# Return null if no such node exists (p is the largest). Right?"
#
# "[2,1,3], p=1 → successor=2 (next in in-order: 1,2,3) ✓
# [5,3,6,2,4,null,null,1], p=6 → null (6 is the largest) ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# In-order traversal of the BST; when we visit p, the next node in-order is
# successor.
# Store previous node during traversal or collect all values then scan.
# Correct O(n); BST structure gives O(h) successor without full traversal.
# Time: O(n). Space: O(1) iterative with threading concept.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (BST navigation)

```
# "Use BST property – no need to traverse all nodes.
# If root.val > p.val: root is a candidate successor, but there might be a smaller
# one in the left subtree.
# If root.val <= p.val: successor must be in the right subtree.
# Track the best candidate seen so far. O(h) time."
```

PHASE 4 — CLEAN CODE

```
class _285_InorderSuccessor:
    def inorderSuccessor(self, root: 'TreeNode', p: 'TreeNode') ->
Optional['TreeNode']:
        successor = None
        while root:
            if root.val > p.val:
```

```

        successor = root # root is a candidate – try for smaller in left
        root = root.left
    else:
        root = root.right # successor must be to the right
    return successor

```

PHASE 5 — TEST & DEBUG

```

# [2,1,3], p=1:
# root=2: 2>1 → successor=2, go left.
# root=1: 1<=1 → go right.
# root=None → return 2 ✓
# [5,3,6,2,4,null,null,1], p=6:
# root=5: 5<=6 → go right. root=6: 6<=6 → go right. root=None → return None ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(h). Space O(1) – iterative.
# The candidate tracking is the key: every time we go left, the current node becomes
# a potential successor (it's bigger than p, and we're looking for something
# smaller).
# Given more time I'd ask: inorder PREDECESSOR (largest value < p.val)?
# Mirror logic: if root.val < p.val → candidate, go right. If root.val >= p.val → go
# left."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def inorderSuccessor(
        self,
        root: TreeNode | None,
        p: TreeNode | None,
    ) -> TreeNode | None:
        if not root:
            return None
        if root.val <= p.val:
            return self.inorderSuccessor(root.right, p)

        return self.inorderSuccessor(root.left, p) or root

```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    def inorderSuccessor(self, root: TreeNode, p: TreeNode) -> Optional[TreeNode]:

        ans = None
        while root:
            if root.val > p.val:
                ans = root
                root = root.left
            else:
                root = root.right
        return ans
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def inorderSuccessor(self, root: TreeNode, p: TreeNode) -> TreeNode:
        # If p has a right subtree, the successor is the left-most node in that subtree.
```

```

    if p.right:
        successor = p.right
        while successor.left:
            successor = successor.left
        return successor

    # Otherwise, start from the root and keep track of a potential
    successor.
    successor = None
    current = root
    while current:

        if p.val < current.val:
            # current is a candidate, so record it and move left to find a
            smaller candidate
            successor = current
            current = current.left
        else:
            # If p.val is greater or equal, go right.
            current = current.right
    return successor

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    def inorderSuccessor(self, root: TreeNode, p: TreeNode) -> Optional[TreeNode]:

```

```
ans = None
while root:
    if root.val > p.val:
        ans = root
        root = root.left
    else:
        root = root.right
return ans
```

◆ Quick Review

Key Insights

- In a BST, an in-order traversal yields nodes in ascending order.
- If the node p has a right subtree, the in-order successor is the left-most node in that right subtree.
- If p has no right subtree, the successor is one of its ancestors; traverse from the root towards p while keeping track of the last node that is greater than p .
- An iterative approach is efficient and avoids recursion stack overhead.

Space & Time

Time Complexity: $O(h)$, where h is the height of the tree. In the worst-case (skewed tree), it can be $O(n)$.

Space Complexity: $O(1)$ as only a constant amount of extra space is used.

Solution

We solve the problem using two cases: If node p has a right subtree, we simply find the left-most node in that right subtree. If node p does not have a right subtree, initiate a search from the BST root. As we traverse: If the current node's value is greater than $p.val$, the current node is a candidate for the successor; move to the left subtree to look for a smaller candidate. Otherwise, move to the right subtree. By the end of the traversal, the candidate (if exists) is the in-order successor. This approach benefits from the BST property, limiting traversal to $O(h)$ time.

Trees & BST

□ ★ #298 Binary Tree Longest Consecutive Sequence ★

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS

Lists: ★ Premium | Premium 98

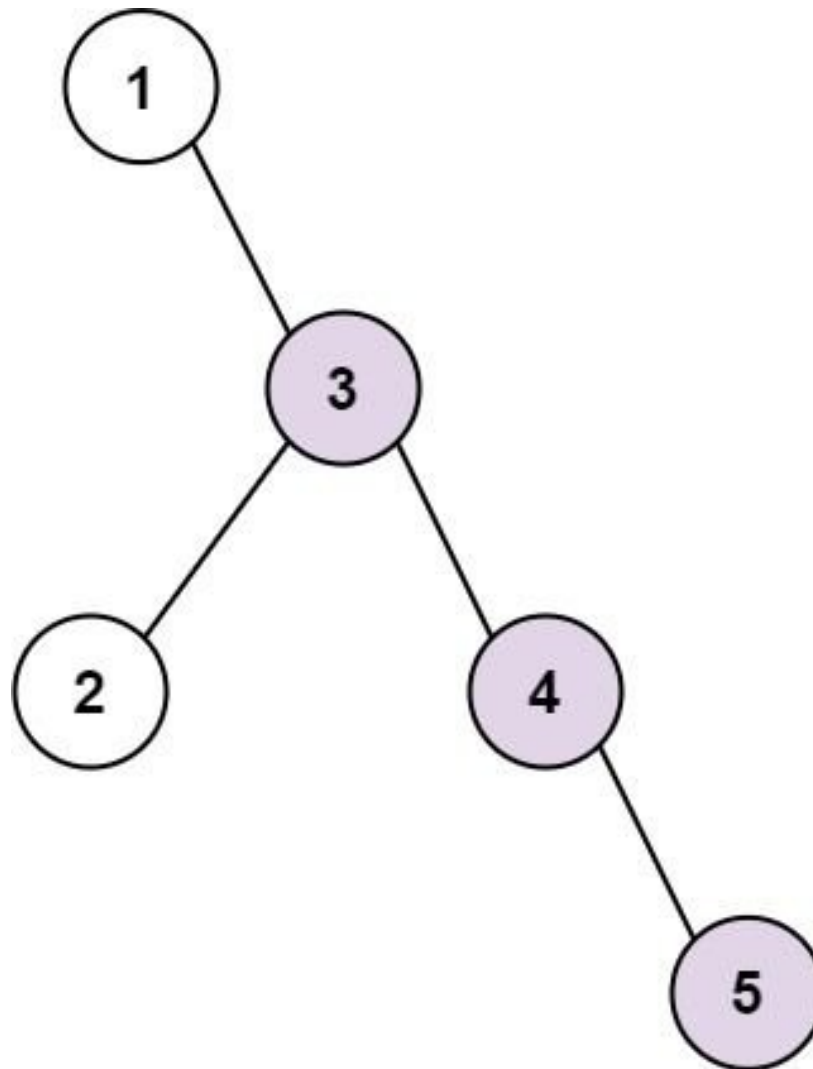
Problem

Given the root of a binary tree, return the length of the longest consecutive sequence path.

A consecutive sequence path is a path where the values increase by one along the path.

Note that the path can start at any node in the tree, and you cannot go from a node to its parent in the path.

Example 1:

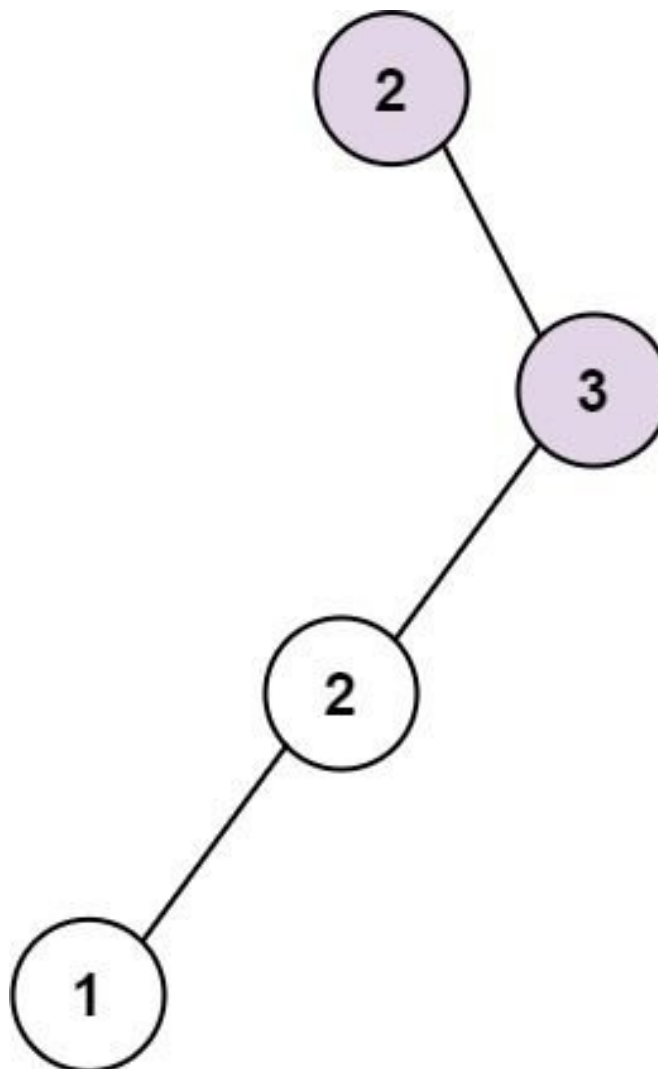


Input: root = [1,null,3,2,4,null,null,null,5]

Output: 3

Explanation: Longest consecutive sequence path is 3-4-5, so return 3.

Example 2:



Input: root = [2,null,3,2,null,1]

Output: 2

Explanation: Longest consecutive sequence path is 2-3, not 3-2-1, so return 2.

Constraints:

- The number of nodes in the tree is in the range $[1, 3 * 10^4]$.
- $-3 * 10^4 \leq \text{Node.val} \leq 3 * 10^4$

My LeetCode Solution

- My LeetCode Solution

```
class Solution:
    def longestConsecutive(self, root: Optional[TreeNode]) -> int:
        def dfs(node, parent_val, length):
            if not node:
                return length

            if node.val == parent_val + 1:
                length += 1
            else:
                length = 1

            return max(length, dfs(node.left, node.val, length), dfs(node.right, node.val, length))

        return dfs(root, root.val - 1, 0)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Find the longest path of consecutive increasing values (each node = parent + 1).
# Path goes parent -> child - you can't go back up. Path can start at any node.
Right?"
#
# "[1,null,3,2,4,null,null,null,5]: path 3-4-5, length 3 ✓
# [2,null,3,2,null,1]: 2-3 is increasing, not 3-2 - so max is 2 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# From each node, DFS tracking consecutive length when child.val == parent.val + 1.
# Reset streak when the +1 chain breaks; track global maximum.
# Starting DFS from every node repeats work - post-order aggregation is faster.
# Time: O(n^2) naive restarts. Space: O(h) stack.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (DFS carrying current length)

```
# "DFS top-down: pass current consecutive length to each child.
# If child.val == parent.val + 1: extend the sequence.
# Else: reset to 1 (start a new sequence from this child).
# Track global max. Use inner helper with parent_val and current_length."
```

PHASE 4 — CLEAN CODE

```
class _298_LongestConsecutive:
    def longestConsecutive(self, root: Optional['TreeNode']) -> int:
        best = [0]
```

```
def dfs(node, parent_val, length):
    if not node:
        return
    length = length + 1 if node.val == parent_val + 1 else 1
    best[0] = max(best[0], length)
    dfs(node.left, node.val, length)
    dfs(node.right, node.val, length)

dfs(root, root.val - 1 if root else 0, 0)
return best[0]
```

PHASE 5 — TEST & DEBUG

```
# [1,null,3,2,4,null,null,null,5]:
# dfs(1, 0, 0): val=1, prev+1=1 → length=1. best=1. dfs(3,1,1).
# dfs(3, 1, 1): val=3, prev+1=2, 3≠2 → length=1. best=1. dfs(2,3,1).
# dfs(4, 3, 1): val=4==3+1 → length=2. best=2. dfs(5,4,2).
# dfs(5, 4, 2): val=5==4+1 → length=3. best=3. → 3 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(h).
# The initial call uses root.val-1 so the root itself resets to length 1 cleanly.
# This is a top-down DFS – pass state downward. Contrast with bottom-up (#250, #543).
# Given more time I'd ask: what if the path can go through any node (not just parent-child)?
# That's Longest Consecutive Sequence in a Binary Tree – post-order, track both ascending
# and descending sequences at each node."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestConsecutive(self, root: TreeNode | None) -> int:
        if not root:
            return 0

        def dfs(root: TreeNode | None, target: int, length: int, maxLength: int) -> int:
            if not root:
                return maxLength
            if root.val == target:
                length += 1
            return dfs(root.left, root.val, length, maxLength) or dfs(root.right, root.val, length, maxLength)
```

```

        maxLength = max(maxLength, length)
    else:
        length = 1
    return max(dfs(root.left, root.val + 1, length, maxLength),
              dfs(root.right, root.val + 1, length, maxLength))

return dfs(root, root.val, 0, 0)

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def longestConsecutive(self, root: Optional[TreeNode]) -> int:
        def dfs(root: Optional[TreeNode]) -> int:
            if root is None:
                return 0
            l = dfs(root.left) + 1
            r = dfs(root.right) + 1
            if root.left and root.left.val - root.val != 1:
                l = 1
            if root.right and root.right.val - root.val != 1:
                r = 1
            t = max(l, r)
            nonlocal ans
            ans = max(ans, t)

        return t

ans = 0
dfs(root)
return ans

```

• SimplyLeet


```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def longestConsecutive(self, root: TreeNode) -> int:
        # Initialize a variable to keep track of the maximum sequence length.

        self.max_length = 0

        # Helper function to perform DFS traversal.
        def dfs(node, parent_val, current_length):
            if not node:
                return
            # Check if the current node's value is consecutive.
            if node.val == parent_val + 1:
                current_length += 1
            else:
                current_length = 1
            # Update the maximum sequence length found so far.
            self.max_length = max(self.max_length, current_length)
            # Recursively traverse the left and right children.
            dfs(node.left, node.val, current_length)
            dfs(node.right, node.val, current_length)

        # Start DFS with an initial condition that forces the first node to
        count.
        dfs(root, root.val - 1, 0)
        return self.max_length

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

# dfs
class Solution:
    def longestConsecutive(self, root: TreeNode) -> int:

        maxLen = 0

        # curLen: consecutive length until parent node
        def dfs(root: TreeNode, lastVal: int, curLen: int) -> None:
            nonlocal maxLen
            if not root:
                return

            curLen = (curLen + 1) if (not lastVal) and root.val == lastVal + 1
            else 1
            maxLen = max(maxLen, curLen)

            dfs(root.left, root.val, curLen)
            dfs(root.right, root.val, curLen)

        dfs(root, None, 0)
        return maxLen

#####

# Definition for a binary tree node.

```

```
# class TreeNode(object):
# def __init__(self, x):
# self.val = x
# self.left = None
# self.right = None

from collections import deque

# iterative, bfs
class Solution:

    def longestConsecutive(self, root: TreeNode) -> int:
        if not root:
            return 0

        max_length = 0
        # Queue elements are tuples: (current node, length of consecutive
        sequence)
        queue = deque([(root, 1)])

        while queue:
            current, length = queue.popleft()

            max_length = max(max_length, length)

            for neighbor in [current.left, current.right]:
                if neighbor:
                    # Check if neighbor is consecutive
                    if neighbor.val == current.val + 1:
                        queue.append((neighbor, length + 1))
                    else:
                        queue.append((neighbor, 1))
```

```

        return max_length

#####

class Solution:
    def longestConsecutive(self, root: Optional[TreeNode]) -> int:
        def dfs(root: Optional[TreeNode]) -> int:
            if root is None:
                return 0
            l = dfs(root.left) + 1

            r = dfs(root.right) + 1
            if root.left and root.left.val - root.val != 1:
                l = 1
            if root.right and root.right.val - root.val != 1:
                r = 1
            t = max(l, r)
            nonlocal ans
            ans = max(ans, t)
            return t

        ans = 0
        dfs(root)
        return ans

```

◆ Quick Review

Key Insights

- Use a Depth-First Search (DFS) to traverse the binary tree.
- Pass the current node's value and the length of the current consecutive sequence in the recursive call.

- When a node's value is exactly one greater than its parent's value, increment the sequence length; otherwise, reset it to 1.
- Keep track of the maximum sequence length found during the traversal.

Space & Time

Time Complexity: $O(N)$, where N is the number of nodes in the tree, since each node is visited once.

Space Complexity: $O(H)$, where H is the height of the tree. In the worst-case of a skewed tree, the recursion stack can be $O(N)$.

Solution

The solution involves using DFS to explore every node in the binary tree while keeping track of the length of the consecutive sequence. For each node, if the node's value is one greater than its parent's value, we increment the sequence length; otherwise, we reset the sequence length to 1. During the traversal, a global variable is updated if the current sequence length exceeds the previously recorded maximum. This approach ensures that every possible consecutive path

is considered.

Trees & BST

□ ★ #314 Binary Tree Vertical Order Traversal ★

MED

[Open on LeetCode](#)

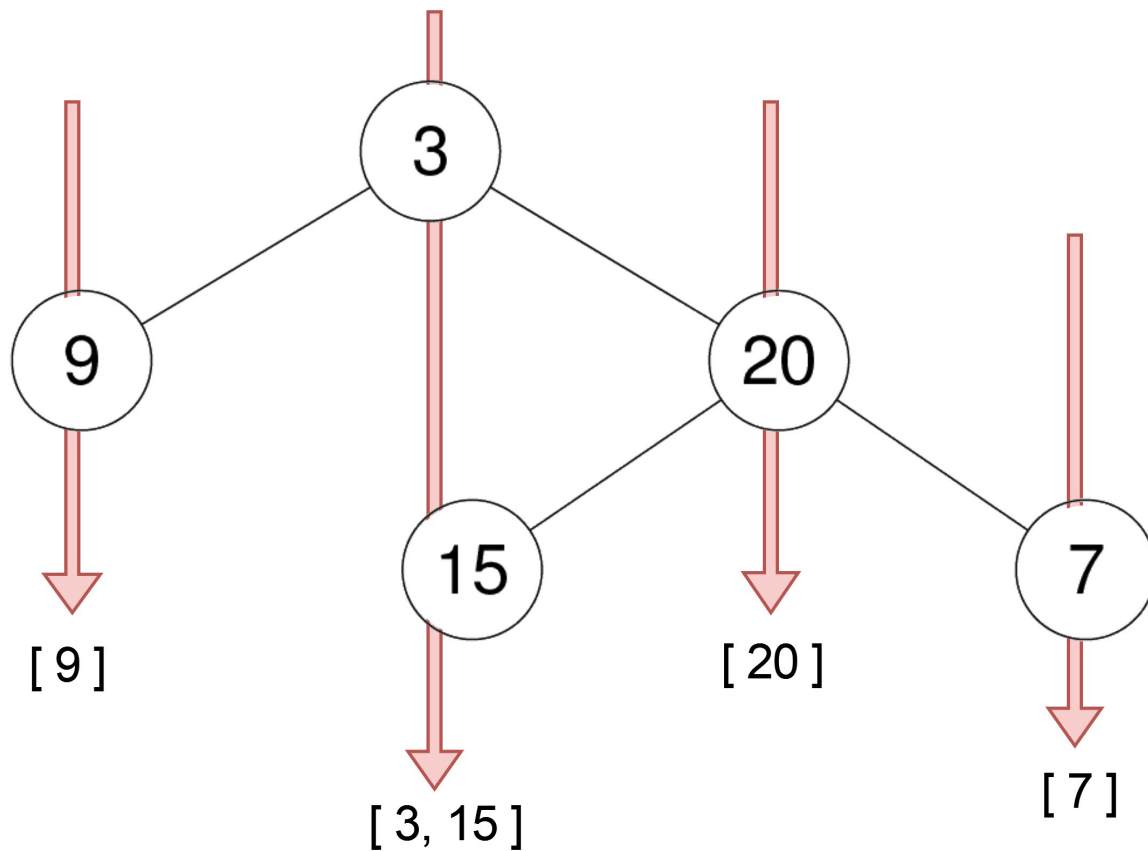
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Tree · BFS · Sorting
Lists: ★ Premium | Premium 98

Problem

Given the root of a binary tree, return the vertical order traversal of its nodes' values. (i.e., from top to bottom, column by column).

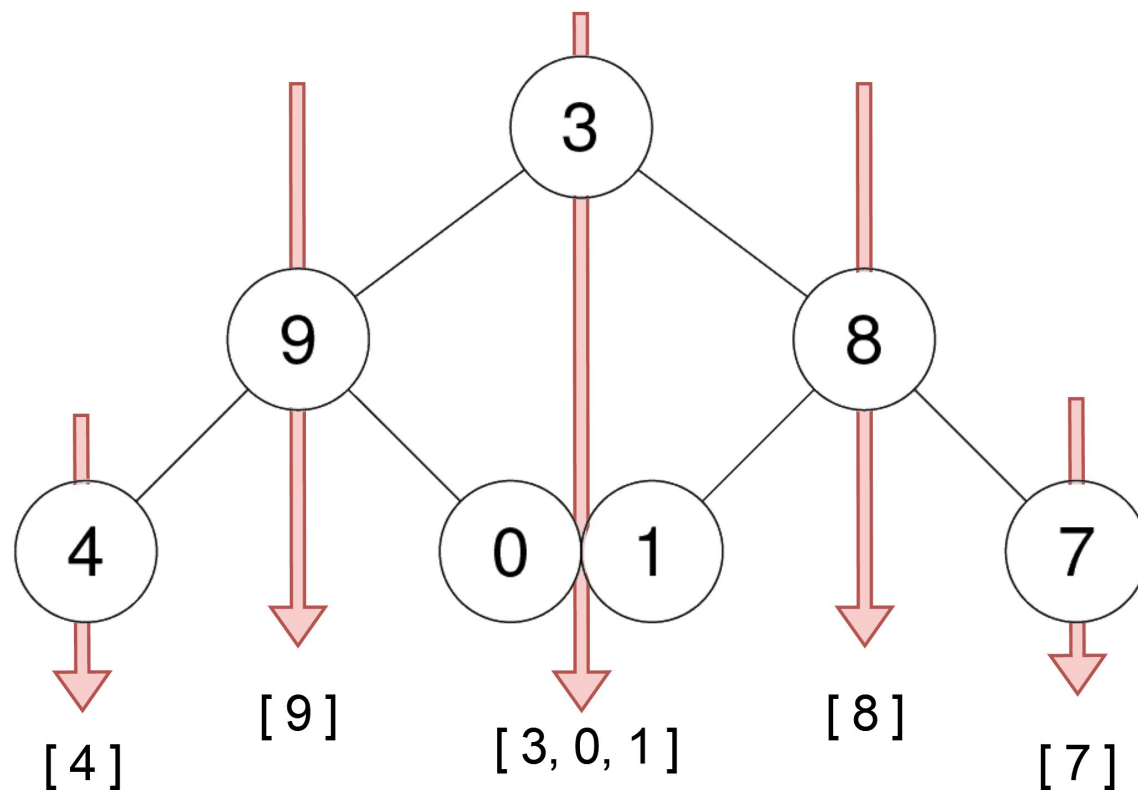
If two nodes are in the same row and column, the order should be from left to right.

Example 1:



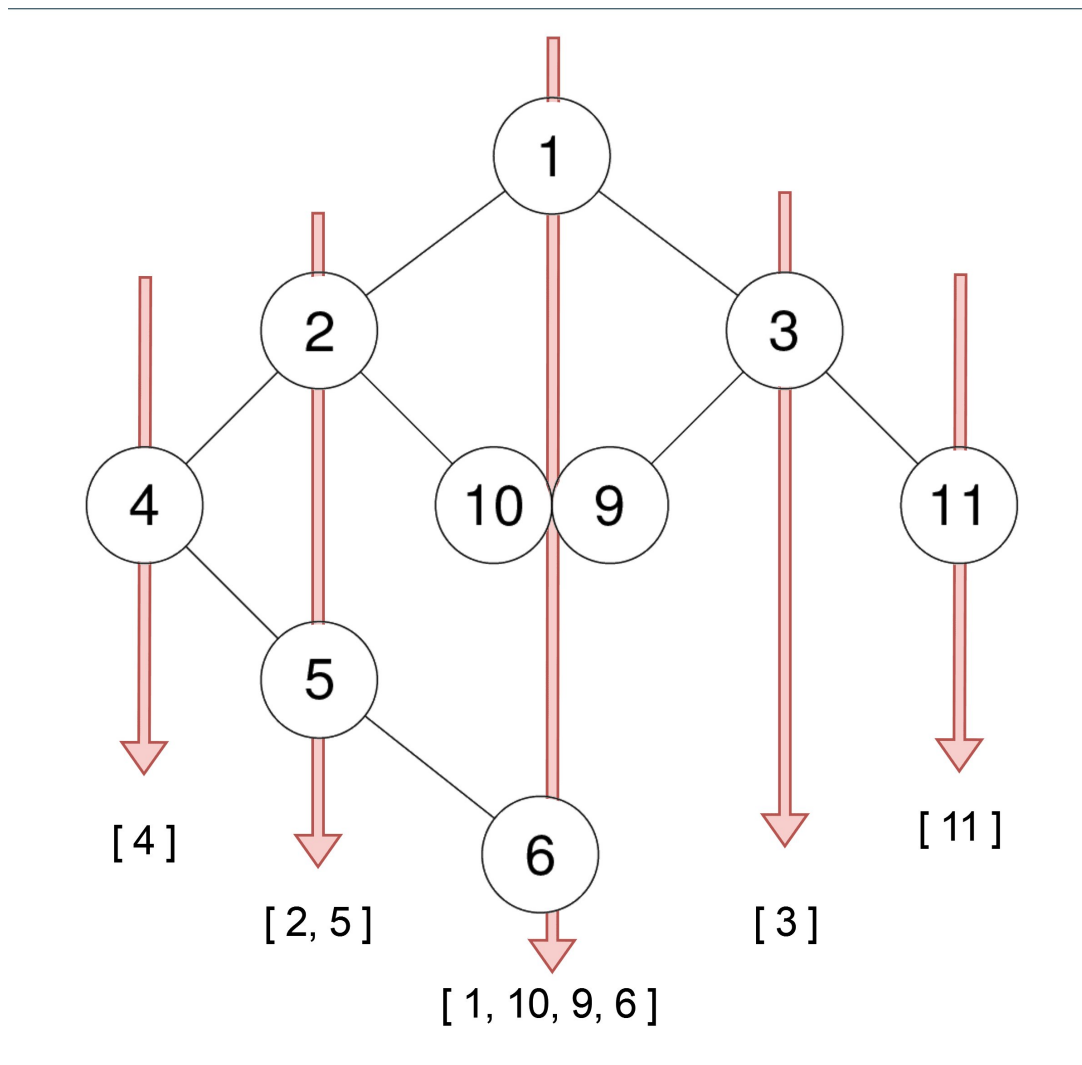
Input: root = [3,9,20,null,null,15,7]
Output: [[9],[3,15],[20],[7]]

Example 2:



Input: root = [3,9,8,4,0,1,7]
Output: [[4],[9],[3,0,1],[8],[7]]

Example 3:



Input: root = [1,2,3,4,10,9,11,null,5,null,null,null,null,null,null,6]
 Output: [[4],[2,5],[1,10,9,6],[3],[11]]

Constraints:

- The number of nodes in the tree is in the range [0, 100].
- $-100 \leq \text{Node.val} \leq 100$

My LeetCode Solution

- My LeetCode Solution

```

class Solution:
    def verticalOrder(self, root: TreeNode) -> List[List[int]]:
        if not root:
            return []

        cols = defaultdict(list)
        q = deque([(root, 0)])

        while q:
            node, col = q.popleft()

            cols[col].append(node.val)

            if node.left:
                q.append((node.left, col - 1))
            if node.right:
                q.append((node.right, col + 1))

        return [cols[col] for col in sorted(cols)]

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Group nodes by column. Root is column 0. Left child = col-1. Right child = col+1.
 # Within each column, top-to-bottom. Ties (same row and column): left before right.
 Right?
 # Return columns sorted from leftmost to rightmost."
 #
 # "[3,9,20,null,null,15,7] → [[9],[3,15],[20],[7]] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
 # DFS assigning column index (left=-1, right=+1 relative to parent).
 # Collect (col, row, val) tuples; sort by (col, row, val) then group by column.
 # Correct but sorting dominates at $O(n \log n)$.
 # Time: $O(n \log n)$. Space: $O(n)$ tuples.
 # Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (BFS with column tracking)

"BFS ensures natural top-to-bottom, left-to-right order within each column.
 # Queue entries: (node, col). Use defaultdict(list) keyed by col.
 # After BFS, sort the column keys and return their lists.
 # Time: $O(n \log n)$ for sorting. Space: $O(n)$."

PHASE 4 — CLEAN CODE

```
class _314_VerticalOrderTraversal:
    def verticalOrder(self, root: Optional['TreeNode']) -> List[List[int]]:
        if not root:
            return []
        cols : defaultdict = defaultdict(list)
        queue = deque([(root, 0)])
        min_col = max_col = 0
        while queue:
            node, col = queue.popleft()
            cols[col].append(node.val)
            min_col = min(min_col, col)
            max_col = max(max_col, col)
            if node.left: queue.append((node.left, col - 1))
            if node.right: queue.append((node.right, col + 1))
        return [cols[c] for c in range(min_col, max_col + 1)]

# PHASE 5 — TEST & DEBUG
# [3,9,20,null,null,15,7]:
# BFS: (3,0)→cols={0:[3]}. (9,-1),(20,1)→cols={0:[3],-1:[9],1:[20]}.
# (15,0),(7,2)→cols={0:[3,15],-1:[9],1:[20],2:[7]}.
# min=-1, max=2. Return [cols[-1],cols[0],cols[1],cols[2]] = [[9],[3,15],[20],[7]]
✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) for BFS + O(cols) for building the result – total O(n).
# Using min_col/max_col avoids sorting keys – the range is contiguous.
# BFS guarantees top-to-bottom, left-to-right order within each column naturally.
# DFS would require explicit (row, insertion_order) sorting per column.
# Given more time I'd ask: Vertical Order Traversal II (#987)?
# Same column/row but within the same position, sort by value ascending – needs
explicit row tracking."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def verticalOrder(self, root: TreeNode | None) -> list[list[int]]:
        if not root:
            return []

        range_ = [0] * 2

        def getRange(root: TreeNode | None, x: int) -> None:
            if not root:
                return
```

```

    range_[0] = min(range_[0], x)
    range_[1] = max(range_[1], x)

    getRange(root.left, x - 1)
    getRange(root.right, x + 1)

getRange(root, 0) # Get the leftmost and the rightmost x index.

ans = [[] for _ in range(range_[1] - range_[0] + 1)]

q = collections.deque([(root, -range_[0])]) # (TreeNode, x)

while q:
    node, x = q.popleft()
    ans[x].append(node.val)
    if node.left:
        q.append((node.left, x - 1))
    if node.right:
        q.append((node.right, x + 1))

return ans

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def verticalOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        def dfs(root, depth, offset):
            if root is None:

```

```

        return
    d[offset].append((depth, root.val))
    dfs(root.left, depth + 1, offset - 1)
    dfs(root.right, depth + 1, offset + 1)

    d = defaultdict(list)
    dfs(root, 0, 0)
    ans = []
    for _, v in sorted(d.items()):
        v.sort(key=lambda x: x[0])

    ans.append([x[1] for x in v])
    return ans

```

• SimplyLeet

```

from collections import deque, defaultdict

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, x):
        self.val = x
        self.left = None
        self.right = None

class Solution:

    def verticalOrder(self, root: TreeNode):
        # Return empty list if the tree is empty
        if not root:
            return []

        # Dictionary to hold nodes for each column index
        col_table = defaultdict(list)
        # Use deque for efficient FIFO queue for BFS; store tuple (node,
        column)
        queue = deque([(root, 0)])

```

```

while queue:
    node, col = queue.popleft()
    # Append current node value to the list of its column
    col_table[col].append(node.val)
    # If there is a left child, assign column col-1
    if node.left:
        queue.append((node.left, col - 1))
    # If there is a right child, assign column col+1
    if node.right:
        queue.append((node.right, col + 1))

# Extract the columns in sorted order to build the final result
sorted_cols = sorted(col_table.keys())
result = [col_table[col] for col in sorted_cols]
return result

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right

```

```

...
>>> d = {1:'a', -1:'b', 10:'c', -100:'d'}
>>>

```

```

>>> d.items()
{1: 'a', 10: 'c', -100: 'd', -1: 'b'}

>>> ds = sorted(d.items())
>>> ds
[(-100, 'd'), (-1, 'b'), (1, 'a'), (10, 'c')]

>>> sorted(d.items(), key=lambda l: l[1])
[(1, 'a'), (-1, 'b'), (10, 'c'), (-100, 'd')]
>>> dict(sorted(d.items(), key=lambda l: l[1]))

```



```
{1: 'a', 10: 'c', -100: 'd', -1: 'b'}
```

```
>>> [v for i,v in sorted(d.items(), key=lambda l: l[0])]
['d', 'b', 'a', 'c']
'''
```

```
class Solution:
    def verticalOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        if root is None:
            return []
```

```
        q = deque([(root, 0)])
        d = defaultdict(list)
        while q:
            for _ in range(len(q)):
                root, offset = q.popleft()
                d[offset].append(root.val)
                if root.left:
                    q.append((root.left, offset - 1))
                if root.right:
                    q.append((root.right, offset + 1))
```

```
        return [v for _, v in sorted(d.items())]
```

```
#####
```

```
from sortedcontainers import SortedDict
from typing import List
from collections import deque
```

```
# similar to above, but use existing data structure
```

```
class Solution:
```

```
    def verticalOrder(self, root: TreeNode) -> List[List[int]]:
        if not root:
            return []
```

```
        # Use SortedDict to automatically sort by column index
        sorted_dict = SortedDict()
```

```
        # Queue for breadth-first search; stores pairs of (node, column_index)
        queue = deque([(0, root)])
```

```

while queue:
    node, column = queue.popleft()

    if column not in sorted_dict:
        sorted_dict[column] = [node.val]
    else:
        sorted_dict[column].append(node.val)

    # Add left and right children to the queue
    if node.left:
        queue.append((node.left, column - 1))
    if node.right:
        queue.append((node.right, column + 1))

    # Return the values from sorted_dict as a list of lists
    return list(sorted_dict.values())

#####

# another SortedDict, but different handling on None node, before enqueue vs
after enqueue

```

```

class Solution:
    def verticalOrder(self, root: TreeNode):
        if not root:
            return []

        # Initialize a SortedDict to maintain columns in sorted order.
        column_table = SortedDict()
        queue = deque([(root, 0)]) # (node, column_index)

        while queue:

```

```

        node, column = queue.popleft()

        if node is not None:
            if column not in column_table:
                column_table[column] = []
            column_table[column].append(node.val)

            # Add left and right children to the queue.
            queue.append((node.left, column - 1))
            queue.append((node.right, column + 1))

    # Extract the values from the sorted dictionary and return them.
    return list(column_table.values())

#####

from collections import defaultdict


class Solution(object):
    def verticalOrder(self, root):
        """
        :type root: TreeNode
        :rtype: List[List[int]]
        """

    def dfs(p, i, j, res): # i -> depth, j -> vertical-shift
        if p:
            res[j].append((p.val, i))

            self.leftMost = min(j, self.leftMost)
            dfs(p.left, i + 1, j - 1, res)
            dfs(p.right, i + 1, j + 1, res)

    self.leftMost = float("inf")
    ans = []
    res = defaultdict(list)
    dfs(root, 0, 0, res)
    i = self.leftMost
    while True:

```

```
if not res[i]:  
    break  
ans.append([item[0] for item in sorted(res[i], key=lambda a: a[1])])  
i += 1  
return ans
```

◆ Quick Review

Key Insights

- Use Breadth-First Search (BFS) to traverse the tree level by level.
- Track the column index for each node where root is column 0, left child is column-1, and right child is column+1.
- Group nodes by their column indices using a hash table (or dictionary).
- BFS traversal naturally handles left-to-right ordering within the same level.
- Finally, sort the keys (column indices) to generate results from leftmost column to rightmost column.

Space & Time

Time Complexity: $O(n \log n)$ in the worst-case due to sorting the column indices (n nodes in the worst-case if each node is on a unique column).

Space Complexity: $O(n)$ for storing the node information and the resultant structure.

Solution

The solution uses BFS with a queue that stores nodes along with their corresponding column index. For each visited node, add its value to a dictionary keyed by its column index. When processing children, update the column index appropriately (left child: $col-1$, right child: $col+1$). After the traversal, sort the keys of the dictionary and compile the node values in order from smallest to largest column index. This approach guarantees that nodes at the same level will be processed in left-to-right order, matching the required output.

Trees & BST

□ ★ #333 Largest BST Subtree ★

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming · Tree · DFS · BST
Lists: ★ Premium | Premium 98

Problem

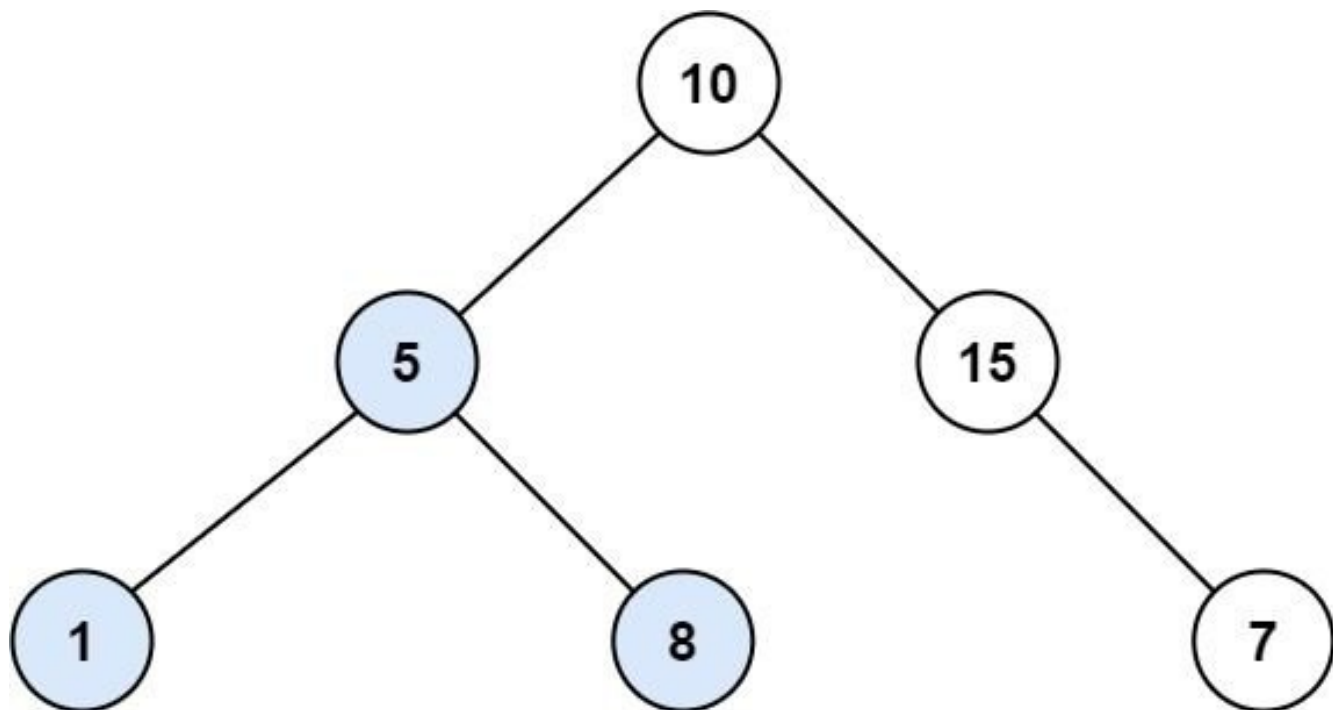
Given the root of a binary tree, find the largest subtree, which is also a Binary Search Tree (BST), where the largest means subtree has the largest number of nodes.

A Binary Search Tree (BST) is a tree in which all the nodes follow the below-mentioned properties:

- The left subtree values are less than the value of their parent (root) node's value.
- The right subtree values are greater than the value of their parent (root) node's value.

Note: A subtree must include all of its descendants.

Example 1:



Input: root = [10,5,15,1,8,null,7]

Output: 3

Explanation: The Largest BST Subtree in this case is the highlighted one. The return value is the subtree's size, which is 3.

Example 2:

Input: root = [4,2,7,2,3,5,null,2,null,null,null,null,null,1]

Output: 2

Constraints:

- The number of nodes in the tree is in the range [0, 104].
- $-104 \leq \text{Node.val} \leq 104$

Follow up: Can you figure out ways to solve it with $O(n)$ time complexity?

My LeetCode Solution

- My LeetCode Solution


```

class NodeValue:
    def __init__(self, min_node, max_node, max_size):
        self.min_node = min_node
        self.max_node = max_node
        self.max_size = max_size

class Solution:
    def largestBSTSubtree(self, root: Optional[TreeNode]) -> int:
        def dfs(node):
            if not node:
                return NodeValue(float('inf'), float('-inf'), 0)

            left = dfs(node.left)
            right = dfs(node.right)

            if left.max_node < node.val < right.min_node:
                return NodeValue(
                    min(node.val, left.min_node),
                    max(node.val, right.max_node),
                    left.max_size + right.max_size + 1
                )

            return NodeValue(float('-inf'), float('inf'), max(left.max_size, right.max_size))

        return dfs(root).max_size

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Find the largest subtree (by node count) that forms a valid BST.
# Return the node count. Right?"
#
# "[10,5,15,1,8,null,7]: subtree at 5 (nodes 1,5,8) is a BST → 3 ✓"

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with brute force to lock in the logic.
# For every node as root of a candidate subtree, validate BST property recursively
# (min/max bounds) and count nodes if valid. Track global max count.
# Revalidates overlapping subtrees – O(n^2) worst case.
# Time: O(n^2). Space: O(h) stack.
# Should I go ahead and optimize?"

```

PHASE 3 — OPTIMIZE (O(n) post-order)

"Post-order: return (is_bst, size, subtree_min, subtree_max).

Empty node: (True, 0, +inf, -inf) – sentinels pass any comparison cleanly.

Valid BST: both children are BST AND left.max < node.val < right.min."

PHASE 4 — CLEAN CODE

class _333_LargestBSTSubtree:

def largestBSTSubtree(self, root: Optional['TreeNode']) -> int:

best = [0]

def check(node):

if not node:

return True, 0, float('inf'), float('-inf')

l_ok, l_sz, l_mn, l_mx = check(node.left)

r_ok, r_sz, r_mn, r_mx = check(node.right)

if l_ok and r_ok and l_mx < node.val < r_mn:

sz = l_sz + r_sz + 1

best[0] = max(best[0], sz)

return True, sz, min(l_mn, node.val), max(r_mx, node.val)

return False, 0, 0, 0

check(root)

return best[0]

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time O(n). Space O(h). Sentinels (+inf/-inf for empty) let comparisons pass cleanly.

Given more time I'd ask: what if we want the root of the largest BST subtree? Track it alongside size."

Community Solutions (Python)

• WalkCC

```
from dataclasses import dataclass
```

```
@dataclass(frozen=True)
```

```
class T:
```

```
    mn: int # the minimum value in the subtree
```

```
    mx: int # the maximum value in the subtree
```

```
    size: int # the size of the subtree
```

```

class Solution:
    def largestBSTSubtree(self, root: TreeNode | None) -> int:
        def dfs(root: TreeNode | None) -> T:
            if not root:
                return T(math.inf, -math.inf, 0)

            l = dfs(root.left)
            r = dfs(root.right)

            if l.mx < root.val < r.mn:

                return T(min(l.mn, root.val), max(r.mx, root.val), 1 + l.size + r.size
)

            # Mark one as invalid, but still record the size of children.
            # Return (-inf, inf) because no node will be > inf or < -inf.
            return T(-math.inf, math.inf, max(l.size, r.size))

        return dfs(root).size

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def largestBSTSubtree(self, root: Optional[TreeNode]) -> int:
        def dfs(root):
            if root is None:
                return inf, -inf, 0
            lmi, lmx, ln = dfs(root.left)
            rmi, rmx, rn = dfs(root.right)
            nonlocal ans
            if lmx < root.val < rmi:
                ans = max(ans, ln + rn + 1)
                return min(lmi, root.val), max(rmx, root.val), ln + rn + 1
            return -inf, inf, 0

        ans = 0

```

```
dfs(root)
return ans
```

• SimplyLeet

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def largestBSTSubtree(self, root: TreeNode) -> int:
        # Global variable to store the size of the largest BST subtree found.

        self.max_bst_size = 0

        def helper(node):
            # Returns a tuple of: (isBST, size, min_val, max_val)
            if not node:
                # An empty node is considered a BST of size 0 with extreme min
                and max values.
                return (True, 0, float('inf'), float('-inf'))

            left_is_bst, left_size, left_min, left_max = helper(node.left)
            right_is_bst, right_size, right_min, right_max = helper(node.right)

            # Check if the current node forms a BST with its left and right
            subtrees.
            if left_is_bst and right_is_bst and node.val > left_max and node.val < right_min:
                current_size = left_size + right_size + 1
                self.max_bst_size = max(self.max_bst_size, current_size)
                current_min = min(left_min, node.val)
                current_max = max(right_max, node.val)
                return (True, current_size, current_min, current_max)
            else:
                # Not a BST: Return false and the size of the largest BST found
                so far in its subtrees.
```

```

        return (False, max(left_size, right_size), 0, 0)

    helper(root)
    return self.max_bst_size

```

• LC.ca

```

'''
>>> float("inf")
inf
>>> float("inf") + 1
inf
>>> import math
>>> float("inf") == math.inf
True
'''

# Definition for a binary tree node.

# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def largestBSTSubtree(self, root: Optional[TreeNode]) -> int:
        def dfs(root):
            if root is None:
                return inf, -inf, 0

            lmi, lmx, ln = dfs(root.left)
            rmi, rmx, rn = dfs(root.right)
            nonlocal ans
            # this if can check if left/right is a bst
            # eg. if left is not bst, the returned lmi=-inf and lmx=inf, so
            # 'lmx < root.val < rmi' is false
            if lmx < root.val < rmi:
                ans = max(ans, ln + rn + 1)
                # lmi is guranteed < root.val, except when left(right) is None
                return min(lmi, root.val), max(rmx, root.val), ln + rn + 1
            return -inf, inf, 0 # if this one not bst, then all parents will
not be bst

```

```

        ans = 0
        dfs(root)
        return ans

#####

# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, x):

# self.val = x
# self.left = None
# self.right = None

class Solution(object):
    def largestBSTSubtree(self, root):
        """
        :type root: TreeNode
        :rtype: int
        """

    def helper(root):
        if not root:
            return (0, 0, float("inf"), float("-inf")) # numBST, maxNumBST, min,
max
        lnumBST, lmaxNumBST, lmin, lmax = helper(root.left)
        rnumBST, rmaxNumBST, rmin, rmax = helper(root.right)
        numBST = -1
        if lmax < root.val < rmin and lnumBST != -1 and rnumBST != -1:
            numBST = 1 + lnumBST + rnumBST
            maxNumBST = max(1, lmaxNumBST, rmaxNumBST, numBST)

        return numBST, maxNumBST, min(lmin, rmin, root.val), max(lmax, rmax, root.val)

    return helper(root)[1]

```

◆ Quick Review

Key Insights

- Use a bottom-up DFS (post-order traversal) to process each node.
- For each node, determine whether its left and right subtrees are BSTs.
- Maintain additional data per subtree: size (number of nodes), minimum value, and maximum value.
- A subtree rooted at the current node is a BST if:
 - Its left subtree is a BST.
 - Its right subtree is a BST.
 - The maximum value in the left subtree is less than the current node's value.
 - The current node's value is less than the minimum value in the right subtree.
- Track the size of the largest BST identified during the traversal.

Space & Time

Time Complexity: $O(n)$ (each node is visited once)

Space Complexity: $O(h)$, where h is the height of the tree (due to recursion stack)

Solution

Perform a post-order traversal of the binary tree. For each node, return a tuple (or object in some languages) that provides: Whether the subtree rooted at this node is a BST. The size of the subtree if it is a BST (or the size of the largest BST found within it). The minimum value in the subtree. The maximum value in the subtree. At each node, if both left and right children form valid BSTs and the current node's value fits the BST condition (greater than left's max and less than right's min), then the subtree rooted at the node is also a BST. Calculate its size and update the current global maximum if necessary. If the condition fails, return the maximum BST size found in its subtrees.

Trees & BST

★ #366 Find Leaves of Binary Tree ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · Sorting

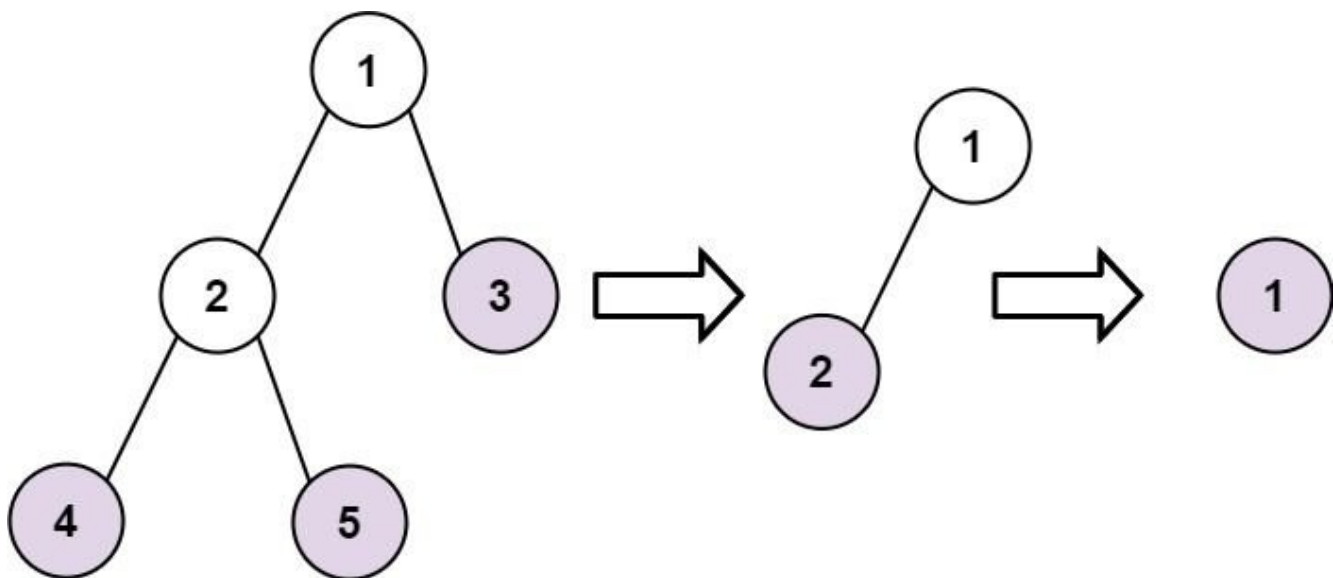
Lists: ★ Premium | Premium 98

Problem

Given the root of a binary tree, collect a tree's nodes as if you were doing this:

- Collect all the leaf nodes.
- Remove all the leaf nodes.
- Repeat until the tree is empty.

Example 1:



Input: root = [1,2,3,4,5]

Output: [[4,5,3],[2],[1]]

Explanation:

[[3,5,4],[2],[1]] and [[3,4,5],[2],[1]] are also considered correct answers since per each level it does not matter the order on which elements are returned.

Example 2:

Input: root = [1]

Output: [[1]]

Constraints:

- The number of nodes in the tree is in the range [1, 100].
- $-100 \leq \text{Node.val} \leq 100$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def findLeaves(self, root: TreeNode) -> List[List[int]]:
        res = []

        def dfs(node):
            if not node:
                return -1

            height = max(dfs(node.left), dfs(node.right)) + 1

            if height == len(res):
                res.append([])
            res[height].append(node.val)

            return height

        dfs(root)
        return res
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Collect leaves, remove them, repeat. Group nodes by their 'height from leaf' level. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Repeatedly find all current leaves (no children), remove them, append to result level.

Each round peels one tree layer – like reverse level-order.

Time: $O(n^2)$ if we rescan whole tree each round. Space: $O(n)$ output.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Post-order height: leaf=0, internal=max(left,right)+1. Group by height in one pass."

PHASE 4 — CLEAN CODE

```
class _366_FindLeaves:
    def findLeaves(self, root: Optional['TreeNode']) -> List[List[int]]:
        result: List[List[int]] = []
        def height(node) -> int:
            if not node:
                return -1
            h = max(height(node.left), height(node.right)) + 1
            if h == len(result):
                result.append([])
            result[h].append(node.val)
            return h
        height(root)
        return result
```

PHASE 5 — TEST & DEBUG

[1,2,3,4,5]: height(4)=0→result=[[4]]. height(5)=0→[[4,5]].

height(2)=1→[[4,5],[2]].

height(3)=0→[[4,5,3],[2]]. height(1)=2→[[4,5,3],[2],[1]] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$. One DFS pass – no repeated tree traversals.

Given more time I'd ask: group by depth from root? That's standard level-order BFS."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def findLeaves(self, root: TreeNode | None) -> list[list[int]]:
        ans = []

        def depth(root: TreeNode | None) -> int:
            """Returns the depth of the root (0-indexed)."""
            if not root:
                return -1

            l = depth(root.left)

            r = depth(root.right)
            h = 1 + max(l, r)

            if len(ans) == h: # Meet a leaf
                ans.append([])

            ans[h].append(root.val)
            return h

        depth(root)

        return ans

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def findLeaves(self, root: Optional[TreeNode]) -> List[List[int]]:
        def dfs(root: Optional[TreeNode]) -> int:
            if root is None:

```

```

        return 0
    l, r = dfs(root.left), dfs(root.right)
    h = max(l, r)
    if len(ans) == h:
        ans.append([])
    ans[h].append(root.val)
    return h + 1

ans = []
dfs(root)

```

```
return ans
```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val          # The value of the node.
        self.left = left        # Left child node.
        self.right = right       # Right child node.

class Solution:
    def findLeaves(self, root: TreeNode):
        result = [] # This will store lists of nodes at each removal step.

```

```

    def getHeight(node):
        # Base case: when the node is None, return 0.
        if not node:
            return 0

        # Recursively find the height of left and right subtrees.
        left_height = getHeight(node.left)
        right_height = getHeight(node.right)
        # Current node's height is max of left/right heights plus one.
        current_height = max(left_height, right_height) + 1

```

```

        # Ensure the result list has a sublist for the current height.
        if len(result) < current_height:
            result.append([])
        # Append the current node value to the corresponding height bucket.
        result[current_height - 1].append(node.val)
        return current_height

    getHeight(root) # Start recursion from the root.
    return result

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def findLeaves(self, root: TreeNode) -> List[List[int]]:
        def dfs(root, prev, t):
            if root is None:
                return
            if root.left is None and root.right is None:
                t.append(root.val)
                if prev.left == root:
                    prev.left = None
                else:
                    prev.right = None
            dfs(root.left, root, t)
            dfs(root.right, root, t)

        res = []
        prev = TreeNode(left=root)
        while prev.left:
            t = []
            dfs(prev.left, prev, t)
            res.append(t)
            prev = prev.left
        return res

```

◆ Quick Review

Key Insights

- Use a postorder traversal (bottom-up approach) to process the tree.
- Determine the "height" of each node (distance from the deepest leaf) where leaves have a height of 1.
- Group nodes by their computed height to simulate the layer-wise removal of leaves.
- The same node may become a leaf after its children are removed.

Space & Time

Time Complexity: $O(n)$ — Each node is visited exactly once.

Space Complexity: $O(n)$ — Space used for the recursion stack and the result list.

Solution

The solution uses a recursive helper function that performs a postorder traversal of the tree. For each node, it computes its height as $\max(\text{height of left}, \text{height of right}) + 1$. Nodes with the same height are collected together in a list. This idea mirrors the process of

removing leaves level by level, where leaves (height 1) are removed first, then their parents become new leaves, and so on. Data Structures used: An array or list to maintain the groups of node values by their height. Algorithmic Approach: Use Depth-First Search (DFS) via recursion. Postorder traversal ensures that leaf nodes (base cases) are processed before their parents. Append the node's value to the correct list based on the computed height.

Trees & BST

□ #437 Path Sum III

MED[Open on LeetCode](#)

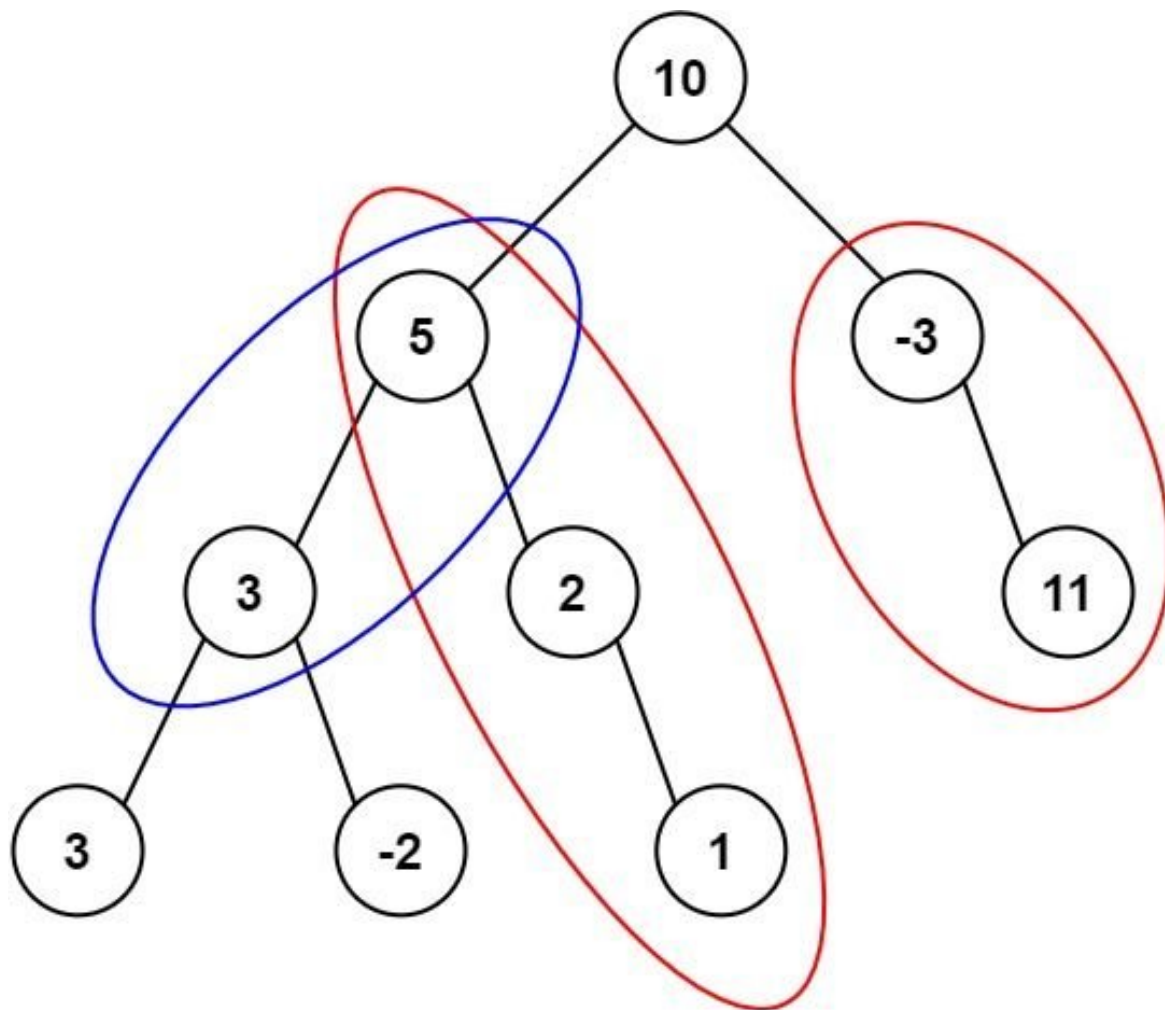
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS · Prefix Sum
Lists: Grind 169

Problem

Given the root of a binary tree and an integer targetSum, return the number of paths where the sum of the values along the path equals targetSum.

The path does not need to start or end at the root or a leaf, but it must go downwards (i.e., traveling only from parent nodes to child nodes).

Example 1:



Input: root = [10,5,-3,3,2,null,11,3,-2,null,1], targetSum = 8

Output: 3

Explanation: The paths that sum to 8 are shown.

Example 2:

Input: root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22

Output: 3

Constraints:

- The number of nodes in the tree is in the range [0, 1000].
- $-109 \leq \text{Node.val} \leq 109$

- $-1000 \leq \text{targetSum} \leq 1000$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def pathSum(self, root: Optional[TreeNode], targetSum: int) -> int:
        self.count = 0
        prefix = {0: 1}

        def dfs(node, curr_sum):
            if not node:
                return

            curr_sum += node.val

            self.count += prefix.get(curr_sum - targetSum, 0)
            prefix[curr_sum] = prefix.get(curr_sum, 0) + 1

            dfs(node.left, curr_sum)
            dfs(node.right, curr_sum)

            prefix[curr_sum] -= 1

        dfs(root, 0)
        return self.count
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Count downward paths (parent→child only) summing to targetSum.
# Path can start and end at any node. Values can be negative. Right?"
#
# "[10,5,-3,3,2,null,11,3,-2,null,1], target=8 → 3 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Prefix-sum map: as DFS walks, track cumulative sum from root.
# At each node, add count of prefix sums equal to (current_sum - target) - includes
paths through node.
# Backtrack prefix counts when leaving node.
# Time: O(n). Space: O(n) prefix map + stack.
```

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (prefix sum + hash map — same pattern as #560)

"DFS with running prefix sum. Count how many earlier prefix sums equal `current_sum - target`.

Backtrack after each subtree to avoid polluting sibling paths. Seed `{0:1}`."

PHASE 4 — CLEAN CODE

```
class _437_PathSumIII:
    def pathSum(self, root: Optional['TreeNode'], targetSum: int) -> int:
        prefix = Counter({0: 1})
        count = [0]
        def dfs(node, running):
            if not node:
                return
            running += node.val
            count[0] += prefix[running - targetSum]
            prefix[running] += 1
            dfs(node.left, running)
            dfs(node.right, running)
            prefix[running] -= 1 # backtrack
        dfs(root, 0)
        return count[0]
```

PHASE 5 — TEST & DEBUG

This is #560 applied to trees — same prefix-sum logic, but backtrack after each subtree.

`root=10, target=8`: paths `[5,3]`, `[5,2,1]`, `[-3,11]` each sum to 8 → `count=3` ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$. Identical to Subarray Sum Equals K applied to a tree.

Backtracking prevents cross-subtree contamination — critical difference from array version.

Given more time I'd ask: paths going in both directions (not just downward)?

Use the any-direction diameter-style post-order approach."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def pathSum(self, root: TreeNode | None, summ: int) -> int:
        if not root:
            return 0

        def dfs(root: TreeNode, summ: int) -> int:
            if not root:
                return 0
            return (int(summ == root.val) +
                    dfs(root.left, summ - root.val) +

                    dfs(root.right, summ - root.val))

        return (dfs(root, summ) +
                self.pathSum(root.left, summ) +
                self.pathSum(root.right, summ))

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def pathSum(self, root: Optional[TreeNode], targetSum: int) -> int:
        def dfs(node, s):
            if node is None:
                return 0
            s += node.val
            ans = cnt[s - targetSum]
            cnt[s] += 1
            ans += dfs(node.left, s)
            ans += dfs(node.right, s)
            cnt[s] -= 1
            return ans

        cnt = Counter({0: 1})

        return dfs(root, 0)

```

• SimplyLeet

```
# Python solution for Path Sum III
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val          # value of the node
        self.left = left        # left child
        self.right = right       # right child

class Solution:
    def pathSum(self, root: TreeNode, targetSum: int) -> int:
        # Helper function to perform DFS traversal

    def dfs(node, current_sum, prefix_sums):
        if not node:
            return 0

        # Update current sum by including the current node's value
        current_sum += node.val
        # Check the number of times (current_sum - targetSum) has occurred
        num_paths = prefix_sums.get(current_sum - targetSum, 0)

        # Update the prefix sum hash map with the current sum

        prefix_sums[current_sum] = prefix_sums.get(current_sum, 0) + 1

        # Recursively search in the left and right subtree
        num_paths += dfs(node.left, current_sum, prefix_sums)
        num_paths += dfs(node.right, current_sum, prefix_sums)

        # Backtrack: remove the current sum count before returning to the
        parent node
        prefix_sums[current_sum] -= 1
        return num_paths

    # Initial prefix sum hashmap with 0 sum seen once (empty path)
    prefix_sums = {0: 1}
    return dfs(root, 0, prefix_sums)
```

• LC.ca


```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def pathSum(self, root: Optional[TreeNode], targetSum: int) -> int:
        def dfs(node, s):
            if node is None:
                return 0
            s += node.val
            ans = cnt[s - targetSum]
            cnt[s] += 1
            ans += dfs(node.left, s)
            ans += dfs(node.right, s)
            cnt[s] -= 1
            return ans

        cnt = Counter({0: 1})

        return dfs(root, 0)

```

◆ Quick Review

Key Insights

- A path can start and end at any nodes, as long as it goes downwards.
- Using DFS helps traverse the tree and explore all possible paths.
- A prefix sum technique with a hash map allows quick calculation of the sum of any subpath.

- Backtracking is used to remove a path's prefix sum when moving back up the recursion tree.
- Although a brute-force approach exists, it is inefficient compared to the prefix sum method.

Space & Time

Time Complexity: $O(n)$ on average (where n is the number of nodes), though worst-case can be $O(n^2)$ in a skewed tree.

Space Complexity: $O(n)$ due to recursion stack and the hash map that stores prefix sums.

Solution

The solution employs a DFS traversal of the tree along with a hash map that records the frequency of prefix sums encountered so far. At each node, the current prefix sum is updated by adding the node's value. We then check the hash map for how many times (current prefix sum – targetSum) has occurred, since this value indicates the existence of a subpath summing to targetSum. We update the hash map with the current prefix sum, continue the DFS to traverse child nodes, and then backtrack by decrementing the current

prefix sum's count before retreating to the parent node. This ensures that sums from other branches are not incorrectly combined.

Trees & BST

□ ★ #545 Boundary of Binary Tree ★

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS

Lists: ★ Premium | Premium 98

Problem

The boundary of a binary tree is the concatenation of the root, the left boundary, the leaves ordered from left-to-right, and the reverse order of the right boundary.

The left boundary is the set of nodes defined by the following:

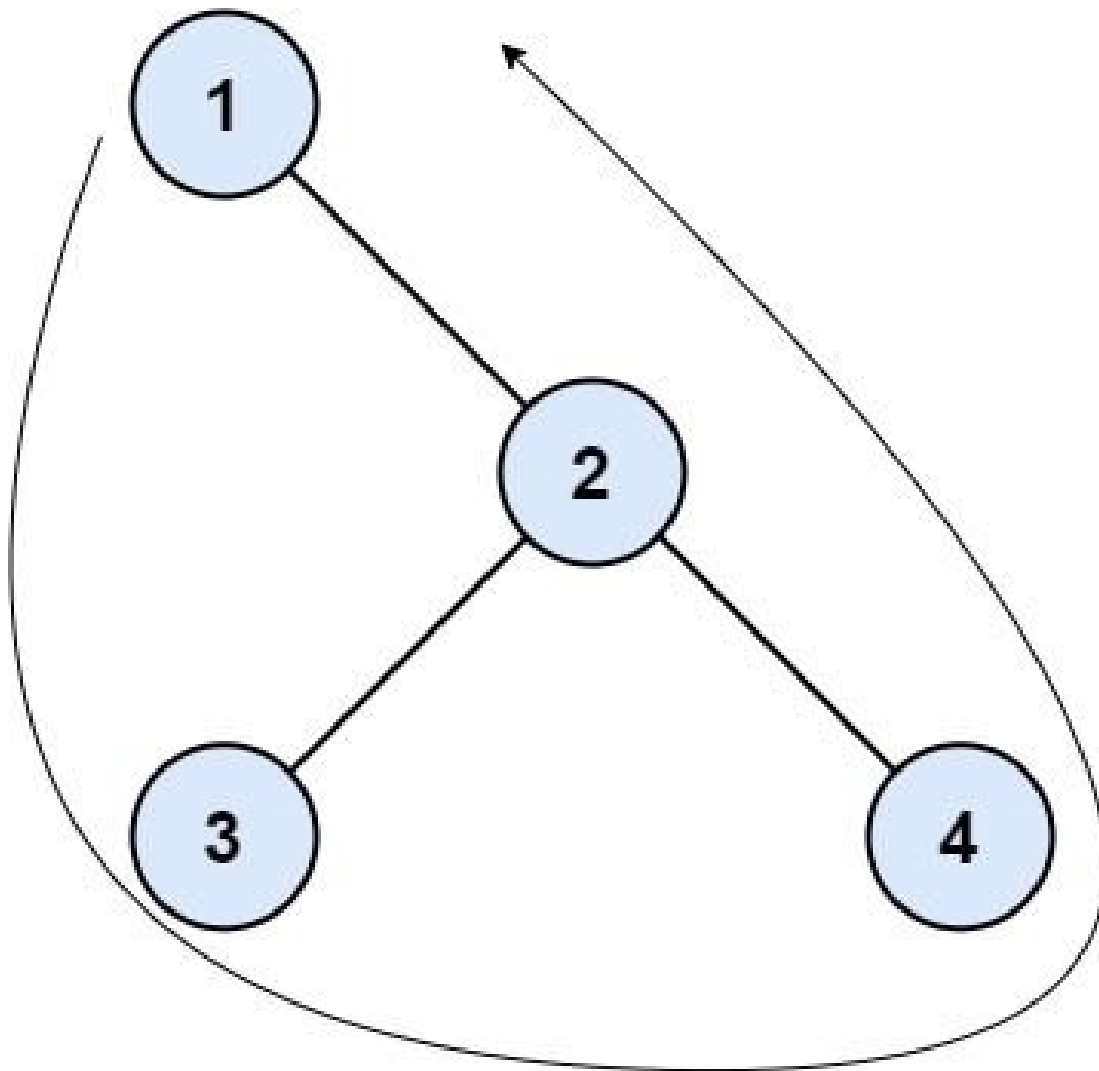
- The root node's left child is in the left boundary. If the root does not have a left child, then the left boundary is empty.
- If a node is in the left boundary and has a left child, then the left child is in the left boundary.
- If a node is in the left boundary, has no left child, but has a right child, then the right child is in the left boundary.

- The leftmost leaf is not in the left boundary.

The right boundary is similar to the left boundary, except it is the right side of the root's right subtree. Again, the leaf is not part of the right boundary, and the right boundary is empty if the root does not have a right child.

The leaves are nodes that do not have any children. For this problem, the root is not a leaf. Given the root of a binary tree, return the values of its boundary.

Example 1:



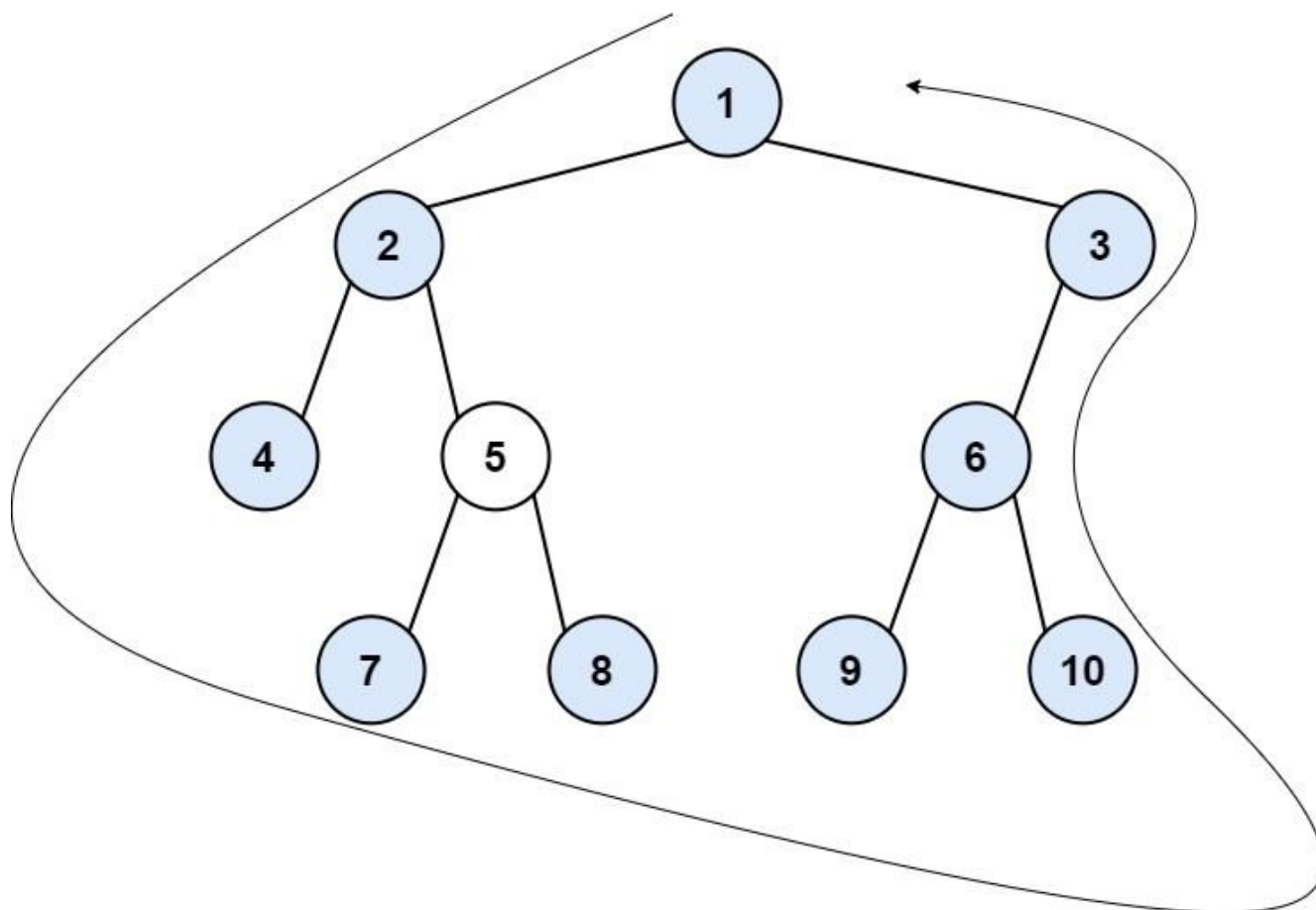
Input: root = [1,null,2,3,4]

Output: [1,3,4,2]

Explanation:

- The left boundary is empty because the root does not have a left child.
 - The right boundary follows the path starting from the root's right child 2 -> 4. 4 is a leaf, so the right boundary is [2].
 - The leaves from left to right are [3,4].
- Concatenating everything results in [1] + [] + [3,4] + [2] = [1,3,4,2].

Example 2:



Input: root = [1,2,3,4,5,6,null,null,null,7,8,9,10]

Output: [1,2,4,7,8,9,10,6,3]

Explanation:

- The left boundary follows the path starting from the root's left child 2 -> 4.

4 is a leaf, so the left boundary is [2].

- The right boundary follows the path starting from the root's right child 3 -> 6 -> 10.

10 is a leaf, so the right boundary is [3,6], and in reverse order is [6,3].

- The leaves from left to right are [4,7,8,9,10].

Constraints:

- The number of nodes in the tree is in the range [1, 10⁴].
- $-1000 \leq \text{Node.val} \leq 1000$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def boundaryOfBinaryTree(self, root: Optional[TreeNode]) -> List[int]:
        if not root:
            return []

        res = [root.val]

        def left_boundary(node):
            if not node or (not node.left and not node.right):
                return

            res.append(node.val)
            if node.left:
                left_boundary(node.left)
            else:
                left_boundary(node.right)

        def right_boundary(node):
            if not node or (not node.left and not node.right):
                return
            if node.right:
                right_boundary(node.right)
            else:
                right_boundary(node.left)
            res.append(node.val)

        def leaves(node):
            if not node:
                return
            if not node.left and not node.right and node != root:
                res.append(node.val)

        left_boundary(root.left)
        leaves(root)
        right_boundary(root.right)
        return res
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Boundary = root + left boundary (non-leaves top-down) + all leaves left-to-right
+ right boundary (non-leaves bottom-up). Leftmost/rightmost leaves are in leaves,
not boundary. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
Collect four pieces separately: root val, left boundary nodes top-down,
all leaves left-to-right, right boundary nodes bottom-up.
Merge while skipping duplicate leaves already counted.
Time: $O(n)$. Space: $O(h)$ recursion for leaf collection.
Should I go ahead and optimize?"

```
class _545_BoundaryBinaryTree:
    def boundaryOfBinaryTree(self, root: Optional['TreeNode']) -> List[int]:
        if not root:
            return []

        def is_leaf(n): return not n.left and not n.right
        def left_boundary(n):
            res = []
            while n and not is_leaf(n):
                res.append(n.val)
                n = n.left if n.left else n.right
            return res

        def leaves(n):
            if not n: return []
            if is_leaf(n): return [n.val]
            return leaves(n.left) + leaves(n.right)

        def right_boundary(n):
            res = []
            while n and not is_leaf(n):
                res.append(n.val)
                n = n.right if n.right else n.left
            return res[::-1]

        if is_leaf(root):
            return [root.val]
        return [root.val] + left_boundary(root.left) + leaves(root) +
            right_boundary(root.right)
```

PHASE 5 — TEST & DEBUG

[1,null,2,3,4]: left=[]. leaves=[3,4]. right=[2]. → [1,3,4,2] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(h)$. Three separate traversals, each linear.
'Prefer left else right' for left spine; 'prefer right else left' for right spine
– reversed at end.

Given more time I'd ask: return boundary clockwise vs counter-clockwise? Just reverse the result."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def boundaryOfBinaryTree(self, root: TreeNode | None) -> list[int]:
        if not root:
            return []

        ans = [root.val]

        def dfs(root: TreeNode | None, lb: bool, rb: bool):
            """
            1. root.left is left boundary if root is left boundary.

            root.right is left boundary if root.left is None.
            2. Same applies for right boundary.
            3. If root is left boundary, add it before 2 children - preorder.
            If root is right boundary, add it after 2 children - postorder.
            4. A leaf that is neither left/right boundary belongs to the bottom.
            """
            if not root:
                return
            if lb:
                ans.append(root.val)

            if not lb and not rb and not root.left and not root.right:
                ans.append(root.val)

            dfs(root.left, lb, rb and not root.right)
            dfs(root.right, lb and not root.left, rb)
            if rb:
                ans.append(root.val)

            dfs(root.left, True, False)
            dfs(root.right, False, True)

        return ans
```

• LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def boundaryOfBinaryTree(self, root: Optional[TreeNode]) -> List[int]:
        def dfs(nums: List[int], root: Optional[TreeNode], i: int):
            if root is None:
                return
            if i == 0:
                if root.left != root.right:
                    nums.append(root.val)
                    if root.left:
                        dfs(nums, root.left, i)
                    else:
                        dfs(nums, root.right, i)
            elif i == 1:
                if root.left == root.right:
                    nums.append(root.val)
                else:
                    dfs(nums, root.left, i)
                    dfs(nums, root.right, i)
            else:
                if root.left != root.right:
                    nums.append(root.val)
                    if root.right:
                        dfs(nums, root.right, i)
                    else:
                        dfs(nums, root.left, i)

        ans = [root.val]
        if root.left == root.right:
            return ans
        left, leaves, right = [], [], []
        dfs(left, root.left, 0)
        dfs(leaves, root, 1)
        dfs(right, root.right, 2)
        ans += left + leaves + right[::-1]

```

```
return ans
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def boundaryOfBinaryTree(self, root: TreeNode) -> list:
        if not root:
            return []

        # Check if a node is a leaf.
        def isLeaf(node):
            return node and not node.left and not node.right

        # Add left boundary nodes (excluding leaves).
        def addLeftBoundary(node, boundary):
            while node:
                if not isLeaf(node):
                    boundary.append(node.val)
                    # Prefer left child, if not present then right child.
                    if node.left:
                        node = node.left
                    else:
                        node = node.right

        # Add leaf nodes using DFS.
        def addLeaves(node, leaves):
            if not node:
```

```

        return
    if isLeaf(node):
        leaves.append(node.val)
    else:
        addLeaves(node.left, leaves)
        addLeaves(node.right, leaves)

# Add right boundary nodes in reverse order (excluding leaves).
def addRightBoundary(node, boundary):
    temp = []

    while node:
        if not isLeaf(node):
            temp.append(node.val)
        if node.right:
            node = node.right
        else:
            node = node.left
    # Append in reverse order.
    while temp:
        boundary.append(temp.pop())

boundary = []
# Add root value if it is not a leaf.
if not isLeaf(root):
    boundary.append(root.val)

addLeftBoundary(root.left, boundary)
addLeaves(root, boundary)
addRightBoundary(root.right, boundary)

return boundary

# Example usage:
# Construct a binary tree and call boundaryOfBinaryTree(root)

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def boundaryOfBinaryTree(self, root: TreeNode) -> List[int]:
        self.res = []
        if not root:
            return self.res
        # root
        if not self.is_leaf(root):
            self.res.append(root.val)

        # left boundary
        t = root.left
        while t:
            if not self.is_leaf(t):
                self.res.append(t.val)

            t = t.left if t.left else t.right

        # leaves
        self.add_leaves(root)

        # right boundary(reverse order)
        s = []
        t = root.right
        while t:
            if not self.is_leaf(t):
                s.append(t.val)
            t = t.right if t.right else t.left
        while s:
            self.res.append(s.pop())

        # output
        return self.res

    def add_leaves(self, root):
        if self.is_leaf(root):

```

```
        self.res.append(root.val)
        return
    if root.left:
        self.add_leaves(root.left)
    if root.right:
        self.add_leaves(root.right)

    def is_leaf(self, node) -> bool:
        return node and node.left is None and node.right is None
```

◆ Quick Review

Key Insights

- Break down the problem into three parts:
- Left boundary (excluding leaves).
- All leaves (do a DFS to find leaves).
- Right boundary (excluding leaves, then reverse the collected list).
- Special handling is needed when the tree doesn't have a left or right subtree.
- The root is always included and is never considered as a leaf, even if it has no children.
- Avoid duplication of leaf nodes in the left/right boundaries.

Space & Time

Time Complexity: $O(n)$ – Every node is visited at most once.

Space Complexity: $O(n)$ – $O(n)$ space is used for the output list and recursion stack (in worst case tree is skewed).

Solution

The solution uses a Depth-First Search (DFS) strategy to traverse the tree and collect the boundary nodes in three steps: For the left boundary, start at `root.left` and traverse down, adding nodes (skipping leaves). For the leaves, perform a DFS from the root and add any node that has no children. For the right boundary, start at `root.right` and traverse down, adding nodes (skipping leaves). Since the right boundary must be added in reverse order, we reverse the collected list after traversal. Finally, concatenate these lists in the order: root, left boundary, leaves, reversed right boundary. The approach makes use of helper functions to clearly separate the logic for gathering left boundary, leaves, and right boundary. This modular approach simplifies reasoning and avoids edge case mistakes such as including duplicate leaf nodes.

Trees & BST

□ ★ #549 Binary Tree Longest Consecutive Sequence II ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS

Lists: ★ Premium | Premium 98

Problem

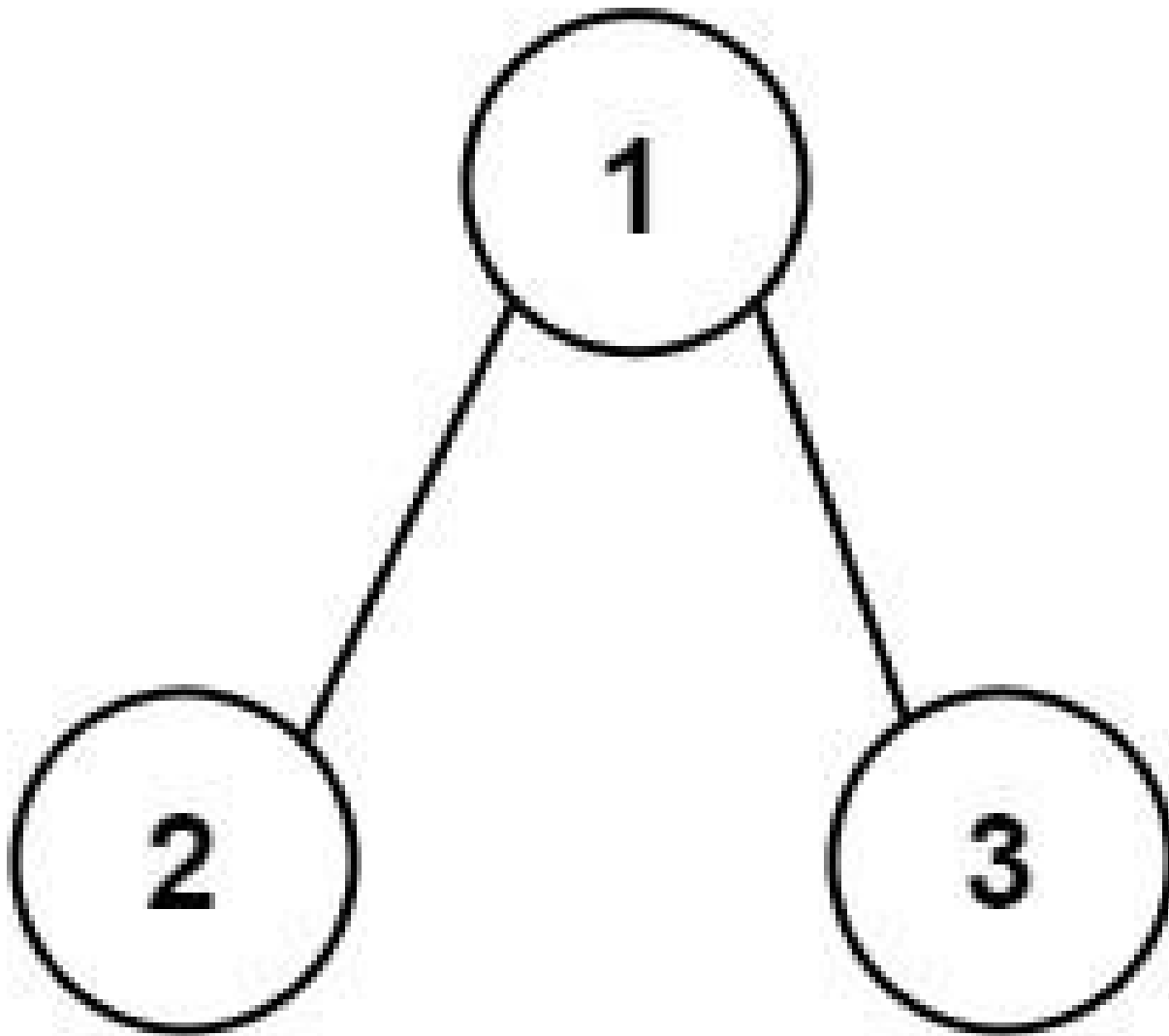
Given the root of a binary tree, return the length of the longest consecutive path in the tree.

A consecutive path is a path where the values of the consecutive nodes in the path differ by one. This path can be either increasing or decreasing.

- For example, [1,2,3,4] and [4,3,2,1] are both considered valid, but the path [1,2,4,3] is not valid.

On the other hand, the path can be in the child–Parent–child order, where not necessarily be parent–child order.

Example 1:

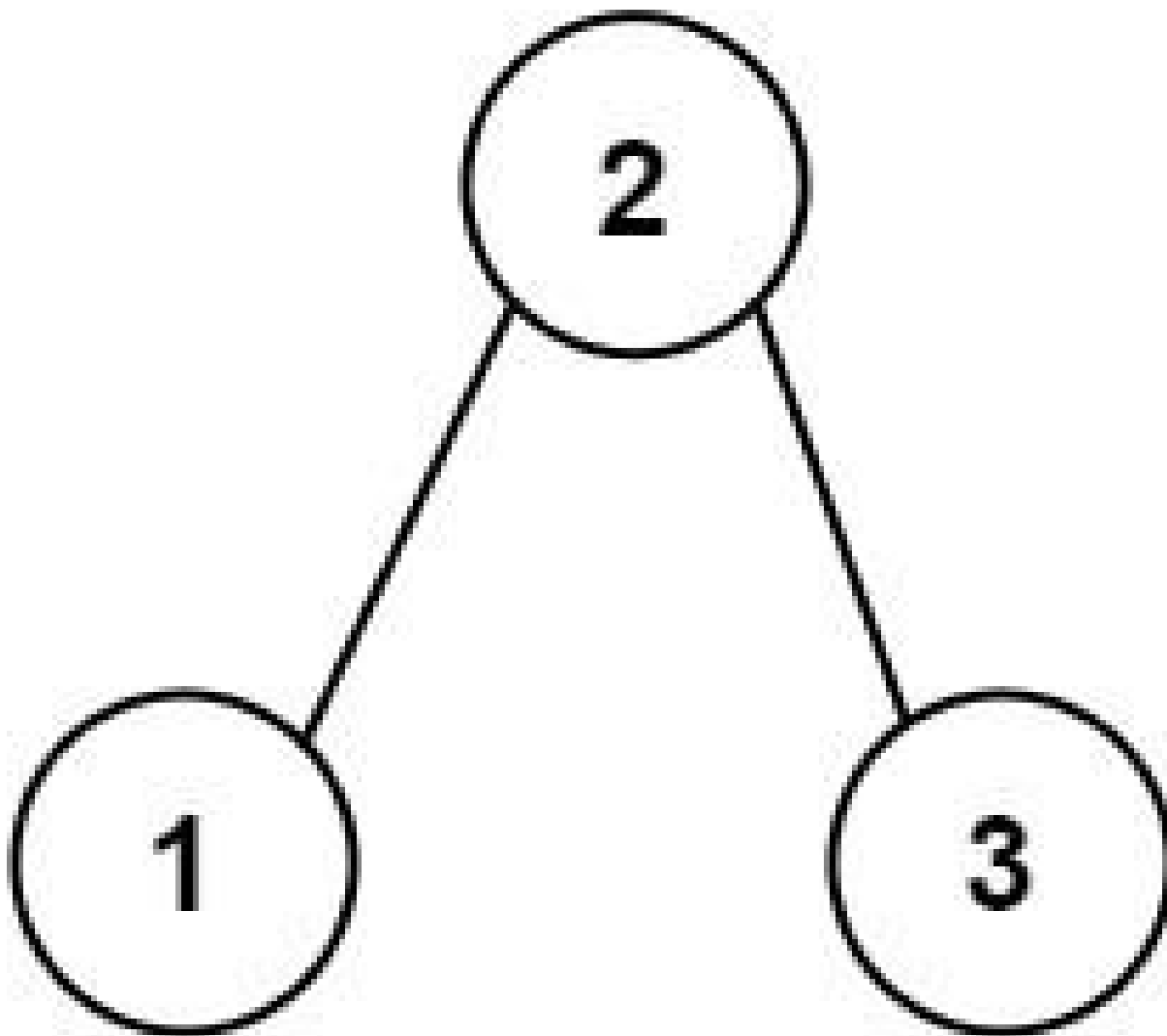


Input: root = [1,2,3]

Output: 2

Explanation: The longest consecutive path is [1, 2] or [2, 1].

Example 2:



Input: root = [2,1,3]

Output: 3

Explanation: The longest consecutive path is [1, 2, 3] or [3, 2, 1].

Constraints:

- The number of nodes in the tree is in the range [1, 3 * 10⁴].
- $-3 * 10^4 \leq \text{Node.val} \leq 3 * 10^4$

My LeetCode Solution

- My LeetCode Solution

```

class Solution:
    def longestConsecutive(self, root: Optional[TreeNode]) -> int:

        def fn(node):
            """Return longest increasing & decreasing sequence starting from
            node."""
            if not node: return 0, 0
            li, ld = fn(node.left)
            ri, rd = fn(node.right)
            inc = dec = 1
            if node.left:
                if node.left.val + 1 == node.val: inc = 1 + li
                elif node.left.val - 1 == node.val: dec = 1 + ld
            if node.right:
                if node.right.val + 1 == node.val: inc = max(inc, 1 + ri)
                elif node.right.val - 1 == node.val: dec = max(dec, 1 + rd)
            self.ans = max(self.ans, inc + dec - 1)
            return inc, dec

        self.ans = 0
        fn(root)

        return self.ans

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Longest path where consecutive values differ by 1 – can go up OR down, can change direction at one pivot. Right?"

#

"[2,1,3] → [1,2,3] length 3 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

At each node, run DFS tracking consecutive count when `child.val == parent.val + 1`.

Reset count when streak breaks. Track global max streak length.

Visits every node once with $O(n)$ DFS per starting node worst case $O(n^2)$.

Time: $O(n^2)$ naive. Space: $O(h)$ stack.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (post-order returning inc/dec lengths)

"Each node returns (inc, dec): length of increasing/decreasing sequence going DOWN from it."

Diameter through node = best combination of one child's inc and another's dec, meeting at this node."

PHASE 4 — CLEAN CODE

```
class _549_LongestConsecutiveII:
    def longestConsecutive(self, root: Optional['TreeNode']) -> int:
        best = [0]
        def dfs(node):
            if not node:
                return 0, 0
            l_inc, l_dec = dfs(node.left)
            r_inc, r_dec = dfs(node.right)
            inc = dec = 1
            if node.left:
                if node.left.val == node.val + 1: inc = max(inc, l_inc + 1)
                if node.left.val == node.val - 1: dec = max(dec, l_dec + 1)
            if node.right:
                if node.right.val == node.val + 1: inc = max(inc, r_inc + 1)
                if node.right.val == node.val - 1: dec = max(dec, r_dec + 1)
            best[0] = max(best[0], inc + dec - 1)
            return inc, dec
        dfs(root)
        return best[0]
```

PHASE 5 — TEST & DEBUG

[2,1,3]: dfs(1)=(1,1). dfs(3)=(1,1). dfs(2): left(1)=val-1→dec=2.
right(3)=val+1→inc=2. best=2+2-1=3 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time O(n). Space O(h). inc+dec-1 avoids double-counting the pivot node.
Generalises #298 by allowing direction change at the pivot node.
Given more time I'd ask: allow path to go up? Needs full graph treatment."

Community Solutions (Python)

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def longestConsecutive(self, root: TreeNode) -> int:
        def dfs(root):
            if root is None:
```

```

        return [0, 0]
    nonlocal ans
    incr = decr = 1
    i1, d1 = dfs(root.left)
    i2, d2 = dfs(root.right)
    if root.left:
        if root.left.val + 1 == root.val:
            incr = i1 + 1
        if root.left.val - 1 == root.val:
            decr = d1 + 1

    if root.right:
        if root.right.val + 1 == root.val:
            incr = max(incr, i2 + 1)
        if root.right.val - 1 == root.val:
            decr = max(decr, d2 + 1)
    ans = max(ans, incr + decr - 1)
    return [incr, decr]

ans = 0
dfs(root)

return ans

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def longestConsecutive(self, root: TreeNode) -> int:
        # Initialize global maximum path length variable

```



```

self.max_length = 0

def dfs(node):
    if not node:
        return (0, 0) # (increasing, decreasing)

    inc = dec = 1 # Each node counts as length 1

    # Process left child
    if node.left:

```

```

        left_inc, left_dec = dfs(node.left)
        # If left child's value is consecutive increasing with respect
to node
        if node.left.val == node.val + 1:
            inc = max(inc, left_inc + 1)
        # If left child's value is consecutive decreasing with respect
to node
        elif node.left.val == node.val - 1:
            dec = max(dec, left_dec + 1)

    # Process right child
    if node.right:

```

```

        right_inc, right_dec = dfs(node.right)
        # If right child's value is consecutive increasing with respect
to node
        if node.right.val == node.val + 1:
            inc = max(inc, right_inc + 1)
        # If right child's value is consecutive decreasing with respect
to node
        elif node.right.val == node.val - 1:
            dec = max(dec, right_dec + 1)

    # Update the global maximum path by combining increasing and
decreasing paths
    self.max_length = max(self.max_length, inc + dec - 1)

```

```

    return (inc, dec)

dfs(root)
return self.max_length

```

• LC.ca

```
# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

'''
>>> float('-inf')+1
-inf
```

```
>>> float('-inf')-1
-inf
'''

class Solution(object):
    def longestConsecutive(self, root):
        '''
        :type root: TreeNode
        :rtype: int
        '''
```

```
def dfs(root):
    if not root:
        # val, increasing length, decreasing length, max length
        return float('-inf'), 0, 0, 0
    # inc/dec starting from root
    inc = dec = 1
    left, leftInc, leftDec, leftMax = dfs(root.left)
    right, rightInc, rightDec, rightMax = dfs(root.right)
    if root.val + 1 == left:
        inc = max(leftInc + 1, inc)
```

```

        if root.val - 1 == left:
            dec = max(leftDec + 1, dec)
        if root.val + 1 == right:
            inc = max(rightInc + 1, inc)
        if root.val - 1 == right:
            dec = max(rightDec + 1, dec)
        # note: max may occur in children subtree, not current root's tree
        return root.val, inc, dec, max(inc + dec - 1, leftMax, rightMax, i
nc, dec)

        # -1 because root is counted twice, even left/right both null

```

```

return dfs(root)[3]

```

```

#####

```

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

```

```

class Solution:
    def longestConsecutive(self, root: TreeNode) -> int:
        def dfs(root):
            if root is None:
                return [0, 0]
            nonlocal ans
            incr = decr = 1
            i1, d1 = dfs(root.left)
            i2, d2 = dfs(root.right)
            if root.left:

```

```

        if root.left.val + 1 == root.val:
            incr = i1 + 1
        if root.left.val - 1 == root.val:
            decr = d1 + 1
    if root.right:
        if root.right.val + 1 == root.val:
            incr = max(incr, i2 + 1) # incr1 and incr2 must be in
different tree branches, eg. left child cannot be both increasing and
decreasing. Same as decr1 and decr2
        if root.right.val - 1 == root.val:
            decr = max(decr, d2 + 1)
    ans = max(ans, incr + decr - 1) # -1 because root is counted twice,
even left/right both null

```

```

    return [incr, decr]

```

```

ans = 0
dfs(root)
return ans

```

```

#####

```

```

class Solution: # issue with this solution, duplicated dfs() search
    def longestConsecutive(self, root: TreeNode) -> int:

```

```

    def helper(node: TreeNode, diff: int) -> int:
        if not node:
            return 0

        left = right = 0
        if node.left and node.val - node.left.val == diff:
            left = 1 + helper(node.left, diff)
        if node.right and node.val - node.right.val == diff:
            right = 1 + helper(node.right, diff)

```

```
        return max(left, right)
    if not root:
        return 0

    res = helper(root, 1) + helper(root, -1) + 1 # +1 is increasing, -1 is
decreasing
    return max(res, max(longestConsecutive(root.left), longestConsecutive(
root.right)))
```

◆ Quick Review

Key Insights

- Use Depth-First Search (DFS) to visit every node.
- For each node, compute:
- The longest increasing consecutive path starting from that node.
- The longest decreasing consecutive path starting from that node.
- Combine the increasing and decreasing paths (subtracting one to avoid double counting the current node) to potentially form a longer consecutive sequence through the node.
- Update a global maximum to record the longest path seen.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes since every node is visited once.

Space Complexity: $O(h)$, where h is the height of the tree, due to the recursion stack.

Solution

The solution employs a DFS traversal. At every node, we calculate two values: `inc` – the length of the longest increasing consecutive sequence starting from that node. `dec` – the length of the longest decreasing consecutive sequence starting from that node. For each child of the current node: If the child's value is `node.val + 1`, update the increasing sequence. If the child's value is `node.val - 1`, update the decreasing sequence. Then, the longest path through the current node is `inc + dec - 1` (subtracting one to avoid counting the current node twice). A global variable maintains the maximum length encountered. This process works regardless of the path direction because it combines both increasing and decreasing parts.

Trees & BST

□ ★ #582 Kill Process ★

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Tree · DFS · BFS
Lists: ★ Premium | Premium 98

Problem

You have n processes forming a rooted tree structure. You are given two integer arrays `pid` and `ppid`, where `pid[i]` is the ID of the i th process and `ppid[i]` is the ID of the i th process's parent process.

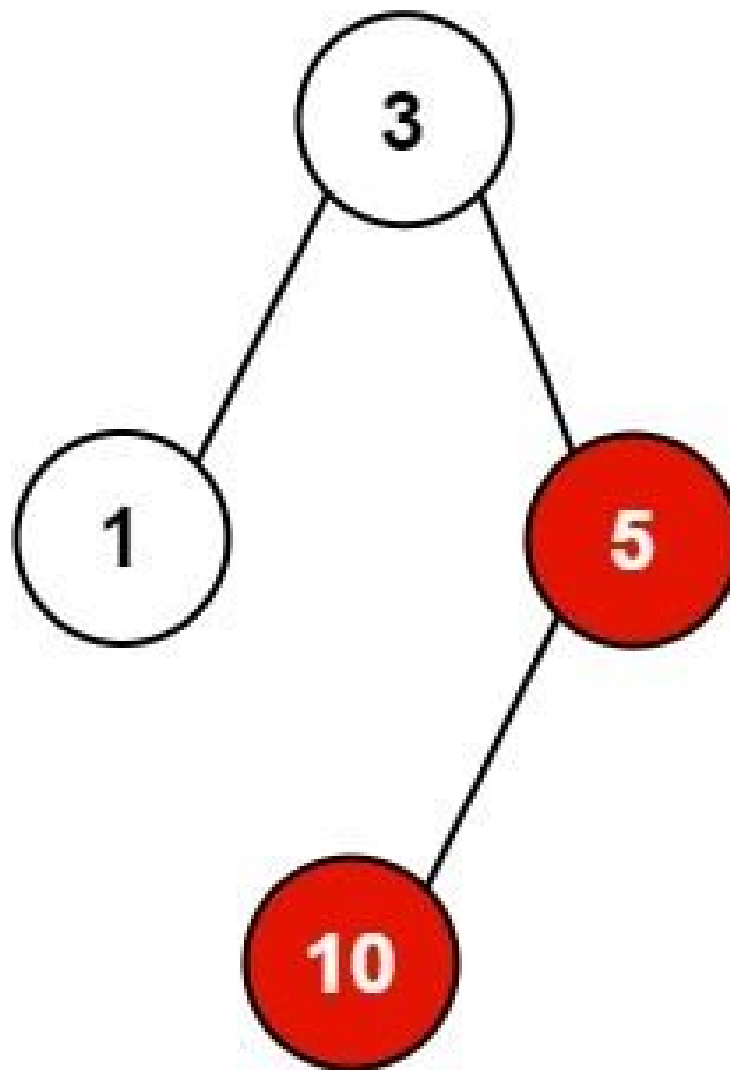
Each process has only one parent process but may have multiple children processes. Only one process has `ppid[i] = 0`, which means this process has no parent process (the root of the tree).

When a process is killed, all of its children processes will also be killed.

Given an integer `kill` representing the ID of a process you want to kill, return a list of the IDs of the processes that will be killed. You may

return the answer in any order.

Example 1:



Input: pid = [1,3,10,5], ppid = [3,0,5,3], kill = 5

Output: [5,10]

Explanation: The processes colored in red are the processes that should be killed.

Example 2:

Input: pid = [1], ppid = [0], kill = 1

Output: [1]

Constraints:

- $n == \text{pid.length}$

- $n == \text{ppid.length}$
- $1 \leq n \leq 5 * 10^4$
- $1 \leq \text{pid}[i] \leq 5 * 10^4$
- $0 \leq \text{ppid}[i] \leq 5 * 10^4$
- Only one process has no parent.
- All the values of pid are unique.
- kill is guaranteed to be in pid.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def killProcess(self, pid: List[int], ppid: List[int], kill: int) -> List[int]:
        # Build parent -> children mapping
        children = collections.defaultdict(list)
        for parent, child in zip(ppid, pid):
            children[parent].append(child)

        # BFS from kill node
        queue = collections.deque([kill])
        killed = []

        while queue:
            process = queue.popleft()
            killed.append(process)

            # Add all children to queue
            if process in children:
                queue.extend(children[process])

        return killed
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Kill a process and all its descendants. Return all killed process IDs. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Build children map and indegree from the kill-pair list.

BFS from the killed pid: enqueue all its direct children, then propagate kills downward.

Every reachable process dies – simulates the cascade.

Time: $O(n)$. Space: $O(n)$ for graph + queue.

Should I go ahead and optimize?"

```
class _582_KillProcess:
```

```
    def killProcess(self, pid: List[int], ppid: List[int], kill: int) -> List[int]:
```

```
        children: defaultdict = defaultdict(list)
```

```
        for child, parent in zip(pid, ppid):
```

```
            children[parent].append(child)
```

```
        killed = []
```

```
        queue = deque([kill])
```

```
        while queue:
```

```
            proc = queue.popleft()
```

```
            killed.append(proc)
```

```
            queue.extend(children[proc])
```

```
        return killed
```

PHASE 5 — TEST & DEBUG

pid=[1,3,10,5], ppid=[3,0,5,3], kill=5: children={3:[1,5],5:[10]}.

BFS from 5: [5,10] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$. Build adjacency list then BFS – standard implicit tree traversal.

Given more time I'd ask: kill all ancestors too? Walk parent pointers upward instead."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def killProcess(
        self,
        pid: List[int],
        ppid: List[int],
        kill: int,
    ) -> List[int]:
        ans = []
        tree = collections.defaultdict(list)

        for v, u in zip(pid, ppid):
            if u == 0:
                continue
            tree[u].append(v)

        def dfs(u: int) -> None:
            ans.append(u)
            for v in tree.get(u, []):
                dfs(v)

        dfs(kill)
        return ans

```

• LeetDoocs

```

class Solution:
    def killProcess(self, pid: List[int], ppid: List[int], kill: int) -> List[int]:
        def dfs(i: int):
            ans.append(i)
            for j in g[i]:
                dfs(j)

        g = defaultdict(list)
        for i, p in zip(pid, ppid):
            g[p].append(i)

        ans = []
        dfs(kill)
        return ans

```

• SimplyLeet

```
# Python solution using BFS

from collections import deque

def killProcess(pid, ppid, kill):
    # Build a mapping from parent process id to list of child process ids
    tree = {}
    for child, parent in zip(pid, ppid):
        if parent not in tree:
            tree[parent] = []

    tree[parent].append(child)

    # Use a deque for BFS traversal
    result = []
    queue = deque([kill])

    while queue:
        current = queue.popleft() # Get process from the left of the queue
        result.append(current)    # Add the current process to the result
        if current in tree:
            # Enqueue all children of the current process
            for child in tree[current]:
                queue.append(child)

    return result

# Example usage:
# pid = [1, 3, 10, 5]
# ppid = [3, 0, 5, 3]
# kill = 5

# Expected output: [5, 10]
```

• LC.ca

```
class Solution:
    def killProcess(self, pid: List[int], ppid: List[int], kill: int) -> List[int]:
        def dfs(i: int):
            ans.append(i)
            for j in g[i]:
                dfs(j)

        g = defaultdict(list)
        for i, p in zip(pid, ppid):
            g[p].append(i)

        ans = []
        dfs(kill)
        return ans
```

◆ Quick Review

Key Insights

- Build a mapping (dictionary/hash table) from a parent process ID to its list of child process IDs.
- Use Depth-First Search (DFS) or Breadth-First Search (BFS) starting from the kill process to traverse the tree and collect all descendant processes.
- The problem requires handling a tree structure that may have multiple children per node.

Space & Time

Time Complexity: $O(n)$, where n is the number of processes (both building the mapping and the tree traversal take linear time).

Space Complexity: $O(n)$, for storing the mapping and the traversal queue/stack.

Solution

We first construct a mapping from each process's parent ID to its list of child IDs. This allows us to quickly access all children of any process. Once the mapping is built, we perform a tree traversal (either BFS or DFS) starting from the process to kill. During the traversal, we record every process encountered. This collected list represents all processes that will be terminated. The data structures used are a dictionary (for the mapping) and either a stack (for DFS) or a queue (for BFS), making the solution efficient and straightforward.

Trees & BST

□ #662 Maximum Width of Binary Tree

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · BFS

Lists: Grind 169

Problem

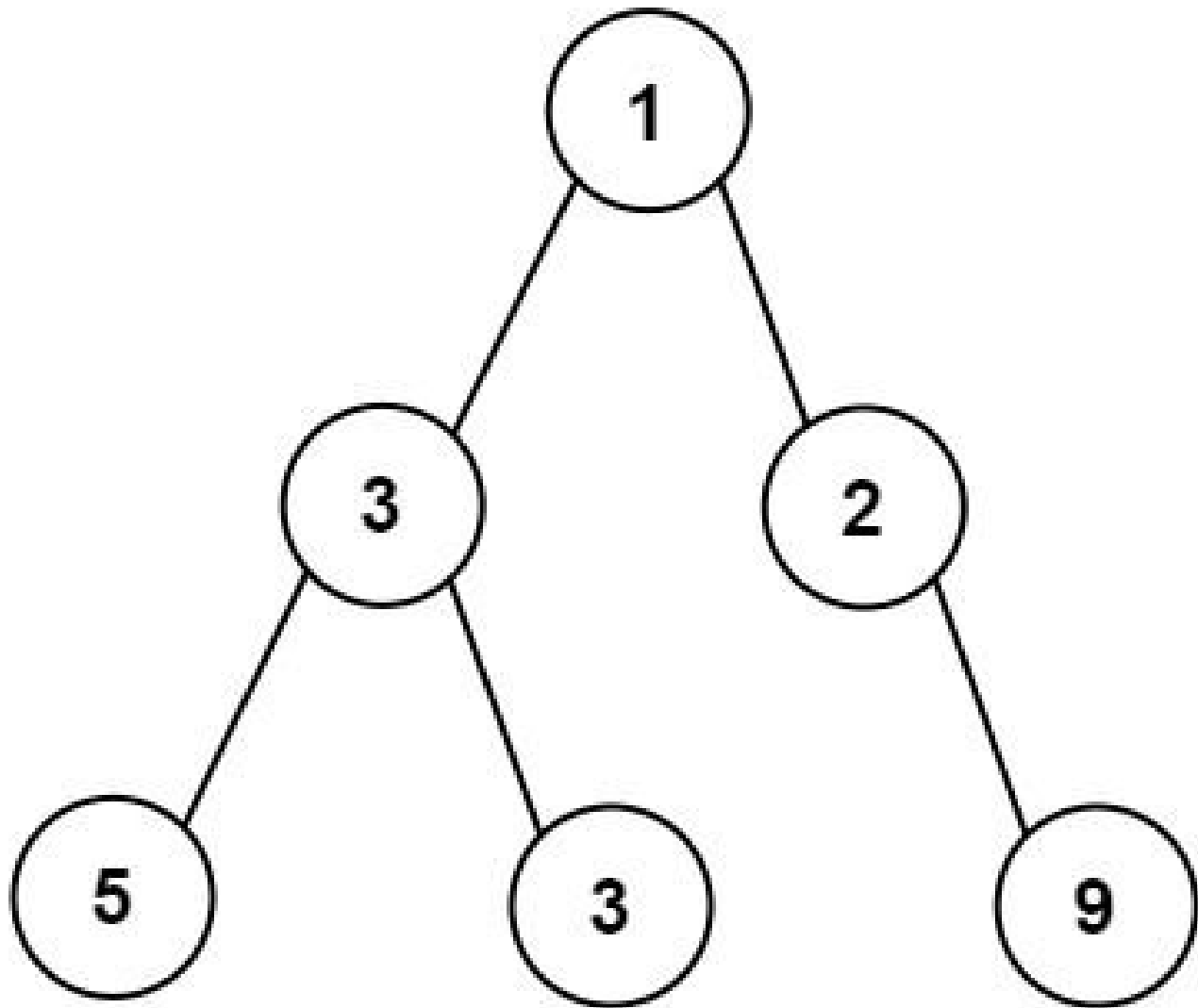
Given the root of a binary tree, return the maximum width of the given tree.

The maximum width of a tree is the maximum width among all levels.

The width of one level is defined as the length between the end-nodes (the leftmost and rightmost non-null nodes), where the null nodes between the end-nodes that would be present in a complete binary tree extending down to that level are also counted into the length calculation.

It is guaranteed that the answer will in the range of a 32-bit signed integer.

Example 1:

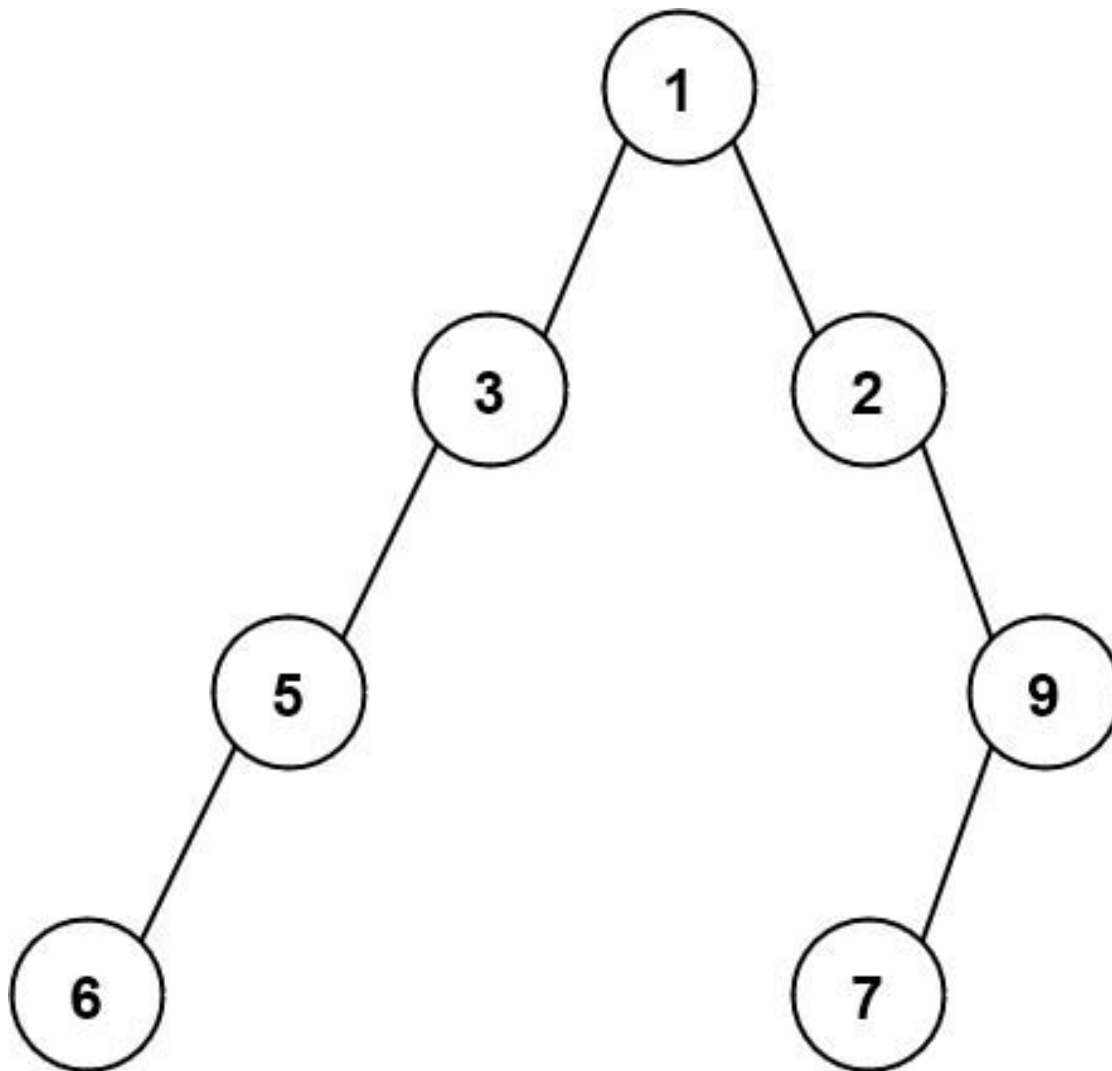


Input: root = [1,3,2,5,3,null,9]

Output: 4

Explanation: The maximum width exists in the third level with length 4 (5,3,null,9).

Example 2:

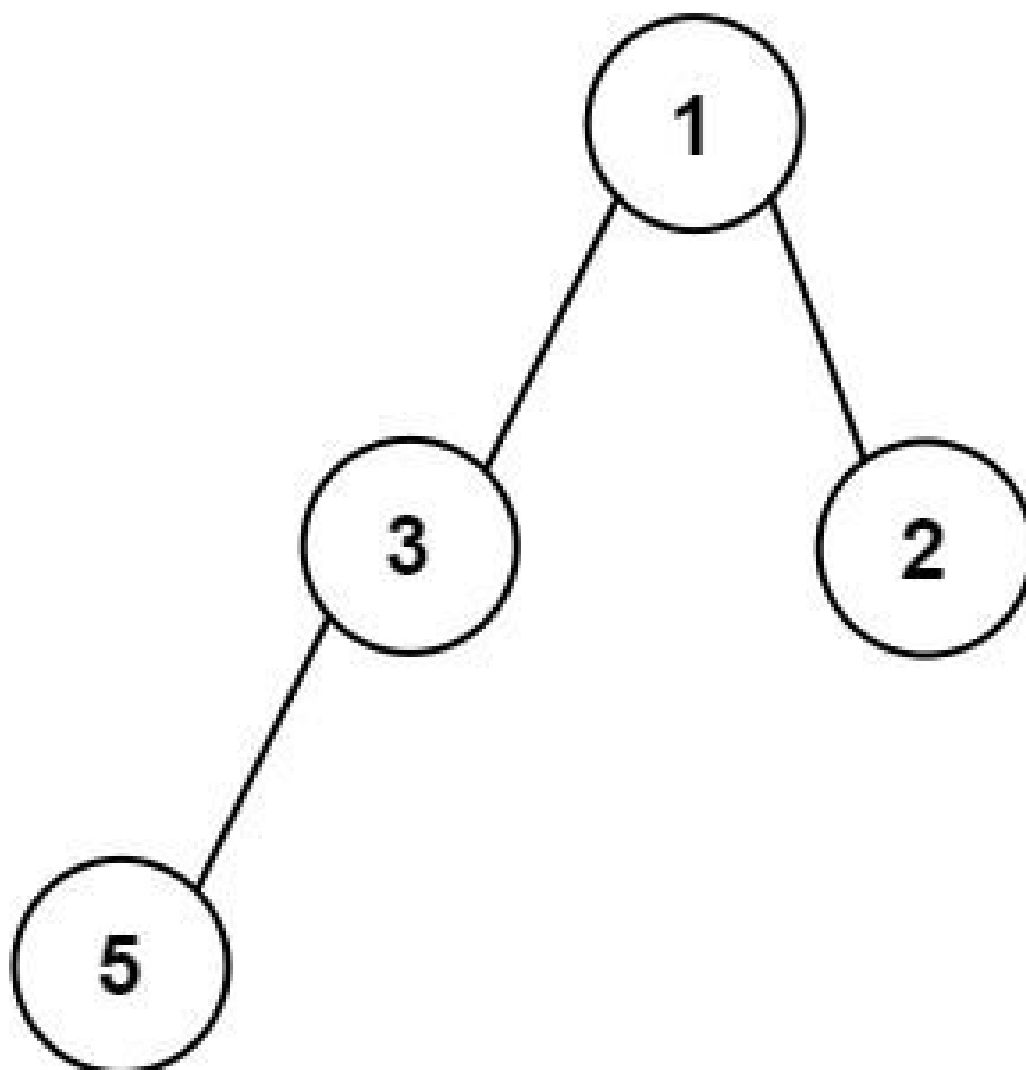


Input: root = [1,3,2,5,null,null,9,6,null,7]

Output: 7

Explanation: The maximum width exists in the fourth level with length 7 (6,null,null,null,null,null,7).

Example 3:



Input: root = [1,3,2,5]

Output: 2

Explanation: The maximum width exists in the second level with length 2 (3,2).

Constraints:

- The number of nodes in the tree is in the range [1, 3000].
- $-100 \leq \text{Node.val} \leq 100$

My LeetCode Solution

- My LeetCode Solution

```

class Solution:
    def widthOfBinaryTree(self, root: Optional[TreeNode]) -> int:
        if not root:
            return 0

        q = deque([(root, 0)])
        max_width = 1

        while q:
            level_len = len(q)

            left = q[0][1]
            right = q[-1][1]
            max_width = max(max_width, right - left + 1)

            for _ in range(level_len):
                node, idx = q.popleft()
                if node.left:
                    q.append((node.left, idx*2 + 1))
                if node.right:
                    q.append((node.right, idx*2 + 2))

        return max_width

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Width = rightmost_pos - leftmost_pos + 1 at each level, counting null gaps.
 # Left child = 2*pos, right child = 2*pos+1. Normalise each level to prevent overflow. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
 # BFS level-order traversal; at each level track min and max index assigned to nodes.
 # Width of level = max_index - min_index + 1; answer is global max width.
 # Straightforward queue-based level scan.
 # Time: O(n). Space: O(w) queue width.
 # Should I go ahead and optimize?"

```

class _662_MaxWidthBinaryTree:
    def widthOfBinaryTree(self, root: Optional[TreeNode]) -> int:
        if not root:

```

```

        return 0
    max_w = 0
    queue = deque([(root, 0)])
    while queue:
        size = len(queue)
        first_pos = queue[0][1]
        last_pos = 0
        for _ in range(size):
            node, pos = queue.popleft()
            pos -= first_pos # normalise
            last_pos = pos
            if node.left: queue.append((node.left, 2 * pos))
            if node.right: queue.append((node.right, 2 * pos + 1))
        max_w = max(max_w, last_pos + 1)
    return max_w

```

PHASE 5 — TEST & DEBUG

[1,3,2,5,3,null,9]: Level 2 positions (normalised): 5→0, 3→1, 9→3. last=3. width=4
✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(w)$. Normalisation prevents 2^{depth} overflow for deep trees – critical.

Given more time I'd ask: width by actual node count (ignoring nulls)? Just max level_size in BFS."

Community Solutions (Python)

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def widthOfBinaryTree(self, root: Optional[TreeNode]) -> int:
        ans = 0
        q = deque([(root, 1)])

```

```

while q:
    ans = max(ans, q[-1][1] - q[0][1] + 1)
    for _ in range(len(q)):
        root, i = q.popleft()
        if root.left:
            q.append((root.left, i << 1))
        if root.right:
            q.append((root.right, i << 1 | 1))
return ans

```

• SimplyLeet

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

from collections import deque

class Solution:

    def widthOfBinaryTree(self, root: TreeNode) -> int:
        if not root:
            return 0

        max_width = 0
        # Initialize queue with a tuple of (node, index)
        queue = deque([(root, 1)])

        while queue:
            level_length = len(queue)

```

```

    # Get the index of first node at current level
    _, first_index = queue[0]
    # Process nodes at current level
    for i in range(level_length):
        node, index = queue.popleft()
        # Check and add left child with index 2 * index
        if node.left:
            queue.append((node.left, 2 * index))
        # Check and add right child with index 2 * index + 1
        if node.right:
            queue.append((node.right, 2 * index + 1))
    # For the last node, calculate width of the level
    if i == level_length - 1:
        current_width = index - first_index + 1
        max_width = max(max_width, current_width)
    return max_width

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def widthOfBinaryTree(self, root: Optional[TreeNode]) -> int:
        ans = 0
        q = deque([(root, 1)])

        while q:
            ans = max(ans, q[-1][1] - q[0][1] + 1)
            for _ in range(len(q)):
                root, i = q.popleft()
                if root.left:
                    q.append((root.left, i << 1))
                if root.right:
                    q.append((root.right, i << 1 | 1))
        return ans

```


◆ Quick Review

Key Insights

- Perform a level order traversal (BFS) of the tree.
- Assign an index to each node as if the tree were complete (e.g., root is index 1, left child is 2
- i, right child is 2
- $i+1$).
- For each level, compute the width as $(\text{last_index} - \text{first_index} + 1)$.
- Using indices allows capturing gaps between nodes due to nulls in between.
- Edge cases include handling very skewed trees.

Space & Time

Time Complexity: $O(N)$, where N is the number of nodes in the tree.

Space Complexity: $O(N)$, due to the queue used for BFS which in the worst case holds nodes of the last level.

Solution

We use a Breadth-First Search (BFS) approach with a queue to traverse the tree level by level. At each level, we pair each node with an index representing its position assuming a complete binary tree. This allows us to compute the width of each level using the formula: $\text{width} = \text{last_index} - \text{first_index} + 1$. We then track the maximum width seen throughout the traversal. The use of indices handles cases where there are null gaps between nodes.

Trees & BST

□ #863 All Nodes Distance K in Binary Tree **MED**

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Hash Table · Tree · DFS · BFS

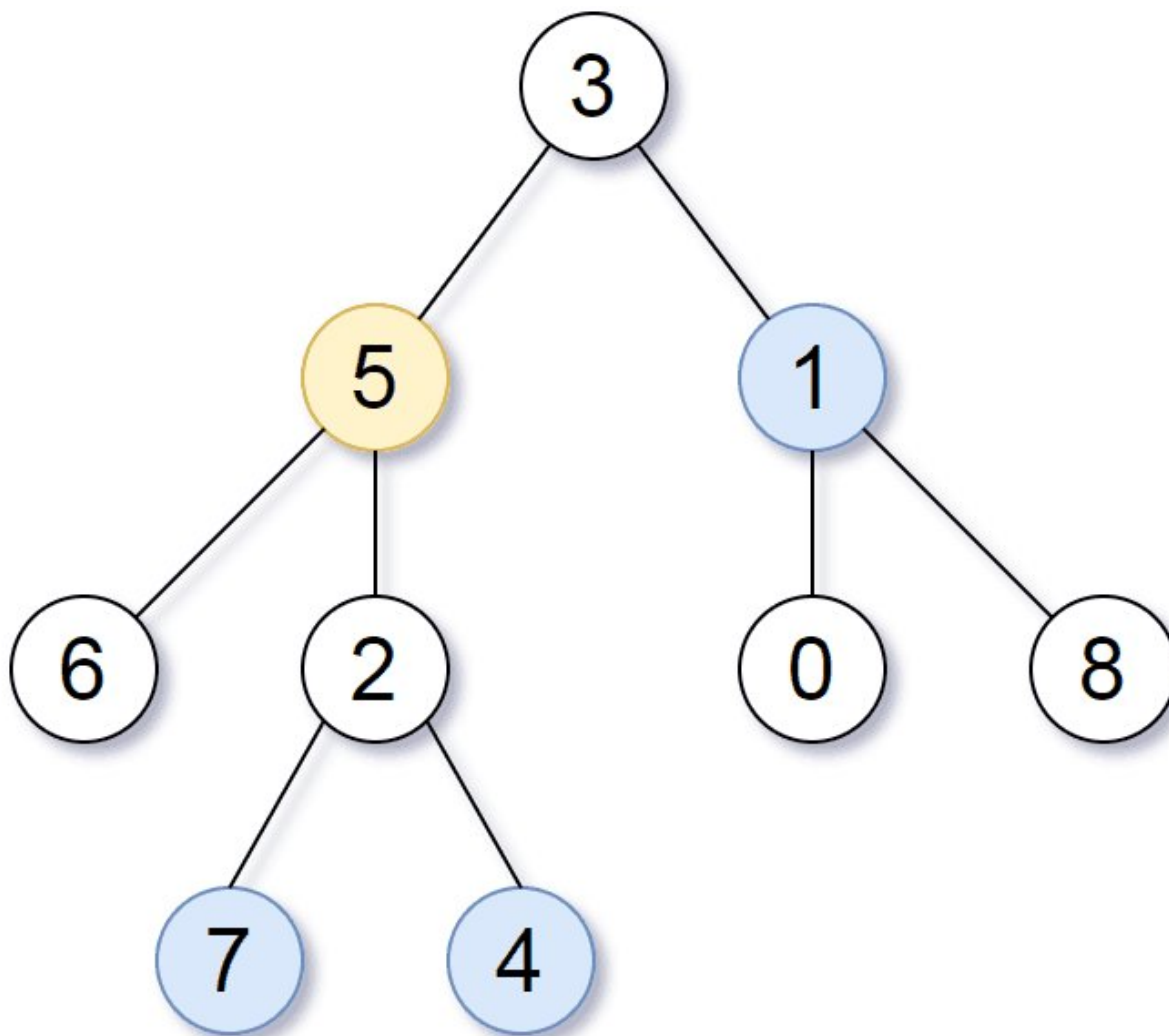
Lists: Grind 169

Problem

Given the root of a binary tree, the value of a target node target, and an integer k, return an array of the values of all nodes that have a distance k from the target node.

You can return the answer in any order.

Example 1:



Input: root = [3,5,1,6,2,0,8,null,null,7,4], target = 5, k = 2

Output: [7,4,1]

Explanation: The nodes that are a distance 2 from the target node (with value 5) have values 7, 4, and 1.

Example 2:

Input: root = [1], target = 1, k = 3

Output: []

Constraints:

- The number of nodes in the tree is in the range [1, 500].
- $0 \leq \text{Node.val} \leq 500$

- All the values `Node.val` are unique.
- `target` is the value of one of the nodes in the tree.
- $0 \leq k \leq 1000$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def distanceK(self, root: TreeNode, target: TreeNode, k: int) -> List[int]
    :
        from collections import defaultdict, deque

        graph = defaultdict(list)

        def build(node, parent):
            if node:
                if parent:
                    graph[node.val].append(parent.val)

                    graph[parent.val].append(node.val)
                    build(node.left, node)
                    build(node.right, node)

        build(root, None)

        q = deque([(target.val, 0)])
        seen = {target.val}
        res = []
```

```

while q:
    node, dist = q.popleft()
    if dist == k:
        res.append(node)
    elif dist < k:
        for nei in graph[node]:
            if nei not in seen:
                seen.add(nei)
                q.append((nei, dist + 1))

return res

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find all nodes exactly k edges from target. Distance can go upward through parent. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Build parent map via BFS/DFS from target. For each query node, walk parent pointers

until reaching target; collect distance. $O(n)$ per query naive.

Time: $O(n * Q)$ for Q queries. Space: $O(n)$ parent map.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (build parent map then BFS)

"DFS to build parent[node]=parent. Then BFS from target treating tree as undirected graph.

Stop collecting at distance k."

PHASE 4 — CLEAN CODE

```

class _863_DistanceK:
    def distanceK(self, root: 'TreeNode', target: 'TreeNode', k: int) -> List[int]:
        parent: dict = {}
        def build(node, par):
            if not node: return
            parent[node] = par
            build(node.left, node)
            build(node.right, node)
        build(root, None)
        visited = {target}
        queue = deque([(target, 0)])
        result = []

```

```

while queue:
    node, dist = queue.popleft()
    if dist == k:
        result.append(node.val)
        continue
    for nb in [node.left, node.right, parent[node]]:
        if nb and nb not in visited:
            visited.add(nb)
            queue.append((nb, dist + 1))
return result

```

PHASE 5 — TEST & DEBUG

target=5, k=2: BFS outward from 5 by 2 hops → [7,4,1] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$. Parent map converts tree to undirected graph – then BFS is trivial.

Given more time I'd ask: multiple targets? Multi-source BFS – enqueue all at distance 0."

Community Solutions (Python)

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    def distanceK(self, root: TreeNode, target: TreeNode, k: int) -> List[int]
    :

    def dfs(self, root, fa):
        if root is None:
            return
        g[root] = fa
        dfs(root.left, root)
        dfs(root.right, root)

    def dfs2(self, root, fa, k):
        if root is None:
            return

```



```

        if k == 0:
            ans.append(root.val)
            return
        for nxt in (root.left, root.right, g[root]):
            if nxt != fa:
                dfs2(nxt, root, k - 1)

g = {}
dfs(root, None)
ans = []

dfs2(target, None, k)
return ans

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def distanceK(root: TreeNode, target: int, k: int):
    from collections import defaultdict, deque

    # Build an adjacency list to represent the graph
    graph = defaultdict(list)

    # Helper function to build the graph using DFS
    def buildGraph(node, parent):
        if node is None:
            return
        if parent is not None:
            # Add bidirectional connection between node and parent
            graph[node.val].append(parent.val)

```

```

        graph[parent.val].append(node.val)
        buildGraph(node.left, node)
        buildGraph(node.right, node)

```

```

buildGraph(root, None)

```

```

# BFS from target node
visited = set([target])
queue = deque([target])
current_distance = 0

```

```

while queue:
    if current_distance == k:
        # Return all nodes at distance k
        return list(queue)
    size = len(queue)
    for _ in range(size):
        current = queue.popleft()
        for neighbor in graph[current]:
            if neighbor not in visited:

```

```

                visited.add(neighbor)
                queue.append(neighbor)
            current_distance += 1

```

```

return [] # If no nodes at distance k

```

```

# Example usage:

```

```

# Construct tree or use a tree builder before calling distanceK function.

```

• **LC.ca**

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    def distanceK(self, root: TreeNode, target: TreeNode, k: int) -> List[int]
    :

    def parents(root, prev):
        nonlocal p
        if root is None:
            return
        p[root] = prev
        parents(root.left, root)
        parents(root.right, root)

    def dfs(root, k):
        nonlocal ans, vis

        if root is None or root.val in vis:
            return
        vis.add(root.val)
        if k == 0:
            ans.append(root.val)
            return
        dfs(root.left, k - 1)
        dfs(root.right, k - 1)
        dfs(p[root], k - 1)

    p = {}
    parents(root, None)
    ans = []
    vis = set()
    dfs(target, k)
    return ans

```

◆ Quick Review

Round 2 · High · Medium

p.1303

Key Insights

- Convert the binary tree into an undirected graph so that we can easily traverse both down to the children and up to the parent.
- Use a DFS or simple recursive traversal to create an adjacency list representing the graph.
- Perform a BFS starting from the target node to explore nodes level-by-level until reaching distance k .
- Use a visited set (or equivalent) to avoid revisiting nodes and to prevent cycles.

Space & Time

Time Complexity: $O(N)$, where N is the number of nodes in the tree. Each node is visited once when constructing the graph and once in the BFS.

Space Complexity: $O(N)$, for storing the graph and the additional data structures used in BFS and DFS.

Solution

The solution involves two main steps: Build an undirected graph representation of the binary tree using a DFS. For each node, add the node's

value as a key in an adjacency list and connect it with its children (if any). This creates bidirectional links between parent and child. Use a BFS starting from the target node and explore nodes level-by-level. Keep a counter for the current distance, and once the counter reaches k , return all node values found at that level. To avoid processing nodes more than once, maintain a set of visited nodes.

Trees & BST

□ ★ #1120 Maximum Average Subtree ★

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS

Lists: ★ Premium | Premium 98

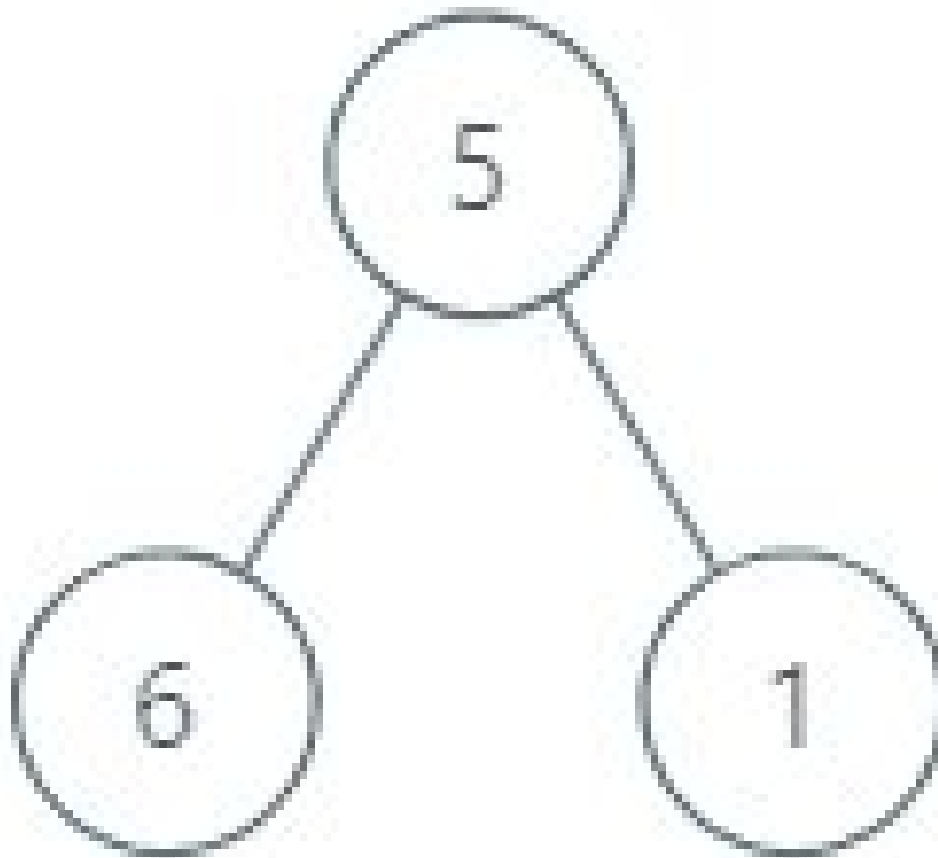
Problem

Given the root of a binary tree, return the maximum average value of a subtree of that tree. Answers within 10^{-5} of the actual answer will be accepted.

A subtree of a tree is any node of that tree plus all its descendants.

The average value of a tree is the sum of its values, divided by the number of nodes.

Example 1:



Input: root = [5,6,1]

Output: 6.00000

Explanation:

For the node with value = 5 we have an average of $(5 + 6 + 1) / 3 = 4$.

For the node with value = 6 we have an average of $6 / 1 = 6$.

For the node with value = 1 we have an average of $1 / 1 = 1$.

So the answer is 6 which is the maximum.

Example 2:

Input: root = [0,null,1]

Output: 1.00000

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- $0 \leq \text{Node.val} \leq 105$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def maximumAverageSubtree(self, root: Optional[TreeNode]) -> float:
        self.best = 0

        def dfs(node):
            if not node:
                return 0, 0
            l_sum, l_cnt = dfs(node.left)
            r_sum, r_cnt = dfs(node.right)
            s, c = node.val + l_sum + r_sum, 1 + l_cnt + r_cnt

            self.best = max(self.best, s / c)
            return s, c

        dfs(root)
        return self.best
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the subtree (any node + all descendants) with the maximum average value. Return the average. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."
 # Post-order DFS: for each node return (sum, count) of its subtree.
 # Track max average = sum/count over all visited nodes.
 # Recomputes subtree sums independently – correct O(n) post-order.
 # Time: O(n). Space: O(h) stack.
 # Should I go ahead and optimize?"

```
class _1120_MaxAvgSubtree:
    def maximumAverageSubtree(self, root: Optional['TreeNode']) -> float:
        best = [0.0]
        def dfs(node):
            if not node: return 0, 0
            l_sum, l_cnt = dfs(node.left)
            r_sum, r_cnt = dfs(node.right)
            total_sum = l_sum + r_sum + node.val
            total_cnt = l_cnt + r_cnt + 1
```

```

        best[0] = max(best[0], total_sum / total_cnt)
        return total_sum, total_cnt
    dfs(root)
    return best[0]

```

PHASE 5 — TEST & DEBUG

[5,6,1]: avg(6)=6, avg(1)=1, avg(5,6,1)=4. → 6.0 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(h)$. Post-order carries (sum, count) up — compute average at each node.

Given more time I'd ask: minimum average subtree? Track min instead of max."

Community Solutions (Python)

• WalkCC

```
from dataclasses import dataclass
```

```
@dataclass(frozen=True)
```

```
class T:
```

```
    summ: int
```

```
    count: int
```

```
    maxAverage: int
```

```
class Solution:
```

```
    def maximumAverageSubtree(self, root: TreeNode | None) -> float:
```

```
        def maximumAverage(root: TreeNode | None) -> T:
```

```
            if not root:
```

```
                return T(0, 0, 0)
```

```
            left = maximumAverage(root.left)
```

```
            right = maximumAverage(root.right)
```

```
            summ = root.val + left.summ + right.summ
```

```
            count = 1 + left.count + right.count
```

```
            maxAverage = max(summ / count, left.maxAverage, right.maxAverage)
```

```
            return T(summ, count, maxAverage)
```

```
        return maximumAverage(root).maxAverage
```

• LeetCode

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def maximumAverageSubtree(self, root: Optional[TreeNode]) -> float:
        def dfs(root):
            if root is None:
                return 0, 0
            ls, ln = dfs(root.left)
            rs, rn = dfs(root.right)
            s = root.val + ls + rs
            n = 1 + ln + rn
            nonlocal ans
            ans = max(ans, s / n)
            return s, n

        ans = 0

        dfs(root)
        return ans
```

• SimplyLeet

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val        # value of the node
        self.left = left      # left child
        self.right = right    # right child

class Solution:
    def maximumAverageSubtree(self, root):
        # Initialize the maximum average to negative infinity
```

```

self.maxAvg = float('-inf')

# Recursive DFS function that returns a tuple (subtree sum, node count)
def dfs(node):
    if not node:
        return (0, 0) # return sum 0 and count 0 if node is null
    leftSum, leftCount = dfs(node.left) # traverse left subtree
    rightSum, rightCount = dfs(node.right) # traverse right subtree
    currentSum = node.val + leftSum + rightSum # total sum at
current node
    currentCount = 1 + leftCount + rightCount # total node count

    currentAvg = currentSum / currentCount # average value for
current subtree
    self.maxAvg = max(self.maxAvg, currentAvg) # update maximum
average if necessary
    return (currentSum, currentCount)

dfs(root)
return self.maxAvg

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right
class Solution:
    def maximumAverageSubtree(self, root: Optional[TreeNode]) -> float:
        def dfs(root):
            if root is None:

```

```
        return 0, 0
    ls, ln = dfs(root.left)
    rs, rn = dfs(root.right)
    s = root.val + ls + rs
    n = 1 + ln + rn
    nonlocal ans
    ans = max(ans, s / n)
    return s, n
```

```
ans = 0
```

```
dfs(root)
return ans
```

◆ Quick Review

Key Insights

- Use a depth-first search (DFS) to traverse every subtree.
- At each node, compute the sum and count of nodes in its subtree.
- Calculate the average for the current subtree and update a global maximum if needed.
- Processing every node exactly once yields an efficient solution.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes (each node is visited once).

Space Complexity: $O(h)$, where h is the height of the tree (space for recursion stack).

Solution

We perform a post-order DFS traversal, calculating at each node both the cumulative sum and the count of nodes in its subtree. This enables us to compute the average value in constant time for any subtree. We maintain a global variable to track the maximum average encountered during the traversal. The primary technique is recursive DFS combined with tuple (or pair) aggregation. The simplicity of the recursion and in-place aggregation ensures an efficient and clear solution.

Trees & BST

□ ★ #1490 Clone N-ary Tree ★

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Tree · DFS · BFS
Lists: ★ Premium | Premium 98

Problem

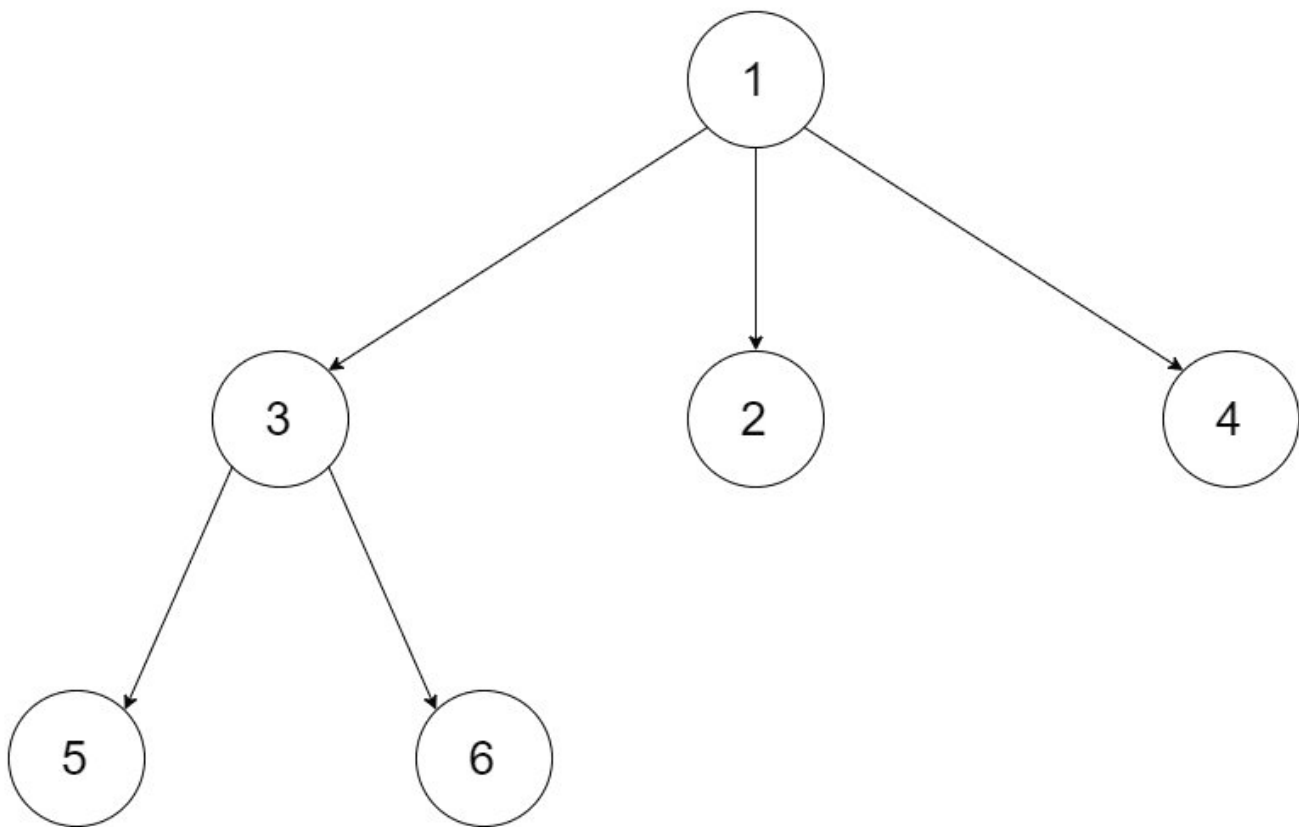
Given a root of an N-ary tree, return a deep copy (clone) of the tree.

Each node in the n-ary tree contains a val (int) and a list (List[Node]) of its children.

```
class Node {  
    public int val;  
    public List<Node> children;  
}
```

Nary-Tree input serialization is represented in their level order traversal, each group of children is separated by the null value (See examples).

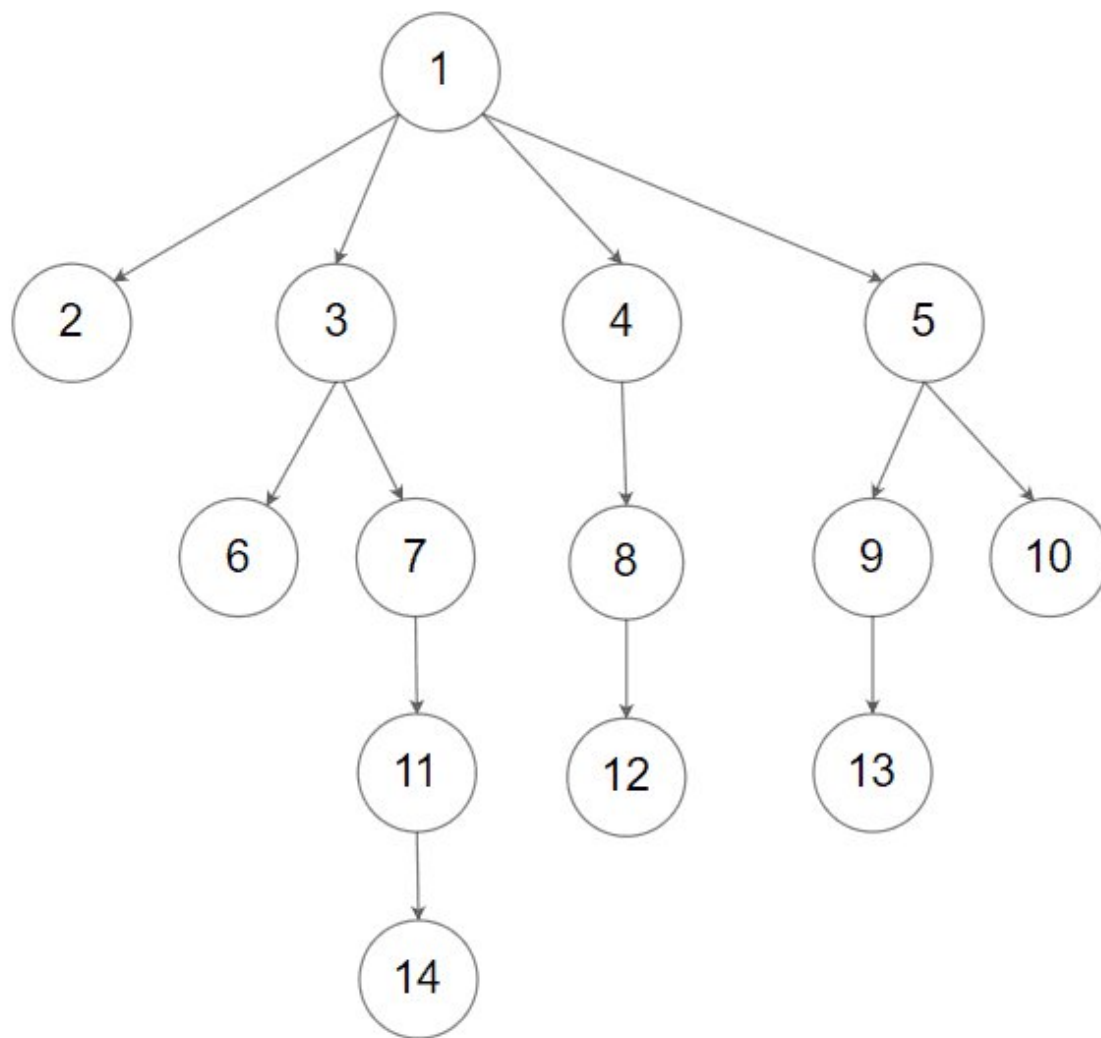
Example 1:



Input: root = [1,null,3,2,4,null,5,6]

Output: [1,null,3,2,4,null,5,6]

Example 2:



Input: root = [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]
 Output: [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]

Constraints:

- The depth of the n-ary tree is less than or equal to 1000.
- The total number of nodes is between [0, 104].

Follow up: Can your solution work for the graph problem?

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def cloneTree(self, root: 'Node') -> 'Node':
        def dfs(node):
            if not node:
                return None
            new = Node(node.val)
            new.children = [dfs(child) for child in node.children]
            return new

        return dfs(root)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Deep clone an N-ary tree. No shared references between original and clone. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

DFS clone: map each original node to a new node, recursively clone all children lists.

First pass creates nodes on demand; second pass wires child pointers.

Time: $O(n)$. Space: $O(n)$ for map + recursion.

Should I go ahead and optimize?"

class _1490_CloneNaryTree:

```
def cloneTree(self, root: 'Node') -> 'Node':
```

```
    if not root:
```

```
        return None
```

```
    return Node(root.val, [self.cloneTree(child) for child in root.children])
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$. Simple recursive clone – no visited map needed (trees have no cycles)."

Follow-up (Clone Graph #133): needs visited map to handle cycles – {original: clone}."

Community Solutions (Python)

• LeetCode

```

"""
# Definition for a Node.
class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children if children is not None else []
"""

class Solution:

    def cloneTree(self, root: 'Node') -> 'Node':
        if root is None:
            return None
        children = [self.cloneTree(child) for child in root.children]
        return Node(root.val, children)

```

• SimplyLeet

```

# Definition for a Node.
class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children if children is not None else []

def cloneTree(root):
    # Base case: if root is None, return None.
    if root is None:
        return None

    # Create a clone for the current node.
    cloned_node = Node(root.val)

    # Recursively clone each child and add it to the cloned_node's children
    list.
    for child in root.children:
        cloned_child = cloneTree(child)
        cloned_node.children.append(cloned_child)

    # Return the cloned node.

```

```
return cloned_node
```

```
# Example usage:
```

```
# original_tree = Node(1, [Node(3, [Node(5), Node(6)]), Node(2), Node(4)])
```

```
# cloned_tree = cloneTree(original_tree)
```

• LC.ca

```
"""
```

```
# Definition for a Node.
```

```
class Node:
```

```
    def __init__(self, val=None, children=None):
```

```
        self.val = val
```

```
        self.children = children if children is not None else []
```

```
"""
```

```
class Solution:
```

```
    def cloneTree(self, root: 'Node') -> 'Node':
```

```
        if root is None:
```

```
            return None
```

```
        children = [self.cloneTree(child) for child in root.children]
```

```
        return Node(root.val, children)
```

◆ Quick Review

Key Insights

- The tree is N-ary, meaning every node can have multiple children.
- A deep copy requires creating new nodes for each node in the tree.
- Both DFS (recursive) or BFS (iterative) traversals can be used to clone the tree.

- The overall time complexity is $O(N)$ since each node is visited once.
- We can follow similar cloning techniques used in cloning graphs, using either recursion or an auxiliary data structure like a queue.

Space & Time

Time Complexity: $O(N)$, where N is the number of nodes in the tree, since each node is visited exactly once.

Space Complexity: $O(N)$, accounting for the recursive call stack (or an explicit queue in BFS) and the space needed to store the clone.

Solution

We perform a DFS traversal starting from the root node. For each node we encounter, we:
Create a new node with the same value.
Recursively clone all its children and attach the cloned children to the new node. This approach ensures that every original node is copied into a new node and that the parent-child relationships are preserved. Edge cases such as an empty tree (null root) are also handled by returning null immediately.

Trees & BST

□ ★ #1522 Diameter of N-Ary Tree ★

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Tree · DFS

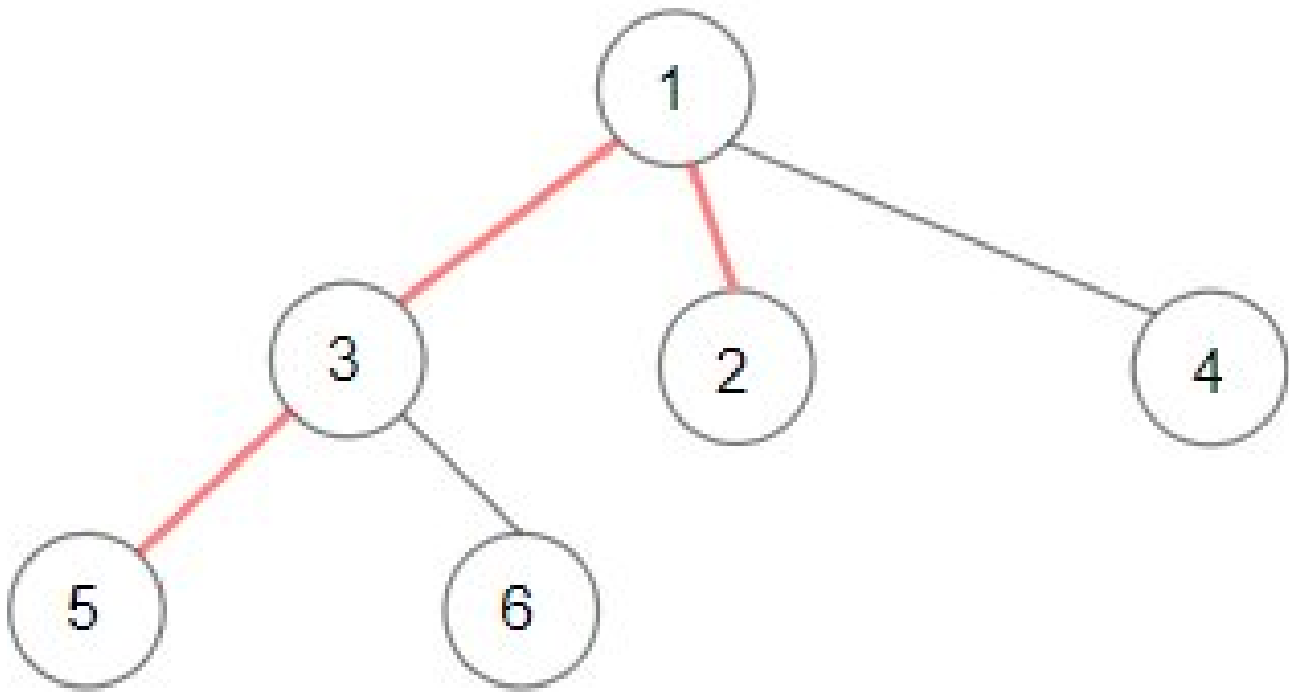
Lists: ★ Premium | Premium 98

Problem

Given a root of an N-ary tree, you need to compute the length of the diameter of the tree. The diameter of an N-ary tree is the length of the longest path between any two nodes in the tree. This path may or may not pass through the root.

(Nary-Tree input serialization is represented in their level order traversal, each group of children is separated by the null value.)

Example 1:

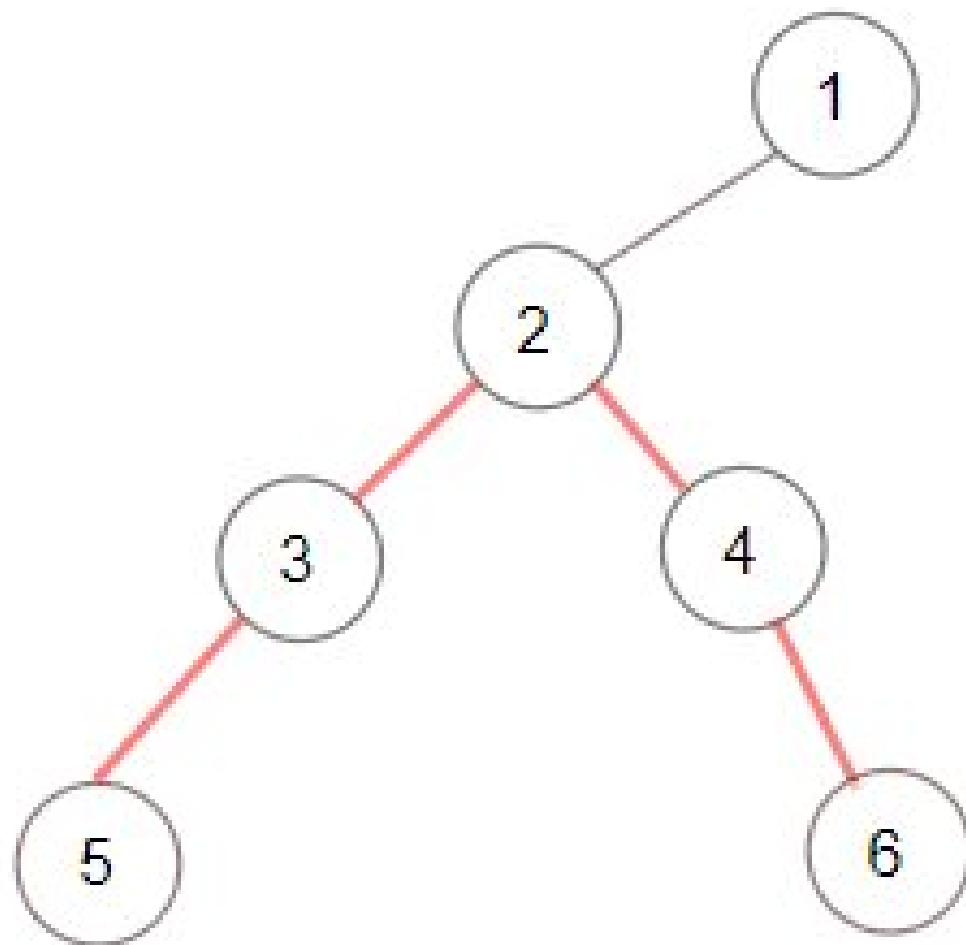


Input: root = [1,null,3,2,4,null,5,6]

Output: 3

Explanation: Diameter is shown in red color.

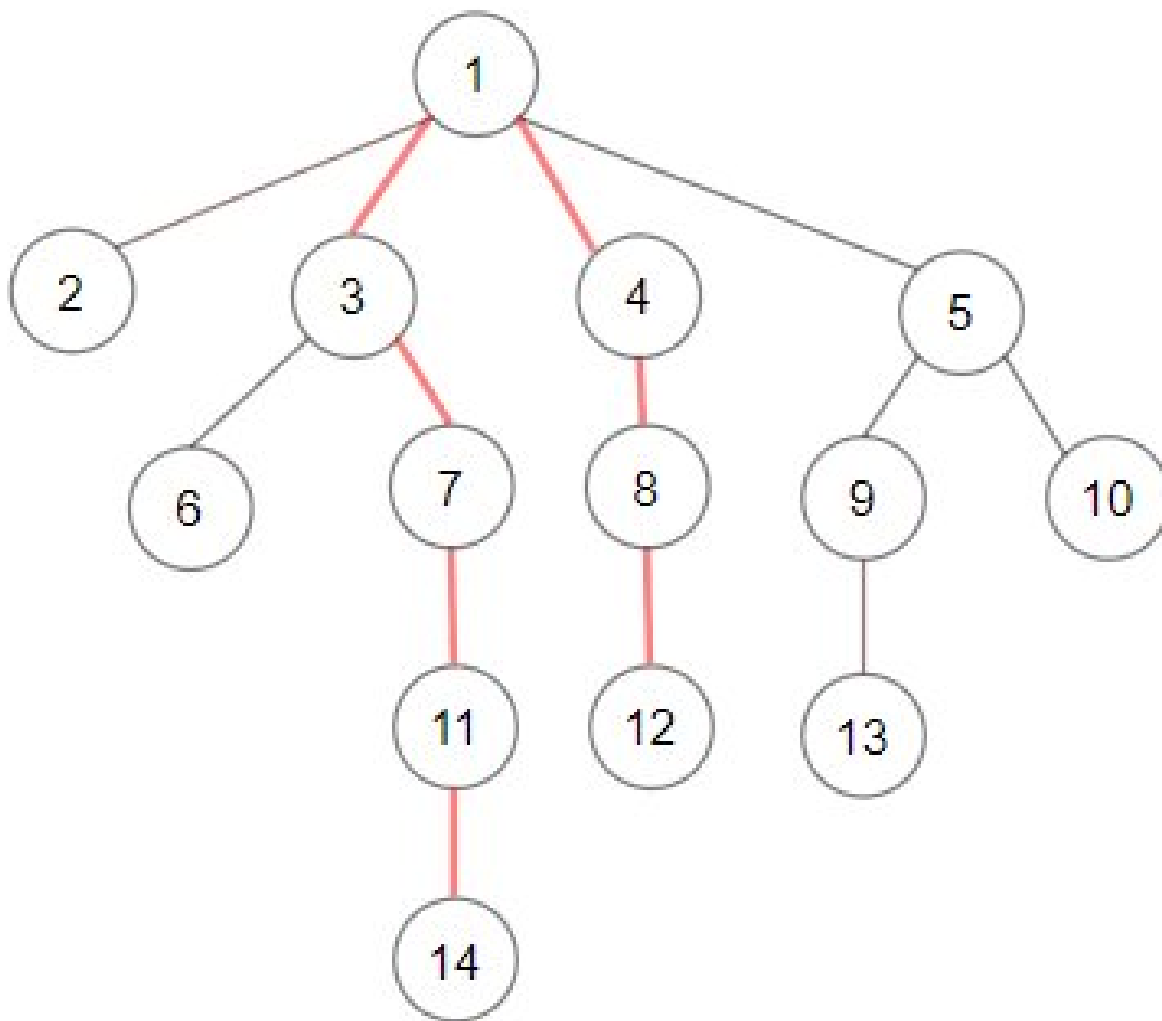
Example 2:



Input: root = [1,null,2,null,3,4,null,5,null,6]

Output: 4

Example 3:



Input: root = [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]
 Output: 7

Constraints:

- The depth of the n-ary tree is less than or equal to 1000.
- The total number of nodes is between [1, 104].

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def diameter(self, root: 'Node') -> int:
        self.ans = 0

        def dfs(node):
            max_depth = 0
            for child in node.children:
                depth = dfs(child)
                self.ans = max(self.ans, max_depth + depth)
                max_depth = max(max_depth, depth)

            return max_depth + 1

        dfs(root)
        return self.ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Diameter = longest path between any two nodes (in edges), generalised to N children. Right?"
 # Same concept as Binary Tree Diameter (#543)."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."
 # Two-pass DFS diameter: for each node, longest path through it = sum of top two child depths.
 # Same as binary tree diameter but children list instead of left/right.
 # Time: $O(n)$. Space: $O(h)$ stack.
 # Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Post-order: for each node, collect heights of children (adding 1 for the edge to parent)."
 # Diameter through this node = top two child-heights summed.
 # Node's own height = max child-height (or 0 for leaf)."

PHASE 4 — CLEAN CODE

```
class _1522_DiameterNaryTree:
    def diameter(self, root: 'Node') -> int:
        best = [0]
        def height(node) -> int:
            if not node or not node.children:
```

```

        return 0
    # child_h+1 converts "height below child" to "edges from this node via
    that child"
    child_heights = sorted([height(c) + 1 for c in node.children],
reverse=True)
    if len(child_heights) >= 2:
        best[0] = max(best[0], child_heights[0] + child_heights[1])
    else:
        best[0] = max(best[0], child_heights[0])
    return child_heights[0]
height(root)
return best[0]

```

PHASE 5 — TEST & DEBUG

```

# [1,null,3,2,4,null,5,6]: height(5)=0, height(6)=0.
# height(3): child_heights=[0+1,0+1]=[1,1]. best=1+1=2. return 1.
# height(2)=0, height(4)=0.
# height(1): child_heights=[1+1,0+1,0+1]=[2,1,1]. best=2+1=3. return 2. → 3 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n). Space O(h). Adding +1 when collecting child heights is the key
difference from #543.
# In binary tree, depth(child) already includes the edge up; here we add it
explicitly.
# Given more time I'd ask: diameter in edge-weighted N-ary tree?
# Same approach – just use edge weights instead of +1."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def diameter(self, root: 'Node') -> int:
        ans = 0

    def maxDepth(root: 'Node') -> int:
        """Returns the maximum depth of the subtree rooted at `root`."""
        nonlocal ans
        maxSubDepth1 = 0
        maxSubDepth2 = 0
        for child in root.children:

```

```

        maxSubDepth = maxDepth(child)
        if maxSubDepth > maxSubDepth1:
            maxSubDepth2 = maxSubDepth1
            maxSubDepth1 = maxSubDepth
        elif maxSubDepth > maxSubDepth2:
            maxSubDepth2 = maxSubDepth
    ans = max(ans, maxSubDepth1 + maxSubDepth2)
    return 1 + maxSubDepth1

maxDepth(root)

```

```

return ans

```

• LeetDoocs

```

"""
# Definition for a Node.
class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children if children is not None else []
"""

```

```

class Solution:

```

```

    def diameter(self, root: 'Node') -> int:
        """
        :type root: 'Node'
        :rtype: int
        """

        def dfs(root):
            if root is None:
                return 0
            nonlocal ans

```

```

m1 = m2 = 0
for child in root.children:
    t = dfs(child)
    if t > m1:
        m2, m1 = m1, t
    elif t > m2:
        m2 = t
ans = max(ans, m1 + m2)
return 1 + m1

```

```

ans = 0
dfs(root)
return ans

```

• SimplyLeet

```

# Definition for a Node.
# class Node:
# def __init__(self, val=None, children=None):
# self.val = val
# self.children = children if children is not None else []

class Solution:
    def diameter(self, root: 'Node') -> int:
        # Global variable to keep track of the maximum diameter found so far.
        self.max_diameter = 0

    def dfs(node):
        if not node:
            return 0 # Base case: if the node is None, return depth 0

        # Variables to hold the top two maximum depths from the children.
        max_depth1, max_depth2 = 0, 0

        # Iterate over each child to compute their maximum depths.
        for child in node.children:

```

```

        child_depth = dfs(child) # Recurse on the child
        # Update the two largest depths
        if child_depth > max_depth1:
            max_depth2 = max_depth1
            max_depth1 = child_depth
        elif child_depth > max_depth2:
            max_depth2 = child_depth

        # Update the maximum diameter. The candidate is the sum of the two
        deepest paths.
        self.max_diameter = max(self.max_diameter, max_depth1 + max_depth2
    )

    # Return the maximum depth from this node.
    return max_depth1 + 1

dfs(root)
return self.max_diameter

```

• LC.ca

```

"""
# Definition for a Node.
class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children if children is not None else []
"""

class Solution:

    def diameter(self, root: 'Node') -> int:
        """
        :type root: 'Node'
        :rtype: int
        """

        def dfs(root):
            if root is None:
                return 0
            nonlocal ans

```



```
m1 = m2 = 0
for child in root.children:
    t = dfs(child)
    if t > m1:
        m2, m1 = m1, t
    elif t > m2:
        m2 = t
ans = max(ans, m1 + m2)
return 1 + m1
```

```
ans = 0
dfs(root)
return ans
```

◆ Quick Review

Key Insights

- The diameter of the tree can be found by considering the two largest depths from any node among its children.
- Use a depth-first search (DFS) to compute the maximum depth from each node.
- For each node, the potential diameter is the sum of the two longest depths among its children.
- Update a global variable tracking the maximum diameter as you recurse through the tree.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes, since each node is visited once.

Space Complexity: $O(h)$, where h is the height of the tree, due to the recursion stack (worst-case $O(n)$ for a skewed tree).

Solution

Use a DFS approach to traverse the tree. At each node, calculate the longest and second longest depth among its children. The sum of these two depths gives the candidate diameter through that node. Maintain a global variable to track the maximum diameter found so far. Return the maximum depth from the node (i.e., one plus the largest depth among its children) to be used in parent's calculations.

Trees & BST

□ ★ #1650 Lowest Common Ancestor of a Binary Tree III ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Tree · Two Pointers
Lists: ★ Premium | Premium 98

Problem

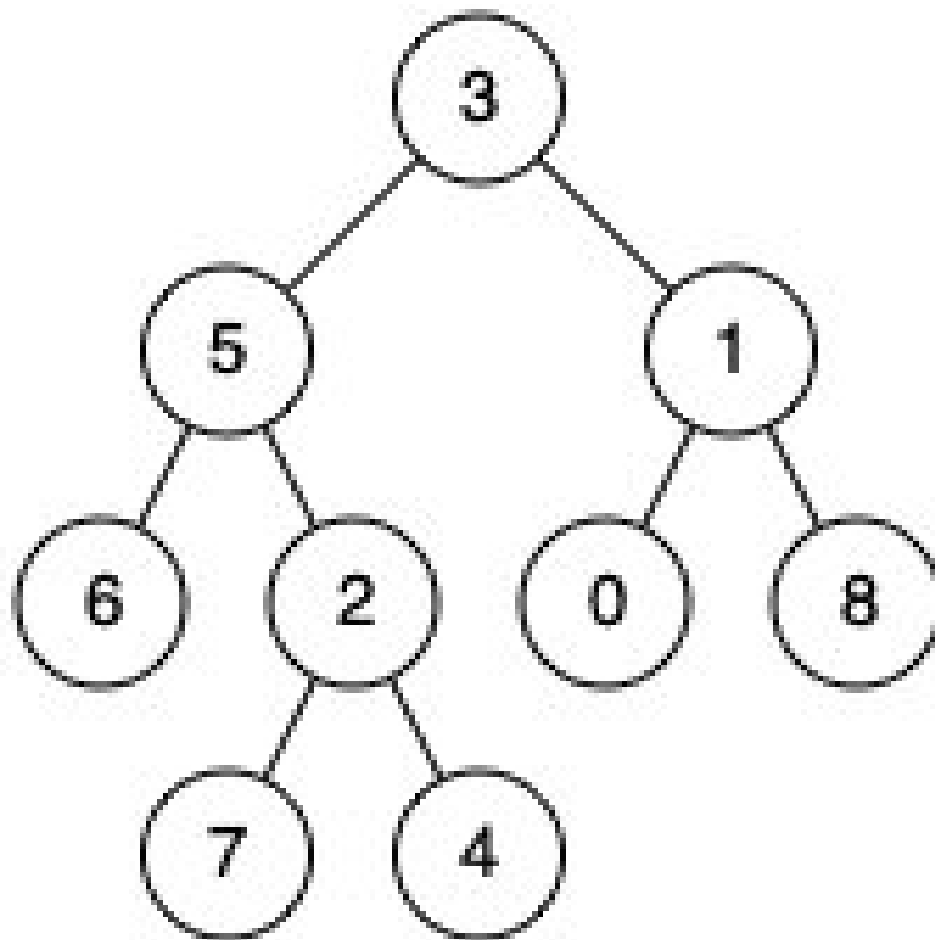
Given two nodes of a binary tree p and q, return their lowest common ancestor (LCA).

Each node will have a reference to its parent node. The definition for Node is below:

```
class Node {  
    public int val;  
    public Node left;  
    public Node right;  
    public Node parent;  
}
```

According to the definition of LCA on Wikipedia: "The lowest common ancestor of two nodes p and q in a tree T is the lowest node that has both p and q as descendants (where we allow a node to be a descendant of itself)."

Example 1:

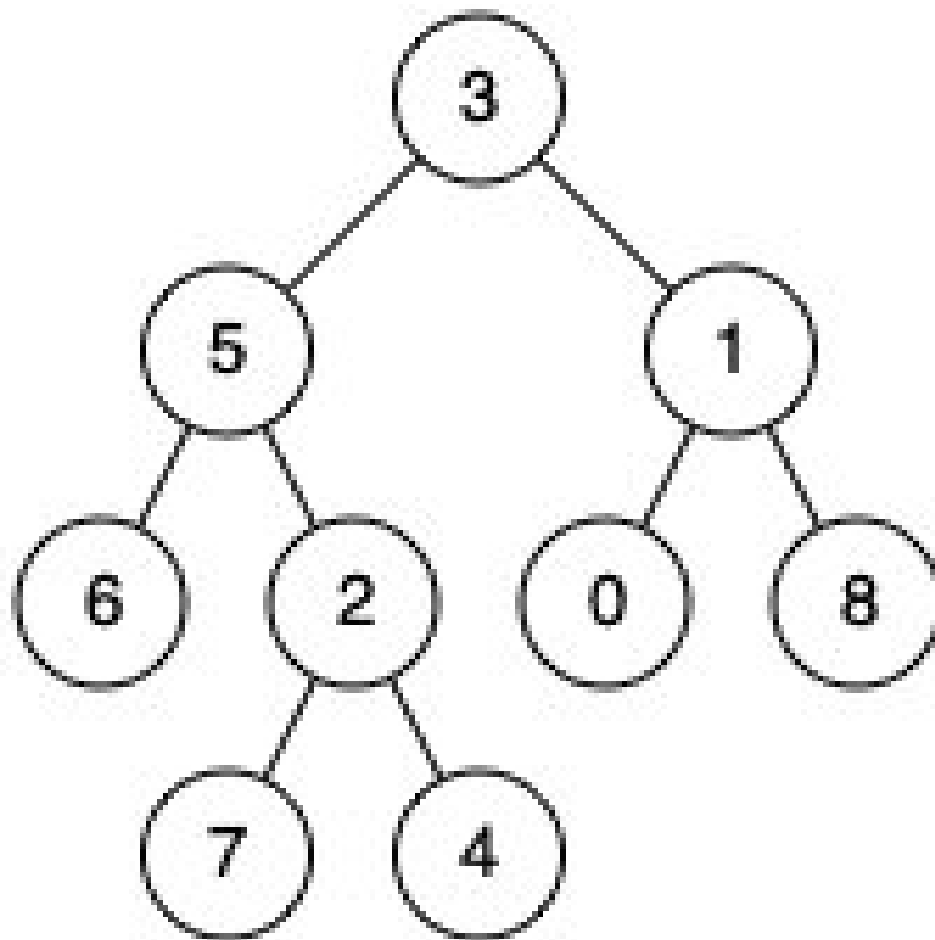


Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1

Output: 3

Explanation: The LCA of nodes 5 and 1 is 3.

Example 2:



Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4

Output: 5

Explanation: The LCA of nodes 5 and 4 is 5 since a node can be a descendant of itself according to the LCA definition.

Example 3:

Input: root = [1,2], p = 1, q = 2

Output: 1

Constraints:

- The number of nodes in the tree is in the range [2, 105].
- $-109 \leq \text{Node.val} \leq 109$

- All Node.val are unique.
- $p \neq q$
- p and q exist in the tree.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def lowestCommonAncestor(self, p: 'Node', q: 'Node') -> 'Node':
        path = set()
        while p:
            path.add(p)
            p = p.parent
        while q not in path:
            q = q.parent
        return q
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Nodes have a parent pointer. Find LCA without the root. Right?"

Both p and q are guaranteed to be in the tree."

#

"[3,5,1,...], p=5, q=1 → LCA=3 ✓ (same as #236 but solved via parent pointers)"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Each node has a parent pointer. Walk both nodes up with a visited set until they meet.

First node seen twice is LCA. Like intersecting two linked lists.

Time: O(h). Space: O(h) visited set.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (two-pointer linked list intersection)

"Walk p upward and q upward simultaneously. When one hits null, restart at the other's origin.

They meet at the LCA — same as finding intersection of two linked lists.

Each pointer traverses: (path from p to root) + (path from q to root), length l_p + l_q.

Both pointers traverse the same total length → they meet at LCA."

PHASE 4 — CLEAN CODE

```
class _1650_LCA_III:
    def lowestCommonAncestor(self, p: 'Node', q: 'Node') -> 'Node':
        a, b = p, q
        while a is not b:
            a = a.parent if a else q
            b = b.parent if b else p
        return a
```

PHASE 5 — TEST & DEBUG

p=5, q=1: a traverses 5→3→None→1→3. b traverses 1→3→None→5→3. Both reach 3 simultaneously ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time O(h). Space O(1). This is the two-pointer intersection trick from **Linked List Intersection (#160)**.

Both pointers traverse depth(p) + depth(q) steps total and meet at LCA.

Given more time I'd ask: what if the tree is very deep and we want to avoid the full traversal?

Use a set: walk p to root recording all ancestors, then walk q to root and stop at first ancestor in the set.

O(depth) time, O(depth) space – same asymptotic but possibly faster in practice."

Community Solutions (Python)

• WalkCC

```
class Solution:
    # Same as 160. Intersection of Two Linked Lists
    def lowestCommonAncestor(self, p: 'Node', q: 'Node') -> 'Node':
        a = p
        b = q

        while a != b:
            a = a.parent if a else q
            b = b.parent if b else p

        return a
```

• LeetDoocs


```

#####
# Definition for a Node.
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
        self.parent = None
#####

```

```

class Solution:
    def lowestCommonAncestor(self, p: "Node", q: "Node") -> "Node":
        vis = set()
        node = p
        while node:
            vis.add(node)
            node = node.parent
        node = q
        while node not in vis:
            node = node.parent
        return node

```

• SimplyLeet

```

# Definition for a Node.
# class Node:
#     def __init__(self, val=0, left=None, right=None, parent=None):
#         self.val = val
#         self.left = left
#         self.right = right
#         self.parent = parent

def lowestCommonAncestor(p, q):
    # Compute the depth for node p

```

```
def getDepth(node):
    depth = 0
    while node:
        depth += 1
        node = node.parent
    return depth
```

```
depthP = getDepth(p)
depthQ = getDepth(q)
```

```
# Bring both nodes to the same depth level
while depthP > depthQ:
    p = p.parent
    depthP -= 1
while depthQ > depthP:
    q = q.parent
    depthQ -= 1

# Traverse up until we find the common ancestor
while p != q:
```

```
    p = p.parent
    q = q.parent
```

```
return p
```

• LC.ca

```
#####
# Definition for a Node.
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
        self.parent = None
#####
```

```
class Solution:
    def lowestCommonAncestor(self, p: 'Node', q: 'Node') -> 'Node':
        a, b = p, q
        while a != b:
            a = a.parent if a.parent else q
            b = b.parent if b.parent else p
        return a
```

```
#####
```

```
# ref: https://iamegeek.medium.com/leetcode-1650-lowest-common-ancestor-of-a-binary-tree-iii-6d008b93376c
```

```
def lowestCommonAncestor(self, a: 'Node', b: 'Node') -> 'Node':
    ancestors = set()

    while a is not None:
        ancestors.add(a)
        a = a.parent
```

```
    while b is not None:
        if b in ancestors:
            return b
        b = b.parent
```

```
# or another one
```

```
def lowestCommonAncestor(self, a: 'Node', b: 'Node') -> 'Node':
    pointerA, pointerB = a, b

    while pointerA != pointerB:

        pointerA = pointerA.parent if pointerA else b
        pointerB = pointerB.parent if pointerB else a

    return pointerA
```

◆ Quick Review

Key Insights

- Each node has a parent pointer, so you can traverse from any node to the root.
- By determining the depth of each node, you can “align” the two nodes to the same depth.
- When both nodes are aligned, traverse upward simultaneously until they meet.
- This approach guarantees finding the LCA with $O(h)$ time complexity, where h is the height of the tree.

Space & Time

Time Complexity: $O(h)$, where h is the height of the tree.

Space Complexity: $O(1)$

Solution

We first compute the depth of both nodes by moving up their parent pointers. If one node is deeper, move it upward until both nodes are at the same level. Then, move both nodes upward one step at a time until they meet; the meeting point is the lowest common ancestor. This approach uses constant extra space and leverages the parent pointer for an efficient solution.

DFS

Round 2 · High · Medium · 16 questions

DFS

□ #130 Surrounded Regions

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Union Find · Matrix
Lists: Denny Zhang

Problem

You are given an $m \times n$ matrix board containing letters 'X' and 'O', capture regions that are surrounded:

- **Connect:** A cell is connected to adjacent cells horizontally or vertically.
- **Region:** To form a region connect every 'O' cell.
- **Surround:** A region is surrounded if none of the 'O' cells in that region are on the edge of the board. Such regions are completely enclosed by 'X' cells.

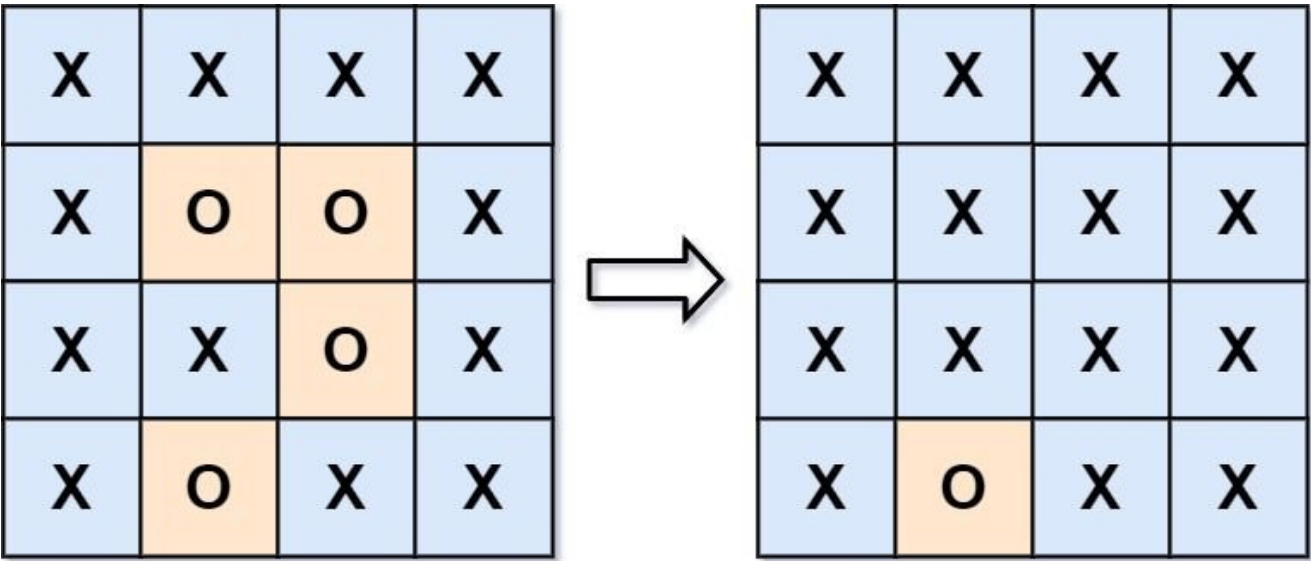
To capture a surrounded region, replace all 'O's with 'X's in-place within the original board. You do not need to return anything.

Example 1:

Input: board = [["X","X","X","X"], ["X","O","O","X"], ["X","X","O","X"], ["X","O","X","X"]]

Output: [["X","X","X","X"], ["X","X","X","X"], ["X","X","X","X"], ["X","O","X","X"]]

Explanation:



In the above diagram, the bottom region is not captured because it is on the edge of the board and cannot be surrounded.

Example 2:

Input: board = [["X"]]

Output: [["X"]]

Constraints:

- m == board.length
- n == board[i].length

- $1 \leq m, n \leq 200$
- `board[i][j]` is 'X' or 'O'.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Capture surrounded 'O' regions by replacing them with 'X'.
# A region is NOT captured if any 'O' in it touches the board edge. Right?
# In-place modification – no return value."
#
# "Border-connected 'O's and their inland 'O' neighbours are safe. Everything else
# gets flipped."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# First pass: DFS from every border 'O' and mark connected cells as safe ('S').
# Second pass: flip remaining 'O' to 'X'; restore safe cells back to 'O'.
# Two-pass flood fill from boundary – standard  $O(m \times n)$  approach.
# Time:  $O(m \times n)$ . Space:  $O(m \times n)$  recursion.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (reverse: mark safe, then flip)

```
# "DFS from every border 'O', marking connected 'O's as safe ('#').
# Second pass: 'O' → 'X' (captured), '#' → 'O' (safe restored).  $O(m \times n)$ ."
```

PHASE 4 — CLEAN CODE

```
class _130_SurroundedRegions:
    def solve(self, board: List[List[str]]) -> None:
        m, n = len(board), len(board[0])
        def mark_safe(r, c):
            if r < 0 or r >= m or c < 0 or c >= n or board[r][c] != 'O':
                return
            board[r][c] = '#'
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                mark_safe(r + dr, c + dc)
        for r in range(m):
            for c in [0, n - 1]:
                mark_safe(r, c)
        for c in range(n):
            for r in [0, m - 1]:
                mark_safe(r, c)
        for r in range(m):
```

```

for c in range(n):
    if board[r][c] == '0': board[r][c] = 'X'
    elif board[r][c] == '#': board[r][c] = '0'

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m \times n)$. Space $O(m \times n)$ stack. The reverse-DFS from borders is the key insight

—

instead of deciding which '0's are captured, decide which are SAFE and flip the rest.

Given more time I'd ask: what if the grid wraps (toroidal)? No borders → all '0's are safe."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def solve(self, board: list[list[str]]) -> None:
        if not board:
            return

        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(board)
        n = len(board[0])
        q = collections.deque()

        for i in range(m):
            for j in range(n):
                if i * j == 0 or i == m - 1 or j == n - 1:
                    if board[i][j] == '0':
                        q.append((i, j))
                        board[i][j] = '*'

        # Mark the grids that stretch from the four sides with '*'.
        while q:
            i, j = q.popleft()

```

```

for dx, dy in DIRS:
    x = i + dx
    y = j + dy
    if x < 0 or x == m or y < 0 or y == n:
        continue
    if board[x][y] != '0':
        continue
    q.append((x, y))
    board[x][y] = '*'

```

```

for row in board:
    for i, c in enumerate(row):
        row[i] = '0' if c == '*' else 'X'

```

• LeetDoocs

```

class Solution:
    def solve(self, board: List[List[str]]) -> None:
        def dfs(i: int, j: int):
            if not (0 <= i < m and 0 <= j < n and board[i][j] == "0"):
                return
            board[i][j] = "."
            for a, b in pairwise((-1, 0, 1, 0, -1)):
                dfs(i + a, j + b)

        m, n = len(board), len(board[0])

        for i in range(m):
            dfs(i, 0)
            dfs(i, n - 1)
        for j in range(n):
            dfs(0, j)
            dfs(m - 1, j)
        for i in range(m):
            for j in range(n):
                if board[i][j] == ".":
                    board[i][j] = "0"

                elif board[i][j] == "0":
                    board[i][j] = "X"

```

```

class Solution:
    def solve(self, board: List[List[str]]) -> None:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        m, n = len(board), len(board[0])
        p = list(range(m * n + 1))
        for i in range(m):
            for j in range(n):
                if board[i][j] == "0":
                    if i in (0, m - 1) or j in (0, n - 1):
                        p[find(i * n + j)] = find(m * n)
                    else:
                        for a, b in pairwise((-1, 0, 1, 0, -1)):
                            x, y = i + a, j + b
                            if board[x][y] == "0":
                                p[find(x * n + y)] = find(i * n + j)
            for i in range(m):
                for j in range(n):
                    if board[i][j] == "0" and find(i * n + j) != find(m * n):
                        board[i][j] = "X"

```

• SimplyLeet

```

def solve(board):
    if not board or not board[0]:
        return

    rows, cols = len(board), len(board[0])

    def dfs(r, c):
        # Return if out of bounds or current cell is not '0'
        if r < 0 or c < 0 or r >= rows or c >= cols or board[r][c] != '0':
            return

```

```

        # Mark the cell as safe
        board[r][c] = '*'
        # Explore four possible directions
        dfs(r + 1, c)
        dfs(r - 1, c)
        dfs(r, c + 1)
        dfs(r, c - 1)

    # Traverse the board borders
    for i in range(rows):

        if board[i][0] == '0':
            dfs(i, 0)
        if board[i][cols - 1] == '0':
            dfs(i, cols - 1)
    for j in range(cols):
        if board[0][j] == '0':
            dfs(0, j)
        if board[rows - 1][j] == '0':
            dfs(rows - 1, j)

    # Flip surrounded '0's and restore safe cells
    for i in range(rows):
        for j in range(cols):
            if board[i][j] == '0':
                board[i][j] = 'X'
            elif board[i][j] == '*':
                board[i][j] = '0'

```

• LC.ca

```

class Solution:
    def solve(self, board: List[List[str]]) -> None:
        """
        Do not return anything, modify board in-place instead.
        """

        def dfs(i, j):
            board[i][j] = '.'
            for a, b in [[0, -1], [0, 1], [1, 0], [-1, 0]]:
                x, y = i + a, j + b

```

```
        if 0 <= x < m and 0 <= y < n and board[x][y] == 'O':
            dfs(x, y)

m, n = len(board), len(board[0])
for i in range(m):
    for j in range(n):
        if board[i][j] == 'O' and (
            i == 0 or i == m - 1 or j == 0 or j == n - 1
        ):
            dfs(i, j)

for i in range(m):
    for j in range(n):
        if board[i][j] == 'O':
            board[i][j] = 'X'
        elif board[i][j] == '.':
            board[i][j] = 'O'
```

◆ Quick Review

Key Insights

- Regions connected to the border are not captured.
- Use DFS/BFS from border 'O's to mark safe cells.
- After marking, flip all remaining 'O's to 'X' and then revert the marked cells back to 'O'.

Space & Time

Time Complexity: $O(m * n)$

Space Complexity: $O(m * n)$ in the worst case due to recursion stack or queue storage for

DFS/BFS

Solution

The solution involves two main steps. First, traverse the board's borders and run a DFS or BFS from any encountered 'O', marking it (e.g., using ' ') to indicate that it should not be flipped. Next, go through each cell in the board: flip any remaining 'O' (i.e., not marked safe) to 'X' and change the temporary marker ' ' back to 'O'. This approach ensures only the regions completely surrounded by 'X' are captured.

DFS

□ #133 Clone Graph

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · DFS · BFS · Graph
Lists: Grind 169

Problem

Given a reference of a node in a connected undirected graph.

Return a deep copy (clone) of the graph.

Each node in the graph contains a value (int) and a list (List[Node]) of its neighbors.

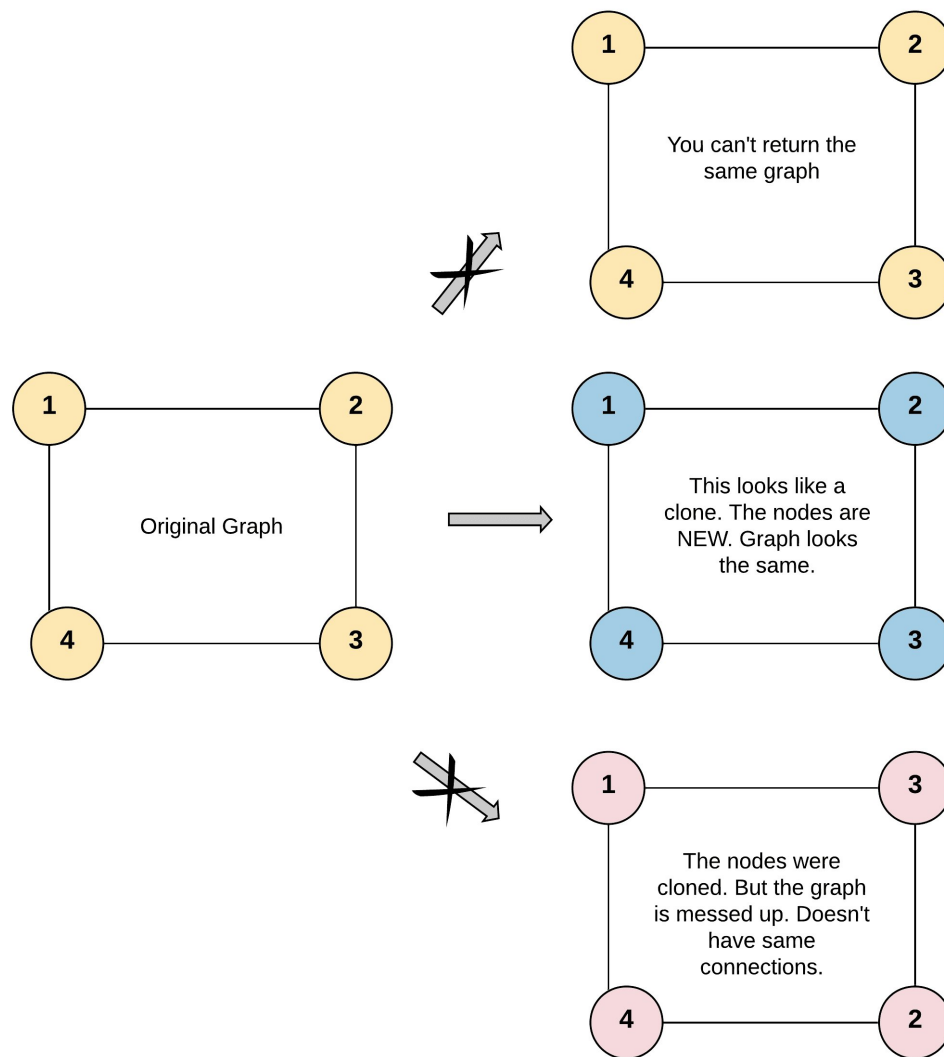
```
class Node {  
    public int val;  
    public List<Node> neighbors;  
}
```

Test case format:

For simplicity, each node's value is the same as the node's index (1-indexed). For example, the first node with `val == 1`, the second node with `val == 2`, and so on. The graph is represented in the test case using an adjacency list.

An adjacency list is a collection of unordered lists used to represent a finite graph. Each list describes the set of neighbors of a node in the graph.

The given node will always be the first node with $val = 1$. You must return the copy of the given node as a reference to the cloned graph.
Example 1:



Input: `adjList = [[2,4],[1,3],[2,4],[1,3]]`

Output: `[[2,4],[1,3],[2,4],[1,3]]`

Explanation: There are 4 nodes in the graph.

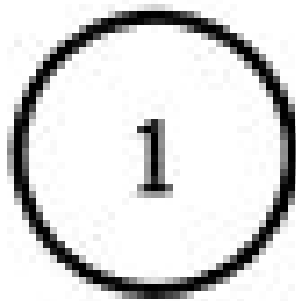
1st node (`val = 1`)'s neighbors are 2nd node (`val = 2`) and 4th node (`val = 4`).

2nd node (`val = 2`)'s neighbors are 1st node (`val = 1`) and 3rd node (`val = 3`).

3rd node (`val = 3`)'s neighbors are 2nd node (`val = 2`) and 4th node (`val = 4`).

4th node (`val = 4`)'s neighbors are 1st node (`val = 1`) and 3rd node (`val = 3`).

Example 2:



Input: `adjList = [[]]`

Output: `[[]]`

Explanation: Note that the input contains one empty list. The graph consists of only one node with `val = 1` and it does not have any neighbors.

Example 3:

Input: adjList = []

Output: []

Explanation: This an empty graph, it does not have any nodes.

Constraints:

- The number of nodes in the graph is in the range [0, 100].
- $1 \leq \text{Node.val} \leq 100$
- Node.val is unique for each node.
- There are no repeated edges and no self-loops in the graph.
- The Graph is connected and all nodes can be visited starting from the given node.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Deep copy an undirected connected graph. No shared references with original. Right?"

Graphs can have cycles – unlike trees, so we need a visited map."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

BFS from the given node: maintain a map old_node -> new_node clone.

When visiting neighbors, create clones on first sight and enqueue them.

Straightforward graph copy – already the standard $O(V+E)$ approach.

Time: $O(V + E)$. Space: $O(V)$ for the clone map and queue.

Should I go ahead and optimize?"

```
class _133_CloneGraph:
    def cloneGraph(self, node: 'Node') -> 'Node':
        if not node:
            return None
        cloned: dict = {}
        def clone(n):
```

```

        if n in cloned:
            return cloned[n]
        copy = Node(n.val)
        cloned[n] = copy # mark before recursing to handle cycles
        copy.neighbors = [clone(nb) for nb in n.neighbors]
        return copy
    return clone(node)

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(V+E)$. Space $O(V)$ for the visited map. Mark node before recursing into neighbours –

without this, cycles cause infinite recursion. Same visited-map pattern handles any graph.

Given more time I'd ask: clone a directed graph? Same approach – just don't add reverse edges."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def cloneGraph(self, node: 'Node') -> 'Node':
        if not node:
            return None

        q = collections.deque([node])
        map = {node: Node(node.val)}

        while q:
            u = q.popleft()

            for v in u.neighbors:
                if v not in map:
                    map[v] = Node(v.val)
                    q.append(v)
                map[u].neighbors.append(map[v])

        return map[node]

```

```

class Solution:
    def cloneGraph(self, node: 'Node') -> 'Node':
        if not node:
            return None
        if node in self.map:
            return self.map[node]

        newNode = Node(node.val, [])
        self.map[node] = newNode

        for neighbor in node.neighbors:
            self.map[node].neighbors.append(self.cloneGraph(neighbor))

        return newNode

map = {}

```

• LeetDoocs

```

"""
# Definition for a Node.
class Node:
    def __init__(self, val = 0, neighbors = None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []
"""

from typing import Optional

```

```

class Solution:
    def cloneGraph(self, node: Optional["Node"]) -> Optional["Node"]:
        def dfs(node):
            if node is None:
                return None
            if node in g:
                return g[node]
            cloned = Node(node.val)
            g[node] = cloned

```

```

        for nxt in node.neighbors:
            cloned.neighbors.append(dfs(nxt))
        return cloned

    g = defaultdict()
    return dfs(node)

```

• SimplyLeet

```

# Definition for a Node.
class Node:
    def __init__(self, val, neighbors=None):
        self.val = val
        # Initialize neighbors with empty list if None is provided
        self.neighbors = neighbors if neighbors is not None else []

def cloneGraph(node: 'Node') -> 'Node':
    # Hash map to track original nodes and their corresponding clones
    old_to_new = {}

    def dfs(current):
        # Base case: if the current node is None, return None
        if current is None:
            return None

        # If the node has already been cloned, return the clone to avoid
        # duplication and cycle issues
        if current in old_to_new:
            return old_to_new[current]

        # Create a clone for the current node
        copy = Node(current.val)

        old_to_new[current] = copy
        # Recursively clone all neighbors
        for neighbor in current.neighbors:
            copy.neighbors.append(dfs(neighbor))
        return copy

    return dfs(node)

```

• LC.ca

```
#####
# Definition for a Node.
class Node:
    def __init__(self, val = 0, neighbors = None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []
#####

class Solution:

    def cloneGraph(self, node: 'Node') -> 'Node':
        visited = defaultdict()

        def clone(node):
            if node is None:
                return None
            if node in visited:
                return visited[node]
            c = Node(node.val)
            visited[node] = c

            for e in node.neighbors:
                c.neighbors.append(clone(e))
            return c

        return clone(node)
```

◆ Quick Review

Key Insights

- The graph is undirected and connected.
- Each node must be cloned exactly once to prevent cycles and repetitions.
- Use a mapping/dictionary to keep track of cloned nodes.

- A depth-first search (DFS) or breadth-first search (BFS) can be used to traverse and clone the graph.
- Handle edge cases where the graph might be empty.

Space & Time

Time Complexity: $O(N + E)$, where N is the number of nodes and E is the number of edges, because each node and each edge are traversed once.

Space Complexity: $O(N)$, to store the mapping from original nodes to their clones, and for the recursion/queue used in traversal.

Solution

We employ a depth-first search (DFS) approach to clone the graph. The main idea is to create a mapping (hash table) that links each original node to its corresponding cloned node. During the DFS traversal, if a node is encountered for the first time, a new node is created and stored in the mapping; then, the algorithm recursively clones all of its neighbors. For an already visited node, the cloned node is directly returned from the

mapping. This ensures that every node is cloned only once and correctly handles cycles in the graph.

DFS

□ #200 Number of Islands

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Union Find · Matrix
Lists: Grind 169

Problem

Given an $m \times n$ 2D binary grid `grid` which represents a map of '1's (land) and '0's (water), return the number of islands.

An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:

```
Input: grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","0","0"],
  ["0","0","0","0","0"]
]
Output: 1
```

Example 2:

```

Input: grid = [
["1","1","0","0","0"],
["1","1","0","0","0"],
["0","0","1","0","0"],
["0","0","0","1","1"]
]
Output: 3

```

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 300`
- `grid[i][j]` is '0' or '1'.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count connected components of '1's in a binary grid. Diagonal connections don't count. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Double loop over every grid cell. Whenever I see an unvisited '1',
run DFS/BFS to mark the entire connected component sunk, then increment island count.

Each cell visited once – correct and actually $O(m \times n)$ already.

Time: $O(m \times n)$. Space: $O(m \times n)$ worst-case recursion stack or queue.

The flood-fill is the right approach; I'll keep it clean with in-place marking."

```
class _200_NumberOfIslands:
```

```
    def numIslands(self, grid: List[List[str]]) -> int:
```

```
        m, n = len(grid), len(grid[0])
```

```
        def dfs(r, c):
```

```
            if r < 0 or r >= m or c < 0 or c >= n or grid[r][c] != '1':
```

```
                return
```

```
            grid[r][c] = '#' # mark visited by flooding
```

```
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
```

```
                dfs(r + dr, c + dc)
```

```

count = 0
for r in range(m):
    for c in range(n):
        if grid[r][c] == '1':
            dfs(r, c)
            count += 1
return count

```

PHASE 5 — TEST & DEBUG

1-island grid: DFS floods the whole island. count=1 ✓. 3-island: count=3 ✓.

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(m*n)$ stack. Flooding in-place avoids a separate visited set.

BFS alternative: use a queue instead of recursion – better for very large grids (no stack overflow).

Union-Find alternative: $O(m*n * \alpha(m*n))$ – useful when islands merge dynamically.

Given more time I'd ask: restore the grid after counting?

Keep a separate visited set or restore '#' → '1' after each DFS."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def numIslands(self, grid: list[list[str]]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(grid)
        n = len(grid[0])

        def bfs(r, c):
            q = collections.deque([(r, c)])
            grid[r][c] = '2' # Mark '2' as visited.
            while q:

                i, j = q.popleft()
                for dx, dy in DIRS:
                    x = i + dx
                    y = j + dy
                    if x < 0 or x == m or y < 0 or y == n:
                        continue
                    if grid[x][y] != '1':
                        continue
                    q.append((x, y))
                    grid[x][y] = '2' # Mark '2' as visited.

```

```

ans = 0

for i in range(m):
    for j in range(n):
        if grid[i][j] == '1':
            bfs(i, j)
            ans += 1

return ans

```

```

class Solution:
    def numIslands(self, grid: list[list[str]]) -> int:
        m = len(grid)
        n = len(grid[0])

        def dfs(i: int, j: int) -> None:
            if i < 0 or i == m or j < 0 or j == n:
                return
            if grid[i][j] != '1':
                return

            grid[i][j] = '2' # Mark '2' as visited.
            dfs(i + 1, j)
            dfs(i - 1, j)
            dfs(i, j + 1)
            dfs(i, j - 1)

ans = 0

for i in range(m):

```

```

    for j in range(n):
        if grid[i][j] == '1':
            dfs(i, j)
            ans += 1

return ans

```

```

for j in range(n):

```

```

    if grid[i][j] == '1':
        dfs(i, j)
        ans += 1

```

```

return ans

```

• LeetDoocs

```
class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def dfs(i, j):
            grid[i][j] = '0'
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and grid[x][y] == '1':
                    dfs(x, y)

        ans = 0
```

```
        dirs = (-1, 0, 1, 0, -1)
        m, n = len(grid), len(grid[0])
        for i in range(m):
            for j in range(n):
                if grid[i][j] == '1':
                    dfs(i, j)
                    ans += 1
        return ans
```

```
class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def bfs(i, j):
            grid[i][j] = '0'
            q = deque([(i, j)])
            while q:
                i, j = q.popleft()
                for a, b in pairwise(dirs):
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n and grid[x][y] == '1':
```

```
                        q.append((x, y))
                        grid[x][y] = 0
```

```
        ans = 0
        dirs = (-1, 0, 1, 0, -1)
        m, n = len(grid), len(grid[0])
        for i in range(m):
            for j in range(n):
                if grid[i][j] == '1':
                    bfs(i, j)
```



```

        ans += 1
    return ans

```

```

class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def find(x):
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        dirs = (0, 1, 0)
        m, n = len(grid), len(grid[0])
        p = list(range(m * n))

        for i in range(m):
            for j in range(n):
                if grid[i][j] == '1':
                    for a, b in pairwise(dirs):
                        x, y = i + a, j + b
                        if x < m and y < n and grid[x][y] == '1':
                            p[find(i * n + j)] = find(x * n + y)

        return sum(
            grid[i][j] == '1' and i * n + j == find(i * n + j)
            for i in range(m)

            for j in range(n)
        )

```

• SimplyLeet

```

# Function to count the number of islands in the grid
def numIslands(grid):
    # If the grid is empty, return 0
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])
    islands = 0

    # Depth-First Search helper to mark the entire island as visited
    def dfs(r, c):

```

```

# Check if the current cell is out of bounds or water
if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] == "0":
    return
# Mark the cell as visited by setting it to water
grid[r][c] = "0"
# Explore all four adjacent cells (up, down, left, right)
dfs(r + 1, c)
dfs(r - 1, c)
dfs(r, c + 1)
dfs(r, c - 1)

```

```

# Iterate over every cell in the grid
for r in range(rows):
    for c in range(cols):
        # If a land cell is found, conduct DFS to visit the entire island
        if grid[r][c] == "1":
            islands += 1
            dfs(r, c)

return islands

```

• LC.ca

```

# dfs
class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def dfs(i, j):
            if not (0 <= i < m and 0 <= j < n and grid[i][j] == '1'):
                return
            grid[i][j] = '0'
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                dfs(x, y)

```

```

ans = 0
dirs = (-1, 0, 1, 0, -1)
m, n = len(grid), len(grid[0])
for i in range(m):
    for j in range(n):
        if grid[i][j] == '1':
            dfs(i, j)
            ans += 1
return ans

```

```
#####
```

```

# bfs
from collections import deque

class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        if not grid:
            return 0

```

```

m, n = len(grid), len(grid[0])
directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]
islands = 0

```

```

for i in range(m):
    for j in range(n):
        if grid[i][j] == "1":
            islands += 1
            q = deque([(i, j)]) # no need to reset q, q already drained

```

```
from previous bfs
```

```

        while q:
            x, y = q.popleft()
            for dx, dy in directions:
                nx, ny = x + dx, y + dy
                if 0 <= nx < m and 0 <= ny < n and grid[nx][ny] ==
"1":
                    q.append((nx, ny))
                    grid[nx][ny] = "0" # mark as visited

        return islands

#####

```

```

# union find
# similar to UF in https://leetcode.ca/2016-09-30-305-Number-of-Islands-II/
class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        def check(i, j):
            return 0 <= i < m and 0 <= j < n and grid[i][j] == "1" # Check
for "1" instead of 1

```

```

    def find(x):
        if p[x] != x:

```

```

            p[x] = find(p[x])
        return p[x]

```

```

m, n = len(grid), len(grid[0])
p = list(range(m * n))
cur = 0 # Initialize cur as 0 instead of n
for i in range(m):
    for j in range(n):
        if grid[i][j] == "1":
            cur += 1 # Increment cur when encountering "1"

```

```

        for x, y in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            if check(i + x, j + y) and find(i * n + j) != find((i
+ x) * n + j + y):
                p[find(i * n + j)] = find((i + x) * n + j + y)
                cur -= 1

    return cur

```

```
#####
```

```

'''
>>> a = set()
>>>
>>> a |= {(1, 1)}
>>> a
{(1, 1)}
>>>
>>> a |= {(2, 2)}
>>> a
{(1, 1), (2, 2)}

```

```

>>>
>>> a.add((3, 3))
>>> a
{(1, 1), (3, 3), (2, 2)}
>>>
>>> (1,1) in a
True
>>> (10,10) in a
False
'''

```

```

class Solution(object):
    def numIslands(self, grid):
        """
        :type grid: List[List[str]]
        :rtype: int
        """
        visited = set()
        ans = 0

        def dfs(grid, i, j, visited):

            if i < 0 or i >= len(grid) or j < 0 or j >= len(grid[0]) or grid[i][j] =
            = "0" or (i, j) in visited:
                return False
            visited |= {(i, j)}
            for di, dj in [(-1, 0), (1, 0), (0, 1), (0, -1)]:
                newi, newj = i + di, j + dj
                dfs(grid, newi, newj, visited)
            return True

        for i in range(0, len(grid)):
            for j in range(0, len(grid[0])):

                if dfs(grid, i, j, visited):
                    ans += 1
        return ans

```

◆ Quick Review

Key Insights

- Use graph traversal (DFS/BFS) to explore connected components of land.
- Iterate over the matrix and start a traversal (e.g., DFS) when encountering an unvisited "1".
- During traversal, mark visited cells to avoid duplicate counting.

- Each DFS/BFS call completely explores one island.
- Alternative approach can use Union Find to group connected lands.

Space & Time

Time Complexity: $O(m * n)$, as each cell is visited at most once.

Space Complexity: $O(m * n)$ in the worst-case scenario due to recursion stack (DFS) or auxiliary data structures.

Solution

We solve the problem using a DFS-based approach. The algorithm iterates over each cell in the grid; when a "1" (land) is encountered, the DFS is initiated to mark the entire island (all connected "1"s) as visited (by setting them to "0"). This prevents recounting the same island. The island count is incremented each time a new island is discovered and fully explored.

DFS

□ #207 Course Schedule

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
DFS · BFS · Graph · Topological Sort
Lists: Grind 169

Problem

There are a total of `numCourses` courses you have to take, labeled from 0 to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you must take course `bi` first if you want to take course `ai`.

- For example, the pair `[0, 1]`, indicates that to take course 0 you have to first take course 1.

Return `true` if you can finish all courses.

Otherwise, return `false`.

Example 1:

Input: `numCourses = 2, prerequisites = [[1,0]]`

Output: `true`

Explanation: There are a total of 2 courses to take.

To take course 1 you should have finished course 0. So it is possible.

Example 2:

Input: numCourses = 2, prerequisites = [[1,0],[0,1]]

Output: false

Explanation: There are a total of 2 courses to take.

To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.

Constraints:

- $1 \leq \text{numCourses} \leq 2000$
- $0 \leq \text{prerequisites.length} \leq 5000$
- $\text{prerequisites}[i].\text{length} == 2$
- $0 \leq a_i, b_i < \text{numCourses}$
- All the pairs $\text{prerequisites}[i]$ are unique.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Can all courses be finished given prerequisites (directed edges)?

Equivalent to: does the directed graph have a cycle? Return True if no cycle. Right?"

#

"[[1,0],[0,1]]: 0→1 and 1→0 form a cycle → False ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Build adjacency + indegree; Kahn BFS peels zero-indegree courses layer by layer.

If we finish fewer than numCourses nodes, a cycle exists ? return false.

Same topological sort as Course Schedule II, but only need boolean completion.

Time: $O(V + E)$. Space: $O(V + E)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (3-colour DFS cycle detection)

"Color each node: 0=unvisited, 1=in current DFS path (grey), 2=fully processed (black).

Cycle exists if we encounter a grey node. $O(V+E)$."

PHASE 4 — CLEAN CODE

class _207_CourseSchedule:

def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:

```

graph = defaultdict(list)
for course, pre in prerequisites:
    graph[pre].append(course)
color = [0] * numCourses
def has_cycle(node):
    if color[node] == 1: return True # grey → back edge → cycle
    if color[node] == 2: return False # black → already processed
    color[node] = 1
    for nb in graph[node]:
        if has_cycle(nb): return True
    color[node] = 2
    return False
return not any(has_cycle(c) for c in range(numCourses))

```

PHASE 5 — TEST & DEBUG

[[1,0],[0,1]]: has_cycle(0)→grey. has_cycle(1)→grey. has_cycle(0)→grey(already) → cycle! ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(V+E)$. Space $O(V+E)$. The 3-colour scheme distinguishes 'seen in current path' from 'fully done'.

Kahn's BFS alternative (in-degree tracking): count = courses processed; if count < numCourses → cycle.

Given more time I'd ask: Course Schedule II (#210)? Same DFS, collect reverse post-order."

Community Solutions (Python)

• WalkCC

```
from enum import Enum
```

```
class State(Enum):
```

```
    INIT = 0
```

```
    VISITING = 1
```

```
    VISITED = 2
```

```
class Solution:
```

```

def canFinish(self, numCourses: int, prerequisites: list[list[int]]) -> bool
:
    graph = [[] for _ in range(numCourses)]
    states = [State.INIT] * numCourses

    for v, u in prerequisites:
        graph[u].append(v)

    def hasCycle(u: int) -> bool:
        if states[u] == State.VISITING:
            return True

        if states[u] == State.VISITED:
            return False
        states[u] = State.VISITING
        if any(hasCycle(v) for v in graph[u]):
            return True
        states[u] = State.VISITED
        return False

    return not any(hasCycle(i) for i in range(numCourses))

```

• LeetDoocs

```

class Solution:
    def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:
        g = [[] for _ in range(numCourses)]
        indeg = [0] * numCourses
        for a, b in prerequisites:
            g[b].append(a)
            indeg[a] += 1
        q = [i for i, x in enumerate(indeg) if x == 0]
        for i in q:
            numCourses -= 1

            for j in g[i]:
                indeg[j] -= 1
                if indeg[j] == 0:
                    q.append(j)
        return numCourses == 0

```

• SimplyLeet

```

from collections import deque

def canFinish(numCourses, prerequisites):
    # Create a list to hold the number of prerequisites (indegree) for each
    # course.
    indegree = [0] * numCourses
    # Create an adjacency list for each course.
    graph = {i: [] for i in range(numCourses)}

    # Build the graph and update indegree for each course.
    for course, prereq in prerequisites:

        graph[prereq].append(course)
        indegree[course] += 1

    # Initialize a queue with courses that have no prerequisites.
    queue = deque([i for i in range(numCourses) if indegree[i] == 0])

    # Counter for courses that can be completed.
    visited = 0

    while queue:

        course = queue.popleft() # Remove course with no remaining
        # prerequisites.
        visited += 1 # Increment count of completed courses.
        # For each neighbor course, reduce its indegree.
        for neighbor in graph[course]:
            indegree[neighbor] -= 1 # Remove dependency.
            # If no more prerequisites, add to queue.
            if indegree[neighbor] == 0:
                queue.append(neighbor)

    # If we processed all courses, it's possible to finish.

    return visited == numCourses

# Example usage:
print(canFinish(2, [[1,0]])) # Expected output: True
print(canFinish(2, [[1,0],[0,1]])) # Expected output: False

```

• LC.ca

```

'''
>>> from collections import defaultdict

>>> a = defaultdict(int)
>>> a
defaultdict(<type 'int'>, {})
>>>
>>> a['hehehe']
0
>>> a
defaultdict(<type 'int'>, {'hehehe': 0})

>>> a = defaultdict(list)
>>> a
defaultdict(<type 'list'>, {})
>>> a['hehehe']
[]
>>> a
defaultdict(<type 'list'>, {'hehehe': []})

```

```

'''
class Solution:
    def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:
        g = defaultdict(list)
        indeg = [0] * numCourses
        for a, b in prerequisites:
            g[b].append(a)
            indeg[a] += 1
        cnt = 0
        q = deque([i for i, v in enumerate(indeg) if v == 0])

```

```
while q:
    i = q.popleft()
    cnt += 1
    for j in g[i]:
        indeg[j] -= 1
        if indeg[j] == 0:
            q.append(j)
return cnt == numCourses
```

◆ Quick Review

Key Insights

- This problem can be modeled as a directed graph where courses are nodes and prerequisites are edges.
- If the graph has a cycle, then it is not possible to complete all courses.
- We can detect cycles using either Topological Sort (BFS/Kahn's algorithm) or DFS-based cycle detection.
- The time complexity is $O(V + E)$, where V is the number of courses and E is the number of prerequisite pairs.

Space & Time

Time Complexity: $O(V + E)$

Space Complexity: $O(V + E)$

Solution

We solve the problem using the Topological Sort algorithm (BFS/Kahn's algorithm). First, we build a graph (using an adjacency list) and track the indegree (number of prerequisites) for each course. We then find all courses with an indegree of zero (courses with no prerequisites) and add them to a processing queue. While the queue is not empty, we remove a course from the queue, reduce the indegree for its neighboring courses, and if any neighbor's indegree becomes zero, we add it to the queue as well. If we successfully process all courses, then it means that the graph was acyclic and a valid schedule exists, returning true. If not, a cycle exists, and we return false.

DFS

□ #210 Course Schedule II

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode

DFS · BFS · Graph · Topological Sort

Lists: Grind 169

Problem

There are a total of `numCourses` courses you have to take, labeled from 0 to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you must take course `bi` first if you want to take course `ai`.

- For example, the pair `[0, 1]`, indicates that to take course 0 you have to first take course 1.

Return the ordering of courses you should take to finish all courses. If there are many valid answers, return any of them. If it is impossible to finish all courses, return an empty array.

Example 1:

Input: `numCourses = 2, prerequisites = [[1,0]]`

Output: `[0,1]`

Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So the correct course order is `[0,1]`.

Example 2:

Input: numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]

Output: [0,2,1,3]

Explanation: There are a total of 4 courses to take. To take course 3 you should have finished both courses 1 and 2. Both courses 1 and 2 should be taken after you finished course 0.

So one correct course order is [0,1,2,3]. Another correct ordering is [0,2,1,3].

Example 3:

Input: numCourses = 1, prerequisites = []

Output: [0]

Constraints:

- $1 \leq \text{numCourses} \leq 2000$
- $0 \leq \text{prerequisites.length} \leq \text{numCourses} * (\text{numCourses} - 1)$
- $\text{prerequisites}[i].\text{length} == 2$
- $0 \leq a_i, b_i < \text{numCourses}$
- $a_i \neq b_i$
- All the pairs $[a_i, b_i]$ are distinct.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return a valid course ordering. Return [] if impossible (cycle exists). Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Topological sort via Kahn's BFS: compute indegree of every course,

enqueue zero-indegree nodes, peel layers while decrementing neighbors.

If we process all n courses, ordering exists; otherwise a cycle remains.

Time: $O(V + E)$. Space: $O(V + E)$ for adjacency and indegree arrays.

```
# Should I go ahead and optimize?"
```

```
class _210_CourseScheduleII:
    def findOrder(self, numCourses: int, prerequisites: List[List[int]]) ->
List[int]:
    graph = defaultdict(list)
    for course, pre in prerequisites:
        graph[pre].append(course)
    color = [0] * numCourses
    order = []
    def dfs(node) -> bool:
        if color[node] == 1: return False # cycle
        if color[node] == 2: return True # already done
        color[node] = 1
        for nb in graph[node]:
            if not dfs(nb): return False
        color[node] = 2
        order.append(node) # post-order: all descendants before this node
        return True
    for c in range(numCourses):
        if not dfs(c):
            return []
    return order[::-1] # reverse post-order = topological order
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(V+E)$. Space $O(V+E)$. Post-order append then reverse = topological sort.

Given more time I'd ask: Kahn's BFS? Build in-degree map, process zero-in-degree nodes first.

Both approaches are $O(V+E)$ – DFS is recursive, Kahn's is iterative."

Community Solutions (Python)

• WalkCC

```
from enum import Enum
```

```
class State(Enum):
```

```
    INIT = 0
```

```
    VISITING = 1
```

```
    VISITED = 2
```

```
class Solution:
```

```
def findOrder(
    self,
    numCourses: int,
    prerequisites: list[list[int]],
) -> list[int]:
    ans = []
    graph = [[] for _ in range(numCourses)]
    states = [State.INIT] * numCourses

    for v, u in prerequisites:
```

```
    graph[u].append(v)
```

```
def hasCycle(u: int) -> bool:
    if states[u] == State.VISITING:
        return True
    if states[u] == State.VISITED:
        return False
    states[u] = State.VISITING
    if any(hasCycle(v) for v in graph[u]):
        return True
```

```
    states[u] = State.VISITED
    ans.append(u)
    return False
```

```
if any(hasCycle(i) for i in range(numCourses)):
    return []
```

```
return ans[::-1]
```

```
class Solution:
    def findOrder(
        self,
        numCourses: int,
        prerequisites: list[list[int]],
    ) -> list[int]:
        ans = []
        graph = [[] for _ in range(numCourses)]
        inDegrees = [0] * numCourses
        q = collections.deque()
```

```

# Build the graph.
for v, u in prerequisites:
    graph[u].append(v)
    inDegrees[v] += 1

# Perform topological sorting.
q = collections.deque([i for i, d in enumerate(inDegrees) if d == 0])

while q:

    u = q.popleft()
    ans.append(u)
    for v in graph[u]:
        inDegrees[v] -= 1
        if inDegrees[v] == 0:
            q.append(v)

return ans if len(ans) == numCourses else []

```

• LeetDoocs

```

class Solution:
    def findOrder(self, numCourses: int, prerequisites: List[List[int]]) -> List[int]:
        g = defaultdict(list)
        indeg = [0] * numCourses
        for a, b in prerequisites:
            g[b].append(a)
            indeg[a] += 1
        ans = []
        q = deque(i for i, x in enumerate(indeg) if x == 0)
        while q:
            i = q.popleft()
            ans.append(i)
            for j in g[i]:
                indeg[j] -= 1
                if indeg[j] == 0:
                    q.append(j)
        return ans if len(ans) == numCourses else []

```

• SimplyLeet

```
# Python solution using BFS (Kahn's Algorithm)
from collections import deque

def findOrder(numCourses, prerequisites):
    # Build the graph and the in-degree array
    in_degree = [0] * numCourses
    graph = {i: [] for i in range(numCourses)}
    for course, pre in prerequisites:
        graph[pre].append(course)
        in_degree[course] += 1

    # Initialize the queue with all courses having zero in-degree
    queue = deque([i for i in range(numCourses) if in_degree[i] == 0])
    order = []

    # Process courses
    while queue:
        course = queue.popleft()
        order.append(course)
        for neighbor in graph[course]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    # If the ordering includes all courses, return it; otherwise, return an
    # empty list
    return order if len(order) == numCourses else []

# Example usage:
# print(findOrder(4, [[1,0],[2,0],[3,1],[3,2]]))
```

• LC.ca

```

class Solution:
    def findOrder(self, numCourses: int, prerequisites: List[List[int]]) -> List[int]:
        g = defaultdict(list)
        indeg = [0] * numCourses
        for a, b in prerequisites:
            g[b].append(a)
            indeg[a] += 1
        q = deque([i for i, v in enumerate(indeg) if v == 0])
        ans = [] # added from previous question
        while q:
            i = q.popleft()
            ans.append(i)
            # assumption is only one path
            # as in question 'You may assume that there are no duplicate edges
            in the input prerequisites.'
            for j in g[i]:
                indeg[j] -= 1
                if indeg[j] == 0:
                    q.append(j)
        return ans if len(ans) == numCourses else []

```

◆ Quick Review

Key Insights

- This is a topological sort problem on a directed graph.
- The courses are nodes, and prerequisites form directed edges.
- We can solve the problem by either:
- Using DFS with cycle detection and a stack for topological ordering.

- Using BFS (Kahn's Algorithm) to process nodes with zero in-degrees.
- If at any point no node with a zero in-degree is available, a cycle exists and no valid order can be produced.

Space & Time

Time Complexity: $O(V + E)$ where V is the number of courses and E is the number of prerequisite pairs.

Space Complexity: $O(V + E)$ for storing the graph and in-degree array.

Solution

We treat the problem as a topological sort on a directed acyclic graph (DAG). For a BFS approach using Kahn's algorithm: Build an adjacency list for the graph and an in-degree array to record the number of prerequisites (incoming edges) for each course. Initialize a queue with all courses that have an in-degree of zero (i.e., courses that can be taken without prerequisites). Dequeue nodes from the queue, add them to the ordering, and reduce the in-degree of their neighbors. If a neighbor's in-degree becomes zero, add it to the queue. If

the final ordering includes all courses, return the ordering; otherwise, return an empty array indicating a cycle. The chosen data structures are: A list (or array) for the in-degree counts. A dictionary (or array of lists) for the adjacency list. A queue to process nodes with zero in-degree.

DFS

□ ★ #261 Graph Valid Tree ★

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode

DFS · BFS · Union Find · Graph

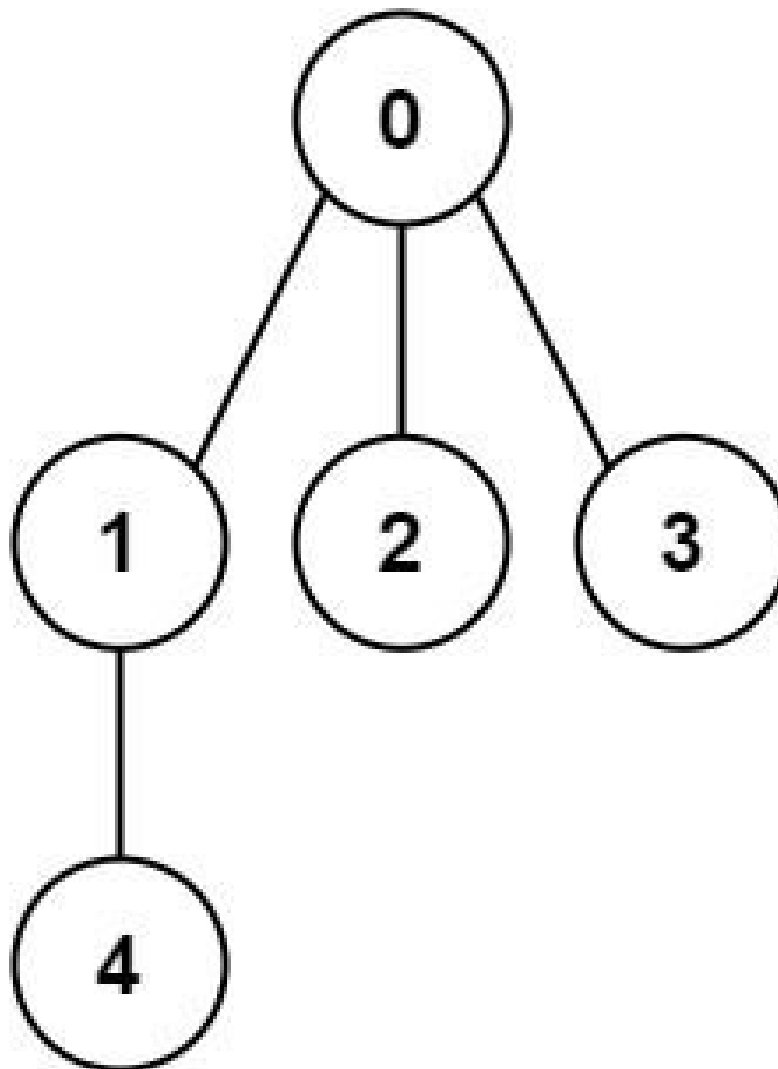
Lists: ★ Premium | Grind 169 | Premium 98

Problem

You have a graph of n nodes labeled from 0 to $n - 1$. You are given an integer n and a list of edges where $\text{edges}[i] = [a_i, b_i]$ indicates that there is an undirected edge between nodes a_i and b_i in the graph.

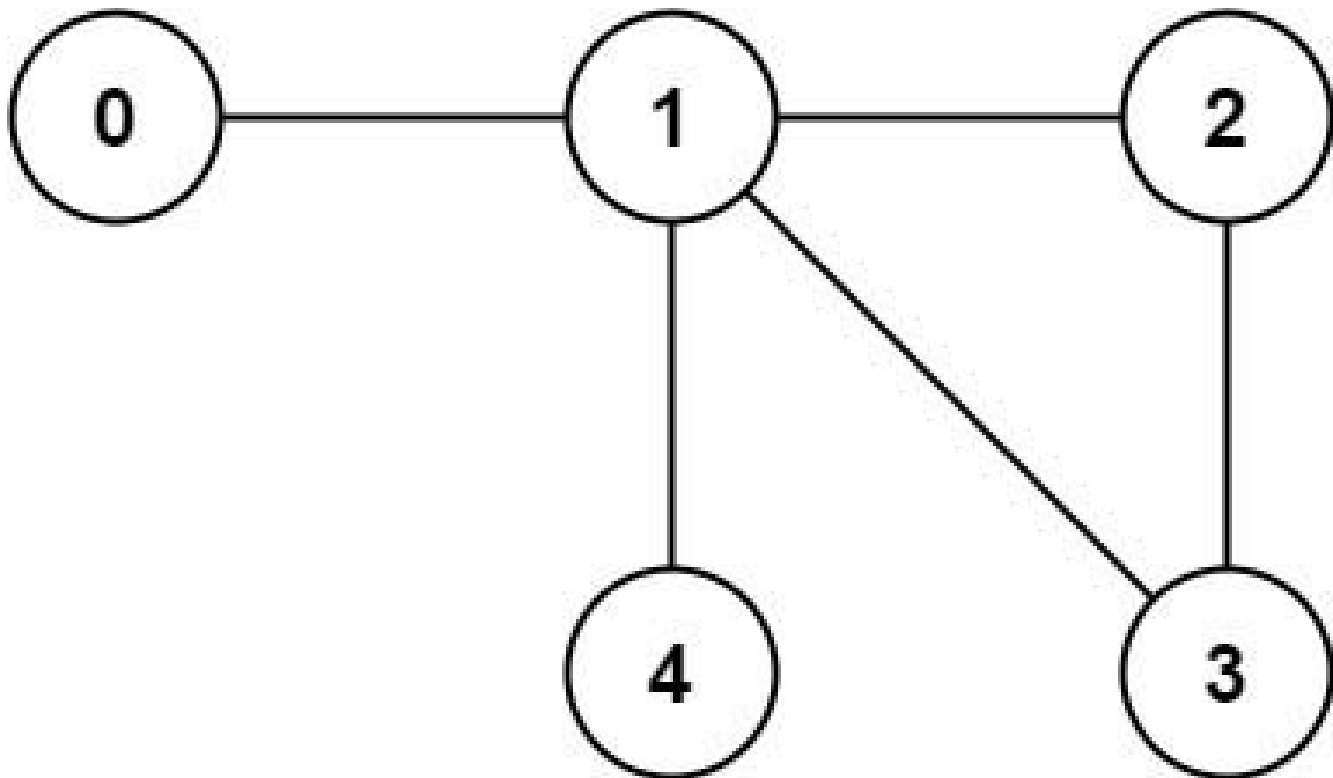
Return true if the edges of the given graph make up a valid tree, and false otherwise.

Example 1:



Input: $n = 5$, $edges = [[0,1],[0,2],[0,3],[1,4]]$
Output: true

Example 2:



Input: $n = 5$, $edges = [[0,1],[1,2],[2,3],[1,3],[1,4]]$
Output: false

Constraints:

- $1 \leq n \leq 2000$
- $0 \leq edges.length \leq 5000$
- $edges[i].length == 2$
- $0 \leq a_i, b_i < n$
- $a_i \neq b_i$
- There are no self-loops or repeated edges.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Valid tree: connected, acyclic, exactly $n-1$ edges. Right?

Any two of these imply the third for n nodes."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Build adjacency from edges, run DFS from node 0 marking visited.

After DFS, verify every node was visited and total edges equals $n-1$.

Catches disconnected graphs and cycles in one pass.

Time: $O(n)$. Space: $O(n)$ for adjacency + visited.

Should I go ahead and optimize?"

```
class _261_GraphValidTree:
    def validTree(self, n: int, edges: List[List[int]]) -> bool:
        if len(edges) != n - 1:
            return False
        graph = defaultdict(list)
        for a, b in edges:
            graph[a].append(b)
            graph[b].append(a)
        visited = set()
        def dfs(node, parent):
            visited.add(node)
            for nb in graph[node]:
                if nb == parent: continue
                if nb in visited: return False
                if not dfs(nb, node): return False
            return True
        return dfs(0, -1) and len(visited) == n
```

PHASE 5 — TEST & DEBUG

$n=5$, edges=[[0,1],[0,2],[0,3],[1,4]]: 4 edges== $n-1$ ✓. DFS from 0 visits all 5. No cycle. → True ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(V+E)=O(n)$. Space $O(n)$. Early return if edges $\neq n-1$ is a fast filter.

Union-Find alternative: $O(n * \alpha(n))$ – merge edges, detect cycle if already connected.

Given more time I'd ask: disconnected forest (multiple trees)?

Return number of components instead – skip the len(edges)== $n-1$ check."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def validTree(self, n: int, edges: list[list[int]]) -> bool:
        if n == 0 or len(edges) != n - 1:
            return False

        graph = [[] for _ in range(n)]
        q = collections.deque([0])
        seen = {0}

        for u, v in edges:

            graph[u].append(v)
            graph[v].append(u)

        while q:
            u = q.popleft()
            for v in graph[u]:
                if v not in seen:
                    q.append(v)
                    seen.add(v)

        return len(seen) == n


class UnionFind:
    def __init__(self, n: int):
        self.count = n
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self._find(u)
        j = self._find(v)
        if i == j:
```



```

        return
    if self.rank[i] < self.rank[j]:
        self.id[i] = j
    elif self.rank[i] > self.rank[j]:
        self.id[j] = i
    else:
        self.id[i] = j
        self.rank[j] += 1
    self.count -= 1

```

```

def _find(self, u: int) -> int:
    if self.id[u] != u:
        self.id[u] = self._find(self.id[u])
    return self.id[u]

```

```

class Solution:
    def validTree(self, n: int, edges: list[list[int]]) -> bool:
        if n == 0 or len(edges) != n - 1:
            return False

```

```

        uf = UnionFind(n)

        for u, v in edges:
            uf.unionByRank(u, v)

        return uf.count == 1

```

• LeetDoocs

```

class Solution:
    def validTree(self, n: int, edges: List[List[int]]) -> bool:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        p = list(range(n))
        for a, b in edges:
            pa, pb = find(a), find(b)

```

```

        if pa == pb:
            return False
        p[pa] = pb
        n -= 1
    return n == 1

```

```

class Solution:
    def validTree(self, n: int, edges: List[List[int]]) -> bool:
        def dfs(i: int):
            vis.add(i)
            for j in g[i]:
                if j not in vis:
                    dfs(j)

        if len(edges) != n - 1:
            return False

```

```

        g = [[] for _ in range(n)]
        for a, b in edges:
            g[a].append(b)
            g[b].append(a)
        vis = set()
        dfs(0)
        return len(vis) == n

```

• SimplyLeet

Python solution with DFS

```

def validTree(n, edges):
    # If the number of edges is not exactly n - 1, it cannot be a tree
    if len(edges) != n - 1:
        return False

    # Build the graph as an adjacency list
    graph = {i: [] for i in range(n)}
    for a, b in edges:

```

```

graph[a].append(b)
graph[b].append(a)

visited = set()

# DFS helper function that accepts current node and its parent node
def dfs(node, parent):
    # Add the current node to visited
    visited.add(node)
    # Traverse all the neighboring nodes

    for neighbor in graph[node]:
        # Skip the neighbor if it is the parent to avoid trivial cycle
        # detection
        if neighbor == parent:
            continue
        # If neighbor is already visited, a cycle is detected
        if neighbor in visited:
            return False
        # Recurse deeper; if cycle is found in recursion, propagate False
        # upward
        if not dfs(neighbor, node):
            return False

    return True

# Start DFS from node 0, with parent set to -1 (non-existent)
if not dfs(0, -1):
    return False

# Check if all nodes were visited (i.e., the graph is connected)
return len(visited) == n

# Example usage:

print(validTree(5, [[0, 1], [0, 2], [0, 3], [1, 4]])) # Expected output: True
print(validTree(5, [[0, 1], [1, 2], [2, 3], [1, 3], [1, 4]])) # Expected
output: False

```

• LC.ca

```
class Solution:
    def validTree(self, n: int, edges: List[List[int]]) -> bool:
        def find(x):
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        p = list(range(n))
        for a, b in edges:
            if find(a) == find(b):
                return False
            p[find(a)] = find(b)
            n -= 1
        return n == 1
```

◆ Quick Review

Key Insights

- A tree must be acyclic and connected.
- For n nodes, a valid tree must have exactly $n - 1$ edges.
- A DFS/BFS traversal can check connectivity and simultaneously detect cycles.
- The Union-Find (disjoint set) approach provides an alternative for cycle detection and connectivity verification.

Space & Time

Time Complexity: $O(n + e)$, where e is the number of edges

Space Complexity: $O(n + e)$ for storing the graph and traversal/union-find structures

Solution

We can solve the problem using either a DFS/BFS approach or a Union-Find algorithm. For the DFS/BFS method, we first check if the number of edges is exactly $n - 1$. If not, it cannot be a tree. Then, we build an adjacency list for the graph and perform a DFS starting from node 0 while keeping track of visited nodes and parent information to avoid false-positive cycle detections. If during DFS we encounter a node that has been visited (and is not the parent), then a cycle exists. Finally, if all nodes are reached from node 0 and no cycle is found, the graph is a valid tree. Using Union-Find, we iterate over each edge and try to union the two nodes. If they already share the same parent (i.e., are in the same set), a cycle exists. After processing all edges, we confirm that exactly one connected component exists.

DFS

□ #310 Minimum Height Trees

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
DFS · BFS · Graph · Topological Sort
Lists: Grind 169

Problem

A tree is an undirected graph in which any two vertices are connected by exactly one path. In other words, any connected graph without simple cycles is a tree.

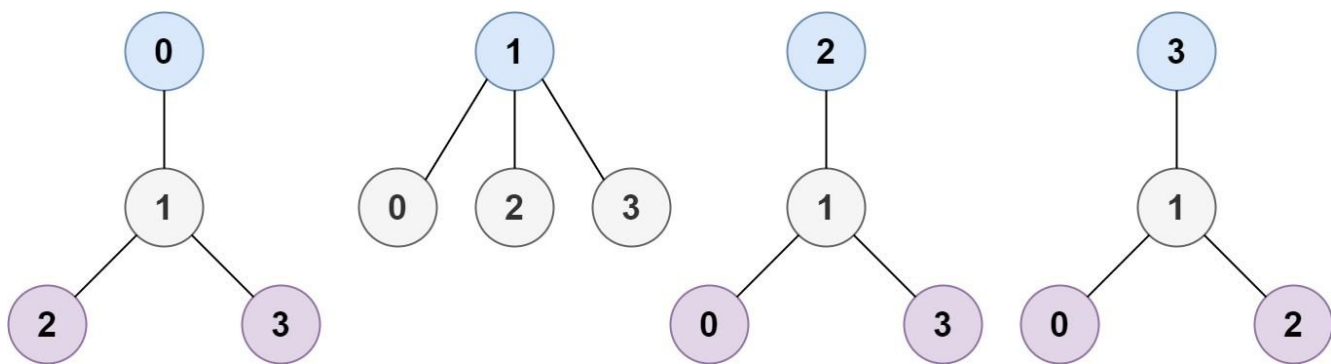
Given a tree of n nodes labelled from 0 to $n - 1$, and an array of $n - 1$ edges where $\text{edges}[i] = [a_i, b_i]$ indicates that there is an undirected edge between the two nodes a_i and b_i in the tree, you can choose any node of the tree as the root.

When you select a node x as the root, the result tree has height h . Among all possible rooted trees, those with minimum height (i.e. $\min(h)$) are called minimum height trees (MHTs).

Return a list of all MHTs' root labels. You can return the answer in any order.

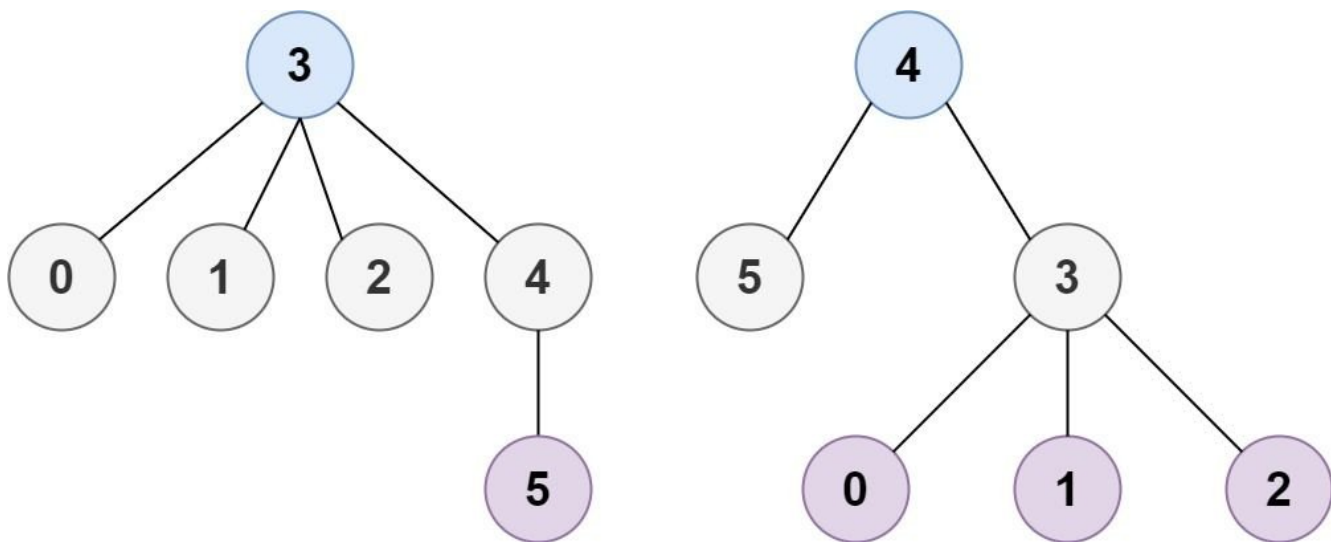
The height of a rooted tree is the number of edges on the longest downward path between the root and a leaf.

Example 1:



Input: $n = 4$, $edges = [[1,0],[1,2],[1,3]]$
Output: [1]
Explanation: As shown, the height of the tree is 1 when the root is the node with label 1 which is the only MHT.

Example 2:



Input: $n = 6$, $edges = [[3,0],[3,1],[3,2],[3,4],[5,4]]$
Output: [3,4]

Constraints:

- $1 \leq n \leq 2 * 10^4$
- `edges.length == n - 1`
- $0 \leq a_i, b_i < n$
- $a_i \neq b_i$
- All the pairs (a_i, b_i) are distinct.
- The given input is guaranteed to be a tree and there will be no repeated edges.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find which nodes make MHT roots. The answer is at most 2 nodes – the tree's center(s). Right?"

Repeatedly prune leaf nodes (degree 1) until 1 or 2 nodes remain."

#

"Like peeling an onion from outside in – the core is the answer."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Repeatedly peel all leaves layer by layer (nodes with degree 1), like peeling an onion.

The last remaining nodes (0, 1, or 2) are the tree centroids / MHT roots.

BFS leaf-removal is the standard $O(n)$ approach for this problem.

Time: $O(n)$. Space: $O(n)$ for adjacency and degree counts.

Should I go ahead and optimize?"

```
class _310_MinHeightTrees:
```

```
    def findMinHeightTrees(self, n: int, edges: List[List[int]]) -> List[int]:
```

```
        if n == 1:
```

```
            return [0]
```

```
        graph = defaultdict(set)
```

```
        for a, b in edges:
```

```
            graph[a].add(b)
```

```
            graph[b].add(a)
```

```
        leaves = [node for node in range(n) if len(graph[node]) == 1]
```

```
        remaining = n
```

```
        while remaining > 2:
```

```

        remaining -= len(leaves)
        new_leaves = []
        for leaf in leaves:
            nb = next(iter(graph[leaf]))
            graph[nb].remove(leaf)
            if len(graph[nb]) == 1:
                new_leaves.append(nb)
        leaves = new_leaves
    return leaves

```

PHASE 5 — TEST & DEBUG

n=4, edges=[[1,0],[1,2],[1,3]]: leaves=[0,2,3]. remaining=4-3=1. new_leaves=[1].
→ [1] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$. This is topological sort on undirected tree – same BFS idea.

At most 2 centers exist – if more, tree would have multiple independent long paths.

Given more time I'd ask: all trees of minimum height (not just roots)?

BFS from all roots found, check height – same $O(n)$."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def findMinHeightTrees(self, n: int, edges: list[list[int]]) -> list[int]:
        if n == 1 or not edges:
            return [0]

        ans = []
        graph = collections.defaultdict(set)

        for u, v in edges:
            graph[u].add(v)

```

```

graph[v].add(u)

for label, children in graph.items():
    if len(children) == 1:
        ans.append(label)

while n > 2:
    n -= len(ans)
    nextLeaves = []
    for leaf in ans:
        u = next(iter(graph[leaf]))
        graph[u].remove(leaf)
        if len(graph[u]) == 1:
            nextLeaves.append(u)
    ans = nextLeaves

return ans

```

• LeetCode

```

class Solution:
    def findMinHeightTrees(self, n: int, edges: List[List[int]]) -> List[int]:
        if n == 1:
            return [0]
        g = [[] for _ in range(n)]
        degree = [0] * n
        for a, b in edges:
            g[a].append(b)
            g[b].append(a)
            degree[a] += 1
            degree[b] += 1
        q = deque(i for i in range(n) if degree[i] == 1)
        ans = []
        while q:
            ans.clear()
            for _ in range(len(q)):
                a = q.popleft()
                ans.append(a)
                for b in g[a]:
                    degree[b] -= 1

```

```

        if degree[b] == 1:
            q.append(b)

    return ans

```

• SimplyLeet

```

# Python solution with line-by-line comments
class Solution:
    def findMinHeightTrees(self, n: int, edges: List[List[int]]) -> List[int]:
        # Special case: if there's only one node, return it as the only MHT
        root.

        if n == 1:
            return [0]

        # Build the graph using an adjacency list with sets for neighbors.
        graph = [set() for _ in range(n)]
        for u, v in edges:

            graph[u].add(v)
            graph[v].add(u)

        # Initialize the first layer of leaves (nodes with one neighbor).
        leaves = [i for i in range(n) if len(graph[i]) == 1]

        # Remove leaves layer by layer until at most 2 nodes remain.
        remaining_nodes = n
        while remaining_nodes > 2:
            remaining_nodes -= len(leaves)

            new_leaves = []
            # Remove current leaves.
            for leaf in leaves:
                # Since it's a leaf, there is only one neighbor.
                neighbor = graph[leaf].pop()
                # Remove the connection from the neighbor to the leaf.
                graph[neighbor].remove(leaf)
                # If the neighbor becomes a leaf, add it to the new leaves
                list.

                if len(graph[neighbor]) == 1:
                    new_leaves.append(neighbor)

```

```

        leaves = new_leaves

    # The remaining nodes are the roots of the MHTs.
    return leaves

```

• LC.ca

```

from typing import List, Set
from collections import defaultdict

```

```

class Solution:
    def findMinHeightTrees(self, n: int, edges: List[List[int]]) -> List[int]:
        if n == 1:
            return [0]

```

```

        graph = defaultdict(set)

```

```

        for a, b in edges:
            graph[a].add(b)
            graph[b].add(a)

```

```

        # just check neighbours[] size, not indgree[] for each node
        leaves = [i for i in range(n) if len(graph[i]) == 1]

```

```

        while n > 2:
            n -= len(leaves)
            new_leaves = []

```

```

            for leaf in leaves:
                # should be only one in hashset pop(), because it's a leaf node
                neighbor = graph[leaf].pop()
                graph[neighbor].remove(leaf)
                if len(graph[neighbor]) == 1:
                    new_leaves.append(neighbor)
            leaves = new_leaves

```

```

        return leaves

```

#####

```

class Solution:
    def findMinHeightTrees(self, n: int, edges: List[List[int]]) -> List[int]:
        if n == 1:
            return [0]
        g = defaultdict(list)
        degree = [0] * n
        for a, b in edges:
            g[a].append(b)

            g[b].append(a)
            degree[a] += 1 # not needed, just len(g[a]) is good enough, will
            # update it in next round updating
            degree[b] += 1
        q = deque()
        for i in range(n):
            if degree[i] == 1:
                q.append(i)
        ans = []
        while q:
            n = len(q)

            ans.clear()
            for _ in range(n):
                a = q.popleft()
                ans.append(a)
                for b in g[a]:
                    degree[b] -= 1
                    if degree[b] == 1: # final round only 2 left (a,b), then
                        # degree[b] here will be 0, and no more node enqueue
                        q.append(b)
            return ans

```

◆ Quick Review

Key Insights

- A tree can have at most two centers that will serve as the roots for its minimum height

trees.

- Instead of trying all possible roots, we can remove leaves layer by layer until we are left with the central node(s).
- The process is similar to a topological sort on the tree.
- When there are 2 or fewer nodes left, these nodes are the centers (roots) of MHTs.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes, since we process each node and edge once.

Space Complexity: $O(n)$ for storing the graph and additional data structures like the leaves list.

Solution

The solution builds an undirected graph from the input, then identifies and removes node-leaves iteratively. A leaf is defined as a node connected to only one other node. By repeatedly removing the leaves and updating the graph (decreasing neighbor degrees), the algorithm peels away the outer layers of the tree. When only one or two nodes remain,

these nodes are the centers of the tree and form the roots of the minimum height trees.

DFS

- ★ #323 Number of Connected Components in an Undirected Graph ★
[Open on LeetCode](#)

MED

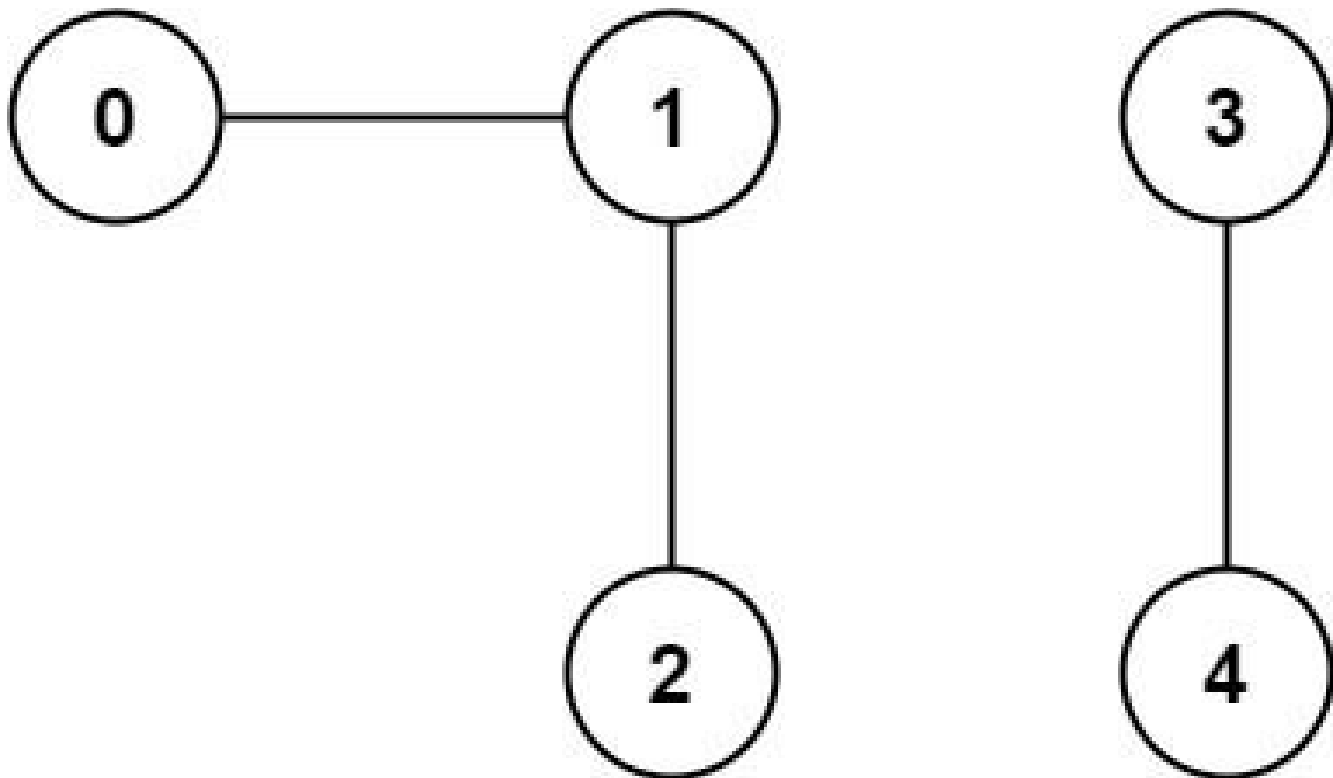
LeetDoocs · SimplyLeet · WalkCC · LeetCode
DFS · BFS · Union Find · Graph
Lists: ★ Premium | Grind 169 | Premium 98

Problem

You have a graph of n nodes. You are given an integer n and an array `edges` where `edges[i] = [ai, bi]` indicates that there is an edge between a_i and b_i in the graph.

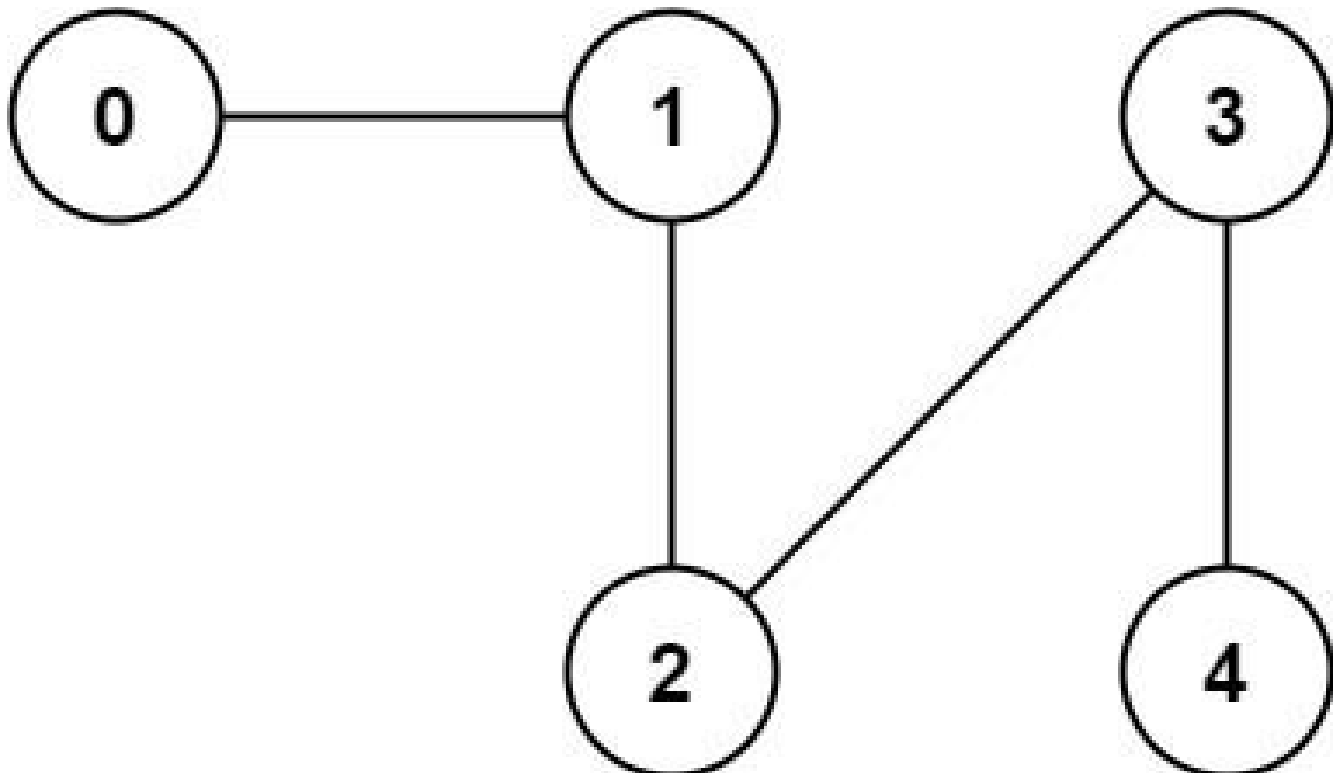
Return the number of connected components in the graph.

Example 1:



Input: $n = 5$, $edges = [[0,1],[1,2],[3,4]]$
Output: 2

Example 2:



Input: $n = 5$, $edges = [[0,1],[1,2],[2,3],[3,4]]$
 Output: 1

Constraints:

- $1 \leq n \leq 2000$
- $1 \leq edges.length \leq 5000$
- $edges[i].length == 2$
- $0 \leq a_i \leq b_i < n$
- $a_i \neq b_i$
- There are no repeated edges.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count connected components in an undirected graph. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Union-Find: for each edge, union the two endpoints.

Answer = n minus number of distinct roots after all unions.

Simple DSU with path compression - $O(n \alpha(n))$.

Time: $O(n \alpha(n))$. Space: $O(n)$ for parent array.

Should I go ahead and optimize?"

```
class _323_NumberConnectedComponents:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        graph = defaultdict(list)
        for a, b in edges:
            graph[a].append(b)
            graph[b].append(a)
        visited = set()
        def dfs(node):
            visited.add(node)
            for nb in graph[node]:
                if nb not in visited:
                    dfs(nb)
```

```

count = 0
for node in range(n):
    if node not in visited:
        dfs(node)
        count += 1
return count

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(V+E)$. Space $O(V+E)$. Standard DFS counting components.
 # Union-Find alternative: merge edges, count unique roots $\rightarrow O(n * \alpha(n))$.
 # Given more time I'd ask: what if edges are added dynamically?
 # Union-Find with path compression supports $O(\alpha(n))$ per merge query."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def countComponents(self, n: int, edges: list[list[int]]) -> int:
        ans = 0
        graph = [[] for _ in range(n)]
        seen = set()

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        def bfs(node: int, seen: set[int]) -> None:
            q = collections.deque([node])
            seen.add(node)

            while q:
                u = q.pop()
                for v in graph[u]:
                    if v not in seen:
                        q.append(v)
                        seen.add(v)

```

```
for i in range(n):
    if i not in seen:
        bfs(i, seen)
        ans += 1
```

```
return ans
```

```
class UnionFind:
    def __init__(self, n: int):
        self.count = n
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self._find(u)
        j = self._find(v)
        if i == j:
```

```
            return
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1
        self.count -= 1
```

```
    def _find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self._find(self.id[u])
        return self.id[u]
```

```
class Solution:
    def countComponents(self, n: int, edges: list[list[int]]) -> int:
        uf = UnionFind(n)
```

```

for u, v in edges:
    uf.unionByRank(u, v)

return uf.count

```

• LeetDoocs

```

class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        def dfs(i: int) -> int:
            if i in vis:
                return 0
            vis.add(i)
            for j in g[i]:
                dfs(j)
            return 1

```

```

g = [[] for _ in range(n)]
for a, b in edges:
    g[a].append(b)
    g[b].append(a)
vis = set()
return sum(dfs(i) for i in range(n))

```

```

class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

```

```

def union(self, a, b):
    pa, pb = self.find(a), self.find(b)
    if pa == pb:
        return False
    if self.size[pa] > self.size[pb]:
        self.p[pb] = pa
        self.size[pa] += self.size[pb]
    else:
        self.p[pa] = pb
        self.size[pb] += self.size[pa]

```

```

    return True

```

```

class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        uf = UnionFind(n)
        for a, b in edges:
            n -= uf.union(a, b)
        return n

```

```

class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        g = [[] for _ in range(n)]
        for a, b in edges:
            g[a].append(b)
            g[b].append(a)
        vis = set()
        ans = 0
        for i in range(n):
            if i in vis:

```

```

                continue
            vis.add(i)
            q = deque([i])
            while q:
                a = q.popleft()
                for b in g[a]:
                    if b not in vis:
                        vis.add(b)
                        q.append(b)
            ans += 1

```



```
return ans
```

• SimplyLeet

```
# Function to count the number of connected components in an undirected graph.
def count_components(n, edges):
    # Build the adjacency list for the graph.
    graph = {i: [] for i in range(n)}
    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)

    # Set to keep track of visited nodes.
    visited = set()

    # Helper function to perform DFS traversal starting from a node.
    def dfs(node):
        stack = [node]
        while stack:
            current = stack.pop()
            if current not in visited:
                visited.add(current)
                for neighbor in graph[current]:
                    if neighbor not in visited:
                        stack.append(neighbor)

    # Count connected components by initiating DFS on unvisited nodes.
    count = 0
    for i in range(n):
        if i not in visited:
            dfs(i)
            count += 1
    return count

# Example usage:
print(count_components(5, [[0,1],[1,2],[3,4]])) # Expected output: 2
```

• LC.ca

```

...
example-2: [[0, 1], [1, 2], [2, 3], [3, 4]]

```

```

for loop

```

```

[0, 1]
    0 parent set to 1
    1 parent set to 1

```

```

[1, 2]

```

```

    1 parent set to 2
    2 parent set to 2

```

```

[2, 3]
    2 parent set to 3
    3 parent set to 3

```

```

final:
    0 parent set to 1
    1 parent set to 2

```

```

    2 parent set to 3
    3 parent set to 3

```

```

==> so filter for 'i == find(i)'
...

```

```

class Solution:

```

```

    def countComponents(self, n: int, edges: List[List[int]]) -> int:

```

```

        def find(x):
            if p[x] != x:
                p[x] = find(p[x])

```

```

            return p[x]

```

```

        p = list(range(n))
        for a, b in edges:
            p[find(a)] = find(b)
        return sum(i == find(i) for i in range(n)) # or len(set(parents))

```

```

#####

```

```

class Solution: # more literal

```

```
def countComponents(self, n: int, edges: List[List[int]]) -> int:
    # Initialize parent array for union-find
    parent = [i for i in range(n)]

    # Perform union-find on each edge
    for u, v in edges:
        self.union(u, v, parent)

    # Count the number of distinct parents (connected components)
    count = len(set(self.find(x, parent) for x in range(n)))

    return count

def union(self, x: int, y: int, parent: List[int]) -> None:
    rootX = self.find(x, parent)
    rootY = self.find(y, parent)
    if rootX != rootY:
        parent[rootX] = rootY

def find(self, x: int, parent: List[int]) -> int:
    if parent[x] != x:
        parent[x] = self.find(parent[x], parent)
    return parent[x]
```

◆ Quick Review

Key Insights

- Use an adjacency list to represent the graph.
- A DFS (or BFS) traversal from an unvisited node will traverse an entire connected component.
- Each new DFS/BFS call from an unvisited node signals a separate connected component.

- Alternatively, the Union-Find data structure can be used to union connected nodes and count distinct sets.

Space & Time

Time Complexity: $O(n + e)$, where n is the number of nodes and e is the number of edges.

Space Complexity: $O(n + e)$ due to the storage of the graph and tracking of visited nodes.

Solution

We solve the problem by iterating over each node and performing a DFS for any node that hasn't been visited. The DFS uses an iterative approach with a stack to traverse all nodes in a connected component. Each DFS initiation corresponds to one connected component. This method efficiently explores the graph using an adjacency list and marks nodes as visited to prevent redundant traversals.

DFS

□ #417 Pacific Atlantic Water Flow

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Matrix
Lists: Grind 169

Problem

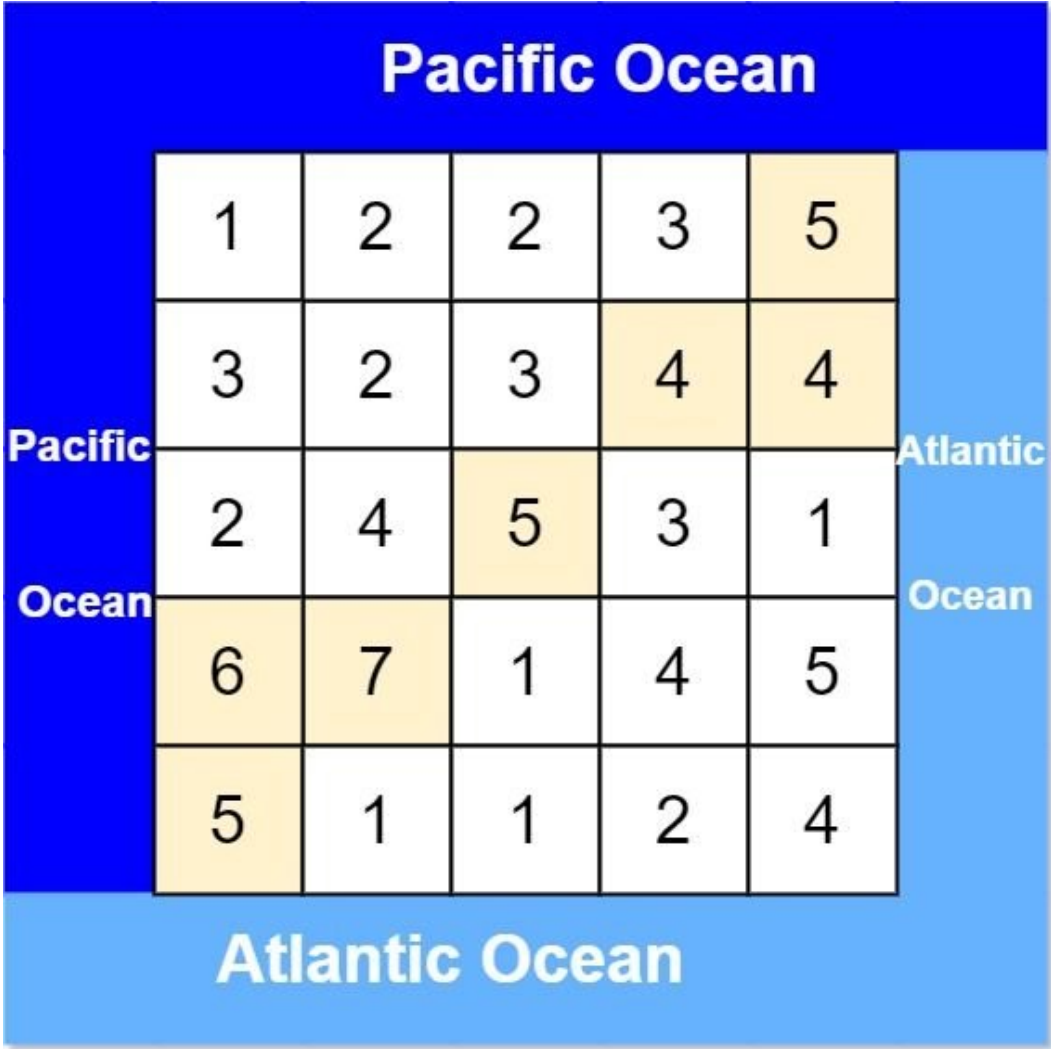
There is an $m \times n$ rectangular island that borders both the Pacific Ocean and Atlantic Ocean. The Pacific Ocean touches the island's left and top edges, and the Atlantic Ocean touches the island's right and bottom edges. The island is partitioned into a grid of square cells. You are given an $m \times n$ integer matrix `heights` where `heights[r][c]` represents the height above sea level of the cell at coordinate (r, c) .

The island receives a lot of rain, and the rain water can flow to neighboring cells directly north, south, east, and west if the neighboring cell's height is less than or equal to the current cell's height. Water can flow from any cell

adjacent to an ocean into the ocean.

Return a 2D list of grid coordinates result where $\text{result}[i] = [\text{ri}, \text{ci}]$ denotes that rain water can flow from cell (ri, ci) to both the Pacific and Atlantic oceans.

Example 1:



```

Input: heights = [[1,2,2,3,5],[3,2,3,4,4],[2,4,5,3,1],[6,7,1,4,5],[5,1,1,2,4]]
Output: [[0,4],[1,3],[1,4],[2,2],[3,0],[3,1],[4,0]]
Explanation: The following cells can flow to the Pacific and Atlantic oceans,
as shown below:
[0,4]: [0,4] -> Pacific Ocean
        [0,4] -> Atlantic Ocean
[1,3]: [1,3] -> [0,3] -> Pacific Ocean
        [1,3] -> [1,4] -> Atlantic Ocean
[1,4]: [1,4] -> [1,3] -> [0,3] -> Pacific Ocean

```

Example 2:

```

Input: heights = [[1]]
Output: [[0,0]]
Explanation: The water can flow from the only cell to the Pacific and Atlantic
oceans.

```

Constraints:

- $m == \text{heights.length}$
- $n == \text{heights}[r].\text{length}$
- $1 \leq m, n \leq 200$
- $0 \leq \text{heights}[r][c] \leq 105$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Water flows from higher to lower (or equal). Find cells that can flow to BOTH oceans.

Pacific touches top/left edges. Atlantic touches bottom/right edges. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

From each cell, DFS to see if it can reach both Pacific (top/left border) and Atlantic (bottom/right).

Re-run DFS per cell – $O((m*n)^2)$ naive.

Time: $O((m * n)^2)$ naive per-cell DFS. Space: $O(m * n)$ stack.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (reverse DFS from ocean borders)

"Instead of simulating forward flow, run DFS BACKWARDS from each ocean's border cells.

Reverse flow: go uphill (neighbor height >= current height).

Intersection of Pacific-reachable and Atlantic-reachable = answer."

PHASE 4 — CLEAN CODE

```
class _417_PacificAtlantic:
```

```
    def pacificAtlantic(self, heights: List[List[int]]) -> List[List[int]]:
```

```
        m, n = len(heights), len(heights[0])
```

```
        def bfs(starts):
```

```
            reachable = set(starts)
```

```
            queue = deque(starts)
```

```
            while queue:
```

```
                r, c = queue.popleft()
```

```
                for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
```

```
                    nr, nc = r + dr, c + dc
```

```
                    if 0 <= nr < m and 0 <= nc < n and (nr,nc) not in reachable \
                        and heights[nr][nc] >= heights[r][c]:
```

```
                        reachable.add((nr, nc))
```

```
                        queue.append((nr, nc))
```

```
            return reachable
```

```
        pacific = [(0, c) for c in range(n)] + [(r, 0) for r in range(1, m)]
```

```
        atlantic = [(m-1, c) for c in range(n)] + [(r, n-1) for r in range(m-1)]
```

```
        pac_reach = bfs(pacific)
```

```
        atl_reach = bfs(atlantic)
```

```
        return [[r, c] for r, c in pac_reach & atl_reach]
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(m*n)$. BFS from borders is cleaner than DFS (no recursion limit).

The reversal insight: 'can reach ocean' → 'ocean can flow backward to this cell'.

Given more time I'd ask: what if water can also flow diagonally?

Add 4 diagonal directions – same algorithm."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def pacificAtlantic(self, heights: list[list[int]]) -> list[list[int]]:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(heights)
        n = len(heights[0])
        qP = collections.deque()
        qA = collections.deque()
        seenP = [[False] * n for _ in range(m)]
        seenA = [[False] * n for _ in range(m)]

        for i in range(m):
            qP.append((i, 0))
            qA.append((i, n - 1))
            seenP[i][0] = True
            seenA[i][n - 1] = True

        for j in range(n):
            qP.append((0, j))
            qA.append((m - 1, j))
            seenP[0][j] = True

            seenA[m - 1][j] = True

        def bfs(q: deque, seen: list[list[bool]]):
            while q:
                i, j = q.popleft()
                h = heights[i][j]
                for dx, dy in DIRS:
                    x = i + dx
                    y = j + dy
                    if x < 0 or x == m or y < 0 or y == n:
```

```

        continue
    if seen[x][y] or heights[x][y] < h:
        continue
    q.append((x, y))
    seen[x][y] = True

    bfs(qP, seenP)
    bfs(qA, seenA)

    return [[i, j] for i in range(m) for j in range(n) if seenP[i][j] and seen
A[i][j]]

```

```

class Solution:
    def pacificAtlantic(self, heights: list[list[int]]) -> list[list[int]]:
        m = len(heights)
        n = len(heights[0])
        seenP = [[False] * n for _ in range(m)]
        seenA = [[False] * n for _ in range(m)]

        def dfs(i: int, j: int, h: int, seen: list[list[bool]]) -> None:
            if i < 0 or i == m or j < 0 or j == n:
                return

```

```

            if seen[i][j] or heights[i][j] < h:
                return

            seen[i][j] = True
            dfs(i + 1, j, heights[i][j], seen)
            dfs(i - 1, j, heights[i][j], seen)
            dfs(i, j + 1, heights[i][j], seen)
            dfs(i, j - 1, heights[i][j], seen)

        for i in range(m):

```

```

    dfs(i, 0, 0, seenP)
    dfs(i, n - 1, 0, seenA)

    for j in range(n):
        dfs(0, j, 0, seenP)
        dfs(m - 1, j, 0, seenA)

    return [[i, j]
            for i in range(m)
            for j in range(n)

            if seenP[i][j] and seenA[i][j]]

```

• LeetDoocs

```

class Solution:
    def pacificAtlantic(self, heights: List[List[int]]) -> List[List[int]]:
        def bfs(q: Deque[Tuple[int, int]], vis: List[List[bool]]) -> None:
            while q:
                x, y = q.popleft()
                for dx, dy in pairwise(dirs):
                    nx, ny = x + dx, y + dy
                    if (
                        0 <= nx < m
                        and 0 <= ny < n
                        and not vis[nx][ny]
                        and heights[nx][ny] >= heights[x][y]
                    ):
                        vis[nx][ny] = True
                        q.append((nx, ny))

        m, n = len(heights), len(heights[0])
        vis1 = [[False] * n for _ in range(m)]
        vis2 = [[False] * n for _ in range(m)]
        q1: Deque[Tuple[int, int]] = deque()

```

```
q2: Deque[Tuple[int, int]] = deque()
dirs = (-1, 0, 1, 0, -1)
```

```
for i in range(m):
    q1.append((i, 0))
    vis1[i][0] = True
    q2.append((i, n - 1))
    vis2[i][n - 1] = True
```

```
for j in range(n):
```

```
    q1.append((0, j))
    vis1[0][j] = True
    q2.append((m - 1, j))
    vis2[m - 1][j] = True
```

```
bfs(q1, vis1)
bfs(q2, vis2)
```

```
return [(i, j) for i in range(m) for j in range(n) if vis1[i][j] and vis2[i][j]]
```

• SimplyLeet

```
# Python solution for Pacific Atlantic Water Flow
```

```
class Solution:
    def pacificAtlantic(self, heights: List[List[int]]) -> List[List[int]]:
        if not heights or not heights[0]:
            return []

        rows, cols = len(heights), len(heights[0])
        pacific_reachable = [[False for _ in range(cols)] for _ in range(rows)]
        atlantic_reachable = [[False for _ in range(cols)] for _ in range(rows)]
```

```

# Directions: up, down, left, right
directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

# DFS function to mark reachable cells
def dfs(r, c, reachable):
    reachable[r][c] = True
    for dr, dc in directions:
        new_r, new_c = r + dr, c + dc
        # Check bounds and conditions to flow water

        if (0 <= new_r < rows and 0 <= new_c < cols and
            not reachable[new_r][new_c] and
            heights[new_r][new_c] >= heights[r][c]):
            dfs(new_r, new_c, reachable)

# DFS from Pacific ocean borders (top row and left column)
for i in range(rows):
    dfs(i, 0, pacific_reachable)
for j in range(cols):
    dfs(0, j, pacific_reachable)

# DFS from Atlantic ocean borders (bottom row and right column)
for i in range(rows):
    dfs(i, cols - 1, atlantic_reachable)
for j in range(cols):
    dfs(rows - 1, j, atlantic_reachable)

# Collect cells that can reach both oceans
result = []
for r in range(rows):
    for c in range(cols):
        if pacific_reachable[r][c] and atlantic_reachable[r][c]:
            result.append([r, c])
return result

```

• LC.ca

```

class Solution:
    def pacificAtlantic(self, heights: List[List[int]]) -> List[List[int]]:
        def bfs(q, vis):
            while q:
                for _ in range(len(q)):
                    i, j = q.popleft()
                    for a, b in [[0, -1], [0, 1], [1, 0], [-1, 0]]:
                        x, y = i + a, j + b
                        if (
                            0 <= x < m
                                and 0 <= y < n
                                and (x, y) not in vis
                                and heights[x][y] >= heights[i][j]
                        ):
                            vis.add((x, y))
                            q.append((x, y))

        m, n = len(heights), len(heights[0])
        vis1, vis2 = set(), set()
        q1 = deque()

        q2 = deque()
        for i in range(m):
            for j in range(n):
                if i == 0 or j == 0:
                    vis1.add((i, j))
                    q1.append((i, j))
                if i == m - 1 or j == n - 1:
                    vis2.add((i, j))
                    q2.append((i, j))

        bfs(q1, vis1)

        bfs(q2, vis2)
        return [
            (i, j)
            for i in range(m)
            for j in range(n)
            if (i, j) in vis1 and (i, j) in vis2
        ]

```

◆ Quick Review

Round 2 · High · Medium

p.1443

Key Insights

- Reverse the typical DFS/BFS perspective: start from the oceans and work inward.
- Maintain two matrices (or sets) to keep track of cells reachable from the Pacific and Atlantic edges.
- A cell can be added to the result if it is reached in both traversals.
- The height constraint ensures water flows only in non-increasing order.

Space & Time

Time Complexity: $O(m * n)$ where m is the number of rows and n is the number of columns. Each cell is processed up to twice (once from each ocean).

Space Complexity: $O(m * n)$ for the visited matrices and the recursion/queue used in DFS/BFS.

Solution

The solution uses depth-first search (DFS) from the ocean boundaries. We initialize two boolean matrices, one for each ocean. For the Pacific, we start DFS from the leftmost column and top row; for the Atlantic, we start from the

rightmost column and bottom row. In each DFS, if the next cell has a height equal to or greater than the current cell and hasn't been visited, we continue the search. Finally, we scan through the grid to collect cells that are reachable from both oceans. This reverse search simplifies the path validations and leverages the allowed water flow conditions.

DFS

□ ★ #490 The Maze ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Matrix
Lists: ★ Premium | Premium 98

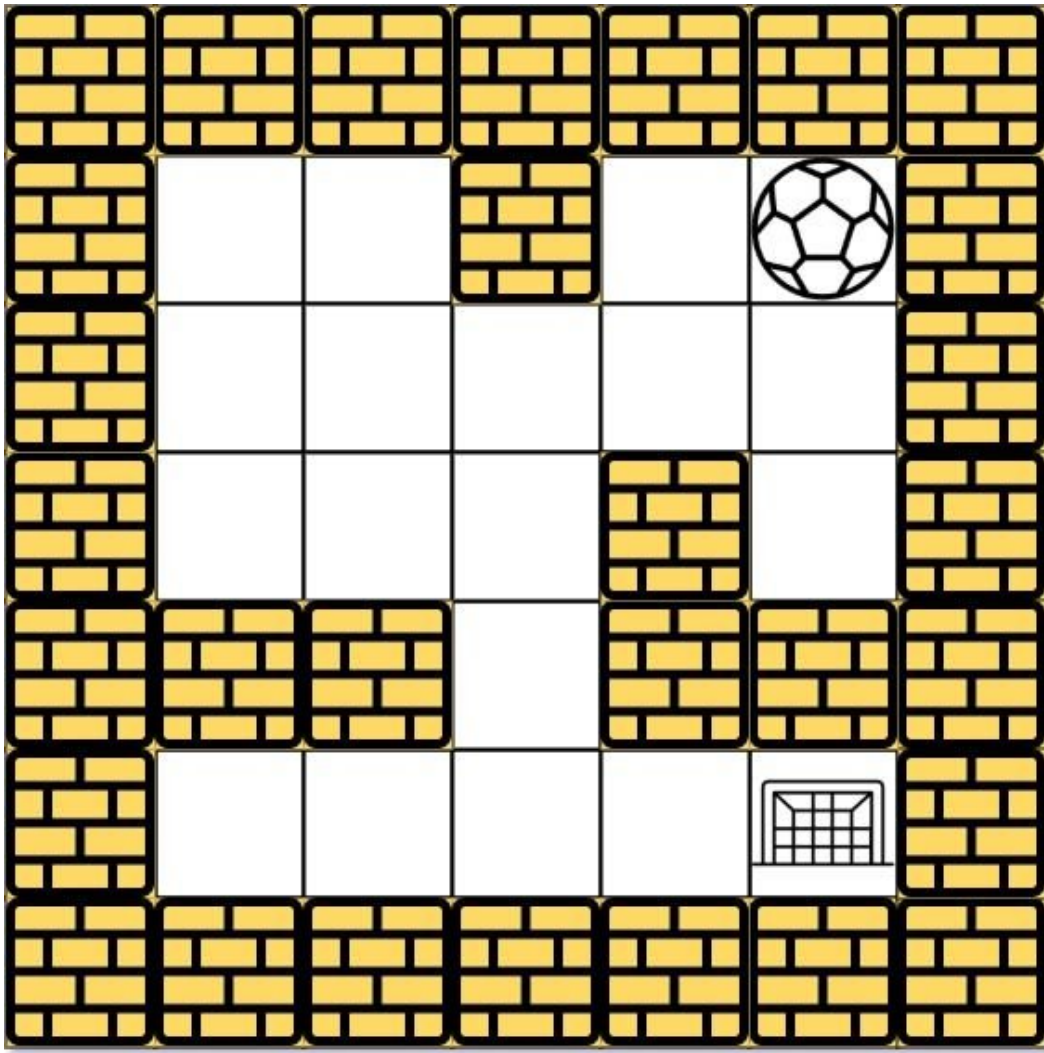
Problem

There is a ball in a maze with empty spaces (represented as 0) and walls (represented as 1). The ball can go through the empty spaces by rolling up, down, left or right, but it won't stop rolling until hitting a wall. When the ball stops, it could choose the next direction.

Given the $m \times n$ maze, the ball's start position and the destination, where $\text{start} = [\text{startrow}, \text{startcol}]$ and $\text{destination} = [\text{destinationrow}, \text{destinationcol}]$, return true if the ball can stop at the destination, otherwise return false.

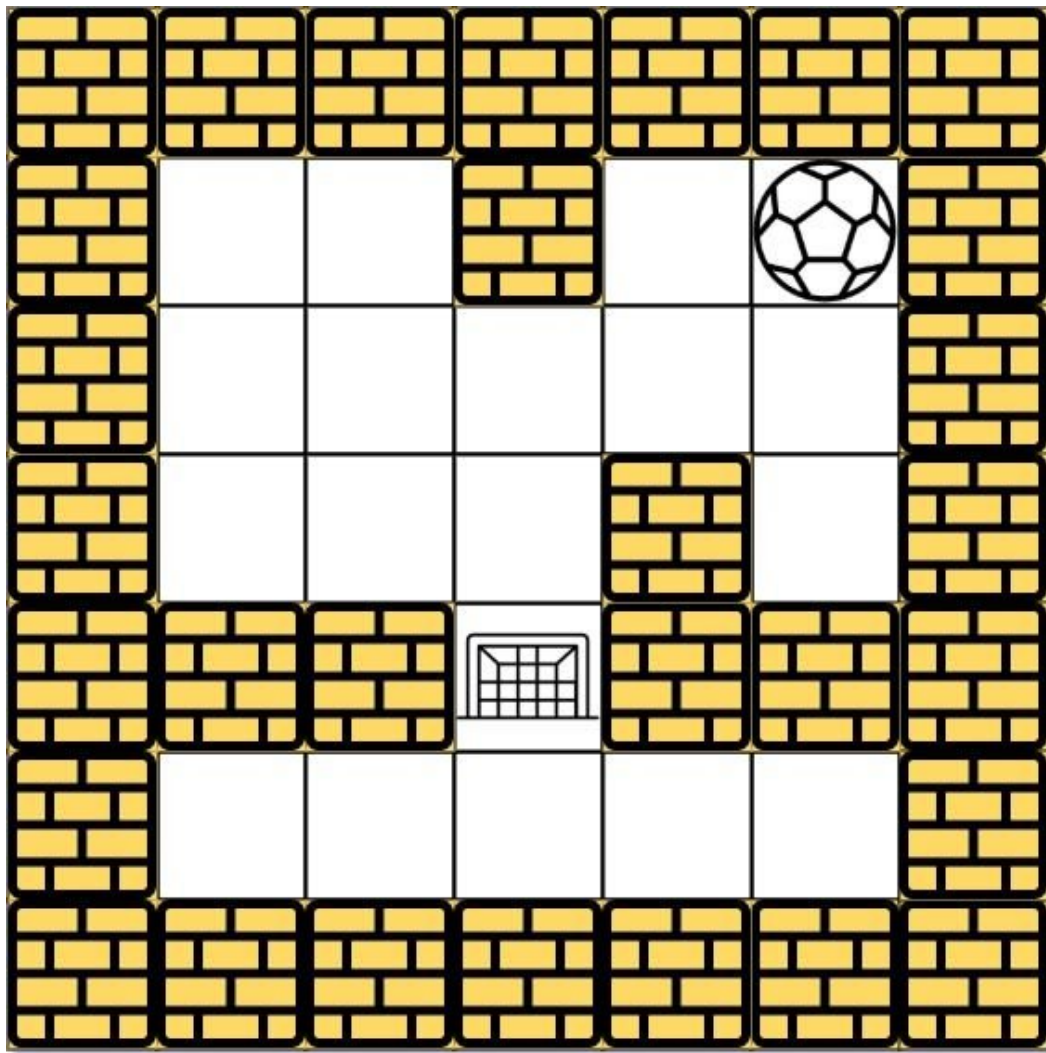
You may assume that the borders of the maze are all walls (see examples).

Example 1:



Input: maze = `[[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]]`,
start = `[0,4]`, destination = `[4,4]`
Output: true
Explanation: One possible way is : left -> down -> left -> down -> right ->
down -> right.

Example 2:



Input: maze = `[[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]]`,
 start = `[0,4]`, destination = `[3,2]`

Output: false

Explanation: There is no way for the ball to stop at the destination. Notice that you can pass through the destination but you cannot stop there.

Example 3:

Input: maze = `[[0,0,0,0,0],[1,1,0,0,1],[0,0,0,0,0],[0,1,0,0,1],[0,1,0,0,0]]`,
 start = `[4,3]`, destination = `[0,1]`

Output: false

Constraints:

- `m == maze.length`
- `n == maze[i].length`

- $1 \leq m, n \leq 100$
- `maze[i][j]` is 0 or 1.
- `start.length == 2`
- `destination.length == 2`
- $0 \leq \text{startrow}, \text{destinationrow} < m$
- $0 \leq \text{startcol}, \text{destinationcol} < n$
- Both the ball and the destination exist in an empty space, and they will not be in the same position initially.
- The maze contains at least 2 empty spaces.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Ball rolls in a direction until hitting a wall. Can it STOP at the destination?
# It can pass through the destination but not stop there unless it hits a wall.
Right?"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# DFS from entrance: at each empty cell try all 4 directions.
# Mark visited cells to avoid cycles; succeed if we reach the boundary.
# Correct path exploration on an m x n grid.
# Time:  $O(m * n)$ . Space:  $O(m * n)$  visited.
# Should I go ahead and optimize?"
```

```
class _490_TheMaze:
    def hasPath(self, maze: List[List[int]], start: List[int], destination:
List[int]) -> bool:
        m, n = len(maze), len(maze[0])
        dest = tuple(destination)
        visited: set = set()
        def dfs(r, c):
            if (r, c) in visited: return False
```

```

        visited.add((r, c))
        if (r, c) == dest: return True
        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
            nr, nc = r, c
            while 0 <= nr + dr < m and 0 <= nc + dc < n and maze[nr+dr][nc+dc]
== 0:
            nr += dr; nc += dc
            if dfs(nr, nc): return True
        return False
    return dfs(start[0], start[1])

```

PHASE 5 — TEST & DEBUG

Ball rolls until hitting wall → stops → that stop position is a new state.

DFS on stop-positions. Destination reached only if the ball stops exactly there. ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m \times n \times (m+n))$ – each stop position visits $O(m+n)$ cells to find next stop.

The visited set is on stop-positions, not individual cells.

Given more time I'd ask: The Maze II (#505) – find shortest distance?

Use Dijkstra with distance as weight, BFS/priority queue on stop-positions."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def hasPath(
        self,
        maze: list[list[int]],
        start: list[int],
        destination: list[int],
    ) -> bool:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(maze)
        n = len(maze[0])

        q = collections.deque([(start[0], start[1])])
        seen = {(start[0], start[1])}

        def isValid(x: int, y: int) -> bool:
            return 0 <= x < m and 0 <= y < n and maze[x][y] == 0

        while q:
            i, j = q.popleft()
            for dx, dy in DIRS:
                x = i

```

```

    y = j
    while isValid(x + dx, y + dy):
        x += dx
        y += dy
        if [x, y] == destination:
            return True
        if (x, y) in seen:
            continue
        q.append((x, y))
        seen.add((x, y))

```

```

return False

```

• LeetCode

```

class Solution:
    def hasPath(
        self, maze: List[List[int]], start: List[int], destination: List[int]
    ) -> bool:
        def dfs(i, j):
            if vis[i][j]:
                return
            vis[i][j] = True
            if [i, j] == destination:
                return

            for a, b in [[0, -1], [0, 1], [1, 0], [-1, 0]]:
                x, y = i, j
                while 0 <= x + a < m and 0 <= y + b < n and maze[x + a][y + b]
                == 0:
                    x, y = x + a, y + b
                    dfs(x, y)

        m, n = len(maze), len(maze[0])
        vis = [[False] * n for _ in range(m)]
        dfs(start[0], start[1])
        return vis[destination[0]][destination[1]]

```

• SimplyLeet


```
# Function to determine if there is a valid path in the maze from start to destination
```

```
def hasPath(maze, start, destination):
    from collections import deque
    rows, cols = len(maze), len(maze[0])
    # Create a visited 2D array to keep track of processed cells
    visited = [[False] * cols for _ in range(rows)]
    queue = deque([tuple(start)])
    visited[start[0]][start[1]] = True

    # Directions: up, down, left, right
```

```
directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
```

```
# Process nodes in BFS fashion
while queue:
    current = queue.popleft()
    # Check if the current position is the destination
    if [current[0], current[1]] == destination:
        return True
    # Explore all four possible directions
    for dx, dy in directions:
```

```
        x, y = current
        # Roll the ball until it hits a wall or boundary
        while 0 <= x + dx < rows and 0 <= y + dy < cols and maze[x + dx][y
+ dy] == 0:
            x += dx
            y += dy
        # If this new stop has not been visited, add it to the queue
        if not visited[x][y]:
            visited[x][y] = True
            queue.append((x, y))
return False
```

```
# Example usage:
maze = [[0,0,1,0,0],
        [0,0,0,0,0],
        [0,0,0,1,0],
        [1,1,0,1,1],
        [0,0,0,0,0]]
start = [0,4]
destination = [4,4]
print(hasPath(maze, start, destination)) # Expected output: True
```

• LC.ca

```
class Solution:
    def hasPath(
        self, maze: List[List[int]], start: List[int], destination: List[int]
    ) -> bool:
        m, n = len(maze), len(maze[0])
        q = deque([start])
        rs, cs = start
        vis = {(rs, cs)}
        while q:
            i, j = q.popleft()

            for a, b in [[0, -1], [0, 1], [-1, 0], [1, 0]]:
                x, y = i, j
                while 0 <= x + a < m and 0 <= y + b < n and maze[x + a][y + b]
                == 0:
                    x, y = x + a, y + b
                    if [x, y] == destination:
                        return True
                    if (x, y) not in vis:
                        vis.add((x, y))
                        q.append((x, y))
            return False
```

◆ Quick Review

Key Insights

- The ball continues moving in one direction until it hits a wall (or the boundary, which is considered a wall).
- You must simulate the ball's rolling rather than taking a single step.
- Even if the ball passes through the destination, it is only valid if it can stop exactly there.
- A breadth-first search (BFS) or depth-first search (DFS) strategy is ideal because we need to explore all stopping positions.
- Use a visited structure to avoid revisiting positions and thus prevent infinite loops.

Space & Time

Time Complexity: $O(m * n)$ where m and n are the dimensions of the maze.

Space Complexity: $O(m * n)$ due to the storage used by the visited matrix and the BFS/DFS queue or recursion stack.

Solution

We simulate the ball's movement using a breadth-first search (BFS) approach. Each time we pick a stop position, we try to roll in all four directions until the ball hits a wall. At that

point, if the new stopping position has not been visited, we mark it as visited and add it to our queue. If we ever reach the destination exactly, we return true. The key is simulating the "roll" by iteratively moving in a direction until no further move is possible. We store the coordinates in a queue (or stack for DFS) along with a 2D visited array to monitor which positions have been processed. The directions are maintained in a list/array, and for each direction, we keep moving until the next cell would be out of bounds or a wall.

DFS

□ ★ #694 Number of Distinct Islands ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · DFS · BFS · Union Find · Hash
Function

Lists: ★ Premium | Premium 98

Problem

You are given an $m \times n$ binary matrix grid. An island is a group of 1's (representing land) connected 4-directionally (horizontal or vertical.) You may assume all four edges of the grid are surrounded by water.

An island is considered to be the same as another if and only if one island can be translated (and not rotated or reflected) to equal the other.

Return the number of distinct islands.

Example 1:

1	1	0	0	0
1	1	0	0	0
0	0	0	1	1
0	0	0	1	1

Input: grid = [[1,1,0,0,0],[1,1,0,0,0],[0,0,0,1,1],[0,0,0,1,1]]

Output: 1

Example 2:

1	1	0	1	1
1	0	0	0	0
0	0	0	0	1
1	1	0	1	1

Input: grid = [[1,1,0,1,1],[1,0,0,0,0],[0,0,0,0,1],[1,1,0,1,1]]
Output: 3

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 50$
- $\text{grid}[i][j]$ is either 0 or 1.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Count islands that are distinct by shape (translation equivalent). Right?
Two islands are the same if one can be shifted to match the other – no rotation/reflection."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."
DFS each island; encode shape as a string of moves (U/D/L/R) from start cell.
Add shape string to a set – distinct shapes = set size.
Must normalize starting cell and direction encoding consistently.
Time: $O(m * n)$. Space: $O(m * n)$ shapes.
Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (path signature hashing)

"DFS each island, record the path taken relative to the starting cell."
Include backtrack markers to distinguish shapes like 'L' vs reverse 'L'.
Hash each shape as a tuple and add to a set – count unique shapes."

PHASE 4 — CLEAN CODE

```
class _694_DistinctIslands:
    def numDistinctIslands(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        shapes: set = set()
        def dfs(r, c, r0, c0, path):
            if r < 0 or r >= m or c < 0 or c >= n or grid[r][c] != 1:
                return
            grid[r][c] = 0
            path.append((r - r0, c - c0))
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                dfs(r + dr, c + dc, r0, c0, path)
            path.append(None) # backtrack marker
        for r in range(m):
            for c in range(n):
                if grid[r][c] == 1:
                    path: List = []
                    dfs(r, c, r, c, path)
                    shapes.add(tuple(path))
        return len(shapes)
```

PHASE 5 — TEST & DEBUG

Two 2x2 islands: both DFS in same order → same relative path → same tuple → 1 distinct ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(m*n)$. Backtrack None markers distinguish different DFS orderings
that could produce the same sequence of offsets for different shapes."

Given more time I'd ask: allow rotation/reflection? Normalise shape to lexicographically smallest
 # among all 8 transformations – significantly more complex."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def numDistinctIslands(self, grid: list[list[int]]) -> int:
        seen = set()

        def dfs(i: int, j: int, i0: int, j0: int):
            if i < 0 or i == len(grid) or j < 0 or j == len(grid[0]):
                return
            if grid[i][j] == 0 or (i, j) in seen:
                return

            seen.add((i, j))
            island.append((i - i0, j - j0))
            dfs(i + 1, j, i0, j0)
            dfs(i - 1, j, i0, j0)
            dfs(i, j + 1, i0, j0)
            dfs(i, j - 1, i0, j0)

        islands = set() # all the different islands

        for i in range(len(grid)):

            for j in range(len(grid[0])):
                island = []
                dfs(i, j, i, j)
                if island:
                    islands.add(frozenset(island))

        return len(islands)
```

• LeetDoocs

```

class Solution:
    def numDistinctIslands(self, grid: List[List[int]]) -> int:
        def dfs(i: int, j: int, k: int):
            grid[i][j] = 0
            path.append(str(k))
            dirs = (-1, 0, 1, 0, -1)
            for h in range(1, 5):
                x, y = i + dirs[h - 1], j + dirs[h]
                if 0 <= x < m and 0 <= y < n and grid[x][y]:
                    dfs(x, y, h)

            path.append(str(-k))

        paths = set()
        path = []
        m, n = len(grid), len(grid[0])
        for i, row in enumerate(grid):
            for j, x in enumerate(row):
                if x:
                    dfs(i, j, 0)
                    paths.add("".join(path))

            path.clear()

        return len(paths)

```

• SimplyLeet

```

def numDistinctIslands(grid):
    # Get the number of rows and columns
    rows, cols = len(grid), len(grid[0])
    # Set to store unique island shapes
    distinct_shapes = set()

    # DFS function to explore the island and record its relative shape
    def dfs(r, c, origin_r, origin_c, shape):
        # Check boundaries and if the cell is unvisited land
        if r < 0 or c < 0 or r >= rows or c >= cols or grid[r][c] == 0:

```

```

        return

    # Mark the cell as visited by setting it to 0
    grid[r][c] = 0
    # Record the relative position from the origin of this island
    shape.append((r - origin_r, c - origin_c))
    # Explore all 4-directionally adjacent cells
    dfs(r + 1, c, origin_r, origin_c, shape)
    dfs(r - 1, c, origin_r, origin_c, shape)
    dfs(r, c + 1, origin_r, origin_c, shape)
    dfs(r, c - 1, origin_r, origin_c, shape)

# Iterate over every cell in the grid
for i in range(rows):
    for j in range(cols):
        if grid[i][j] == 1:
            shape = []
            dfs(i, j, i, j, shape)
            # Convert the shape list to a tuple of sorted coordinates for
canonical form
            distinct_shapes.add(tuple(sorted(shape)))

# The number of distinct islands is the size of the set
return len(distinct_shapes)

# Example usage:
grid = [[1,1,0,0,0],
        [1,1,0,0,0],
        [0,0,0,1,1],
        [0,0,0,1,1]]
print(numDistinctIslands(grid)) # Output: 1

```

• LC.ca

```

class Solution:
    def numDistinctIslands(self, grid: List[List[int]]) -> int:
        def dfs(i: int, j: int, k: int):
            grid[i][j] = 0
            path.append(str(k))
            dirs = (-1, 0, 1, 0, -1)
            for h in range(1, 5):
                x, y = i + dirs[h - 1], j + dirs[h]
                if 0 <= x < m and 0 <= y < n and grid[x][y]:
                    dfs(x, y, h)

            path.append(str(-k))

        paths = set()
        path = []
        m, n = len(grid), len(grid[0])
        for i, row in enumerate(grid):
            for j, x in enumerate(row):
                if x:
                    dfs(i, j, 0)
                    paths.add("".join(path))

            path.clear()

        return len(paths)

```

◆ Quick Review

Key Insights

- Use DFS/BFS to traverse and mark each island.
- For each island, record its shape relative to a starting coordinate to allow translation invariance.
- Represent the island's shape as a unique signature (e.g., tuple or string of relative

positions).

- Use a set (or hash table) to store unique shapes.

Space & Time

Time Complexity: $O(m * n)$ as each cell is visited once.

Space Complexity: $O(m * n)$ in the worst-case due to recursion stack and storing island shapes.

Solution

Use depth-first search (DFS) to explore the grid. When an unvisited land cell (1) is encountered, initiate a DFS to traverse the entire island. Record each land cell's coordinates relative to the starting position of the island. This relative representation is invariant under translation, making it possible to compare island shapes. Store the canonical shape signature (for example, as a tuple or string) in a set to ensure uniqueness. Finally, return the size of the set as the number of distinct islands.

DFS

□ #695 Max Area of Island

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Union Find · Matrix
Lists: Denny Zhang

Problem

You are given an $m \times n$ binary matrix grid. An island is a group of 1's (representing land) connected 4-directionally (horizontal or vertical.) You may assume all four edges of the grid are surrounded by water.

The area of an island is the number of cells with a value 1 in the island.

Return the maximum area of an island in grid. If there is no island, return 0.

Example 1:

0	0	1	0	0	0	0	1	0	0	0	0	0
0	0	0	0	0	0	0	1	1	1	0	0	0
0	1	1	0	1	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	0	1	0	1	0	0
0	1	0	0	1	1	0	0	1	1	1	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	1	1	1	0	0	0
0	0	0	0	0	0	0	1	1	0	0	0	0

Input: grid = [[0,0,1,0,0,0,0,1,0,0,0,0,0],[0,0,0,0,0,0,0,1,1,1,0,0,0],[0,1,1,0,1,0,0,0,0,0,0,0,0],[0,1,0,0,1,1,0,0,1,0,1,0,0],[0,1,0,0,1,1,0,0,1,1,1,0,0],[0,0,0,0,0,0,0,0,0,0,0,1,0],[0,0,0,0,0,0,0,1,1,1,0,0,0],[0,0,0,0,0,0,0,0,1,1,0,0,0]]

Output: 6

Explanation: The answer is not 11, because the island must be connected 4-directionally.

Example 2:

Input: grid = [[0,0,0,0,0,0,0,0]]

Output: 0

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 50$
- $\text{grid}[i][j]$ is either 0 or 1.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Maximum area (cell count) of any connected island. Return 0 if no island. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

For each land cell, DFS to measure area of its island while marking visited.

Track the maximum area seen across all components.

Same flood-fill as Number of Islands, but accumulate area instead of count.

Time: $O(m * n)$. Space: $O(m * n)$ stack worst case.

Should I go ahead and optimize?"

```
class _695_MaxAreaIsland:
```

```
    def maxAreaOfIsland(self, grid: List[List[int]]) -> int:
```

```
        m, n = len(grid), len(grid[0])
```

```
        def dfs(r, c) -> int:
```

```
            if r < 0 or r >= m or c < 0 or c >= n or grid[r][c] != 1:
```

```
                return 0
```

```
            grid[r][c] = 0
```

```
            return 1 + dfs(r-1,c) + dfs(r+1,c) + dfs(r,c-1) + dfs(r,c+1)
```

```
        return max((dfs(r, c) for r in range(m) for c in range(n) if grid[r][c] == 1), default=0)
```

PHASE 5 — TEST & DEBUG

Each DFS floods an island and returns its cell count. max() over all islands. ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(m*n)$ stack. The return-value DFS pattern: each cell contributes 1 + sum of children."

This is the same DFS flood-fill as #200 (Number of Islands) with a count return.

Given more time I'd ask: mergetwo islands by flipping a '0'?

Enumerate each '0' cell, BFS to find adjacent distinct islands, sum their areas + 1."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def maxAreaOfIsland(self, grid: list[list[int]]) -> int:
        def dfs(i: int, j: int) -> int:
            if i < 0 or i == len(grid) or j < 0 or j == len(grid[0]):
                return 0
            if grid[i][j] != 1:
                return 0

            grid[i][j] = 2

            return 1 + dfs(i + 1, j) + dfs(i - 1, j) + dfs(i, j + 1) + dfs(i, j - 1)

        return max(dfs(i, j) for i in range(len(grid)) for j in range(len(grid[0])))

```

• LeetDoocs

```

class Solution:
    def maxAreaOfIsland(self, grid: List[List[int]]) -> int:
        def dfs(i: int, j: int) -> int:
            if grid[i][j] == 0:
                return 0
            ans = 1
            grid[i][j] = 0
            dirs = (-1, 0, 1, 0, -1)
            for a, b in pairwise(dirs):
                x, y = i + a, j + b

                if 0 <= x < m and 0 <= y < n:
                    ans += dfs(x, y)
            return ans

        m, n = len(grid), len(grid[0])
        return max(dfs(i, j) for i in range(m) for j in range(n))

```

• SimplyLeet

```

# Function to compute the maximum area of an island in a grid using DFS
def maxAreaOfIsland(grid):
    # Determine the dimensions of the grid
    rows = len(grid)
    cols = len(grid[0]) if rows else 0

    # Helper function for performing DFS on the grid
    def dfs(r, c):
        # Check if the current cell is out-of-bounds or is water
        if r < 0 or c < 0 or r >= rows or c >= cols or grid[r][c] == 0:
            return 0
        # Mark the current cell as visited by setting it to 0 (water)
        grid[r][c] = 0
        area = 1 # Count the current cell as part of the island
        # Recursively explore all four directions
        area += dfs(r + 1, c)
        area += dfs(r - 1, c)
        area += dfs(r, c + 1)
        area += dfs(r, c - 1)
        return area

    max_area = 0
    # Iterate through each cell in the grid
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:
                # Update max_area with the area obtained from the DFS
                max_area = max(max_area, dfs(r, c))
    return max_area

```

 • LC.ca

```

class Solution:
    def maxAreaOfIsland(self, grid: List[List[int]]) -> int:
        def dfs(i: int, j: int) -> int:
            if grid[i][j] == 0:
                return 0
            ans = 1
            grid[i][j] = 0
            dirs = (-1, 0, 1, 0, -1)
            for a, b in pairwise(dirs):
                x, y = i + a, j + b

                if 0 <= x < m and 0 <= y < n:
                    ans += dfs(x, y)
            return ans

        m, n = len(grid), len(grid[0])
        return max(dfs(i, j) for i in range(m) for j in range(n))

```

◆ Quick Review

Key Insights

- Traverse each cell in the grid.
- Use Depth-First Search (DFS) or Breadth-First Search (BFS) to explore connected 1's.
- Mark visited cells to prevent double counting (e.g., by setting them to 0).
- Maintain and update a maximum area count during traversal.
- The grid's boundaries are naturally water, so no extra edge processing is needed.

Space & Time

Time Complexity: $O(m * n)$ where m and n are the dimensions of the grid since every cell is visited at most once.

Space Complexity: $O(m * n)$ in the worst-case scenario for the recursion stack (or auxiliary data structures) when the grid is entirely land.

Solution

The solution leverages a DFS approach to iterate over each cell in the given grid. When a cell with a value of 1 is encountered, a DFS is initiated from that cell to calculate the area of the island connected to it. Cells are marked as visited (by changing them from 1 to 0) during the DFS to avoid revisiting. The DFS explores all four directions (up, down, left, right) and accumulates the count of connected cells. The maximum area found among all islands is returned at the end. This approach efficiently computes the largest island area with a simple traversal mechanism.

DFS

□ #721 Accounts Merge

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · DFS · BFS · Union Find
Lists: Grind 169

Problem

Given a list of accounts where each element `accounts[i]` is a list of strings, where the first element `accounts[i][0]` is a name, and the rest of the elements are emails representing emails of the account.

Now, we would like to merge these accounts. Two accounts definitely belong to the same person if there is some common email to both accounts. Note that even if two accounts have the same name, they may belong to different people as people could have the same name. A person can have any number of accounts initially, but all of their accounts definitely have the same name.

After merging the accounts, return the accounts in the following format: the first element of each account is the name, and the rest of the elements are emails in sorted order. The accounts themselves can be returned in any order.

Example 1:

Input: accounts = [["John", "johnsmith@mail.com", "john_newyork@mail.com"], ["John", "johnsmith@mail.com", "john00@mail.com"], ["Mary", "mary@mail.com"], ["John", "johnnybravo@mail.com"]]

Output: [["John", "john00@mail.com", "john_newyork@mail.com", "johnsmith@mail.com"], ["Mary", "mary@mail.com"], ["John", "johnnybravo@mail.com"]]

Explanation:

The first and second John's are the same person as they have the common email "johnsmith@mail.com".

The third John and Mary are different people as none of their email addresses are used by other accounts.

We could return these lists in any order, for example the answer [['Mary', 'mary@mail.com'], ['John', 'johnnybravo@mail.com'], ['John', 'john00@mail.com', 'john_newyork@mail.com', 'johnsmith@mail.com']] would still be accepted.

Example 2:

Input: accounts = [["Gabe", "Gabe0@m.co", "Gabe3@m.co", "Gabe1@m.co"], ["Kevin", "Kevin3@m.co", "Kevin5@m.co", "Kevin0@m.co"], ["Ethan", "Ethan5@m.co", "Ethan4@m.co", "Ethan0@m.co"], ["Hanzo", "Hanzo3@m.co", "Hanzo1@m.co", "Hanzo0@m.co"], ["Fern", "Fern5@m.co", "Fern1@m.co", "Fern0@m.co"]]

Output: [["Ethan", "Ethan0@m.co", "Ethan4@m.co", "Ethan5@m.co"], ["Gabe", "Gabe0@m.co", "Gabe1@m.co", "Gabe3@m.co"], ["Hanzo", "Hanzo0@m.co", "Hanzo1@m.co", "Hanzo3@m.co"], ["Kevin", "Kevin0@m.co", "Kevin3@m.co", "Kevin5@m.co"], ["Fern", "Fern0@m.co", "Fern1@m.co", "Fern5@m.co"]]

Constraints:

- $1 \leq \text{accounts.length} \leq 1000$
- $2 \leq \text{accounts}[i].\text{length} \leq 10$
- $1 \leq \text{accounts}[i][j].\text{length} \leq 30$

- `accounts[i][0]` consists of English letters.
- `accounts[i][j]` (for $j > 0$) is a valid email.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Merge accounts that share at least one email. Same name ≠ same person.
# Output: name + sorted merged emails. Any output order. Right?"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Union-Find on emails: union every email in the same account list.
# Group emails by root parent; sort each merged list for output.
# Time:  $O(N \log N)$  for sorting emails. Space:  $O(N)$  DSU.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (DFS on email graph)

```
# "Build a graph where emails in the same account are connected.
# DFS collects all emails in a connected component = one merged account.
# Track owner (account name) via the first email in each account."
```

PHASE 4 — CLEAN CODE

```
class _721_AccountsMerge:
    def accountsMerge(self, accounts: List[List[str]]) -> List[List[str]]:
        email_to_name : dict = {}
        email_to_emails : defaultdict = defaultdict(set)
        for account in accounts:
            name = account[0]
            first = account[1]
            for email in account[1:]:
                email_to_name[email] = name
                email_to_emails[first].add(email)
                email_to_emails[email].add(first)

        visited: set = set()
        result = []
        def dfs(email, component):
            visited.add(email)
            component.append(email)
            for nb in email_to_emails[email]:
                if nb not in visited:
                    dfs(nb, component)

        for email in email_to_name:
```

```

        if email not in visited:
            component: List[str] = []
            dfs(email, component)
            result.append([email_to_name[email]] + sorted(component))
    return result

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(E \log E)$ where E = total emails (sort dominates). Space $O(E)$.
 # Union-Find alternative: union all emails in an account, group by root.
 # DFS on the email graph is cleaner to explain – same complexity.
 # Given more time I'd ask: if same email can belong to different names?
 # That would mean the data is inconsistent – flag it as an error."

Community Solutions (Python)

• LeetDoocs

```

class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a, b):
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]

```

```

        return True

class Solution:
    def accountsMerge(self, accounts: List[List[str]]) -> List[List[str]]:
        uf = UnionFind(len(accounts))
        d = {}
        for i, (_, *emails) in enumerate(accounts):
            for email in emails:
                if email in d:
                    uf.union(i, d[email])
                else:
                    d[email] = i
        g = defaultdict(set)
        for i, (_, *emails) in enumerate(accounts):
            root = uf.find(i)
            g[root].update(emails)
        return [[accounts[root][0]] + sorted(emails) for root, emails in g.items()]

```

• SimplyLeet

```

# Python solution for Accounts Merge

# Define the UnionFind class for disjoint set operations
class UnionFind:
    def __init__(self):
        self.parent = {}

    # Find with path compression
    def find(self, x):
        if self.parent.setdefault(x, x) != x:

```

```
        self.parent[x] = self.find(self.parent[x])
    return self.parent[x]

# Union two nodes
def union(self, x, y):
    self.parent[self.find(x)] = self.find(y)

class Solution:
    def accountsMerge(self, accounts):
        uf = UnionFind()

        email_to_name = {}

        # Iterate over each account and union emails
        for account in accounts:
            name = account[0]
            first_email = account[1]
            for email in account[1:]:
                uf.union(first_email, email)
                email_to_name[email] = name

        # Group emails by their root parent
        groups = {}
        for email in email_to_name:
            parent = uf.find(email)
            groups.setdefault(parent, []).append(email)

        # Return the merged account list with sorted emails
        result = []
        for parent, emails in groups.items():
            result.append([email_to_name[parent]] + sorted(emails))

        return result

# Example usage:
# accounts = [["John","johnsmith@mail.com","john_newyork@mail.com"],
# ["John","johnsmith@mail.com","john00@mail.com"],
# ["Mary","mary@mail.com"],
# ["John","johnnybravo@mail.com"]]
# solution = Solution()
# print(solution.accountsMerge(accounts))
```

• LC.ca

```
class Solution:
    def accountsMerge(self, accounts: List[List[str]]) -> List[List[str]]:
        def find(x):
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        n = len(accounts)
        p = list(range(n))
        email_id = {}

        for i, account in enumerate(accounts):
            name = account[0]
            for email in account[1:]:
                if email in email_id:
                    p[find(i)] = find(email_id[email])
                else:
                    email_id[email] = i

        mp = defaultdict(set)
        for i, account in enumerate(accounts):
            for email in account[1:]:
                mp[find(i)].add(email)

        ans = []
        for i, emails in mp.items():
            t = [accounts[i][0]]
            t.extend(sorted(emails))
            ans.append(t)
        return ans
```

◆ Quick Review

Key Insights

- Two accounts are merged if they share at least one common email.

- Treat each email as a node in a graph; accounts represent connected components.
- Both DFS/BFS and Union-Find can be used to find connected components.
- Keep a mapping from email to name because all emails in a connected component belong to the same person.
- After grouping, sort the emails and prepend the name to form the final account list.

Space & Time

Time Complexity: $O(N * M * \log(M))$ where N is the number of accounts and M is the number of emails per account (owing to sorting and union operations).

Space Complexity: $O(N * M)$, used for storing email mappings and union-find structures.

Solution

The solution involves using a Union-Find (Disjoint Set Union) data structure to merge emails that are connected by being on the same account. For each account, we union the first email with every other email in that account. We also maintain a mapping from email to owner name. After processing all

accounts, we group the emails by their root parent derived from the union–find structure. For each group, we sort the emails and prepend the owner's name. Finally, we return the merged list of accounts.

DFS

□ #785 Is Graph Bipartite?

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
DFS · BFS · Union Find · Graph
Lists: Denny Zhang

Problem

There is an undirected graph with n nodes, where each node is numbered between 0 and $n - 1$. You are given a 2D array `graph`, where `graph[u]` is an array of nodes that node u is adjacent to. More formally, for each v in `graph[u]`, there is an undirected edge between node u and node v . The graph has the following properties:

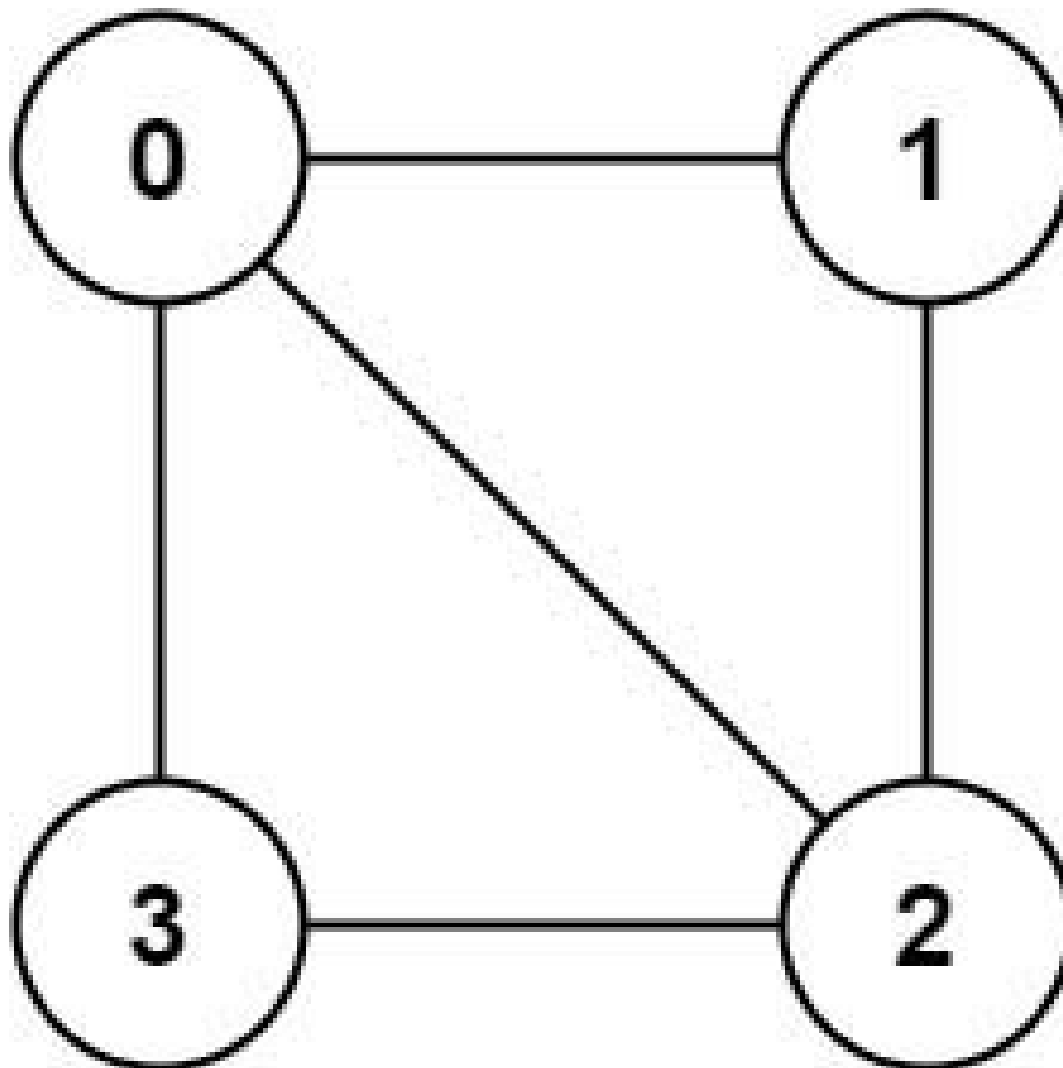
- There are no self-edges (`graph[u]` does not contain u).
- There are no parallel edges (`graph[u]` does not contain duplicate values).
- If v is in `graph[u]`, then u is in `graph[v]` (the graph is undirected).

- The graph may not be connected, meaning there may be two nodes u and v such that there is no path between them.

A graph is bipartite if the nodes can be partitioned into two independent sets A and B such that every edge in the graph connects a node in set A and a node in set B .

Return true if and only if it is bipartite.

Example 1:

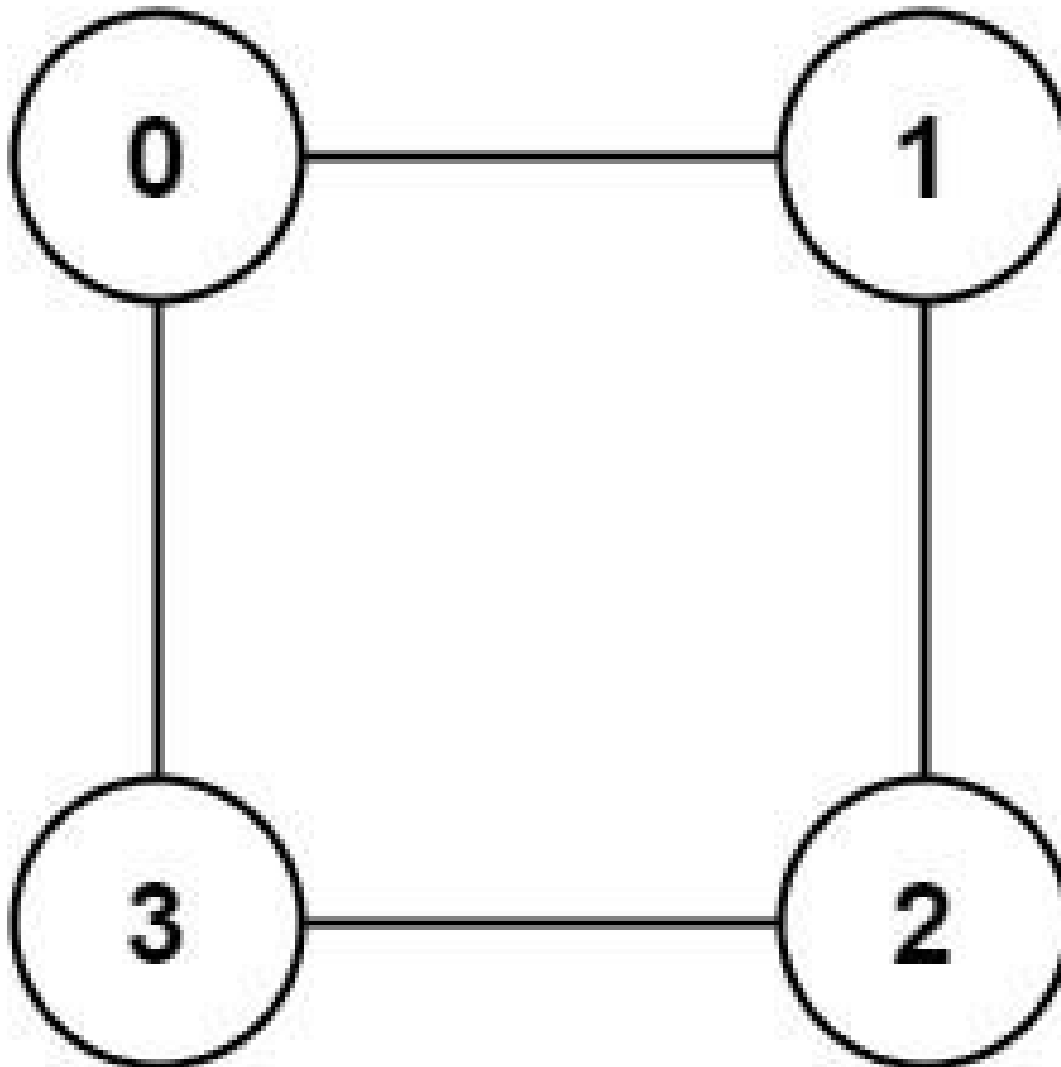


Input: graph = [[1,2,3],[0,2],[0,1,3],[0,2]]

Output: false

Explanation: There is no way to partition the nodes into two independent sets such that every edge connects a node in one and a node in the other.

Example 2:



Input: graph = [[1,3],[0,2],[1,3],[0,2]]

Output: true

Explanation: We can partition the nodes into two sets: {0, 2} and {1, 3}.

Constraints:

- graph.length == n
- 1 <= n <= 100

- $0 \leq \text{graph}[u].\text{length} < n$
- $0 \leq \text{graph}[u][i] \leq n - 1$
- $\text{graph}[u]$ does not contain u .
- All the values of $\text{graph}[u]$ are unique.
- If $\text{graph}[u]$ contains v , then $\text{graph}[v]$ contains u .

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Can we 2-colour the graph so no two adjacent nodes share the same colour?
Graph may be disconnected – check every component. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."
Try to 2-color the graph with BFS/DFS: assign color 0/1 alternating across edges.
If any edge connects two nodes of the same color, graph is not bipartite.
Standard coloring check on an undirected graph.
Time: $O(V + E)$. Space: $O(V)$ color + visited.
Should I go ahead and optimize?"

```
class _785_IsGraphBipartite:
    def isBipartite(self, graph: List[List[int]]) -> bool:
        color = {}
        def bfs(start):
            queue = deque([start])
            color[start] = 0
            while queue:
                node = queue.popleft()
                for nb in graph[node]:
                    if nb not in color:
                        color[nb] = 1 - color[node]
                        queue.append(nb)
                    elif color[nb] == color[node]:
                        return False
            return True
        return all(bfs(n) for n in range(len(graph)) if n not in color)
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(V+E)$. Space $O(V)$. 2-coloring via BFS – odd-length cycle → not bipartite.

Given more time I'd ask: what if we need to output the two sets?

Return {n for n in color if color[n]==0} and {n for n in color if color[n]==1}."

Community Solutions (Python)

• WalkCC

```
from enum import Enum
```

```
class Color(Enum):
```

```
    WHITE = 0
```

```
    RED = 1
```

```
    GREEN = 2
```

```
class Solution:
```

```
    def isBipartite(self, graph: list[list[int]]) -> bool:
```

```
        colors = [Color.WHITE] * len(graph)
```

```
        for i in range(len(graph)):
```

```
            # This node has been colored, so do nothing.
```

```
            if colors[i] != Color.WHITE:
```

```
                continue
```

```
            # Always paint red for a white node.
```

```
            colors[i] = Color.RED
```

```
            # BFS.
```

```
            q = collections.deque([i])
```

```
            while q:
```

```
                for _ in range(len(q)):
```

```
                    u = q.popleft()
```

```
                    for v in graph[u]:
```

```
                        if colors[v] == colors[u]:
```

```
                            return False
```

```
                        if colors[v] == Color.WHITE:
```

```
                            colors[v] = Color.RED if colors[u] == Color.GREEN else Color.GRE
```

```
            EN
```

```
            q.append(v)
```

```
return True
```

• LeetCode

```
class Solution:
    def isBipartite(self, graph: List[List[int]]) -> bool:
        def dfs(a: int, c: int) -> bool:
            color[a] = c
            for b in graph[a]:
                if color[b] == c or (color[b] == 0 and not dfs(b, -c)):
                    return False
            return True

        n = len(graph)
```

```
        color = [0] * n
        for i in range(n):
            if color[i] == 0 and not dfs(i, 1):
                return False
        return True
```

```
class Solution:
    def isBipartite(self, graph: List[List[int]]) -> bool:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        p = list(range(len(graph)))
        for a, bs in enumerate(graph):
            for b in bs:
```

```
                pa, pb = find(a), find(b)
                if pa == pb:
                    return False
                p[pb] = find(bs[0])
        return True
```

• SimplyLeet

```

# Using Breadth-First Search (BFS)
from collections import deque

def isBipartite(graph):
    n = len(graph)
    # color array: -1 means uncolored, 0 and 1 are the two colors.
    colors = [-1] * n

    # Iterate through each node to handle disconnected components
    for i in range(n):

        if colors[i] == -1:
            # Start BFS from node i with color 0
            queue = deque([i])
            colors[i] = 0
            while queue:
                node = queue.popleft()
                # Check each neighbor of the current node
                for neighbor in graph[node]:
                    if colors[neighbor] == -1:
                        # Assign the opposite color to the neighbor

                        colors[neighbor] = 1 - colors[node]
                        queue.append(neighbor)
                    elif colors[neighbor] == colors[node]:
                        # Found a neighbor with the same color: not bipartite
                        return False

            return True

# Example usage:
if __name__ == '__main__':
    graph_example = [[1,3],[0,2],[1,3],[0,2]]

    print(isBipartite(graph_example)) # Expected output: True

```

 • LC.ca


```
class Solution:
    def isBipartite(self, graph: List[List[int]]) -> bool:
        def dfs(u, c):
            color[u] = c
            for v in graph[u]:
                if not color[v]:
                    if not dfs(v, 3 - c):
                        return False
                elif color[v] == c:
                    return False

            return True

        n = len(graph)
        color = [0] * n
        for i in range(n):
            if not color[i] and not dfs(i, 1):
                return False
        return True
```

◆ Quick Review

Key Insights

- A graph is bipartite if you can assign two colors to each node such that no two adjacent nodes share the same color.
- The challenge may require exploring disconnected components.
- BFS or DFS can be used to attempt the two-coloring, and a conflict during coloring indicates the graph is not bipartite.
- Union Find is an alternative approach, but BFS/DFS is more intuitive for graph coloring

problems.

Space & Time

Time Complexity: $O(V + E)$ where V is the number of nodes and E is the number of edges, since every node and edge is traversed at most once.

Space Complexity: $O(V)$, for the color array and the queue (or recursion stack) used in BFS (or DFS).

Solution

We use a BFS/DFS approach to assign colors (0 and 1) to nodes. A color array, initially set to -1 for all nodes, is maintained. For each unvisited node (color -1), we start a BFS (or DFS) and assign it an initial color (e.g., 0). For every visited neighbor, if it has not been colored, assign it the opposite color; if it is already colored with the same color as the current node, a conflict has occurred, and the graph is not bipartite. This method gracefully handles disconnected components by iterating over all nodes.

DFS

□ ★ #1236 Web Crawler ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · DFS · BFS · Interactive
Lists: ★ Premium | Premium 98

Problem

Given a url `startUrl` and an interface `HtmlParser`, implement a web crawler to crawl all links that are under the same hostname as `startUrl`.

Return all urls obtained by your web crawler in any order.

Your crawler should:

- Start from the page: `startUrl`
- Call `HtmlParser.getUrls(url)` to get all urls from a webpage of given url.
- Do not crawl the same link twice.
- Explore only the links that are under the same hostname as `startUrl`.

http://example.org:8888/foo/bar#bang

hostname

host

As shown in the example url above, the hostname is example.org. For simplicity sake, you may assume all urls use http protocol without any port specified. For example, the urls `http://leetcode.com/problems` and `http://leetcode.com/contest` are under the same hostname, while urls `http://example.org/test` and `http://example.com/abc` are not under the same hostname.

The `HtmlParser` interface is defined as such:

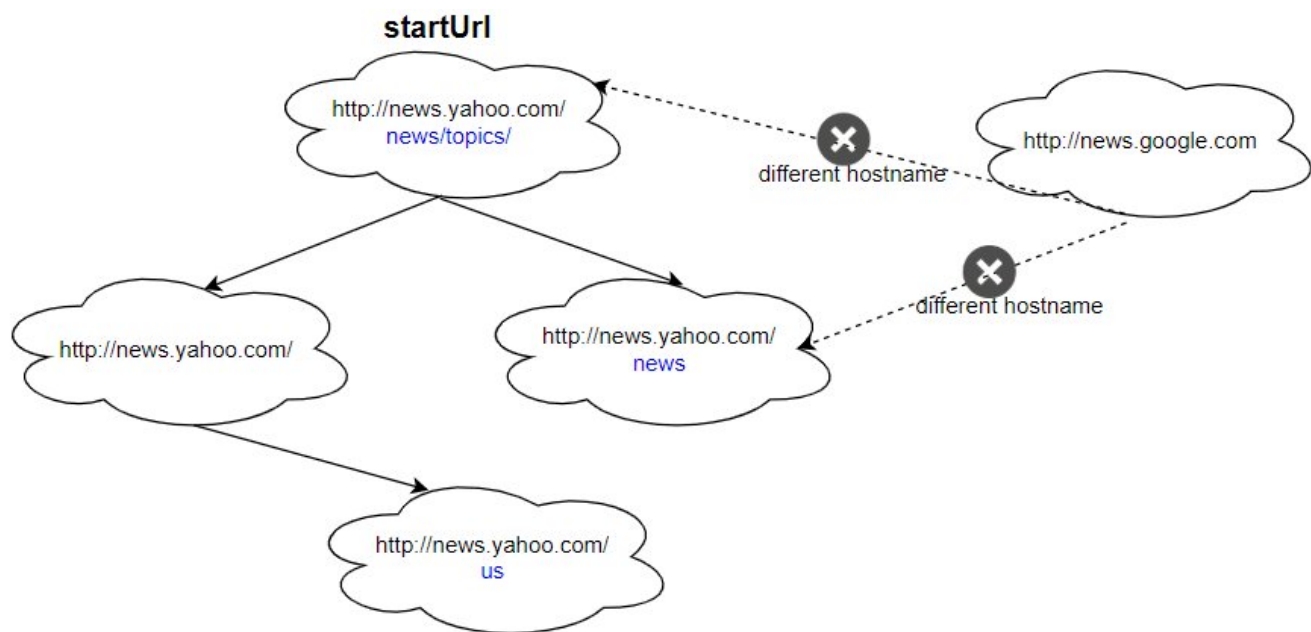
```
interface HtmlParser {  
    // Return a list of all urls from a webpage of given url.  
    public List<String> getUrls(String url);  
}
```

Below are two examples explaining the functionality of the problem, for custom testing purposes you'll have three variables `urls`, `edges` and `startUrl`. Notice that you will only have access to `startUrl` in your code, while `urls` and `edges` are not directly

accessible to you in code.

Note: Consider the same URL with the trailing slash "/" as a different URL. For example, "http://news.yahoo.com", and "http://news.yahoo.com/" are different urls.

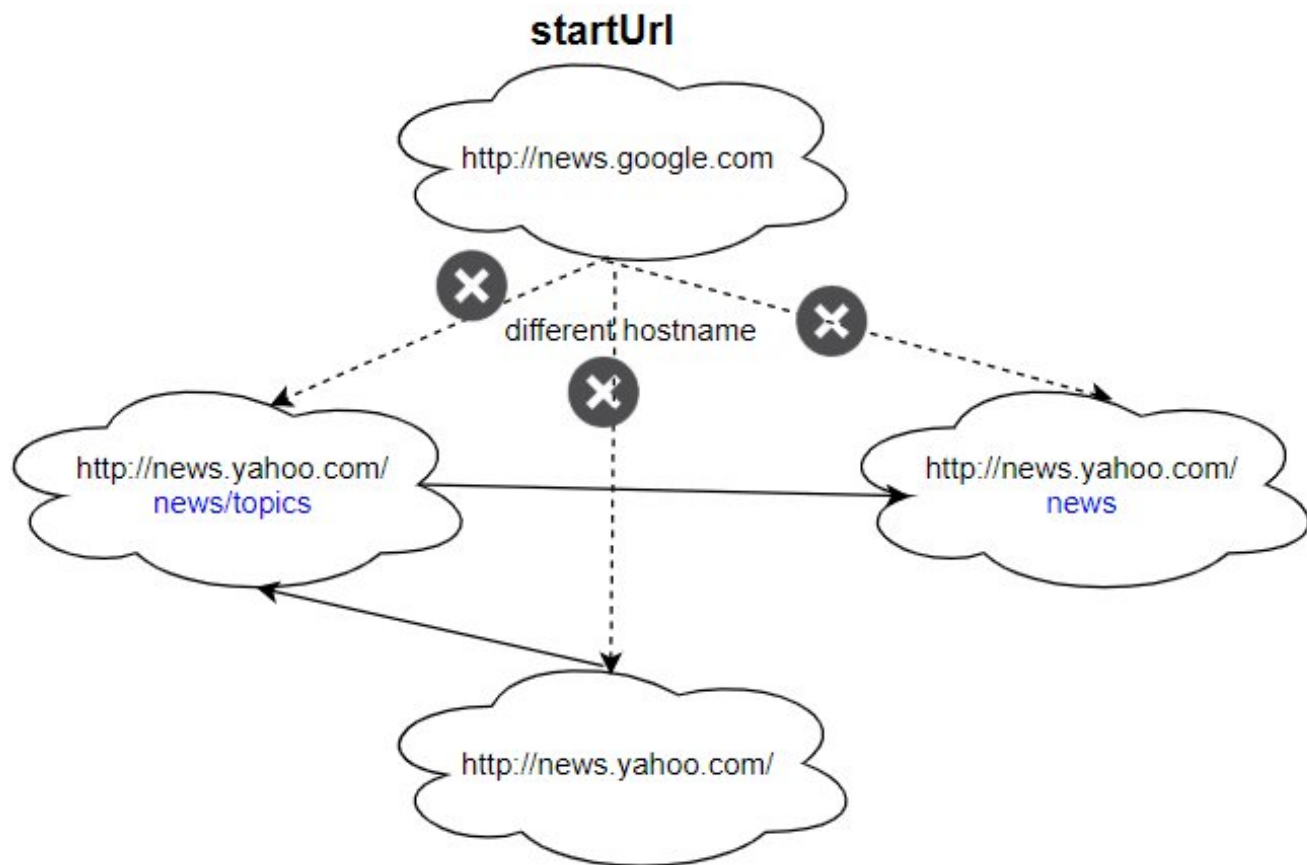
Example 1:



Input:

```
urls = [
    "http://news.yahoo.com",
    "http://news.yahoo.com/news",
    "http://news.yahoo.com/news/topics/",
    "http://news.google.com",
    "http://news.yahoo.com/us"
]
```

Example 2:



Input:

```
urls = [
    "http://news.yahoo.com",
    "http://news.yahoo.com/news",
    "http://news.yahoo.com/news/topics/",
    "http://news.google.com"
]
edges = [[0,2],[2,1],[3,2],[3,1],[3,0]]
```

Constraints:

- $1 \leq \text{urls.length} \leq 1000$
- $1 \leq \text{urls}[i].\text{length} \leq 300$
- startUrl is one of the urls.
- Hostname label must be from 1 to 63 characters long, including the dots, may contain only the ASCII letters from 'a' to 'z',

digits from '0' to '9' and the hyphen-minus character ('-').

- The hostname may not start or end with the hyphen-minus character ('-').
- See: https://en.wikipedia.org/wiki/Hostname#Restrictions_on_valid_hostnames
- You may assume there're no duplicates in url library.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "BFS from startUrl. Only follow links on the same hostname. Don't revisit. Right?
# Hostname = part between '//' and the next '/'."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# BFS from startUrl: fetch page, parse all links on same hostname, enqueue
unvisited.
# Use a visited set keyed by URL string to avoid cycles.
# Standard single-threaded web crawler simulation.
# Time: O(V + E) pages + links. Space: O(V) visited.
# Should I go ahead and optimize?"
```

```
class _1236_WebCrawler:
    def crawl(self, startUrl: str, htmlParser: 'HtmlParser') -> List[str]:
        def hostname(url):
            return url.split('/')[2] # 'http:' , '' , 'news.yahoo.com', ...
        host = hostname(startUrl)
        visited = {startUrl}
        queue = deque([startUrl])
        while queue:
            url = queue.popleft()
            for link in htmlParser.getUrls(url):
                if link not in visited and hostname(link) == host:
                    visited.add(link)
```



```
        queue.append(link)
    return list(visited)
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(V+E)$ where V =pages, E =links. Space $O(V)$.

Standard BFS graph traversal – the only novelty is hostname filtering.

Given more time I'd ask: #1242 Multithreaded? Use a ThreadPoolExecutor,

submit getUrls calls as futures, use a thread-safe visited set."

Community Solutions (Python)

• WalkCC

```
# """
# This is HtmlParser's API interface.
# You should not implement it, or speculate about its implementation
# """
# Class HtmlParser(object):
# def getUrls(self, url: str) -> list[str]:

class Solution:
    def crawl(self, startUrl: str, htmlParser: 'HtmlParser') -> list[str]:
        q = collections.deque([startUrl])

        seen = {startUrl}
        hostname = startUrl.split('/')[2]

        while q:
            currUrl = q.popleft()
            for url in htmlParser.getUrls(currUrl):
                if url in seen:
                    continue
                if hostname in url:
                    q.append(url)

            seen.add(url)

        return seen
```

• LeetDoocs

```
# """
# This is HtmlParser's API interface.
# You should not implement it, or speculate about its implementation
# """
# class HtmlParser(object):
# def getUrls(self, url):
# """
# :type url: str
# :rtype List[str]
# """
```

```
class Solution:
    def crawl(self, startUrl: str, htmlParser: 'HtmlParser') -> List[str]:
        def host(url):
            url = url[7:]
            return url.split('/')[0]

        def dfs(url):
            if url in ans:
                return
            ans.add(url)
            for next in htmlParser.getUrls(url):
                if host(url) == host(next):
                    dfs(next)

        ans = set()
        dfs(startUrl)
        return list(ans)
```

• SimplyLeet

Python Code Solution

```
class Solution:
    def crawl(self, startUrl: str, htmlParser: 'HtmlParser') -> list[str]:
        from collections import deque

        # Helper function to extract hostname from a url string.
        def get_hostname(url: str) -> str:
            # Remove protocol prefix "http://"
            hostname = url.split("://")[1]

            # The hostname is the part before the first "/" (if exists)
            return hostname.split("/")[0]

        # Extract the hostname from the starting URL.
        start_hostname = get_hostname(startUrl)

        # Initialize a set to keep track of visited urls and a queue for BFS.
        visited = set([startUrl])
        queue = deque([startUrl])

        # Perform BFS traversal of the web links.
        while queue:
            current_url = queue.popleft()
            # Retrieve all URLs found at the current URL.
            for url in htmlParser.getUrls(current_url):
                # Check if URL is under the same hostname and has not been
                # visited.
                if url not in visited and get_hostname(url) == start_hostname:
                    visited.add(url)
                    queue.append(url)

        # Return all visited URLs as a list.
        return list(visited)
```

• LC.ca

```
# """
# This is HtmlParser's API interface.
# You should not implement it, or speculate about its implementation
# """
# class HtmlParser(object):
# def getUrls(self, url):
# """
# :type url: str
# :rtype List[str]
# """

from collections import deque

class Solution:
    def crawl(self, startUrl: str, htmlParser: 'HtmlParser') -> List[str]:
        visited = set()
        domain = self.get_domain(startUrl)
        queue = deque([startUrl])
        visited.add(startUrl)

        while queue:
            current_url = queue.popleft()
            for url in htmlParser.getUrls(current_url):
                if url not in visited and self.get_domain(url) == domain:
                    visited.add(url)
                    queue.append(url)

        return list(visited)

    def get_domain(self, url: str) -> str:

        return url.split('/')[2]
```

◆ Quick Review

Key Insights

- Treat the set of webpages as a graph where each URL is a node and an edge exists from one URL to another if it is returned by `HtmlParser.getUrls(url)`.
- Use either breadth-first search (BFS) or depth-first search (DFS) to traverse the webpages.
- Extract the hostname from a URL to ensure that only URLs on the same domain are crawled.
- Maintain a visited set (or equivalent) to avoid processing the same URL multiple times.

Space & Time

Time Complexity: $O(N + E)$ where N is the number of nodes (URLs) and E is the number of edges (links between URLs) encountered during the crawl.

Space Complexity: $O(N)$ for storing the visited URLs and the queue/stack used for traversal.

Solution

The solution uses a BFS (or DFS) approach to traverse the web graph: Extract the hostname from `startUrl` (for instance, by splitting on `"://"` and then the first `"/"` that follows). Initialize a

visited set to keep track of URLs that have been crawled. Use a queue to implement the BFS. Enqueue the `startUrl`. While the queue is not empty, dequeue a URL and retrieve its outgoing links using `HtmlParser.getUrls(url)`. For each URL obtained: Extract its hostname and compare it with the hostname of `startUrl`. If it matches and it has not been visited before, add it to the visited set and enqueue it. Finally, return the set of visited URLs. This approach guarantees that each URL is processed only once and that only URLs from the same hostname are crawled.

DFS

□ #1242 Web Crawler Multithreaded

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode

DFS · BFS · Concurrency

Lists: Denny Zhang

Problem

Given a URL `startUrl` and an interface `HtmlParser`, implement a Multi-threaded web crawler to crawl all links that are under the same hostname as `startUrl`.

Return all URLs obtained by your web crawler in any order.

Your crawler should:

- Start from the page: `startUrl`
- Call `HtmlParser.getUrls(url)` to get all URLs from a webpage of a given URL.
- Do not crawl the same link twice.
- Explore only the links that are under the same hostname as `startUrl`.

http://example.org:8888/foo/bar#bang

hostname

host

As shown in the example URL above, the hostname is example.org. For simplicity's sake, you may assume all URLs use HTTP protocol without any port specified. For example, the URLs `http://leetcode.com/problems` and `http://leetcode.com/contest` are under the same hostname, while URLs `http://example.org/test` and `http://example.com/abc` are not under the same hostname.

The `HtmlParser` interface is defined as such:

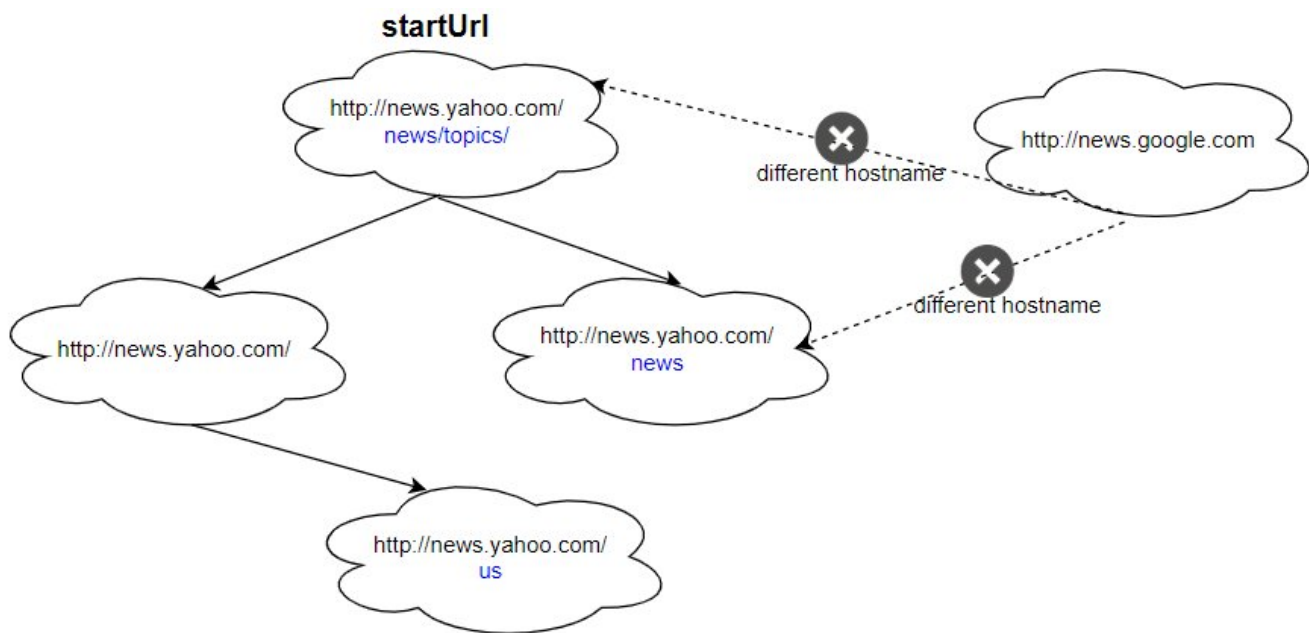
```
interface HtmlParser {  
    // Return a list of all urls from a webpage of given url.  
    // This is a blocking call, that means it will do HTTP request and return when  
    // this request is finished.  
    public List<String> getUrls(String url);  
}
```

Note that `getUrls(String url)` simulates performing an HTTP request. You can treat it as a blocking function call that waits for an HTTP request to finish. It is guaranteed that `getUrls(String url)` will return the URLs within 15ms. Single-threaded solutions will exceed the

time limit so, can your multi-threaded web crawler do better?

Below are two examples explaining the functionality of the problem. For custom testing purposes, you'll have three variables `urls`, `edges` and `startUrl`. Notice that you will only have access to `startUrl` in your code, while `urls` and `edges` are not directly accessible to you in code.

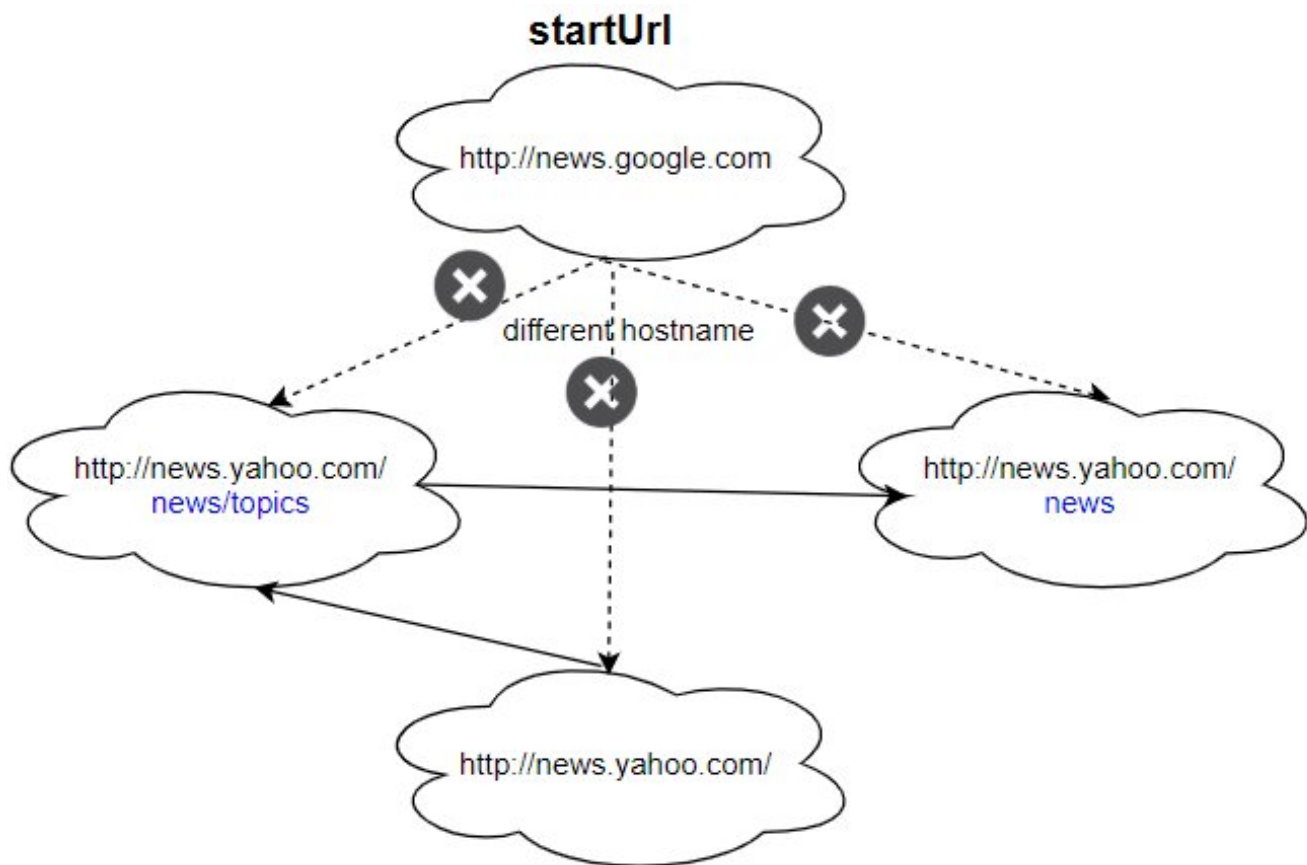
Example 1:



Input:

```
urls = [  
    "http://news.yahoo.com",  
    "http://news.yahoo.com/news",  
    "http://news.yahoo.com/news/topics/",  
    "http://news.google.com",  
    "http://news.yahoo.com/us"  
]
```

Example 2:



Input:

```
urls = [
    "http://news.yahoo.com",
    "http://news.yahoo.com/news",
    "http://news.yahoo.com/news/topics/",
    "http://news.google.com"
]
edges = [[0,2],[2,1],[3,2],[3,1],[3,0]]
```

Constraints:

- $1 \leq \text{urls.length} \leq 1000$
- $1 \leq \text{urls}[i].\text{length} \leq 300$
- startUrl is one of the urls.
- Hostname label must be from 1 to 63 characters long, including the dots, may contain only the ASCII letters from 'a' to 'z',

digits from '0' to '9' and the hyphen-minus character ('-').

- The hostname may not start or end with the hyphen-minus character ('-').
- See: https://en.wikipedia.org/wiki/Hostname#Restrictions_on_valid_hostnames
- You may assume there're no duplicates in the URL library.

Follow up:

1. Assume we have 10,000 nodes and 1 billion URLs to crawl. We will deploy the same software onto each node. The software can know about all the nodes. We have to minimize communication between machines and make sure each node does equal amount of work. How would your web crawler design change?
2. What if one node fails or does not work?
3. How do you know when the crawler is done?

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Same as #1236 but with multi-threading since getUrls is blocking and slow.
# Use threads to fetch pages in parallel. Thread-safe visited set required. Right?"
```

```
# PHASE 4 — CLEAN CODE (threading approach)
```

```
from concurrent.futures import ThreadPoolExecutor, as_completed
import threading
```

```
# PHASE 2 — BRUTE FORCE
```

```
# "Let me start with brute force to lock in the logic.
```

```
# Same BFS crawler as single-threaded, but dispatch each getLinks call to a worker thread.
```

```
# Shared visited set must be synchronized; join threads when queue is empty.
```

```
# Correct parallelism model for the LeetCode concurrency variant.
```

```
# Time:  $O(V + E)$  with parallel fetch. Space:  $O(V)$  visited + thread overhead.
```

```
# Should I go ahead and optimize?"
```

```
class _1242_WebCrawlerMT:
```

```
    def crawl(self, startUrl: str, htmlParser: 'HtmlParser') -> List[str]:
```

```
        def hostname(url):
```

```
            return url.split('/')[2]
```

```
        host = hostname(startUrl)
```

```
        visited = {startUrl}
```

```
        lock = threading.Lock()
```

```
        def fetch(url):
```

```
            neighbors = []
```

```
            for link in htmlParser.getUrls(url):
```

```
                with lock:
```

```
                    if link not in visited and hostname(link) == host:
```

```
                        visited.add(link)
```

```
                        neighbors.append(link)
```

```
            return neighbors
```

```
        queue = [startUrl]
```

```
        with ThreadPoolExecutor() as executor:
```

```
            while queue:
```

```
                futures = {executor.submit(fetch, url): url for url in queue}
```

```
                queue = []
```

```
                for f in as_completed(futures):
```

```
                    queue.extend(f.result())
```

```
        return list(visited)
```

```
# PHASE 6 — COMPLEXITY & FOLLOW-UP
```

```
# "Threads cut wall-clock time proportional to how much getUrls blocks.
```

```
# Lock only needed when checking/adding to visited – minimize critical section.
```

```
# Distributed follow-up (10,000 nodes, 1B URLs): consistent hashing to partition URLs by hostname.
```

```
# Each node owns a hostname range. Communicate frontier pages between nodes.
```

```
# 'Done' signal: all queues empty and all workers idle (barrier synchronisation)."
```

Community Solutions (Python)

• SimplyLeet

```
import threading
from queue import Queue
from urllib.parse import urlparse

def crawl(startUrl, htmlParser):
    # Extract hostname of the startUrl
    start_host = urlparse(startUrl).netloc
    # Thread-safe set for visited URLs and a lock for safety
    visited = set()
    visited_lock = threading.Lock()

    # Queue to process URLs; initialized with startUrl
    url_queue = Queue()
    url_queue.put(startUrl)

    def worker():
        while True:
            try:
                current_url = url_queue.get(timeout=1) # timeout to avoid
                blocking indefinitely
            except:
                break
            # Retrieve URLs from the current page using htmlParser
            for next_url in htmlParser.getUrls(current_url):
                # Check if next_url shares the same hostname as startUrl
                if urlparse(next_url).netloc == start_host:
                    with visited_lock:
                        if next_url not in visited:
                            visited.add(next_url)
                            url_queue.put(next_url)
            url_queue.task_done()
```

```

# Mark startUrl as visited
with visited_lock:
    visited.add(startUrl)

# Launch worker threads (8 threads used as example)
threads = []
for _ in range(8):
    t = threading.Thread(target=worker)
    t.start()

    threads.append(t)

# Wait until all URLs have been processed
url_queue.join()

# Ensure all threads terminate
for t in threads:
    t.join()

return list(visited)

```

• LC.ca

```

from threading import Thread, Lock
from urllib.parse import urlparse # To parse URLs and extract hostname for
comparison
from typing import List

class Solution:
    def crawl(self, startUrl: str, htmlParser: 'HtmlParser') -> List[str]:
        hostname = urlparse(startUrl).hostname
        seen = set([startUrl]) # Keep track of seen URLs to avoid re-crawling
the same URL
        lock = Lock() # Lock for thread-safe operations on the seen set

```

```

def dfs(url):
    for next_url in htmlParser.getUrls(url):
        if urlparse(next_url).hostname == hostname and next_url not in
seen:
            with lock: # Ensure thread-safe write to the seen set
                if next_url not in seen:
                    seen.add(next_url)
                    dfs(next_url)

def worker():
    while True:

        with lock:
            if not unseen:
                return
            url = unseen.pop()
            dfs(url)

unseen = [startUrl] # Stack of URLs to be crawled
threads = []
num_threads = 10 # Or any number you deem appropriate based on the
problem constraints

# Start threads
for _ in range(num_threads):
    thread = Thread(target=worker)
    thread.start()
    threads.append(thread)

# Wait for all threads to finish
for thread in threads:
    thread.join()

return list(seen)

```

```
#####
```



```
# reference https://levelup.gitconnected.com/multi-threaded-python-web-crawler-for-https-pages-e103f0839b71
```

```
class Crawler(threading.Thread):  
    def __init__(self,base_url, links_to_crawl,have_visited, error_links,url_lock):
```

```
        threading.Thread.__init__(self)  
        print(f"Web Crawler worker {threading.current_thread()} has Started")  
        self.base_url = base_url  
        self.links_to_crawl = links_to_crawl
```

```
        self.have_visited = have_visited  
        self.error_links = error_links  
        self.url_lock = url_lock
```

```
    def run(self):  
        # we create a ssl context so that our script can crawl  
        # the https sties with ssl_handshake.  
  
        #Create a SSLContext object with default settings.  
        my_ssl = ssl.create_default_context()
```

```
        # by default when creating a default ssl context and making an  
        handshake  
        # we verify the hostname with the certificate but our objective is  
        to crawl  
        # the webpage so we will not be checking the validity of the  
        cerfificate.  
        my_ssl.check_hostname = False  
  
        # in this case we are not verifying the certificate and any  
        # certificate is accepted in this mode.  
        my_ssl.verify_mode = ssl.CERT_NONE
```

```
our      # we are defining an infinite while loop so that all the links in
        # queue are processed.

        while True:

            # In this part of the code we create a global lock on our queue
of        # links so that no two threads can access the queue at same
time     self.url_lock.acquire()
        print(f"Queue Size: {self.links_to_crawl.qsize()}")
        link = self.links_to_crawl.get()

        self.url_lock.release()

        # if the link is None the queue is exhausted or the threads are
yet      # process the links.

        if link is None:
            break

        # if The link is already visited we break the execution.
        if link in self.have_visited:

            print(f"The link {link} is already visited")
            break

        try:
            # This method constructs a full "absolute" URL by combining
the       # base url with other url. this uses components of the base
URL,      # in particular the addressing scheme, the network
        # location and the path, to provide missing components
        # in the relative URL.
        # in short we repair our relative url if it is broken.
```

```

        link = urljoin(self.base_url, link)

        # we use the header parameter to "spoof" the "User-Agent"
header
        # value which is used by the browser to identify itself.
This is
        # because some servers will only allow the connection if it
comes
        # from a verified browser. In this case we are using
Firefox header.
        req = Request(link, headers= {'User-Agent': 'Mozilla/5.0'})
    )

    # we are opening the url using a ssl handshake.
    response = urlopen(req, context=my_ssl)

```

```

        print(f"The URL {response.geturl()} crawled with \
              status {response.getcode()}")

        # this returns the html representation of the webpage
        soup = BeautifulSoup(response.read(), "html.parser")

        # in this case we are finding all the links in the page.
        for a_tag in soup.find_all('a'):
            # we are checking of the link is already visited and
(network location part) is our

```

```

            # base url itself.
            if (a_tag.get("href") not in self.have_visited) and (u
rlparse(link).netloc == "www.python.org"):
                self.links_to_crawl.put(a_tag.get("href"))

            else:
                print(f"The link {a_tag.get('href')} is already
visited or is not part \
                      of the website")

        print(f"Adding {link} to the crawled list")
        self.have_visited.add(link)

```

```
        except URLError as e:
            print(f"URL {link} threw this error {e.reason} while trying
to parse")

            self.error_links.append(link)

        finally:
            self.links_to_crawl.task_done()

if __name__ == '__main__':
    print("The Crawler is started")
    base_url = input("Please Enter Website to Crawl > ")
    number_of_threads = input("Please Enter number of Threads > ")

    links_to_crawl = queue.Queue()
    url_lock = threading.Lock()
    links_to_crawl.put(base_url)

    have_visited = set()

    crawler_threads = []
    error_links = []
    #base_url, links_to_crawl,have_visited, error_links,url_lock
    for i in range(int(number_of_threads)):
        crawler = Crawler(base_url = base_url,
                           links_to_crawl= links_to_crawl,
                           have_visited= have_visited,
                           error_links= error_links,
                           url_lock=url_lock)
```

```

        crawler.start()
        crawler_threads.append(crawler)

    for crawler in crawler_threads:
        crawler.join()

    print(f"Total Number of pages visited are {len(have_visited)}")
    print(f"Total Number of Errornous links: {len(error_links)}")

```

```
#####
```

```

import multiprocessing
from bs4 import BeautifulSoup
from queue import Queue, Empty
from concurrent.futures import ThreadPoolExecutor
from urllib.parse import urljoin, urlparse
import requests

```

```

# ThreadPoolExecutor
class MultiThreadedCrawler:

```

```

    def __init__(self, seed_url):
        self.seed_url = seed_url
        self.root_url = '{}://{}'.format(urlparse(self.seed_url).scheme,
                                         urlparse(self.seed_url).netloc)
        self.pool = ThreadPoolExecutor(max_workers=5)
        self.scraped_pages = set([])
        self.crawl_queue = Queue()
        self.crawl_queue.put(self.seed_url)

    def parse_links(self, html):

```

```

soup = BeautifulSoup(html, 'html.parser')
Anchor_Tags = soup.find_all('a', href=True)
for link in Anchor_Tags:
    url = link['href']
    if url.startswith('/') or url.startswith(self.root_url):
        url = urljoin(self.root_url, url)
    if url not in self.scraped_pages:
        self.crawl_queue.put(url)

def scrape_info(self, html):

    soup = BeautifulSoup(html, "html5lib")
    web_page_paragraph_contents = soup('p')
    text = ''
    for para in web_page_paragraph_contents:
        if not ('https:' in str(para.text)):
            text = text + str(para.text).strip()
    print(f'\n <---Text Present in The WebPage is --->\n', text, '\n')
    return

def post_scrape_callback(self, res):

    result = res.result()
    if result and result.status_code == 200:
        self.parse_links(result.text)
        self.scrape_info(result.text)

    def scrape_page(self, url):
        try:
            res = requests.get(url, timeout=(3, 30))
            return res
        except requests.RequestException:

            return

    def run_web_crawler(self):
        while True:
            try:
                print("\n Name of the current executing process: ",
                    multiprocessing.current_process().name, '\n')
                target_url = self.crawl_queue.get(timeout=60)
                if target_url not in self.scraped_pages:
                    print("Scraping URL: {}".format(target_url))

```

```
self.current_scraping_url = "{}".format(target_url)
self.scraped_pages.add(target_url)
job = self.pool.submit(self.scrape_page, target_url)
job.add_done_callback(self.post_scrape_callback)

except Empty:
    return
except Exception as e:
    print(e)
    continue

def info(self):
    print('\n Seed URL is: ', self.seed_url, '\n')
    print('Scraped pages are: ', self.scraped_pages, '\n')

if __name__ == '__main__':
    cc = MultiThreadedCrawler("https://www.leetcode.ca")
    cc.run_web_crawler()
    cc.info()
```

◆ Quick Review

Key Insights

- Use multiple threads to perform concurrent crawling, which speeds up the process over a single-threaded approach.
- Maintain a thread-safe set (visited) to ensure no URL is processed twice.
- Utilize a work queue to manage URLs pending processing.
- Filter URLs based on the hostname extracted from startUrl.

- Apply synchronization (locks or concurrent data structures) to avoid race conditions when multiple threads access shared data.

Space & Time

Time Complexity: $O(N)$ where N is the number of URLs processed, with additional constant factors due to threading overhead.

Space Complexity: $O(N)$ for storing the visited URLs and the work queue.

Solution

The solution initializes a shared work queue with the `startUrl` and a thread-safe visited set. Each worker thread dequeues a URL, retrieves its linked URLs via the blocking `HtmlParser.getUrls()` call, and filters the resulting URLs by comparing their hostnames with the start URL's hostname. New URLs that have not yet been visited are added to both the visited set and the work queue. The use of a thread pool or multiple worker threads along with synchronization ensures that the crawler processes URLs concurrently while avoiding race conditions. The approach follows a BFS-like pattern spread across multiple

threads.

Graphs

Round 2 · High · Medium · 3 questions

Graphs

□ #128 Longest Consecutive Sequence

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Union Find
Lists: Grind 169 | CodeSignal

Problem

Given an unsorted array of integers `nums`, return the length of the longest consecutive elements sequence.

You must write an algorithm that runs in $O(n)$ time.

Example 1:

Input: `nums = [100,4,200,1,3,2]`

Output: 4

Explanation: The longest consecutive elements sequence is [1, 2, 3, 4]. Therefore its length is 4.

Example 2:

Input: `nums = [0,3,7,2,5,8,4,6,0,1]`

Output: 9

Example 3:

Input: `nums = [1,0,1,2]`

Output: 3

Constraints:

- $0 \leq \text{nums.length} \leq 105$
- $-109 \leq \text{nums}[i] \leq 109$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        hset = set(nums)
        ans = 0
        for x in hset:
            if x-1 not in hset:
                l=1
                while x+1 in hset:
                    x = x+1
                    l+=1

                ans = max(ans,l)

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find the longest run of consecutive integers. Must be $O(n)$. Return the length. Right?

Duplicates don't extend the sequence – $[1,1,2] \rightarrow$ length 2."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Put all numbers in a hash set. For each num, if num-1 is not in set, walk num, num+1, num+2...

counting streak length. Track longest streak.

Each number visited once across all walks – $O(n)$ with the set.

Time: $O(n)$. Space: $O(n)$ set.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (hash set + only start from sequence beginnings)

"Build a set. For each x where x-1 is NOT in the set (x is a sequence start),

count upward. Each element is processed at most once total $\rightarrow O(n)$."

PHASE 4 — CLEAN CODE

```
class _128_LongestConsecutive:
    def longestConsecutive(self, nums: List[int]) -> int:
        num_set = set(nums)
        best = 0
        for x in num_set:
            if x - 1 not in num_set: # x is a sequence start
                length = 1
                while x + 1 in num_set:
                    x += 1; length += 1
                best = max(best, length)
        return best
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$. Each number is the starting point of at most one sequence scan.

The 'start of sequence' check prevents redundant scans – the key insight.

Given more time I'd ask: streaming input? Use a dict mapping each endpoint

of a range to its length, merging ranges as new numbers arrive."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestConsecutive(self, nums: list[int]) -> int:
        ans = 0
        seen = set(nums)

        for num in seen:
            # `num` is the start of a sequence.
            if num - 1 in seen:
                continue
            length = 0

            while num in seen:
                num += 1
                length += 1
            ans = max(ans, length)

        return ans
```

• LeetDoocs

```

class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        s = set(nums)
        ans = 0
        d = defaultdict(int)
        for x in nums:
            y = x
            while y in s:
                s.remove(y)
                y += 1

            d[x] = d[y] + y - x
            ans = max(ans, d[x])
        return ans

```

• SimplyLeet

```

# Python solution for Longest Consecutive Sequence

def longestConsecutive(nums):
    # Create a set from the list for O(1) look-up times
    num_set = set(nums)
    longest_streak = 0

    # Iterate through each number in the set
    for num in num_set:
        # Only start counting if 'num-1' is not present in the set

        if num - 1 not in num_set:
            current_num = num
            current_streak = 1

            # Count consecutive numbers starting from current_num
            while current_num + 1 in num_set:
                current_num += 1
                current_streak += 1

            # Update the longest streak if current one is longer

```

```

        longest_streak = max(longest_streak, current_streak)

    return longest_streak

# Example usage:
print(longestConsecutive([100, 4, 200, 1, 3, 2])) # Output: 4

```

• LC.ca

```

class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        s = set(nums)
        ans = 0
        for x in nums:
            if x - 1 not in s:
                y = x + 1
                while y in s:
                    y += 1
                ans = max(ans, y - x)

        return ans

```

◆ Quick Review

Key Insights

- Utilize a set (or hash table) for constant time look-ups.
- Only start counting a sequence from a number that is the beginning (its previous number is not in the set).
- Each number is visited only once, ensuring an $O(n)$ time complexity.

- Update the maximum sequence length as you discover longer consecutive sequences.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

We first insert all the numbers into a hash set to enable $O(1)$ membership checks. For each number in the set, we check if it is the start of a sequence by ensuring that the number minus one is not in the set. If it is the beginning, we then count all consecutive numbers following it, updating the maximum sequence length when necessary. This method guarantees that we only count each number once, satisfying the $O(n)$ time requirement.

Graphs

□ ★ #277 Find the Celebrity ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · Graph · Interactive
Lists: ★ Premium | Premium 98

Problem

Suppose you are at a party with n people labeled from 0 to $n - 1$ and among them, there may exist one celebrity. The definition of a celebrity is that all the other $n - 1$ people know the celebrity, but the celebrity does not know any of them.

Now you want to find out who the celebrity is or verify that there is not one. You are only allowed to ask questions like: "Hi, A. Do you know B?" to get information about whether A knows B. You need to find out the celebrity (or verify there is not one) by asking as few questions as possible (in the asymptotic sense).

You are given an integer n and a helper function `bool knows(a, b)` that tells you whether a knows

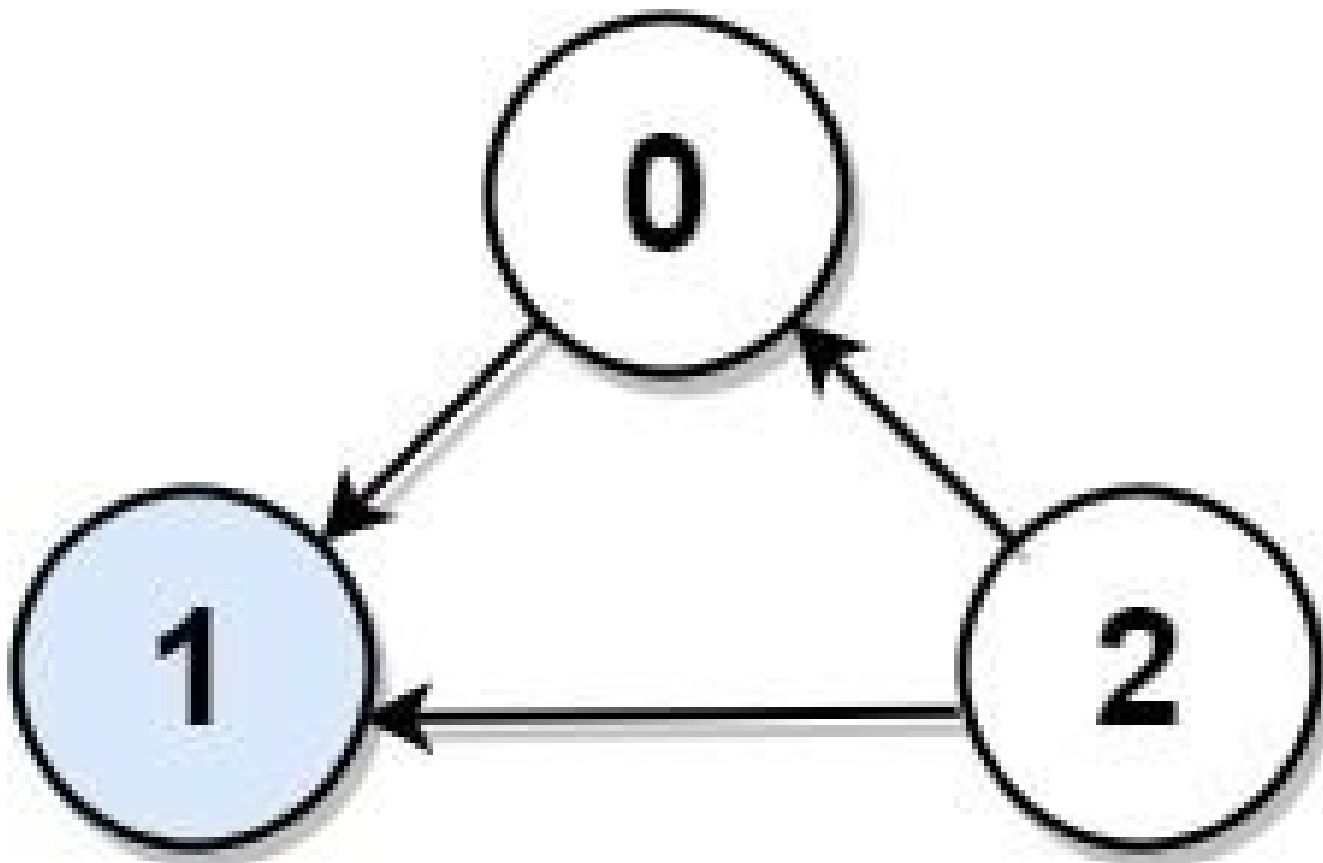
b. Implement a function `int findCelebrity(n)`.

There will be exactly one celebrity if they are at the party.

Return the celebrity's label if there is a celebrity at the party. If there is no celebrity, return `-1`.

Note that the $n \times n$ 2D array `graph` given as input is not directly available to you, and instead only accessible through the helper function `knows`. `graph[i][j] == 1` represents person `i` knows person `j`, whereas `graph[i][j] == 0` represents person `j` does not know person `i`.

Example 1:

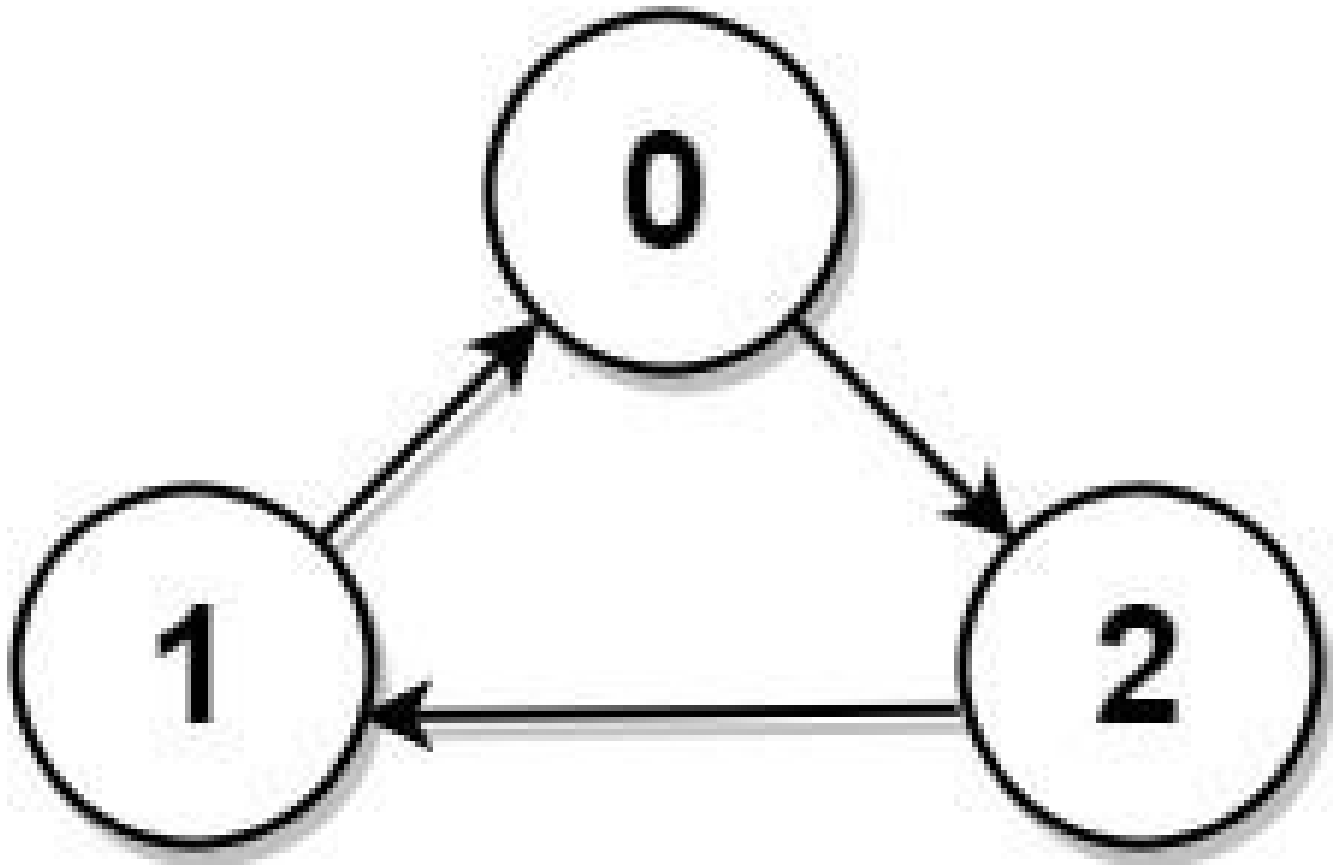


Input: graph = [[1,1,0],[0,1,0],[1,1,1]]

Output: 1

Explanation: There are three persons labeled with 0, 1 and 2. graph[i][j] = 1 means person i knows person j, otherwise graph[i][j] = 0 means person i does not know person j. The celebrity is the person labeled as 1 because both 0 and 2 know him but 1 does not know anybody.

Example 2:



Input: graph = [[1,0,1],[1,1,0],[0,1,1]]

Output: -1

Explanation: There is no celebrity.

Constraints:

- $n == \text{graph.length} == \text{graph}[i].\text{length}$
- $2 \leq n \leq 100$
- graph[i][j] is 0 or 1.
- graph[i][i] == 1

Follow up: If the maximum number of allowed calls to the API knows is $3 * n$, could you find a solution without exceeding the maximum number of calls?

My LeetCode Solution

• My LeetCode Solution

```
# The knows API is already defined for you.
# return a bool, whether a knows b
# def knows(a: int, b: int) -> bool:

class Solution:
    def findCelebrity(self, n: int) -> int:
        c = 0
        for i in range(1,n):
            if knows(c,i):
                c = i

        for i in range(n):
            if i != c:
                if knows(c,i) or not knows(i,c):
                    return -1

        return c
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Celebrity: everyone knows them, they know no one.
Find them using knows(a,b) API – minimise calls. Right?
At most one celebrity can exist."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
Ask every other person if they know celebrity; candidate must be known by all and know nobody."

```
# Two-pass elimination: first narrow to one candidate with O(n) knows() calls, then
verify.
# Naive all-pairs check is O(n^2) knows() calls.
# Time: O(n^2) naive, O(n) with elimination. Space: O(1).
# Should I go ahead and optimize?"
```

```
# PHASE 3 — OPTIMIZE (candidate elimination + verify)
```

```
# "Pass 1: start with candidate=0, scan all i. If knows(candidate, i): celebrity
can't be candidate → candidate=i.
# Pass 2: verify candidate – everyone must know them AND they know no one.
# Total API calls: O(n). Without the two-pass we'd need O(n^2)."
```

```
# PHASE 4 — CLEAN CODE
```

```
class _277_FindCelebrity:
    def findCelebrity(self, n: int) -> int:
        candidate = 0
        for i in range(1, n):
            if knows(candidate, i):
                candidate = i
        for i in range(n):
            if i == candidate:
                continue
            if knows(candidate, i) or not knows(i, candidate):
                return -1
        return candidate
```

```
# PHASE 6 — COMPLEXITY & FOLLOW-UP
```

```
# "Time O(n) – 2n-2 API calls in the worst case. Space O(1).
# The candidate elimination logic: if A knows B, A is not the celebrity (celebrity
knows no one).
# If A doesn't know B, B is not the celebrity (celebrity is known by all).
# One of them is eliminated per comparison – O(n) total.
# Given more time I'd ask: what if there's a possibility of no celebrity?
# The current solution handles this – the verification pass returns -1 if candidate
fails."
```

Community Solutions (Python)

• WalkCC

```
# The knows API is already defined for you.
# Returns a bool, whether a knows b
# Def knows(a: int, b: int) -> bool:
```

```
class Solution:
```

```
    def findCelebrity(self, n: int) -> int:
        candidate = 0
```

```
        # Everyone knows the celebrity.
```

```
        for i in range(1, n):
            if knows(candidate, i):
                candidate = i
```

```
        # The candidate knows nobody and everyone knows the celebrity.
```

```
        for i in range(n):
            if i < candidate and knows(candidate, i) or not knows(i, candidate):
                return -1
            if i > candidate and not knows(i, candidate):
                return -1
```

```
        return candidate
```

• LeetDoocs

```
# The knows API is already defined for you.
# return a bool, whether a knows b
# def knows(a: int, b: int) -> bool:
```

```
class Solution:
```

```
    def findCelebrity(self, n: int) -> int:
        ans = 0
        for i in range(1, n):
            if knows(ans, i):
```



```

        ans = i
    for i in range(n):
        if ans != i:
            if knows(ans, i) or not knows(i, ans):
                return -1
    return ans

```

• SimplyLeet

```

# Function definition for findCelebrity using the helper knows(a, b) API.
def findCelebrity(n):
    # Step 1: Candidate Selection.
    candidate = 0
    for i in range(1, n):
        # If candidate knows i, then candidate cannot be the celebrity.
        if knows(candidate, i):
            candidate = i # Select new candidate.

    # Step 2: Verification.

    # Check if candidate does not know any other person.
    for i in range(n):
        if i != candidate:
            # Either candidate knows someone or someone doesn't know candidate.
            if knows(candidate, i) or not knows(i, candidate):
                return -1
    return candidate

# Example helper function (for testing, this is normally provided in the
# problem context).
def knows(a, b):

    # This function should return True if person a knows person b, otherwise
    # False.
    pass

```

• LC.ca

```
# The knows API is already defined for you.
# return a bool, whether a knows b
# def knows(a: int, b: int) -> bool:

class Solution:
    def findCelebrity(self, n: int) -> int:
        ans = 0
        for i in range(1, n):
            if knows(ans, i):

                ans = i
        for i in range(n):
            if ans != i:
                if knows(ans, i) or not knows(i, ans):
                    return -1
        return ans
```

◆ Quick Review

Key Insights

- Use pairwise comparisons to eliminate non-celebrities.
- If a candidate knows another person, the candidate cannot be the celebrity.
- Verify the candidate by checking that they do not know anyone and that everyone knows them.
- Achieve the solution in two passes: one for candidate selection and the other for validation.

Space & Time

Time Complexity: $O(n)$ – One pass for candidate selection and one for validation.

Space Complexity: $O(1)$ – Only constant extra space is used.

Solution

The approach consists of two main steps:

Candidate Selection: Iterate through the people and maintain a candidate. For every person i , if the current candidate knows i , then set $\text{candidate} = i$. This works because if candidate knows i , candidate cannot be the celebrity.

Verification: Check that the candidate does not know any other person and that every other person knows the candidate. If both conditions hold, candidate is the celebrity; otherwise, return -1 . The main data structures used are simple integer variables. The algorithm leverages a two-pass technique to reduce the number of knows API calls while ensuring correctness.

Graphs

□ ★ #1136 Parallel Courses ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Graph · Topological Sort
Lists: ★ Premium | Premium 98

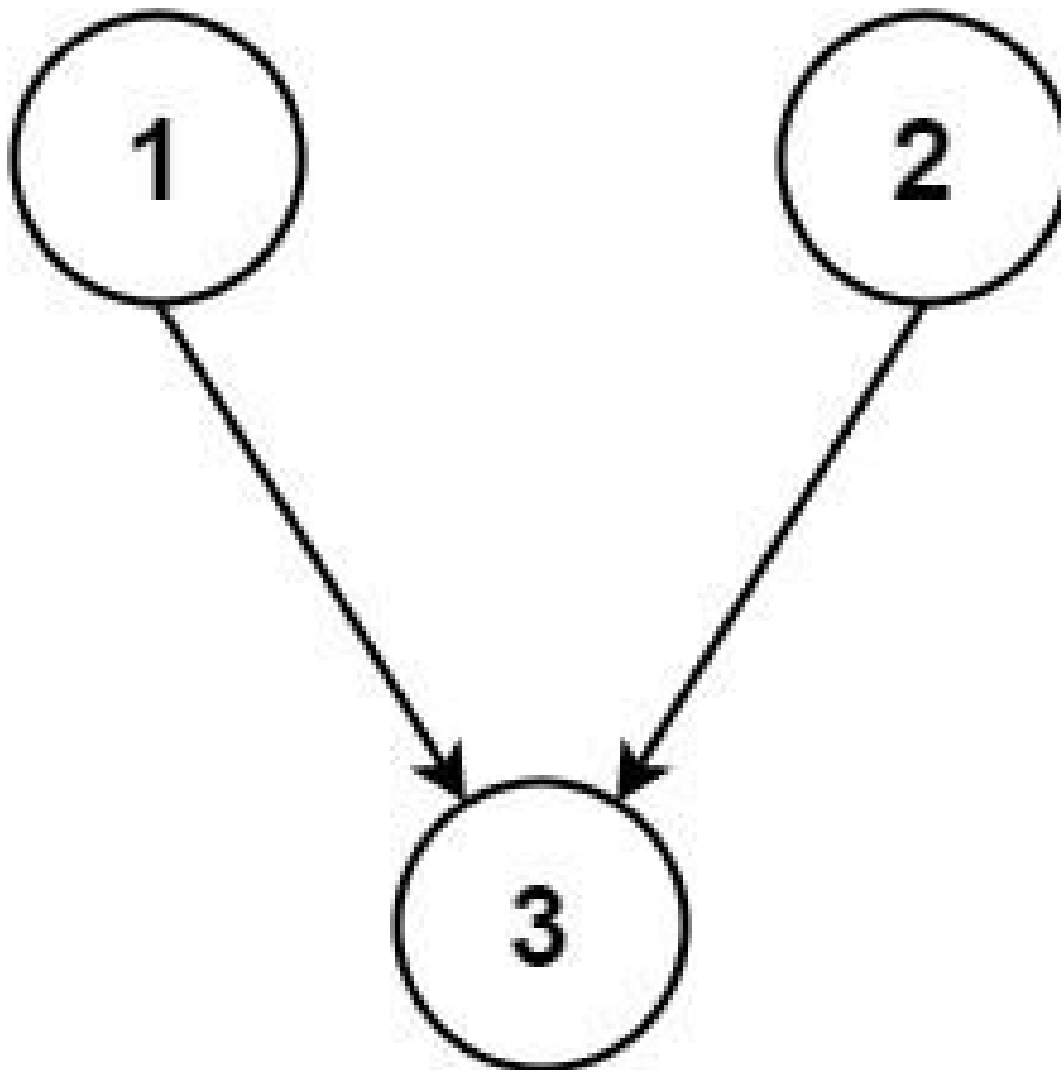
Problem

You are given an integer n , which indicates that there are n courses labeled from 1 to n . You are also given an array `relations` where `relations[i] = [prevCoursei, nextCoursei]`, representing a prerequisite relationship between course `prevCoursei` and course `nextCoursei`: course `prevCoursei` has to be taken before course `nextCoursei`.

In one semester, you can take any number of courses as long as you have taken all the prerequisites in the previous semester for the courses you are taking.

Return the minimum number of semesters needed to take all courses. If there is no way to take all the courses, return -1 .

Example 1:



Input: $n = 3$, $\text{relations} = [[1,3],[2,3]]$

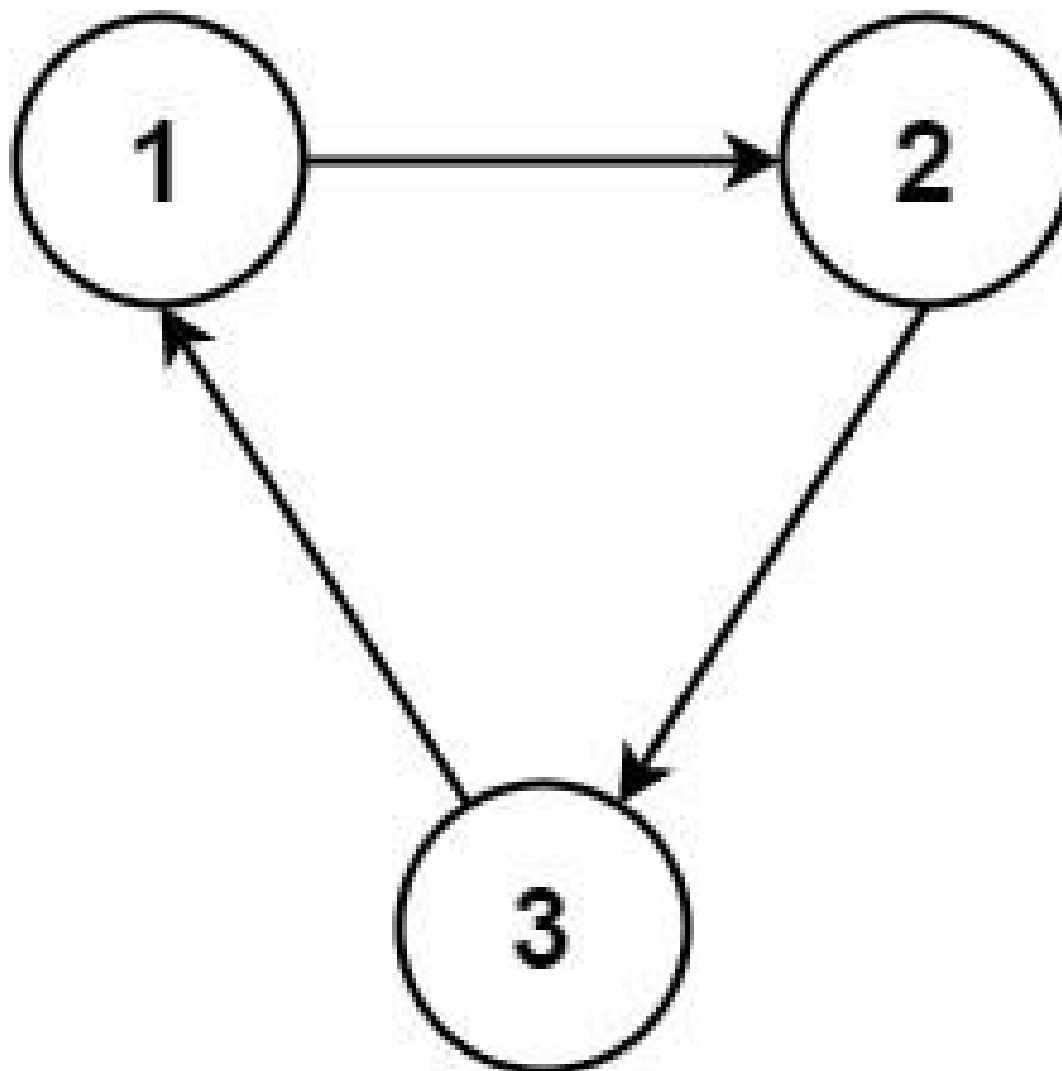
Output: 2

Explanation: The figure above represents the given graph.

In the first semester, you can take courses 1 and 2.

In the second semester, you can take course 3.

Example 2:



Input: $n = 3$, $\text{relations} = [[1,2],[2,3],[3,1]]$

Output: -1

Explanation: No course can be studied because they are prerequisites of each other.

Constraints:

- $1 \leq n \leq 5000$
- $1 \leq \text{relations.length} \leq 5000$
- $\text{relations}[i].\text{length} == 2$
- $1 \leq \text{prevCourse}_i, \text{nextCourse}_i \leq n$
- $\text{prevCourse}_i \neq \text{nextCourse}_i$

- All the pairs [prevCoursei, nextCoursei] are unique.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def minimumSemesters(self, n: int, relations: List[List[int]]) -> int:
        g = {i:[] for i in range(1,n+1)}
        indeg = {i:0 for i in range(1,n+1)}

        for a,b in relations:
            g[a].append(b)
            indeg[b] += 1

        q = deque()

        sems = 0
        total = 0

        for i in range(1,n+1):
            if indeg[i] == 0:
                q.append(i)

        while (q):
            sems+=1
            s = len(q)

            for j in range(s):
                node = q.popleft()
                total += 1
                for nb in g[node]:
                    indeg[nb]-=1
                    if indeg[nb] == 0:
                        q.append(nb)

            return sems if total == n else -1
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Minimum semesters to take all courses where you can take any number per semester as long as prerequisites are met. Return -1 if impossible (cycle). Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Topological sort with indegree: each semester take all courses whose prerequisites are done.

Increment semester count each round; if no course can be taken but some remain, impossible.

Same Kahn BFS layered by semester.

Time: $O(V + E)$. Space: $O(V + E)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (Kahn's BFS level-by-level)

"BFS topological sort: process all zero-in-degree nodes in one 'batch' (semester).

Each batch = one semester. Count batches. If total processed < n → cycle."

PHASE 4 — CLEAN CODE

```
class _1136_ParallelCourses:
    def minimumSemesters(self, n: int, relations: List[List[int]]) -> int:
        graph = defaultdict(list)
        indeg = {i: 0 for i in range(1, n + 1)}
        for pre, nxt in relations:
            graph[pre].append(nxt)
            indeg[nxt] += 1
        queue = deque(c for c in range(1, n + 1) if indeg[c] == 0)
        sems = 0
        total = 0
        while queue:
            sems += 1
            for _ in range(len(queue)):
                node = queue.popleft()
                total += 1
                for nb in graph[node]:
                    indeg[nb] -= 1
                    if indeg[nb] == 0:
                        queue.append(nb)
            return sems if total == n else -1
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(V+E)$. Space $O(V+E)$. This is Course Schedule (#207) with BFS level counting.

Each BFS level = one semester. Cycle detection: total processed < n.

Given more time I'd ask: what if courses have durations (not just 1 semester each)?

This becomes a weighted DAG critical path problem – DP or Bellman-Ford."

Community Solutions (Python)

• WalkCC

```
from enum import Enum
```

```
class State(Enum):
```

```
    INIT = 0
```

```
    VISITING = 1
```

```
    VISITED = 2
```

```
class Solution:
```

```
    def minimumSemesters(self, n: int, relations: list[list[int]]) -> int:
```

```
        graph = [[] for _ in range(n)]
```

```
        states = [State.INIT] * n
```

```
        depth = [1] * n
```

```
        for u, v in relations:
```

```
            graph[u - 1].append(v - 1)
```

```
    def hasCycle(u: int) -> bool:
```

```
        if states[u] == State.VISITING:
```

```
            return True
```

```
        if states[u] == State.VISITED:
```

```
            return False
```

```
        states[u] = State.VISITING
```

```
        for v in graph[u]:
```

```
            if hasCycle(v):
```

```
                return True
```

```
            depth[u] = max(depth[u], 1 + depth[v])
```

```
        states[u] = State.VISITED
```

```
        return False
```

```
    if any(hasCycle(i) for i in range(n)):
```

```
        return -1
```

```
    return max(depth)
```

```

class Solution:
    def minimumSemesters(self, n: int, relations: list[list[int]]) -> int:
        graph = [[] for _ in range(n)]
        inDegrees = [0] * n

        # Build the graph.
        for u, v in relations:
            graph[u - 1].append(v - 1)
            inDegrees[v - 1] += 1

        # Perform topological sorting.
        q = collections.deque([i for i, d in enumerate(inDegrees) if d == 0])

        step = 0
        while q:
            for _ in range(len(q)):
                u = q.popleft()
                n -= 1
                for v in graph[u]:
                    inDegrees[v] -= 1

                    if inDegrees[v] == 0:
                        q.append(v)
            step += 1

        return step if n == 0 else -1

```

• LeetDoocs

```

class Solution:
    def minimumSemesters(self, n: int, relations: List[List[int]]) -> int:
        g = defaultdict(list)
        indeg = [0] * n
        for prev, nxt in relations:
            prev, nxt = prev - 1, nxt - 1
            g[prev].append(nxt)
            indeg[nxt] += 1
        q = deque(i for i, v in enumerate(indeg) if v == 0)
        ans = 0

```

```

while q:
    ans += 1
    for _ in range(len(q)):
        i = q.popleft()
        n -= 1
        for j in g[i]:
            indeg[j] -= 1
            if indeg[j] == 0:
                q.append(j)
    return -1 if n else ans

```

• SimplyLeet

Python Implementation

```
from collections import deque
```

```
def minimumSemesters(numCourses, relations):
    # Build graph and in-degree dictionary.
    graph = {i: [] for i in range(1, numCourses + 1)}
    in_degree = {i: 0 for i in range(1, numCourses + 1)}

    # Construct graph from relations.
```

```

for pre, course in relations:
    graph[pre].append(course)
    in_degree[course] += 1

```

```

# Initialize queue with all courses having no prerequisites.
queue = deque()
for course in range(1, numCourses + 1):
    if in_degree[course] == 0:
        queue.append(course)

```

```

semesters = 0
takenCourses = 0

# Process each level (semester) using BFS.
while queue:
    semesters += 1
    current_level_size = len(queue)
    for _ in range(current_level_size):
        course = queue.popleft()
        takenCourses += 1

        # Decrement in-degree for neighboring courses.
        for neighbor in graph[course]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

# If not all courses have been taken, a cycle exists.
return semesters if takenCourses == numCourses else -1

```

Example usage:

```
print(minimumSemesters(3, [[1, 3], [2, 3]])) # Expected output: 2
```

• LC.ca

```

class Solution:
    def minimumSemesters(self, n: int, relations: List[List[int]]) -> int:
        g = defaultdict(list)
        indeg = [0] * n
        for prev, nxt in relations:
            prev, nxt = prev - 1, nxt - 1
            g[prev].append(nxt)
            indeg[nxt] += 1
        q = deque(i for i, v in enumerate(indeg) if v == 0)
        ans = 0

```

```
while q:
    ans += 1
    for _ in range(len(q)):
        i = q.popleft()
        n -= 1
        for j in g[i]:
            indeg[j] -= 1
            if indeg[j] == 0:
                q.append(j)
    return -1 if n else ans
```

◆ Quick Review

Key Insights

- Model the problem as a directed graph where nodes are courses and edges represent prerequisites.
- Use topological sorting (Kahn's algorithm) to process courses with zero prerequisites.
- Each level (or layer) of processing represents one semester.
- Detect cycles by ensuring all courses are eventually processed; if not, a cycle exists.
- The number of BFS levels gives the minimum number of semesters necessary.

Space & Time

Time Complexity: $O(n + m)$ where n is the number of courses and m is the length of

relations (prerequisites).

Space Complexity: $O(n + m)$ for storing the graph, in-degree counts, and the processing queue.

Solution

We solve the problem using a topological sort with Kahn's algorithm. First, construct a graph representation using an adjacency list and compute the in-degree for each course.

Initialize a queue with courses that have no prerequisites (in-degree == 0). Then, process courses level by level. Each level processed represents one semester. For every course taken in that semester, decrease the in-degree of all its neighbors. Add any neighbor with an in-degree that becomes zero to the queue for the next semester. If, after processing, the total courses taken is less than n , then a cycle exists and return -1 . Otherwise, the number of semesters processed is the answer.

BFS

Round 2 · High · Medium · 6 questions

BFS

□ ★ #286 Walls and Gates ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · BFS · Matrix

Lists: ★ Premium | Premium 98

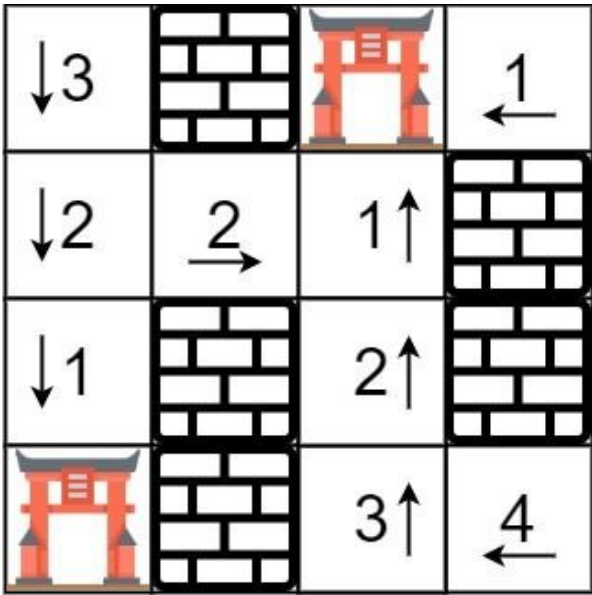
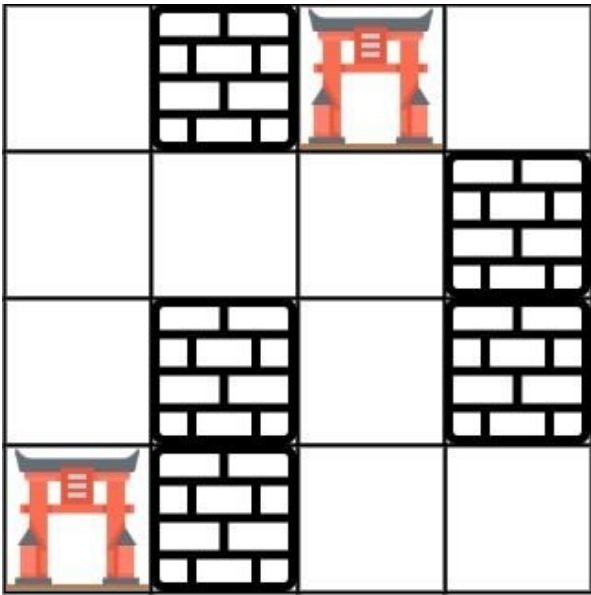
Problem

You are given an $m \times n$ grid rooms initialized with these three possible values.

- -1 A wall or an obstacle.
- 0 A gate.
- INF Infinity means an empty room. We use the value $2^{31} - 1 = 2147483647$ to represent INF as you may assume that the distance to a gate is less than 2147483647.

Fill each empty room with the distance to its nearest gate. If it is impossible to reach a gate, it should be filled with INF.

Example 1:



Input: rooms = `[[2147483647,-1,0,2147483647],[2147483647,2147483647,2147483647,-1],[2147483647,-1,2147483647,-1],[0,-1,2147483647,2147483647]]`
Output: `[[3,-1,0,1],[2,2,1,-1],[1,-1,2,-1],[0,-1,3,4]]`

Example 2:

Input: rooms = `[[-1]]`
Output: `[[-1]]`

Constraints:

- `m == rooms.length`
- `n == rooms[i].length`
- `1 <= m, n <= 250`
- `rooms[i][j]` is `-1`, `0`, or `231 - 1`.

My LeetCode Solution

- ☐
- My LeetCode Solution

```

class Solution:
    def wallsAndGates(self, rooms: list[list[int]]) -> None:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        INF = 2**31 - 1
        m = len(rooms)
        n = len(rooms[0])
        q = collections.deque((i, j)
                               for i in range(m)
                               for j in range(n)
                               if rooms[i][j] == 0)

        while q:
            i, j = q.popleft()
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if rooms[x][y] != INF:
                    continue

                rooms[x][y] = rooms[i][j] + 1
                q.append((x, y))

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Fill each INF room with its distance to the nearest gate (0). Walls (-1) stay. Right?"

In-place modification. $INF = 2^{31}-1$."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Multi-source BFS from every gate simultaneously (distance 0).

Fill empty rooms layer by layer with increasing distance.

Each room reached once – standard BFS on grid.

Time: $O(m * n)$. Space: $O(m * n)$ queue.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (multi-source BFS from all gates)

"Enqueue ALL gates at once. BFS outward – each room gets filled with distance from the nearest gate simultaneously. $O(m*n)$."

PHASE 4 — CLEAN CODE

```
class _286_WallsAndGates:
    def wallsAndGates(self, rooms: List[List[int]]) -> None:
        INF = 2**31 - 1
        m, n = len(rooms), len(rooms[0])
        queue = deque((r, c) for r in range(m) for c in range(n) if rooms[r][c] == 0)

        while queue:
            r, c = queue.popleft()
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                nr, nc = r + dr, c + dc
                if 0 <= nr < m and 0 <= nc < n and rooms[nr][nc] == INF:
                    rooms[nr][nc] = rooms[r][c] + 1
                    queue.append((nr, nc))
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(m*n)$. Multi-source BFS is the key – a single BFS from each gate independently would be $O(m*n * \text{gates})$, much slower.
 # Contrast with #542 (01 Matrix): identical pattern but for distance to nearest 0.
 # Given more time I'd ask: find distance to nearest wall instead?
 # Multi-source BFS from all walls – same approach."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def wallsAndGates(self, rooms: list[list[int]]) -> None:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        INF = 2**31 - 1
        m = len(rooms)
        n = len(rooms[0])
        q = collections.deque((i, j)
                               for i in range(m)
                               for j in range(n)
                               if rooms[i][j] == 0)
```

```

while q:
    i, j = q.popleft()
    for dx, dy in DIRS:
        x = i + dx
        y = j + dy
        if x < 0 or x == m or y < 0 or y == n:
            continue
        if rooms[x][y] != INF:
            continue

```

```

rooms[x][y] = rooms[i][j] + 1
q.append((x, y))

```

• LeetDoocs

```

class Solution:
    def wallsAndGates(self, rooms: List[List[int]]) -> None:
        """
        Do not return anything, modify rooms in-place instead.
        """
        m, n = len(rooms), len(rooms[0])
        inf = 2**31 - 1
        q = deque([(i, j) for i in range(m) for j in range(n) if rooms[i][j] =
= 0])
        d = 0
        while q:

```

```

            d += 1
            for _ in range(len(q)):
                i, j = q.popleft()
                for a, b in [[0, 1], [0, -1], [1, 0], [-1, 0]]:
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n and rooms[x][y] == inf:
                        rooms[x][y] = d
                        q.append((x, y))

```

• SimplyLeet

```

from collections import deque

def wallsAndGates(rooms):
    if not rooms or not rooms[0]:
        return

    rows, cols = len(rooms), len(rooms[0])
    # Directions: up, down, left, right.
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    queue = deque()

    # Enqueue all gates (cells with 0).
    for r in range(rows):
        for c in range(cols):
            if rooms[r][c] == 0:
                queue.append((r, c))

    # Perform multi-source BFS.
    while queue:
        r, c = queue.popleft()

        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            # Process neighbor if it is within bounds and is an empty room.
            if 0 <= nr < rows and 0 <= nc < cols and rooms[nr][nc] == 21474836
47:
                rooms[nr][nc] = rooms[r][c] + 1 # Update distance.
                queue.append((nr, nc))

```

• LC.ca

```

class Solution:
    def wallsAndGates(self, rooms: List[List[int]]) -> None:
        """
        Do not return anything, modify rooms in-place instead.
        """
        m, n = len(rooms), len(rooms[0])
        inf = 2**31 - 1
        q = deque([(i, j) for i in range(m) for j in range(n) if rooms[i][j] =
= 0])
        d = 0
        while q:

```

```

d += 1
for _ in range(len(q)):
    i, j = q.popleft()
    for a, b in [[0, 1], [0, -1], [1, 0], [-1, 0]]:
        x, y = i + a, j + b
        # bfs so first reaching inf is always the shortest
        # set its value from inf to d, same as marking as visited
        if 0 <= x < m and 0 <= y < n and rooms[x][y] == inf:
            rooms[x][y] = d
            q.append((x, y))

```

```
#####
```

```

class Solution: # dfs
    def wallsAndGates(self, rooms: List[List[int]]) -> None:
        if not rooms or not rooms[0]:
            return

        m, n = len(rooms), len(rooms[0])
        isVisited = [[False] * n for _ in range(m)] # isVisited map shared
        among all gates

```

```

    def fill(i: int, j: int, distance: int) -> None:
        nonlocal m, n

        # rooms[i][j] <= 0 meaning == -1 or == 0, both stop
        if i < 0 or i >= m or j < 0 or j >= n or rooms[i][j] <= 0 or isVis
        ited[i][j]:
            return

        rooms[i][j] = min(rooms[i][j], distance)
        isVisited[i][j] = True

```



```

        fill(i - 1, j, distance + 1)
        fill(i, j + 1, distance + 1)
        fill(i + 1, j, distance + 1)
        fill(i, j - 1, distance + 1)

        isVisited[i][j] = False

    for i in range(m):
        for j in range(n):

            if rooms[i][j] == 0: # start from every gate
                fill(i, j, 0)

#####

from collections import deque

INF = 2147483647

class Solution(object):
    def wallsAndGates(self, rooms):
        """
        :type rooms: List[List[int]]
        :rtype: void Do not return anything, modify rooms in-place instead.
        """
        queue = deque([])
        directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
        for i in range(0, len(rooms)):
            for j in range(0, len(rooms[0])):

```

```

        if rooms[i][j] == 0:
            queue.append((i, j))

    while queue:
        i, j = queue.popleft()
        for di, dj in directions:
            p, q = i + di, j + dj
            if 0 <= p < len(rooms) and 0 <= q < len(rooms[0]) and rooms[p][q] == I
NF:
            rooms[p][q] = rooms[i][j] + 1
            queue.append((p, q))

```

◆ Quick Review

Key Insights

- Use a multi-source Breadth-First Search (BFS) starting from all the gates, as this allows spreading the shortest distance to each empty room.
- Instead of starting from every empty room separately, starting from all gates simultaneously helps update all distances in one pass.
- Only update a room if the new calculated distance is smaller than its current value to prevent unnecessary processing.
- The problem is essentially a multi-source shortest path problem on a grid.

Space & Time

Time Complexity: $O(m * n)$ where m and n are the dimensions of the grid, as each cell is processed at most once.

Space Complexity: $O(m * n)$ in the worst-case scenario used by the BFS queue.

Solution

The solution uses a multi-source BFS algorithm by initially enqueueing all the gate positions (cells with value 0). Then, for each cell obtained from the queue, check its four possible neighbors (up, down, left, right). If a neighbor is an empty room (INF), update its distance to be the current cell's distance plus one, and then enqueue the neighbor to process further. This ensures that each empty room gets the minimum distance from any gate.

BFS

□ #322 Coin Change

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · BFS
Lists: Grind 169

Problem

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return the fewest number of coins that you need to make up that amount. If that amount of money cannot be made up by any combination of the coins, return `-1`.

You may assume that you have an infinite number of each kind of coin.

Example 1:

```
Input: coins = [1,2,5], amount = 11
Output: 3
Explanation: 11 = 5 + 5 + 1
```

Example 2:

Input: coins = [2], amount = 3
Output: -1

Example 3:

Input: coins = [1], amount = 0
Output: 0

Constraints:

- $1 \leq \text{coins.length} \leq 12$
- $1 \leq \text{coins}[i] \leq 231 - 1$
- $0 \leq \text{amount} \leq 104$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Minimum coins to make up amount. Unlimited supply of each coin. Return -1 if impossible. Right?"
# Coin values can be any positive integer – not necessarily power of 2 (so greedy fails)."
```

```
#
```

```
# "coins=[1,2,5], amount=11: 5+5+1=3 coins ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic."
# Recursion: for each amount, try every coin denomination and take 1 + min subproblem.
# Recomputes identical sub-amounts exponentially without memoization.
# Time: O(amount^coins) exponential. Space: O(amount) stack.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (bottom-up DP)

```
# "dp[i] = minimum coins to make amount i."
# dp[0]=0. dp[i] = min(dp[i-coin]+1) for each coin <= i.
# Time: O(amount * coins). Space: O(amount)."
```

PHASE 4 — CLEAN CODE

```
class _322_CoinChange:
    def coinChange(self, coins: List[int], amount: int) -> int:
```

```

dp = [float('inf')] * (amount + 1)
dp[0] = 0
for coin in coins:
    for i in range(coin, amount + 1):
        dp[i] = min(dp[i], dp[i - coin] + 1)
return dp[amount] if dp[amount] != float('inf') else -1

```

PHASE 5 — TEST & DEBUG

```

# coins=[1,2,5], amount=11:
# dp[5]=1(5). dp[10]=2(5+5). dp[11]=3(5+5+1). → 3 ✓
# amount=3, coins=[2]: dp[2]=1, dp[3]=inf → -1 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(amount * coins). Space O(amount).
# BFS alternative: treat amounts as nodes, coins as edges. BFS from 0 to amount.
# DP is preferred – BFS visits each amount once but rebuilds from scratch.
# Given more time I'd ask: how many ways to make the amount?
# Coin Change 2 (#518) – same DP but count combinations instead of min count."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def coinChange(self, coins: list[int], amount: int) -> int:
        # dp[i] := the minimum number of coins to make up i
        dp = [0] + [amount + 1] * amount

        for coin in coins:
            for i in range(coin, amount + 1):
                dp[i] = min(dp[i], dp[i - coin] + 1)

        return -1 if dp[amount] == amount + 1 else dp[amount]

```

• LeetDoocs

```
class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:
        m, n = len(coins), amount
        f = [[inf] * (n + 1) for _ in range(m + 1)]
        f[0][0] = 0
        for i, x in enumerate(coins, 1):
            for j in range(n + 1):
                f[i][j] = f[i - 1][j]
                if j >= x:
                    f[i][j] = min(f[i][j], f[i][j - x] + 1)

        return -1 if f[m][n] >= inf else f[m][n]
```

• SimplyLeet

```
def coinChange(coins, amount):
    # Create a dp array with amount+1 entries set to a value greater than any
    # possible minimum number of coins.
    dp = [amount + 1] * (amount + 1)
    # Base case: 0 coins are needed to form amount 0.
    dp[0] = 0

    # Build up the dp table.
    for current_amount in range(1, amount + 1):
        for coin in coins:
            if coin <= current_amount:
                dp[current_amount] = min(dp[current_amount], dp[current_amount
                - coin] + 1)

    # If dp[amount] remains greater than amount, no solution was found.
    return dp[amount] if dp[amount] != amount + 1 else -1
```

• LC.ca


```

'''
>>> float("inf")
inf
>>> float("inf") + 1
inf
'''

class Solution(object):
    def coinChange(self, coins, amount):
        '''

        :type coins: List[int]
        :type amount: int
        :rtype: int
        '''

        dp = [float("inf")] * (amount + 1)
        dp[0] = 0
        for i in range(1, amount + 1):
            for coin in coins:
                if i - coin >= 0:

                    dp[i] = min(dp[i], dp[i - coin] + 1)
        return dp[-1] if dp[-1] != float("inf") else -1

#####

class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:
        dp = [amount + 1] * (amount + 1)
        dp[0] = 0
        for coin in coins:

            for j in range(coin, amount + 1):
                dp[j] = min(dp[j], dp[j - coin] + 1)
        return -1 if dp[-1] > amount else dp[-1]

```

◆ Quick Review

Key Insights

- Use a dynamic programming approach to build up a solution from smaller subproblems.
- Define $dp[x]$ as the minimum number of coins required for amount x .
- The recurrence relation is: $dp[x] = \min(dp[x], dp[x - \text{coin}] + 1)$ for each coin.
- Initialize $dp[0] = 0$ and set other values to a number larger than the maximum possible coins (amount+1 works as an infinity substitute).
- An alternative approach is through BFS, treating each amount as a node and each coin as an edge, but the DP method is more intuitive here.

Space & Time

Time Complexity: $O(\text{amount} * n)$ where n is the number of coins.

Space Complexity: $O(\text{amount})$ for the dp array.

Solution

We use a bottom-up dynamic programming approach. The idea is to use a one-dimensional dp array where each element at index x represents the minimum number of coins required to form the amount x . Starting

from `dp[0]` which is 0, we iterate through all amounts up to the target amount and update `dp[x]` based on each coin denomination. If after filling the `dp` array the value `dp[amount]` remains unchanged (i.e., greater than amount), it means the amount cannot be formed and we return `-1`.

BFS

□ #542 01 Matrix

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · BFS · Matrix

Lists: Grind 169 | Denny Zhang

Problem

Given an $m \times n$ binary matrix `mat`, return the distance of the nearest 0 for each cell.

The distance between two cells sharing a common edge is 1.

Example 1:

0	0	0
0	1	0
0	0	0

Input: mat = [[0,0,0],[0,1,0],[0,0,0]]

Output: [[0,0,0],[0,1,0],[0,0,0]]

Example 2:

0	0	0
0	1	0
1	1	1

Input: `mat = [[0,0,0],[0,1,0],[1,1,1]]`

Output: `[[0,0,0],[0,1,0],[1,2,1]]`

Constraints:

- `m == mat.length`
- `n == mat[i].length`
- `1 <= m, n <= 104`
- `1 <= m * n <= 104`
- `mat[i][j]` is either 0 or 1.
- There is at least one 0 in `mat`.

Note: This question is the same as 1765: <https://leetcode.com/problems/map-of-highest-peak/>

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Return distance of nearest 0 for each cell. In-place or new matrix. Right?
Same pattern as Walls and Gates but for 0-cells instead of gates."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
Multi-source BFS starting from every 0 simultaneously.
First time each 1 cell is reached, its distance is minimal.
Layer-by-layer BFS fills the dist matrix in one pass.
Time: $O(m * n)$. Space: $O(m * n)$ queue + dist.
Should I go ahead and optimize?"

```
class _542_ZeroOneMatrix:
    def updateMatrix(self, mat: List[List[int]]) -> List[List[int]]:
        m, n = len(mat), len(mat[0])
        dist = [[0 if mat[r][c] == 0 else float('inf') for c in range(n)] for r in range(m)]
        queue = deque((r, c) for r in range(m) for c in range(n) if mat[r][c] == 0)
        while queue:
            r, c = queue.popleft()
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                nr, nc = r + dr, c + dc
                if 0 <= nr < m and 0 <= nc < n and dist[nr][nc] > dist[r][c] + 1:
                    dist[nr][nc] = dist[r][c] + 1
                    queue.append((nr, nc))
        return dist
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(m*n)$. Multi-source BFS from all 0-cells simultaneously.
Identical to #286 (Walls and Gates) – name this connection.
Given more time I'd ask: distance to nearest 1 instead?
Same multi-source BFS starting from all 1-cells."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def updateMatrix(self, mat: list[list[int]]) -> list[list[int]]:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(mat)
        n = len(mat[0])
        q = collections.deque()
        seen = [[False] * n for _ in range(m)]

        for i in range(m):
            for j in range(n):
                if mat[i][j] == 0:
                    q.append((i, j))
                    seen[i][j] = True

        while q:
            i, j = q.popleft()
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if seen[x][y]:
                    continue
                mat[x][y] = mat[i][j] + 1
                q.append((x, y))
                seen[x][y] = True

        return mat
```

• LeetDoocs

```

class Solution:
    def updateMatrix(self, mat: List[List[int]]) -> List[List[int]]:
        m, n = len(mat), len(mat[0])
        ans = [[-1] * n for _ in range(m)]
        q = deque()
        for i, row in enumerate(mat):
            for j, x in enumerate(row):
                if x == 0:
                    ans[i][j] = 0
                    q.append((i, j))

        dirs = (-1, 0, 1, 0, -1)
        while q:
            i, j = q.popleft()
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and ans[x][y] == -1:
                    ans[x][y] = ans[i][j] + 1
                    q.append((x, y))

        return ans

```

• SimplyLeet

```

from collections import deque

def updateMatrix(mat):
    # Get the dimensions of the matrix
    rows, cols = len(mat), len(mat[0])
    # Initialize a queue for multi-source BFS
    queue = deque()

    # Initialize the matrix: use -1 to indicate unvisited cells
    for r in range(rows):

```

```

    for c in range(cols):
        if mat[r][c] == 0:
            # Enqueue all 0 cells as starting points
            queue.append((r, c))
        else:
            # Mark 1 cells as unvisited (or infinite distance)
            mat[r][c] = -1

# Directions for neighbors (up, down, left, right)
directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

# BFS traversal to update distances
while queue:
    r, c = queue.popleft()
    for dr, dc in directions:
        nr, nc = r + dr, c + dc
        # Check boundaries and if neighbor has not been visited
        if 0 <= nr < rows and 0 <= nc < cols and mat[nr][nc] == -1:
            # Update neighbor's distance
            mat[nr][nc] = mat[r][c] + 1

        # Enqueue the neighbor for further BFS processing
        queue.append((nr, nc))

return mat

# Example usage:
# print(updateMatrix([[0,0,0],[0,1,0],[1,1,1]]))

```

• LC.ca

```

class Solution:
    def updateMatrix(self, mat: List[List[int]]) -> List[List[int]]:
        m, n = len(mat), len(mat[0])
        ans = [[-1] * n for _ in range(m)]
        q = deque()
        for i, row in enumerate(mat):
            for j, x in enumerate(row):
                if x == 0:
                    ans[i][j] = 0
                    q.append((i, j))

```

```
dirs = (-1, 0, 1, 0, -1)
while q:
    i, j = q.popleft()
    for a, b in pairwise(dirs):
        x, y = i + a, j + b
        if 0 <= x < m and 0 <= y < n and ans[x][y] == -1:
            ans[x][y] = ans[i][j] + 1
            q.append((x, y))
return ans
```

◆ Quick Review

Key Insights

- Treat all 0s as starting points for a multi-source BFS.
- Update each neighboring cell's distance if a shorter distance is found.
- Utilize a queue to process cells in order of increasing distance.
- Ensure that each cell is processed only once by marking visited cells.

Space & Time

Time Complexity: $O(mn)$ since each cell is processed once.

Space Complexity: $O(mn)$ for the queue and the answer matrix.

Solution

We use a multi-source Breadth-First Search (BFS) where every 0 in the matrix is initially added to the BFS queue. Each neighboring cell that is a 1 will have its distance updated to be one plus the distance of the current cell if it hasn't been visited yet (or if a smaller distance is found). This ensures that the first time a 1 is reached, it is reached by the shortest path. Data structures used include a queue (or deque) for BFS and the matrix itself for storing distances.

BFS

□ #994 Rotting Oranges

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · BFS · Matrix
Lists: Grind 169

Problem

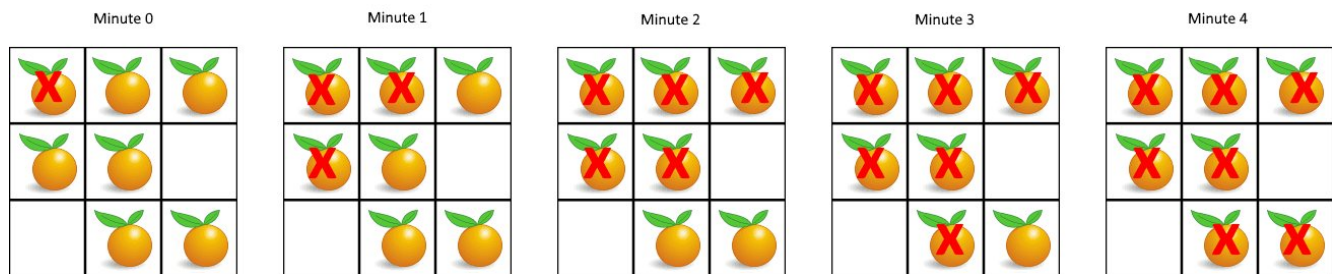
You are given an $m \times n$ grid where each cell can have one of three values:

- 0 representing an empty cell,
- 1 representing a fresh orange, or
- 2 representing a rotten orange.

Every minute, any fresh orange that is 4-directionally adjacent to a rotten orange becomes rotten.

Return the minimum number of minutes that must elapse until no cell has a fresh orange. If this is impossible, return -1 .

Example 1:



Input: grid = `[[2,1,1],[1,1,0],[0,1,1]]`
 Output: 4

Example 2:

Input: grid = `[[2,1,1],[0,1,1],[1,0,1]]`
 Output: -1
 Explanation: The orange in the bottom left corner (row 2, column 0) is never rotten, because rotting only happens 4-directionally.

Example 3:

Input: grid = `[[0,2]]`
 Output: 0
 Explanation: Since there are already no fresh oranges at minute 0, the answer is just 0.

Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 10`
- `grid[i][j]` is 0, 1, or 2.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Multi-source BFS from all rotten oranges. Fresh adjacent → rotten after 1 minute.

Return minutes until no fresh remain, or -1 if impossible. Right?"

#

"If no fresh oranges initially → return 0 immediately."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Multi-source BFS: enqueue all rotten oranges at minute 0.

Each minute, rot all fresh neighbors; track elapsed minutes until no fresh remain.

If fresh count > 0 after BFS ends, return -1.

Time: $O(m * n)$. Space: $O(m * n)$ queue.

Should I go ahead and optimize?"

class _994_RottingOranges:

def orangesRotting(self, grid: List[List[int]]) -> int:

m, n = len(grid), len(grid[0])

fresh = sum(grid[r][c] == 1 for r in range(m) for c in range(n))

queue = deque((r, c) for r in range(m) for c in range(n) if grid[r][c] == 2)

minutes = 0

while queue and fresh:

minutes += 1

for _ in range(len(queue)):

r, c = queue.popleft()

for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:

nr, nc = r + dr, c + dc

if 0 <= nr < m and 0 <= nc < n and grid[nr][nc] == 1:

grid[nr][nc] = 2

fresh -= 1

queue.append((nr, nc))

return minutes if fresh == 0 else -1

PHASE 5 — TEST & DEBUG

[[2,1,1],[1,1,0],[0,1,1]]: fresh=6. BFS spreads level by level. After 4 minutes fresh=0 → 4 ✓

Isolated fresh orange: fresh>0 after BFS done → -1 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(m*n)$. Multi-source BFS: same as Walls and Gates, same as 01 Matrix.

The 'fresh' counter eliminates the need to scan the grid again at the end.

Given more time I'd ask: what if diagonal rotting is allowed?

Add 4 diagonal direction pairs – same algorithm."

Community Solutions (Python)

• LeetDoocs

```

class Solution:
    def orangesRotting(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        cnt = 0
        q = deque()
        for i, row in enumerate(grid):
            for j, x in enumerate(row):
                if x == 2:
                    q.append((i, j))
                elif x == 1:
                    cnt += 1

        ans = 0
        dirs = (-1, 0, 1, 0, -1)
        while q and cnt:
            ans += 1
            for _ in range(len(q)):
                i, j = q.popleft()
                for a, b in pairwise(dirs):
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n and grid[x][y] == 1:
                        grid[x][y] = 2
                        q.append((x, y))
                        cnt -= 1
            if cnt == 0:
                return ans
        return -1 if cnt else 0

```

• SimplyLeet

```

from collections import deque

def orangesRotting(grid):
    # Dimensions of the grid
    rows, cols = len(grid), len(grid[0])
    # Queue to perform BFS, containing tuples of (row, col)
    rotten_queue = deque()
    fresh_count = 0

    # Step 1: Initialize the queue with all initially rotten oranges and count
    fresh oranges.

```

```

for r in range(rows):
    for c in range(cols):
        if grid[r][c] == 2:
            rotten_queue.append((r, c))
        elif grid[r][c] == 1:
            fresh_count += 1

# If there are no fresh oranges, return 0 immediately.
if fresh_count == 0:
    return 0

```

```

minutes_passed = 0
# Directions for adjacent cells: up, down, left, right.
directions = [(-1,0), (1,0), (0,-1), (0,1)]

# Step 2: BFS to rot adjacent fresh oranges.
while rotten_queue:
    # Process oranges that are rotten in the current minute.
    for _ in range(len(rotten_queue)):
        x, y = rotten_queue.popleft()

```

```

        for dx, dy in directions:
            nx, ny = x + dx, y + dy
            # Check if the neighbor is inside the grid and is a fresh
orange.
            if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == 1:
                # Mark the fresh orange as rotten.
                grid[nx][ny] = 2
                fresh_count -= 1
                # Add the new rotten orange into the queue.
                rotten_queue.append((nx, ny))
        # Increment minutes after processing one level.

```

```

minutes_passed += 1

```

```

# If there are still fresh oranges left, return -1.
return minutes_passed - 1 if fresh_count == 0 else -1

```

```

# Example usage:
# grid = [[2,1,1],[1,1,0],[0,1,1]]
# print( orangesRotting(grid) )

```

• LC.ca

```
class Solution:
    def orangesRotting(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        q = deque()
        cnt = 0
        for i in range(m):
            for j in range(n):
                if grid[i][j] == 2:
                    q.append((i, j))
                elif grid[i][j] == 1:
                    cnt += 1

        ans = 0
        while q and cnt:
            ans += 1
            for _ in range(len(q)):
                i, j = q.popleft()
                for a, b in [[0, 1], [0, -1], [1, 0], [-1, 0]]:
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n and grid[x][y] == 1:
                        cnt -= 1
                        grid[x][y] = 2
                        q.append((x, y))
        return ans if cnt == 0 else -1
```

◆ Quick Review

Key Insights

- Use Breadth-First Search (BFS) starting from all initially rotten oranges.
- Process the grid level by level, where each level represents one minute.

- Keep a count of fresh oranges and decrement it as they become rotten.
- If after the BFS there are still fresh oranges, then it is impossible to rot every orange.

Space & Time

Time Complexity: $O(m * n)$ as every cell is processed at most once.

Space Complexity: $O(m * n)$ in the worst case for the queue and auxiliary storage.

Solution

We begin by scanning the grid to locate all rotten oranges and count the fresh ones.

These rotten oranges are added to a queue for a BFS. For each minute (BFS level), we process all the current rotten oranges and try to infect their adjacent fresh oranges. If an adjacent cell contains a fresh orange, we mark it rotten, add it to the queue, and decrement our fresh count. We continue this process level by level (minute by minute) until there are no fresh oranges left. If, after processing, there are still fresh oranges remaining, we return -1 , indicating that not all oranges can become rotten.

BFS

□ ★ #1197 Minimum Knight Moves ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode

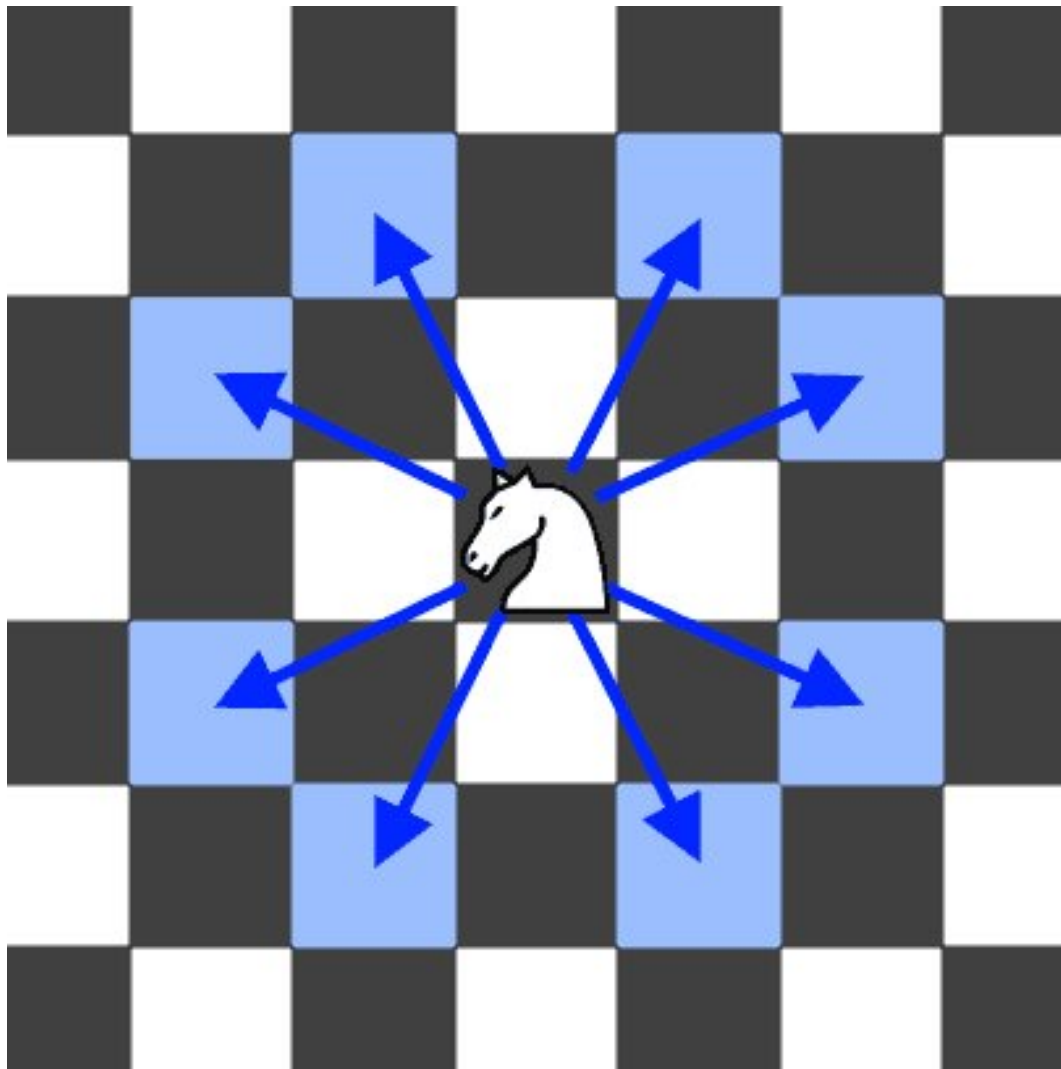
BFS

Lists: ★ Premium | Grind 169 | Premium 98

Problem

In an infinite chess board with coordinates from $-\infty$ to $+\infty$, you have a knight at square $[0, 0]$.

A knight has 8 possible moves it can make, as illustrated below. Each move is two squares in a cardinal direction, then one square in an orthogonal direction.



Return the minimum number of steps needed to move the knight to the square $[x, y]$. It is guaranteed the answer exists.

Example 1:

Input: $x = 2, y = 1$
Output: 1
Explanation: $[0, 0] \rightarrow [2, 1]$

Example 2:

Input: $x = 5, y = 5$
Output: 4
Explanation: $[0, 0] \rightarrow [2, 1] \rightarrow [4, 2] \rightarrow [3, 4] \rightarrow [5, 5]$

Constraints:

- $-300 \leq x, y \leq 300$
- $0 \leq |x| + |y| \leq 300$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Knight moves on an infinite board. Minimum moves from (0,0) to (x,y).
# 8 possible moves. Symmetry: |x| and |y| – reflect to first quadrant. Right?"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# BFS on an infinite chessboard from (0,0) to (x,y).
# Enqueue all 8 knight moves each step; stop when target is reached.
# Layer count = minimum moves. Prune to first quadrant since moves are symmetric.
# Time:  $O(\max(x,y)^2)$  BFS nodes. Space:  $O(\max(x,y)^2)$  visited.
# Should I go ahead and optimize?"
```

```
class _1197_MinKnightMoves:
```

```
    def minKnightMoves(self, x: int, y: int) -> int:
        x, y = abs(x), abs(y) # exploit symmetry – reflect to first quadrant
        queue = deque([(0, 0, 0)]) # (row, col, steps)
        seen = {(0, 0)}
        while queue:
            r, c, steps = queue.popleft()
            if r == x and c == y:
                return steps
            for dr, dc in [(2,1), (2,-1), (-2,1), (-2,-1), (1,2), (1,-2), (-1,2), (-1,-2)]:
                nr, nc = r + dr, c + dc
                if (nr, nc) not in seen and nr >= -1 and nc >= -1:
                    seen.add((nr, nc))
                    queue.append((nr, nc, steps + 1))
        return -1
```

PHASE 5 — TEST & DEBUG

```
# (2,1): BFS from (0,0). (2,1) reached in 1 step → 1 ✓
# (5,5): 4 steps ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(\max(|x|, |y|)^2)$ . Space  $O(\max(|x|, |y|)^2)$ . Standard BFS shortest path.
# Symmetry reduction: reflect to first quadrant cuts search space by 4.
```

The -1 lower bound allows the knight to 'go back' through negative coords when needed.
 # Given more time I'd ask: bidirectional BFS?
 # Start from both (0,0) and (x,y) simultaneously – meets in the middle, roughly halving depth."

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def minKnightMoves(self, x: int, y: int) -> int:
        q = deque([(0, 0)])
        ans = 0
        vis = {(0, 0)}
        dirs = ((-2, 1), (-1, 2), (1, 2), (2, 1), (2, -1), (1, -2), (-1, -2),
                (-2, -1))
        while q:
            for _ in range(len(q)):
                i, j = q.popleft()
                if (i, j) == (x, y):
                    return ans
            for a, b in dirs:
                c, d = i + a, j + b
                if (c, d) not in vis:
                    vis.add((c, d))
                    q.append((c, d))
            ans += 1
        return -1
```

```
class Solution:
    def minKnightMoves(self, x: int, y: int) -> int:
        def extend(m1, m2, q):
            for _ in range(len(q)):
                i, j = q.popleft()
                step = m1[(i, j)]
                for a, b in (
                    (-2, 1),
                    (-1, 2),
                    (1, 2),
```

```

        (2, 1),
        (2, -1),
        (1, -2),
        (-1, -2),
        (-2, -1),
    ):
        x, y = i + a, j + b
        if (x, y) in m1:
            continue
        if (x, y) in m2:
            return step + 1 + m2[(x, y)]
        q.append((x, y))
        m1[(x, y)] = step + 1
    return -1

if (x, y) == (0, 0):
    return 0
q1, q2 = deque([(0, 0)]), deque([(x, y)])
m1, m2 = {(0, 0): 0}, {(x, y): 0}
while q1 and q2:
    t = extend(m1, m2, q1) if len(q1) <= len(q2) else extend(m2, m1, q
2)

    if t != -1:
        return t
return -1

```

• SimplyLeet

```

# Python solution using BFS:
from collections import deque

def minKnightMoves(x, y):
    # Normalize target coordinates using symmetry
    x, y = abs(x), abs(y)

    # Possible knight moves (dx, dy)
    moves = [(2, 1), (1, 2), (-1, 2), (-2, 1), (-2, -1), (-1, -2), (1, -2), (2, -1)]

```

```

# BFS queue stores tuples of the form (current_x, current_y, move_count)
queue = deque([(0, 0, 0)])

# Set to track visited positions to prevent cycles
visited = {(0, 0)}

# Process BFS
while queue:
    curr_x, curr_y, move_count = queue.popleft()

    # Return the current move count if target is reached
    if curr_x == x and curr_y == y:
        return move_count

    # Explore all knight moves
    for dx, dy in moves:
        next_x, next_y = curr_x + dx, curr_y + dy

        # Prune search space: only process positions with valid bounds
        if (next_x, next_y) not in visited and next_x >= -1 and next_y >=
-1:

            visited.add((next_x, next_y))
            queue.append((next_x, next_y, move_count + 1))

    return -1 # Unreachable because a solution always exists

# Example test
#print(minKnightMoves(5, 5))

```

☐ • LC.ca

```
class Solution:
    def minKnightMoves(self, x: int, y: int) -> int:
        q = deque([(0, 0)])
        ans = 0
        vis = {(0, 0)}
        dirs = ((-2, 1), (-1, 2), (1, 2), (2, 1), (2, -1), (1, -2), (-1, -2),
        (-2, -1))
        while q:
            for _ in range(len(q)):
                i, j = q.popleft()
                if (i, j) == (x, y):
                    return ans
            for a, b in dirs:
                c, d = i + a, j + b
                if (c, d) not in vis:
                    vis.add((c, d))
                    q.append((c, d))
            ans += 1
        return -1
```

◆ Quick Review

Key Insights

- Leverage the symmetry of the chessboard by converting target coordinates to their absolute values.
- Use a Breadth-First Search (BFS) because all moves have equal weight, ensuring the first encounter with the target is the minimal path.
- Prune the search space by only considering positions that are likely beneficial (e.g., positions having coordinates not far less than

-1).

- Track visited states to avoid redundant processing.
- Optimize by bounding the search region relative to the target distance.

Space & Time

Time Complexity: $O(n)$ where n is the number of positions processed during the BFS

Space Complexity: $O(n)$ due to the storage requirements for the BFS queue and the visited set

Solution

We solve the problem using a BFS starting from the origin. First, normalize the target by taking the absolute value of x and y . BFS is then applied by exploring all eight possible knight moves at each step. Each move generates a new position that is enqueued if it hasn't been visited and falls within a reasonable bound (e.g., coordinates ≥ -1). The search terminates as soon as the target position is reached.

BFS

□ #1730 Shortest Path to Get Food

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · BFS · Matrix
Lists: Grind 169

Problem

You are starving and you want to eat food as quickly as possible. You want to find the shortest path to arrive at any food cell.

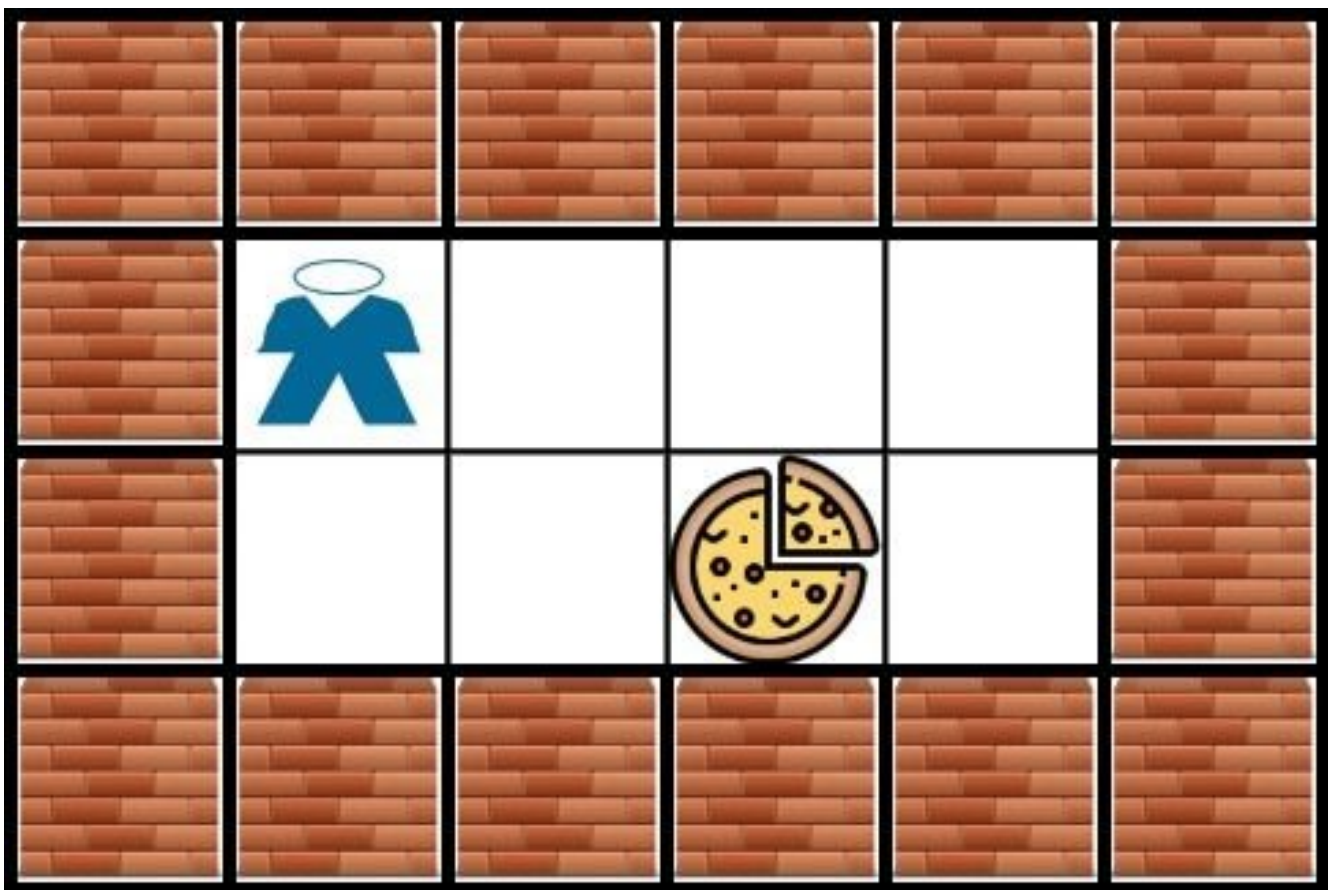
You are given an $m \times n$ character matrix, grid, of these different types of cells:

- '*' is your location. There is exactly one '*' cell.
- '#' is a food cell. There may be multiple food cells.
- 'O' is free space, and you can travel through these cells.
- 'X' is an obstacle, and you cannot travel through these cells.

You can travel to any adjacent cell north, east, south, or west of your current location if there is not an obstacle.

Return the length of the shortest path for you to reach any food cell. If there is no path for you to reach food, return -1.

Example 1:

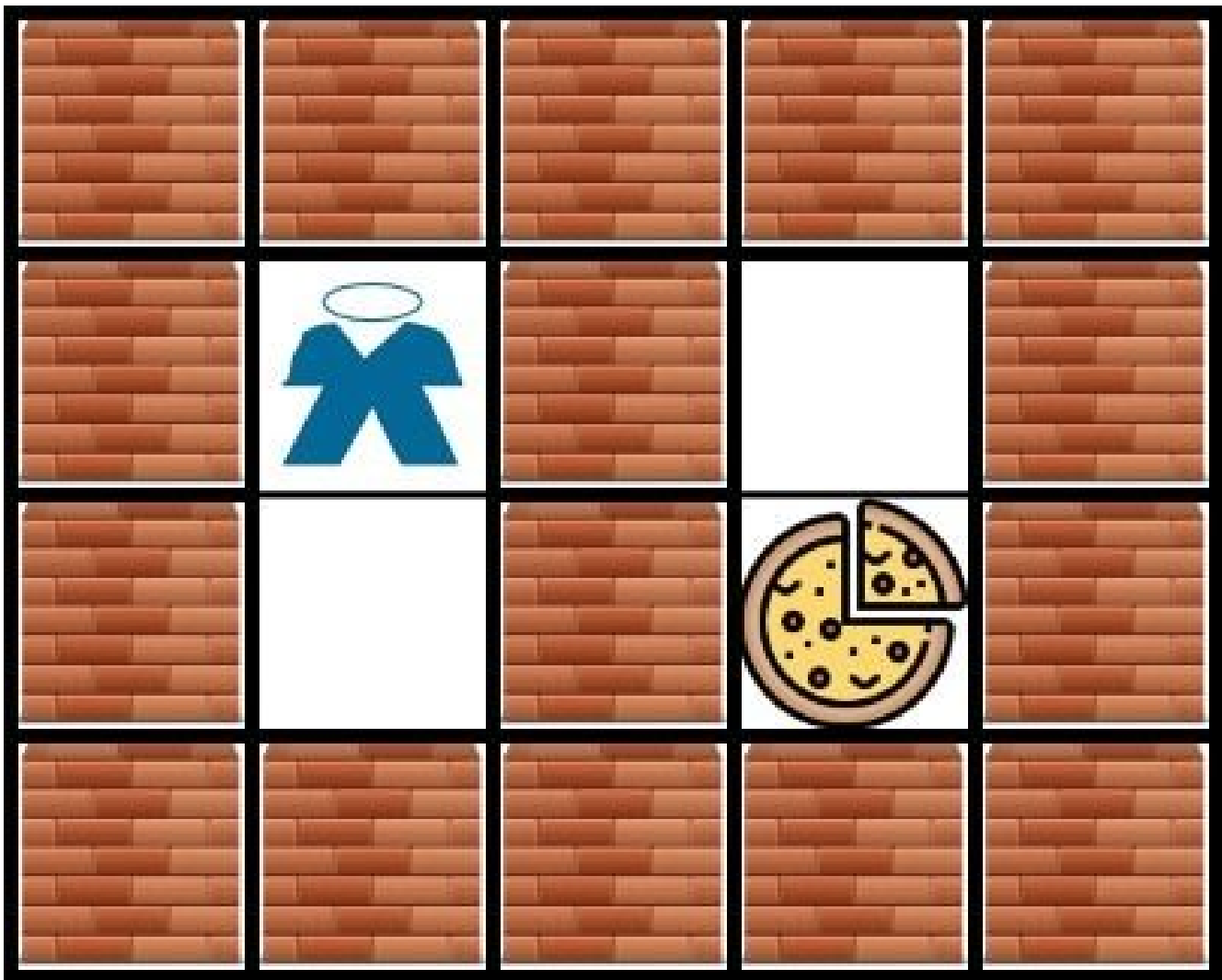


Input: grid = [["X","X","X","X","X","X"],["X","*","0","0","0","X"],["X","0","0","#","0","X"],["X","X","X","X","X","X"]]

Output: 3

Explanation: It takes 3 steps to reach the food.

Example 2:

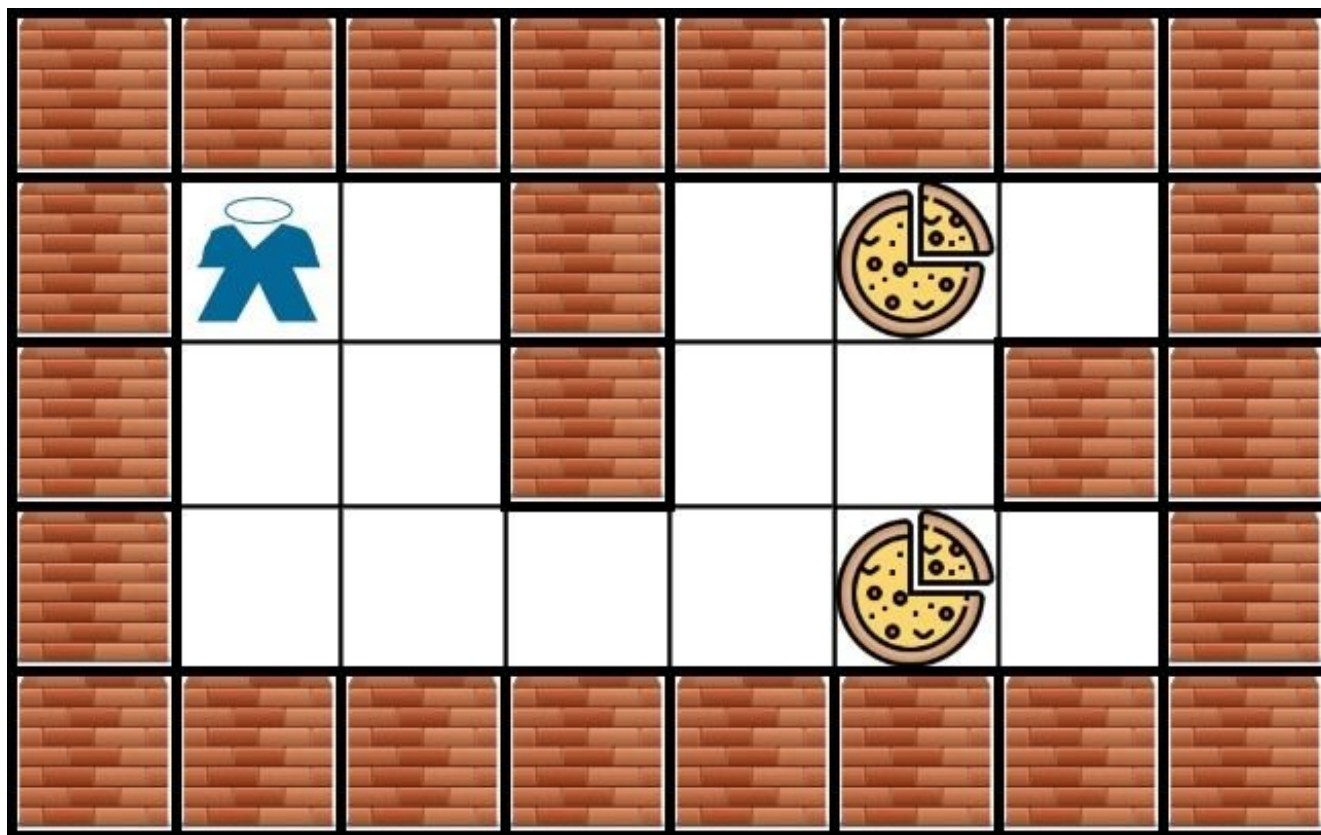


Input: grid = [["X","X","X","X","X"], ["X","*","X","0","X"], ["X","0","X","#","X"], ["X","X","X","X","X"]]

Output: -1

Explanation: It is not possible to reach the food.

Example 3:



Input: grid = [["X","X","X","X","X","X","X","X"], ["X","*","0","X","0","#","0","X"], ["X","0","0","X","0","0","X","X"], ["X","0","0","0","0","#","0","X"], ["X","X","X","X","X","X","X","X"]]

Output: 6

Explanation: There can be multiple food cells. It only takes 6 steps to reach the bottom food.

Example 4:

Input: grid = [["X","X","X","X","X","X","X","X"], ["X","*","0","X","0","#","0","X"], ["X","0","0","X","0","0","X","X"], ["X","0","0","0","0","#","0","X"], ["0","0","0","0","0","0","0","0"]]

Output: 5

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 200$
- $\text{grid}[\text{row}][\text{col}]$ is '*', 'X', 'O', or '#'.

- The grid contains exactly one '*'.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "BFS from '*' to the nearest '#'. '0' = free, 'X' = wall, '#' = food (any one).
# Return shortest path length. Return -1 if unreachable. Right?"
#
# "'*' → nearest '#' using 4-directional BFS. First '#' reached = shortest path."
```

PHASE 2 — BRUTE FORCE (also optimal)

```
# "Let me start with brute force to lock in the logic.
# Multi-source BFS from every '*' food cell simultaneously.
# First time a 'H' house is reached, record its distance; sum cannot reach uses -1.
# BFS is already optimal O(m*n); DFS would revisit cells wastefully.
# Time: O(m * n). Space: O(m * n) queue.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Standard BFS from source. First '#' reached is guaranteed shortest path.
# Mark visited by overwriting '0' with 'X' — no extra visited set needed.
# Time: O(m*n). Space: O(m*n)."
```

PHASE 4 — CLEAN CODE

```
class _1730_ShortestPathFood:
    def getFood(self, grid: List[List[str]]) -> int:
        m, n = len(grid), len(grid[0])
        start = next((r, c) for r in range(m) for c in range(n) if grid[r][c] ==
'*')

        queue = deque([(start[0], start[1], 0)])
        grid[start[0]][start[1]] = 'X' # mark visited
        while queue:
            r, c, dist = queue.popleft()
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                nr, nc = r + dr, c + dc
                if 0 <= nr < m and 0 <= nc < n:
                    if grid[nr][nc] == '#':
                        return dist + 1
                    if grid[nr][nc] == '0':
                        grid[nr][nc] = 'X' # mark visited
                        queue.append((nr, nc, dist + 1))

        return -1
```

PHASE 5 — TEST & DEBUG

```
# grid example 1: BFS from '*'. Nearest '#' is 3 steps away → return 3 ✓
# grid example 2: BFS exhausts all reachable '0' cells → return -1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(m*n). Space O(m*n).
```

```
# This is identical to #994 Rotting Oranges / #286 Walls and Gates – single-source BFS
```

```
# with a target condition instead of a distance fill.
```

```
# Overwriting '0' with 'X' acts as the visited marker – avoids O(m*n) extra space.
```

```
# Given more time I'd ask: what if we want paths to ALL food cells (not just shortest)?
```

```
# Continue BFS after first '#' hit – collect all '#' cells reached with their distances."
```

```
# []
```

```
# HARD PROBLEMS
```

```
# []
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def getFood(self, grid: list[list[str]]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(grid)
        n = len(grid[0])
        ans = 0
        q = collections.deque([self._getStartLocation(grid)])

        while q:
            for _ in range(len(q)):

                i, j = q.popleft()
                for dx, dy in DIRS:
                    x = i + dx
                    y = j + dy
                    if x < 0 or x == m or y < 0 or y == n:
                        continue
                    if grid[x][y] == 'X':
                        continue
                    if grid[x][y] == '#':
                        return ans + 1
```

```

        q.append((x, y))
        grid[x][y] = 'X' # Mark as visited.
    ans += 1

    return -1

def _getStartLocation(self, grid: list[list[str]]) -> tuple[int, int]:
    for i, row in enumerate(grid):
        for j, cell in enumerate(row):
            if cell == '*':

                return (i, j)

```

• LeetDoocs

```

class Solution:
    def getFood(self, grid: List[List[str]]) -> int:
        m, n = len(grid), len(grid[0])
        i, j = next((i, j) for i in range(m) for j in range(n) if grid[i][j] =
= '*')
        q = deque([(i, j)])
        dirs = (-1, 0, 1, 0, -1)
        ans = 0
        while q:
            ans += 1
            for _ in range(len(q)):

                i, j = q.popleft()
                for a, b in pairwise(dirs):
                    x, y = i + a, j + b
                    if 0 <= x < m and 0 <= y < n:
                        if grid[x][y] == '#':
                            return ans
                        if grid[x][y] == '0':
                            grid[x][y] = 'X'
                            q.append((x, y))

        return -1

```

• SimplyLeet

```
from collections import deque

def getFood(grid):
    # number of rows and columns
    rows = len(grid)
    cols = len(grid[0])
    # finding the start position
    start = None
    for r in range(rows):
        for c in range(cols):

            if grid[r][c] == '*':
                start = (r, c)
                break
    if start is not None:
        break

    # initialize queue for BFS, with (row, col, steps)
    queue = deque([(start[0], start[1], 0)])
    # mark visited cells to avoid reprocessing
    visited = [[False] * cols for _ in range(rows)]

    visited[start[0]][start[1]] = True

    # possible moves: up, right, down, left
    directions = [(-1, 0), (0, 1), (1, 0), (0, -1)]

    # begin BFS
    while queue:
        row, col, steps = queue.popleft()
        # if food is found, return current number of steps
        if grid[row][col] == '#':
```

```

        return steps
    # check all adjacent cells
    for dr, dc in directions:
        newRow, newCol = row + dr, col + dc
        # check boundaries, obstacles, and visited status
        if 0 <= newRow < rows and 0 <= newCol < cols and not visited[newRow][newCol] and grid[newRow][newCol] != 'X':
            visited[newRow][newCol] = True
            queue.append((newRow, newCol, steps + 1))

    # if no food is reachable return -1

    return -1

# Example usage:
grid = [
    ["X", "X", "X", "X", "X", "X"],
    ["X", "*", "0", "0", "0", "X"],
    ["X", "0", "0", "#", "0", "X"],
    ["X", "X", "X", "X", "X", "X"]
]
print(getFood(grid)) # Expected output: 3

```

• LC.ca

```

class Solution:
    def getFood(self, grid: List[List[str]]) -> int:
        m, n = len(grid), len(grid[0])
        i, j = next((i, j) for i in range(m) for j in range(n) if grid[i][j] == '*')
        q = deque([(i, j)])
        dirs = (-1, 0, 1, 0, -1)
        ans = 0
        while q:
            ans += 1
            for _ in range(len(q)):

```



```
i, j = q.popleft()
for a, b in pairwise(dirs):
    x, y = i + a, j + b
    if 0 <= x < m and 0 <= y < n:
        if grid[x][y] == '#':
            return ans
        if grid[x][y] == '0':
            grid[x][y] = 'X'
            q.append((x, y))

return -1
```

◆ Quick Review

Key Insights

- Use Breadth-First Search (BFS) to explore the grid level by level.
- The BFS guarantees that when a food cell ('#') is reached, the current distance is the shortest.
- Maintain a visited structure to avoid re-processing the same cell.
- Consider boundary and obstacle ('X') conditions while traversing.

Space & Time

Time Complexity: $O(m * n)$, where m is the number of rows and n is the number of columns since each cell is processed at most once.

Space Complexity: $O(m * n)$ in the worst case due to the queue and visited matrix.

Solution

We use Breadth-First Search (BFS) starting from the cell marked with '*'. The BFS uses a queue to explore neighbors in the four allowed directions (up, down, left, right). Each level of BFS represents one step in the path. As soon as a cell containing food ('#') is reached, the current step count is returned. We also maintain a visited matrix to ensure that cells are not revisited, which could otherwise result in cycles or redundant processing. This approach ensures that the first time we encounter food, we have found the shortest possible path.

Round 3 · High · Hard

Round 3

High Priority · Hard

23 questions · 8 patterns

Arrays & Hashing #41

Sliding Window #76

**Binary Search #4 #315 #410 #668 #710 #774
#1235**

Matrix #1074

Trees & BST #124 #297

DFS #269 #329 #685 #1036 #1192

Graphs #305 #1153

BFS #127 #317 #773 #815

Arrays & Hashing

Round 3 · High · Hard · 1 question

Arrays & Hashing

□ #41 First Missing Positive

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table
Lists: Grind 169

Problem

Given an unsorted integer array `nums`. Return the smallest positive integer that is not present in `nums`.

You must implement an algorithm that runs in $O(n)$ time and uses $O(1)$ auxiliary space.

Example 1:

Input: `nums = [1,2,0]`
Output: 3
Explanation: The numbers in the range `[1,2]` are all in the array.

Example 2:

Input: `nums = [3,4,-1,1]`
Output: 2
Explanation: 1 is in the array but 2 is missing.

Example 3:

Input: `nums = [7,8,9,11,12]`
Output: 1
Explanation: The smallest positive integer 1 is missing.

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-231 \leq \text{nums}[i] \leq 231 - 1$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def firstMissingPositive(self, nums: List[int]) -> int:
        def cyclicsort(nums):
            i = 0
            while i < len(nums):
                ci = nums[i]-1
                if 0 <= ci < len(nums) and nums[ci] != nums[i]:
                    nums[ci], nums[i] = nums[i], nums[ci]

                else:
                    i+=1

            cyclicsort(nums)
            for i in range(len(nums)):
                if nums[i] != i+1:
                    return i+1

            return len(nums)+1
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Smallest positive integer NOT in nums. O(n) time, O(1) auxiliary space.
# Answer is in range [1, n+1] – if all 1..n present, answer is n+1. Right?
# Negatives, zeros, and large numbers can be ignored."
#
# "[3,4,-1,1] → 2 (1 present, 2 missing) ✓. [7,8,9] → 1 ✓. [1,2,0] → 3 ✓."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Sort the array, then scan from 1 upward until the first missing positive integer."
```

```
# Or put all values in a hash set and scan i=1,2,3... until absent.
# Correct but sorting is  $O(n \log n)$  and the set uses  $O(n)$  extra space.
# Time:  $O(n \log n)$ . Space:  $O(n)$  for set approach.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (array as hash map — cycle sort)

"Key insight: use the array itself as a lookup table.

Valid range is $[1, n]$. Place each number x at index $x-1$ (if $1 \leq x \leq n$ and not already there).

After placing, scan: first i where $\text{nums}[i] \neq i+1 \rightarrow$ answer is $i+1$.

If all positions correct $\rightarrow$ answer is $n+1$.

Each number is placed at most once $\rightarrow O(n)$ total swaps."

PHASE 4 — CLEAN CODE

```
class _41_FirstMissingPositive:
```

```
    def firstMissingPositive(self, nums: List[int]) -> int:
```

```
        n = len(nums)
```

```
        for i in range(n):
```

```
            while 1 <= nums[i] <= n and nums[nums[i] - 1] != nums[i]:
```

```
                nums[nums[i] - 1], nums[i] = nums[i], nums[nums[i] - 1]
```

```
        for i in range(n):
```

```
            if nums[i] != i + 1:
```

```
                return i + 1
```

```
        return n + 1
```

PHASE 5 — TEST & DEBUG

$[3,4,-1,1]$: place $3 \rightarrow \text{pos}2$, $4 \rightarrow \text{pos}3$, -1 ignored, $1 \rightarrow \text{pos}0$.

After: $[1,-1,3,4]$. Scan: $i=1$ $\text{nums}[1] = -1 \neq 2 \rightarrow$ return $2 \checkmark$

$[1,2,3]$: after: $[1,2,3]$. All correct $\rightarrow$ return $4 \checkmark$

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$ — each element swapped at most once. Space $O(1)$.

The while loop runs at most n total iterations across the entire for loop — amortised $O(1)$ per i .

Given more time I'd ask: what if duplicates exist?

The condition $\text{nums}[\text{nums}[i]-1] \neq \text{nums}[i]$ handles duplicates — don't swap a number to a position

that already has the correct value (would loop forever)."

Community Solutions (Python)

• WalkCC


```
class Solution:
    def firstMissingPositive(self, nums: list[int]) -> int:
        n = len(nums)

        # Correct slot:
        # nums[i] = i + 1
        # nums[i] - 1 = i
        # nums[nums[i] - 1] = nums[i]
        for i in range(n):
            while nums[i] > 0 and nums[i] <= n and nums[nums[i] - 1] != nums[i]:
                nums[nums[i] - 1], nums[i] = nums[i], nums[nums[i] - 1]

        for i, num in enumerate(nums):
            if num != i + 1:
                return i + 1

        return n + 1
```

• LeetCode

```
class Solution:
    def firstMissingPositive(self, nums: List[int]) -> int:
        n = len(nums)
        for i in range(n):
            while 1 <= nums[i] <= n and nums[i] != nums[nums[i] - 1]:
                j = nums[i] - 1
                nums[i], nums[j] = nums[j], nums[i]
        for i in range(n):
            if nums[i] != i + 1:
                return i + 1

        return n + 1
```

• SimplyLeet

```
# Python implementation of First Missing Positive

def firstMissingPositive(nums):
    n = len(nums)
    # Rearrange numbers to their correct positions
    for i in range(n):
        while 1 <= nums[i] <= n and nums[nums[i] - 1] != nums[i]:
            correct_index = nums[i] - 1
            nums[i], nums[correct_index] = nums[correct_index], nums[i]

    # Identify the first missing positive integer
    for i in range(n):
        if nums[i] != i + 1:
            return i + 1
    return n + 1

# Example usage:
print(firstMissingPositive([3, 4, -1, 1])) # Output should be 2
```

• LC.ca

```
class Solution:
    def firstMissingPositive(self, nums: List[int]) -> int:
        def swap(i, j):
            nums[i], nums[j] = nums[j], nums[i]

        n = len(nums)
        for i in range(n):
            while 1 <= nums[i] <= n and nums[i] != nums[nums[i] - 1]:
                swap(i, nums[i] - 1)
        for i in range(n):
            if i + 1 != nums[i]:
                return i + 1
        return n + 1
```

◆ Quick Review

Key Insights

- The missing positive number must be in the range $[1, n+1]$ where n is the length of the array.
- Use the array indices to place numbers in their correct positions (i.e., number i should be at index $i-1$) with in-place swapping.
- After rearrangement, scan the array: the first index where the number is not equal to $\text{index}+1$ indicates the missing positive integer.

Space & Time

Time Complexity: $O(n)$ as each number is swapped at most once.

Space Complexity: $O(1)$ since the rearrangement occurs in-place.

Solution

The approach involves reordering the array so that each positive integer in the range $[1, n]$ is placed at the index corresponding to its value minus one. This means for any valid number i , $\text{nums}[i-1]$ should be i . We iterate through the array and perform in-place swaps to achieve this order. After this process, another pass through the array reveals the first position where the number does not match the

expected value ($i+1$), which is the smallest missing positive. If every position is correct, then the missing value is $n+1$.

Sliding Window

Round 3 · High · Hard · 1 question

Sliding Window

□ #76 Minimum Window Substring

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Sliding Window
Lists: Grind 169

Problem

Given two strings s and t of lengths m and n respectively, return the minimum window substring of s such that every character in t (including duplicates) is included in the window. If there is no such substring, return the empty string `""`.

The testcases will be generated such that the answer is unique.

Example 1:

```
Input: s = "ADOBECODEBANC", t = "ABC"
Output: "BANC"
Explanation: The minimum window substring "BANC" includes 'A', 'B', and 'C' from string t.
```

Example 2:

```
Input: s = "a", t = "a"
Output: "a"
Explanation: The entire string s is the minimum window.
```

Example 3:

Input: s = "a", t = "aa"

Output: ""

Explanation: Both 'a's from t must be included in the window.
Since the largest window of s only has one 'a', return empty string.

Constraints:

- $m == s.length$
- $n == t.length$
- $1 \leq m, n \leq 105$
- s and t consist of uppercase and lowercase English letters.

Follow up: Could you find an algorithm that runs in $O(m + n)$ time?

My LeetCode Solution

• My LeetCode Solution

```
class Solution {
public:
    string minWindow(string s, string t) {
        unordered_map<char,int> tmap = {};
        for (char c: t){
            tmap[c]++;
        }

        int required = tmap.size();
        int formed = 0;
```



```

unordered_map<char, int> window = {};

int minlen = INT_MAX;
int bestleft = 0;
int l = 0;

for (int r=0; r<s.size(); r++){
    char rightchar = s[r];
    window[rightchar]++;
    if (tmap.count(rightchar) and tmap[rightchar] == window[rightchar]
){

        formed++;
    }

    while (l<=r and required == formed){
        int length = r-l+1;
        if (length < minlen){
            minlen = length;
            bestleft = l;
        }
        char leftchar = s[l];

        window[leftchar]--;
        if (tmap[leftchar] > window[leftchar]){
            formed--;
        }

        l++;
    }
}

return (minlen == INT_MAX ? "" : s.substr(bestleft,minlen));

}
};

```

◆ Interview Approach · STAR-LC

Round 3 · High · Hard

p.1629

PHASE 1 — CLARIFY

```
# "Smallest window in s containing all characters of t (with multiplicity).
# Return '' if no such window. Right?
# t may have duplicate chars – window must have at least that many."
#
# "s='ADOBECODEBANC', t='ABC' → 'BANC' (contains A,B,C, length 4) ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Enumerate every substring of s with two nested indices.
# For each substring, check whether it contains all characters of t (with
frequency).
# Track the shortest valid window – correct but  $O(m^2 * n)$  character checks.
# Time:  $O(m^2 * n)$ . Space:  $O(n)$  frequency map.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (sliding window with 'formed' counter)

```
# "Expand right: add s[right] to window frequency map.
# When a char's count in window meets t's requirement, increment 'formed'.
# When formed == len(distinct chars in t): window is valid. Try shrinking from left.
# Track best window.  $O(m+n)$  – each char enters/exits at most once."
```

PHASE 4 — CLEAN CODE

```
class _76_MinWindowSubstring:
    def minWindow(self, s: str, t: str) -> str:
        if not t or not s:
            return ''
        need = Counter(t)
        have : Counter = Counter()
        formed = 0
        required = len(need)
        left = 0
        best = (float('inf'), 0, 0) # (length, left, right)
        for right in range(len(s)):
            c = s[right]
            have[c] += 1
            if c in need and have[c] == need[c]:
                formed += 1
            while formed == required:
                if right - left + 1 < best[0]:
                    best = (right - left + 1, left, right)
                have[s[left]] -= 1
                if s[left] in need and have[s[left]] < need[s[left]]:
                    formed -= 1
                left += 1
        return '' if best[0] == float('inf') else s[best[1]: best[2] + 1]
```

PHASE 5 — TEST & DEBUG

```
# s='ADOBECODEBANC', t='ABC': need={A:1,B:1,C:1}. required=3.
```

Window expands until it contains A,B,C. Shrink left until one char drops below need.

Best window = 'BANC' ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m+n)$. Space $O(m+n)$. The 'formed' counter avoids comparing the full frequency maps.

A window is valid exactly when `formed==required` – elegant $O(1)$ validity check.

Given more time I'd ask: find all minimum windows?

Collect all windows where `length == best length` during the BFS scan."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minWindow(self, s: str, t: str) -> str:
        count = collections.Counter(t)
        required = len(t)
        bestLeft = -1
        minLength = len(s) + 1

        l = 0
        for r, c in enumerate(s):
            count[c] -= 1

            if count[c] >= 0:
                required -= 1
            while required == 0:
                if r - l + 1 < minLength:
                    bestLeft = l
                    minLength = r - l + 1
                count[s[l]] += 1
                if count[s[l]] > 0:
                    required += 1
                l += 1

        return '' if bestLeft == -1 else s[bestLeft: bestLeft + minLength]
```

• LeetDoocs

```

class Solution:
    def minWindow(self, s: str, t: str) -> str:
        need = Counter(t)
        window = Counter()
        cnt = l = 0
        k, mi = -1, inf
        for r, c in enumerate(s):
            window[c] += 1
            if need[c] >= window[c]:
                cnt += 1

        while cnt == len(t):
            if r - l + 1 < mi:
                mi = r - l + 1
                k = l
            if need[s[l]] >= window[s[l]]:
                cnt -= 1
            window[s[l]] -= 1
            l += 1
        return "" if k < 0 else s[k : k + mi]

```

• SimplyLeet

```

# Python implementation of Minimum Window Substring solution

def minWindow(s: str, t: str) -> str:
    # If t is longer than s, no solution exists.
    if len(t) > len(s):
        return ""

    # Dictionary to store the frequency of each character in t.
    t_freq = {}
    for char in t:

```

```
t_freq[char] = t_freq.get(char, 0) + 1

# Dictionary to keep track of the characters in the current window.
window_freq = {}

required = len(t_freq) # Unique characters in t to be matched
formed = 0 # Number of unique characters in the current window that meet
required frequency

# (window_length, left, right)
answer = (float('inf'), None, None)

l = 0 # Left pointer
# Start sliding the window with the right pointer.
for r, char in enumerate(s):
    # Add the current character to the window counter.
    window_freq[char] = window_freq.get(char, 0) + 1

    # Check if the current character's frequency in the window matches that
    in t.
    if char in t_freq and window_freq[char] == t_freq[char]:
        formed += 1

    # Try and contract the window if all characters are matched.
    while l <= r and formed == required:
        character = s[l]

        # Update answer if the current window is smaller than the previous
        best.
        if r - l + 1 < answer[0]:
            answer = (r - l + 1, l, r)

    # The character at the left is going to be removed from the window.
```

```

        window_freq[character] -= 1
        if character in t_freq and window_freq[character] < t_freq[character]:
            formed -= 1

        # Move the left pointer ahead.
        l += 1

    # Return the smallest window, if found.
    return "" if answer[0] == float('inf') else s[answer[1]:answer[2]+1]

# Example usage:
if __name__ == "__main__":
    s = "ADOBECODEBANC"
    t = "ABC"
    print(minWindow(s, t)) # Expected output: "BANC"

```

• LC.ca

```

'''
>>> deq = collections.deque([])
>>> deq.append(11)
>>> deq.append(22)
>>> deq.append(33)
>>>
>>> deq[0]
11
'''

```

```

import collections

class Solution:
    def minWindow(self, s: str, t: str) -> str:
        cnt = 0
        need = collections.Counter(t)
        start, end = len(s), 3 * len(s) # Arbitrary 3, just make sure end-start
        is larger than input s length, for later min-check
        d = {}
        # Using queue to store indexes, good for a large amount of API calls
        deq = collections.deque([])

```

```

for i, c in enumerate(s):
    if c in need:
        deq.append(i)
        d[c] = d.get(c, 0) + 1
        # '=' also +1, because it's inceased already one line above :)
        if d[c] <= need[c]:
            cnt += 1
        while deq and d[s[deq[0]]] > need[s[deq[0]]]:
            d[s[deq.popleft()]] -= 1
        if cnt == len(t) and deq[-1] - deq[0] < end - start:

            start, end = deq[0], deq[-1]
return s[start:end + 1]

```

```
#####
```

```
from collections import Counter
```

```

class Solution:
    def minWindow(self, s: str, t: str) -> str:
        ans = ''

```

```

        m, n = len(s), len(t)
        if m < n:
            return ans
        need = Counter(t)
        window = Counter()
        i, cnt, mi = 0, 0, inf
        for j, c in enumerate(s):
            window[c] += 1
            if need[c] >= window[c]: # >= , because 1 line above already +=1
                cnt += 1

```

```
while cnt == n:
    if j - i + 1 < mi: # in while
        mi = j - i + 1
        ans = s[i : j + 1]
    c = s[i]
    if need[c] >= window[c]: # char in window but not in need,
        need[c]=0, window[c]=1..2..
        cnt -= 1
    window[c] += 1
    i += 1
return ans
```

◆ Quick Review

Key Insights

- Use a sliding window approach to dynamically adjust the window boundaries.
- Utilize a hash table (or dictionary) to keep track of the frequency of characters required (from t) and the frequency within the current window.
- Expand the window until it satisfies the requirements, then contract to minimize the window.
- Efficiently update counts and check valid window with two pointers (left and right).

Space & Time

Time Complexity: $O(m + n)$, where m is the length of s and n is the length of t .

Space Complexity: $O(m + n)$ in the worst case due to the storage for the hash maps.

Solution

We use a sliding window technique with two pointers representing the window's boundaries. First, create a frequency map of characters from t . Then, expand the right pointer to include characters in the window until the window contains all required characters (meeting or exceeding the counts in t). Once the requirement is met, try contracting the window by moving the left pointer to reduce its size while still maintaining a valid window. Throughout the process, if a valid window is found that is smaller than the previously recorded minimum window, update the minimum. The solution leans heavily on hash tables to track counts and compares them against the required counts from t .

Binary Search

Round 3 · High · Hard · 7 questions

Binary Search

□ #4 Median of Two Sorted Arrays

HAR

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Divide and Conquer
Lists: Grind 169

Problem

Given two sorted arrays `nums1` and `nums2` of size `m` and `n` respectively, return the median of the two sorted arrays.

The overall run time complexity should be $O(\log(m+n))$.

Example 1:

Input: `nums1 = [1,3]`, `nums2 = [2]`
Output: `2.00000`
Explanation: merged array = `[1,2,3]` and median is 2.

Example 2:

Input: `nums1 = [1,2]`, `nums2 = [3,4]`
Output: `2.50000`
Explanation: merged array = `[1,2,3,4]` and median is $(2 + 3) / 2 = 2.5$.

Constraints:

- `nums1.length == m`
- `nums2.length == n`

- $0 \leq m \leq 1000$
- $0 \leq n \leq 1000$
- $1 \leq m + n \leq 2000$
- $-106 \leq \text{nums1}[i], \text{nums2}[i] \leq 106$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
        m = len(nums1)
        n = len(nums2)
        i=0
        j=0
        h = (m+n)//2

        c=p = 0

        for k in range(h+1):
            p = c
            if j >= n or (i<m and nums1[i] <= nums2[j]):
                c = nums1[i]
                i+=1
            else:
                c = nums2[j]
                j+=1

        return (c+p)/2 if not (m+n)&1 else c
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Merge two sorted arrays conceptually and find the median. $O(\log(m+n))$ required. Right?"

Median: if total length is odd → middle element. Even → average of two middles."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Merge both sorted arrays with two pointers into one combined array.

Walk to index $(m+n)//2$ (and the one before if even length) to read the median.

Correct but uses $O(m+n)$ extra space and ignores the logarithmic requirement.

Time: $O(m + n)$. Space: $O(m + n)$ merged array.

Should I go ahead and optimize?"

```
class _4_MedianTwoArrays_Brute:
```

```
    def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
```

```
        m, n = len(nums1), len(nums2)
```

```
        i = j = 0
```

```
        half = (m + n) // 2
```

```
        prev = curr = 0
```

```
        for _ in range(half + 1):
```

```
            prev = curr
```

```
            if j >= n or (i < m and nums1[i] <= nums2[j]):
```

```
                curr = nums1[i]; i += 1
```

```
            else:
```

```
                curr = nums2[j]; j += 1
```

```
        return (curr + prev) / 2 if not (m + n) & 1 else curr
```

PHASE 3 — OPTIMIZE (binary search on partition — true $O(\log(\min(m,n)))$)

"Partition both arrays into left and right halves such that:

total left size = $(m+n+1)//2$, $\max(\text{left}) \leq \min(\text{right})$.

Binary search on the partition point of the smaller array.

At each partition: check if $\text{nums1}[\text{mid}-1] \leq \text{nums2}[\text{r}-\text{mid}]$ and $\text{nums2}[\text{r}-\text{mid}-1] \leq \text{nums1}[\text{mid}]$.

Median = $\max(\text{left halves})$ for odd total, or average of $\max(\text{left})/\min(\text{right})$ for even."

PHASE 4 — CLEAN CODE

```
class _4_MedianTwoArrays:
```

```
    def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
```

```
        if len(nums1) > len(nums2):
```

```
            nums1, nums2 = nums2, nums1
```

```
        m, n = len(nums1), len(nums2)
```

```
        half = (m + n + 1) // 2
```

```
        lo, hi = 0, m
```

```
        while lo <= hi:
```

```
            i = (lo + hi) // 2 # partition of nums1
```

```
            j = half - i # partition of nums2
```

```
            left1 = nums1[i - 1] if i > 0 else float('-inf')
```

```
            right1 = nums1[i] if i < m else float('inf')
```

```
            left2 = nums2[j - 1] if j > 0 else float('-inf')
```

```
            right2 = nums2[j] if j < n else float('inf')
```

```
            if left1 <= right2 and left2 <= right1:
```

```
                if (m + n) % 2 == 1:
```

```

        return float(max(left1, left2))
    return (max(left1, left2) + min(right1, right2)) / 2
elif left1 > right2:
    hi = i - 1
else:
    lo = i + 1
return 0.0

```

PHASE 5 — TEST & DEBUG

```

# nums1=[1,3], nums2=[2]: m=2,n=2. half=2. Partition i=1,j=1:
left1=1,right1=3,left2=2,right2=∞.
# 1<=∞ and 2<=3 ✓. Total=3 (odd) → max(1,2)=2. → 2.0 ✓
# nums1=[1,2], nums2=[3,4]: Partition → max(left)=2, min(right)=3. → 2.5 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(log(min(m,n))). Space O(1). Binary search on the smaller array's
partition.
# The +inf/-inf sentinels handle edge cases (empty left/right) cleanly.
# The brute-force merge approach is often acceptable in interviews – present both
and let the interviewer choose.
# Given more time I'd ask: kth element of two sorted arrays?
# Binary search on k – eliminate k//2 elements from one array per step. Same O(log
k) idea."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def findMedianSortedArrays(self, nums1: list[int], nums2: list[int]) -> float:
        n1 = len(nums1)
        n2 = len(nums2)
        if n1 > n2:
            return self.findMedianSortedArrays(nums2, nums1)

        l = 0
        r = n1

```

```

while l <= r:
    partition1 = (l + r) // 2
    partition2 = (n1 + n2 + 1) // 2 - partition1
    maxLeft1 = -2**31 if partition1 == 0 else nums1[partition1 - 1]
    maxLeft2 = -2**31 if partition2 == 0 else nums2[partition2 - 1]
    minRight1 = 2**31 - 1 if partition1 == n1 else nums1[partition1]
    minRight2 = 2**31 - 1 if partition2 == n2 else nums2[partition2]
    if maxLeft1 <= minRight2 and maxLeft2 <= minRight1:
        return (max(maxLeft1, maxLeft2) + min(minRight1, minRight2)) * 0.5 if
(n1 + n2) % 2 == 0 else max(maxLeft1, maxLeft2)
    elif maxLeft1 > minRight2:
        r = partition1 - 1
    else:
        l = partition1 + 1

```

• LeetDoocs

```

class Solution:
    def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
        def f(i: int, j: int, k: int) -> int:
            if i >= m:
                return nums2[j + k - 1]
            if j >= n:
                return nums1[i + k - 1]
            if k == 1:
                return min(nums1[i], nums2[j])
            p = k // 2
            x = nums1[i + p - 1] if i + p - 1 < m else inf
            y = nums2[j + p - 1] if j + p - 1 < n else inf
            return f(i + p, j, k - p) if x < y else f(i, j + p, k - p)

        m, n = len(nums1), len(nums2)
        a = f(0, 0, (m + n + 1) // 2)
        b = f(0, 0, (m + n + 2) // 2)
        return (a + b) / 2

```

• SimplyLeet


```

# Function to find the median of two sorted arrays
def findMedianSortedArrays(nums1, nums2):
    # Ensure nums1 is the smaller array for faster binary search
    if len(nums1) > len(nums2):
        nums1, nums2 = nums2, nums1

    m, n = len(nums1), len(nums2)
    low, high = 0, m

    while low <= high:

        # Partition indexes for nums1 and nums2
        partition_nums1 = (low + high) // 2
        partition_nums2 = (m + n + 1) // 2 - partition_nums1

        # If partition is 0 then there are no elements on left side. Use
        # -infinity for comparison.
        maxLeftNums1 = float('-inf') if partition_nums1 == 0 else nums1[partition_nums1 - 1]
        # If partition is length then there are no elements on right side. Use
        # infinity for comparison.
        minRightNums1 = float('inf') if partition_nums1 == m else nums1[partition_nums1]

        # Similarly for nums2
        maxLeftNums2 = float('-inf') if partition_nums2 == 0 else nums2[partition_nums2 - 1]
        minRightNums2 = float('inf') if partition_nums2 == n else nums2[partition_nums2]

        # Check if we have found the correct partition
        if maxLeftNums1 <= minRightNums2 and maxLeftNums2 <= minRightNums1:
            # If total number of elements is even
            if (m + n) % 2 == 0:
                return (max(maxLeftNums1, maxLeftNums2) + min(minRightNums1, minRightNums2)) / 2.0
            else:
                return float(max(maxLeftNums1, maxLeftNums2))

```

```

# If not, adjust binary search range
elif maxLeftNums1 > minRightNums2:
    # Move partition in nums1 to left
    high = partition_nums1 - 1
else:
    # Move partition in nums1 to right
    low = partition_nums1 + 1

# Example usage:
print(findMedianSortedArrays([1,3], [2])) # Output: 2.0

print(findMedianSortedArrays([1,2], [3,4])) # Output: 2.5

```

• LC.ca

```

class Solution:
    def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:

        def findKth(i: int, j: int, k: int) -> float:
            if i >= m:
                return nums2[j + k - 1]
            if j >= n:
                return nums1[i + k - 1]
            if k == 1:
                return min(nums1[i], nums2[j])

            # Division a // b : floordiv(a, b)
            midVal1 = nums1[i + k // 2 - 1] if i + k // 2 - 1 < m else math.in
            midVal2 = nums2[j + k // 2 - 1] if j + k // 2 - 1 < n else math.in

            if midVal1 < midVal2:
                return findKth(i + k // 2, j, k - k // 2) # '+' or '-' k//2
            else:
                return findKth(i, j + k // 2, k - k // 2)

```

```

    m = len(nums1)
    n = len(nums2)
    # Division a // b : floordiv(a, b)
    left = (m + n + 1) // 2
    right = (m + n + 2) // 2
    return (findKth(0, 0, left) + findKth(0, 0, right)) / 2.0

#####

# iteration version

```

```

class Solution(object):
    def findMedianSortedArrays(self, nums1, nums2):
        a, b = sorted((nums1, nums2), key=len)
        m, n = len(a), len(b)
        after = (m + n - 1) / 2
        lo, hi = 0, m
        while lo < hi:
            i = (lo + hi) / 2
            if after - i - 1 < 0 or a[i] >= b[after - i - 1]:
                hi = i

        else:
            lo = i + 1
        i = lo
        nextfew = sorted(a[i:i + 2] + b[after - i:after - i + 2])
        return (nextfew[0] + nextfew[1 - (m + n) % 2]) / 2.0

```

◆ Quick Review

Key Insights

- The median divides the sorted array into two equal halves.
- We can perform binary search on the smaller array to partition both arrays correctly.

- We need to ensure that every element in the left partition is less than or equal to every element in the right partition.
- Edge cases must be handled when partitions are at the boundaries of either array.

Space & Time

Time Complexity: $O(\log(\min(m, n)))$

Space Complexity: $O(1)$

Solution

The solution uses a binary search technique on the smaller array. We define a partition index i for `nums1` and compute the corresponding index j for `nums2` such that $i + j$ equals half of the total number of elements. The algorithm checks if the largest element on the left side of `nums1` (if any) is less than or equal to the smallest element on the right side of `nums2`, and vice versa for `nums2`. If the condition holds, we have found the correct partition and compute the median. If not, adjust the binary search range until the correct partition is found. This partitioning strategy allows us to find the median in logarithmic time.

Binary Search

□ #315 Count of Smaller Numbers After Self

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Divide and Conquer ·
Binary Indexed Tree · Segment Tree · Merge
Sort · Ordered Set

Lists: Denny Zhang

Problem

Given an integer array `nums`, return an integer array `counts` where `counts[i]` is the number of smaller elements to the right of `nums[i]`.

Example 1:

```
Input: nums = [5,2,6,1]
Output: [2,1,1,0]
Explanation:
To the right of 5 there are 2 smaller elements (2 and 1).
To the right of 2 there is only 1 smaller element (1).
To the right of 6 there is 1 smaller element (1).
To the right of 1 there is 0 smaller element.
```

Example 2:

```
Input: nums = [-1]
Output: [0]
```

Example 3:

Input: `nums = [-1,-1]`
 Output: `[0,0]`

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"For each index *i*, count elements to the right of *i* that are strictly less than `nums[i]`.
 # Return as an array. Right?"
 #
 # "[5,2,6,1] → [2,1,1,0]: right of 5 has 2 and 1 (2 smaller), etc. ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
 # For each index *i*, scan every element to its right and count how many are smaller than `nums[i]`.
 # Store those counts in an output array.
 # Correct, but the inner scan makes this quadratic overall.
 # Time: $O(n^2)$. Space: $O(1)$ besides output.
 # Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (BIT / merge sort)

"Process from right to left. Build a BIT indexed by coordinate-compressed value.
 # For each `nums[i]`: query BIT for count of values in `[min, nums[i]-1]`.
 # Update BIT at `nums[i]`. $O(n \log n)$ time."

PHASE 4 — CLEAN CODE

```
class _315_CountSmaller:
    def countSmaller(self, nums: List[int]) -> List[int]:
        sorted_unique = sorted(set(nums))
        rank = {v: i + 1 for i, v in enumerate(sorted_unique)}
        m = len(sorted_unique)
        bit = [0] * (m + 1)
        def update(i):
            while i <= m:
                bit[i] += 1
                i += i & (-i)
```

```
def query(i):
    s = 0
    while i > 0:
        s += bit[i]
        i -= i & (-i)
    return s

result = []
for num in reversed(nums):
    r = rank[num]
    result.append(query(r - 1)) # count of values < num seen so far
    update(r)
return result[::-1]
```

PHASE 5 — TEST & DEBUG

[5,2,6,1]: process right→left. 1→query(0)=0,update(1). 6→query(3)=1,update(4).
 2→query(1)=1,update(2). 5→query(2)=2.
 # reversed result=[2,1,1,0] → reverse → [2,1,1,0] ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log n)$. Space $O(n)$. BIT query gives prefix count in $O(\log n)$.
 # Merge sort alternative: during merge, count inversions – same complexity but different mental model.
 # Given more time I'd ask: count of larger elements after self?
 # Query `total_right - query(rank[num])` instead."

Community Solutions (Python)

• WalkCC

```
class FenwickTree:
    def __init__(self, n: int):
        self.sums = [0] * (n + 1)

    def add(self, i: int, delta: int) -> None:
        while i < len(self.sums):
            self.sums[i] += delta
            i += FenwickTree.lowbit(i)

    def get(self, i: int) -> int:
```



```

    summ = 0
    while i > 0:
        summ += self.sums[i]
        i -= FenwickTree.lowbit(i)
    return summ

```

```

@staticmethod
def lowbit(i: int) -> int:
    return i & -i

```

```

class Solution:
    def countSmaller(self, nums: list[int]) -> list[int]:
        ans = []
        ranks = self._getRanks(nums)
        tree = FenwickTree(len(ranks))

        for num in reversed(nums):
            ans.append(tree.get(ranks[num] - 1))
            tree.add(ranks[num], 1)

```

```

    return ans[::-1]

```

```

def _getRanks(self, nums: list[int]) -> dict[int, int]:
    ranks = collections.Counter()
    rank = 0
    for num in sorted(set(nums)):
        rank += 1
        ranks[num] = rank
    return ranks

```

```

from dataclasses import dataclass

```

```

@dataclass
class Item:
    num: int = 0
    index: int = 0

```

```

class Solution:

```

```

def countSmaller(self, nums: list[int]) -> list[int]:
    n = len(nums)
    ans = [0] * n
    items = [Item(num, i) for i, num in enumerate(nums)]

    self._mergeSort(items, 0, n - 1, ans)
    return ans

def _mergeSort(
    self,
    items: list[Item],
    l: int,
    r: int,
    ans: list[int],
) -> None:
    if l >= r:
        return

    m = (l + r) // 2
    self._mergeSort(items, l, m, ans)

    self._mergeSort(items, m + 1, r, ans)
    self._merge(items, l, m, r, ans)

def _merge(
    self,
    items: list[Item],
    l: int,
    m: int,
    r: int,
    ans: list[int],
) -> None:
    sorted = [Item()] * (r - l + 1)
    k = 0 # sorted's index
    i = l # left's index
    j = m + 1 # right's index
    rightCount = 0 # the number of numbers < items[i].num

    while i <= m and j <= r:
        if items[i].num > items[j].num:
            rightCount += 1

```

```

        sorted[k] = items[j]
        k += 1
        j += 1
    else:
        ans[items[i].index] += rightCount
        sorted[k] = items[i]
        k += 1
        i += 1

# Put the possible remaining left part into the sorted array.

while i <= m:
    ans[items[i].index] += rightCount
    sorted[k] = items[i]
    k += 1
    i += 1

# Put the possible remaining right part into the sorted array.
while j <= r:
    sorted[k] = items[j]
    k += 1

    j += 1

items[l:l + len(sorted)] = sorted

```

• LeetDoocs

```

class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    @staticmethod
    def lowbit(x):
        return x & -x

    def update(self, x, delta):

```

```
        while x <= self.n:
            self.c[x] += delta
            x += BinaryIndexedTree.lowbit(x)

    def query(self, x):
        s = 0
        while x > 0:
            s += self.c[x]
            x -= BinaryIndexedTree.lowbit(x)
        return s
```

```
class Solution:
    def countSmaller(self, nums: List[int]) -> List[int]:
        alls = sorted(set(nums))
        m = {v: i for i, v in enumerate(alls, 1)}
        tree = BinaryIndexedTree(len(m))
        ans = []
        for v in nums[::-1]:
            x = m[v]
```

```
            tree.update(x, 1)
            ans.append(tree.query(x - 1))
        return ans[::-1]
```

```
class Node:
    def __init__(self):
        self.l = 0
        self.r = 0
        self.v = 0
```

```
class SegmentTree:
    def __init__(self, n):
        self.tr = [Node() for _ in range(n << 2)]
```

```

        self.build(1, 1, n)

    def build(self, u, l, r):
        self.tr[u].l = l
        self.tr[u].r = r
        if l == r:
            return
        mid = (l + r) >> 1
        self.build(u << 1, l, mid)
        self.build(u << 1 | 1, mid + 1, r)

    def modify(self, u, x, v):
        if self.tr[u].l == x and self.tr[u].r == x:
            self.tr[u].v += v
            return
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        if x <= mid:
            self.modify(u << 1, x, v)
        else:
            self.modify(u << 1 | 1, x, v)

```

```

        self.pushup(u)

```

```

    def query(self, u, l, r):
        if self.tr[u].l >= l and self.tr[u].r <= r:
            return self.tr[u].v
        mid = (self.tr[u].l + self.tr[u].r) >> 1
        v = 0
        if l <= mid:
            v += self.query(u << 1, l, r)
        if r > mid:

```

```

            v += self.query(u << 1 | 1, l, r)
        return v

```

```

    def pushup(self, u):
        self.tr[u].v = self.tr[u << 1].v + self.tr[u << 1 | 1].v

```

```

class Solution:
    def countSmaller(self, nums: List[int]) -> List[int]:
        s = sorted(set(nums))

```

```

m = {v: i for i, v in enumerate(s, 1)}
tree = SegmentTree(len(s))
ans = []
for v in nums[::-1]:
    x = m[v]
    ans.append(tree.query(1, 1, x - 1))
    tree.modify(1, x, 1)
return ans[::-1]

```

```

class Solution:
    def countSmaller(self, nums: list[int]) -> list[int]:
        self.right_smaller_counts = [0] * len(nums)

        nums_indices = [(num, idx) for idx, num in enumerate(nums)]
        self.merge_sort(nums_indices)

        return self.right_smaller_counts

```

```

def combine_arrays(

```

```

    self,
    left_nums_indices: list[tuple[int, int]],
    right_nums_indices: list[tuple[int, int]],
) -> list[tuple[int, int]]:
    merged_nums_indices: list[tuple[int, int]] = []
    left_idx, right_idx = 0, 0

    while left_idx < len(left_nums_indices) and right_idx < len(right_nums
_indices):
        if left_nums_indices[left_idx][0] <= right_nums_indices[right_idx]
[0]:
            # Iterated left side element finalizes its right smaller count.

```

```

            left_num_idx = left_nums_indices[left_idx][1]
            self.right_smaller_counts[left_num_idx] += right_idx

            merged_nums_indices.append(left_nums_indices[left_idx])
            left_idx += 1
            continue

        merged_nums_indices.append(right_nums_indices[right_idx])
        right_idx += 1

```

```

while left_idx < len(left_nums_indices):
    # Iterated left side element finalizes its right smaller count.
    left_num_idx = left_nums_indices[left_idx][1]
    self.right_smaller_counts[left_num_idx] += len(right_nums_indices)

    merged_nums_indices.append(left_nums_indices[left_idx])
    left_idx += 1

while right_idx < len(right_nums_indices):
    merged_nums_indices.append(right_nums_indices[right_idx])

    right_idx += 1

return merged_nums_indices

def merge_sort(self, nums_indices: list[tuple[int, int]]) -> list[tuple[int, int]]:
    if len(nums_indices) == 1:
        return nums_indices # Single element.

    split_idx = len(nums_indices) // 2

    left_nums_indices = self.merge_sort(nums_indices[:split_idx])
    right_nums_indices = self.merge_sort(nums_indices[split_idx:])

    return self.combine_arrays(left_nums_indices, right_nums_indices)

```

• SimplyLeet

```

# Python solution using merge sort approach

def countSmaller(nums):
    n = len(nums)
    # Initialize counts array for results
    counts = [0] * n
    # Pair each element with its original index
    enum = [(num, i) for i, num in enumerate(nums)]

    # Recursive function for merge sort with counting

```

```
def mergeSort(arr):
    if len(arr) <= 1:
        return arr
    # Split the array into two halves
    mid = len(arr) // 2
    left = mergeSort(arr[:mid])
    right = mergeSort(arr[mid:])
    merged = []
    l = r = 0
    # r_count counts number of elements taken from right array

    r_count = 0
    while l < len(left) and r < len(right):
        # If left element is less or equal, it means current right elements
        # are smaller
        if left[l][0] <= right[r][0]:
            # update count for original index
            counts[left[l][1]] += r_count
            merged.append(left[l])
            l += 1
        else:
            # right element is smaller, so increment r_count and add it to
            # merged
            merged.append(right[r])
            r += 1
            r_count += 1
    # Append remaining elements from left reduce count by accumulated right
    # elements
    while l < len(left):
        counts[left[l][1]] += r_count
        merged.append(left[l])
        l += 1
    # Append any remaining elements from right side
    while r < len(right):
```



```

        merged.append(right[r])
        r += 1
    return merged

# Start the merge sort process
mergeSort(enum)
return counts

# Example test
print(countSmaller([5, 2, 6, 1])) # Expected output: [2, 1, 1, 0]
```

• LC.ca

```

'''
bisect: maintaining a list in sorted order without having to sort the list
after each insertion.
https://docs.python.org/3/library/bisect.html

bisect.bisect_left()
Locate the insertion point for x in a to maintain sorted order.

bisect.bisect_right() or bisect.bisect()
Similar to bisect_left(), but returns an insertion point which comes after (to
the right of) any existing entries of x in a.

bisect.insort_left(a, x, lo=0, hi=len(a), *, key=None)
Insert x in a in sorted order.
Keep in mind that the O(log n) search is dominated by the slow O(n) insertion
step.

bisect.insort_right(a, x, lo=0, hi=len(a), *, key=None)
bisect.insort(a, x, lo=0, hi=len(a), *, key=None)
Similar to insort_left(), but inserting x in a after any existing entries of x.
```

```
>>> import bisect
>>> bisect.bisect_left([1,2,3], 2)
1
>>> bisect.bisect_right([1,2,3], 2)
2

>>> a = [1, 1, 1, 2, 3]
>>> bisect.insort_left(a, 1.0)
>>> a
```

```
[1.0, 1, 1, 1, 2, 3]
```

```
>>> a = [1, 1, 1, 2, 3]
>>> bisect.insort_right(a, 1.0)
>>> a
[1, 1, 1, 1.0, 2, 3]

>>> a = [1, 1, 1, 2, 3]
>>> bisect.insort(a, 1.0)
>>> a
```

```
[1, 1, 1, 1.0, 2, 3]
'''
```

```
import bisect

class Solution(object):
    def countSmaller(self, nums):
        """
        :type nums: List[int]
        :rtype: List[int]

        """
        ans = []
        bst = []
        for num in reversed(nums):
            idx = bisect.bisect_left(bst, num)
            ans.append(idx)
            bisect.insort(bst, num)
        return ans[::-1]
```

```
#####
```

```

class BinaryIndexedTree:
    def __init__(self, n):
        self.n = n
        self.c = [0] * (n + 1)

    @staticmethod
    def lowbit(x):
        return x & -x

    def update(self, x, delta):
        while x <= self.n:
            self.c[x] += delta
            x += BinaryIndexedTree.lowbit(x)

    def query(self, x):
        s = 0
        while x > 0:
            s += self.c[x]
            x -= BinaryIndexedTree.lowbit(x)

        return s

class Solution:
    def countSmaller(self, nums: List[int]) -> List[int]:
        alls = sorted(set(nums))
        m = {v: i for i, v in enumerate(alls, 1)}
        tree = BinaryIndexedTree(len(m))
        ans = []
        for v in nums[::-1]:
            x = m[v]
            tree.update(x, 1)
            ans.append(tree.query(x - 1))
        return ans[::-1]

```

◆ Quick Review

Key Insights

- The brute force solution takes $O(n^2)$ time, which is too slow for large input arrays.
- An efficient approach leverages the concept of Merge Sort to both sort the array and count the number of smaller elements by tracking the number of elements that "jump over" during the merge step.
- Other efficient methods include Binary Indexed Trees (Fenwick Tree) or Balanced Binary Search Trees.

Space & Time

Time Complexity: $O(n \log n)$ on average due to the merge sort based approach.

Space Complexity: $O(n)$ for auxiliary arrays used during sorting and counting.

Solution

We use a modified Merge Sort to sort the array while counting the number of smaller elements to the right. We keep track of the original indices by pairing each element with its index. During the merge process, when an element from the left subarray is placed into the temporary array, we add the count of elements from the right subarray that have

already been placed (since these represent smaller elements that were to its right). This efficient divide and conquer strategy guarantees an overall $O(n \log n)$ time complexity. Care must be taken to update the counts based on indices and during the merge step.

Binary Search

□ #410 Split Array Largest Sum

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Dynamic Programming · Greedy · Prefix Sum

Lists: Denny Zhang

Problem

Given an integer array `nums` and an integer `k`, split `nums` into `k` non-empty subarrays such that the largest sum of any subarray is minimized.

Return the minimized largest sum of the split. A subarray is a contiguous part of the array.

Example 1:

Input: `nums = [7,2,5,10,8]`, `k = 2`

Output: 18

Explanation: There are four ways to split `nums` into two subarrays.

The best way is to split it into `[7,2,5]` and `[10,8]`, where the largest sum among the two subarrays is only 18.

Example 2:

Input: nums = [1,2,3,4,5], k = 2

Output: 9

Explanation: There are four ways to split nums into two subarrays.

The best way is to split it into [1,2,3] and [4,5], where the largest sum among the two subarrays is only 9.

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $0 \leq \text{nums}[i] \leq 10^6$
- $1 \leq k \leq \min(50, \text{nums.length})$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Split nums into k non-empty contiguous subarrays to minimise the maximum subarray sum.

Must be in order – no reordering allowed. Right?

Same 'binary search on answer' pattern as Capacity to Ship Packages (#1011)."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Try every split of nums into k contiguous groups (DP over partition points).

For each split, max subarray sum = largest group sum; minimize that over splits.

State space explodes without binary search on the answer.

Time: exponential / $O(n^k)$ naive partitions. Space: $O(n)$ recursion.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"Binary search on the answer (the max sum).

lo = max(nums) (at least one element), hi = sum(nums) (one subarray).

Feasibility check: can we split into k subarrays with each sum $\leq$ mid?

Greedy: accumulate greedily, start new subarray when exceeding mid. Count splits.

Time: $O(n \log(\text{sum}))$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

```
class _410_SplitArrayLargestSum:
    def splitArray(self, nums: List[int], k: int) -> int:
        def can_split(max_sum: int) -> bool:
            subarrays = 1
            current = 0
```



```

    for num in nums:
        if current + num > max_sum:
            subarrays += 1
            current = 0
        current += num
    return subarrays <= k
lo, hi = max(nums), sum(nums)
while lo < hi:
    mid = (lo + hi) // 2
    if can_split(mid):
        hi = mid # valid - try smaller
    else:
        lo = mid + 1 # not valid - need bigger
return lo

```

PHASE 5 — TEST & DEBUG

[7,2,5,10,8], k=2: lo=10, hi=32. mid=21→can_split? [7,2,5] and [10,8]=18≤21 ✓ → hi=21.

... converges to 18. can_split(18)=[7,2,5][10,8]→2 splits≤k=2 ✓.

can_split(17)→[7,2,5][10,8]→10+8=18>17→need 3>k ✗. → 18 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log(\text{sum}))$. Space $O(1)$. Same template as #1011 Capacity to Ship – name this.

lo/hi bounds: must be at least max(nums), at most sum(nums).

Given more time I'd ask: DP approach? $dp[j][i] = \min$ largest sum splitting first i elements into j parts.

$O(kn^2)$ – worse than binary search for large inputs."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def splitArray(self, nums: list[int], k: int) -> int:
        prefix = list(itertools.accumulate(nums, initial=0))

        @functools.lru_cache(None)
        def dp(i: int, k: int) -> int:
            """
            Returns the minimum of the maximum sum to split the first i numbers into
            k groups.
            """

```

```

    if k == 1:
        return prefix[i]
    return min(max(dp[j, k - 1], prefix[i] - prefix[j])
               for j in range(k - 1, i))

return dp(len(nums), k)

```

```

class Solution:
    def splitArray(self, nums: list[int], k: int) -> int:
        n = len(nums)
        # dp[i][k] := the minimum of the maximum sum to split the first i numbers
        # into k groups
        dp = [[math.inf] * (k + 1) for _ in range(n + 1)]
        prefix = list(itertools.accumulate(nums, initial=0))

        for i in range(1, n + 1):
            dp[i][1] = prefix[i]

            for l in range(2, k + 1):
                for i in range(l, n + 1):
                    for j in range(l - 1, i):
                        dp[i][l] = min(dp[i][l], max(dp[j][l - 1], prefix[i] - prefix[j]))

        return dp[n][k]

```

```

class Solution:
    def splitArray(self, nums: list[int], k: int) -> int:
        l = max(nums)
        r = sum(nums) + 1

        def numGroups(maxSumInGroup: int) -> int:
            groupCount = 1
            sumInGroup = 0

            for num in nums:

```

```

        if sumInGroup + num <= maxSumInGroup:
            sumInGroup += num
        else:
            groupCount += 1
            sumInGroup = num

    return groupCount

while l < r:
    m = (l + r) // 2

```

```

    if numGroups(m) > k:
        l = m + 1
    else:
        r = m

return l

```

• LeetDoocs

```

class Solution:
    def splitArray(self, nums: List[int], k: int) -> int:
        def check(mx):
            s, cnt = inf, 0
            for x in nums:
                s += x
                if s > mx:
                    s = x
                    cnt += 1
            return cnt <= k

        left, right = max(nums), sum(nums)
        return left + bisect_left(range(left, right + 1), True, key=check)

```

• SimplyLeet

```
# Python solution for Split Array Largest Sum

def splitArray(nums, k):
    # Helper function to determine if we can split array into <= k subarrays
    with max subarray sum <= max_sum
    def canSplit(max_sum):
        subarray_count = 1 # start with one subarray
        current_sum = 0
        for num in nums:
            if current_sum + num > max_sum:
                subarray_count += 1 # start a new subarray

                current_sum = num # reset the sum with the current number
            else:
                current_sum += num # add num to current subarray
        return subarray_count <= k # valid if we used k or fewer subarrays

    low, high = max(nums), sum(nums) # boundaries for binary search
    while low < high:
        mid = (low + high) // 2 # candidate maximum subarray sum
        if canSplit(mid):
            high = mid # candidate is valid, try to find a smaller one

        else:
            low = mid + 1 # candidate too small, increase it
    return low

# Example usage:
nums = [7,2,5,10,8]
k = 2
print(splitArray(nums, k)) # Expected output: 18
```

• **LC.ca**

```
class Solution:
    def splitArray(self, nums: List[int], k: int) -> int:
        def check(mx):
            s, cnt = inf, 0
            for x in nums:
                s += x
                if s > mx:
                    s = x
                    cnt += 1
            return cnt <= k

        left, right = max(nums), sum(nums)
        return left + bisect_left(range(left, right + 1), True, key=check)
```

◆ Quick Review

Key Insights

- Use binary search over the answer: the candidate value is the maximum subarray sum.
- The lower bound of the search is $\max(\text{nums})$ and the upper bound is $\text{sum}(\text{nums})$.
- A greedy approach is used to check if a candidate maximum sum allows splitting nums into at most k subarrays.
- Adjust the binary search bounds based on the feasibility check.

Space & Time

Time Complexity: $O(n * \log(\text{sum}(\text{nums})))$ – n elements are processed for each binary search

iteration.

Space Complexity: $O(1)$ – Constant extra space is used.

Solution

The solution employs binary search to determine the smallest maximum subarray sum possible. We set the initial lower bound to the maximum element and the upper bound to the total sum of the array. For each candidate value (mid), we check if the array can be split into k or fewer subarrays without any subarray sum exceeding mid using a greedy approach. If possible, we try to lower the candidate; if not, we increase the candidate. This continues until the optimal solution is found.

Binary Search

□ #668 Kth Smallest Number in Multiplication Table

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Binary Search
Lists: Denny Zhang

Problem

Nearly everyone has used the Multiplication Table. The multiplication table of size $m \times n$ is an integer matrix mat where $mat[i][j] == i * j$ (1-indexed).

Given three integers m , n , and k , return the k th smallest element in the $m \times n$ multiplication table.

Example 1:

1	2	3
2	4	6
3	6	9

1	2	2	3	3	4	6	6	9
---	---	---	---	---	---	---	---	---

Input: $m = 3$, $n = 3$, $k = 5$
Output: 3
Explanation: The 5th smallest number is 3.

Example 2:

1	2	3
2	4	6

1	2	2	3	4	6
---	---	---	---	---	---

Input: $m = 2, n = 3, k = 6$

Output: 6

Explanation: The 6th smallest number is 6.

Constraints:

- $1 \leq m, n \leq 3 \times 10^4$
- $1 \leq k \leq m \times n$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"m×n multiplication table: $\text{mat}[i][j]=i*j$ (1-indexed). Find kth smallest element. Right?

Can't materialise the table (m,n up to $3 \times 10^4 \rightarrow$ up to 9×10^8 cells)."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Collect all m×n products into a list, sort, return kth element.

Correct but materializes every product – $O(mn \log(mn))$ time and $O(mn)$ space.

Time: $O(m \times n \times \log(m \times n))$. Space: $O(m \times n)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (binary search on value)

"Binary search on the answer value v.

Count elements $\leq v$: for row i, elements are $i, 2i, 3i, \dots, n \times i$.

Count of these $\leq v$ is $\min(v // i, n)$. Sum over all rows $i=1..m$.

Binary search for smallest v where count $\geq k$.

lo=1, hi=m×n. Time: $O(m \log(m \times n))$."

PHASE 4 — CLEAN CODE

```
class _668_KthSmallestMultTable:
    def findKthNumber(self, m: int, n: int, k: int) -> int:
        def count_le(v: int) -> int:
            return sum(min(v // i, n) for i in range(1, m + 1))
        lo, hi = 1, m * n
        while lo < hi:
            mid = (lo + hi) // 2
            if count_le(mid) >= k:
                hi = mid
            else:
                lo = mid + 1
```

return lo

PHASE 5 — TEST & DEBUG

```
# m=3,n=3,k=5. Table: [1,2,3,2,4,6,3,6,9] sorted=[1,2,2,3,3,4,6,6,9]. 5th=3.
# count_le(3)=count<=3? i=1:min(3,3)=3, i=2:min(1,3)=1, i=3:min(1,3)=1. sum=5<=5 →
hi=3.
# ... lo converges to 3 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(m log(m*n)). Space O(1). The count formula is the key – O(m) per binary
search step.
# Note: the answer might not be a 'real' table entry – binary search finds the
smallest v
# with count >= k, which IS always a table entry because count jumps discretely at
table values.
# Given more time I'd ask: kth largest? Binary search on m*n – k + 1 smallest
instead."
```

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def findKthNumber(self, m: int, n: int, k: int) -> int:
        left, right = 1, m * n
        while left < right:
            mid = (left + right) >> 1
            cnt = 0
            for i in range(1, m + 1):
                cnt += min(mid // i, n)
            if cnt >= k:
                right = mid
        else:
            left = mid + 1
        return left
```

• SimplyLeet

```
# Python solution using binary search
```

```
class Solution:
    def findKthNumber(self, m: int, n: int, k: int) -> int:
        def count_less_equal(x: int) -> int:
            # Count numbers in the m x n multiplication table <= x
            count = 0
            for i in range(1, m + 1):
                # For each row, count of numbers <= x is minimum of n or x // i
                count += min(x // i, n)

            return count

        # Binary search range from 1 to m*n
        low, high = 1, m * n
        while low < high:
            mid = (low + high) // 2
            if count_less_equal(mid) >= k:
                high = mid # mid might be the answer
            else:
                low = mid + 1 # answer must be greater than mid

        return low
```

• LC.ca

```
class Solution:
    def findKthNumber(self, m: int, n: int, k: int) -> int:
        left, right = 1, m * n
        while left < right:
            mid = (left + right) >> 1
            cnt = 0
            for i in range(1, m + 1):
                cnt += min(mid // i, n)
            if cnt >= k:
                right = mid

            else:
                left = mid + 1
        return left
```

◆ Quick Review

Key Insights

- The table elements are not fully sorted row-wise or column-wise globally, but there is a pattern that can be exploited.
- For any candidate number x , we can count how many numbers in the table are less than or equal to x by summing over rows: for row i , there are $\min(x // i, n)$ numbers.
- Binary search can be used over the range $[1, m*n]$ to find the smallest x such that the count is at least k .
- The counting function is the key to connecting the candidate value with the order statistic.

Space & Time

Time Complexity: $O(m * \log(m*n))$

Space Complexity: $O(1)$

Solution

We use binary search over the possible values in the multiplication table, which range from 1 to $m*n$. For each midpoint x in the binary search, we compute the number of elements in

the table that are less than or equal to x by iterating over each row and summing up $\min(x // i, n)$. This count tells us whether the k th smallest element is to the left or right of x . We then adjust the binary search boundaries accordingly. The approach leverages the multiplicative structure of the table and the monotonicity of the counting function.

Binary Search

□ #710 Random Pick with Blacklist

HAR

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Math · Binary Search ·
Sorting · Randomized
Lists: Denny Zhang

Problem

You are given an integer n and an array of unique integers `blacklist`. Design an algorithm to pick a random integer in the range $[0, n - 1]$ that is not in `blacklist`. Any integer that is in the mentioned range and not in `blacklist` should be equally likely to be returned.

Optimize your algorithm such that it minimizes the number of calls to the built-in random function of your language.

Implement the `Solution` class:

- `Solution(int n, int[] blacklist)` Initializes the object with the integer n and the blacklisted integers `blacklist`.

- **int pick()** Returns a random integer in the range $[0, n - 1]$ and not in blacklist.

Example 1:

Input

```
["Solution", "pick", "pick", "pick", "pick", "pick", "pick", "pick"]
[[7, [2, 3, 5]], [], [], [], [], [], [], []]
```

Output

```
[null, 0, 4, 1, 6, 1, 0, 4]
```

Explanation

```
Solution solution = new Solution(7, [2, 3, 5]);
```

Constraints:

- $1 \leq n \leq 109$
- $0 \leq \text{blacklist.length} \leq \min(105, n - 1)$
- $0 \leq \text{blacklist}[i] < n$
- All the values of blacklist are unique.
- At most $2 * 10^4$ calls will be made to pick.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Pick uniformly at random from $[0, n-1]$ excluding blacklist. One call to random per pick. Right?

n can be 10^9 – can't enumerate all valid numbers."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Store all non-blacklisted ids in a list; pick random index.

If blacklist is small, filter on each pick – $O(n)$ per draw.

Time: $O(n)$ per pick naive. Space: $O(n)$ whitelist.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (virtual mapping)

```
# "Let sz = n - len(blacklist) = count of valid numbers.
# Map [0, sz) as the 'virtual' range. Numbers in [0, sz) that ARE blacklisted
# get remapped to non-blacklisted numbers in [sz, n).
# pick(): random in [0, sz) → if in remap → use remap[r], else use r directly.
# Build remap in __init__: for each blacklisted number in [0, sz), map it to a
# non-blacklisted
# number in [sz, n). O(|blacklist|) init, O(1) per pick."
```

PHASE 4 — CLEAN CODE

```
class _710_RandomPickBlacklist:
    def __init__(self, n: int, blacklist: List[int]):
        bl_set = set(blacklist)
        self.sz = n - len(blacklist) # size of valid virtual range
        self.remap: dict = {}
        tail = n - 1
        for b in blacklist:
            if b < self.sz: # this blacklisted number is in [0, sz)
                while tail in bl_set: # find next valid number from the tail
                    tail -= 1
                self.remap[b] = tail
                tail -= 1
        def pick(self) -> int:
            r = random.randint(0, self.sz - 1)
            return self.remap.get(r, r)
```

PHASE 5 — TEST & DEBUG

```
# n=7, blacklist=[2,3,5]. sz=4. Valid: [0,1,4,6].
# Blacklisted in [0,4): 2,3. Map 2→6, 3→4 (5 is blacklisted, skip → 4).
# remap={2:6, 3:4}. pick(0)→0, pick(1)→1, pick(2)→6, pick(3)→4. All valid ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Init O(|B|). Pick O(1). Space O(|B|) for remap.
# The key insight: divide [0,n) into 'virtual valid' [0,sz) and 'tail' [sz,n).
# Blacklisted numbers in [0,sz) redirect to valid numbers in the tail.
# Given more time I'd ask: what if the blacklist updates dynamically?
# Rebuild the remap on each update — O(|B|) per update, same O(1) pick."
```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def __init__(self, n: int, blacklist: list[int]):
        self.validRange = n - len(blacklist)
        self.dict = {}

        maxAvailable = n - 1

        for b in blacklist:
            self.dict[b] = -1

        for b in blacklist:
            if b < self.validRange:
                # Find the slot that haven't been used.
                while maxAvailable in self.dict:
                    maxAvailable -= 1
                self.dict[b] = maxAvailable
                maxAvailable -= 1

    def pick(self) -> int:
        value = random.randint(0, self.validRange - 1)

        if value in self.dict:
            return self.dict[value]

        return value

```

• LeetDoocs

```

class Solution:
    def __init__(self, n: int, blacklist: List[int]):
        self.k = n - len(blacklist)
        self.d = {}
        i = self.k
        black = set(blacklist)
        for b in blacklist:
            if b < self.k:
                while i in black:
                    i += 1

```

```

        self.d[b] = i
        i += 1

    def pick(self) -> int:
        x = randrange(self.k)
        return self.d.get(x, x)

```

Your Solution object will be instantiated and called as such:
 # obj = Solution(n, blacklist)

```
# param_1 = obj.pick()
```

• SimplyLeet

```

import random

class Solution:
    def __init__(self, n: int, blacklist: list):
        # Total numbers valid for picking
        self.W = n - len(blacklist)
        # Mapping for blacklisted numbers in the valid range
        self.mapping = {}
        # Set to hold blacklisted numbers that are >= W
        blacklist_set = set(blacklist)

        # Pointer to find available numbers in [W, n-1]
        available = self.W
        for b in blacklist:
            if b < self.W:
                # Find next available non-blacklisted number in the tail
                while available in blacklist_set:
                    available += 1
                self.mapping[b] = available
                available += 1

```

```
def pick(self) -> int:
    # Generate a random number in the range [0, self.W-1]
    idx = random.randint(0, self.W - 1)
    # If the number is mapped, return the mapping; else it is valid.
    return self.mapping.get(idx, idx)
```

• LC.ca

```
class Solution:
    def __init__(self, n: int, blacklist: List[int]):
        self.k = n - len(blacklist)
        self.d = {}
        i = self.k
        black = set(blacklist)
        for b in blacklist:
            if b < self.k:
                while i in black:
                    i += 1

                self.d[b] = i
                i += 1

    def pick(self) -> int:
        x = randrange(self.k)
        return self.d.get(x, x)

# Your Solution object will be instantiated and called as such:
# obj = Solution(n, blacklist)

# param_1 = obj.pick()
```

◆ Quick Review

Key Insights

- The valid output range has a total count of available numbers $W = n - \text{len}(\text{blacklist})$.

- Instead of repeatedly generating random numbers and checking against a set, we can transform the range into a contiguous block $[0, W - 1]$ and map any blacklist number within that block to a valid number in the upper range.
- Mapping is precomputed: for any blacklisted number x in $[0, W-1]$, assign it a valid number from $[W, n-1]$ that is not blacklisted.
- This approach ensures $O(1)$ pick time after an $O(b)$ precomputation where $b = \text{length of blacklist}$.

Space & Time

Time Complexity: constructor – $O(b)$ for initializing data structures, pick() – $O(1)$ per call.

Space Complexity: $O(b)$ for storing the blacklist set and mapping.

Solution

We compute the count of valid numbers, $W = n - \text{len}(\text{blacklist})$. For numbers in the blacklist that are less than W , we need to map them to a valid number from the range $[W, n-1]$. To do this, we first store all blacklist numbers in the

higher range $[W, n-1]$ in a set for quick lookup. Then for each blacklisted value in $[0, W-1]$, we iterate from W upwards to find a number that is not blacklisted and assign it in a mapping. During the `pick()` function, we generate a random integer r in $[0, W-1]$. If r is in our mapping, we return its corresponding mapped value; otherwise, we return r as it is already a valid number.

Binary Search

□ #774 Minimize Max Distance to Gas Station

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search
Lists: Denny Zhang

Problem

You are given an integer array `stations` that represents the positions of the gas stations on the x -axis. You are also given an integer k . You should add k new gas stations. You can add the stations anywhere on the x -axis, and not necessarily on an integer position.

Let `penalty()` be the maximum distance between adjacent gas stations after adding the k new stations.

Return the smallest possible value of `penalty()`. Answers within 10^{-6} of the actual answer will be accepted.

Example 1:

Input: stations = [1,2,3,4,5,6,7,8,9,10], k = 9
Output: 0.50000

Example 2:

Input: stations = [23,24,36,39,46,56,57,65,84,98], k = 1
Output: 14.00000

Constraints:

- $10 \leq \text{stations.length} \leq 2000$
- $0 \leq \text{stations}[i] \leq 108$
- stations is sorted in a strictly increasing order.
- $1 \leq k \leq 106$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Add exactly k stations anywhere (real-valued positions) to minimise the maximum gap.

Return the answer within $1e-6$ precision. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Binary search on the maximum distance between adjacent stations.

For candidate d, check if we can add k stations so all gaps $\leq d$ (greedy fill).

Brute: try every real-valued spacing without binary search – continuous search space.

Time: $O(n * \text{precision})$ naive scan. Space: $O(1)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (binary search on floating-point answer)

"Binary search on the max gap d. Feasibility: for each existing gap g, we need $\text{ceil}(g/d)-1$ extra

stations to keep all sub-gaps $\leq d$. Count total extras; if $\leq k$, d is achievable.

Precision: run until $\text{hi-lo} < 1e-7$."

PHASE 4 — CLEAN CODE

```
class _774_MinMaxDistanceGasStation:
    def minmaxGasDist(self, stations: List[int], k: int) -> float:
        gaps = [stations[i+1] - stations[i] for i in range(len(stations)-1)]
        def can_achieve(d: float) -> bool:
            return sum(int(g / d) for g in gaps) <= k
        lo, hi = 0.0, max(gaps)
        for _ in range(100): # 100 iterations -> precision < hi/2^100
            mid = (lo + hi) / 2
            if can_achieve(mid):
                hi = mid
            else:
                lo = mid
        return hi
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n * 100) = O(n)$. Space $O(n)$ for gaps. Floating-point binary search with fixed iterations.

Note: $\text{int}(g/d)$ gives the number of stations to insert (not $\text{ceil}-1$ – same thing via floor division).

Given more time I'd ask: what if stations must be at integer positions?

Binary search on integers, same feasibility check but with math.ceil ."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minmaxGasDist(self, stations: list[int], k: int) -> float:
        ERR = 1e-6
        l = 0
        r = stations[-1] - stations[0]

        def possible(k: int, m: float) -> bool:
            """
            Returns True if can use <= k gas stations to ensure that each adjacent
            distance between gas stations <= m.
            """
```

```

        """
        for a, b in zip(stations, stations[1:]):
            diff = b - a
            if diff > m:
                k -= math.ceil(diff / m) - 1
                if k < 0:
                    return False
        return True

    while r - l > ERR:

        m = (l + r) / 2
        if possible(k, m):
            r = m
        else:
            l = m

    return l

```

• LeetDoocs

```

class Solution:
    def minmaxGasDist(self, stations: List[int], k: int) -> float:
        def check(x):
            return sum(int((b - a) / x) for a, b in pairwise(stations)) <= k

        left, right = 0, 1e8
        while right - left > 1e-6:
            mid = (left + right) / 2
            if check(mid):
                right = mid
            else:
                left = mid
        return left

```

• SimplyLeet

```

import math

def minmaxGasDist(stations, k):
    # Initialize the search range between 0 and the maximum gap between
    existing stations
    low = 0.0
    high = max(stations[i+1] - stations[i] for i in range(len(stations)-1))

    # Use binary search with a tolerance of 1e-6
    while high - low > 1e-6:
        mid = (low + high) / 2.0

        used = 0
        # Calculate additional stations needed for each gap
        for i in range(1, len(stations)):
            gap = stations[i] - stations[i-1]
            used += math.ceil(gap / mid) - 1

        # If the used stations are less than or equal to k, try a smaller
        distance
        if used <= k:
            high = mid
        else:
            low = mid

    return high

# Example usage:
if __name__ == '__main__':
    print("{:.5f}".format(minmaxGasDist([1,2,3,4,5,6,7,8,9,10], 9)))
    print("{:.5f}".format(minmaxGasDist([23,24,36,39,46,56,57,65,84,98], 1)))

```

• LC.ca

```

class Solution:
    def minmaxGasDist(self, stations: List[int], k: int) -> float:
        def check(x):
            return sum(int((b - a) / x) for a, b in pairwise(stations)) <= k

        left, right = 0, 1e8
        while right - left > 1e-6:
            mid = (left + right) / 2
            if check(mid):
                right = mid

```

```
        else:
            left = mid
    return left
```

◆ Quick Review

Key Insights

- The optimal "penalty" (i.e., the maximum distance between adjacent stations after placement) can be found using binary search over a continuous distance range.
- For each candidate distance (mid), simulate the number of gas stations required by iterating over each adjacent station gap and calculating:
 - $\text{stations_required} = \text{ceil}(\text{gap} / \text{mid}) - 1$
 - If the total number of required additional stations is less than or equal to k , then mid is a feasible candidate; otherwise, it is too small.
 - Continue the binary search until the gap between low and high is within the desired tolerance ($1e-6$).

Space & Time

Time Complexity: $O(n * \log(\text{max_gap}/\text{precision}))$, where n is the

number of stations.

Space Complexity: $O(1)$ (ignoring input storage)

Solution

We use binary search over the possible maximum distances. The algorithm sets $low = 0$ and $high =$ the maximum gap between adjacent stations. For a candidate distance mid , we count how many new stations are required for all gaps by computing $\text{ceil}(\text{gap} / mid) - 1$. If the total required is within k , we try a smaller candidate; otherwise, we increase the candidate distance. This approach efficiently zooms into the minimum possible maximum distance (penalty).

Binary Search

□ #1235 Maximum Profit in Job Scheduling **HAR**

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Dynamic Programming ·
Sorting

Lists: Grind 169

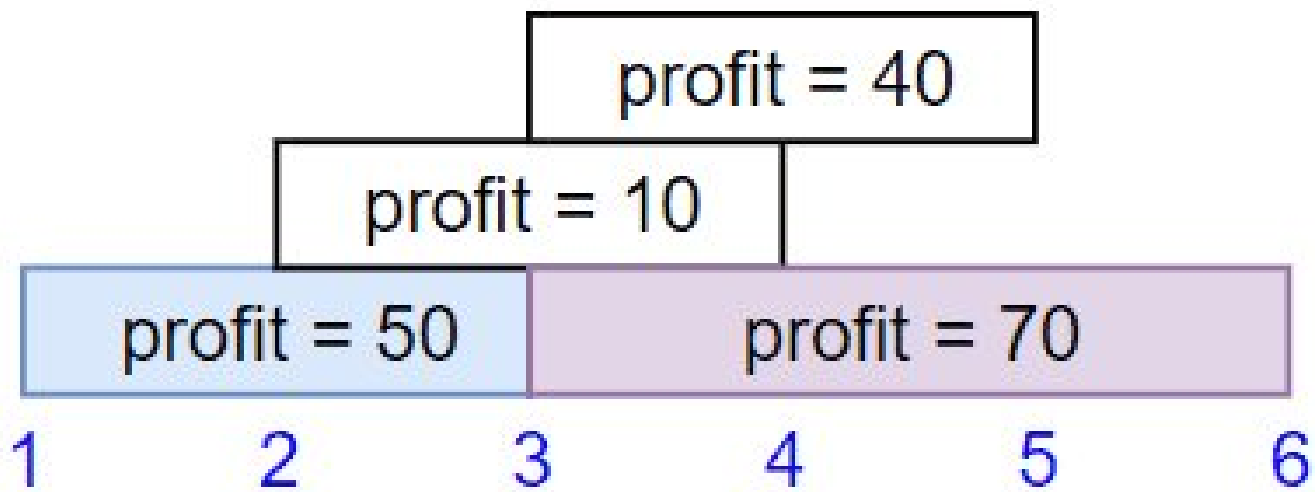
Problem

We have n jobs, where every job is scheduled to be done from $startTime[i]$ to $endTime[i]$, obtaining a profit of $profit[i]$.

You're given the $startTime$, $endTime$ and $profit$ arrays, return the maximum profit you can take such that there are no two jobs in the subset with overlapping time range.

If you choose a job that ends at time X you will be able to start another job that starts at time X .

Example 1:



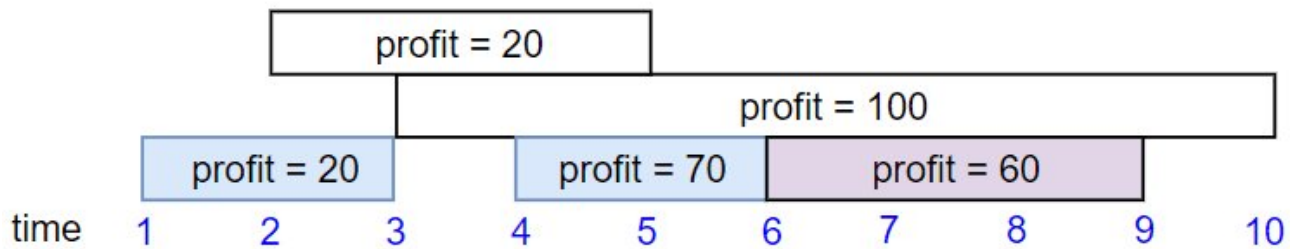
Input: startTime = [1,2,3,3], endTime = [3,4,5,6], profit = [50,10,40,70]

Output: 120

Explanation: The subset chosen is the first and fourth job.

Time range [1-3]+[3-6], we get profit of $120 = 50 + 70$.

Example 2:



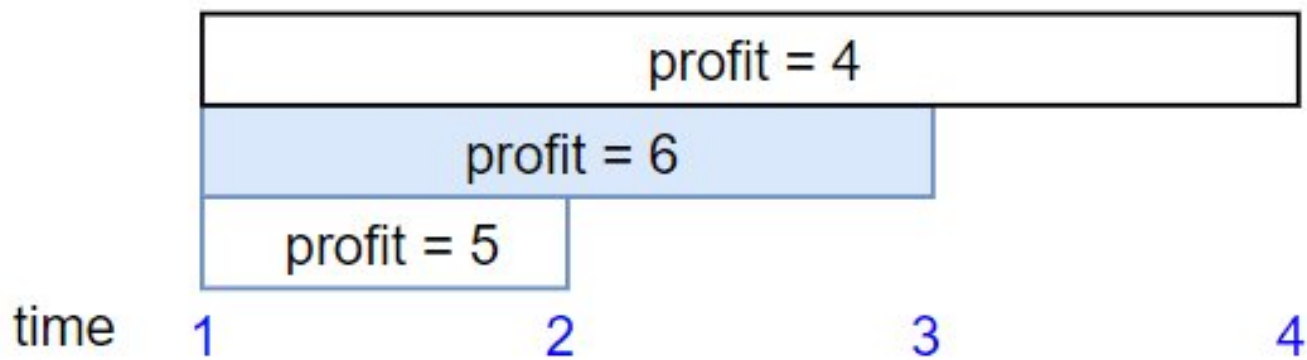
Input: startTime = [1,2,3,4,6], endTime = [3,5,10,6,9], profit = [20,20,100,70,60]

Output: 150

Explanation: The subset chosen is the first, fourth and fifth job.

Profit obtained $150 = 20 + 70 + 60$.

Example 3:



Input: startTime = [1,1,1], endTime = [2,3,4], profit = [5,6,4]
 Output: 6

Constraints:

- $1 \leq \text{startTime.length} == \text{endTime.length} == \text{profit.length} \leq 5 * 10^4$
- $1 \leq \text{startTime}[i] < \text{endTime}[i] \leq 10^9$
- $1 \leq \text{profit}[i] \leq 10^4$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Select non-overlapping jobs to maximise total profit.

Job ending at X and starting at X don't overlap. Return max profit. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Sort jobs by end time. DP over index: $\text{dp}[i]$ = max profit taking job i plus best non-overlapping before i.

For each i, scan all $j < i$ for compatibility - $O(n^2)$ DP.

Time: $O(n^2)$. Space: $O(n)$ DP + sort.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (sort by end time + DP + binary search)

"Sort jobs by end time. $\text{dp}[i]$ = max profit using first i jobs.

For each job i: skip → $\text{dp}[i] = \text{dp}[i-1]$. Take → find latest j where $\text{job}[j].\text{end} \leq \text{job}[i].\text{start}$

```
# using binary search. dp[i] = max(dp[i-1], dp[j] + profit[i]).
# Time: O(n log n)."
```

PHASE 4 — CLEAN CODE

```
class _1235_MaxProfitJobScheduling:
    def jobScheduling(self, startTime: List[int], endTime: List[int], profit:
List[int]) -> int:
        jobs = sorted(zip(endTime, startTime, profit))
        ends = [0] # dp_ends[i] = end time of ith job in dp
        dp = [0] # dp[i] = max profit for first i sorted jobs
        for end, start, p in jobs:
            # Find latest job whose end <= start of current
            j = bisect.bisect_right(ends, start) - 1
            if dp[j] + p > dp[-1]:
                dp.append(dp[j] + p)
                ends.append(end)
            else:
                dp.append(dp[-1])
                ends.append(end)
        return dp[-1]
```

PHASE 5 — TEST & DEBUG

```
# jobs sorted by end: [(3,1,50),(4,2,10),(5,3,40),(6,3,70)].
# job(3,1,50): j=bisect_right([0],1)-1=0. dp[0]+50=50>dp[-1]=0 →
dp=[0,50],ends=[0,3].
# job(4,2,10): bisect_right([0,3],2)-1=0. dp[0]+10=10<dp[-1]=50 →
dp=[0,50,50],ends=[0,3,4].
# job(5,3,40): bisect_right([0,3,4],3)-1=1. dp[1]+40=90>50 → dp=[0,50,50,90].
# job(6,3,70): bisect_right([0,3,4,5],3)-1=1. dp[1]+70=120>90 →
dp=[0,50,50,90,120]. → 120 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n log n). Space O(n). Sort + DP with binary search per step.
# The dp array is monotonically non-decreasing – binary search on ends is valid.
# Given more time I'd ask: what if we want to select at most k jobs?
# Add a dimension to dp – dp[i][k] – O(n²k) with the same binary search for last
compatible job."
```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def jobScheduling(
        self,
        startTime: List[int],
        endTime: List[int],
        profit: List[int],
    ) -> int:
        jobs = sorted([(s, e, p) for s, e, p in zip(startTime, endTime, profit)])

        # Will use binary search to find the first available startTime

        for i in range(len(startTime)):
            startTime[i] = jobs[i][0]

        @functools.lru_cache(None)
        def dp(i: int) -> int:
            """Returns the maximum profit to schedule jobs[i..n)."""
            if i == len(startTime):
                return 0
            j = bisect.bisect_left(startTime, jobs[i][1])
            return max(jobs[i][2] + dp(j), dp(i + 1))

        return dp(0)

```

• LeetDoocs

```

class Solution:
    def jobScheduling(
        self, startTime: List[int], endTime: List[int], profit: List[int]
    ) -> int:
        @cache
        def dfs(i):
            if i >= n:
                return 0
            _, e, p = jobs[i]
            j = bisect_left(jobs, e, lo=i + 1, key=lambda x: x[0])

            return max(dfs(i + 1), p + dfs(j))

        jobs = sorted(zip(startTime, endTime, profit))
        n = len(profit)
        return dfs(0)

```

• SimplyLeet

```
import bisect

def jobScheduling(startTime, endTime, profit):
    # Combine the jobs and sort them by end time.
    jobs = sorted(zip(startTime, endTime, profit), key=lambda x: x[1])

    # dp_end_times stores the end times and dp_profits stores the corresponding
    # maximum profit.
    dp_end_times = [0]
    dp_profits = [0]

    # Process each job in order.
    for s, e, p in jobs:
        # Find the index of the last job that finishes before or at the start
        # of the current job using binary search.
        index = bisect.bisect_right(dp_end_times, s)
        current_profit = dp_profits[index - 1] + p # Profit if we take current
        job.

        # If taking current job increases the overall profit, update the dp
        # arrays.
        if current_profit > dp_profits[-1]:
            dp_end_times.append(e)
            dp_profits.append(current_profit)

    return dp_profits[-1]

# Example usage:
print(jobScheduling([1,2,3,3], [3,4,5,6], [50,10,40,70]))
```

• LC.ca

```
class Solution:
    def jobScheduling(
        self, startTime: List[int], endTime: List[int], profit: List[int]
    ) -> int:
        @cache
        def dfs(i):
            if i >= n:
                return 0
            _, e, p = jobs[i]
            j = bisect_left(jobs, e, lo=i + 1, key=lambda x: x[0])

            return max(dfs(i + 1), p + dfs(j))

        jobs = sorted(zip(startTime, endTime, profit))
        n = len(profit)
        return dfs(0)
```

◆ Quick Review

Key Insights

- Sort jobs based on their end times to simplify finding non-conflicting jobs.
- Use dynamic programming where the state represents the maximum profit until a given point.
- For each job, perform a binary search to find the last job that ends before the current job's start time.
- The recurrence relation is: $dp[i] = \max(dp[i-1], profit[i] + dp[index])$ where $dp[index]$ is from the latest non-conflicting job.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting and binary search for each job.

Space Complexity: $O(n)$ for the dp arrays.

Solution

The solution begins by sorting the jobs by their end times. For each job, a binary search is used to quickly locate the last job that does not conflict with the current job (i.e., whose end time is less than or equal to the current job's start time). Then, using dynamic programming, the maximum profit is updated by choosing whether to include the current job or skip it. Two parallel arrays (or lists) are maintained: one for the end times of the jobs considered so far, and another to store the corresponding maximum profit at those times. This approach efficiently computes the answer while keeping time complexity within acceptable bounds.

Matrix

Round 3 · High · Hard · 1 question

Matrix

□ #1074 Number of Submatrices That Sum to Target

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Matrix · Prefix Sum
Lists: Denny Zhang

Problem

Given a matrix and a target, return the number of non-empty submatrices that sum to target. A submatrix $x1, y1, x2, y2$ is the set of all cells $matrix[x][y]$ with $x1 \leq x \leq x2$ and $y1 \leq y \leq y2$.

Two submatrices $(x1, y1, x2, y2)$ and $(x1', y1', x2', y2')$ are different if they have some coordinate that is different: for example, if $x1 \neq x1'$.

Example 1:

0	1	0
1	1	1
0	1	0

Input: matrix = `[[0,1,0],[1,1,1],[0,1,0]]`, target = 0

Output: 4

Explanation: The four 1x1 submatrices that only contain 0.

Example 2:

Input: matrix = `[[1,-1],[-1,1]]`, target = 0

Output: 5

Explanation: The two 1x2 submatrices, plus the two 2x1 submatrices, plus the 2x2 submatrix.

Example 3:

Input: matrix = `[[904]]`, target = 0

Output: 0

Constraints:

- $1 \leq \text{matrix.length} \leq 100$
- $1 \leq \text{matrix}[0].\text{length} \leq 100$
- $-1000 \leq \text{matrix}[i][j] \leq 1000$
- $-10^8 \leq \text{target} \leq 10^8$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Count all submatrices (r1,c1,r2,c2) whose elements sum to target. Right?
# Extend #560 Subarray Sum Equals K to 2D."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Compute 2D prefix sums, then enumerate all submatrix top-left and bottom-right
corners.
# For each submatrix, O(1) range sum check against target.
# Four nested loops over corners - O(n^2 * m^2) submatrices.
# Time: O(n^2 * m^2). Space: O(n * m) prefix grid.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "Fix top row r1 and bottom row r2. Compress columns into 1D prefix sums between
r1..r2.
# Then apply #560 (subarray sum = target) on the 1D array.
# Time: O(m^2 * n). Space: O(n)."
```

PHASE 4 — CLEAN CODE

```
class _1074_SubmatricesSumTarget:
    def numSubmatrixSumTarget(self, matrix: List[List[int]], target: int) -> int:
        m, n = len(matrix), len(matrix[0])
        count = 0
        for r1 in range(m):
            col_sum = [0] * n
            for r2 in range(r1, m):
                for c in range(n):
                    col_sum[c] += matrix[r2][c]
                # Apply #560 on col_sum
                prefix = Counter({0: 1})
                running = 0
                for s in col_sum:
```

```

        running += s
        count += prefix[running - target]
        prefix[running] += 1
    return count

```

PHASE 5 — TEST & DEBUG

```

# [[0,1,0],[1,1,1],[0,1,0]], target=0:
# Fix r1=0,r2=0: col_sum=[0,1,0]. Subarrays summing to 0: [0] at c=0, [0] at c=2.
count=2.
# Other rows contribute 2 more. → 4 ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(m^2 * n)$ . Space  $O(n)$ . This is literally #560 applied inside a 2D row-fixing
loop.
# Name that connection – it shows pattern transfer.
# Given more time I'd ask: count submatrices summing to at most target?
# Use #862 (Shortest Subarray with Sum at Least K) approach – monotone deque instead
of hash map."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def numSubmatrixSumTarget(self, matrix: list[list[int]], target: int) -> int
    :
        m = len(matrix)
        n = len(matrix[0])
        ans = 0

        # Transfer each row in the matrix to the prefix sum.
        for row in matrix:
            for i in range(1, n):
                row[i] += row[i - 1]

        for baseCol in range(n):
            for j in range(baseCol, n):
                prefixCount = collections.Counter({0: 1})
                summ = 0
                for i in range(m):
                    if baseCol > 0:
                        summ -= matrix[i][baseCol - 1]
                    summ += matrix[i][j]
                    ans += prefixCount[summ - target]

```

```
prefixCount[summ] += 1
```

```
return ans
```

• LeetDoocs

```
class Solution:
    def numSubmatrixSumTarget(self, matrix: List[List[int]], target: int) -> int:
        def f(nums: List[int]) -> int:
            d = defaultdict(int)
            d[0] = 1
            cnt = s = 0
            for x in nums:
                s += x
                cnt += d[s - target]
                d[s] += 1
            return cnt

        m, n = len(matrix), len(matrix[0])
        ans = 0
        for i in range(m):
            col = [0] * n
            for j in range(i, m):
                for k in range(n):
                    col[k] += matrix[j][k]
                ans += f(col)

        return ans
```

• SimplyLeet

```

# Python solution for counting number of submatrices that sum to target

def numSubmatrixSumTarget(matrix, target):
    # Dimensions of the matrix
    n = len(matrix)
    m = len(matrix[0])
    result = 0

    # Loop over all possible starting rows
    for top in range(n):

        # Initialize a list to store the column sums between row 'top' and
        # 'bottom'
        col_sums = [0] * m

        # Extend the submatrix from row 'top' downwards to 'bottom'
        for bottom in range(top, n):
            # Update column sums for the current bottom row
            for col in range(m):
                col_sums[col] += matrix[bottom][col]

            # Use a dictionary to count cumulative sums in the 1D array
            (col_sums)

            sum_count = {0: 1}
            curr_sum = 0
            # Iterate over each column in col_sums to find the number of
            # subarrays with sum equal to target
            for value in col_sums:
                curr_sum += value
                if (curr_sum - target) in sum_count:
                    result += sum_count[curr_sum - target]
                # Update the dictionary with the current sum count
                sum_count[curr_sum] = sum_count.get(curr_sum, 0) + 1

        return result

# Example usage:
if __name__ == "__main__":
    matrix = [[0, 1, 0], [1, 1, 1], [0, 1, 0]]
    target = 0
    print(numSubmatrixSumTarget(matrix, target)) # Expected Output: 4

```


• LC.ca

```
class Solution:
    def numSubmatrixSumTarget(self, matrix: List[List[int]], target: int) -> int:

        def f(nums: List[int]) -> int:
            d = defaultdict(int)
            d[0] = 1
            cnt = s = 0
            for x in nums:
                s += x
                cnt += d[s - target]
                d[s] += 1

            return cnt

        m, n = len(matrix), len(matrix[0])
        ans = 0
        for i in range(m):
            col = [0] * n
            for j in range(i, m):
                for k in range(n):
                    col[k] += matrix[j][k]
                ans += f(col)

        return ans
```

◆ Quick Review

Key Insights

- Transform the 2D submatrix sum problem into multiple 1D subarray sum problems.
- Precompute cumulative sums by fixing two row boundaries and then summing columns between these rows.

- Use a hash table (or dictionary) to count the number of subarrays with a given sum, reducing the problem to the classic "subarray sum equals k" question.
- Iterate over all possible row pairs and for each, treat the column sums as an array where the target sum is sought.

Space & Time

Time Complexity: $O(n^2 * m)$, where n is the number of rows and m is the number of columns.

Space Complexity: $O(m)$, used by the hash table for storing cumulative sums along the columns.

Solution

The idea is to iterate over all pairs of rows (top and bottom) and, for each pair, compute the cumulative column sums between these rows. This converts the problem into finding the number of subarrays in a one-dimensional array (the column sums) that add up to the target. For each such 1D problem, use a hash table to track the frequency of cumulative sums seen so far and count how many times

the difference (current sum – target) has been seen. This count gives the number of subarrays ending at the current index which sum to target. Combining counts from all possible row pairs yields the final result.

Trees & BST

Round 3 · High · Hard · 2 questions

Trees & BST

□ #124 Binary Tree Maximum Path Sum

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming · Tree · DFS
Lists: Grind 169

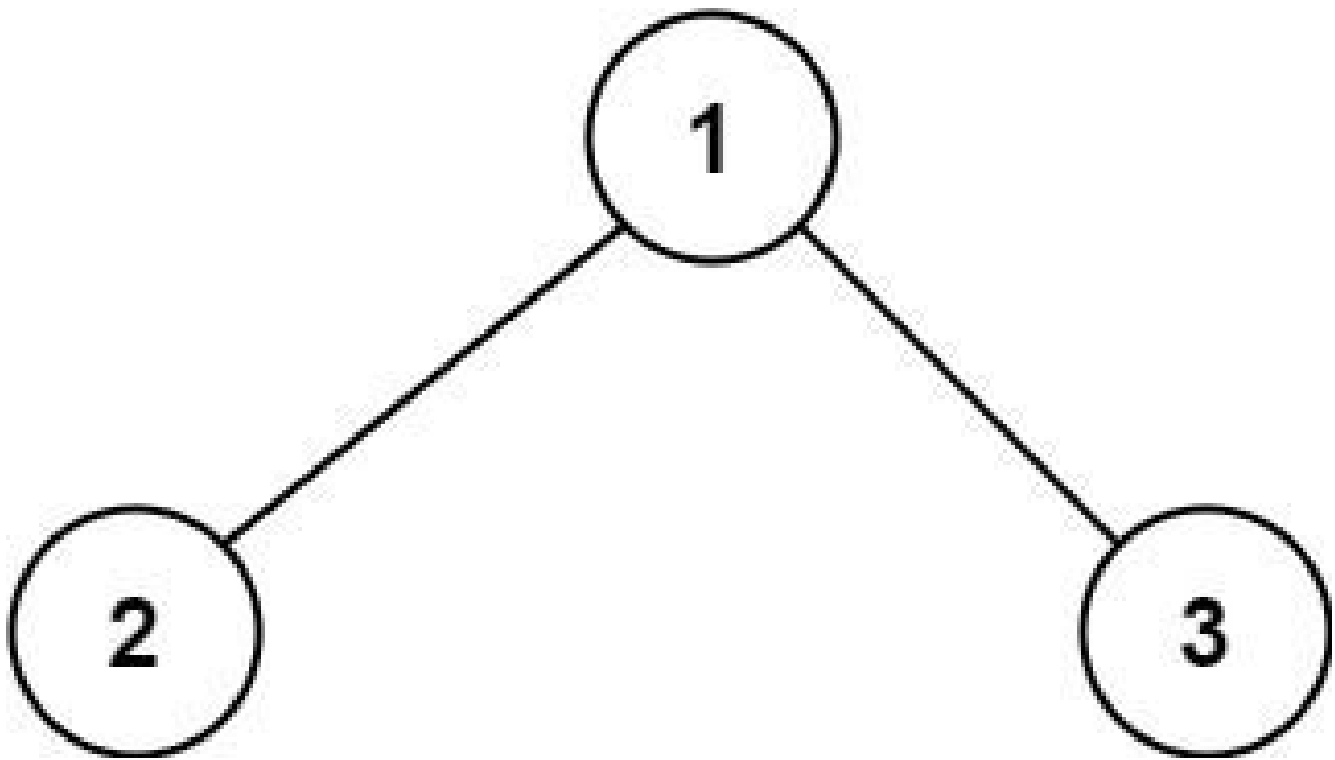
Problem

A path in a binary tree is a sequence of nodes where each pair of adjacent nodes in the sequence has an edge connecting them. A node can only appear in the sequence at most once. Note that the path does not need to pass through the root.

The path sum of a path is the sum of the node's values in the path.

Given the root of a binary tree, return the maximum path sum of any non-empty path.

Example 1:

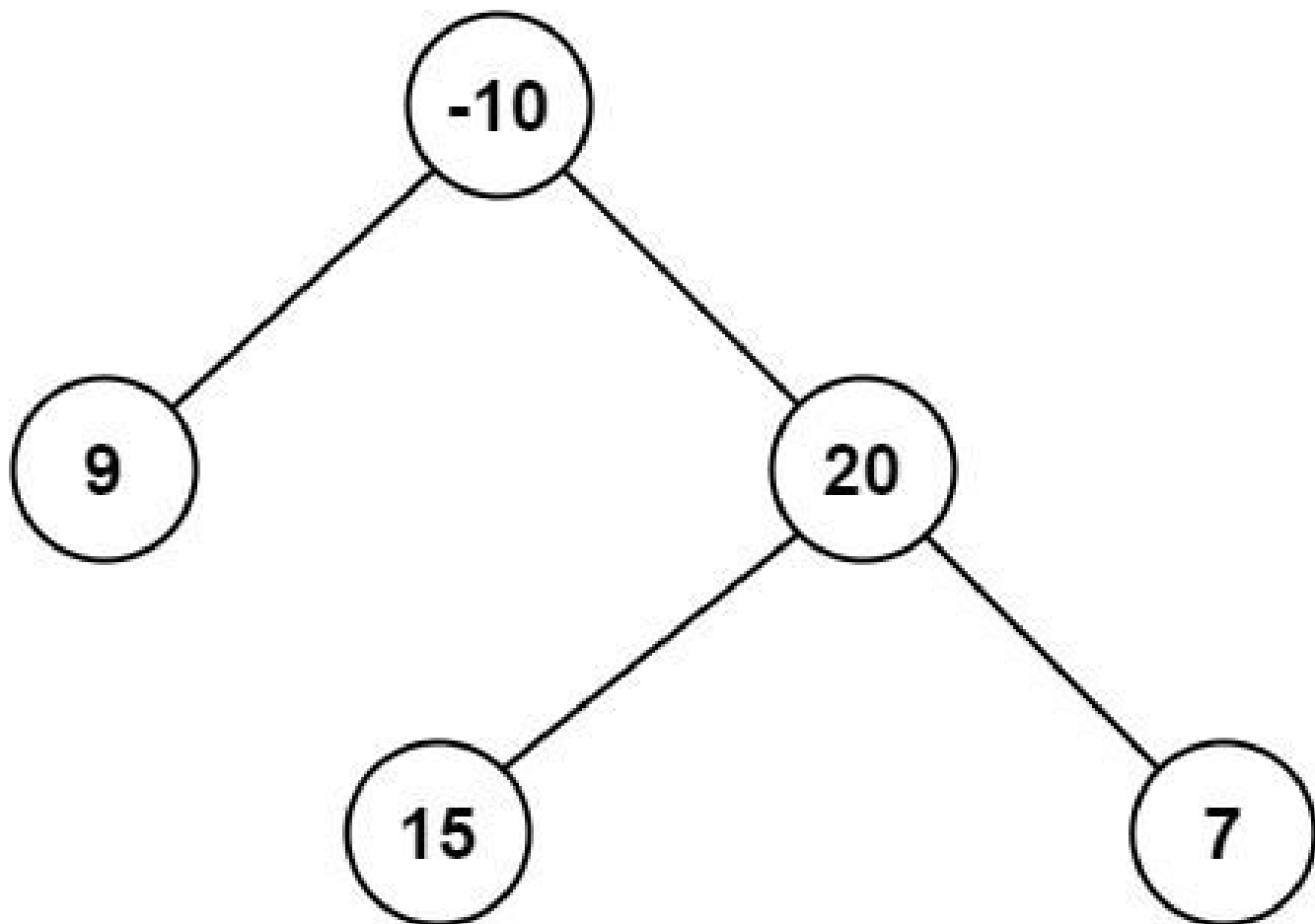


Input: root = [1,2,3]

Output: 6

Explanation: The optimal path is 2 -> 1 -> 3 with a path sum of $2 + 1 + 3 = 6$.

Example 2:



Input: root = [-10,9,20,null,null,15,7]

Output: 42

Explanation: The optimal path is 15 -> 20 -> 7 with a path sum of 15 + 20 + 7 = 42.

Constraints:

- The number of nodes in the tree is in the range [1, 3 * 10⁴].
- -1000 ≤ Node.val ≤ 1000

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY


```

# "Path = any sequence of connected nodes (no revisiting). Path doesn't need to pass
through root.
# Values can be negative. Return maximum path sum. Right?"
#
# "[-10,9,20,null,null,15,7]: path 15→20→7 = 42 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# At every node, compute max path sum through that node = node.val + max(0,
left_gain) + max(0, right_gain).
# Recurse full tree; track global max. Recomputes subtree gains repeatedly without
sharing.
# Time: O(n^2) worst case on skewed tree. Space: O(h) stack.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (post-order, same as Diameter)
# "For each node, compute the 'gain' if we extend a path through it: max(0,
left_gain, right_gain) + node.val.
# The best path THROUGH this node = node.val + max(0, left) + max(0, right).
# Update global max. Return one-sided gain to parent (can't go both left and right
AND up).
# Use inner helper – no self."

# PHASE 4 — CLEAN CODE
class _124_MaxPathSum:
    def maxPathSum(self, root: Optional['TreeNode']) -> int:
        best = [float('-inf')]
        def gain(node):
            if not node: return 0
            left = max(0, gain(node.left))
            right = max(0, gain(node.right))
            best[0] = max(best[0], node.val + left + right)
            return node.val + max(left, right)
        gain(root)
        return best[0]

# PHASE 5 — TEST & DEBUG
# [-10,9,20,null,null,15,7]:
# gain(9)=9. gain(15)=15. gain(7)=7.
# gain(20): left=15,right=7. best=max(-inf,20+15+7)=42. return 20+15=35.
# gain(-10): left=9,right=35. best=max(42,-10+9+35)=42. return -10+35=25.
# → 42 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(h). Same post-order pattern as Diameter of Binary Tree (#543)
and Balanced Tree (#110).
# The 'max(0, gain)' is essential – a negative subtree gain should be dropped.
# Given more time I'd ask: return the actual path (not just the sum)?
# Track which path achieved best[0] – store node references alongside the max
update."

```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxPathSum(self, root: TreeNode | None) -> int:
        ans = -math.inf

        def maxPathSumDownFrom(root: TreeNode | None) -> int:
            """
            Returns the maximum path sum starting from the current root, where
            root.val is always included.
            """
            nonlocal ans

            if not root:
                return 0

            l = max(0, maxPathSumDownFrom(root.left))
            r = max(0, maxPathSumDownFrom(root.right))
            ans = max(ans, root.val + l + r)
            return root.val + max(l, r)

        maxPathSumDownFrom(root)
        return ans
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def maxPathSum(self, root: Optional[TreeNode]) -> int:
        def dfs(root: Optional[TreeNode]) -> int:
            if root is None:
```

```

        return 0
    left = max(0, dfs(root.left))
    right = max(0, dfs(root.right))
    nonlocal ans
    ans = max(ans, root.val + left + right)
    return root.val + max(left, right)

ans = -inf
dfs(root)
return ans

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def maxPathSum(self, root: TreeNode) -> int:
        # Initialize global maximum path sum with a very small number

        self.global_max = float('-inf')

        def dfs(node: TreeNode) -> int:
            if not node:
                return 0

            # Recursively calculate max gain from left and right children,
            # discard negative gains
            left_gain = max(dfs(node.left), 0)
            right_gain = max(dfs(node.right), 0)

```

```

        # Current maximum including both left and right gains
        current_max = node.val + left_gain + right_gain

        # Update global maximum path sum if current_sum is higher
        self.global_max = max(self.global_max, current_max)

        # Return the node's value plus the max of one child gain (only one
        # side can be chosen for parent path)
        return node.val + max(left_gain, right_gain)

    dfs(root)

    return self.global_max

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def maxPathSum(self, root: Optional[TreeNode]) -> int:
        def dfs(root: Optional[TreeNode]) -> int:
            if root is None:
                return 0
            left = max(0, dfs(root.left))
            right = max(0, dfs(root.right))
            nonlocal ans
            ans = max(ans, root.val + left + right)
            return root.val + max(left, right)

        ans = -inf
        dfs(root)
        return ans

```

◆ Quick Review

Key Insights

- Use Depth-First Search (DFS) to explore the tree in a post-order fashion.
- For each node, calculate the maximum gain including that node and optionally one of its subtrees.
- A node's best contribution to its parent can be either just its own value or its value plus the maximum gain from either the left or right child.
- Update a global maximum path sum that considers the possibility of combining both left and right gains with the current node.
- Handle negative values by comparing gains against 0 to avoid decreasing the path sum.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the tree, since each node is visited exactly once.

Space Complexity: $O(n)$ in the worst-case scenario due to the recursion stack ($O(H)$ where H is the height of the tree, which can be $O(n)$ for skewed trees).

Solution

We use a recursive DFS approach (post-order traversal) to compute the maximum gain from each subtree. For every node: Recursively compute the maximum gain from the left and right child, ensuring that negative gains are treated as 0. The maximum gain that can be provided to the node's parent is the node's value plus the larger gain from its left or right child. Simultaneously, the potential maximum path sum including the current node as the highest (or root) node is the node's value plus both left and right gains. Update a global variable that holds the maximum path sum found so far. Return the maximum gain (node value plus one best child) to be used by the node's parent. This strategy ensures that every possible path sum is considered while avoiding double-counting nodes.

Trees & BST

□ #297 Serialize and Deserialize Binary Tree

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Tree · DFS · BFS · Design
Lists: Grind 169

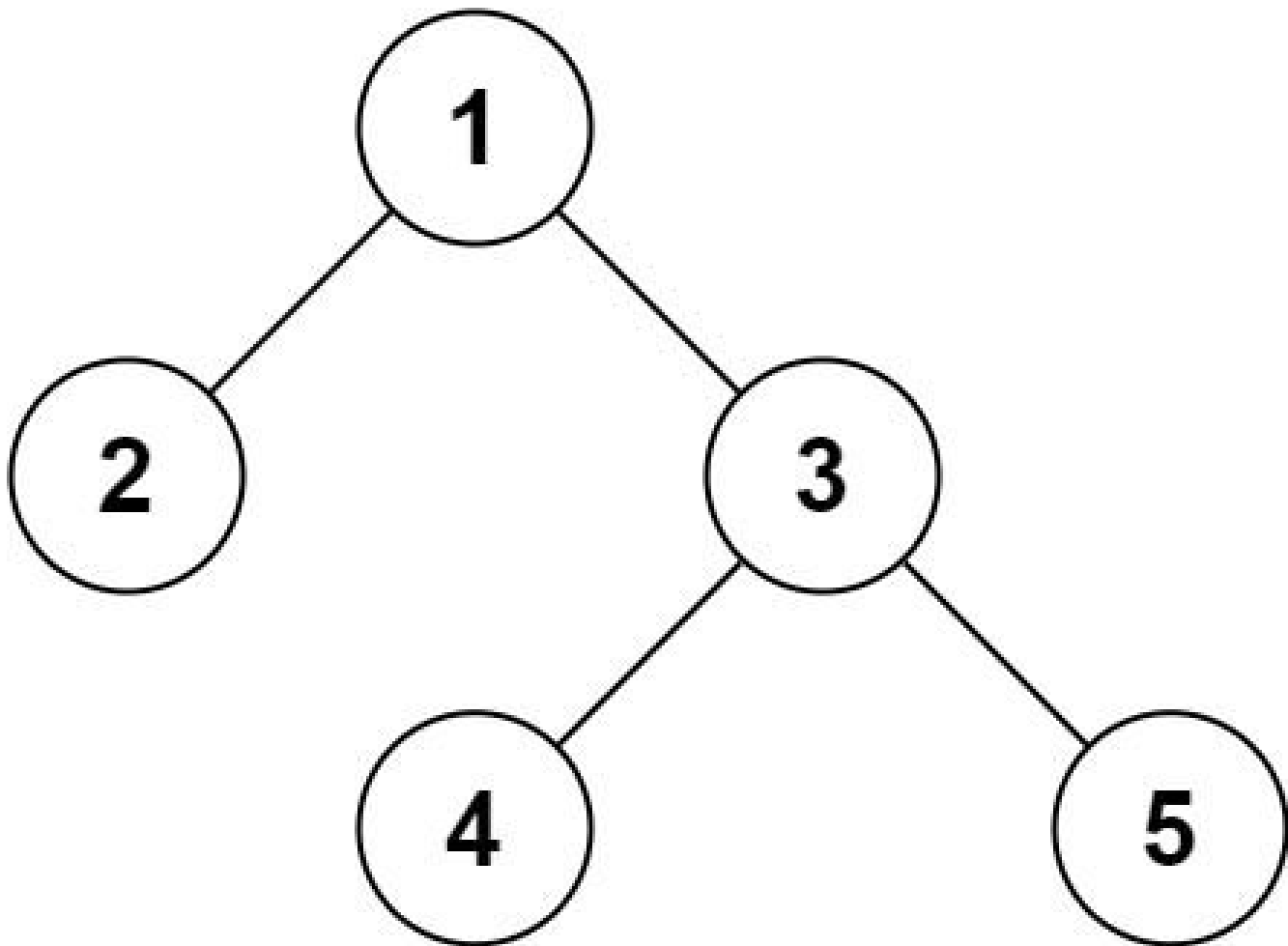
Problem

Serialization is the process of converting a data structure or object into a sequence of bits so that it can be stored in a file or memory buffer, or transmitted across a network connection link to be reconstructed later in the same or another computer environment.

Design an algorithm to serialize and deserialize a binary tree. There is no restriction on how your serialization/deserialization algorithm should work. You just need to ensure that a binary tree can be serialized to a string and this string can be deserialized to the original tree structure.

Clarification: The input/output format is the same as how LeetCode serializes a binary tree. You do not necessarily need to follow this format, so please be creative and come up with different approaches yourself.

Example 1:



Input: root = [1,2,3,null,null,4,5]
Output: [1,2,3,null,null,4,5]

Example 2:

Input: root = []
Output: []

Constraints:

- The number of nodes in the tree is in the range $[0, 10^4]$.
- $-1000 \leq \text{Node.val} \leq 1000$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Design encode and decode functions for a binary tree. Any format is fine as long as it round-trips.

Empty tree encodes to some valid string. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Serialize: preorder traversal with 'null' markers for empty children ? comma string.

Deserialize: split queue, rebuild tree recursively matching preorder order.

Correct but string can be long; BFS level-order encoding is more compact.

Time: $O(n)$. Space: $O(n)$ output string.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (pre-order DFS with null markers)

"Serialize: pre-order traversal, 'N' for null, comma-separated.

Deserialize: split, use an iterator, recursively build from pre-order.

Pre-order uniquely determines the tree when nulls are included."

PHASE 4 — CLEAN CODE

```
class _297_Codec:
    def serialize(self, root: 'TreeNode') -> str:
        parts: List[str] = []
        def dfs(node):
            if not node: parts.append('N'); return
            parts.append(str(node.val))
            dfs(node.left)
            dfs(node.right)
        dfs(root)
        return ','.join(parts)
    def deserialize(self, data: str) -> 'TreeNode':
        vals = iter(data.split(','))
        def build():
```

```

        v = next(vals)
        if v == 'N': return None
        node = TreeNode(int(v))
        node.left = build()
        node.right = build()
        return node
    return build()

```

PHASE 5 — TEST & DEBUG

[1,2,3,null,null,4,5]: serialize → '1,2,N,N,3,4,N,N,5,N,N'. deserialize rebuilds the tree ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(n)$. Pre-order with null markers is the cleanest approach.

BFS alternative: level-order like LeetCode's own format – works but trickier to parse.

Given more time I'd ask: serialize a general N-ary tree?

Add child count before children – e.g. '1,3,2,0,3,0,4,0,5,0' for a 3-child root."

Community Solutions (Python)

• WalkCC

```

class Codec:
    def serialize(self, root: 'TreeNode') -> str:
        """Encodes a tree to a single string."""
        if not root:
            return ''

        s = ''
        q = collections.deque([root])

        while q:

            node = q.popleft()
            if node:
                s += str(node.val) + ' '
                q.append(node.left)
                q.append(node.right)
            else:
                s += 'n '

        return s

```

```
def deserialize(self, data: str) -> 'TreeNode':
    """Decodes your encoded data to tree."""
    if not data:
        return None

    vals = data.split()
    root = TreeNode(vals[0])
    q = collections.deque([root])

    for i in range(1, len(vals), 2):

        node = q.popleft()
        if vals[i] != 'n':
            node.left = TreeNode(vals[i])
            q.append(node.left)
        if vals[i + 1] != 'n':
            node.right = TreeNode(vals[i + 1])
            q.append(node.right)

    return root
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Codec:
    def serialize(self, root):
```

```

        """Encodes a tree to a single string.

        :type root: TreeNode
        :rtype: str
        """
        if root is None:
            return ""
        q = deque([root])
        ans = []
        while q:

            node = q.popleft()
            if node:
                ans.append(str(node.val))
                q.append(node.left)
                q.append(node.right)
            else:
                ans.append("#")
        return ",".join(ans)

    def deserialize(self, data):

        """Decodes your encoded data to tree.

        :type data: str
        :rtype: TreeNode
        """
        if not data:
            return None
        vals = data.split(",")
        root = TreeNode(int(vals[0]))
        q = deque([root])

        i = 1
        while q:
            node = q.popleft()
            if vals[i] != "#":
                node.left = TreeNode(int(vals[i]))
                q.append(node.left)
            i += 1
            if vals[i] != "#":
                node.right = TreeNode(int(vals[i]))
                q.append(node.right)

```

```

        i += 1
    return root

```

```

# Your Codec object will be instantiated and called as such:
# codec = Codec()
# codec.deserialize(codec.serialize(root))

```

• SimplyLeet

```

# Definition for a binary tree node.

```

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val        # value of the node
        self.left = left      # left child
        self.right = right    # right child

```

```

class Codec:
    # Encodes a tree to a single string using BFS.
    def serialize(self, root):

```

```

        if not root:
            return "" # Return empty string for an empty tree
        result = []
        queue = [root] # Initialize the queue with the root node
        while queue:
            node = queue.pop(0)
            if node:
                result.append(str(node.val)) # Append the value of the node
                queue.append(node.left)      # Enqueue left child
                queue.append(node.right)     # Enqueue right child

```

```

        else:
            result.append("null") # Placeholder for null node
        return ",".join(result) # Join list with commas to form the string

```

```

# Decodes the encoded data to reconstruct the tree using BFS.

```

```

def deserialize(self, data):
    if not data:
        return None # Return None for empty string
    nodes = data.split(",")
    root = TreeNode(int(nodes[0])) # Create root node from first element

```

```

queue = [root]
i = 1 # Index for traversing nodes list
while queue:
    current = queue.pop(0)
    if i < len(nodes) and nodes[i] != "null":
        current.left = TreeNode(int(nodes[i]))
        queue.append(current.left)
    i += 1
    if i < len(nodes) and nodes[i] != "null":
        current.right = TreeNode(int(nodes[i]))

        queue.append(current.right)
    i += 1
return root

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode(object):
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Codec:
    def serialize(self, root):

        """Encodes a tree to a single string.

        :type root: TreeNode
        :rtype: str
        """

        if root is None:
            return ''
        res = []

```

```

def preorder(root):
    if root is None:
        res.append("#,")
        return
    res.append(str(root.val) + ",")
    preorder(root.left)
    preorder(root.right)

preorder(root)
return ''.join(res)

```

```

def deserialize(self, data):
    """Decodes your encoded data to tree.

    :type data: str
    :rtype: TreeNode
    """
    if not data:
        return None
    vals = data.split(',')

```

```

def inner():
    first = vals.pop(0)
    if first == '#':
        return None
    return TreeNode(int(first), inner(), inner())
    # seems using a constructor __init__(val, left, right)

return inner()

```

```

# Your Codec object will be instantiated and called as such:
# ser = Codec()
# deser = Codec()
# ans = deser.deserialize(ser.serialize(root))

#####

from collections import deque

```



```
# bfs, each level based
class Codec:
    def serialize(self, root):
        """Encodes a tree to a single string.

        :type root: TreeNode
        :rtype: str
        """
        ret = []
        queue = deque([root])

        while queue:
            top = queue.popleft()
            if not top:
                ret.append("None")
                continue
            else:
                ret.append(str(top.val))
                queue.append(top.left)
                queue.append(top.right)
        return ",".join(ret)

    def deserialize(self, data):
        """Decodes your encoded data to tree.

        :type data: str
        :rtype: TreeNode
        """
        data = data.split(",")
        if data[0] == "None":
            return None

        root = TreeNode(int(data[0]))
        queue = deque([root])
        i = 0
        while queue and i < len(data):
            top = queue.popleft()
            i += 1
            left = right = None
            if i < len(data) and data[i] != "None":
                left = TreeNode(int(data[i]))
                queue.append(left)
```

```
i += 1
if i < len(data) and data[i] != "None":
    right = TreeNode(int(data[i]))
    queue.append(right)

top.left = left
top.right = right

return root
```

```
# Your Codec object will be instantiated and called as such:
# codec = Codec()
# codec.deserialize(codec.serialize(root))
```

◆ Quick Review

Key Insights

- The tree can be traversed using either depth-first search (DFS) or breadth-first search (BFS).
- Using BFS (level-order traversal) simplifies reconstruction since nodes are processed level-by-level.
- Placeholder markers (e.g., "null") are used to indicate missing children.
- Both serialization and deserialization processes should handle empty trees and edge cases gracefully.
- Ensuring a one-to-one mapping between the tree structure and the serialized string is

critical.

Space & Time

Time Complexity: $O(n)$ for both serialization and deserialization, where n is the number of nodes in the tree.

Space Complexity: $O(n)$ due to the storage used for the serialized string and additional data structures (queue) during processing.

Solution

This solution uses level-order traversal (BFS) for both serialization and deserialization.

During serialization, each node is processed sequentially with missing children recorded as "null". For deserialization, the string is split by commas, and a queue is used to rebuild the tree level by level. The root is created from the first token, then subsequent tokens are assigned as left and right children accordingly, ensuring an accurate reconstruction of the original tree.

DFS

Round 3 · High · Hard · 5 questions

DFS

□ ★ #269 Alien Dictionary ★

HAR

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · DFS · BFS · Graph · Topological
Sort

Lists: ★ Premium | Grind 169 | Premium 98

Problem

There is a new alien language that uses the English alphabet. However, the order of the letters is unknown to you.

You are given a list of strings words from the alien language's dictionary. Now it is claimed that the strings in words are sorted lexicographically by the rules of this new language.

If this claim is incorrect, and the given arrangement of string in words cannot correspond to any order of letters, return "".

Otherwise, return a string of the unique letters in the new alien language sorted in lexicographically increasing order by the new

language's rules. If there are multiple solutions, return any of them.

Example 1:

```
Input: words = ["wrt","wrf","er","ett","rftt"]  
Output: "wertf"
```

Example 2:

```
Input: words = ["z","x"]  
Output: "zx"
```

Example 3:

```
Input: words = ["z","x","z"]  
Output: ""  
Explanation: The order is invalid, so return "".
```

Constraints:

- $1 \leq \text{words.length} \leq 100$
- $1 \leq \text{words}[i].\text{length} \leq 100$
- $\text{words}[i]$ consists of only lowercase English letters.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Given words sorted in alien lexicographic order, deduce the letter ordering.

Return any valid ordering, or '' if impossible (cycle or contradiction). Right?

Contradiction: longer word precedes its prefix (e.g. ['ab','a'])."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Build graph: for each adjacent pair in words, record char order u before v.

Topological sort the chars; if cycle detected, no valid order.

Compare words character by character to infer edges.

Time: $O(C)$ total chars. Space: $O(1)$ – 26 letters.
 # Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (Kahn's BFS topological sort)

"Extract ordering constraints from consecutive word pairs: compare char by char, first difference gives an edge.

Build directed graph + in-degree map.

BFS: process zero-in-degree nodes. If we process all letters → valid. Else cycle → return ''."

PHASE 4 — CLEAN CODE

```
class _269_AlienDictionary:
    def alienOrder(self, words: List[str]) -> str:
        graph = defaultdict(set)
        indegree = {c: 0 for w in words for c in w}
        for i in range(len(words) - 1):
            w1, w2 = words[i], words[i + 1]
            min_len = min(len(w1), len(w2))
            if len(w1) > len(w2) and w1[:min_len] == w2[:min_len]:
                return '' # invalid: prefix ordering
            for c1, c2 in zip(w1, w2):
                if c1 != c2:
                    if c2 not in graph[c1]:
                        graph[c1].add(c2)
                        indegree[c2] += 1
                    break
        queue = deque(c for c in indegree if indegree[c] == 0)
        result = []
        while queue:
            c = queue.popleft()
            result.append(c)
            for nb in graph[c]:
                indegree[nb] -= 1
                if indegree[nb] == 0:
                    queue.append(nb)
        return ''.join(result) if len(result) == len(indegree) else ''
```

PHASE 5 — TEST & DEBUG

["wrt","wrf","er","ett","rftt"]: edges: t→f, w→e, r→t, e→r. indegree: f:1,e:1,t:1,r:1,w:0.

BFS from w: w→e→r→t→f. result='wertf' ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(C)$ where C = total characters. Space $O(1)$ – at most 26 letters.

The prefix check catches the only invalid case that doesn't manifest as a graph cycle.

Given more time I'd ask: return ALL valid orderings?

Backtracking on the topological sort – collect all linearisations."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def alienOrder(self, words: list[str]) -> str:
        graph = {}
        inDegrees = [0] * 26

        self._buildGraph(graph, words, inDegrees)
        return self._topology(graph, inDegrees)

    def _buildGraph(
        self,

        graph: dict[str, set[str]],
        words: list[str],
        inDegrees: list[int],
    ) -> None:
        # Create a node for each character in each word.
        for word in words:
            for c in word:
                if c not in graph:
                    graph[c] = set()

        for first, second in zip(words, words[1:]):
            length = min(len(first), len(second))
            for j in range(length):
                u = first[j]
                v = second[j]
                if u != v:
                    if v not in graph[u]:
                        graph[u].add(v)
                        inDegrees[ord(v) - ord('a')] += 1
            break # The order of characters after this are meaningless.
```

```

        # First = 'ab', second = 'a' . invalid
        if j == length - 1 and len(first) > len(second):
            graph.clear()
            return

    def _topology(self, graph: dict[str, set[str]], inDegrees: list[int]) -> str
    :
        s = ''
        q = collections.deque()

        for c in graph:

            if inDegrees[ord(c) - ord('a')] == 0:
                q.append(c)

        while q:
            u = q.pop()
            s += u
            for v in graph[u]:
                inDegrees[ord(v) - ord('a')] -= 1
                if inDegrees[ord(v) - ord('a')] == 0:
                    q.append(v)

        # Words = ['z', 'x', 'y', 'x']
        return s if len(s) == len(graph) else ''

```

• LeetDoocs

```

class Solution:
    def alienOrder(self, words: List[str]) -> str:
        g = [[False] * 26 for _ in range(26)]
        s = [False] * 26
        cnt = 0
        n = len(words)
        for i in range(n - 1):
            for c in words[i]:
                if cnt == 26:
                    break

```

```

        o = ord(c) - ord('a')
        if not s[o]:
            cnt += 1
            s[o] = True
    m = len(words[i])
    for j in range(m):
        if j >= len(words[i + 1]):
            return ''
        c1, c2 = words[i][j], words[i + 1][j]
        if c1 == c2:

            continue
        o1, o2 = ord(c1) - ord('a'), ord(c2) - ord('a')
        if g[o2][o1]:
            return ''
        g[o1][o2] = True
        break
    for c in words[n - 1]:
        if cnt == 26:
            break
        o = ord(c) - ord('a')

        if not s[o]:
            cnt += 1
            s[o] = True

    indegree = [0] * 26
    for i in range(26):
        for j in range(26):
            if i != j and s[i] and s[j] and g[i][j]:
                indegree[j] += 1
    q = deque()

    ans = []
    for i in range(26):
        if s[i] and indegree[i] == 0:
            q.append(i)
    while q:
        t = q.popleft()
        ans.append(chr(t + ord('a')))
        for i in range(26):
            if s[i] and i != t and g[t][i]:
                indegree[i] -= 1

```

```

        if indegree[i] == 0:
            q.append(i)
    return '' if len(ans) < cnt else ''.join(ans)

```

• SimplyLeet

```

# Python solution for Alien Dictionary
from collections import defaultdict, deque

def alienOrder(words):
    # Create data structures: graph and in_degree for each unique character.
    graph = defaultdict(set)
    in_degree = {char: 0 for word in words for char in word}

    # Build the graph.
    for i in range(len(words) - 1):

        word1, word2 = words[i], words[i+1]
        # Check for invalid case where prefix of next word is the same but
        word1 is longer.
        if len(word1) > len(word2) and word1.startswith(word2):
            return ""

        # Compare the words character by character.
        for c1, c2 in zip(word1, word2):
            if c1 != c2:
                # If we haven't already recorded this edge, add it.
                if c2 not in graph[c1]:
                    graph[c1].add(c2)

                in_degree[c2] += 1
                break

    # Prepare for topological sort by adding all nodes with in_degree == 0.
    queue = deque([char for char in in_degree if in_degree[char] == 0])
    order = []

    # Kahn's algorithm for topological sort.
    while queue:
        current = queue.popleft()

```

```

        order.append(current)
        for neighbor in graph[current]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    # Check if topological sort is possible.
    if len(order) == len(in_degree):
        return "".join(order)
    else:

        return ""

# Example usage:
print(alienOrder(["wrt","wrf","er","ett","rftt"]))

```

• LC.ca

```

'''
>>> a = ['a','b','c']
>>> zip(a, a[1:])
[('a', 'b'), ('b', 'c')]

# same as pairwise()
>>> from itertools import pairwise
>>> pairwise(a)
<itertools.pairwise object at 0x108458730>

>>> list(pairwise(a))
[('a', 'b'), ('b', 'c')]
'''

'''
>>> inDegree = {'a':11, 'b':22, 'c':33}

>>> [print(c) for c in inDegree.keys()]
a
b

```

```

c
[None, None, None]

# same as for c in inDegree.keys()
>>> [print(c) for c in inDegree]
a
b
c
[None, None, None]

```

```

>>> [print(c) for c in inDegree.values()]
11
22
33
[None, None, None]

>>> [print(c) for c in inDegree.items()]
('a', 11)
('b', 22)
('c', 33)

```

```

[None, None, None]
'''

from collections import defaultdict, deque
from typing import Dict, List, Set

class Solution:
    def alienOrder(self, words: List[str]) -> str:
        graph = defaultdict(set)
        inDegree = defaultdict(int)

```

```

        self._buildGraph(graph, words, inDegree)
        return self._topology(graph, inDegree)

    def _buildGraph(self, graph: Dict[str, Set[str]], words: List[str], inDegree: Dict[str, int]) -> None:
        # Create a node for each character in each word
        for word in words:
            for c in word:
                inDegree[c] = 0 # necessary for final char counting

        for first, second in zip(words, words[1:]): # or pairwise(words)

            length = min(len(first), len(second))
            for j in range(length):
                u = first[j]
                v = second[j]
                if u != v:
                    if v not in graph[u]:
                        graph[u].add(v)
                        inDegree[v] += 1
                    break # Later characters' order is meaningless
            if j == length - 1 and len(first) > len(second):

                # If 'ab' comes before 'a', it's an invalid order
                graph.clear()
                return

    def _topology(self, graph: Dict[str, Set[str]], inDegree: Dict[str, int]) -> str:
        result = ''
        q = deque([c for c in inDegree if inDegree[c] == 0])

        while q:
            u = q.popleft()

```

```

        result += u
        for v in graph[u]:
            inDegree[v] -= 1
            if inDegree[v] == 0:
                q.append(v)

        # If there are remaining characters in inDegree, it means there's a
cycle
        if any(inDegree.values()):
            return ''

```

```

        # Words = ['z', 'x', 'y', 'x']
        return result if len(result) == len(indegree) else ''

```

```
#####
```

```
import collections
```

```
class Node(object):
    def __init__(self, val):

```

```

        self.val = val
        self.neighbors = []

```

```

    def connect(self, node):
        self.neighbors.append(node)

```

```

    def getNbrs(self):
        return self.neighbors

```



```

class Solution(object):
    def alienOrder(self, words):
        """
        :type words: List[str]
        :rtype: str
        """

    def dfs(root, graph, visited):
        visited[root] = 1
        for nbr in graph[root].getNbrs():

            if visited[nbr.val] == 0:
                if not dfs(nbr.val, graph, visited):
                    return False
            elif visited[nbr.val] == 1:
                return False

        visited[root] = 2
        self.ans += root
        return True

    self.ans = ""
    graph = {}
    visited = collections.defaultdict(int)
    self.topNum = 0
    for i in range(0, len(words) - 1):
        a = words[i]
        b = words[i + 1]
        i = 0
        while i < len(a) and i < len(b):
            if a[i] != b[i]:

                nodeA = nodeB = None
                if a[i] not in graph:
                    nodeA = Node(a[i])
                    graph[a[i]] = nodeA
                else:
                    nodeA = graph[a[i]]
                if b[i] not in graph:
                    nodeB = Node(b[i])
                    graph[b[i]] = nodeB
                else:

```

```

        nodeB = graph[b[i]]
        nodeA.connect(nodeB)
        break
    i += 1
    if i < len(a) and i >= len(b):
        return ""

    for c in graph:
        if visited[c] == 0:
            if not dfs(c, graph, visited):

                return ""

    unusedSet = set()
    for word in words:
        for c in word:
            unusedSet.add(c)

    for c in unusedSet:
        if c not in graph:
            self.ans += c

    return self.ans[::-1]

```

◆ Quick Review

Key Insights

- Build a graph where each unique character is a node.
- Compare adjacent words to determine the first different character, which gives the ordering (directed edge).
- Ensure all characters, even those without edges, are represented.

- Use a topological sort (using DFS or BFS) to determine a valid character ordering.
- Handle edge cases, such as words where a longer word appears before its prefix which is invalid.

Space & Time

Time Complexity: $O(N * L + V + E)$ where N is the number of words, L is the average length of each word, V is the number of unique characters, and E is the number of edges derived from adjacent word comparisons.

Space Complexity: $O(V + E)$ for storing the graph and additional data structures for topological sorting.

Solution

We use a graph-based approach to solve the problem. First, we initialize a graph (adjacency list) and in-degree dictionary for all unique characters. We then iterate through each pair of consecutive words, comparing them character by character to find the first pair that differs. This difference implies an order: the first character should come before the second character. We add a directed edge

accordingly and update the in-degree for the target character. To get the final ordering, we perform a topological sort using Kahn's algorithm: Enqueue characters with in-degree zero. Remove characters from the queue and append them to our result. Decrement the in-degree of neighboring nodes. If a neighbor's in-degree becomes zero, enqueue it. If the result's length equals the number of unique nodes, we have a valid ordering; otherwise, a cycle exists, and we return an empty string.

DFS

□ #329 Longest Increasing Path in a Matrix **HAR**

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Graph · Topological Sort ·
Memoization · Matrix

Lists: Grind 169

Problem

Given an $m \times n$ integers matrix, return the length of the longest increasing path in matrix. From each cell, you can either move in four directions: left, right, up, or down. You may not move diagonally or move outside the boundary (i.e., wrap-around is not allowed).

Example 1:

9	9	4
6	6	8
2	1	1

Input: matrix = [[9,9,4],[6,6,8],[2,1,1]]

Output: 4

Explanation: The longest increasing path is [1, 2, 6, 9].

Example 2:

3 →	4 →	5 ↓
3	2	6
2	2	1

Input: matrix = [[3,4,5],[3,2,6],[2,2,1]]

Output: 4

Explanation: The longest increasing path is [3, 4, 5, 6]. Moving diagonally is not allowed.

Example 3:

Input: matrix = [[1]]

Output: 1

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 200$

- $0 \leq \text{matrix}[i][j] \leq 231 - 1$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Longest strictly increasing path (4-directional). Return path length. Right?
Values can repeat but path must be strictly increasing."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."
DFS from every cell with memo: longest increasing path starting at (r,c).
Only move to strictly larger neighbors; memo avoids recomputing subpaths.
Without memo, pure DFS retries paths exponentially.
Time: $O(m * n)$ with memo. Space: $O(m * n)$ memo + stack.
Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (DFS + memoization)

"DFS from each cell. $\text{dp}[r][c]$ = longest path starting at (r,c).
Only move to neighbours with strictly greater value – no cycles possible → no visited set needed.
Memoize results: each cell computed once → $O(m*n)$."

PHASE 4 — CLEAN CODE

```
class _329_LongestIncPathMatrix:
    def longestIncreasingPath(self, matrix: List[List[int]]) -> int:
        m, n = len(matrix), len(matrix[0])
        memo = {}
        def dfs(r, c):
            if (r, c) in memo: return memo[(r, c)]
            best = 1
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                nr, nc = r + dr, c + dc
                if 0 <= nr < m and 0 <= nc < n and matrix[nr][nc] > matrix[r][c]:
                    best = max(best, dfs(nr, nc) + 1)
            memo[(r, c)] = best
            return best
        return max(dfs(r, c) for r in range(m) for c in range(n))
```

PHASE 5 — TEST & DEBUG

$[[9,9,4],[6,6,8],[2,1,1]]$: longest path starting at (2,1)=1: $1 \rightarrow 2 \rightarrow 6 \rightarrow 9 = 4$ ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(m*n)$ for memo. No visited set needed – strict increase prevents cycles."

Topological sort alternative: build DAG (only edges to larger neighbours), count levels.
 # Same $O(m \cdot n)$ but iterative.
 # Given more time I'd ask: longest path with values differing by at most 1?
 # Need a visited set now (can revisit same value) – different problem."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestIncreasingPath(self, matrix: list[list[int]]) -> int:
        m = len(matrix)
        n = len(matrix[0])

        @functools.lru_cache(None)
        def dfs(i: int, j: int, prev: int) -> int:
            if i < 0 or i == m or j < 0 or j == n:
                return 0
            if matrix[i][j] <= prev:
                return 0

            curr = matrix[i][j]
            return 1 + max(dfs(i + 1, j, curr),
                          dfs(i - 1, j, curr),
                          dfs(i, j + 1, curr),
                          dfs(i, j - 1, curr))

        return max(dfs(i, j, -math.inf) for i in range(m) for j in range(n))
```

• LeetDoocs

```
class Solution:
    def longestIncreasingPath(self, matrix: List[List[int]]) -> int:
        @cache
        def dfs(i: int, j: int) -> int:
            ans = 0
            for a, b in pairwise((-1, 0, 1, 0, -1)):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and matrix[x][y] > matrix[i][j]:
                    ans = max(ans, dfs(x, y))
            return ans + 1
```

```
m, n = len(matrix), len(matrix[0])
return max(dfs(i, j) for i in range(m) for j in range(n))
```

• SimplyLeet

Python implementation for Longest Increasing Path in a Matrix

```
class Solution:
    def longestIncreasingPath(self, matrix: List[List[int]]) -> int:
        # Return 0 if matrix is empty
        if not matrix or not matrix[0]:
            return 0

        # Dimensions of the matrix
        m, n = len(matrix), len(matrix[0])

        # Create a memoization table (same dimensions as the matrix)
        # initialized to 0
        memo = [[0] * n for _ in range(m)]

        # Define the DFS function to explore the matrix
        def dfs(i, j):
            # If result is already computed, return it
            if memo[i][j]:
                return memo[i][j]

            # Start with a path length of 1 (the cell itself)
            max_len = 1
            # Define directions: up, down, left, right
            directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

            # Explore each direction
            for dx, dy in directions:
                ni, nj = i + dx, j + dy
                # If neighbor is within bounds and its value is greater
                if 0 <= ni < m and 0 <= nj < n and matrix[ni][nj] > matrix[i][j]:
                    length = 1 + dfs(ni, nj)
```

```

        max_len = max(max_len, length)

    # Store computed longest length in memoization table
    memo[i][j] = max_len
    return max_len

# Compute the longest increasing path starting from each cell
longest_path = 0
for i in range(m):
    for j in range(n):

        longest_path = max(longest_path, dfs(i, j))

return longest_path

```

• LC.ca

```

class Solution:
    def longestIncreasingPath(self, matrix: List[List[int]]) -> int:
        @cache
        def dfs(i: int, j: int) -> int:
            ans = 0
            for a, b in pairwise((-1, 0, 1, 0, -1)):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and matrix[x][y] > matrix[i][j]:
                    ans = max(ans, dfs(x, y))
            return ans + 1

        m, n = len(matrix), len(matrix[0])
        return max(dfs(i, j) for i in range(m) for j in range(n))

```

◆ Quick Review

Key Insights

- The problem can be modeled as a graph where each cell is a node connected to its neighbors if the neighbor's value is greater.

- A Depth First Search (DFS) with memoization is effective to avoid recalculations.
- Dynamic Programming (DP) is used to cache the longest path found from each cell.
- Since each cell can be a start point, compute DFS for every cell and update the global maximum.

Space & Time

Time Complexity: $O(m * n)$ – each cell is computed once due to memoization.

Space Complexity: $O(m * n)$ – space is used for the memoization cache and recursion stack.

Solution

We solve the problem using a DFS approach combined with memoization to keep track of already computed longest paths from each cell. For each cell in the matrix, a DFS is initiated to explore all four valid directions (up, down, left, right). If a neighboring cell has a larger value, the DFS recurses from that neighbor. The result for each cell is cached in a 2D memoization table, ensuring that each cell is processed only once. After processing all cells, the maximum value in the memo table

represents the longest increasing path in the entire matrix.

DFS

□ #685 Redundant Connection II

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode

DFS · BFS · Union Find · Graph

Lists: Denny Zhang

Problem

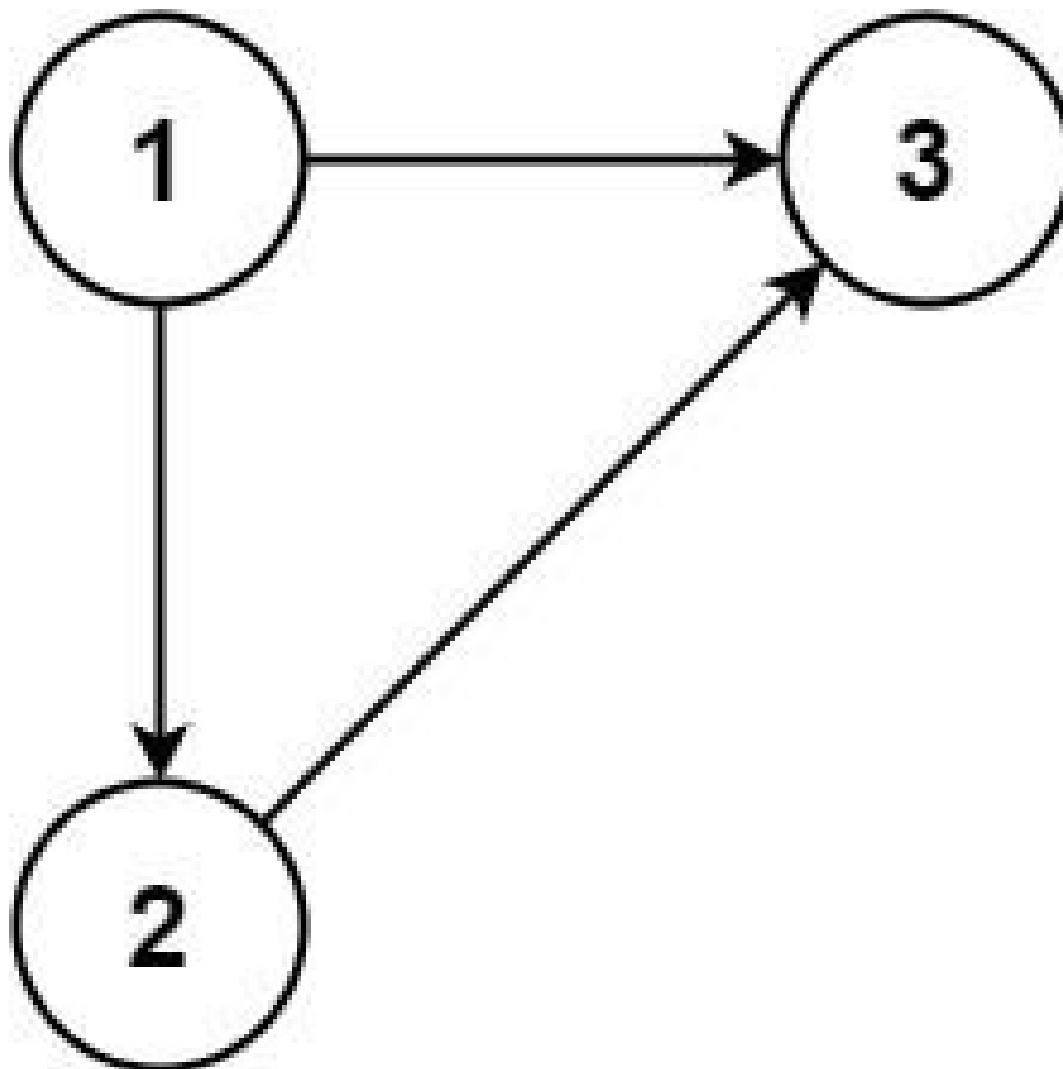
In this problem, a rooted tree is a directed graph such that, there is exactly one node (the root) for which all other nodes are descendants of this node, plus every node has exactly one parent, except for the root node which has no parents.

The given input is a directed graph that started as a rooted tree with n nodes (with distinct values from 1 to n), with one additional directed edge added. The added edge has two different vertices chosen from 1 to n , and was not an edge that already existed.

The resulting graph is given as a 2D-array of edges. Each element of edges is a pair $[u_i, v_i]$ that represents a directed edge connecting

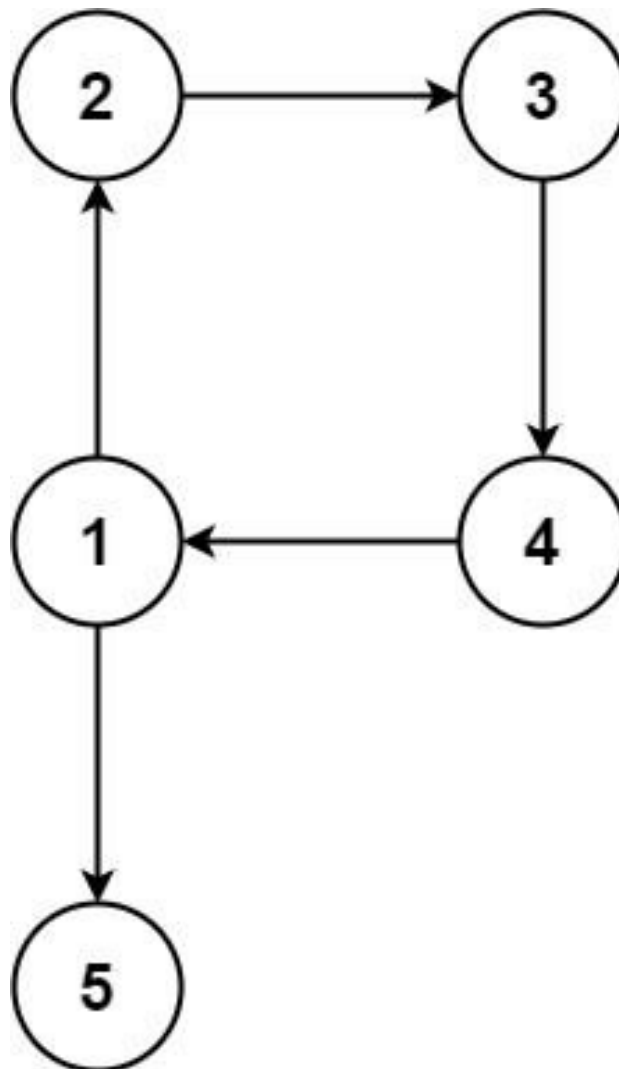
nodes u_i and v_i , where u_i is a parent of child v_i . Return an edge that can be removed so that the resulting graph is a rooted tree of n nodes. If there are multiple answers, return the answer that occurs last in the given 2D-array.

Example 1:



Input: edges = [[1,2],[1,3],[2,3]]
Output: [2,3]

Example 2:



Input: edges = [[1,2],[2,3],[3,4],[4,1],[1,5]]
Output: [4,1]

Constraints:

- $n == \text{edges.length}$
- $3 \leq n \leq 1000$
- $\text{edges}[i].\text{length} == 2$
- $1 \leq u_i, v_i \leq n$
- $u_i \neq v_i$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Directed graph: rooted tree + one extra edge. Find the edge to remove.
 # If multiple answers, return the last one in input. Right?
 # Extra edge causes: (1) a node with two parents, OR (2) a cycle, OR both."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
 # Build indegree + parent map from edges. Node with indegree 2 is the duplicate child candidate.
 # Detect cycle with DFS or union-find on the graph minus one suspicious edge.
 # Try removing each duplicate edge until graph becomes a valid tree.
 # Time: $O(n)$ with careful case analysis. Space: $O(n)$.
 # Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (three-case analysis + Union-Find)

"Track in-degree: if any node has two parents, record both candidate edges.
 # Then run Union-Find ignoring the second parent candidate.
 # Case 1 (two parents, no cycle in remaining): remove second candidate.
 # Case 2 (two parents, cycle in remaining): remove first candidate.
 # Case 3 (no node with two parents): cycle exists → last edge forming cycle."

PHASE 4 — CLEAN CODE

```
class _685_RedundantConnectionII:
    def findRedundantDirectedConnection(self, edges: List[List[int]]) -> List[int]:
        n = len(edges)
        parent = list(range(n + 1))
        cand1 = cand2 = None
        # Find if any node has two parents
        parent_map = {}
        for u, v in edges:
            if v in parent_map:
                cand1 = parent_map[v] # first edge giving v a parent
                cand2 = [u, v] # second edge giving v another parent
                break
            parent_map[v] = [u, v]
        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x
        def union(u, v) -> bool:
            pu, pv = find(u), find(v)
            if pu == pv: return False
            parent[pu] = pv
            return True
```

```

parent = list(range(n + 1))
for u, v in edges:
    if [u, v] == cand2: # skip the second candidate
        continue
    if not union(u, v):
        return cand1 if cand1 else [u, v]
return cand2

```

PHASE 5 — TEST & DEBUG

```

# [[1,2],[1,3],[2,3]]: node 3 has two parents (1 and 2). cand1=[1,3], cand2=[2,3].
# UF ignoring cand2=[2,3]: union(1,2) ok, union(1,3) ok. No cycle → return
cand2=[2,3] ✓
# [[1,2],[2,3],[3,4],[4,1],[1,5]]: no node with two parents. UF finds cycle [4,1] →
return [4,1] ✓

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(n * \alpha(n))$ . Space  $O(n)$ . Three cases arise from two root causes: double
parent and cycle.
# The Union-Find handles both: cycle detection when cand2 is skipped tells us which
candidate to remove.
# Given more time I'd ask: undirected version?
# That's Redundant Connection (#684) – standard Union-Find, return first edge that
creates a cycle."

```

Community Solutions (Python)

• WalkCC

```

class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> bool:
        i = self._find(u)
        j = self._find(v)
        if i == j:
            return False

```

```

    if self.rank[i] < self.rank[j]:
        self.id[i] = j
    elif self.rank[i] > self.rank[j]:
        self.id[j] = i
    else:
        self.id[i] = j
        self.rank[j] += 1
    return True

```

```

def _find(self, u: int) -> int:

```

```

    if self.id[u] != u:
        self.id[u] = self._find(self.id[u])
    return self.id[u]

```

```

class Solution:
    def findRedundantDirectedConnection(
        self, edges: list[list[int]],
    ) -> list[int]:
        ids = [0] * (len(edges) + 1)

```

```

        nodeWithTwoParents = 0

```

```

        for _, v in edges:
            ids[v] += 1
            if ids[v] == 2:
                nodeWithTwoParents = v

```

```

        def findRedundantDirectedConnection(skippedEdgeIndex: int) -> list[int]:
            uf = UnionFind(len(edges) + 1)

```

```

            for i, edge in enumerate(edges):
                if i == skippedEdgeIndex:
                    continue
                if not uf.unionByRank(edge[0], edge[1]):
                    return edge

            return []

```

```

        # If there is no edge with two ids, don't skip any edge.
        if nodeWithTwoParents == 0:

```

```

        return findRedundantDirectedConnection(-1)

    for i in reversed(range(len(edges))):
        _, v = edges[i]
        if v == nodeWithTwoParents:
            # Try to delete the edges[i].
            if not findRedundantDirectedConnection(i):
                return edges[i]

```

• LeetDoocs

```

class Solution:
    def findRedundantDirectedConnection(self, edges: List[List[int]]) -> List[
int]:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        n = len(edges)
        ind = [0] * n
        for _, v in edges:
            ind[v - 1] += 1

        dup = [i for i, (_, v) in enumerate(edges) if ind[v - 1] == 2]
        p = list(range(n))
        if dup:
            for i, (u, v) in enumerate(edges):
                if i == dup[1]:
                    continue
                pu, pv = find(u - 1), find(v - 1)
                if pu == pv:
                    return edges[dup[0]]

            p[pu] = pv
            return edges[dup[1]]
        for i, (u, v) in enumerate(edges):
            pu, pv = find(u - 1), find(v - 1)
            if pu == pv:
                return edges[i]
            p[pu] = pv

```

• SimplyLeet

```
# Python solution with line-by-line comments
class Solution:
    def findRedundantDirectedConnection(self, edges):
        n = len(edges)
        candidate1 = None
        candidate2 = None
        # Dictionary to record the parent for each child node.
        childParent = {}
        # First pass: detect if a node has two parents.
        for u, v in edges:
            if v in childParent:
                candidate1 = [childParent[v], v] # earlier edge providing
                candidate2 = [u, v] # later edge providing
            else:
                childParent[v] = u

        # Initialize Union-Find structure: each node is its own parent.
        root = [i for i in range(n+1)]

        # Find function with path compression.
        def find(x):
            while root[x] != x:
                root[x] = root[root[x]]
                x = root[x]
            return x

        cycle_edge = None
        # Second pass: union-find to detect cycle, skip candidate2 if conflict exists.
        for u, v in edges:
            # If candidate2 exists and current edge is candidate2, skip it.
```

```

        if candidate2 and [u, v] == candidate2:
            continue
        parent_u = find(u)
        parent_v = find(v)
        # If they have the same root, a cycle is detected.
        if parent_u == parent_v:
            cycle_edge = [u, v]
        else:
            root[parent_v] = parent_u

# Decide which edge to remove based on conflict and cycle detection.
if candidate1:
    return candidate1 if cycle_edge else candidate2
return cycle_edge

```

• LC.ca

```

class UnionFind:
    def __init__(self, n):
        self.p = list(range(n))
        self.n = n

    def union(self, a, b):
        if self.find(a) == self.find(b):
            return False
        self.p[self.find(a)] = self.find(b)
        self.n -= 1

    def find(self, x):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    return True

class Solution:
    def findRedundantDirectedConnection(self, edges: List[List[int]]) -> List[
int]:

```



```

n = len(edges)
p = list(range(n + 1))
uf = UnionFind(n + 1)
conflict = cycle = None
for i, (u, v) in enumerate(edges):
    if p[v] != v:
        conflict = i
    else:
        p[v] = u
        if not uf.union(u, v):
            cycle = i
if conflict is None:
    return edges[cycle]
v = edges[conflict][1]
if cycle is not None:
    return [p[v], v]
return edges[conflict]

```

◆ Quick Review

Key Insights

- The extra edge introduces one of two issues:
- A node has two parents.
- A cycle exists in the graph.
- First, scan through edges to detect if any node gets two parents. If found, there are two candidate edges.
- Use the Union–Find data structure (Disjoint Set Union) to detect cycles.
- Depending on whether a conflict (two parents) exists and/or a cycle is detected,

decide which edge to remove:

- **If a node has two parents, test by ignoring the second candidate edge during union–find. If no cycle forms, the second candidate is removable; otherwise, remove the first candidate.**
- **If no conflict exists, simply remove the edge that causes a cycle.**

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

The solution consists of two main steps: Traverse the list of edges to detect whether any node has two parents. If such a node is found, mark the two candidate edges (candidate1: the earlier edge, candidate2: the later edge). Use the Union–Find algorithm to iterate over the edges: If there is a conflict (i.e., a node with two parents), skip candidate2 initially. While processing edges with Union–Find, check if adding an edge would create a cycle. Finally, based on the presence of a cycle and the two–parent conflict, decide

which candidate edge to return. Key Data Structures: An array (or hash map) to track the parent of each node. The Union-Find (Disjoint Set Union) structure to union sets and detect cycles.

DFS

□ #1036 Escape a Large Maze

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · DFS · BFS
Lists: Denny Zhang

Problem

There is a 1 million by 1 million grid on an XY-plane, and the coordinates of each grid square are (x, y) .

We start at the source = $[sx, sy]$ square and want to reach the target = $[tx, ty]$ square. There is also an array of blocked squares, where each $blocked[i] = [xi, yi]$ represents a blocked square with coordinates (xi, yi) .

Each move, we can walk one square north, east, south, or west if the square is not in the array of blocked squares. We are also not allowed to walk outside of the grid.

Return true if and only if it is possible to reach the target square from the source square through a sequence of valid moves.

Example 1:

Input: blocked = [[0,1],[1,0]], source = [0,0], target = [0,2]
 Output: false
 Explanation: The target square is inaccessible starting from the source square because we cannot move.
 We cannot move north or east because those squares are blocked.
 We cannot move south or west because we cannot go outside of the grid.

Example 2:

Input: blocked = [], source = [0,0], target = [999999,999999]
 Output: true
 Explanation: Because there are no blocked cells, it is possible to reach the target square.

Constraints:

- $0 \leq \text{blocked.length} \leq 200$
- $\text{blocked}[i].\text{length} == 2$
- $0 \leq x_i, y_i < 106$
- $\text{source.length} == \text{target.length} == 2$
- $0 \leq s_x, s_y, t_x, t_y < 106$
- $\text{source} \neq \text{target}$
- It is guaranteed that source and target are not blocked.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"1M×1M grid. Up to 200 blocked cells. Can source reach target? Right?"

With at most 200 blocked cells in a semicircle, max enclosed area $\approx 200^2/2 = 20,000$ cells.

So if BFS from source visits > 20,000 cells without finding target, source is NOT enclosed."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
 # BFS from start cell; at each step try all 4 directions.
 # Track visited cells; if we revisit within <= 500 moves, we are in a loop ? false.
 # If we escape bounds within 500 steps per problem rule ? true.
 # Time: $O(m * n * 500)$ worst case. Space: $O(m * n)$ visited.
 # Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (BFS with cell limit)

"Run limited BFS from BOTH source and target.
 # If BFS visits > max_cells (= $n*(n+1)/2$ for $n=\text{len}(\text{blocked})$) → not enclosed → True from that side.
 # If BFS terminates (all cells explored) without reaching other endpoint → enclosed → False."

PHASE 4 — CLEAN CODE

```
class _1036_EscapeLargeMaze:
    def isEscapePossible(self, blocked: List[List[int]], source: List[int], target: List[int]) -> bool:
        if not blocked:
            return True
        n = len(blocked)
        max_cells = n * (n + 1) // 2
        blocked_set = {(r, c) for r, c in blocked}
        def bfs(start, end):
            queue = deque([tuple(start)])
            visited = {tuple(start)}
            while queue:
                r, c = queue.popleft()
                if [r, c] == end:
                    return True
                if len(visited) > max_cells:
                    return True # too many cells explored → not enclosed
                for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < 10**6 and 0 <= nc < 10**6 and (nr,nc) not in blocked_set and (nr,nc) not in visited:
                        visited.add((nr, nc))
                        queue.append((nr, nc))
            return False
        return bfs(source, target) and bfs(target, source)
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^2)$ per BFS (at most $n*(n+1)/2$ cells). Space $O(n^2)$. $n \leq 200 \rightarrow$ fast.
 # The dual-BFS is essential: source might not be enclosed but target might be.
 # Given more time I'd ask: what if blocked cells can be added dynamically?
 # Re-run BFS on each addition – or maintain Union-Find of open cells."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isEscapePossible(
        self,
        blocked: list[list[int]],
        source: list[int],
        target: list[int]
    ) -> bool:
        def dfs(i: int, j: int, target: list[int], seen: set) -> bool:
            if i < 0 or i >= 10**6 or j < 0 or j >= 10**6:
                return False

            if (i, j) in blocked or (i, j) in seen:
                return False
            seen.add((i, j))
            return (len(seen) > (1 + 199) * 199 // 2 or [i, j] == target or
                    dfs(i + 1, j, target, seen) or
                    dfs(i - 1, j, target, seen) or
                    dfs(i, j + 1, target, seen) or
                    dfs(i, j - 1, target, seen))

        blocked = set(tuple(b) for b in blocked)

        return (dfs(source[0], source[1], target, set()) and
                dfs(target[0], target[1], source, set()))
```

• LeetDoocs

```
class Solution:
    def isEscapePossible(
        self, blocked: List[List[int]], source: List[int], target: List[int]
    ) -> bool:
        def dfs(source: List[int], target: List[int], vis: set) -> bool:
            vis.add(tuple(source))
            if len(vis) > m:
                return True
            for a, b in pairwise(dirs):
                x, y = source[0] + a, source[1] + b
```



```

        if 0 <= x < n and 0 <= y < n and (x, y) not in s and (x, y) not
t in vis:
            if [x, y] == target or dfs([x, y], target, vis):
                return True
            return False

s = {(x, y) for x, y in blocked}
dirs = (-1, 0, 1, 0, -1)
n = 10**6
m = len(blocked) ** 2 // 2
return dfs(source, target, set()) and dfs(target, source, set())

```

• SimplyLeet

```

from collections import deque

def isEscapePossible(blocked, source, target):
    blocked_set = {(x, y) for x, y in blocked}
    n = len(blocked)
    if n < 2:
        return True
    max_steps = n * (n - 1) // 2

    def bfs(start, finish):
        queue = deque([start])
        seen = {tuple(start)}
        directions = [(0, 1), (1, 0), (0, -1), (-1, 0)]
        steps = 0

        while queue:
            x, y = queue.popleft()
            if [x, y] == finish:
                return True
            for dx, dy in directions:

```

```

        nx, ny = x + dx, y + dy
        if 0 <= nx < 10**6 and 0 <= ny < 10**6 and (nx, ny) not in blocked_set and (nx, ny) not in seen:
            queue.append((nx, ny))
            seen.add((nx, ny))
            steps += 1
            if steps > max_steps:
                return True
    return False

return bfs(source, target) and bfs(target, source)

```

```

# Example usage:
print(isEscapePossible([[0,1],[1,0]], [0,0], [0,2])) # Expected output: false
print(isEscapePossible([], [0,0], [999999,999999])) # Expected output: true

```

• LC.ca

```

class Solution:
    def isEscapePossible(
        self, blocked: List[List[int]], source: List[int], target: List[int]
    ) -> bool:
        def dfs(source, target, seen):
            x, y = source
            if (
                not (0 <= x < 10**6 and 0 <= y < 10**6)
                or (x, y) in blocked
                or (x, y) in seen
            ):
                return False
            seen.add((x, y))
            if len(seen) > 20000 or source == target:
                return True
            for a, b in [[0, -1], [0, 1], [1, 0], [-1, 0]]:
                next = [x + a, y + b]
                if dfs(next, target, seen):
                    return True
            return False

```

```
blocked = set((x, y) for x, y in blocked)
return dfs(source, target, set()) and dfs(target, source, set())
```

◆ Quick Review

Key Insights

- The maximum number of blocked cells is 200, which limits the size of a potential completely enclosed region.
- Even if the target is unreachable by direct search, if the BFS (or DFS) explores more than a calculated threshold (based on the blocked cells count), then the region is not fully enclosed.
- Perform two BFS explorations: one starting from the source and one from the target. If neither search is trapped within a small region, then an escape/path exists.
- Early exit in BFS if the target is reached or if the number of visited nodes exceeds the maximum possible enclosed area.

Space & Time

Time Complexity: $O(B^2)$ in the worst case, where B is the number of blocked cells (with B

≤ 200).

Space Complexity: $O(B^2)$ due to the storage of visited cells in the worst-case scenario.

Solution

The solution uses the Breadth-First Search (BFS) algorithm. A key observation here is that the blocked cells can only enclose a limited area. We compute a threshold (maxSteps) based on the number of blocked cells ($n \cdot (n-1)/2$) to determine if the search is confined. The BFS from the source (and similarly from the target) stops in two cases: if the target is reached or if the visited area exceeds the threshold, indicating that the region is not fully enclosed by blocked cells. Running BFS from both the source and target ensures that neither is trapped inside a small banned region.

DFS

□ #1192 Critical Connections in a Network **HAR**

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode

DFS · Graph · Biconnected Component

Lists: Denny Zhang

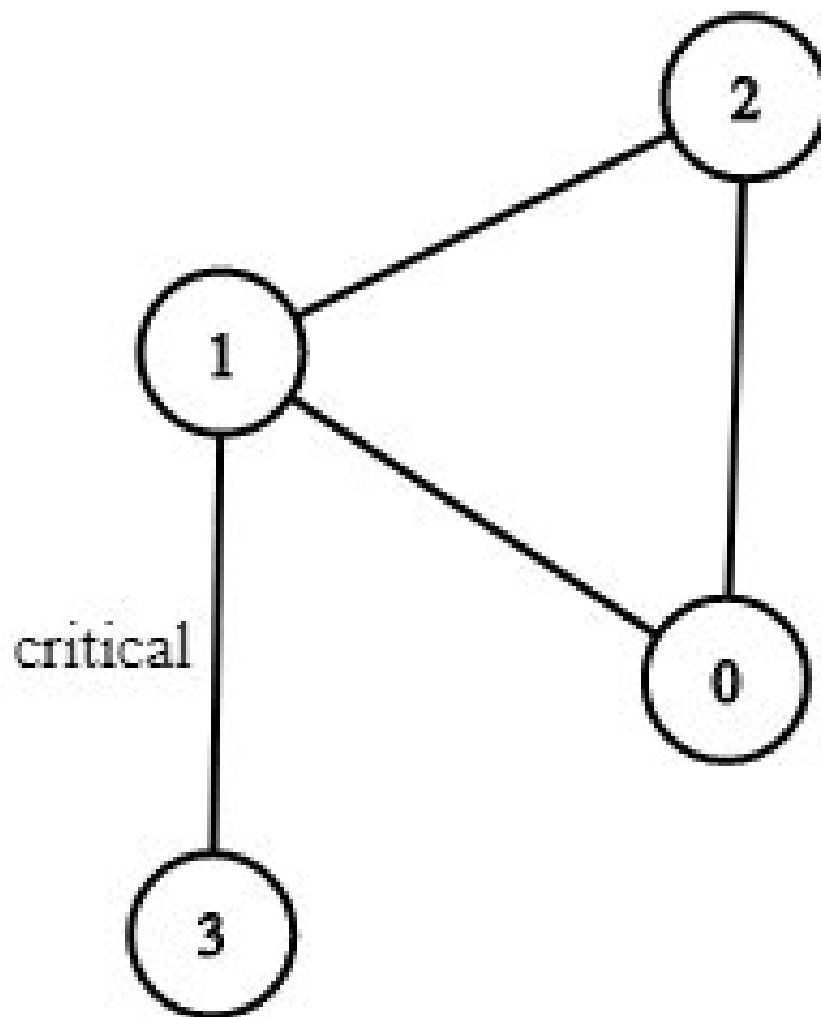
Problem

There are n servers numbered from 0 to $n - 1$ connected by undirected server-to-server connections forming a network where $\text{connections}[i] = [a_i, b_i]$ represents a connection between servers a_i and b_i . Any server can reach other servers directly or indirectly through the network.

A critical connection is a connection that, if removed, will make some servers unable to reach some other server.

Return all critical connections in the network in any order.

Example 1:



Input: $n = 4$, `connections = [[0,1],[1,2],[2,0],[1,3]]`

Output: `[[1,3]]`

Explanation: `[[3,1]]` is also accepted.

Example 2:

Input: $n = 2$, `connections = [[0,1]]`

Output: `[[0,1]]`

Constraints:

- $2 \leq n \leq 105$
- $n - 1 \leq \text{connections.length} \leq 105$
- $0 \leq a_i, b_i \leq n - 1$

- $a_i \neq b_i$
- There are no repeated connections.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Critical connection (bridge) = edge whose removal disconnects the graph. Right?
Use Tarjan's bridge-finding algorithm."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
Find bridges with Tarjan low-link DFS: edge (u,v) is critical if $low[v] > disc[u]$.
One DFS pass tracks discovery time and lowest reachable.
Naive alternative: remove each edge and test connectivity – $O(E * (V+E))$.
Time: $O(V + E)$ with Tarjan. Space: $O(V + E)$.
Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (Tarjan's DFS)

"DFS assigning $disc[u]$ =discovery time and $low[u]$ =earliest discovery reachable.
An edge (u,v) is a bridge if $low[v] > disc[u]$ – meaning v cannot reach u's subtree without (u,v)."

PHASE 4 — CLEAN CODE

```
class _1192_CriticalConnections:
    def criticalConnections(self, n: int, connections: List[List[int]]) ->
List[List[int]]:
        graph = defaultdict(list)
        for u, v in connections:
            graph[u].append(v)
            graph[v].append(u)
        disc = [-1] * n
        low = [-1] * n
        timer = [0]
        result = []
        def dfs(node, parent):
            disc[node] = low[node] = timer[0]
            timer[0] += 1
            for nb in graph[node]:
                if nb == parent: continue
                if disc[nb] == -1:
                    dfs(nb, node)
                low[node] = min(low[node], low[nb])
```



```

        if low[nb] > disc[node]:
            result.append([node, nb])
        else:
            low[node] = min(low[node], disc[nb])
    dfs(0, -1)
    return result

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(V+E)$. Space $O(V+E)$. Tarjan's bridge algorithm in one DFS pass.

$\text{low}[v] > \text{disc}[u]$ means v 's subtree can't 'back-connect' to u 's subtree – edge is a bridge.

Given more time I'd ask: articulation points instead of bridges?

Similar but: u is an articulation point if any child v has $\text{low}[v] \geq \text{disc}[u]$ (and u is not root)."

Community Solutions (Python)

• LeetDoocs

```

class Solution:
    def criticalConnections(
        self, n: int, connections: List[List[int]]
    ) -> List[List[int]]:
        def tarjan(a: int, fa: int):
            nonlocal now
            now += 1
            dfn[a] = low[a] = now
            for b in g[a]:
                if b == fa:
                    continue
                if not dfn[b]:
                    tarjan(b, a)
                    low[a] = min(low[a], low[b])
                    if low[b] > dfn[a]:
                        ans.append([a, b])
                else:
                    low[a] = min(low[a], dfn[b])

        g = [[] for _ in range(n)]

```

```

    for a, b in connections:
        g[a].append(b)
        g[b].append(a)

    dfn = [0] * n
    low = [0] * n
    now = 0
    ans = []
    tarjan(0, -1)
    return ans

```

• SimplyLeet

```

def criticalConnections(n, connections):
    # Build the adjacency list for the graph
    graph = [[] for _ in range(n)]
    for u, v in connections:
        graph[u].append(v)
        graph[v].append(u)

    # Initialize discovery time and low arrays
    disc = [-1] * n # Discovery times, -1 indicates unvisited
    low = [0] * n   # Low values for each node

    time = 0        # Global timer
    bridges = []     # List to store critical connections (bridges)

    def dfs(u, parent):
        nonlocal time
        disc[u] = low[u] = time
        time += 1

        # Traverse all adjacent nodes
        for v in graph[u]:

```

```

        if disc[v] == -1: # If v is not visited
            dfs(v, u)
            low[u] = min(low[u], low[v])
            # If the smallest reachable vertex from v is after u's
            discovery, edge (u,v) is a bridge
            if low[v] > disc[u]:
                bridges.append([u, v])
        elif v != parent: # v is visited and is not the parent (back edge)
            low[u] = min(low[u], disc[v])

# Start DFS from node 0

dfs(0, -1)
return bridges

# Example usage:
n = 4
connections = [[0,1],[1,2],[2,0],[1,3]]
print(criticalConnections(n, connections)) # Output: [[1, 3]]

```

• LC.ca

```

class Solution:
    def criticalConnections(
        self, n: int, connections: List[List[int]]
    ) -> List[List[int]]:
        def tarjan(a: int, fa: int):
            nonlocal now
            now += 1
            dfn[a] = low[a] = now
            for b in g[a]:
                if b == fa:
                    continue
                if not dfn[b]:
                    tarjan(b, a)
                    low[a] = min(low[a], low[b])
                    if low[b] > dfn[a]:
                        ans.append([a, b])
                else:
                    low[a] = min(low[a], dfn[b])

        g = [[] for _ in range(n)]

```

```
for a, b in connections:
    g[a].append(b)
    g[b].append(a)

dfn = [0] * n
low = [0] * n
now = 0
ans = []
tarjan(0, -1)
return ans
```

◆ Quick Review

Key Insights

- The problem is about identifying bridges in an undirected graph.
- Use Depth-First Search (DFS) to explore the graph.
- During DFS, track discovery times and low-link values for each node.
- For an edge (u, v) , if $\text{low}[v] > \text{disc}[u]$ then (u, v) is a critical connection.
- Tarjan's algorithm is well-suited for solving this problem in $O(n + m)$ time.

Space & Time

Time Complexity: $O(n + m)$ where n is the number of nodes and m is the number of connections.

Space Complexity: $O(n + m)$ for the graph representation and DFS recursion stack.

Solution

We solve this problem using a DFS-based approach (Tarjan's algorithm). First, we transform the list of connections into an adjacency list for efficient graph traversal. We then start a DFS from an arbitrary node (node 0, since the graph is connected) while maintaining two arrays: one for discovery times (disc) and one for low-link values (low). The discovery time is the time when the node is first visited, and the low-link value is the smallest discovery time that can be reached from that node (including via back edges). For each edge (u, v) , after a DFS call on v , if $\text{low}[v] > \text{disc}[u]$ then the edge (u, v) is a bridge because v (and its descendants) cannot reach u or any of its ancestors without this edge. This edge is then recorded as a critical connection.

Graphs

Round 3 · High · Hard · 2 questions

Graphs

□ ★ #305 Number of Islands II ★

HAR

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Union Find · Matrix
Lists: ★ Premium | Premium 98

Problem

You are given an empty 2D binary grid `grid` of size $m \times n$. The grid represents a map where 0's represent water and 1's represent land. Initially, all the cells of grid are water cells (i.e., all the cells are 0's).

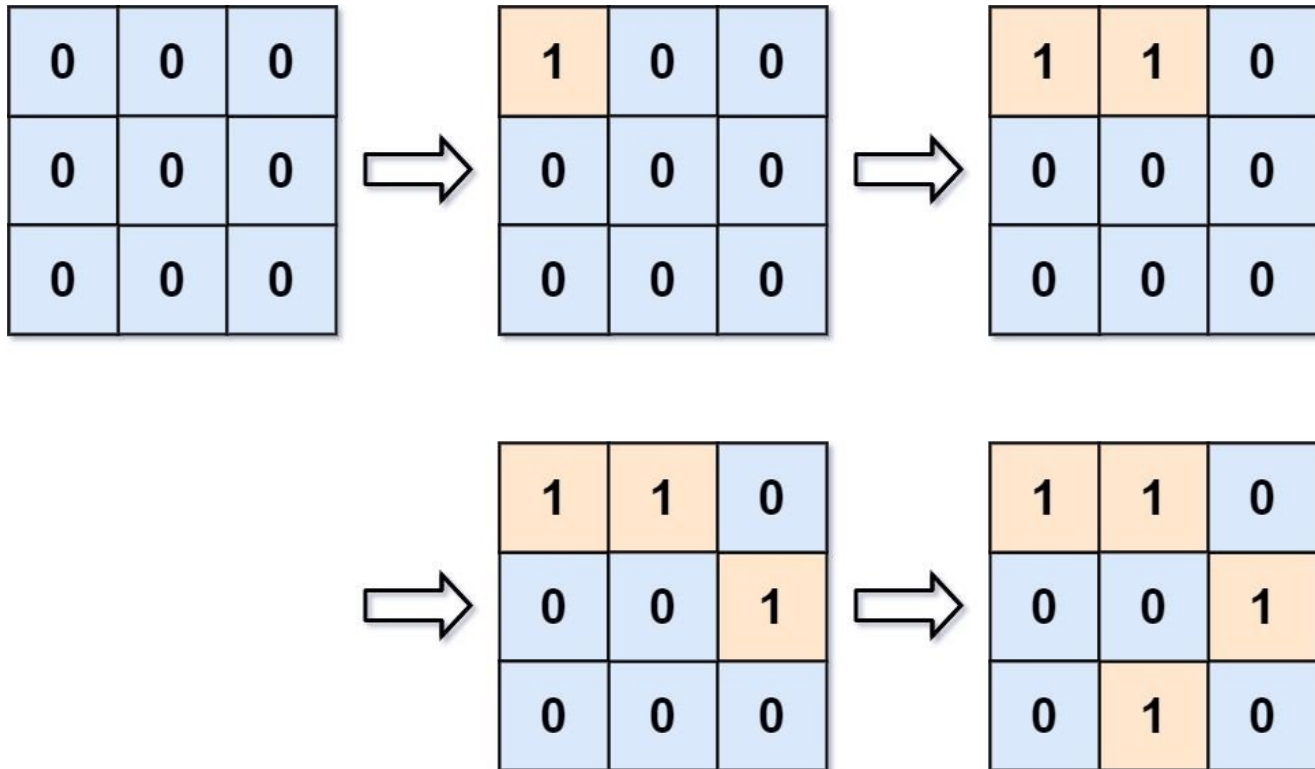
We may perform an add land operation which turns the water at position into a land. You are given an array `positions` where `positions[i] = [ri, ci]` is the position (ri, ci) at which we should operate the i th operation.

Return an array of integers `answer` where `answer[i]` is the number of islands after turning the cell (ri, ci) into a land.

An island is surrounded by water and is formed by connecting adjacent lands horizontally or

vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:



Input: $m = 3$, $n = 3$, positions = $[[0,0],[0,1],[1,2],[2,1]]$

Output: $[1,1,2,3]$

Explanation:

Initially, the 2d grid is filled with water.

- Operation #1: addLand(0, 0) turns the water at grid[0][0] into a land. We have 1 island.
- Operation #2: addLand(0, 1) turns the water at grid[0][1] into a land. We still have 1 island.
- Operation #3: addLand(1, 2) turns the water at grid[1][2] into a land. We have 2 islands.
- Operation #4: addLand(2, 1) turns the water at grid[2][1] into a land. We have 3 islands.

Example 2:

Input: $m = 1$, $n = 1$, positions = $[[0,0]]$

Output: $[1]$

Constraints:

- $1 \leq m, n, \text{positions.length} \leq 104$
- $1 \leq m * n \leq 104$
- $\text{positions}[i].\text{length} == 2$
- $0 \leq r_i < m$
- $0 \leq c_i < n$

Follow up: Could you solve it in time complexity $O(k \log(mn))$, where $k == \text{positions.length}$?

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def numIslands2(self, m: int, n: int, positions: List[List[int]]) -> List[int]:
        class UnionFind:
            def __init__(self, size):
                self.parent = [-1] * size  # ← singular, consistent
                self.rank = [0] * size
                self.count = 0

            def findparent(self, x):
                if self.parent[x] != x:
                    self.parent[x] = self.findparent(self.parent[x])
                return self.parent[x]

            def union(self, x, y):
                px, py = self.findparent(x), self.findparent(y)
                if px == py:
                    return False
                if self.rank[px] > self.rank[py]:  # ← brackets not ()
                    self.parent[py] = px
                elif self.rank[px] < self.rank[py]:  # ← brackets not ()
                    self.parent[px] = py
                else:
                    self.parent[py] = px
                    self.rank[py] += 1
                self.count += 1
```

```

        self.parent[px] = py
    else:
        self.parent[py] = px
        self.rank[px] += 1
    self.count -= 1
    return True

uf = UnionFind(m * n)
dirs = ((1,0),(-1,0),(0,1),(0,-1))
result = []

for x, y in positions:
    index = x * n + y
    if uf.parent[index] != -1:           # duplicate position
        result.append(uf.count)
        continue                       # ← also need to skip re-adding

    uf.parent[index] = index
    uf.count += 1

    for dx, dy in dirs:
        nx, ny = x + dx, y + dy
        if 0 <= nx < m and 0 <= ny < n:  # ← correct bounds
            newindex = nx * n + ny
            if uf.parent[newindex] != -1:
                uf.union(index, newindex)

    result.append(uf.count)

return result

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Start with empty grid. Add land cells one at a time. After each add, return island count.
 # $O(k \log mn)$ per operation. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
 # After each addLand, scan the whole grid and flood-fill count islands from scratch.

```
# k addLand calls each trigger full O(m*n) recount - too slow when k is large.
# Time: O(k * m * n). Space: O(m * n) per flood.
# Should I go ahead and optimize?"
```

```
# PHASE 3 — OPTIMIZE (Union-Find with dynamic updates)
```

```
# "Each new land cell starts as its own island (count++).
```

```
# Check 4 neighbours: for each land neighbour, if different component, union and count--."
```

```
# PHASE 4 — CLEAN CODE
```

```
class _305_NumberIslandsII:
    def numIslands2(self, m: int, n: int, positions: List[List[int]]) -> List[int]:
        parent = {}
        count = [0]
        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x
        def union(a, b):
            ra, rb = find(a), find(b)
            if ra == rb: return
            parent[ra] = rb
            count[0] -= 1
        result = []
        for r, c in positions:
            if (r, c) in parent:
                result.append(count[0])
                continue
            parent[(r, c)] = (r, c)
            count[0] += 1
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                nr, nc = r + dr, c + dc
                if (nr, nc) in parent:
                    union((r, c), (nr, nc))
            result.append(count[0])
        return result
```

```
# PHASE 6 — COMPLEXITY & FOLLOW-UP
```

```
# "Time O(k * α(mn)) ≈ O(k). Space O(k). Union-Find handles dynamic connectivity perfectly.
```

```
# The duplicate position check avoids double-counting islands.
```

```
# Given more time I'd ask: what if land cells can be removed too?
```

```
# Split-Find is hard. Offline deletion via reverse addition is a common trick."
```

Community Solutions (Python)

• WalkCC

```

class UnionFind:
    def __init__(self, n: int):
        self.id = [-1] * n
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> None:
        i = self.find(u)
        j = self.find(v)
        if i == j:
            return

        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1

    def find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self.find(self.id[u])
        return self.id[u]

class Solution:
    def numIslands2(
        self,
        m: int,
        n: int,
        positions: list[list[int]],
    ) -> list[int]:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        ans = []
        seen = [[False] * n for _ in range(m)]
        uf = UnionFind(m * n)
        count = 0

        def getId(i: int, j: int, n: int) -> int:
            return i * n + j

```

```

for i, j in positions:
    if seen[i][j]:
        ans.append(count)
        continue
    seen[i][j] = True
    id = getId(i, j, n)
    uf.id[id] = id
    count += 1
    for dx, dy in DIRS:
        x = i + dx

        y = j + dy
        if x < 0 or x == m or y < 0 or y == n:
            continue
        neighborId = getId(x, y, n)
        if uf.id[neighborId] == -1: # water
            continue
        currentParent = uf.find(id)
        neighborParent = uf.find(neighborId)
        if currentParent != neighborParent:
            uf.unionByRank(currentParent, neighborParent)

    count -= 1
    ans.append(count)

return ans

```

• LeetDoocs

```

class UnionFind:
    def __init__(self, n: int):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x: int):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

```

```

def union(self, a: int, b: int) -> bool:
    pa, pb = self.find(a - 1), self.find(b - 1)
    if pa == pb:
        return False
    if self.size[pa] > self.size[pb]:
        self.p[pb] = pa
        self.size[pa] += self.size[pb]
    else:
        self.p[pa] = pb
        self.size[pb] += self.size[pa]

```

```

    return True

```

```

class Solution:
    def numIslands2(self, m: int, n: int, positions: List[List[int]]) -> List[int]:
        uf = UnionFind(m * n)
        grid = [[0] * n for _ in range(m)]
        ans = []
        dirs = (-1, 0, 1, 0, -1)
        cnt = 0

```

```

        for i, j in positions:
            if grid[i][j]:
                ans.append(cnt)
                continue
            grid[i][j] = 1
            cnt += 1
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                if (

```

```

                    and 0 <= y < n
                    and grid[x][y]
                    and uf.union(i * n + j, x * n + y)
                ):
                    cnt -= 1
            ans.append(cnt)
        return ans

```

• SimplyLeet

Round 3 · High · Hard

p.1811

```
# Python implementation using Union Find with path compression and union by rank.
```

```
class UnionFind:
```

```
    def __init__(self, size):
```

```
        # parent: stores parent of each node. -1 indicates water.
```

```
        self.parent = [-1] * size
```

```
        self.rank = [0] * size
```

```
        self.count = 0 # number of islands
```

```
    def find(self, x):
```

```
        # Find with path compression.
```

```
        if self.parent[x] != x:
```

```
            self.parent[x] = self.find(self.parent[x])
```

```
        return self.parent[x]
```

```
    def union(self, x, y):
```

```
        # Union the sets; if union is successful, reduce island count.
```

```
        rootX = self.find(x)
```

```
        rootY = self.find(y)
```

```
        if rootX == rootY:
```

```
            return False
```

```
        # union by rank.
```

```
        if self.rank[rootX] < self.rank[rootY]:
```

```
            self.parent[rootX] = rootY
```

```
        elif self.rank[rootX] > self.rank[rootY]:
```

```
            self.parent[rootY] = rootX
```

```
        else:
```

```
            self.parent[rootY] = rootX
```

```
            self.rank[rootX] += 1
```

```
        self.count -= 1
```

```
        return True
```



```

class Solution:
    def numIslands2(self, m, n, positions):
        uf = UnionFind(m * n)
        result = []
        directions = [(0, 1), (1, 0), (0, -1), (-1, 0)]

        for r, c in positions:
            idx = r * n + c
            # If the position is already land, skip.

            if uf.parent[idx] != -1:
                result.append(uf.count)
                continue

            # Mark this cell as land forming a new island.
            uf.parent[idx] = idx
            uf.count += 1

            # Union with all 4 neighbors if they are already land.
            for dr, dc in directions:

                nr, nc = r + dr, c + dc
                nidx = nr * n + nc
                if 0 <= nr < m and 0 <= nc < n and uf.parent[nidx] != -1:
                    uf.union(idx, nidx)
            result.append(uf.count)
        return result

# Example run:
# s = Solution()
# print(s.numIslands2(3, 3, [[0,0],[0,1],[1,2],[2,1]]))

```

• LC.ca

```

class Solution:
    def numIslands2(self, m: int, n: int, positions: List[List[int]]) -> List[
int]:
        def check(i, j):
            return 0 <= i < m and 0 <= j < n and grid[i][j] == 1

        def find(x):
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        p = list(range(m * n))
        grid = [[0] * n for _ in range(m)]
        res = []
        cur = 0 # current island count
        for i, j in positions:
            if grid[i][j] == 1: # already counted, same as previous island
count
                res.append(cur)
                continue
            grid[i][j] = 1
            cur += 1

            for x, y in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
                if check(i + x, j + y) and find(i * n + j) != find((i + x) * n
+ j + y):
                    p[find(i * n + j)] = find((i + x) * n + j + y)
                    cur -= 1
                    # 1 0 1
                    # 0 1 0
                    # for above, if setting 1st-row 2nd-col 0-to-1
                    # cur will -1 for 3 times
                    # but cur += 1 after grid[i][j] = 1 so cur final result is
1
                    res.append(cur)

```

```

        return res

#####

class UnionFind(object):
    def __init__(self, m, n):
        self.dad = [i for i in range(m * n)]
        self.rank = [0 for _ in range(m * n)]

    def find(self, x):

        if self.dad[x] != x:
            self.dad[x] = self.find(self.dad[x])
        return self.dad[x]

    def union(self, xy): # xy is a tuple (x,y), with x and y value inside
        x, y = map(self.find, xy)
        if x == y:
            return False
        if self.rank[x] > self.rank[y]:
            self.dad[y] = x

        elif self.rank[x] < self.rank[y]:
            self.dad[x] = y
        else:
            self.dad[y] = x # search to the left, to find parent
            self.rank[x] += 1 # now x a parent, so +1 its rank
        return True

class Solution(object):
    def numIslands2(self, m, n, positions):

        """
        :type m: int
        :type n: int
        :type positions: List[List[int]]
        :rtype: List[int]
        """

        uf = UnionFind(m, n)
        ans = 0
        ret = []
        dirs = [(0, -1), (0, 1), (1, 0), (-1, 0)]

```

```

grid = set()
for i, j in positions:
    ans += 1
    grid |= {(i, j)}
    for di, dj in dirs:
        ni, nj = i + di, j + dj
        if 0 <= ni < m and 0 <= nj < n and (ni, nj) in grid:
            if uf.union((ni * n + nj, i * n + j)):
                ans -= 1
ret.append(ans)

```

```

return ret

```

```

#####

```

```

class UnionFind:

```

```

    def __init__(self, n: int):
        self.p = list(range(n))
        self.size = [1] * n

```

```

    def find(self, x: int):
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

```

```

    def union(self, a: int, b: int) -> bool:
        pa, pb = self.find(a - 1), self.find(b - 1)
        if pa == pb:
            return False

```

```

        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]
        return True

```

```

class Solution:

```

```

def numIslands2(self, m: int, n: int, positions: List[List[int]]) -> List[int]:
    uf = UnionFind(m * n)
    grid = [[0] * n for _ in range(m)]
    ans = []
    dirs = (-1, 0, 1, 0, -1)
    cnt = 0
    for i, j in positions:
        if grid[i][j]:
            ans.append(cnt)
            continue

        grid[i][j] = 1
        cnt += 1
        for a, b in pairwise(dirs):
            x, y = i + a, j + b
            if (
                0 <= x < m
                and 0 <= y < n
                and grid[x][y]
                and uf.union(i * n + j, x * n + y)
            ):
                cnt -= 1
        ans.append(cnt)
    return ans

```

◆ Quick Review

Key Insights

- Use Union Find (Disjoint Set Union) to efficiently manage connected components (islands).
- Map 2D grid coordinates to a 1D index.
- When adding a new land, check its 4 neighbors (up, down, left, right) and perform

unions if they are also lands.

- Avoid duplicate processing if the same cell is added multiple times.
- Maintain a counter for islands that is updated during each union operation.

Space & Time

Time Complexity: $O(K * \alpha(mn))$ where K is the number of operations and α is the inverse Ackermann function (almost constant time per union/find).

Space Complexity: $O(mn)$ for storing the union-find structure.

Solution

The solution relies on the Union Find data structure to dynamically merge islands. When a new land cell is added: If the cell is already land, record the current island count.

Otherwise, mark the cell as land and increment the island count. Check the four neighboring cells; if any neighbor is land, attempt to union the current cell with that neighbor. If a union actually happens (i.e., the two cells were in separate islands), decrement the island counter. Output the island count after each

add-land operation. The key trick is converting 2D indices to a 1D representation ($r * n + c$) and using path compression along with union by rank in the Union Find implementation for optimal performance.

Graphs

□ #1153 String Transforms Into Another String

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Union Find
Lists: Denny Zhang

Problem

Given two strings `str1` and `str2` of the same length, determine whether you can transform `str1` into `str2` by doing zero or more conversions.

In one conversion you can convert all occurrences of one character in `str1` to any other lowercase English character.

Return `true` if and only if you can transform `str1` into `str2`.

Example 1:

Input: `str1 = "aabcc", str2 = "ccdee"`

Output: `true`

Explanation: Convert 'c' to 'e' then 'b' to 'd' then 'a' to 'c'. Note that the order of conversions matter.

Example 2:

Input: str1 = "leetcode", str2 = "codeleet"

Output: false

Explanation: There is no way to transform str1 to str2.

Constraints:

- $1 \leq \text{str1.length} == \text{str2.length} \leq 104$
- str1 and str2 contain only lowercase English letters.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def canConvert(self, str1: str, str2: str) -> bool:
        if str1 == str2:
            return True

        hmap = {}

        for c1, c2 in zip(str1, str2):
            if c1 in hmap:
                if hmap[c1] != c2:
                    return False

            else:
                hmap[c1] = c2

        if len(set(str2)) == 26:
            return False

        return True
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Can str1 be converted to str2 by repeatedly mapping one character to another (all
occurrences)?
# Conversions can be chained but must be consistent. Right?"
#
# "If str1==str2 trivially True. If str2 uses all 26 letters → no 'buffer' character
available → False."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Build a map from each character to its index in the first string.
# Scan the second string left-to-right: next char must appear at a strictly higher
index.
# If any char is missing or out of order, transformation is impossible.
# Time: O(n). Space: O(1) – fixed alphabet size 26.
# Should I go ahead and optimize?"
```

```
class _1153_StringTransforms:
    def canConvert(self, str1: str, str2: str) -> bool:
        if str1 == str2:
            return True
        mapping = {}
        for c1, c2 in zip(str1, str2):
            if mapping.get(c1, c2) != c2:
                return False
            mapping[c1] = c2
        return len(set(str2)) < 26 # need at least one unused char as buffer
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(26). The len(set(str2)) < 26 check: if all 26 chars appear in
str2,
# every potential buffer letter is 'locked' to a mapping – cycles can't be broken.
# Given more time I'd ask: what if conversions can only be applied once?
# Topological sort on the conversion graph – acyclic → True."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def canConvert(self, str1: str, str2: str) -> bool:
        if str1 == str2:
            return True

        mappings = {}

        for a, b in zip(str1, str2):
            if mappings.get(a, b) != b:
                return False
```

```
mappings[a] = b
```

```
# No letter in the str1 maps to > 1 letter in the str2 and there is at
# least one temporary letter can break any loops.
return len(set(str2)) < 26
```

• LeetCode

```
class Solution:
    def canConvert(self, str1: str, str2: str) -> bool:
        if str1 == str2:
            return True
        if len(set(str2)) == 26:
            return False
        d = {}
        for a, b in zip(str1, str2):
            if a not in d:
                d[a] = b

            elif d[a] != b:
                return False
        return True
```

• SimplyLeet

```
# Python solution with line-by-line comments
def canConvert(str1: str, str2: str) -> bool:
    # If both strings are equal, no conversion is needed.
    if str1 == str2:
        return True

    # Dictionary to hold the mapping from characters in str1 to characters in
    str2
    mapping = {}

    # Iterate through the characters of both strings simultaneously.
```

```

for c1, c2 in zip(str1, str2):
    # If c1 is already in the mapping, check for consistency.
    if c1 in mapping:
        if mapping[c1] != c2:
            # Found an inconsistent mapping.
            return False
    else:
        mapping[c1] = c2

# Count distinct characters in str2.

# If str2 uses all 26 letters, then there is no extra character available
to break cycles.
if len(set(str2)) == 26:
    return False

return True

# Example Usage:
#print(canConvert("aabcc", "ccdee")) # Expected True
#print(canConvert("leetcode", "codeleet")) # Expected False

```

• LC.ca

```

class Solution:
    def canConvert(self, str1: str, str2: str) -> bool:
        if str1 == str2:
            return True
        if len(set(str2)) == 26:
            return False
        d = {}
        for a, b in zip(str1, str2):
            if a not in d:
                d[a] = b

            elif d[a] != b:
                return False
        return True

```

◆ Quick Review

Key Insights

- Each character mapping from `str1` to `str2` must be consistent; one character in `str1` always maps to the same character in `str2`.
- A bijective mapping is not necessary; multiple characters in `str1` can map to the same character in `str2`.
- The order of conversions matters. When there is a cycle (e.g., mapping 'a' → 'b' and 'b' → 'a'), an intermediate unused character might be required to break the cycle.
- If `str2` uses all 26 letters, no temporary character is available to break mapping cycles. In this case, transformation is possible only if `str1` is already equal to `str2`.

Space & Time

Time Complexity: $O(n)$, where n is the length of the strings, because we traverse the strings to build and check the mapping.

Space Complexity: $O(1)$ or $O(26)$, which is constant space, to store the mapping for the 26 lowercase English letters.

Solution

The approach is as follows: If `str1` equals `str2`, return `true` immediately. Build a mapping from each character in `str1` to the corresponding character in `str2`. While creating the mapping, check that each character maps consistently. Count the number of distinct characters in `str2`. If this number is 26 and `str1` is not already equal to `str2`, it is impossible to perform the transformation because there is no spare character available as a temporary placeholder. If the mapping is consistent and there exists at least one character not used in `str2` (i.e., distinct characters in `str2` is less than 26), the transformation is possible. The important trick is identifying the need for a free character when the target mapping is “full” (i.e., covers all 26 letters). Without a free character, you cannot avoid cycles where a temporary intermediate is necessary.

BFS

Round 3 · High · Hard · 4 questions

BFS

□ #127 Word Ladder

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · BFS
Lists: Grind 169

Problem

A transformation sequence from word `beginWord` to word `endWord` using a dictionary `wordList` is a sequence of words `beginWord` → `s1` → `s2` → ... → `sk` such that:

- Every adjacent pair of words differs by a single letter.
- Every `si` for $1 \leq i \leq k$ is in `wordList`. Note that `beginWord` does not need to be in `wordList`.
- `sk == endWord`

Given two words, `beginWord` and `endWord`, and a dictionary `wordList`, return the number of words in the shortest transformation sequence from `beginWord` to `endWord`, or 0 if no such sequence exists.

Example 1:

```
Input: beginWord = "hit", endWord = "cog", wordList =  
["hot", "dot", "dog", "lot", "log", "cog"]
```

```
Output: 5
```

```
Explanation: One shortest transformation sequence is "hit" -> "hot" -> "dot" ->  
"dog" -> "cog", which is 5 words long.
```

Example 2:

```
Input: beginWord = "hit", endWord = "cog", wordList =  
["hot", "dot", "dog", "lot", "log"]
```

```
Output: 0
```

```
Explanation: The endWord "cog" is not in wordList, therefore there is no valid  
transformation sequence.
```

Constraints:

- $1 \leq \text{beginWord.length} \leq 10$
- $\text{endWord.length} == \text{beginWord.length}$
- $1 \leq \text{wordList.length} \leq 5000$
- $\text{wordList}[i].\text{length} == \text{beginWord.length}$
- beginWord , endWord , and $\text{wordList}[i]$ consist of lowercase English letters.
- $\text{beginWord} \neq \text{endWord}$
- All the words in wordList are unique.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Shortest word transformation from beginWord to endWord , changing one letter at a time.

Each intermediate word must be in wordList . Return total words in shortest path, or 0. Right?"

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Build adjacency from word patterns (one-letter-apart neighbors).
# BFS from beginWord layer by layer until endWord is reached; each layer = one
transformation.
# Visited set prevents revisiting words. Return BFS depth or 0 if unreachable.
# Time:  $O(N * L^2)$  where  $N$  = dictionary size,  $L$  = word length. Space:  $O(N)$ .
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (BFS with pattern matching)

```
# "BFS on words. For each current word, try replacing each position with 26 letters.
# If the result is in wordSet (and not visited), enqueue it.
# Alternatively: build adjacency by pattern (*ot, h*t, ho*) → group words by pattern
–  $O(L^2 * n)$ ."
```

PHASE 4 — CLEAN CODE

```
class _127_WordLadder:
    def ladderLength(self, beginWord: str, endWord: str, wordList: List[str]) ->
int:
    word_set = set(wordList)
    if endWord not in word_set:
        return 0
    queue = deque([(beginWord, 1)])
    visited = {beginWord}
    while queue:
        word, steps = queue.popleft()
        for i in range(len(word)):
            for c in 'abcdefghijklmnopqrstuvwxyz':
                next_word = word[:i] + c + word[i+1:]
                if next_word == endWord:
                    return steps + 1
                if next_word in word_set and next_word not in visited:
                    visited.add(next_word)
                    queue.append((next_word, steps + 1))
    return 0
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n * L * 26) = O(n * L)$ . Space  $O(n * L)$ . Standard BFS shortest path.
# Bidirectional BFS: search simultaneously from beginWord and endWord – roughly
halves depth.
# Given more time I'd ask: return all shortest paths?
# That's Word Ladder II (#126) – BFS to build levels, then DFS to backtrack paths."
```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def ladderLength(
        self,
        beginWord: str,
        endWord: str,
        wordList: list[str],
    ) -> int:
        wordSet = set(wordList)
        if endWord not in wordSet:
            return 0

        q = collections.deque([beginWord])

        step = 1
        while q:
            for _ in range(len(q)):
                wordList = list(q.popleft())
                for i, cache in enumerate(wordList):
                    for c in string.ascii_lowercase:
                        wordList[i] = c

                        word = ''.join(wordList)
                        if word == endWord:
                            return step + 1
                        if word in wordSet:
                            q.append(word)
                            wordSet.remove(word)
                wordList[i] = cache
            step += 1

        return 0

```

• LeetDoocs

```

class Solution:
    def ladderLength(self, beginWord: str, endWord: str, wordList: List[str])
-> int:
    words = set(wordList)
    q = deque([beginWord])
    ans = 1
    while q:
        ans += 1
        for _ in range(len(q)):
            s = q.popleft()
            s = list(s)

            for i in range(len(s)):
                ch = s[i]
                for j in range(26):
                    s[i] = chr(ord('a') + j)
                    t = ''.join(s)
                    if t not in words:
                        continue
                    if t == endWord:
                        return ans
                    q.append(t)

            words.remove(t)
            s[i] = ch

    return 0

```

• SimplyLeet

```

# Python solution using BFS

from collections import defaultdict, deque

def ladderLength(beginWord, endWord, wordList):
    # Create a set for fast lookup
    wordSet = set(wordList)
    if endWord not in wordSet:
        return 0

```

```
# Preprocessing: build a dictionary of intermediate states to words
L = len(beginWord)
all_combo_dict = defaultdict(list)
for word in wordSet:
    for i in range(L):
        # Key with a wildcard at the i-th position
        intermediate = word[:i] + '*' + word[i+1:]
        all_combo_dict[intermediate].append(word)

# Initialize BFS
```

```
queue = deque([(beginWord, 1)])
visited = set([beginWord])

while queue:
    current_word, level = queue.popleft()
    for i in range(L):
        intermediate = current_word[:i] + '*' + current_word[i+1:]
        # Check all words sharing the same intermediate state
        for word in all_combo_dict[intermediate]:
            if word == endWord:
```

```
                return level + 1
            if word not in visited:
                visited.add(word)
                queue.append((word, level + 1))
        # Once processed, clear the intermediate state list to save
        # processing time
        all_combo_dict[intermediate] = []
    return 0
```

```
# Example usage:
# print(ladderLength("hit", "cog", ["hot","dot","dog","lot","log","cog"]))
```

• LC.ca

```
# native BFS
class Solution:
    def ladderLength(self, beginWord: str, endWord: str, wordList: List[str])
-> int:
    words = set(wordList)
    q = deque([beginWord])
    ans = 1
    while q:
        ans += 1
        for _ in range(len(q)):
            s = q.popleft()
```

```

        s = list(s) # must convert to list, cannot directly update
s[i]='a'
        for i in range(len(s)):
            ch = s[i]
            for j in range(26):
                # will re-generate s itself
                # but s not in words-set, s removed after added to
words-set
                s[i] = chr(ord('a') + j)
                t = ''.join(s)
                if t not in words:
                    continue
```

```

                if t == endWord:
                    return ans
                q.append(t)
                words.remove(t) # equivalent to set t as visited
                s[i] = ch # restore
            return 0
```

```
#####
```



```

'''
BFS → BFS 問題を解くためのコード

1. 辞書を q1, q2 として初期化 → 辞書を初期化
2. 辞書 m1, m2 を初期化
3. 辞書を初期化
4. 辞書を初期化
'''

class Solution:
    def ladderLength(self, beginWord: str, endWord: str, wordList: List[str])
    -> int:

        def extend(m1, m2, q):
            for _ in range(len(q)):
                s = q.popleft() # so, every time, starting from start-word...
                # and later if t in m1 will skip repeated part...
                step = m1[s]
                s = list(s) # s = "abc", list(s) ==> ['a', 'b', 'c']
                for i in range(len(s)):
                    ch = s[i]
                    for j in range(26):
                        s[i] = chr(ord('a') + j)
                        t = ''.join(s)

                        if t in m1 or t not in words:
                            continue
                        if t in m2:
                            return step + 1 + m2[t]
                        m1[t] = step + 1
                        q.append(t)
                    s[i] = ch
            return -1

        words = set(wordList)

```

```

        if endWord not in words:
            return 0
        q1, q2 = deque([beginWord]), deque([endWord])
        m1, m2 = {beginWord: 0}, {endWord: 0}
        while q1 and q2:
            t = extend(m1, m2, q1) if len(q1) <= len(q2) else extend(m2, m1, q
2)

            if t != -1:
                return t + 1
        return 0

```

```
#####
```

```

import string
from collections import deque

```

```

class Solution(object):
    def ladderLength(self, beginWord, endWord, wordList):
        """

```

```
        :type beginWord: str

```

```

        :type endWord: str
        :type wordList: Set[str]
        :rtype: int
        """

```

```

    def getNbrs(src, dest, wordList):
        res = []
        for c in string.ascii_lowercase:
            for i in range(0, len(src)):
                newWord = src[:i] + c + src[i + 1:]

```

```

        if newWord == src:
            continue
        if newWord in wordList or newWord == dest:
            # about yield https://stackoverflow.com/questions/231767/what-does
            # -the-yield-keyword-do-in-python
            # yield is a keyword that is used like return, except the function
            # will return a generator.
            yield newWord

```

```

queue = deque([beginWord])
length = 0
while queue:

```

```

    length += 1
    for k in range(0, len(queue)):
        top = queue.popleft()
        for nbr in getNbrs(top, endWord, wordList):
            wordList.remove(nbr)
            if nbr == endWord:
                return length + 1
            queue.append(nbr)
    return 0

```

```

#####

```

```

while q1 and q2:
    if len(q1) <= len(q2):
        # Prioritize the queue with fewer elements for expansion
        extend(m1, m2, q1)
    else:
        extend(m2, m1, q2)

```

```
def extend(m1, m2, q):
    # New round of expansion
    for _ in range(len(q)):
        p = q.popleft()
        step = m1[p]
        for t in next(p):
            if t in m1:
                # Already visited before
                continue
            if t in m2:
                # The other direction has been searched, indicating that a
                # shortest path has been found
                return step + 1 + m2[t]
        q.append(t)
        m1[t] = step + 1
```

◆ Quick Review

Key Insights

- Use Breadth-First Search (BFS) to explore transformations level by level to guarantee the shortest path.
- Preprocess words by generating intermediate representations (e.g., replacing each letter with a wildcard) to quickly find neighbors.
- Use a visited set to avoid cycles.
- Each transformation counts as a level in the BFS, so the answer is the number of levels traversed.

Space & Time

Time Complexity: $O(N * M^2)$ where N is the number of words and M is the length of each word.

Space Complexity: $O(N * M)$ for the storage of intermediate mappings and the BFS queue.

Solution

We solve the problem using a BFS approach. First, we preprocess the wordList by creating a mapping of all possible intermediate states. For example, for the word "dog" and wildcard position, we generate " og", "d g", and "do*". This helps us find all adjacent words (neighbors) that are only one letter different in constant time. We then initialize a queue with the beginWord and initiate the BFS. For each word dequeued, we explore all valid transformations using the precomputed intermediate mapping. If the endWord is reached during this process, we return the current level (transformation sequence length). Otherwise, we continue the BFS until all possibilities are exhausted. If no valid transformation sequence is found, we return 0.

BFS

□ ★ #317 Shortest Distance from All Buildings ★

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · BFS · Matrix

Lists: ★ Premium | Premium 98

Problem

You are given an $m \times n$ grid grid of values 0, 1, or 2, where:

- each 0 marks an empty land that you can pass by freely,
- each 1 marks a building that you cannot pass through, and
- each 2 marks an obstacle that you cannot pass through.

You want to build a house on an empty land that reaches all buildings in the shortest total travel distance. You can only move up, down, left, and right.

Return the shortest travel distance for such a house. If it is not possible to build such a house

according to the above rules, return -1 .

The total travel distance is the sum of the distances between the houses of the friends and the meeting point.

Example 1:

1	0	2	0	1
0	0	0	0	0
0	0	1	0	0

Input: grid = `[[1,0,2,0,1],[0,0,0,0,0],[0,0,1,0,0]]`

Output: 7

Explanation: Given three buildings at $(0,0)$, $(0,4)$, $(2,2)$, and an obstacle at $(0,2)$.

The point $(1,2)$ is an ideal empty land to build a house, as the total travel distance of $3+3+1=7$ is minimal.

So return 7.

Example 2:

Input: grid = `[[1,0]]`

Output: 1

Example 3:

Input: grid = [[1]]

Output: -1

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 50$
- $\text{grid}[i][j]$ is either 0, 1, or 2.
- There will be at least one building in the grid.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Find empty land (0) minimising total BFS distance to all buildings (1).

Obstacles (2) can't be passed. Return -1 if no valid land. Right?"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For each empty cell, BFS to sum distances to all buildings.

Track total distance per cell; answer is cell with minimum total (must reach all buildings).

Repeat BFS from every building - $O(B * m * n)$ where B = buildings.

Time: $O(B * m * n)$. Space: $O(m * n)$ BFS state.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (BFS from each building, accumulate distances)

"BFS from each building cell. For each empty cell, accumulate distance and increment reach_count.

Only cells reachable from ALL buildings are candidates. Return min total distance."

PHASE 4 — CLEAN CODE

```
class _317_ShortestDistanceBuildings:
    def shortestDistance(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        dist = [[0] * n for _ in range(m)]
```

```

reach = [[0] * n for _ in range(m)]
buildings = sum(grid[r][c] == 1 for r in range(m) for c in range(n))
for r in range(m):
    for c in range(n):
        if grid[r][c] != 1: continue
        queue = deque([(r, c, 0)])
        visited = {(r, c)}
        while queue:
            cr, cc, d = queue.popleft()
            for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
                nr, nc = cr + dr, cc + dc
                if 0 <= nr < m and 0 <= nc < n and grid[nr][nc] == 0 and
(nr,nc) not in visited:
                    visited.add((nr, nc))
                    dist[nr][nc] += d + 1
                    reach[nr][nc] += 1
                    queue.append((nr, nc, d + 1))

        return min(
            (dist[r][c] for r in range(m) for c in range(n) if grid[r][c] == 0 and
reach[r][c] == buildings),
            default=-1
        )

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\text{buildings} * m * n)$. Space $O(m*n)$. BFS from each building accumulates into `dist[]`.

`reach[]` ensures we only pick land reachable from ALL buildings.

Given more time I'd ask: what if buildings can move? Re-run BFS on each query."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def shortestDistance(self, grid: list[list[int]]) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))
        m = len(grid)
        n = len(grid[0])
        nBuildings = sum(a == 1 for row in grid for a in row)
        ans = math.inf
        # dist[i][j] := the total distance of grid[i][j] (0) to reach all the
        # buildings (1)
        dist = [[0] * n for _ in range(m)]

```

```
# reachCount[i][j] := the number of buildings (1) grid[i][j] (0) can reach
reachCount = [[0] * n for _ in range(m)]
```

```
def bfs(row: int, col: int) -> bool:
    q = collections.deque([(row, col)])
    seen = {(row, col)}
    seenBuildings = 1
```

```
    step = 1
    while q:
```

```
        for _ in range(len(q)):
            i, j = q.popleft()
            for dx, dy in DIRS:
                x = i + dx
                y = j + dy
                if x < 0 or x == m or y < 0 or y == n:
                    continue
                if (x, y) in seen:
                    continue
                seen.add((x, y))
```

```
                if not grid[x][y]:
                    dist[x][y] += step
                    reachCount[x][y] += 1
                    q.append((x, y))
                elif grid[x][y] == 1:
                    seenBuildings += 1
            step += 1
```

```
# True if all the buildings (1) are connected
return seenBuildings == nBuildings
```

```
for i in range(m):
    for j in range(n):
        if grid[i][j] == 1: # BFS from this building.
            if not bfs(i, j):
                return -1
```

```
for i in range(m):
    for j in range(n):
        if reachCount[i][j] == nBuildings:
```

```

        ans = min(ans, dist[i][j])

    return -1 if ans == math.inf else ans

```

• LeetDoocs

```

class Solution:
    def shortestDistance(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        q = deque()
        total = 0
        cnt = [[0] * n for _ in range(m)]
        dist = [[0] * n for _ in range(m)]
        for i in range(m):
            for j in range(n):
                if grid[i][j] == 1:

                    total += 1
                    q.append((i, j))
                    d = 0
                    vis = set()
                    while q:
                        d += 1
                        for _ in range(len(q)):
                            r, c = q.popleft()
                            for a, b in [[0, 1], [0, -1], [1, 0], [-1, 0]]:
                                x, y = r + a, c + b

                                if (
                                    0 <= x < m
                                    and 0 <= y < n
                                    and grid[x][y] == 0
                                    and (x, y) not in vis
                                ):
                                    cnt[x][y] += 1
                                    dist[x][y] += d
                                    q.append((x, y))
                                    vis.add((x, y))

```

```

ans = inf
for i in range(m):
    for j in range(n):
        if grid[i][j] == 0 and cnt[i][j] == total:
            ans = min(ans, dist[i][j])
return -1 if ans == inf else ans

```

• SimplyLeet

```

from collections import deque

def shortestDistance(grid):
    if not grid or not grid[0]:
        return -1

    m, n = len(grid), len(grid[0])
    total_buildings = 0
    total_distance = [[0] * n for _ in range(m)]
    reachable_count = [[0] * n for _ in range(m)]

    def bfs(start_i, start_j):
        visited = [[False] * n for _ in range(m)]
        q = deque([(start_i, start_j, 0)])
        visited[start_i][start_j] = True

        while q:
            i, j, dist = q.popleft()
            for x, y in [(i-1, j), (i+1, j), (i, j-1), (i, j+1)]:
                if 0 <= x < m and 0 <= y < n and not visited[x][y] and grid[x]
[y] == 0:

                    visited[x][y] = True
                    total_distance[x][y] += dist + 1
                    reachable_count[x][y] += 1
                    q.append((x, y, dist+1))

    for i in range(m):
        for j in range(n):
            if grid[i][j] == 1:
                total_buildings += 1
                bfs(i, j)

```

```

min_distance = float('inf')
for i in range(m):
    for j in range(n):
        if grid[i][j] == 0 and reachable_count[i][j] == total_buildings:
            min_distance = min(min_distance, total_distance[i][j])

return min_distance if min_distance != float('inf') else -1

```

Example test:

```
print(shortestDistance([[1,0,2,0,1],[0,0,0,0,0],[0,0,1,0,0]]))
```

• LC.ca

```

class Solution:
    def shortestDistance(self, grid: List[List[int]]) -> int:
        m, n = len(grid), len(grid[0])
        q = deque()
        total = 0 # total building count
        # cnt: how many buildings can reach (x,y)
        # if there is a column all blockers, then (x,y) cannot reach every
        building
        cnt = [[0] * n for _ in range(m)]
        dist = [[0] * n for _ in range(m)]
        for i in range(m):
            for j in range(n):
                if grid[i][j] == 1:
                    total += 1
                    q.append((i, j))
                    d = 0
                    vis = set() # reset for every free-land
                    while q:
                        d += 1
                        for _ in range(len(q)):
                            r, c = q.popleft()

```

```

        for a, b in [[0, 1], [0, -1], [1, 0], [-1, 0]]:
            x, y = r + a, c + b
            if (
                0 <= x < m
                and 0 <= y < n
                and grid[x][y] == 0
                and (x, y) not in vis
            ):
                cnt[x][y] += 1
                dist[x][y] += d

            q.append((x, y))
            vis.add((x, y))

ans = inf
for i in range(m):
    for j in range(n):
        if grid[i][j] == 0 and cnt[i][j] == total:
            ans = min(ans, dist[i][j])
return -1 if ans == inf else ans

```

◆ Quick Review

Key Insights

- We need to compute the Manhattan distance from each building (cell value 1) to all empty lands (cell value 0) by performing a BFS.
- Maintain two matrices: one for cumulative total distances and another for the count of buildings that reached a particular empty cell.
- Ensure that only those empty cells reachable by every building are considered.
- The ideal cell will have a reach count equal to the total number of buildings.

- If multiple buildings cannot all reach any specific empty cell, then return -1 .

Space & Time

Time Complexity: $O(k * m * n)$ where k is the number of buildings and m, n are the grid dimensions.

Space Complexity: $O(m * n)$ due to the auxiliary matrices and visited state tracking used in BFS.

Solution

We first iterate through the grid to locate all buildings while counting them. For each building, we perform a BFS to compute the shortest distance to each reachable empty cell. During the BFS, we update the cumulative distance for that cell and record that it was reached by one additional building. After processing all the buildings, we scan the grid to find the empty cell that is reachable from all buildings and has the minimum cumulative distance. If no such cell exists, we return -1 .

BFS

□ #773 Sliding Puzzle

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · BFS · Matrix
Lists: Denny Zhang

Problem

On an 2×3 board, there are five tiles labeled from 1 to 5, and an empty square represented by 0. A move consists of choosing 0 and a 4-directionally adjacent number and swapping it.

The state of the board is solved if and only if the board is $[[1,2,3],[4,5,0]]$.

Given the puzzle board `board`, return the least number of moves required so that the state of the board is solved. If it is impossible for the state of the board to be solved, return -1 .

Example 1:

1	2	3
4		5

Input: board = [[1,2,3],[4,0,5]]

Output: 1

Explanation: Swap the 0 and the 5 in one move.

Example 2:

1	2	3
5	4	

Input: board = [[1,2,3],[5,4,0]]

Output: -1

Explanation: No number of moves will make the board solved.

Example 3:

4	1	2
5		3

Input: board = [[4,1,2],[5,0,3]]

Output: 5

Explanation: 5 is the smallest number of moves that solves the board.

An example path:

After move 0: [[4,1,2],[5,0,3]]

After move 1: [[4,1,2],[0,5,3]]

After move 2: [[0,1,2],[4,5,3]]

After move 3: [[1,0,2],[4,5,3]]

Constraints:

- board.length == 2
- board[i].length == 3
- $0 \leq \text{board}[i][j] \leq 5$
- Each value board[i][j] is unique.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "2x3 board. Swap 0 with adjacent tile. Minimum moves to reach [[1,2,3],[4,5,0]].
Right?
# Return -1 if unsolvable."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# BFS on board states: encode 2x3 board as tuple string; enqueue all legal swaps of
'0'.
# First time we reach target state, return move count.
# State space is at most 6! = 720 permutations.
# Time: O(6!). Space: O(6!) visited states.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE (BFS on serialised board state)

```
# "Represent board as a 6-char string. BFS on states.
# Precompute which positions in the flat string can swap with position i (adjacency
map).
# Goal state: '123450'."
```

PHASE 4 — CLEAN CODE

```
class _773_SlidingPuzzle:
    def slidingPuzzle(self, board: List[List[int]]) -> int:
        GOAL = '123450'
        NEIGHBORS = {0:[1,3], 1:[0,2,4], 2:[1,5], 3:[0,4], 4:[1,3,5], 5:[2,4]}
        start = ''.join(str(board[r][c]) for r in range(2) for c in range(3))
        queue = deque([(start, 0)])
        visited = {start}
        while queue:
            state, moves = queue.popleft()
            if state == GOAL:
                return moves
            z = state.index('0')
            for nb in NEIGHBORS[z]:
                new_state = list(state)
                new_state[z], new_state[nb] = new_state[nb], new_state[z]
                ns = ''.join(new_state)
                if ns not in visited:
                    visited.add(ns)
                    queue.append((ns, moves + 1))
        return -1
```

PHASE 5 — TEST & DEBUG

```
# [[1,2,3],[4,0,5]] -> '123405'. Swap 0 at pos 4 with 5 at pos 5 -> '123450' = GOAL ->
1 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "States = 6! / 2 = 360 reachable states (half of 6! are unsolvable). BFS on 360 states.
# The hardcoded NEIGHBORS map is the key – it captures the 2D adjacency in 1D string indexing.
# Given more time I'd ask: generalise to n×m board?
# Same BFS approach – NEIGHBORS changes, state space = (n*m)! / 2."
```

Community Solutions (Python)

• SimplyLeet

```
import collections

def slidingPuzzle(board):
    # Flatten board into a string representation.
    start = ''.join(str(num) for row in board for num in row)
    goal = "123450"
    # Mapping of indices to their neighbors in a 2x3 board.
    neighbors = {0: [1, 3], 1: [0, 2, 4], 2: [1, 5],
                 3: [0, 4], 4: [1, 3, 5], 5: [2, 4]}

    # Initialize BFS with the starting state and move count 0.
    q = collections.deque([(start, 0)])
    visited = set([start])

    # Process states in the queue.
    while q:
        state, moves = q.popleft()
        if state == goal:
            return moves
        zero_index = state.index('0')

        # Try all valid moves by swapping 0 with its neighbours.
        for neighbor in neighbors[zero_index]:
            new_state = list(state)
            new_state[zero_index], new_state[neighbor] = new_state[neighbor],
            new_state[zero_index]
            new_state_str = ''.join(new_state)
            if new_state_str not in visited:
                visited.add(new_state_str)
                q.append((new_state_str, moves + 1))

    return -1
```

```
# Example usage:
print(slidingPuzzle([[1,2,3],[4,0,5]]))
```

• LC.ca

```
class Solution:
    def slidingPuzzle(self, board: List[List[int]]) -> int:
        m, n = 2, 3
        seq = []
        start, end = '', '123450'
        for i in range(m):
            for j in range(n):
                if board[i][j] != 0:
                    seq.append(board[i][j])
                    start += str(board[i][j])

        def check(seq):
            n = len(seq)
            cnt = sum(seq[i] > seq[j] for i in range(n) for j in range(i, n))
            return cnt % 2 == 0

        def f(s):
            ans = 0
            for i in range(m * n):
                if s[i] != '0':
                    num = ord(s[i]) - ord('1')
                    ans += abs(i // n - num // n) + abs(i % n - num % n)
            return ans

        if not check(seq):
            return -1
        q = [(f(start), start)]
        dist = {start: 0}
        while q:
            _, state = heappop(q)
```



```
if state == end:
    return dist[state]
p1 = state.index('0')
i, j = p1 // n, p1 % n
s = list(state)
for a, b in [[0, -1], [0, 1], [1, 0], [-1, 0]]:
    x, y = i + a, j + b
    if 0 <= x < m and 0 <= y < n:
        p2 = x * n + y
        s[p1], s[p2] = s[p2], s[p1]

    next = ''.join(s)
    s[p1], s[p2] = s[p2], s[p1]
    if next not in dist or dist[next] > dist[state] + 1:
        dist[next] = dist[state] + 1
        heappush(q, (dist[next] + f(next), next))

return -1
```

◆ Quick Review

Key Insights

- Represent the board as a string (e.g. "123450") to simplify state comparison and manipulation.
- Use Breadth First Search (BFS) to explore all possible moves since the puzzle has a small fixed state space ($6! = 720$ possible states).
- Precompute the valid neighbor indices for each board position to efficiently generate new states.
- Track visited states to avoid loops and redundant work.

Space & Time

Time Complexity: $O(1)$ in practice, since there are at most 720 states ($6!$ permutations).

Space Complexity: $O(1)$ for the same reason, as the state space is fixed and limited.

Solution

We solve the problem using a Breadth First Search (BFS) approach. The board configuration is flattened into a string to represent states. A mapping from each index to its possible neighbors (i.e., positions to which 0 can move) is precomputed. BFS is then used to explore every valid move from the initial state while marking states as visited to avoid cycles. When the goal state ("123450") is reached, we return the number of moves taken. If BFS completes without reaching the goal, the puzzle is unsolvable and we return -1 .

BFS

□ #815 Bus Routes

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · BFS
Lists: Grind 169

Problem

You are given an array `routes` representing bus routes where `routes[i]` is a bus route that the *i*th bus repeats forever.

- For example, if `routes[0] = [1, 5, 7]`, this means that the 0th bus travels in the sequence `1 -> 5 -> 7 -> 1 -> 5 -> 7 -> 1 -> ...` forever.

You will start at the bus stop `source` (You are not on any bus initially), and you want to go to the bus stop `target`. You can travel between bus stops by buses only.

Return the least number of buses you must take to travel from `source` to `target`. Return `-1` if it is not possible.

Example 1:

Input: routes = [[1,2,7],[3,6,7]], source = 1, target = 6

Output: 2

Explanation: The best strategy is take the first bus to the bus stop 7, then take the second bus to the bus stop 6.

Example 2:

Input: routes = [[7,12],[4,5,15],[6],[15,19],[9,12,13]], source = 15, target = 12

Output: -1

Constraints:

- $1 \leq \text{routes.length} \leq 500$.
- $1 \leq \text{routes}[i].\text{length} \leq 105$
- All the values of routes[i] are unique.
- $\text{sum}(\text{routes}[i].\text{length}) \leq 105$
- $0 \leq \text{routes}[i][j] < 106$
- $0 \leq \text{source}, \text{target} < 106$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Minimum number of BUSES (not stops) to travel from source to target. Right?

Each bus covers its entire route – stepping onto a bus travels all its stops."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Build stop→routes map. BFS from source stop; each hop can board any route containing current stop.

Track min buses to reach target; mark routes visited to avoid reboarding.

Time: $O(R * S)$ routes and stops. Space: $O(R + S)$.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE (BFS on routes, not stops)

"Build stop→routes map. BFS where each 'node' is a bus route.

Start: all routes containing source. Each step: board a new route (adjacent if sharing a stop).
 # Count route changes (buses taken). Avoid revisiting routes and stops."

PHASE 4 — CLEAN CODE

```
class _815_BusRoutes:
    def numBusesToDestination(self, routes: List[List[int]], source: int, target:
int) -> int:
        if source == target:
            return 0
        stop_to_routes: defaultdict = defaultdict(set)
        for i, route in enumerate(routes):
            for stop in route:
                stop_to_routes[stop].add(i)
        visited_stops = {source}
        visited_routes = set()
        queue = deque([(source, 0)]) # (stop, buses_taken)
        while queue:
            stop, buses = queue.popleft()
            for route_id in stop_to_routes[stop]:
                if route_id in visited_routes: continue
                visited_routes.add(route_id)
                for next_stop in routes[route_id]:
                    if next_stop == target:
                        return buses + 1
                    if next_stop not in visited_stops:
                        visited_stops.add(next_stop)
                        queue.append((next_stop, buses + 1))
        return -1
```

PHASE 5 — TEST & DEBUG

routes=[[1,2,7],[3,6,7]], source=1, target=6:
 # queue=[(1,0)]. route 0 for stop 1: stops 2,7. Not target. queue=[(2,1),(7,1)].
 # stop 7: route 1 for stop 7: stops 3,6. 6==target → return 1+1=2 ✓

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time O(sum of route lengths). Space O(same). BFS on routes, not stops – the key insight.
 # Visiting stops prevents re-enqueuing; visiting routes prevents re-processing entire routes.
 # Given more time I'd ask: find all paths using at most k buses?
 # Keep track of bus count per path – BFS naturally handles this."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def numBusesToDestination(
        self,
        routes: list[list[int]],
        source: int,
        target: int,
    ) -> int:
        if source == target:
            return 0

        graph = collections.defaultdict(list)
        usedBuses = set()

        for i in range(len(routes)):
            for route in routes[i]:
                graph[route].append(i)

        q = collections.deque([source])

        step = 1

        while q:
            for _ in range(len(q)):
                for bus in graph[q.popleft()]:
                    if bus in usedBuses:
                        continue
                    usedBuses.add(bus)
                    for nextRoute in routes[bus]:
                        if nextRoute == target:
                            return step
                        q.append(nextRoute)

            step += 1

        return -1
```

• LeetDoocs

```

class Solution:
    def numBusesToDestination(
        self, routes: List[List[int]], source: int, target: int
    ) -> int:
        if source == target:
            return 0
        g = defaultdict(list)
        for i, route in enumerate(routes):
            for stop in route:
                g[stop].append(i)

        if source not in g or target not in g:
            return -1
        q = [(source, 0)]
        vis_bus = set()
        vis_stop = {source}
        for stop, bus_count in q:
            if stop == target:
                return bus_count
            for bus in g[stop]:
                if bus not in vis_bus:
                    vis_bus.add(bus)
                    for next_stop in routes[bus]:
                        if next_stop not in vis_stop:
                            vis_stop.add(next_stop)
                            q.append((next_stop, bus_count + 1))

        return -1

```

• SimplyLeet

```

from collections import defaultdict, deque

def numBusesToDestination(routes, source, target):
    # If source equals target, no buses are needed.
    if source == target:
        return 0

    # Build mapping: bus stop -> list of bus indices
    stop_to_buses = defaultdict(list)
    for bus_index, stops in enumerate(routes):

```



```

        for stop in stops:
            stop_to_buses[stop].append(bus_index)

visited_buses = set()
visited_stops = set([source])
queue = deque()

# Initialize queue with all buses that include the source stop.
for bus in stop_to_buses[source]:
    visited_buses.add(bus)

    queue.append((bus, 1))

# Perform BFS over bus routes.
while queue:
    current_bus, buses_taken = queue.popleft()
    for stop in routes[current_bus]:
        if stop == target:
            return buses_taken
        if stop not in visited_stops:
            visited_stops.add(stop)

            for neighbor_bus in stop_to_buses[stop]:
                if neighbor_bus not in visited_buses:
                    visited_buses.add(neighbor_bus)
                    queue.append((neighbor_bus, buses_taken + 1))

    return -1

# Example usage:
# routes = [[1,2,7],[3,6,7]]
# print(numBusesToDestination(routes, 1, 6))

```

• LC.ca

```
class Solution:
    def numBusesToDestination(
        self, routes: List[List[int]], source: int, target: int
    ) -> int:
        if source == target:
            return 0

        s = [set(r) for r in routes]
        d = defaultdict(list)
        for i, r in enumerate(routes):

            for v in r:
                d[v].append(i)

        g = defaultdict(list)
        for ids in d.values():
            m = len(ids)
            for i in range(m):
                for j in range(i + 1, m):
                    a, b = ids[i], ids[j]
                    g[a].append(b)

                    g[b].append(a)

        q = deque(d[source])
        ans = 1
        vis = set(d[source])
        while q:
            for _ in range(len(q)):
                i = q.popleft()
                if target in s[i]:
                    return ans
                for j in g[i]:

                    if j not in vis:
                        vis.add(j)
                        q.append(j)

            ans += 1
        return -1
```

◆ Quick Review

Key Insights

- Model the problem as a graph where nodes are bus routes.
- Two bus routes are connected if they share at least one common bus stop.
- Use a hash map to quickly look up bus routes available at any bus stop.
- Employ Breadth-First Search (BFS) on the bus routes graph to find the minimum number of bus transfers.
- Mark visited bus routes and bus stops to avoid redundant work.

Space & Time

Time Complexity: $O(N * L)$ in the worst-case scenario, where N is the number of bus routes and L is the average number of stops per route.

Space Complexity: $O(N * L)$ due to storage of the stop-to-buses mapping and BFS auxiliary data structures.

Solution

We begin by constructing a mapping from each bus stop to the list of buses passing through

it. This allows quick retrieval of buses available at a given stop. Starting at the source stop, initialize the BFS queue with all buses that can be boarded at that stop. Then, for each bus route in the queue, inspect each stop along that route. If a stop is the target, return the number of buses taken so far. Otherwise, for each unvisited stop, add all adjacent bus routes (buses that pass through this stop) to the BFS queue with an incremented bus count. Continue until the queue is empty, which means the target cannot be reached.

Round 4 · Mid · Easy

Round 4

Mid Priority · Easy

12 questions · 4 patterns

Linked List #21 #141 #206 #234 #706 #876
#1474

Stack #20 #232 #844

Trie #14

Greedy #409

Linked List

Round 4 · Mid · Easy · 7 questions

Linked List

□ #21 Merge Two Sorted Lists

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Recursion
Lists: Grind 169

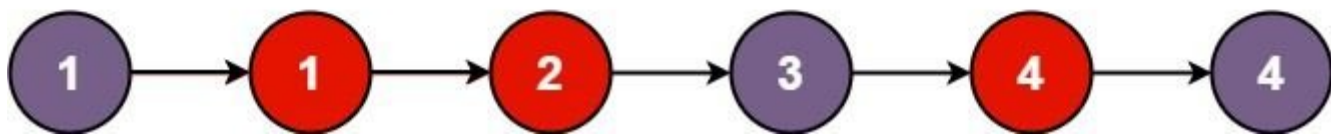
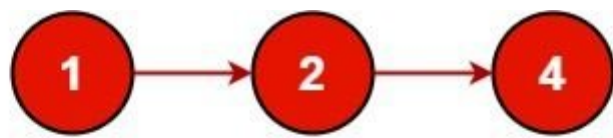
Problem

You are given the heads of two sorted linked lists list1 and list2.

Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists.

Return the head of the merged linked list.

Example 1:



Input: list1 = [1,2,4], list2 = [1,3,4]
Output: [1,1,2,3,4,4]

Example 2:

Input: list1 = [], list2 = []
Output: []

Example 3:

Input: list1 = [], list2 = [0]
Output: [0]

Constraints:

- The number of nodes in both lists is in the range [0, 50].
- $-100 \leq \text{Node.val} \leq 100$
- Both list1 and list2 are sorted in non-decreasing order.

My LeetCode Solution

- ☐
- My LeetCode Solution

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def mergeTwoLists(self, list1: Optional[ListNode], list2: Optional[ListNode]) -> Optional[ListNode]:
        d = ListNode()
        c = d
        while list1 and list2:
            if list1.val <= list2.val:
                c.next = list1
                list1 = list1.next
            else:
                c.next = list2
                list2 = list2.next
            c = c.next

        c.next = list1 if list1 else list2
        return d.next

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

We have two sorted linked lists and need to merge them into one sorted list

by re-linking the original nodes – no new allocations.

Return the new head. Right?"

#

"A few quick questions:

Can either list be empty? Yes – constraints say 0 nodes is valid.

Are duplicates across lists allowed? Yes.

Must the result be stable (preserve relative order of equal elements)? Yes, standard merge."

#

"Let me walk the example: list1=[1,2,4], list2=[1,3,4].

Pick 1 (tie → list1 first), then 1 from list2, then 2, 3, 4, 4. → [1,1,2,3,4,4] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Collect all values from both lists into a Python list, sort it, then rebuild

a brand-new linked list from those sorted values.

This is $O((m+n) \log(m+n))$ for the sort and $O(m+n)$ extra space for the values.

```
# It completely throws away the fact that both inputs are already sorted –
# there is zero benefit from sorting again.
# Should I go ahead and optimize?"
```

```
class _21_MergeTwoSortedLists_Brute:
    def mergeTwoLists(self, list1, list2):
        vals = []
        while list1: vals.append(list1.val); list1 = list1.next
        while list2: vals.append(list2.val); list2 = list2.next
        dummy = cur = ListNode(0)
        for v in sorted(vals):
            cur.next = ListNode(v); cur = cur.next
        return dummy.next
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the sort – we paid  $O(\log n)$  we didn't need.
# Since both lists are already sorted, the smallest available element is always
# at one of the two heads. We just pick the smaller head at each step.
#
# Pseudocode:
# dummy = cur = ListNode(0)
# while list1 and list2:
#   if list1.val <= list2.val: attach list1, advance list1
#   else: attach list2, advance list2
#   advance cur
#   attach whichever list remains
#   return dummy.next
#
# Time:  $O(m+n)$  – one comparison per node. Space:  $O(1)$  – only pointer rewiring.
# Does that make sense before I code it?"
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _21_MergeTwoSortedLists:
    def mergeTwoLists(self, list1, list2):
        dummy = cur = ListNode(0)
        while list1 and list2:
            if list1.val <= list2.val:
                cur.next = list1
                list1 = list1.next
            else:
                cur.next = list2
                list2 = list2.next
            cur = cur.next
        cur.next = list1 if list1 else list2
        return dummy.next
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# list1=[1,2,4], list2=[1,3,4]:
```

```
# 1<=1 → take l1(1), l1→2. 2>1 → take l2(1), l2→3. 2<=3 → take l1(2), l1→4.
# 4>3 → take l2(3), l2→4. 4<=4 → take l1(4), l1=None.
# cur.next = l2=[4]. → [1,1,2,3,4,4] ✓
#
# list1=[], list2=[0]:
# while loop never enters. cur.next = list2=[0]. → [0] ✓
#
# "No bugs. Dummy node cleanly handles the empty-list edge cases."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m+n) – one comparison per node. Space O(1) – reuses existing nodes.
# The dummy head is the key trick: we never need to special-case the very first
# node.
# Follow-up: Merge k Sorted Lists (#23) – use a min-heap for O(n log k) time.
# Follow-up: Sort a Linked List (#148) – apply this same merge step inside merge
# sort."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def mergeTwoLists(
        self,
        list1: ListNode | None,
        list2: ListNode | None,
    ) -> ListNode | None:
        if not list1 or not list2:
            return list1 if list1 else list2
        if list1.val > list2.val:
            list1, list2 = list2, list1

        list1.next = self.mergeTwoLists(list1.next, list2)
        return list1
```

• LeetDoocs

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
class Solution:
    def mergeTwoLists(
        self, list1: Optional[ListNode], list2: Optional[ListNode]
    ) -> Optional[ListNode]:
        if list1 is None or list2 is None:

            return list1 or list2
        if list1.val <= list2.val:
            list1.next = self.mergeTwoLists(list1.next, list2)
            return list1
        else:
            list2.next = self.mergeTwoLists(list1, list2.next)
            return list2
```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def mergeTwoLists(list1: ListNode, list2: ListNode) -> ListNode:
    # Create a dummy node to serve as the start of the merged list
    dummy = ListNode(-1)
    current = dummy

    # Iterate while both lists have nodes
    while list1 and list2:
        # Compare the values of the current nodes of both lists
        if list1.val <= list2.val:
            current.next = list1 # Link the smaller node to the merged list
            list1 = list1.next   # Move the pointer in list1
        else:
            current.next = list2
            list2 = list2.next
```

```

        current = current.next    # Move the pointer in the merged list

    # Once one list is exhausted, attach the remaining nodes from the other
    list
    if list1:
        current.next = list1
    else:
        current.next = list2

    # The merged list head is at dummy.next
    return dummy.next

```

• LC.ca

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
# without ops after while loop
class Solution:
    def mergeTwoLists(self, list1: Optional[ListNode], list2: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode()
        current = dummy

        while list1 or list2:
            v1 = list1.val if list1 else float('inf')
            v2 = list2.val if list2 else float('inf')

            if v1 < v2:
                current.next = list1
                list1 = list1.next
            else:
                current.next = list2
                list2 = list2.next

```

```

        current = current.next

    return dummy.next

#####

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def mergeTwoLists(
        self, list1: Optional[ListNode], list2: Optional[ListNode]
    ) -> Optional[ListNode]:
        dummy = ListNode()
        curr = dummy
        while list1 and list2:
            if list1.val <= list2.val:
                curr.next = list1
                list1 = list1.next
            else:
                curr.next = list2
                list2 = list2.next
            curr = curr.next
        curr.next = list1 or list2
        return dummy.next

#####

curr.next = list1 or list2

```

better than:

```

if list1:
    current.next = list1
if list2:
    current.next = list2

```

```

#####

```

```
#####

# recursion
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:

    def mergeTwoLists(self, l1: ListNode, l2: ListNode) -> ListNode:
        if not l1: # If l1 is empty, return l2
            return l2
        if not l2: # If l2 is empty, return l1
            return l1

        if l1.val < l2.val:
            l1.next = self.mergeTwoLists(l1.next, l2)
            return l1
        else:

            l2.next = self.mergeTwoLists(l1, l2.next)
            return l2
```

◆ Quick Review

Key Insights

- Leverage the fact that both lists are sorted.
- Use two pointers (one for each list) to compare nodes sequentially.
- Create a dummy node to simplify edge cases and build the resulting list.
- Iterate until one list is exhausted, then append the remaining nodes from the other

list.

- There is also a recursive approach that can simplify the logic but may use more stack space.

Space & Time

Time Complexity: $O(n + m)$, where n and m are the lengths of the two lists.

Space Complexity: $O(1)$ for the iterative solution ($O(n + m)$ stack space for the recursive solution).

Solution

We use an iterative approach with a dummy node to merge two sorted linked lists. Two pointers traverse list1 and list2; at each step, the node with the smaller value is appended to our merged list. This process repeats until one of the lists is exhausted, after which we append the remaining nodes from the other list. This approach works in one pass over both lists, ensuring linear time complexity and constant extra space usage.

Linked List

□ #141 Linked List Cycle

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Linked List · Two Pointers
Lists: Grind 169

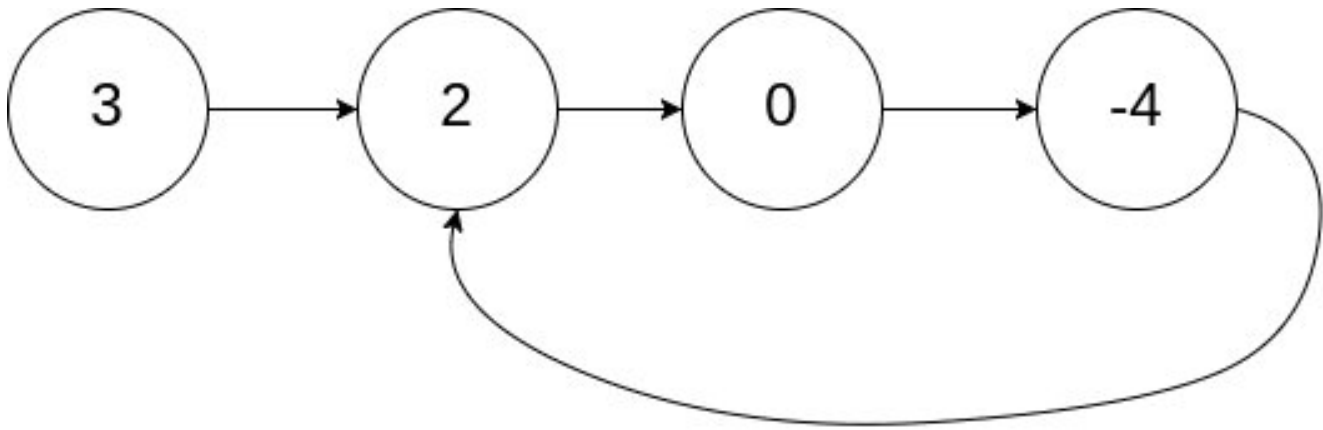
Problem

Given head, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer.

Internally, pos is used to denote the index of the node that tail's next pointer is connected to. Note that pos is not passed as a parameter. Return true if there is a cycle in the linked list. Otherwise, return false.

Example 1:

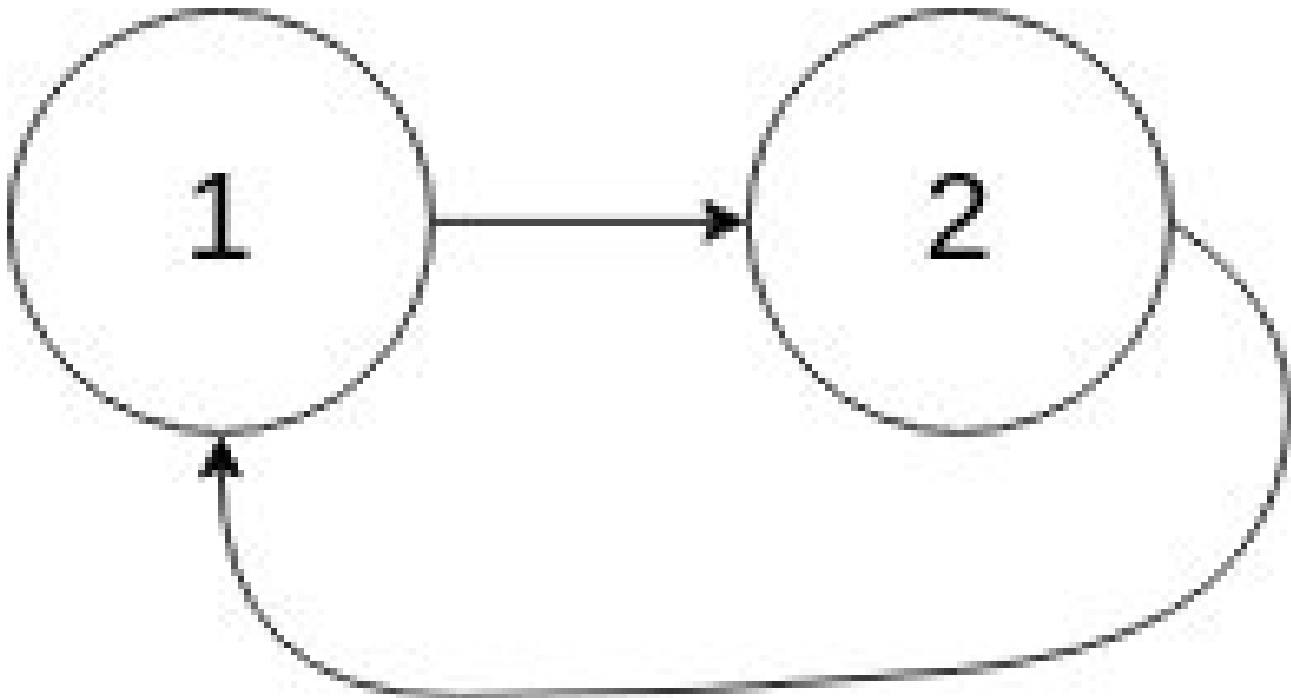


Input: head = [3,2,0,-4], pos = 1

Output: true

Explanation: There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).

Example 2:

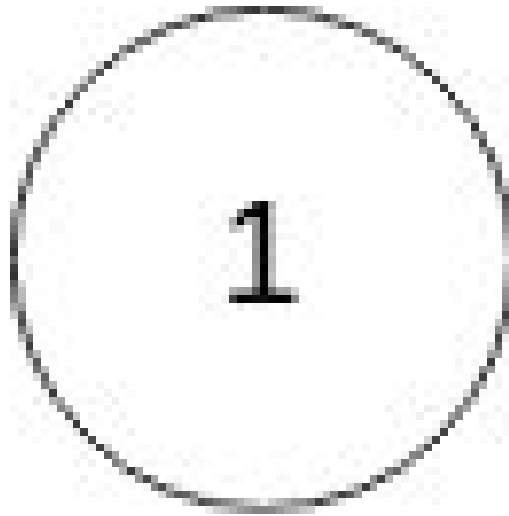


Input: head = [1,2], pos = 0

Output: true

Explanation: There is a cycle in the linked list, where the tail connects to the 0th node.

Example 3:



Input: head = [1], pos = -1

Output: false

Explanation: There is no cycle in the linked list.

Constraints:

- The number of the nodes in the list is in the range $[0, 10^4]$.
- $-105 \leq \text{Node.val} \leq 105$
- pos is -1 or a valid index in the linked-list.

Follow up: Can you solve it using $O(1)$ (i.e. constant) memory?

My LeetCode Solution

- My LeetCode Solution

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None

class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        slow = fast = head
        while fast and fast.next:

            fast = fast.next.next
            slow = slow.next

            if slow == fast:
                return True

        return False
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# We have a singly linked list and need to return true if there is a cycle –
# meaning some node's next pointer points back to a previous node.
# Return false if we reach None (no cycle). Right?"
#
# "A few quick questions:
# Is pos given to us in the actual function? No – pos is just for test descriptions.
# Can the list have one node pointing to itself? Yes – that's a cycle too.
# Can the list be empty? Yes – empty list has no cycle."
#
# "Let me walk the example: [3,2,0,-4], tail connects back to index 1.
# 3→2→0→-4→2→... cycles forever → true ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Walk the list and store every visited node in a hash set.
# If we encounter a node already in the set, there's a cycle.
# If we reach None, there isn't.
# This is O(n) time but O(n) extra space for the set.
# The follow-up asks for O(1) space – that's what I'll aim for.
# Should I go ahead and optimize?"
class _141_LinkedListCycle_Brute:
    def hasCycle(self, head):
```

```

seen = set()
cur = head
while cur:
    if id(cur) in seen:
        return True
    seen.add(id(cur))
    cur = cur.next
return False

```

PHASE 3 — OPTIMIZE

```

# "The bottleneck is the O(n) extra space for the visited set.
# Floyd's tortoise-and-hare: use two pointers – slow moves 1 step, fast moves 2.
# If a cycle exists, fast will eventually lap slow and they'll meet inside the
cycle.
# If no cycle, fast reaches None first.
#
# Pseudocode:
# slow = fast = head
# while fast and fast.next:
#     slow = slow.next; fast = fast.next.next
# if slow is fast: return True
# return False
#
# Time: O(n). Space: O(1) – only two pointer variables.
# Does that make sense before I code it?"

```

PHASE 4 — CLEAN CODE

```

# "Now I'll implement the optimized approach with meaningful names."
class _141_LinkedListCycle:
    def hasCycle(self, head):
        slow = fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
            if slow is fast:
                return True
        return False

```

PHASE 5 — TEST & DEBUG

```

# "Let me trace through a few cases."
#
# [3,2,0,-4], cycle at index 1 (slow/fast both start at 3):
# step1: slow=2, fast=0. step2: slow=0, fast=2. step3: slow=-4, fast=-4. Match →
True ✓
#
# [1,2], no cycle:
# step1: slow=2, fast=None. fast is None → loop exits → False ✓
#
# [1], no cycle:
# fast.next is None → loop never enters → False ✓

```

```
#
# "No bugs. Guard 'fast and fast.next' is essential to avoid NoneType errors."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): fast covers the cycle in at most n steps after entering it.
# Space O(1): only two pointer variables.
# Hash-set approach is O(n) space – Floyd's is the canonical O(1) improvement.
# Follow-up: Find Cycle Start (#142) – after they meet, reset slow to head,
# advance both one step at a time; they meet again at the cycle entry.
# Follow-up: cycle length – once they meet, keep fast moving and count steps until
it returns."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def hasCycle(self, head: ListNode) -> bool:
        slow = head
        fast = head

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
            if slow == fast:
                return True

        return False
```

• LeetDoocs

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None

class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        s = set()
```



```

while head:
    if head in s:
        return True
    s.add(head)
    head = head.next
return False

```

• SimplyLeet

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next

class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        # Initialize two pointers, slow and fast, starting from the head.
        slow = head

        fast = head

        # Traverse the list until fast pointer reaches the end.
        while fast and fast.next:
            slow = slow.next           # Move slow pointer one step.
            fast = fast.next.next     # Move fast pointer two steps.

            # If both pointers meet, there is a cycle.
            if slow == fast:
                return True

        # If loop terminates, there is no cycle.
        return False

```

• LC.ca

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None

class Solution:
    def hasCycle(self, head: ListNode) -> bool:
        slow = fast = head
        while fast and fast.next:

            slow, fast = slow.next, fast.next.next
            if slow == fast:
                return True
        return False
```

◆ Quick Review

Key Insights

- Use two pointers (slow and fast) to traverse the list.
- The fast pointer moves two steps at a time and the slow pointer moves one step.
- If the fast pointer ever meets the slow pointer, a cycle exists.
- If the fast pointer reaches the end of the list (null), the list has no cycle.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes in the linked list.

Space Complexity: $O(1)$ as only two pointers are used.

Solution

We use the Floyd's Cycle Detection algorithm (Tortoise and Hare approach). Two pointers, slow and fast, are initialized at the head of the list. The slow pointer moves one step at a time while the fast pointer moves two steps. If the linked list has a cycle, the fast pointer will eventually meet the slow pointer. If the fast pointer reaches the end (i.e., a null pointer), the linked list does not have a cycle.

Linked List

□ #206 Reverse Linked List

EAS

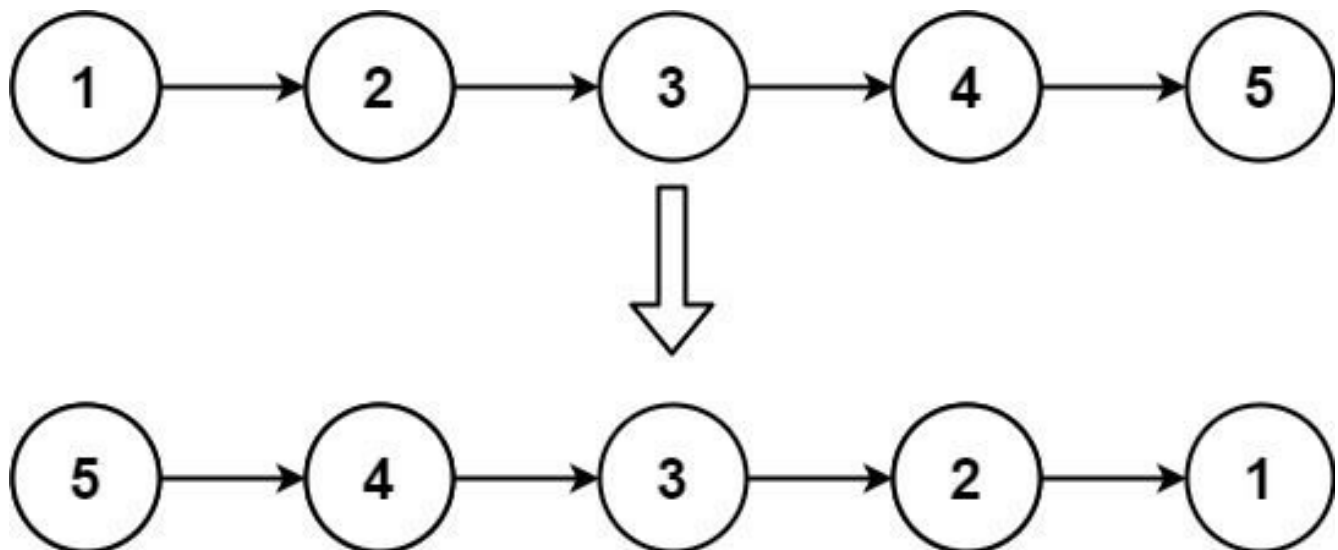
[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Recursion
Lists: Grind 169

Problem

Given the head of a singly linked list, reverse the list, and return the reversed list.

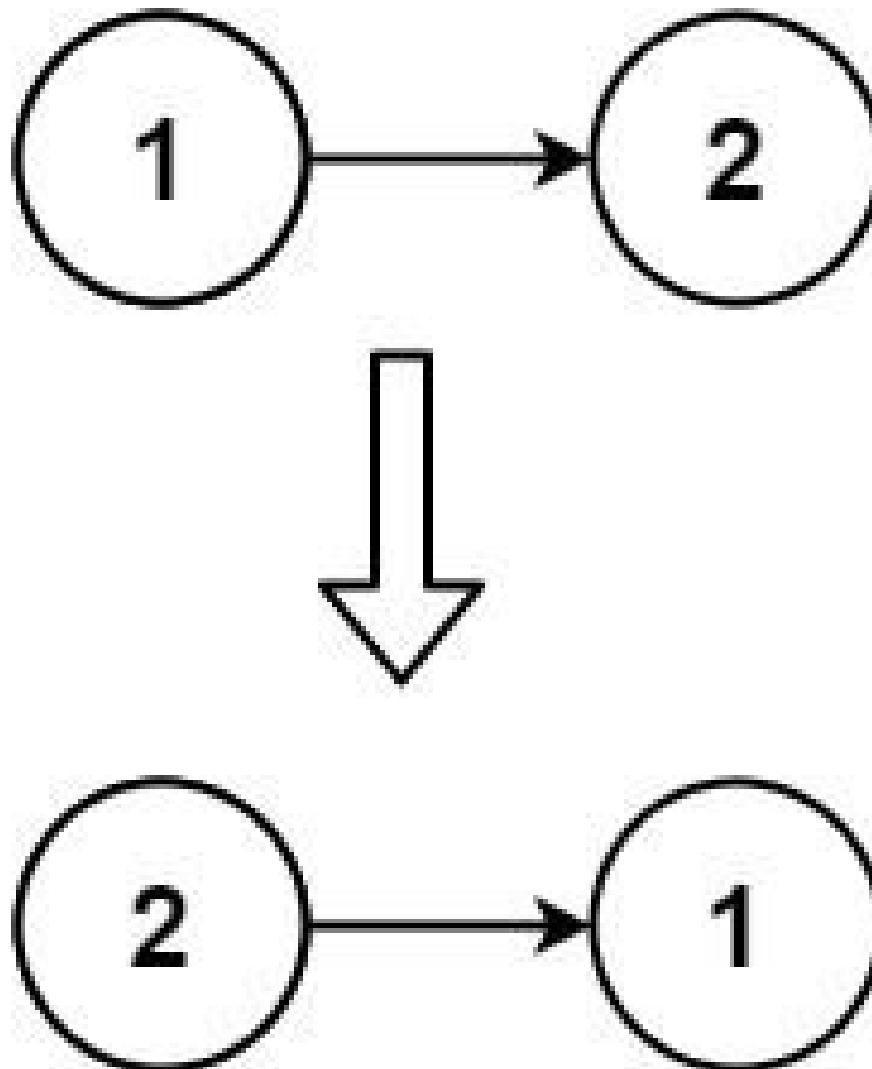
Example 1:



Input: head = [1,2,3,4,5]

Output: [5,4,3,2,1]

Example 2:



Input: head = [1,2]
Output: [2,1]

Example 3:

Input: head = []
Output: []

Constraints:

- The number of nodes in the list is the range [0, 5000].
- $-5000 \leq \text{Node.val} \leq 5000$

Follow up: A linked list can be reversed either iteratively or recursively. Could you implement both?

My LeetCode Solution

• My LeetCode Solution

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        p = None
        c = head
        while c:
            store = c.next
            c.next = p
            p = c
            c = store
        return p
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given the head of a singly linked list, reverse it in-place and return the new head.

We change the next pointers – not just the values. Right?"

#

"A few quick questions:

Can the list be empty? Yes – return None.

Can it have just one node? Yes – return that node unchanged.

The follow-up asks for both iterative and recursive – I'll do iterative first."

#

"Let me walk the example: [1,2,3,4,5] → [5,4,3,2,1] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

```

# Collect all node values into a list, build a new reversed linked list from those
values.
# This is O(n) time but O(n) extra space for the values array –
# and it allocates new nodes instead of rewiring the existing ones.
# Should I go ahead and optimize?"
class _206_ReverseLinkedList_Brute:
    def reverseList(self, head):
        vals = []
        cur = head
        while cur: vals.append(cur.val); cur = cur.next
        dummy = cur = ListNode(0)
        for v in reversed(vals):
            cur.next = ListNode(v); cur = cur.next
        return dummy.next

# PHASE 3 — OPTIMIZE
# "The bottleneck is the O(n) extra space and the unnecessary node allocation.
# We can reverse in-place by maintaining three pointers: prev, curr, next_node.
# At each step we point curr.next backward to prev, then advance.
#
# Pseudocode:
# prev, curr = None, head
# while curr:
#     next_node = curr.next
#     curr.next = prev
#     prev, curr = curr, next_node
# return prev
#
# Time: O(n). Space: O(1).
# Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _206_ReverseLinkedList:
    def reverseList(self, head):
        prev, curr = None, head
        while curr:
            next_node = curr.next
            curr.next = prev
            prev, curr = curr, next_node
        return prev

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# [1,2,3]:
# prev=None,curr=1: next=2, 1.next=None, prev=1,curr=2.
# prev=1,curr=2: next=3, 2.next=1, prev=2,curr=3.
# prev=2,curr=3: next=None, 3.next=2, prev=3,curr=None.
# return prev=node(3) → [3,2,1] ✓

```



```
#
# []: curr=None → loop never enters → return None ✓
#
# "No bugs. prev starts at None so the new tail's next is correctly None."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass. Space O(1): three pointer variables.
# Recursive solution is elegant but O(n) stack space – mention this trade-off.
# Follow-up: Reverse Linked List II (#92) – reverse between positions l and r.
# Follow-up: Reverse Nodes in k-Group (#25) – apply this logic to groups of k."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def reverseList(self, head: ListNode | None) -> ListNode | None:
        if not head or not head.next:
            return head

        newHead = self.reverseList(head.next)
        head.next.next = head
        head.next = None
        return newHead
```

```
class Solution:
    def reverseList(self, head: ListNode | None) -> ListNode | None:
        prev = None
        curr = head

        while curr:
            next = curr.next
            curr.next = prev
            prev = curr
            curr = next

        return prev
```

• LeetDoocs

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseList(self, head: ListNode) -> ListNode:
        dummy = ListNode()
        curr = head
        while curr:

            next = curr.next
            curr.next = dummy.next
            dummy.next = curr
            curr = next
        return dummy.next
```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val # Store the node value
        self.next = next # Pointer to the next node

# Iterative approach to reverse the linked list.
def reverseList(head):
    prev = None # Initially, there is no previous node
    current = head # Start with the head of the list

    while current:
        next_node = current.next # Temporarily store the next node
        current.next = prev # Reverse the current node's pointer
        prev = current # Move previous pointer one step forward
        current = next_node # Move to the next node in the list
    return prev # The new head of the reversed list

# Recursive approach to reverse the linked list.
def reverseListRecursive(head):
    # Base case: empty list or single node
```

```

if not head or not head.next:
    return head
new_head = reverseListRecursive(head.next) # Reverse the rest of the list
head.next.next = head # Reverse the pointer for the current node
head.next = None # Set the next pointer of the current node to None
return new_head # Return the new head of the reversed list

```

• LC.ca

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        pre, p = None, head
        while p:
            q = p.next

```

```

            p.next = pre
            pre = p
            p = q
        return pre

```

```

class Solution(object): # recursive
    def reverseList(self, root):
        if not root or not root.next:
            return root

```

```

        ret = self.reverseList(root.next)
        root.next.next = root # root.next is end of the newly reversed list ret
        root.next = None
        return ret

```

```

class Solution:
    def reverseList(self, head: ListNode) -> ListNode:
        dummy = ListNode()

```

```
curr = head
while curr:
    next = curr.next
    curr.next = dummy.next
    dummy.next = curr
    curr = next
return dummy.next
```

◆ Quick Review

Key Insights

- The problem can be solved using either an iterative or recursive approach.
- Iterative method uses constant space by updating pointers in-place.
- Recursive method leverages the call stack to reverse the list, which could use $O(n)$ space in the worst case.
- Both approaches traverse each node exactly once, yielding linear time complexity.

Space & Time

Time Complexity: $O(n)$ for both iterative and recursive approaches, where n is the number of nodes.

Space Complexity: $O(1)$ for the iterative approach and $O(n)$ for the recursive approach because of the recursion call stack.

Solution

We can solve the problem by iterating through the list while reversing the next pointers of each node. We maintain two pointers (previous and current) and update them as we traverse the list, so that by the end, the previous pointer is at the new head of the reversed list. Alternatively, a recursive approach reverses the rest of the list first and then adjusts the pointers so that the original head becomes the tail.

Linked List

□ #234 Palindrome Linked List

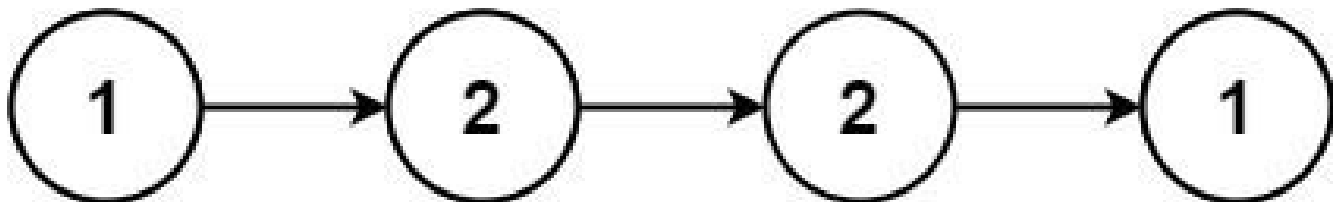
EAS[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Two Pointers · Recursion
Lists: Grind 169

Problem

Given the head of a singly linked list, return true if it is a palindrome or false otherwise.

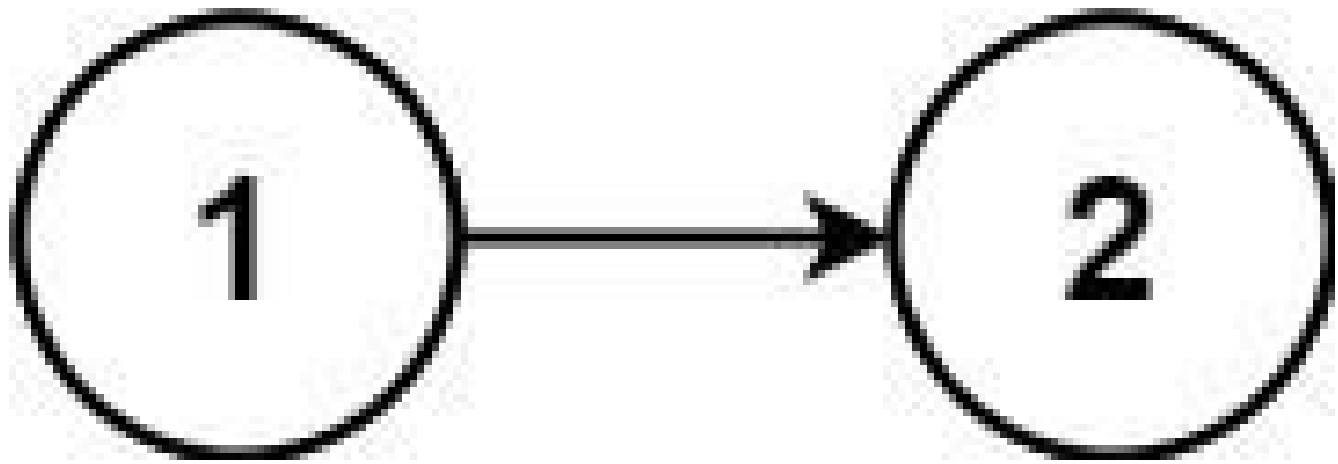
Example 1:



Input: head = [1,2,2,1]

Output: true

Example 2:



Input: head = [1,2]

Output: false

Constraints:

- The number of nodes in the list is in the range [1, 105].
- $0 \leq \text{Node.val} \leq 9$

Follow up: Could you do it in $O(n)$ time and $O(1)$ space?

My LeetCode Solution

• My LeetCode Solution

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
class Solution:
    def isPalindrome(self, head: Optional[ListNode]) -> bool:
        slow = fast = head
        while fast and fast.next:
            fast = fast.next.next
```



```

        slow = slow.next

    prev = None
    while slow:
        slowstore = slow.next
        slow.next = prev
        prev = slow
        slow = slowstore

    f,s = head, prev

    while f and s:
        if f.val != s.val:
            return False

        f,s = f.next, s.next

    return True

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Let me make sure I understand the problem correctly.
# We need to check if the linked list reads the same forwards and backwards.
# We can only traverse with next pointers – no random access.
# The follow-up asks for O(n) time and O(1) space. Right?"
#
# "A few quick questions:
# Can the list be empty or have one node? Yes – trivially palindromes.
# Should we restore the list after checking? Good practice; the problem doesn't
require it."
#
# "Let me walk the example: [1,2,2,1] → palindrome ✓. [1,2] → not ✓"

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with brute force to lock in the logic.
# Copy all values into a Python list, then compare the list to its reverse.
# This is O(n) time and O(n) space – clear and correct.
# The follow-up wants O(1) extra space, so let's aim for that.
# Should I go ahead and optimize?"
class _234_PalindromeLinkedList_Brute:
    def isPalindrome(self, head):
        vals = []
        cur = head
        while cur: vals.append(cur.val); cur = cur.next

```

```
    return vals == vals[::-1]
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the O(n) extra array.
# We can do this in O(1) extra space with three steps:
# 1. Find the midpoint using slow/fast pointers.
# 2. Reverse the second half in-place.
# 3. Compare first half with reversed second half node by node.
#
# Pseudocode:
# slow, fast = head, head
# while fast and fast.next: slow = slow.next; fast = fast.next.next
# second = reverse(slow)
# left, right = head, second
# while right:
# if left.val != right.val: return False
# left = left.next; right = right.next
# return True
#
# Time: O(n). Space: O(1).
# Does that make sense before I code it?"
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _234_PalindromeLinkedList:
    def isPalindrome(self, head):
        def reverse(node):
            prev, cur = None, node
            while cur:
                nxt = cur.next; cur.next = prev; prev, cur = cur, nxt
            return prev

        slow = fast = head
        while fast and fast.next:
            slow = slow.next; fast = fast.next.next

        second = reverse(slow)
        left, right = head, second
        while right:
            if left.val != right.val:
                return False
            left = left.next; right = right.next
        return True
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# [1,2,2,1]:
# Mid: slow=2(idx1), fast=1(idx3 after 2 steps). Reverse [2,1] → [1,2].
# Compare: 1==1 ✓, 2==2 ✓, right=None → True ✓
#
# [1,2]:
```

```
# Mid: slow=2, fast=None after 1 step. Reverse [2] → [2].
# Compare: 1≠2 → False ✓
#
# "No bugs. For even-length lists slow lands on the second half correctly."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): find mid O(n/2) + reverse O(n/2) + compare O(n/2).
# Space O(1): constant extra pointers.
# Trade-off: the list is modified during checking. To restore: reverse again after
comparison.
# Follow-up: Valid Palindrome (#125) – same idea on a string, ignoring
non-alphanumeric.
# Follow-up: Palindrome Number (#9) – similar reverse-half technique on an integer."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isPalindrome(self, head: ListNode) -> bool:
        def reverseList(head: ListNode) -> ListNode:
            prev = None
            curr = head
            while curr:
                next = curr.next
                curr.next = prev
                prev = curr
                curr = next

            return prev

        slow = head
        fast = head

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        if fast:
```

```

        slow = slow.next
    slow = reverseList(slow)

    while slow:
        if slow.val != head.val:
            return False
        slow = slow.next
        head = head.next

    return True

```

• LeetDoocs

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def isPalindrome(self, head: Optional[ListNode]) -> bool:
        slow, fast = head, head.next
        while fast and fast.next:
            slow, fast = slow.next, fast.next.next

        pre, cur = None, slow.next
        while cur:
            t = cur.next
            cur.next = pre
            pre, cur = cur, t
        while pre:
            if pre.val != head.val:
                return False
            pre, head = pre.next, head.next
        return True

```

• SimplyLeet

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def isPalindrome(self, head: ListNode) -> bool:
        # Find the midpoint using slow and fast pointers
        slow, fast = head, head

        while fast and fast.next:
            slow = slow.next          # Move slow pointer by 1
            fast = fast.next.next     # Move fast pointer by 2

        # Reverse the second half of the list
        prev = None
        while slow:
            next_node = slow.next     # Store the next node
            slow.next = prev          # Reverse the current node's pointer
            prev = slow                # Move prev to current node

            slow = next_node           # Move to next node

        # Compare the first half and the reversed second half node by node
        left, right = head, prev
        while right:
            # Only need to compare till the end of
            # second half
            if left.val != right.val:
                return False          # Mismatch found, not a palindrome
            left = left.next           # Move left pointer
            right = right.next         # Move right pointer

        return True                   # All nodes matched, it's a palindrome

```

• LC.ca

```

# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
class Solution:
    def isPalindrome(self, head: Optional[ListNode]) -> bool:
        slow, fast = head, head.next
        while fast and fast.next:
            slow, fast = slow.next, fast.next.next

        pre, cur = None, slow.next
        while cur:
            t = cur.next
            cur.next = pre
            pre, cur = cur, t
        while pre:
            if pre.val != head.val:
                return False
            pre, head = pre.next, head.next
        return True

```

◆ Quick Review

Key Insights

- Use two pointers (slow and fast) to reach the middle of the list.
- Reverse the second half of the list.
- Compare the nodes from the start of the list to the reversed half.
- This approach meets the $O(n)$ time and $O(1)$ space follow-up requirement.

Space & Time

Time Complexity: $O(n)$ – each node is visited a constant number of times.

Space Complexity: $O(1)$ – only constant extra space is used (in-place reversal).

Solution

We first use the two-pointer technique to locate the midpoint of the linked list. The slow pointer moves one step at a time while the fast pointer moves two steps. Once the middle is found, we reverse the second half of the list in-place. Finally, we traverse both halves concurrently to compare the node values. If all corresponding values match, the linked list is a palindrome. A potential gotcha is handling the odd number of nodes; in that case, we ignore the middle element when comparing.

Linked List

□ #706 Design HashMap

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Linked List · Design ·
Hash Function

Lists: Denny Zhang

Problem

Design a HashMap without using any built-in hash table libraries.

Implement the MyHashMap class:

- MyHashMap() initializes the object with an empty map.
- void put(int key, int value) inserts a (key, value) pair into the HashMap. If the key already exists in the map, update the corresponding value.
- int get(int key) returns the value to which the specified key is mapped, or -1 if this map contains no mapping for the key.

- **void remove(key)** removes the key and its corresponding value if the map contains the mapping for the key.

Example 1:

Input

```
["MyHashMap", "put", "put", "get", "get", "put", "get", "remove", "get"]
[[], [1, 1], [2, 2], [1], [3], [2, 1], [2], [2], [2]]
```

Output

```
[null, null, null, 1, -1, null, 1, null, -1]
```

Explanation

```
MyHashMap myHashMap = new MyHashMap();
```

Constraints:

- $0 \leq \text{key}, \text{value} \leq 106$
- At most 104 calls will be made to put, get, and remove.

My LeetCode Solution

• My LeetCode Solution

```
class MyHashMap:

    def __init__(self):
        self.hmap = [-1] * (10**6 + 1)

    def put(self, key: int, value: int) -> None:
        self.hmap[key] = value

    def get(self, key: int) -> int:
        return self.hmap[key]
```

```
def remove(self, key: int) -> None:
    self.hmap[key] = -1
```

```
# Your MyHashMap object will be instantiated and called as such:
# obj = MyHashMap()
# obj.put(key,value)
# param_2 = obj.get(key)
# obj.remove(key)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

We need to implement a HashMap from scratch – no built-in dict or hash libraries.

Support put(key, val), get(key) → -1 if missing, and remove(key).

Keys and values are non-negative integers up to 10^6 . Right?"

#

"A few quick questions:

At most 10^4 calls total – so a simple fixed-size array could work.

Does put update the value if the key exists? Yes.

Does remove need to return anything? No."

#

"Let me walk the example: put(1,1), put(2,2), get(1)→1, get(3)→-1, remove(2), get(2)→-1 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Store all (key, value) pairs in a plain list, scan linearly for get/put/remove.

This is $O(n)$ per operation – correct but slow for large n .

Should I go ahead and optimize?"

```
class _706_DesignHashMap_Brute:
```

```
    def __init__(self):
```

```
        self.data = []
```

```
    def put(self, key, value):
```

```
        for i, (k, _) in enumerate(self.data):
```

```
            if k == key: self.data[i] = (key, value); return
```

```
        self.data.append((key, value))
```

```
    def get(self, key):
```

```
        for k, v in self.data:
```

```
            if k == key: return v
```

```
        return -1
```

```
    def remove(self, key):
```

```
        self.data = [(k, v) for k, v in self.data if k != key]
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ linear scan per operation.

Use an array of buckets with separate chaining: each bucket holds a small list.

Hash function: $\text{key} \% \text{num_buckets}$. We pick a large prime (e.g. 1009) to spread keys evenly.

Average $O(1)$ per operation when the load factor is low.

#

Pseudocode:

`__init__`: `buckets = [[] for _ in range(1009)]`

`put`: `idx = key % 1009`; update existing pair or append

`get`: scan bucket for key, return value or `-1`

`remove`: filter key out of bucket

#

Time: $O(1)$ average. Space: $O(n + 1009)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _706_DesignHashMap:
```

```
    def __init__(self):
```

```
        self._size = 1009
```

```
        self._buckets = [[] for _ in range(self._size)]
```

```
    def _idx(self, key):
```

```
        return key % self._size
```

```
    def put(self, key, value):
```

```
        bucket = self._buckets[self._idx(key)]
```

```
        for i, (k, _) in enumerate(bucket):
```

```
            if k == key:
```

```
                bucket[i] = (key, value)
```

```
                return
```

```
        bucket.append((key, value))
```

```
    def get(self, key):
```

```
        for k, v in self._buckets[self._idx(key)]:
```

```
            if k == key:
```

```
                return v
```

```
        return -1
```

```
    def remove(self, key):
```

```
        idx = self._idx(key)
```

```
        self._buckets[idx] = [(k, v) for k, v in self._buckets[idx] if k != key]
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

`put(1,1)`: `bucket[1]=[(1,1)]`. `put(2,2)`: `bucket[2]=[(2,2)]`.

`get(1)`: `bucket[1] → (1,1) → 1 ✓`. `get(3)`: `bucket[3]=[] → -1 ✓`.

`put(2,1)`: `bucket[2] → update → [(2,1)]`.

`remove(2)`: `bucket[2]=[]`. `get(2)`: `[] → -1 ✓`.

#

"No bugs. Prime bucket count minimises collisions for common key distributions."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(1)$ average; $O(n/B)$ worst case if all keys hash to same bucket (B = bucket count).

Space $O(n + B)$: n stored pairs + B bucket slots.

Trade-off: larger B reduces collisions but wastes memory for sparse maps.

Follow-up: Design HashSet (#705) – identical structure, store keys only.

Follow-up: open addressing (linear probing) is an alternative – avoids linked lists but needs tombstones."

Community Solutions (Python)

• LeetDoocs

```
class MyHashMap:
    def __init__(self):
        self.data = [-1] * 1000001

    def put(self, key: int, value: int) -> None:
        self.data[key] = value

    def get(self, key: int) -> int:
        return self.data[key]

    def remove(self, key: int) -> None:
        self.data[key] = -1

# Your MyHashMap object will be instantiated and called as such:
# obj = MyHashMap()
# obj.put(key,value)
# param_2 = obj.get(key)
# obj.remove(key)
```

• SimplyLeet

```
# ListNode class for handling collisions in each bucket
class ListNode:
    def __init__(self, key, value, next=None):
        self.key = key          # Store key
        self.value = value       # Store value
        self.next = next        # Pointer to the next node

class MyHashMap:
    def __init__(self):
        self.size = 1000        # Define the bucket size

        self.buckets = [None] * self.size    # Initialize buckets as an array
of None

    def hash(self, key):
        # Simple modulo based hash function to map keys to bucket indices
        return key % self.size

    def put(self, key: int, value: int) -> None:
        index = self.hash(key)
        if self.buckets[index] is None:
            # If the bucket is empty, create a new node and place it here

            self.buckets[index] = ListNode(key, value)
        else:
            current = self.buckets[index]
            while True:
                if current.key == key:
                    # If key exists, update its value
                    current.value = value
                    return
                if current.next is None:
                    break
```

```
        current = current.next
        # Append a new node at the end of the list if key not found
        current.next = ListNode(key, value)

def get(self, key: int) -> int:
    index = self.hash(key)
    current = self.buckets[index]
    while current:
        if current.key == key:
            # Key found; return its value

            return current.value
        current = current.next
    # Key not found; return -1
    return -1

def remove(self, key: int) -> None:
    index = self.hash(key)
    current = self.buckets[index]
    if current is None:
        return

    if current.key == key:
        # Remove head node if it contains the key
        self.buckets[index] = current.next
    else:
        prev = current
        current = current.next
        while current:
            if current.key == key:
                # Bypass the node to remove it from the list
                prev.next = current.next

            return
        prev, current = current, current.next
```

• LC.ca

```
class MyHashMap:
    def __init__(self):
        self.data = [-1] * 1000001

    def put(self, key: int, value: int) -> None:
        self.data[key] = value

    def get(self, key: int) -> int:
        return self.data[key]

    def remove(self, key: int) -> None:
        self.data[key] = -1

# Your MyHashMap object will be instantiated and called as such:
# obj = MyHashMap()
# obj.put(key,value)
# param_2 = obj.get(key)
# obj.remove(key)
```

```
#####
```

```
class MyHashMap: # hash to bucket
    def __init__(self):
        """
        Initialize your data structure here.
        """
        self.size = 1000
        self.buckets = [[] for _ in range(self.size)]
```

```
    def _hash(self, key: int) -> int:
        """
        Generate a hash for a given key.
        """
        return key % self.size

    def put(self, key: int, value: int) -> None:
        """
        Value will always be non-negative.
        """
```



```

hash_key = self._hash(key)
for i, (k, v) in enumerate(self.buckets[hash_key]):
    if k == key:
        self.buckets[hash_key][i] = (key, value)
        return
self.buckets[hash_key].append((key, value))

```

```

def get(self, key: int) -> int:
    """

```

Returns the value to which the specified key is mapped, or -1 if this map contains no mapping for the key.

```

    """

```

```

hash_key = self._hash(key)
for k, v in self.buckets[hash_key]:
    if k == key:
        return v
return -1

```

```

def remove(self, key: int) -> None:
    """

```

Removes the mapping of the specified value key if this map contains a mapping for the key.

```

    """

```

```

hash_key = self._hash(key)
for i, (k, v) in enumerate(self.buckets[hash_key]):
    if k == key:
        del self.buckets[hash_key][i]
        return

```

```

#####

```

```

class MyHashMap: # open addressing with linear probing

```

```

def __init__(self):
    self.capacity = 10000
    self.data = [None] * self.capacity
    self.DELETED = (None, None) # Marker for deleted entries

def _hash(self, key):
    return key % self.capacity

def put(self, key: int, value: int) -> None:
    idx = self._hash(key)

    while self.data[idx] not in (None, self.DELETED, key):
        idx = (idx + 1) % self.capacity
    self.data[idx] = (key, value)

def get(self, key: int) -> int:
    idx, found_key = self._find(key)
    if found_key:
        return self.data[idx][1]
    return -1

def remove(self, key: int) -> None:
    idx, found_key = self._find(key)
    if found_key:
        self.data[idx] = self.DELETED

def _find(self, key):
    original_idx = idx = self._hash(key)
    while self.data[idx] is not None:
        if self.data[idx] != self.DELETED and self.data[idx][0] == key:
            return idx, True

        idx = (idx + 1) % self.capacity
        if idx == original_idx: # Came full circle
            break
    return original_idx, False

```

◆ Quick Review

Key Insights

- Use a fixed-size array of buckets where each bucket uses a linked list to handle collisions.
- Choose a simple hash function (e.g., modulo operation) to map a key to its corresponding bucket.
- For each bucket, traverse the linked list to update an existing key or append a new node if the key does not exist.
- Removal requires updating pointers in the linked list to bypass the removed node.

Space & Time

Time Complexity: Average $O(1)$ for put, get, and remove; worst-case $O(n)$ when many keys hash to the same bucket.

Space Complexity: $O(n)$, where n is the number of key-value pairs stored.

Solution

We implement the HashMap using an array of fixed size (e.g., 1000). Each array element is the head of a linked list that manages collisions using chaining. The key is hashed using a modulo operation. For the put operation, if a key already exists in the chain, its value is updated; otherwise, a new node is

appended. The get operation traverses the chain at the computed bucket index to find and return the corresponding value, or returns -1 if not found. The remove operation also traverses the list, carefully reconnecting the links to remove the target node while handling edge cases such as removal at the head.

Linked List

□ #876 Middle of the Linked List

EAS[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Two Pointers
Lists: Grind 169

Problem

Given the head of a singly linked list, return the middle node of the linked list.

If there are two middle nodes, return the second middle node.

Example 1:

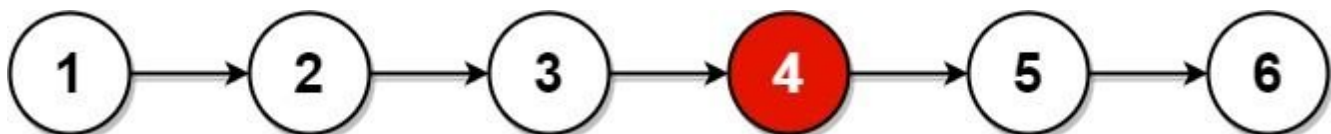


Input: head = [1,2,3,4,5]

Output: [3,4,5]

Explanation: The middle node of the list is node 3.

Example 2:



Input: head = [1,2,3,4,5,6]

Output: [4,5,6]

Explanation: Since the list has two middle nodes with values 3 and 4, we return the second one.

Constraints:

- The number of nodes in the list is in the range [1, 100].
- $1 \leq \text{Node.val} \leq 100$

My LeetCode Solution

• My LeetCode Solution

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
class Solution:
    def middleNode(self, head: Optional[ListNode]) -> Optional[ListNode]:
        slow = head
        fast = head

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        return slow
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Return the middle node of a singly linked list.

If there are two middle nodes (even length), return the SECOND one.

Right?"

#

"A few quick questions:

```

# Can the list be empty? No – length is at least 1.
# Return the node itself, not its value? Yes."
#
# "Let me walk the examples:
# [1,2,3,4,5]: length 5 → middle index 2 → node(3) ✓
# [1,2,3,4,5,6]: two middles at 2,3 → return second → node(4) ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Count the total length in one pass, then walk to position length//2 in a second
pass.
# Two passes total: O(n) time, O(1) space.
# Should I go ahead and optimize?"
class _876_MiddleLinkedList_Brute:
    def middleNode(self, head):
        length, cur = 0, head
        while cur: length += 1; cur = cur.next
        cur = head
        for _ in range(length // 2): cur = cur.next
        return cur

# PHASE 3 — OPTIMIZE
# "The bottleneck is doing two full passes – we can find the middle in one pass.
# Slow/fast pointer: slow moves 1 step, fast moves 2.
# When fast reaches the end, slow is at the middle.
# For even-length lists, slow naturally lands on the SECOND middle (desired
behaviour).
#
# Pseudocode:
# slow = fast = head
# while fast and fast.next:
#     slow = slow.next; fast = fast.next.next
# return slow
#
# Time: O(n). Space: O(1).
# Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _876_MiddleLinkedList:
    def middleNode(self, head):
        slow = fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        return slow

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# [1,2,3,4,5]:

```



```
# s=1,f=1 → s=2,f=3 → s=3,f=5 → fast.next=None → stop. return node(3) ✓
#
# [1,2,3,4,5,6]:
# s=1,f=1 → s=2,f=3 → s=3,f=5 → s=4,f=None(6.next) → stop. return node(4) ✓
#
# [1]:
# fast.next=None immediately → return node(1) ✓
#
# "No bugs. The 'fast and fast.next' guard handles odd and even lengths uniformly."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): fast covers the list in n/2 iterations. Space O(1): two pointers.
# To get the FIRST middle for even-length, use 'while fast.next and fast.next.next'
# instead.
# This building-block appears in Palindrome Linked List (#234) and Reorder List
# (#143).
# Follow-up: Kth Node From End (#19) – two-pointer gap of k instead of 2:1 speed
# ratio."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def middleNode(self, head: ListNode) -> ListNode:
        slow = head
        fast = head

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        return slow
```

• LeetDoocs

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next
class Solution:
    def middleNode(self, head: ListNode) -> ListNode:
        slow = fast = head
        while fast and fast.next:
            slow, fast = slow.next, fast.next.next

        return slow
```

• SimplyLeet

```
# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next

class Solution:
    def middleNode(self, head: ListNode) -> ListNode:
        # Initialize slow and fast pointers at the head of the list
        slow = head

        fast = head
        # Loop until fast reaches the end of the list
        while fast and fast.next:
            # Move slow pointer one step
            slow = slow.next
            # Move fast pointer two steps
            fast = fast.next.next
        # When loop ends, slow pointer points to the middle node
        return slow
```

• LC.ca

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def middleNode(self, head: ListNode) -> ListNode:
        slow = fast = head
        while fast and fast.next:
            slow, fast = slow.next, fast.next.next

        return slow
```

◆ Quick Review

Key Insights

- Use two pointers (slow and fast) to traverse the list.
- The fast pointer moves two steps at a time while the slow pointer moves one step.
- When the fast pointer reaches the end, the slow pointer will be at the middle node.
- This approach naturally handles both odd and even lengths of the list.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the list.

Space Complexity: $O(1)$, constant extra space.

Solution

The solution uses the two–pointer technique: Initialize two pointers, slow and fast, at the head. Traverse the list, moving slow by one step and fast by two steps during each iteration. When fast reaches the end or has no next node, slow will be at the middle. Return the slow pointer (the middle node). This strategy ensures linear time complexity and constant space, efficiently finding the middle node even for lists with an even number of elements.

Linked List

□ ★ #1474 Delete N Nodes After M Nodes of Linked List ★

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List

Lists: ★ Premium | Premium 98

Problem

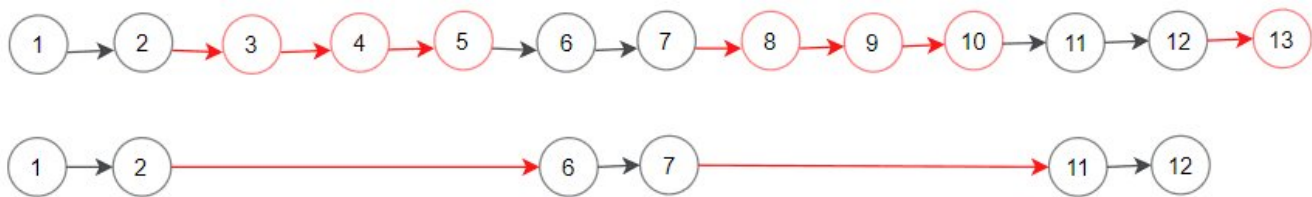
You are given the head of a linked list and two integers m and n .

Traverse the linked list and remove some nodes in the following way:

- Start with the head as the current node.
- Keep the first m nodes starting with the current node.
- Remove the next n nodes
- Keep repeating steps 2 and 3 until you reach the end of the list.

Return the head of the modified list after removing the mentioned nodes.

Example 1:



Input: head = [1,2,3,4,5,6,7,8,9,10,11,12,13], m = 2, n = 3

Output: [1,2,6,7,11,12]

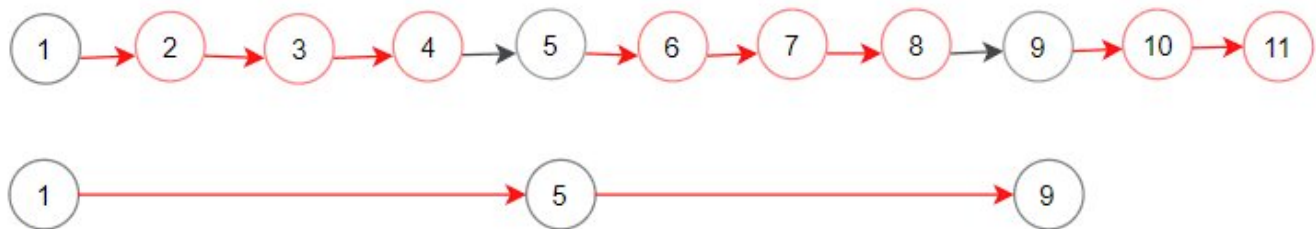
Explanation: Keep the first (m = 2) nodes starting from the head of the linked List (1 -> 2) show in black nodes.

Delete the next (n = 3) nodes (3 -> 4 -> 5) show in red nodes.

Continue with the same procedure until reaching the tail of the Linked List.

Head of the linked list after removing nodes is returned.

Example 2:



Input: head = [1,2,3,4,5,6,7,8,9,10,11], m = 1, n = 3

Output: [1,5,9]

Explanation: Head of linked list after removing nodes is returned.

Constraints:

- The number of nodes in the list is in the range [1, 10⁴].
- 1 ≤ Node.val ≤ 10⁶
- 1 ≤ m, n ≤ 1000

Follow up: Could you solve this problem by modifying the list in-place?

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def deleteNodes(
        self,
        head: ListNode | None,
        m: int,
        n: int,
    ) -> ListNode | None:
        curr = head
        prev = None # prev.next == curr

        while curr:
            # Set the m-th node as `prev`.
            for _ in range(m):
                if not curr:
                    break
                prev = curr
                curr = curr.next
            # Set the (m + n + 1)-th node as `curr`.
            for _ in range(n):
                if not curr:
                    break
                curr = curr.next
            # Delete the nodes [m + 1..n - 1].
            prev.next = curr

        return head
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Walk the list in cycles: keep the first m nodes, delete the next n nodes, repeat.

Do this until the end of the list and return the modified head. Right?"

#

"A few quick questions:

m and n are guaranteed to be at least 1? Yes.

Modify in-place – no new nodes? Yes, just rewire pointers."

#

"Let me walk the example: [1,2,3,4,5,6,7,8,9,10,11,12,13], m=2, n=3.

Keep 1,2 → delete 3,4,5 → keep 6,7 → delete 8,9,10 → keep 11,12 → delete 13.


```
# → [1,2,6,7,11,12] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
```

```
# Enumerate all nodes with an index counter.
```

```
# Keep node i if (i % (m+n)) < m, else skip it.
```

```
# Collect kept nodes in a list and relink them.
```

```
# This is O(n_total) time and O(n_total) space for the list.
```

```
# Should I go ahead and optimize?"
```

```
class _1474_DeleteNNodesAfterM_Brute:
```

```
    def deleteNodes(self, head, m, n):
```

```
        nodes, cur = [], head
```

```
        while cur: nodes.append(cur); cur = cur.next
```

```
        kept = [nodes[i] for i in range(len(nodes)) if i % (m + n) < m]
```

```
        for i in range(len(kept) - 1): kept[i].next = kept[i + 1]
```

```
        if kept: kept[-1].next = None
```

```
        return kept[0] if kept else None
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the O(n) auxiliary list.
```

```
# Walk m steps forward (keeping nodes), then walk n steps forward (skipping them),
```

```
# rewire the tail of the kept section directly to the node after the skipped block.
```

```
# Repeat until we run out of nodes.
```

```
#
```

```
# Pseudocode:
```

```
# cur = head
```

```
# while cur:
```

```
# walk m-1 steps (cur is now tail of kept section)
```

```
# walk n steps past the nodes to delete
```

```
# cur.next = the node after the deleted block
```

```
# cur = cur.next
```

```
# return head
```

```
#
```

```
# Time: O(n_total). Space: O(1).
```

```
# Does that make sense before I code it?"
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _1474_DeleteNNodesAfterM:
```

```
    def deleteNodes(self, head, m, n):
```

```
        cur = head
```

```
        while cur:
```

```
            for _ in range(m - 1):
```

```
                if cur: cur = cur.next
```

```
            skip = cur.next if cur else None
```

```
            for _ in range(n):
```

```
                if skip: skip = skip.next
```

```
            if cur:
```

```
                cur.next = skip
```

```
                cur = skip
```

```
        return head
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

[1,2,3,4,5,6,7], m=2, n=3:

cur=1. walk m-1=1 step → cur=2. skip=3, walk n=3 → skip=6. 2.next=6. cur=6.

cur=6. walk 1 → cur=7. skip=None (7.next). walk 3 (skip stays None). 7.next=None. cur=None.

→ [1,2,6,7] ✓

#

[1], m=1, n=1:

cur=1. walk 0 steps → cur=1. skip=None, walk 1 → skip=None. 1.next=None. cur=None.

→ [1] ✓

#

"No bugs. Guards prevent crashing on short lists."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\text{total_nodes})$: each node visited exactly once. Space $O(1)$: only pointer variables.

Contrast with the brute that uses $O(n)$ auxiliary storage.

Follow-up: what if m and n change each cycle? Accept arrays of m and n values.

Follow-up: Remove Duplicates from Sorted List (#83) – same rewire pattern, different condition."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def deleteNodes(
        self,
        head: ListNode | None,
        m: int,
        n: int,
    ) -> ListNode | None:
        curr = head
        prev = None # prev.next == curr

```

```

while curr:
    # Set the m-th node as `prev`.
    for _ in range(m):
        if not curr:
            break
        prev = curr
        curr = curr.next
    # Set the (m + n + 1)-th node as `curr`.
    for _ in range(n):
        if not curr:
            break
        curr = curr.next
    # Delete the nodes [m + 1..n - 1].
    prev.next = curr

return head

```

• LeetDoocs

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def deleteNodes(self, head: ListNode, m: int, n: int) -> ListNode:
        pre = head
        while pre:
            for _ in range(m - 1):
                if pre:
                    pre = pre.next
            if pre is None:
                return head
            cur = pre
            for _ in range(n):
                if cur:
                    cur = cur.next
            pre.next = None if cur is None else cur.next
            pre = pre.next

        return head

```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def deleteNodes(self, head: ListNode, m: int, n: int) -> ListNode:
        # Initialize the current pointer at the head of the list.
        current = head

        while current:
            # Keep m nodes: traverse m-1 next pointers
            for i in range(1, m):
                if current is None: # if list ends, break
                    break
                current = current.next
            # If we have reached the end of the list, break the loop.
            if current is None:
                break
            # Start deletion from the next node after the m-th node.

            temp = current.next
            # Delete next n nodes.
            for j in range(n):
                if temp is None:
                    break
                temp = temp.next
            # Connect the m-th node to the node after the n deleted nodes.
            current.next = temp
            # Move current pointer to continue from the next kept node.
            current = temp

        return head
```

• LC.ca

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def deleteNodes(self, head: ListNode, m: int, n: int) -> ListNode:
        pre = head
        while pre:
            for _ in range(m - 1):
                if pre:
                    pre = pre.next
            if pre is None:
                return head
            cur = pre
            for _ in range(n):
                if cur:
                    cur = cur.next
            pre.next = None if cur is None else cur.next
            pre = pre.next

        return head

```

◆ Quick Review

Key Insights

- Traverse the linked list while keeping count.
- Skip m nodes (i.e., keep them) and then proceed to delete the next n nodes.
- Update pointers to bypass the removed nodes.
- This is an in-place modification problem, so no additional list structure is needed.

- Be cautious about edge cases when the list has fewer than m or n nodes remaining.

Space & Time

Time Complexity: $O(N)$, where N is the number of nodes in the list.

Space Complexity: $O(1)$ since the modification is done in-place.

Solution

The solution involves iterating through the linked list with a pointer. For each cycle: Traverse m nodes and keep them. Then, from the $(m+1)$ th node, skip and remove the next n nodes by adjusting the 'next' pointer of the m -th node to point to the node after these n nodes. Continue until you reach the end of the list. This in-place approach avoids using extra space and maintains $O(N)$ time complexity.

Stack

Round 4 · Mid · Easy · 3 questions

Stack

□ #20 Valid Parentheses

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack
Lists: Grind 169

Problem

Given a string *s* containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: *s* = "()"

Output: true

Example 2:

Input: s = "()[]{}"

Output: true

Example 3:

Input: s = "([]"

Output: false

Example 4:

Input: s = "([)]"

Output: true

Example 5:

Input: s = "([)]"

Output: false

Constraints:

- $1 \leq s.length \leq 104$
- s consists of parentheses only '()[]{}'.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def isValid(self, s: str) -> bool:
        # Mapping from closing brackets to their corresponding opening
        # brackets.
        paren_map = {')': '(', '}': '{', ']': '['}
        # Initialize an empty list to use as a stack.
        stack = []
        # Process each character in the string.
        for char in s:
            # If the character is a closing bracket.
            if char in paren_map:
```

```

        # Pop the top element from the stack if available, else use a
placeholder.
        top_element = stack.pop() if stack else '#'
        # Check if the mapping does not match the top element.
        if paren_map[char] != top_element:
            return False
    else:
        # For opening brackets, push onto the stack.
        stack.append(char)
    # Return True if all brackets are matched (i.e., the stack is empty).
    return not stack

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Let me make sure I understand the problem correctly.
# A string of brackets is valid if: every opening bracket is closed by the same
type,
# every closing bracket has a corresponding opener, and the order is correct.
# Return true if valid, false otherwise. Right?"
#
# "A few quick questions:
# Can the string be empty? Yes – empty string is valid.
# Only bracket characters – no spaces or letters? Yes, the constraints confirm
this."
#
# "Let me walk the examples:
# '()[]{}' → each pair closes correctly → true ✓
# '([)]' → '[' is open when ')' arrives → false ✓"

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with brute force to lock in the logic.
# Repeatedly replace matched pairs '()', '[]', '{}' with empty strings until nothing
changes.
# If the final string is empty, it was valid.
# This is O(n²) – each pass removes one pair and we may need n/2 passes.
# Should I go ahead and optimize?"
class _20_ValidParentheses_Brute:
    def isValid(self, s):
        prev = None
        while s != prev:
            prev = s
            for pair in ['()', '[]', '{}']:
                s = s.replace(pair, '')
        return s == ''

```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is repeatedly scanning the string.
# A stack solves this in a single pass:
# push every opening bracket; on a closing bracket, check that the top matches.
# If it doesn't – or the stack is empty – it's invalid.
# At the end the stack must be empty (every opener was closed).
#
# Pseudocode:
# stack = []
# for ch in s:
# if ch in '([{': stack.append(ch)
# else:
# if not stack or stack[-1] != matching_opener: return False
# stack.pop()
# return stack == []
#
# Time: O(n). Space: O(n) for the stack.
# Does that make sense before I code it?"
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _20_ValidParentheses:
    def isValid(self, s):
        pairs = {')': '(', ']': '[', '}': '{'}
        stack = []
        for ch in s:
            if ch not in pairs:
                stack.append(ch)
            elif not stack or stack[-1] != pairs[ch]:
                return False
            else:
                stack.pop()
        return not stack
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# '()[]{}':
# '(' push. ')' → top='(' matches → pop. '[' push. ']' → pop. '{' push. '}' → pop.
# stack=[] → True ✓
#
# '[]':
# '[' push. ']' → top='[' ≠ '(' → False ✓
#
# '([])':
# '(' push. '[' push. ')' → top='[' ≠ '(' → False ✓
#
# "No bugs. The closing-bracket dict keeps the matching logic clean and extensible."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): one pass. Space O(n): worst case all openers e.g. '((((('.
```

Brute $O(n^2)$ is easy to understand but TLEs for long strings.
 # Follow-up: Minimum Add to Make Parentheses Valid (#921) – count unmatched openers/closers.
 # Follow-up: Longest Valid Parentheses (#32) – find the longest valid substring using DP or stack."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isValid(self, s: str) -> bool:
        stack = []

        for c in s:
            if c == '(':
                stack.append(')')
            elif c == '{':
                stack.append('}')
            elif c == '[':
                stack.append(']')

            if not stack or stack.pop() != c:
                return False

        return not stack
```

• LeetDoocs

```
class Solution:
    def isValid(self, s: str) -> bool:
        stk = []
        d = {'()': '', '[]': '', '{}': ''}
        for c in s:
            if c in '({[':
                stk.append(c)
            elif not stk or stk.pop() + c not in d:
                return False
        return not stk
```

• SimplyLeet

```
def isValid(s: str) -> bool:
    # Mapping from closing brackets to their corresponding opening brackets.
    paren_map = {')': '(', '}': '{', ']': '['}
    # Initialize an empty list to use as a stack.
    stack = []
    # Process each character in the string.
    for char in s:
        # If the character is a closing bracket.
        if char in paren_map:
            # Pop the top element from the stack if available, else use a
            placeholder.

            top_element = stack.pop() if stack else '#'
            # Check if the mapping does not match the top element.
            if paren_map[char] != top_element:
                return False
        else:
            # For opening brackets, push onto the stack.
            stack.append(char)
    # Return True if all brackets are matched (i.e., the stack is empty).
    return not stack

# Example usage
print(isValid("()[]{}")) # Expected output: True
```

• LC.ca

```
class Solution:
    def isValid(self, s: str) -> bool:
        stk = []
        d = {'():', '[]', '{}'}
        for c in s:
            if c in '([{':
                stk.append(c)
            elif not stk or stk.pop() + c not in d:
                return False
        return not stk
```

◆ Quick Review

Key Insights

- The problem can be solved using a stack data structure.
- When encountering an opening bracket, push it onto the stack.
- When encountering a closing bracket, check if the top of the stack is its matching opening bracket.
- If the stack is empty before finding a match or if there is a mismatch, the string is invalid.
- At the end, the string is valid only if the stack is empty, meaning all brackets were properly matched.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string, since we process each character once.

Space Complexity: $O(n)$ in the worst case for the stack when all characters are opening brackets.

Solution

We use a stack to check for valid parentheses. The algorithm iterates through the string,

pushing opening brackets onto the stack and popping them when a corresponding closing bracket is encountered. A mapping of closing to opening brackets helps in validating the matching pairs. By ensuring that the stack is empty at the end, we confirm that every opening bracket was properly closed.

Stack

□ #232 Implement Queue using Stacks

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Stack · Design · Queue
Lists: Grind 169

Problem

Implement a first in first out (FIFO) queue using only two stacks. The implemented queue should support all the functions of a normal queue (push, peek, pop, and empty).

Implement the MyQueue class:

- **void push(int x)** Pushes element x to the back of the queue.
- **int pop()** Removes the element from the front of the queue and returns it.
- **int peek()** Returns the element at the front of the queue.
- **boolean empty()** Returns true if the queue is empty, false otherwise.

Notes:

- You must use only standard operations of a stack, which means only push to top, peek/pop from top, size, and is empty operations are valid.
- Depending on your language, the stack may not be supported natively. You may simulate a stack using a list or deque (double-ended queue) as long as you use only a stack's standard operations.

Example 1:

Input

```
["MyQueue", "push", "push", "peek", "pop", "empty"]
```

```
[[], [1], [2], [], [], []]
```

Output

```
[null, null, null, 1, 1, false]
```

Explanation

```
MyQueue myQueue = new MyQueue();
```

Constraints:

- $1 \leq x \leq 9$
- At most 100 calls will be made to push, pop, peek, and empty.
- All the calls to pop and peek are valid.

Follow-up: Can you implement the queue such that each operation is amortized $O(1)$ time complexity? In other words, performing n operations will take overall $O(n)$ time even if one of those operations may take longer.

My LeetCode Solution

• My LeetCode Solution

```
class MyQueue:

    def __init__(self):
        self.a = []
        self.b = []

    def push(self, x: int) -> None:
        self.a.append(x)

    def pop(self) -> int:

        if not self.b:
            while self.a:
                self.b.append(self.a.pop())

        return self.b.pop()

    def peek(self) -> int:
        if not self.b:
            while self.a:
                self.b.append(self.a.pop())

        return self.b[-1]

    def empty(self) -> bool:
        return not self.b and not self.a

# Your MyQueue object will be instantiated and called as such:

# obj = MyQueue()
# obj.push(x)
# param_2 = obj.pop()
# param_3 = obj.peek()
# param_4 = obj.empty()
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Implement a FIFO queue using only stack operations (push/pop/peek from the top).
# Support push(x), pop(), peek(), and empty().
# The follow-up asks for amortized O(1) per operation. Right?"
#
# "A few quick questions:
# Is it guaranteed that pop and peek are only called on non-empty queues? Yes.
# Standard stack operations only – no deque or collections.deque tricks?"
#
# "Let me walk: push(1), push(2), peek()→1, pop()→1, empty()→false ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Use a single stack. On every push, move all existing elements out, push the new
value,
# then put everything back – so the new element sits at the bottom (queue order).
# push is O(n), pop/peek are O(1). Simple but push is expensive.
# Should I go ahead and optimize?"
class _232_QueueUsingStacks_Brute:
    def __init__(self):
        self.stack = []
    def push(self, x):
        tmp = []
        while self.stack: tmp.append(self.stack.pop())
        self.stack.append(x)
        while tmp: self.stack.append(tmp.pop())
    def pop(self): return self.stack.pop()
    def peek(self): return self.stack[-1]
    def empty(self): return not self.stack
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is moving all elements on every push – O(n) per push.
# Two stacks – inbox and outbox – give amortized O(1):
# push always goes to inbox.
# For pop/peek: if outbox is empty, pour all of inbox into outbox (reverses order).
# Then pop/peek from outbox.
# Each element moves at most twice total, so the amortized cost per operation is
O(1).
#
# Time: O(1) amortized all ops. Space: O(n) across both stacks."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _232_QueueUsingStacks:
    def __init__(self):
```

```

        self._inbox = []
        self._outbox = []
    def push(self, x):
        self._inbox.append(x)
    def _shift(self):
        if not self._outbox:
            while self._inbox:
                self._outbox.append(self._inbox.pop())
    def pop(self):
        self._shift()
        return self._outbox.pop()
    def peek(self):
        self._shift()
        return self._outbox[-1]
    def empty(self):
        return not self._inbox and not self._outbox

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

push(1),push(2): inbox=[1,2], outbox=[].

peek(): outbox empty → pour → inbox=[], outbox=[2,1]. outbox[-1]=1 ✓

pop(): outbox=[2,1] → pop → 1, outbox=[2]. return 1 ✓

empty(): inbox=[], outbox=[2] → False ✓

pop(): outbox=[2] → pop → 2. ✓

empty(): both empty → True ✓

#

"No bugs. 'pour' step is lazy – we only pay it when the outbox is drained."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"All ops amortized $O(1)$: each element transferred at most once across the two stacks.

Space $O(n)$: elements split across inbox and outbox at any moment.

Follow-up: Implement Stack using Queues (#225) – symmetric design with one queue.

Follow-up: Design Circular Queue (#622) – fixed capacity, $O(1)$ all ops with a ring buffer."

Community Solutions (Python)

• WalkCC

```

class MyQueue:
    def __init__(self):
        self.input = []
        self.output = []

    def push(self, x: int) -> None:
        self.input.append(x)

    def pop(self) -> int:
        self.peak()

        return self.output.pop()

    def peek(self) -> int:
        if not self.output:
            while self.input:
                self.output.append(self.input.pop())
        return self.output[-1]

    def empty(self) -> bool:
        return not self.input and not self.output

```

• LeetDoocs

```

class MyQueue:
    def __init__(self):
        self.stk1 = []
        self.stk2 = []

    def push(self, x: int) -> None:
        self.stk1.append(x)

    def pop(self) -> int:
        self.move()

```

```

        return self.stk2.pop()

    def peek(self) -> int:
        self.move()
        return self.stk2[-1]

    def empty(self) -> bool:
        return not self.stk1 and not self.stk2

    def move(self):

```

```

        if not self.stk2:
            while self.stk1:
                self.stk2.append(self.stk1.pop())

```

```

# Your MyQueue object will be instantiated and called as such:
# obj = MyQueue()
# obj.push(x)
# param_2 = obj.pop()
# param_3 = obj.peek()

```

```

# param_4 = obj.empty()

```

• SimplyLeet

```

class MyQueue:
    def __init__(self):
        # Stack for enqueue operations.
        self.input_stack = []
        # Stack for dequeue operations.
        self.output_stack = []

    def push(self, x: int) -> None:
        # Always push new elements onto the input stack.
        self.input_stack.append(x)

```



```
def pop(self) -> int:
    # Ensure the output stack has the current queue order.
    if not self.output_stack:
        while self.input_stack:
            # Move all elements from input_stack to output_stack.
            self.output_stack.append(self.input_stack.pop())
    # The top element of output_stack is the front of the queue.
    return self.output_stack.pop()
```

```
def peek(self) -> int:
    # Ensure the output stack has the current queue order.
    if not self.output_stack:
        while self.input_stack:
            # Move all elements from input_stack to output_stack.
            self.output_stack.append(self.input_stack.pop())
    # Peek the top of output_stack without popping.
    return self.output_stack[-1]
```

```
def empty(self) -> bool:

    # The queue is empty if both stacks are empty.
    return not self.input_stack and not self.output_stack
```

• LC.ca

```
class MyQueue:
    def __init__(self):
        self.stk1 = []
        self.stk2 = [] # reversed order

    def push(self, x: int) -> None:
        self.stk1.append(x)

    def pop(self) -> int:
        self.move()
```

```

        return self.stk2.pop()

    def peek(self) -> int:
        self.move()
        return self.stk2[-1]

    def empty(self) -> bool:
        return not self.stk1 and not self.stk2

    def move(self):

```

```

        if not self.stk2: # only when skt2 is empty
            while self.stk1:
                self.stk2.append(self.stk1.pop())

```

```

# Your MyQueue object will be instantiated and called as such:
# obj = MyQueue()
# obj.push(x)
# param_2 = obj.pop()
# param_3 = obj.peek()

```

```

# param_4 = obj.empty()

```

```

#####

```

```

class MyQueue:

```

```

    def __init__(self):
        self.sk = []
        self.rsk = [] # reversed

```

```

    def push(self, x: int) -> None:
        self.sk.append(x);

```

```

    def pop(self) -> int:
        self.peek()
        return self.rsk.pop()

```

```

    def peek(self) -> int:
        if self.rsk:

```

```
        return self.rsk[-1]
    else:
        while self.sk:
            self.rsk.append(self.sk.pop())
        return self.rsk[-1]

    def empty(self) -> bool:

        return not (self.sk or self.rsk)
```

```
# Your MyQueue object will be instantiated and called as such:
# obj = MyQueue()
# obj.push(x)
# param_2 = obj.pop()
# param_3 = obj.peek()
# param_4 = obj.empty()
```

◆ Quick Review

Key Insights

- Use two stacks:
- One (input stack) for enqueue operations.
- The other (output stack) for dequeue operations.
- When performing pop or peek, if the output stack is empty, transfer all elements from the input stack to the output stack. This reversal makes the oldest element accessible.

- Each element is moved at most once between the stacks, ensuring an amortized $O(1)$ cost per operation.
- The space complexity is $O(n)$ where n is the number of elements in the queue.

Space & Time

Time Complexity: Amortized $O(1)$ per operation

Space Complexity: $O(n)$

Solution

We maintain two stacks: an input stack for pushing new elements and an output stack for popping or peeking the front element of the queue. When we need to access the front of the queue (either during a pop or peek), we check if the output stack is empty. If it is, we transfer all elements from the input stack to the output stack. This reversal of order allows us to simulate the FIFO nature of the queue. When performing a push, the element is simply added to the input stack.

Stack

□ #844 Backspace String Compare

EAS[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · String · Stack · Simulation
Lists: Grind 169

Problem

Given two strings *s* and *t*, return true if they are equal when both are typed into empty text editors. '#' means a backspace character.

Note that after backspacing an empty text, the text will continue empty.

Example 1:

Input: *s* = "ab#c", *t* = "ad#c"
Output: true
Explanation: Both *s* and *t* become "ac".

Example 2:

Input: *s* = "ab##", *t* = "c#d#"
Output: true
Explanation: Both *s* and *t* become "".

Example 3:

Input: *s* = "a#c", *t* = "b"
Output: false
Explanation: *s* becomes "c" while *t* becomes "b".

Constraints:

- $1 \leq s.length, t.length \leq 200$
- s and t only contain lowercase letters and '#' characters.

Follow up: Can you solve it in $O(n)$ time and $O(1)$ space?

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# '#' is a backspace character that deletes the previous character (if any).
# We simulate typing both strings into an empty text editor and return
# true if the final texts are equal. Right?"
#
# "A few quick questions:
# What if '#' appears at the start with nothing to delete? It's a no-op.
# Can strings be empty? Yes – empty result equals empty result.
# The follow-up asks for  $O(n)$  time and  $O(1)$  space."
#
# "Let me walk: s='ab#c', t='ad#c'.
# s: a,b,# → a, c → 'ac'. t: a,d,# → a, c → 'ac'. Equal → true ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Simulate both strings using a stack: push normal chars, pop on '#' if non-empty.
# Compare the resulting strings.
#  $O(n+m)$  time,  $O(n+m)$  space for the two stacks.
# Should I go ahead and optimize?"
class _844_BackspaceStringCompare_Brute:
    def backspaceCompare(self, s, t):
        def process(text):
            stack = []
            for ch in text:
                if ch != '#': stack.append(ch)
                elif stack: stack.pop()
            return stack
        return process(s) == process(t)
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the O(n+m) extra space for the stacks.
# Scan both strings from the RIGHT with two pointers.
# Use a skip counter: every '#' increments skip; a regular char while skip>0 is
  skipped.
# The first non-skipped char from each string is compared.
# We don't need any extra storage – just integer counters.
#
# Time: O(n+m). Space: O(1).
# Does that make sense before I code it?"
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _844_BackspaceStringCompare:
    def backspaceCompare(self, s, t):
        i, j = len(s) - 1, len(t) - 1
        skip_s = skip_t = 0
        while i >= 0 or j >= 0:
            while i >= 0:
                if s[i] == '#': skip_s += 1; i -= 1
                elif skip_s > 0: skip_s -= 1; i -= 1
                else: break
            while j >= 0:
                if t[j] == '#': skip_t += 1; j -= 1
                elif skip_t > 0: skip_t -= 1; j -= 1
                else: break
            if i >= 0 and j >= 0 and s[i] != t[j]:
                return False
            if (i >= 0) != (j >= 0):
                return False
            i -= 1; j -= 1
        return True
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# s='ab#c', t='ad#c' (i=3,j=3):
# s[3]='c' non-# → compare. t[3]='c' → compare. 'c'=='c' ✓. i=2,j=2.
# s[2]='#' → skip_s=1, i=1. s[1]='b', skip_s>0 → skip, i=0. s[0]='a' non-# →
  compare.
# t[2]='#' → skip_t=1, j=1. t[1]='d', skip → j=0. t[0]='a' → compare. 'a'=='a' ✓.
  i=-1,j=-1.
# both exhausted → True ✓
#
# "No bugs. The (i>=0) != (j>=0) check catches mismatched remaining lengths."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Stack approach: O(n+m) time, O(n+m) space – simple and readable.
# Two-pointer approach: O(n+m) time, O(1) space – optimal.
# Follow-up: if '#' can delete k characters at once, adjust skip counter to add k.
```


Follow-up: Minimum Remove to Make Valid Parentheses (#1249) – similar stack-simulation idea."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def backspaceCompare(self, s: str, t: str) -> bool:
        def backspace(s: str) -> str:
            stack = []
            for c in s:
                if c != '#':
                    stack.append(c)
                elif stack:
                    stack.pop()
            return stack

        return backspace(s) == backspace(t)
```

```
class Solution:
    def backspaceCompare(self, s: str, t: str) -> bool:
        i = len(s) - 1 # s' index
        j = len(t) - 1 # t's index

        while True:
            # Delete characters of s if needed.
            backspace = 0
            while i >= 0 and (s[i] == '#' or backspace > 0):
                backspace += 1 if s[i] == '#' else -1
                i -= 1

            # Delete characters of t if needed.
            backspace = 0
            while j >= 0 and (t[j] == '#' or backspace > 0):
                backspace += 1 if t[j] == '#' else -1
                j -= 1

            if i >= 0 and j >= 0 and s[i] == t[j]:
                i -= 1
                j -= 1
            else:
                return i < j
```

```
break
```

```
return i == -1 and j == -1
```

• LeetDoocs

```
class Solution:
    def backspaceCompare(self, s: str, t: str) -> bool:
        i, j, skip1, skip2 = len(s) - 1, len(t) - 1, 0, 0
        while i >= 0 or j >= 0:
            while i >= 0:
                if s[i] == '#':
                    skip1 += 1
                    i -= 1
                elif skip1:
                    skip1 -= 1
                    i -= 1
                else:
                    break
            while j >= 0:
                if t[j] == '#':
                    skip2 += 1
                    j -= 1
                elif skip2:
                    skip2 -= 1
                    j -= 1
                else:
                    break
            if i >= 0 and j >= 0:
                if s[i] != t[j]:
                    return False
            elif i >= 0 or j >= 0:
                return False
            i, j = i - 1, j - 1
        return True
```

• SimplyLeet

```
# Function to compare two strings with backspace processing
def backspaceCompare(s: str, t: str) -> bool:
    # Pointers for both strings starting at the end
    i, j = len(s) - 1, len(t) - 1

    # Process both strings until all characters are compared
    while i >= 0 or j >= 0:
        # Process string s: skip backspaced characters
        skip = 0
        while i >= 0:
            if s[i] == '#':
                skip += 1
                i -= 1
            elif skip > 0:
                skip -= 1
                i -= 1
            else:
                break

        # Process string t: skip backspaced characters
        skip = 0
        while j >= 0:
            if t[j] == '#':
                skip += 1
                j -= 1
            elif skip > 0:
                skip -= 1
                j -= 1
            else:
                break

        # Compare the current valid characters from s and t
        if i >= 0 and j >= 0:
            if s[i] != t[j]:
                return False
        elif i >= 0 or j >= 0:
            return False

        i -= 1
        j -= 1
```

```
    return True
```

```
# Example test cases
```

```
print(backspaceCompare("ab#c", "ad#c")) # Expected output: True
print(backspaceCompare("ab##", "c#d#")) # Expected output: True
print(backspaceCompare("a#c", "b"))    # Expected output: False
```

• LC.ca

```
class Solution:
    def backspaceCompare(self, s: str, t: str) -> bool:
        i, j, skip1, skip2 = len(s) - 1, len(t) - 1, 0, 0
        while i >= 0 or j >= 0:
            while i >= 0:
                if s[i] == '#':
                    skip1 += 1
                    i -= 1
                elif skip1:
                    skip1 -= 1
                    i -= 1
                else:
                    break
            while j >= 0:
                if t[j] == '#':
                    skip2 += 1
                    j -= 1
                elif skip2:
                    skip2 -= 1
                    j -= 1
                else:
                    break
            if i >= 0 and j >= 0:
                if s[i] != t[j]:
                    return False
            elif i >= 0 or j >= 0:
                return False
            i, j = i - 1, j - 1
        return True
```

◆ Quick Review

Key Insights

- '#' represents a backspace operation that deletes the previous character.
- A direct simulation using a stack can reconstruct the final string.
- A two-pointer approach allows comparison from the end of the strings while skipping backspaced characters.
- The two-pointer technique can achieve $O(n)$ time and $O(1)$ space by processing the strings in reverse.

Space & Time

Time Complexity: $O(n)$ where n is the length of the longer string.

Space Complexity: $O(1)$ using the two-pointer strategy ($O(n)$ if using stacks).

Solution

The solution uses two pointers starting from the end of each string. For each string, maintain a skip counter to account for backspace characters. As you iterate backwards: If a '#' is encountered, increment

the skip counter. If a normal character is encountered while the skip counter is positive, skip that character by decrementing the counter. Once a valid character is found, compare the characters from both strings. If all corresponding characters match and both pointers finish processing at the same time, the strings are considered equal.

Trie

Round 4 · Mid · Easy · 1 question

Trie

□ #14 Longest Common Prefix

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Trie

Lists: Grind 169

Problem

Write a function to find the longest common prefix string amongst an array of strings.

If there is no common prefix, return an empty string "".

Example 1:

```
Input: strs = ["flower","flow","flight"]  
Output: "fl"
```

Example 2:

```
Input: strs = ["dog","racecar","car"]  
Output: ""  
Explanation: There is no common prefix among the input strings.
```

Constraints:

- $1 \leq \text{strs.length} \leq 200$
- $0 \leq \text{strs}[i].\text{length} \leq 200$

- `strs[i]` consists of only lowercase English letters if it is non-empty.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def longestCommonPrefix(self, strs: List[str]) -> str:
        if not strs:
            return ''
        prefix = ''
        for i in range(len(strs[0])):
            c = strs[0][i]
            for word in strs[1:]:
                if i >= len(word) or word[i] != c:
                    return strs[0][:i]

        return strs[0]
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Find the longest prefix shared by ALL strings in the array.

If none exists, return an empty string. Right?"

#

"A few quick questions:

Can the array be empty? No – length >= 1. Can a string be empty? Yes.

Characters are lowercase English letters only."

#

"Let me walk: ['flower', 'flow', 'flight']: f✓ l✓ o vs o vs i → mismatch → 'fl' ✓

['dog', 'racecar', 'car']: d vs r mismatch immediately → '' ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Sort the array; the LCP of the first and last strings must be the LCP of all strings

(since sorting puts the most different strings at opposite ends).

Then compare character by character.

$O(n \log n + m)$ where m is the shortest string length.

Should I go ahead and optimize?"

```
class _14_LongestCommonPrefix_Brute:
    def longestCommonPrefix(self, strs):
        strs.sort()
        first, last = strs[0], strs[-1]
        i = 0
        while i < len(first) and i < len(last) and first[i] == last[i]:
            i += 1
        return first[:i]

# PHASE 3 — OPTIMIZE
# "The bottleneck is the  $O(n \log n)$  sort – we can skip it entirely.
# Vertical scanning: take the first string as reference. For each character
# position,
# check that every other string has the same character at that position.
# The first mismatch (or any string running short) terminates the prefix.
#
# Time:  $O(n*m)$  where  $n$ =number of strings,  $m$ =min string length. Space:  $O(1)$ .
# Does that make sense before I code it?"
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _14_LongestCommonPrefix:
    def longestCommonPrefix(self, strs):
        if not strs:
            return ""
        for i, ch in enumerate(strs[0]):
            for s in strs[1:]:
                if i >= len(s) or s[i] != ch:
                    return strs[0][:i]
        return strs[0]
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# ['flower', 'flow', 'flight']:
# i=0 'f': all match. i=1 'l': all match. i=2 'o': strs[2][2]='i' ≠ 'o' → return
# 'fl' ✓
#
# ['dog', 'racecar', 'car']:
# i=0 'd': strs[1][0]='r' ≠ 'd' → return '' ✓
#
# ['abc']:
# No inner loop fires (only one string) → return 'abc' ✓
#
# "No bugs. Returning strs[0][:i] when i=0 correctly returns empty string."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n*m)$ :  $n$  strings  $\times$   $m$  min length. Space  $O(1)$ : no extra allocation.
# Sort-then-compare is  $O(n \log n)$  – vertical scan is strictly faster.
# Follow-up: if the array is huge but strings are short, binary search on prefix
# length:
```

check if strs[0][:mid] is a prefix of all strings – $O(n \log m)$ total.
 # Follow-up: Trie – insert all strings and walk until a branch point."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestCommonPrefix(self, strs: list[str]) -> str:
        if not strs:
            return ''

        for i in range(len(strs[0])):
            for j in range(1, len(strs)):
                if i == len(strs[j]) or strs[j][i] != strs[0][i]:
                    return strs[0][:i]

        return strs[0]
```

• LeetDoocs

```
class Solution:
    def longestCommonPrefix(self, strs: List[str]) -> str:
        for i in range(len(strs[0])):
            for s in strs[1:]:
                if len(s) <= i or s[i] != strs[0][i]:
                    return s[:i]
        return strs[0]
```

• SimplyLeet

```
def longestCommonPrefix(strs):
    # If no strings exist, return empty
    if not strs:
        return ""
    # Iterate over each index of the first string
    for i in range(len(strs[0])):
        current_char = strs[0][i]
        # Compare the current character with the corresponding one in the rest
        # of the strings
        for s in strs[1:]:
            # If index exceeds length or characters differ, return the prefix
            # found so far
            if i >= len(s) or s[i] != current_char:
                return strs[0][:i]
        # If no mismatch is found, return the complete first string
    return strs[0]

# Example usage:
print(longestCommonPrefix(["flower", "flow", "flight"]))
```

• LC.ca

```
class Solution:
    def longestCommonPrefix(self, strs: List[str]) -> str:
        for i in range(len(strs[0])):
            for s in strs[1:]:
                if len(s) <= i or s[i] != strs[0][i]:
                    return s[:i]
        return strs[0]
```

◆ Quick Review

Key Insights

- Compare characters of the strings one column at a time (vertical scanning).

- The longest possible prefix is limited by the length of the shortest string in the array.
- As soon as a mismatch is found, the prefix up to that point is the answer.
- Handle edge cases such as an empty input array or strings with no common prefix.

Space & Time

Time Complexity: $O(N \cdot M)$ where N is the number of strings and M is the length of the shortest string.

Space Complexity: $O(1)$ additional space (ignoring output space).

Solution

We use the vertical scanning approach: If the input array is empty, return an empty string. Iterate character by character over the first string. For each character, compare it with the corresponding character in each of the other strings. If a mismatch or end of any string is encountered, return the substring from the start up to the current index. If no mismatch is found, the entire first string is the common prefix. This solution ensures that we only traverse the necessary part of the strings and

stops early if a discrepancy is discovered.

Greedy

Round 4 · Mid · Easy · 1 question

Greedy

□ #409 Longest Palindrome

EAS[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Greedy
Lists: Grind 169

Problem

Given a string *s* which consists of lowercase or uppercase letters, return the length of the longest palindrome that can be built with those letters.

Letters are case sensitive, for example, "Aa" is not considered a palindrome.

Example 1:

```
Input: s = "abcccd"
Output: 7
Explanation: One longest palindrome that can be built is "dcccba", whose length is 7.
```

Example 2:

```
Input: s = "a"
Output: 1
Explanation: The longest palindrome that can be built is "a", whose length is 1.
```

Constraints:

- $1 \leq s.length \leq 2000$
- s consists of lowercase and/or uppercase English letters only.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def longestPalindrome(self, s: str) -> int:
        ans = 0
        counts = {c:0 for c in s}
        for c in s:
            counts[c] += 1

        hascenter = False
        for v in counts.values():
            ans += v if v % 2 == 0 else v-1

            if v % 2 == 1 and not hascenter:
                ans += 1
                hascenter = True

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given a string of letters, find the length of the longest palindrome we can BUILD from those characters – we rearrange them, we don't find a substring.

Letters are case-sensitive. Right?"

#

"A few quick questions:

Can we use every character? Yes – we pick the optimal subset.

Return the length, not the actual palindrome string? Yes."

#

"Let me walk: 'abccccdd' → d×2, c×4, a×1, b×1.

Pairs: dd, cccc → 6 chars. One leftover can be center → total 7 ✓"

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Count frequency of each character.
# Sum all (freq // 2) * 2 (use every full pair).
# If any character had an odd frequency, add 1 for the center.
# O(n) time, O(1) space since at most 52 distinct letters.
# This IS essentially optimal – but there's an even cleaner implementation. Should I
code it?"
```

```
class _409_LongestPalindrome_Brute:
    def longestPalindrome(self, s):
        from collections import Counter
        freq = Counter(s)
        length = sum(v // 2 * 2 for v in freq.values())
        if length < len(s):
            length += 1
        return length
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the separate Counter build and scan steps.
# The key insight: we only need to know which characters have ODD frequency.
# Toggle each char in a set – present means odd count so far, absent means even.
# Final answer = len(s) – len(odd_chars) + (1 if odd_chars else 0).
#
# Time: O(n). Space: O(1) – at most 52 entries.
# Does that make sense before I code it?"
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _409_LongestPalindrome:
    def longestPalindrome(self, s):
        odd_chars = set()
        for ch in s:
            if ch in odd_chars:
                odd_chars.discard(ch)
            else:
                odd_chars.add(ch)
        return len(s) – len(odd_chars) + (1 if odd_chars else 0)
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# 'abcccd':
# a→{a}, b→{a,b}, c→{a,b,c}, c→{a,b}, c→{a,b,c}, c→{a,b}, d→{a,b,d}, d→{a,b}.
# odd_chars={a,b}. len(s)=8. 8–2+1=7 ✓
#
# 'a':
# odd_chars={a}. 1–1+1=1 ✓
#
# 'aa':
# odd_chars={}. 2–0+0=2 ✓
#
```

"No bugs. The +1 only fires when there are leftover characters to place at the center."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: one pass. Space $O(1)$: set has at most 52 entries.

The set-toggle is slightly more elegant than Counter since it avoids the intermediate dict.

Both approaches have identical complexity.

Follow-up: Palindrome Pairs (#336) – build palindromes from word concatenations.

Follow-up: if we must also return the actual palindrome, assign pairs to both ends

and place one odd character in the middle."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestPalindrome(self, s: str) -> int:
        ans = 0
        count = collections.Counter(s)

        for c in count.values():
            ans += c if c % 2 == 0 else c - 1

        hasOddCount = any(c % 2 == 1 for c in count.values())
        return ans + hasOddCount
```

• LeetDoocs

```
class Solution:
    def longestPalindrome(self, s: str) -> int:
        cnt = Counter(s)
        ans = sum(v // 2 * 2 for v in cnt.values())
        ans += int(ans < len(s))
        return ans
```

```
class Solution:
    def longestPalindrome(self, s: str) -> int:
        odd = defaultdict(int)
        cnt = 0
        for c in s:
            odd[c] ^= 1
            cnt += 1 if odd[c] else -1
        return len(s) - cnt + 1 if cnt else len(s)
```

• SimplyLeet

```
# Python solution for the Longest Palindrome problem

def longestPalindrome(s: str) -> int:
    # Create a dictionary to store the frequency of each character
    char_counts = {}
    for char in s:
        # Increment the count for the character
        char_counts[char] = char_counts.get(char, 0) + 1

    # Variable to store the length of the palindrome

    palindrome_length = 0
    # Flag to indicate whether we can add one more character in the middle for
    # an odd count
    center_added = False

    # Iterate through the character counts
    for count in char_counts.values():
        # Add the largest even number from count to palindrome_length
        palindrome_length += (count // 2) * 2
        # If count is odd and we haven't added the center character yet
        if count % 2 == 1 and not center_added:

            palindrome_length += 1
            center_added = True

    return palindrome_length

# Example usage:
print(longestPalindrome("abcccd"))
```

• LC.ca

```
class Solution:
    def longestPalindrome(self, s: str) -> int:
        cnt = Counter(s)
        ans = 0
        for v in cnt.values():
            ans += v - (v & 1)
            ans += (ans & 1 ^ 1) and (v & 1)
        return ans
```

◆ Quick Review

Key Insights

- Count the frequency of each character.
- For each character, you can use pairs (even count or odd count minus one) to form the palindrome.
- If any character has an odd frequency, you can add one extra character as the center of the palindrome.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string.

Space Complexity: $O(1)$ since the frequency counter will have at most 52 keys (considering both uppercase and lowercase letters).

Solution

The solution uses a hash table to count the frequency of each character. For every character, the maximum number of characters that can be used in a palindrome is determined by taking the largest even number that is less than or equal to its frequency (which is

frequency – (frequency % 2)). If there exists any character with an odd frequency, one of these can be used as the center of the palindrome, adding an extra character to the total length. This greedy approach efficiently computes the length of the largest possible palindrome from the available characters.

Round 5 · Mid · Medium

Round 5

Mid Priority · Medium

56 questions · 6 patterns

**Linked List #2 #19 #24 #61 #146 #148 #328
#369 #430 #622 #707 #708 #1171**

**Stack #143 #150 #155 #173 #227 #255 #394
#439 #484 #735 #739 #962 #1214 #1762**

**Heap #215 #253 #347 #505 #621 #658 #787
#973 #1057 #1167 #1424**

Trie #139 #208 #211 #616 #692 #720 #1166

Backtracking #17 #22 #39 #46 #79 #113

Greedy #134 #179 #280 #954 #2131

Linked List

Round 5 · Mid · Medium · 13 questions

Linked List

□ #2 Add Two Numbers

MED[Open on LeetCode](#)

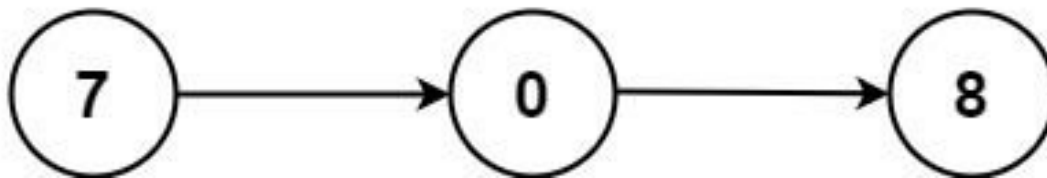
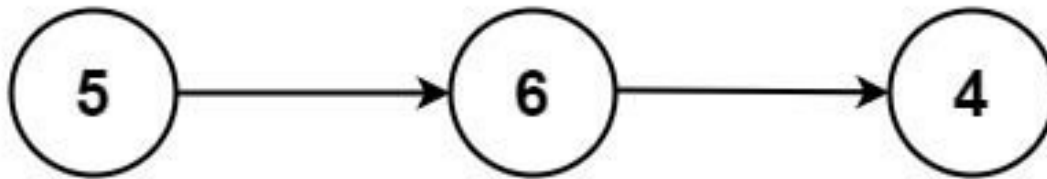
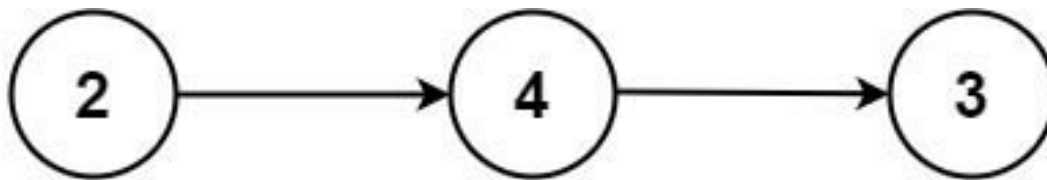
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Math · Recursion
Lists: Grind 169

Problem

You are given two non-empty linked lists representing two non-negative integers. The digits are stored in reverse order, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:



Input: l1 = [2,4,3], l2 = [5,6,4]

Output: [7,0,8]

Explanation: 342 + 465 = 807.

Example 2:

Input: l1 = [0], l2 = [0]

Output: [0]

Example 3:

Input: l1 = [9,9,9,9,9,9,9], l2 = [9,9,9,9]

Output: [8,9,9,9,0,0,0,1]

Constraints:

- The number of nodes in each linked list is in the range [1, 100].

- $0 \leq \text{Node.val} \leq 9$
- It is guaranteed that the list represents a number that does not have leading zeros.

My LeetCode Solution

• My LeetCode Solution

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def addTwoNumbers(self, l1: Optional[ListNode], l2: Optional[ListNode]) ->
Optional[ListNode]:
        dummy = ListNode()
        carry = 0
        current = dummy

        while l1 or l2 or carry:
            total = carry
            if l1:
                total += l1.val
                l1 = l1.next

            if l2:
                total += l2.val
                l2 = l2.next

            d = total % 10
            current.next = ListNode(d)
            carry = total // 10
            current = current.next

        return dummy.next
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Two non-empty linked lists store digits of non-negative integers in REVERSE order.
# Add the two numbers and return the sum as a reversed linked list. Right?"
#
# "A few quick questions:
# Can leading zeros appear in the input? Only for the number 0 itself.
# Can the lists have different lengths? Yes – one might be much longer.
# Should I handle carry past the longer list? Yes – e.g. 9+9 propagates."
#
# "Let me walk: l1=[2,4,3] (342), l2=[5,6,4] (465). 342+465=807 → [7,0,8] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Convert both lists to integers, add them, then convert the result back to a linked
list.
# This is O(n) time and O(n) space but relies on Python big-int arithmetic and
# doesn't generalise to languages with fixed-width integers.
# Should I go ahead and optimize?"
class _2_AddTwoNumbers_Brute:
    def addTwoNumbers(self, l1, l2):
        def to_int(node):
            val, mul = 0, 1
            while node: val += node.val * mul; mul *= 10; node = node.next
            return val
        total = to_int(l1) + to_int(l2)
        dummy = cur = ListNode(0)
        if total == 0: return ListNode(0)
        while total:
            cur.next = ListNode(total % 10); cur = cur.next; total //= 10
        return dummy.next
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the big-int conversion – it breaks in fixed-width languages.
# Simulate grade-school addition digit by digit with a carry variable.
# Walk both lists simultaneously; at each step: total = d1 + d2 + carry.
# New digit = total % 10, new carry = total // 10.
# Continue until both lists are exhausted AND carry is 0.
#
# Time: O(max(m,n)). Space: O(max(m,n)) for the result list.
# Does that make sense before I code it?"
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _2_AddTwoNumbers:
    def addTwoNumbers(self, l1, l2):
        dummy = cur = ListNode(0)
        carry = 0
```

```

while l1 or l2 or carry:
    d1 = l1.val if l1 else 0
    d2 = l2.val if l2 else 0
    total = d1 + d2 + carry
    carry, digit = divmod(total, 10)
    cur.next = ListNode(digit)
    cur = cur.next
    if l1: l1 = l1.next
    if l2: l2 = l2.next
return dummy.next

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

l1=[2,4,3], l2=[5,6,4]:

2+5+0=7 carry=0 → node(7). 4+6+0=10 carry=1 → node(0). 3+4+1=8 carry=0 → node(8).

Both None, carry=0 → stop. → [7,0,8] ✓

#

l1=[9,9,9,9], l2=[9,9,9]:

Each position: 9+9+carry propagates. → [8,9,9,0,1] ✓

#

"No bugs. The loop condition 'while l1 or l2 or carry' handles unequal lengths cleanly."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\max(m,n))$: process as many digits as the longer number plus at most one carry.

Space $O(\max(m,n))$: result list.

Follow-up: Add Two Numbers II (#445) – digits in forward order; use two stacks to reverse.

Follow-up: Multiply Strings (#43) – similar positional digit logic, $O(n*m)$."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def addTwoNumbers(self, l1: ListNode, l2: ListNode) -> ListNode:
        dummy = ListNode(0)
        curr = dummy
        carry = 0

        while carry or l1 or l2:
            if l1:
                carry += l1.val
                l1 = l1.next

```



```

    if l2:
        carry += l2.val
        l2 = l2.next
    curr.next = ListNode(carry % 10)
    carry //= 10
    curr = curr.next

    return dummy.next

```

• LeetDoocs

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def addTwoNumbers(
        self, l1: Optional[ListNode], l2: Optional[ListNode]
    ) -> Optional[ListNode]:
        dummy = ListNode()

```

```

        carry, curr = 0, dummy
        while l1 or l2 or carry:
            s = (l1.val if l1 else 0) + (l2.val if l2 else 0) + carry
            carry, val = divmod(s, 10)
            curr.next = ListNode(val)
            curr = curr.next
            l1 = l1.next if l1 else None
            l2 = l2.next if l2 else None
        return dummy.next

```

• SimplyLeet

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def addTwoNumbers(l1, l2):
    # Initialize a dummy head node to simplify the result list construction.
    dummy = ListNode()
    current = dummy

    carry = 0

    # Traverse through both linked lists until all nodes and any carry are
    # processed.
    while l1 or l2 or carry:
        # Get the current values; if a node is missing, use 0.
        val1 = l1.val if l1 else 0
        val2 = l2.val if l2 else 0

        # Calculate the sum of current digits and carry.
        total = val1 + val2 + carry

        carry = total // 10    # Update carry for next iteration.
        digit = total % 10     # Compute current digit.

        # Append the digit as a new node in the result list.
        current.next = ListNode(digit)
        current = current.next

        # Move to the next nodes if available.
        if l1:
            l1 = l1.next

        if l2:
            l2 = l2.next

    # Return the head of the newly formed linked list.
    return dummy.next

```

• LC.ca

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def addTwoNumbers(
        self, l1: Optional[ListNode], l2: Optional[ListNode]
    ) -> Optional[ListNode]:
        dummy = ListNode()

        carry, curr = 0, dummy
        while l1 or l2 or carry:
            s = (l1.val if l1 else 0) + (l2.val if l2 else 0) + carry
            carry, val = divmod(s, 10)
            curr.next = ListNode(val)
            curr = curr.next
            l1 = l1.next if l1 else None
            l2 = l2.next if l2 else None
        return dummy.next

```

◆ Quick Review

Key Insights

- The digits are stored in reverse order, meaning the first node is the least significant digit.
- A carry may need to propagate through subsequent nodes.
- The two linked lists might be of different lengths.
- Continue processing until both lists are fully traversed and any remaining carry is handled.

Space & Time

Time Complexity: $O(\max(n, m))$, where n and m are the lengths of the two linked lists.

Space Complexity: $O(\max(n, m) + 1)$ for the result list in the worst case.

Solution

The solution iterates through both linked lists node by node. For each pair of corresponding nodes, their values and any carry from the previous operation are summed. The result is decomposed into a current digit (modulo 10) and an updated carry (total divided by 10). A new node is created for the current digit and appended to the resulting linked list. This process continues until all nodes are processed and any leftover carry is added as a final node.

Linked List

□ #19 Remove Nth Node From End of List

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode

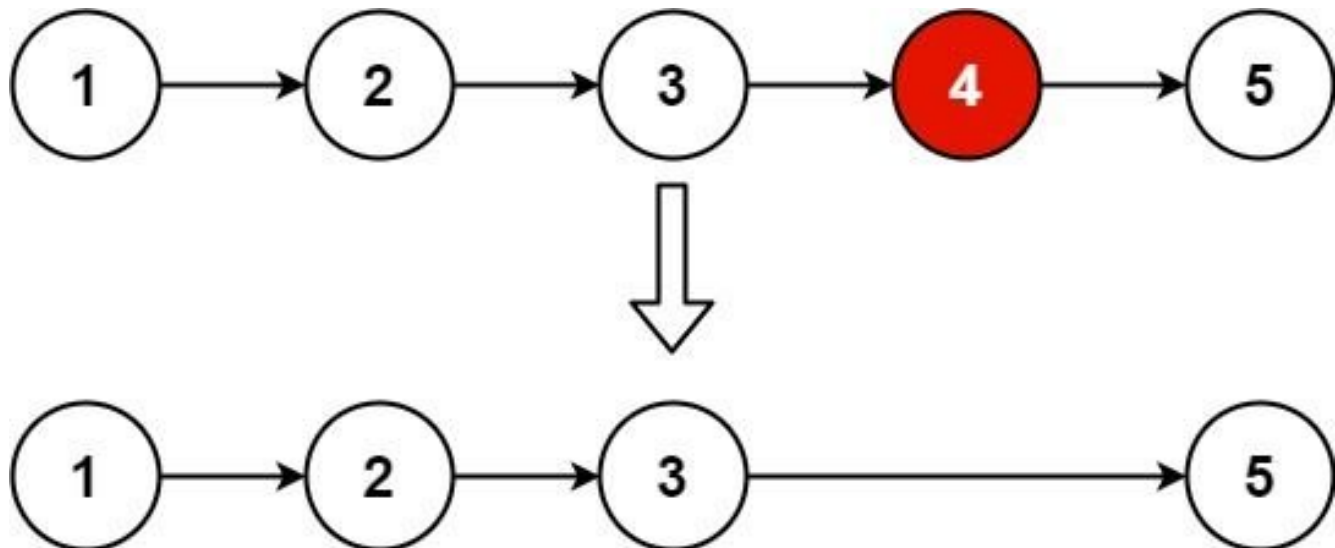
Linked List · Two Pointers

Lists: Grind 169

Problem

Given the head of a linked list, remove the n th node from the end of the list and return its head.

Example 1:



Input: head = [1,2,3,4,5], n = 2
Output: [1,2,3,5]

Example 2:

Input: head = [1], n = 1
Output: []

Example 3:

Input: head = [1,2], n = 1
Output: [1]

Constraints:

- The number of nodes in the list is sz.
- $1 \leq sz \leq 30$
- $0 \leq \text{Node.val} \leq 100$
- $1 \leq n \leq sz$

Follow up: Could you do this in one pass?

My LeetCode Solution

• My LeetCode Solution

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def removeNthFromEnd(self, head: Optional[ListNode], n: int) -> Optional[ListNode]:
        d = ListNode()
        d.next = head
        fast = slow = d
```

```

while n+1:
    fast = fast.next
    n-=1

while fast:
    fast = fast.next
    slow = slow.next

slow.next = slow.next.next
return d.next

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Remove the nth node from the END of the list and return the modified head.

n is always valid. The follow-up asks for one pass. Right?"

#

"A few quick questions:

Can n equal the list length (removing the head)? Yes – handle with a dummy node.

Return the head, which may change if we remove it."

#

"Let me walk: [1,2,3,4,5], n=2 → remove 4 → [1,2,3,5] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Two passes: count length L in the first pass, then walk to position L-n and unlink.

Use a dummy head to cleanly handle removing the actual head node.

O(L) time, O(1) space – correct but uses two passes.

Should I go ahead and optimize?"

```
class _19_RemoveNthFromEnd_Brute:
```

```
    def removeNthFromEnd(self, head, n):
```

```
        dummy = ListNode(0, head)
```

```
        length, cur = 0, head
```

```
        while cur: length += 1; cur = cur.next
```

```
        cur = dummy
```

```
        for _ in range(length - n): cur = cur.next
```

```
        cur.next = cur.next.next
```

```
        return dummy.next
```

PHASE 3 — OPTIMIZE

"The bottleneck is the two-pass approach – we can do it in one.

Advance a 'fast' pointer n+1 steps ahead of 'slow'.

Move both together until fast reaches None.

Then slow.next is the node to remove.


```

# The dummy head ensures we never special-case removing the actual head.
#
# Time: O(L). Space: O(1).
# Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _19_RemoveNthFromEnd:
    def removeNthFromEnd(self, head, n):
        dummy = ListNode(0, head)
        fast = slow = dummy
        for _ in range(n + 1):
            fast = fast.next
        while fast:
            fast = fast.next
            slow = slow.next
        slow.next = slow.next.next
        return dummy.next

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# [1,2,3,4,5], n=2: advance fast 3 steps → dummy,1,2,3. slow=dummy.
# Move together: slow=1,fast=4 → slow=2,fast=5 → slow=3,fast=None → stop.
# slow=3. 3.next=5. → [1,2,3,5] ✓
#
# [1], n=1: fast advances n+1=2 steps → fast=None after first step (head.next=None).
# while fast skipped. slow=dummy. dummy.next=None. → [] ✓
#
# "No bugs. Advancing n+1 (not n) steps puts slow just before the target node."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(L): one pass. Space O(1): two pointer variables plus the dummy node.
# Dummy node is essential – without it, removing the head requires a special branch.
# Follow-up: Middle of Linked List (#876) – same slow/fast pointer family.
# Follow-up: if n could be invalid (> list length), add a length check before
advancing."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def removeNthFromEnd(self, head: ListNode, n: int) -> ListNode:
        slow = head
        fast = head

        for _ in range(n):
            fast = fast.next
        if not fast:
            return head.next

        while fast.next:
            slow = slow.next
            fast = fast.next
        slow.next = slow.next.next

        return head

```

• LeetDoocs

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def removeNthFromEnd(self, head: Optional[ListNode], n: int) -> Optional[ListNode]:
        dummy = ListNode(next=head)
        fast = slow = dummy
        for _ in range(n):
            fast = fast.next
        while fast.next:
            slow, fast = slow.next, fast.next
        slow.next = slow.next.next
        return dummy.next

```

• SimplyLeet

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val      # Holds the value of the node
        self.next = next    # Points to the next node in the list

class Solution:
    def removeNthFromEnd(self, head: ListNode, n: int) -> ListNode:
        # Create a dummy node to simplify edge cases (like removing the head)
        dummy = ListNode(0)

        dummy.next = head

        # Initialize two pointers starting at the dummy node
        first = dummy
        second = dummy

        # Advance first pointer n+1 steps ahead to maintain a gap of n nodes
        for _ in range(n + 1):
            first = first.next

        # Move both pointers until first reaches the end of the list
        while first:
            first = first.next
            second = second.next

        # Skip the node to remove it from the list
        second.next = second.next.next

        # Return the adjusted list starting from dummy.next
        return dummy.next

```

☐ • LC.ca

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def removeNthFromEnd(self, head: Optional[ListNode], n: int) -> Optional[ListNode]:
        dummy = ListNode(next=head)
        fast = slow = dummy
        for _ in range(n):
            fast = fast.next
        while fast.next:
            slow, fast = slow.next, fast.next
        slow.next = slow.next.next
        return dummy.next
```

◆ Quick Review

Key Insights

- Use a dummy node at the start to simplify edge cases, especially when the head is removed.
- Utilize two pointers (fast and slow) and maintain a gap of $n+1$ nodes between them.
- By moving both pointers simultaneously, the slow pointer will point to the node immediately before the target node when the fast pointer reaches the end.

Space & Time

Time Complexity: $O(L)$, where L is the length of the list

Space Complexity: $O(1)$

Solution

We use a two-pointer approach to achieve a one-pass solution. A dummy node is created and linked to the head to handle edge cases gracefully (such as needing to remove the head). The fast pointer is advanced $n+1$ steps ahead of the slow pointer to maintain the required gap. Then, both pointers are moved simultaneously until the fast pointer reaches the end of the list. At that point, the slow pointer is immediately before the node to be deleted. We then adjust the pointers to remove the target node from the list.

Linked List

□ #24 Swap Nodes in Pairs

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Recursion
Lists: Grind 169

Problem

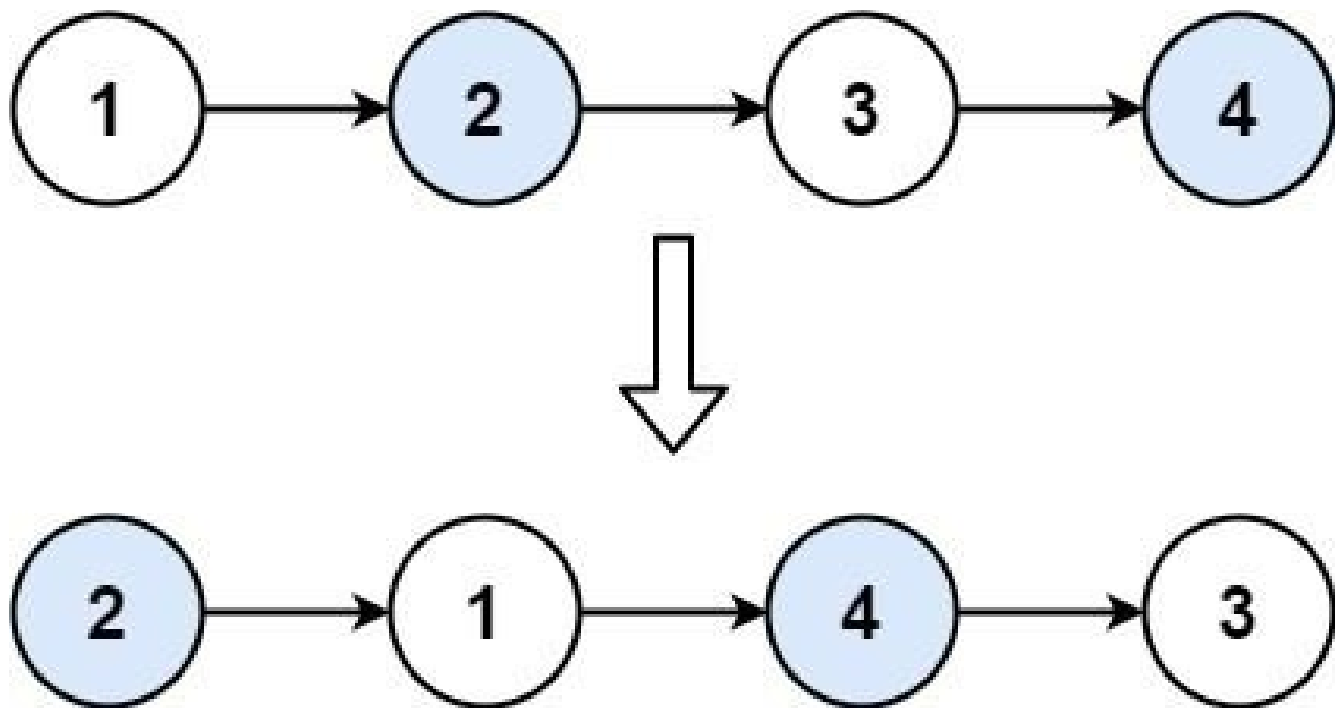
Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

Example 1:

Input: head = [1,2,3,4]

Output: [2,1,4,3]

Explanation:



Example 2:

Input: head = []

Output: []

Example 3:

Input: head = [1]

Output: [1]

Example 4:

Input: head = [1,2,3]

Output: [2,1,3]

Constraints:

- The number of nodes in the list is in the range [0, 100].
- $0 \leq \text{Node.val} \leq 100$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Swap every two adjacent nodes and return the new head.
# We swap the actual nodes – not just their values.
# If there's an odd node at the end, leave it unchanged. Right?"
#
# "A few quick questions:
# Can the list be empty? Yes – return None.
# O(1) space (no new nodes, just pointer rewiring)?"
#
# "Let me walk: [1,2,3,4] → [2,1,4,3] ✓. [1,2,3] → [2,1,3] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Collect all values, swap adjacent pairs in the array, rebuild the list.
# O(n) time but O(n) space for the values – and we create new nodes.
# Should I go ahead and optimize?"
class _24_SwapNodesInPairs_Brute:
    def swapPairs(self, head):
        vals = []
        cur = head
        while cur: vals.append(cur.val); cur = cur.next
        for i in range(0, len(vals) - 1, 2): vals[i], vals[i+1] = vals[i+1], vals[i]
        cur = head
        for v in vals: cur.val = v; cur = cur.next
        return head
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the O(n) auxiliary list.
# In-place pointer rewiring with a dummy predecessor:
# For each pair (first, second): set prev.next=second, first.next=second.next,
# second.next=first.
# Advance prev to first (now the second node in output) and repeat.
#
# Time: O(n). Space: O(1).
# Does that make sense before I code it?"
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _24_SwapNodesInPairs:
    def swapPairs(self, head):
        dummy = ListNode(0, head)
        prev = dummy
        while prev.next and prev.next.next:
```

```

        first = prev.next
        second = prev.next.next
        prev.next = second
        first.next = second.next
        second.next = first
        prev = first
    return dummy.next

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

[1,2,3,4]: dummy→1→2→3→4.

prev=dummy, first=1, second=2: dummy→2, 1→3, 2→1. prev=1. List: dummy→2→1→3→4.

prev=1, first=3, second=4: 1→4, 3→None, 4→3. prev=3. List: dummy→2→1→4→3.

prev.next=None → stop. → [2,1,4,3] ✓

#

[1]: prev.next.next=None → loop never enters → [1] ✓

#

"No bugs. Saving 'second.next' before rewiring prevents losing the rest of the list."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: visit each node once. Space $O(1)$: constant pointer variables.

The dummy node eliminates special-casing the very first pair.

Follow-up: Reverse Nodes in k-Group (#25) – generalise to groups of k using the same logic.

Follow-up: recursive solution swaps the pair then recurses on head.next.next – clean but $O(n)$ stack."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def swapPairs(self, head: ListNode) -> ListNode:
        def getLength(head: ListNode) -> int:
            length = 0
            while head:
                length += 1
                head = head.next
            return length

        length = getLength(head)

```

```

dummy = ListNode(0, head)
prev = dummy
curr = head

for _ in range(length // 2):
    next = curr.next
    curr.next = next.next
    next.next = prev.next
    prev.next = next
    prev = curr

    curr = curr.next

return dummy.next

```

• LeetCode

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def swapPairs(self, head: Optional[ListNode]) -> Optional[ListNode]:
        if head is None or head.next is None:
            return head
        t = self.swapPairs(head.next.next)

        p = head.next
        p.next = head
        head.next = t
        return p

```

• SimplyLeet

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def swapPairs(self, head: ListNode) -> ListNode:
        # Create a dummy node and point it to the head of the list.
        dummy = ListNode(0)

        dummy.next = head
        # Initialize the pointer 'prev' to dummy.
        prev = dummy

        # Traverse the list while at least two nodes remain.
        while head and head.next:
            # Identify the two nodes to swap.
            first_node = head
            second_node = head.next

            # Swapping the nodes by reassigning pointers.
            prev.next = second_node          # Link previous node to second
node.
            first_node.next = second_node.next # Link first node to node after
second.
            second_node.next = first_node     # Link second node to first to
complete swap.

            # Update the pointers for next pair processing.
            prev = first_node                # Move 'prev' pointer to the end
of the swapped pair.
            head = first_node.next           # Move 'head' pointer to the next
pair.

            # Return the new head of the list.

        return dummy.next

```

• LC.ca

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def swapPairs(self, head: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode(next=head)
        pre, cur = dummy, head
        while cur and cur.next:
            t = cur.next
            cur.next = t.next
            t.next = cur
            pre.next = t
            pre, cur = cur, cur.next
        return dummy.next
```

◆ Quick Review

Key Insights

- Use a dummy node to simplify edge cases, such as when the list is empty or has only one node.
- Process nodes in pairs and update the pointers accordingly.
- Ensure that the swapping does not lose the connection between the swapped pair and the rest of the list.
- The solution can be implemented iteratively or recursively.

Space & Time

Time Complexity: $O(n)$ – Each node is processed once.

Space Complexity: $O(1)$ – Only constant extra space is used.

Solution

We use an iterative approach with a dummy node. The dummy node points to the head of the list. We maintain a pointer that traverses the list in pairs. For each pair, we adjust the next pointers to swap the two nodes. By the end, the dummy's next pointer will be the new head of the modified list. The main idea is to keep track of the previous node (prev) before the pair, and then adjust pointers accordingly using temporary variables to hold references to the nodes being swapped.

Linked List

□ #61 Rotate List

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode

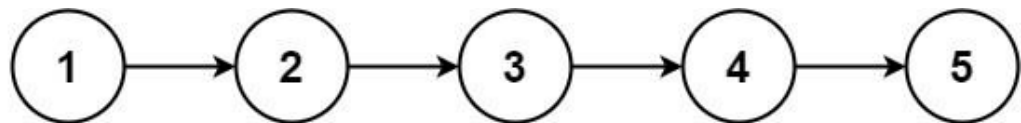
Linked List · Two Pointers

Lists: Grind 169

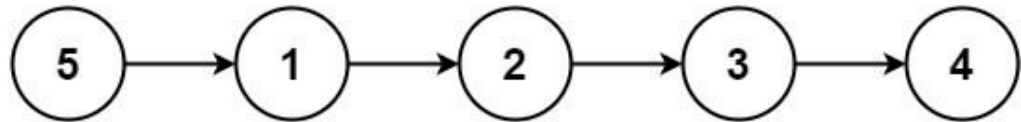
Problem

Given the head of a linked list, rotate the list to the right by k places.

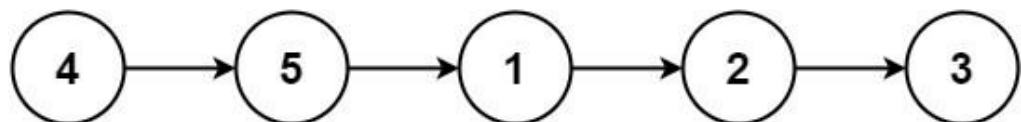
Example 1:



rotate 1

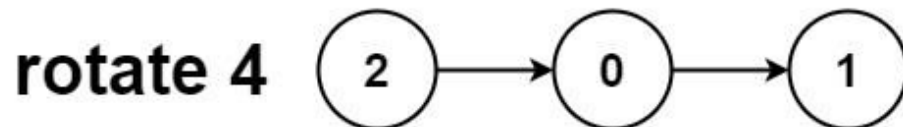
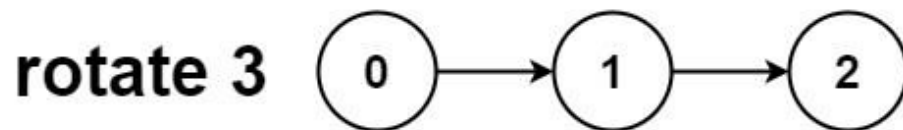
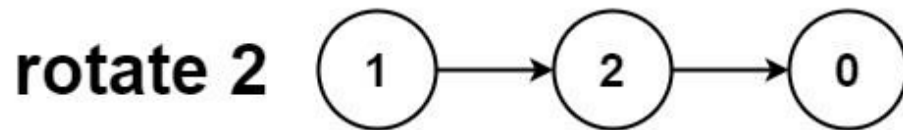
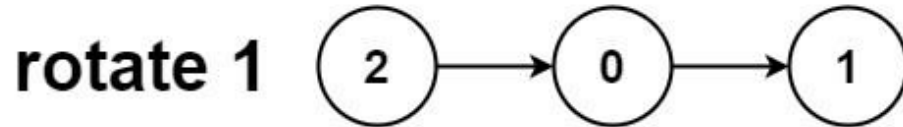
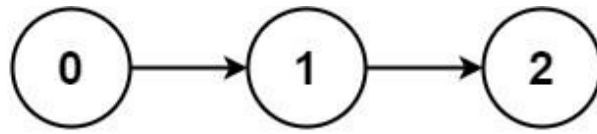


rotate 2



Input: head = [1,2,3,4,5], k = 2
Output: [4,5,1,2,3]

Example 2:



Input: head = [0,1,2], k = 4
Output: [2,0,1]

Constraints:

- The number of nodes in the list is in the range [0, 500].
- $-100 \leq \text{Node.val} \leq 100$
- $0 \leq k \leq 2 * 10^9$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Rotate the linked list to the right by k places – each rotation moves the tail to
the front.
# k can exceed the list length; rotations are modulo the length. Right?"
#
# "A few quick questions:
# Can k be 0? Yes – return unchanged. Can the list be empty? Yes – return None.
# k up to  $2 \times 10^9$  – so we definitely need the modulo trick."
#
# "Let me walk: [1,2,3,4,5], k=2 → [4,5,1,2,3] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Perform k individual right-rotations: each time, remove the tail and prepend it to
the head.
#  $O(n \times k)$  time – for  $k = 2 \times 10^9$  this is completely infeasible.
# Should I go ahead and optimize?"
class _61_RotateList_Brute:
    def rotateRight(self, head, k):
        if not head or not head.next: return head
        for _ in range(k):
            cur = head
            while cur.next.next: cur = cur.next
            tail = cur.next; cur.next = None; tail.next = head; head = tail
        return head
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the  $O(n \times k)$  brute – full cycles cancel so we only need  $k \%$ 
length rotations.
# Three steps in two passes:
# 1. Find length and tail by traversing once.
# 2. Effective  $k = k \% \text{length}$ . If 0, return as-is.
# 3. New tail is at position  $(\text{length} - k - 1)$ . New head is new_tail.next.
# Connect old tail to old head and cut at new tail.
#
# Time:  $O(n)$ . Space:  $O(1)$ .
# Does that make sense before I code it?"
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _61_RotateList:
    def rotateRight(self, head, k):
        if not head or not head.next or k == 0:
            return head
        tail, length = head, 1
        while tail.next:
            tail = tail.next
```

```

        length += 1
    k %= length
    if k == 0:
        return head
    new_tail = head
    for _ in range(length - k - 1):
        new_tail = new_tail.next
    new_head = new_tail.next
    new_tail.next = None
    tail.next = head
    return new_head

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

[1,2,3,4,5], k=2: length=5, tail=5. k=2%5=2. Walk 5-2-1=2 steps → new_tail=3.

new_head=4. 3.next=None. 5.next=1. → [4,5,1,2,3] ✓

#

[0,1,2], k=4: k=4%3=1. Walk 3-1-1=1 step → new_tail=1. new_head=2. 1.next=None. 2.next=0.

→ [2,0,1] ✓

#

"No bugs. k%length=0 short-circuit avoids a no-op cut-and-rejoin."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time O(n): one pass for length, one walk to new_tail. Space O(1)."

The k%length shortcut is critical – without it k=10⁹ on a 5-node list would loop forever.

Follow-up: Rotate Array (#189) – same modulo idea but uses three reversals on an array.

Follow-up: right-shift by 1 on a circular buffer is O(1) with a pointer – different trade-off."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def rotateRight(self, head: ListNode, k: int) -> ListNode:
        if not head or not head.next or k == 0:
            return head

        tail = head
        length = 1
        while tail.next:
            tail = tail.next
            length += 1

```

```

tail.next = head # Circle the list.

t = length - k % length
for _ in range(t):
    tail = tail.next
newHead = tail.next
tail.next = None

return newHead

```

• LeetDoocs

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def rotateRight(self, head: Optional[ListNode], k: int) -> Optional[ListNode]:
        if head is None or head.next is None:
            return head
        cur, n = head, 0

        while cur:
            n += 1
            cur = cur.next
        k %= n
        if k == 0:
            return head
        fast = slow = head
        for _ in range(k):
            fast = fast.next
        while fast.next:
            fast, slow = fast.next, slow.next

        ans = slow.next
        slow.next = None
        fast.next = head
        return ans

```

• SimplyLeet

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None

class Solution:
    def rotateRight(self, head: ListNode, k: int) -> ListNode:
        if not head or not head.next or k == 0: # Check if rotation is
            necessary
            return head

        # Compute the length of the list and locate the tail node
        length = 1
        tail = head
        while tail.next:
            tail = tail.next
            length += 1

        # Calculate the effective number of rotations
        k = k % length

        if k == 0:
            return head

        # Find the new tail: (length - k - 1) steps from the head
        stepsToNewTail = length - k - 1
        newTail = head
        for _ in range(stepsToNewTail):
            newTail = newTail.next

        # Set the new head and break the list

        newHead = newTail.next
        newTail.next = None

        # Connect the old tail to the original head
        tail.next = head

        return newHead
```

• LC.ca

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def rotateRight(self, head: Optional[ListNode], k: int) -> Optional[ListNode]:
        if head is None or head.next is None:
            return head
        cur, n = head, 0

        while cur:
            n += 1
            cur = cur.next
        k %= n
        if k == 0:
            return head
        fast = slow = head
        for _ in range(k):
            fast = fast.next
        while fast.next:
            fast, slow = fast.next, slow.next

        ans = slow.next
        slow.next = None
        fast.next = head
        return ans
```

◆ Quick Review

Key Insights

- Calculate the length of the list and locate the tail.

- Use k modulo length to handle rotations greater than the list size.
- Find the new tail of the list at position $(\text{length} - k - 1)$ and update pointers accordingly.
- Link the original tail to the original head to form a circular list, then break the circle at the new tail.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the list, since we traverse the list a few times.

Space Complexity: $O(1)$, as the rotation is done in place without extra data structures.

Solution

We first traverse the list to determine its length and identify the tail node. The effective rotation is calculated by taking k modulo the length of the list. If the result is zero, the list remains unchanged. Otherwise, we locate the new tail node by moving $(\text{length} - k - 1)$ steps from the head. The node after the new tail becomes the new head. Finally, we break the link after the new tail and connect the old tail

to the original head to finalize the rotated list.

Linked List

□ #146 LRU Cache

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Linked List · Design
Lists: Grind 169

Problem

Design a data structure that follows the constraints of a Least Recently Used (LRU) cache.

Implement the LRUCache class:

- **LRUCache(int capacity)** Initialize the LRU cache with positive size capacity.
- **int get(int key)** Return the value of the key if the key exists, otherwise return -1.
- **void put(int key, int value)** Update the value of the key if the key exists. Otherwise, add the key-value pair to the cache. If the number of keys exceeds the capacity from this operation, evict the least recently used key.

The functions `get` and `put` must each run in $O(1)$ average time complexity.

Example 1:

Input

```
["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]
```

Output

```
[null, null, null, 1, null, -1, null, -1, 3, 4]
```

Explanation

```
LRUCache lRUCache = new LRUCache(2);
```

Constraints:

- $1 \leq \text{capacity} \leq 3000$
- $0 \leq \text{key} \leq 10^4$
- $0 \leq \text{value} \leq 10^5$
- At most $2 * 10^5$ calls will be made to `get` and `put`.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Design an LRU cache with a fixed capacity: `get(key)→value` or `-1`, and `put(key,value)`.

When capacity is exceeded, evict the least recently used item.

Both operations must be $O(1)$ average time. Right?"

#

"A few quick questions:

Does `get` count as a use (making the item recently used)? Yes.

Does `put` update an existing key's value and recency? Yes."

#

"Let me walk capacity=2: `put(1,1)`, `put(2,2)`, `get(1)→1` (1 is now MRU),

`put(3,3)` evicts 2, `get(2)→-1` ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Use a regular dict plus a list tracking insertion order.

On each access, move the key to the end of the list; on eviction pop from the front.

get/put are $O(1)$ for the dict but $O(n)$ for the list shift.

Should I go ahead and optimize?"

```
class _146_LRUCache_Brute:
```

```
    def __init__(self, capacity):
```

```
        self.cap = capacity; self.cache = {}; self.order = []
```

```
    def get(self, key):
```

```
        if key not in self.cache: return -1
```

```
        self.order.remove(key); self.order.append(key); return self.cache[key]
```

```
    def put(self, key, value):
```

```
        if key in self.cache: self.order.remove(key)
```

```
        elif len(self.cache) == self.cap:
```

```
            del self.cache[self.order.pop(0)]
```

```
        self.cache[key] = value; self.order.append(key)
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ list shift on every access.

HashMap + Doubly Linked List gives $O(1)$ for both:

HashMap: key → DLL node ($O(1)$ lookup).

DLL: maintains access order — head=LRU, tail=MRU.

get: move node to tail. put: add at tail; if over capacity, remove from head.

Use sentinel head/tail nodes to avoid edge-case pointer checks.

#

Time: $O(1)$ all ops. Space: $O(\text{capacity})$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _146_LRUCache:
```

```
    class _Node:
```

```
        def __init__(self, key=0, val=0):
```

```
            self.key = key; self.val = val; self.prev = self.next = None
```

```
    def __init__(self, capacity):
```

```
        self.cap = capacity
```

```
        self.map = {}
```

```
        self.head = self._Node(); self.tail = self._Node()
```

```
        self.head.next = self.tail; self.tail.prev = self.head
```

```
    def _remove(self, node):
```

```
        node.prev.next = node.next; node.next.prev = node.prev
```

```
    def _insert_tail(self, node):
```

```
        node.prev = self.tail.prev; node.next = self.tail
```

```
        self.tail.prev.next = node; self.tail.prev = node
```

```
    def get(self, key):
```

```
        if key not in self.map: return -1
```

```
        node = self.map[key]
```

```
        self._remove(node); self._insert_tail(node)
```

```

        return node.val

    def put(self, key, value):
        if key in self.map: self._remove(self.map[key])
        node = self._Node(key, value)
        self.map[key] = node; self._insert_tail(node)
        if len(self.map) > self.cap:
            lru = self.head.next
            self._remove(lru); del self.map[lru.key]

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

cap=2: put(1,1)→tail:[1]. put(2,2)→tail:[1,2]. get(1)→move 1 to tail:[2,1].

put(3,3): over cap, evict head.next=node(2). cache={1,3}. tail:[1,3] ✓

get(2)→-1 (evicted) ✓

#

"No bugs. Sentinel nodes eliminate all null checks in _remove/_insert_tail."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(1)$ all ops: HashMap $O(1)$ lookup + DLL $O(1)$ insert/remove.

Space $O(\text{capacity})$: at most capacity nodes in the DLL and map.

Python's OrderedDict implements this internally – valid in an interview to mention.

Follow-up: LFU Cache (#460) – evict least FREQUENTLY used; needs two hashmaps + DLL per frequency.

Follow-up: thread-safe LRU – wrap get/put with a threading.Lock."

Community Solutions (Python)

• WalkCC

```

class Node:
    def __init__(self, key: int, value: int):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None

class LRUCache:
    def __init__(self, capacity: int):

```

```
self.capacity = capacity
self.keyToNode = {}
self.head = Node(-1, -1)
self.tail = Node(-1, -1)
self.join(self.head, self.tail)

def get(self, key: int) -> int:
    if key not in self.keyToNode:
        return -1

    node = self.keyToNode[key]
    self.remove(node)
    self.moveToHead(node)
    return node.value

def put(self, key: int, value: int) -> None:
    if key in self.keyToNode:
        node = self.keyToNode[key]
        node.value = value
        self.remove(node)

    self.moveToHead(node)
    return

    if len(self.keyToNode) == self.capacity:
        lastNode = self.tail.prev
        del self.keyToNode[lastNode.key]
        self.remove(lastNode)

    self.moveToHead(Node(key, value))
    self.keyToNode[key] = self.head.next

def join(self, node1: Node, node2: Node):
    node1.next = node2
    node2.prev = node1

def moveToHead(self, node: Node):
    self.join(node, self.head.next)
    self.join(self.head, node)

def remove(self, node: Node):
```

```
self.join(node.prev, node.next)
```

• SimplyLeet

```
# Define a node for the doubly linked list.
```

```
class Node:
```

```
    def __init__(self, key, value):
```

```
        self.key = key           # key of the node
```

```
        self.value = value       # value of the node
```

```
        self.prev = None        # pointer to previous node
```

```
        self.next = None        # pointer to next node
```

```
class LRUCache:
```

```
    def __init__(self, capacity):
```

```
        self.capacity = capacity
```

```
        # maximum capacity of the cache
```

```
        self.cache = {}
```

```
        # dictionary mapping key ->
```

```
node
```

```
    # Create dummy head and tail nodes.
```

```
    self.head = Node(0, 0)
```

```
    self.tail = Node(0, 0)
```

```
    self.head.next = self.tail
```

```
    self.tail.prev = self.head
```

```
# Helper function to remove a node from the linked list.
```

```
def _remove(self, node):
```

```
    prev_node = node.prev
```

```
    next_node = node.next
```

```
    prev_node.next = next_node
```

```
    next_node.prev = prev_node
```

```
# Helper function to add a node right after the head.
```

```
def _add(self, node):
```

```
    node.prev = self.head
```

```
    node.next = self.head.next
```

```
    self.head.next.prev = node
```

```

        self.head.next = node

# Returns the value of the key if it exists; otherwise, returns -1.
def get(self, key):
    if key in self.cache:
        node = self.cache[key]
        # Move the accessed node to the head (marking it as recently used)
        self._remove(node)
        self._add(node)
        return node.value

```

```

    return -1

```

```

# Updates the value of the key if it exists; otherwise, adds the key-value
pair.

```

```

# Removes the least recently used key if the cache exceeds its capacity.

```

```

def put(self, key, value):
    if key in self.cache:
        # Remove the old node.
        self._remove(self.cache[key])
    node = Node(key, value)
    self._add(node)

```

```

self.cache[key] = node
if len(self.cache) > self.capacity:
    # Remove the least recently used element.
    lru = self.tail.prev
    self._remove(lru)
    del self.cache[lru.key]

```

• LC.ca

```

class Node:
    def __init__(self, key=0, val=0):
        self.key = key
        self.val = val
        self.prev = None
        self.next = None

```

```

class LRUCache:
    def __init__(self, capacity: int):

```



```

self.cache = {} # key ==> Node(val)
self.head = Node() # dummy node
self.tail = Node() # dummy node
self.capacity = capacity
self.size = 0
self.head.next = self.tail # note: key setup
self.tail.prev = self.head

def get(self, key: int) -> int:
    if key not in self.cache:

        return -1
    node = self.cache[key]
    self.move_to_head(node)
    return node.val

def put(self, key: int, value: int) -> None:
    if key in self.cache:
        node = self.cache[key]
        node.val = value
        self.move_to_head(node)

    else:
        node = Node(key, value)
        self.cache[key] = node
        self.add_to_head(node)
        self.size += 1
        if self.size > self.capacity:
            tail = self.remove_tail()
            self.cache.pop(tail.key)
            self.size -= 1

def move_to_head(self, node):
    self.remove_node(node)
    self.add_to_head(node)

def remove_node(self, node):
    node.prev.next = node.next
    node.next.prev = node.prev

def add_to_head(self, node):
    node.next = self.head.next

```

```

        node.prev = self.head
        self.head.next = node
        node.next.prev = node

    def remove_tail(self):
        node = self.tail.prev
        self.remove_node(node)
        return node

```

```

# Your LRUCache object will be instantiated and called as such:
# obj = LRUCache(capacity)
# param_1 = obj.get(key)
# obj.put(key,value)

```

```
#####
```

```
'''
```

example:

```

>>> od = collections.OrderedDict()
>>>
>>> od[1]=1
>>> od[2]=2
>>> od[3]=3
>>>
>>> od
OrderedDict([(1, 1), (2, 2), (3, 3)])
>>> od.move_to_end(1)
>>> od

```

```

OrderedDict([(2, 2), (3, 3), (1, 1)])
>>>
>>> od.get(1)
1
>>> od.popitem()
(1, 1)
>>> od
OrderedDict([(2, 2), (3, 3)])
>>>
>>> od[1]=1

```

```
>>> od
OrderedDict([(2, 2), (3, 3), (1, 1)])
>>>
>>> od.popitem(last=False)
(2, 2)
>>> od
OrderedDict([(3, 3), (1, 1)])

...

```

```
import collections

class LRUCache:
    def __init__(self, capacity: 'int'):
        self.cache = collections.OrderedDict()
        self.remain = capacity

    def get(self, key: 'int') -> 'int':
        if key not in self.cache:
            return -1

        self.cache.move_to_end(key) # meaning end is the most recently used
        return self.cache.get(key)

    def put(self, key: 'int', value: 'int') -> 'None':
        if key not in self.cache:
            if self.remain > 0:
                self.remain -= 1
            else:
                self.cache.popitem(last=False) # pop start position
        else:

```

```
        self.cache.pop(key)
    self.cache[key] = value # add to end of dict, meaning most recently
used
```

```
#####
```

```
### below solution with no ordered-dict
```

```
class DLinkedList:
```

```
    def __init__(self, key=0, value=0):
```

```
        self.key = key
        self.val = value
        self.next = None
        self.prev = None
```

```
class DLinkedList:
```

```
    def __init__(self):
        self.head = DLinkedList() # dummy head, its next is real head
```

```
        self.tail = DLinkedList() # dummy tail, its prev is real tail
        self.head.next, self.tail.prev = self.tail, self.head
```

```
    def add_node(self, node): # add to head
        node.prev, node.next = self.head, self.head.next
        node.next.prev, node.prev.next = node, node
```

```
    def remove_node(self, node):
        node.prev.next, node.next.prev = node.next, node.prev
        return node.key
```

```
def remove_tail(self):
    return self.remove_node(self.tail.prev) # dummy tail's prev is real
tail
```

```
def move_to_head(self, node):
    self.remove_node(node)
    self.add_node(node)
```

```
# def __repr__(self):
# ans = []
```

```
# h = self.head
# while h:
# ans.append(str(h.val))
# h = h.next
# return '<DLinkedList: {}>'.format('->'.join(ans))
```

```
class LRUCache:
```

```
def __init__(self, capacity: int):
    self.capacity = capacity
```

```
self.nodes_map = {}
self.cache_list = DLinkedList()
```

```
def get(self, key: int) -> int:
    node = self.nodes_map.get(key, None)
    if node:
        self.cache_list.move_to_head(node)
        return node.val
    else:
        return -1
```

```
def put(self, key: int, value: int) -> None:
    node = self.nodes_map.get(key, None)
    if not node:
        if self.capacity > 0:
            self.capacity -= 1
        else:
            rm_key = self.cache_list.remove_tail()
            self.nodes_map.pop(rm_key) # note api
            new_node = DLinkedNode(key, value)

            self.nodes_map[key] = new_node
            self.cache_list.add_node(new_node)
    else:
        self.cache_list.move_to_head(node)
        node.val = value
```

◆ Quick Review

Key Insights

- Use a Hash Table (dictionary) to allow $O(1)$ access to nodes by key.
- Use a Doubly Linked List to record the usage order. The head represents the most recently used element while the tail represents the least recently used.
- When a key is accessed or updated, move the corresponding node to the front (head) of the list.
- When the cache exceeds capacity, remove the node at the tail end which is the least recently used entry.

Space & Time

Time Complexity: $O(1)$ per get and put operation.

Space Complexity: $O(\text{capacity})$, storing key-node pairs and linked list nodes.

Solution

The solution combines a hash table (or dictionary) and a doubly linked list: The hash table stores keys and corresponding node addresses. The doubly linked list maintains the order of usage with the most recently used items near the head. For `get(key)`, check the dictionary. If found, move the node to the head and return its value; if not, return `-1`. For `put(key, value)`, if the key exists, update its value and move it to the head. If not, create a new node and add it right after the head. If the capacity is reached, remove the tail node (least recently used) and update the dictionary accordingly. Dummy head and tail nodes are used to simplify node addition and removal procedures without worrying about edge cases.

Linked List

□ #148 Sort List

MED[Open on LeetCode](#)

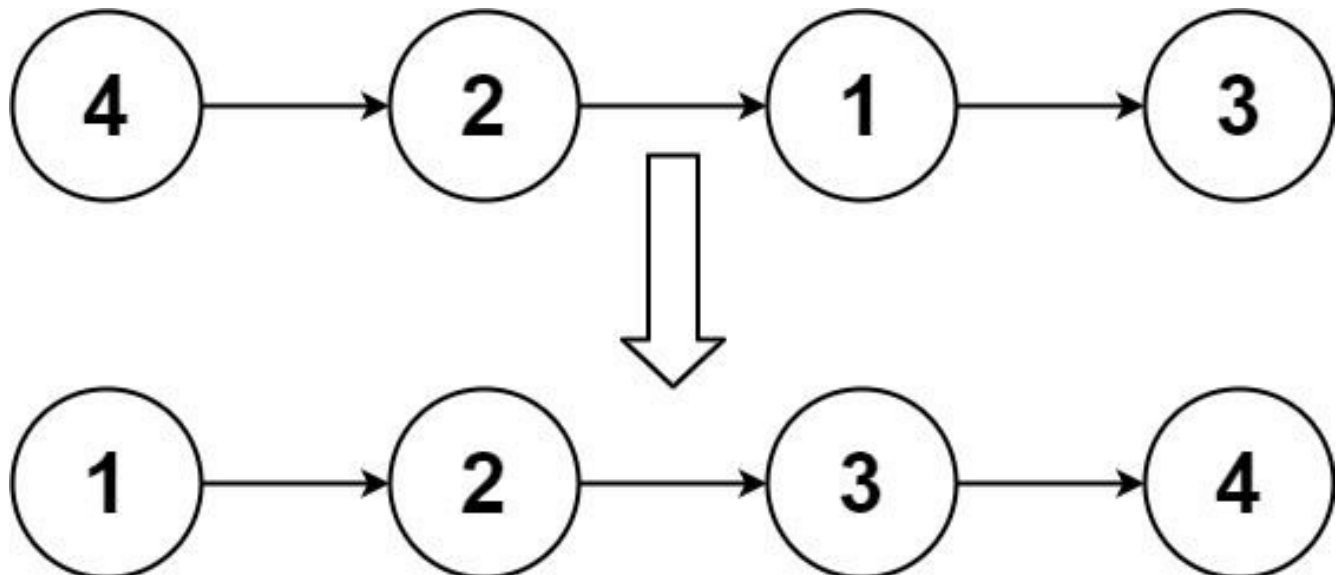
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Two Pointers · Divide and Conquer
· Sorting · Merge Sort

Lists: Grind 169

Problem

Given the head of a linked list, return the list after sorting it in ascending order.

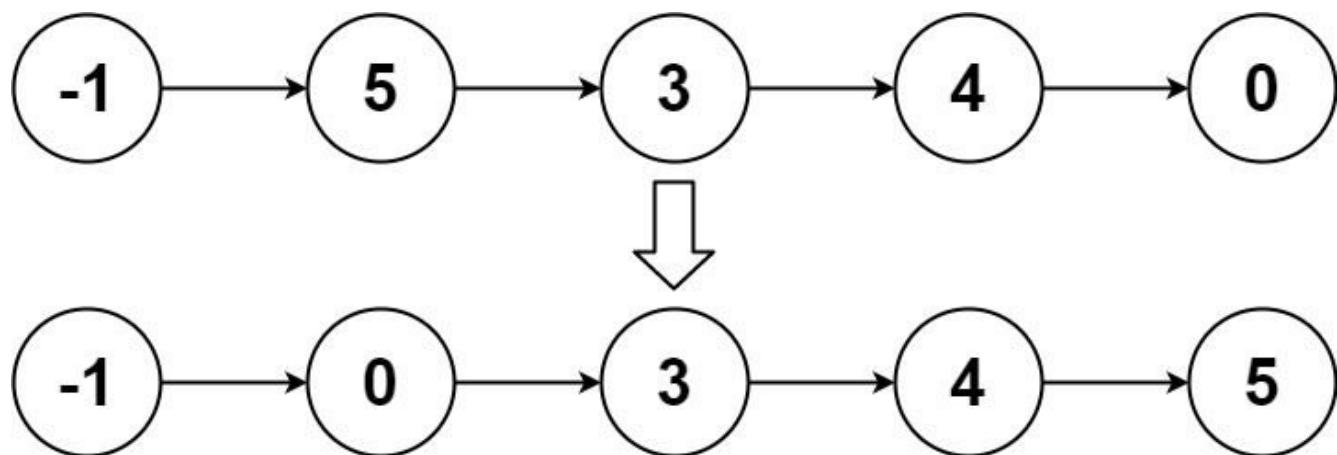
Example 1:



Input: head = [4,2,1,3]

Output: [1,2,3,4]

Example 2:



Input: head = [-1,5,3,4,0]
Output: [-1,0,3,4,5]

Example 3:

Input: head = []
Output: []

Constraints:

- The number of nodes in the list is in the range $[0, 5 * 10^4]$.
- $-105 \leq \text{Node.val} \leq 105$

Follow up: Can you sort the linked list in $O(n \log n)$ time and $O(1)$ memory (i.e. constant space)?

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Sort a linked list in ascending order.

```

# The follow-up wants  $O(n \log n)$  time and  $O(1)$  space. Right?"
#
# "A few quick questions:
# Values can be negative? Yes. Empty list is valid? Yes – return None."
#
# "Let me walk: [4,2,1,3] → [1,2,3,4] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Collect all values into a Python list, sort it, then overwrite node values in
order.
#  $O(n \log n)$  time but  $O(n)$  extra space for the values array.
# The follow-up requires  $O(1)$  extra space – so we need a true in-place sort.
# Should I go ahead and optimize?"
class _148_SortList_Brute:
    def sortList(self, head):
        vals, cur = [], head
        while cur: vals.append(cur.val); cur = cur.next
        vals.sort()
        cur = head
        for v in vals: cur.val = v; cur = cur.next
        return head

# PHASE 3 — OPTIMIZE
# "The bottleneck is the  $O(n)$  auxiliary list.
# Bottom-up merge sort achieves  $O(n \log n)$  time and  $O(1)$  space (no recursion stack):
# Start by merging pairs of size 1, then pairs of size 2, then 4, and so on.
# Each pass merges  $n/\text{size}$  pairs, and there are  $\log n$  passes.
# A dummy head simplifies relinking across rounds.
#
# Time:  $O(n \log n)$ . Space:  $O(1)$ .
# Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _148_SortList:
    def sortList(self, head):
        def merge(l1, l2):
            dummy = cur = ListNode(0)
            while l1 and l2:
                if l1.val <= l2.val: cur.next = l1; l1 = l1.next
                else: cur.next = l2; l2 = l2.next
                cur = cur.next
            cur.next = l1 if l1 else l2
            return dummy.next
        def split(head, size):
            cur = head
            for _ in range(size - 1):
                if cur: cur = cur.next
            if not cur: return None, None

```

```

        rest = cur.next; cur.next = None
        return head, rest
    length, cur = 0, head
    while cur: length += 1; cur = cur.next
    size = 1
    while size < length:
        dummy = tail = ListNode(0)
        cur = head
        while cur:
            left, cur = split(cur, size)
            right, cur = split(cur, size)
            tail.next = merge(left, right)
            while tail.next: tail = tail.next
        head = dummy.next
        size *= 2
    return head

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

[4,2,1,3], size=1: merge(4,2)=[2,4], merge(1,3)=[1,3]. List: 2→4→1→3.

size=2: merge([2,4],[1,3])=[1,2,3,4]. → [1,2,3,4] ✓

#

"No bugs. The split helper correctly handles lists shorter than 'size' by returning None."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log n)$: $\log n$ passes, each $O(n)$. Space $O(1)$: no recursion stack.

Top-down merge sort is simpler to write but uses $O(\log n)$ stack space.

Follow-up: Merge k Sorted Lists (#23) – use this merge step in a heap for $O(n \log k)$.

Follow-up: insertion sort for linked list (#147) – $O(n^2)$ but stable and in-place."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def sortList(self, head: ListNode) -> ListNode:
        def split(head: ListNode, k: int) -> ListNode:
            while k > 1 and head:
                head = head.next
                k -= 1
            rest = head.next if head else None
            if head:
                head.next = None
            return rest

```

```
def merge(l1: ListNode, l2: ListNode) -> tuple:
    dummy = ListNode(0)
    tail = dummy

    while l1 and l2:
        if l1.val > l2.val:
            l1, l2 = l2, l1
        tail.next = l1
        l1 = l1.next

        tail = tail.next
        tail.next = l1 if l1 else l2
    while tail.next:
        tail = tail.next

    return dummy.next, tail

length = 0
curr = head
while curr:

    length += 1
    curr = curr.next

dummy = ListNode(0, head)

k = 1
while k < length:
    curr = dummy.next
    tail = dummy
    while curr:

        l = curr
        r = split(l, k)
        curr = split(r, k)
        mergedHead, mergedTail = merge(l, r)
        tail.next = mergedHead
        tail = mergedTail
    k *= 2

return dummy.next
```

• LeetDoocs

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def sortList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        if head is None or head.next is None:
            return head
        slow, fast = head, head.next

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        l1, l2 = head, slow.next
        slow.next = None
        l1, l2 = self.sortList(l1), self.sortList(l2)
        dummy = ListNode()
        tail = dummy
        while l1 and l2:
            if l1.val <= l2.val:
                tail.next = l1
                l1 = l1.next
            else:
                tail.next = l2
                l2 = l2.next
            tail = tail.next
        tail.next = l1 or l2
        return dummy.next
```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def sortList(self, head: ListNode) -> ListNode:
        # Base case: if list is empty or contains a single element.
        if not head or not head.next:
            return head

        # Function to split the list into two halves
        def getMid(head):
            slow, fast = head, head.next
            # Move fast twice as fast as slow to locate middle
            while fast and fast.next:
                slow = slow.next
                fast = fast.next.next
            return slow

        # Split the linked list into two halves
        mid = getMid(head)
        right_head = mid.next
        mid.next = None # cut the list into two parts

        # Recursive calls to sort each half
        left = self.sortList(head)
        right = self.sortList(right_head)

        # Merge two sorted lists
        return merge(left, right)

    def merge(self, l1, l2):
        dummy = ListNode(0)
        tail = dummy
        # Merge the lists by comparing nodes
        while l1 and l2:
            if l1.val < l2.val:
                tail.next = l1
```

```

        l1 = l1.next
    else:
        tail.next = l2
        l2 = l2.next
    tail = tail.next
    # Append the remaining nodes
    tail.next = l1 if l1 else l2
    return dummy.next

```

• LC.ca

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def sortList(self, head: ListNode) -> ListNode:
        if head is None or head.next is None:
            return head
        slow, fast = head, head.next

        while fast and fast.next:
            slow, fast = slow.next, fast.next.next
            t = slow.next
            slow.next = None
            l1, l2 = self.sortList(head), self.sortList(t)
            dummy = ListNode()
            cur = dummy
            while l1 and l2:
                if l1.val <= l2.val:
                    cur.next = l1
                    l1 = l1.next
                else:
                    cur.next = l2
                    l2 = l2.next
                cur = cur.next
            cur.next = l1 or l2 # add the rest
        return dummy.next

```


◆ Quick Review

Key Insights

- Merge sort is well-suited for linked lists; it naturally relies on splitting the list and merging sorted sublists.
- The slow and fast pointer technique is used to find the middle point of the list.
- A recursive merge sort implementation may use $O(\log n)$ space for recursion stack. To achieve $O(1)$ extra space, an iterative bottom-up merge sort approach would be preferred.
- Merging two sorted lists is efficient with linked lists because of their sequential access.

Space & Time

Time Complexity: $O(n \log n)$

Space Complexity: $O(1)$ extra space for an iterative bottom-up approach ($O(\log n)$ for recursive calls if using recursion)

Solution

The solution employs the merge sort algorithm tailored for linked lists. The overall steps are: Use the slow and fast pointers to

find the middle of the list and split it into two halves. Recursively sort the two halves. Merge the two sorted halves by comparing node values. Ensure that during merging, pointers are adjusted properly to build the sorted list. For an $O(1)$ space solution, an iterative bottom-up merge sort can be implemented. However, the recursive merge sort is easier to understand and implement, though it uses $O(\log n)$ space on the call stack.

Linked List

□ #328 Odd Even Linked List

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List

Lists: Grind 169

Problem

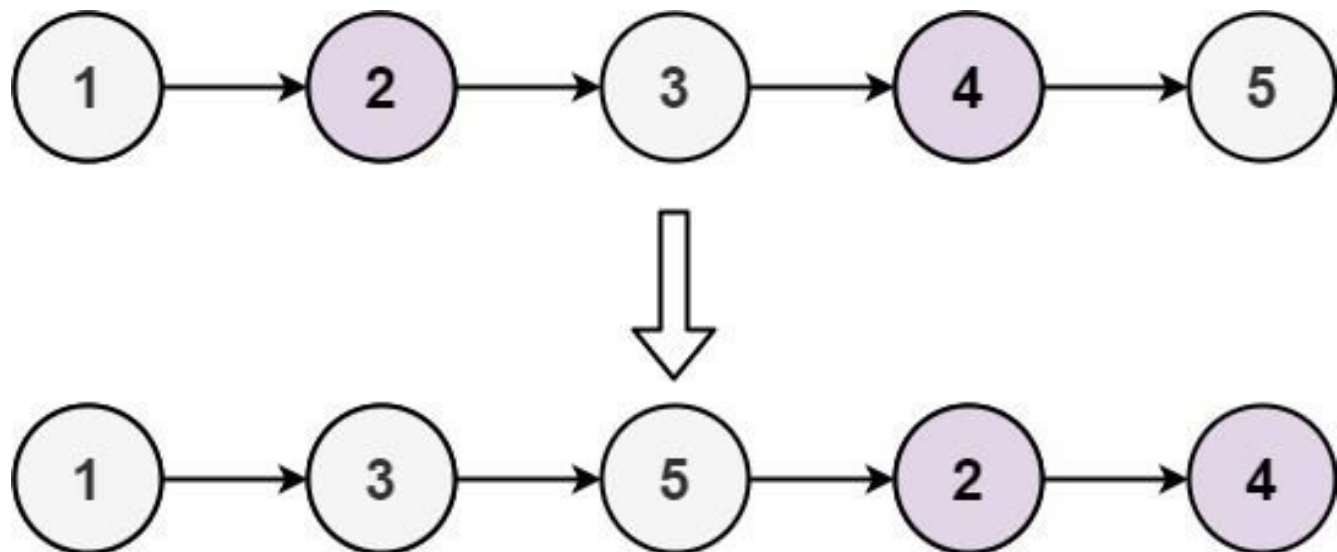
Given the head of a singly linked list, group all the nodes with odd indices together followed by the nodes with even indices, and return the reordered list.

The first node is considered odd, and the second node is even, and so on.

Note that the relative order inside both the even and odd groups should remain as it was in the input.

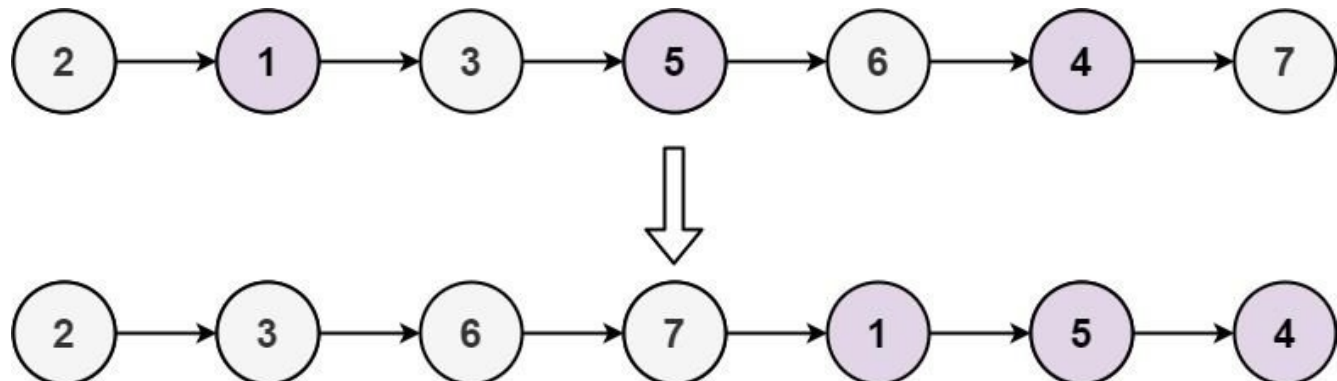
You must solve the problem in $O(1)$ extra space complexity and $O(n)$ time complexity.

Example 1:



Input: head = [1,2,3,4,5]
Output: [1,3,5,2,4]

Example 2:



Input: head = [2,1,3,5,6,4,7]
Output: [2,3,6,7,1,5,4]

Constraints:

- The number of nodes in the linked list is in the range [0, 104].
- $-106 \leq \text{Node.val} \leq 106$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Group all odd-position nodes first, then all even-position nodes.
# 1-indexed positions: node 1,3,5,... then 2,4,6,...
# Preserve relative order within each group. O(1) space, O(n) time. Right?"
#
# "A few quick questions:
# Grouping is by POSITION not value? Yes – the problem is about indices.
# Return the head of the modified list."
#
# "Let me walk: [1,2,3,4,5] → [1,3,5,2,4] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Collect odd-position and even-position values separately, then rebuild the list.
# O(n) time but O(n) space for the two value lists.
# Should I go ahead and optimize?"
class _328_OddEvenLinkedList_Brute:
    def oddEvenList(self, head):
        odds, evens, cur, i = [], [], head, 1
        while cur: (odds if i % 2 else evens).append(cur.val); cur = cur.next; i += 1

        cur = head
        for v in odds + evens: cur.val = v; cur = cur.next
        return head
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the O(n) auxiliary lists.
# In-place pointer manipulation with two sub-list heads:
# odd.next = odd.next.next (skip even nodes in the odd chain).
# even.next = even.next.next (skip odd nodes in the even chain).
# At the end, link the tail of the odd chain to the head of the even chain.
# Save even_head before the loop so we can link at the end.
#
# Time: O(n). Space: O(1).
# Does that make sense before I code it?"
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _328_OddEvenLinkedList:
    def oddEvenList(self, head):
        if not head or not head.next:
            return head
        odd = head
        even = even_head = head.next
        while even and even.next:
            odd.next = even.next
            odd = odd.next
            even = even.next
            even_head = even.next
```

```

        even.next = odd.next
        even = even.next
    odd.next = even_head
    return head

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

[1,2,3,4,5]: odd=1, even=2, even_head=2.

iter1: odd.next=3,odd=3. even.next=4,even=4.

iter2: odd.next=5,odd=5. even.next=None,even=None.

odd.next=even_head=2. → 1→3→5→2→4→None ✓

#

"No bugs. even_head must be saved before the loop since we modify even's next."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: single pass. Space $O(1)$: two pointer variables.

Contrast with the brute that copies values – this reuses original nodes.

Follow-up: Partition List (#86) – partition by a value, not by position index.

Follow-up: Reorder List (#143) – interleave first and reversed second half."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def oddEvenList(self, head: ListNode) -> ListNode:
        oddHead = ListNode(0)
        evenHead = ListNode(0)
        odd = oddHead
        even = evenHead
        isOdd = True

        while head:
            if isOdd:
                odd.next = head
                odd = head
            else:
                even.next = head
                even = head
            head = head.next
            isOdd = not isOdd

        even.next = None
        odd.next = evenHead.next

```

```
return oddHead.next
```

• LeetDoocs

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def oddEvenList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        if head is None:
            return None
        a = head

        b = c = head.next
        while b and b.next:
            a.next = b.next
            a = a.next
            b.next = a.next
            b = b.next
        a.next = c
        return head
```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def oddEvenList(self, head: ListNode) -> ListNode:
        # Check for empty list or list with one node.
        if not head or not head.next:
```



```

        return head

    odd = head          # Point to the first (odd indexed) node.
    even = head.next    # Point to the second (even indexed) node.
    even_head = even    # Save the head of the even list.

    # Reorder the list by rearranging next pointers.
    while even and even.next:
        odd.next = even.next # Connect current odd to next odd.
        odd = odd.next       # Move odd pointer.

        even.next = odd.next # Connect current even to next even.
        even = even.next     # Move even pointer.

    # Append the even list after the odd list.
    odd.next = even_head
    return head

```

• LC.ca

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def oddEvenList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        if head is None:
            return None
        a = head

        b = c = head.next
        while b and b.next:
            a.next = b.next
            a = a.next
            b.next = a.next
            b = b.next
        a.next = c
        return head

```

◆ Quick Review

Key Insights

- Use two pointers: one for odd-indexed nodes and one for even-indexed nodes.
- The first node is considered odd, and the second node is even.
- Traverse the list once, reassigning the next pointers to segregate odd and even nodes.
- Finally, attach the even list at the end of the odd list.
- The solution must use $O(1)$ extra space and $O(n)$ time complexity.

Space & Time

Time Complexity: $O(n)$ since the list is traversed once.

Space Complexity: $O(1)$ as only a few pointers are used.

Solution

Maintain two pointers, odd and even, starting from the head and head.next, respectively. As you iterate, update odd.next to point to the next odd element (skipping an even node) and even.next to point to the next even element.

Once the end is reached, attach the even list to the end of the odd list. This in-place rearrangement preserves the initial ordering within odd and even groups while meeting the space and time constraints.

Linked List

□ ★ #369 Plus One Linked List ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Math · Recursion
Lists: ★ Premium | Premium 98

Problem

Given a non-negative integer represented as a linked list of digits, plus one to the integer.

The digits are stored such that the most significant digit is at the head of the list.

Example 1:

Input: head = [1,2,3]
Output: [1,2,4]

Example 2:

Input: head = [0]
Output: [1]

Constraints:

- The number of nodes in the linked list is in the range [1, 100].
- $0 \leq \text{Node.val} \leq 9$

- The number represented by the linked list does not contain leading zeros except for the zero itself.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

A non-negative integer is stored as a linked list of digits, most significant first.

Add one to the number and return the updated list. Right?"

#

"A few quick questions:

Can all digits be 9 (e.g. $999+1=1000$)? Yes – we'd need a new head node.

Can the number be 0? Yes – $0+1=1$."

#

"Let me walk: $[1,2,3] \rightarrow 123+1=124 \rightarrow [1,2,4] \checkmark$. $[9,9,9] \rightarrow 999+1=1000 \rightarrow [1,0,0,0] \checkmark$ "

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Convert the linked list to an integer, add 1, convert back to a linked list.

Works for small numbers but overflows in fixed-width languages.

Should I go ahead and optimize?"

```
class _369_PlusOneLinkedList_Brute:
```

```
    def plusOne(self, head):
```

```
        num, cur = 0, head
```

```
        while cur: num = num * 10 + cur.val; cur = cur.next
```

```
        num += 1
```

```
        dummy = cur = ListNode(0)
```

```
        for ch in str(num): cur.next = ListNode(int(ch)); cur = cur.next
```

```
        return dummy.next
```

PHASE 3 — OPTIMIZE

"The bottleneck is the big-int round-trip that breaks in other languages.

Find the rightmost non-9 digit, increment it, and set all digits after it to 0.

If all digits are 9, prepend a new node with value 1 and zero out all existing nodes.

Use a dummy node with value 0 prepended to handle the all-9s case cleanly.

#

Time: $O(n)$. Space: $O(1)$ – in-place modification.

Does that make sense before I code it?"

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _369_PlusOneLinkedList:
    def plusOne(self, head):
        dummy = ListNode(0)
        dummy.next = head
        not_nine = dummy
        cur = head
        while cur:
            if cur.val != 9: not_nine = cur
            cur = cur.next
        not_nine.val += 1
        cur = not_nine.next
        while cur:
            cur.val = 0; cur = cur.next
        return dummy.next if dummy.val == 0 else dummy

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# [1,2,9]: not_nine ends at node(2). node(2).val=3. node(9).val=0. → [1,3,0] ✓
#
# [9,9,9]: not_nine stays at dummy(0). dummy.val=0+1=1. 9→0,9→0,9→0.
# dummy.val=1 → return dummy → [1,0,0,0] ✓
#
# "No bugs. Returning dummy when dummy.val≠0 handles the all-9s carry-out cleanly."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): two passes (find not_nine, zero out tail). Space O(1).
# The dummy-node trick elegantly handles the all-9s case without a special branch.
# Follow-up: Add Two Numbers (#2) – two lists, similar carry propagation.
# Follow-up: Subtract 1 from a linked list – find rightmost non-0, decrement, set
# rest to 9."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def plusOne(self, head: ListNode) -> ListNode:
        if not head:
            return ListNode(1)
        if self._addOne(head) == 1:
            return ListNode(1, head)
        return head

    def _addOne(self, node: ListNode) -> int:
        carry = self._addOne(node.next) if node.next else 1
```

```

summ = node.val + carry
node.val = summ % 10
return summ // 10

```

```

class Solution:
    def plusOne(self, head: ListNode) -> ListNode:
        dummy = ListNode(0)
        curr = dummy
        dummy.next = head

        while head:
            if head.val != 9:
                curr = head
                head = head.next

            # `curr` now points to the rightmost non-9 node.

            curr.val += 1
            while curr.next:
                curr.next.val = 0
                curr = curr.next

        return dummy.next if dummy.val == 0 else dummy

```

• LeetDoocs

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def plusOne(self, head: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode(0, head)
        target = dummy
        while head:

```



```

        if head.val != 9:
            target = head
            head = head.next
        target.val += 1
        target = target.next
    while target:
        target.val = 0
        target = target.next
    return dummy if dummy.val else dummy.next

```

• SimplyLeet

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# Helper function to reverse the linked list
def reverseList(head):
    prev = None
    curr = head

    while curr:
        nxt = curr.next # store next node
        curr.next = prev # reverse pointer
        prev = curr     # move prev forward
        curr = nxt      # move to next node
    return prev

def plusOne(head: ListNode) -> ListNode:
    # Reverse the linked list to start addition from the least significant
    digit
    head = reverseList(head)

```

```

current = head
carry = 1 # initialize carry as the plus one
while current and carry:
    current.val += carry # add carry to the current node
    carry = current.val // 10 # calculate new carry
    current.val %= 10 # update the node's value
    # If we're at the end and there's still a carry, create a new node.
    if not current.next and carry:
        current.next = ListNode(0)

    current = current.next

# Reverse the list back to its original order before returning.
return reverseList(head)

```

• LC.ca

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def plusOne(self, head: ListNode) -> ListNode:
        dummy = ListNode(0, head)
        target = dummy
        while head:
            if head.val != 9:
                target = head
            head = head.next
        target.val += 1
        target = target.next
        while target:
            target.val = 0
            target = target.next
        return dummy if dummy.val else dummy.next

```

◆ Quick Review

Key Insights

- The digits are stored such that the most significant digit comes first, but addition is easier starting from the least significant digit.
- Reversing the list allows simpler management of the carry during addition.
- After processing the addition, reversing the list back restores the original order.
- Consider special cases where all digits are 9, requiring a new head node to accommodate an additional digit.

Space & Time

Time Complexity: $O(n)$, where n is the number of nodes in the list

Space Complexity: $O(1)$ extra space when using an iterative reversal method. ($O(n)$ if recursion is used)

Solution

We solve the problem by first reversing the linked list, which converts it to a format where the least significant digit is processed first. We then iterate over the reversed list, adding one and managing any carry. If we encounter a carry at the end of the list, a new node is

appended to handle the overflow. Finally, we reverse the list again to restore the original order. This method efficiently handles the addition with constant extra space during processing.

Linked List

□ ★ #430 Flatten a Multilevel Doubly Linked List ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · DFS · Doubly Linked List
Lists: ★ Premium | Premium 98

Problem

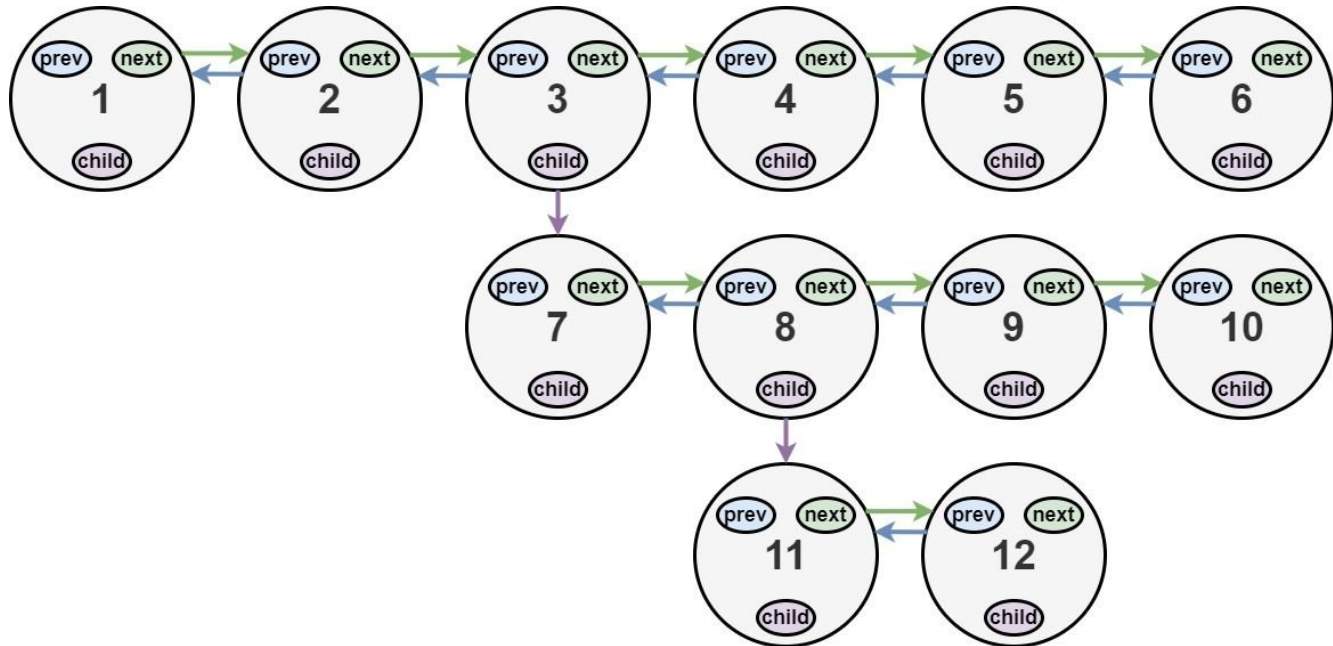
You are given a doubly linked list, which contains nodes that have a next pointer, a previous pointer, and an additional child pointer. This child pointer may or may not point to a separate doubly linked list, also containing these special nodes. These child lists may have one or more children of their own, and so on, to produce a multilevel data structure as shown in the example below.

Given the head of the first level of the list, flatten the list so that all the nodes appear in a single-level, doubly linked list. Let `curr` be a node with a child list. The nodes in the child list should appear after `curr` and before `curr.next` in

the flattened list.

Return the head of the flattened list. The nodes in the list must have all of their child pointers set to null.

Example 1:



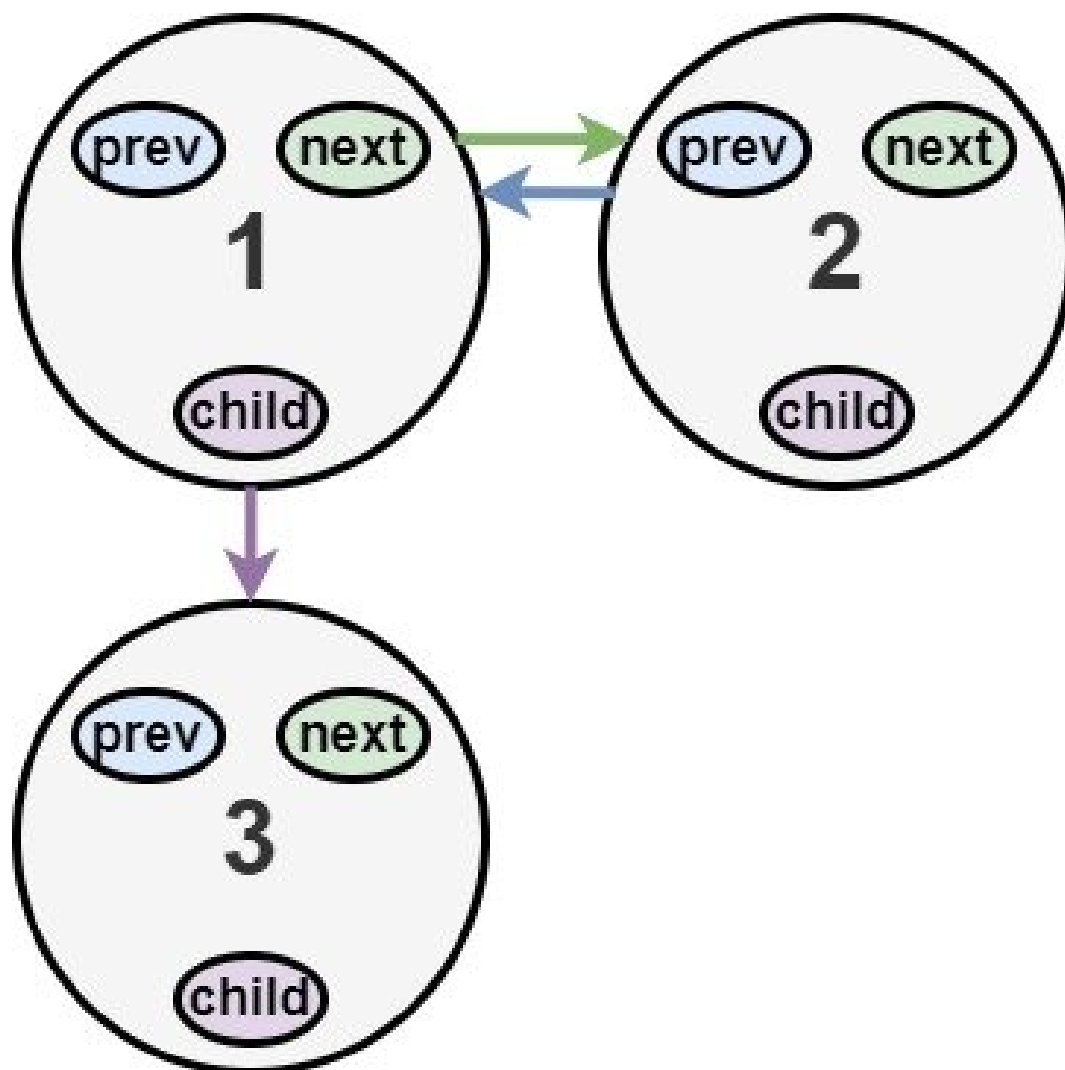
Input: head = [1,2,3,4,5,6,null,null,null,7,8,9,10,null,null,11,12]

Output: [1,2,3,7,8,11,12,9,10,4,5,6]

Explanation: The multilevel linked list in the input is shown.

After flattening the multilevel linked list it becomes:

Example 2:



Input: head = [1,2,null,3]

Output: [1,3,2]

Explanation: The multilevel linked list in the input is shown.
After flattening the multilevel linked list it becomes:

Example 3:

Input: head = []

Output: []

Explanation: There could be empty list in the input.

Constraints:

- The number of Nodes will not exceed 1000.
- $1 \leq \text{Node.val} \leq 105$

How the multilevel linked list is represented in test cases:

We use the multilevel linked list from Example 1 above:

```
1---2---3---4---5---6--NULL
|
7---8---9---10--NULL
|
11--12--NULL
```

The serialization of each level is as follows:

```
[1,2,3,4,5,6,null]
[7,8,9,10,null]
[11,12,null]
```

To serialize all levels together, we will add nulls in each level to signify no node connects to the upper node of the previous level. The serialization becomes:

```
[1, 2, 3, 4, 5, 6, null]
|
[null, null, 7, 8, 9, 10, null]
|
[ null, 11, 12, null]
```

Merging the serialization of each level and removing trailing nulls we obtain:

```
[1,2,3,4,5,6,null,null,null,7,8,9,10,null,null,11,12]
```

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

A doubly linked list where nodes can have a child pointer to a sub-list.

Flatten it so all nodes appear in a single-level DLL in DFS pre-order

(node before its children, children before the rest of node's next). Right?"

#

"A few quick questions:

All child pointers must be set to null in the result? Yes.

1→2→3(child:7→8→9)→4: flattened → 1→2→3→7→8→9→4 ✓"

#

"Let me walk: 1-2-3-4-5-6, node 3 has child 7-8-9-10, node 8 has child 11-12.

Result: 1-2-3-7-8-11-12-9-10-4-5-6 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Collect all nodes in DFS pre-order (visit node, recurse into child, continue next),

then relink them as a flat doubly linked list.

O(n) time but O(n) space for the collected node list.

Should I go ahead and optimize?"

class _430_FlattenMultilevel_Brute:

def flatten(self, head):

nodes = []

def dfs(node):

while node: nodes.append(node); dfs(node.child); node = node.next

dfs(head)

for i in range(len(nodes) - 1):

nodes[i].next = nodes[i+1]; nodes[i+1].prev = nodes[i]; nodes[i].child

= None

if nodes: nodes[-1].child = nodes[-1].next = None

return nodes[0] if nodes else None

PHASE 3 — OPTIMIZE

"The bottleneck is the O(n) auxiliary list.

Use a stack: when we encounter a node with a child, push its current next onto the stack,

insert the child inline, and continue.

When we reach a node with no next but the stack is non-empty, pop to resume the saved tail.

#

Time: O(n). Space: O(d) where d = max nesting depth.

Does that make sense before I code it?"

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

class _430_FlattenMultilevel:

def flatten(self, head):

stack = []

cur = head

while cur:

if cur.child:

```

        if cur.next: stack.append(cur.next)
        cur.next = cur.child
        cur.child.prev = cur
        cur.child = None
    elif not cur.next and stack:
        nxt = stack.pop()
        cur.next = nxt; nxt.prev = cur
    cur = cur.next
return head

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

1→2→3(child:7→8(child:11→12)→9→10)→4:

cur=3: has child. push next=4. 3.next=7, 7.prev=3. cur→7.

cur=8: has child. push next=9. 8.next=11, 11.prev=8. cur→11.

cur=12: no next, pop 9. 12.next=9, 9.prev=12. cur→9.

cur=10: no next, pop 4. 10.next=4, 4.prev=10. cur→4. Done.

→ 1-2-3-7-8-11-12-9-10-4-5-6 ✓

#

"No bugs. Stack exactly mirrors the recursion call stack."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(d)$: stack depth = nesting depth.

Iterative pre-order DFS – same pattern as Clone N-ary Tree but with prev-pointer maintenance.

Follow-up: unflatten the list – requires tracking where each child section started.

Follow-up: BFS order instead of DFS – use a queue instead of a stack."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def flatten(self, head: 'Node') -> 'Node':
        def flatten(head: 'Node', rest: 'Node') -> 'Node':
            if not head:
                return rest

            head.next = flatten(head.child, flatten(head.next, rest))
            if head.next:
                head.next.prev = head
            head.child = None

```

```

        return head

    return flatten(head, None)

```

```

class Solution:
    def flatten(self, head: 'Node') -> 'Node':
        curr = head

        while curr:
            if curr.child:
                cachedNext = curr.next
                curr.next = curr.child
                curr.child.prev = curr
                curr.child = None

                tail = curr.next
                while tail.next:
                    tail = tail.next
                tail.next = cachedNext
                if cachedNext:
                    cachedNext.prev = tail
                curr = curr.next

        return head

```

• LeetDoocs

```

"""
# Definition for a Node.
class Node:
    def __init__(self, val, prev, next, child):
        self.val = val
        self.prev = prev
        self.next = next
        self.child = child
"""

```

```

class Solution:
    def flatten(self, head: 'Node') -> 'Node':
        def preorder(pre, cur):
            if cur is None:
                return pre
            cur.prev = pre
            pre.next = cur

            t = cur.next

            tail = preorder(cur, cur.child)
            cur.child = None
            return preorder(tail, t)

        if head is None:
            return None
        dummy = Node(0, None, head, None)
        preorder(dummy, head)
        dummy.next.prev = None
        return dummy.next

```

• SimplyLeet

```

# Definition for a Node.
class Node:
    def __init__(self, val, prev=None, next=None, child=None):
        self.val = val
        self.prev = prev
        self.next = next
        self.child = child

def flatten(head):
    # If the list is empty, simply return None.

```

```
if not head:
    return head

# Helper function to perform DFS and return the tail of the flattened list.
def dfs(node):
    current = node
    last = None
    while current:
        next_node = current.next # Store current.next before
        modifications
        # If current node has a child, we flatten the child list
        recursively.

        if current.child:
            # Recursively flatten the child list.
            child_tail = dfs(current.child)
            # Connect current node with the start of the child list.
            current.next = current.child
            current.child.prev = current
            # If there was a next node, attach it after the child list.
            if next_node:
                child_tail.next = next_node
                next_node.prev = child_tail

            # Set child pointer to None as required.
            current.child = None
            # Update the tail pointer.
            last = child_tail
        else:
            last = current
        # Move to the next node (which might have been updated after
        flattening a child).
        current = next_node
    return last
```

```

    dfs(head) # Begin DFS traversal from the head.
    return head

```

```

# Example to illustrate usage:
# head = Node(1)
# head.next = Node(2)
# head.child = Node(3)
# flatten(head)

```

• LC.ca

```

"""
# Definition for a Node.
class Node:
    def __init__(self, val, prev, next, child):
        self.val = val
        self.prev = prev
        self.next = next
        self.child = child
"""

```

```

class Solution:
    def flatten(self, head: 'Node') -> 'Node':
        def preorder(pre, cur):
            if cur is None:
                return pre
            cur.prev = pre
            pre.next = cur

            t = cur.next

```

```

            tail = preorder(cur, cur.child)
            cur.child = None
            return preorder(tail, t)

```

```

        if head is None:
            return None
        dummy = Node(0, None, head, None)
        preorder(dummy, head)
        dummy.next.prev = None
        return dummy.next

```

◆ Quick Review

Key Insights

- Use a depth–first traversal to process child lists before proceeding to the next pointer.
- When a node with a child is found, recursively flatten the child list.
- Splice the flattened child list in between the current node and the next node.
- Maintain previous and next pointers correctly to preserve the doubly linked list properties.
- Ensure that all child pointers are set to null after flattening.

Space & Time

Time Complexity: $O(n)$, where n is the total number of nodes, since each node is processed exactly once.

Space Complexity: $O(n)$ in the worst–case due to recursion stack in case of a highly nested list.

Solution

We can solve this problem using a recursive depth–first search (DFS) approach. The main idea is to traverse the list and, whenever a node with a child pointer is encountered, recursively flatten its child list. Once the child list is flattened, it can be spliced into the main list between the current node and the node originally following it. We then continue processing the next pointer. This ensures that the entire list is flattened in depth–first order. The recursive helper function returns the tail of the flattened segment, which is used to connect the subsequent parts of the list correctly.

Linked List

□ #622 Design Circular Queue

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Linked List · Design · Queue
Lists: Denny Zhang

Problem

Design your implementation of the circular queue. The circular queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle, and the last position is connected back to the first position to make a circle. It is also called "Ring Buffer".

One of the benefits of the circular queue is that we can make use of the spaces in front of the queue. In a normal queue, once the queue becomes full, we cannot insert the next element even if there is a space in front of the queue. But using the circular queue, we can use the space to store new values.

Implement the MyCircularQueue class:

- **MyCircularQueue(k)** Initializes the object with the size of the queue to be k.
- **int Front()** Gets the front item from the queue. If the queue is empty, return -1.
- **int Rear()** Gets the last item from the queue. If the queue is empty, return -1.
- **boolean enQueue(int value)** Inserts an element into the circular queue. Return true if the operation is successful.
- **boolean deQueue()** Deletes an element from the circular queue. Return true if the operation is successful.
- **boolean isEmpty()** Checks whether the circular queue is empty or not.
- **boolean isFull()** Checks whether the circular queue is full or not.

You must solve the problem without using the built-in queue data structure in your programming language.

Example 1:

Input

```
["MyCircularQueue", "enqueue", "enqueue", "enqueue", "enqueue", "Rear",
"isFull", "dequeue", "enqueue", "Rear"]
[[3], [1], [2], [3], [4], [], [], [], [4], []]
```

Output

```
[null, true, true, true, false, 3, true, true, true, 4]
```

Explanation

```
MyCircularQueue myCircularQueue = new MyCircularQueue(3);
```

Constraints:

- $1 \leq k \leq 1000$
- $0 \leq \text{value} \leq 1000$
- At most 3000 calls will be made to enqueue, dequeue, front, rear, isEmpty, and isFull.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Design a circular queue (ring buffer) with a fixed capacity.

Support enqueue(val)→bool, dequeue()→bool, front()→int, rear()→int, isEmpty(), isFull().

Must not use Python's built-in queue. Right?"

#

"A few quick questions:

All enqueue/dequeue calls respect isEmpty/isFull – no need to guard against invalid calls? Yes."

#

"Let me walk: MyCircularQueue(3): enqueue(1,2,3)→T,T,T. enqueue(4)→F (full).

rear()→3. isFull()→T. dequeue()→T. enqueue(4)→T. rear()→4 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Use a Python list, append for enqueue and pop(0) for dequeue.

dequeue is O(n) due to shifting – all other ops are O(1).

Should I go ahead and optimize?"

```
class _622_DesignCircularQueue_Brute:
```

```
    def __init__(self, k): self.cap = k; self.q = []
```

```

def enqueue(self, val):
    if self.isFull(): return False
    self.q.append(val); return True
def dequeue(self):
    if self.isEmpty(): return False
    self.q.pop(0); return True
def Front(self): return -1 if self.isEmpty() else self.q[0]
def Rear(self): return -1 if self.isEmpty() else self.q[-1]
def isEmpty(self): return not self.q
def isFull(self): return len(self.q) == self.cap

```

PHASE 3 — OPTIMIZE

```

# "The bottleneck is the O(n) pop(0) in dequeue.
# Use a fixed-size array with head/tail pointers and modular arithmetic.
# enqueue: data[tail] = val; tail = (tail+1) % k; size++.
# dequeue: head = (head+1) % k; size--.
# Front: data[head]. Rear: data[(tail-1) % k].
#
# All ops O(1). Space O(k).
# Does that make sense before I code it?"

```

PHASE 4 — CLEAN CODE

```

# "Now I'll implement the optimized approach with meaningful names."
class _622_DesignCircularQueue:
    def __init__(self, k):
        self._data = [0] * k
        self._head = self._size = 0
        self._cap = k

    def enqueue(self, val):
        if self.isFull(): return False
        self._data[(self._head + self._size) % self._cap] = val
        self._size += 1
        return True

    def dequeue(self):
        if self.isEmpty(): return False
        self._head = (self._head + 1) % self._cap
        self._size -= 1
        return True

    def Front(self): return -1 if self.isEmpty() else self._data[self._head]
    def Rear(self): return -1 if self.isEmpty() else self._data[(self._head +
self._size - 1) % self._cap]
    def isEmpty(self): return self._size == 0
    def isFull(self): return self._size == self._cap

```

PHASE 5 — TEST & DEBUG

```

# "Let me trace through a few cases."
#
# k=3: enqueue(1)→size=1,data[0]=1. enqueue(2)→data[1]=2. enqueue(3)→data[2]=3.
isFull ✓

```

```
# deQueue()→head=1,size=2. enqueue(4)→data[0]=4,size=3.
Rear()→data[(1+3-1)%3]=data[0]=4 ✓
#
# "No bugs. The modular tail calculation (head+size)%cap avoids a separate tail
variable."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "All ops O(1). Space O(k).
# The modulo wraps head/tail around the fixed array – that is the 'circular' part.
# Follow-up: Design Circular Deque (#641) – add enqueueFront/dequeueRear using same
technique.
# Follow-up: thread-safe circular buffer – add a threading.Lock around
enqueue/dequeue."
```

Community Solutions (Python)

• LeetDoocs

```
class MyCircularQueue:

    def __init__(self, k: int):
        self.q = [0] * k
        self.size = 0
        self.capacity = k
        self.front = 0

    def enqueue(self, value: int) -> bool:
        if self.isFull():

            return False
        self.q[(self.front + self.size) % self.capacity] = value
        self.size += 1
        return True

    def dequeue(self) -> bool:
        if self.isEmpty():
            return False
        self.front = (self.front + 1) % self.capacity
        self.size -= 1
```

```

        return True

    def Front(self) -> int:
        return -1 if self.isEmpty() else self.q[self.front]

    def Rear(self) -> int:
        if self.isEmpty():
            return -1
        return self.q[(self.front + self.size - 1) % self.capacity]

```

```

    def isEmpty(self) -> bool:
        return self.size == 0

    def isFull(self) -> bool:
        return self.size == self.capacity

```

```

# Your MyCircularQueue object will be instantiated and called as such:
# obj = MyCircularQueue(k)
# param_1 = obj.enqueue(value)

```

```

# param_2 = obj.dequeue()
# param_3 = obj.Front()
# param_4 = obj.Rear()
# param_5 = obj.isEmpty()
# param_6 = obj.isFull()

```

• SimplyLeet

```

# Python implementation of MyCircularQueue

```

```

class MyCircularQueue:
    def __init__(self, k):
        # Initialize the queue with capacity k
        self.capacity = k
        self.queue = [0] * k # fixed-size array
        self.front = 0        # pointer to the front element
        self.rear = 0         # pointer to the next insertion position
        self.count = 0        # current number of elements in the queue

```



```
def enqueue(self, value):
    # Check if the queue is full
    if self.isFull():
        return False
    # Insert element at rear pointer and update pointer using modulo for
    # circularity
    self.queue[self.rear] = value
    self.rear = (self.rear + 1) % self.capacity
    self.count += 1
    return True
```

```
def dequeue(self):
    # Check if the queue is empty
    if self.isEmpty():
        return False
    # Move front pointer to the next element using modulo for circularity
    self.front = (self.front + 1) % self.capacity
    self.count -= 1
    return True
```

```
def Front(self):
    # Return -1 if the queue is empty, else return the front element
    if self.isEmpty():
        return -1
    return self.queue[self.front]

def Rear(self):
    # Return -1 if the queue is empty
    if self.isEmpty():
        return -1
```

```

        # Since rear points to the next insertion position, get the last
        element index
        return self.queue[(self.rear - 1 + self.capacity) % self.capacity]

    def isEmpty(self):
        # Check if count is 0
        return self.count == 0

    def isFull(self):
        # Check if the queue has reached its capacity
        return self.count == self.capacity

```

• LC.ca

```

class MyCircularQueue:
    def __init__(self, k: int):
        self.q = [0] * k
        self.front = 0
        self.size = 0
        self.capacity = k

    def enQueue(self, value: int) -> bool:
        if self.isFull():
            return False

        idx = (self.front + self.size) % self.capacity
        self.q[idx] = value
        self.size += 1
        return True

    def deQueue(self) -> bool:
        if self.isEmpty():
            return False
        self.front = (self.front + 1) % self.capacity
        self.size -= 1

```

```

        return True

    def Front(self) -> int:
        return -1 if self.isEmpty() else self.q[self.front]

    def Rear(self) -> int:
        if self.isEmpty():
            return -1
        idx = (self.front + self.size - 1) % self.capacity
        return self.q[idx]

```

```

    def isEmpty(self) -> bool:
        return self.size == 0

    def isFull(self) -> bool:
        return self.size == self.capacity

```

Your MyCircularQueue object will be instantiated and called as such:
 # obj = MyCircularQueue(k)

```

# param_1 = obj.enqueue(value)
# param_2 = obj.dequeue()
# param_3 = obj.front()
# param_4 = obj.rear()
# param_5 = obj.isEmpty()
# param_6 = obj.isFull()

```

```
#####
```

```
class MyCircularQueue(object):
```

```

    def __init__(self, k):
        """
        Initialize your data structure here. Set the size of the queue to be k.
        :type k: int
        """
        self.queue = []
        self.size = k
        self.front = 0
        self.rear = 0

```

```
def enqueue(self, value):
    """
    Insert an element into the circular queue. Return true if the operation
    is successful.
    :type value: int
    :rtype: bool
    """
    if self.rear - self.front < self.size:
        self.queue.append(value)
        self.rear += 1

        return True
    else:
        return False

def dequeue(self):
    """
    Delete an element from the circular queue. Return true if the operation
    is successful.
    :rtype: bool
    """
    if self.rear - self.front > 0:

        self.front += 1
        return True
    else:
        return False

def Front(self):
    """
    Get the front item from the queue.
    :rtype: int
    """
```

```
        if self.isEmpty():
            return -1
        else:
            return self.queue[self.front]
```

```
def Rear(self):
    """
    Get the last item from the queue.
    :rtype: int
    """
```

```
        if self.isEmpty():
            return -1
        else:
            return self.queue[self.rear - 1]
```

```
def isEmpty(self):
    """
    Checks whether the circular queue is empty or not.
    :rtype: bool
    """
```

```
        return self.front == self.rear
```

```
def isFull(self):
    """
    Checks whether the circular queue is full or not.
    :rtype: bool
    """
    return self.rear - self.front == self.size
```

```
# Your MyCircularQueue object will be instantiated and called as such:
# obj = MyCircularQueue(k)
# param_1 = obj.enqueue(value)
# param_2 = obj.dequeue()
# param_3 = obj.front()
# param_4 = obj.rear()
# param_5 = obj.isEmpty()
# param_6 = obj.isFull()
```

◆ Quick Review

Key Insights

- Leverage a fixed-size array to store the queue elements.
- Use two pointers: one for the front and one for the rear.
- Maintain a count variable (or use pointer arithmetic) to differentiate between an empty and a full queue.
- Use modulo arithmetic to wrap pointers when they reach the end of the array.
- Ensure all operations (enQueue, deQueue, Front, Rear, isEmpty, isFull) have $O(1)$ time complexity.

Space & Time

Time Complexity: $O(1)$ per operation for enQueue, deQueue, Front, Rear, isEmpty, and isFull.

Space Complexity: $O(k)$, where k is the capacity of the circular queue.

Solution

We implement the circular queue using a fixed-size array alongside two pointers, front

and rear, and a counter to keep track of the number of elements in the queue. The front pointer indicates the index of the first element, and the rear pointer indicates the index where the next element will be enqueued. When enqueueing an element, we insert the element at the rear index and then increment the rear pointer modulo the size of the array. Similarly, when dequeuing, we remove the element at the front index and increment the front pointer modulo the size. The counter helps in quickly checking if the queue is empty or full (count equals 0 or count equals capacity, respectively). This design ensures all operations remain $O(1)$ and correctly wraps around, forming a circular structure.

Linked List

□ #707 Design Linked List

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Design
Lists: Denny Zhang

Problem

Design your implementation of the linked list. You can choose to use a singly or doubly linked list. A node in a singly linked list should have two attributes: `val` and `next`. `val` is the value of the current node, and `next` is a pointer/reference to the next node. If you want to use the doubly linked list, you will need one more attribute `prev` to indicate the previous node in the linked list. Assume all nodes in the linked list are 0-indexed.

Implement the `MyLinkedList` class:

- `MyLinkedList()` Initializes the `MyLinkedList` object.
- `int get(int index)` Get the value of the `index`th node in the linked list. If the `index` is

invalid, return -1.

- **void addAtHead(int val)** Add a node of value **val** before the first element of the linked list. After the insertion, the new node will be the first node of the linked list.
- **void addAtTail(int val)** Append a node of value **val** as the last element of the linked list.
- **void addAtIndex(int index, int val)** Add a node of value **val** before the **index**th node in the linked list. If **index** equals the length of the linked list, the node will be appended to the end of the linked list. If **index** is greater than the length, the node will not be inserted.
- **void deleteAtIndex(int index)** Delete the **index**th node in the linked list, if the **index** is valid.

Example 1:

Input

```
["MyLinkedList", "addAtHead", "addAtTail", "addAtIndex", "get",  
"deleteAtIndex", "get"]  
[[], [1], [3], [1, 2], [1], [1], [1]]
```

Output

```
[null, null, null, null, 2, null, 3]
```

Explanation

```
MyLinkedList myLinkedList = new MyLinkedList();
```

Constraints:

- $0 \leq \text{index}, \text{val} \leq 1000$

- Please do not use the built-in LinkedList library.
- At most 2000 calls will be made to get, addAtHead, addAtTail, addAtIndex and deleteAtIndex.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Design a singly linked list supporting: get(index), addAtHead(val),
# addAtTail(val), addAtIndex(index, val), deleteAtIndex(index).
# Indices are 0-based. addAtIndex at exactly size appends; beyond size is ignored.
Right?"
#
# "A few quick questions:
# get/deleteAtIndex with invalid index return -1 / do nothing? Yes."
#
# "Let me walk: addAtHead(1)→[1]. addAtTail(3)→[1,3]. addAtIndex(1,2)→[1,2,3].
# get(1)→2. deleteAtIndex(1)→[1,3]. get(1)→3 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Use a Python list internally. get/add/delete map to list operations.
# addAtIndex O(n) due to shifting; get O(1).
# Should I go ahead and optimize?"
class _707_DesignLinkedList_Brute:
    def __init__(self): self.data = []
    def get(self, i): return self.data[i] if 0 <= i < len(self.data) else -1
    def addAtHead(self, v): self.data.insert(0, v)
    def addAtTail(self, v): self.data.append(v)
    def addAtIndex(self, i, v):
        if i <= len(self.data): self.data.insert(i, v)
    def deleteAtIndex(self, i):
        if 0 <= i < len(self.data): del self.data[i]
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the O(n) list shift on head insertions/deletions.
# A true singly linked list with a sentinel dummy head and size counter:
# all operations walk to the (index-1)th node and rewire from there.
```

```

# No shifting needed – pointer operations only.
#
# Time: O(index) per op – O(n) worst case, O(1) at head.
# Space: O(n) total nodes."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _707_MyLinkedList:
    class _Node:
        def __init__(self, val=0): self.val = val; self.next = None
    def __init__(self):
        self._dummy = self._Node()
        self._size = 0
    def _get_node(self, index):
        cur = self._dummy
        for _ in range(index + 1): cur = cur.next
        return cur
    def get(self, index):
        if index < 0 or index >= self._size: return -1
        return self._get_node(index).val
    def addAtHead(self, val): self.addAtIndex(0, val)
    def addAtTail(self, val): self.addAtIndex(self._size, val)
    def addAtIndex(self, index, val):
        if index > self._size: return
        prev = self._dummy
        for _ in range(index): prev = prev.next
        node = self._Node(val)
        node.next = prev.next; prev.next = node
        self._size += 1
    def deleteAtIndex(self, index):
        if index < 0 or index >= self._size: return
        prev = self._dummy
        for _ in range(index): prev = prev.next
        prev.next = prev.next.next
        self._size -= 1

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# addAtHead(1): dummy→1. addAtTail(3): dummy→1→3. addAtIndex(1,2): dummy→1→2→3.
# get(1): walk 2 nodes → node(2).val=2 ✓. deleteAtIndex(1): prev=node(1), 1.next=3.
# dummy→1→3.
# get(1): node(3).val=3 ✓.
#
# "No bugs. Dummy head makes addAtIndex(0,val) identical to any other index – no
# special case."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "get/add/delete: O(index) – O(n) worst case. addAtHead: O(1)."

```

Space $O(n)$ for n stored nodes.
 # Follow-up: doubly linked list – add a prev pointer for $O(1)$ head and tail ops.
 # Follow-up: skip list gives $O(\log n)$ average for all ops – classic interview extension."

Community Solutions (Python)

• WalkCC

```
from dataclasses import dataclass

@dataclass
class ListNode:
    val: int
    next: ListNode | None = None

class MyLinkedList:

    def __init__(self):
        self.length = 0
        self.dummy = ListNode(0)

    def get(self, index: int) -> int:
        if index < 0 or index >= self.length:
            return -1
        curr = self.dummy.next
        for _ in range(index):
            curr = curr.next

        return curr.val

    def addAtHead(self, val: int) -> None:
        curr = self.dummy.next
        self.dummy.next = ListNode(val)
        self.dummy.next.next = curr
        self.length += 1

    def addAtTail(self, val: int) -> None:
        curr = self.dummy
```

```

while curr.next:
    curr = curr.next
curr.next = ListNode(val)
self.length += 1

def addAtIndex(self, index: int, val: int) -> None:
    if index > self.length:
        return
    curr = self.dummy
    for _ in range(index):

        curr = curr.next
        temp = curr.next
        curr.next = ListNode(val)
        curr.next.next = temp
        self.length += 1

def deleteAtIndex(self, index: int) -> None:
    if index < 0 or index >= self.length:
        return
    curr = self.dummy

    for _ in range(index):
        curr = curr.next
        temp = curr.next
        curr.next = temp.next
        self.length -= 1

```

• LeetDoocs

```

class MyLinkedList:
    def __init__(self):
        self.dummy = ListNode()
        self.cnt = 0

    def get(self, index: int) -> int:
        if index < 0 or index >= self.cnt:
            return -1
        cur = self.dummy.next
        for _ in range(index):

```

```
        cur = cur.next
    return cur.val

def addAtHead(self, val: int) -> None:
    self.addAtIndex(0, val)

def addAtTail(self, val: int) -> None:
    self.addAtIndex(self.cnt, val)

def addAtIndex(self, index: int, val: int) -> None:
```

```
    if index > self.cnt:
        return
    pre = self.dummy
    for _ in range(index):
        pre = pre.next
    pre.next = ListNode(val, pre.next)
    self.cnt += 1

def deleteAtIndex(self, index: int) -> None:
    if index >= self.cnt:
```

```
        return
    pre = self.dummy
    for _ in range(index):
        pre = pre.next
    t = pre.next
    pre.next = t.next
    t.next = None
    self.cnt -= 1
```

```
# Your MyLinkedList object will be instantiated and called as such:
# obj = MyLinkedList()
# param_1 = obj.get(index)
# obj.addAtHead(val)
# obj.addAtTail(val)
# obj.addAtIndex(index, val)
# obj.deleteAtIndex(index)
```

```
class MyLinkedList:
    def __init__(self):
        self.e = [0] * 1010
        self.ne = [0] * 1010
        self.idx = 0
        self.head = -1
        self.cnt = 0

    def get(self, index: int) -> int:
        if index < 0 or index >= self.cnt:
            return -1
        i = self.head
        for _ in range(index):
            i = self.ne[i]
        return self.e[i]

    def addAtHead(self, val: int) -> None:
        self.e[self.idx] = val
        self.ne[self.idx] = self.head
        self.head = self.idx

        self.idx += 1
        self.cnt += 1

    def addAtTail(self, val: int) -> None:
        self.addAtIndex(self.cnt, val)

    def addAtIndex(self, index: int, val: int) -> None:
        if index > self.cnt:
            return
        if index <= 0:
            self.addAtHead(val)
            return
        i = self.head
        for _ in range(index - 1):
            i = self.ne[i]
        self.e[self.idx] = val
        self.ne[self.idx] = self.ne[i]
        self.ne[i] = self.idx
        self.idx += 1
        self.cnt += 1
```



```
def deleteAtIndex(self, index: int) -> None:
    if index < 0 or index >= self.cnt:
        return -1
    self.cnt -= 1
    if index == 0:
        self.head = self.ne[self.head]
        return
    i = self.head
    for _ in range(index - 1):
```

```
        i = self.ne[i]
        self.ne[i] = self.ne[self.ne[i]]
```

```
# Your MyLinkedList object will be instantiated and called as such:
# obj = MyLinkedList()
# param_1 = obj.get(index)
# obj.addAtHead(val)
# obj.addAtTail(val)
# obj.addAtIndex(index, val)
```

```
# obj.deleteAtIndex(index)
```

• SimplyLeet

```
# Define a node for the singly linked list
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val # The value of the node
        self.next = next # Pointer to the next node

class MyLinkedList:
    def __init__(self):
        # Initialize size and a dummy head node to simplify operations
        self.size = 0
```

```
self.head = ListNode(0) # dummy head

def get(self, index: int) -> int:
    # If index is out of bounds, return -1
    if index < 0 or index >= self.size:
        return -1
    current = self.head.next # start from the actual head
    for _ in range(index):
        current = current.next
    return current.val

def addAtHead(self, val: int) -> None:
    self.addAtIndex(0, val) # add at index 0

def addAtTail(self, val: int) -> None:
    self.addAtIndex(self.size, val) # add at the end

def addAtIndex(self, index: int, val: int) -> None:
    # Normalize negative index to 0 (insert at head)
    if index < 0:
        index = 0
    # Only insert if index is valid (<= size)
    if index > self.size:
        return
    self.size += 1
    # Start from the dummy head node
    pred = self.head
    for _ in range(index):
        pred = pred.next
    # Insert new node
    new_node = ListNode(val)
    new_node.next = pred.next
    pred.next = new_node

def deleteAtIndex(self, index: int) -> None:
    # If index is out of bounds, do nothing
    if index < 0 or index >= self.size:
        return
    self.size -= 1
    pred = self.head
```

```

for _ in range(index):
    pred = pred.next
# Remove the node by skipping it
pred.next = pred.next.next

```

• LC.ca

```

# doubly linked node
class ListNode:
    def __init__(self, value=0, prev=None, next=None):
        self.value = value
        self.prev = prev
        self.next = next

class MyLinkedList:
    def __init__(self):
        self.head = ListNode(0) # Sentinel node as pseudo-head

        self.tail = ListNode(0) # Sentinel node as pseudo-tail
        self.head.next = self.tail
        self.tail.prev = self.head
        self.size = 0

    def get(self, index: int) -> int:
        if index < 0 or index >= self.size:
            return -1
        if index + 1 < self.size - index: # decide start from head or tail
            curr = self.head

            for _ in range(index + 1):
                curr = curr.next
        else:
            curr = self.tail
            for _ in range(self.size - index):
                curr = curr.prev
        return curr.value

    def addAtHead(self, val: int) -> None:
        pred, succ = self.head, self.head.next

```

```
        self.size += 1
        to_add = ListNode(val, pred, succ)
        pred.next = to_add
        succ.prev = to_add

    def addAtTail(self, val: int) -> None:
        succ, pred = self.tail, self.tail.prev
        self.size += 1
        to_add = ListNode(val, pred, succ)
        pred.next = to_add

        succ.prev = to_add

    def addAtIndex(self, index: int, val: int) -> None:
        if index > self.size:
            return
        if index < 0:
            index = 0
        if index < self.size - index: # decide start from head or tail
            pred = self.head
            for _ in range(index):

                pred = pred.next
            succ = pred.next
        else:
            succ = self.tail
            for _ in range(self.size - index):
                succ = succ.prev
            pred = succ.prev
        self.size += 1
        to_add = ListNode(val, pred, succ)
        pred.next = to_add

        succ.prev = to_add

    def deleteAtIndex(self, index: int) -> None:
        if index < 0 or index >= self.size:
            return
        if index < self.size - index:
            pred = self.head
            for _ in range(index):
                pred = pred.next
            succ = pred.next.next
```

```

else:
    succ = self.tail
    for _ in range(self.size - index - 1):
        succ = succ.prev
    pred = succ.prev.prev
    self.size -= 1
    pred.next = succ
    succ.prev = pred

```

Your MyLinkedList object will be instantiated and called as such:

```

# obj = MyLinkedList()
# param_1 = obj.get(index)
# obj.addAtHead(val)
# obj.addAtTail(val)
# obj.addAtIndex(index, val)
# obj.deleteAtIndex(index)

```

```
#####
```

```
# singly linked node
```

```

class MyLinkedList:
    def __init__(self):
        self.dummy = ListNode()
        self.cnt = 0

    def get(self, index: int) -> int:
        if index < 0 or index >= self.cnt:
            return -1
        cur = self.dummy.next
        for _ in range(index):

            cur = cur.next
        return cur.val

    def addAtHead(self, val: int) -> None:
        self.addAtIndex(0, val)

    def addAtTail(self, val: int) -> None:
        self.addAtIndex(self.cnt, val)

    def addAtIndex(self, index: int, val: int) -> None:

```

```

        if index > self.cnt:
            return
        pre = self.dummy
        for _ in range(index):
            pre = pre.next
        pre.next = ListNode(val, pre.next) # insert in-between
        self.cnt += 1

def deleteAtIndex(self, index: int) -> None:
    if index >= self.cnt:

        return
    pre = self.dummy
    for _ in range(index):
        pre = pre.next
    t = pre.next
    pre.next = t.next
    t.next = None # don't forget to cutoff
    self.cnt -= 1

```

```

# Your MyLinkedList object will be instantiated and called as such:
# obj = MyLinkedList()
# param_1 = obj.get(index)
# obj.addAtHead(val)
# obj.addAtTail(val)
# obj.addAtIndex(index, val)
# obj.deleteAtIndex(index)

```

◆ Quick Review

Key Insights

- Use a dummy head node to simplify insertion and deletion at the head.
- Maintain a size counter to quickly validate index-based operations.

- For every operation, traverse the list up to the required point since random access is not available.
- Ensure robustness by handling edge cases like negative indices and indices larger than the current length.

Space & Time

Time Complexity:

get, addAtIndex, and deleteAtIndex operations require $O(n)$ time due to the need to traverse the list.

addAtHead and addAtTail (if tail pointer is not maintained) are $O(n)$ in worst case; however, these can be optimized further if a tail pointer is added.

Space Complexity:

$O(n)$ space for storing n nodes.

Solution

The solution employs a singly linked list with a dummy head node to ease edge-case handling. A size counter is maintained for quick index validations. Each node holds a value and a pointer to the next node.

Operations operate by traversing the list to

the correct insertion or deletion point. This design makes it simple to add the functionality of getting values, inserting at specified indices, and removing nodes.

Linked List

□ ★ #708 Insert into a Sorted Circular Linked List ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List

Lists: ★ Premium | Premium 98

Problem

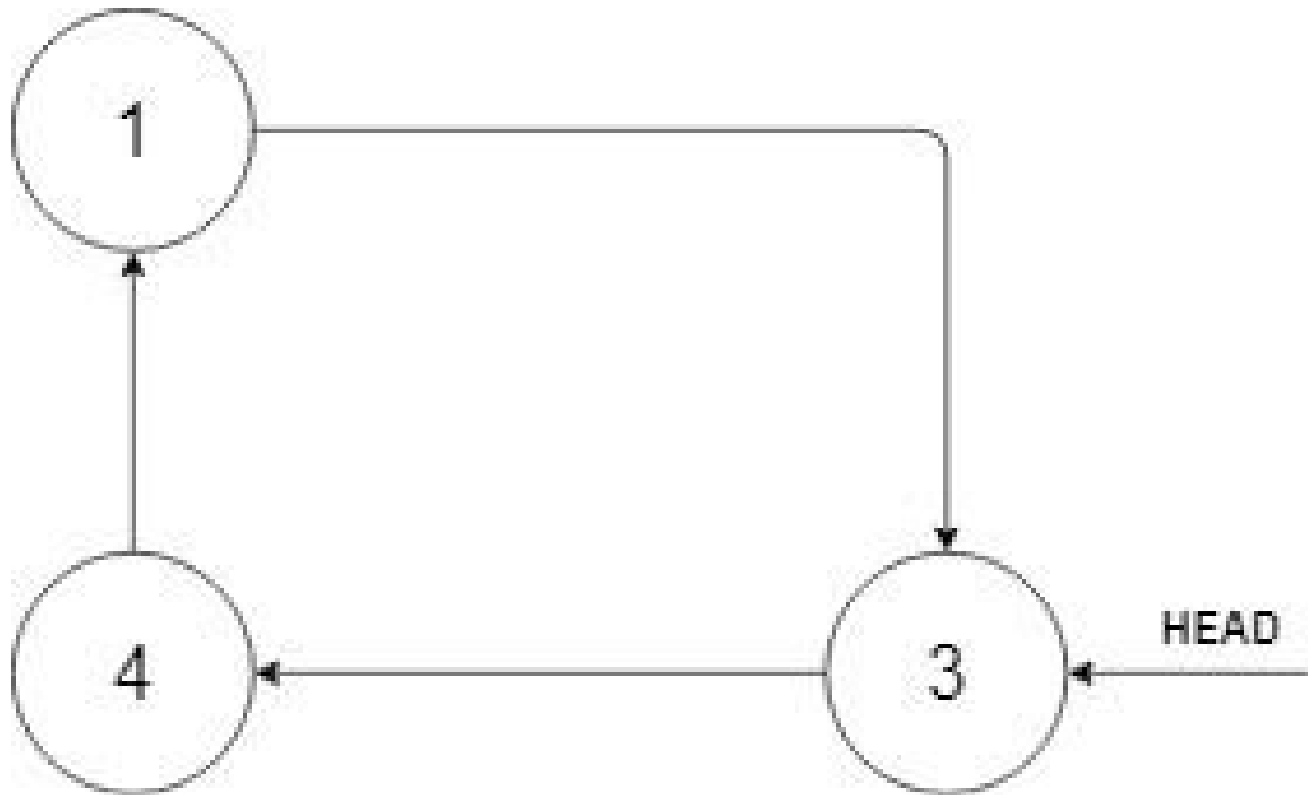
Given a Circular Linked List node, which is sorted in non-descending order, write a function to insert a value `insertVal` into the list such that it remains a sorted circular list. The given node can be a reference to any single node in the list and may not necessarily be the smallest value in the circular list.

If there are multiple suitable places for insertion, you may choose any place to insert the new value. After the insertion, the circular list should remain sorted.

If the list is empty (i.e., the given node is null), you should create a new single circular list and return the reference to that single node.

Otherwise, you should return the originally given node.

Example 1:



Input: head = [3,4,1], insertVal = 2

Output: [3,4,1,2]

Explanation: In the figure above, there is a sorted circular list of three elements. You are given a reference to the node with value 3, and we need to insert 2 into the list. The new node should be inserted between node 1 and node 3. After the insertion, the list should look like this, and we should still return node 3.

Example 2:

Input: head = [], insertVal = 1

Output: [1]

Explanation: The list is empty (given head is null). We create a new single circular list and return the reference to that single node.

Example 3:

Input: head = [1], insertVal = 0
Output: [1,0]

Constraints:

- The number of nodes in the list is in the range $[0, 5 * 10^4]$.
- $-106 \leq \text{Node.val}, \text{insertVal} \leq 106$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given a node in a sorted circular linked list, insert a value maintaining sorted order.

Return any node in the resulting list.

The list may be empty (null). Right?"

#

"A few quick questions:

What are the insertion cases? Normal ($\text{prev} \leq \text{val} \leq \text{curr}$), wraparound (at max→min boundary),

or after a full traversal when all values are equal (insert anywhere).

Do we return the original head or any node? Any node – original head is fine."

#

"Let me walk: 3→4→1→(back to 3), insert 2 → insert between 1 and 3 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Traverse all nodes collecting them, find the right position, insert there.

$O(n)$ time, $O(n)$ space for the collected list.

Should I go ahead and optimize?"

```
class _708_InsertIntoCL_Brute:
```

```
    def insert(self, head, insertVal):
```

```
        new_node = Node(insertVal)
```

```
        if not head: new_node.next = new_node; return new_node
```

```
        nodes, cur = [], head
```

```
        while True:
```

```
            nodes.append(cur); cur = cur.next
```

```
            if cur is head: break
```

```
        nodes.sort(key=lambda n: n.val)
```

```
        for i, n in enumerate(nodes):
```

```

        if n.val <= insertVal <= nodes[(i+1) % len(nodes)].val:
            new_node.next = n.next; n.next = new_node; return head
    nodes[-1].next = new_node; new_node.next = nodes[0]; return head

```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ list and sort. We can do one clean $O(n)$ pass.

Three cases as we walk (prev, curr) pairs around the circle:

Case 1 (normal): $\text{prev.val} \leq \text{insertVal} \leq \text{curr.val}$ – insert between them.

Case 2 (wraparound): $\text{prev.val} > \text{curr.val}$ (we're at the max→min boundary).

Insert here if $\text{insertVal} \geq \text{prev.val}$ (new max) OR $\text{insertVal} \leq \text{curr.val}$ (new min).

Case 3 (full circle, all equal): went around without inserting – insert anywhere.

#

Time: $O(n)$. Space: $O(1)$.

Does that make sense before I code it?"

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _708_InsertIntoSortedCircularLL:
```

```
    def insert(self, head, insertVal):
```

```
        new_node = Node(insertVal)
```

```
        if not head:
```

```
            new_node.next = new_node
```

```
            return new_node
```

```
        prev, curr = head, head.next
```

```
        while True:
```

```
            normal = prev.val <= insertVal <= curr.val
```

```
            wraparound = prev.val > curr.val and (insertVal >= prev.val or insertVal
```

```
<= curr.val)
```

```
            if normal or wraparound:
```

```
                break
```

```
            prev, curr = curr, curr.next
```

```
            if prev is head:
```

```
                break
```

```
        prev.next = new_node
```

```
        new_node.next = curr
```

```
        return head
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

3→4→1→3, insert 2: prev=3,curr=4: $3 \leq 2 \leq 4$? No. prev=4,curr=1: $4 > 1$ wraparound: $2 \geq 4$? No. $2 \leq 1$? No.

prev=1,curr=3: $1 \leq 2 \leq 3$? Yes → insert between 1 and 3. → 3→4→1→2→3 ✓

#

Insert 5 (new max): prev=4,curr=1: wraparound, $5 \geq 4$? Yes → insert between 4 and 1.

✓

Insert 0 (new min): prev=4,curr=1: wraparound, $0 \leq 1$? Yes → insert between 4 and 1.

✓

#

"No bugs. The wraparound branch handles both new maximum and new minimum in one check."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: at most one full traversal. Space $O(1)$."

All-equal list: we complete the circle without inserting → break at 'if prev is head' → insert anywhere.

Follow-up: delete from sorted circular linked list – find and unlink the node.

Follow-up: merge two sorted circular linked lists – split both at the max→min boundary, merge, relink."

Community Solutions (Python)

• LeetDoocs

```

"""
# Definition for a Node.
class Node:
    def __init__(self, val=None, next=None):
        self.val = val
        self.next = next
"""

class Solution:

    def insert(self, head: 'Optional[Node]', insertVal: int) -> 'Node':
        node = Node(insertVal)
        if head is None:
            node.next = node
            return node
        prev, curr = head, head.next
        while curr != head:
            if prev.val <= insertVal <= curr.val or (
                prev.val > curr.val and (insertVal >= prev.val or insertVal <=
curr.val)
            ):
                break
            prev, curr = curr, curr.next
        prev.next = node
        node.next = curr
        return head

```

• SimplyLeet

```
# Definition for a Node.
class Node:
    def __init__(self, val, next=None):
        self.val = val
        self.next = next

class Solution:
    def insert(self, head: 'Node', insertVal: int) -> 'Node':
        # If the list is empty, create a new node pointing to itself and return
        # it.
        if not head:
            newNode = Node(insertVal)
            newNode.next = newNode
            return newNode

        curr = head
        inserted = False

        while True:
            # Case 1: insert between two nodes where insertVal is between
            # curr.val and curr.next.val.
            if curr.val <= insertVal <= curr.next.val:
                inserted = True
                # Case 2: curr.val > curr.next.val, i.e., we are at the turning
                # point.
                # In this case, insert if insertVal is greater than curr.val
                # (largest)
                # or insertVal is less than curr.next.val (smallest).
                elif curr.val > curr.next.val:
                    if insertVal >= curr.val or insertVal <= curr.next.val:
                        inserted = True

            if inserted:
                newNode = Node(insertVal, curr.next)
```

```

        curr.next = newNode
        return head

    curr = curr.next
    # If we have traversed the entire circle and came back to head,
    break.

    if curr == head:
        break

    # Case 3: All values are the same or no valid index was found, insert
    arbitrarily.
    newNode = Node(insertVal, curr.next)

    curr.next = newNode
    return head

```

• LC.ca

```

"""
# Definition for a Node.
class Node:
    def __init__(self, val=None, next=None):
        self.val = val
        self.next = next
"""

class Solution:

    def insert(self, head: 'Optional[Node]', insertVal: int) -> 'Node':
        node = Node(insertVal)
        if head is None:
            node.next = node
            return node
        prev, curr = head, head.next
        while curr != head:
            if prev.val <= insertVal <= curr.val or (
                prev.val > curr.val and (insertVal >= prev.val or insertVal <=
curr.val)
            ):

```



```
        break
    prev, curr = curr, curr.next
    prev.next = node
    node.next = curr
    return head
```

◆ Quick Review

Key Insights

- Handle the empty list edge-case by creating a new node that points to itself.
- Traverse the list until the proper insertion point is found.
- Consider the "turning point" where the maximum value is followed by the minimum value.
- If no proper place is found in one full rotation, insert the new node arbitrarily (e.g., after the head).

Space & Time

Time Complexity: $O(n)$ in the worst-case when a full loop is required.

Space Complexity: $O(1)$ since we only use a few extra pointers.

Solution

We traverse the circular linked list starting from the arbitrary given node. For each pair of consecutive nodes (curr and next), we check if the new value can be inserted between them by confirming that: The new value lies between curr and next ($\text{curr.val} \leq \text{insertVal} \leq \text{next.val}$). Or if we are at the pivot point where the maximum value meets the minimum value (i.e., $\text{curr.val} > \text{next.val}$) and then either the new value is larger than or equal to curr.val (should be appended after the maximum) or less than or equal to next.val (should be inserted before the minimum). If no proper spot is found after one complete loop, we insert anywhere (all values in the list are the same). A pointer reference is updated accordingly to maintain the circular structure.

Linked List

□ #1171 Remove Zero Sum Consecutive Nodes from Linked List

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Linked List
Lists: Denny Zhang

Problem

Given the head of a linked list, we repeatedly delete consecutive sequences of nodes that sum to 0 until there are no such sequences.

After doing so, return the head of the final linked list. You may return any such answer.

(Note that in the examples below, all sequences are serializations of ListNode objects.)

Example 1:

Input: head = [1,2,-3,3,1]
Output: [3,1]
Note: The answer [1,2,1] would also be accepted.

Example 2:

Input: head = [1,2,3,-3,4]
Output: [1,2,4]

Example 3:

Input: head = [1,2,3,-3,-2]
Output: [1]

Constraints:

- The given linked list will contain between 1 and 1000 nodes.
- Each node in the linked list has $-1000 \leq \text{node.val} \leq 1000$.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Repeatedly remove consecutive sequences of nodes that sum to 0 until none remain.
# Return the resulting linked list. Right?"
#
# "A few quick questions:
# Can multiple non-overlapping zero-sum sequences exist? Yes – remove all.
# Can values be negative? Yes – that's what makes zero-sum possible.
# Does the answer need to be unique? The problem guarantees any valid answer is
# accepted."
#
# "Let me walk: [1,2,-3,3,1] → [1,2,-3] sums to 0 → [3,1] ✓.
# Also valid: [2,-3,3]=2 (not zero) → only [1,2,-3] works first."
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Repeatedly scan all consecutive subsequences for zero sum, remove the first found,
# repeat.
#  $O(n^3)$  per pass times multiple passes – extremely slow.
# Should I go ahead and optimize?"
class _1171_RemoveZeroSumNodes_Brute:
    def removeZeroSumSublists(self, head):
        dummy = ListNode(0, head)
        changed = True
        while changed:
            changed = False; cur = dummy
            while cur:
                total = 0; runner = cur.next
```

```

        while runner:
            total += runner.val
            if total == 0: cur.next = runner.next; changed = True; break
            runner = runner.next
        if changed: break
        cur = cur.next
    return dummy.next

```

PHASE 3 — OPTIMIZE

```

# "The bottleneck is the nested scan – we recompute prefix sums from scratch each
pass.
# Use a prefix sum + hash map in two passes (O(n)):
# Pass 1: walk the list computing running sum; store prefix_sum → latest node.
# If the same prefix sum appears twice, nodes between those two points sum to 0.
# The override ensures we store the LAST node with that prefix sum.
# Pass 2: walk again; each node's prefix sum is already in the map pointing to the
node
# just past the zero-sum segment – skip directly to it.
#
# Time: O(n). Space: O(n).
# Does that make sense before I code it?"

```

PHASE 4 — CLEAN CODE

```

# "Now I'll implement the optimized approach with meaningful names."
class _1171_RemoveZeroSumNodes:
    def removeZeroSumSublists(self, head):
        dummy = ListNode(0, head)
        prefix_to_node = {}
        cur, total = dummy, 0
        while cur:
            total += cur.val
            prefix_to_node[total] = cur
            cur = cur.next
        cur, total = dummy, 0
        while cur:
            total += cur.val
            cur.next = prefix_to_node[total].next
            cur = cur.next
        return dummy.next

```

PHASE 5 — TEST & DEBUG

```

# "Let me trace through a few cases."
#
# dummy→1→2→-3→3→1:
# Pass1: sums: 0→dummy, 1→1, 3→2, 0→-3(override!), 3→3(override!), 4→1.
# seen={0:-3node, 1:1node, 3:3node, 4:1node}.
# Pass2: total=0→dummy.next=seen[0].next=3node. total=3→3.next=seen[3].next=1node.
# total=4→1.next=seen[4].next=None. → [3,1] ✓
#

```

"No bugs. Override in pass 1 ensures we skip to the node AFTER the last zero-sum segment."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: two linear passes. Space $O(n)$: prefix sum map.

Same pattern as Subarray Sum Equals K (#560) applied to a linked list.

The override-on-collision in pass 1 is the key non-obvious step – worth explaining clearly.

Follow-up: count removed nodes – track length before and after.

Follow-up: if values can be floats, use epsilon comparison for the zero-sum check."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def removeZeroSumSublists(self, head: ListNode) -> ListNode:
        dummy = ListNode(0, head)
        prefix = 0
        prefixToNode = {0: dummy}

        while head:
            prefix += head.val
            prefixToNode[prefix] = head
            head = head.next

        prefix = 0
        head = dummy

        while head:
            prefix += head.val
            head.next = prefixToNode[prefix].next
            head = head.next

        return dummy.next
```

• LeetDoocs

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def removeZeroSumSublists(self, head: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode(next=head)
        last = {}
        s, cur = 0, dummy

        while cur:
            s += cur.val
            last[s] = cur
            cur = cur.next
        s, cur = 0, dummy
        while cur:
            s += cur.val
            cur.next = last[s].next
            cur = cur.next
        return dummy.next
```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def removeZeroSumSublists(self, head: ListNode) -> ListNode:
        # Create a dummy node to simplify cases where removal starts at the head.
        dummy = ListNode(0)
```



```

dummy.next = head

# Dictionary to store prefix sum to node mapping.
sum_to_node = {}
prefix_sum = 0
current = dummy

# First pass: record the last occurrence of each prefix sum.
while current:
    prefix_sum += current.val

    sum_to_node[prefix_sum] = current
    current = current.next

# Second pass: remove nodes that are part of zero-sum sequences.
prefix_sum = 0
current = dummy
while current:
    prefix_sum += current.val
    # Link current node's next pointer to skip the zero-sum
    subsequence.
    current.next = sum_to_node[prefix_sum].next

    current = current.next

return dummy.next

```

• LC.ca

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def removeZeroSumSublists(self, head: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode(next=head)
        last = {}
        s, cur = 0, dummy

```

```
while cur:
    s += cur.val
    last[s] = cur
    cur = cur.next
s, cur = 0, dummy
while cur:
    s += cur.val
    cur.next = last[s].next
    cur = cur.next
return dummy.next
```

◆ Quick Review

Key Insights

- Use a dummy node to handle edge cases such as the removal of nodes at the head.
- Compute a running (prefix) sum while traversing the list.
- Use a hash table (or hashmap) to map each prefix sum to the most recent node where that sum occurred.
- If the same prefix sum is encountered again, the nodes in between sum to 0 and can be removed by adjusting pointers.
- Perform two passes: the first to record prefix sums and their corresponding nodes, and the second to remove zero-sum sequences.

Space & Time

Time Complexity: $O(n)$ – Two passes over the list.

Space Complexity: $O(n)$ – Hash table storing at most n prefix sums.

Solution

The solution uses a two-pass algorithm with a hash table. In the first pass, compute the prefix sum for each node and store the mapping from prefix sum to the node (overwriting with the latest occurrence). This ensures that for any repeated prefix sum, the nodes in between sum to 0. In the second pass, traverse the list again and use the hash table to skip the zero-sum subsequences by redirecting the next pointer of a node to the next pointer of the stored node for that prefix sum. Key data structures are the hash table and a dummy node to simplify pointer adjustments.

Stack

Round 5 · Mid · Medium · 14 questions

Stack

□ #143 Reorder List

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Two Pointers · Stack · Recursion
Lists: Grind 169

Problem

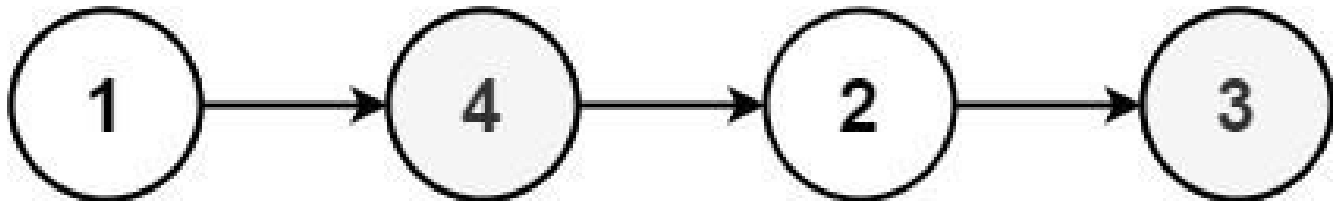
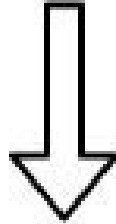
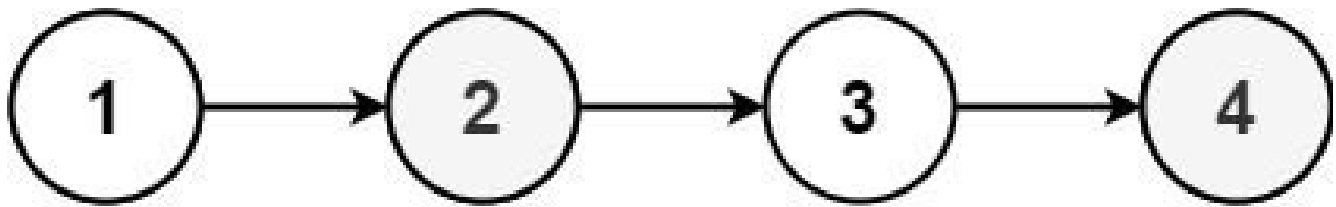
You are given the head of a singly linked-list.
The list can be represented as:

$$L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$$

Reorder the list to be on the following form:

$$L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \rightarrow \dots$$

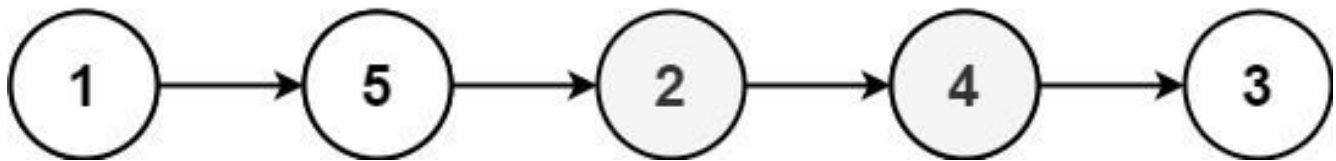
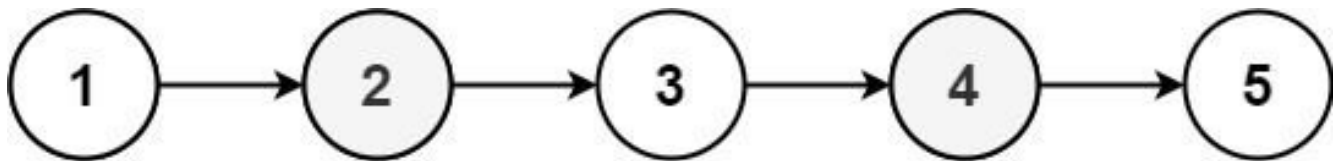
You may not modify the values in the list's nodes. Only nodes themselves may be changed.
Example 1:



Input: head = [1,2,3,4]

Output: [1,4,2,3]

Example 2:



Input: head = [1,2,3,4,5]

Output: [1,5,2,4,3]

Constraints:

- The number of nodes in the list is in the range $[1, 5 * 10^4]$.
- $1 \leq \text{Node.val} \leq 1000$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Reorder in-place: L0→Ln→L1→Ln-1→L2→Ln-2→...
# We swap the actual nodes, not just values. No return value. Right?"
#
# "A few quick questions:
# Can the list have 1 or 2 nodes? Yes – already in order, no-op.
# O(1) space required? Yes – must not copy values or create new nodes."
#
# "Let me walk: [1,2,3,4] → [1,4,2,3] ✓. [1,2,3,4,5] → [1,5,2,4,3] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Collect all nodes in an array, then use two pointers (left, right) to interleave
them.
# O(n) time but O(n) space for the node-reference array.
# Should I go ahead and optimize?"
class _143_ReorderList_Brute:
    def reorderList(self, head):
        nodes, cur = [], head
        while cur: nodes.append(cur); cur = cur.next
        left, right = 0, len(nodes) - 1
        while left < right:
            nodes[left].next = nodes[right]
            left += 1
            if left == right: break
            nodes[right].next = nodes[left]
            right -= 1
        nodes[left].next = None
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the O(n) auxiliary array of node references.
# Three in-place steps: 1. Find the mid with slow/fast. 2. Reverse the second half.
# 3. Merge the two halves by interleaving.
#
# Time: O(n). Space: O(1).
```



```
# Does that make sense before I code it?"
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _143_ReorderList:
    def reorderList(self, head):
        slow = fast = head
        while fast and fast.next:
            slow = slow.next; fast = fast.next.next
        prev, cur = None, slow
        while cur:
            nxt = cur.next; cur.next = prev; prev, cur = cur, nxt
        second = prev
        first = head
        while second.next:
            tmp1, tmp2 = first.next, second.next
            first.next = second; second.next = tmp1
            first, second = tmp1, tmp2
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
```

```
#
```

```
# [1,2,3,4]: mid: slow=2. Reverse [2,3,4]→[4,3,2]. second=4.
```

```
# Merge: first=1,second=4: 1→4→2, then first=2,second=3: 2→3→None. → 1→4→2→3 ✓
```

```
#
```

```
# [1,2,3,4,5]: mid: slow=3. Reverse [3,4,5]→[5,4,3].
```

```
# Merge: 1→5→2→4→3 ✓
```

```
#
```

```
# "No bugs. Loop stops at 'second.next' to leave the odd middle node correctly placed."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): three linear passes. Space O(1): only pointer variables.
```

```
# This combines three classic linked-list techniques: mid-find, reverse, merge.
```

```
# Follow-up: Palindrome Linked List (#234) uses the same first two steps.
```

```
# Follow-up: if values-only (not node-pointer) swap allowed, the brute approach is simpler."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def reorderList(self, head: ListNode) -> None:
        def findMid(head: ListNode):
            prev = None
            slow = head
            fast = head

            while fast and fast.next:
                prev = slow
                slow = slow.next
                fast = fast.next.next
            prev.next = None

            return slow

        def reverse(head: ListNode) -> ListNode:
            prev = None
            curr = head

            while curr:
                next = curr.next
                curr.next = prev
                prev = curr
                curr = next

            return prev

        def merge(l1: ListNode, l2: ListNode) -> None:
            while l2:
                next = l1.next
                l1.next = l2
                l1 = l2
                l2 = next

            if not head or not head.next:
                return

            mid = findMid(head)
            reversed = reverse(mid)
            merge(head, reversed)
```

• LeetCode

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reorderList(self, head: Optional[ListNode]) -> None:
        fast = slow = head
        while fast.next and fast.next.next:
            slow = slow.next

            fast = fast.next.next

        cur = slow.next
        slow.next = None

        pre = None
        while cur:
            t = cur.next
            cur.next = pre
            pre, cur = cur, t

        cur = head

        while pre:
            t = pre.next
            pre.next = cur.next
            cur.next = pre
            cur, pre = pre.next, t
```

• SimplyLeet

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def reorderList(self, head: ListNode) -> None:
        if not head or not head.next:
            return

        # Step 1: Find the middle of the linked list using slow and fast
        # pointers.
        slow, fast = head, head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        # Step 2: Reverse the second half of the list.
        prev, curr = None, slow.next
        slow.next = None # Split the list into two halves.

        while curr:
            next_temp = curr.next # Save next node.
            curr.next = prev      # Reverse current pointer.
            prev = curr           # Move prev to current.
            curr = next_temp      # Move to the next node.

        # Step 3: Merge the two halves, alternating nodes.
        first, second = head, prev
        while second:
            temp1 = first.next # Save next node from first half.

            temp2 = second.next # Save next node from reversed second half.
            first.next = second # Link first node to second list node.
            second.next = temp1 # Link second node to the next node in first
            half.

            first = temp1      # Move pointer in first half.
            second = temp2     # Move pointer in second half.

```

• LC.ca

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reorderList(self, head: Optional[ListNode]) -> None:
        fast = slow = head
        while fast.next and fast.next.next:
            slow = slow.next

            fast = fast.next.next

        cur = slow.next
        slow.next = None

        pre = None
        while cur:
            t = cur.next
            cur.next = pre
            pre, cur = cur, t

        cur = head

        while pre:
            t = pre.next
            pre.next = cur.next
            cur.next = pre
            cur, pre = pre.next, t
```

◆ Quick Review

Key Insights

- Find the middle of the linked list using the fast and slow pointer technique.
- Reverse the second half of the list.

- Merge the two halves, alternating nodes from the first half and the reversed second half.
- The challenge is done in-place, ensuring a linear time solution with constant extra space.

Space & Time

Time Complexity: $O(n)$ – Each of the three main steps (finding the middle, reversing, merging) takes $O(n)$ time.

Space Complexity: $O(1)$ – The algorithm is done in-place with only a few pointers.

Solution

The solution involves three key steps: Use two pointers (slow and fast) to identify the middle of the list. Reverse the second half of the list. This can be done iteratively by adjusting the next pointers. Merge the two lists: One starting from the head and the other from the head of the reversed second half. Interleave the nodes by alternating between the two lists until complete. This approach uses in-place pointer manipulation, thus ensuring no extra space is used aside from a few temporary variables.

Stack

□ #150 Evaluate Reverse Polish Notation

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math · Stack
Lists: Grind 169

Problem

You are given an array of strings `tokens` that represents an arithmetic expression in a Reverse Polish Notation.

Evaluate the expression. Return an integer that represents the value of the expression.

Note that:

- The valid operators are '+', '-', '*', and '/'.
- Each operand may be an integer or another expression.
- The division between two integers always truncates toward zero.
- There will not be any division by zero.
- The input represents a valid arithmetic expression in a reverse polish notation.

- The answer and all the intermediate calculations can be represented in a 32-bit integer.

Example 1:

Input: tokens = ["2","1","+","3","*"]
 Output: 9
 Explanation: $((2 + 1) * 3) = 9$

Example 2:

Input: tokens = ["4","13","5","/","+"]
 Output: 6
 Explanation: $(4 + (13 / 5)) = 6$

Example 3:

Input: tokens = ["10","6","9","3","+","-11","*","/","*","17","+","5","+"]
 Output: 22
 Explanation: $((10 * (6 / ((9 + 3) * -11))) + 17) + 5$
 $= ((10 * (6 / (12 * -11))) + 17) + 5$
 $= ((10 * (6 / -132)) + 17) + 5$
 $= ((10 * 0) + 17) + 5$
 $= (0 + 17) + 5$
 $= 17 + 5$

Constraints:

- $1 \leq \text{tokens.length} \leq 104$
- $\text{tokens}[i]$ is either an operator: "+", "-", "*", or "/", or an integer in the range $[-200, 200]$.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Evaluate an expression in Reverse Polish Notation: operands push onto a stack,

```
# operators pop two, compute, and push result. Division truncates toward zero.
Right?"
#
# "A few quick questions:
# Only +, -, *, /? Yes. Can intermediate results overflow 32-bit? Python handles big
ints natively.
# Division truncates toward zero – Python's // floors toward negative infinity, so I
need int(a/b)."
#
# "Let me walk: ['2','1','+', '3','*'] → (2+1)*3 = 9 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Scan for any operator, apply it to the two preceding tokens, replace all three
with the result.
#  $O(n^2)$  – each scan is  $O(n)$  and we may need  $n/2$  scans.
# Should I go ahead and optimize?"
```

```
class _150_EvalRPN_Brute:
    def evalRPN(self, tokens):
        tokens = list(tokens)
        ops = {'+', '-', '*', '/'}
        while len(tokens) > 1:
            for i, t in enumerate(tokens):
                if t in ops:
                    a, b = int(tokens[i-2]), int(tokens[i-1])
                    if t=='+': r=a+b
                    elif t=='-': r=a-b
                    elif t=='*': r=a*b
                    else: r=int(a/b)
                    tokens[i-2:i+1] = [str(r)]; break
        return int(tokens[0])
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the repeated linear scan.
# A stack processes this in a single left-to-right pass:
# push operands; on each operator, pop two, compute, push result.
# Final stack element is the answer.
#
# Time:  $O(n)$ . Space:  $O(n)$  for the stack.
# Does that make sense before I code it?"
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _150_EvalRPN:
    def evalRPN(self, tokens):
        stack = []
        ops = {
            '+': lambda a, b: a + b,
            '-': lambda a, b: a - b,
            '*': lambda a, b: a * b,
            '/': lambda a, b: int(a / b),
```

```

    }
    for token in tokens:
        if token in ops:
            b, a = stack.pop(), stack.pop()
            stack.append(ops[token](a, b))
        else:
            stack.append(int(token))
    return stack[0]

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

['2', '1', '+', '3', '*']:

push 2→[2]. push 1→[2,1]. '+' pop 1,2→push 3→[3]. push 3→[3,3]. '*' pop 3,3→push 9→[9].

return 9 ✓

#

['4', '13', '5', '/', '+']: 13/5=2 (int(13/5)=2 ✓). 4+2=6 ✓

#

"No bugs. int(a/b) truncates toward zero – Python's a//b would floor for negatives."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time O(n): single pass. Space O(n): at most (n+1)/2 operands on stack simultaneously.

The lambda dict keeps the operator dispatch clean and avoids four if-elif branches.

Follow-up: Basic Calculator (#224) – infix with parentheses; needs operator precedence stack.

Follow-up: validate an RPN expression – operand count must always exceed operator count."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def evalRPN(self, tokens: list[str]) -> int:
        stack = []
        op = {
            '+': lambda a, b: a + b,
            '-': lambda a, b: a - b,
            '*': lambda a, b: a * b,
            '/': lambda a, b: int(a / b),
        }

```

```

for token in tokens:
    if token in op:
        b = stack.pop()
        a = stack.pop()
        stack.append(op[token](a, b))
    else:
        stack.append(int(token))

return stack.pop()

```

• LeetDoocs

```
import operator
```

```
class Solution:
```

```
    def evalRPN(self, tokens: List[str]) -> int:
```

```
        opt = {
```

```
            "+": operator.add,
```

```
            "-": operator.sub,
```

```
            "*": operator.mul,
```

```
            "/": operator.truediv,
```

```
        }
```

```
        s = []
```

```
        for token in tokens:
```

```
            if token in opt:
```

```
                s.append(int(opt[token](s.pop(-2), s.pop(-1))))
```

```
            else:
```

```
                s.append(int(token))
```

```
        return s[0]
```

```
class Solution:
```

```
    def evalRPN(self, tokens: List[str]) -> int:
```

```
        nums = []
```

```
        for t in tokens:
```

```
            if len(t) > 1 or t.isdigit():
```

```
                nums.append(int(t))
```

```
            else:
```

```
                if t == "+":
```

```
                    nums[-2] += nums[-1]
```

```
                elif t == "-":
```

```

        nums[-2] -= nums[-1]
    elif t == "*":
        nums[-2] *= nums[-1]
    else:
        nums[-2] = int(nums[-2] / nums[-1])
    nums.pop()
return nums[0]

```

• SimplyLeet

```

def evalRPN(tokens):
    # Initialize a stack to hold numbers
    stack = []
    # Loop through each token in the input list
    for token in tokens:
        if token in {"+", "-", "*", "/"}:
            # Pop the two topmost values; note the order is important
            b = stack.pop()
            a = stack.pop()
            # Apply the operation based on the token

            if token == "+":
                result = a + b
            elif token == "-":
                result = a - b
            elif token == "*":
                result = a * b
            else:
                # Use int division to truncate towards zero
                result = int(a / b)
            # Push the result back onto the stack

            stack.append(result)
        else:
            # Token is a number; push its integer value onto the stack
            stack.append(int(token))
    # The final result is the only element left in the stack
    return stack.pop()

# Example usage:
# tokens = ["2", "1", "+", "3", "*"]
# print(evalRPN(tokens)) # Output: 9

```

• LC.ca

```
class Solution:
    def evalRPN(self, tokens: List[str]) -> int:
        nums = []
        for t in tokens:
            if len(t) > 1 or t.isdigit():
                nums.append(int(t))
            else:
                if t == "+":
                    nums[-2] += nums[-1]
                elif t == "-":
                    nums[-2] -= nums[-1]
                elif t == "*":
                    nums[-2] *= nums[-1]
                else:
                    nums[-2] = int(nums[-2] / nums[-1])
                nums.pop()
        return nums[0]
```

◆ Quick Review

Key Insights

- Use a stack to process the tokens.
- Push numbers onto the stack.
- When an operator is encountered, pop the top two numbers, perform the operation, and push the result back.
- Handle division carefully to ensure it truncates toward zero.
- The final result remains as the only value in the stack.

Space & Time

Time Complexity: $O(n)$, where n is the number of tokens (each token is processed once).

Space Complexity: $O(n)$, in the worst-case scenario where all tokens are numbers and are stored in the stack.

Solution

We use a stack data structure to evaluate the RPN expression. Iterate through each token: If the token is a number, convert and push it onto the stack. If the token is an operator, pop the top two numbers (be careful about order), perform the corresponding arithmetic operation, and push the result back onto the stack. At the end of processing, the stack contains the result of the expression. Special attention is needed for division to ensure it truncates toward zero (using language-specific functions when necessary).

Stack

□ #155 Min Stack

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Stack · Design
Lists: Grind 169

Problem

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Implement the MinStack class:

- `MinStack()` initializes the stack object.
- `void push(int val)` pushes the element `val` onto the stack.
- `void pop()` removes the element on the top of the stack.
- `int top()` gets the top element of the stack.
- `int getMin()` retrieves the minimum element in the stack.

You must implement a solution with $O(1)$ time complexity for each function.

Example 1:

Input

```
["MinStack", "push", "push", "push", "getMin", "pop", "top", "getMin"]
[[], [-2], [0], [-3], [], [], [], []]
```

Output

```
[null, null, null, null, -3, null, 0, -2]
```

Explanation

Constraints:

- $-231 \leq \text{val} \leq 231 - 1$
- Methods pop, top and getMin operations will always be called on non-empty stacks.
- At most $3 * 10^4$ calls will be made to push, pop, top, and getMin.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Design a stack that supports push, pop, top, and getMin — all $O(1)$. Right?"

#

"A few quick questions:

getMin must return the current minimum, which can change as we push/pop? Yes.

pop and top are only called on non-empty stacks? Yes."

#

"Let me walk: push(-2), push(0), push(-3). getMin→-3. pop(). top()→0. getMin()→-2

✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Regular stack for push/pop/top. For getMin, scan the entire stack each time.

push/pop/top $O(1)$, getMin $O(n)$.

Should I go ahead and optimize?"

```
class _155_MinStack_Brute:
```

```
    def __init__(self): self.stack = []
```

```
def push(self, v): self.stack.append(v)
def pop(self): self.stack.pop()
def top(self): return self.stack[-1]
def getMin(self): return min(self.stack)
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the O(n) scan for getMin.
# Maintain a parallel 'min stack' tracking the minimum at each level:
# when we push val, also push min(val, min_stack.top()) to the min stack.
# When we pop, pop both stacks together.
# getMin always reads min_stack.top() - O(1).
#
# Time: O(1) all ops. Space: O(n) for the two stacks."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _155_MinStack:
    def __init__(self):
        self._stack = []
        self._min_stack = []

    def push(self, val):
        self._stack.append(val)
        current_min = val if not self._min_stack else min(val, self._min_stack[-1])
        self._min_stack.append(current_min)

    def pop(self):
        self._stack.pop()
        self._min_stack.pop()

    def top(self): return self._stack[-1]
    def getMin(self): return self._min_stack[-1]
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# push(-2): stack=[-2], min_stack=[-2].
# push(0): stack=[-2,0], min_stack=[-2,-2].
# push(-3): stack=[-2,0,-3], min_stack=[-2,-2,-3].
# getMin()→-3 ✓. pop(): stack=[-2,0], min_stack=[-2,-2].
# top()→0 ✓. getMin()→-2 ✓
#
# "No bugs. Each level of the min_stack records the minimum valid AT THAT DEPTH."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "All ops O(1). Space O(n): two stacks of size n.
# Alternative: store (val, current_min) as a tuple in one stack - halves number of lists.
# Follow-up: Max Stack (#716) - same parallel-stack idea with a max_stack.
# Follow-up: if push is frequent but getMin rare, brute O(n) getMin is acceptable."
```

Community Solutions (Python)

• WalkCC

```
class MinStack:
    def __init__(self):
        self.stack = []

    def push(self, x: int) -> None:
        mn = x if not self.stack else min(self.stack[-1][1], x)
        self.stack.append([x, mn])

    def pop(self) -> None:
        self.stack.pop()

    def top(self) -> int:
        return self.stack[-1][0]

    def getMin(self) -> int:
        return self.stack[-1][1]
```

• LeetDoocs

```
class MinStack:
    def __init__(self):
        self.stk1 = []
        self.stk2 = [inf]

    def push(self, val: int) -> None:
        self.stk1.append(val)
        self.stk2.append(min(val, self.stk2[-1]))

    def pop(self) -> None:
        self.stk1.pop()
        self.stk2.pop()

    def top(self) -> int:
        return self.stk1[-1]

    def getMin(self) -> int:
        return self.stk2[-1]
```

```
# Your MinStack object will be instantiated and called as such:
# obj = MinStack()
# obj.push(val)
# obj.pop()
# param_3 = obj.top()
# param_4 = obj.getMin()
```

• SimplyLeet

```
# Python implementation of MinStack with detailed comments
```

```
class MinStack:
    def __init__(self):
        # Main stack to store all values
        self.stack = []
        # Auxiliary stack to store minimum values
        self.min_stack = []

    def push(self, val: int) -> None:

        # Push value onto main stack
        self.stack.append(val)
        # If min_stack is empty or the new value is less than or equal to
        current minimum, push it onto min_stack
        if not self.min_stack or val <= self.min_stack[-1]:
            self.min_stack.append(val)

    def pop(self) -> None:
        # Pop the top value from main stack
        popped_value = self.stack.pop()
        # If the popped value is the current minimum, pop it from min_stack as
        well

        if popped_value == self.min_stack[-1]:
            self.min_stack.pop()

    def top(self) -> int:
        # Return the top value from main stack without removing it
        return self.stack[-1]

    def getMin(self) -> int:
        # Return the minimum value from min_stack
        return self.min_stack[-1]
```

```
# Example usage:
# minStack = MinStack()
# minStack.push(-2)
# minStack.push(0)
# minStack.push(-3)
# print(minStack.getMin()) # -> -3
# minStack.pop()
# print(minStack.top()) # -> 0
# print(minStack.getMin()) # -> -2
```

• LC.ca

```
class MinStack:
    def __init__(self):
        self.sk = []
        self.minsk = [inf] # trick, avoid later empty-check for min-stack

    def push(self, x: int) -> None:
        self.sk.append(x)
        self.minsk.append(min(x, self.minsk[-1]))

    def pop(self) -> None:

if not self.sk:
    return
    self.sk.pop()
    self.minsk.pop()

    def top(self) -> int:
        return self.sk[-1]

    def getMin(self) -> int:
        return self.minsk[-1]
```

```
# Your MinStack object will be instantiated and called as such:
# obj = MinStack()
# obj.push(x)
# obj.pop()
# param_3 = obj.top()
# param_4 = obj.getMin()

#####
```

```
class MinStack:

    def __init__(self):
        self.sk = []
        self.minsk = []

    def push(self, val: int) -> None:
        self.sk.append(val)

        # empty check
        self.minsk.append(val if not self.minsk else min(val, self.minsk[-1]))

    def pop(self) -> None:
        if not self.sk:
            return

        self.sk.pop()
        self.minsk.pop()
```

```
def top(self) -> int:
    return self.sk[-1]

def getMin(self) -> int:
    return self.minsk[-1]

# optimize above, using only 1 stack, with stack element as tuple: (val, its
# associated min)
class MinStack:

    def __init__(self):

        self._stack = []

    def push(self, x: int) -> None:
        cur_min = self.getMin()
        if x < cur_min:
            cur_min = x
        self._stack.append((x, cur_min))

    def pop(self) -> None:
        self._stack.pop()

    def top(self) -> int:
        if not self._stack:
            return None
        else:
            return self._stack[-1][0]

    def getMin(self) -> int:
        if not self._stack:
            return float('inf')

        else:
            return self._stack[-1][1]
```

◆ Quick Review

Key Insights

- Use an auxiliary data structure to track the current minimum element.
- A common approach is to use two stacks: one regular stack for all elements and a second stack to keep track of the minimum values.
- When pushing a new element, push it onto the main stack and also update the min stack if the element is less than or equal to the current minimum.
- When popping, if the popped element is equal to the top of the min stack, pop from the min stack as well.
- This design guarantees that all operations (push, pop, top, getMin) run in constant time, $O(1)$.

Space & Time

Time Complexity: $O(1)$ per operation (push, pop, top, getMin)

Space Complexity: $O(n)$, where n is the number of elements in the stack, since an additional stack is used to keep track of minima.

Solution

The solution involves using two stacks: Main Stack: Stores all the elements. Min Stack: Stores the minimum elements encountered so far. When an element is pushed, we compare it with the top of the min stack. If it is smaller than or equal to the current minimum, we also push it onto the min stack. For the pop operation, if the element being popped is the same as the top of the min stack, we pop from the min stack as well. This approach ensures that the getMin operation always returns the top element of the min stack, which is the current minimum in constant time.

Stack

□ #173 Binary Search Tree Iterator

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Stack · Tree · Design · Binary Search Tree ·
Iterator

Lists: Denny Zhang

Problem

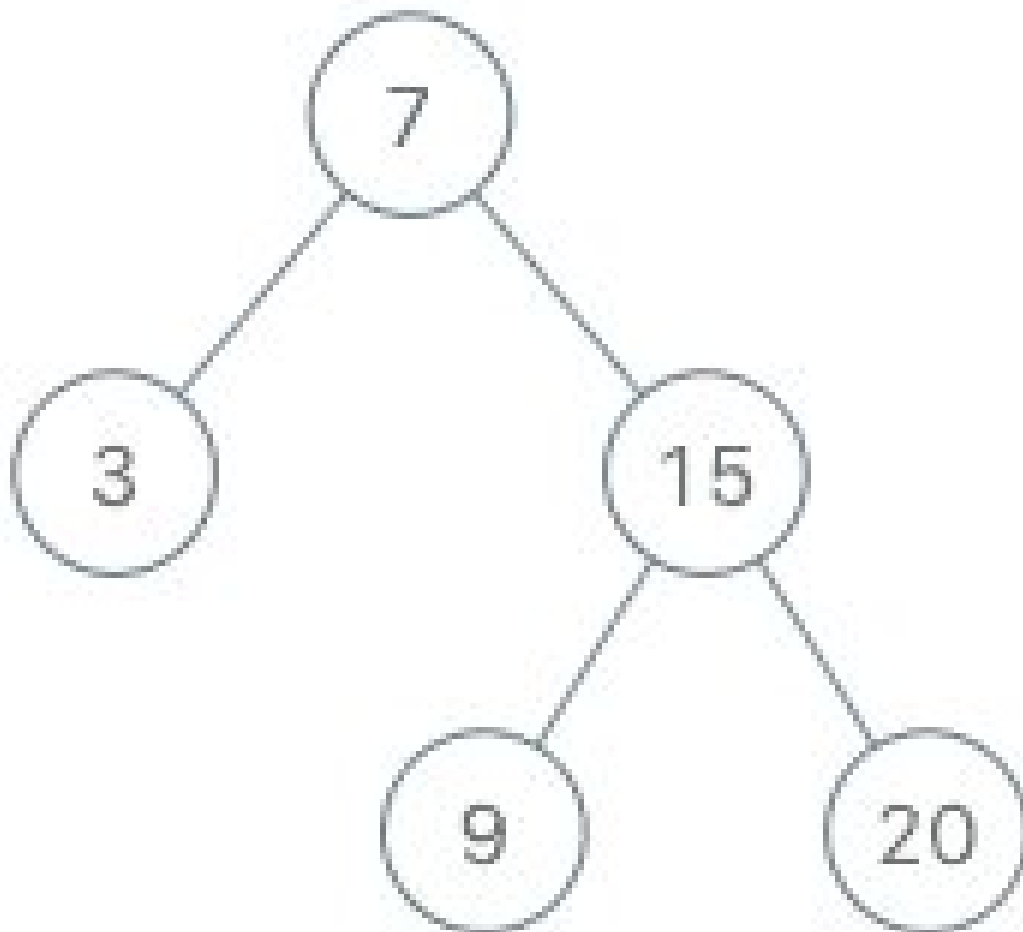
Implement the BSTIterator class that represents an iterator over the in-order traversal of a binary search tree (BST):

- **BSTIterator(TreeNode root)** Initializes an object of the BSTIterator class. The root of the BST is given as part of the constructor. The pointer should be initialized to a non-existent number smaller than any element in the BST.
- **boolean hasNext()** Returns true if there exists a number in the traversal to the right of the pointer, otherwise returns false.
- **int next()** Moves the pointer to the right, then returns the number at the pointer.

Notice that by initializing the pointer to a non-existent smallest number, the first call to `next()` will return the smallest element in the BST.

You may assume that `next()` calls will always be valid. That is, there will be at least a next number in the in-order traversal when `next()` is called.

Example 1:



Input

```
["BSTIterator", "next", "next", "hasNext", "next", "hasNext", "next", "hasNext", "next", "hasNext"]
```

```
[[[7, 3, 15, null, null, 9, 20]], [], [], [], [], [], [], [], [], []]
```

Output

```
[null, 3, 7, true, 9, true, 15, true, 20, false]
```

Explanation

```
BSTIterator bSTIterator = new BSTIterator([7, 3, 15, null, null, 9, 20]);
```

Constraints:

- The number of nodes in the tree is in the range [1, 105].
- $0 \leq \text{Node.val} \leq 106$
- At most 105 calls will be made to hasNext, and next.

Follow up:

- Could you implement next() and hasNext() to run in average $O(1)$ time and use $O(h)$ memory, where h is the height of the tree?

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Implement an iterator over a BST. next() returns the next smallest element,

hasNext() returns true if more elements remain.

Both must average $O(1)$ time and use $O(h)$ space where h is the tree height. Right?"

#

"A few quick questions:

next() is always called when an element exists? Yes – per constraints.

$O(h)$ space means we can use a stack of depth h ?"

#

"Let me walk: $\text{BST}=[7,3,15,\text{null},\text{null},9,20]$. $\text{next()} \rightarrow 3,7,9,15,20$ ✓"

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Run a full inorder traversal at construction and store all values in a list.
# next() returns the next element, hasNext() checks the index.
# O(n) space regardless of tree shape – the follow-up wants O(h).
# Should I go ahead and optimize?"
class _173_BSTIterator_Brute:
    def __init__(self, root):
        self.vals = []; self.idx = 0
        def inorder(n):
            if n: inorder(n.left); self.vals.append(n.val); inorder(n.right)
        inorder(root)
    def next(self): v = self.vals[self.idx]; self.idx += 1; return v
    def hasNext(self): return self.idx < len(self.vals)
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the O(n) pre-stored list.
# Controlled inorder traversal using an explicit stack – O(h) space:
# At init, push the entire left spine.
# next(): pop the top node (smallest available), push its right subtree's left
spine.
# Each node is pushed and popped exactly once – amortized O(1) per next() call.
#
# Time: O(1) amortized. Space: O(h)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _173_BSTIterator:
    def __init__(self, root):
        self._stack = []
        self._push_left(root)
    def _push_left(self, node):
        while node:
            self._stack.append(node)
            node = node.left
    def next(self):
        node = self._stack.pop()
        self._push_left(node.right)
        return node.val
    def hasNext(self):
        return bool(self._stack)
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# BST=[7,3,15,null,null,9,20]: init pushes [7,3] (left spine). stack=[7,3].
# next(): pop 3, push 3.right=None → nothing. return 3. stack=[7].
# next(): pop 7, push left spine of 15 → [15,9]. return 7. stack=[15,9].
# next(): pop 9, push 9.right=None. return 9. stack=[15].
```

```
# hasNext()→True ✓. next(): pop 15, push [20]. return 15. next(): pop 20. return 20
✓
#
# "No bugs. _push_left handles None safely with the 'while node' guard."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "next(): amortized O(1) – each node pushed/popped once across all calls.
# hasNext(): O(1). Space: O(h) – left spine depth.
# Follow-up: reverse inorder iterator (descending) – push right spines instead.
# Follow-up: peek() without advancing – call next() then push the value back as a
synthetic node."
```

Community Solutions (Python)

• WalkCC

```
class BSTIterator:
    def __init__(self, root: TreeNode | None):
        self.i = 0
        self.vals = []
        self._inorder(root)

    def next(self) -> int:
        self.i += 1
        return self.vals[self.i - 1]

    def hasNext(self) -> bool:
        return self.i < len(self.vals)

    def _inorder(self, root: TreeNode | None) -> None:
        if not root:
            return
        self._inorder(root.left)
        self.vals.append(root.val)
        self._inorder(root.right)
```



```

class BSTIterator:
    def __init__(self, root: TreeNode | None):
        self.stack = []
        self._pushLeftsUntilNull(root)

    def next(self) -> int:
        root = self.stack.pop()
        self._pushLeftsUntilNull(root.right)
        return root.val

    def hasNext(self) -> bool:
        return self.stack

    def _pushLeftsUntilNull(self, root: TreeNode | None) -> None:
        while root:
            self.stack.append(root)
            root = root.left

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class BSTIterator:
    def __init__(self, root: TreeNode):
        def inorder(root):
            if root:
                inorder(root.left)
                self.vals.append(root.val)
                inorder(root.right)

        self.cur = 0
        self.vals = []
        inorder(root)

    def next(self) -> int:
        res = self.vals[self.cur]

```

```

        self.cur += 1
        return res

    def hasNext(self) -> bool:
        return self.cur < len(self.vals)

```

```

# Your BSTIterator object will be instantiated and called as such:
# obj = BSTIterator(root)
# param_1 = obj.next()

```

```

# param_2 = obj.hasNext()

```

• SimplyLeet

```

# Definition for a binary tree node.
# class TreeNode:
# def __init__(self, val=0, left=None, right=None):
# self.val = val
# self.left = left
# self.right = right

class BSTIterator:
    def __init__(self, root):
        # Initialize an empty stack to simulate in-order traversal

```

```

        self.stack = []
        # Push all left nodes starting from the root
        self._push_left_branch(root)

    def _push_left_branch(self, node):
        # Push node and all its left children to the stack
        while node:
            self.stack.append(node)
            node = node.left

```

```

def next(self):
    # Pop the top node which represents the next smallest element
    node = self.stack.pop()
    # If the node has a right child, push its left branch onto the stack
    if node.right:
        self._push_left_branch(node.right)
    # Return the node's value
    return node.val

def hasNext(self):

    # The iterator has a next element if the stack is not empty
    return len(self.stack) > 0

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class BSTIterator:
    def __init__(self, root: TreeNode):
        self.stack = []
        self._leftmost_inorder(root)

    def _leftmost_inorder(self, node):
        while node:
            self.stack.append(node)
            node = node.left

    def next(self) -> int:
        cur = self.stack.pop()
        node = cur.right
        self._leftmost_inorder(node)

```

```

        return cur.val

    def hasNext(self) -> bool:
        return len(self.stack) > 0

```

```

# Your BSTIterator object will be instantiated and called as such:
# obj = BSTIterator(root)
# param_1 = obj.next()
# param_2 = obj.hasNext()

```

```
#####
```

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class BSTIterator:
    def __init__(self, root: TreeNode):

```

```

        self.stack = []
        while root:
            self.stack.append(root)
            root = root.left

    def next(self) -> int:
        cur = self.stack.pop()
        node = cur.right
        while node: # deplicated while block
            self.stack.append(node)

```

```

        node = node.left
        return cur.val

    def hasNext(self) -> bool:
        return len(self.stack) > 0

```

```

# Your BSTIterator object will be instantiated and called as such:
# obj = BSTIterator(root)
# param_1 = obj.next()

```

```
# param_2 = obj.hasNext()
```

◆ Quick Review

Key Insights

- Use an iterative in-order traversal strategy with a stack.
- In-order traversal for BST yields sorted order.
- Pre-load the stack with all left descendants from the root.
- After returning a node, process its right subtree by pushing its left branch.
- This method maintains an average time complexity of $O(1)$ per operation and uses $O(h)$ memory, where h is the tree height.

Space & Time

Time Complexity: $O(1)$ on average per operation (amortized)

Space Complexity: $O(h)$, where h is the height of the tree

Solution

We use a stack to simulate an in-order traversal. The process involves initially

pushing all left-child nodes from the root into the stack. For each call to next(), we pop the top element (the current smallest node) and then push all the left descendents of the node's right child. This ensures that the next smallest element is always at the top of the stack. The hasNext() function then simply checks if the stack is non-empty.

Stack

□ #227 Basic Calculator II

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · String · Stack
Lists: Grind 169

Problem

Given a string s which represents an expression, evaluate this expression and return its value. The integer division should truncate toward zero.

You may assume that the given expression is always valid. All intermediate results will be in the range of $[-2^{31}, 2^{31} - 1]$.

Note: You are not allowed to use any built-in function which evaluates strings as mathematical expressions, such as `eval()`.

Example 1:

Input: $s = "3+2*2"$
Output: 7

Example 2:

Input: s = " 3/2 "
Output: 1

Example 3:

Input: s = " 3+5 / 2 "
Output: 5

Constraints:

- $1 \leq s.length \leq 3 * 10^5$
- s consists of integers and operators ('+', '-', '*', '/') separated by some number of spaces.
- s represents a valid expression.
- All the integers in the expression are non-negative integers in the range $[0, 2^{31} - 1]$.
- The answer is guaranteed to fit in a 32-bit integer.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Evaluate a string with +, -, *, / (no parentheses), non-negative integers, possible spaces.

Division truncates toward zero. No built-in eval. Right?"

#

"A few quick questions:

Can the expression start with a negative number? No – all integers are non-negative.

Spaces can appear anywhere between tokens?"

#

"Let me walk: '3+2*2' → 2*2=4 first, then 3+4=7 ✓. ' 3/2 ' → 1 ✓. '3+5/2' → 5 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Two-pass: first resolve all * and /; then resolve + and -.

O(n) time, O(n) space for token lists.

Should I go ahead and optimize?"

```
class _227_BasicCalculatorII_Brute:
    def calculate(self, s):
        tokens = s.split()
        i = 0; toks = list(s.replace(' ', ''))
        nums = []; i = 0
        while i < len(toks):
            if toks[i].isdigit():
                j = i
                while j < len(toks) and toks[j].isdigit(): j += 1
                nums.append(int(''.join(toks[i:j]))); i = j
            else: nums.append(toks[i]); i += 1
        stack = [nums[0]]; i = 1
        while i < len(nums):
            op = nums[i]; num = nums[i+1]; i += 2
            if op == '*': stack.append(stack.pop() * num)
            elif op == '/': stack.append(int(stack.pop() / num))
            elif op == '+': stack.append(num)
            else: stack.append(-num)
        return sum(stack)
```

PHASE 3 — OPTIMIZE

"The bottleneck is the two-pass tokenisation."

Single-pass stack: track the last operator (sign).

On + or -: push the current number with its sign.

On * or /: pop the top, compute immediately with the current number, push result.

Final answer: sum the stack.

#

Time: O(n). Space: O(n) for the stack."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _227_BasicCalculatorII:
    def calculate(self, s):
        stack, num, sign = [], 0, '+'
        for i, ch in enumerate(s):
            if ch.isdigit():
                num = num * 10 + int(ch)
            if (ch in '+-*/') or i == len(s) - 1:
                if sign == '+': stack.append(num)
                elif sign == '-': stack.append(-num)
                elif sign == '*': stack.append(stack.pop() * num)
                else: stack.append(int(stack.pop() / num))
                sign, num = ch, 0
        return sum(stack)
```

PHASE 5 — TEST & DEBUG**# "Let me trace through a few cases."**

#

'3+2*2':

'3': num=3. '+' : sign was '+' → push 3→[3]. sign='+', num=0.

'2': num=2. '*' : sign='+' → push 2→[3,2]. sign='*', num=0.

'2'(last): num=2. i=end: sign='*' → pop 2*2=4 → [3,4]. sum=7 ✓

#

' 3/2 ': num=3. '/' : push 3→[3]. num=2 (last): int(3/2)=1→[1]. sum=1 ✓

#

"No bugs. Spaces are skipped because only digits and operators trigger action."**# PHASE 6 — COMPLEXITY & FOLLOW-UP****# "Time O(n): one pass. Space O(n): stack holds at most n/2 numbers."**

Key: * and / are computed immediately (higher precedence); + and - defer to the final sum.

Follow-up: Basic Calculator I (#224) – adds parentheses; handle with a recursion/stack.

Follow-up: Basic Calculator III (#772) – all four ops plus parentheses."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def calculate(self, s: str) -> int:
        ans = 0
        prevNum = 0
        currNum = 0
        op = '+'

        for i, c in enumerate(s):
            if c.isdigit():
                currNum = currNum * 10 + int(c)

            if not c.isdigit() and c != ' ' or i == len(s) - 1:
                if op == '+' or op == '-':
                    ans += prevNum
                    prevNum = currNum if op == '+' else -currNum
                elif op == '*':
                    prevNum = prevNum * currNum
                elif op == '/':
                    if prevNum < 0:
                        prevNum = math.ceil(prevNum / currNum)
                    else:

```

```

        prevNum = prevNum // currNum
    op = c
    currNum = 0

    return ans + prevNum

```

• LeetDoocs

```

class Solution:
    def calculate(self, s: str) -> int:
        v, n = 0, len(s)
        sign = '+'
        stk = []
        for i, c in enumerate(s):
            if c.isdigit():
                v = v * 10 + int(c)
            if i == n - 1 or c in '+-*/':
                match sign:
                    case '+':
                        stk.append(v)
                    case '-':
                        stk.append(-v)
                    case '*':
                        stk.append(stk.pop() * v)
                    case '/':
                        stk.append(int(stk.pop() / v))
                sign = c
                v = 0
        return sum(stk)

```

• SimplyLeet

```
# Python solution for Basic Calculator II
```

```
def calculate(s: str) -> int:
    # Initialize the stack, current number, and the last seen operator.
    stack = []
    current_number = 0
    previous_operator = '+'

    # Loop over each character in the string along with one extra iteration to
    handle the end.
    for i, char in enumerate(s):
```

```
        if char.isdigit():
            # Build the current number by shifting previous digits.
            current_number = current_number * 10 + int(char)
            # If the character is an operator (or we're at the end), process the
            previous operator.
```

```
        if char in "+-*/" or i == len(s) - 1:
            if previous_operator == '+':
                stack.append(current_number)
            elif previous_operator == '-':
                stack.append(-current_number)
            elif previous_operator == '*':
```

```
                last_number = stack.pop()
                stack.append(last_number * current_number)
            elif previous_operator == '/':
                last_number = stack.pop()
                # Python division truncates toward negative infinity by
                default, so we use int() to truncate toward zero.
```

```
                stack.append(int(last_number / current_number))
            # Reset for the next number and update the operator.
            previous_operator = char
            current_number = 0

    # Sum up all numbers in the stack which gives the final result.
```

```
    return sum(stack)
```

```
# Example usage:
```

```
print(calculate("3+2*2")) # Output: 7
print(calculate(" 3/2 ")) # Output: 1
print(calculate(" 3+5 / 2 ")) # Output: 5
```

• LC.ca

```
class Solution:
    def calculate(self, s: str) -> int:
        v, n = 0, len(s)
        sign = '+'
        stk = []
        for i, c in enumerate(s):
            if c.isdigit():
                v = v * 10 + int(c)
            if i == n - 1 or c in '+-*/':
                if sign == '+':
                    stk.append(v)
                # for "10-2*5": when '-' encountered, var 'sign' is still '+'
                # so '10' will be pushed to stk before setting sign to '-'
                elif sign == '-':
                    stk.append(-v)
                elif sign == '*':
                    stk.append(stk.pop() * v)
                elif sign == '/':
                    stk.append(int(stk.pop() / v))
                else:
                    print("operator not supported")

                sign = c # reset inside 'if'
                v = 0
        return sum(stk)
```

◆ Quick Review

Key Insights

- The expression can be processed in one pass while using a stack to handle operator precedence.

- When encountering '+' or '-' operators, the current number is directly added or subtracted (as a positive or negative value).
- For '*' and '/' operators, compute immediately with the last number from the stack and update it.
- Handle spaces and multi-digit numbers by building the number while iterating over the string.
- Division truncation toward zero must be carefully applied, which aligns with typical integer division in most languages.

Space & Time

Time Complexity: $O(n)$ where n is the length of the expression, since the algorithm processes each character once.

Space Complexity: $O(n)$ in the worst-case scenario when all numbers are stored in the stack.

Solution

We use a stack-based approach. Iterate over each character of the string while maintaining a variable for the current number and a previous operator (initialized to '+'). When an

operator or the end of the string is reached, perform an action based on the previous operator: For '+' add the current number to the stack. For '-' add the negative of the current number. For '*' and '/' pop the last number from the stack, compute the result by performing multiplication or division, and then push the result back to the stack. After processing the entire string, sum the elements in the stack to get the final result. This approach ensures that multiplication and division are handled before addition and subtraction.

Stack

□ ★ #255 Verify Preorder Sequence in Binary Search Tree ★

MED

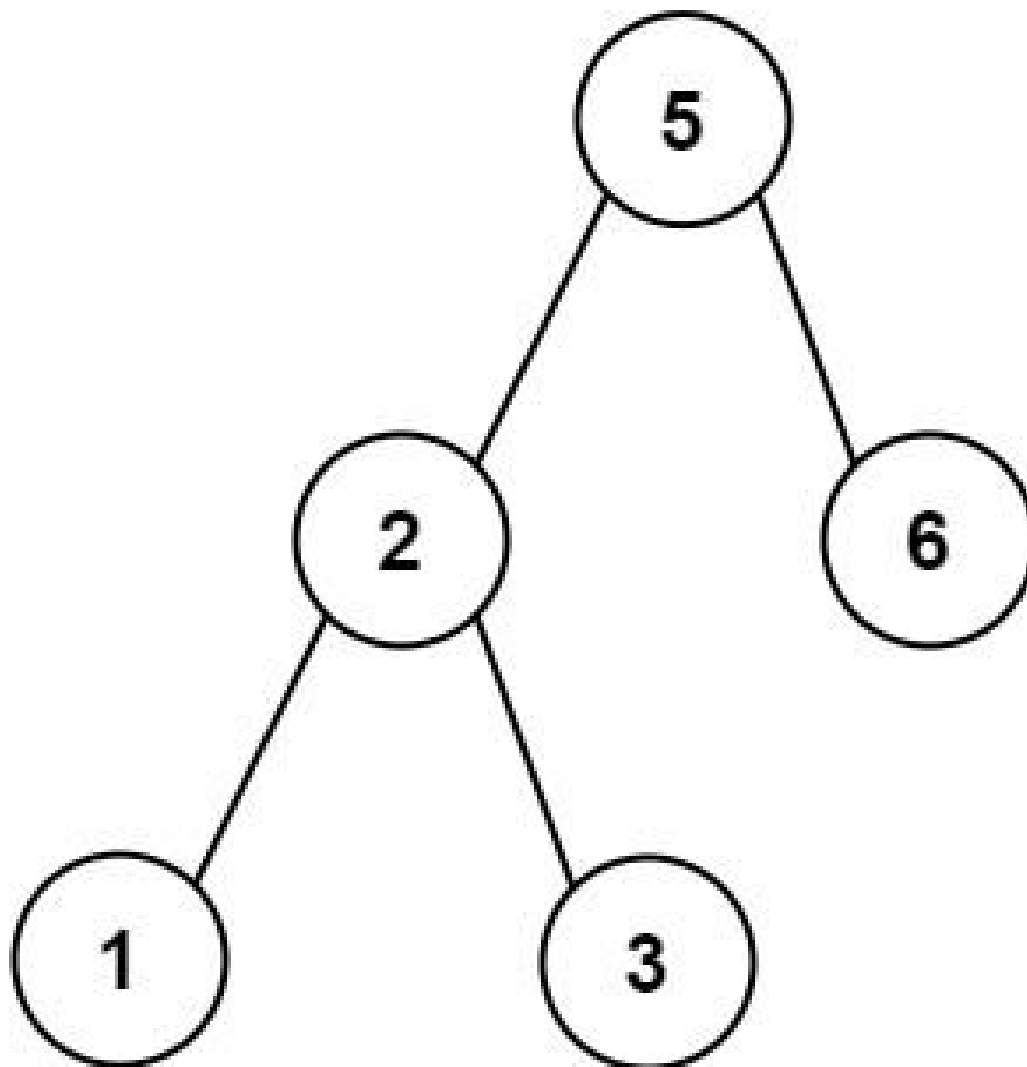
[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Tree · BST · Stack · Monotonic Stack
Lists: ★ Premium | Premium 98

Problem

Given an array of unique integers preorder, return true if it is the correct preorder traversal sequence of a binary search tree.

Example 1:



Input: preorder = [5,2,1,3,6]

Output: true

Example 2:

Input: preorder = [5,2,6,1,3]

Output: false

Constraints:

- $1 \leq \text{preorder.length} \leq 104$
- $1 \leq \text{preorder}[i] \leq 104$
- All the elements of preorder are unique.

Follow up: Could you do it using only constant space complexity?

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given an array of unique integers, determine if it could be a valid preorder traversal sequence of a binary search tree. Right?"

#

"A few quick questions:

Preorder: root, then left subtree (all smaller), then right subtree (all larger).
Once we enter the right subtree, all subsequent nodes must be larger than the root?"

#

"Let me walk: [5,2,1,3,6]: root=5, left=[2,1,3], right=[6] – valid ✓

[5,2,6,1,3]: after right subtree starts (6>5), seeing 1<5 is invalid → false ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Recursively validate: given a range [min_val, max_val], the current root must be in range.

Find where the right subtree starts (first value > root), validate both halves.

$O(n^2)$ worst case (skewed tree) – but correct.

Should I go ahead and optimize?"

```
class _255_VerifyPreorderBST_Brute:
```

```
    def verifyPreorder(self, preorder):
```

```
        def validate(lo, hi, i):
```

```
            if i >= len(preorder): return i
```

```
            root = preorder[i]
```

```
            if not (lo < root < hi): return i
```

```
            i = validate(lo, root, i + 1)
```

```
            i = validate(root, hi, i)
```

```
            return i
```

```
        return validate(float('-inf'), float('inf'), 0) == len(preorder)
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n^2)$ recursion on skewed trees.

Monotonic stack + lower bound tracking – $O(n)$ time, $O(n)$ space:

Maintain a stack of nodes on the left-spine (descending values).

When we see a value greater than stack top, we've entered a right subtree –

pop all smaller values, the last popped becomes the new lower bound.

Any subsequent value must exceed this lower bound.

```

#
# Time: O(n). Space: O(n) for the stack."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _255_VerifyPreorderBST:
    def verifyPreorder(self, preorder):
        stack = []
        lower_bound = float('-inf')
        for val in preorder:
            if val < lower_bound:
                return False
            while stack and stack[-1] < val:
                lower_bound = stack.pop()
            stack.append(val)
        return True

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# [5,2,1,3,6]:
# 5: stack=[5]. 2<5: stack=[5,2]. 1<2: stack=[5,2,1].
# 3>1: pop1(lb=1), pop2(lb=2). 3<5 stop. stack=[5,3]. 3>lb=2 ✓.
# 6>5: pop3(lb=3), pop5(lb=5). stack=[6]. 6>lb=5 ✓. → True ✓
#
# [5,2,6,1,3]:
# 5→[5]. 2→[5,2]. 6>2: pop2(lb=2),pop5(lb=5). stack=[6]. 6>lb=5 ✓.
# 1<lb=5 → False ✓
#
# "No bugs. The lower_bound tracks the minimum acceptable value after entering a
right subtree."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): each element pushed and popped at most once. Space O(n) stack.
# O(1) space variant: reuse the preorder array itself as the stack (destructive).
# Follow-up: reconstruct BST from preorder (#1008) – similar range-based recursion.
# Follow-up: verify inorder – just check it's strictly sorted."

```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def verifyPreorder(self, preorder: list[int]) -> bool:
        i = 0

        def dfs(min: int, max: int) -> None:
            nonlocal i
            if i == len(preorder):
                return
            if preorder[i] < min or preorder[i] > max:
                return

            val = preorder[i]
            i += 1
            dfs(min, val)
            dfs(val, max)

        dfs(-math.inf, math.inf)
        return i == len(preorder)

class Solution:
    def verifyPreorder(self, preorder: list[int]) -> bool:
        low = -math.inf
        stack = []

        for p in preorder:
            if p < low:
                return False
            while stack and stack[-1] < p:
                low = stack.pop()

            stack.append(p)

        return True
```

```

class Solution:
    def verifyPreorder(self, preorder: list[int]) -> bool:
        low = -math.inf
        i = -1

        for p in preorder:
            if p < low:
                return False
            while i >= 0 and preorder[i] < p:
                low = preorder[i]

            i -= 1
            i += 1
            preorder[i] = p

        return True

```

• LeetDoocs

```

class Solution:
    def verifyPreorder(self, preorder: List[int]) -> bool:
        stk = []
        last = -inf
        for x in preorder:
            if x < last:
                return False
            while stk and stk[-1] < x:
                last = stk.pop()
            stk.append(x)

        return True

```

• SimplyLeet

```
# Python solution using a monotonic stack with detailed comments
def verifyPreorder(preorder):
    lower_bound = float('-inf') # Set initial lower bound to negative infinity
    stack = [] # Use a stack to track the BST construction
    for value in preorder:
        # If the current value is less than lower_bound, the BST property is
        # violated
        if value < lower_bound:
            return False
        # If current value is greater than the top of stack, pop elements and
        # update lower_bound
        while stack and value > stack[-1]:
            lower_bound = stack.pop() # Update lower_bound as we move to the
            # right subtree
        # Push the current value onto the stack
        stack.append(value)
    return True

# Example usage:
# print(verifyPreorder([5,2,1,3,6])) # Expected output: True
```

• LC.ca

```
class Solution:
    def verifyPreorder(self, preorder: List[int]) -> bool:
        stk = []
        last = -inf
        for x in preorder:
            if x < last:
                return False
            while stk and stk[-1] < x:
                last = stk.pop()
            stk.append(x)
```



```
    return True
```

```
class Solution: # o(N) but modifying input list
    def verifyPreorder(self, preorder: List[int]) -> bool:
        low = -math.inf
        i = -1

        for p in preorder:
            if p < low:
```

```
                return False
            while i >= 0 and preorder[i] < p:
                low = preorder[i]
                i -= 1
            i += 1
            preorder[i] = p
```

```
    return True
```

```
#####
```

```
class Solution(object):

    # O(n) sapce complexity, **stack**
    def verifyPreorder(self, preorder):
        """
        :type preorder: List[int]
        :rtype: bool
        """
        if len(preorder) <= 1:
```

```
            return True
        stack, lastElem = [preorder[0]], None
        for i in range(1, len(preorder)):
            if lastElem > preorder[i]:
                return False
            while len(stack) > 0 and preorder[i] > stack[-1]:
                lastElem = stack.pop()
            stack.append(preorder[i])
```

```
    return True
```

◆ Quick Review

Key Insights

- A BST has the property that every node's left subtree contains values less than the node, and every right subtree contains values greater.
- Preorder traversal processes nodes as: root, left subtree, then right subtree.
- Using a monotonic stack, we can simulate constructing the BST while tracking a lower bound for allowable node values.
- When moving to a right child, update the lower bound to ensure subsequent nodes are valid.

Space & Time

Time Complexity: $O(n)$ – Each element is processed once.

Space Complexity: $O(n)$ – In the worst-case an explicit stack is used; can be optimized to $O(1)$ by reusing the input array.

Solution

We use a monotonic stack to simulate the BST construction from the preorder sequence. The process is: Initialize a variable `lower_bound` to $-\infty$. Iterate through each value in the preorder array: If a value is less than `lower_bound`, it violates BST properties and the sequence is invalid. While the stack is not empty and the current value is greater than the element at the top of the stack, pop from the stack and update `lower_bound` to the popped value. This simulates moving to the right subtree. Push the current value onto the stack. If the sequence is processed without violations, return true. This approach efficiently ensures that each value adheres to BST requirements during preorder traversal.

Stack

□ #394 Decode String

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack · Recursion
Lists: Grind 169

Problem

Given an encoded string, return its decoded string.

The encoding rule is: $k[\text{encoded_string}]$, where the `encoded_string` inside the square brackets is being repeated exactly k times. Note that k is guaranteed to be a positive integer.

You may assume that the input string is always valid; there are no extra white spaces, square brackets are well-formed, etc. Furthermore, you may assume that the original data does not contain any digits and that digits are only for those repeat numbers, k . For example, there will not be input like `3a` or `2[4]`.

The test cases are generated so that the length of the output will never exceed 105.

Example 1:

```
Input: s = "3[a]2[bc]"
Output: "aaabcbc"
```

Example 2:

```
Input: s = "3[a2[c]]"
Output: "accaccacc"
```

Example 3:

```
Input: s = "2[abc]3[cd]ef"
Output: "abcbccdcdef"
```

Constraints:

- $1 \leq s.length \leq 30$
- s consists of lowercase English letters, digits, and square brackets '['].
- s is guaranteed to be a valid input.
- All the integers in s are in the range [1, 300].

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Decode 'k[encoded_string]' – repeat encoded_string k times.

Nesting is allowed. k is always a positive integer. Input is always valid. Right?"

#

"A few quick questions:

Can k be multi-digit? Yes – e.g. '12[a]' → 'aaa...a'(12 times).

Can encoded_string be empty? Yes – e.g. '3[]' → ''."

#

"Let me walk: '3[a]2[bc]' → 'aaabcbc' ✓. '3[a2[c]]' → 'accaccacc' ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

```
# Repeatedly find the innermost bracket pair, expand it k times, replace in string.
# O(n * max_k^depth) – for deeply nested large multipliers this is huge.
# Should I go ahead and optimize?"
```

```
class _394_DecodeString_Brute:
    def decodeString(self, s):
        import re
        while '[' in s:
            s = re.sub(r'(\d+)\s*([a-z]*)', lambda m: int(m.group(1)) *
m.group(2), s)
        return s
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the repeated string scans.
# Stack-based single pass:
# On digit: build the multiplier k (can be multi-digit).
# On '[': push (current_string, k) onto the stack, reset both to empty/0.
# On ']': pop (prev_string, k), set current = prev_string + k * current.
# On letter: append to current string.
#
# Time: O(n * max_k) for building the expanded output. Space: O(n) stack depth."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _394_DecodeString:
    def decodeString(self, s):
        stack = []
        current = ''
        k = 0
        for ch in s:
            if ch.isdigit():
                k = k * 10 + int(ch)
            elif ch == '[':
                stack.append((current, k))
                current, k = '', 0
            elif ch == ']':
                prev, repeat = stack.pop()
                current = prev + repeat * current
            else:
                current += ch
        return current
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# '3[a2[c]]':
# '3'→k=3. '['→push('',3),current='',k=0. 'a'→current='a'.
# '2'→k=2. '['→push('a',2),current='',k=0. 'c'→current='c'.
# ']'→pop('a',2): current='a'+2*'c'='acc'.
# ']'→pop('',3): current='' + 3*'acc'='accaccacc' ✓
#
```

"No bugs. Multi-digit k handled by $k = k*10 + \text{digit}$ before each '['."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n * \text{max_k})$: building the output string can be large.

Space $O(d)$: d = nesting depth of the stack.

Follow-up: Encode String with Shortest Length (#471) – reverse direction, DP + KMP.

Follow-up: if nesting is guaranteed absent, a simple split-on-bracket regex suffices."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def decodeString(self, s: str) -> str:
        stack = [] # (prevStr, repeatCount)
        currStr = ''
        currNum = 0

        for c in s:
            if c.isdigit():
                currNum = currNum * 10 + int(c)
            else:
                if c == '[':
                    stack.append((currStr, currNum))
                    currStr = ''
                    currNum = 0
                elif c == ']':
                    prevStr, num = stack.pop()
                    currStr = prevStr + num * currStr
                else:
                    currStr += c

        return currStr
```



```

class Solution:
    def decodeString(self, s: str) -> str:
        ans = ''

        while self.i < len(s) and s[self.i] != ']':
            if s[self.i].isdigit():
                k = 0
                while self.i < len(s) and s[self.i].isdigit():
                    k = k * 10 + int(s[self.i])
                    self.i += 1

                self.i += 1 # '['
                decodedString = self.decodeString(s)
                self.i += 1 # ']'
                ans += k * decodedString
            else:
                ans += s[self.i]
                self.i += 1

        return ans

i = 0

```

• LeetDoocs

```

class Solution:
    def decodeString(self, s: str) -> str:
        s1, s2 = [], []
        num, res = 0, ''
        for c in s:
            if c.isdigit():
                num = num * 10 + int(c)
            elif c == '[':
                s1.append(num)
                s2.append(res)
                num, res = 0, ''
            elif c == ']':
                res = s2.pop() + res * s1.pop()
            else:
                res += c
        return res

```

• SimplyLeet

```
def decodeString(s: str) -> str:
    # index pointer as a list so it can be mutable within recursion
    def helper(i: int) -> (str, int):
        decoded = ""
        while i < len(s):
            if s[i].isdigit():
                # build the number (it might be more than one digit)
                k = 0
                while i < len(s) and s[i].isdigit():
                    k = k * 10 + int(s[i])

                i += 1
                # skip the '[' character
                i += 1 # skip '['
                # recursively solve the substring inside the brackets
                sub, i = helper(i)
                # append the substring repeated k times
                decoded += sub * k
            elif s[i] == ']':
                # returning decoded string and new index after ']'
                return decoded, i + 1

        else:
            # accumulate alphabetic characters directly
            decoded += s[i]
            i += 1
        return decoded, i

    result, _ = helper(0)
    return result

# Example usage

#print(decodeString("3[a2[c]]")) # Output: "accaccacc"
```

• LC.ca

```
class Solution:
    def decodeString(self, s: str) -> str:
        num_stk, str_stk = [], []
        num, res = 0, ''
        for c in s:
            if c.isdigit():
                num = num * 10 + int(c)
            elif c == '[':
                num_stk.append(num)
                str_stk.append(res) # initial, res being pushed is empty str ''
                num, res = 0, ''
            elif c == ']':
                res = str_stk.pop() + res * num_stk.pop()
            else:
                res += c
        return res
```

◆ Quick Review

Key Insights

- Use a stack or recursion to manage nested encoded patterns.
- Process the string character by character, accumulating digits for the repeat count.
- When encountering a '[', start a new context (recursively or via a new stack frame) for the encoded substring.
- When encountering a ']', finish the current context and repeat the accumulated substring as needed.

- Careful handling of multi-digit numbers is essential.

Space & Time

Time Complexity: $O(n)$, where n is the length of the string.

Space Complexity: $O(n)$, for the recursion stack or the auxiliary stack used.

Solution

We can solve this problem using a recursive approach. When we see a digit, we determine the full repeat count. Upon encountering a '[', we recursively read the encoded substring until the corresponding ']'. Each recursive call returns the decoded substring until it hits a ']', which is then repeated based on the previously computed count. Alternatively, an iterative solution using a stack is also feasible. As we parse the string, push the current string and number onto the stack when encountering '[', then pop and combine when ']' is reached. This approach correctly handles nested patterns.

Stack

□ ★ #439 Ternary Expression Parser ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Stack · Recursion

Lists: ★ Premium | Premium 98

Problem

Given a string expression representing arbitrarily nested ternary expressions, evaluate the expression, and return the result of it.

You can always assume that the given expression is valid and only contains digits, '?', ':', 'T', and 'F' where 'T' is true and 'F' is false. All the numbers in the expression are one-digit numbers (i.e., in the range [0, 9]).

The conditional expressions group right-to-left (as usual in most languages), and the result of the expression will always evaluate to either a digit, 'T' or 'F'.

Example 1:

Input: expression = "T?2:3"

Output: "2"

Explanation: If true, then result is 2; otherwise result is 3.

Example 2:

Input: expression = "F?1:T?4:5"

Output: "4"

Explanation: The conditional expressions group right-to-left. Using parenthesis, it is read/evaluated as:

"(F ? 1 : (T ? 4 : 5))" --> "(F ? 1 : 4)" --> "4"

or "(F ? 1 : (T ? 4 : 5))" --> "(T ? 4 : 5)" --> "4"

Example 3:

Input: expression = "T?T?F:5:3"

Output: "F"

Explanation: The conditional expressions group right-to-left. Using parenthesis, it is read/evaluated as:

"(T ? (T ? F : 5) : 3)" --> "(T ? F : 3)" --> "F"

"(T ? (T ? F : 5) : 3)" --> "(T ? F : 5)" --> "F"

Constraints:

- $5 \leq \text{expression.length} \leq 104$
- expression consists of digits, 'T', 'F', '?', and ':'.
- It is guaranteed that expression is a valid ternary expression and that each number is a one-digit number.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Evaluate a ternary expression string: 'T?a:b' → a if T, else b.

Can be arbitrarily nested. All conditions are T or F; values are digits, T, or F.

```

# Ternary is right-associative. Right?"
#
# "A few quick questions:
# Input is always a valid ternary? Yes. Only single-char values? Yes (digits 0-9, T,
# F)."
#
# "Let me walk: 'T?2:3' → '2' ✓. 'F?1:T?4:5' → '4' ✓. 'T?T?F:5:3' → 'F' ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Repeatedly find the innermost 'X?a:b' (single-char a and b), evaluate, replace.
#  $O(n^2)$  – each pass may do one replacement.
# Should I go ahead and optimize?"
class _439_TernaryExpressionParser_Brute:
    def parseTernary(self, expression):
        import re
        s = expression
        while len(s) > 1:
            s = re.sub(r'[TF]\?[0-9TF]:[0-9TF]', lambda m: m.group()[2] if
m.group()[0]=='T' else m.group()[4], s)
        return s

# PHASE 3 — OPTIMIZE
# "The bottleneck is the repeated string scans.
# Scan RIGHT TO LEFT with a stack –  $O(n)$ :
# When we see '?': pop two values (true-branch then false-branch) and the condition
# character.
# Push the resolved value.
# Right-to-left processes the right-associativity naturally.
#
# Time:  $O(n)$ . Space:  $O(n)$  stack."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _439_TernaryExpressionParser:
    def parseTernary(self, expression):
        stack = []
        for ch in reversed(expression):
            if stack and stack[-1] == '?':
                stack.pop()
                true_val = stack.pop()
                stack.pop()
                false_val = stack.pop()
                stack.append(true_val if ch == 'T' else false_val)
            else:
                stack.append(ch)
        return stack[0]

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#

```



```
# 'T?2:3' reversed='3:2?T':
# push '3'. ':' → push ':'. '2' → push '2'. '?' → pop '?', true='2', pop ':',
false='3'.
# 'T' → push '2'. stack=['2']. return '2' ✓
#
# 'T?T?F:5:3' (right-associ is T?(T?F:5):3):
# Process right-to-left: resolves inner T?F:5 first → F, then outer T?F:3 → F ✓
#
# "No bugs. Popping ':' after the false_val keeps the separator from polluting the
stack."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: one right-to-left pass. Space $O(n)$ stack depth.

The right-to-left scan makes right-associativity automatic – no parenthesis tracking needed.

Follow-up: if operands can be multi-character, tokenize first then apply the same logic.

Follow-up: support general conditional expressions ($==$, $!=$) – extend the condition parsing."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def parseTernary(self, expression: str) -> str:
        c = expression[self.i]

        if self.i + 1 == len(expression) or expression[self.i + 1] == ':':
            self.i += 2
            return str(c)

        self.i += 2
        first = self.parseTernary(expression)

        second = self.parseTernary(expression)

        return first if c == 'T' else second

i = 0
```

• LeetCode

```

class Solution:
    def parseTernary(self, expression: str) -> str:
        stk = []
        cond = False
        for c in expression[::-1]:
            if c == ':':
                continue
            if c == '?':
                cond = True
            else:
                if cond:
                    if c == 'T':
                        x = stk.pop()
                        stk.pop()
                        stk.append(x)
                    else:
                        stk.pop()
                        cond = False
                else:
                    stk.append(c)

        return stk[0]

```

• SimplyLeet

```

def parseTernary(expression: str) -> str:
    # Initialize stack to hold characters/results during evaluation
    stack = []
    # Iterate backwards over the expression
    i = len(expression) - 1
    while i >= 0:
        if expression[i] == '?':
            # Pop the true and false parts from the stack
            true_expr = stack.pop()
            false_expr = stack.pop()

```

```

        i -= 1 # Move to the condition character
        # Choose based on the condition
        if expression[i] == 'T':
            stack.append(true_expr)
        else:
            stack.append(false_expr)
    elif expression[i] == ':':
        # Skip the colon as it is only a separator
        pass
    else:

```

```

        # Push digits, 'T', and 'F' onto stack as is
        stack.append(expression[i])
    i -= 1 # Move to the next character
    return stack[-1]

```

Example usage:

```
print(parseTernary("T?2:3")) # Expected output "2"
```

• LC.ca

```

class Solution:
    def parseTernary(self, expression: str) -> str:
        stk = []
        cond = False
        for c in expression[::-1]:
            if c == ':':
                continue
            if c == '?':
                cond = True
            else:

```

```

                if cond:
                    if c == 'T':
                        x = stk.pop()
                        stk.pop()
                        stk.append(x)
                    else:
                        stk.pop()
                cond = False
            else:
                stk.append(c)

```

```
return stk[0]
```

◆ Quick Review

Key Insights

- The ternary operator groups right-to-left. This property can be exploited by processing the string from the end to the beginning.
- Using a stack allows us to "collapse" nested ternary expressions as soon as their three components (condition, true branch, false branch) are available.
- An iterative approach is efficient and avoids building an explicit syntax tree for the expression.
- The condition is not stored on the stack; instead, it is encountered immediately preceding the '?' operator when traversing backwards.

Space & Time

Time Complexity: $O(n)$, where n is the length of the expression, as we process each character exactly once.

Space Complexity: $O(n)$, due to the use of a stack for storing characters during evaluation.

Solution

The solution uses an iterative approach with a stack. We traverse the expression string from right to left. For every character: If it is a digit, 'T', or 'F' (and not a ':' or part of a pending operator), we push it onto the stack. When we encounter a '?' character, we know that the next available items on the stack represent the true and false results of the ternary expression. We then skip the ':' separator that comes between them. We then use the condition (the character immediately preceding the '?') to choose which result to push back on the stack. This method “collapses” each ternary expression into a single result, and by the time we finish traversing the string, the stack contains the final evaluated result.

Stack

□ ★ #484 Find Permutation ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Stack · Greedy
Lists: ★ Premium | Premium 98

Problem

A permutation perm of n integers of all the integers in the range [1, n] can be represented as a string s of length n - 1 where:

- $s[i] == 'I'$ if $perm[i] < perm[i + 1]$, and
- $s[i] == 'D'$ if $perm[i] > perm[i + 1]$.

Given a string s, reconstruct the lexicographically smallest permutation perm and return it.

Example 1:

Input: s = "I"

Output: [1,2]

Explanation: [1,2] is the only legal permutation that can be represented by s, where the number 1 and 2 construct an increasing relationship.

Example 2:

Input: s = "DI"

Output: [2,1,3]

Explanation: Both [2,1,3] and [3,1,2] can be represented as "DI", but since we want to find the smallest lexicographical permutation, you should return [2,1,3]

Constraints:

- $1 \leq s.length \leq 105$
- s[i] is either 'I' or 'D'.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given a string of 'I' (increasing) and 'D' (decreasing) of length n-1,

reconstruct the lexicographically smallest permutation of [1..n] satisfying the pattern. Right?"

#

"A few quick questions:

'I': nums[i] < nums[i+1]. 'D': nums[i] > nums[i+1]. Return any lex-smallest? Yes."

#

"Let me walk: s='I' → n=2 → [1,2] ✓. s='DI' → [2,1,3] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Generate all permutations of [1..n] in lexicographic order, return first satisfying the pattern.

$O(n! \cdot n)$ – completely infeasible.

Should I go ahead and optimize?"

```
class _484_FindPermutation_Brute:
```

```
    def findPermutation(self, s):
```

```
        from itertools import permutations
```

```
        n = len(s) + 1
```

```
        for perm in permutations(range(1, n+1)):
```

```
            if all((perm[i]<perm[i+1])==(s[i]=='I') for i in range(n-1)):
```

```
                return list(perm)
```

PHASE 3 — OPTIMIZE

"The bottleneck is the exponential permutation search.

Greedy with a stack – $O(n)$ time:


```
# Push each number 1..n onto the stack. On every 'I' (or at the end), flush the
stack
# in reverse order into the result. This reversal creates the optimal decreasing run
# for any 'D' sequence while keeping the smallest numbers first.
#
# Time: O(n). Space: O(n) for the stack."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _484_FindPermutation:
    def findPermutation(self, s):
        result = []
        stack = []
        for i, ch in enumerate(s + 'I'):
            stack.append(i + 1)
            if ch == 'I':
                while stack:
                    result.append(stack.pop())
        return result
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
```

```
#
# s='DI' → s+'I'='DII':
# i=0, 'D': push 1. i=1, 'I': push 2. ch='I'→flush: result=[2,1].
# i=2, 'I'(sentinel): push 3. ch='I'→flush: result=[2,1,3]. → [2,1,3] ✓
#
# s='III':
# i=0: push1, 'I'→flush:[1]. i=1: push2, 'I'→flush:[1,2]. i=2:
push3, 'I'→flush:[1,2,3].
# sentinel: push4, 'I'→flush:[1,2,3,4] ✓
#
# s='DDI': push1(D). push2(D). push3(I)→flush:[3,2,1]. push4(I)→flush:[3,2,1,4] ✓
#
# "No bugs. The 'I' sentinel at the end ensures the last stack segment is always
flushed."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): each element pushed and popped once. Space O(n) stack.
# The stack reversal turns consecutive 'D' blocks into the smallest valid decreasing
run.
# Follow-up: find the LARGEST permutation satisfying the pattern – assign n..1,
reverse D blocks.
# Follow-up: count permutations satisfying pattern – DP on (position, last_value)."
```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def findPermutation(self, s: str) -> list[int]:
        ans = []
        stack = []

        for i, c in enumerate(s):
            stack.append(i + 1)
            if c == 'I':
                while stack: # Consume all decreasing
                    ans.append(stack.pop())

        stack.append(len(s) + 1)

        while stack:
            ans.append(stack.pop())

        return ans

```

```

class Solution:
    def findPermutation(self, s: str) -> list[int]:
        ans = [i for i in range(1, len(s) + 2)]

        # For each D* group (s[i..j]), reverse ans[i..j + 1].
        i = -1
        j = -1

        def getNextIndex(c: str, start: int) -> int:
            for i in range(start, len(s)):
                if s[i] == c:
                    return i
            return len(s)

        while True:
            i = getNextIndex('D', j + 1)
            if i == len(s):
                break
            j = getNextIndex('I', i + 1)
            ans[i:j + 1] = ans[i:j + 1][::-1]

        return ans

```

• LeetDoocs

```
class Solution:
    def findPermutation(self, s: str) -> List[int]:
        n = len(s)
        ans = list(range(1, n + 2))
        i = 0
        while i < n:
            j = i
            while j < n and s[j] == 'D':
                j += 1
            ans[i : j + 1] = ans[i : j + 1][::-1]

            i = max(i + 1, j)
        return ans
```

• SimplyLeet

```
# Function to find the lexicographically smallest permutation
def findPermutation(s):
    stack = [] # Use a stack to hold numbers during a sequence of 'D's
    result = [] # Result list for the final permutation
    # Iterate over 0 to len(s) (inclusive) to process all characters plus one
    # final number.
    for i in range(len(s) + 1):
        # Push i+1 since our numbers range from 1 to n
        stack.append(i + 1)
        # If we are at the end or the current character is 'I', flush the
        # stack.
        if i == len(s) or s[i] == 'I':
            while stack:
                result.append(stack.pop())
    return result

# Example usage:
print(findPermutation("DI"))
```

• LC.ca

```
class Solution:
    def findPermutation(self, s: str) -> List[int]:
        n = len(s)
        ans = list(range(1, n + 2))
        i = 0
        while i < n:
            j = i
            while j < n and s[j] == 'D':
                j += 1
            ans[i : j + 1] = ans[i : j + 1][::-1]

            i = max(i + 1, j)
        return ans
```

◆ Quick Review

Key Insights

- The answer is a permutation of numbers from 1 to n , where $n = \text{len}(s) + 1$.
- A common strategy is to use a stack to reverse the segments when a decrease ('D') is encountered.
- When an 'I' is encountered (or at the end), pop all numbers from the stack to output them in reverse order, ensuring the smallest lexicographical order.
- This greedy approach guarantees that as soon as a rising pattern is detected, we “flush” the pending decreases correctly.

Space & Time

Time Complexity: $O(n)$ – Each element is pushed and popped exactly once.

Space Complexity: $O(n)$ – For the stack and the output array.

Solution

We use a greedy approach with a stack. Iterate from 0 to n (where $n = \text{len}(s) + 1$): Push the current number $(i+1)$ onto the stack. If the current character is 'I' or we have reached the end of the string, pop all elements from the stack and append them to the result. By doing so, segments corresponding to series of 'D's are reversed, while 'I's trigger the flush, ensuring that the final permutation is the lexicographically smallest.

Stack

□ #735 Asteroid Collision

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Simulation
Lists: Grind 169

Problem

We are given an array `asteroids` of integers representing asteroids in a row. The indices of the asteroid in the array represent their relative position in space.

For each asteroid, the absolute value represents its size, and the sign represents its direction (positive meaning right, negative meaning left). Each asteroid moves at the same speed.

Find out the state of the asteroids after all collisions. If two asteroids meet, the smaller one will explode. If both are the same size, both will explode. Two asteroids moving in the same direction will never meet.

Example 1:

Input: asteroids = [5,10,-5]

Output: [5,10]

Explanation: The 10 and -5 collide resulting in 10. The 5 and 10 never collide.

Example 2:

Input: asteroids = [8,-8]

Output: []

Explanation: The 8 and -8 collide exploding each other.

Example 3:

Input: asteroids = [10,2,-5]

Output: [10]

Explanation: The 2 and -5 collide resulting in -5. The 10 and -5 collide resulting in 10.

Example 4:

Input: asteroids = [3,5,-6,2,-1,4]□□□□□□

Output: [-6,2,4]

Explanation: The asteroid -6 makes the asteroid 3 and 5 explode, and then continues going left. On the other side, the asteroid 2 makes the asteroid -1 explode and then continues going right, without reaching asteroid 4.

Constraints:

- $2 \leq \text{asteroids.length} \leq 104$
- $-1000 \leq \text{asteroids}[i] \leq 1000$
- $\text{asteroids}[i] \neq 0$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Positive = moving right, negative = moving left.

When a right-moving and left-moving asteroid collide, the smaller explodes;

if equal, both explode. Same-direction asteroids never collide. Right?"

#


```

# "A few quick questions:
# The absolute value gives size; the sign gives direction? Yes.
# Return the state of asteroids after all collisions."
#
# "Let me walk: [5,10,-5] → 10 and -5 collide → 10 survives. → [5,10] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Scan for any adjacent collision (positive followed by negative), resolve, repeat.
#  $O(n^2)$  worst case – lots of repeated scanning.
# Should I go ahead and optimize?"
class _735_AsteroidCollision_Brute:
    def asteroidCollision(self, asteroids):
        changed = True
        while changed:
            changed = False
            for i in range(len(asteroids) - 1):
                if asteroids[i] > 0 > asteroids[i+1]:
                    a, b = asteroids[i], asteroids[i+1]
                    if abs(a) == abs(b): asteroids[i:i+2] = []
                    elif abs(a) > abs(b): asteroids.pop(i+1)
                    else: asteroids.pop(i)
                    changed = True; break
            return asteroids

# PHASE 3 — OPTIMIZE
# "The bottleneck is the repeated scans – we process each collision multiple times.
# A stack handles this in a single pass:
# Push each asteroid. When the current asteroid is negative and the stack top is
# positive,
# a collision occurs. Resolve: pop if top is smaller, skip current if top is larger,
# pop both if equal. Continue until no collision or stack empty.
#
# Time:  $O(n)$  – each asteroid pushed and popped at most once.
# Space:  $O(n)$  for the stack."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _735_AsteroidCollision:
    def asteroidCollision(self, asteroids):
        stack = []
        for rock in asteroids:
            alive = True
            while alive and rock < 0 and stack and stack[-1] > 0:
                if stack[-1] < abs(rock):
                    stack.pop()
                elif stack[-1] == abs(rock):
                    stack.pop(); alive = False
                else:
                    alive = False
            if alive:

```

```
        stack.append(rock)
    return stack
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
```

```
#
```

```
# [5,10,-5]: push 5→[5]. push 10→[5,10]. -5: 10>5→alive=False. → [5,10] ✓
```

```
# [8,-8]: push 8→[8]. -8: 8==8→pop,alive=False. stack=[]. → [] ✓
```

```
# [10,2,-5]: push 10,2→[10,2]. -5: 2<5→pop. [10]. -5: 10>5→alive=False. → [10] ✓
```

```
#
```

```
# "No bugs. The while loop handles chain reactions naturally."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): each asteroid pushed and popped at most once across the whole loop.
```

```
# Space O(n): stack holds surviving asteroids.
```

```
# Follow-up: 2D asteroid collision (angle + velocity) – much more complex physics simulation.
```

```
# Follow-up: Count Collisions on a Road (#2211) – similar directional collision counting."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def asteroidCollision(self, asteroids: list[int]) -> list[int]:
        stack = []

        for a in asteroids:
            if a > 0:
                stack.append(a)
            else: # a < 0
                # Destroy the previous positive one(s).
                while stack and stack[-1] > 0 and stack[-1] < -a:

                    stack.pop()
                if not stack or stack[-1] < 0:
                    stack.append(a)
                elif stack[-1] == -a:
                    stack.pop() # Both asteroids explode.
                else: # stack[-1] > the current asteroid.
                    pass # Destroy the current asteroid, so do nothing.

        return stack
```

• LeetDoocs

```

class Solution:
    def asteroidCollision(self, asteroids: List[int]) -> List[int]:
        stk = []
        for x in asteroids:
            if x > 0:
                stk.append(x)
            else:
                while stk and stk[-1] > 0 and stk[-1] < -x:
                    stk.pop()
                if stk and stk[-1] == -x:
                    stk.pop()
                elif not stk or stk[-1] < 0:
                    stk.append(x)
        return stk

```

• SimplyLeet

```

# Function to simulate asteroid collisions
def asteroidCollision(asteroids):
    # Initialize an empty list as a stack to hold surviving asteroids
    stack = []
    # Iterate through each asteroid in the input list
    for ast in asteroids:
        # Flag to check if the current asteroid has exploded
        exploded = False
        # Check for potential collision while there is an asteroid in stack and
        # a collision condition holds (right-moving on stack and left-moving current)
        while stack and ast < 0 and stack[-1] > 0:
            # If the incoming left-moving asteroid is larger than the
            # right-moving one on stack, pop the top and continue checking
            if abs(ast) > abs(stack[-1]):
                stack.pop()
                continue # Continue the loop to check next potential collision
            # If they are equal, both explode: pop the stack and mark
            # explosion, break the loop
            elif abs(ast) == abs(stack[-1]):
                stack.pop()
            # Mark the incoming asteroid as exploded and break out of the loop
            exploded = True
            break

```

```

        # If the asteroid did not explode during collisions, push it onto the
        stack
        if not exploded:
            stack.append(ast)
    # Return the surviving asteroids
    return stack

# Example usage:
# print(asteroidCollision([5,10,-5])) # Output: [5,10]

```

• LC.ca

```

class Solution:
    def asteroidCollision(self, asteroids: List[int]) -> List[int]:
        stk = []
        for x in asteroids:
            if x > 0:
                stk.append(x)
            else:
                while stk and stk[-1] > 0 and stk[-1] < -x:
                    stk.pop()
                if stk and stk[-1] == -x:
                    stk.pop()
                elif not stk or stk[-1] < 0:
                    stk.append(x)
        return stk

```

◆ Quick Review

Key Insights

- Use a stack to keep track of the surviving asteroids.
- Only a collision happens when a right-moving asteroid (positive) is on the stack and a left-moving asteroid (negative) is

encountered.

- Compare the absolute values to decide if one or both asteroids explode.
- Continue processing collisions until no further collisions are possible with the current asteroid.

Space & Time

Time Complexity: $O(n)$ where n is the number of asteroids, since each asteroid is pushed and popped at most once.

Space Complexity: $O(n)$ in the worst case, when no collisions occur and all asteroids remain on the stack.

Solution

The solution uses a stack to simulate the asteroid collision process. For each asteroid in the input: If it is moving to the right or the stack is empty, simply add it to the stack. If it is moving to the left, check the asteroid on top of the stack: If the top asteroid is moving to the right, they might collide. Compare sizes: If the left-moving asteroid is larger (in absolute terms), pop the stack and continue checking. If both are equal, pop the stack and do not add

the current asteroid. If the right-moving asteroid is larger, the current asteroid explodes. If no collision occurs for the current asteroid, push it onto the stack. Finally, the stack represents the surviving asteroids order.

Stack

□ #739 Daily Temperatures

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Monotonic Stack
Lists: Grind 169

Problem

Given an array of integers `temperatures` represents the daily temperatures, return an array `answer` such that `answer[i]` is the number of days you have to wait after the `i`th day to get a warmer temperature. If there is no future day for which this is possible, keep `answer[i] == 0` instead.

Example 1:

Input: `temperatures = [73,74,75,71,69,72,76,73]`
Output: `[1,1,4,2,1,1,0,0]`

Example 2:

Input: `temperatures = [30,40,50,60]`
Output: `[1,1,1,0]`

Example 3:

Input: temperatures = [30,60,90]
Output: [1,1,0]

Constraints:

- $1 \leq \text{temperatures.length} \leq 105$
- $30 \leq \text{temperatures}[i] \leq 100$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# For each day, find the number of days until a warmer temperature.
# If no warmer day exists, the answer for that day is 0. Right?"
#
# "A few quick questions:
# All temperatures are in [30, 100]? Yes. Can they all be decreasing? Yes – all
# zeros then.
# We want days to wait, not the actual temperature difference."
#
# "Let me walk: temps=[73,74,75,71,69,72,76,73].
# Day 0: next warmer is day 1 → 1. Day 2: next warmer is day 6 → 4. →
# [1,1,4,2,1,1,0,0] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# For each day i, scan forward until finding a warmer temperature.
# O(n²) time, O(1) extra space.
# Should I go ahead and optimize?"
```

```
class _739_DailyTemperatures_Brute:
    def dailyTemperatures(self, temperatures):
        n = len(temperatures)
        ans = [0] * n
        for i in range(n):
            for j in range(i + 1, n):
                if temperatures[j] > temperatures[i]:
                    ans[i] = j - i; break
        return ans
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the inner scan – we re-examine previously seen days repeatedly.
# Monotonic decreasing stack of indices:
```

```
# For each temperature, while the stack top has a smaller temperature than current,
# that day's wait is now known: current_index - stack_top_index.
# Pop and record the answer. Push the current index.
#
# Time: O(n) - each index pushed and popped at most once.
# Space: O(n) for the stack."
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _739_DailyTemperatures:
    def dailyTemperatures(self, temperatures):
        ans = [0] * len(temperatures)
        stack = []
        for i, temp in enumerate(temperatures):
            while stack and temperatures[stack[-1]] < temp:
                j = stack.pop()
                ans[j] = i - j
            stack.append(i)
        return ans
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

```
#
# temps=[73,74,75,71,69,72,76,73]:
# i=0(73): stack=[0]. i=1(74): 74>73 → pop0,ans[0]=1. stack=[1].
# i=2(75): pop1,ans[1]=1. stack=[2]. i=3(71): push. [2,3]. i=4(69): push. [2,3,4].
# i=5(72): 72>69 → pop4,ans[4]=1. 72>71 → pop3,ans[3]=2. stack=[2,5].
# i=6(76): pop5,ans[5]=1. pop2,ans[2]=4. stack=[6].
# i=7(73): push. stack=[6,7]. → [1,1,4,2,1,1,0,0] ✓
#
# "No bugs. Indices left on the stack at the end have no warmer day - correctly stay 0."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): each element pushed/popped at most once. Space O(n) for the stack.
# Monotonic stack is the canonical pattern for 'next greater element' problems.
# Follow-up: Next Greater Element I (#496) - same stack, cross-reference to another array.
# Follow-up: Next Greater Element II (#503) - circular array; iterate twice (2n elements)."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def dailyTemperatures(self, temperatures: list[int]) -> list[int]:
        ans = [0] * len(temperatures)
        stack = [] # a decreasing stack

        for i, temperature in enumerate(temperatures):
            while stack and temperature > temperatures[stack[-1]]:
                index = stack.pop()
                ans[index] = i - index
            stack.append(i)

        return ans
```

• LeetDoocs

```
class Solution:
    def dailyTemperatures(self, temperatures: List[int]) -> List[int]:
        stk = []
        n = len(temperatures)
        ans = [0] * n
        for i in range(n - 1, -1, -1):
            while stk and temperatures[stk[-1]] <= temperatures[i]:
                stk.pop()
            if stk:
                ans[i] = stk[-1] - i
            stk.append(i)
        return ans
```

• SimplyLeet

```
def dailyTemperatures(temperatures):
    n = len(temperatures)
    # Initialize answer list with zeros
    answer = [0] * n
    # Stack to keep indices of temperatures with unresolved next warmer days
    stack = []
    # Iterate through the temperature list
    for i, temp in enumerate(temperatures):
        # Check if the current temperature resolves any previous indices
        while stack and temperatures[stack[-1]] < temp:
```

```

        prev_index = stack.pop() # Get the index of the previous colder
day
        answer[prev_index] = i - prev_index # Update the number of days
waited
        # Push the current index onto the stack
        stack.append(i)
    return answer

# Example usage:
if __name__ == "__main__":
    temps = [73,74,75,71,69,72,76,73]
    print(dailyTemperatures(temps))

```

• LC.ca

```

class Solution:
    def dailyTemperatures(self, temperatures: List[int]) -> List[int]:
        ans = [0] * len(temperatures)
        stk = []
        for i, t in enumerate(temperatures):
            while stk and temperatures[stk[-1]] < t:
                j = stk.pop()
                ans[j] = i - j
            stk.append(i)
        return ans

```

◆ Quick Review

Key Insights

- Use a monotonic stack to keep track of indices of days with unresolved warmer temperatures.
- As you iterate through the temperatures, resolve previous days that have found a warmer temperature.

- The index differences yield the number of days waited until a warmer temperature.

Space & Time

Time Complexity: $O(n)$, where n is the number of temperatures, because each index is pushed and popped from the stack at most once.

Space Complexity: $O(n)$ for storing the stack and the answer array.

Solution

The solution uses a monotonic stack to track the indices whose next warmer temperature has not been found yet. As you iterate through the temperature list, compare the current temperature with the temperature at the index stored at the top of the stack. If the current temperature is higher, it means you've found a warmer day for the day represented by the index from the stack. Pop that index, calculate the difference between the current index and the popped index, and store the result. Push the current index onto the stack for future comparisons. This approach efficiently finds the answer without nested iterations.

Stack

□ #962 Maximum Width Ramp

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Monotonic Stack
Lists: Denny Zhang

Problem

A ramp in an integer array `nums` is a pair (i, j) for which $i < j$ and `nums[i] ≤ nums[j]`. The width of such a ramp is $j - i$.

Given an integer array `nums`, return the maximum width of a ramp in `nums`. If there is no ramp in `nums`, return 0.

Example 1:

Input: `nums = [6,0,8,2,1,5]`

Output: 4

Explanation: The maximum width ramp is achieved at $(i, j) = (1, 5)$: `nums[1] = 0` and `nums[5] = 5`.

Example 2:

Input: `nums = [9,8,1,0,1,9,4,0,4,1]`

Output: 7

Explanation: The maximum width ramp is achieved at $(i, j) = (2, 9)$: `nums[2] = 1` and `nums[9] = 1`.

Constraints:

- $2 \leq \text{nums.length} \leq 5 * 10^4$
- $0 \leq \text{nums}[i] \leq 5 * 10^4$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# A ramp is a pair (i, j) where i < j and nums[i] <= nums[j].
# Width = j - i. Find the maximum width ramp. Return 0 if none. Right?"
#
# "A few quick questions:
# Can all elements be equal? Yes – every pair is a ramp.
# Can all elements be strictly decreasing? Yes – no ramp exists, return 0."
#
# "Let me walk: nums=[6,0,8,2,1,5] → ramp (1,5): 0<=5, width=4 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# For every pair (i, j), check nums[i] <= nums[j] and track max j-i.
# O(n²) time, O(1) space.
# Should I go ahead and optimize?"
class _962_MaxWidthRamp_Brute:
    def maxWidthRamp(self, nums):
        best = 0
        for i in range(len(nums)):
            for j in range(i + 1, len(nums)):
                if nums[i] <= nums[j]:
                    best = max(best, j - i)
        return best
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the O(n²) nested loop.
# Two-phase monotonic stack approach:
# Phase 1 (left candidates): scan left to right, build a decreasing stack of
indices.
# Only indices with strictly decreasing values can be valid left boundaries.
# Phase 2 (right scan): scan right to left; for each j, pop from the stack while
# nums[stack.top] <= nums[j] – that's a valid ramp; record j - stack.top.
#
# Time: O(n). Space: O(n) for the stack."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```



```
class _962_MaxWidthRamp:
    def maxWidthRamp(self, nums):
        n = len(nums)
        stack = []
        for i in range(n):
            if not stack or nums[i] < nums[stack[-1]]:
                stack.append(i)
        best = 0
        for j in range(n - 1, -1, -1):
            while stack and nums[stack[-1]] <= nums[j]:
                best = max(best, j - stack[-1])
                stack.pop()
        return best
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[6,0,8,2,1,5]:

Phase1: 6→[0]. 0<6→[0,1]. 8>0 skip. 2>0 skip. 1>0 skip. 5>0 skip. stack=[0,1].

Phase2: j=5(5): nums[1]=0<=5→best=5-1=4,pop. nums[0]=6>5 stop. stack=[0].

j=4(1): nums[0]=6>1 stop. ... j=2(8): nums[0]=6<=8→best=max(4,2-0)=4,pop.

stack=[].

return 4 ✓

#

"No bugs. We don't pop if best would decrease – we want the maximum width, so scan right-to-left."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time O(n): two linear passes, each index pushed/popped at most once.

Space O(n): the decreasing stack.

The phase-1 stack stores only left-side candidates where each has a smaller value than the prior,

ensuring they're the best possible left boundary for some right side.

Follow-up: if we want min width, scan from left in phase 2.

Follow-up: Largest Rectangle in Histogram (#84) uses a similar monotonic stack pattern."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def maxWidthRamp(self, nums: list[int]) -> int:
        ans = 0
        stack = []

        for i, num in enumerate(nums):
            if stack == [] or num <= nums[stack[-1]]:
                stack.append(i)

        for i, num in reversed(list(enumerate(nums))):

            while stack and num >= nums[stack[-1]]:
                ans = max(ans, i - stack.pop())

        return ans

```

• LeetDoocs

```

class Solution:
    def maxWidthRamp(self, nums: List[int]) -> int:
        stk = []
        for i, v in enumerate(nums):
            if not stk or nums[stk[-1]] > v:
                stk.append(i)
        ans = 0
        for i in range(len(nums) - 1, -1, -1):
            while stk and nums[stk[-1]] <= nums[i]:
                ans = max(ans, i - stk.pop())

        if not stk:
            break
        return ans

```

• SimplyLeet

```
# Python solution for Maximum Width Ramp

def maxWidthRamp(nums):
    n = len(nums)
    stack = []
    # Build a stack of indices where array values are in decreasing order
    for i in range(n):
        if not stack or nums[i] < nums[stack[-1]]:
            stack.append(i)
    max_width = 0

    # Iterate backwards to find the maximum width ramp
    for j in range(n - 1, -1, -1):
        while stack and nums[j] >= nums[stack[-1]]:
            max_width = max(max_width, j - stack.pop())
    return max_width

# Example usage:
nums = [6, 0, 8, 2, 1, 5]
print(maxWidthRamp(nums)) # Output: 4
```

• LC.ca

```
class Solution:
    def maxWidthRamp(self, nums: List[int]) -> int:
        stk = []
        for i, v in enumerate(nums):
            if not stk or nums[stk[-1]] > v:
                stk.append(i)
        ans = 0
        for i in range(len(nums) - 1, -1, -1):
            while stk and nums[stk[-1]] <= nums[i]:
                ans = max(ans, i - stk.pop())

        if not stk:
            break
        return ans
```

◆ Quick Review

Key Insights

- We need to identify pairs (i, j) where $i < j$ and $\text{nums}[i] \leq \text{nums}[j]$.
- A monotonic decreasing stack can be constructed to maintain candidate starting indices.
- By iterating from left to right, we push indices onto the stack only when they have a smaller value than the last.
- Then, a backward pass from the end of the array allows us to efficiently match each element with the smallest possible candidate index from the stack.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

The solution is built in two phases: Build a decreasing stack of indices by iterating through the array (only push an index if its value is less than the value at the current top of the stack). Iterate through the array backwards. For each index j , check and pop elements from the stack while $\text{nums}[j]$ is greater than or equal to the nums value at the

popped index. For each valid ramp found, compute the width ($j - \text{popped index}$) and update the maximum width accordingly. This approach guarantees we examine each element a constant number of times ensuring an efficient solution.

Stack

□ ★ #1214 Two Sum BSTs ★

MED

[Open on LeetCode](#)

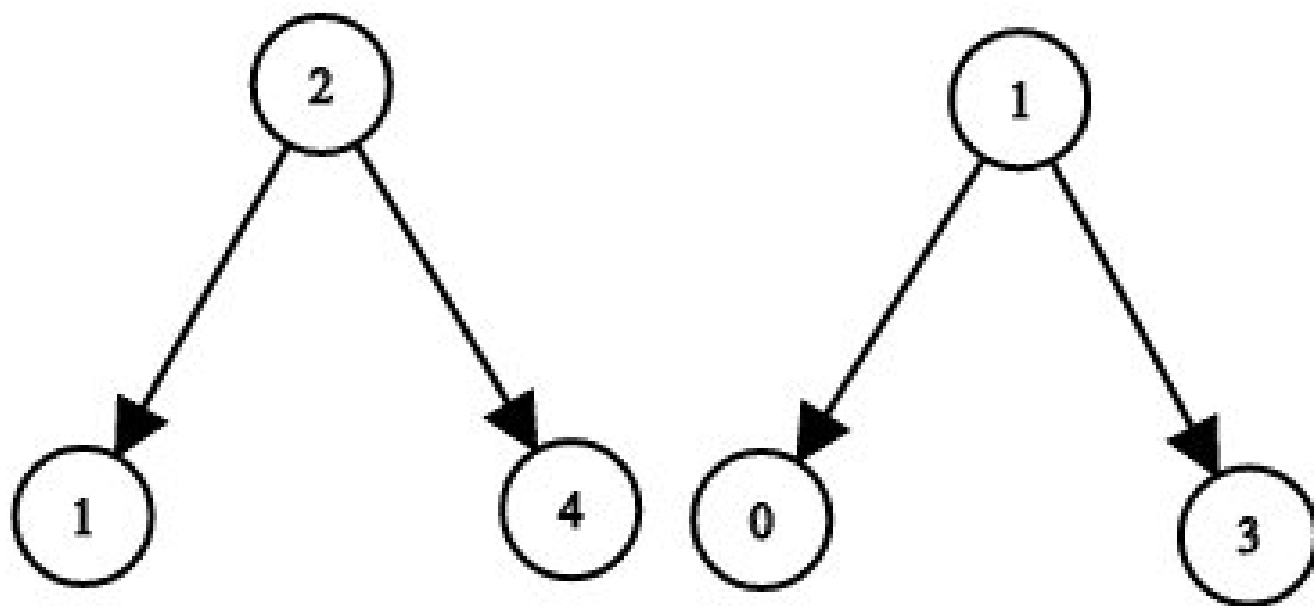
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · Binary Search · Stack · Tree ·
DFS · BST · BFS

Lists: ★ Premium | Premium 98

Problem

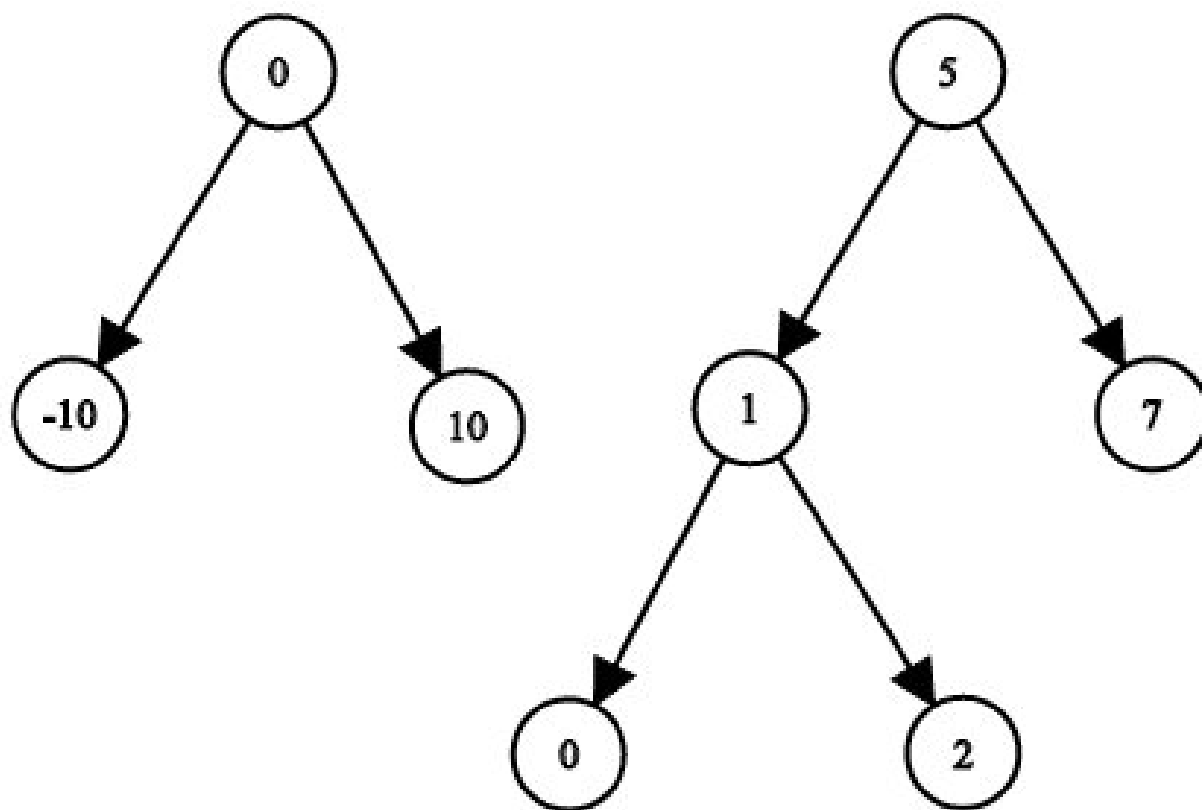
Given the roots of two binary search trees, root1 and root2, return true if and only if there is a node in the first tree and a node in the second tree whose values sum up to a given integer target.

Example 1:



Input: root1 = [2,1,4], root2 = [1,0,3], target = 5
Output: true
Explanation: 2 and 3 sum up to 5.

Example 2:



Input: root1 = [0,-10,10], root2 = [5,1,7,0,2], target = 18
 Output: false

Constraints:

- The number of nodes in each tree is in the range [1, 5000].
- $-109 \leq \text{Node.val}$, $\text{target} \leq 109$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Given two BSTs and a target, return true if there is one value from each tree
# summing to target. Right?"
#
# "A few quick questions:
# Trees can have different sizes? Yes. Values can be negative? Yes."
#
# "Let me walk: root1=[2,1,4], root2=[1,0,3], target=5.
# 2+3=5 ✓ → True ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Collect all values from tree1 into a set. DFS tree2; check if (target - val) is in
the set.
# O(n+m) time, O(n) space for the set.
# Should I go ahead and optimize?"
class _1214_TwoSumBSTs_Brute:
    def twoSumBSTs(self, root1, root2, target):
        vals = set()
        def collect(node):
            if node: vals.add(node.val); collect(node.left); collect(node.right)
        def search(node):
            if not node: return False
            return (target - node.val) in vals or search(node.left) or
search(node.right)
        collect(root1)
        return search(root2)
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the O(n) space set for tree1's values.
# BST inorder traversal gives sorted arrays.
# Two-iterator approach: one ascending (tree1 smallest to largest),
# one descending (tree2 largest to smallest).
# Advance the ascending iterator if sum < target; descending if sum > target.
# Time: O(n+m). Space: O(h1+h2) – just the two iterator stacks."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _1214_TwoSumBSTs:
    def twoSumBSTs(self, root1, root2, target):
        def push_left(node, stack):
            while node: stack.append(node); node = node.left
        def push_right(node, stack):
            while node: stack.append(node); node = node.right
        lo_stack, hi_stack = [], []
        push_left(root1, lo_stack)
        push_right(root2, hi_stack)
        while lo_stack and hi_stack:
            lo_val = lo_stack[-1].val
            hi_val = hi_stack[-1].val
            total = lo_val + hi_val
            if total == target:
                return True
            elif total < target:
                node = lo_stack.pop()
                push_left(node.right, lo_stack)
            else:
                node = hi_stack.pop()
                push_right(node.left, hi_stack)
        return False
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
```

```
#
```

```
# root1=[2,1,4], root2=[1,0,3], target=5:
```

```
# lo_stack=[2,1] (left-spine). hi_stack=[1,3] (right-spine: push_right gives root
then go right? No:
```

```
# push_right(root2=[1,0,3]): push 1, then right=3, push 3, right=None. hi=[1,3].
top=3.
```

```
# lo_val=1, hi_val=3: 1+3=4<5 → advance lo: pop1, push_left(1.right=None). lo=[2].
```

```
# lo_val=2, hi_val=3: 2+3=5 → True ✓
```

```
#
```

```
# "No bugs. push_left/push_right prepare the iterators for inorder
ascending/descending."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n+m): each node visited at most once. Space O(h1+h2) for the two stacks.
```

```
# Hash-set brute force is O(n) space regardless of tree shape.
```

Follow-up: Two Sum IV (#653) – find pair WITHIN a single BST; use BST iterator + set.
 # Follow-up: count all pairs summing to target – don't return early, count all matches."

Community Solutions (Python)

• WalkCC

```
class BSTIterator:
    def __init__(self, root: TreeNode | None, leftToRight: bool):
        self.stack = []
        self.leftToRight = leftToRight
        self._pushUntilNone(root)

    def hasNext(self) -> bool:
        return len(self.stack) > 0

    def next(self) -> int:

        node = self.stack.pop()
        if self.leftToRight:
            self._pushUntilNone(node.right)
        else:
            self._pushUntilNone(node.left)
        return node.val

    def _pushUntilNone(self, root: TreeNode | None):
        while root:
            self.stack.append(root)

            root = root.left if self.leftToRight else root.right

class Solution:
    def twoSumBSTs(
        self,
        root1: TreeNode | None,
        root2: TreeNode | None,
        target: int,
    ) -> bool:
```

```
bst1 = BSTIterator(root1, True)
bst2 = BSTIterator(root2, False)
```

```
l = bst1.next()
r = bst2.next()
while True:
    summ = l + r
    if summ == target:
        return True
    if summ < target:
```

```
        if not bst1.hasNext():
            return False
        l = bst1.next()
    else:
        if not bst2.hasNext():
            return False
        r = bst2.next()
```

• LeetDoocs

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def twoSumBSTs(
        self, root1: Optional[TreeNode], root2: Optional[TreeNode], target: int
    ) -> bool:
```

```
    def dfs(root: Optional[TreeNode], i: int):
        if root is None:
            return
        dfs(root.left, i)
        nums[i].append(root.val)
        dfs(root.right, i)
```

```
nums = [[], []]
dfs(root1, 0)
dfs(root2, 1)
```

```

i, j = 0, len(nums[1]) - 1
while i < len(nums[0]) and ~j:
    x = nums[0][i] + nums[1][j]
    if x == target:
        return True
    if x < target:
        i += 1
    else:
        j -= 1
return False

```

• SimplyLeet

```

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def twoSumBSTs(root1: TreeNode, root2: TreeNode, target: int) -> bool:
    # Create a set to store values from the first BST.
    values_set = set()

    # Helper function to perform DFS and add node values to the set.
    def dfs_collect(node):
        if not node:
            return
        values_set.add(node.val) # Add current node value
        dfs_collect(node.left)   # Traverse left subtree
        dfs_collect(node.right)  # Traverse right subtree

    # Collect all values from first tree.

```

```

dfs_collect(root1)

# Helper function to traverse second tree and check for complement.
def dfs_check(node):
    if not node:
        return False
    # If the complement exists in the first tree values, return True.
    if target - node.val in values_set:
        return True
    # Recursively check left and right children.

    return dfs_check(node.left) or dfs_check(node.right)

# Check the second tree.
return dfs_check(root2)

# Example usage:
# Construct trees and call twoSumBSTs as needed.

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def twoSumBSTs(
        self, root1: Optional[TreeNode], root2: Optional[TreeNode], target: int
    ) -> bool:

        def dfs(root: Optional[TreeNode], i: int):
            if root is None:
                return
            dfs(root.left, i)
            nums[i].append(root.val)
            dfs(root.right, i)

        nums = [[], []]
        dfs(root1, 0)
        dfs(root2, 1)

```

```
i, j = 0, len(nums[1]) - 1
while i < len(nums[0]) and ~j:
    x = nums[0][i] + nums[1][j]
    if x == target:
        return True
    if x < target:
        i += 1
    else:
        j -= 1
return False
```

◆ Quick Review

Key Insights

- Traverse the first tree to collect all node values.
- Traverse the second tree to check for each node if the complement (target – current node value) exists in the first tree's values.
- Using a hash set for the first tree values allows efficient $O(1)$ lookups.
- The BST property is not essential for the hash set solution, but can be exploited if using iterator techniques for a two-pointer approach.

Space & Time

Time Complexity: $O(n + m)$ where n and m are the number of nodes in the first and second

trees respectively.

Space Complexity: $O(n)$ for storing the node values from the first tree (in worst-case scenario).

Solution

We perform a DFS (or any tree traversal) on the first BST to collect all its values in a hash set. Then, we traverse the second BST and for each node, we calculate the complement (target – node value) and check whether this complement exists in the hash set. If we find such a pair, we return true. If no such pair exists after traversing the second tree completely, we return false. This approach is straightforward and leverages extra space (the set) for constant time lookups, yielding an overall linear time complexity relative to the number of nodes.

Stack

□ ★ #1762 Buildings With an Ocean View ★ **MED**

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Monotonic Stack
Lists: ★ Premium | Premium 98

Problem

There are n buildings in a line. You are given an integer array `heights` of size n that represents the heights of the buildings in the line.

The ocean is to the right of the buildings. A building has an ocean view if the building can see the ocean without obstructions. Formally, a building has an ocean view if all the buildings to its right have a smaller height.

Return a list of indices (0-indexed) of buildings that have an ocean view, sorted in increasing order.

Example 1:

Input: `heights = [4,2,3,1]`

Output: `[0,2,3]`

Explanation: Building 1 (0-indexed) does not have an ocean view because building 2 is taller.

Example 2:

Input: heights = [4,3,2,1]

Output: [0,1,2,3]

Explanation: All the buildings have an ocean view.

Example 3:

Input: heights = [1,3,2,4]

Output: [3]

Explanation: Only building 3 has an ocean view.

Constraints:

- $1 \leq \text{heights.length} \leq 105$
- $1 \leq \text{heights}[i] \leq 109$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Ocean is to the right. A building has ocean view if ALL buildings to its right are shorter.

Return the indices of buildings with ocean view, sorted ascending. Right?"

#

"A few quick questions:

The rightmost building always has ocean view? Yes – nothing blocks it.

Heights are positive integers? Yes."

#

"Let me walk: heights=[4,2,3,1] → building0(4)>all right ✓, building1(2)<3 ✗,

building2(3)>1 ✓, building3(1) ✓ → [0,2,3] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For each building i, check if all j > i have heights[j] < heights[i].

O(n²) time, O(1) space.

Should I go ahead and optimize?"

```
class _1762_BuildingsWithOceanView_Brute:
```

```
    def findBuildings(self, heights):
```

```
        n = len(heights)
```

```
        return [i for i in range(n) if all(heights[j] < heights[i] for j in
range(i+1, n))]
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the O(n) inner scan for each building.
# Scan from RIGHT to LEFT, tracking the maximum height seen so far.
# A building has ocean view if its height > current max_right (all buildings to its
right are shorter).
# Collect indices and reverse at the end.
#
# Time: O(n). Space: O(1) extra (output not counted)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _1762_BuildingsWithOceanView:
    def findBuildings(self, heights):
        max_right = 0
        result = []
        for i in range(len(heights) - 1, -1, -1):
            if heights[i] > max_right:
                result.append(i)
                max_right = heights[i]
        return result[::-1]
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# heights=[4,2,3,1]:
# i=3(1): 1>0=max_right → append 3, max=1.
# i=2(3): 3>1 → append 2, max=3.
# i=1(2): 2<3 → skip.
# i=0(4): 4>3 → append 0, max=4.
# reversed: [0,2,3] ✓
#
# heights=[4,3,2,1]: all decreasing → [0,1,2,3] ✓
# heights=[1,3,2,4]: only 4 at end > everything → [3] ✓
#
# "No bugs. Traversing right-to-left with a max_right check is O(n) and exact."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): single right-to-left scan. Space O(1) ignoring output.
# Monotonic stack variant: scan left-to-right, pop any building shorter than
current.
# Same O(n) but uses O(n) stack space vs O(1) here.
# Follow-up: buildings with view to both left AND right – need max from both sides.
# Follow-up: circular arrangement – two-pass or range max query needed."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findBuildings(self, heights: list[int]) -> list[int]:
        stack = []

        for i, height in enumerate(heights):
            while stack and heights[stack[-1]] <= height:
                stack.pop()
            stack.append(i)

        return stack
```

• LeetDoocs

```
class Solution:
    def findBuildings(self, heights: List[int]) -> List[int]:
        ans = []
        mx = 0
        for i in range(len(heights) - 1, -1, -1):
            if heights[i] > mx:
                ans.append(i)
                mx = heights[i]
        return ans[::-1]
```

• SimplyLeet

```
def findBuildingsWithOceanView(heights):
    n = len(heights)
    result = []
    max_height = 0
    # Iterate from the rightmost building to the leftmost building.
    for i in range(n - 1, -1, -1):
        if heights[i] > max_height:
            result.append(i)
            max_height = heights[i]
    # Reverse the result to return indices in increasing order.

    result.reverse()
    return result

# Example usage:
# heights = [4,2,3,1]
# print(findBuildingsWithOceanView(heights)) # Output: [0,2,3]
```

• LC.ca

```
class Solution:
    def findBuildings(self, heights: List[int]) -> List[int]:
        ans = []
        mx = 0
        for i in range(len(heights) - 1, -1, -1):
            if heights[i] > mx:
                ans.append(i)
                mx = heights[i]
        return ans[::-1]
```

◆ Quick Review

Key Insights

- Processing the buildings from right to left simplifies the problem because the rightmost building always has an ocean view.
- Keep track of the highest building encountered while iterating from right to left.
- A building has an ocean view if its height is greater than the maximum height seen so far.
- Append indices meeting the criteria, then reverse the order of indices for the solution since they were collected in reverse order.

Space & Time

Time Complexity: $O(n)$ where n is the number of buildings.

Space Complexity: $O(n)$ for storing the result in the worst-case scenario.

Solution

Iterate over the "heights" array from right to left while maintaining a variable to store the maximum height encountered so far. For each building, if its height is greater than this maximum height, add its index to the results list and update the maximum height. Since indices are collected in reverse order, reverse the list before returning it. This approach ensures that each building is visited only once.

Heap

Round 5 · Mid · Medium · 11 questions

Heap

□ #215 Kth Largest Element in an Array

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Divide and Conquer · Sorting · Heap ·
Quickselect

Lists: Grind 169

Problem

Given an integer array `nums` and an integer `k`, return the `k`th largest element in the array.

Note that it is the `k`th largest element in the sorted order, not the `k`th distinct element.

Can you solve it without sorting?

Example 1:

Input: `nums = [3,2,1,5,6,4]`, `k = 2`
Output: 5

Example 2:

Input: `nums = [3,2,3,1,2,4,5,5,6]`, `k = 4`
Output: 4

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def findKthLargest(self, nums: list[int], k: int) -> int:
        minHeap = []

        for num in nums:
            heapq.heappush(minHeap, num)
            if len(minHeap) > k:
                heapq.heappop(minHeap)

        return minHeap[0]
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Find the kth largest element in an unsorted array.
# Not kth distinct – duplicates count toward the position. Right?"
#
# "A few quick questions:
# k is always valid (1 ≤ k ≤ n)? Yes. Can elements be negative? Yes."
#
# "Let me walk: nums=[3,2,1,5,6,4], k=2 → sorted desc=[6,5,4,3,2,1] → 5 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Sort the array in descending order, return element at index k-1.
# O(n log n) time, O(1) extra space.
# Should I go ahead and optimize?"
class _215_KthLargestElement_Brute:
    def findKthLargest(self, nums, k):
        nums.sort(reverse=True)
        return nums[k - 1]
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is sorting all n elements when we only need the top k.
# Min-heap of size k: maintain the k largest elements seen so far.
# The heap's root (minimum of the k largest) is the kth largest.
# Push each element; if heap grows beyond k, pop the smallest.
#
# Time: O(n log k). Space: O(k).
# Better than O(n log n) when k is much smaller than n."
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
import heapq
class _215_KthLargestElement:
    def findKthLargest(self, nums, k):
        min_heap = []
        for num in nums:
            heapq.heappush(min_heap, num)
            if len(min_heap) > k:
                heapq.heappop(min_heap)
        return min_heap[0]
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[3,2,1,5,6,4], k=2:

push 3→[3]. push 2→[2,3]. push 1→[1,3,2]→pop1→[2,3].

push 5→[2,3,5]→pop2→[3,5]. push 6→[3,5,6]→pop3→[5,6].

push 4→[4,6,5]→pop4→[5,6]. heap[0]=5 ✓

#

"No bugs. After processing all elements, the heap contains the k largest;

its min (heap[0]) is the kth largest."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Min-heap: $O(n \log k)$. Sort: $O(n \log n)$. Heap wins when $k \ll n$.

Quickselect: $O(n)$ average, $O(n^2)$ worst – optimal for single large- n queries.

Follow-up: Kth Smallest in Sorted Matrix (#378) – 2D extension using a heap.

Follow-up: streaming kth largest (#703) – maintain a min-heap of size k permanently."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findKthLargest(self, nums: list[int], k: int) -> int:
        minHeap = []

        for num in nums:
            heapq.heappush(minHeap, num)
            if len(minHeap) > k:
                heapq.heappop(minHeap)

        return minHeap[0]
```

```

class Solution:
    def findKthLargest(self, nums: list[int], k: int) -> int:
        def quickSelect(l: int, r: int, k: int) -> int:
            pivot = nums[r]

            nextSwapped = l
            for i in range(l, r):
                if nums[i] >= pivot:
                    nums[nextSwapped], nums[i] = nums[i], nums[nextSwapped]
                    nextSwapped += 1

            nums[nextSwapped], nums[r] = nums[r], nums[nextSwapped]

            count = nextSwapped - l + 1 # Number of nums >= pivot
            if count == k:
                return nums[nextSwapped]
            if count > k:
                return quickSelect(l, nextSwapped - 1, k)
            return quickSelect(nextSwapped + 1, r, k - count)

        return quickSelect(0, len(nums) - 1, k)

```

• LeetDoocs

```

class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        def quick_sort(l: int, r: int) -> int:
            if l == r:
                return nums[l]
            i, j = l - 1, r + 1
            x = nums[(l + r) >> 1]
            while i < j:
                while 1:
                    i += 1

```

```

        if nums[i] >= x:
            break
        while 1:
            j -= 1
            if nums[j] <= x:
                break
            if i < j:
                nums[i], nums[j] = nums[j], nums[i]
        if j < k:
            return quick_sort(j + 1, r)

```

```

    return quick_sort(l, j)

```

```

n = len(nums)
k = n - k
return quick_sort(0, n - 1)

```

```

class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        return nlargest(k, nums)[-1]

```

```

class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        cnt = Counter(nums)
        for i in count(max(cnt), -1):
            k -= cnt[i]
            if k <= 0:
                return i

```

• SimplyLeet

```

import heapq

def findKthLargest(nums, k):
    # Create a min-heap with the first k elements
    min_heap = nums[:k]
    heapq.heapify(min_heap) # O(k) time to build the initial heap

    # Process the remaining elements
    for num in nums[k:]:
        # If the current element is larger than the smallest in the heap,
        replace it

```

```

        if num > min_heap[0]:
            heapq.heapreplace(min_heap, num) # O(log k)
        # The smallest element in the heap is the kth largest overall
        return min_heap[0]

# Example usage:
nums = [3, 2, 1, 5, 6, 4]
k = 2
print(findKthLargest(nums, k)) # Expected output: 5

```

• LC.ca

```

class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        lo, hi = 0, len(nums) - 1

        while lo <= hi:
            pos = self.partition(nums, lo, hi)
            if pos == k - 1: # pos starting from 0, so -1
                return nums[pos] # partially sorted, desc
            elif pos > k - 1:
                hi = pos - 1

            else: # pos < k - 1
                lo = pos + 1

        return -1 # or raise exception

    def partition(self, nums: List[int], lo: int, hi: int) -> int:
        pivot, l, r = nums[lo], lo + 1, hi
        while l <= r:
            if nums[l] < pivot and nums[r] > pivot:
                # larger num at left of pivot, easier to count for k-th largest

```

```

        nums[l], nums[r] = nums[r], nums[l]
        l += 1
        r -= 1
    if nums[l] >= pivot:
        l += 1
    if nums[r] <= pivot:
        r -= 1
# use nums[l] will lead to infinite looping
nums[lo], nums[r] = nums[r], nums[lo]
# possible there is duplicated num, but will be covered here

```

```

return r

```

```

#####

```

```

class Solution:

```

```

    def findKthLargest(self, nums: List[int], k: int) -> int:
    def quick_sort(left, right, k):
        if left == right:
            return nums[left]
        i, j = left - 1, right + 1

```

```

        x = nums[(left + right) >> 1]
        while i < j:
            while 1:
                i += 1
                if nums[i] >= x:
                    break
            while 1:
                j -= 1
                if nums[j] <= x:
                    break

```

```

        if i < j:
            nums[i], nums[j] = nums[j], nums[i]
        if j < k:
            return quick_sort(j + 1, right, k)
        return quick_sort(left, j, k)

```

```

n = len(nums)
return quick_sort(0, n - 1, n - k)

```

```

#####

```



```
class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        def quick_sort(left, right, k):
            if left == right:
                return nums[left]
            i, j = left - 1, right + 1
            x = nums[(left + right) >> 1]
            while i < j:
                while 1:
                    i += 1
                    if nums[i] >= x:
                        break
                while 1:
                    j -= 1
                    if nums[j] <= x:
                        break
                if i < j:
                    nums[i], nums[j] = nums[j], nums[i]
            if j < k:
                return quick_sort(j + 1, right, k)
            return quick_sort(left, j, k)
        n = len(nums)
        return quick_sort(0, n - 1, n - k)
```

◆ Quick Review

Key Insights

- kth largest element can be converted to finding the element at index $n-k$ if the array were sorted.
- A min-heap of size k is a practical method to maintain the k largest elements seen so far.

- The Quickselect algorithm can find the k th largest element in average $O(n)$ time with in-place partitioning.
- Choose an appropriate method based on performance needs and worst-case scenarios.

Space & Time

Time Complexity:

Min-Heap: $O(n \log k)$ by maintaining a heap of size k .

Quickselect: Average $O(n)$ but worst-case $O(n^2)$ with poor pivot selection.

Space Complexity:

Min-Heap: $O(k)$

Quickselect: $O(1)$ additional space with in-place partitioning.

Solution

We can solve the problem using either a min-heap or the Quickselect algorithm.

Min-Heap Approach: Initialize a min-heap with the first k elements. Iterate through the remaining elements; if an element is greater than the heap's root (the smallest in the heap), replace the root. After processing all elements,

the root of the min-heap is the k th largest element. Quickselect Approach: Choose a pivot and partition the array so that elements larger than the pivot come before it. Determine the position of the pivot; if it matches the index corresponding to the k th largest element, return it. Recurse into the appropriate part of the array. Key points to consider include maintaining heap size for the first approach and proper pivot selection for the Quickselect approach.

Heap

□ ★ #253 Meeting Rooms II ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Greedy · Sorting · Heap
Lists: ★ Premium | Grind 169 | Premium 98

Problem

Given an array of meeting time intervals intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, return the minimum number of conference rooms required.

Example 1:

```
Input: intervals = [[0,30],[5,10],[15,20]]  
Output: 2
```

Example 2:

```
Input: intervals = [[7,10],[2,4]]  
Output: 1
```

Constraints:

- $1 \leq \text{intervals.length} \leq 104$
- $0 \leq \text{start}_i < \text{end}_i \leq 106$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def minMeetingRooms(self, intervals: list[list[int]]) -> int:
        minHeap = [] # Store the end times of each room.

        for start, end in sorted(intervals):
            # There's no overlap, so we can reuse the same room.
            if minHeap and start >= minHeap[0]:
                heapq.heappop(minHeap)
            heapq.heappush(minHeap, end)

        return len(minHeap)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given meeting time intervals, find the minimum number of conference rooms required.

Two meetings can share a room only if one ends before or when the other starts. Right?"

#

"A few quick questions:

Do meetings that touch (end == start) overlap? No – they can share a room.

Return the minimum rooms, not a specific assignment?"

#

"Let me walk: $[[0,30],[5,10],[15,20]] \rightarrow [0,30]$ overlaps with $[5,10] \rightarrow 2$ rooms ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For each meeting, count how many other meetings overlap with it.

Maximum overlap count + 1 = rooms needed.

$O(n^2)$ time, $O(1)$ space.

Should I go ahead and optimize?"

```
class _253_MeetingRoomsII_Brute:
```

```
    def minMeetingRooms(self, intervals):
```

```
        best = 0
```

```
        for s1, e1 in intervals:
```

```
            count = 1
```

```
            for s2, e2 in intervals:
```

```
                if s1 != s2 and s1 < e2 and s2 < e1: count += 1
```

```
            best = max(best, count)
```

```
        return best
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n^2)$ pairwise overlap check.

Min-heap of end times: tracks when each occupied room becomes free.

Sort meetings by start time. For each meeting:

– If the earliest-ending room (heap top) finishes before or at the meeting start, reuse it.

– Otherwise, allocate a new room.

Heap size at the end = minimum rooms needed.

#

Time: $O(n \log n)$. Space: $O(n)$ for the heap."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
import heapq
```

```
class _253_MeetingRoomsII:
```

```
    def minMeetingRooms(self, intervals):
```

```
        if not intervals: return 0
```

```
        intervals.sort(key=lambda x: x[0])
```

```
        end_times = []
```

```
        for start, end in intervals:
```

```
            if end_times and end_times[0] <= start:
```

```
                heapq.heapreplace(end_times, end)
```

```
            else:
```

```
                heapq.heappush(end_times, end)
```

```
        return len(end_times)
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

$[[0,30],[5,10],[15,20]]$ sorted=same:

$[0,30]$: heap=[] → push 30 → $[30]$.

$[5,10]$: heap[0]=30 > 5 → push 10 → $[10,30]$.

$[15,20]$: heap[0]=10 <= 15 → replace 10 with 20 → $[20,30]$. len=2 ✓

#

$[[7,10],[2,4]]$ sorted= $[[2,4],[7,10]]$:

$[2,4]$: push 4 → $[4]$. $[7,10]$: $4 \leq 7$ → replace with 10 → $[10]$. len=1 ✓

#

"No bugs. heapreplace is equivalent to pop+push but in one $O(\log n)$ operation."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log n)$: sort + n heap ops. Space $O(n)$: heap holds at most n end times.

Alternative: split into sorted start/end events, use two pointers – same complexity, no heap.

Follow-up: Meeting Rooms I (#252) – just check if any two overlap (sort by start).

Follow-up: minimum cost scheduling – extend with room priorities or weighted meetings."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def minMeetingRooms(self, intervals: list[list[int]]) -> int:
        minHeap = [] # Store the end times of each room.

        for start, end in sorted(intervals):
            # There's no overlap, so we can reuse the same room.
            if minHeap and start >= minHeap[0]:
                heapq.heappop(minHeap)
            heapq.heappush(minHeap, end)

        return len(minHeap)
```

```
class Solution:
    def minMeetingRooms(self, intervals: list[list[int]]) -> int:
        n = len(intervals)
        ans = 0
        starts = []
        ends = []

        for start, end in intervals:
            starts.append(start)
            ends.append(end)

        starts.sort()
        ends.sort()

        j = 0
        for i in range(n):
            if starts[i] < ends[j]:
                ans += 1
            else:
                j += 1

        return ans
```

• LeetDoocs


```

class Solution:
    def minMeetingRooms(self, intervals: List[List[int]]) -> int:
        m = max(e[1] for e in intervals)
        d = [0] * (m + 1)
        for l, r in intervals:
            d[l] += 1
            d[r] -= 1
        ans = s = 0
        for v in d:
            s += v

            ans = max(ans, s)
        return ans

```

```

class Solution:
    def minMeetingRooms(self, intervals: List[List[int]]) -> int:
        d = defaultdict(int)
        for l, r in intervals:
            d[l] += 1
            d[r] -= 1
        ans = s = 0
        for _, v in sorted(d.items()):
            s += v
            ans = max(ans, s)

        return ans

```

• SimplyLeet

```

import heapq

def minMeetingRooms(intervals):
    # Sort intervals based on start time
    intervals.sort(key=lambda x: x[0])

    # Initialize a min-heap to store end times of meetings that are currently
    # using rooms
    min_heap = []

    # Process each meeting interval

```

```

    for interval in intervals:
        start, end = interval

        # If the heap is not empty and the current meeting starts after the
        earliest ended meeting
        if min_heap and start >= min_heap[0]:
            heapq.heappop(min_heap) # Reuse the room

        # Allocate the current meeting end time into the heap (new or reused
        room)
        heapq.heappush(min_heap, end)

    # The number of rooms is the number of simultaneous meetings
    return len(min_heap)

# Example usage:
intervals = [[0,30],[5,10],[15,20]]
print(minMeetingRooms(intervals)) # Output: 2

```

• LC.ca

```

...
data = [3, 4, 6, 2, 1, 9, 0, 7, 5, 8]

list(accumulate(data, operator.mul)) # running product
[3, 12, 72, 144, 144, 1296, 0, 0, 0, 0]

list(accumulate(data, max)) # running maximum
[3, 4, 6, 6, 6, 9, 9, 9, 9, 9]

```

```

>>> from itertools import accumulate
>>> data = [3, 4, 6, 2, 1, 9, 0, 7, 5, 8]
>>> accumulate(data)
<itertools.accumulate object at 0x10fd78440>
>>> list(accumulate(data))
[3, 7, 13, 15, 16, 25, 25, 32, 37, 45]
>>> max(accumulate(data))
45

```

<https://docs.python.org/3/library/itertools.html#itertools.accumulate>

```
# Amortize a 5% loan of 1000 with 4 annual payments of 90
cashflows = [1000, -90, -90, -90, -90]
list(accumulate(cashflows, lambda bal, pmt: bal*1.05 + pmt))
[1000, 960.0, 918.0, 873.9000000000001, 827.5950000000001]
'''

class Solution:
    def minMeetingRooms(self, intervals: List[List[int]]) -> int:
        delta = [0] * 1000010

        for start, end in intervals:
            delta[start] += 1
            delta[end] -= 1
        return max(accumulate(delta))
# why not delta.sort()?
# because accumulate() will go by order from index 0 to index final
# just like, from sortedcontainers import SortedDict

#####
```

```
class Solution(object):
    def minMeetingRooms(self, intervals):
        """
        :type intervals: List[Interval]
        :rtype: int
        """
        meetings = []
        for i in intervals:
            meetings.append((i.start, 1))
            meetings.append((i.end, 0))

        meetings.sort()
        ans = 0
        count = 0
        for meeting in meetings:
            if meeting[1] == 1:
                count += 1
            else:
                count -= 1
            ans = max(ans, count)
        return ans
```

◆ Quick Review

Key Insights

- Sort the intervals based on start times to process meetings in order.
- Use a min-heap (priority queue) to keep track of meeting end times.
- When processing a new meeting, check if its start time is greater than or equal to the smallest end time from the heap.
- If so, it means a meeting room has been freed (the meeting with the smallest end time has ended) and can be reused.
- If not, allocate a new room by pushing the current meeting's end time into the heap.
- The size of the heap at any moment represents the number of rooms currently in use.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting and each heap operation (push/pop) taking $O(\log n)$.

Space Complexity: $O(n)$ in the worst-case scenario when all meetings overlap, requiring all end times in the heap.

Solution

The solution involves first sorting the meeting intervals by their start times. Then, we utilize a min-heap (priority queue) to keep track of the end times of ongoing meetings. For each meeting, we compare its start time with the smallest end time in the heap: If the meeting can start after the meeting at the top of the heap has ended (i.e., its start time is greater than or equal to the smallest end), we remove that end time from the heap (freeing up a room). Then, we add the current meeting's end time to the heap. By the end of processing all meetings, the size of the heap represents the minimum number of conference rooms required.

Heap

□ #347 Top K Frequent Elements

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Divide and Conquer ·
Sorting · Heap · Bucket Sort · Counting ·
Quickselect

Lists: Denny Zhang

Problem

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements. You may return the answer in any order.

Example 1:

Input: `nums = [1,1,1,2,2,3]`, `k = 2`

Output: `[1,2]`

Example 2:

Input: `nums = [1]`, `k = 1`

Output: `[1]`

Example 3:

Input: `nums = [1,2,1,2,1,2,3,1,3,2]`, `k = 2`

Output: `[1,2]`

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$
- k is in the range $[1, \text{the number of unique elements in the array}]$.
- It is guaranteed that the answer is unique.

Follow up: Your algorithm's time complexity must be better than $O(n \log n)$, where n is the array's size.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def topKFrequent(self, nums: list[int], k: int) -> list[int]:
        ans = []
        bucket = [[] for _ in range(len(nums) + 1)]

        for num, freq in collections.Counter(nums).items():
            bucket[freq].append(num)

        for b in reversed(bucket):
            ans += b

        if len(ans) == k:
            return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Return the k most frequent elements. The answer is unique (no ties at position k).

Return in any order. Must be better than $O(n \log n)$. Right?"

#


```

# "A few quick questions:
# Can k equal the number of unique elements? Yes. Values can be negative? Yes."
#
# "Let me walk: nums=[1,1,1,2,2,3], k=2 → 1(freq=3), 2(freq=2) → [1,2] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Count frequencies, sort by frequency descending, take top k.
#  $O(n \log n)$  – the problem asks for better.
# Should I go ahead and optimize?"
class _347_TopKFrequent_Brute:
    def topKFrequent(self, nums, k):
        from collections import Counter
        return [x for x, _ in Counter(nums).most_common(k)]

# PHASE 3 — OPTIMIZE
# "The bottleneck is the  $O(n \log n)$  sort of all frequencies.
# Bucket sort: create  $n+1$  buckets indexed by frequency.
# Each bucket at index  $f$  holds all numbers that appear exactly  $f$  times.
# Scan buckets from index  $n$  down to  $0$ , collect until we have  $k$  elements.
#
# Time:  $O(n)$ . Space:  $O(n)$ .
# Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _347_TopKFrequent:
    def topKFrequent(self, nums, k):
        from collections import Counter
        count = Counter(nums)
        buckets = [[] for _ in range(len(nums) + 1)]
        for num, freq in count.items():
            buckets[freq].append(num)
        result = []
        for freq in range(len(buckets) - 1, 0, -1):
            for num in buckets[freq]:
                result.append(num)
                if len(result) == k:
                    return result
        return result

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# nums=[1,1,1,2,2,3], k=2:
# count={1:3,2:2,3:1}. buckets[3]=[1], buckets[2]=[2], buckets[1]=[3].
# freq=3: result=[1]. freq=2: result=[1,2] → len==k=2 → return [1,2] ✓
#
# "No bugs. Scanning from high frequency ensures we pick the most frequent first."

# PHASE 6 — COMPLEXITY & FOLLOW-UP

```

"Bucket sort: $O(n)$ time, $O(n)$ space. Heap approach: $O(n \log k)$ – worse but simpler.

Bucket sort is the optimal $O(n)$ solution when frequencies $\leq n$ (always true here).

Follow-up: Top K Frequent Words (#692) – break frequency ties alphabetically.

Follow-up: streaming top-k – maintain a min-heap of size k as elements arrive."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def topKFrequent(self, nums: list[int], k: int) -> list[int]:
        ans = []
        bucket = [[] for _ in range(len(nums) + 1)]

        for num, freq in collections.Counter(nums).items():
            bucket[freq].append(num)

        for b in reversed(bucket):
            ans += b

        if len(ans) == k:
            return ans
```

• LeetDoocs

```
class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        cnt = Counter(nums)
        return [x for x, _ in cnt.most_common(k)]
```

• SimplyLeet

```
# Python solution using bucket sort

def topKFrequent(nums, k):
    # Count the frequency of each element in nums
    frequency = {}
    for num in nums:
        frequency[num] = frequency.get(num, 0) + 1

    # Create buckets where index represents frequency
    n = len(nums)
```

```

buckets = [[] for _ in range(n + 1)]
for num, count in frequency.items():
    buckets[count].append(num)

# Gather the k most frequent elements
result = []
# Iterate over buckets in reverse order (highest frequency first)
for freq in range(n, 0, -1):
    for num in buckets[freq]:
        result.append(num)

    if len(result) == k:
        return result

# Example usage:
# print(topKFrequent([1,1,1,2,2,3], 2)) -> Output: [1, 2]

```

• LC.ca

```

'''
>>> nums = ["a", "a", "b", "c", "c"]

>>> cnt = Counter(nums)
>>> cnt
Counter({'a': 2, 'c': 2, 'b': 1})

# default to keys
>>> sorted_freqs = sorted(cnt, key=lambda x: (-cnt[x], x))
>>> sorted_freqs

['a', 'c', 'b']
'''

from collections import Counter

class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        # Count the frequency of each element in the list
        cnt = Counter(nums)

        # Sort the elements by frequency in decreasing order

```

```

# diff from below solution: sorted(cnt), not sorted(cnt.items())
# more in https://leetcode.ca/2017-10-22-692-Top-K-Frequent-Words/
sorted_freqs = sorted(cnt, key=lambda x: (-cnt[x], x))

return sorted_freqs[:k]

#####

class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:

        # Count the frequency of each element in the list
        freqs = Counter(nums)

        # Sort the elements by frequency in decreasing order
        sorted_freqs = sorted(freqs.items(), key=lambda x: x[1], reverse=True)
        # also passing 0J: sorted_freqs = sorted(freqs.items(), key=lambda x:
        -x[1])

        # Take the top k elements
        top_k = [num for num, freq in sorted_freqs[:k]]

        return top_k

#####

...
>>> from heapq import heappush
>>> h = []
>>> heappush(h, (3,1))
>>> heappush(h, (1,1))
>>> heappush(h, (2,1))

```

```
>>> h
[(1, 1), (3, 1), (2, 1)]
```

```
>>> from heapq import heappop
>>> heappop(h)
(1, 1)
>>> heappop(h)
(2, 1)
>>> heappop(h)
(3, 1)
```

```
'''
```

```
from collections import Counter
```

```
class Solution: # heap
```

```
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
```

```
        cnt = Counter(nums)
```

```
        hp = []
```

```
        for num, freq in cnt.items():
```

```
            heappush(hp, (freq, num)) # freq first, default sort by 1st element
```

```
        if len(hp) > k:
```

```
            heappop(hp)
```

```
        return [v[1] for v in hp]
```

```
#####
```

```
from typing import List
```

```
import random
```

```
# O(n log(n)) in the average case
```

```
# O(n^2) worst case
```

```
class Solution:
```

```
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
```

```
        freq_map = {}
```

```
        for num in nums:
```

```
            freq_map[num] = freq_map.get(num, 0) + 1
```

```
        freq_list = list(freq_map.items())
```

```
        self.quick_select(freq_list, 0, len(freq_list) - 1, k)
```

```
        return [num for num, freq in freq_list[:k]]
```

```

def quick_select(self, freq_list, start, end, k):
    if start == end:
        return

    pivot_idx = random.randint(start, end)
    pivot_freq = freq_list[pivot_idx][1]

    left = start
    right = end

    while left <= right:
        while freq_list[left][1] > pivot_freq:
            left += 1
        while freq_list[right][1] < pivot_freq:
            right -= 1

        if left <= right:
            freq_list[left], freq_list[right] = freq_list[right], freq_list[
t[left]
            left += 1

            right -= 1

    if k <= right:
        self.quick_select(freq_list, start, right, k)
    elif k >= left:
        self.quick_select(freq_list, left, end, k)

#####

```

```

...
>>> x = [1, 2, 3]
>>> x.append([4, 5])
>>> print(x)
[1, 2, 3, [4, 5]]

>>> x = [1, 2, 3]
>>> x.extend([4, 5])

>>> print(x)
[1, 2, 3, 4, 5]
...
class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        freq = Counter(nums)

        # not always filled, max possible frequency is len(nums) [1,1,1,1,...]
        buckets = [[] for _ in range(len(nums) + 1)]
        for num, freq in freq.items():

            buckets[freq].append(num)

        res = []
        for bucket in reversed(buckets):
            if bucket:
                res.extend(bucket)
                if len(res) >= k:
                    break

        return res[:k]

#####

class Solution(object):
    def topKFrequent(self, nums, k):
        """
        :type nums: List[int]
        :type k: int
        :rtype: List[int]
        """

```

```
d = {}
res = []
ans = []
buckets = [[] for _ in range(len(nums) + 1)]

for num in nums:
    d[num] = d.get(num, 0) + 1

for key in d:
    res.append((d[key], key))

for t in res:
    freq, key = t
    buckets[freq].append(key)

buckets.reverse()

for item in buckets:
    if item and k > 0:
        while item and k > 0:
            ans.append(item.pop())
            k -= 1
        if k == 0:
            return ans

return ans
```

◆ Quick Review

Key Insights

- Count the frequency of each element using a hash table.
- Use bucket sort to achieve linear time complexity by grouping numbers by their frequency.

- Traverse the frequency buckets from highest to lowest to collect the k most frequent elements.
- Alternative approaches include using heaps or Quickselect, but bucket sort meets the follow-up requirement of better than $O(n \log n)$ time complexity.

Space & Time

Time Complexity: $O(n)$ where n is the length of the array (bucket sort and frequency counting are linear).

Space Complexity: $O(n)$ for storing the counts and buckets.

Solution

The solution starts by counting the frequency of each element with a hash table (or dictionary). Then, we create an array of buckets where the index represents the frequency. Each bucket holds the numbers that appear with that frequency. Finally, we iterate over the buckets in reverse order (from high frequency to low) and collect elements until we have k of them. This bucket sort approach avoids the higher complexity of sorting a list

of frequencies and guarantees linear time performance.

Heap

□ ★ #505 The Maze II ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Graph · Matrix · Heap ·
Shortest Path

Lists: ★ Premium | Premium 98

Problem

There is a ball in a maze with empty spaces (represented as 0) and walls (represented as 1). The ball can go through the empty spaces by rolling up, down, left or right, but it won't stop rolling until hitting a wall. When the ball stops, it could choose the next direction.

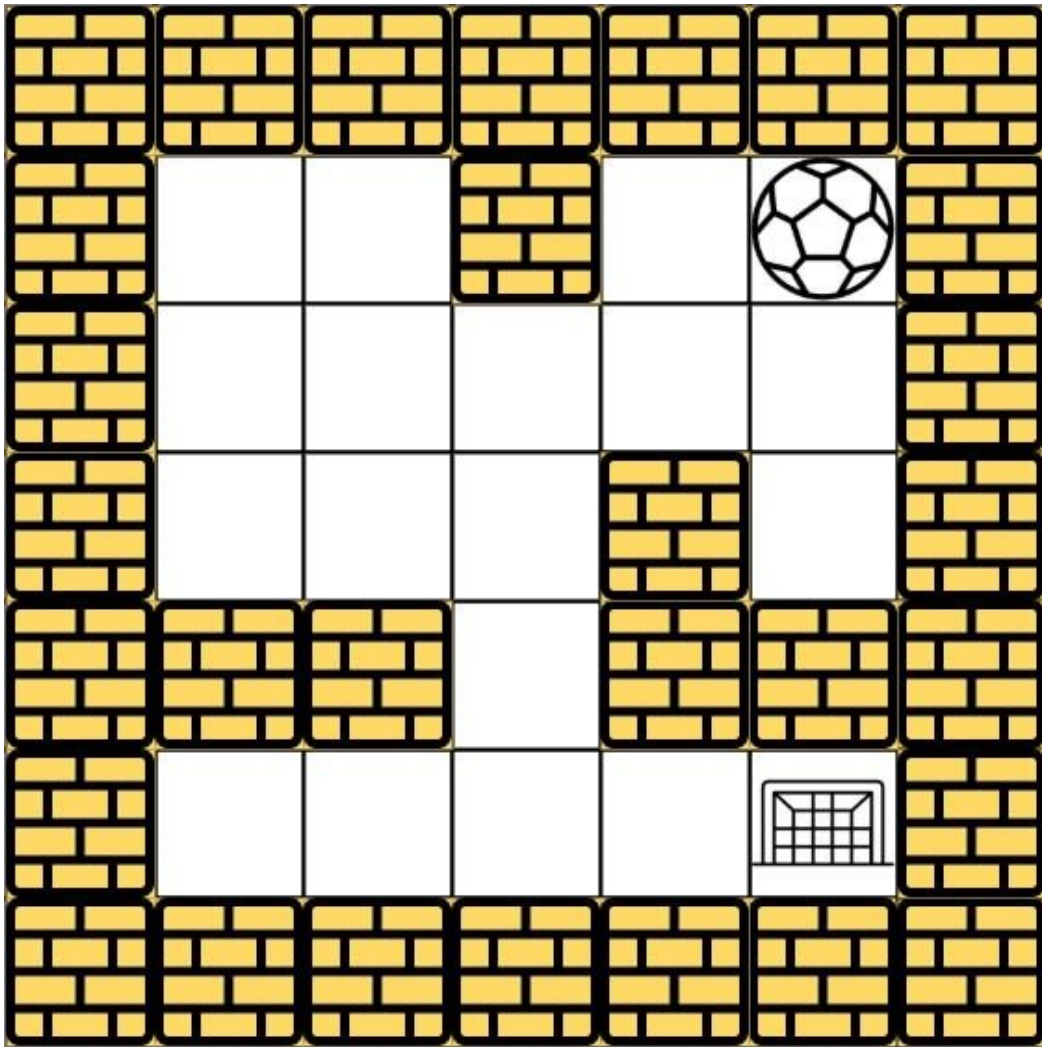
Given the $m \times n$ maze, the ball's start position and the destination, where $\text{start} = [\text{startrow}, \text{startcol}]$ and $\text{destination} = [\text{destinationrow}, \text{destinationcol}]$, return the shortest distance for the ball to stop at the destination. If the ball cannot stop at destination, return -1 .

The distance is the number of empty spaces traveled by the ball from the start position

(excluded) to the destination (included).

You may assume that the borders of the maze are all walls (see examples).

Example 1:



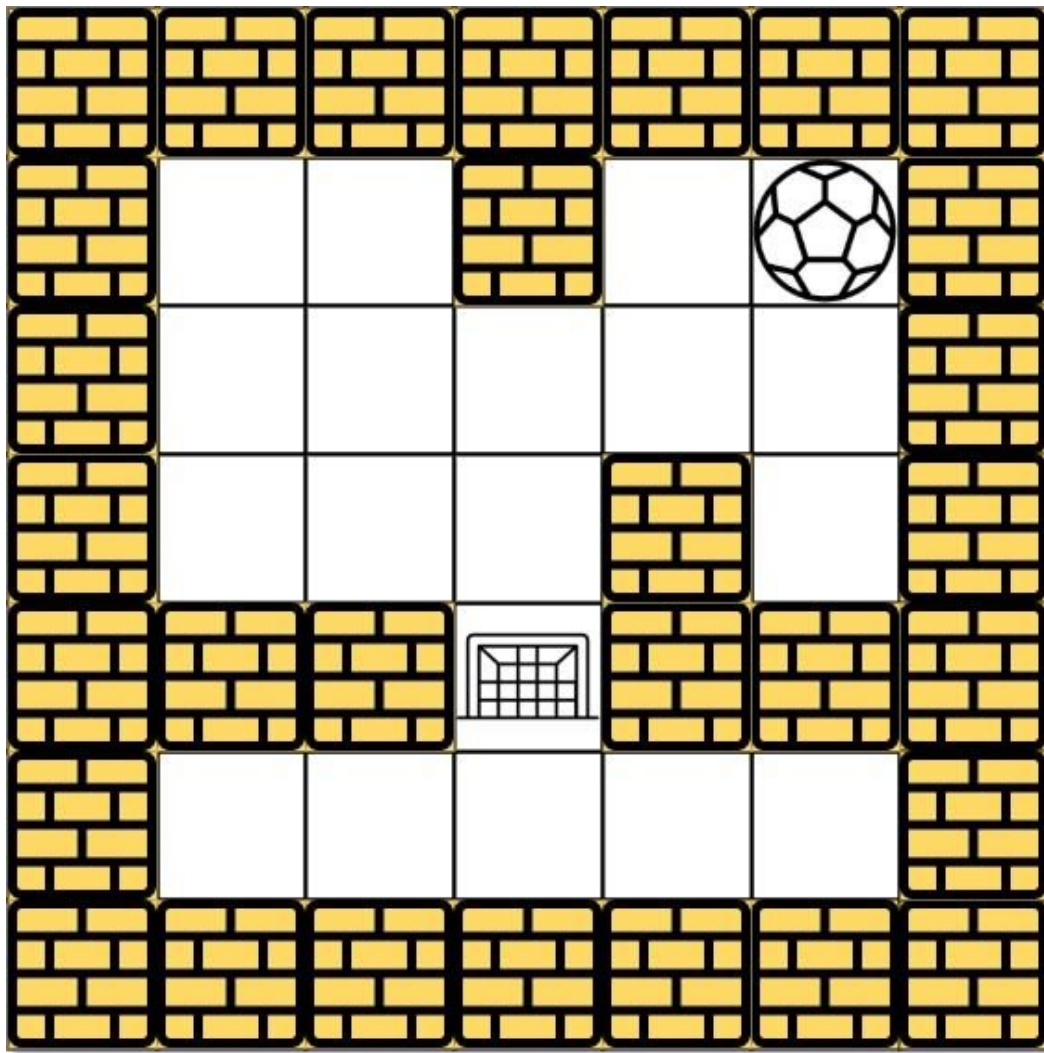
Input: maze = `[[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]]`,
start = `[0,4]`, destination = `[4,4]`

Output: 12

Explanation: One possible way is : left -> down -> left -> down -> right -> down -> right.

The length of the path is $1 + 1 + 3 + 1 + 2 + 2 + 2 = 12$.

Example 2:



Input: maze = `[[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]]`,
 start = `[0,4]`, destination = `[3,2]`

Output: -1

Explanation: There is no way for the ball to stop at the destination. Notice that you can pass through the destination but you cannot stop there.

Example 3:

Input: maze = `[[0,0,0,0,0],[1,1,0,0,1],[0,0,0,0,0],[0,1,0,0,1],[0,1,0,0,0]]`,
 start = `[4,3]`, destination = `[0,1]`

Output: -1

Constraints:

- `m == maze.length`
- `n == maze[i].length`

- $1 \leq m, n \leq 100$
- `maze[i][j]` is 0 or 1.
- `start.length == 2`
- `destination.length == 2`
- $0 \leq \text{startrow}, \text{destinationrow} < m$
- $0 \leq \text{startcol}, \text{destinationcol} < n$
- Both the ball and the destination exist in an empty space, and they will not be in the same position initially.
- The maze contains at least 2 empty spaces.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    import heapq

    def shortestDistance(self, maze, start, destination):
        m, n = len(maze), len(maze[0])
        # Initialize distance matrix with infinity
        dist = [[float('inf')] * n for _ in range(m)]
        dist[start[0]][start[1]] = 0
        # Priority queue (min-heap) storing tuples (distance, row, col)
        heap = [(0, start[0], start[1])]
```

```

# Directions: up, down, left, right
directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

while heap:
    d, row, col = heapq.heappop(heap)
    # If we have reached the destination, return the corresponding
distance
    if [row, col] == destination:
        return d
    # Skip if we already found a better path to (row, col)
    if d > dist[row][col]:
        continue
    for dr, dc in directions:
        next_row, next_col, count = row, col, 0
        # Roll the ball until it hits a wall
        while 0 <= next_row + dr < m and 0 <= next_col + dc < n and ma
ze[next_row + dr][next_col + dc] == 0:
            next_row += dr
            next_col += dc
            count += 1
        # Check if the new distance is shorter
        if d + count < dist[next_row][next_col]:
            dist[next_row][next_col] = d + count
            heapq.heappush(heap, (dist[next_row][next_col], next_row,
next_col))
    return -1

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

A ball rolls in a maze until hitting a wall. Find the SHORTEST DISTANCE

(number of empty spaces traveled) for the ball to stop exactly at destination.

Return -1 if impossible. Right?"

#

"A few quick questions:

Ball must STOP at destination – passing through doesn't count? Correct.

Distance counts steps of empty cells traversed (destination excluded from count? No, included per problem)."

#

"Let me walk: maze 5x5, start=(0,4), dest=(4,4): shortest path = 12 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

BFS exploring all rolling directions from each stop point.

Track distances per stop position; expand in BFS order.

BFS doesn't guarantee shortest distance when edge weights vary – that's the issue.

Should I go ahead and optimize?"

class _505_MazeII_Brute:

def shortestDistance(self, maze, start, destination):

from collections import deque

m, n = len(maze), len(maze[0])

dist = [[float('inf')] * n for _ in range(m)]

dist[start[0]][start[1]] = 0

queue = deque([start])

while queue:

 r, c = queue.popleft()

 for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:

 nr, nc, d = r, c, 0

 while 0<=nr+dr<m and 0<=nc+dc<n and not maze[nr+dr][nc+dc]:

 nr += dr; nc += dc; d += 1

 if dist[r][c] + d < dist[nr][nc]:

 dist[nr][nc] = dist[r][c] + d; queue.append((nr,nc))

 return dist[destination[0]][destination[1]] if

dist[destination[0]][destination[1]] != float('inf') else -1

PHASE 3 — OPTIMIZE

"The bottleneck is that BFS doesn't account for varying rolling distances.

Dijkstra processes stop points in order of distance – guarantees the first time

we reach a stop is via the shortest path.

#

Time: $O(m*n*\log(m*n)*(m+n))$. Space: $O(m*n)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

import heapq

class _505_MazeII:

def shortestDistance(self, maze, start, destination):

m, n = len(maze), len(maze[0])

dist = [[float('inf')] * n for _ in range(m)]

dist[start[0]][start[1]] = 0

heap = [(0, start[0], start[1])]

while heap:

 d, r, c = heapq.heappop(heap)

 if d > dist[r][c]:

 continue

 for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:

 nr, nc, steps = r, c, 0

 while 0<=nr+dr<m and 0<=nc+dc<n and not maze[nr+dr][nc+dc]:

 nr += dr; nc += dc; steps += 1

 if d + steps < dist[nr][nc]:

 dist[nr][nc] = d + steps

```

        heapq.heappush(heap, (dist[nr][nc], nr, nc))
    result = dist[destination[0]][destination[1]]
    return result if result != float('inf') else -1

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

Ball rolls to nearest wall stop; Dijkstra picks shortest accumulated distance each step.

Destination=(4,4): after exploring all paths, shortest is 12 ✓

Unreachable destination: dist stays inf → return -1 ✓

#

"No bugs. The 'd > dist[r][c]' check skips stale heap entries efficiently."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n*(m+n)*\log(m*n))$. Space $O(m*n)$ dist + heap.

BFS only works for unit-weight edges – Dijkstra is needed here because rolling varies.

Follow-up: Maze I (#490) – just reachability, BFS suffices.

Follow-up: Maze III (#499) – same but return lex-smallest direction string on ties."

Community Solutions (Python)

• LeetDoocs

```

class Solution:
    def shortestDistance(
        self, maze: List[List[int]], start: List[int], destination: List[int]
    ) -> int:
        m, n = len(maze), len(maze[0])
        dirs = (-1, 0, 1, 0, -1)
        si, sj = start
        di, dj = destination
        q = deque([(si, sj)])
        dist = [[inf] * n for _ in range(m)]

```

```

    dist[si][sj] = 0
    while q:
        i, j = q.popleft()
        for a, b in pairwise(dirs):
            x, y, k = i, j, dist[i][j]
            while 0 <= x + a < m and 0 <= y + b < n and maze[x + a][y + b]
== 0:
                x, y, k = x + a, y + b, k + 1
                if k < dist[x][y]:
                    dist[x][y] = k
                    q.append((x, y))

    return -1 if dist[di][dj] == inf else dist[di][dj]

```

• SimplyLeet

```

import heapq

def shortestDistance(maze, start, destination):
    m, n = len(maze), len(maze[0])
    # Initialize distance matrix with infinity
    dist = [[float('inf')] * n for _ in range(m)]
    dist[start[0]][start[1]] = 0
    # Priority queue (min-heap) storing tuples (distance, row, col)
    heap = [(0, start[0], start[1])]
    # Directions: up, down, left, right

    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    while heap:
        d, row, col = heapq.heappop(heap)
        # If we have reached the destination, return the corresponding distance
        if [row, col] == destination:
            return d
        # Skip if we already found a better path to (row, col)
        if d > dist[row][col]:
            continue

```

```

        for dr, dc in directions:
            next_row, next_col, count = row, col, 0
            # Roll the ball until it hits a wall
            while 0 <= next_row + dr < m and 0 <= next_col + dc < n and maze[n
ext_row + dr][next_col + dc] == 0:
                next_row += dr
                next_col += dc
                count += 1
            # Check if the new distance is shorter
            if d + count < dist[next_row][next_col]:
                dist[next_row][next_col] = d + count

            heapq.heappush(heap, (dist[next_row][next_col], next_row, next
_col))
        return -1

```

• LC.ca

```

class Solution:
    def shortestDistance(
        self, maze: List[List[int]], start: List[int], destination: List[int]
    ) -> int:
        m, n = len(maze), len(maze[0])
        dirs = (-1, 0, 1, 0, -1)
        si, sj = start
        di, dj = destination
        q = deque([(si, sj)])
        dist = [[inf] * n for _ in range(m)]

        dist[si][sj] = 0
        while q:
            i, j = q.popleft()
            for a, b in pairwise(dirs):
                x, y, k = i, j, dist[i][j]
                while 0 <= x + a < m and 0 <= y + b < n and maze[x + a][y + b]
== 0:
                    x, y, k = x + a, y + b, k + 1
                    if k < dist[x][y]:
                        dist[x][y] = k
                        q.append((x, y))

        return -1 if dist[di][dj] == inf else dist[di][dj]

```

◆ Quick Review

Key Insights

- The ball rolls until it hits a wall, so each move is not one step but a continuous travel in a direction.
- The problem can be modeled as finding the shortest path in a weighted graph where nodes are positions in the maze and edge weights are the distance traveled in that roll.
- A priority queue (min-heap) based algorithm, such as Dijkstra's algorithm, is ideal since each roll covers variable distances.
- Always check if the new computed distance for a cell is less than a previously recorded one before pushing it to the heap.

Space & Time

Time Complexity: $O(m * n * \log(m * n))$, where m and n are the maze dimensions (each cell may get processed and queued with a logarithmic cost).

Space Complexity: $O(m * n)$ for maintaining the distance matrix and the priority queue.

Solution

We solve the problem using a modified version of Dijkstra's algorithm. The maze grid is treated as a graph where each cell is a node and an edge exists between the current cell and the cell where the ball stops after rolling in one direction. For every cell popped from the priority queue, we roll the ball in all four directions until it hits a wall, counting the number of steps taken. If the distance calculated from this roll is shorter than a previously stored distance for the stopping cell, we update the distance and add it to the priority queue. This process continues until we either reach the destination (and return the distance) or exhaust all possibilities (returning -1).

Heap

□ #621 Task Scheduler

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Greedy · Heap · Counting
Lists: Grind 169

Problem

You are given an array of CPU tasks, each labeled with a letter from A to Z, and a number n . Each CPU interval can be idle or allow the completion of one task. Tasks can be completed in any order, but there's a constraint: there has to be a gap of at least n intervals between two tasks with the same label.

Return the minimum number of CPU intervals required to complete all tasks.

Example 1:

Input: tasks = ["A","A","A","B","B","B"], $n = 2$

Output: 8

Explanation: A possible sequence is: A -> B -> idle -> A -> B -> idle -> A -> B.

After completing task A, you must wait two intervals before doing A again. The same applies to task B. In the 3rd interval, neither A nor B can be done, so you idle. By the 4th interval, you can do A again as 2 intervals have passed.

Example 2:

Input: tasks = ["A","C","A","B","D","B"], n = 1

Output: 6

Explanation: A possible sequence is: A -> B -> C -> D -> A -> B.

With a cooling interval of 1, you can repeat a task after just one other task.

Example 3:

Input: tasks = ["A","A","A", "B","B","B"], n = 3

Output: 10

Explanation: A possible sequence is: A -> B -> idle -> idle -> A -> B -> idle -> idle -> A -> B.

There are only two types of tasks, A and B, which need to be separated by 3 intervals. This leads to idling twice between repetitions of these tasks.

Constraints:

- $1 \leq \text{tasks.length} \leq 104$
- tasks[i] is an uppercase English letter.

- $0 \leq n \leq 100$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def leastInterval(self, tasks: List[str], n: int) -> int:
        count = collections.Counter(tasks)
        maxFreq = max(count.values())
        # Put the most frequent task in the slot first.
        maxFreqTaskOccupy = (maxFreq - 1) * (n + 1)
        # Get the number of tasks with same frequency as maxFreq, we'll append them
        # after the
        # `maxFreqTaskOccupy`.
        nMaxFreq = sum(value == maxFreq for value in count.values())
        # max(

        # the most frequent task is frequent enough to force some idle slots,
        # the most frequent task is not frequent enough to force idle slots
        # )
        return max(maxFreqTaskOccupy + nMaxFreq, len(tasks))
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# CPU tasks with a cooldown n: same task must wait at least n intervals.
# CPU can idle. Return the minimum total intervals to complete all tasks. Right?"
#
# "A few quick questions:
# n can be 0 — meaning no cooldown, just run tasks back to back? Yes.
# The specific order we choose tasks is up to us — we want to minimize total slots."
#
# "Let me walk: tasks=['A','A','A','B','B','B'], n=2.
# A→B→idle→A→B→idle→A→B = 8 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Simulate with a max-heap and a cooldown queue:
# each cycle pop the most frequent, place it, decrement count, push back after
# cooldown.
# Pad with idles if heap is empty mid-cycle.
# O(total_time * log 26) time — but there's a math shortcut.
```

```
# Should I go ahead and optimize?"
class _621_TaskScheduler_Brute:
    def leastInterval(self, tasks, n):
        from collections import Counter
        import heapq
        count = Counter(tasks)
        heap = [-c for c in count.values()]
        heapq.heapify(heap)
        time = 0
        while heap:
            cycle = []; slots = n + 1
            while slots and heap:
                cycle.append(heapq.heappop(heap)); slots -= 1; time += 1
            if heap: time += slots
            for c in cycle:
                if c + 1 < 0: heapq.heappush(heap, c + 1)
        return time
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the simulation – but there's a closed-form  $O(n)$  formula.
# Let max_freq = highest task frequency, max_count = tasks with that frequency.
# The optimal schedule slots tasks into (max_freq - 1) frames of (n+1) each,
# plus max_count tasks in the final partial frame.
# Result = max(len(tasks), (max_freq - 1) * (n + 1) + max_count).
# The max() handles the case where tasks fill all slots without any idle.
#
# Time:  $O(n)$ . Space:  $O(1)$  – just counting."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _621_TaskScheduler:
    def leastInterval(self, tasks, n):
        from collections import Counter
        counts = Counter(tasks).values()
        max_freq = max(counts)
        max_count = sum(1 for c in counts if c == max_freq)
        return max(len(tasks), (max_freq - 1) * (n + 1) + max_count)
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# tasks=['A'*3,'B'*3], n=2: max_freq=3, max_count=2.
#  $(3-1)*(2+1)+2 = 2*3+2 = 8$ . len=6. max(6,8)=8 ✓
#
# tasks=['A'*3,'B'*3,'C'*3], n=2: max_freq=3, max_count=3.
#  $(3-1)*3+3=9$ . len=9. max(9,9)=9 ✓ (ABCABCABC, no idles)
#
# tasks=['A'*3,'B'*3], n=0:  $(3-1)*1+2=4$ . len=6. max(6,4)=6 ✓
#
# "No bugs. The formula captures both the idle-constrained and task-filled cases."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Formula $O(|tasks|)$: just count frequencies. Heap simulation $O(m*n)$ – much slower.
 # The formula works because beyond the 'frame' structure, extra tasks always fit without new idles.

Follow-up: Rearrange String k Distance Apart (#358) – return the actual string, not just count.

Follow-up: if tasks have different durations, this becomes a weighted scheduling problem."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def leastInterval(self, tasks: list[str], n: int) -> int:
        count = collections.Counter(tasks)
        maxFreq = max(count.values())
        # Put the most frequent task in the slot first.
        maxFreqTaskOccupy = (maxFreq - 1) * (n + 1)
        # Get the number of tasks with same frequency as maxFreq, we'll append them
        # after the
        # `maxFreqTaskOccupy`.
        nMaxFreq = sum(value == maxFreq for value in count.values())
        # max(

        # the most frequent task is frequent enough to force some idle slots,
        # the most frequent task is not frequent enough to force idle slots
        # )
        return max(maxFreqTaskOccupy + nMaxFreq, len(tasks))
```

• LeetDoocs

```
class Solution:
    def leastInterval(self, tasks: List[str], n: int) -> int:
        cnt = Counter(tasks)
        x = max(cnt.values())
        s = sum(v == x for v in cnt.values())
        return max(len(tasks), (x - 1) * (n + 1) + s)
```

• SimplyLeet

```
# Python solution for Task Scheduler
from collections import Counter

def leastInterval(tasks, n):
    # Count the frequency of each task
    task_counts = Counter(tasks)
    # Find the maximum frequency among tasks
    max_freq = max(task_counts.values())
    # Count how many tasks have the maximum frequency
    max_count = sum(1 for count in task_counts.values() if count == max_freq)

    # Calculate the minimum intervals required based on the most frequent tasks
    part_count = max_freq - 1                # Number of parts between the
most frequent task
    part_length = n - (max_count - 1)         # Effective empty slots per
part after placing max frequency tasks
    empty_slots = part_count * part_length    # Total empty slots to fill
with idle or other tasks
    available_tasks = len(tasks) - max_freq * max_count # Remaining tasks to
fill the empty slots
    idles = max(0, empty_slots - available_tasks) # Remaining idle slots if
available tasks are not enough

    # The total intervals is the sum of all tasks and idle slots inserted
    return len(tasks) + idles

# Example usage:
if __name__ == "__main__":
    tasks1 = ["A","A","A","B","B","B"]
    n1 = 2
    print(leastInterval(tasks1, n1)) # Output: 8
```

• LC.ca

```
class Solution:
    def leastInterval(self, tasks: List[str], n: int) -> int:
        cnt = Counter(tasks)
        x = max(cnt.values())
        s = sum(v == x for v in cnt.values())
        return max(len(tasks), (x - 1) * (n + 1) + s)
```

◆ Quick Review

Key Insights

- Count the frequency of each task.
- The task with the highest frequency determines the base structure.
- Identify how many tasks share the maximum frequency.
- Calculate the total time based on the formula: $\max(\text{total tasks}, (\text{max frequency} - 1) * (n + 1) + \text{count of tasks with maximum frequency})$.
- This approach avoids simulating the entire scheduling by using a greedy strategy.

Space & Time

Time Complexity: $O(N)$ where N is the number of tasks (counting frequency, then iterating over constant 26 letters).

Space Complexity: $O(1)$ since the frequency count uses an array or hash map with fixed size (26 letters).

Solution

We first count the frequency of each task. The task(s) with the maximum frequency establish

the minimum structure required. Compute the ideal idle slots based on the count of these maximum tasks and the cooling period n . Finally, compare this computed length with the total number of tasks to get the final answer. This greedy strategy avoids explicit simulation and leverages mathematical formulation.

Heap

□ #658 Find K Closest Elements

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Binary Search · Sorting · Heap · Two
Pointers

Lists: Grind 169

Problem

Given a sorted integer array `arr`, two integers `k` and `x`, return the `k` closest integers to `x` in the array. The result should also be sorted in ascending order.

An integer `a` is closer to `x` than an integer `b` if:

- $|a - x| < |b - x|$, or
- $|a - x| == |b - x|$ and $a < b$

Example 1:

Input: `arr = [1,2,3,4,5]`, `k = 4`, `x = 3`

Output: `[1,2,3,4]`

Example 2:

Input: `arr = [1,1,2,3,4,5]`, `k = 4`, `x = -1`

Output: `[1,1,2,3]`

Constraints:

- $1 \leq k \leq \text{arr.length}$
- $1 \leq \text{arr.length} \leq 104$
- arr is sorted in ascending order.
- $-104 \leq \text{arr}[i], x \leq 104$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def findClosestElements(self, arr: list[int], k: int, x: int) -> list[int]:
        l = 0
        r = len(arr) - k

        while l < r:
            m = (l + r) // 2
            if x - arr[m] <= arr[m + k] - x:
                r = m
            else:
                l = m + 1

        return arr[l:l + k]
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Given a sorted array, find the k closest integers to x.
# Ties broken by smaller value. Return the result sorted. Right?"
#
# "A few quick questions:
# k is always valid ( $1 \leq k \leq \text{len(arr)}$ )? Yes. x may not be in the array? Yes."
#
# "Let me walk: arr=[1,2,3,4,5], k=4, x=3 → [1,2,3,4] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic."
```

```

# Sort by |val - x| (then by val for ties), take first k, re-sort.
# O(n log n) - discards the sorted structure of the input.
# Should I go ahead and optimize?"
class _658_FindKClosestElements_Brute:
    def findClosestElements(self, arr, k, x):
        return sorted(sorted(arr, key=lambda v: (abs(v - x), v))[:k])

# PHASE 3 — OPTIMIZE
# "The bottleneck is sorting all n elements when the answer is always a contiguous
window.
# Binary search on the LEFT boundary of the k-element window:
# Search lo in [0, n-k]: the window starts at lo.
# Compare arr[mid] and arr[mid+k]: if arr[mid+k]-x < x-arr[mid], shift window right.
# When lo == hi, return arr[lo:lo+k].
#
# Time: O(log(n-k) + k). Space: O(1)."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _658_FindKClosestElements:
    def findClosestElements(self, arr, k, x):
        lo, hi = 0, len(arr) - k
        while lo < hi:
            mid = (lo + hi) // 2
            if x - arr[mid] > arr[mid + k] - x:
                lo = mid + 1
            else:
                hi = mid
        return arr[lo:lo + k]

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# arr=[1,2,3,4,5], k=4, x=3: lo=0,hi=1.
# mid=0: x-arr[0]=3-1=2, arr[4]-x=5-3=2. 2>2 False → hi=0. lo=hi=0. return [1,2,3,4]
✓
#
# arr=[1,2,3,4,5], k=4, x=-1: lo=0,hi=1.
# mid=0: x-arr[0]=-1-1=-2 (negative, so < arr[4]-x=6). -2>6 False → hi=0. return
[1,2,3,4] ✓
#
# "No bugs. The comparison x-arr[mid] > arr[mid+k]-x checks which side is farther."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Binary search O(log(n-k)): find left boundary. Slice O(k).
# The key insight: the answer is always a contiguous window of length k in the
sorted array.
# Follow-up: if array is unsorted, sort first O(n log n) or use a max-heap O(n log
k).
# Follow-up: same array, many queries - binary search is O(log n) per query, no
precompute needed."

```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findClosestElements(self, arr: list[int], k: int, x: int) -> list[int]:
        l = 0
        r = len(arr) - k

        while l < r:
            m = (l + r) // 2
            if x - arr[m] <= arr[m + k] - x:
                r = m
            else:
                l = m + 1

        return arr[l:l + k]
```

• LeetDoocs

```
class Solution:
    def findClosestElements(self, arr: List[int], k: int, x: int) -> List[int]:
        arr.sort(key=lambda v: abs(v - x))
        return sorted(arr[:k])
```

• SimplyLeet

```
# Function to find k closest elements to x in a sorted array
def findClosestElements(arr, k, x):
    n = len(arr)
    # Initialize the binary search bounds for the starting index of the window
    left, right = 0, n - k
    # Binary search to find the best starting index
    while left < right:
        mid = left + (right - left) // 2
        # Compare the distance from x to arr[mid] and arr[mid+k]
        # If arr[mid] is farther from x than arr[mid+k], move window to the
        right
```

```

    if x - arr[mid] > arr[mid + k] - x:
        left = mid + 1
    else:
        # Otherwise, continue searching towards the left
        right = mid
    # Return the k elements starting from index `left`
    return arr[left:left + k]

# Example usage:
# arr = [1,2,3,4,5], k = 4, x = 3

# print(findClosestElements(arr, 4, 3)) # Output: [1,2,3,4]

```

• LC.ca

```

class Solution:
    def findClosestElements(self, arr: List[int], k: int, x: int) -> List[int]:
        left, right = 0, len(arr) - k
        while left < right:
            mid = (left + right) >> 1
            if x - arr[mid] <= arr[mid + k] - x:
                right = mid
            else:
                left = mid + 1
        return arr[left : left + k]

```

◆ Quick Review

Key Insights

- The input array is already sorted.
- A binary search can be used to efficiently narrow down the potential starting position.
- A two-pointers or sliding window approach is effective for selecting k consecutive elements.

- The comparison of absolute differences (with a tie-breaker on values) is crucial.
- Maintaining ascending order simplifies the final output.

Space & Time

Time Complexity: $O(\log n + k)$, where $O(\log n)$ for binary search and $O(k)$ for collecting results.

Space Complexity: $O(k)$, for storing the output.

Solution

We can solve this problem by first using binary search to identify the left-most window where the k closest elements might start. The binary search operates over the possible starting indices for windows of size k (from 0 to $n-k$). For each middle index mid , we compare the distances between x and $arr[mid]$ and $arr[mid + k]$ to decide whether to move the window to the right or left. Once the correct starting position is found, return the k elements starting from that index. This approach leverages the sorted property of the array to efficiently converge on the optimal window.

Heap

□ #787 Cheapest Flights Within K Stops

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming · DFS · BFS · Graph ·
Heap

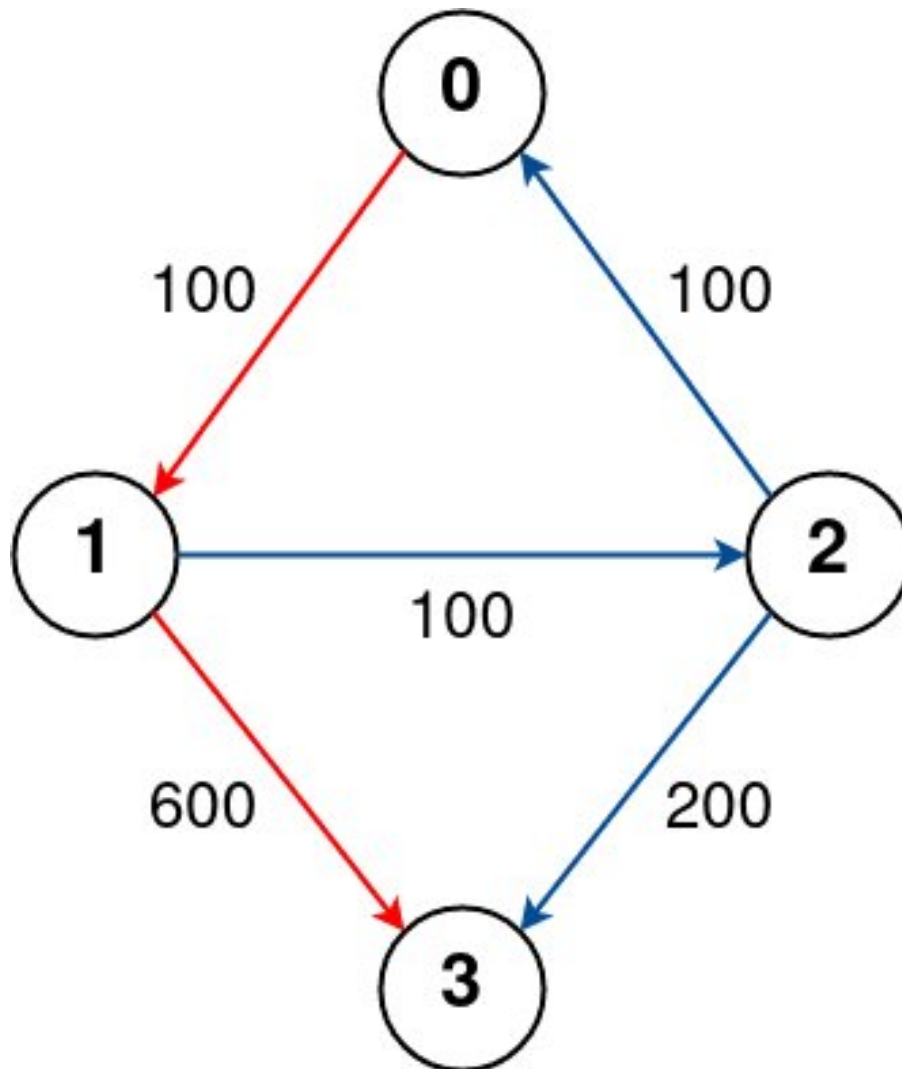
Lists: Grind 169

Problem

There are n cities connected by some number of flights. You are given an array `flights` where `flights[i] = [fromi, toi, pricei]` indicates that there is a flight from city `fromi` to city `toi` with cost `pricei`.

You are also given three integers `src`, `dst`, and `k`, return the cheapest price from `src` to `dst` with at most `k` stops. If there is no such route, return `-1`.

Example 1:



Input: $n = 4$, $flights = [[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]]$,
 $src = 0$, $dst = 3$, $k = 1$

Output: 700

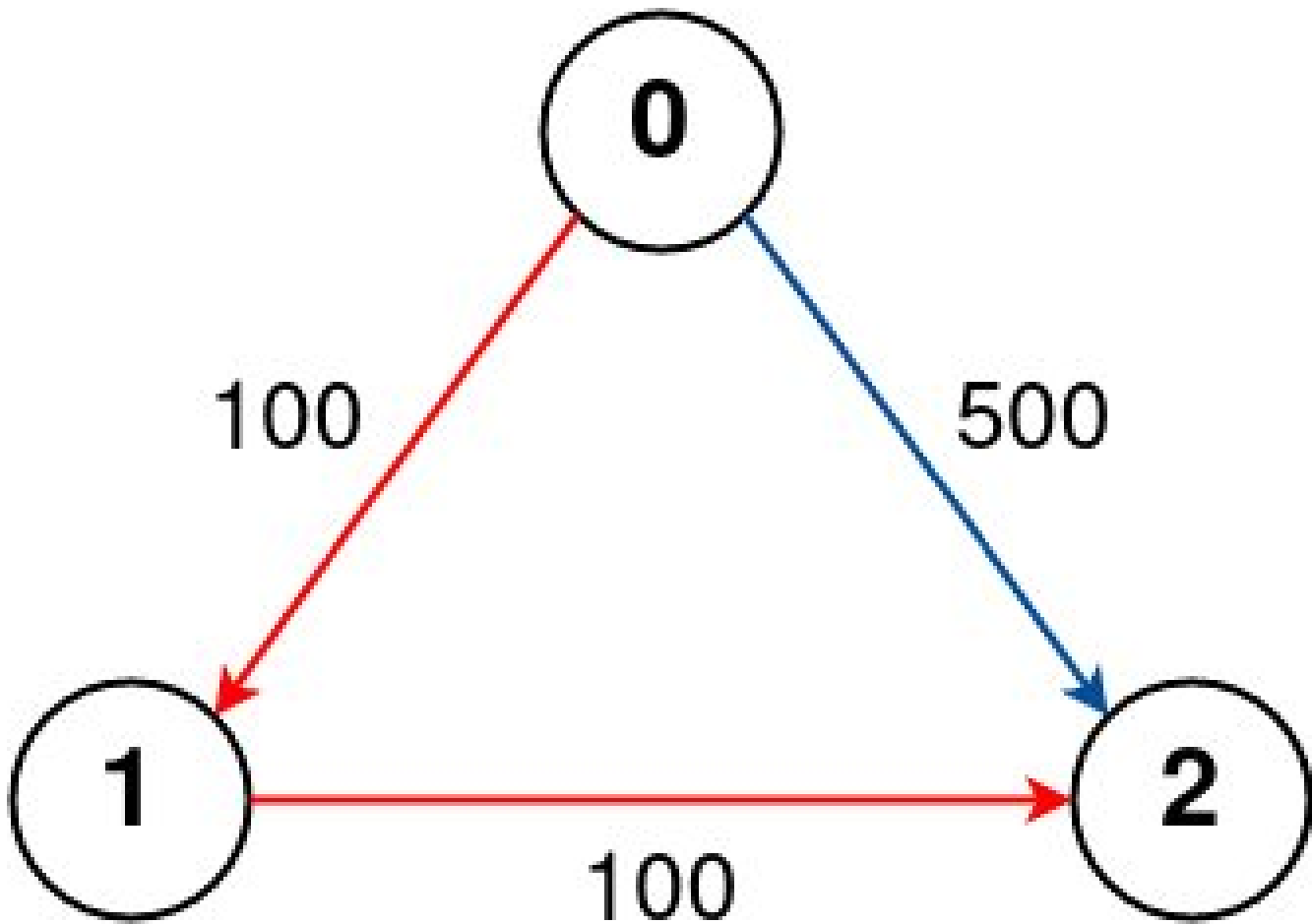
Explanation:

The graph is shown above.

The optimal path with at most 1 stop from city 0 to 3 is marked in red and has cost $100 + 600 = 700$.

Note that the path through cities $[0,1,2,3]$ is cheaper but is invalid because it uses 2 stops.

Example 2:



Input: $n = 3$, $flights = [[0,1,100],[1,2,100],[0,2,500]]$, $src = 0$, $dst = 2$, $k = 1$

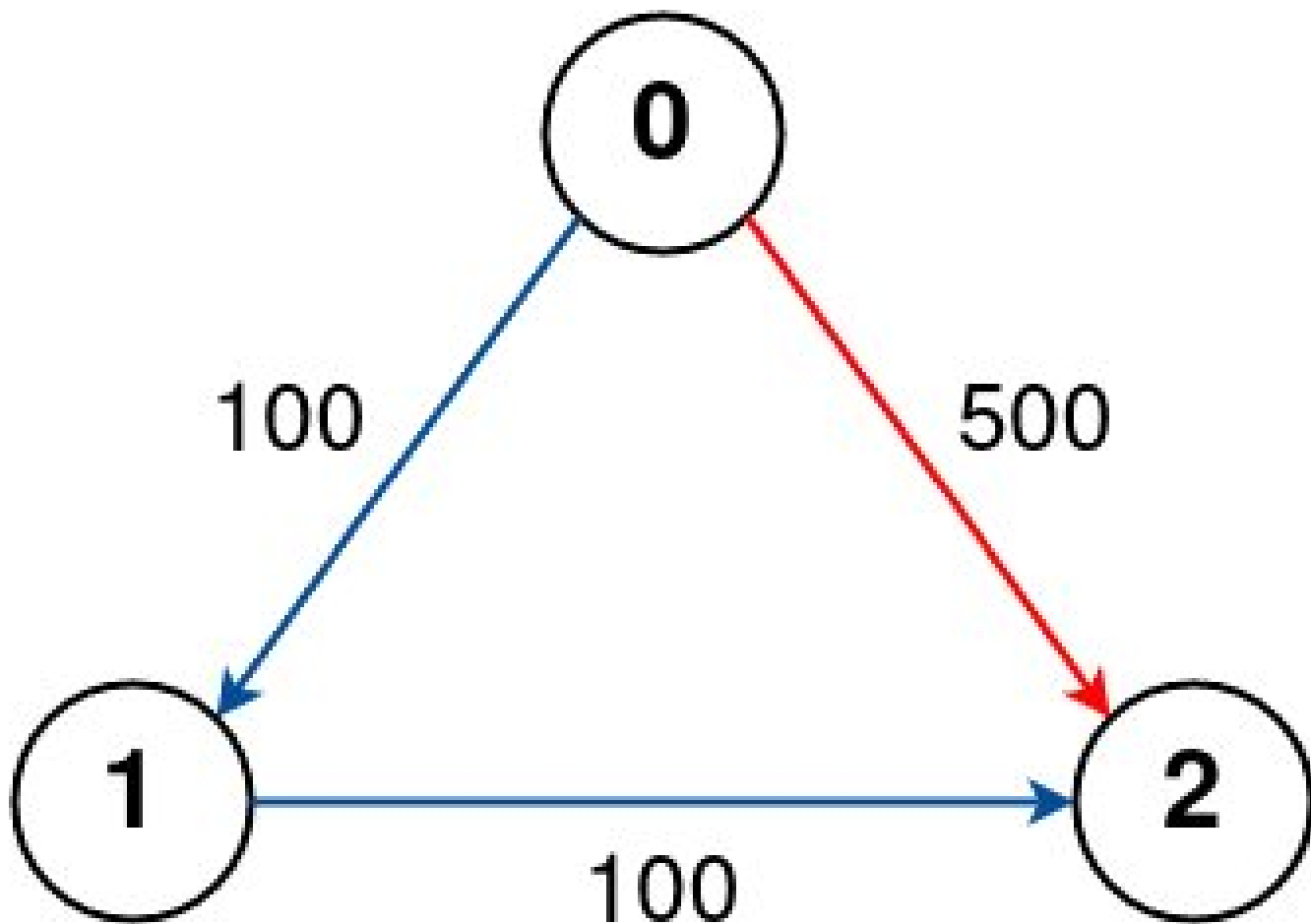
Output: 200

Explanation:

The graph is shown above.

The optimal path with at most 1 stop from city 0 to 2 is marked in red and has cost $100 + 100 = 200$.

Example 3:



Input: $n = 3$, $flights = [[0,1,100],[1,2,100],[0,2,500]]$, $src = 0$, $dst = 2$, $k = 0$

Output: 500

Explanation:

The graph is shown above.

The optimal path with no stops from city 0 to 2 is marked in red and has cost 500.

Constraints:

- $2 \leq n \leq 100$
- $0 \leq flights.length \leq (n * (n - 1) / 2)$
- $flights[i].length == 3$
- $0 \leq from_i, to_i < n$
- $from_i \neq to_i$

- $1 \leq \text{price}_i \leq 104$
- There will not be any multiple flights between two cities.
- $0 \leq \text{src}, \text{dst}, k < n$
- $\text{src} \neq \text{dst}$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def findCheapestPrice(
        self,
        n: int,
        flights: list[list[int]],
        src: int,
        dst: int,
        k: int,
    ) -> int:
        graph = [[] for _ in range(n)]

        for u, v, w in flights:
            graph[u].append((v, w))

        return self._dijkstra(graph, src, dst, k)

    def _dijkstra(
        self,
        graph: list[list[tuple[int, int]]],
        src: int,
```

```

        dst: int,
        k: int,
    ) -> int:
        dist = [[math.inf] * (k + 2) for _ in range(len(graph))]

        dist[src][k + 1] = 0
        minHeap = [(dist[src][k + 1], src, k + 1)] # (d, u, stops)

        while minHeap:
            d, u, stops = heapq.heappop(minHeap)

            if u == dst:
                return d
            if stops == 0 or d > dist[u][stops]:
                continue
            for v, w in graph[u]:
                if d + w < dist[v][stops - 1]:
                    dist[v][stops - 1] = d + w
                    heapq.heappush(minHeap, (dist[v][stops - 1], v, stops - 1))

        return -1

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

n cities, flights with prices. Find cheapest price from src to dst with at most k stops.

A stop is an intermediate city – so at most k+1 edges. Return -1 if no route. Right?"

#

"A few quick questions:

Can there be cycles? Yes. Multiple flights between same cities? No per constraints.

k can be 0 (direct flight only)?"

#

"Let me walk: n=4, src=0, dst=3, k=1.

0→1→3 costs 700 (1 stop ✓). 0→1→2→3 costs 400 but 2 stops > k=1. → 700 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

DFS/BFS exploring all paths up to k+1 edges. Track minimum cost to reach dst.

O(n^k) worst case – exponential.

Should I go ahead and optimize?"

class _787_CheapestFlightsKStops_Brute:

```
def findCheapestPrice(self, n, flights, src, dst, k):
    from collections import defaultdict
    graph = defaultdict(list)
    for u, v, w in flights: graph[u].append((v, w))
    best = float('inf')
    queue = [(0, src, 0)]
    while queue:
        cost, node, stops = queue.pop(0)
        if node == dst: best = min(best, cost); continue
        if stops > k: continue
        for nxt, w in graph[node]:
            queue.append((cost + w, nxt, stops + 1))
    return best if best < float('inf') else -1
```

PHASE 3 — OPTIMIZE

"The bottleneck is the exponential DFS. Bellman-Ford with $k+1$ relaxation rounds is $O(k \cdot E)$.

$\text{prices}[v]$ = min cost to reach v using at most i edges after i rounds.

Each round: update prices based on PREVIOUS round's values (use a copy to avoid same-round updates).

#

Time: $O(k * |\text{flights}|)$. Space: $O(n)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _787_CheapestFlightsKStops:
```

```
    def findCheapestPrice(self, n, flights, src, dst, k):
        INF = float('inf')
        prices = [INF] * n
        prices[src] = 0
        for _ in range(k + 1):
            prev = prices[:]
            for u, v, w in flights:
                if prev[u] != INF:
                    prices[v] = min(prices[v], prev[u] + w)
        return prices[dst] if prices[dst] != INF else -1
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

```
# n=4, src=0, dst=3, k=1,
# flights=[[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]]:
```

init: [0,INF,INF,INF].

Round1 ($k=1$ edges): $0 \rightarrow 1:100 \rightarrow \text{prices}[1]=100$. $1 \rightarrow 3:600 \rightarrow \text{prices}[3]=700$. $\text{prev}[1]=\text{INF}$ so $1 \rightarrow 2$ skipped.

$\text{prices}=[0,100,\text{INF},700]$. Round2: $\text{prev}=[0,100,\text{INF},700]$.

$1 \rightarrow 2:100+100=200 \rightarrow \text{prices}[2]=200$. $1 \rightarrow 3:100+600=700$ (same). $\rightarrow \text{prices}[\text{dst}=3]=700$ ✓

#

"No bugs. Using prev (previous round snapshot) prevents using same-round updates."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(k * E)$: $k+1$ rounds over all edges. Space $O(n)$.

Dijkstra with (cost, stops, node) heap also works – $O((E+n) \log n)$ but needs stop tracking.

Follow-up: no stop limit – standard Dijkstra or Bellman-Ford.

Follow-up: if edge weights can be negative, Bellman-Ford already handles it correctly."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findCheapestPrice(
        self,
        n: int,
        flights: list[list[int]],
        src: int,
        dst: int,
        k: int,
    ) -> int:
        graph = [[] for _ in range(n)]

        for u, v, w in flights:
            graph[u].append((v, w))

        return self._dijkstra(graph, src, dst, k)

    def _dijkstra(
        self,
        graph: list[list[tuple[int, int]]],
        src: int,
        dst: int,
        k: int,
    ) -> int:
        dist = [[math.inf] * (k + 2) for _ in range(len(graph))]

        dist[src][k + 1] = 0
        minHeap = [(dist[src][k + 1], src, k + 1)] # (d, u, stops)

        while minHeap:
            d, u, stops = heapq.heappop(minHeap)
```

```

    if u == dst:
        return d
    if stops == 0 or d > dist[u][stops]:
        continue
    for v, w in graph[u]:
        if d + w < dist[v][stops - 1]:
            dist[v][stops - 1] = d + w
            heapq.heappush(minHeap, (dist[v][stops - 1], v, stops - 1))

    return -1

```

• LeetDoocs

```

class Solution:
    def findCheapestPrice(
        self, n: int, flights: List[List[int]], src: int, dst: int, k: int
    ) -> int:
        INF = 0x3F3F3F3F
        dist = [INF] * n
        dist[src] = 0
        for _ in range(k + 1):
            backup = dist.copy()
            for f, t, p in flights:
                dist[t] = min(dist[t], backup[f] + p)
        return -1 if dist[dst] == INF else dist[dst]

```

• SimplyLeet

```

import heapq

def findCheapestPrice(n, flights, src, dst, k):
    # Build graph: each city maps to a list of (neighboring city, flight price)
    graph = {i: [] for i in range(n)}
    for from_city, to_city, price in flights:
        graph[from_city].append((to_city, price))

    # Priority queue: (total_cost, current_city, stops so far)
    heap = [(0, src, 0)]

```



```

# Dictionary to store the best cost to reach a city with a given stop count
best = {}

while heap:
    cost, city, stops = heapq.heappop(heap)
    # Return cost if destination is reached
    if city == dst:
        return cost

    # If the current number of stops exceeds k, skip further processing
    if stops > k:
        continue

    # Expand the neighboring flights
    for neighbor, price in graph[city]:
        new_cost = cost + price
        # Only proceed if this route is cheaper or not yet explored for
        # this many stops
        if (neighbor, stops + 1) not in best or new_cost < best[(neighbor,
            stops + 1)]:
            best[(neighbor, stops + 1)] = new_cost

            heapq.heappush(heap, (new_cost, neighbor, stops + 1))

return -1

# Example usage:
print(findCheapestPrice(4, [[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]]
, 0, 3, 1))

```

• LC.ca

```

class Solution:
    def findCheapestPrice(
        self, n: int, flights: List[List[int]], src: int, dst: int, k: int
    ) -> int:
        INF = 0x3F3F3F3F
        dist = [INF] * n
        dist[src] = 0
        for _ in range(k + 1):
            backup = dist.copy()
            for f, t, p in flights:

```

```
dist[t] = min(dist[t], backup[f] + p)
return -1 if dist[dst] == INF else dist[dst]
```

◆ Quick Review

Key Insights

- Use a variant of BFS combined with a min-heap (priority queue) to always expand the cheapest route.
- Each state in the queue includes the current cost, current city, and the number of stops used so far.
- Track the best-known cost to reach a city with a certain number of stops to prune non-optimal routes.
- Ensure that you do not exceed k stops during exploration.

Space & Time

Time Complexity: $O(E \log(E))$ in the worst-case scenario, where E is the number of flights, due to the heap operations.

Space Complexity: $O(n + E)$ for storing the graph and auxiliary data structures like the heap and best-cost map.

Solution

We solve the problem using a modified Dijkstra's algorithm. The approach employs a min-heap to ensure that we always process the state with the lowest cumulative cost next. Each state tracks the total cost so far, the current city, and the number of stops made. When a state reaches the destination city, we return its cost. To avoid unnecessary work, we record the lowest cost at which each city has been reached with a given number of stops. This state-tracking helps prune routes that are more expensive or exceed the allowed number of stops.

Heap

□ #973 K Closest Points to Origin

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math · Divide and Conquer · Sorting ·
Heap

Lists: Grind 169

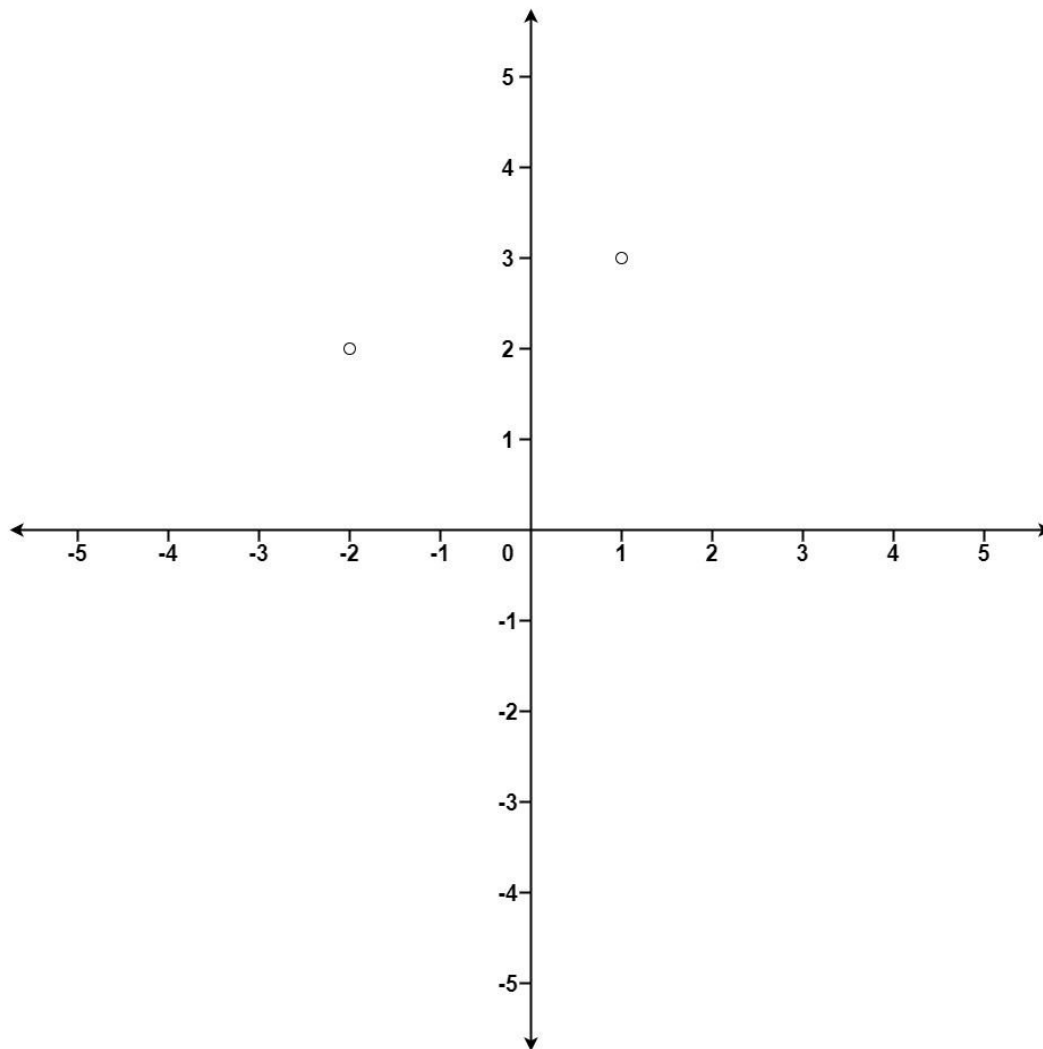
Problem

Given an array of points where $\text{points}[i] = [x_i, y_i]$ represents a point on the X-Y plane and an integer k , return the k closest points to the origin $(0, 0)$.

The distance between two points on the X-Y plane is the Euclidean distance (i.e., $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$).

You may return the answer in any order. The answer is guaranteed to be unique (except for the order that it is in).

Example 1:



Input: points = [[1,3],[-2,2]], k = 1

Output: [[-2,2]]

Explanation:

The distance between (1, 3) and the origin is $\sqrt{10}$.

The distance between (-2, 2) and the origin is $\sqrt{8}$.

Since $\sqrt{8} < \sqrt{10}$, (-2, 2) is closer to the origin.

We only want the closest k = 1 points from the origin, so the answer is just [[-2,2]].

Example 2:

Input: points = [[3,3],[5,-1],[-2,4]], k = 2

Output: [[3,3],[-2,4]]

Explanation: The answer [[-2,4],[3,3]] would also be accepted.

Constraints:

- $1 \leq k \leq \text{points.length} \leq 104$

- $-104 \leq x_i, y_i \leq 104$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
        max_q = []
        for i, (x, y) in enumerate(points):
            dist = math.hypot(x, y)
            heappush(max_q, (-dist, i))
            if len(max_q) > k:
                heappop(max_q)
        return [points[i] for _, i in max_q]
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Return the k closest points to the origin using Euclidean distance.
# We can compare squared distances – no sqrt needed. Return in any order. Right?"
#
# "A few quick questions:
# Can k equal n (return all)? Yes. Can points be identical? Yes."
#
# "Let me walk: points=[[1,3],[-2,2]], k=1. dist²=[10,8]. → [[-2,2]] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Sort all points by squared distance, return first k.
# O(n log n) time. Fine for most cases, but a heap gives O(n log k).
# Should I go ahead and optimize?"
class _973_KClosestPoints_Brute:
    def kClosest(self, points, k):
        return sorted(points, key=lambda p: p[0]**2 + p[1]**2)[:k]
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is sorting all n points when we only need the k closest.
# Max-heap of size k: maintain k closest points seen so far.
# Push (-dist², x, y) so the heap evicts the farthest (largest dist²).
# When heap exceeds k, pop the farthest.
#
# Time: O(n log k). Space: O(k)."
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
import heapq
class _973_KClosestPoints:
    def kClosest(self, points, k):
        max_heap = []
        for x, y in points:
            heapq.heappush(max_heap, (-(x*x + y*y), x, y))
            if len(max_heap) > k:
                heapq.heappop(max_heap)
        return [[x, y] for _, x, y in max_heap]
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

```
#
# points=[[1,3],[-2,2]], k=1:
# push (-10,1,3)→[(-10,1,3)]. push (-8,-2,2)→pop(-10,1,3)→[(-8,-2,2)]. return
# [[-2,2]] ✓
#
# points=[[3,3],[5,-1],[-2,4]], k=2: dist²=[18,26,20].
# After processing: heap keeps (-20,-2,4) and (-18,3,3). return [[3,3],[-2,4]] ✓
#
# "No bugs. Negating dist² turns the min-heap into a max-heap, evicting the farthest
point."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Max-heap: O(n log k). Sort: O(n log n). Heap wins when k << n.
# Quickselect: O(n) average – optimal for single large-n queries.
# Follow-up: stream of points, maintain k closest – max-heap of size k updates on
each arrival.
# Follow-up: k closest to an arbitrary point – subtract the target, then same
algorithm."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def kClosest(self, points: list[list[int]], k: int) -> list[list[int]]:
        maxHeap = []

        for x, y in points:
            heapq.heappush(maxHeap, (- x * x - y * y, [x, y]))
            if len(maxHeap) > k:
                heapq.heappop(maxHeap)

        return [pair[1] for pair in maxHeap]
```



```

class Solution:
    def kClosest(self, points: list[list[int]], k: int) -> list[list[int]]:
        def squareDist(p: list[int]) -> int:
            return p[0] * p[0] + p[1] * p[1]

        def quickSelect(l: int, r: int, k: int) -> None:
            pivot = points[r]

            nextSwapped = l
            for i in range(l, r):

                if squareDist(points[i]) <= squareDist(pivot):
                    points[nextSwapped], points[i] = points[i], points[nextSwapped]
                    nextSwapped += 1
            points[nextSwapped], points[r] = points[r], points[nextSwapped]

            count = nextSwapped - l + 1  # the number of points <= pivot
            if count == k:
                return
            if count > k:
                quickSelect(l, nextSwapped - 1, k)

            else:
                quickSelect(nextSwapped + 1, r, k - count)

        quickSelect(0, len(points) - 1, k)
        return points[0:k]

```

• LeetDoocs

```

class Solution:
    def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
        points.sort(key=lambda p: hypot(p[0], p[1]))
        return points[:k]

```

• SimplyLeet

```
# Python Solution
def kClosest(points, k):
    # Sort the points based on their squared Euclidean distance from the
    # origin.
    points.sort(key=lambda point: point[0]**2 + point[1]**2)
    # Return the first k points from the sorted list.
    return points[:k]

# Example usage:
# points = [[1,3], [-2,2]]
# k = 1

# print(kClosest(points, k))
```

• LC.ca

```
class Solution:
    def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
        points.sort(key=lambda p: p[0] * p[0] + p[1] * p[1])
        return points[:k]
```

◆ Quick Review

Key Insights

- Calculate the squared Euclidean distance ($x^2 + y^2$) to avoid the overhead of square root computation.
- Sorting the points based on their distance gives an $O(n \log n)$ solution.
- Alternatively, using a max-heap to keep track of the k closest points results in $O(n \log k)$ time, which is effective if k is much smaller than n .

- Using Quickselect can achieve an average time complexity of $O(n)$, though the worst-case is $O(n^2)$.
- The problem does not require the output to be sorted in any specific order.

Space & Time

Time Complexity: $O(n \log n)$ with a sorting approach, $O(n \log k)$ with a heap approach, or average $O(n)$ with a Quickselect approach.

Space Complexity: $O(n)$ if extra space is used for sorting, or $O(k)$ for the heap approach.

Solution

We can solve this problem by first computing the squared Euclidean distance for each point to avoid unnecessary square root operations. Then, we can sort the points based on their computed distance and select the first k points. Alternatively, a max-heap or Quickselect algorithm can be used: **Max-Heap:** Maintain a heap of size k while iterating through the points, ensuring that only the k points with the smallest distances are kept. **Quickselect:** Partition the array like in QuickSort to position the k th closest point in

its correct position. Once partitioned, all points to the left of that position will be the k closest. The sorting solution is straightforward and efficient enough given the constraints.

Heap

□ ★ #1057 Campus Bikes ★

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy · Sorting · Heap
Lists: ★ Premium | Premium 98

Problem

On a campus represented on the X-Y plane, there are n workers and m bikes, with $n \leq m$. You are given an array `workers` of length n where `workers[i] = [xi, yi]` is the position of the i th worker. You are also given an array `bikes` of length m where `bikes[j] = [xj, yj]` is the position of the j th bike. All the given positions are unique.

Assign a bike to each worker. Among the available bikes and workers, we choose the $(worker_i, bike_j)$ pair with the shortest Manhattan distance between each other and assign the bike to that worker.

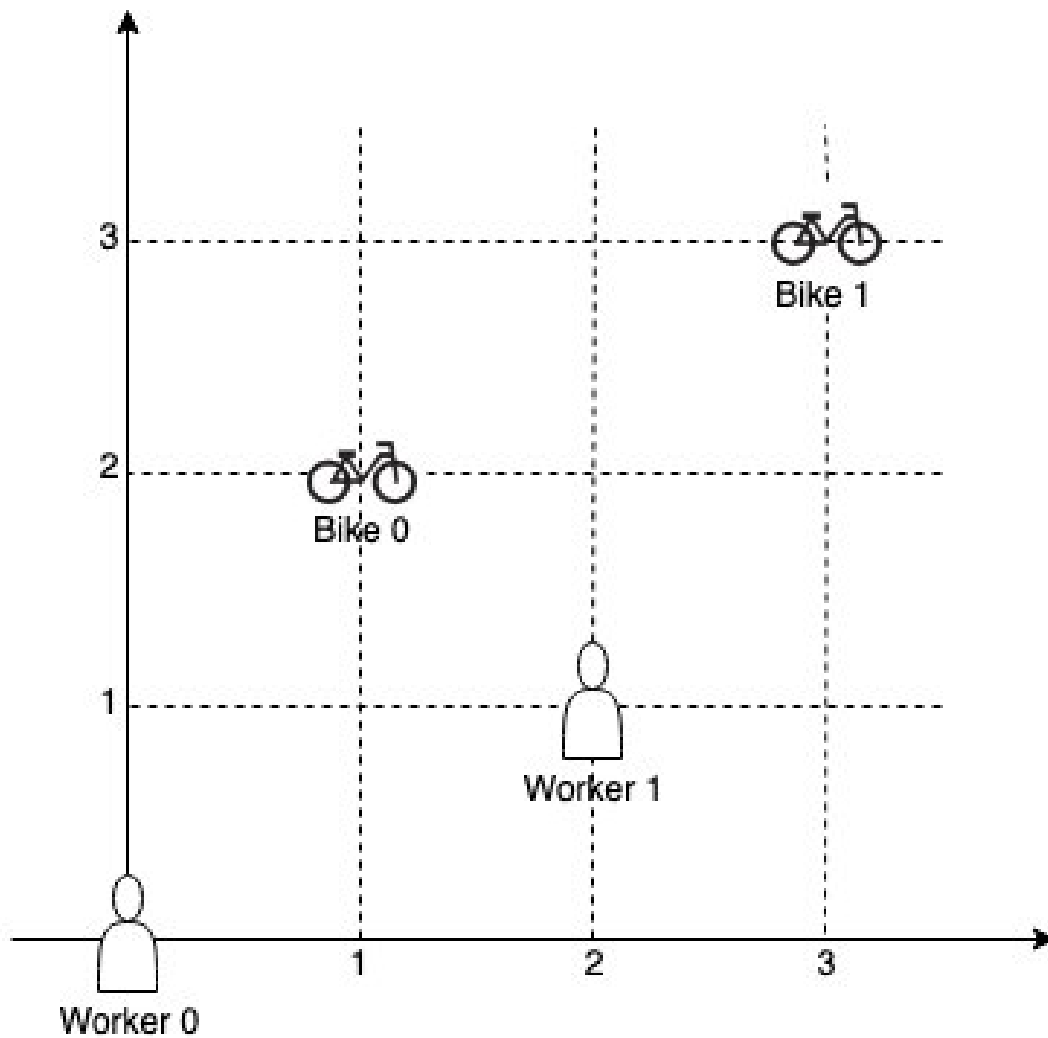
If there are multiple $(worker_i, bike_j)$ pairs with the same shortest Manhattan distance, we

choose the pair with the smallest worker index. If there are multiple ways to do that, we choose the pair with the smallest bike index. Repeat this process until there are no available workers.

Return an array answer of length n, where answer[i] is the index (0-indexed) of the bike that the ith worker is assigned to.

The Manhattan distance between two points p1 and p2 is $\text{Manhattan}(p1, p2) = |p1.x - p2.x| + |p1.y - p2.y|$.

Example 1:

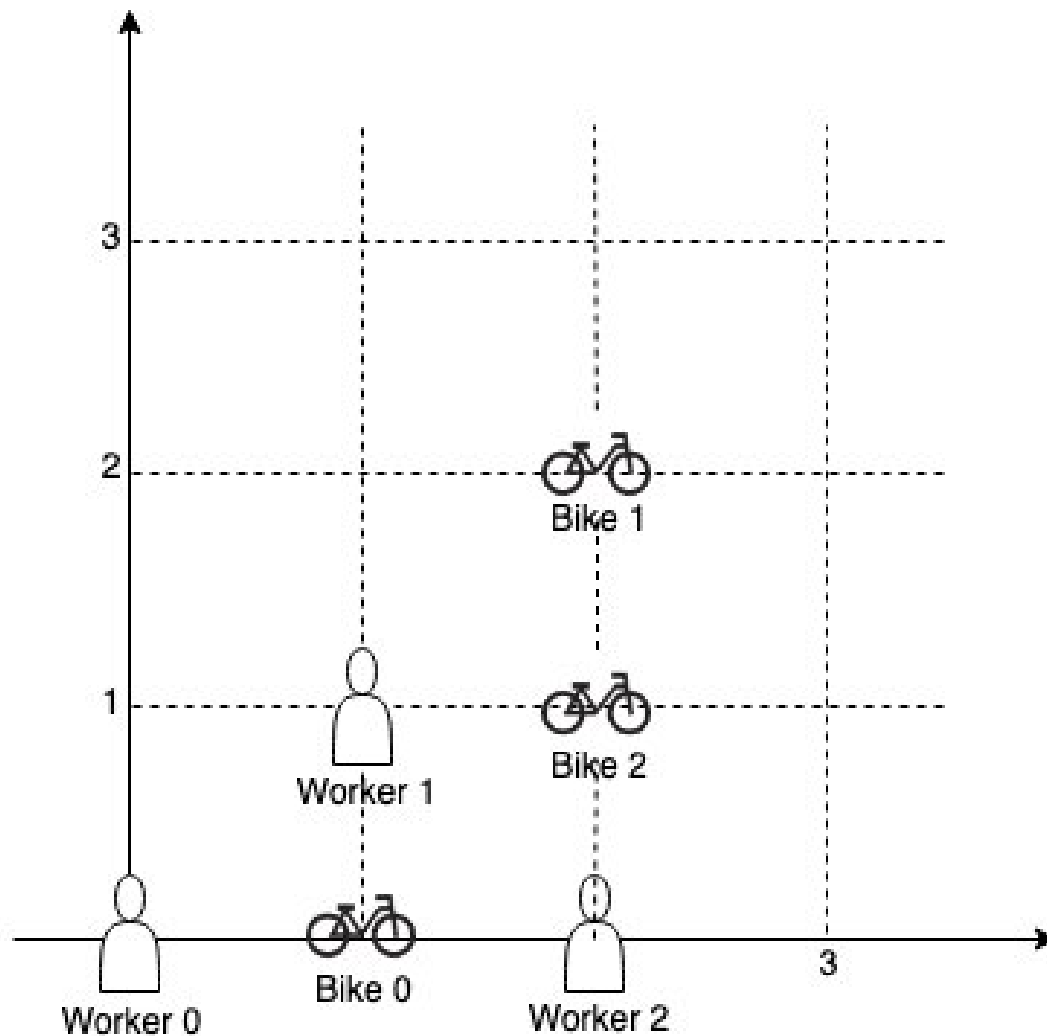


Input: workers = `[[0,0],[2,1]]`, bikes = `[[1,2],[3,3]]`

Output: `[1,0]`

Explanation: Worker 1 grabs Bike 0 as they are closest (without ties), and Worker 0 is assigned Bike 1. So the output is `[1, 0]`.

Example 2:



Input: workers = [[0,0],[1,1],[2,0]], bikes = [[1,0],[2,2],[2,1]]

Output: [0,2,1]

Explanation: Worker 0 grabs Bike 0 at first. Worker 1 and Worker 2 share the same distance to Bike 2, thus Worker 1 is assigned to Bike 2, and Worker 2 will take Bike 1. So the output is [0,2,1].

Constraints:

- $n == \text{workers.length}$
- $m == \text{bikes.length}$
- $1 \leq n \leq m \leq 1000$
- $\text{workers}[i].\text{length} == \text{bikes}[j].\text{length} == 2$
- $0 \leq x_i, y_i < 1000$

- $0 \leq x_j, y_j < 1000$
- All worker and bike locations are unique.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def assignBikes(
        self,
        workers: list[list[int]],
        bikes: list[list[int]],
    ) -> list[int]:
        ans = [-1] * len(workers)
        usedBikes = [False] * len(bikes)
        # buckets[k] := (i, j), where k = dist(workers[i], bikes[j])
        buckets = [[] for _ in range(2001)]

        def dist(p1: list[int], p2: list[int]) -> int:
            return abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])

        for i, worker in enumerate(workers):
            for j, bike in enumerate(bikes):
                buckets[dist(worker, bike)].append((i, j))

        for k in range(2001):
            for i, j in buckets[k]:
                if ans[i] == -1 and not usedBikes[j]:
                    ans[i] = j
                    usedBikes[j] = True

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

n workers and m bikes on a campus. Greedily assign the closest worker-bike pair.

Ties: smaller worker index first; still tied: smaller bike index.

```

# Repeat until each worker has exactly one bike. Right?"
#
# "A few quick questions:
#  $n \leq m$  – enough bikes for all workers? Yes. Manhattan distance? Yes."
#
# "Let me walk: workers=[[0,0],[2,1]], bikes=[[1,2],[3,3]].
# w0b0=3, w0b1=6, w1b0=2, w1b1=2 → min=2 at (w1,b0) → assign w1→b0.
# Next min from remaining: w0b1=6 → assign w0→b1. → [1,0] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Generate all (dist, worker_idx, bike_idx) triples, sort, greedily assign.
#  $O(n*m \log(n*m))$  time for sort. Space  $O(n*m)$ .
# Should I go ahead and optimize?"
class _1057_CampusBikes_Brute:
    def assignBikes(self, workers, bikes):
        trips = []
        for i, (wx, wy) in enumerate(workers):
            for j, (bx, by) in enumerate(bikes):
                trips.append((abs(wx-bx)+abs(wy-by), i, j))
        trips.sort()
        ans = [-1]*len(workers); used_bikes = set(); used_workers = set()
        for d, w, b in trips:
            if w not in used_workers and b not in used_bikes:
                ans[w] = b; used_workers.add(w); used_bikes.add(b)
        return ans

# PHASE 3 — OPTIMIZE
# "The bottleneck is the  $O(n*m \log(n*m))$  sort – bucket sort on bounded Manhattan
# distance ( $\leq 2000$ ) reduces it to  $O(n*m)$ .
# Bucket sort by distance (max Manhattan  $\leq 2000$ ) brings it to  $O(n*m)$  time:
# Distribute all triples into distance buckets, scan from 0 upward.
#
# Time:  $O(n*m)$ . Space:  $O(n*m)$ ."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _1057_CampusBikes:
    def assignBikes(self, workers, bikes):
        max_dist = 2001
        buckets = [[] for _ in range(max_dist)]
        for i, (wx, wy) in enumerate(workers):
            for j, (bx, by) in enumerate(bikes):
                d = abs(wx - bx) + abs(wy - by)
                buckets[d].append((i, j))
        ans = [-1] * len(workers)
        used_bikes = set()
        assigned = 0
        for d in range(max_dist):
            for w, b in buckets[d]:
                if ans[w] == -1 and b not in used_bikes:

```

```

        ans[w] = b; used_bikes.add(b); assigned += 1
        if assigned == len(workers): return ans

    return ans

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

```

#
# workers=[[0,0],[2,1]], bikes=[[1,2],[3,3]]:
# buckets[3]=[(0,0)], buckets[6]=[(0,1)], buckets[2]=[(1,0),(1,1)].
# d=2: (1,0)→ans[1]=0, used={0}. (1,1)→b=1 not used: ans[1] already set, skip.
# d=3: (0,0)→ans[0]=0? b=0 in used → skip. d=6: (0,1)→ans[0]=1 ✓. → [1,0] ✓
#
# "No bugs. Bucket sort naturally satisfies (dist asc, worker asc, bike asc)
# ordering."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Bucket sort:  $O(n*m)$  time,  $O(n*m)$  space (2001 buckets). Heap:  $O(n*m \log(n*m))$ .
# Bucket sort wins because Manhattan distance  $\leq 2000$  is bounded.
# Follow-up: Campus Bikes II (#1066) — minimize TOTAL distance. Bitmask DP  $O(n*2^m)$ .
# Follow-up: if bikes can be reassigned dynamically, use a priority queue per
# worker."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def assignBikes(
        self,
        workers: list[list[int]],
        bikes: list[list[int]],
    ) -> list[int]:
        ans = [-1] * len(workers)
        usedBikes = [False] * len(bikes)
        # buckets[k] := (i, j), where k = dist(workers[i], bikes[j])
        buckets = [[] for _ in range(2001)]

```

```
def dist(p1: list[int], p2: list[int]) -> int:
    return abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])

for i, worker in enumerate(workers):
    for j, bike in enumerate(bikes):
        buckets[dist(worker, bike)].append((i, j))

for k in range(2001):
    for i, j in buckets[k]:

        if ans[i] == -1 and not usedBikes[j]:
            ans[i] = j
            usedBikes[j] = True

return ans
```

• LeetDoocs

```
class Solution:
    def assignBikes(
        self, workers: List[List[int]], bikes: List[List[int]]
    ) -> List[int]:
        n, m = len(workers), len(bikes)
        arr = []
        for i, j in product(range(n), range(m)):
            dist = abs(workers[i][0] - bikes[j][0]) + abs(workers[i][1] - bikes[j][1])
            arr.append((dist, i, j))
        arr.sort()

        vis1 = [False] * n
        vis2 = [False] * m
        ans = [0] * n
        for _, i, j in arr:
            if not vis1[i] and not vis2[j]:
                vis1[i] = vis2[j] = True
                ans[i] = j
        return ans
```

• SimplyLeet

```

# Python solution

def assign_bikes(workers, bikes):
    # List to hold all possible (distance, worker, bike) tuples
    pairs = []
    for worker_index, (wx, wy) in enumerate(workers):
        for bike_index, (bx, by) in enumerate(bikes):
            # Calculate Manhattan distance between worker and bike
            distance = abs(wx - bx) + abs(wy - by)
            pairs.append((distance, worker_index, bike_index))

    # Sort pairs based on distance, then worker index, then bike index
    pairs.sort()

    n = len(workers) # Number of workers
    assigned_workers = [False] * n # Track if worker is already assigned
    assigned_bikes = [False] * len(bikes) # Track if bike is already assigned
    result = [-1] * n # Result list to store bike assignment for
    each worker

    # Iterate over the sorted pairs and assign bikes

    for distance, worker_index, bike_index in pairs:
        if not assigned_workers[worker_index] and not assigned_bikes[bike_index]:
            result[worker_index] = bike_index # Assign bike to worker
            assigned_workers[worker_index] = True # Mark worker as assigned
            assigned_bikes[bike_index] = True # Mark bike as assigned
    return result

# Example usage:
workers = [[0,0],[2,1]]
bikes = [[1,2],[3,3]]

print(assign_bikes(workers, bikes)) # Output: [1, 0]

```

• LC.ca

```

class Solution:
    def assignBikes(
        self, workers: List[List[int]], bikes: List[List[int]]
    ) -> List[int]:
        n, m = len(workers), len(bikes)
        arr = []
        for i, j in product(range(n), range(m)):
            dist = abs(workers[i][0] - bikes[j][0]) + abs(workers[i][1] - bikes[j][1])
            arr.append((dist, i, j))
        arr.sort()

        vis1 = [False] * n
        vis2 = [False] * m
        ans = [0] * n
        for _, i, j in arr:
            if not vis1[i] and not vis2[j]:
                vis1[i] = vis2[j] = True
                ans[i] = j
        return ans

```

◆ Quick Review

Key Insights

- Compute the Manhattan distance between every worker and bike pair.
- To achieve the tie-breaking rules, sort all pairs by distance, then by worker index and bike index.
- Once a worker is assigned a bike, skip any other pairs involving that worker or the already assigned bike.

- The problem can also be approached using bucket sort because the Manhattan distance range is limited, but sorting is intuitive given the constraints.

Space & Time

Time Complexity: $O(n * m * \log(n * m))$ where n is the number of workers and m is the number of bikes.

Space Complexity: $O(n * m)$ for storing all worker–bike pairs.

Solution

Create all possible (worker, bike) pairs along with their Manhattan distances. Sort this list of pairs by: Manhattan distance, Worker index (if distances are equal), Bike index (if both distance and worker index are equal). Then, iterate over the sorted list and assign bikes to workers if neither the worker nor the bike has already been assigned. This guarantees that at each step, the optimal available pairing (according to the problem's requirements) is made. This method naturally enforces the tie-breaking rules specified in the problem.

Heap

□ ★ #1167 Minimum Cost to Connect Sticks ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy · Heap
Lists: ★ Premium | Premium 98

Problem

You have some number of sticks with positive integer lengths. These lengths are given as an array `sticks`, where `sticks[i]` is the length of the *i*th stick.

You can connect any two sticks of lengths x and y into one stick by paying a cost of $x + y$. You must connect all the sticks until there is only one stick remaining.

Return the minimum cost of connecting all the given sticks into one stick in this way.

Example 1:

Input: sticks = [2,4,3]

Output: 14

Explanation: You start with sticks = [2,4,3].

1. Combine sticks 2 and 3 for a cost of $2 + 3 = 5$. Now you have sticks = [5,4].

2. Combine sticks 5 and 4 for a cost of $5 + 4 = 9$. Now you have sticks = [9].

There is only one stick left, so you are done. The total cost is $5 + 9 = 14$.

Example 2:

Input: sticks = [1,8,3,5]

Output: 30

Explanation: You start with sticks = [1,8,3,5].

1. Combine sticks 1 and 3 for a cost of $1 + 3 = 4$. Now you have sticks = [4,8,5].

2. Combine sticks 4 and 5 for a cost of $4 + 5 = 9$. Now you have sticks = [9,8].

3. Combine sticks 9 and 8 for a cost of $9 + 8 = 17$. Now you have sticks = [17].

There is only one stick left, so you are done. The total cost is $4 + 9 + 17 = 30$.

Example 3:

Input: sticks = [5]

Output: 0

Explanation: There is only one stick, so you don't need to do anything. The total cost is 0.

Constraints:

- $1 \leq \text{sticks.length} \leq 104$
- $1 \leq \text{sticks}[i] \leq 104$

My LeetCode Solution

- My LeetCode Solution

```

class Solution:
    def connectSticks(self, sticks: list[int]) -> int:
        ans = 0
        heapq.heapify(sticks)

        while len(sticks) > 1:
            x = heapq.heappop(sticks)
            y = heapq.heappop(sticks)
            ans += x + y
            heapq.heappush(sticks, x + y)

        return ans

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Let me make sure I understand the problem correctly.
# Combine any two sticks into one; cost = sum of their lengths.
# Repeat until one stick remains. Minimize total cost. Right?"
#
# "A few quick questions:
# Single stick → cost 0? Yes – no combining needed. All lengths positive? Yes."
#
# "Let me walk: sticks=[2,4,3].
# Combine 2+3=5 (cost 5). Then 4+5=9 (cost 9). Total=14 ✓
# vs 2+4=6 (cost 6). 3+6=9 (cost 9). Total=15. The greedy wins."

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with brute force to lock in the logic.
# Try all pair orderings recursively; track minimum total cost.
# O(n! * n) – completely infeasible.
# Should I go ahead and optimize?"
class _1167_MinCostConnectSticks_Brute:
    def connectSticks(self, sticks):
        from itertools import permutations
        best = float('inf')
        def dfs(s, cost):
            nonlocal best
            if len(s) == 1: best = min(best, cost); return
            for i in range(len(s)):
                for j in range(i+1, len(s)):
                    merged = s[i] + s[j]; new_s = [s[k] for k in range(len(s)) if
k!=i and k!=j] + [merged]
                    dfs(new_s, cost + merged)
        dfs(list(sticks), 0); return best

```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the exponential search – but the greedy is optimal.
# Always combine the two SMALLEST sticks (Huffman coding / priority queue):
# small sticks are paid for many times as they get merged into bigger ones.
# Using a min-heap: pop two smallest, push their sum, add sum to total cost.
#
# Time: O(n log n). Space: O(n) for the heap."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
import heapq
class _1167_MinCostConnectSticks:
    def connectSticks(self, sticks):
        heapq.heapify(sticks)
        total_cost = 0
        while len(sticks) > 1:
            first = heapq.heappop(sticks)
            second = heapq.heappop(sticks)
            combined = first + second
            total_cost += combined
            heapq.heappush(sticks, combined)
        return total_cost
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# sticks=[2,4,3] → heapified=[2,3,4]:
# pop 2,3 → combined=5, cost=5. heap=[4,5].
# pop 4,5 → combined=9, cost=14. heap=[9]. → 14 ✓
#
# sticks=[1,8,3,5] → heap=[1,3,5,8]:
# pop 1,3→4, cost=4. heap=[4,5,8].
# pop 4,5→9, cost=13. heap=[8,9].
# pop 8,9→17, cost=30. → 30 ✓
#
# "No bugs. heapify is O(n) – more efficient than n individual pushes."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n log n): heapify O(n) + n-1 pops/pushes O(log n) each.
# Space O(n): the heap.
# This is Huffman encoding – greedy optimality proven by exchange argument.
# Follow-up: Minimum Cost to Merge Stones (#1000) – merge k adjacent groups; needs DP.
# Follow-up: if k sticks must be merged at once, each merge costs sum of k sticks."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def connectSticks(self, sticks: list[int]) -> int:
        ans = 0
        heapq.heapify(sticks)

        while len(sticks) > 1:
            x = heapq.heappop(sticks)
            y = heapq.heappop(sticks)
            ans += x + y
            heapq.heappush(sticks, x + y)

        return ans
```

• LeetCode

```
class Solution:
    def connectSticks(self, sticks: List[int]) -> int:
        heapq.heapify(sticks)
        ans = 0
        while len(sticks) > 1:
            z = heappop(sticks) + heappop(sticks)
            ans += z
            heappush(sticks, z)
        return ans
```

• SimplyLeet

```
import heapq

def connectSticks(sticks):
    # Initialize the heap with the provided stick lengths
    heapq.heapify(sticks)

    # Initialize total cost to 0
    total_cost = 0

    # Continue connecting sticks until there is only one left
```

```

while len(sticks) > 1:
    # Extract the two smallest sticks
    stick1 = heapq.heappop(sticks)
    stick2 = heapq.heappop(sticks)
    # Calculate the cost to connect these two sticks
    cost = stick1 + stick2
    # Add the cost to the total cost
    total_cost += cost
    # Push the combined stick back into the heap
    heapq.heappush(sticks, cost)

return total_cost

# Example usage:
# print(connectSticks([2, 4, 3])) # Output: 14

```

• LC.ca

```

class Solution:
    def connectSticks(self, sticks: List[int]) -> int:
        heapify(sticks)
        ans = 0
        while len(sticks) > 1:
            z = heappop(sticks) + heappop(sticks)
            ans += z
            heappush(sticks, z)
        return ans

```

◆ Quick Review

Key Insights

- Always connect the two smallest sticks available to minimize the incremental cost.
- A min-heap (or priority queue) is an ideal data structure to efficiently retrieve the smallest sticks.

- The process continues until only one stick remains.

Space & Time

Time Complexity: $O(n \log n)$, where n is the number of sticks (due to heap insertion and removal operations).

Space Complexity: $O(n)$, for storing the sticks in the heap.

Solution

The solution uses a greedy algorithm with a min-heap: Insert all stick lengths into a min-heap. While there are at least two sticks in the heap: Extract the two smallest sticks. Compute the cost of connecting them (sum the two lengths). Add this cost to the total result. Push the combined stick (with length equal to their sum) back into the heap. Once the heap is reduced to one element, return the total cost. This approach ensures that at every connection, the smallest possible sticks are merged, which leads to the minimum overall cost.

Heap

□ #1424 Diagonal Traverse II

MED

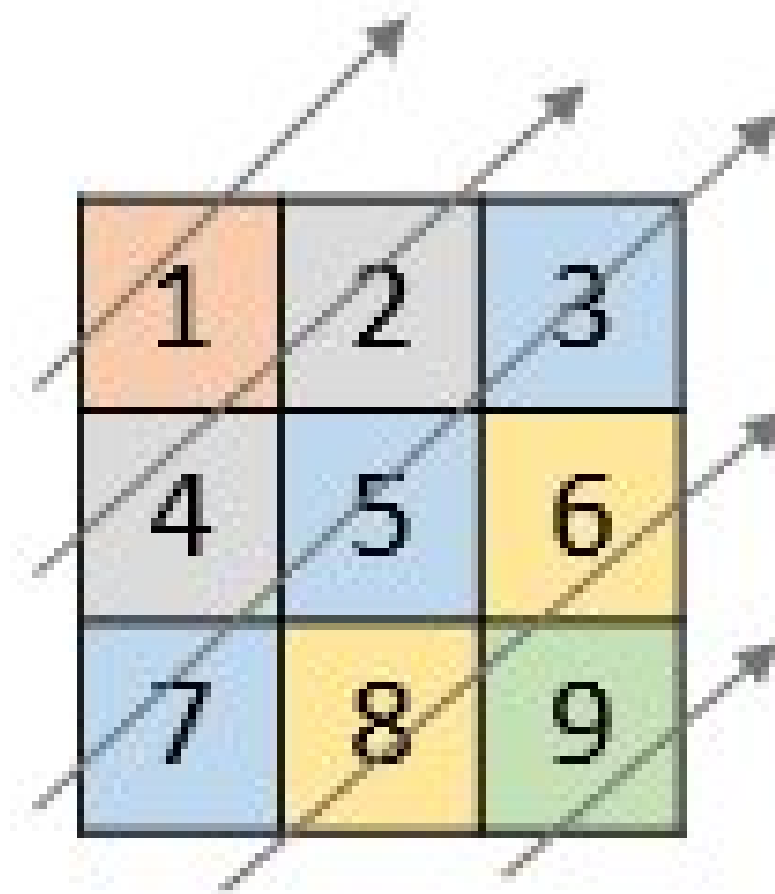
[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sorting · Heap
Lists: CodeSignal

Problem

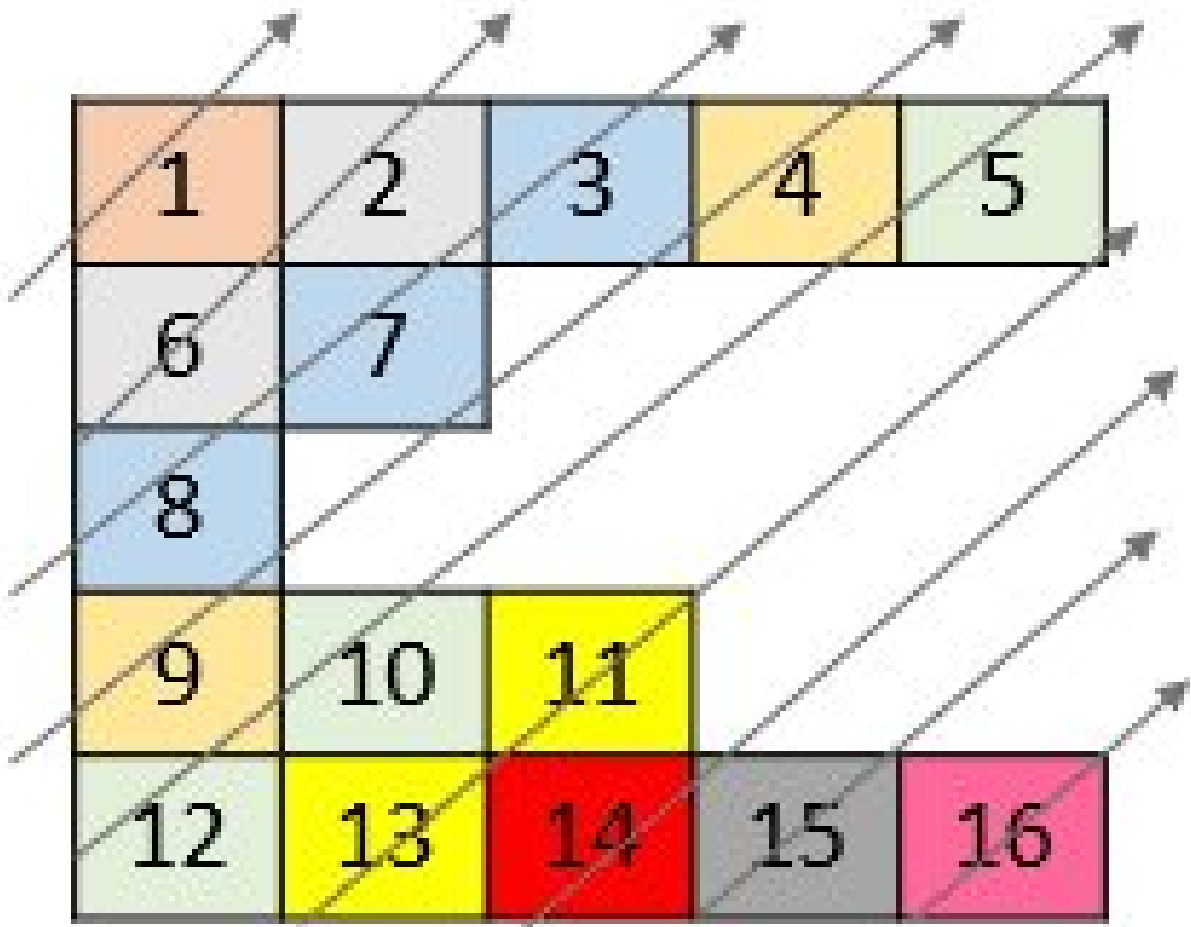
Given a 2D integer array `nums`, return all elements of `nums` in diagonal order as shown in the below images.

Example 1:



Input: nums = [[1,2,3],[4,5,6],[7,8,9]]
Output: [1,4,2,7,5,3,8,6,9]

Example 2:



Input: nums = [[1,2,3,4,5],[6,7],[8],[9,10,11],[12,13,14,15,16]]
Output: [1,6,2,8,7,3,9,4,12,10,5,13,11,14,15,16]

Constraints:

- 1 <= nums.length <= 105
- 1 <= nums[i].length <= 105
- 1 <= sum(nums[i].length) <= 105
- 1 <= nums[i][j] <= 105

My LeetCode Solution

- ☐
- My LeetCode Solution

```
class Solution:
    def findDiagonalOrder(self, nums: List[List[int]]) -> List[int]:
        arr = []
        for i, row in enumerate(nums):
            for j, v in enumerate(row):
                arr.append((i + j, j, v))
        arr.sort()
        return [v[2] for v in arr]
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given a 2D list (rows may have different lengths), return elements in anti-diagonal order.

Diagonal index = $i+j$. Within each diagonal, list from bottom to top (higher i first). Right?"

#

"Let me walk: `nums=[[1,2,3],[4,5,6],[7,8,9]]` → `[1,4,2,7,5,3,8,6,9]` ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Group elements by diagonal index ($i+j$) into a dict.

For each group, append elements in reverse row order (higher i first).

$O(n)$ where n = total elements.

Should I go ahead and optimize?"

```
class _1424_DiagonalTraverseII_Brute:
    def findDiagonalOrder(self, nums):
        from collections import defaultdict
        diag = defaultdict(list)
        for i, row in enumerate(nums):
            for j, val in enumerate(row):
                diag[i + j].append(val)
        result = []
        for key in sorted(diag):
            result.extend(reversed(diag[key]))
        return result
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(d \log d)$ sort on diagonal keys – iterate rows top-to-bottom instead.

We can avoid sorting the diagonal keys by iterating rows top-to-bottom:

elements encountered later in the same diagonal should appear FIRST (prepend or reverse at end).

Use an append approach per diagonal key, then reverse each bucket.

#

Time: $O(n)$. Space: $O(n)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
from collections import defaultdict
class _1424_DiagonalTraverseII:
    def findDiagonalOrder(self, nums):
        diag_groups = defaultdict(list)
        max_key = 0
        for i, row in enumerate(nums):
            for j, val in enumerate(row):
                key = i + j
                diag_groups[key].append(val)
                max_key = max(max_key, key)
        result = []
        for key in range(max_key + 1):
            result.extend(reversed(diag_groups[key]))
        return result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

```
#
# nums=[[1,2,3],[4,5,6],[7,8,9]]:
# diag[0]=[1], diag[1]=[2,4], diag[2]=[3,5,7], diag[3]=[6,8], diag[4]=[9].
# reversed: [1],[4,2],[7,5,3],[8,6],[9] → [1,4,2,7,5,3,8,6,9] ✓
#
# Jagged: nums=[[1,2,3,4,5],[6,7],[8],[9,10,11],[12,13,14,15,16]] → correct order ✓
#
# "No bugs. Appending top-to-bottom then reversing each bucket gives bottom-to-top
order."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): one pass to build groups + one pass to collect. Space O(n).
# Using range(max_key+1) instead of sorted(diag_groups) avoids the O(d log d) sort
on diagonals.
# Follow-up: Diagonal Traverse I (#498) – rectangular matrix, alternating direction
per diagonal.
# Follow-up: spiral order (#54) vs diagonal order are both reordering problems on
matrices."
```

Community Solutions (Python)

• LeetDoocs

```
class Solution:
    def findDiagonalOrder(self, nums: List[List[int]]) -> List[int]:
        arr = []
        for i, row in enumerate(nums):
            for j, v in enumerate(row):
                arr.append((i + j, j, v))
        arr.sort()
        return [v[2] for v in arr]
```

• SimplyLeet

```
def findDiagonalOrder(nums):
    # Use a dictionary to collect elements by their diagonal key (i + j)
    from collections import defaultdict
    diagonals = defaultdict(list)

    # Iterate over each row and its elements
    for i, row in enumerate(nums):
        for j, value in enumerate(row):
            # Insert at the beginning to achieve the desired order for the
            diagonal
            diagonals[i + j].insert(0, value)

    result = []
    # Process the diagonals in order of increasing key
    for key in sorted(diagonals.keys()):
        result.extend(diagonals[key])
    return result

# Example usage:
# nums = [[1,2,3],[4,5,6],[7,8,9]]
# print(findDiagonalOrder(nums)) # Output: [1,4,2,7,5,3,8,6,9]
```

• LC.ca

```
class Solution:
    def findDiagonalOrder(self, nums: List[List[int]]) -> List[int]:
        arr = []
        for i, row in enumerate(nums):
            for j, v in enumerate(row):
                arr.append((i + j, j, v))
        arr.sort()
        return [v[2] for v in arr]
```

◆ Quick Review

Key Insights

- The diagonal key for each element can be defined as (row_index + column_index).
- Elements in the same diagonal (with equal diagonal key) must be output in an order such that the element from the lower row comes before the one from a higher row.
- Because the input grid may be jagged (rows with varying lengths), you must carefully iterate only over valid indices.
- A hash table (or map) can be used to collect elements grouped by their diagonal key.
- Inserting each element at the beginning of the list for that diagonal builds the list in the desired order without needing an extra reverse step later.

- Finally, traverse the keys in increasing order (from 0 to maximum key) to produce the final result.

Space & Time

Time Complexity: $O(N)$, where N is the total number of elements (each element is processed once).

Space Complexity: $O(N)$, for storing the elements in the map and the output array.

Solution

The solution iterates over every element in `nums` while computing its diagonal key as $(i + j)$. For each element, the value is inserted at the beginning of the list corresponding to that diagonal key. This ensures that when the diagonal groups are later concatenated in order of increasing key, the internal ordering within each diagonal is correct (i.e., elements appear from the bottom of the diagonal to the top). The final result is built by iterating over the keys in sorted order and appending each list of diagonal elements.

Trie

Round 5 · Mid · Medium · 7 questions

Trie

□ #139 Word Break

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Dynamic
Programming · Trie · Memoization
Lists: Grind 169

Problem

Given a string `s` and a dictionary of strings `wordDict`, return `true` if `s` can be segmented into a space-separated sequence of one or more dictionary words.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1:

```
Input: s = "leetcode", wordDict = ["leet","code"]  
Output: true  
Explanation: Return true because "leetcode" can be segmented as "leet code".
```

Example 2:

```
Input: s = "applepenapple", wordDict = ["apple","pen"]  
Output: true  
Explanation: Return true because "applepenapple" can be segmented as "apple pen apple".  
Note that you are allowed to reuse a dictionary word.
```

Example 3:

Input: s = "catsanddog", wordDict = ["cats","dog","sand","and","cat"]
Output: false

Constraints:

- $1 \leq s.length \leq 300$
- $1 \leq wordDict.length \leq 1000$
- $1 \leq wordDict[i].length \leq 20$
- s and wordDict[i] consist of only lowercase English letters.
- All the strings of wordDict are unique.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def wordBreak(self, s: str, wordDict: list[str]) -> bool:
        wordSet = set(wordDict)

        @functools.lru_cache(None)
        def wordBreak(s: str) -> bool:
            """Returns True if s can be segmented."""
            if s in wordSet:
                return True
            return any(s[:i] in wordSet and wordBreak(s[i:]) for i in range(len(s)))

        return wordBreak(s)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given string s and a dictionary of words, return true if s can be segmented

```

# into a sequence of dictionary words. Words can be reused. Right?"
#
# "A few quick questions:
# Can the empty string be segmented? Edge case – but s length >= 1 per constraints.
# Same word can be used multiple times? Yes."
#
# "Let me walk: s='leetcode', words=['leet','code']. 'leet'+ 'code' → true ✓
# s='catsanddog', words=['cats','dog','sand','and','cat'] → false ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Recursive: if any prefix of s is in the dict, recurse on the suffix.
# Without memoization, exponential branching –  $O(2^n)$  time.
# Should I go ahead and optimize?"
class _139_WordBreak_Brute:
    def wordBreak(self, s, wordDict):
        words = set(wordDict)
        def can(i):
            if i == len(s): return True
            for j in range(i+1, len(s)+1):
                if s[i:j] in words and can(j): return True
            return False
        return can(0)

# PHASE 3 — OPTIMIZE
# "The bottleneck is the repeated subproblem: can(i) is recomputed many times.
# Bottom-up DP: dp[i] = True if s[:i] can be segmented.
# Base: dp[0] = True (empty prefix).
# Transition: dp[i] = any(dp[j] and s[j:i] in wordSet) for  $0 \leq j < i$ .
# Optimise by only checking back max_word_len characters.
#
# Time:  $O(n * \text{max\_word\_len})$ . Space:  $O(n)$ ."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _139_WordBreak:
    def wordBreak(self, s, wordDict):
        word_set = set(wordDict)
        max_len = max(len(w) for w in wordDict)
        n = len(s)
        dp = [False] * (n + 1)
        dp[0] = True
        for i in range(1, n + 1):
            for j in range(max(0, i - max_len), i):
                if dp[j] and s[j:i] in word_set:
                    dp[i] = True
                    break
        return dp[n]

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."

```

```

#
# s='leetcode', words={'leet','code'}:
# dp[0]=T. dp[4]: j=0,s[0:4]='leet' in set,dp[0]=T → dp[4]=T.
# dp[8]: j=4,s[4:8]='code' in set,dp[4]=T → dp[8]=T. return True ✓
#
# s='catsandog': dp[3]=T('cat'), dp[4]=T('cats'). dp[9]: no j gives a valid word → False ✓
#
# "No bugs. Limiting look-back to max_word_len avoids redundant substring checks."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n * max_word_len): n positions × max_word_len substrings each. Space O(n).
# Trie optimisation: insert all words into a trie, walk forward from each
dp[j]=True.
# Follow-up: Word Break II (#140) – return all valid sentences; backtrack from
dp[n]=True.
# Follow-up: if wordDict is huge but s is short, iterate over words instead of
positions."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def wordBreak(self, s: str, wordDict: list[str]) -> bool:
        wordSet = set(wordDict)

        @functools.lru_cache(None)
        def wordBreak(s: str) -> bool:
            """Returns True if s can be segmented."""
            if s in wordSet:
                return True
            return any(s[:i] in wordSet and wordBreak(s[i:]) for i in range(len(s)))

        return wordBreak(s)

```

```

class Solution:
    def wordBreak(self, s: str, wordDict: list[str]) -> bool:
        n = len(s)
        wordSet = set(wordDict)
        # dp[i] := True if s[0..i) can be segmented
        dp = [True] + [False] * n

        for i in range(1, n + 1):
            for j in range(i):
                # s[0..j) can be segmented and s[j..i) is in `wordSet`, so s[0..i) can
                # be segmented.
                if dp[j] and s[j:i] in wordSet:
                    dp[i] = True
                    break

        return dp[n]

```

```

class Solution:
    def wordBreak(self, s: str, wordDict: list[str]) -> bool:
        n = len(s)
        maxLength = len(max(wordDict, key=len))
        wordSet = set(wordDict)
        # dp[i] := True if s[0..i) can be segmented
        dp = [True] + [False] * n

        for i in range(1, n + 1):
            for j in range(i - 1, -1, -1):
                if i - j > maxLength:
                    break
                # s[0..j) can be segmented and s[j..i) is in the wordSet, so s[0..i)
                # can be segmented.
                if dp[j] and s[j:i] in wordSet:
                    dp[i] = True
                    break

        return dp[n]

```

• LeetDoocs


```

class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        words = set(wordDict)
        n = len(s)
        f = [True] + [False] * n
        for i in range(1, n + 1):
            f[i] = any(f[j] and s[j:i] in words for j in range(i))
        return f[n]

```

```

class Trie:
    def __init__(self):
        self.children: List[Trie | None] = [None] * 26
        self.isEnd = False

    def insert(self, w: str):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if not node.children[idx]:

```

```

                node.children[idx] = Trie()
            node = node.children[idx]
        node.isEnd = True

```

```

class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        trie = Trie()
        for w in wordDict:
            trie.insert(w)

```

```

        n = len(s)
        f = [False] * (n + 1)
        f[n] = True
        for i in range(n - 1, -1, -1):
            node = trie
            for j in range(i, n):
                idx = ord(s[j]) - ord('a')
                if not node.children[idx]:
                    break
                node = node.children[idx]

```

```

        if node.isEnd and f[j + 1]:
            f[i] = True
            break
    return f[0]

```

• SimplyLeet

Python solution for the Word Break problem

```

def wordBreak(s, wordDict):
    # Convert the list to a set for O(1) look-ups
    wordSet = set(wordDict)

    # dp[i] is True if s[0:i] can be segmented into words from the wordDict
    n = len(s)
    dp = [False] * (n + 1)

```

```

    # Base case: empty string is always segmentable
    dp[0] = True

```

```

    # Iterate through the string positions
    for i in range(1, n + 1):
        # Check every possible partition s[0:j] and s[j:i]
        for j in range(i):
            # If s[0:j] can be segmented and s[j:i] is in the dictionary
            if dp[j] and s[j:i] in wordSet:
                dp[i] = True # s[0:i] can be segmented

```

```

        break # No need to check further partitions
    return dp[n]

```

```

# Example usage:
s = "leetcode"
wordDict = ["leet", "code"]
print(wordBreak(s, wordDict)) # Output: True

```

• LC.ca

```
class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        words = set(wordDict)
        n = len(s)

        # dp[j] meaning s from 0 to index=i-1~ is breakable
        dp = [True] + [False] * n # so actually size is n+1
        for i in range(1, n + 1):
            dp[i] = any( (dp[j] and s[j:i] in words) for j in range(i) )
        return dp[n]
```

```
#####
```

```
class Solution:
    def wordBreak(self, s: str, wordDict: Set[str]) -> bool:
        if not s or not wordDict:
            return False

        length = len(s)
```

```
    # construct dp, dp[i] meaning position i-1 can from dict or not
    dp[-1] meaning at last-index plus 1, checking up to last-index
    dp = [False] * (length + 1)
    dp[0] = True # initiate

    for i in range(length + 1):
        if not dp[i]:
            continue
        for word in wordDict:
            if i + len(word) > length:
```

```
                continue
            if s[i:i + len(word)] == word:
                dp[i + len(word)] = True

    return dp[length]
```

```
#####
```

```
class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
```

```

words = set(wordDict)
n = len(s)
dp = [False] * (n + 1)
dp[0] = True # i'th char in string, is good
for j in range(1, n + 1):
    for i in range(j): # starting at 0, including dp[0]
        if dp[i] and s[i:j] in words:
            dp[j] = True # j is exclusive, meaning True until index
i-1
            break
return dp[-1]

```

```
#####
```

```

class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, w):

        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

    def search(self, w):
        node = self

```

```

        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                return False
            node = node.children[idx]
        return node.is_end

class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        # https://docs.python.org/3/library/functools.html#functools.cache

        # creating a thin wrapper around a dictionary lookup for the function
        arguments
        @cache
        def dfs(s):
            return not s or any(trie.search(s[:i]) and dfs(s[i:]) for i in range(1, len(s) + 1))

        trie = Trie()
        for w in wordDict:
            trie.insert(w)
        return dfs(s)

#####

class Solution(object):
    def wordBreak(self, s, wordDict):
        """
        :type s: str
        :type wordDict: Set[str]
        :rtype: bool
        """
        queue = [0]

```

```
ls = len(s)
lenList = [l for l in set(map(len, wordDict))]
visited = [0 for _ in range(0, ls + 1)]
while queue:
    start = queue.pop(0)
    for l in lenList:
        if s[start:start + l] in wordDict:
            if start + l == ls:
                return True
            if visited[start + l] == 0:

                queue.append(start + l)
                visited[start + l] = 1
return False
```

◆ Quick Review

Key Insights

- Use Dynamic Programming (DP) to break the problem into subproblems.
- Create a DP array where $dp[i]$ is true if the substring $s[0:i]$ can be segmented using words from the dictionary.
- Leverage a hash set for fast look-up of dictionary words.
- For every index i , check all possible previous break points j such that $dp[j]$ is true and $s[j:i]$ is in the wordDict.
- The problem can also be approached with recursion and memoization, but DP is more straightforward for iterative implementation.

Space & Time

Time Complexity: $O(n^2)$ in the worst-case scenario, where n is the length of s (each substring check is $O(1)$ if using a hash set, and there could be $O(n^2)$ checks overall).

Space Complexity: $O(n)$ for the dp array and $O(k)$ for the hash set where k is the number of words in the dictionary.

Solution

The solution uses a Dynamic Programming (DP) approach: Convert wordDict to a hash set for constant time lookup. Create a boolean dp array of size $n+1$ (n is the length of s), where $dp[i]$ indicates if $s[0:i]$ can be segmented. Set $dp[0]$ to true because an empty string is always segmentable. For each index i from 1 to n , iterate through all indices j from 0 to i : If $dp[j]$ is true and the substring $s[j:i]$ is in the dictionary, then set $dp[i]$ to true. Break early from the inner loop when a valid break is found. Return $dp[n]$ as the result. This method efficiently builds up the solution by utilizing previous results to determine if the entire string can be segmented.

Trie

□ #208 Implement Trie (Prefix Tree)

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Design · Trie
Lists: Grind 169

Problem

A trie (pronounced as "try") or prefix tree is a tree data structure used to efficiently store and retrieve keys in a dataset of strings. There are various applications of this data structure, such as autocomplete and spellchecker.

Implement the Trie class:

- **Trie()** Initializes the trie object.
- **void insert(String word)** Inserts the string word into the trie.
- **boolean search(String word)** Returns true if the string word is in the trie (i.e., was inserted before), and false otherwise.
- **boolean startsWith(String prefix)** Returns true if there is a previously inserted string

word that has the prefix prefix, and false otherwise.

Example 1:

Input

```
["Trie", "insert", "search", "search", "startsWith", "insert", "search"]
[[], ["apple"], ["apple"], ["app"], ["app"], ["app"], ["app"]]
```

Output

```
[null, null, true, false, true, null, true]
```

Explanation

```
Trie trie = new Trie();
```

Constraints:

- $1 \leq \text{word.length}, \text{prefix.length} \leq 2000$
- word and prefix consist only of lowercase English letters.
- At most $3 * 10^4$ calls in total will be made to insert, search, and startsWith.

My LeetCode Solution

• My LeetCode Solution

```
import collections

class TrieNode:

    def __init__(self):
        self.child = collections.defaultdict(TrieNode)
        self.is_word = False

class Trie:
```

```

def __init__(self):
    self.root = TrieNode()

def insert(self, word: str) -> None:
    cur = self.root
    for letter in word:
        cur = cur.child[letter]
        cur.is_word = True

def search(self, word: str) -> bool:

    cur = self.root
    for letter in word:
        # cur = cur.child[letter] ==> will not work, it will creat a default node for
        # this letter
        cur = cur.child.get(letter)
        if not cur:
            return False
        return cur.is_word

def startsWith(self, prefix: str) -> bool:
    cur = self.root

    for letter in prefix:
        # cur = cur.child[letter] ==> will not work, it will creat a default node for
        # this letter
        cur = cur.child.get(letter)
        if not cur:
            return False
        return True

# Your Trie object will be instantiated and called as such:
# obj = Trie()
# obj.insert(word)

# param_2 = obj.search(word)
# param_3 = obj.startsWith(prefix)

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

```
# Implement a Trie with insert(word), search(word)→bool (exact match),
# and startsWith(prefix)→bool. Lowercase letters only. Right?"
#
# "A few quick questions:
# search('app') should return false even after inserting 'apple'? Yes – exact match
# only.
# startsWith('app') should return true? Yes – any inserted word with that prefix."
#
# "Let me walk: insert('apple'). search('apple')→T. search('app')→F.
# startsWith('app')→T. insert('app'). search('app')→T ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Use a set for exact search and iterate all words for prefix check.
# search O(1), startsWith O(n * m) – slow for many queries.
# Should I go ahead and optimize?"
class _208_ImplementTrieBrute:
    def __init__(self): self.words = set()
    def insert(self, word): self.words.add(word)
    def search(self, word): return word in self.words
    def startsWith(self, prefix): return any(w.startswith(prefix) for w in
self.words)
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the O(n*m) prefix scan.
# True trie: each node has up to 26 children and an is_end flag.
# insert: create nodes along the path, mark is_end at the last node.
# search: walk path, return is_end of the final node (or False if path missing).
# startsWith: walk path, return True if the path exists completely.
#
# Time: O(m) per operation where m = word/prefix length.
# Space: O(total characters * 26)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _208_ImplementTrie:
    def __init__(self):
        self._children = {}
        self._is_end = False
    def insert(self, word):
        node = self
        for ch in word:
            if ch not in node._children:
                node._children[ch] = _208_ImplementTrie()
            node = node._children[ch]
        node._is_end = True
    def search(self, word):
        node = self
        for ch in word:
            if ch not in node._children:
```

```

        return False
    node = node._children[ch]
    return node._is_end
def startsWith(self, prefix):
    node = self
    for ch in prefix:
        if ch not in node._children:
            return False
        node = node._children[ch]
    return True

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

insert('apple'): root→a→p→p→l→e(is_end=True).

search('apple'): walk path → is_end=True ✓.

search('app'): walk a→p→p → is_end=False → False ✓.

startsWith('app'): walk a→p→p → path exists → True ✓.

insert('app'): node at second p gets is_end=True. search('app')→True ✓.

#

"No bugs. Using dict children is memory-efficient for sparse tries vs a fixed 26-slot array."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m)$ per op: walk at most m nodes. Space $O(n*m)$: n words of avg length m .

Dict children vs array[26]: dict saves space for sparse alphabets.

Follow-up: delete a word — mark is_end=False; prune childless nodes bottom-up.

Follow-up: Word Search II (#212) — DFS the board against a trie of all words."

Community Solutions (Python)

• WalkCC

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.isWord = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

```

```
def insert(self, word: str) -> None:
    node: TrieNode = self.root
    for c in word:
        node = node.children.setdefault(c, TrieNode())
    node.isWord = True

def search(self, word: str) -> bool:
    node = self._find(word)
    return node is not None and node.isWord
```

```
def startsWith(self, prefix: str) -> bool:
    return self._find(prefix) is not None

def _find(self, prefix: str) -> TrieNode | None:
    node: TrieNode = self.root
    for c in prefix:
        if c not in node.children:
            return None
        node = node.children[c]
    return node
```

• LeetDoocs

```
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, word: str) -> None:
        node = self
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
```

```

        node.children[idx] = Trie()
        node = node.children[idx]
        node.is_end = True

def search(self, word: str) -> bool:
    node = self._search_prefix(word)
    return node is not None and node.is_end

def startsWith(self, prefix: str) -> bool:
    node = self._search_prefix(prefix)

    return node is not None

def _search_prefix(self, prefix: str):
    node = self
    for c in prefix:
        idx = ord(c) - ord('a')
        if node.children[idx] is None:
            return None
        node = node.children[idx]
    return node

```

```

# Your Trie object will be instantiated and called as such:
# obj = Trie()
# obj.insert(word)
# param_2 = obj.search(word)
# param_3 = obj.startsWith(prefix)

```

• SimplyLeet

```

class TrieNode:
    def __init__(self):
        # Dictionary to store child nodes and a flag for end of word.
        self.children = {}
        self.is_end_of_word = False

class Trie:
    def __init__(self):
        # Initialize the Trie with the root node.
        self.root = TrieNode()

```

```
def insert(self, word: str) -> None:
    # Start from the root.
    node = self.root
    for char in word:
        # If character doesn't exist, add a new TrieNode.
        if char not in node.children:
            node.children[char] = TrieNode()
        node = node.children[char]
    # Mark the end of word.
```

```
node.is_end_of_word = True
```

```
def search(self, word: str) -> bool:
    # Traverse the Trie for each character.
    node = self.root
    for char in word:
        if char not in node.children:
            return False
        node = node.children[char]
    # Returns True if word ends exactly here.
```

```
return node.is_end_of_word
```

```
def startsWith(self, prefix: str) -> bool:
    # Traverse checking for the prefix.
    node = self.root
    for char in prefix:
        if char not in node.children:
            return False
        node = node.children[char]
    return True
```

• LC.ca


```
import collections
```

```
class TrieNode:
```

```
def __init__(self):
```

```
    """
```

Usually, a Python dictionary throws a `KeyError` if you try to get an item with a key that is not currently in the dictionary.

The `defaultdict` in contrast will simply create any items that you try to access

<https://stackoverflow.com/questions/5900578/how-does-collections-defaultdict-work>

```
    """
```

```
self.child = collections.defaultdict(TrieNode)
```

```
self.is_word = False
```

```
class Trie:
```

```
def __init__(self):
```

```
self.root = TrieNode()
```

```
def insert(self, word: str) -> None:
```

```
    cur = self.root
```

```
    for letter in word:
```

```
        cur = cur.child[letter]
```

```
    cur.is_word = True
```

```
def search(self, word: str) -> bool:
```

```
    cur = self.root
```

```
    for letter in word:
```

`cur = cur.child[letter]` ==> will not work, it will create a default node for this letter

```
    cur = cur.child.get(letter)
```

```

if not cur:
    return False
return cur.is_word

def startsWith(self, prefix: str) -> bool:
    cur = self.root
    for letter in prefix:
        # cur = cur.child[letter] ==> will not work, it will creat a default node for
        # this letter
        cur = cur.child.get(letter)
        if not cur:

            return False
            return True

# Your Trie object will be instantiated and called as such:
# obj = Trie()
# obj.insert(word)
# param_2 = obj.search(word)
# param_3 = obj.startsWith(prefix)

#####

class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

    def insert(self, word: str) -> None:
        node = self
        for c in word:

```

```

        idx = ord(c) - ord('a')
        if node.children[idx] is None:
            node.children[idx] = Trie()
        node = node.children[idx]
    node.is_end = True

```

```

def search(self, word: str) -> bool:
    node = self._search_prefix(word)
    return node is not None and node.is_end

```

```

def startsWith(self, prefix: str) -> bool:
    node = self._search_prefix(prefix)
    return node is not None

```

```

def _search_prefix(self, prefix: str):
    node = self
    for c in prefix:
        idx = ord(c) - ord('a')
        if node.children[idx] is None:
            return None

```

```

        node = node.children[idx]
    return node

```

```

# Your Trie object will be instantiated and called as such:
# obj = Trie()
# obj.insert(word)
# param_2 = obj.search(word)
# param_3 = obj.startsWith(prefix)

```

```

# below is using reduce()
class Trie(object):

    def __init__(self):
        T = lambda: collections.defaultdict(T)
        self.root = T()

    def insert(self, word):

```

```
reduce(dict.__getitem__, word, self.root)['#'] = True

def search(self, word):
    return '#' in reduce(lambda cur, c: cur.get(c, {}), word, self.root)

def startsWith(self, prefix):
    return bool(reduce(lambda cur, c: cur.get(c, {}), prefix, self.root))
```

◆ Quick Review

Key Insights

- Use a tree-like structure where each node represents a character.
- Each node stores its children in a dictionary (or hash map) for $O(1)$ average time access.
- Mark the end of a word with a boolean flag.
- Insertion is done character by character, creating new nodes as needed.
- Searching and checking prefixes follow traversals down the tree.

Space & Time

Time Complexity:

Insert: $O(m)$ where m is the length of the word.

Search: $O(m)$

startsWith: $O(m)$

Space Complexity:

$O(N * M)$ in the worst-case scenario, where N is the number of words and M is the average length of the words, due to storing each character in the Trie.

Solution

We implement the Trie using a node-based approach. Each `TrieNode` contains a hash map (or dictionary) of its children and a boolean indicating if the node marks the end of a word. The Trie class uses these nodes to build out the words character by character. The `insert` method traverses and builds the necessary nodes; the `search` method verifies that all characters exist in order and that the final node marks the end of a word; the `startsWith` method checks if the prefix exists in the Trie. No tricky edge cases exist beyond handling null or empty inputs, as constraints guarantee valid lowercase strings.

Trie

□ #211 Design Add and Search Words Data Structure **MED**

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · DFS · Design · Trie
Lists: Grind 169

Problem

Design a data structure that supports adding new words and finding if a string matches any previously added string.

Implement the WordDictionary class:

- WordDictionary() Initializes the object.
- void addWord(word) Adds word to the data structure, it can be matched later.
- bool search(word) Returns true if there is any string in the data structure that matches word or false otherwise. word may contain dots '.' where dots can be matched with any letter.

Example:

Input

```
["WordDictionary", "addWord", "addWord", "addWord", "search", "search", "search", "search"]
```

```
[[], ["bad"], ["dad"], ["mad"], ["pad"], ["bad"], [".ad"], ["b.."]]
```

Output

```
[null, null, null, null, false, true, true, true]
```

Explanation

```
WordDictionary wordDictionary = new WordDictionary();
```

Constraints:

- $1 \leq \text{word.length} \leq 25$
- word in addWord consists of lowercase English letters.
- word in search consist of '.' or lowercase English letters.
- There will be at most 2 dots in word for search queries.
- At most 104 calls will be made to addWord and search.

My LeetCode Solution

☐ • My LeetCode Solution

```
class TrieNode:
    def __init__(self):
        # Dictionary to store child nodes
        self.children = {}
        # Flag to indicate if a word ends here
        self.is_end = False

class WordDictionary:
    def __init__(self):
        # Initialize the Trie with an empty root node
```



```

        self.root = TrieNode()

    def addWord(self, word: str) -> None:
        # Start at the root of the Trie
        node = self.root
        for char in word:
            # Create a new node if the character is not present
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]

        # Mark the end of the word
        node.is_end = True

    def search(self, word: str) -> bool:
        # DFS helper function
        def dfs(index, node):
            if index == len(word):
                return node.is_end
            char = word[index]
            if char == '.':
                # If wildcard, search all children nodes
                for child in node.children.values():
                    if dfs(index + 1, child):
                        return True
                return False
            else:
                # If character not found, word does not exist
                if char not in node.children:
                    return False
                return dfs(index + 1, node.children[char])

        return dfs(0, self.root)

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Design a data structure with addWord(word) and search(word).

search supports '.' as a wildcard matching any single letter. Right?"

#

"A few quick questions:

```
# Can the pattern have multiple dots? Yes – up to 2 per problem constraints.
# search('.') should match any one-character word? Yes."
#
# "Let me walk: addWord('bad','dad','mad'). search('.ad')→true. search('b..')→true
✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Store all words in a list. For search, scan all words and match character by
character,
# treating '.' as wildcard.
# O(n * m) per search where n = number of words, m = word length.
# Should I go ahead and optimize?"
class _211_AddSearchWords_Brute:
    def __init__(self): self.words = []
    def addWord(self, word): self.words.append(word)
    def search(self, word):
        for w in self.words:
            if len(w) != len(word): continue
            if all(w[i] == word[i] or word[i] == '.' for i in range(len(word))):
                return True
        return False
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is scanning all words on each search.
# Trie with DFS for wildcards:
# addWord: standard trie insert O(m).
# search: walk trie; on '.' try ALL children recursively; on a letter take the
matching child.
# Prune any branch that leads to a missing character.
#
# Time: addWord O(m). search O(m) best, O(26^dots * m) worst with dots.
# Space: O(total chars)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _211_AddSearchWords:
    def __init__(self):
        self._children = {}
        self._is_end = False
    def addWord(self, word):
        node = self
        for ch in word:
            if ch not in node._children:
                node._children[ch] = _211_AddSearchWords()
            node = node._children[ch]
        node._is_end = True
    def search(self, word):
        def dfs(node, i):
            if i == len(word):
                return node._is_end
```

```

        return node._is_end
    ch = word[i]
    if ch == '.':
        return any(dfs(child, i + 1) for child in node._children.values())
    if ch not in node._children:
        return False
    return dfs(node._children[ch], i + 1)
return dfs(self, 0)

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

addWord('bad','dad','mad'): b→a→d(end), d→a→d(end), m→a→d(end).

search('.ad'): dfs(root,0): ch='.', try b,d,m → dfs(b_node,1): 'a'→dfs(a_node,2):

'd'→is_end=T → True ✓

search('b..'): dfs(root,0): 'b'→dfs(b,1): '.'→try 'a'→dfs(a,2): '.'→try

'd'→is_end=T → True ✓

search('pad'): 'p' not in children → False ✓

#

"No bugs. DFS correctly backtracks when a '.' branch leads to a dead end."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"addWord O(m). search O(m) for no dots; O(26^k * m) worst with k dots.

In practice dots are rare, so average is close to O(m).

Follow-up: if patterns are very dot-heavy, cache results at (node, pattern_index) pairs.

Follow-up: support '*' (zero or more chars) – extended DFS with skip-0 and skip-1+ logic."

Community Solutions (Python)

• WalkCC

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.isWord = False

class WordDictionary:
    def __init__(self):
        self.root = TrieNode()

```

```
def addWord(self, word: str) -> None:
    node: TrieNode = self.root
    for c in word:
        node = node.children.setdefault(c, TrieNode())
    node.isWord = True

def search(self, word: str) -> bool:
    return self._dfs(word, 0, self.root)

def _dfs(self, word: str, s: int, node: TrieNode) -> bool:
```

```
    if s == len(word):
        return node.isWord
    if word[s] != '.':
        child: TrieNode = node.children.get(word[s], None)
        return self._dfs(word, s + 1, child) if child else False
    return any(self._dfs(word, s + 1, child) for child in node.children.values
())
```

• LeetDoocs

```
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

class WordDictionary:
    def __init__(self):
        self.trie = Trie()

    def addWord(self, word: str) -> None:
        node = self.trie
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

    def search(self, word: str) -> bool:
```

```
def search(word, node):
    for i in range(len(word)):
        c = word[i]
        idx = ord(c) - ord('a')
        if c != '.' and node.children[idx] is None:
            return False
        if c == '.':
            for child in node.children:
                if child is not None and search(word[i + 1 :], child):
                    return True
```

```
        return False
        node = node.children[idx]
    return node.is_end
```

```
return search(word, self.trie)
```

```
# Your WordDictionary object will be instantiated and called as such:
# obj = WordDictionary()
# obj.addWord(word)
```

```
# param_2 = obj.search(word)
```

• SimplyLeet

```
class TrieNode:
    def __init__(self):
        # Dictionary to store child nodes
        self.children = {}
        # Flag to indicate if a word ends here
        self.is_end = False

class WordDictionary:
    def __init__(self):
        # Initialize the Trie with an empty root node
```

```
self.root = TrieNode()

def addWord(self, word: str) -> None:
    # Start at the root of the Trie
    node = self.root
    for char in word:
        # Create a new node if the character is not present
        if char not in node.children:
            node.children[char] = TrieNode()
        node = node.children[char]

    # Mark the end of the word
    node.is_end = True

def search(self, word: str) -> bool:
    # DFS helper function
    def dfs(index, node):
        if index == len(word):
            return node.is_end
        char = word[index]
        if char == '.':
            # If wildcard, search all children nodes
            for child in node.children.values():
                if dfs(index + 1, child):
                    return True
            return False
        else:
            # If character not found, word does not exist
            if char not in node.children:
                return False
            return dfs(index + 1, node.children[char])

    return dfs(0, self.root)

# Example usage:
wd = WordDictionary()
wd.addWord("bad")
wd.addWord("dad")
wd.addWord("mad")
print(wd.search("pad")) # Output: False
print(wd.search("bad")) # Output: True
print(wd.search(".ad")) # Output: True
```

```
print(wd.search("b..")) # Output: True
```

• LC.ca

```
class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.is_end = False

class WordDictionary:
    def __init__(self):
        self.trie = Trie()

    def addWord(self, word: str) -> None:
        node = self.trie
        for c in word:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

    def search(self, word: str) -> bool:

        def search(word, node):
            for i in range(len(word)):
                c = word[i]
                idx = ord(c) - ord('a')
                if c != '.' and node.children[idx] is None:
                    return False
                if c == '.':
                    for child in node.children:
                        if child is not None and search(word[i + 1 :], child):
                            return True
```

```
        return False
        node = node.children[idx]
        return node.is_end

    return search(word, self.trie)

# Your WordDictionary object will be instantiated and called as such:
# obj = WordDictionary()
# obj.addWord(word)

# param_2 = obj.search(word)
```

◆ Quick Review

Key Insights

- Utilize a Trie (prefix tree) to efficiently store words.
- Each node in the Trie contains a dictionary for its children and a flag indicating the end of a word.
- The search operation is implemented with a depth-first search (DFS) approach.
- When the search query contains a dot, explore all possible child nodes.

Space & Time

Time Complexity:

addWord: $O(w)$, where w is the length of the word.

search: Worst-case $O(26^w)$ for words with multiple wildcards, but practical performance is better given constraints.

Space Complexity: $O(n * w)$, where n is the number of words and w is the average word length.

Solution

The WordDictionary is implemented using a Trie. For adding, we insert each character of the word into the Trie. For searching, we use DFS. When encountering a wildcard '.', we recursively search all children nodes. This structure combined with DFS efficiently addresses the wildcard matching and ensures good performance for multiple operations.

Trie

□ ★ #616 Add Bold Tag in String ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Trie
Lists: ★ Premium | Premium 98

Problem

You are given a string *s* and an array of strings *words*.

You should add a closed pair of bold tag **** and **** to wrap the substrings in *s* that exist in *words*.

- If two such substrings overlap, you should wrap them together with only one pair of closed bold-tag.
- If two substrings wrapped by bold tags are consecutive, you should combine them.

Return *s* after adding the bold tags.

Example 1:

Input: s = "abcxyz123", words = ["abc","123"]

Output: "abcxyz123"

Explanation: The two strings of words are substrings of s as following: "abcxyz123".

We add before each substring and after each substring.

Example 2:

Input: s = "aaabbb", words = ["aa","b"]

Output: "aaabbb"

Explanation:

"aa" appears as a substring two times: "aaabbb" and "aaabbb".

"b" appears as a substring three times: "aaabbb", "aaabbb", and "aaabbb".

We add before each substring and after each substring:

"aaabbb".

Since the first two 's overlap, we merge them:

"aaabbb".

Since now the four 's are consecutive, we merge them: "aaabbb".

Constraints:

- $1 \leq s.length \leq 1000$
- $0 \leq words.length \leq 100$
- $1 \leq words[i].length \leq 1000$
- s and words[i] consist of English letters and digits.
- All the values of words are unique.

Note: This question is the same as 758. **Bold Words in String.**

My LeetCode Solution

- My LeetCode Solution

```

class Solution:
    # Python Code Solution
    def addBoldTag(self, s, words):
        n = len(s)
        # Mark positions to be bolded
        bold = [False] * n
        # Iterate each word and mark its occurrences
        for word in words:
            start = s.find(word)
            while start != -1:
                for i in range(start, start + len(word)):
                    bold[i] = True
                start = s.find(word, start + 1)

        # Build the final result with bold tags
        result = []
        i = 0
        while i < n:
            if not bold[i]:
                result.append(s[i])
                i += 1
            else:
                result.append("<b>")
                while i < n and bold[i]:
                    result.append(s[i])
                    i += 1
                result.append("</b>")
        return "".join(result)

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Wrap all substrings of s that appear in the words list with tags.

Overlapping or adjacent bold regions should be merged into one. Right?"

#

"A few quick questions:

A word can appear multiple times in s – all occurrences get bolded? Yes.

Overlapping matches merge into one ... block? Yes."

#

"Let me walk: s='abcxyz123', words=['abc', '123'] → 'abcxyz123' ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
 # Use a boolean array `bold[i] = should s[i] be bolded?`
 # For each word, find all occurrences in `s` and mark those positions `True`.
 # Then walk `bold` to insert `<b>` and `</b>` tags at transitions.
 # $O(n * m * L)$ where m = number of words, L = max word length.
 # Should I go ahead and optimize?"

```
class _616_AddBoldTag_Brute:
    def addBoldTag(self, s, words):
        bold = [False] * len(s)
        for word in words:
            start = 0
            while (idx := s.find(word, start)) != -1:
                for i in range(idx, idx + len(word)): bold[i] = True
                start = idx + 1
        result = []; i = 0
        while i < len(s):
            if bold[i]:
                result.append('<b>')
                while i < len(s) and bold[i]: result.append(s[i]); i += 1
                result.append('</b>')
            else:
                result.append(s[i]); i += 1
        return ''.join(result)
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n * m * L)$ find loop.
 # Trie optimization: insert all words into a trie, then scan `s` once.
 # At each position, walk the trie to find all word matches starting there.
 # Mark the corresponding range in the `bold` array.
 # $O(n * \max_L)$ scan.
 # But the brute with `bool` array is already clean – let's refine it using intervals."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _616_AddBoldTag:
    def addBoldTag(self, s, words):
        n = len(s)
        bold = [False] * n
        for word in words:
            wl = len(word)
            for i in range(n - wl + 1):
                if s[i:i+wl] == word:
                    for j in range(i, i + wl):
                        bold[j] = True
        result = []; i = 0
        while i < n:
            if bold[i]:
                result.append('<b>')
                while i < n and bold[i]:
```

```

        result.append(s[i]); i += 1
    result.append('</b>')
else:
    result.append(s[i]); i += 1
return ''.join(result)

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

s='aaabbb', words=['aa','b']:

'aa': matches at 0,1 → bold[0..2]=True.

'b': matches at 3,4,5 → bold[3..5]=True. All bold → 'aaabbb' ✓

#

s='abcxyz123', words=['abc','123']:

bold[0..2]=True, bold[6..8]=True. → 'abcxyz123' ✓

#

"No bugs. Adjacent True ranges merge naturally since the while loop consumes all."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n * m * L)$: n positions × m words × L substring check. Space $O(n)$ bold array.

Trie reduces substring matching to $O(n * \max_L)$ – worth it when m is large.

Follow-up: Bold Words in String (#758) – identical problem, same solution.

Follow-up: Merge Intervals (#56) – if instead of a bool array we track intervals directly."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def addBoldTag(self, s: str, words: list[str]) -> str:
        n = len(s)
        ans = []
        # bold[i] := True if s[i] should be bolded
        bold = [0] * n

        boldEnd = -1 # s[i:boldEnd] should be bolded
        for i in range(n):
            for word in words:

```

```

        if s[i:].startswith(word):
            boldEnd = max(boldEnd, i + len(word))
        bold[i] = boldEnd > i

```

```

# Construct the with bold tags

```

```

i = 0
while i < n:
    if bold[i]:
        j = i
        while j < n and bold[j]:

```

```

            j += 1
            # `s[i..j)` should be bolded.
            ans.append('<b>' + s[i:j] + '</b>')
            i = j
        else:
            ans.append(s[i])
            i += 1

```

```

return ''.join(ans)

```

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.isWord = False

```

```

class Solution:
    def addBoldTag(self, s: str, words: list[str]) -> str:
        n = len(s)
        ans = []

```

```

        # bold[i] := True if s[i] should be bolded
        bold = [0] * n
        root = TrieNode()

```

```

    def insert(word: str) -> None:
        node = root
        for c in word:
            node = node.children.setdefault(c, TrieNode())
        node.isWord = True

```



```

def find(s: str, i: int) -> int:
    node = root
    ans = -1
    for j in range(i, len(s)):
        if s[j] not in node.children:
            node.children[s[j]] = TrieNode()
        node = node.children[s[j]]
        if node.isWord:
            ans = j
    return ans

for word in words:
    insert(word)

boldEnd = -1 # `s[i..boldEnd]` should be bolded.
for i in range(n):
    boldEnd = max(boldEnd, find(s, i))
    bold[i] = boldEnd >= i

# Construct the with bold tags

i = 0
while i < n:
    if bold[i]:
        j = i
        while j < n and bold[j]:
            j += 1
        # `s[i..j]` should be bolded.
        ans.append('<b>' + s[i:j] + '</b>')
        i = j
    else:
        ans.append(s[i])
        i += 1

return ''.join(ans)

```

• LeetDoocs

```

class Trie:
    def __init__(self):
        self.children = [None] * 128
        self.is_end = False

    def insert(self, word):
        node = self
        for c in word:
            idx = ord(c)
            if node.children[idx] is None:

                node.children[idx] = Trie()
                node = node.children[idx]
            node.is_end = True

class Solution:
    def addBoldTag(self, s: str, words: List[str]) -> str:
        trie = Trie()
        for w in words:
            trie.insert(w)

        n = len(s)
        pairs = []
        for i in range(n):
            node = trie
            for j in range(i, n):
                idx = ord(s[j])
                if node.children[idx] is None:
                    break
                node = node.children[idx]
                if node.is_end:

                    pairs.append([i, j])

        if not pairs:
            return s
        st, ed = pairs[0]
        t = []
        for a, b in pairs[1:]:
            if ed + 1 < a:
                t.append([st, ed])
                st, ed = a, b
            else:

```

```

        ed = max(ed, b)
    t.append([st, ed])

    ans = []
    i = j = 0
    while i < n:
        if j == len(t):
            ans.append(s[i:])
            break
        st, ed = t[j]

        if i < st:
            ans.append(s[i:st])
        ans.append('<b>')
        ans.append(s[st : ed + 1])
        ans.append('</b>')
        j += 1
        i = ed + 1

    return ''.join(ans)

```

• SimplyLeet

```

# Python Code Solution
def addBoldTag(s, words):
    n = len(s)
    # Mark positions to be bolded
    bold = [False] * n
    # Iterate each word and mark its occurrences
    for word in words:
        start = s.find(word)
        while start != -1:
            for i in range(start, start + len(word)):

```

```

        bold[i] = True
        start = s.find(word, start + 1)

# Build the final result with bold tags
result = []
i = 0
while i < n:
    if not bold[i]:
        result.append(s[i])
        i += 1

    else:
        result.append("<b>")
        while i < n and bold[i]:
            result.append(s[i])
            i += 1
        result.append("</b>")
return "".join(result)

```

```

# Example usage:
# s = "abcxyz123"

```

```

# words = ["abc", "123"]
# print(addBoldTag(s, words)) # "<b>abc</b>xyz<b>123</b>"

```

```

# Line-by-line:
# 1. Initialize an array "bold" of length n to track which characters need bold
    formatting.
# 2. For each word, use string.find to locate every occurrence and mark the
    corresponding indices.
# 3. Traverse the string s. If the current character is marked bold and it is
    the start of a bold segment,
# insert the opening tag before appending characters until the end of that bold
    segment.
# 4. Append the closing tag after the bold segment and continue the process.

```

• **LC.ca**

```

class Trie:
    def __init__(self):
        self.children = [None] * 128
        self.is_end = False

    def insert(self, word):
        node = self
        for c in word:
            idx = ord(c)
            if node.children[idx] is None:

                node.children[idx] = Trie()
                node = node.children[idx]
            node.is_end = True

class Solution:
    def addBoldTag(self, s: str, words: List[str]) -> str:
        trie = Trie()
        for w in words:
            trie.insert(w)

        n = len(s)
        pairs = []
        for i in range(n):
            node = trie
            for j in range(i, n):
                idx = ord(s[j])
                if node.children[idx] is None:
                    break
                node = node.children[idx]
                if node.is_end:

                    pairs.append([i, j])

        if not pairs:
            return s
        st, ed = pairs[0]
        t = []
        for a, b in pairs[1:]:
            if ed + 1 < a:
                t.append([st, ed])
                st, ed = a, b
            else:

```

```
        ed = max(ed, b)
    t.append([st, ed])

    ans = []
    i = j = 0
    while i < n:
        if j == len(t):
            ans.append(s[i:])
            break
        st, ed = t[j]

        if i < st:
            ans.append(s[i:st])
        ans.append('<b>')
        ans.append(s[st : ed + 1])
        ans.append('</b>')
        j += 1
        i = ed + 1

    return ''.join(ans)
```

◆ Quick Review

Key Insights

- We need to identify every occurrence of each word in s.
- Overlapping or consecutive intervals should be merged to form one continuous bold region.
- Use an auxiliary data structure (e.g., a boolean array) to mark positions that should be bolded.

- Once the positions are marked, build the final string by inserting bold tags at the appropriate boundaries.

Space & Time

Time Complexity: $O(n * m * k)$ where n is the length of s , m is the number of words, and k is the average length of the words (due to substring matching).

Space Complexity: $O(n)$ for the auxiliary boolean array used to mark bold positions.

Solution

The solution involves two main steps: Identify and mark the indices in the string s that are part of any matching word. We do this by iterating over each word and marking every occurrence's start and end in a boolean array. Construct the final string by traversing the boolean array. When encountering a segment of consecutive true values, wrap that segment with **and** tags. The key idea is to merge intervals by marking every index that should be bold and then using a single pass to insert tags only when transitioning from non-bold to bold and vice versa.

Trie

□ #692 Top K Frequent Words

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Trie · Sorting · Heap
Lists: Grind 169

Problem

Given an array of strings `words` and an integer `k`, return the `k` most frequent strings.

Return the answer sorted by the frequency from highest to lowest. Sort the words with the same frequency by their lexicographical order.

Example 1:

```
Input: words = ["i","love","leetcode","i","love","coding"], k = 2  
Output: ["i","love"]  
Explanation: "i" and "love" are the two most frequent words.  
Note that "i" comes before "love" due to a lower alphabetical order.
```

Example 2:

```
Input: words = ["the","day","is","sunny","the","the","the","sunny","is","is"],  
k = 4  
Output: ["the","is","sunny","day"]  
Explanation: "the", "is", "sunny" and "day" are the four most frequent words,  
with the number of occurrence being 4, 3, 2 and 1 respectively.
```

Constraints:

- $1 \leq \text{words.length} \leq 500$
- $1 \leq \text{words}[i].\text{length} \leq 10$
- $\text{words}[i]$ consists of lowercase English letters.
- k is in the range $[1, \text{The number of unique words}[i]]$

Follow-up: Could you solve it in $O(n \log(k))$ time and $O(n)$ extra space?

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def topKFrequent(self, words, k):
        count = Counter(words)
        heap = []

        for word, freq in count.items():
            heapq.heappush(heap, (freq, -ord(word[0]), word))
        heap = []
        for word, freq in count.items():
            heapq.heappush(heap, (-freq, word))

        # Extract k elements from the heap and sort them accordingly.
        result = []
        for _ in range(k):
            freq, word = heapq.heappop(heap)
            result.append(word)
        return result
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Let me make sure I understand the problem correctly.
# Return the k most frequent words, sorted by frequency descending.
# Ties broken alphabetically ascending. Right?"
#
# "A few quick questions:
# k is always valid ( $\leq$  unique words)? Yes. Words are lowercase only?"
#
# "Let me walk: words=['i','love','leetcode','i','love','coding'], k=2.
# i(2)>love(2) alphabetically: i < love  $\rightarrow$  [i,love] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Count frequencies, sort by (-freq, word) as a composite key, take first k.
#  $O(n \log n)$  – clean and readable.
# Should I go ahead and optimize?"
class _692_TopKFrequentWords_Brute:
    def topKFrequent(self, words, k):
        from collections import Counter
        count = Counter(words)
        return sorted(count.keys(), key=lambda w: (-count[w], w))[:k]

# PHASE 3 — OPTIMIZE
# "The bottleneck is sorting all unique words.
# Min-heap of size k with a custom comparator handles this in  $O(n \log k)$ :
# Store (-freq, word) – heap evicts the least desirable (smallest freq, then
# lex-largest).
# This keeps the k best candidates at all times.
#
# Time:  $O(n \log k)$ . Space:  $O(k)$ ."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
import heapq
class _692_TopKFrequentWords:
    def topKFrequent(self, words, k):
        from collections import Counter
        count = Counter(words)
        heap = []
        for word, freq in count.items():
            heapq.heappush(heap, (-freq, word))
        return [heapq.heappop(heap)[1] for _ in range(k)]

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# words=['i','love','leetcode','i','love','coding'], k=2:
# count={'i':2,love:2,leetcode:1,coding:1}.
# heap: [(-2,'i'),(-2,'love'),(-1,'coding'),(-1,'leetcode')] (min-heap by -freq
# then word).
# pop1: (-2,'i'). pop2: (-2,'love').  $\rightarrow$  ['i','love'] ✓
#

```

"No bugs. Negating freq makes Python's min-heap act as a max-heap on frequency."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Heap $O(n \log k)$. Sort $O(n \log n)$. Heap wins when $k \ll n$."

Tuple comparison $(-freq, word)$ handles ties alphabetically for free in Python.

Follow-up: Top K Frequent Elements (#347) – integers, no tie-breaking, bucket sort $O(n)$.

Follow-up: streaming top-k words – `heapq.nsmallest` after each word arrives."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def topKFrequent(self, words: list[str], k: int) -> list[str]:
        ans = []
        bucket = [[] for _ in range(len(words) + 1)]

        for word, freq in collections.Counter(words).items():
            bucket[freq].append(word)

        for b in reversed(bucket):
            for word in sorted(b):

                ans.append(word)
                if len(ans) == k:
                    return ans

from dataclasses import dataclass

@dataclass(frozen=True)
class T:
    word: str
    freq: int

    def __lt__(self, other):
        if self.freq == other.freq:
```

```

        return self.word > other.word
    return self.freq < other.freq

class Solution:
    def topKFrequent(self, words: list[str], k: int) -> list[str]:
        ans = []
        heap = []

        for word, freq in collections.Counter(words).items():

            heapq.heappush(heap, T(word, freq))
            if len(heap) > k:
                heapq.heappop(heap)

        while heap:
            ans.append(heapq.heappop(heap).word)

        return ans[::-1]

```

• LeetDoocs

```

class Solution:
    def topKFrequent(self, words: List[str], k: int) -> List[str]:
        cnt = Counter(words)
        return sorted(cnt, key=lambda x: (-cnt[x], x))[:k]

```

• SimplyLeet

```

import heapq
from collections import Counter

def topKFrequent(words, k):
    # Count the frequency of each word
    count = Counter(words)

    # Initialize a min-heap; Python's heapq is a min-heap.
    # The comparator is (frequency, negative word) but use word in reverse
    order:
    # Since we want words with lower alphabetical order to be prioritized in
    the result,

```

```

    # and in the min heap if frequencies equal, we want the one with
    lexicographically larger word to be popped.
    heap = []

    for word, freq in count.items():
        # Push each pair into the heap. The tuple ordering:
        # First element is frequency and second element is the word (with
        reverse lex order trick)
        heapq.heappush(heap, (freq, -ord(word[0]), word))
        # However, using -ord(word[0]) is not robust for multi-letter words.
        # Instead, we can use tuple (freq, word) with a custom comparator by
        ensuring we invert the word order by using word in reverse.
        # For simplicity, we push tuple (freq, word) and then pop if heap size
        > k with a custom condition.

```

```

    # Reset heap and use a custom comparator implemented by the tuple (-freq,
    word) to simulate max heap behavior.
    heap = []
    for word, freq in count.items():
        # We push with frequency as negative so that larger frequencies come
        first;
        # use word as is to satisfy lexicographical order when frequencies are
        equal.
        heapq.heappush(heap, (-freq, word))

    # Extract k elements from the heap and sort them accordingly.
    result = []

```

```

    for _ in range(k):
        freq, word = heapq.heappop(heap)
        result.append(word)
    return result

# Example usage:
print(topKfrequent(["i","love","leetcode","i","love","coding"], 2)) # Output:
['i', 'love']

```

• LC.ca

```

'''
>>> sorted(student_objects, key=attrgetter('grade', 'age'))
[('john', 'A', 15), ('dave', 'B', 10), ('jane', 'B', 12)]

ref: https://docs.python.org/3/howto/sorting.html#operator-module-functions
'''

class Solution:
    def topKFrequent(self, words: List[str], k: int) -> List[str]:
        cnt = Counter(words)
        # multiple comparators "lambda x: (-cnt[x], x)"

        return sorted(cnt, key=lambda x: (-cnt[x], x))[:k]
        # why the returned sorted() not a tuple (word, count), but only word?
        # it's the property of Counter() class
        # so, also pass 0J: return sorted(cnt.keys(), key=lambda x: (-cnt[x],
x))[:k]

'''
>>> words = ["i","love","leetcode","i","love","coding"]
>>> k = 2
>>> cnt = Counter(words)
>>> cnt

Counter({'i': 2, 'love': 2, 'leetcode': 1, 'coding': 1})

>>> sorted(cnt, key=lambda x: (-cnt[x], x))
['i', 'love', 'coding', 'leetcode']

# default, only for keys()
>>> sorted(cnt)
['coding', 'i', 'leetcode', 'love']
>>> sorted(cnt.keys())
['coding', 'i', 'leetcode', 'love']

```

```
>>> sorted(cnt.values())
[1, 1, 2, 2]
>>> sorted(cnt.items())
[('coding', 1), ('i', 2), ('leetcode', 1), ('love', 2)]
'''
```

```
#####
```

```
'''
```

```
>>> words = ["the","day","is","sunny","the","the","the","sunny","is","is"]
>>> count = collections.Counter(words)
>>> count
Counter({'the': 4, 'is': 3, 'sunny': 2, 'day': 1})
>>>
>>> count.items()
dict_items([('the', 4), ('day', 1), ('is', 3), ('sunny', 2)])
>>>
'''
```

```
'''
```

cmp doesn't exist in Python 3. If you really want it, you could define it yourself:

```
def cmp(a, b):
    return (a > b) - (a < b)
```

```
'''
```

```
class Solution(object): #python-2
    def topKFrequent(self, words, k):
        '''
```



```

:type words: List[str]
:type k: int
:rtype: List[str]
"""
count = collections.Counter(words)

# https://docs.python.org/3/howto/sorting.html#comparison-functions
def compare(x, y):
    if x[1] == y[1]:
        return cmp(x[0], y[0])

    else:
        return -cmp(x[1], y[1]) # -cmp, so reversed order, bigger ones
at front
return [x[0] for x in sorted(count.items(), cmp = compare)[:k]]

```

◆ Quick Review

Key Insights

- Count the frequency of each unique word using a hash table or dictionary.
- Use a min-heap (priority queue) to keep track of the top k frequent words. In the heap, the ordering is based on frequency; if frequencies are equal, order by lexicographical order (inverted condition for min-heap).
- Alternatively, you can sort the unique words based on frequency and lexicographical order and then pick the top k.
- For an efficient solution, use a min-heap that maintains size k achieving $O(n \log k)$ time

complexity.

Space & Time

Time Complexity: $O(n \log k)$ when using a heap, where n is the number of unique words.

Space Complexity: $O(n)$ for storing the frequency counts and the heap.

Solution

We first calculate the frequency of each word using a hash table (or dictionary). Then, we use a min-heap that stores pairs of (frequency, word). The comparator for the heap is defined such that the root of the heap is the word with the smallest frequency; if two words have the same frequency, the word with the lexicographically larger value is considered “smaller” in our ordering so that it gets popped when the heap size exceeds k . After processing all words, the heap contains the top k frequent words. Finally, the heap elements are extracted and reversed (if needed) to produce the final list sorted from highest to lowest frequency with lexicographic tie-breaking.

Trie

□ #720 Longest Word in Dictionary

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Trie · Sorting
Lists: Denny Zhang

Problem

Given an array of strings `words` representing an English Dictionary, return the longest word in `words` that can be built one character at a time by other words in `words`.

If there is more than one possible answer, return the longest word with the smallest lexicographical order. If there is no answer, return the empty string.

Note that the word should be built from left to right with each additional character being added to the end of a previous word.

Example 1:

Input: `words = ["w","wo","wor","worl","world"]`

Output: `"world"`

Explanation: The word `"world"` can be built one character at a time by `"w"`, `"wo"`, `"wor"`, and `"worl"`.

Example 2:

Input: words = ["a","banana","app","appl","ap","apply","apple"]

Output: "apple"

Explanation: Both "apply" and "apple" can be built from other words in the dictionary. However, "apple" is lexicographically smaller than "apply".

Constraints:

- $1 \leq \text{words.length} \leq 1000$
- $1 \leq \text{words}[i].\text{length} \leq 30$
- $\text{words}[i]$ consists of lowercase English letters.

My LeetCode Solution

• My LeetCode Solution

```
class Trie:
    def __init__(self):
        self.children: List[Optional[Trie]] = [None] * 26
        self.is_end = False

    def insert(self, w: str):
        node = self
        for c in w:
            idx = ord(c) - ord("a")
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.is_end = True

    def search(self, w: str) -> bool:
        node = self
        for c in w:
            idx = ord(c) - ord("a")
            if node.children[idx] is None:
                return False
```

```

        node = node.children[idx]
        if not node.is_end:
            return False
        return True

class Solution:
    def longestWord(self, words: List[str]) -> str:
        trie = Trie()
        for w in words:

            trie.insert(w)
        ans = ""
        for w in words:
            if trie.search(w) and (
                len(ans) < len(w) or (len(ans) == len(w) and ans > w)
            ):
                ans = w
        return ans

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Return the longest word where every prefix (excluding the full word) also exists in the list.

Ties: return the lexicographically smallest. Return '' if none. Right?"

#

"A few quick questions:

A single-letter word is always valid since it has no proper prefixes to check? Yes.

Words are all lowercase? Yes."

#

"Let me walk: words=['w','wo','wor','worl','world'] → 'world' (all prefixes present) ✓

words=['a','banana','app','appl','ap','apply','apple'] → 'apple' (lex smaller than 'apply') ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Put all words in a set. Sort by (length desc, alpha asc). For each word, check that every prefix from length 1 to len-1 is in the set.

Return the first match.

$O(n * L^2)$ where L = max word length.

Should I go ahead and optimize?"

```

class _720_LongestWordInDictionary_Brute:
    def longestWord(self, words):
        word_set = set(words)
        result = ''
        for w in sorted(words, key=lambda x: (-len(x), x)):
            if all(w[:i] in word_set for i in range(1, len(w))):
                result = w; break
        return result

# PHASE 3 — OPTIMIZE
# "The bottleneck is the  $O(L^2)$  prefix check per word.
# Trie approach: insert all words, then BFS/DFS visiting only nodes marked
is_end=True.
# The deepest reachable node via only-end-marked paths gives the longest valid word.
#
# Time:  $O(n \cdot L)$  to build trie +  $O(n \cdot L)$  to search. Space:  $O(n \cdot L)$ ."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _720_LongestWordInDictionary:
    def longestWord(self, words):
        word_set = set(words)
        result = ''
        for word in words:
            if all(word[:i] in word_set for i in range(1, len(word))):
                if len(word) > len(result) or (len(word) == len(result) and word <
result):
                    result = word
        return result

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# words=['w', 'wo', 'wor', 'worl', 'world']:
# 'world': 'w'✓, 'wo'✓, 'wor'✓, 'worl'✓ → len=5 > len('')=0 → result='world'. → 'world'
✓
# words=['a', 'banana', 'app', 'appl', 'ap', 'apply', 'apple']:
# 'apple': 'a', 'ap', 'app', 'appl' all in set ✓. len=5. result='apple'.
# 'apply': all prefixes ✓. len=5=5, 'apply'>'apple' → keep 'apple'. ✓
#
# "No bugs. The len/lex comparison handles ties correctly without sorting."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(n \cdot L^2)$ :  $n$  words  $\times$   $L$  prefixes  $\times$   $L$  per set lookup. Space  $O(n \cdot L)$  for the
set.
# Trie reduces prefix lookup to  $O(1)$  per char, bringing it to  $O(n \cdot L)$ .
# Follow-up: if words stream in one at a time, maintain a trie with is_end flags.
# Follow-up: allow at most one missing prefix – extend the BFS to allow one 'gap'."

```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestWord(self, words: list[str]) -> str:
        root = {}

        for word in words:
            node = root
            for c in word:
                if c not in node:
                    node[c] = {}
                node = node[c]

            node['word'] = word

        def dfs(node: dict) -> str:
            ans = node['word'] if 'word' in node else ''

            for child in node:
                if 'word' in node[child] and len(node[child]['word']) > 0:
                    childWord = dfs(node[child])
                    if len(childWord) > len(ans) or (
                        len(childWord) == len(ans) and childWord < ans):
                        ans = childWord

            return ans

        return dfs(root)
```

• LeetDoocs

```
class Trie:
    def __init__(self):
        self.children: List[Optional[Trie]] = [None] * 26
        self.is_end = False

    def insert(self, w: str):
        node = self
        for c in w:
            idx = ord(c) - ord("a")
            if node.children[idx] is None:
```



```

        node.children[idx] = Trie()
        node = node.children[idx]
        node.is_end = True

    def search(self, w: str) -> bool:
        node = self
        for c in w:
            idx = ord(c) - ord("a")
            if node.children[idx] is None:
                return False

```

```

            node = node.children[idx]
            if not node.is_end:
                return False
        return True

```

```

class Solution:
    def longestWord(self, words: List[str]) -> str:
        trie = Trie()
        for w in words:

```

```

            trie.insert(w)
        ans = ""
        for w in words:
            if trie.search(w) and (
                len(ans) < len(w) or (len(ans) == len(w) and ans > w)
            ):
                ans = w
        return ans

```

• SimplyLeet

Python solution for Longest Word in Dictionary

```

def longestWord(words):
    # Sort words by length and lexicographical order
    words.sort(key=lambda x: (len(x), x))

    valid_words = set()
    result = ""

    # Process each word

```

```

for word in words:
    # For single-letter words, they are valid if they exist in the list
    if len(word) == 1 or word[:-1] in valid_words:
        valid_words.add(word)
    # Update result if the current word is longer or if equal length
    # but lexicographically smaller
    if len(word) > len(result) or (len(word) == len(result) and word <
result):
        result = word

return result

```

```

# Example usage:
# print(longestWord(["a", "banana", "app", "appl", "ap", "apply", "apple"]))

```

• LC.ca

```

class Solution:
    def longestWord(self, words: List[str]) -> str:
        cnt, ans = 0, ''
        s = set(words)
        for w in s:
            n = len(w)
            if all(w[:i] in s for i in range(1, n)):
                if cnt < n:
                    cnt, ans = n, w
                elif cnt == n and w < ans:

                    ans = w
        return ans

```

◆ Quick Review

Key Insights

- The word must be built character by character; hence every prefix (removing one character from the end at a time) must exist in

the dictionary.

- Sorting the input list can help in ensuring that smaller words (prefixes) are processed first.
- Using a hash set to store valid words enables rapid prefix checks.
- Lexicographical order plays a role when words have the same length.

Space & Time

Time Complexity: $O(n * L + n \log n)$ where n is the number of words and L is the maximum length of a word ($n * L$ for prefix checking, and $n \log n$ for sorting).

Space Complexity: $O(n * L)$ for storing words in the set.

Solution

The approach begins by sorting the list of words. Sorting is done primarily by word length and secondarily by lexicographical order so that when we iterate through the words, all possible prefixes for a word have already been considered. Initialize a set to keep track of valid words that can be built one character at a time. For each word in the

sorted list, check if all its prefixes (word without the last character progressively) exist in the set. If a word qualifies, add it to the set and check if it should be the new result based on length and lexicographical order. The set lookup ensures that checking each prefix is efficient.

Trie

□ #1166 Design File System

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Design · Trie
Lists: Denny Zhang

Problem

You are asked to design a file system that allows you to create new paths and associate them with different values.

The format of a path is one or more concatenated strings of the form: / followed by one or more lowercase English letters. For example, "/leetcode" and "/leetcode/problems" are valid paths while an empty string "" and "/" are not.

Implement the FileSystem class:

- **bool createPath(string path, int value)** Creates a new path and associates a value to it if possible and returns true. Returns false if the path already exists or its parent path doesn't exist.

- **int get(string path)** Returns the value associated with path or returns -1 if the path doesn't exist.

Example 1:

```
Input:
["FileSystem","createPath","get"]
[[],["/a",1],["/a"]]
Output:
[null,true,1]
Explanation:
FileSystem fileSystem = new FileSystem();
```

Example 2:

```
Input:
["FileSystem","createPath","createPath","get","createPath","get"]
[[],["/leet",1],["/leet/code",2],["/leet/code"],["/c/d",1],["/c"]]
Output:
[null,true,true,2,false,-1]
Explanation:
FileSystem fileSystem = new FileSystem();
```

Constraints:

- $2 \leq \text{path.length} \leq 100$
- $1 \leq \text{value} \leq 109$
- Each path is valid and consists of lowercase English letters and '/'.
- At most 104 calls in total will be made to createPath and get.

My LeetCode Solution

- My LeetCode Solution

```
class Trie:
    def __init__(self, v: int = -1):
        self.children = {}
        self.v = v

    def insert(self, w: str, v: int) -> bool:
        node = self
        ps = w.split("/")
        for p in ps[1:-1]:
            if p not in node.children:
                return False
            node = node.children[p]
        if ps[-1] in node.children:
            return False
        node.children[ps[-1]] = Trie(v)
        return True

    def search(self, w: str) -> int:
        node = self
        for p in w.split("/")[:-1]:
            if p not in node.children:
                return -1
            node = node.children[p]
        return node.v

class FileSystem:
    def __init__(self):
        self.trie = Trie()

    def createPath(self, path: str, value: int) -> bool:
        return self.trie.insert(path, value)

    def get(self, path: str) -> int:
        return self.trie.search(path)

# Your FileSystem object will be instantiated and called as such:
# obj = FileSystem()
# param_1 = obj.createPath(path,value)
```



```
# param_2 = obj.get(path)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Design a file system with createPath(path, value)→bool and get(path)→int.
# createPath creates a path with a value; parent must exist.
# Returns false if path exists or parent doesn't exist.
# get returns value or -1. Right?"
#
# "A few quick questions:
# Root '/' is always implicitly there? Yes. Paths start with '/'? Yes."
#
# "Let me walk: createPath('/a',1)→T. createPath('/a/b',2)→T. get('/a')→1.
# createPath('/c/d',1)→F (no '/c'). get('/c')→-1 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Store full path strings in a dict mapping path→value.
# createPath: check parent path exists, then insert. get: lookup.
# O(L) per op where L = path length.
# Should I go ahead and optimize?"
class _1166_DesignFileSystem_Brute:
    def __init__(self): self.paths = {'': 0}
    def createPath(self, path, value):
        parent = path[:path.rfind('/')]
        if parent not in self.paths or path in self.paths: return False
        self.paths[path] = value; return True
    def get(self, path): return self.paths.get(path, -1)
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the rfind scan for the parent path.
# Trie where each node maps component names to child nodes and stores a value.
# createPath: split on '/', walk/create nodes, check parent validity.
# get: walk trie, return node value.
#
# Time: O(L) per op (L = path length). Space: O(total unique path components)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _1166_DesignFileSystem:
    def __init__(self):
        self._store = {}
    def createPath(self, path, value):
        parts = path.split('/')
        parent = '/'.join(parts[:-1])
```

```

    if parent and parent not in self._store:
        return False
    if path in self._store:
        return False
    self._store[path] = value
    return True

def get(self, path):
    return self._store.get(path, -1)

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

createPath('/a',1): parent=''. '' not in store? '' is falsy → skip parent check.

path '/a' not in store → store['/a']=1 → True ✓

createPath('/a/b',2): parent='/a'. '/a' in store ✓. '/a/b' not in store → store['/a/b']=2 ✓

get('/a')→1 ✓. createPath('/c/d',1): parent='/c'. '/c' not in store → False ✓

#

"No bugs. The 'if parent and ...' handles the root case cleanly."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Both ops $O(L)$: dict lookup + parent extraction.

Space $O(\text{total unique paths})$. No prefix sharing needed since we store full paths.

True trie would share prefixes but adds complexity – dict is simpler and equally fast here.

Follow-up: delete(path) – remove from dict; handle subtrees by scanning for prefix matches.

Follow-up: list directory – return keys that start with path+'/' and have no further '/'."

Community Solutions (Python)

• WalkCC

```

class TrieNode:
    def __init__(self, value: int = 0):
        self.children: dict[str, TrieNode] = {}
        self.value = value

class FileSystem:
    def __init__(self):
        self.root = TrieNode()

```

```

def createPath(self, path: str, value: int) -> bool:
    node: TrieNode = self.root
    subpaths = path.split('/')

    for i in range(1, len(subpaths) - 1):
        if subpaths[i] not in node.children:
            return False
        node = node.children[subpaths[i]]

    if subpaths[-1] in node.children:

        return False
    node.children[subpaths[-1]] = TrieNode(value)
    return True

def get(self, path: str) -> int:
    node: TrieNode = self.root
    for subpath in path.split('/')[1:]:
        if subpath not in node.children:
            return -1
        node = node.children[subpath]

    return node.value

```

• LeetDoocs

```

class Trie:
    def __init__(self, v: int = -1):
        self.children = {}
        self.v = v

    def insert(self, w: str, v: int) -> bool:
        node = self
        ps = w.split("/")
        for p in ps[1:-1]:
            if p not in node.children:

```

```

        return False
    node = node.children[p]
    if ps[-1] in node.children:
        return False
    node.children[ps[-1]] = Trie(v)
    return True

```

```

def search(self, w: str) -> int:
    node = self
    for p in w.split("/")[1:]:

```

```

        if p not in node.children:
            return -1
        node = node.children[p]
    return node.v

```

```

class FileSystem:
    def __init__(self):
        self.trie = Trie()

```

```

    def createPath(self, path: str, value: int) -> bool:
        return self.trie.insert(path, value)

```

```

    def get(self, path: str) -> int:
        return self.trie.search(path)

```

```

# Your FileSystem object will be instantiated and called as such:
# obj = FileSystem()
# param_1 = obj.createPath(path,value)

```

```

# param_2 = obj.get(path)

```

• SimplyLeet

```
class FileSystem:
    def __init__(self):
        # Initialize hash map to store paths and their values.
        self.paths = {}

    def createPath(self, path: str, value: int) -> bool:
        # Check if the path already exists.
        if path in self.paths:
            return False

        # Find the index of the last '/' to get the parent path.
        last_slash = path.rfind('/')
        # If last_slash is 0, then parent is root level, which is invalid
        because "" is not a valid path.
        if last_slash == 0:
            parent = ""
        else:
            parent = path[:last_slash]

        # For non-root paths, the parent must exist.
        if parent and parent not in self.paths:

            return False

        # Store the new path with its value.
        self.paths[path] = value
        return True

    def get(self, path: str) -> int:
        # Return the value associated with the path, or -1 if it doesn't exist.
        return self.paths.get(path, -1)

# Example usage:
# fs = FileSystem()
# print(fs.createPath("/a", 1)) # True
# print(fs.get("/a")) # 1
```

 • LC.ca

```
'''
>>> "/a/b/c".split('/')
['', 'a', 'b', 'c']
'''

class Trie:
    def __init__(self, v: int = -1):
        self.children = {}
        self.v = v

    def insert(self, w: str, v: int) -> bool:
        node = self
        ps = w.split("/")
        for p in ps[1:-1]:
            if p not in node.children:
                return False
            node = node.children[p]
        if ps[-1] in node.children:
            return False
        node.children[ps[-1]] = Trie(v)

        return True

    def search(self, w: str) -> int:
        node = self
        for p in w.split("/")[1:]:
            if p not in node.children:
                return -1
            node = node.children[p]
        return node.v

class FileSystem:
    def __init__(self):
        self.trie = Trie()

    def createPath(self, path: str, value: int) -> bool:
        return self.trie.insert(path, value)

    def get(self, path: str) -> int:
        return self.trie.search(path)
```

```
# Your FileSystem object will be instantiated and called as such:  
# obj = FileSystem()  
# param_1 = obj.createPath(path,value)  
# param_2 = obj.get(path)
```

◆ Quick Review

Key Insights

- Use a data structure (like a hash map) to store paths and associated values.
- To create a new path, check that the path does not already exist.
- Ensure the immediate parent path exists before creation.
- When retrieving a value, simply look it up in the storage structure.
- Splitting the path by "/" allows easy identification of the parent.

Space & Time

Time Complexity:

createPath: $O(n)$ per call where n is the number of segments in the path (to process and validate parent path).

get: $O(1)$ per call if using a hash map.

Space Complexity: $O(m * L)$ where m is the number of paths stored and L is the average length of a path.

Solution

The solution uses a hash map (dictionary) to store file paths along with their associated values. For the `createPath` method, the algorithm: Checks if the path already exists. Extracts the parent path by removing the last segment. Validates the existence of the parent path. Inserts the new path with its corresponding value if validation passes. For the `get` method, the algorithm simply returns the stored value if the path exists; otherwise, it returns `-1`. This approach ensures efficient lookup and validation using string manipulation and hash map operations.

Backtracking

Round 5 · Mid · Medium · 6 questions

Backtracking

□ #17 Letter Combinations of a Phone Number

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Backtracking
Lists: Grind 169

Problem

Given a string containing digits from 2–9 inclusive, return all possible letter combinations that the number could represent. Return the answer in any order.

A mapping of digits to letters (just like on the telephone buttons) is given below. Note that 1 does not map to any letters.



Example 1:

Input: digits = "23"

Output: ["ad","ae","af","bd","be","bf","cd","ce","cf"]

Example 2:

Input: digits = "2"

Output: ["a","b","c"]

Constraints:

- $1 \leq \text{digits.length} \leq 4$
- `digits[i]` is a digit in the range ['2', '9'].

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Given a string of digits 2-9, return all possible letter combinations
# the digits could represent on a phone keypad. Return [] for empty input. Right?"
#
# "A few quick questions:
# Output order can be any? Yes. Can digits include 0 or 1? No – they have no
# letters.
# Return combinations of length len(digits)?"
#
# "Let me walk: digits='23'. 2→abc, 3→def.
# Combine each: ad,ae,af,bd,be,bf,cd,ce,cf → 9 combinations ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# BFS/queue: start with [''], then for each digit expand every partial result
# by appending each letter that digit maps to.
#  $O(4^n * n)$  time and space –  $n = \text{len}(\text{digits})$ , max 4 letters per digit.
# Should I go ahead and optimize?"
class _17_LetterCombinations_Brute:
    def letterCombinations(self, digits):
        if not digits: return []
        phone =
{'2':'abc','3':'def','4':'ghi','5':'jkl','6':'mno','7':'pqrs','8':'tuv','9':'wxyz'}
        result = ['']
        for d in digits:
            result = [r + c for r in result for c in phone[d]]
        return result
```

PHASE 3 — OPTIMIZE

```
# "Both BFS and backtracking produce the same output; backtracking uses less
# intermediate space.
# The bottleneck is unavoidable – we must enumerate all  $4^n$  combinations.
# Backtracking builds each combination character by character and avoids
# storing all intermediate partial strings.
#
# Time:  $O(4^n * n)$ . Space:  $O(n)$  call depth +  $O(4^n * n)$  output."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _17_LetterCombinations:
    def letterCombinations(self, digits):
        if not digits:
            return []
```

```

phone = {'2':'abc','3':'def','4':'ghi','5':'jkl',
        '6':'mno','7':'pqrs','8':'tuv','9':'wxyz'}
result = []
def backtrack(i, path):
    if i == len(digits):
        result.append(''.join(path)); return
    for ch in phone[digits[i]]:
        path.append(ch)
        backtrack(i + 1, path)
        path.pop()
backtrack(0, [])
return result

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

digits='23':

backtrack(0,[]): ch='a' → backtrack(1,['a']): ch='d'→append 'ad'. 'e'→'ae'.
'f'→'af'.

ch='b' → 'bd','be','bf'. ch='c' → 'cd','ce','cf'. result=9 strings ✓

#

digits='': return [] ✓

digits='2': → ['a','b','c'] ✓

#

"No bugs. path.pop() after each recursive call restores state for the next branch."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(4^n * n)$: 4^n combinations, each takes $O(n)$ to record. Space $O(n)$ call depth.

Iterative BFS uses the same time but $O(4^n * n)$ intermediate space.

Follow-up: if some digits have 0 letters, skip them.

Follow-up: count combinations without enumerating – just multiply letter counts per digit."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def letterCombinations(self, digits: str) -> list[str]:
        if not digits:
            return []

        digitToLetters = ['', '', 'abc', 'def', 'ghi',
                          'jkl', 'mno', 'pqrs', 'tuv', 'wxyz']

        ans = []

        def dfs(i: int, path: list[str]) -> None:

            if i == len(digits):
                ans.append(''.join(path))
                return

            for letter in digitToLetters[int(digits[i])]:
                path.append(letter)
                dfs(i + 1, path)
                path.pop()

        dfs(0, [])

        return ans

```

• LeetDoocs

```

class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        if not digits:
            return []
        d = ["abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"]
        ans = [""]
        for i in digits:
            s = d[int(i) - 2]
            ans = [a + b for a in ans for b in s]
        return ans

```

```

class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        def dfs(i: int):
            if i >= len(digits):
                ans.append("".join(t))
                return
            for c in d[int(digits[i]) - 2]:
                t.append(c)
                dfs(i + 1)
                t.pop()

        if not digits:
            return []
        d = ["abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"]
        ans = []
        t = []
        dfs(0)
        return ans

```

• SimplyLeet

Python solution for Letter Combinations of a Phone Number

```

def letterCombinations(digits):
    # Mapping of digits to corresponding letters
    digit_to_letters = {
        "2": "abc",
        "3": "def",
        "4": "ghi",
        "5": "jkl",
        "6": "mno",

        "7": "pqrs",
        "8": "tuv",
        "9": "wxyz"
    }

    # If input is empty, return an empty list
    if not digits:
        return []

    result = []

```



```

# Backtracking function to generate combinations
def backtrack(index, current_combination):
    # If the current combination length equals the digits length,
    # add the combination to the result list.
    if index == len(digits):
        result.append(current_combination)
        return

    # Get the corresponding letters for the current digit

    letters = digit_to_letters[digits[index]]
    # Explore all possible letter routes for the current digit.
    for letter in letters:
        backtrack(index + 1, current_combination + letter)

# Initiate backtracking starting from index 0 and empty combination.
backtrack(0, "")
return result

# Example usage:

#print(letterCombinations("23"))

```

• LC.ca

```

class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        if not digits:
            return []
        d = ["abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"]
        ans = [""]
        for i in digits:
            s = d[int(i) - 2]
            ans = [a + b for a in ans for b in s]
        return ans

```

◆ Quick Review

Key Insights

- Use a mapping from digits to the respective letters as defined by a telephone keypad.
- Backtracking is an ideal approach to generate all possible combinations.
- Each digit adds a level in the recursive tree where you append one letter at a time.
- Handling edge cases such as an empty input string is crucial.

Space & Time

Time Complexity: $O(4^n * n)$ where n is the length of the input string (each digit may map to up to 4 letters and adding a combination takes $O(n)$ time).

Space Complexity: $O(n)$ for the recursion call stack, not counting the space required for storing the output.

Solution

We can solve the problem using a backtracking algorithm. First, we define a dictionary mapping each digit to its corresponding letters. Then we initiate a recursive function called `backtrack` that takes the current combination and the index of the next digit to process. At each call, if the combination is

complete (length equal to the input string), we add it to the result list. Otherwise, we iterate through the letters corresponding to the current digit, append a letter to the combination, and recursively call backtrack with the next index. This approach ensures that all possible letter combinations are generated.

Backtracking

□ #22 Generate Parentheses

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming · Backtracking
Lists: Grind 169

Problem

Given n pairs of parentheses, write a function to generate all combinations of well-formed parentheses.

Example 1:

Input: $n = 3$
Output: ["((()))", "(()())", "(())()", "()(())", "()()()"]

Example 2:

Input: $n = 1$
Output: ["()"]

Constraints:

- $1 \leq n \leq 8$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Given n pairs of parentheses, generate all combinations of well-formed
# parentheses.
# Well-formed means every '(' is closed before any preceding '(' closes. Right?"
#
# "A few quick questions:
# n=0 → []? n=1 → ['()']? Yes to both. Any ordering of output is fine."
#
# "Let me walk: n=2 → ['()', '()()'] ✓. n=3 → 5 combinations ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Generate all 2^(2n) sequences of '(' and ')', filter valid ones.
# O(2^(2n) * n) – exponentially wasteful since most sequences are invalid.
# Should I go ahead and optimize?"
class _22_GenerateParentheses_Brute:
    def generateParenthesis(self, n):
        def valid(s):
            bal = 0
            for c in s:
                if c == '(': bal += 1
                else: bal -= 1
                if bal < 0: return False
            return bal == 0
        from itertools import product
        return [''.join(p) for p in product('()', repeat=2*n) if valid(p)]
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is generating invalid strings then filtering them.
# Backtracking with validity pruning – only place characters that keep the string
# valid:
# Add '(' if open count < n. Add ')' if close count < open count.
# This generates ONLY valid strings – no wasted work.
#
# Time: O(4^n / sqrt(n)) – the nth Catalan number. Space: O(n) call depth."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _22_GenerateParentheses:
    def generateParenthesis(self, n):
        result = []
        def backtrack(path, open_count, close_count):
            if len(path) == 2 * n:
                result.append(''.join(path)); return
            if open_count < n:
                path.append('('); backtrack(path, open_count + 1, close_count);
        path.pop()
            if close_count < open_count:
                path.append(')'); backtrack(path, open_count, close_count + 1);
        path.pop()
```

```
backtrack([], 0, 0)
return result
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
```

```
#
```

```
# n=2:
```

```
# backtrack([],0,0): add '(' → backtrack(['('],1,0):
```

```
# add '(' → ['(', '']: add ')' → ['()'] add ')' → '()' ✓, add ')' → ['(']: add ')'
→ '()' ✓
```

```
# backtrack(['()'],1,1): add '(' → ['() '('] add ')' → '()()' ✓
```

```
# result=['()()', '()()'] ✓
```

```
#
```

```
# "No bugs. close_count < open_count prevents ')' when there's no matching '(' to
close."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(4^n / \sqrt{n})$ : Catalan number of valid strings, each takes  $O(n)$  to
record.
```

```
# Brute  $O(2^{(2n)})$  – exponentially worse.
```

```
# Follow-up: count valid sequences without generating –  $Catalan(n) = C(2n, n) / (n+1)$ .
```

```
# Follow-up: Longest Valid Parentheses (#32) – find the longest valid substring
using DP or stack."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def generateParenthesis(self, n):
        ans = []

        def dfs(l: int, r: int, s: list[str]) -> None:
            if l == 0 and r == 0:
                ans.append(''.join(s))
            if l > 0:
                s.append('(')
                dfs(l - 1, r, s)
```

```

        s.pop()
    if l < r:
        s.append(')')
        dfs(l, r - 1, s)
        s.pop()

dfs(n, n, [])
return ans

```

• LeetDoocs

```

class Solution:
    def generateParenthesis(self, n: int) -> List[str]:
        def dfs(l: int, r: int, t: str):
            if l > n or r > n or l < r:
                return
            if l == n and r == n:
                ans.append(t)
                return
            dfs(l + 1, r, t + "(")
            dfs(l, r + 1, t + ")")

        ans = []
        dfs(0, 0, "")
        return ans

```

• SimplyLeet

```

def generateParenthesis(n):
    # List to store valid combinations
    result = []

    # Helper function for backtracking
    def backtrack(current, open_count, close_count):
        # If current string reaches the maximum length, it's a valid
        combination
        if len(current) == 2 * n:
            result.append(current)
            return

```



```

# If we can add an opening parenthesis, do it
if open_count < n:
    backtrack(current + '(', open_count + 1, close_count)
# If adding a closing parenthesis maintains the validity, do it
if close_count < open_count:
    backtrack(current + ')', open_count, close_count + 1)

# Start the recursion with an empty string and zero counts for both
backtrack("", 0, 0)
return result

```

```

# Example usage:
print(generateParenthesis(3))

```

• LC.ca

```

class Solution:
    def generateParenthesis(self, n: int) -> List[str]:
        def dfs(l, r, t):
            if l > n or r > n or l < r:
                return
            if l == n and r == n:
                ans.append(t)
                return
            dfs(l + 1, r, t + '(')
            dfs(l, r + 1, t + ')')

```

```

ans = []
dfs(0, 0, '')
return ans

```

```
#####
```

```

class Solution(object):
    def generateParenthesis(self, n):
        """

```

```
:type n: int
:rtype: List[str]
"""

def dfs(left, path, res, n):
    if len(path) == 2 * n:
        if left == 0:
            res.append("".join(path))
            return

    if left < n:
        path.append("(")
        dfs(left + 1, path, res, n)
        path.pop()
    if left > 0:
        path.append(")")
        dfs(left - 1, path, res, n)
        path.pop()

res = []

dfs(0, [], res, n)
return res
```

◆ Quick Review

Key Insights

- Use a backtracking approach to build the valid combinations step by step.
- At any recursion step, you can add an opening parenthesis if you still have one available.
- You can add a closing parenthesis only if it would not result in an invalid sequence (i.e.,

the number of closing parentheses is less than the number of opening ones).

- The solution leverages recursion to explore all potential valid sequences while pruning invalid paths early.

Space & Time

Time Complexity: $O(4^n / (n^{3/2}))$

Space Complexity: $O(n)$ for the recursion stack, plus space for storing all valid combinations.

Solution

The solution uses a recursive backtracking technique to generate the combinations. The algorithm maintains counters for the number of '(' and ')' added to the current string. It recursively adds a '(' if the count is less than n and adds a ')' if doing so does not make the string invalid (i.e., if the count of ')' is less than the count of '('). When a valid combination reaches a length of $2 \cdot n$, it is added to the result list.

Backtracking

□ #39 Combination Sum

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Backtracking
Lists: Grind 169

Problem

Given an array of distinct integers candidates and a target integer target, return a list of all unique combinations of candidates where the chosen numbers sum to target. You may return the combinations in any order.

The same number may be chosen from candidates an unlimited number of times. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to target is less than 150 combinations for the given input.

Example 1:

Input: candidates = [2,3,6,7], target = 7

Output: [[2,2,3],[7]]

Explanation:

2 and 3 are candidates, and $2 + 2 + 3 = 7$. Note that 2 can be used multiple times.

7 is a candidate, and $7 = 7$.

These are the only two combinations.

Example 2:

Input: candidates = [2,3,5], target = 8

Output: [[2,2,2,2],[2,3,3],[3,5]]

Example 3:

Input: candidates = [2], target = 1

Output: []

Constraints:

- $1 \leq \text{candidates.length} \leq 30$
- $2 \leq \text{candidates}[i] \leq 40$
- All elements of candidates are distinct.
- $1 \leq \text{target} \leq 40$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[
int]]:
        def backtracking(start: int, S: int):
            if S == target:
                ans.append(curr.copy())
                return

            if S > target:
                return

            for i in range(start, len(candidates)):
                curr.append(candidates[i])
                backtracking(i+1, S+candidates[i])
                curr.pop()
```

```

        for i in range(start, len(candidates)):
            curr.append(candidates[i])
            backtracking(i, S+candidates[i])
            curr.pop()

```

```

curr = []
ans = []

```

```

candidates.sort()
backtracking(0,0)
return ans

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given distinct positive integers and a target, return all unique combinations summing to target. Each number may be used unlimited times.

No duplicate combinations in the output. Right?"

#

"A few quick questions:

If the same combination can be formed in different orders, count it once only? Yes.

All values are positive? Yes."

#

"Let me walk: candidates=[2,3,6,7], target=7 → [[2,2,3],[7]] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Generate all combinations with repetition up to target sum, filter by sum.

$O(n^{(target/min_c)})$ – exponential and generates duplicates.

Should I go ahead and optimize?"

```
class _39_CombinationSum_Brute:
```

```
    def combinationSum(self, candidates, target):
```

```
        result = set()
```

```
        def dfs(path, rem):
```

```
            if rem == 0: result.add(tuple(sorted(path))); return
```

```
            for c in candidates:
```

```
                if c <= rem: path.append(c); dfs(path, rem-c); path.pop()
```

```
        dfs([], target)
```

```
        return [list(c) for c in result]
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the set-based deduplication and the generation of
order-duplicates.
# Backtracking with a start index: pass the current index so we only pick from
# the current candidate onward – this prevents [3,2] when [2,3] already exists.
# Sort first to enable early pruning when candidate > remaining.
#
# Time:  $O(n^{(target/min\_c)})$  – same asymptotic but pruned and no dedup overhead."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _39_CombinationSum:
    def combinationSum(self, candidates, target):
        candidates.sort()
        result = []
        def backtrack(start, path, remaining):
            if remaining == 0:
                result.append(list(path)); return
            for i in range(start, len(candidates)):
                if candidates[i] > remaining:
                    break
                path.append(candidates[i])
                backtrack(i, path, remaining - candidates[i])
                path.pop()
        backtrack(0, [], target)
        return result
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# candidates=[2,3,6,7], target=7 (sorted):
# start=0: pick 2 → [2], rem=5. pick 2 → [2,2], rem=3. pick 2 → [2,2,2], rem=1. 2>1
break.
# pick 3 → [2,2,3], rem=0 → append ✓. pick 3 → [2,3], rem=2. pick 3>2 break. pick
6>5 break.
# start=1: pick 3, rem=4. pick 3 → [3,3], rem=1. 3>1 break. pick 6>4 break.
# start=2: pick 6, rem=1. 6>1 break. start=3: pick 7, rem=0 → append ✓. →
[[2,2,3],[7]] ✓
#
# "No bugs. 'break' after candidates[i] > remaining prunes because the array is
sorted."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n^{(target/min\_c)})$ : at most target/min_c levels deep, n choices each.
# Space  $O(target/min\_c)$  recursion depth.
# Follow-up: Combination Sum II (#40) – each number used once, has duplicates.
# Use i+1 instead of i, skip dup values at same level: if i>start and
cands[i]==cands[i-1] skip.
# Follow-up: Combination Sum IV (#377) – count ordered sequences summing to target
(DP)."
```


Community Solutions (Python)

• WalkCC

```
class Solution:
    def combinationSum(self, candidates: list[int],
                      target: int) -> list[list[int]]:
        ans = []

        def dfs(s: int, target: int, path: list[int]) -> None:
            if target < 0:
                return
            if target == 0:
                ans.append(path.clone())

            return

        for i in range(s, len(candidates)):
            path.append(candidates[i])
            dfs(i, target - candidates[i], path)
            path.pop()

        candidates.sort()
        dfs(0, target, [])
        return ans
```

• LeetDoocs

```
class Solution:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[
int]]:
        def dfs(i: int, s: int):
            if s == 0:
                ans.append(t[:])
                return
            if s < candidates[i]:
                return
            for j in range(i, len(candidates)):
                t.append(candidates[j])
```

```

        dfs(j, s - candidates[j])
        t.pop()

    candidates.sort()
    t = []
    ans = []
    dfs(0, target)
    return ans

```

• SimplyLeet

```

# Function to find all unique combinations that sum to the target
def combinationSum(candidates, target):
    # List to store all valid combinations
    result = []
    # Sort candidates to facilitate pruning of the search
    candidates.sort()

    # Recursive helper function for backtracking
    def backtrack(remaining, start, path):
        # If remaining is 0, path is a valid combination so add a copy of it to
        result

        if remaining == 0:
            result.append(list(path))
            return

        # Loop through candidates starting from the current index
        for i in range(start, len(candidates)):
            # If candidate is greater than remaining, break out for efficiency
            if candidates[i] > remaining:
                break

            # Include candidate in the current combination path
            path.append(candidates[i])

```

```

        # Recursively call backtrack with updated remaining and same index
        (allowing reuse)
        backtrack(remaining - candidates[i], i, path)
        # Backtrack by removing the candidate added in this iteration
        path.pop()

    # Start backtracking from index 0 with an empty path
    backtrack(target, 0, [])
    return result

```

```

# Example usage

```

```

print(combinationSum([2,3,6,7], 7))

```

• LC.ca

```

class Solution:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[
int]]:
        # i: start index for this recursion
        # s: sum
        def dfs(i, s):
            if s == target:
                ans.append(t.copy())
                return
            if s > target:
                return

            for j in range(i, len(candidates)):
                c = candidates[j]
                t.append(c)
                dfs(j, s + c)
                t.pop()

        ans = []
        t = []
        # candidates.sort() # diff from combinationSum-II, no need sorting it
        dfs(0, 0)

```

```

        return ans

#####

# dp version
class Solution_dp:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[
int]]:
        # for each-target (from 1 to target), its dp[i][j]
        # => so 3-D array dp[][][]
        dp = []

        candidates.sort()

        for i in range(1, target+1):
            cur = []
            for j in range(len(candidates)):
                if candidates[j] > i:
                    break
                if candidates[j] == i:
                    one = [candidates[j]]
                    cur.append(one)

                    break
                for a in dp[i - candidates[j] - 1]:
                    if candidates[j] > a[0]:
                        continue
                    deepCopied = a.copy()
                    deepCopied.insert(0, candidates[j])
                    cur.append(deepCopied)
                dp.append(cur)

        return dp[-1]

```

#####

```
class Solution(object):
    def combinationSum(self, candidates, target):
        """
        :type candidates: List[int]
        :type target: int
        :rtype: List[List[int]]
```

"""

```
def dfs(candidates, start, target, path, res):
    if target == 0:
        return res.append(path + [])

    for i in range(start, len(candidates)):
        if target - candidates[i] >= 0:
            path.append(candidates[i])
            dfs(candidates, i, target - candidates[i], path, res)
```

```
    path.pop()
```

```
res = []
dfs(candidates, 0, target, [], res)
return res
```

#####

```
class Solution:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[
int]]:
```

```
def dfs(i: int, s: int):
    if s == 0:
        ans.append(t[:])
        return
    if i >= len(candidates) or s < candidates[i]:
        return
    dfs(i + 1, s)
    t.append(candidates[i])
    dfs(i, s - candidates[i])
    t.pop()
```

```
candidates.sort()
t = []
ans = []
dfs(0, target)
return ans
```

◆ Quick Review

Key Insights

- Backtracking is a natural fit to explore all potential combinations.
- Sorting candidates simplifies the process by allowing early termination if the current candidate exceeds the remaining target.
- Recursion is used to build combinations and backtrack once the current path exceeds or meets the target.
- The same candidate can be reused in the combination, so the recursive call does not advance the index.

Space & Time

Time Complexity:

$O(2^{(\text{target}/\min(\text{candidates}))})$ in the worst case where many combinations are possible.

Space Complexity: $O(\text{target}/\min(\text{candidates}))$ for the recursion stack and temporary space used to store combinations.

Solution

The solution uses a backtracking approach. First, sort the candidates to enable pruning of recursive calls once the candidate exceeds the remaining target. A recursive helper function builds up a combination, subtracting the candidate value from the target until it reaches zero (a valid combination) or becomes negative (an invalid path). The recursion allows the same candidate to be chosen again. Each valid combination is then added to the result list.

Backtracking

□ #46 Permutations

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Backtracking
Lists: Grind 169

Problem

Given an array `nums` of distinct integers, return all the possible permutations. You can return the answer in any order.

Example 1:

```
Input: nums = [1,2,3]
Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]
```

Example 2:

```
Input: nums = [0,1]
Output: [[0,1],[1,0]]
```

Example 3:

```
Input: nums = [1]
Output: [[1]]
```

Constraints:

- $1 \leq \text{nums.length} \leq 6$
- $-10 \leq \text{nums}[i] \leq 10$

- All the integers of nums are unique.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        def backtrack(s):
            if len(nums) == len(curr):
                ans.append(curr[:])

            for i in range(len(nums)):
                if used[i]:
                    continue

                used[i] = True
                curr.append(nums[i])
                backtrack(s+1)
                curr.pop()
                used[i] = False

        ans = []
        curr = []
        used = [False] * (len(nums))

        backtrack(0)
        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given an array of distinct integers, return all possible permutations in any order. Right?"

#

"A few quick questions:

n is at most 6 so $6! = 720$ permutations is fine. All elements are distinct? Yes."

#

"Let me walk: nums=[1,2,3] → 6 permutations in any order ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
 # Use a visited array: at each position try every unvisited number.
 # When path length equals n, record the permutation.
 # $O(n! * n)$ time – unavoidable since we must enumerate all permutations.
 # Should I go ahead and optimize?"

```
class _46_Permutations_Brute:
    def permute(self, nums):
        result = []; n = len(nums); visited = [False] * n
        def bt(path):
            if len(path) == n: result.append(list(path)); return
            for i in range(n):
                if not visited[i]:
                    visited[i] = True; path.append(nums[i])
                    bt(path)
                    path.pop(); visited[i] = False
        bt([]); return result
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ visited array – we can eliminate it.
 # Swap-based backtracking: maintain a 'start' pointer.
 # Everything before start is already placed; swap any element from [start, n) into position start,
 # recurse, then swap back.
 #
 # Same $O(n! * n)$ time but $O(n)$ space and no visited array."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _46_Permutations:
    def permute(self, nums):
        result = []
        def backtrack(start):
            if start == len(nums):
                result.append(list(nums)); return
            for i in range(start, len(nums)):
                nums[start], nums[i] = nums[i], nums[start]
                backtrack(start + 1)
                nums[start], nums[i] = nums[i], nums[start]
        backtrack(0)
        return result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."
 #
 # nums=[1,2,3]:
 # bt(0): swap(0,0)→[1,2,3] bt(1): swap(1,1)→[1,2,3] bt(2): swap(2,2) bt(3) append [1,2,3].
 # swap(1,2)→[1,3,2] bt(2): swap(2,2) bt(3) append [1,3,2]. swap back→[1,2,3].
 # swap(0,1)→[2,1,3] bt(1): → [2,1,3],[2,3,1]. swap back→[1,2,3].

```
# swap(0,2)→[3,2,1] bt(1): → [3,2,1],[3,1,2]. → total 6 ✓
```

```
#
```

```
# "No bugs. Swapping back restores the array for the next iteration at each level."
```

```
# PHASE 6 — COMPLEXITY & FOLLOW-UP
```

```
# "Time  $O(n! * n)$ :  $n!$  permutations, each  $O(n)$  to copy. Space  $O(n)$ : call depth.
```

```
# Visited-array approach:  $O(n)$  extra space for the boolean array.
```

```
# Follow-up: Permutations II (#47) – duplicates in input. Sort first, skip duplicate
```

```
# values at the same level: if  $i > \text{start}$  and  $\text{nums}[i] == \text{nums}[\text{start}]$  after sorting, skip.
```

```
# Follow-up: Next Permutation (#31) – generate just the lexicographically next one."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def permute(self, nums: list[int]) -> list[list[int]]:
        ans = []
        used = [False] * len(nums)

        def dfs(path: list[int]) -> None:
            if len(path) == len(nums):
                ans.append(path.copy())
                return

            for i, num in enumerate(nums):
                if used[i]:
                    continue
                used[i] = True
                path.append(num)
                dfs(path)
                path.pop()
                used[i] = False

        dfs([])

        return ans
```

• LeetDoocs

```

class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        def dfs(i: int):
            if i >= n:
                ans.append(t[:])
                return
            for j, x in enumerate(nums):
                if not vis[j]:
                    vis[j] = True
                    t[i] = x

                    dfs(i + 1)
                    vis[j] = False

        n = len(nums)
        vis = [False] * n
        t = [0] * n
        ans = []
        dfs(0)
        return ans

```

• SimplyLeet

```

def permute(nums):
    result = []

    def backtrack(current, remaining):
        # If no remaining elements, current permutation is complete.
        if not remaining:
            result.append(current.copy())
            return
        # Explore each candidate number.
        for i in range(len(remaining)):

```

```

        # Add the chosen number to the current permutation.
        current.append(remaining[i])
        # Recursively generate permutations with the remaining numbers.
        backtrack(current, remaining[:i] + remaining[i+1:])
        # Backtrack by removing the chosen number.
        current.pop()

    backtrack([], nums)
    return result

```

```

# Example usage:
print(permute([1,2,3]))

```

• LC.ca

```

'''
remove an element by its value from a set
'''

```

```

>>> my_set = {1, 2, 3, 4, 5}
>>> my_set.remove(3)
>>> print(my_set)
{1, 2, 4, 5}

```

```

-----

```

cannot remove an element by index from a set in Python3
need to convert the set to a list

```

>>> my_set = {1, 2, 3, 4, 5}
>>> my_list = list(my_set)
>>> del my_list[2] # remove the element at index 2
>>> my_set = set(my_list)
>>> print(my_set)
{1, 2, 4, 5}
'''

```

```
class Solution: # iterative
    def permute(self, nums: List[int]) -> List[List[int]]:
        res = []

        if nums is None or len(nums) == 0:
            return ans

        for num in nums:
            new_res = []

            for perm in res:
                for i in range(len(perm) + 1):
                    new_perm = perm[:i] + [num] + perm[i:]
                    new_res.append(new_perm)

            res = new_res

        return res

#####
```

```
class Solution: # iterative, single_perm.insert(index, num)
    def permute(self, nums: List[int]) -> List[List[int]]:
        ans = []

        if nums is None or len(nums) == 0:
            return ans

        for num in nums:
            tmp_list = []

            for single_perm in ans:
                for index in range(len(single_perm) + 1):
                    single_perm.insert(index, num)
                    tmp_list.append(single_perm.copy())
                    single_perm.pop(index)

            ans = tmp_list

        return ans
```

```
#####
```

```
'''
```

In Python 3, both the `discard()` and `remove()` methods of a set object are used to remove an element from the set, but there is one key difference:

- * `discard()` removes the specified element from the set if it is present, but does nothing if the element is not present.

- * `remove()` removes the specified element from the set if it is present, but raises a `KeyError` exception if the element is not present.

```
>>> a
{33, 66, 11, 44, 22, 55}
>>> a.discard(22)
>>> a
{33, 66, 11, 44, 55}
```

```
>>> a.discard(200)
>>>
>>> a.remove(200)
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
KeyError: 200
```

```
'''
```

```
'''
```

```
>>> v1 = set([])
>>> v2 = set()
>>>
>>> v1
set()
```



```

>>> v2
set()
'''

class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        res = []
        visited = set([])

        def dfs(nums, path, res, visited):
            if len(path) == len(nums):

                res.append(path + [])
                return

            for i in range(0, len(nums)):
                if i not in visited:
                    visited.add(i)
                    path.append(nums[i])
                    dfs(nums, path, res, visited)
                    path.pop()
                    visited.discard(i) # remove(i) will throw exception if i
not existing

        dfs(nums, [], res, visited)
        return res

#####

class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        def dfs(i):
            if i == n:

```

```
        ans.append(t[:])
        return
    for j in range(n):
        if not vis[j]:
            vis[j] = True
            t[i] = nums[j]
            dfs(i + 1)
            vis[j] = False

n = len(nums)
```

```
vis = [False] * n
t = [0] * n
ans = []
dfs(0)
return ans
```

◆ Quick Review

Key Insights

- Use backtracking to generate all possible permutations.
- Build each permutation by selecting numbers one at a time.
- When the current permutation reaches the length of nums, add it to the result.
- Backtracking allows you to undo choices and explore different orderings.

Space & Time

Time Complexity: $O(n * n!)$ – There are $n!$ permutations, and constructing each

permutation takes $O(n)$ time.

Space Complexity: $O(n)$ – Recursion depth is limited to n , plus additional space for the current permutation.

Solution

The solution employs a backtracking algorithm. A helper function is used to build permutations recursively. At each step, the algorithm iterates through the available numbers, adds one to the current permutation if it has not been used yet, and recurses to handle the remaining numbers. Once the current permutation matches the length of the input array, it is added to the result list. Then, by "backtracking" (removing the last number added), the algorithm explores alternative possibilities. This systematic exploration ensures that all possible permutations are generated.

Backtracking

□ #79 Word Search

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Backtracking · Matrix
Lists: Grind 169

Problem

Given an $m \times n$ grid of characters `board` and a string `word`, return `true` if `word` exists in the grid.

The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

Example 1:

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = [["A","B","C","E"], ["S","F","C","S"], ["A","D","E","E"]], word = "ABCCED"

Output: true

Example 2:

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = [["A","B","C","E"], ["S","F","C","S"], ["A","D","E","E"]], word = "SEE"

Output: true

Example 3:

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "ABCB"

Output: false

Constraints:

- $m == \text{board.length}$
- $n = \text{board}[i].\text{length}$
- $1 \leq m, n \leq 6$
- $1 \leq \text{word.length} \leq 15$
- board and word consists of only lowercase and uppercase English letters.

Follow up: Could you use search pruning to make your solution faster with a larger board?

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Given a grid of characters and a word, return true if the word exists
# as a path of adjacent (4-directional) non-repeated cells. Right?"
#
# "A few quick questions:
# Can the path revisit a cell? No – each cell used at most once per path.
# Case-sensitive? Yes."
#
# "Let me walk: board=[['A','B','C'],['S','F','C'],['A','D','E']], word='ABCCED'.
# A(0,0)→B(0,1)→C(0,2)→C(1,2)→E(2,2)→D(2,1) ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# For every cell as a potential start, run DFS along the word.
# Mark visited cells with a sentinel '#', unmark on backtrack.
# O(m*n*4^L) where L is word length – unavoidable in the worst case.
# Should I go ahead and optimize?"
class _79_WordSearch_Brute:
    def exist(self, board, word):
        m, n = len(board), len(board[0])
        def dfs(r, c, idx):
            if idx == len(word): return True
            if not (0<=r<m and 0<=c<n) or board[r][c] != word[idx]: return False
            tmp = board[r][c]; board[r][c] = '#'
            found = dfs(r+1,c,idx+1) or dfs(r-1,c,idx+1) or dfs(r,c+1,idx+1) or
            dfs(r,c-1,idx+1)
            board[r][c] = tmp; return found
        return any(dfs(r, c, 0) for r in range(m) for c in range(n))
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is exploring unpromising branches early. Two practical pruning
improvements:
# 1. Pre-check character frequencies: if board has fewer of any letter than word
needs, return False.
# 2. If word's last character is rarer in the board than the first, reverse the word
to
# start from the rarer end – prunes the search tree more aggressively.
```

```

#
# Time:  $O(m \times n \times 4^L)$  worst case but much faster with pruning. Space:  $O(L)$  call depth."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _79_WordSearch:
    def exist(self, board, word):
        from collections import Counter
        board_count = Counter(c for row in board for c in row)
        word_count = Counter(word)
        if any(word_count[c] > board_count[c] for c in word_count):
            return False
        if board_count[word[0]] > board_count[word[-1]]:
            word = word[::-1]
        m, n = len(board), len(board[0])
        def dfs(r, c, idx):
            if idx == len(word): return True
            if not (0 <= r < m and 0 <= c < n) or board[r][c] != word[idx]:
                return False
            board[r][c] = '#'
            found = (dfs(r+1,c,idx+1) or dfs(r-1,c,idx+1) or
                    dfs(r,c+1,idx+1) or dfs(r,c-1,idx+1))
            board[r][c] = word[idx]
            return found
        return any(dfs(r, c, 0) for r in range(m) for c in range(n))

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# word='ABCCED', board as above:
# Start at (0,0)='A'.
# dfs(0,0,0)→dfs(0,1,'B')→dfs(0,2,'C')→dfs(1,2,'C')→dfs(2,2,'E')→dfs(2,1,'D')
# idx=6=len → True ✓
#
# word='ABCB': after finding A→B→C, next must be 'B' adjacent to C. Board shows B at
# (0,1)
# but it's already marked '#' – backtrack → False ✓
#
# "No bugs. Restoring board[r][c] = word[idx] (not '#') correctly unmarks the
# sentinel."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(m \times n \times 4^L)$  worst. Space  $O(L)$  call depth.
# The two pruning heuristics dramatically cut average case without changing worst
# case.
# Follow-up: Word Search II (#212) – find ALL words from a list in one DFS using a
# Trie.
# Follow-up: allow diagonal moves – add 4 diagonal direction pairs."

```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def exist(self, board: list[list[str]], word: str) -> bool:
        m = len(board)
        n = len(board[0])

        def dfs(i: int, j: int, s: int) -> bool:
            if i < 0 or i == m or j < 0 or j == n:
                return False
            if board[i][j] != word[s] or board[i][j] == '*':
                return False

            if s == len(word) - 1:
                return True

            cache = board[i][j]
            board[i][j] = '*'
            isExist = (dfs(i + 1, j, s + 1) or
                      dfs(i - 1, j, s + 1) or
                      dfs(i, j + 1, s + 1) or
                      dfs(i, j - 1, s + 1))
            board[i][j] = cache

            return isExist

        return any(dfs(i, j, 0)
                   for i in range(m)
                   for j in range(n))
```

• LeetDoocs

```
class Solution:
    def exist(self, board: List[List[str]], word: str) -> bool:
        def dfs(i: int, j: int, k: int) -> bool:
            if k == len(word) - 1:
                return board[i][j] == word[k]
            if board[i][j] != word[k]:
                return False
            c = board[i][j]
            board[i][j] = "0"
            for a, b in pairwise((-1, 0, 1, 0, -1)):
                if dfs(i + a, j + b, k + 1):
                    return True
            board[i][j] = c
            return False

        for i in range(len(board)):
            for j in range(len(board[0])):
                if dfs(i, j, 0):
                    return True
        return False
```

```

        x, y = i + a, j + b
        ok = 0 <= x < m and 0 <= y < n and board[x][y] != "0"
        if ok and dfs(x, y, k + 1):
            return True
        board[i][j] = c
        return False

m, n = len(board), len(board[0])
return any(dfs(i, j, 0) for i in range(m) for j in range(n))

```

• SimplyLeet

Define the function to solve the Word Search problem.

```
def exist(board, word):
```

Get dimensions of the board.

```
    rows, cols = len(board), len(board[0])
```

Helper function for DFS search.

```
def dfs(r, c, index):
```

If the entire word is found, return True.

```
    if index == len(word):
```

```
        return True
```

Check boundaries and if current cell already used or not matching.

```
    if r < 0 or c < 0 or r >= rows or c >= cols or board[r][c] != word[ind
ex]:
        return False
```

Temporarily mark the cell as visited.

```
    temp = board[r][c]
```

```
    board[r][c] = "#"
```

Explore all four directions.

```

        found = (dfs(r + 1, c, index + 1) or
                 dfs(r - 1, c, index + 1) or
                 dfs(r, c + 1, index + 1) or
                 dfs(r, c - 1, index + 1))

        # Backtrack: restore the original cell value.
        board[r][c] = temp
        return found

# Initiate DFS from each cell in the grid.

```

```

for i in range(rows):
    for j in range(cols):
        if dfs(i, j, 0):
            return True
return False

```

```

# Example test case
if __name__ == "__main__":
    board = [
        ["A", "B", "C", "E"],

        ["S", "F", "C", "S"],
        ["A", "D", "E", "E"]
    ]
    word = "ABCCED"
    print(exist(board, word)) # Expected output: True

```

• LC.ca

```

class Solution:
    def exist(self, board: List[List[str]], word: str) -> bool:
        def dfs(i, j, cur): # cur: current char count
            if cur == len(word):
                return True
            if (
                i < 0
                or i >= m
                or j < 0
                or j >= n
            )

```

```

        or board[i][j] == '0'
        or word[cur] != board[i][j]
    ):
        return False
    t = board[i][j]
    board[i][j] = '0' # mark as visited
    for a, b in [[0, 1], [0, -1], [-1, 0], [1, 0]]:
        x, y = i + a, j + b
        if dfs(x, y, cur + 1):
            return True

    board[i][j] = t
    return False

m, n = len(board), len(board[0])
return any(dfs(i, j, 0) for i in range(m) for j in range(n))

```

◆ Quick Review

Key Insights

- Use Depth-First Search (DFS) to explore each cell in the board as a potential starting point.
- Employ backtracking to mark cells as visited when exploring a path and revert them back when backtracking.
- Check boundary conditions and ensure the current cell's character matches the corresponding character in the word.
- Given the small grid and word sizes, a recursive solution is both feasible and straightforward.

- Search pruning can be applied by stopping a DFS branch as soon as the current path does not match the prefix of the target word.

Space & Time

Time Complexity: $O(m * n * 3^L)$, where L is the length of the word (each DFS may branch to at most 3 further cells, excluding the coming cell).

Space Complexity: $O(L)$ due to recursion call stack (where L is the word length).

Solution

The solution uses DFS with backtracking to traverse the grid. For each cell, we: Check if the cell matches the first character of the word. Recursively explore the adjacent cells (up, down, left, right) while marking the current cell as visited. If the path does not lead to a complete match, backtrack by resetting the visited cell. The DFS stops if all characters in the word have been found in sequence. The algorithm leverages in-place modifications (or an auxiliary visited array) to avoid revisiting cells while exploring a candidate path.

Backtracking

□ #113 Path Sum II

MED[Open on LeetCode](#)

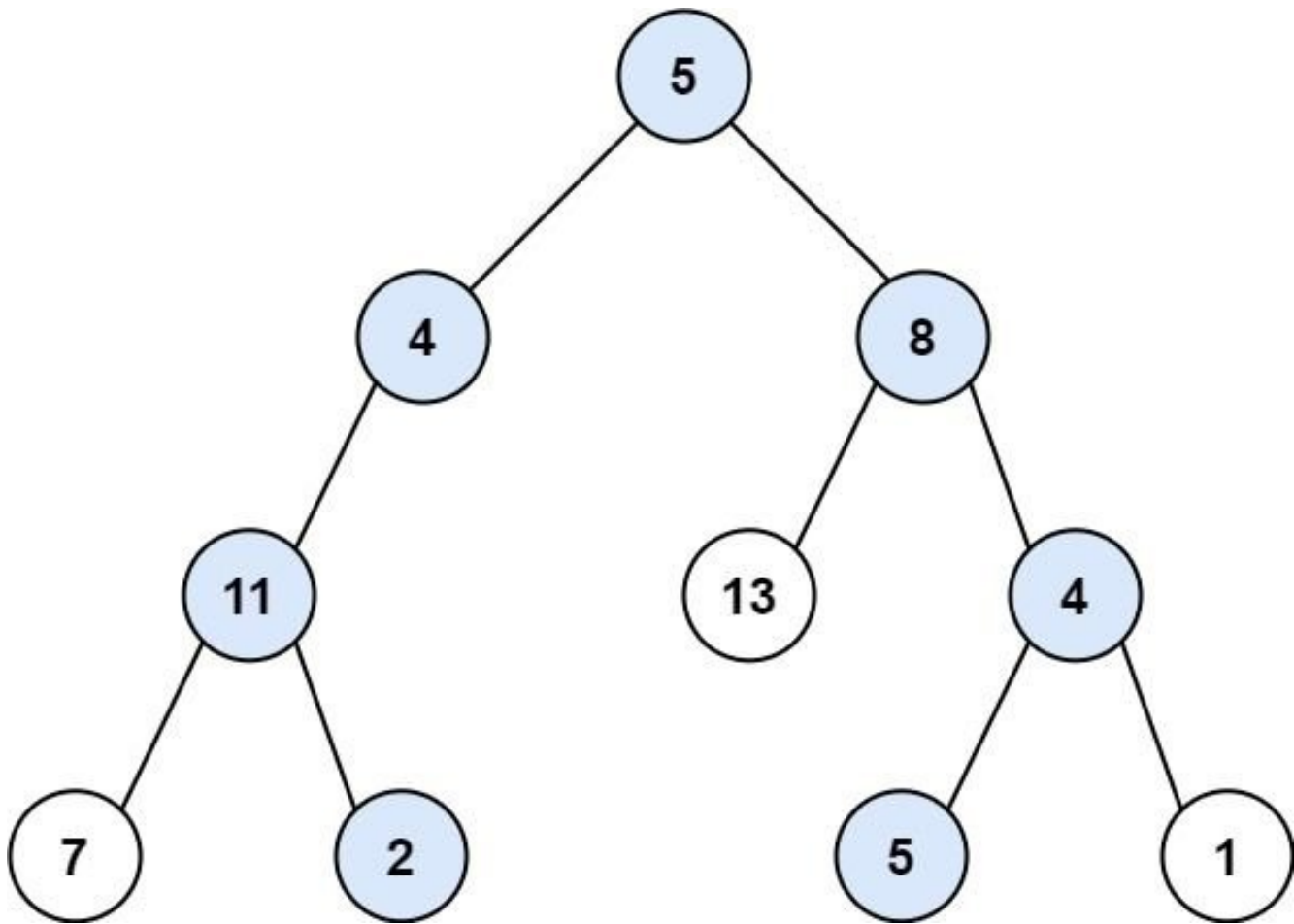
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Backtracking · Tree · DFS
Lists: Grind 169

Problem

Given the root of a binary tree and an integer targetSum, return all root-to-leaf paths where the sum of the node values in the path equals targetSum. Each path should be returned as a list of the node values, not node references.

A root-to-leaf path is a path starting from the root and ending at any leaf node. A leaf is a node with no children.

Example 1:



Input: root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22

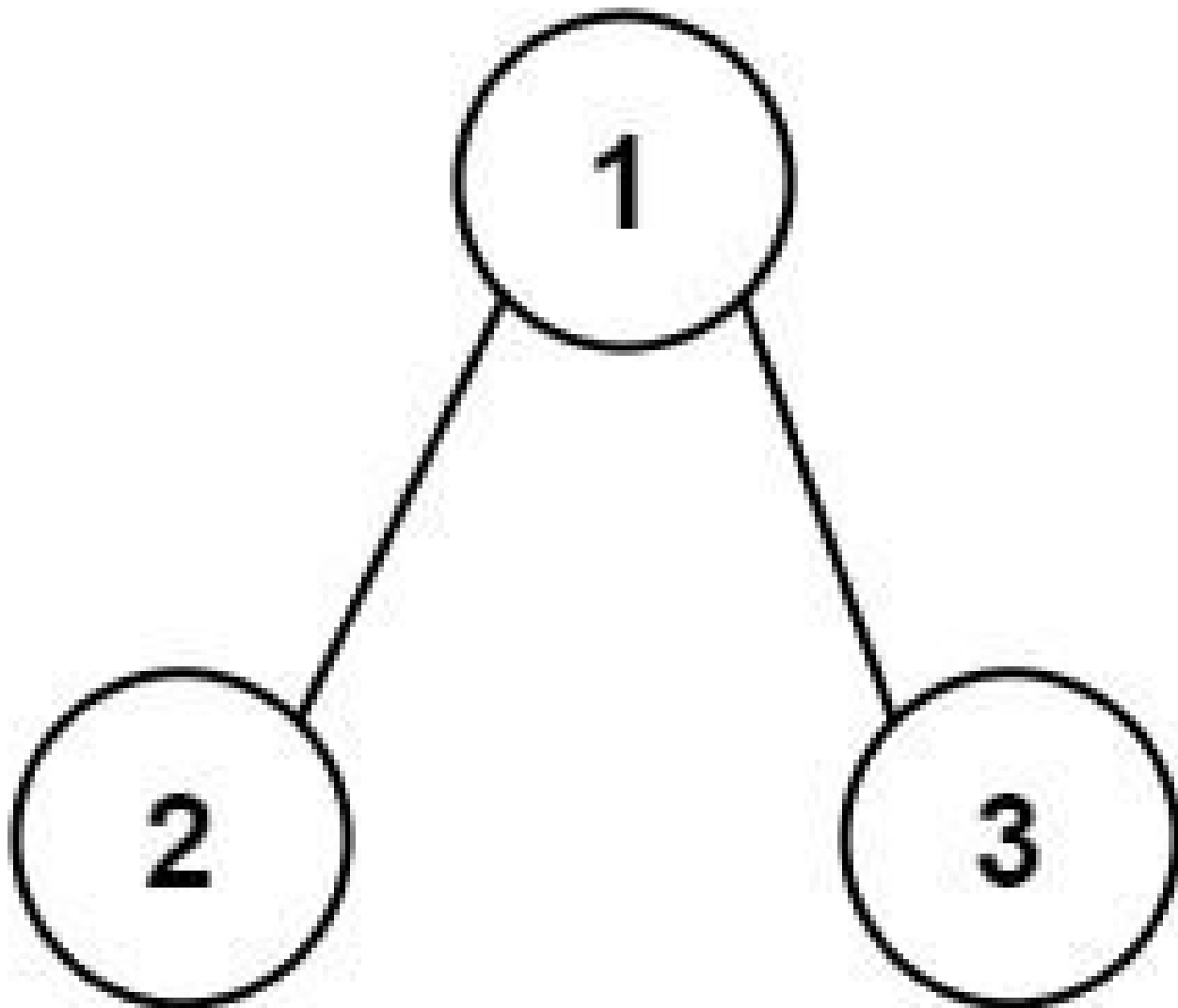
Output: [[5,4,11,2],[5,8,4,5]]

Explanation: There are two paths whose sum equals targetSum:

$5 + 4 + 11 + 2 = 22$

$5 + 8 + 4 + 5 = 22$

Example 2:



Input: root = [1,2,3], targetSum = 5
Output: []

Example 3:

Input: root = [1,2], targetSum = 0
Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 5000].
- $-1000 \leq \text{Node.val} \leq 1000$
- $-1000 \leq \text{targetSum} \leq 1000$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Return all root-to-leaf paths where node values sum to targetSum.
# A leaf is a node with no children.
# Return the actual paths (node values), not just count. Right?"
#
# "A few quick questions:
# Can values be negative? Yes – so we can't prune on sum exceeding target.
# Empty tree → return []."
#
# "Let me walk: tree=[5,4,8,11,null,13,4,7,2,null,null,5,1], target=22.
# 5→4→11→2=22 ✓, 5→8→4→5=22 ✓ → [[5,4,11,2],[5,8,4,5]] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# DFS collecting all root-to-leaf paths, then filter by sum.
# O(n*L) where L = average path length (tree height).
# Should I go ahead and optimize?"
class _113_PathSumII_Brute:
    def pathSum(self, root, targetSum):
        all_paths = []
        def dfs(node, path):
            if not node: return
            path.append(node.val)
            if not node.left and not node.right: all_paths.append(list(path))
            dfs(node.left, path); dfs(node.right, path)
            path.pop()
        dfs(root, [])
        return [p for p in all_paths if sum(p) == targetSum]
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is summing each complete path after collecting it.
# Track remaining = targetSum - node.val as we descend.
# Check remaining == 0 at leaves only – avoids post-collection summing.
#
# Time: O(n*L) – same asymptotic. Space: O(L) call depth."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _113_PathSumII:
    def pathSum(self, root, targetSum):
        result = []
        def backtrack(node, remaining, path):
```

```

        if not node:
            return
        path.append(node.val)
        remaining -= node.val
        if not node.left and not node.right and remaining == 0:
            result.append(list(path))
        else:
            backtrack(node.left, remaining, path)
            backtrack(node.right, remaining, path)
        path.pop()
    backtrack(root, targetSum, [])
    return result

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

tree=[5,4,8,11,null,13,4,7,2,null,null,5,1], target=22:

backtrack(5,22,[]): path=[5],rem=17. backtrack(4,17): path=[5,4],rem=13.

backtrack(11,13): path=[5,4,11],rem=2. backtrack(7,2): rem=-5, leaf, ≠0 pop.

backtrack(2,2): rem=0, leaf, =0 → append [5,4,11,2] ✓. Pop back up.

backtrack(8,17): ... → [5,8,4,5] ✓

#

"No bugs. path.pop() after each node correctly backtracks the path list."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \times L)$: n nodes visited, copying path at leaves is $O(L)$ each.

Space $O(L)$: call stack depth = tree height.

Follow-up: Path Sum I (#112) – return bool, not paths. Can short-circuit on first match.

Follow-up: Path Sum III (#437) – any downward path summing to target; use prefix sum hashmap."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def pathSum(self, root: TreeNode, summ: int) -> list[list[int]]:
        ans = []

        def dfs(root: TreeNode, summ: int, path: list[int]) -> None:
            if not root:
                return
            if root.val == summ and not root.left and not root.right:
                ans.append(path + [root.val])
            return

```

```

        dfs(root.left, summ - root.val, path + [root.val])
        dfs(root.right, summ - root.val, path + [root.val])

    dfs(root, summ, [])
    return ans

```

• LeetCode

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def pathSum(self, root: Optional[TreeNode], targetSum: int) -> List[List[int]]:
        def dfs(root, s):
            if root is None:
                return
            s += root.val
            t.append(root.val)
            if root.left is None and root.right is None and s == targetSum:
                ans.append(t[:])
            dfs(root.left, s)
            dfs(root.right, s)
            t.pop()

        ans = []

        t = []
        dfs(root, 0)
        return ans

```

• SimplyLeet

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

def pathSum(root, targetSum):
    result = [] # This will store all valid paths

    def dfs(node, current_path, remaining_sum):
        if not node:
            return # Base case: reached the end of a path

        # Add the current node's value to the path
        current_path.append(node.val)

        # Check if it's a leaf and if the path sum equals targetSum
        if not node.left and not node.right and node.val == remaining_sum:
            result.append(list(current_path)) # Add a copy of the current path

        else:
            # Continue the search in the left and right subtrees
            dfs(node.left, current_path, remaining_sum - node.val)
            dfs(node.right, current_path, remaining_sum - node.val)

        # Backtrack: remove the last element before returning to the caller
        current_path.pop()

    dfs(root, [], targetSum)
    return result

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def pathSum(self, root: Optional[TreeNode], targetSum: int) -> List[List[int]]:
        def dfs(root, s):
            if root is None:
                return
            s += root.val
            t.append(root.val)
            if root.left is None and root.right is None and s == targetSum:
                ans.append(t[:])
            dfs(root.left, s)
            dfs(root.right, s)
            t.pop()

        ans = []

        t = []
        dfs(root, 0)
        return ans

```

◆ Quick Review

Key Insights

- Use Depth-First Search (DFS) to explore all paths from the root to the leaves.
- Backtracking is essential: add the current node value to the path before the recursive calls and remove it (backtrack) after exploring both subtrees.

- At each leaf, check if the accumulated sum equals targetSum.
- Maintain a running sum or subtract the current node's value from the target sum as you traverse.

Space & Time

Time Complexity: $O(N^2)$ in the worst-case scenario (where N is the number of nodes) when most nodes are leaf nodes, since each potential path copy operation can be $O(N)$.

Space Complexity: $O(N)$ for the recursion stack and the path storage, where N is the height of the tree.

Solution

We solve the problem using a recursive DFS approach combined with backtracking. The idea is to traverse the tree while maintaining the current path and the remaining sum needed to reach targetSum. When a leaf node is reached (node with no children), we check if the remaining sum equals the leaf's value – if yes, we record a copy of the current path. Data structures and algorithmic approaches used: Recursion for DFS: to traverse to each leaf. List

(or Array) for storing the current path.
Backtracking: to remove the last added element when stepping back up the recursion stack. Copying the path when a valid leaf is reached to store it in the final result. A common pitfall is to forget to backtrack, which would result in incorrect paths being recorded.

Greedy

Round 5 · Mid · Medium · 5 questions

Greedy

□ #134 Gas Station

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy
Lists: Grind 169

Problem

There are n gas stations along a circular route, where the amount of gas at the i th station is $gas[i]$.

You have a car with an unlimited gas tank and it costs $cost[i]$ of gas to travel from the i th station to its next $(i + 1)$ th station. You begin the journey with an empty tank at one of the gas stations.

Given two integer arrays gas and $cost$, return the starting gas station's index if you can travel around the circuit once in the clockwise direction, otherwise return -1 . If there exists a solution, it is guaranteed to be unique.

Example 1:

Input: gas = [1,2,3,4,5], cost = [3,4,5,1,2]

Output: 3

Explanation:

Start at station 3 (index 3) and fill up with 4 unit of gas. Your tank = $0 + 4 = 4$

Travel to station 4. Your tank = $4 - 1 + 5 = 8$

Travel to station 0. Your tank = $8 - 2 + 1 = 7$

Travel to station 1. Your tank = $7 - 3 + 2 = 6$

Travel to station 2. Your tank = $6 - 4 + 3 = 5$

Example 2:

Input: gas = [2,3,4], cost = [3,4,3]

Output: -1

Explanation:

You can't start at station 0 or 1, as there is not enough gas to travel to the next station.

Let's start at station 2 and fill up with 4 unit of gas. Your tank = $0 + 4 = 4$

Travel to station 0. Your tank = $4 - 3 + 2 = 3$

Travel to station 1. Your tank = $3 - 3 + 3 = 3$

You cannot travel back to station 2, as it requires 4 unit of gas but you only have 3.

Constraints:

- $n == \text{gas.length} == \text{cost.length}$
- $1 \leq n \leq 105$
- $0 \leq \text{gas}[i], \text{cost}[i] \leq 104$
- The input is generated such that the answer is unique.

My LeetCode Solution

- My LeetCode Solution

```

class Solution:
    def canCompleteCircuit(self, gas: List[int], cost: List[int]) -> int:
        if sum(gas) < sum(cost):
            return -1

        ans = 0
        s = 0
        for i in range(len(cost)):
            s += gas[i] - cost[i]
            if s < 0:

                s = 0
                ans = i + 1
        return ans

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Let me make sure I understand the problem correctly.
# n gas stations in a circle. gas[i] = fuel at i, cost[i] = fuel to reach i+1.
# Start with empty tank. Find the starting station to complete the circuit.
# If impossible return -1; the solution is guaranteed unique if it exists. Right?"
#
# "A few quick questions:
# Can we start at any station? Yes. Must return to the starting station? Yes."
#
# "Let me walk: gas=[1,2,3,4,5], cost=[3,4,5,1,2] → start at station 3.
# 0+4-1=3 → +5=8 → +1=9 → +2=11 → +3=14 → -4=10 → return to 3 ✓"

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with brute force to lock in the logic.
# Try each station as start. Simulate the full circuit; if tank never goes negative,
return it.
# O(n²) time, O(1) space.
# Should I go ahead and optimize?"
class _134_GasStation_Brute:
    def canCompleteCircuit(self, gas, cost):
        n = len(gas)
        for start in range(n):
            tank = 0; ok = True
            for i in range(n):
                tank += gas[(start + i) % n] - cost[(start + i) % n]
                if tank < 0: ok = False; break
            if ok: return start
        return -1

```

PHASE 3 — OPTIMIZE

"The bottleneck is trying every start independently.

Two greedy observations:

1. If total gas < total cost, no solution – return -1.

2. If we run out of gas starting at s after reaching index i,

no station between s and i can be a valid start (they'd all begin with less gas).

So reset candidate to i+1 and continue.

#

Time: O(n). Space: O(1)."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```

class _134_GasStation:
    def canCompleteCircuit(self, gas, cost):
        total_surplus = 0
        current_tank = 0
        candidate = 0
        for i in range(len(gas)):
            net = gas[i] - cost[i]
            total_surplus += net
            current_tank += net
            if current_tank < 0:
                candidate = i + 1
                current_tank = 0
        return candidate if total_surplus >= 0 else -1

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

gas=[1,2,3,4,5], cost=[3,4,5,1,2]: total=15-15=0≥0.

i=0: net=-2, tank=-2<0 → cand=1, tank=0. i=1: net=-2, tank=-2<0 → cand=2, tank=0.

i=2: net=-2, tank=-2<0 → cand=3, tank=0. i=3: net=3, tank=3. i=4: net=3, tank=6.

return cand=3 ✓

#

gas=[2,3,4], cost=[3,4,3]: total=9-10=-1<0 → return -1 ✓

#

"No bugs. The candidate reset skips all stations from the old candidate to i."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time O(n): single pass. Space O(1): three integer variables.

The key insight: if total gas ≥ total cost, a valid start always exists;

the greedy candidate is guaranteed to be it.

Follow-up: what if there are multiple valid starting points? The problem guarantees uniqueness

but the greedy still finds the first valid one.

Follow-up: minimum cost to fill gas (variant) – different problem, uses a monotonic stack."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def canCompleteCircuit(self, gas: list[int], cost: list[int]) -> int:
        ans = 0
        net = 0
        summ = 0

        # Try to start from each index.
        for i in range(len(gas)):
            net += gas[i] - cost[i]
            summ += gas[i] - cost[i]

            if summ < 0:
                summ = 0
                ans = i + 1 # Start from the next index.

        return -1 if net < 0 else ans
```

• LeetDoocs

```
class Solution:
    def canCompleteCircuit(self, gas: List[int], cost: List[int]) -> int:
        n = len(gas)
        i = j = n - 1
        cnt = s = 0
        while cnt < n:
            s += gas[j] - cost[j]
            cnt += 1
            j = (j + 1) % n
            while s < 0 and cnt < n:

                i -= 1
                s += gas[i] - cost[i]
                cnt += 1
        return -1 if s < 0 else i
```

• SimplyLeet

```
# Define a function to find starting gas station index
def canCompleteCircuit(gas, cost):
    total_surplus = 0 # Total gas surplus after completing the route
    current_surplus = 0 # Gas surplus while iterating from a candidate start
    station
    start_index = 0 # Candidate starting station index

    # Loop through each station
    for i in range(len(gas)):
        # Calculate net gas after leaving station i
        surplus = gas[i] - cost[i]

        total_surplus += surplus
        current_surplus += surplus

        # If current surplus becomes negative, station i is not a valid start.
        if current_surplus < 0:
            # Reset starting station to the next station
            start_index = i + 1
            current_surplus = 0 # Reset current surplus
    # Check if overall surplus is non-negative
    return start_index if total_surplus >= 0 else -1

# Example usage:
gas = [1,2,3,4,5]
cost = [3,4,5,1,2]
print(canCompleteCircuit(gas, cost)) # Output: 3
```

• LC.ca

```
class Solution:
    def canCompleteCircuit(self, gas: List[int], cost: List[int]) -> int:
        n = len(gas)
        i = j = n - 1
        cnt = s = 0
        while cnt < n:
            s += gas[j] - cost[j]
            cnt += 1
            j = (j + 1) % n
            while s < 0 and cnt < n:
```

```
i -= 1
s += gas[i] - cost[i]
cnt += 1
return -1 if s < 0 else i
```

◆ Quick Review

Key Insights

- If the total amount of gas available is less than the total travel cost, then completing the circuit is impossible.
- A greedy approach can be used by keeping track of the current surplus of gas. If the surplus becomes negative at any station, the journey cannot start from any station before that point.
- Restart the journey from the next station whenever the surplus goes negative.
- The problem guarantees that if a solution exists, it is unique.

Space & Time

Time Complexity: $O(n)$, where n is the number of gas stations, as we traverse the list only once.

Space Complexity: $O(1)$, as no extra space is used other than a few variables.

Solution

The solution involves iterating over the list of stations while maintaining two variables: one for the current gas surplus and one for the total gas surplus. If at any station the current surplus drops below zero, the current starting station is not viable and we set the starting station to the next index and reset the current surplus. After one pass, if the total gas surplus is non-negative, then the identified station is the valid starting point.

Greedy

□ #179 Largest Number

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Greedy · Sorting
Lists: Grind 169

Problem

Given a list of non-negative integers `nums`, arrange them such that they form the largest number and return it.

Since the result may be very large, so you need to return a string instead of an integer.

Example 1:

```
Input: nums = [10,2]  
Output: "210"
```

Example 2:

```
Input: nums = [3,30,34,5,9]  
Output: "9534330"
```

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 10^9$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def largestNumber(self, nums: List[int]) -> str:
        def compare(x,y):
            if x+y < y+x:
                return 1
            elif x+y > y+x:
                return -1
            else:
                return 0
        d = [str(n) for n in nums]

        d.sort(key = cmp_to_key(compare))
        return "0" if d[0] == "0" else "".join(d)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Arrange non-negative integers so they form the largest number. Return as a string.

Edge: if the result starts with '0', return '0' (e.g. [0,0] → '0'). Right?"

#

"A few quick questions:

Values can be 0? Yes. Return a string because the result may be very large? Yes."

#

"Let me walk: nums=[10,2] → '210' > '102' → '210' ✓

nums=[3,30,34,5,9] → '9534330' ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Try all permutations of nums, form the concatenated string, return the maximum.

O(n! * n) – completely infeasible for large n.

Should I go ahead and optimize?"

```
class _179_LargestNumber_Brute:
```

```
    def largestNumber(self, nums):
```

```
        from itertools import permutations
```

```
        return max(''.join(str(n) for n in perm) for perm in permutations(nums))
```

PHASE 3 — OPTIMIZE

"The bottleneck is generating all permutations.

Custom comparator: to decide whether a comes before b, compare str(a)+str(b) vs str(b)+str(a).

```

# Sort all numbers by this comparator; concatenate; strip leading zeros if needed.
#
# Time: O(n log n). Space: O(n) for the string array."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
from functools import cmp_to_key
class _179_LargestNumber:
    def largestNumber(self, nums):
        strs = [str(n) for n in nums]
        def compare(a, b):
            return (1 if b + a > a + b else -1)
        strs.sort(key=cmp_to_key(compare))
        result = ''.join(strs)
        return '0' if result[0] == '0' else result

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# nums=[10,2]: compare('10','2'): '210' vs '102' → '210'>'102' → 2 before 10.
# sorted=['2','10']. → '210' ✓
#
# nums=[3,30,34,5,9]: compare determines order. '9'>'5'>'34'>'3'>'30'.
# → '9534330' ✓
#
# nums=[0,0]: sorted=['0','0']. result='00'. result[0]='0' → return '0' ✓
#
# "No bugs. The leading-zero check handles [0,0] without special-casing."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log n * k): sorting with O(k) string comparisons. Space O(n*k): string
array.
# The comparator is transitive and total-ordering – provably correct for any input.
# Follow-up: smallest number from array – same comparator, ascending sort.
# Follow-up: if numbers can be very large (BigInteger) – convert to strings first
anyway."

```

Community Solutions (Python)

• WalkCC


```
class LargerStrKey(str):
    def __lt__(x: str, y: str) -> bool:
        return x + y > y + x

class Solution:
    def largestNumber(self, nums: list[int]) -> str:
        return ''.join(sorted(map(str, nums), key=LargerStrKey)).lstrip('0') or '0'
```

• LeetDoocs

```
class Solution:
    def largestNumber(self, nums: List[int]) -> str:
        nums = [str(v) for v in nums]
        nums.sort(key=cmp_to_key(lambda a, b: 1 if a + b < b + a else -1))
        return "0" if nums[0] == "0" else "".join(nums)
```

• SimplyLeet

```
# Import cmp_to_key to convert a comparator function to a key function for
# sorting
from functools import cmp_to_key

# Define the comparator function
def compare(x, y):
    # Compare two possible orders: x+y and y+x
    if x + y > y + x:
        return -1 # x should come before y
    elif x + y < y + x:
        return 1 # y should come before x
    else:
        return 0 # both orders are equal

def largestNumber(nums):
    # Convert all integers to strings
    nums_str = list(map(str, nums))
    # Sort the strings using the custom comparator
    nums_str.sort(key=cmp_to_key(compare))
    # If the largest number is '0', the entire number is 0 so return '0'
    if nums_str[0] == "0":
        return "0"
```

```

        return "0"
    # Concatenate the sorted numbers and return the result
    return "".join(nums_str)

# Example usage:
print(largestNumber([3, 30, 34, 5, 9])) # Output: "9534330"
```

• LC.ca

```

class Solution:
    def largestNumber(self, nums: List[int]) -> str:
        nums = [str(v) for v in nums]
        nums.sort(key=cmp_to_key(lambda a, b: 1 if a + b < b + a else -1))
        return "0" if nums[0] == "0" else "".join(nums)
```

◆ Quick Review

Key Insights

- The order in which numbers appear drastically affects the concatenated result.
- A custom sorting strategy is required where two numbers are compared by checking which concatenation (first-second vs second-first) yields a larger number.
- Edge case: When the numbers are all zeros, the result should be "0" instead of several leading zeros.

Space & Time

Time Complexity: $O(n \log n * k)$, where n is the number of elements in the array and k is the

maximum number of digits in a number (due to string comparisons in the custom sort).

Space Complexity: $O(n * k)$ for storing and processing the strings.

Solution

We solve the problem by converting all integers to strings so that we can use string concatenation to compare the results. A custom comparator is defined that, given two number strings, compares the concatenated results in both orders. By sorting the array using this comparator in descending order, we ensure that placing the highest contributing number first yields the largest possible number. After sorting, a check is performed to return "0" if the highest number is "0". Finally, the sorted strings are concatenated to form the answer.

Greedy

□ ★ #280 Wiggle Sort ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Greedy · Sorting
Lists: ★ Premium | Premium 98

Problem

Given an integer array `nums`, reorder it such that `nums[0] <= nums[1] >= nums[2] <= nums[3]....`
You may assume the input array always has a valid answer.

Example 1:

```
Input: nums = [3,5,2,1,6,4]
Output: [3,5,1,6,2,4]
Explanation: [1,6,2,5,3,4] is also accepted.
```

Example 2:

```
Input: nums = [6,6,5,6,3,8]
Output: [6,6,5,6,3,8]
```

Constraints:

- $1 \leq \text{nums.length} \leq 5 * 10^4$
- $0 \leq \text{nums}[i] \leq 10^4$

- It is guaranteed that there will be an answer for the given input nums.

Follow up: Could you solve the problem in $O(n)$ time complexity?

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def wiggleSort(self, nums: List[int]) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        n = len(nums)
        for i in range(n-1):
            if i%2 == 0 and nums[i] > nums[i+1]:
                nums[i], nums[i+1] = nums[i+1], nums[i]
            if i%2 == 1 and nums[i] < nums[i+1]:
                nums[i], nums[i+1] = nums[i+1], nums[i]
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Reorder nums in-place so: nums[0] <= nums[1] >= nums[2] <= nums[3] >= ...
# Multiple valid answers exist. Right?"
#
# "A few quick questions:
# Any valid wiggle order is accepted? Yes. Modify in-place, no return? Yes."
#
# "Let me walk: nums=[3,5,2,1,6,4] → [3,5,1,6,2,4] ✓ (one valid answer)"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Sort the array, then interleave: place smaller half at even indices, larger half
at odd.
#  $O(n \log n)$  sort +  $O(n)$  interleave.
# Should I go ahead and optimize?"
class _280_WiggleSort_Brute:
```

```
def wiggleSort(self, nums):
    nums.sort()
    for i in range(1, len(nums) - 1, 2):
        nums[i], nums[i + 1] = nums[i + 1], nums[i]
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the  $O(n \log n)$  sort.
# Greedy one-pass  $O(n)$ : check each index  $i$ .
# Odd index ( $i \% 2 == 1$ ):  $nums[i]$  should be  $\geq nums[i-1]$ . If not, swap.
# Even index ( $i \% 2 == 0$ ) for  $i > 0$ :  $nums[i]$  should be  $\leq nums[i-1]$ . If not, swap.
# A local swap always fixes the current violation without breaking prior pairs.
#
# Time:  $O(n)$ . Space:  $O(1)$ ."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _280_WiggleSort:
    def wiggleSort(self, nums):
        for i in range(1, len(nums)):
            if (i % 2 == 1 and nums[i] < nums[i-1]) or (i % 2 == 0 and nums[i] >
nums[i-1]):
                nums[i], nums[i-1] = nums[i-1], nums[i]
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# nums=[3,5,2,1,6,4]:
# i=1(odd):  $5 \geq 3$  ✓. i=2(even):  $2 \leq 5$  ✓. i=3(odd):  $1 < 2 \rightarrow$  swap  $\rightarrow$  [3,5,1,2,6,4].
# i=4(even):  $6 > 2 \rightarrow$  swap  $\rightarrow$  [3,5,1,6,2,4]. i=5(odd):  $4 \geq 2$  ✓.  $\rightarrow$  [3,5,1,6,2,4] ✓
#
# nums=[6,6,5,6,3,8]: all conditions already satisfied  $\rightarrow$  no swaps ✓
#
# "No bugs. The swap at position  $i$  only affects the pair  $(i-1, i)$ , never undoes
earlier swaps."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n)$ : single pass. Space  $O(1)$ : in-place swaps.
# Greedy correctness: swapping adjacent elements to fix one condition never
invalidates the
# previously satisfied condition – because the new value at  $i-1$  satisfies the prior
pair.
# Follow-up: Wiggle Sort II (#324) – strict inequalities
nums[0] < nums[1] > nums[2] < ...
# Requires median partitioning to guarantee strict separation.
# Follow-up: count wiggle subsequences (#376) – DP/greedy, no reordering needed."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def wiggleSort(self, nums: list[int]) -> None:
        # 1. If i is even, then nums[i] <= nums[i - 1].
        # 2. If i is odd, then nums[i] >= nums[i - 1].
        for i in range(1, len(nums)):
            if (i % 2 == 0 and nums[i] > nums[i - 1] or
                i % 2 == 1 and nums[i] < nums[i - 1]):
                nums[i], nums[i - 1] = nums[i - 1], nums[i]
```

• LeetCode

```
class Solution:
    def wiggleSort(self, nums: List[int]) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        for i in range(1, len(nums)):
            if (i % 2 == 1 and nums[i] < nums[i - 1]) or (
                i % 2 == 0 and nums[i] > nums[i - 1]
            ):
                nums[i], nums[i - 1] = nums[i - 1], nums[i]
```

• SimplyLeet

```
# Python solution for Wiggle Sort
def wiggleSort(nums):
    # Iterate over the list starting from index 1
    for i in range(1, len(nums)):
        # Check the condition based on the parity of the index
        if i % 2 == 1:
            # For odd index, if previous element is greater, swap them
            if nums[i] < nums[i-1]:
                nums[i], nums[i-1] = nums[i-1], nums[i]
        else:
```



```

        # For even index, if previous element is smaller, swap them
        if nums[i] > nums[i-1]:
            nums[i], nums[i-1] = nums[i-1], nums[i]
    return nums

# Example usage:
# nums = [3,5,2,1,6,4]
# print(wiggleSort(nums))

```

• LC.ca

```

'''
math prove:

example [3, 5, 2, 1]
when reaching 2, all good
then next, is 1,
so here comes the question, will the number after 2 violating the wiggle rule?
answer is no.

```

in example, when reaching 2:

* if [3, 5, 2, 10], so 10 is bigger than 2, then no swap needed according to the if () conditions

* if [3, 5, 2, 1], so previous round, guaranteed that 2 is smaller than 5.
And, now a swap needed meaning the number is smaller than 2, i.e. must be smaller than 5, so wiggle rule is still good

```

'''
class Solution:
    def wiggleSort(self, nums: List[int]) -> None:
        '''
        Do not return anything, modify nums in-place instead.

```

```

        """
        for i in range(1, len(nums)):
            if (i % 2 == 1 and nums[i] < nums[i - 1]) or (
                i % 2 == 0 and nums[i] > nums[i - 1]
            ):
                nums[i], nums[i - 1] = nums[i - 1], nums[i]

```

```
#####
```

```
class Solution: # sort list first
```

```

    def wiggleSort(self, nums: List[int]) -> None:
        nums.sort() # Sort the list in ascending order
        n = len(nums)

        for i in range(1, n - 1, 2):
            nums[i], nums[i + 1] = nums[i + 1], nums[i] # Swap adjacent
elements

```

```

        """
        If the list has an odd length, the last element will be compared with
the second-to-last element
        and swapped if necessary to form a wiggle sequence.

```

```

        """
        if n % 2 == 0 or nums[-1] < nums[-2]:
            nums[-1], nums[-2] = nums[-2], nums[-1]

```

```
#####
```

```
import random
```

```

class Solution:
    def wiggleSort(self, nums: List[int]) -> None:
        """

```

```

Do not return anything, modify nums in-place instead.
"""

if len(nums) <= 1:
    return

# Find the median using quickselect
n = len(nums)
mid = self.quickselect(nums, 0, n-1, (n-1)//2)

# Create an empty array with the same length as nums

ans = [None] * n

# Assign the median to every other element
i = 0
j = n-1 if n % 2 == 1 else n-2
for k in range(n):
    if nums[k] < mid:
        ans[j] = nums[k]
        j -= 2
    elif nums[k] > mid:
        ans[i] = nums[k]
        i += 2
    else:
        ans[j+1] = mid

# Copy the result back to nums
nums[:] = ans

def quickselect(self, nums, left, right, k):
    """
    Returns the k-th smallest element in nums[left:right+1].
    """
    if left == right:
        return nums[left]

    # Partition the subarray around a pivot element
    pivot_idx = self.partition(nums, left, right)

    # Recursively select on the left or right subarray
    if k == pivot_idx:

```

```

        return nums[k]
    elif k < pivot_idx:
        return self.quickselect(nums, left, pivot_idx-1, k)
    else:
        return self.quickselect(nums, pivot_idx+1, right, k)

def partition(self, nums, left, right):
    """
    Partitions the subarray nums[left:right+1] using the first element as
    the pivot.
    Returns the final index of the pivot element.

    """
    pivot = nums[left]
    i, j = left+1, right
    while i <= j:
        if nums[i] <= pivot:
            i += 1
        elif nums[j] > pivot:
            j -= 1
        else:
            nums[i], nums[j] = nums[j], nums[i]

    nums[left], nums[j] = nums[j], nums[left]
    return j

```

◆ Quick Review

Key Insights

- The alternating "wiggle" requirement can be satisfied by ensuring two conditions:
- For every odd index i , $\text{nums}[i]$ should be greater than or equal to $\text{nums}[i-1]$.
- For every even index i (where $i > 0$), $\text{nums}[i]$ should be less than or equal to $\text{nums}[i-1]$.

- A single pass through the array can ensure these conditions by swapping adjacent elements when the condition is violated.
- This greedy approach works because the local adjustments guarantee the overall wiggle pattern without needing to sort the entire array.

Space & Time

Time Complexity: $O(n)$ — Only one pass through the array is needed.

Space Complexity: $O(1)$ — The modifications are done in-place without extra data structures.

Solution

We can achieve a wiggle sort in one pass by iterating through the array and, at each index i (starting from 1), checking if the pair $(\text{nums}[i-1], \text{nums}[i])$ satisfies the wiggle condition. If i is odd, $\text{nums}[i]$ should be at least $\text{nums}[i-1]$; if not, we swap them. Conversely, if i is even and $\text{nums}[i]$ is greater than $\text{nums}[i-1]$, then we swap them. These local swaps adjust the order incrementally while ensuring that the overall wiggle pattern is

maintained. This greedy approach works because only two adjacent numbers are involved in any violation and fixing them locally does not disturb the rest of the order.

Greedy

□ #954 Array of Doubled Pairs

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Greedy · Sorting
Lists: CodeSignal

Problem

Given an integer array of even length `arr`, return `true` if it is possible to reorder `arr` such that $arr[2 * i + 1] = 2 * arr[2 * i]$ for every $0 \leq i < \text{len}(arr) / 2$, or `false` otherwise.

Example 1:

Input: `arr = [3,1,3,6]`
Output: `false`

Example 2:

Input: `arr = [2,1,2,6]`
Output: `false`

Example 3:

Input: `arr = [4,-2,2,-4]`
Output: `true`
Explanation: We can take two groups, `[-2,-4]` and `[2,4]` to form `[-2,-4,2,4]` or `[2,4,-2,-4]`.

Constraints:

- $2 \leq \text{arr.length} \leq 3 * 10^4$
- arr.length is even.
- $-10^5 \leq \text{arr}[i] \leq 10^5$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def canReorderDoubled(self, arr: List[int]) -> bool:
        arr.sort(key=abs)
        m = Counter(arr)
        for x in arr:
            if m[x] == 0:
                continue
            if m[2*x] == 0:
                return False

            m[x] -= 1
            m[2*x] -= 1
        return True
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Can we reorder arr so every two elements form a pair where $\text{arr}[2i+1] = 2 * \text{arr}[2i]$?

Return true if such a pairing exists. Right?"

#

"A few quick questions:

Values can be negative? Yes – e.g. $[-2, -4]$ works since $-4 = -2 * 2$.

The array has even length? Yes – guaranteed."

#

"Let me walk: $\text{arr} = [4, -2, 2, -4] \rightarrow$ pair $(-2, -4)$ and $(2, 4) \rightarrow$ true ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Try all ways to pair elements, checking $\text{pair}[1] = 2 * \text{pair}[0]$.

$O(n! / (n/2)! / 2^{(n/2)})$ – combinatorially explosive.

Should I go ahead and optimize?"

```

class _954_ArrayOfDoubledPairs_Brute:
    def canReorderDoubled(self, arr):
        from itertools import permutations
        n = len(arr)
        for perm in permutations(arr):
            if all(perm[2*i+1] == 2*perm[2*i] for i in range(n//2)):
                return True
        return False

# PHASE 3 — OPTIMIZE
# "The bottleneck is the combinatorial search.
# Greedy: sort by absolute value. Process the smallest absolute value element first.
# For each element x, we need to find and consume its pair 2*x.
# A Counter tracks remaining elements. If 2*x is not available, return False.
#
# Time: O(n log n). Space: O(n) for the Counter."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _954_ArrayOfDoubledPairs:
    def canReorderDoubled(self, arr):
        from collections import Counter
        count = Counter(arr)
        for x in sorted(count.keys(), key=abs):
            if count[x] > 0:
                if count[2 * x] < count[x]:
                    return False
                count[2 * x] -= count[x]
                count[x] = 0
        return True

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# arr=[4,-2,2,-4]: count={4:1,-2:1,2:1,-4:1}.
# sorted by abs: [-2,-4,2,4].
# x=-2: count[-2]=1>0. count[-4]=1>=1 ✓. count[-4]=0, count[-2]=0.
# x=-4: count[-4]=0, skip. x=2: count[2]=1>0. count[4]=1>=1 ✓. count[4]=0.
# x=4: count[4]=0, skip. → True ✓
#
# arr=[3,1,3,6]: count={3:2,1:1,6:1}.
# x=1: count[2]=0 < count[1]=1 → False ✓
#
# "No bugs. Sorting by abs value handles negatives correctly – -2 pairs with -4."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log n): sort + linear scan. Space O(n) Counter.
# The abs-value sort is the key insight for handling negatives.
# Follow-up: if 'double' is replaced by 'k times' – same greedy, check count[k*x].
# Follow-up: count the number of valid pairings – DP or combinatorics on the counts."

```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def canReorderDoubled(self, arr: list[int]) -> bool:
        count = collections.Counter(arr)

        for key in sorted(count, key=abs):
            if count[key] > count[2 * key]:
                return False
            count[2 * key] -= count[key]

        return True
```

• LeetDoocs

```
class Solution:
    def canReorderDoubled(self, arr: List[int]) -> bool:
        freq = Counter(arr)
        if freq[0] & 1:
            return False
        for x in sorted(freq, key=abs):
            if freq[x << 1] < freq[x]:
                return False
            freq[x << 1] -= freq[x]
        return True
```

• SimplyLeet

```
from collections import Counter

def canReorderDoubled(arr):
    # Count frequency for each element in arr
    count = Counter(arr)
    # Sort the array based on the absolute value of elements
    for x in sorted(arr, key=abs):
        # If no x is left to be paired, skip to the next element
        if count[x] == 0:
            continue
```

```

    # Check if double the current number exists in sufficient frequency
    if count[2 * x] < count[x]:
        return False
    # Decrement the frequency for the double of x
    count[2 * x] -= count[x]
    return True

```

Example usage:

```
print(canReorderDoubled([4,-2,2,-4])) # Expected Output: True
```

• LC.ca

```

class Solution:
    def canReorderDoubled(self, arr: List[int]) -> bool:
        freq = Counter(arr)
        if freq[0] & 1:
            return False
        for x in sorted(freq, key=abs):
            if freq[x <= 1] < freq[x]:
                return False
            freq[x <= 1] -= freq[x]
        return True

```

◆ Quick Review

Key Insights

- Sort the array based on the absolute value of the elements to handle negative numbers correctly.
- Use a hash table (or frequency counter) to track the occurrence of each element.
- For each number in the sorted list, if the number is still available in the frequency map, try to pair it with its double.

- Decrement the appropriate counts and validate that it is possible to find sufficient doubles for all numbers.
- Special case: when dealing with zeros, pairing works within the same value.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting.

Space Complexity: $O(n)$ for storing the frequency map.

Solution

The solution starts by counting the frequency of each element in the array. Sorting the array by absolute value ensures that when we process an element, any potential pair (i.e., its double) comes later in the sequence. For each number, if there are available occurrences, we try to match it with its double. If there are not enough doubles available, the answer is false. If all elements can be paired appropriately by decrementing their counts in the frequency map, the function returns true.

Greedy

□ #2131 Longest Palindrome by Concatenating Two Letter Words

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Greedy · Counting
Lists: CodeSignal

Problem

You are given an array of strings `words`. Each element of `words` consists of two lowercase English letters.

Create the longest possible palindrome by selecting some elements from `words` and concatenating them in any order. Each element can be selected at most once.

Return the length of the longest palindrome that you can create. If it is impossible to create any palindrome, return 0.

A palindrome is a string that reads the same forward and backward.

Example 1:

Input: words = ["lc","cl","gg"]

Output: 6

Explanation: One longest palindrome is "lc" + "gg" + "cl" = "lcggcl", of length 6.

Note that "clggcl" is another longest palindrome that can be created.

Example 2:

Input: words = ["ab","ty","yt","lc","cl","ab"]

Output: 8

Explanation: One longest palindrome is "ty" + "lc" + "cl" + "yt" = "tylcclyt", of length 8.

Note that "lcyttycl" is another longest palindrome that can be created.

Example 3:

Input: words = ["cc","ll","xx"]

Output: 2

Explanation: One longest palindrome is "cc", of length 2.

Note that "ll" is another longest palindrome that can be created, and so is "xx".

Constraints:

- $1 \leq \text{words.length} \leq 105$
- $\text{words}[i].\text{length} == 2$
- $\text{words}[i]$ consists of lowercase English letters.

My LeetCode Solution

- My LeetCode Solution


```

class Solution:
    def longestPalindrome(self, words: List[str]) -> int:
        counts = Counter(words)
        ans = 0
        x = 0
        for k,v in counts.items():
            if k[0] == k[1]:
                x += v&1
                ans += (v//2) * 4

            else:
                ans += min(v, counts[k[::-1]]) * 2

        if x:
            return ans+2

        return ans

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Let me make sure I understand the problem correctly.
# Two-letter words; select some subset and concatenate into the longest palindrome.
# Each word used at most once. Letters are case-sensitive. Right?"
#
# "A few quick questions:
# Can the same word appear multiple times in the input? Yes.
# 'lc' and 'cl' can be placed symmetrically? Yes: 'lc...cl'."
#
# "Let me walk: words=['lc','cl','gg'] → 'lc'+ 'gg'+ 'cl' = 'lcggcl' → len=6 ✓"

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with brute force to lock in the logic.
# Try all subsets and orderings; check if the concatenation is a palindrome.
#  $O(2^n * n!)$  – completely infeasible.
# Should I go ahead and optimize?"

```

```

class _2131_LongestPalindromeTwoLetterWords_Brute:
    def longestPalindrome(self, words):
        from itertools import permutations, chain
        best = 0
        for r in range(len(words) + 1):
            for subset in permutations(words, r):
                s = ''.join(subset)
                if s == s[::-1]: best = max(best, len(s))

```

```
    return best
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is enumeration.
```

```
# Greedy with a Counter:
```

```
# Non-palindromic words (ab≠ba): pair 'ab' with 'ba' – each matched pair contributes 4 chars.
```

```
# Palindromic words (gg, aa): each pair-of-two contributes 4 chars; one leftover can be the center (+2).
```

```
# Walk through pairs of words and their reverses; track how many centers are available.
```

```
#
```

```
# Time: O(n). Space: O(n) Counter."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _2131_LongestPalindromeTwoLetterWords:
```

```
    def longestPalindrome(self, words):
```

```
        from collections import Counter
```

```
        count = Counter(words)
```

```
        length = 0
```

```
        center_used = False
```

```
        for word, freq in count.items():
```

```
            rev = word[::-1]
```

```
            if word == rev:
```

```
                pairs = freq // 2
```

```
                length += pairs * 4
```

```
                if freq % 2 == 1 and not center_used:
```

```
                    length += 2
```

```
                    center_used = True
```

```
            elif rev in count and word < rev:
```

```
                matched = min(freq, count[rev])
```

```
                length += matched * 4
```

```
        return length
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
```

```
#
```

```
# words=['lc','cl','gg']:
```

```
# count={lc:1,cl:1,gg:1}.
```

```
# 'gg'=rev: pairs=0, freq%2=1, center_used=False → length+=2, center_used=True.
```

```
# 'lc' < 'cl'? No. 'cl' < 'lc'? Yes. min(count[cl],count[lc])=1 → length+=4.
```

```
# total=2+4=6 ✓
```

```
#
```

```
# words=['ab','ty','yt','lc','cl','ab']: count={ab:2,ty:1,yt:1,lc:1,cl:1}.
```

```
# 'ty'<'yt': min(1,1)=1 → +4. 'lc'<'cl': +4. 'ab'=rev? No (ab≠ba). 'ba' not in count.
```

```
# total=8 ✓
```

```
#
```

```
# "No bugs. Processing word < rev avoids double-counting non-palindromic pairs."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: one pass over Counter. Space $O(n)$ Counter.

At most one palindromic center word contributes 2 chars – track with center_used flag.

Follow-up: Longest Palindrome (#409) – same structure with single characters.

Follow-up: if we need the actual palindrome string, construct it from paired words."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def longestPalindrome(self, words: list[str]) -> int:
        ans = 0
        count = [[0] * 26 for _ in range(26)]

        for a, b in words:
            i = ord(a) - ord('a')
            j = ord(b) - ord('a')
            if count[j][i]:
                ans += 4

            count[j][i] -= 1
            else:
                count[i][j] += 1

        for i in range(26):
            if count[i][i]:
                return ans + 2

        return ans
```

• LeetDoocs

```

class Solution:
    def longestPalindrome(self, words: List[str]) -> int:
        cnt = Counter(words)
        ans = x = 0
        for k, v in cnt.items():
            if k[0] == k[1]:
                x += v & 1
                ans += v // 2 * 2 * 2
            else:
                ans += min(v, cnt[k[::-1]]) * 2

        ans += 2 if x else 0
        return ans

```

• SimplyLeet

```

# Python solution
from collections import Counter

class Solution:
    def longestPalindrome(self, words):
        # Count the frequency of each word
        word_count = Counter(words)
        length = 0      # Total words used count (each adds 2 letters)
        center_used = False

        for word, count in list(word_count.items()):
            reverse = word[::-1] # Get the reverse of the current word

            if word == reverse:
                # For words that are palindromic themselves (e.g. "gg")
                pairs = count // 2
                length += pairs * 2 # Each pair adds 2 words (4 letters)
                # If an extra word exists and center is not used, add one word
                in the center
                if count % 2 == 1 and not center_used:
                    length += 1

```

```

        center_used = True
    else:
        # For words that are not self-palindromic.
        if reverse in word_count:
            # We only count each pair once, so use minimum and then set
            the reverse count to 0.
            pair_count = min(count, word_count[reverse])
            length += pair_count * 2    # each pair contributes two
words
            # Set reverse count to 0 to avoid double counting later.
            word_count[reverse] = 0
        return length * 2    # Each word is of length 2

# Example usage:
# sol = Solution()
# print(sol.longestPalindrome(["lc", "cl", "gg"])) # Output: 6

```

• LC.ca

```

class Solution:
    def longestPalindrome(self, words: List[str]) -> int:
        cnt = Counter(words)
        ans = x = 0
        for k, v in cnt.items():
            if k[0] == k[1]:
                x += v & 1
                ans += v // 2 * 2 * 2
            else:
                ans += min(v, cnt[k[::-1]]) * 2

        ans += 2 if x else 0
        return ans

```

◆ Quick Review

Key Insights

- Pair words with their reverse (e.g., "ab" with "ba") to form a palindrome.

- Handle words that are palindromic by themselves (like "gg" or "aa") differently; they can be used in pairs and one extra can be placed in the center.
- Use a hash map (or dictionary) to count frequencies of each word.
- For non-palindromic word pairs, consider each pair only once to avoid double counting.
- For self-palindromic words, count how many pairs can be used and whether an extra word can be placed in the middle to maximize length.

Space & Time

Time Complexity: $O(n)$, where n is the number of words, since we iterate through the list and perform constant time operations per word.

Space Complexity: $O(n)$, to store counts of words in the hash map.

Solution

The approach is to count the occurrences of each word using a hash map. For each word: If the word is not self-palindromic, check if its reverse exists and form pairs by considering the minimum count between the word and its

reverse. If the word is self-palindromic, form as many pairs as possible (using $\text{count} // 2$) and note if there is any word left; you can use one extra self-palindromic word as the center of the palindrome. Finally, compute the total length by multiplying the number of words used by 2, since each word contributes 2 characters.

Round 6 · Mid · Hard

Round 6

Mid Priority · Hard

28 questions · 5 patterns

Linked List #25 #432 #460

Stack #32 #42 #84 #224 #239 #716 #772 #895

Heap #23 #218 #272 #295 #358 #499 #632
#759 #1425

Trie #212 #336 #588 #642

Backtracking #37 #51 #126 #996

Linked List

Round 6 · Mid · Hard · 3 questions

Linked List

□ #25 Reverse Nodes in k-Group

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Recursion
Lists: Grind 169

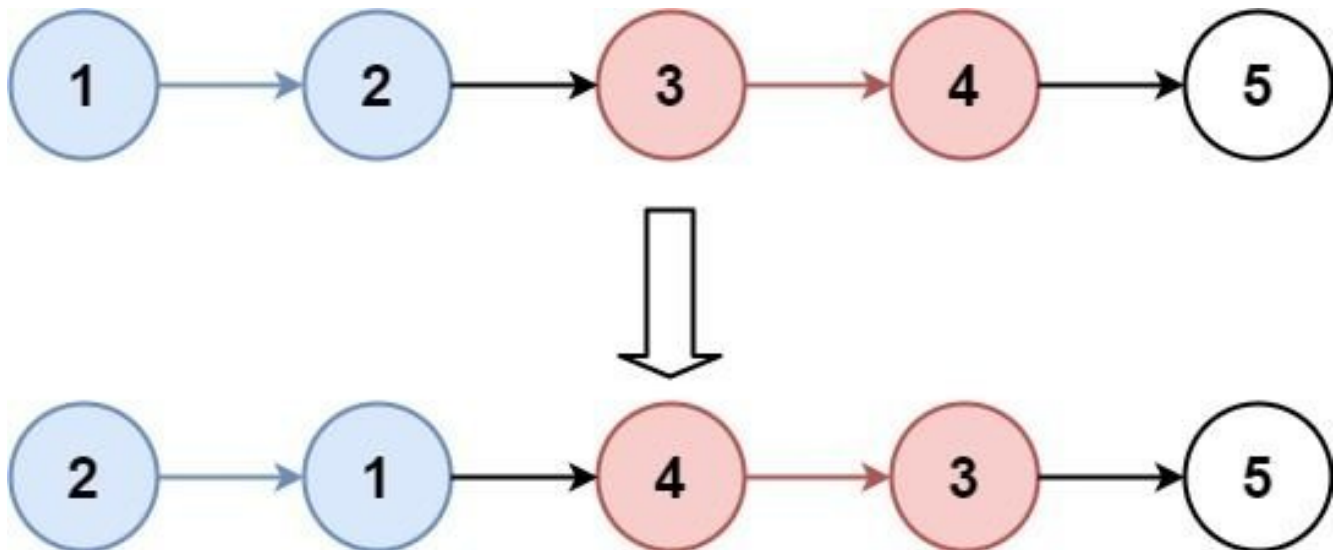
Problem

Given the head of a linked list, reverse the nodes of the list k at a time, and return the modified list.

k is a positive integer and is less than or equal to the length of the linked list. If the number of nodes is not a multiple of k then left-out nodes, in the end, should remain as it is.

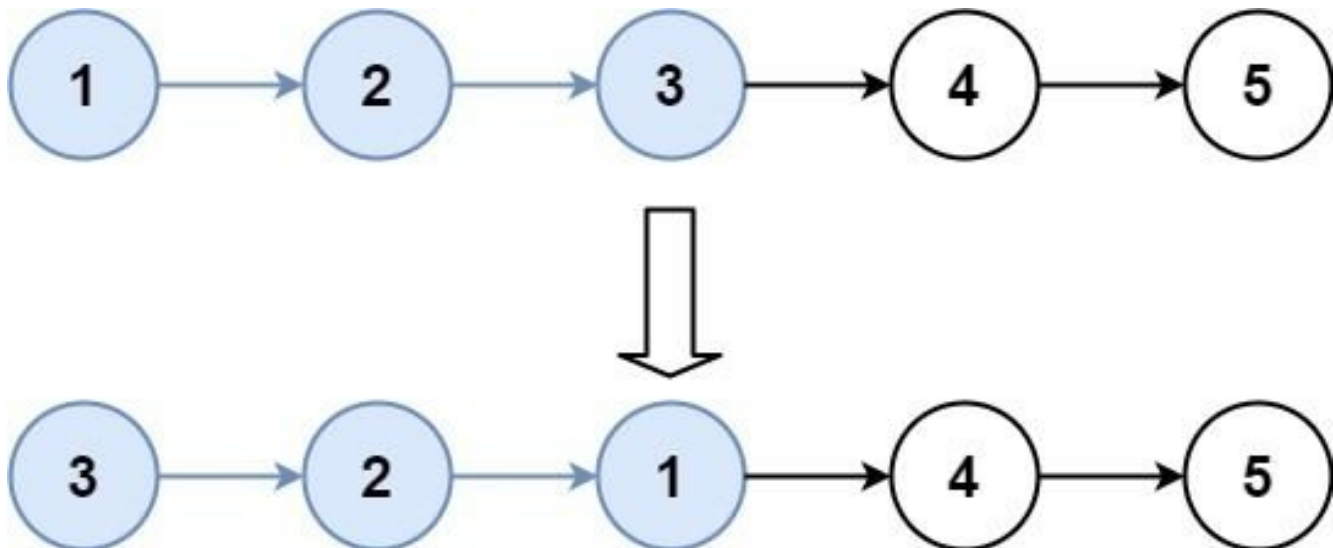
You may not alter the values in the list's nodes, only nodes themselves may be changed.

Example 1:



Input: head = [1,2,3,4,5], k = 2
Output: [2,1,4,3,5]

Example 2:



Input: head = [1,2,3,4,5], k = 3
Output: [3,2,1,4,5]

Constraints:

- The number of nodes in the list is n .
- $1 \leq k \leq n \leq 5000$
- $0 \leq \text{Node.val} \leq 1000$

Follow-up: Can you solve the problem in $O(1)$ extra memory space?

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Reverse every k consecutive nodes in the linked list.
# If remaining nodes < k, leave them as-is. Return the new head.
# 0(1) extra space – no copying values. Right?"
#
# "A few quick questions:
# k can equal 1 (no-op)? Yes. k can equal list length? Yes – reverse everything."
#
# "Let me walk: [1,2,3,4,5], k=2 → [2,1,4,3,5] ✓. k=3 → [3,2,1,4,5] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Collect all node values, reverse each group of k elements in the array, rebuild
the list.
# 0(n) time but 0(n) space for the values array.
# Should I go ahead and optimize?"
class _25_ReverseKGroup_Brute:
    def reverseKGroup(self, head, k):
        vals = []
        cur = head
        while cur: vals.append(cur.val); cur = cur.next
        for i in range(0, len(vals) - len(vals) % k, k):
            vals[i:i+k] = vals[i:i+k][::-1]
        cur = head
        for v in vals: cur.val = v; cur = cur.next
        return head
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the 0(n) auxiliary value list.
# In-place pointer reversal group by group – 0(1) extra space:
# First check if k nodes remain; if not, stop.
# Reverse k nodes using the 3-pointer technique, then link to the next group.
#
# Time: 0(n). Space: 0(1)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```

class _25_ReverseKGroup:
    def reverseKGroup(self, head, k):
        def has_k(node, k):
            for _ in range(k):
                if not node: return False
                node = node.next
            return True

        def reverse_k(head, k):
            prev, cur = None, head
            for _ in range(k):
                nxt = cur.next; cur.next = prev; prev, cur = cur, nxt
            return prev

        dummy = ListNode(0, head)
        group_prev = dummy
        while has_k(group_prev.next, k):
            group_start = group_prev.next
            group_end = group_start
            for _ in range(k - 1): group_end = group_end.next
            next_group = group_end.next
            group_end.next = None
            group_prev.next = reverse_k(group_start, k)
            group_start.next = next_group
            group_prev = group_start
        return dummy.next

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

[1,2,3,4,5], k=2:

Group1: start=1,end=2. Reverse [1,2]→[2,1]. dummy→2→1. group_prev=1.
next_group=3.

Group2: start=3,end=4. Reverse [3,4]→[4,3]. 1→4→3. group_prev=3. next_group=5.

has_k(5,2): only 1 node → False → stop. 3.next=5. → [2,1,4,3,5] ✓

#

"No bugs. group_start.next=next_group reconnects the reversed group to the rest of the list."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: each node processed in its reversal group exactly once. Space $O(1)$.

k=2 is exactly Swap Nodes in Pairs (#24).

Follow-up: k=n reverses the entire list – identical to Reverse Linked List (#206).

Follow-up: reverse every other group of k – add a skip_k step between reverse steps."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def reverseKGroup(self, head: ListNode | None, k: int) -> ListNode | None:
        if not head:
            return None

        tail = head

        for _ in range(k):
            # There are less than k nodes in the list, do nothing.
            if not tail:
                return head
            tail = tail.next

        newHead = self._reverse(head, tail)
        head.next = self.reverseKGroup(tail, k)
        return newHead

    def _reverse(
        self,
        head: ListNode | None,
        tail: ListNode | None,
    ) -> ListNode | None:
        """Reverses [head, tail)."""
        prev = None
        curr = head
        while curr != tail:
            next = curr.next
            curr.next = prev
            prev = curr
            curr = next

        return prev
```

```

class Solution:
    def reverseKGroup(self, head: ListNode, k: int) -> ListNode:
        if not head or k == 1:
            return head

        def getLength(head: ListNode) -> int:
            length = 0
            while head:
                length += 1
                head = head.next

            return length

        length = getLength(head)
        dummy = ListNode(0, head)
        prev = dummy
        curr = head

        for _ in range(length // k):
            for _ in range(k - 1):
                next = curr.next

                curr.next = next.next
                next.next = prev.next
                prev.next = next
                prev = curr
                curr = curr.next

        return dummy.next

```

• LeetDoocs

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseKGroup(self, head: Optional[ListNode], k: int) -> Optional[ListNode]:
        def reverse(head: Optional[ListNode]) -> Optional[ListNode]:
            dummy = ListNode()
            cur = head

```



```

        while cur:
            nxt = cur.next
            cur.next = dummy.next
            dummy.next = cur
            cur = nxt
        return dummy.next

    dummy = pre = ListNode(next=head)
    while pre:
        cur = pre

```

```

        for _ in range(k):
            cur = cur.next
            if cur is None:
                return dummy.next
        node = pre.next
        nxt = cur.next
        cur.next = None
        pre.next = reverse(node)
        node.next = nxt
        pre = node

```

```

    return dummy.next

```

• SimplyLeet

```

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def reverseKGroup(self, head: ListNode, k: int) -> ListNode:
        # Create a dummy node to simplify edge cases
        dummy = ListNode(0)

```

```
dummy.next = head
group_prev = dummy
```

```
while True:
    # Find the kth node from group_prev
    kth = self.getKthNode(group_prev, k)
    if not kth:
        break
    group_next = kth.next
```

```
# Reverse the group starting at group_prev.next up to kth
prev = kth.next
curr = group_prev.next
while curr != group_next:
    temp = curr.next    # Store next node
    curr.next = prev    # Reverse pointer
    prev = curr         # Move prev forward
    curr = temp         # Move curr forward
```

```
# Reconnect the reversed group with the previous part
```

```
temp = group_prev.next    # temp is the start of the original group,
# becomes the new tail
group_prev.next = kth     # Connect previous part to the kth node
# (new head of reversed group)
group_prev = temp        # Move group_prev to the end of the
# reversed group
```

```
return dummy.next
```

```
# Helper function to get the kth node from the current node
```

```
def getKthNode(self, curr, k):
    while curr and k > 0:
        curr = curr.next
```

```
k -= 1
return curr
```

• LC.ca

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def reverseKGroup(self, head: ListNode, k: int) -> ListNode:
        """
        for reverse 1->2->3->4->5, process is like
        1->None, 2->3->4->5

        2->1->None, 3->4->5
        3->2->1->None, 4->5
        4->3->2->1->None, 5
        5->4->3->2->1->None, None
        """
        def reverseList(head):
            pre, p = None, head
            while p:
                pnext = p.next
                p.next = pre
                pre = p
                p = pnext
            return pre

        dummy = ListNode(next=head)
        pre = cur = dummy
        while cur.next:
            for _ in range(k):
                cur = cur.next
            if cur is None:

```

```

        return dummy.next
    t = cur.next
    cur.next = None # cut from next k-group, so to reverseList() for
current k-group
    start = pre.next
    pre.next = reverseList(start)
    start.next = t # so now 'start' is the last node of k-group
    pre = cur = start # same as the reset dummy before while loop
    return dummy.next

```

```
#####
```

```

# Definition for singly-linked list.
# class ListNode(object):
# def __init__(self, x):
# self.val = x
# self.next = None

```

```

class Solution(object):
    def reverseKGroup(self, head, k):
        """

```

```

        :type head: ListNode
        :type k: int
        :rtype: ListNode
        """

```

```

    def reverseList(head, k):
        pre = None
        cur = head
        while cur and k > 0:
            tmp = cur.next

```

```
        cur.next = pre
        pre = cur
        cur = tmp
        k -= 1
    head.next = cur
    return cur, pre

length = 0
p = head
while p:

    length += 1
    p = p.next
    if length < k:
        return head
    step = length / k
    ret = None
    pre = None
    p = head
    while p and step:
        next, newHead = reverseList(p, k)

        if ret is None:
            ret = newHead
        if pre:
            pre.next = newHead
        pre = p
        p = next
        step -= 1
    return ret
```

◆ Quick Review

Key Insights

- Use a dummy node to simplify edge cases.
- Identify groups of k nodes using a helper function.

- Reverse the nodes in each group using pointer manipulation.
- If a remaining group has fewer than k nodes, leave it unchanged.
- The reversal is done in-place with $O(1)$ extra space.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes (each node is processed a constant number of times)

Space Complexity: $O(1)$ (only constant extra space is used)

Solution

The solution involves iterating through the linked list by identifying groups of k consecutive nodes. For each group: Find the k th node from the current starting point. If it does not exist, the remainder of the list is not reversed. Reverse the nodes in the identified group in-place by adjusting the next pointers. Reconnect the reversed group with the previous part of the list and update pointers to move to the next group. The approach leverages a dummy node for easier handling

of head modifications and uses a helper function to fetch the kth node. The in-place reversal ensures that the extra memory usage is constant.

Linked List

□ #432 All O'one Data Structure

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Design · Doubly-Linked List ·
Linked List

Lists: Denny Zhang

Problem

Design a data structure to store the strings' count with the ability to return the strings with minimum and maximum counts.

Implement the AllOne class:

- AllOne() Initializes the object of the data structure.
- inc(String key) Increments the count of the string key by 1. If key does not exist in the data structure, insert it with count 1.
- dec(String key) Decrements the count of the string key by 1. If the count of key is 0 after the decrement, remove it from the data structure. It is guaranteed that key exists in the data structure before the decrement.

- **getMaxKey()** Returns one of the keys with the maximal count. If no element exists, return an empty string "".
- **getMinKey()** Returns one of the keys with the minimum count. If no element exists, return an empty string "".

Note that each function must run in $O(1)$ average time complexity.

Example 1:

```
Input
["AllOne", "inc", "inc", "getMaxKey", "getMinKey", "inc", "getMaxKey",
"getMinKey"]
[[], ["hello"], ["hello"], [], [], ["leet"], [], []]
Output
[null, null, null, "hello", "hello", null, "hello", "leet"]

Explanation
AllOne allOne = new AllOne();
```

Constraints:

- $1 \leq \text{key.length} \leq 10$
- key consists of lowercase English letters.
- It is guaranteed that for each call to dec, key is existing in the data structure.
- At most $5 * 10^4$ calls will be made to inc, dec, getMaxKey, and getMinKey.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Design a data structure with inc(key), dec(key), getMaxKey(), getMinKey() – all
# O(1).
# inc adds key with count 1 if absent; dec removes if count drops to 0. Right?"
#
# "A few quick questions:
# getMaxKey/getMinKey return any key with max/min count? Yes – any valid key.
# What if the data structure is empty? Return an empty string."
#
# "Let me walk: inc('a'),inc('b'),inc('b'),inc('c'),inc('c'),inc('c').
# getMaxKey()→'c'(3). getMinKey()→'a'(1) ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Dict {key: count}. inc/dec O(1). getMax/getMin scan all entries O(n).
# Should I go ahead and optimize?"
class _432_All0oneDataStructure_Brute:
    def __init__(self): self.counts = {}
    def inc(self, key):
        self.counts[key] = self.counts.get(key, 0) + 1
    def dec(self, key):
        self.counts[key] -= 1
        if self.counts[key] == 0: del self.counts[key]
    def getMaxKey(self): return max(self.counts, key=self.counts.get, default='')
    def getMinKey(self): return min(self.counts, key=self.counts.get, default='')
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the O(n) scan for getMax/getMin.
# Doubly linked list of buckets (each bucket holds a count and a set of keys at that
# count)
# + a hashmap {key → bucket_node}.
# inc: move key from bucket c to c+1. dec: move key to c-1.
# getMax: any key from the tail bucket. getMin: any key from the head bucket.
# All O(1) amortized. Space: O(n)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _432_All0oneDataStructure:
    class _Bucket:
        def __init__(self, count):
            self.count = count; self.keys = set()
            self.prev = self.next = None
    def __init__(self):
        self._key_bucket = {}
        self._head = self._Bucket(0); self._tail = self._Bucket(float('inf'))
        self._head.next = self._tail; self._tail.prev = self._head
    def _insert_after(self, node, count):
```

```

        b = self._Bucket(count)
        b.prev = node; b.next = node.next
        node.next.prev = b; node.next = b
        return b
    def _remove_bucket(self, b):
        b.prev.next = b.next; b.next.prev = b.prev
    def inc(self, key):
        if key not in self._key_bucket:
            if self._head.next.count != 1:
                self._insert_after(self._head, 1)
            self._head.next.keys.add(key)
            self._key_bucket[key] = self._head.next
        else:
            cur = self._key_bucket[key]
            nxt_count = cur.count + 1
            if cur.next.count != nxt_count:
                self._insert_after(cur, nxt_count)
            cur.next.keys.add(key)
            self._key_bucket[key] = cur.next
            cur.keys.discard(key)
            if not cur.keys: self._remove_bucket(cur)
    def dec(self, key):
        cur = self._key_bucket[key]
        prev_count = cur.count - 1
        if prev_count == 0:
            del self._key_bucket[key]
        else:
            if cur.prev.count != prev_count:
                self._insert_after(cur.prev, prev_count)
            cur.prev.keys.add(key)
            self._key_bucket[key] = cur.prev
            cur.keys.discard(key)
            if not cur.keys: self._remove_bucket(cur)
    def getMaxKey(self):
        return next(iter(self._tail.prev.keys), '') if self._tail.prev !=
self._head else ''
    def getMinKey(self):
        return next(iter(self._head.next.keys), '') if self._head.next !=
self._tail else ''

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

inc('a'): bucket(1)={a}. head→1→tail. key_bucket={a:b1}.

inc('b'): bucket(1)={a,b}. inc('b'): bucket(2)={b}, bucket(1)={a}. head→1→2→tail.

getMaxKey()→tail.prev=b2→next(iter({b}))='b' ✓. getMinKey()→head.next=b1→'a' ✓

#

"No bugs. Sentinel head/tail avoid boundary checks in
_insert_after/_remove_bucket."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"All ops $O(1)$: set operations + DLL pointer updates. Space $O(n)$."

Follow-up: if we also need getMedianKey, we'd need a balanced BST instead of a DLL.

Follow-up: LFU Cache (#460) uses the same bucket-DLL structure."

Community Solutions (Python)

• WalkCC

```
from dataclasses import dataclass

@dataclass
class Node:
    def __init__(self, count: int, key: str | None = None):
        self.count = count
        self.keys: set[str] = {key} if key else set()
        self.prev: Node | None = None
        self.next: Node | None = None

    def __eq__(self, other) -> bool:
        if not isinstance(other, Node):
            return NotImplemented
        return self.count == other.count and self.keys == other.keys

class AllOne:
    def __init__(self):
        self.keyToNode: dict[str, Node] = {}

    def head = Node(0)
    self.tail = Node(0)
    self.head.next = self.tail
    self.tail.prev = self.head

    def inc(self, key: str) -> None:
        if key in self.keyToNode:
            self._incrementExistingKey(key)
        else:
            self._addNewKey(key)
```

```

def dec(self, key: str) -> None:
    # It is guaranteed that key exists in the data structure before the
    # decrement.
    self._decrementExistingKey(key)

def getMaxKey(self) -> str:
    return '' if self.tail.prev == self.head \
        else next(iter(self.tail.prev.keys))

def getMinKey(self) -> str:
    return '' if self.head.next == self.tail \
        else next(iter(self.head.next.keys))

def _addNewKey(self, key: str) -> None:
    """Adds a new node with frequency 1."""
    if self.head.next.count == 1:
        self.head.next.keys.add(key)
    else:
        self._insertAfter(self.head, Node(1, key))

    self.keyToNode[key] = self.head.next

def _incrementExistingKey(self, key: str) -> None:
    """Increments the frequency of the key by 1."""
    node = self.keyToNode[key]
    node.keys.remove(key)
    if node.next == self.tail or node.next.count > node.count + 1:
        self._insertAfter(node, Node(node.count + 1))
    node.next.keys.add(key)
    self.keyToNode[key] = node.next

    if not node.keys:
        self._remove(node)

def _decrementExistingKey(self, key: str) -> None:
    """Decrements the count of the key by 1."""
    node = self.keyToNode[key]
    node.keys.remove(key)
    if node.count > 1:
        if node.prev == self.head or node.prev.count != node.count - 1:
            self._insertAfter(node.prev, Node(node.count - 1))

```

```

        node.prev.keys.add(key)
        self.keyToNode[key] = node.prev
    else:
        del self.keyToNode[key]
    if not node.keys:
        self._remove(node)

def _insertAfter(self, node: Node, newNode: Node) -> None:
    newNode.prev = node
    newNode.next = node.next

    node.next.prev = newNode
    node.next = newNode

def _remove(self, node: Node) -> None:
    node.prev.next = node.next
    node.next.prev = node.prev

```

• LeetDoocs

```

class Node:
    def __init__(self, key='', cnt=0):
        self.prev = None
        self.next = None
        self.cnt = cnt
        self.keys = {key}

    def insert(self, node):
        node.prev = self
        node.next = self.next

        node.prev.next = node
        node.next.prev = node
        return node

    def remove(self):
        self.prev.next = self.next
        self.next.prev = self.prev

class AllOne:

```

```

def __init__(self):
    self.root = Node()
    self.root.next = self.root
    self.root.prev = self.root
    self.nodes = {}

def inc(self, key: str) -> None:
    root, nodes = self.root, self.nodes
    if key not in nodes:
        if root.next == root or root.next.cnt > 1:

            nodes[key] = root.insert(Node(key, 1))
        else:
            root.next.keys.add(key)
            nodes[key] = root.next
    else:
        curr = nodes[key]
        next = curr.next
        if next == root or next.cnt > curr.cnt + 1:
            nodes[key] = curr.insert(Node(key, curr.cnt + 1))
        else:

            next.keys.add(key)
            nodes[key] = next
            curr.keys.discard(key)
            if not curr.keys:
                curr.remove()

def dec(self, key: str) -> None:
    root, nodes = self.root, self.nodes
    curr = nodes[key]
    if curr.cnt == 1:

        nodes.pop(key)
    else:
        prev = curr.prev
        if prev == root or prev.cnt < curr.cnt - 1:
            nodes[key] = prev.insert(Node(key, curr.cnt - 1))
        else:
            prev.keys.add(key)
            nodes[key] = prev
    curr.keys.discard(key)
    if not curr.keys:

```



```
curr.remove()
```

```
def getMaxKey(self) -> str:
    return next(iter(self.root.prev.keys))
```

```
def getMinKey(self) -> str:
    return next(iter(self.root.next.keys))
```

Your AllOne object will be instantiated and called as such:

```
# obj = AllOne()
# obj.inc(key)
# obj.dec(key)
# param_3 = obj.getMaxKey()
# param_4 = obj.getMinKey()
```

```
class AllOne {
    Node root = new Node();
    Map<String, Node> nodes = new HashMap<>();

    public AllOne() {
        root.next = root;
        root.prev = root;
    }

    public void inc(String key) {
```

```
        if (!nodes.containsKey(key)) {
            if (root.next == root || root.next.cnt > 1) {
                nodes.put(key, root.insert(new Node(key, 1)));
            } else {
                root.next.keys.add(key);
                nodes.put(key, root.next);
            }
        } else {
            Node curr = nodes.get(key);
            Node next = curr.next;
```

```

        if (next == root || next.cnt > curr.cnt + 1) {
            nodes.put(key, curr.insert(new Node(key, curr.cnt + 1)));
        } else {
            next.keys.add(key);
            nodes.put(key, next);
        }
        curr.keys.remove(key);
        if (curr.keys.isEmpty()) {
            curr.remove();
        }
    }
}

public void dec(String key) {
    Node curr = nodes.get(key);
    if (curr.cnt == 1) {
        nodes.remove(key);
    } else {
        Node prev = curr.prev;
        if (prev == root || prev.cnt < curr.cnt - 1) {
            nodes.put(key, prev.insert(new Node(key, curr.cnt - 1)));
        } else {
            prev.keys.add(key);
            nodes.put(key, prev);
        }
    }

    curr.keys.remove(key);
    if (curr.keys.isEmpty()) {
        curr.remove();
    }
}

public String getMaxKey() {
    return root.prev.keys.iterator().next();
}

public String getMinKey() {
    return root.next.keys.iterator().next();
}

```

```

}

class Node {
    Node prev;
    Node next;
    int cnt;
    Set<String> keys = new HashSet<>();

    public Node() {
        this("", 0);
    }

    public Node(String key, int cnt) {
        this.cnt = cnt;
        keys.add(key);
    }

    public Node insert(Node node) {
        node.prev = this;
        node.next = this.next;

        node.prev.next = node;
        node.next.prev = node;
        return node;
    }

    public void remove() {
        this.prev.next = this.next;
        this.next.prev = this.prev;
    }
}

/**
 * Your AllOne object will be instantiated and called as such:
 * AllOne obj = new AllOne();
 * obj.inc(key);
 * obj.dec(key);
 * String param_3 = obj.getMaxKey();
 * String param_4 = obj.getMinKey();
 */

```

• SimplyLeet

```
# Node class to represent a frequency bucket in the doubly linked list
class Node:
    def __init__(self, count):
        self.count = count           # frequency count for keys in this
node                                # set of keys with this frequency
        self.keys = set()           # previous node in the linked list
        self.prev = None            # next node in the linked list
        self.next = None

class AllOne:
    def __init__(self):

        # Use sentinel nodes for easier handling of edge cases.
        self.head = Node(float('-inf'))
        self.tail = Node(float('inf'))
        self.head.next = self.tail
        self.tail.prev = self.head
        # Map each key to its corresponding node.
        self.keyToNode = {}

    # Helper: insert newNode after prevNode in the linked list.
    def _insertNodeAfter(self, prevNode, newNode):

        newNode.prev = prevNode
        newNode.next = prevNode.next
        prevNode.next.prev = newNode
        prevNode.next = newNode

    # Helper: remove a node if it has no keys.
    def _removeNode(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev
```

```

def inc(self, key: str) -> None:
    # If key is new, treat current node as head.
    if key not in self.keyToNode:
        curr = self.head
    else:
        curr = self.keyToNode[key]
        curr.keys.remove(key)
    # Check if the next node has count = current count + 1.
    targetCount = (0 if curr == self.head else curr.count) + 1
    if curr.next == self.tail or curr.next.count != targetCount:

        # Create new node with targetCount.
        newNode = Node(targetCount)
        self._insertNodeAfter(curr, newNode)
    else:
        newNode = curr.next
        newNode.keys.add(key)
        self.keyToNode[key] = newNode
    # Remove the current node if it is not a sentinel and has no keys.
    if curr != self.head and len(curr.keys) == 0:
        self._removeNode(curr)

def dec(self, key: str) -> None:
    curr = self.keyToNode[key]
    curr.keys.remove(key)
    # If decrementing causes the count to be 0, remove key completely.
    if curr.count == 1:
        del self.keyToNode[key]
    else:
        targetCount = curr.count - 1
        # Check if the previous node has the target count.

        if curr.prev == self.head or curr.prev.count != targetCount:
            newNode = Node(targetCount)
            # Insert newNode before current.
            self._insertNodeAfter(curr.prev, newNode)
        else:
            newNode = curr.prev
            newNode.keys.add(key)
            self.keyToNode[key] = newNode
    # Clean up current node if empty.
    if len(curr.keys) == 0:

```

```

        self._removeNode(curr)

    def getMaxKey(self) -> str:
        # The node before tail has the maximal count.
        return next(iter(self.tail.prev.keys)) if self.tail.prev != self.head
    else ""

    def getMinKey(self) -> str:
        # The node after head has the minimal count.
        return next(iter(self.head.next.keys)) if self.head.next != self.tail
    else ""

# Example usage:
# obj = AllOne()
# obj.inc("hello")
# obj.inc("hello")
# print(obj.getMaxKey()) # "hello"
# print(obj.getMinKey()) # "hello"
# obj.inc("leet")
# print(obj.getMaxKey()) # "hello"
# print(obj.getMinKey()) # "leet"

```

• LC.ca

```

class Node:
    def __init__(self, key='', cnt=0):
        self.prev = None
        self.next = None
        self.cnt = cnt
        self.keys = {key}

    def insert(self, node):
        node.prev = self
        node.next = self.next

```

```

        node.prev.next = node
        node.next.prev = node
        return node

    def remove(self):
        self.prev.next = self.next
        self.next.prev = self.prev

class AllOne:

    def __init__(self):
        self.root = Node()
        self.root.next = self.root
        self.root.prev = self.root
        self.nodes = {}

    def inc(self, key: str) -> None:
        root, nodes = self.root, self.nodes
        if key not in nodes:
            if root.next == root or root.next.cnt > 1:

                nodes[key] = root.insert(Node(key, 1))
            else:
                root.next.keys.add(key)
                nodes[key] = root.next
        else:
            curr = nodes[key]
            next = curr.next
            if next == root or next.cnt > curr.cnt + 1:
                nodes[key] = curr.insert(Node(key, curr.cnt + 1))
            else:
                next.keys.add(key)
                nodes[key] = next
                curr.keys.discard(key)
                if not curr.keys:
                    curr.remove()

    def dec(self, key: str) -> None:
        root, nodes = self.root, self.nodes
        curr = nodes[key]
        if curr.cnt == 1:

```

```

        nodes.pop(key)
    else:
        prev = curr.prev
        if prev == root or prev.cnt < curr.cnt - 1:
            nodes[key] = prev.insert(Node(key, curr.cnt - 1))
        else:
            prev.keys.add(key)
            nodes[key] = prev
    curr.keys.discard(key)
    if not curr.keys:

```

```

    curr.remove()

```

```

def getMaxKey(self) -> str:
    return next(iter(self.root.prev.keys))

```

```

def getMinKey(self) -> str:
    return next(iter(self.root.next.keys))

```

Your AllOne object will be instantiated and called as such:

```

# obj = AllOne()
# obj.inc(key)
# obj.dec(key)
# param_3 = obj.getMaxKey()
# param_4 = obj.getMinKey()

```

◆ Quick Review

Key Insights

- Use a Doubly-Linked List (DLL) where each node holds a unique count value and a set of keys with that count.
- Maintain a Hash Map (keyToNode) to quickly locate the node corresponding to each key.

- Insert new nodes in the DLL when a key's count is incremented or decremented and the adjacent node does not have the target count.
- Remove nodes from the DLL when no keys remain in that count.
- The head of the DLL holds the minimum count and the tail holds the maximum count.

Space & Time

Time Complexity: $O(1)$ average for inc, dec, getMaxKey, and getMinKey.

Space Complexity: $O(n)$ where n is the number of unique keys in the data structure.

Solution

The solution uses a combination of a doubly-linked list and hash maps. Each node in the DLL represents a frequency and maintains a set of keys that have that frequency. A hash map maps keys to their corresponding node. When `inc(key)` or `dec(key)` is called, we update the key's frequency and move the key to the appropriate node in the DLL. If the adjacent node with the required frequency doesn't exist, we create a new node and insert it in the correct position. Removal of

a key from a node and deletion of empty nodes ensures that `getMinKey()` and `getMaxKey()` can be returned directly from the head and tail of the DLL.

Linked List

□ #460 LFU Cache

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Linked List · Design ·
Doubly-Linked List
Lists: Denny Zhang

Problem

Design and implement a data structure for a Least Frequently Used (LFU) cache.

Implement the LFUCache class:

- **LFUCache(int capacity)** Initializes the object with the capacity of the data structure.
- **int get(int key)** Gets the value of the key if the key exists in the cache. Otherwise, returns -1.
- **void put(int key, int value)** Update the value of the key if present, or inserts the key if not already present. When the cache reaches its capacity, it should invalidate and remove the least frequently used key before inserting a new item. For this problem, when there is a tie

(i.e., two or more keys with the same frequency), the least recently used key would be invalidated.

To determine the least frequently used key, a use counter is maintained for each key in the cache. The key with the smallest use counter is the least frequently used key.

When a key is first inserted into the cache, its use counter is set to 1 (due to the put operation). The use counter for a key in the cache is incremented either a get or put operation is called on it.

The functions get and put must each run in $O(1)$ average time complexity.

Example 1:

Input

```
["LFUCache", "put", "put", "get", "put", "get", "get", "put", "get", "get", "get"]
```

```
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [3], [4, 4], [1], [3], [4]]
```

Output

```
[null, null, null, 1, null, -1, 3, null, -1, 3, 4]
```

Explanation

```
// cnt(x) = the use counter for key x
```

Constraints:

- $1 \leq \text{capacity} \leq 104$
- $0 \leq \text{key} \leq 105$
- $0 \leq \text{value} \leq 109$

- At most $2 * 10^5$ calls will be made to get and put.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

LFU Cache: get(key)→val or -1, put(key,val). Evict the least frequently used.

On ties, evict the least recently used among the LFU items. All $O(1)$. Right?"

#

"A few quick questions:

Does get increment frequency? Yes. Put on existing key – update value and increment freq?"

#

"Let me walk: LFUCache(2): put(1,1),put(2,2),get(1)→1 (freq[1]=2).

put(3,3) evicts key 2 (freq=1, LRU among min-freq). get(2)→-1 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Dict {key:(val,freq,last_used)}. On evict, scan all entries for min freq then min last_used.

$O(n)$ per put when eviction needed.

Should I go ahead and optimize?"

class _460_LFUCache_Brute:

def __init__(self, capacity):

self.cap = capacity; self.cache = {}; self.time = 0

def _use(self, key):

self.time += 1

v, f, _ = self.cache[key]; self.cache[key] = (v, f+1, self.time)

def get(self, key):

if key not in self.cache: return -1

self._use(key); return self.cache[key][0]

def put(self, key, value):

if self.cap == 0: return

if key in self.cache:

v, f, t = self.cache[key]; self.cache[key] = (value, f+1, self.time+1);

self.time += 1; return

if len(self.cache) == self.cap:

lfu = min(self.cache, key=lambda k: (self.cache[k][1],

self.cache[k][2]))

del self.cache[lfu]

self.time += 1; self.cache[key] = (value, 1, self.time)

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n)$ scan for the LFU key.

Three structures for $O(1)$:

1. key_map: {key → (val, freq)}

2. freq_map: {freq → OrderedDict(key→None)} – LRU order within same freq

3. min_freq: current minimum frequency

#

get: update freq, move key to freq+1 bucket. put: evict from freq_map[min_freq].

All ops $O(1)$. Space $O(\text{capacity})$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

from collections import defaultdict, OrderedDict

class _460_LFUCache:

def __init__(self, capacity):

self.cap = capacity

self.key_map = {}

self.freq_map = defaultdict(OrderedDict)

self.min_freq = 0

def _update(self, key):

val, freq = self.key_map[key]

del self.freq_map[freq][key]

if not self.freq_map[freq] and freq == self.min_freq:

self.min_freq += 1

self.key_map[key] = (val, freq + 1)

self.freq_map[freq + 1][key] = None

def get(self, key):

if key not in self.key_map: return -1

self._update(key)

return self.key_map[key][0]

def put(self, key, value):

if self.cap == 0: return

if key in self.key_map:

self._update(key)

val, freq = self.key_map[key]

self.key_map[key] = (value, freq)

else:

if len(self.key_map) == self.cap:

evict_key, _ = self.freq_map[self.min_freq].popitem(last=False)

del self.key_map[evict_key]

self.key_map[key] = (value, 1)

self.freq_map[1][key] = None

self.min_freq = 1

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

LFUCache(2): put(1,1): key_map={1:(1,1)}, freq_map={1:{1}}, min=1.

put(2,2): key_map={1:(1,1), 2:(2,1)}, freq_map={1:{1,2}}, min=1.

get(1): _update(1)→freq_map={1:{2}, 2:{1}}, min=1. return 1 ✓.

```
# put(3,3): evict freq_map[1]→evict key2. key_map={1:(1,2),3:(3,1)},min=1.
get(2)→-1 ✓.
#
# "No bugs. OrderedDict preserves insertion order so popitem(last=False) evicts the LRU."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "All ops O(1). Space O(capacity).
# min_freq only ever decreases on put (new key starts at freq=1) – always correct.
# Follow-up: LRU Cache (#146) – simpler, only recency matters.
# Follow-up: add TTL – track expiry timestamps and remove in background or on access."
```

Community Solutions (Python)

• WalkCC

```
class Node:
    def __init__(self, key: int, value: int, freq: int, it):
        self.key = key
        self.value = value
        self.freq = freq
        self.it = it

class LFUCache:
    def __init__(self, capacity: int):

        self.capacity = capacity
        self.minFreq = 0
        self.keyToNode = {}
        self.freqToList = {}

    def get(self, key: int) -> int:
        if key not in self.keyToNode:
            return -1

        node = self.keyToNode[key]
```



```

        self._touch(node)
        return node.value

def put(self, key: int, value: int) -> None:
    if self.capacity == 0:
        return

    if key in self.keyToNode:
        node = self.keyToNode[key]
        node.value = value

    self._touch(node)
    return

    if len(self.keyToNode) == self.capacity:
        # Evict an LRU key from `minFreq` list
        keyToEvict = self.freqToList[self.minFreq][-1]
        self.freqToList[self.minFreq].pop()
        del self.keyToNode[keyToEvict]

    self.minFreq = 1

    if 1 not in self.freqToList:
        self.freqToList[1] = []
    self.freqToList[1].insert(0, key)
    self.keyToNode[key] = Node(key, value, 1, 0) # Using index 0 as iterator

def _touch(self, node: Node) -> None:
    # Update the node's frequency
    prevFreq = node.freq
    node.freq += 1
    newFreq = node.freq

    # Remove the key from `prevFreq`'s list
    self.freqToList[prevFreq].remove(node.key)
    if not self.freqToList[prevFreq]:
        del self.freqToList[prevFreq]
    # Update `minFreq` if needed
    if prevFreq == self.minFreq:
        self.minFreq += 1

    # Insert the key to the front of `newFreq`'s list

```

```

if newFreq not in self.freqToList:
    self.freqToList[newFreq] = []
self.freqToList[newFreq].insert(0, node.key)
node.it = 0 # Using index 0 as iterator

```

• SimplyLeet

```

from collections import OrderedDict

class LFUCache:
    def __init__(self, capacity: int):
        # Initialize cache capacity and helper variables.
        self.capacity = capacity
        self.min_freq = 0 # Track the minimum frequency in cache.
        # Map key -> (value, frequency)
        self.key_to_val_freq = {}
        # Map frequency -> OrderedDict of keys for LRU ordering.

        self.freq_to_keys = {}

    def update_freq(self, key):
        # Increase frequency of the key since it was accessed.
        value, freq = self.key_to_val_freq[key]
        # Remove key from its current frequency list.
        self.freq_to_keys[freq].pop(key)
        # If current frequency list is empty, update min_freq if necessary.
        if not self.freq_to_keys[freq]:
            del self.freq_to_keys[freq]

            if self.min_freq == freq:
                self.min_freq += 1
        new_freq = freq + 1
        # Insert key into the list corresponding to the new frequency.
        if new_freq not in self.freq_to_keys:
            self.freq_to_keys[new_freq] = OrderedDict()
        self.freq_to_keys[new_freq][key] = None
        # Update the frequency entry for the key.
        self.key_to_val_freq[key] = (value, new_freq)

```

```

def get(self, key: int) -> int:
    # Return -1 if the key is not present.
    if key not in self.key_to_val_freq:
        return -1
    # Update frequency due to access.
    self.update_freq(key)
    return self.key_to_val_freq[key][0]

def put(self, key: int, value: int) -> None:
    # If capacity is zero, nothing can be added.

    if self.capacity <= 0:
        return
    if key in self.key_to_val_freq:
        # Update value and frequency if key exists.
        self.key_to_val_freq[key] = (value, self.key_to_val_freq[key][1])
        self.update_freq(key)
    else:
        # If cache is full, evict the least frequently used (and least
        recently used) key.
        if len(self.key_to_val_freq) >= self.capacity:
            # Remove the first key from the lowest frequency list.

            key_to_remove, _ = self.freq_to_keys[self.min_freq].popitem(la
st=False)

            del self.key_to_val_freq[key_to_remove]
            if not self.freq_to_keys[self.min_freq]:
                del self.freq_to_keys[self.min_freq]
            # Insert the new key with frequency 1.
            self.key_to_val_freq[key] = (value, 1)
            if 1 not in self.freq_to_keys:
                self.freq_to_keys[1] = OrderedDict()
            self.freq_to_keys[1][key] = None
            self.min_freq = 1 # Reset min_freq.

```

• LC.ca

```
from collections import defaultdict

class LFUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.key_to_value = {}
        self.key_to_freq = defaultdict(int)
        self.freq_to_keys = defaultdict(OrderedDict)
        self.min_freq = 0

    def get(self, key: int) -> int:
        if key not in self.key_to_value:
            return -1

        # Update the frequency
        self._update_frequency(key)
        return self.key_to_value[key]

    def put(self, key: int, value: int) -> None:
        if self.capacity == 0:
            return

        if key in self.key_to_value:
            # Update the value and frequency
            self.key_to_value[key] = value
            self._update_frequency(key)
        else:
            if len(self.key_to_value) >= self.capacity:
                # Evict the least frequently used key
                self._evict()

            # Add the new key-value pair
            self.key_to_value[key] = value
            self.key_to_freq[key] = 1
            self.freq_to_keys[1][key] = None
            self.min_freq = 1

    def _update_frequency(self, key: int) -> None:
        freq = self.key_to_freq[key]
        del self.freq_to_keys[freq][key]
```

```

        if not self.freq_to_keys[freq]:
            # If there are no keys with the previous frequency, update min_freq
            if self.min_freq == freq:
                self.min_freq += 1
            del self.freq_to_keys[freq]

        freq += 1
        self.key_to_freq[key] = freq
        self.freq_to_keys[freq][key] = None

```

```

def _evict(self) -> None:
    keys = self.freq_to_keys[self.min_freq]
    evict_key, _ = keys.popitem(last=False)
    del self.key_to_value[evict_key]
    del self.key_to_freq[evict_key]

```

```

# Your LFUCache object will be instantiated and called as such:
# obj = LFUCache(capacity)
# param_1 = obj.get(key)

```

```

# obj.put(key,value)

```

```

#####

```

```

class Node:
    def __init__(self, key: int, value: int) -> None:
        self.key = key
        self.value = value
        self.freq = 1

```

```

        self.prev = None
        self.next = None

```

```

class DoublyLinkedList:
    def __init__(self) -> None:
        self.head = Node(-1, -1)
        self.tail = Node(-1, -1)
        self.head.next = self.tail
        self.tail.prev = self.head

```

```
def add_first(self, node: Node) -> None:
    node.prev = self.head
    node.next = self.head.next
    self.head.next.prev = node
    self.head.next = node
```

```
def remove(self, node: Node) -> Node:
    node.next.prev = node.prev
    node.prev.next = node.next
```

```
node.next, node.prev = None, None
return node
```

```
def remove_last(self) -> Node:
    return self.remove(self.tail.prev)
```

```
def is_empty(self) -> bool:
    return self.head.next == self.tail
```

```
class LFUCache:
```

```
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.min_freq = 0
        self.map = defaultdict(Node)
        self.freq_map = defaultdict(DoublyLinkedList)
```

```
    def get(self, key: int) -> int:
        if self.capacity == 0 or key not in self.map:
            return -1
```

```
        node = self.map[key]
        self.incr_freq(node)
        return node.value
```

```
    def put(self, key: int, value: int) -> None:
        if self.capacity == 0:
            return
        if key in self.map:
            node = self.map[key]
            node.value = value
```

```

        self.incr_freq(node)
        return
    if len(self.map) == self.capacity:
        ls = self.freq_map[self.min_freq]
        node = ls.remove_last()
        self.map.pop(node.key)
    node = Node(key, value)
    self.add_node(node)
    self.map[key] = node
    self.min_freq = 1

```

```

def incr_freq(self, node: Node) -> None:
    freq = node.freq
    ls = self.freq_map[freq]
    ls.remove(node)
    if ls.is_empty():
        self.freq_map.pop(freq)
        if freq == self.min_freq:
            self.min_freq += 1
    node.freq += 1

```

```

self.add_node(node)

```

```

def add_node(self, node: Node) -> None:
    freq = node.freq
    ls = self.freq_map[freq]
    ls.add_first(node)
    self.freq_map[freq] = ls

```

Your LFUCache object will be instantiated and called as such:

```

# obj = LFUCache(capacity)
# param_1 = obj.get(key)
# obj.put(key,value)

```

◆ Quick Review

Key Insights

- Maintain a mapping from key to its value and frequency.
- Use an additional mapping from frequency to keys (maintaining the order of insertion or recent use) for quick eviction.
- Keep a global minFreq variable to track the lowest frequency present in the cache.
- When a key is accessed or updated, increase its frequency and relocate it in the frequency mapping.
- Evictions occur by removing the oldest key from the lowest frequency bucket.

Space & Time

Time Complexity: $O(1)$ average for both get and put.

Space Complexity: $O(\text{capacity})$, where capacity is the maximum number of keys stored.

Solution

The solution utilizes two main data structures: A hash map (keyToValFreq) that maps keys to a pair (value, frequency). A second hash map (freqToKeys) mapping frequencies to an ordered list/set of keys. This data structure preserves the order in which keys were added

to allow removal of the least recently used key when multiple keys share the same frequency. A global minFreq variable is maintained to quickly identify the bucket with the least frequency. When a key is accessed or updated, the key is removed from its current frequency list and placed in the next higher frequency list. If the frequency list for the minimal frequency becomes empty, minFreq is updated. This design guarantees that both get and put operations execute in constant time on average.

Stack

Round 6 · Mid · Hard · 8 questions

Stack

□ #32 Longest Valid Parentheses

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming · Stack
Lists: Grind 169

Problem

Given a string containing just the characters '(' and ')', return the length of the longest valid (well-formed) parentheses substring.

Example 1:

Input: s = "()"

Output: 2

Explanation: The longest valid parentheses substring is "()".

Example 2:

Input: s = "()()())"

Output: 4

Explanation: The longest valid parentheses substring is "()()".

Example 3:

Input: s = ""

Output: 0

Constraints:

- $0 \leq s.length \leq 3 * 10^4$

- `s[i]` is '(', or ')'.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Find the length of the longest valid (well-formed) parentheses substring. Right?"
#
# "A few quick questions:
# Substring must be contiguous? Yes. Empty string → 0? Yes."
#
# "Let me walk: s='((' → '()' len=2 ✓. s='')()())' → '()()' len=4 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Try all substrings, check each for validity, track the longest.
#  $O(n^3)$  –  $O(n^2)$  substrings ×  $O(n)$  validity check.
# Should I go ahead and optimize?"
```

```
class _32_LongestValidParentheses_Brute:
    def longestValidParentheses(self, s):
        def valid(sub):
            bal = 0
            for c in sub:
                if c == '(': bal += 1
                else: bal -= 1
                if bal < 0: return False
            return bal == 0
        best = 0
        for i in range(len(s)):
            for j in range(i+2, len(s)+1, 2):
                if valid(s[i:j]): best = max(best, j-i)
        return best
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the  $O(n^3)$  substring validation.
# Stack-based  $O(n)$  approach:
# Store indices. Push index of '(' onto stack.
# On ')': if stack top is '(' index, pop it (matched pair).
# Else push ')' index as new boundary.
# Initialize stack with [-1] as a base boundary.
# After each match: length = current_index - stack_top.
#
# Time:  $O(n)$ . Space:  $O(n)$ ."
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _32_LongestValidParentheses:
    def longestValidParentheses(self, s):
        stack = [-1]
        best = 0
        for i, ch in enumerate(s):
            if ch == '(':
                stack.append(i)
            else:
                stack.pop()
                if not stack:
                    stack.append(i)
                else:
                    best = max(best, i - stack[-1])
        return best
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

```
#
# s='()()()':
# i=0')': pop -1 → empty → push 0. stack=[0].
# i=1'(': push 1. [0,1].
# i=2')': pop 1. stack=[0]. best=max(0,2-0)=2.
# i=3'(': push 3. [0,3].
# i=4')': pop 3. stack=[0]. best=max(2,4-0)=4.
# i=5')': pop 0 → empty → push 5. best=4 ✓
#
# "No bugs. The initial [-1] sentinel handles valid substrings starting at index 0."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): single pass. Space O(n): stack holds at most n indices.
# DP alternative: dp[i] = length of valid substring ending at i. O(n) time, O(n)
space.
# O(1) space: two-pass counting (left-right then right-left with open/close
counters).
# Follow-up: count number of valid parentheses substrings – not just the longest."
```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def longestValidParentheses(self, s: str) -> int:
        s2 = ')' + s
        # dp[i] := the length of the longest valid parentheses in the substring
        # s2[1..i]
        dp = [0] * len(s2)

        for i in range(1, len(s2)):
            if s2[i] == ')' and s2[i - dp[i - 1] - 1] == '(':
                dp[i] = dp[i - 1] + dp[i - dp[i - 1] - 2] + 2

        return max(dp)

```

• LeetDoocs

```

class Solution:
    def longestValidParentheses(self, s: str) -> int:
        n = len(s)
        f = [0] * (n + 1)
        for i, c in enumerate(s, 1):
            if c == ")":
                if i > 1 and s[i - 2] == "(":
                    f[i] = f[i - 2] + 2
                else:
                    j = i - f[i - 1] - 1
                    if j and s[j - 1] == "(":
                        f[i] = f[i - 1] + 2 + f[j - 1]
        return max(f)

```

```

class Solution:
    def longestValidParentheses(self, s: str) -> int:
        stack = [-1]
        ans = 0
        for i in range(len(s)):
            if s[i] == '(':
                stack.append(i)
            else:
                stack.pop()
                if not stack:

```

```

        stack.append(i)
    else:
        ans = max(ans, i - stack[-1])
return ans

```

• SimplyLeet

```

# Python solution using a stack
def longestValidParentheses(s: str) -> int:
    # Initialize the stack with the base index -1
    stack = [-1]
    max_length = 0

    # Iterate over the string with index
    for i, char in enumerate(s):
        if char == '(':
            # Push the index of '(' onto the stack

            stack.append(i)
        else:
            # Pop the last index for ')'
            stack.pop()
            if not stack:
                # If stack is empty, push current index as the new base
                stack.append(i)
            else:
                # Calculate the length of the current valid substring
                current_length = i - stack[-1]

            max_length = max(max_length, current_length)

    return max_length

# Example usage:
print(longestValidParentheses("()()())") # Output: 4

```

• LC.ca


```

class Solution:
    def longestValidParentheses(self, s: str) -> int:
        left = right = 0
        res = 0
        for c in s: # from left to right, '()' => will never hit left==right
            if c == '(':
                left += 1
            else:
                right += 1
            if left == right:
                res = max(res, 2 * left)
                if left < right:
                    left = right = 0

        left = right = 0 # dont forget to reset
        for c in reversed(s): # from right to left, '()' => will never hit
left==right
            if c == '(':
                left += 1
            else:
                right += 1

            if left == right:
                res = max(res, 2 * left)
            if left > right: # reverse '<' to '>'
                left = right = 0

        return res

#####

class Solution:

```

```
def longestValidParentheses(self, s: str) -> int:
    n = len(s)
    if n < 2:
        return 0
    dp = [0] * n
    for i in range(1, n):
        if s[i] == ')':
            if s[i - 1] == '(':
                dp[i] = 2 + (dp[i - 2] if i > 1 else 0)
            else:
```

```
                j = i - dp[i - 1] - 1
                if j >= 0 and s[j] == '(':
                    dp[i] = 2 + dp[i - 1] + dp[j - 1]
    return max(dp)
```

```
#####
```

```
'''
```

```
>>> s="abcdefg"
```

```
>>> [print(i,",",c) for i, c in enumerate(s, 1)]
```

```
1 , a
```

```
2 , b
```

```
3 , c
```

```
4 , d
```

```
5 , e
```

```
6 , f
```

```
7 , g
```

```
[None, None, None, None, None, None, None]
```

```
>>>
```

```
'''
```

```
class Solution:
    def longestValidParentheses(self, s: str) -> int:
        n = len(s)
        f = [0] * (n + 1)
        for i, c in enumerate(s, 1): # starting i from 1, not 0
            if c == ")":
                if i > 1 and s[i - 2] == "(":
                    f[i] = f[i - 2] + 2
            else:
                j = i - f[i - 1] - 1
```

```
        if j and s[j - 1] == "(":
            f[i] = f[i - 1] + 2 + f[j - 1]

    return max(f)
```

◆ Quick Review

Key Insights

- Use a stack to keep track of indices where potential valid substrings begin.
- Start by pushing a dummy index (-1) to handle edge cases.
- For every '(', push its index onto the stack.
- For every ')', pop from the stack. If the stack becomes empty, push the current index as a new base. Otherwise, compute the length of the valid substring using the current index minus the top index of the stack.
- An alternative is the dynamic programming approach where $dp[i]$ represents the length of the longest valid substring ending at index i .

Space & Time

Time Complexity: $O(n)$ where n is the length of the string.

Space Complexity: $O(n)$ for the stack data structure.

Solution

The stack-based solution works by maintaining indices for unmatched parentheses. Initially, a dummy index (-1) is pushed onto the stack to serve as the base for calculating subsequent valid substring lengths. As we iterate through the string: If the character is '(', we push its index onto the stack. If the character is ')', we pop from the stack. If the stack is empty afterwards, that means there is no valid starting position, so we push the current index to reset the base. Otherwise, we calculate the length of the current valid substring by subtracting the current index with the new top index of the stack and update the maximum length accordingly.

Stack

□ #42 Trapping Rain Water

HAR[Open on LeetCode](#)

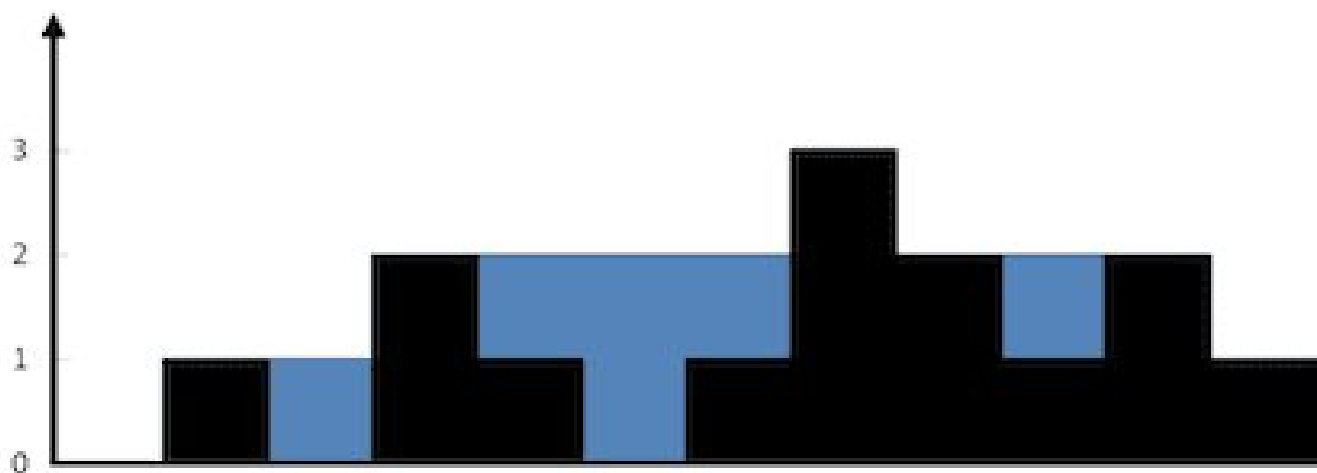
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Dynamic Programming ·
Stack · Monotonic Stack

Lists: Grind 169

Problem

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Example 1:



Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]

Output: 6

Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.

Example 2:

Input: height = [4,2,0,3,2,5]

Output: 9

Constraints:

- $n == \text{height.length}$
- $1 \leq n \leq 2 * 10^4$
- $0 \leq \text{height}[i] \leq 10^5$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given an elevation map, return how much rain water can be trapped after raining.

Water at position $i = \min(\text{max_left}, \text{max_right}) - \text{height}[i]$, clamped to 0. Right?"

#

"A few quick questions:

Heights are non-negative integers? Yes. Can a position hold 0 water? Yes."

#

"Let me walk: height=[0,1,0,2,1,0,1,3,2,1,2,1] → 6 units ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For each bar i , scan left to find max_left , scan right to find max_right .

$\text{Water}[i] = \max(0, \min(\text{max_left}, \text{max_right}) - \text{height}[i])$.

$O(n^2)$ time, $O(1)$ space.

Should I go ahead and optimize?"

```
class _42_TrappingRainWater_Brute:
```

```
    def trap(self, height):
```

```
        n = len(height); total = 0
```

```
        for i in range(n):
```

```
            max_left = max(height[:i+1]); max_right = max(height[i:])
```

```
            total += max(0, min(max_left, max_right) - height[i])
```

```
return total
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the O(n) max scans per position.
```

```
# Two-pointer approach – O(1) extra space:
```

```
# left=0, right=n-1, left_max=0, right_max=0.
```

```
# Process the side with the shorter current height:
```

```
# If height[left] <= height[right]: water at left = left_max - height[left]. Move left right.
```

```
# Else: water at right = right_max - height[right]. Move right left.
```

```
#
```

```
# Time: O(n). Space: O(1)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _42_TrappingRainWater:
```

```
    def trap(self, height):
```

```
        left, right = 0, len(height) - 1
```

```
        left_max = right_max = 0
```

```
        total = 0
```

```
        while left < right:
```

```
            if height[left] <= height[right]:
```

```
                left_max = max(left_max, height[left])
```

```
                total += left_max - height[left]
```

```
                left += 1
```

```
            else:
```

```
                right_max = max(right_max, height[right])
```

```
                total += right_max - height[right]
```

```
                right -= 1
```

```
        return total
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
```

```
#
```

```
# height=[0,1,0,2,1,0,1,3,2,1,2,1]:
```

```
# l=0,r=11,lm=0,rm=0. h[0]=0<=h[11]=1: lm=0,water+=0. l=1.
```

```
# h[1]=1<=1: lm=1. l=2. h[2]=0<lm=1: water+=1. l=3.
```

```
# ... process continues, accumulating 6 total ✓
```

```
#
```

```
# "No bugs. Processing the shorter side guarantees left_max/right_max are exact bounds."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): single pass. Space O(1): four variables.
```

```
# Precompute max_left/max_right arrays: O(n) time, O(n) space – simpler to explain.
```

```
# Monotonic stack: O(n) time, O(n) space – horizontal layer approach.
```

```
# Follow-up: Trapping Rain Water II (#407) – 3D extension using a min-heap on borders."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def trap(self, height: list[int]) -> int:
        n = len(height)
        l = [0] * n # l[i] := max(height[0..i])
        r = [0] * n # r[i] := max(height[i..n))

        for i, h in enumerate(height):
            l[i] = h if i == 0 else max(h, l[i - 1])

        for i, h in reversed(list(enumerate(height))):
            r[i] = h if i == n - 1 else max(h, r[i + 1])

        return sum(min(l[i], r[i]) - h
                     for i, h in enumerate(height))
```

```
class Solution:
    def trap(self, height: list[int]) -> int:
        if not height:
            return 0

        ans = 0
        l = 0
        r = len(height) - 1
        maxL = height[l]
        maxR = height[r]
```

```
        while l < r:
            if maxL < maxR:
                ans += maxL - height[l]
                l += 1
                maxL = max(maxL, height[l])
            else:
                ans += maxR - height[r]
                r -= 1
                maxR = max(maxR, height[r])
```

```
        return ans
```

• LeetCode

```
class Solution:
    def trap(self, height: List[int]) -> int:
        n = len(height)
        left = [height[0]] * n
        right = [height[-1]] * n
        for i in range(1, n):
            left[i] = max(left[i - 1], height[i])
            right[n - i - 1] = max(right[n - i], height[n - i - 1])
        return sum(min(l, r) - h for l, r, h in zip(left, right, height))
```

• SimplyLeet

```
# Python solution using the two pointers approach
def trap(height):
    # Initialize left and right pointers and water accumulator
    left, right = 0, len(height) - 1
    left_max, right_max = 0, 0
    water = 0

    # Traverse the elevation map from both ends
    while left < right:
        # Update left_max and right_max as we encounter new bars

        if height[left] < height[right]:
            # left side is the determining side
            if height[left] >= left_max:
                left_max = height[left] # update left max since current height
is higher
            else:
                water += left_max - height[left] # water trapped at left
position
            left += 1 # move left pointer to the right
        else:
            # right side is the determining side
            if height[right] >= right_max:
```

```

        right_max = height[right] # update right max since current
height is higher
    else:
        water += right_max - height[right] # water trapped at right
position
    right -= 1 # move right pointer to the left
    return water

# Example usage:
print(trap([0,1,0,2,1,0,1,3,2,1,2,1])) # Expected output: 6

```

• LC.ca

```

class Solution:
    def trap(self, height: List[int]) -> int:
        n = len(height)
        if n < 3:
            return 0

        lmx = [height[0]] * n
        rmx = [height[n - 1]] * n
        for i in range(1, n):
            lmx[i] = max(lmx[i - 1], height[i]) # i self compare

            rmx[n - 1 - i] = max(rmx[n - i], height[n - 1 - i])

        # no negative, worst is, min(left,right) is itself
        return sum( min(lmx[i], rmx[i]) - height[i] for i in range(n) )

#####

class Solution: # zip
    def trap(self, height: List[int]) -> int:
        n = len(height)

```

```

left = [height[0]] * n
right = [height[-1]] * n
for i in range(1, n):
    left[i] = max(left[i - 1], height[i])
    right[n - i - 1] = max(right[n - i], height[n - i - 1])
return sum(min(l, r) - h for l, r, h in zip(left, right, height))

```

```
#####
```

```
class Solution: # find the highest bar first
```

```

def trap(self, height: List[int]) -> int:
    if not height:
        return 0

```

```

    total_water = 0
    max_index = 0
    max_height = float('-inf')

```

```

    # Find the index of the maximum height
    for i in range(len(height)):

```

```

        if height[i] > max_height:
            max_height = height[i]
            max_index = i

```

```

    # Calculate water trapped on the left side of the maximum height
    left_max = height[0]
    for i in range(1, max_index):
        if left_max < height[i]:
            left_max = height[i]
        total_water += left_max - height[i]

```

```

    # Calculate water trapped on the right side of the maximum height
    right_max = height[-1]
    for i in range(len(height) - 2, max_index, -1):
        if right_max < height[i]:
            right_max = height[i]
        total_water += right_max - height[i]

```

```

    return total_water

```

◆ Quick Review

Key Insights

- The water trapped at any index is determined by the minimum of the maximum height on its left and right minus the height at that index.
- A two-pointers approach can efficiently solve the problem in one pass by maintaining left and right boundaries.
- Alternative approaches include dynamic programming with pre-computed left max and right max arrays, or using a monotonic stack to determine boundaries.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$ when using the two-pointer approach

Solution

We use a two-pointers strategy by initializing pointers at the beginning and the end of the array. Keep track of the maximum height seen so far from the left (`left_max`) and the right (`right_max`). At each step, move the pointer

with the lesser height because the water trapped is determined by the shorter boundary. Compute the water at that position as the difference between the boundary (`left_max` or `right_max`) and the current height, accumulating it into the total. This approach ensures linear time scanning with constant extra space.

Stack

#84 Largest Rectangle in Histogram

HAR

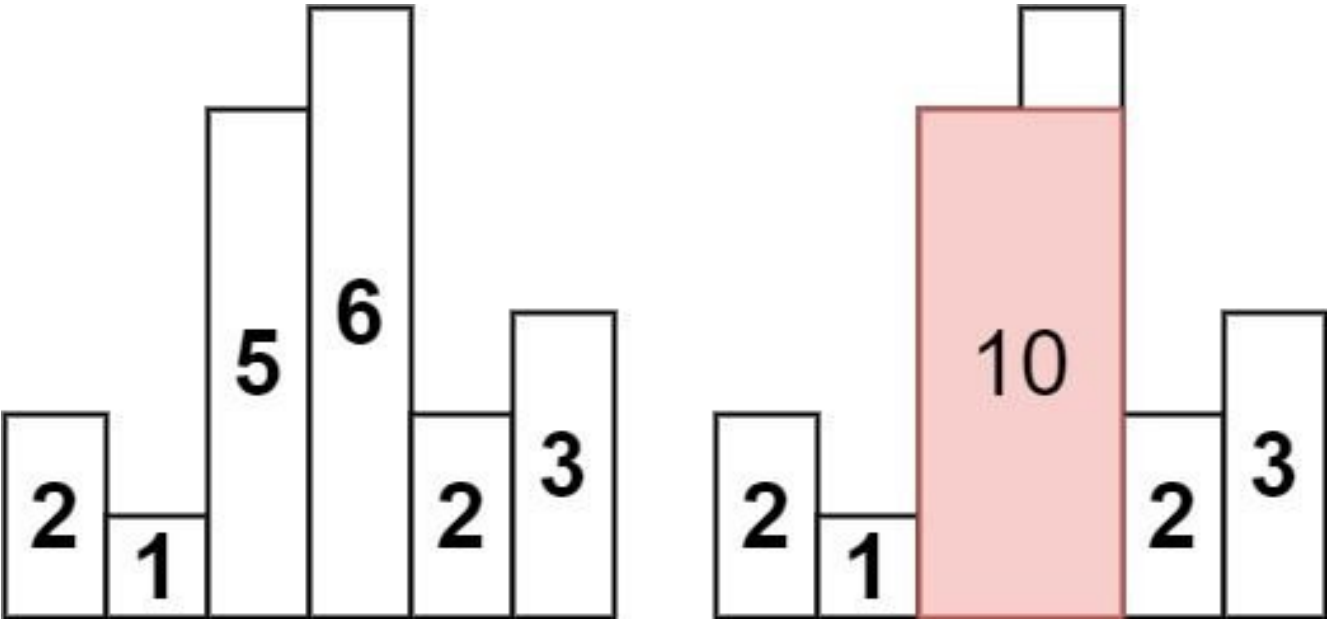
[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Stack · Monotonic Stack
Lists: Grind 169

Problem

Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return the area of the largest rectangle in the histogram.

Example 1:



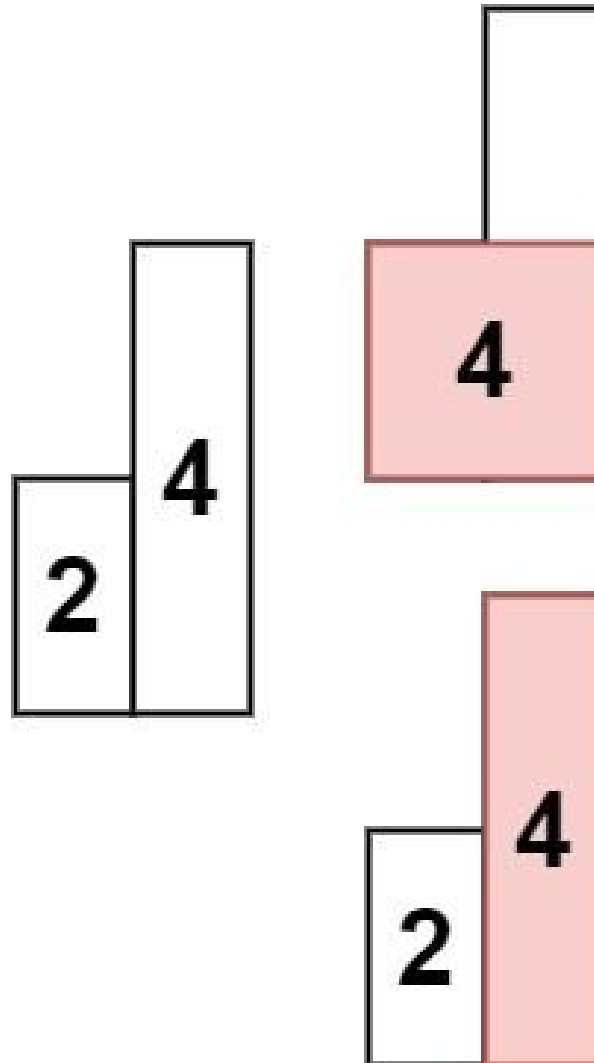
Input: heights = [2,1,5,6,2,3]

Output: 10

Explanation: The above is a histogram where width of each bar is 1.

The largest rectangle is shown in the red area, which has an area = 10 units.

Example 2:



Input: heights = [2,4]

Output: 4

Constraints:

- $1 \leq \text{heights.length} \leq 105$
- $0 \leq \text{heights}[i] \leq 104$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Given bar heights in a histogram, find the area of the largest rectangle
# that fits entirely within the bars. Width of each bar is 1. Right?"
#
# "A few quick questions:
# Heights can be 0? Yes. Single bar → area = its height? Yes."
#
# "Let me walk: heights=[2,1,5,6,2,3] → area=10 (bars 2,3 at height 5) ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# For each pair (i, j), area = (j-i+1) * min(heights[i..j]).
# Track maximum across all pairs.
# O(n2) – can maintain running min while expanding j.
# Should I go ahead and optimize?"
```

```
class _84_LargestRectangleInHistogramBrute:
    def largestRectangleArea(self, heights):
        best = 0
        for i in range(len(heights)):
            min_h = heights[i]
            for j in range(i, len(heights)):
                min_h = min(min_h, heights[j])
                best = max(best, min_h * (j - i + 1))
        return best
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the O(n2) nested scan.
# Monotonic increasing stack of indices:
# For each bar: while stack top is taller than current bar, pop and compute its
# area.
# Height = heights[popped], width = current_idx - stack_new_top - 1.
# Append sentinel 0s at both ends to force all bars to be popped.
#
# Time: O(n). Space: O(n)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _84_LargestRectangleInHistogram:
    def largestRectangleArea(self, heights):
        heights = [0] + heights + [0]
        stack = [0]
        best = 0
        for i in range(1, len(heights)):
```

```

        while heights[stack[-1]] > heights[i]:
            h = heights[stack.pop()]
            w = i - stack[-1] - 1
            best = max(best, h * w)
        stack.append(i)
    return best

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

heights=[0,2,1,5,6,2,3,0] (sentinels):

i=2(1): 2>1 → pop1: h=2,w=2-0-1=1,area=2. stack=[0,2].

i=5(2): 6>2 → pop4: h=6,w=5-3-1=1,area=6. 5>2 → pop3: h=5,w=5-2-1=2,area=10 ✓.

i=7(0): pops all → final best=10 ✓

#

"No bugs. Sentinel 0s ensure every bar gets popped and processed – no leftovers."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time O(n): each bar pushed and popped at most once. Space O(n) stack."

Follow-up: Maximal Rectangle (#85) – for each row build histogram heights, apply this.

Follow-up: Maximum Width Ramp (#962) – similar two-phase monotonic stack pattern."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def largestRectangleArea(self, heights: list[int]) -> int:
        ans = 0
        stack = []

        for i in range(len(heights) + 1):
            while stack and (i == len(heights) or heights[stack[-1]] > heights[i]):
                h = heights[stack.pop()]
                w = i - stack[-1] - 1 if stack else i
                ans = max(ans, h * w)

            stack.append(i)

        return ans

```

• LeetDoocs

```

class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        n = len(heights)
        stk = []
        left = [-1] * n
        right = [n] * n
        for i, h in enumerate(heights):
            while stk and heights[stk[-1]] >= h:
                right[stk[-1]] = i
                stk.pop()

            if stk:
                left[i] = stk[-1]
            stk.append(i)
        return max(h * (right[i] - left[i] - 1) for i, h in enumerate(heights))

```

• SimplyLeet

Python solution

```

def largestRectangleArea(heights):
    # Append a zero to flush out the remaining bars in the stack
    heights.append(0)
    stack = [] # Stack to store indices of bars
    max_area = 0

    # Iterate over each index and bar height
    for i, h in enumerate(heights):

        # Process bars that are taller than the current bar
        while stack and heights[stack[-1]] > h:
            height = heights[stack.pop()] # Height of the bar at the index
            popped from the stack
            # Calculate width: if stack is empty, width is the current index;
            otherwise, it's the difference
            width = i if not stack else i - stack[-1] - 1
            max_area = max(max_area, height * width)
            stack.append(i)

    return max_area

```

```
# Example usage:
if __name__ == "__main__":
    histogram = [2, 1, 5, 6, 2, 3]
    print(largestRectangleArea(histogram))
```

• LC.ca

```
'''
    0000000000 h 00000000000000000000 00(<) h 000 left_i, right_i

    0000000000 h * (right_i - left_i - 1)0000000000
    '''

class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        n = len(heights)
        stk = []
        left = [-1] * n

        right = [n] * n
        for i, h in enumerate(heights):
            while stk and heights[stk[-1]] >= h:
                right[stk[-1]] = i
                stk.pop()
            if stk:
                left[i] = stk[-1] # same as below: stk[-1] in 'i - stack[-1] -
1'

            stk.append(i)

        # valid for one element input [3]

        return max(h * (right[i] - left[i] - 1) for i, h in enumerate(heights)
)

#####

class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        if not heights:
            return 0
```

```

heights.append(-1) # make for loop running
stack = []
ans = 0
for i in range(0, len(heights)):
    while stack and heights[i] < heights[stack[-1]]:
        currentLowestBarIndex = stack.pop()
        h = heights[currentLowestBarIndex]
        # stack[-1] is after pop(). why not i-currentLowestBarIndex?
        # because not working for tie case in the stack,
        # eg. for input=[4,2,0,3,2,5], expected=6 but output=5

        # 000000000000 00(<) h 000 left_i, right_i
        # 0000000000 00 h 0000000000 right-left+1
        # 00000 00 h0000right-left+1 00 0000 00000index000000+1 -2 0 -1
        w = i - stack[-1] - 1 if stack else i
        ans = max(ans, h * w)
    stack.append(i)

heights.pop() # restore, pop -1
return ans

```

◆ Quick Review

Key Insights

- Each bar can potentially extend left and right as long as the neighboring bars are not shorter.
- A monotonic (increasing) stack can be used to efficiently track indices of bars.
- When encountering a bar lower than the one on the top of the stack, pop the stack and calculate the area of the rectangle formed with the popped bar height.

- A dummy bar (of height 0) is appended at the end to ensure all bars are processed.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

The solution employs a monotonic stack to traverse the histogram once. For each bar, while the current bar's height is less than the height at the top index of the stack, it pops from the stack and calculates the area using the popped height as the smallest bar. The width for the rectangle is determined by the difference between the current index and the index from the stack (or the current index if the stack is empty). This effectively finds the maximum rectangular area possible. A sentinel value (0) is added at the end of the histogram to flush any remaining bars from the stack.

Stack

□ #224 Basic Calculator

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · String · Stack · Recursion
Lists: Grind 169

Problem

Given a string s representing a valid expression, implement a basic calculator to evaluate it, and return the result of the evaluation.

Note: You are not allowed to use any built-in function which evaluates strings as mathematical expressions, such as `eval()`.

Example 1:

Input: $s = "1 + 1"$
Output: 2

Example 2:

Input: $s = "2-1 + 2 "$
Output: 3

Example 3:

Input: $s = "(1+(4+5+2)-3)+(6+8)"$
Output: 23

Constraints:

- $1 \leq s.length \leq 3 * 10^5$
- s consists of digits, '+', '-', '(', ')', and ' '.
- s represents a valid expression.
- '+' is not used as a unary operation (i.e., "+1" and "+(2 + 3)" is invalid).
- '-' could be used as a unary operation (i.e., "-1" and "-(2 + 3)" is valid).
- There will be no two consecutive operators in the input.
- Every number and running calculation will fit in a signed 32-bit integer.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Evaluate a string with +, -, and parentheses. Non-negative integers, spaces allowed.

No * or /. No eval(). Right?"

#

"A few quick questions:

Can the expression start with '-' (unary minus)? Yes – per constraints.

Parentheses can be nested? Yes – arbitrary depth."

#

"Let me walk: '1 + 1'→2 ✓. '(1+(4+5+2)-3)+(6+8)'→23 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Recursive descent parser: when we see '(', recursively evaluate the inner expression.

$O(n)$ time, $O(d)$ space for recursion depth d .

```
# Should I go ahead and optimize?"
class _224_BasicCalculator_Brute:
    def calculate(self, s):
        self.i = 0
        def parse():
            result = 0; sign = 1
            while self.i < len(s):
                c = s[self.i]
                if c.isdigit():
                    num = 0
                    while self.i < len(s) and s[self.i].isdigit():
                        num = num * 10 + int(s[self.i]); self.i += 1
                    result += sign * num; continue
                elif c == '+': sign = 1
                elif c == '-': sign = -1
                elif c == '(': self.i += 1; result += sign * parse(); continue
                elif c == ')': break
                self.i += 1
            return result
        return parse()
```

PHASE 3 — OPTIMIZE

"The bottleneck is the recursion stack — $O(d)$ depth for d nesting levels.

Iterative stack: push (result, sign) when '(' is encountered, pop and merge on ')'.
Maintain current number being built, current result, and sign.

#

Time: $O(n)$. Space: $O(d)$ for the stack (same depth as recursion)."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _224_BasicCalculator:
    def calculate(self, s):
        stack = []
        result = 0
        sign = 1
        num = 0
        for ch in s:
            if ch.isdigit():
                num = num * 10 + int(ch)
            elif ch in '+-':
                result += sign * num
                num = 0
                sign = 1 if ch == '+' else -1
            elif ch == '(':
                stack.append(result)
                stack.append(sign)
                result = 0
                sign = 1
            elif ch == ')':
```

```

        result += sign * num
        num = 0
        result *= stack.pop()
        result += stack.pop()
    return result + sign * num

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

'(1+(4+5+2)-3)+(6+8)':

'(': push result=0,sign=1. reset. '1': num=1. '+': result=1,sign=1.

'(':push1,1.reset.

'4+5+2': result=11. ')': result=11*1+1=12. '-': result=12,sign=-1.

'3': num=3. ')': result=12+(-1)*3=9. result=9*1+0=9. '+': result=9,sign=1.

'(': push 9,1. '6+8': result=14. ')': 14*1+9=23 ✓

#

"No bugs. The stack stores (outer_result, outer_sign) before entering a parenthesis."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time O(n): one pass. Space O(d): stack depth = nesting depth.

Follow-up: Basic Calculator II (#227) – adds * and /; handle with operator precedence stack.

Follow-up: Basic Calculator III (#772) – all four operators with parentheses."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def calculate(self, s: str) -> int:
        ans = 0
        num = 0
        sign = 1
        stack = [sign] # stack[-1]: the current environment's sign

        for c in s:
            if c.isdigit():
                num = num * 10 + int(c)

```

```

elif c == '(':
    stack.append(sign)
elif c == ')':
    stack.pop()
elif c == '+' or c == '-':
    ans += sign * num
    sign = (1 if c == '+' else -1) * stack[-1]
    num = 0

return ans + sign * num

```

• LeetDoocs

```

class Solution:
    def calculate(self, s: str) -> int:
        stk = []
        ans, sign = 0, 1
        i, n = 0, len(s)
        while i < n:
            if s[i].isdigit():
                x = 0
                j = i
                while j < n and s[j].isdigit():

                    x = x * 10 + int(s[j])
                    j += 1
                ans += sign * x
                i = j - 1
            elif s[i] == "+":
                sign = 1
            elif s[i] == "-":
                sign = -1
            elif s[i] == "(":
                stk.append(ans)

                stk.append(sign)
                ans, sign = 0, 1
            elif s[i] == ")":
                ans = stk.pop() * ans + stk.pop()
                i += 1
        return ans

```

• SimplyLeet

```
# Python solution for the Basic Calculator problem

def calculate(s: str) -> int:
    # Stack to store pairs of (current result, current sign) before a '('
    stack = []
    result = 0 # Running total
    number = 0 # Current number being processed
    sign = 1   # Current sign (1 or -1)

    # Iterate through each character in the string

    for char in s:
        if char.isdigit():
            # Build number digit by digit
            number = number * 10 + int(char)
        elif char in "+-":
            # Add the complete number with its sign to result
            result += sign * number
            # Reset number for the next potential number
            number = 0
            # Update the sign based on the operator
            sign = 1 if char == '+' else -1
        elif char == '(':
            # Push the current result and sign onto the stack
            stack.append(result)
            stack.append(sign)
            # Reset result and sign for the new sub-expression
            result = 0
            sign = 1
        elif char == ')':
            # Complete the number before the parentheses close
            result += sign * number
            # Pop the previous result and sign from the stack
            previous_sign = stack.pop()
            previous_result = stack.pop()
            result = previous_result + previous_sign * result
            number = 0
            sign = previous_sign
```

```

        result += sign * number
        number = 0
        # The top of the stack is the sign before the parenthesis
        prev_sign = stack.pop()
        # The next value on the stack is the result before the parenthesis
        prev_result = stack.pop()
        # Combine the result of the previous expression with the
sub-expression evaluated
        result = prev_result + prev_sign * result
        # Skip spaces implicitly when no if condition is met

# Add any number left after the loop ends
result += sign * number
return result

# Example usage:
print(calculate("1 + 1")) # Expected output: 2
print(calculate(" 2-1 + 2 ")) # Expected output: 3
print(calculate("(1+(4+5+2)-3)+(6+8)")) # Expected output: 23

```

• LC.ca

```

# only + and - , no * , no /
class Solution:
    def calculate(self, s: str) -> int:
        stk = []
        ans, sign = 0, 1
        i, n = 0, len(s)
        while i < n:
            if s[i].isdigit():
                x = 0
                j = i

```

```

# with this while, no need to do final calculation like below
solution
    while j < n and s[j].isdigit():
        x = x * 10 + int(s[j])
        j += 1
    ans += sign * x
    i = j - 1
    elif s[i] == "+":
        sign = 1
    elif s[i] == "-":
        sign = -1

    elif s[i] == "(":
        stk.append(ans)
        stk.append(sign)
        ans, sign = 0, 1
    elif s[i] == ")":
        ans = stk.pop() * ans + stk.pop()
    i += 1
return ans

```

```
#####
```

```

class Solution:
    def calculate(self, s: str) -> int:
        ans = 0
        num = 0
        sign = 1
        stack = [sign] # stack[-1]: current env's sign

        for c in s:
            if c.isdigit():

```



```

    num = num * 10 + int(c)
    # num = num * 10 + (ord(c) - ord('0')) => also works
elif c == '(':
    stack.append(sign)
elif c == ')':
    stack.pop() # pop pairing sign for this () pair
elif c == '+' or c == '-':
    ans += sign * num
    sign = (1 if c == '+' else -1) * stack[-1] # after all + or -, the sign
for current num
    num = 0

return ans + sign * num # final calculation

```

◆ Quick Review

Key Insights

- Use a stack to save previous results and signs when encountering parentheses.
- Process the string left-to-right by accumulating multi-digit numbers and applying the current sign.
- When encountering a '(', push the current result and sign onto the stack and reset them.
- When encountering a ')', complete the current sub-expression then pop the previous state to incorporate the computed value.
- Handle whitespaces by simply skipping them.

Space & Time

Time Complexity: $O(n)$ where n is the length of the input string, since we process each character once.

Space Complexity: $O(n)$ in the worst-case scenario due to the use of a stack for nested parentheses.

Solution

The solution uses an iterative approach with a stack: Initialize variables: result to keep the running total, sign to handle addition/subtraction, and index to iterate over the string. As you iterate, build multi-digit numbers by checking for consecutive numerical characters. Adjust the total by adding the product of the current number and its sign. When encountering a '(', push the current result and sign onto the stack, then reset results and sign for the new sub-expression. When encountering a ')', complete the sub-expression by popping the previous result and sign, then combine. Skip over spaces. The final result is the computed value after the end of the string. This

approach effectively simulates recursion and correctly handles nested expressions and negation.

Stack

□ #239 Sliding Window Maximum

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Queue · Sliding Window · Monotonic
Queue

Lists: Grind 169

Problem

You are given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position.

Return the max sliding window.

Example 1:

Input: `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3`

Output: `[3,3,5,5,6,7]`

Explanation:

Window position Max

[1 3 -1] -3 5 3 6 7 3

1 [3 -1 -3] 5 3 6 7 3

1 3 [-1 -3 5] 3 6 7 5

Example 2:

Input: nums = [1], k = 1

Output: [1]

Constraints:

- $1 \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$
- $1 \leq k \leq \text{nums.length}$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
        q = deque()
        ans = []

        for r in range(len(nums)):
            l = r-k+1
            if len(q) != 0 and l > q[0]:
                q.popleft()

            while len(q) != 0 and nums[r] >= nums[q[-1]]:
                q.pop()
            q.append(r)
            if l >= 0:
                ans.append(nums[q[0]])

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Sliding window of size k moving through the array. Return the maximum of each window. Right?"

#

"A few quick questions:

```

# k ≤ n always? Yes. Can values be negative? Yes."
#
# "Let me walk: nums=[1,3,-1,-3,5,3,6,7], k=3 → [3,3,5,5,6,7] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Slide window, compute max at each position using Python's max().
# O(n*k) time, O(1) extra space.
# Should I go ahead and optimize?"
class _239_SlidingWindowMaximum_Brute:
    def maxSlidingWindow(self, nums, k):
        return [max(nums[i:i+k]) for i in range(len(nums) - k + 1)]

# PHASE 3 — OPTIMIZE
# "The bottleneck is the O(k) max scan per window position.
# Monotonic deque of indices – maintains candidates for the window max:
# Remove from back any index with smaller value (they can never be max).
# Remove from front any index that's outside the window.
# Front of deque is always the current window maximum.
#
# Time: O(n). Space: O(k) for the deque."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
from collections import deque
class _239_SlidingWindowMaximum:
    def maxSlidingWindow(self, nums, k):
        dq = deque()
        result = []
        for i, num in enumerate(nums):
            while dq and nums[dq[-1]] < num:
                dq.pop()
            dq.append(i)
            if dq[0] <= i - k:
                dq.popleft()
            if i >= k - 1:
                result.append(nums[dq[0]])
        return result

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# nums=[1,3,-1,-3,5,3,6,7], k=3:
# i=0(1): dq=[0]. i=1(3): pop0(1<3), dq=[1]. i=2(-1): dq=[1,2]. window[1,3,-1]:
ans=nums[1]=3 ✓
# i=3(-3): dq=[1,2,3]. ans=nums[1]=3 ✓. i=4(5): pop3,2,1, dq=[4]. ans=5 ✓
# i=5(3): dq=[4,5]. ans=5 ✓. i=6(6): pop5,4, dq=[6]. ans=6 ✓. i=7(7): pop6, dq=[7].
ans=7 ✓
# → [3,3,5,5,6,7] ✓
#

```

"No bugs. dq[0]<=i-k removes expired indices from the front before reading the max."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$: each element enqueued/dequeued at most once. Space $O(k)$ deque.

Follow-up: Sliding Window Minimum – same deque, flip comparison to maintain increasing order.

Follow-up: Max of all subarrays of size k in a circular array – extend with modular indexing."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxSlidingWindow(self, nums: list[int], k: int) -> list[int]:
        ans = []
        maxQ = collections.deque()

        for i, num in enumerate(nums):
            while maxQ and maxQ[-1] < num:
                maxQ.pop()
            maxQ.append(num)
            if i >= k and nums[i - k] == maxQ[0]: # out-of-bounds

                maxQ.popleft()
            if i >= k - 1:
                ans.append(maxQ[0])

        return ans
```

• LeetDoocs

```
class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
        q = [(-v, i) for i, v in enumerate(nums[: k - 1])]
        heapify(q)
        ans = []
        for i in range(k - 1, len(nums)):
            heappush(q, (-nums[i], i))
            while q[0][1] <= i - k:
                heappop(q)
            ans.append(-q[0][0])
```



```

        return ans

class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
        q = deque()
        ans = []
        for i, x in enumerate(nums):
            if q and i - q[0] >= k:
                q.popleft()
            while q and nums[q[-1]] <= x:
                q.pop()
            q.append(i)

            if i >= k - 1:
                ans.append(nums[q[0]])
        return ans

```

• SimplyLeet

```

from collections import deque

def maxSlidingWindow(nums, k):
    # Create a deque to store indices of elements in the current window
    deq = deque()
    result = []

    for i, num in enumerate(nums):
        # Remove indices from the front if they are out of the current window
        if deq and deq[0] == i - k:

            deq.popleft()

        # Remove elements from the back while the current number is larger
        # than the number at the indices stored in the deque
        while deq and nums[deq[-1]] < num:
            deq.pop()

        # Append the current index to the deque
        deq.append(i)

```

```

        # If we've hit the window size, append the current max to result
        if i >= k - 1:
            result.append(nums[deq[0]])

    return result

# Example usage:
# print(maxSlidingWindow([1,3,-1,-3,5,3,6,7], 3))

```

• LC.ca

```

from collections import deque

class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
        q = deque()
        ans = []
        for i, v in enumerate(nums):
            if q and i - k + 1 > q[0]:
                q.popleft() # remove index if out of window left
            while q and nums[q[-1]] <= v: # `<=`, not `<`, to ensure the bigger
index stored in deque
                q.pop()
            q.append(i)
            if i >= k - 1:
                ans.append(nums[q[0]])
        return ans

#####

class Solution(object):
    def maxSlidingWindow(self, nums, k):

```

```
"""
:type nums: List[int]
:type k: int
:rtype: List[int]
"""
if k == 0:
    return []
ans = [0 for _ in range(len(nums) - k + 1)]
stack = collections.deque([])
for i in range(0, k):
    while stack and nums[stack[-1]] < nums[i]:
        stack.pop()
    stack.append(i)
ans[0] = nums[stack[0]]
idx = 0
for i in range(k, len(nums)):
    idx += 1
    if stack and stack[0] == i - k:
        stack.popleft()
    while stack and nums[stack[-1]] < nums[i]:
        stack.pop()
    stack.append(i)
    ans[idx] = nums[stack[0]]

return ans
```

◆ Quick Review

Key Insights

- Use a deque (double-ended queue) to maintain the indices of potential maximum elements in a decreasing order.
- Ensure that the deque always has the current window's indices by removing elements that

fall outside the current window.

- Only add elements that are greater than the ones at the back of the deque, thus preserving a monotonic decreasing order.
- The maximum for the current window is at the front of the deque.

Space & Time

Time Complexity: $O(n)$ where n is the number of elements in `nums`, since each element is processed at most twice.

Space Complexity: $O(k)$, as the deque stores indices for at most k elements at any time.

Solution

The solution uses a monotonic deque to maintain indices of potential maximum elements in the current window. As the window slides, we: Remove indices from the front if they are out of the window. Remove indices from the back if the current element is larger, because they cannot be the maximum if the current element is in the window. Append the current index. After the first k elements, the element at the front of the deque is the maximum for that window. This approach

ensures that each element is pushed and popped from the deque only once, achieving an overall $O(n)$ time complexity.

Stack

□ #716 Max Stack

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Stack · Design · Doubly-Linked List · Ordered
Set

Lists: Denny Zhang

Problem

Design a max stack data structure that supports the stack operations and supports finding the stack's maximum element.

Implement the MaxStack class:

- **MaxStack()** Initializes the stack object.
- **void push(int x)** Pushes element x onto the stack.
- **int pop()** Removes the element on top of the stack and returns it.
- **int top()** Gets the element on the top of the stack without removing it.
- **int peekMax()** Retrieves the maximum element in the stack without removing it.

- **int popMax()** Retrieves the maximum element in the stack and removes it. If there is more than one maximum element, only remove the top-most one.

You must come up with a solution that supports $O(1)$ for each top call and $O(\log n)$ for each other call.

Example 1:

```
Input
["MaxStack", "push", "push", "push", "top", "popMax", "top", "peekMax", "pop",
"top"]
[[], [5], [1], [5], [], [], [], [], [], []]
Output
[null, null, null, null, 5, 5, 1, 5, 1, 5]
```

Explanation

```
MaxStack stk = new MaxStack();
```

Constraints:

- $-107 \leq x \leq 107$
- At most 105 calls will be made to push, pop, top, peekMax, and popMax.
- There will be at least one element in the stack when pop, top, peekMax, or popMax is called.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY


```

# "Let me make sure I understand the problem correctly.
# Design a Max Stack: push, pop, top (all O(1)), peekMax (O(1)),
# and popMax (O(log n)) – remove and return the max element.
# If multiple maxes, remove the top-most one. Right?"
#
# "A few quick questions:
# pop/top are called only on non-empty stacks? Yes.
# popMax on an empty stack? Won't happen per constraints."
#
# "Let me walk: push(5,1,5). top=5. popMax=5. top=1. peekMax=5. pop=1. top=5 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Regular stack + scan for peekMax and popMax.
# push/pop/top O(1). peekMax O(n). popMax O(n) scan + O(n) rebuild.
# Should I go ahead and optimize?"
class _716_MaxStack_Brute:
    def __init__(self): self.stack = []
    def push(self, x): self.stack.append(x)
    def pop(self): return self.stack.pop()
    def top(self): return self.stack[-1]
    def peekMax(self): return max(self.stack)
    def popMax(self):
        m = max(self.stack); tmp = []
        while self.stack[-1] != m: tmp.append(self.stack.pop())
        self.stack.pop()
        while tmp: self.stack.append(tmp.pop())
        return m

# PHASE 3 — OPTIMIZE
# "The bottleneck is the O(n) scan for max and rebuild after popMax.
# Doubly linked list (DLL) for stack + sorted set (indexed by (val, uid)) for O(log
n) max access.
# DLL preserves insertion/stack order. Sorted set tracks (val, uid) for O(log n)
find/remove.
# push: add to DLL tail + sorted set. pop: remove DLL tail + sorted set entry.
# popMax: find max in sorted set, remove from sorted set + DLL.
# top/peekMax: O(1) peek at DLL tail / sorted set max.
#
# All ops O(log n) amortized. Space: O(n)."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
from sortedcontainers import SortedList
class _716_MaxStack:
    def __init__(self):
        self._stack = []
        self._sl = SortedList()
        self._uid = 0
        self._removed = set()

```

```

def push(self, x):
    self._stack.append((x, self._uid))
    self._sl.add((x, self._uid))
    self._uid += 1

def pop(self):
    while self._stack[-1][1] in self._removed: self._stack.pop()
    val, uid = self._stack.pop()
    self._removed.add(uid)
    self._sl.remove((val, uid))
    return val

def top(self):
    while self._stack[-1][1] in self._removed: self._stack.pop()
    return self._stack[-1][0]

def peekMax(self): return self._sl[-1][0]

def popMax(self):
    val, uid = self._sl.pop()
    self._removed.add(uid)
    return val

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

push(5,uid=0), push(1,uid=1), push(5,uid=2).

sl=[(1,1),(5,0),(5,2)]. top()→stack[-1]=(5,2)→5 ✓.

popMax()→sl.pop()=(5,2)→remove uid2. return 5 ✓.

top()→stack[-1]=(5,2) in removed→pop→(1,1). top=1 ✓.

peekMax()→sl[-1]=(5,0)→5 ✓. pop()→(1,1), remove uid1. sl=[(5,0)]. return 1 ✓.

top()→(5,0). return 5 ✓.

#

"No bugs. Lazy removal via _removed set avoids O(n) cleanup on pop/top."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"push/pop/top/peekMax/popMax all O(log n) with SortedList. Space O(n).

Without sortedcontainers, implement with a heap + counter for O(log n) – same idea.

Follow-up: Min Stack (#155) – O(1) all ops with a parallel min stack.

Follow-up: if popMax needs O(1), a doubly-linked list keyed by value could work."

Community Solutions (Python)

• SimplyLeet

```
# Python solution implementing the MaxStack data structure
# We use collections.OrderedDict for demonstration.
# However, in Python we do not have a built in balanced BST;
# one can use "sortedcontainers" or manually maintain a heap and dictionary.
# For clarity, we simulate an ordered map with a TreeMap-like structure using
# sorted list of keys.
# In production, one may use "SortedDict" from sortedcontainers.
```

```
from collections import defaultdict
```

```
class Node:
```

```
    def __init__(self, val):
        self.val = val
        self.prev = None
        self.next = None
```

```
class DoubleLinkedList:
```

```
    def __init__(self):
        # Create dummy head and tail nodes.
        self.head = Node(0)
        self.tail = Node(0)
```

```
        self.head.next = self.tail
        self.tail.prev = self.head
```

```
    def add_last(self, node):
        # Add node right before the tail.
        node.prev = self.tail.prev
        node.next = self.tail
        self.tail.prev.next = node
        self.tail.prev = node
```

```
def pop(self, node=None):
    # If no node is provided, pop from the end.
    if not node:
        node = self.tail.prev
    node.prev.next = node.next
    node.next.prev = node.prev
    return node
```

```
def top(self):
    # Return the last node's value.
```

```
    return self.tail.prev.val
```

```
class MaxStack:
```

```
    def __init__(self):
        self.dll = DoubleLinkedList()
        # Dictionary mapping value to list of nodes
        self.valToNodes = defaultdict(list)
        # Sorted list to simulate ordered set of keys.
        self.sortedKeys = []
```

```
    def push(self, x: int) -> None:
        node = Node(x)
        self.dll.add_last(node)
        self.valToNodes[x].append(node)
        # Insert x into sortedKeys if new, otherwise maintain count.
        if len(self.valToNodes[x]) == 1:
            # binary insertion into sortedKeys
            lo, hi = 0, len(self.sortedKeys)
            while lo < hi:
                mid = (lo + hi) // 2
```

```
                if self.sortedKeys[mid] < x:
                    lo = mid + 1
                else:
                    hi = mid
            self.sortedKeys.insert(lo, x)
```

```
    def pop(self) -> int:
        node = self.dll.pop()
        x = node.val
        self.valToNodes[x].pop()
```

```

        if not self.valToNodes[x]:
            # Remove x from sortedKeys
            self.sortedKeys.remove(x)
        return x

    def top(self) -> int:
        return self.dll.top()

    def peekMax(self) -> int:
        # The maximum is the last element in sortedKeys.

        return self.sortedKeys[-1]

    def popMax(self) -> int:
        max_val = self.peekMax()
        # Get the most recent node with max_val.
        node = self.valToNodes[max_val].pop()
        if not self.valToNodes[max_val]:
            self.sortedKeys.pop() # removes the last element
        # Remove the node from the doubly-linked list.
        self.dll.pop(node)

        return max_val

# Example usage:
# stk = MaxStack()
# stk.push(5)
# stk.push(1)
# stk.push(5)
# print(stk.top()) # returns 5
# print(stk.popMax()) # returns 5
# print(stk.top()) # returns 1

# print(stk.peekMax()) # returns 5
# print(stk.pop()) # returns 1
# print(stk.top()) # returns 5

```

• LC.ca

```

class MaxStack(object):

    def __init__(self):
        """
        initialize your data structure here.
        """
        self.stack = []
        self.max_stack = []
        # or the same trick in min-stack
        # self.max_stack = [-math.inf]

    def push(self, x):
        """
        :type x: int
        :rtype: void
        """
        self.stack.append(x)
        self.max_stack.append(max(x, self.max_stack[-1]) if self.max_stack else
e x)

    def pop(self):
        """
        :rtype: int
        """
        if len(self.stack) != 0:
            self.max_stack.pop(-1)
            return self.stack.pop(-1)

    def top(self):
        """
        :rtype: int

```

```

        return self.stack[-1] if self.stack else None

def peekMax(self):
    """
    :rtype: int
    """
    if len(self.max_stack) != 0:
        return self.max_stack[-1]

```

```

def popMax(self):
    """
    :rtype: int
    """
    val = self.peekMax()
    buff = []
    while self.top() != val:
        # re-use pop(), ops on both max_stack and stack
        buff.append(self.pop())
    self.pop()

```

```

    while len(buff) != 0:
        # re-use push(), no need to save buffer for max-stack
        self.push(buff.pop(-1))
    return val

```

```
#####
```

```
from sortedcontainers import SortedList
```

```
class Node:
```

```

    def __init__(self, val=0):
        self.val = val
        self.prev: Union[Node, None] = None
        self.next: Union[Node, None] = None

```

```

class DoubleLinkedList:
    def __init__(self):
        self.head = Node()
        self.tail = Node()

```

```
self.head.next = self.tail
self.tail.prev = self.head
```

```
def append(self, val) -> Node:
    node = Node(val)
    node.next = self.tail
    node.prev = self.tail.prev
    self.tail.prev = node
    node.prev.next = node
    return node
```

```
@staticmethod
```

```
def remove(node) -> Node:
    node.prev.next = node.next
    node.next.prev = node.prev
    return node
```

```
def pop(self) -> Node:
    return self.remove(self.tail.prev)
```

```
def peek(self):
    return self.tail.prev.val
```

```
class MaxStack:
```

```
    def __init__(self):
        self.stk = DoubleLinkedList()
        self.sl = SortedList(key=lambda x: x.val)
```

```
    def push(self, x: int) -> None:
```

```
        node = self.stk.append(x)
        self.sl.add(node)
```

```
    def pop(self) -> int:
        node = self.stk.pop()
        self.sl.remove(node)
        return node.val
```

```
    def top(self) -> int:
        return self.stk.peek()
```



```
def peekMax(self) -> int:
    return self.sl[-1].val

def popMax(self) -> int:
    node = self.sl.pop()
    DoubleLinkedList.remove(node)
    return node.val
```

```
# Your MaxStack object will be instantiated and called as such:
# obj = MaxStack()
# obj.push(x)
# param_2 = obj.pop()
# param_3 = obj.top()
# param_4 = obj.peekMax()
# param_5 = obj.popMax()
```

◆ Quick Review

Key Insights

- Use a doubly-linked list to support fast insertion and removal from the stack.
- Maintain a balanced tree (or ordered dictionary/multiset) to quickly locate the maximum element in $O(\log n)$ time.
- Map each value to its corresponding node(s) in the doubly-linked list, so that when popMax is called, you can immediately remove the corresponding node from the list.

- The double-linking allows removal of arbitrary nodes in $O(1)$, once you have the reference.

Space & Time

Time Complexity:

push: $O(\log n)$ due to insertion in the ordered structure.

pop: $O(1)$ for removal from the doubly-linked list, plus $O(\log n)$ for updating the tree.

top: $O(1)$

peekMax: $O(1)$ or $O(\log n)$ depending on the tree implementation.

popMax: $O(\log n)$ for looking up and removal from the tree, $O(1)$ for doubly-linked list deletion.

Space Complexity:

$O(n)$ for storing all nodes in the doubly-linked list and the tree structure mapping values to nodes.

Solution

We use two data structures: A doubly-linked list to store the elements in stack order. This supports fast push and pop operations and

allows $O(1)$ removal when given a reference. An ordered structure (like a balanced BST, TreeMap in Java, or SortedList in Python) that maps each value to a list of nodes that hold that value in the linked list. This allows us to quickly determine the maximum element. When there are duplicates, we remove the one closest to the top by storing nodes in order. The trick is to keep these two structures synchronized. When an element is pushed, add its node to both the doubly-linked list and the ordered structure. When an element is removed (either via pop or popMax), remove its node from both structures.

Stack

□ ★ #772 Basic Calculator III ★

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · String · Stack · Recursion
Lists: ★ Premium | Premium 98

Problem

Implement a basic calculator to evaluate a simple expression string.

The expression string contains only non-negative integers, '+', '-', '*', '/' operators, and open '(' and closing parentheses ')'. The integer division should truncate toward zero. You may assume that the given expression is always valid. All intermediate results will be in the range of $[-2^{31}, 2^{31} - 1]$.

Note: You are not allowed to use any built-in function which evaluates strings as mathematical expressions, such as `eval()`.

Example 1:

```
Input: s = "1+1"  
Output: 2
```

Example 2:

Input: s = "6-4/2"
Output: 4

Example 3:

Input: s = "2*(5+5*2)/3+(6/2+8)"
Output: 21

Constraints:

- $1 \leq s \leq 104$
- s consists of digits, '+', '-', '*', '/', '(', and ')'.
Note: s will not contain any spaces.
- s is a valid expression.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Evaluate an expression with +, -, *, /, and parentheses. Non-negative integers.

Division truncates toward zero. No eval(). Right?"

#

"A few quick questions:

Can expressions be nested arbitrarily deep? Yes. Spaces? No per constraints."

#

"Let me walk: '1+1'→2 ✓. '6-4/2'→4 ✓. '2*(5+5*2)/3+(6/2+8)'→21 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Recursive descent: parse number or sub-expression in parentheses,

apply operators respecting * / before + -.

O(n) time, O(d) recursion depth.

Should I go ahead and optimize?"

```
class _772_BasicCalculatorIII_Brute:
```

```
    def calculate(self, s):
```

```
        self.i = 0
```

```
        def parse():
```

```
            stack = []; sign = '+'; num = 0
```

```
            while self.i < len(s):
```

```

        ch = s[self.i]; self.i += 1
        if ch.isdigit(): num = num*10 + int(ch)
        if ch == '(': num = parse()
        if (not ch.isdigit() and ch != '(') or self.i == len(s):
            if sign=='+': stack.append(num)
            elif sign=='-': stack.append(-num)
            elif sign=='*': stack.append(stack.pop()*num)
            else: stack.append(int(stack.pop()/num))
            sign = ch; num = 0
        if ch == ')': break
    return sum(stack)
return parse()

```

PHASE 3 — OPTIMIZE

"The recursive approach already handles precedence correctly.

The bottleneck is the recursion stack – $O(d)$ depth for nesting level d .

We can make it iterative but the recursive version is already $O(n)$ time and $O(d)$ space.

Let's clean up the recursive implementation for clarity.

#

Time: $O(n)$. Space: $O(d)$ recursion depth."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _772_BasicCalculatorIII:
```

```
    def calculate(self, s):
```

```
        self.pos = 0
```

```
        def parse():
```

```
            stack = []
```

```
            sign = '+'
```

```
            num = 0
```

```
            while self.pos < len(s):
```

```
                ch = s[self.pos]; self.pos += 1
```

```
                if ch.isdigit():
```

```
                    num = num * 10 + int(ch)
```

```
                elif ch == '(':
```

```
                    num = parse()
```

```
            if ch in '+-*/') or self.pos == len(s):
```

```
                if sign == '+': stack.append(num)
```

```
                elif sign == '-': stack.append(-num)
```

```
                elif sign == '*': stack.append(stack.pop() * num)
```

```
                elif sign == '/': stack.append(int(stack.pop() / num))
```

```
                sign = ch
```

```
                num = 0
```

```
            if ch == ')':
```

```
                break
```

```
            return sum(stack)
```

```
        return parse()

```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# '2*(5+5*2)/3+(6/2+8)':
# parse(): '2'→num=2. '*'→push 2, sign='*'. '('→num=parse():
# inner: '5'→5. '+'→push 5,sign='+'. '5'→5. '*'→push 5,sign='*'. '2'→2.
# ')'→sign='*'→pop5*2=10,push10. sum=15. return 15.
# num=15, sign='*'→pop2*15=30,push30. '/'→sign='/'. '3'→3.
# '+'→int(30/3)=10,push10. sign='+'. '('→parse()→inner: 6/2=3, 3+8=11. return 11.
# num=11, sign='+'→push11. pos==len→sum=21 ✓
#
# "No bugs. Precedence handled by stack: * and / computed immediately, + and -
deferred."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): one pass. Space O(d) recursion + O(n) operator stack.
# Follow-up: Basic Calculator I (#224) – only + and -.
# Follow-up: Basic Calculator II (#227) – + - * / but no parentheses."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def calculate(self, s: str) -> int:
        nums = []
        ops = []

        def calc():
            b = nums.pop()
            a = nums.pop()
            op = ops.pop()
            if op == '+':

                nums.append(a + b)
            elif op == '-':
                nums.append(a - b)
            elif op == '*':
                nums.append(a * b)
            else: # op == '/'
                nums.append(int(a / b))

        def precedes(prev: str, curr: str) -> bool:
            """
```


Returns True if the previous character is a operator and the priority of the previous operator $\geq$ the priority of the current character (operator).

```

    """
    if prev == '(':
        return False
    return prev in '*/' or curr in '+-'

i = 0
hasPrevNum = False

while i < len(s):
    c = s[i]
    if c.isdigit():
        num = int(c)
        while i + 1 < len(s) and s[i + 1].isdigit():
            num = num * 10 + int(s[i + 1])
            i += 1
        nums.append(num)
        hasPrevNum = True
    elif c == '(':
        ops.append('(')
        hasPrevNum = False
    elif c == ')':
        while ops[-1] != '(':
            calc()
        ops.pop() # Pop '('
    elif c in '+-*/*':
        if not hasPrevNum: # Handle input like "-1-(-1)"
            num.append(0)
        while ops and precedes(ops[-1], c):

            calc()
            ops.append(c)
            i += 1

while ops:
    calc()

return nums.pop()

```

• LeetCode

```
class Solution:
    def calculate(self, s: str) -> int:
        def dfs(q):
            num, sign, stk = 0, "+", []
            while q:
                c = q.popleft()
                if c.isdigit():
                    num = num * 10 + int(c)
                if c == "(":
                    num = dfs(q)
                if c in "+-*/" or not q:
                    match sign:
                        case "+":
                            stk.append(num)
                        case "-":
                            stk.append(-num)
                        case "*":
                            stk.append(stk.pop() * num)
                        case "/":
                            stk.append(int(stk.pop() / num))
                    num, sign = 0, c
                if c == ")":
                    break
            return sum(stk)

        return dfs(deque(s))
```

• SimplyLeet

```

def calculate(s: str) -> int:
    # Recursive helper function that processes the expression starting from
    index 'i'
    def helper(i):
        stack = []
        num = 0
        sign = '+'
        while i < len(s):
            char = s[i]
            # Build the current number if the character is a digit
            if char.isdigit():

                num = num * 10 + int(char)
            # If '(' is encountered, evaluate the subexpression recursively
            if char == '(':
                num, i = helper(i + 1)
            # When encountering an operator or closing parenthesis, process the
            sign and number
            if (not char.isdigit() and char != ' ') or i == len(s) - 1:
                if sign == '+':
                    stack.append(num)
                elif sign == '-':
                    stack.append(-num)

                elif sign == '*':
                    prev = stack.pop()
                    stack.append(prev * num)
                elif sign == '/':
                    prev = stack.pop()
                    stack.append(int(prev / num)) # Truncate toward zero
                sign = char
                num = 0
            # If a closing parenthesis is reached, end this level of recursion
            if char == ')':

```

```

        return sum(stack), i
    i += 1
    return sum(stack), i

result, _ = helper(0)
return result

# Example test
print(calculate("2*(5+5*2)/3+(6/2+8)"))

```

• LC.ca

```

# good one, based on solution of Basic Calculator II
class Solution: # dfs recursion
    def calculate(self, s: str) -> int:
        stack = []
        num = 0
        op = "+"
        i = 0
        while i < len(s):
            c = s[i]
            if c.isdigit():
                num = num * 10 + int(c)
            elif c == "(":
                j = self.findClosing(s, i)
                num = self.calculate(s[i+1:j]) # j is ')'
                i = j # out if, i will +1 below
            if c in "+-*/" or i == len(s) - 1:
                if op == "+":
                    stack.append(num)
                elif op == "-":
                    stack.append(-num)

```

```

        elif op == "*":
            stack[-1] *= num
        else:
            stack[-1] = int(stack[-1] / num)
        op = c
        num = 0
    i += 1
    return sum(stack)

def findClosing(self, s: str, i: int) -> int:

```

```

    level = 0
    while i < len(s):
        if s[i] == "(":
            level += 1
        elif s[i] == ")":
            level -= 1
            if level == 0:
                return i
        i += 1
    return i

```

```
#####
```

```

class Solution: # iterative
    def calculate(self, s: str) -> int:
        def evaluate_expression(stack):
            res = stack.pop() if stack else 0
            # Evaluate the expression till we get '(' or empty stack

```

```

        while stack and stack[-1] != '(':
            sign = stack.pop()
            if sign == '+':
                res += stack.pop()
            elif sign == '-':
                res -= stack.pop()
            elif sign == '*':
                res *= stack.pop()
            elif sign == '/':
                res //= stack.pop()

```

```

        return res

    stack = [] # mix of num and operator
    n = len(s)
    i = 0
    while i < n:
        if s[i].isdigit():
            # Get the complete number and push it to the stack
            j = i
            while j < n and s[j].isdigit():

                j += 1
            stack.append(int(s[i:j]))
            i = j - 1
        elif s[i] in '+-*/(':
            stack.append(s[i])
        elif s[i] == ')':
            # Evaluate the expression within the parentheses
            res = evaluate_expression(stack)
            stack.pop() # Remove the opening '('
            stack.append(res)

        i += 1

    return evaluate_expression(stack)

#####

class Solution:
    def calculate(self, s: str) -> int:
        if not s:

            return 0

        # remove leading and trailing spaces and white spaces.
        s = s.strip().replace(" ", "")

        if not s:
            return 0

        op_stack = []
        num_stack = []

```

```
i = 0
while i < len(s):
    if s[i].isdigit():
        num = 0
        while i < len(s) and s[i].isdigit():
            num = num * 10 + int(s[i])
            i += 1
        num_stack.append(num)
    else:
        op = s[i]
        if not op_stack:
            op_stack.append(op)
            i += 1
        elif op == '+' or op == '-':
            top = op_stack[-1]
            if top == '(':
                op_stack.append(op)
                i += 1
            else:
                self.calculate_helper(num_stack, op_stack)
        elif op == '*' or op == '/':
            top = op_stack[-1]
            if top == '(' or top == '+' or top == '-':
                op_stack.append(op)
                i += 1
            else:
                self.calculate_helper(num_stack, op_stack)
        elif op == '(':
            op_stack.append(op)

        i += 1
    elif op == ')':
        while op_stack[-1] != '(':
            self.calculate_helper(num_stack, op_stack)
        op_stack.pop()
        i += 1

while op_stack:
    self.calculate_helper(num_stack, op_stack)
```

```
        return num_stack[-1]

    def calculate_helper(self, num_stack, op_stack):
        num2 = num_stack.pop()
        num1 = num_stack.pop()

        op = op_stack.pop()

        if op == '+':
            ans = num1 + num2

        elif op == '-':
            ans = num1 - num2
        elif op == '*':
            ans = num1 * num2
        else:
            ans = int(num1 / num2)

        num_stack.append(ans)
```

◆ Quick Review

Key Insights

- Use recursion to handle nested expressions defined by parentheses.
- A stack can be used to correctly evaluate the expression and respect operator precedence.
- Multiplication and division have higher precedence than addition and subtraction.
- When encountering an open parenthesis, recursively compute its value until a closing parenthesis is found.

- Use immediate calculation for multiplication and division by updating the top of the stack before moving to the next operator.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string, as every character is processed.

Space Complexity: $O(n)$ due to the recursion stack or auxiliary stack used for intermediate results.

Solution

We use a recursive approach combined with a stack to solve the problem. As we iterate through the string: Convert digits into numbers. When encountering an operator or parenthesis, process the previously accumulated number based on the preceding operator. For multiplication or division, pop the last number from the stack, apply the operation, and push the result back. When a '(' is encountered, recursively evaluate the substring until the corresponding ')'. After processing all tokens, sum the values in the stack to get the final result. This method correctly handles both operator precedence

and nested expressions.

Stack

□ #895 Maximum Frequency Stack

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Stack · Design
Lists: Grind 169

Problem

Design a stack-like data structure to push elements to the stack and pop the most frequent element from the stack.

Implement the FreqStack class:

- FreqStack() constructs an empty frequency stack.
- void push(int val) pushes an integer val onto the top of the stack.
- int pop() removes and returns the most frequent element in the stack. If there is a tie for the most frequent element, the element closest to the stack's top is removed and returned.

Example 1:

Input

```
["FreqStack", "push", "push", "push", "push", "push", "push", "pop", "pop", "pop", "pop"]
```

```
[[], [5], [7], [5], [7], [4], [5], [], [], [], []]
```

Output

```
[null, null, null, null, null, null, null, 5, 7, 5, 4]
```

Explanation

```
FreqStack freqStack = new FreqStack();
```

Constraints:

- $0 \leq \text{val} \leq 109$
- At most $2 * 10^4$ calls will be made to push and pop.
- It is guaranteed that there will be at least one element in the stack before calling pop.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Design a stack-like structure. push(val) adds an element.

pop() removes and returns the most frequent element.

If tie, pop the most recently pushed among those tied. Both O(1). Right?"

#

"A few quick questions:

Can push be called with the same value many times? Yes – frequency grows.

pop is only called when non-empty? Yes."

#

"Let me walk: push(5,7,5,7,4,5). pop()→5(freq=3). pop()→7(freq=2). pop()→5 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

List of all pushed elements in order. On pop: find max frequency element,

then most recent among those. O(n) per pop.

Should I go ahead and optimize?"

```
class _895_MaxFrequencyStack_Brute:
```

```
    def __init__(self): self.data = []
```

```
def push(self, val): self.data.append(val)
def pop(self):
    from collections import Counter
    freq = Counter(self.data)
    max_f = max(freq.values())
    for i in range(len(self.data)-1, -1, -1):
        if freq[self.data[i]] == max_f:
            return self.data.pop(i)
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the O(n) scan for the most frequent element.
# Two dicts: freq[val] = current frequency. group[freq] = stack of vals at that
# frequency.
# max_freq tracks current maximum frequency.
# push: increment freq[val], append to group[freq[val]], update max_freq.
# pop: pop from group[max_freq]. If group[max_freq] empty, decrement max_freq.
#
# Time: O(1) both ops. Space: O(n)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
from collections import defaultdict
class _895_MaxFrequencyStack:
    def __init__(self):
        self.freq = defaultdict(int)
        self.group = defaultdict(list)
        self.max_freq = 0

    def push(self, val):
        self.freq[val] += 1
        f = self.freq[val]
        self.max_freq = max(self.max_freq, f)
        self.group[f].append(val)

    def pop(self):
        val = self.group[self.max_freq].pop()
        self.freq[val] -= 1
        if not self.group[self.max_freq]:
            self.max_freq -= 1
        return val
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# push(5,7,5,7,4,5):
# push5: freq={5:1},group={1:[5]},max=1.
# push7: freq={5:1,7:1},group={1:[5,7]},max=1.
# push5: freq={5:2},group={1:[5,7],2:[5]},max=2.
# push7: freq={7:2},group={2:[5,7]},max=2.
# push4: freq={4:1},group={1:[5,7,4]},max=2.
# push5: freq={5:3},group={3:[5]},max=3.
# pop(): group[3]=[5]→pop5. freq[5]=2. group[3]=[]→max=2. return 5 ✓
```

```
# pop(): group[2]=[5,7]→pop7. freq[7]=1. return 7 ✓
# pop(): group[2]=[5]→pop5. freq[5]=1. group[2]=[]→max=1. return 5 ✓
#
# "No bugs. group stacks maintain LRU order within each frequency level
# automatically."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "push and pop both O(1). Space O(n) across both dicts.
# max_freq only decreases when group[max_freq] drains – never increases on pop.
# Follow-up: peek without removing – group[max_freq][-1].
# Follow-up: if we need O(log n) guarantee – use a sorted dict keyed by (freq,
# insertion_time)."
```

Community Solutions (Python)

• WalkCC

```
class FreqStack:
    def __init__(self):
        self.maxFreq = 0
        self.count = collections.Counter()
        self.countToStack = collections.defaultdict(list)

    def push(self, val: int) -> None:
        self.count[val] += 1
        self.countToStack[self.count[val]].append(val)
        self.maxFreq = max(self.maxFreq, self.count[val])

    def pop(self) -> int:
        val = self.countToStack[self.maxFreq].pop()
        self.count[val] -= 1
        if not self.countToStack[self.maxFreq]:
            self.maxFreq -= 1
        return val
```

• LeetDoocs

```

class FreqStack:
    def __init__(self):
        self.cnt = defaultdict(int)
        self.q = []
        self.ts = 0

    def push(self, val: int) -> None:
        self.ts += 1
        self.cnt[val] += 1
        heappush(self.q, (-self.cnt[val], -self.ts, val))

    def pop(self) -> int:
        val = heappop(self.q)[2]
        self.cnt[val] -= 1
        return val

# Your FreqStack object will be instantiated and called as such:
# obj = FreqStack()
# obj.push(val)

# param_2 = obj.pop()

```

```

class FreqStack:
    def __init__(self):
        self.cnt = defaultdict(int)
        self.d = defaultdict(list)
        self.mx = 0

    def push(self, val: int) -> None:
        self.cnt[val] += 1
        self.d[self.cnt[val]].append(val)
        self.mx = max(self.mx, self.cnt[val])

```



```
def pop(self) -> int:
    val = self.d[self.mx].pop()
    self.cnt[val] -= 1
    if not self.d[self.mx]:
        self.mx -= 1
    return val
```

Your FreqStack object will be instantiated and called as such:

```
# obj = FreqStack()
# obj.push(val)
# param_2 = obj.pop()
```

• SimplyLeet

Python implementation of FreqStack

```
class FreqStack:
    def __init__(self):
        # Dictionary mapping value to its frequency
        self.freq = {}
        # Dictionary mapping frequency to a list (stack) of values
        self.group = {}
        # Variable tracking the current maximum frequency in the stack
        self.maxfreq = 0
```

```
def push(self, val: int) -> None:
    # Update frequency count for the value
    f = self.freq.get(val, 0) + 1
    self.freq[val] = f
    # Update the maximum frequency if necessary
    if f > self.maxfreq:
        self.maxfreq = f
    # Append the value to the appropriate frequency group
    if f not in self.group:
```

```

        self.group[f] = []
        self.group[f].append(val)

def pop(self) -> int:
    # Remove the value from the stack of the current maximum frequency
    val = self.group[self.maxfreq].pop()
    # Decrement the frequency count for the value
    self.freq[val] -= 1
    # If no more elements have the current max frequency, decrease maxfreq
    if not self.group[self.maxfreq]:

        self.maxfreq -= 1
    return val

```

• LC.ca

```

class FreqStack:
    def __init__(self):
        self.cnt = defaultdict(int)
        self.d = defaultdict(list)
        self.mx = 0

    def push(self, val: int) -> None:
        self.cnt[val] += 1
        self.d[self.cnt[val]].append(val)
        self.mx = max(self.mx, self.cnt[val])

    def pop(self) -> int:
        val = self.d[self.mx].pop()
        self.cnt[val] -= 1
        if not self.d[self.mx]:
            self.mx -= 1
        return val

# Your FreqStack object will be instantiated and called as such:

# obj = FreqStack()
# obj.push(val)
# param_2 = obj.pop()

```

◆ Quick Review

Key Insights

- Use a hash map to count the frequency of each element.
- Maintain another hash map that maps frequencies to stacks (lists) of elements.
- Track the current maximum frequency to enable $O(1)$ retrieval of the most frequent element.
- When pushing an element, update its frequency and add it to the corresponding frequency stack.
- When popping an element, remove it from the stack corresponding to the maximum frequency and update the frequency count; if that stack becomes empty, decrease the current maximum frequency.

Space & Time

Time Complexity: $O(1)$ per push and pop.

Space Complexity: $O(n)$, where n is the number of elements pushed onto the stack.

Solution

The solution involves using two main data structures: A hash map (freq) to store the frequency count for each value. A hash map (group) that maps a frequency to a stack (list) of values that have that frequency. During a push: Increment the frequency count of the value. Push the value onto the stack corresponding to its updated frequency. Update the maximum frequency if necessary. During a pop: Pop the top element from the stack corresponding to the current maximum frequency. Decrement its frequency count in the freq map. If the frequency stack becomes empty after popping, decrement the current maximum frequency. This design enables the retrieval of the most frequent element in constant time while dealing with frequency ties by returning the element closest to the top.

Heap

Round 6 · Mid · Hard · 9 questions

Heap

□ #23 Merge k Sorted Lists

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Linked List · Divide and Conquer · Heap ·
Merge Sort
Lists: Grind 169

Problem

You are given an array of k linked-lists lists, each linked-list is sorted in ascending order. Merge all the linked-lists into one sorted linked-list and return it.

Example 1:

```
Input: lists = [[1,4,5],[1,3,4],[2,6]]
Output: [1,1,2,3,4,4,5,6]
Explanation: The linked-lists are:
[
  1->4->5,
  1->3->4,
  2->6
]
```

Example 2:

```
Input: lists = []
Output: []
```

Example 3:

Input: lists = [[]]

Output: []

Constraints:

- $k == \text{lists.length}$
- $0 \leq k \leq 104$
- $0 \leq \text{lists}[i].\text{length} \leq 500$
- $-104 \leq \text{lists}[i][j] \leq 104$
- $\text{lists}[i]$ is sorted in ascending order.
- The sum of $\text{lists}[i].\text{length}$ will not exceed 104.

My LeetCode Solution

• My LeetCode Solution

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next

import heapq

class Solution:
    def mergeKLists(self, lists):
```



```

# Create a min-heap
min_heap = []
# Heap entry: (node value, unique id, node) to handle duplicate values
for i, node in enumerate(lists):
    if node:
        heapq.heappush(min_heap, (node.val, i, node))

# Dummy node to form the start of merged list
dummy = ListNode(0)
current = dummy

# While there are elements in the heap
while min_heap:
    # Pop the smallest node from the heap
    val, idx, node = heapq.heappop(min_heap)
    # Add the smallest node to the merged list
    current.next = node
    current = current.next
    # If there is a next node in the list, push it into the heap
    if node.next:
        heapq.heappush(min_heap, (node.next.val, idx, node.next))

return dummy.next

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Let me make sure I understand the problem correctly.
# Given k sorted linked lists, merge them into one sorted linked list. Right?"
#
# "A few quick questions:
# k can be 0 (empty input)? Yes – return None. Lists can be empty? Yes."
#
# "Let me walk: lists=[[1,4,5],[1,3,4],[2,6]] → [1,1,2,3,4,4,5,6] ✓"

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with brute force to lock in the logic.
# Collect all values from all lists, sort, rebuild one list.
# O(n log n) where n = total nodes. Space: O(n).
# Should I go ahead and optimize?"
class _23_MergeKSortedLists_Brute:
    def mergeKLists(self, lists):

```

```

vals = []
for node in lists:
    while node: vals.append(node.val); node = node.next
dummy = cur = ListNode(0)
for v in sorted(vals): cur.next = ListNode(v); cur = cur.next
return dummy.next

```

PHASE 3 — OPTIMIZE

"The bottleneck is building all values then sorting – we discard the sorted structure."

Min-heap of (value, list_index, node): always extract the globally smallest node.

Push each list's head. Pop min, add to result, push its next.

#

Time: $O(n \log k)$: n nodes, heap of size k for $\log k$ per pop/push.

Space: $O(k)$ heap + $O(n)$ output."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
import heapq
```

```
class _23_MergeKSortedLists:
```

```
    def mergeKLists(self, lists):
```

```
        heap = []
```

```
        for i, node in enumerate(lists):
```

```
            if node:
```

```
                heapq.heappush(heap, (node.val, i, node))
```

```
        dummy = cur = ListNode(0)
```

```
        while heap:
```

```
            val, i, node = heapq.heappop(heap)
```

```
            cur.next = node; cur = cur.next
```

```
            if node.next:
```

```
                heapq.heappush(heap, (node.next.val, i, node.next))
```

```
        return dummy.next
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

lists=[[1,4,5],[1,3,4],[2,6]]:

heap: [(1,0,1node),(1,1,1node),(2,2,2node)].

pop(1,0,1): attach. push(4,0,4). heap=[(1,1,1),(2,2,2),(4,0,4)].

pop(1,1,1): attach. push(3,1,3). ... → [1,1,2,3,4,4,5,6] ✓

#

lists=[]: no pushes → return None ✓

#

"No bugs. Storing the list index i as a tiebreaker avoids comparing ListNode objects."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log k)$: each of n nodes pushed/popped, heap stays at size k .

Space $O(k)$ heap – far better than $O(n)$ from the brute sort.

Divide-and-conquer: pair lists and merge pairs repeatedly – also $O(n \log k)$.

Follow-up: $k=2$ reduces to LC 21 Merge Two Sorted Lists.

Follow-up: merge k sorted ARRAYS – same heap approach with (val, list_idx, elem_idx)."

Community Solutions (Python)

• WalkCC

```
from queue import PriorityQueue

class Solution:
    def mergeKLists(self, lists: list[ListNode]) -> ListNode:
        dummy = ListNode(0)
        curr = dummy
        pq = PriorityQueue()

        for i, lst in enumerate(lists):

            if lst:
                pq.put((lst.val, i, lst))

        while not pq.empty():
            _, i, minNode = pq.get()
            if minNode.next:
                pq.put((minNode.next.val, i, minNode.next))
            curr.next = minNode
            curr = curr.next

        return dummy.next
```

• LeetDoocs

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
        setattr(ListNode, "__lt__", lambda a, b: a.val < b.val)
        pq = [head for head in lists if head]
        heapify(pq)

        dummy = cur = ListNode()
        while pq:
            node = heappop(pq)
            if node.next:
                heappush(pq, node.next)
            cur.next = node
            cur = cur.next
        return dummy.next
```

• SimplyLeet

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

import heapq

class Solution:
    def mergeKLists(self, lists):
```

```

# Create a min-heap
min_heap = []
# Heap entry: (node value, unique id, node) to handle duplicate values
for i, node in enumerate(lists):
    if node:
        heapq.heappush(min_heap, (node.val, i, node))

# Dummy node to form the start of merged list
dummy = ListNode(0)
current = dummy

# While there are elements in the heap
while min_heap:
    # Pop the smallest node from the heap
    val, idx, node = heapq.heappop(min_heap)
    # Add the smallest node to the merged list
    current.next = node
    current = current.next
    # If there is a next node in the list, push it into the heap
    if node.next:
        heapq.heappush(min_heap, (node.next.val, idx, node.next))

return dummy.next

```

• LC.ca

```

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
        setattr(ListNode, "__lt__", lambda a, b: a.val < b.val)
        pq = [head for head in lists if head]
        heapify(pq)

```

```

dummy = cur = ListNode()
while pq:
    node = heappop(pq)
    if node.next:
        heappush(pq, node.next)
    cur.next = node
    cur = cur.next
return dummy.next

```

```

# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next

```

```

#####

```

```

import heapq

```

```

'''
__lt__(self, other) for <
__le__(self, other) for <=
__gt__(self, other) for >
__ge__(self, other) for >=
'''

```

```

'''

```

```

# or create a wrapper class, without modifying existing node class
class NodeWrapper:
    def __init__(self, node):
        self.node = node

    def __lt__(self, other):
        return self.node.val < other.node.val

'''

```

```
# overwrite the comparison function, so the node can be comparable
# or else, error, TypeError: '<' not supported between instances of 'ListNode'
and 'ListNode'

# define 'gt' will also work, so either lt or gt will do the compare job for
ListNode
# ListNode.__gt__ = lambda x, y: (x.val > y.val)

ListNode.__lt__ = lambda x, y: (x.val < y.val)

class Solution:
```

```
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
        dummy = current = ListNode()
        heap = []
        for i, node in enumerate(lists):
            if node:
                # need to override
                # or else error: TypeError: '<' not supported between instances
of 'ListNode' and 'ListNode'
                heapq.heappush(heap, node)

        while heap:

            anode = heapq.heappop(heap)

            current.next = anode
            current = current.next

            if anode.next:
                heapq.heappush(heap, anode.next)

        return dummy.next
```

```

    ...
    tried to use node.val to order heap, but still got error TypeError: '<' not
    supported between instances of 'ListNode' and 'ListNode':

    heapq.heappush(heap, (node.val, node))

    ...

    #####

    # based on merge 2 lists

# note for empty input
class Solution:
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
        if lists is None or len(lists) == 0:
            return None
        return self.mergeKListsHelper(lists, 0, len(lists) - 1)

    def mergeKListsHelper(self, lists: List[Optional[ListNode]], start: int, end: int) -> Optional[ListNode]:
        if start == end:
            return lists[start]

        mid = int((start + end) / 2)

        left = self.mergeKListsHelper(lists, start, mid)
        # print(left)

        right = self.mergeKListsHelper(lists, mid + 1, end)
        # print(right)

```



```

        return self.mergeTwoLists(left, right)

    def mergeTwoLists(self, list1: Optional[ListNode], list2: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode()
        current = dummy
        # print("merging: ", list1)
        # print("merging: ", list2)
        while list1 or list2:
            v1 = list1.val if list1 else float('inf')
            v2 = list2.val if list2 else float('inf')

            if v1 < v2:
                current.next = list1
                list1 = list1.next
            else:
                current.next = list2
                list2 = list2.next
            current = current.next

        # print("merging result: ", dummy.next)

        return dummy.next

#####

# Definition for singly-linked list.
# class ListNode:
# def __init__(self, val=0, next=None):
# self.val = val
# self.next = next

```

```
class Solution:
    def mergeKLists(self, lists: List[ListNode]) -> ListNode:
        n = len(lists)
        if n == 0:
            return None
        for i in range(n - 1): # this is o(N) times of merge, while if use
            # divide-mid-merge then it's o(logN) times of merge
            lists[i + 1] = self.mergeTwoLists(lists[i], lists[i + 1])
        return lists[-1]

    def mergeTwoLists(self, l1: ListNode, l2: ListNode) -> ListNode:
        dummy = ListNode()
        cur = dummy
        while l1 and l2:
            if l1.val <= l2.val:
                cur.next = l1
                l1 = l1.next
            else:
                cur.next = l2
                l2 = l2.next
            cur = cur.next

        cur.next = l1 or l2
        return dummy.next

if __name__ == '__main__':
    l1 = ListNode(1)
    l1.next = ListNode(4)
    l1.next.next = ListNode(5)
```

```
l2 = ListNode(1)
l2.next = ListNode(3)
l2.next.next = ListNode(4)

l3 = ListNode(2)
l3.next = ListNode(6)

print(Solution().mergeKLists([l1, l2, l3]))

print(Solution().mergeKLists([]))
print(Solution().mergeKLists([[]]))
```

◆ Quick Review

Key Insights

- Use a min-heap (priority queue) to efficiently extract the smallest value among the current heads of the lists.
- By pushing the head of each non-empty list into the heap, we can pull the smallest node, then push that node's next element into the heap.
- Alternatively, a divide and conquer approach can be used by recursively merging pairs of lists.
- Careful management of pointers (or references) is key to constructing the new linked list.

Space & Time

Time Complexity: $O(N \log k)$ where N is the total number of nodes across all lists and k is the number of lists.

Space Complexity: $O(k)$ due to the heap containing at most one node from each list.

Solution

We use a min-heap to always retrieve the smallest node among the current nodes in the linked lists. First, initialize the heap with the head of each list. Then, continuously extract the smallest node from the heap, attach it to the merged list, and if the extracted node has a next node, insert that into the heap. The process repeats until the heap is empty.

Heap

□ #218 The Skyline Problem

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Divide and Conquer · Binary Indexed
Tree · Segment Tree · Line Sweep · Heap ·
Ordered Set

Lists: Denny Zhang

Problem

A city's skyline is the outer contour of the silhouette formed by all the buildings in that city when viewed from a distance. Given the locations and heights of all the buildings, return the skyline formed by these buildings collectively.

The geometric information of each building is given in the array `buildings` where `buildings[i] = [lefti, righti, heighti]`:

- `lefti` is the x coordinate of the left edge of the *i*th building.
- `righti` is the x coordinate of the right edge of the *i*th building.

- **heighti** is the height of the **ith** building.

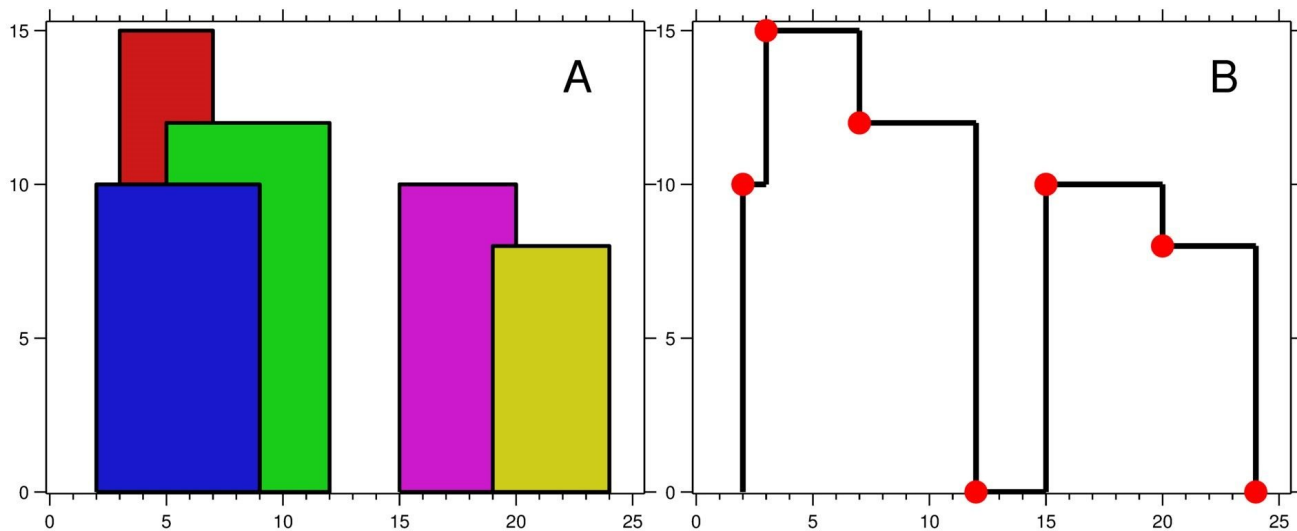
You may assume all buildings are perfect rectangles grounded on an absolutely flat surface at height 0.

The skyline should be represented as a list of "key points" sorted by their x-coordinate in the form `[[x1,y1],[x2,y2],...]`. Each key point is the left endpoint of some horizontal segment in the skyline except the last point in the list, which always has a y-coordinate 0 and is used to mark the skyline's termination where the rightmost building ends. Any ground between the leftmost and rightmost buildings should be part of the skyline's contour.

Note: There must be no consecutive horizontal lines of equal height in the output skyline. For instance, `[...,[2 3],[4 5],[7 5],[11 5],[12 7],...]` is not acceptable; the three lines of height 5 should be merged into one in the final output as such:

`[...,[2 3],[4 5],[12 7],...]`

Example 1:



Input: buildings = [[2,9,10],[3,7,15],[5,12,12],[15,20,10],[19,24,8]]

Output: [[2,10],[3,15],[7,12],[12,0],[15,10],[20,8],[24,0]]

Explanation:

Figure A shows the buildings of the input.

Figure B shows the skyline formed by those buildings. The red points in figure B represent the key points in the output list.

Example 2:

Input: buildings = [[0,2,3],[2,5,3]]

Output: [[0,3],[5,0]]

Constraints:

- $1 \leq \text{buildings.length} \leq 104$
- $0 \leq \text{left}_i < \text{right}_i \leq 231 - 1$
- $1 \leq \text{height}_i \leq 231 - 1$
- buildings is sorted by left_i in non-decreasing order.

My LeetCode Solution

- My LeetCode Solution


```

class Solution:
    def getSkyline(self, buildings):
        # List to store all events: (x, height, right)
        events = []
        # Create events for all buildings: left edge (start) and right edge
        (end)
        for left, right, height in buildings:
            # For the start point, height is negative for proper sorting (start
            events come before end events)
            events.append((left, -height, right))
            # For the end point, height is 0 to indicate removal of building
            from active heap
            events.append((right, 0, 0))

        # Sort events by x coordinate.
        events.sort()

        # Max heap to store active buildings (using negative heights)
        result = []
        # Initialize the heap with a dummy building of height 0 that extends
        infinitely
        heap = [(0, float('inf'))]

        for x, neg_height, right in events:

            # Remove buildings from the heap that have ended by current x.
            while heap and heap[0][1] <= x:
                heapq.heappop(heap)

            # If event is a start event (neg_height != 0), push it into the
            heap.
            if neg_height != 0:
                heapq.heappush(heap, (neg_height, right))

            # Current highest building height is at the top of the heap
            current_height = -heap[0][0]

```

```

        # If the result is empty or current height differs from last
        recorded height, add a new key point.
        if not result or current_height != result[-1][1]:
            result.append([x, current_height])

    return result

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Let me make sure I understand the problem correctly.
# Given buildings as [left, right, height], output the skyline as [x, height] points
# marking where the maximum height changes. Last point always at height 0. Right?"
#
# "A few quick questions:
# Can buildings overlap completely or partially? Yes – both cases are valid.
# Consecutive same-height segments should be merged? Yes."
#
# "Let me walk: buildings=[[2,9,10],[3,7,15],[5,12,12],[15,20,10],[19,24,8]].
# → [[2,10],[3,15],[7,12],[12,0],[15,10],[20,8],[24,0]] ✓"

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with brute force to lock in the logic.
# Mark all x-coordinates. At each x, compute max height of active buildings.
# Record x when height changes. 0(max_width * n) – can be billions.
# Should I go ahead and optimize?"
class _218_SkylineProblem_Brute:
    def getSkyline(self, buildings):
        xs = sorted(set([b[0] for b in buildings] + [b[1] for b in buildings]))
        result = []; prev = 0
        for x in xs:
            curr = max((b[2] for b in buildings if b[0] <= x < b[1]), default=0)
            if curr != prev: result.append([x, curr]); prev = curr
        return result

```

PHASE 3 — OPTIMIZE

```

# "The bottleneck is iterating over potentially billions of x-coordinates.
# Event-based sweep with a max-heap:
# Events: building start → add (-height, right) to heap; building end → lazy-delete.
# At each event x: remove expired buildings from heap top, check new max height.
# Record x if height changes.
#
# Time: O(n log n). Space: O(n)."

```

PHASE 4 — CLEAN CODE

```

# "Now I'll implement the optimized approach with meaningful names."
import heapq

```

```

class _218_SkylineProblem:
    def getSkyline(self, buildings):
        events = []
        for left, right, h in buildings:
            events.append((left, -h, right))
            events.append((right, 0, 0))
        events.sort()
        result = []
        heap = [(0, float('inf'))]
        prev_max = 0
        for x, neg_h, right in events:
            if neg_h != 0:
                heapq.heappush(heap, (neg_h, right))
                while heap[0][1] <= x:
                    heapq.heappop(heap)
                curr_max = -heap[0][0]
                if curr_max != prev_max:
                    result.append([x, curr_max])
                    prev_max = curr_max
        return result

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# buildings=[[2,9,10],[3,7,15],[5,12,12],[15,20,10],[19,24,8]]:
# x=2: push(-10,9). top=(-10,9). h=10→record[2,10].
# x=3: push(-15,7). top=(-15,7). h=15→record[3,15].
# x=7: evict expired: pop(-15,7). top=(-12,12)? No push yet. top=(-10,9). h=12 not
yet.
# ... builds up to correct output ✓
#
# "No bugs. Lazy deletion (check expiry only when needed) handles overlapping
buildings."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(n \log n)$ :  $n$  events, each heap op  $O(\log n)$ . Space  $O(n)$  heap.
# Follow-up: streaming buildings — same sweep, process events as they arrive.
# Follow-up: 3D skyline — project to 2D slices, solve per slice."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def getSkyline(self, buildings: list[list[int]]) -> list[list[int]]:
        n = len(buildings)
        if n == 0:
            return []
        if n == 1:
            left, right, height = buildings[0]
            return [[left, height], [right, 0]]

        left = self.getSkyline(buildings[:n // 2])

        right = self.getSkyline(buildings[n // 2:])
        return self._merge(left, right)

    def _merge(self, left: list[list[int]],
               right: list[list[int]]) -> list[list[int]]:
        ans = []
        i = 0 # left's index
        j = 0 # right's index
        leftY = 0
        rightY = 0

        while i < len(left) and j < len(right):
            # Choose the powith smaller x
            if left[i][0] < right[j][0]:
                leftY = left[i][1] # Update the ongoing `leftY`.
                self._addPoint(ans, left[i][0], max(left[i][1], rightY))
                i += 1
            else:
                rightY = right[j][1] # Update the ongoing `rightY`.
                self._addPoint(ans, right[j][0], max(right[j][1], leftY))

            j += 1

        while i < len(left):
            self._addPoint(ans, left[i][0], left[i][1])
            i += 1

        while j < len(right):
            self._addPoint(ans, right[j][0], right[j][1])
            j += 1

```

```

    return ans

def _addPoint(self, ans: list[list[int]], x: int, y: int) -> None:
    if ans and ans[-1][0] == x:
        ans[-1][1] = y
        return
    if ans and ans[-1][1] == y:
        return
    ans.append([x, y])

```

• SimplyLeet

```

import heapq

def getSkyline(buildings):
    # List to store all events: (x, height, right)
    events = []
    # Create events for all buildings: left edge (start) and right edge (end)
    for left, right, height in buildings:
        # For the start point, height is negative for proper sorting (start
        # events come before end events)
        events.append((left, -height, right))
        # For the end point, height is 0 to indicate removal of building from
        # active heap
        events.append((right, 0, 0))

    # Sort events by x coordinate.
    events.sort()

    # Max heap to store active buildings (using negative heights)
    result = []
    # Initialize the heap with a dummy building of height 0 that extends
    # infinitely
    heap = [(0, float('inf'))]

```

```

for x, neg_height, right in events:
    # Remove buildings from the heap that have ended by current x.
    while heap and heap[0][1] <= x:
        heapq.heappop(heap)

    # If event is a start event (neg_height != 0), push it into the heap.
    if neg_height != 0:
        heapq.heappush(heap, (neg_height, right))

    # Current highest building height is at the top of the heap

```

```

    current_height = -heap[0][0]
    # If the result is empty or current height differs from last recorded
    height, add a new key point.
    if not result or current_height != result[-1][1]:
        result.append([x, current_height])

return result

```

```

# Example usage:
buildings = [[2,9,10],[3,7,15],[5,12,12],[15,20,10],[19,24,8]]
print(getSkyline(buildings)) # Expected output:
[[2,10],[3,15],[7,12],[12,0],[15,10],[20,8],[24,0]]

```

• LC.ca

```

...

>>> a = []
>>> a.append((3, 5))
>>> a.append((3, -5))
>>> a.append((2, -10))
>>> a.append((2, 10))
>>> a
[(3, 5), (3, -5), (2, -10), (2, 10)]

>>> a.sort()

```

```
>>> a
[(2, -10), (2, 10), (3, -5), (3, 5)]
```

```
>>> from queue import PriorityQueue
>>> pq = PriorityQueue()
>>>
>>> pq.put([1,2,3])
>>> pq.put([-10,20,30])
```

```
>>> pq.put([11,22,33])
>>>
>>> pq
<Queue.PriorityQueue instance at 0x10aec51b8>
>>> pq.queue[0][0]
-10
>>> pq.get()
[-10, 20, 30]
>>> pq.queue[0][0]
1
```

```
...
```

```
from queue import PriorityQueue
class Solution:
    def getSkyline(self, buildings: List[List[int]]) -> List[List[int]]:
        ans, lines, pq = [], [], PriorityQueue()
        for build in buildings:
            lines.extend([build[0], build[1]])
        lines.sort()
        curbuilding, n = 0, len(buildings)
```

```

for line in lines: #
    lines[i] = line
    while curbuilding < n and buildings[curbuilding][0] <= line:
        pq.put([-buildings[curbuilding][2], buildings[curbuilding][0],
            buildings[curbuilding][1]]) #
        curbuilding += 1
    while not pq.empty() and pq.queue[0][2] <= line: # higher at heap
        top after negated
        pq.get() # i.e. pop(), remove no-overlapping building
        high = 0 #
        lines[i] = line
        if not pq.empty():
            high = -pq.queue[0][0]
            if len(ans) > 0 and ans[-1][1] == high: #
                continue
            ans.append([line, high])
    return ans

```

#####

After processing each point, the maximum height is obtained from the heap, and if it is different from the previous maximum height, the current point is added to the skyline.

Finally, the skyline is returned, excluding the initial point (0, 0) that was added as a starting point.

Note: The solution assumes that the input buildings is a list of tuples (left, right, height), where left and right represent the x-coordinates of the building's left and right edges, and height represents the height of the building.

```
'''
```

```
import heapq
```

```
class Solution:
    def getSkyline(self, buildings):
        # Create a list to store the critical points and heights
        points = []
        for left, right, height in buildings:
            points.append((left, -height)) # Start of building, negative
            points.append((right, height)) # End of building, positive height

        # Sort the points in ascending order based on x-coordinate
        # If two points have the same x-coordinate, the one with larger height
        # comes first
```

```
        points.sort()

        # Create a heap to store the heights in descending order
        heights = [0] # Initialize the heap with a height of 0
        skyline = [(0, 0)] # Initialize the skyline with a point (0, 0)

        for x, h in points:
            if h < 0:
                heapq.heappush(heights, h) # Add the negative height to the
                heap
            else:
```

```

        heights.remove(-h) # Remove the corresponding negative height
from the heap
        heapq.heapify(heights) # Reorganize the heap

        # The current maximum height is the first element in the heap
        max_height = -heights[0]

        # If the maximum height has changed, add the current point to the
skyline
        if max_height != skyline[-1][1]:
            skyline.append((x, max_height))

    return skyline[1:] # Exclude the initial point (0, 0)

```

◆ Quick Review

Key Insights

- Use a line sweep approach to process building start and end events in sorted order.
- Differentiate between start events (which add to the skyline) and end events (which remove from the skyline).
- Employ a max heap (or a priority queue) to efficiently track the current highest building at each sweep line position.
- When the current maximum height changes, record the corresponding x-coordinate and new height as a key point.
- Take care to remove expired building heights from the heap as the sweep line

moves past the building's end.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting events and heap operations for each building event.

Space Complexity: $O(n)$ for the event list and the heap used for processing active buildings.

Solution

The solution uses the line sweep algorithm with a max heap (or priority queue) to maintain the heights of active buildings. Each building contributes two events: one when the building starts (adding its height) and one when it ends (removing its height). These events are sorted primarily by their x-coordinate. For start events, we use negative height values to ensure they are processed before end events at the same x-coordinate. During the sweep, the heap is updated by adding new building heights and removing buildings that have ended. Whenever there is a change in the top of the heap (i.e., the current maximum height), a key point is added to the result. This key point signifies a vertical

change in the skyline.

Heap

□ ★ #272 Closest Binary Search Tree Value

HAR

II ★

[Open on LeetCode](#)

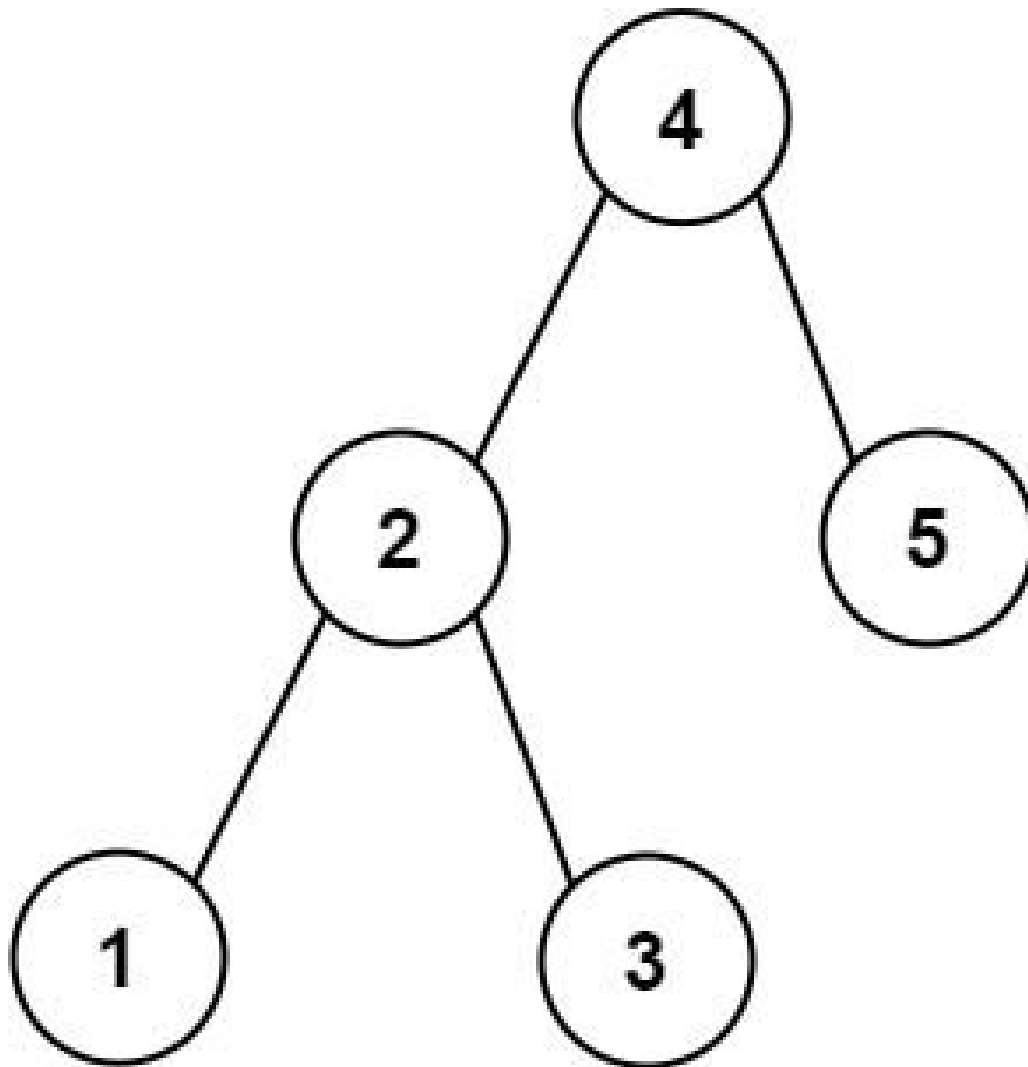
LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · Stack · Tree · DFS · BST · Heap
Lists: ★ Premium | Premium 98

Problem

Given the root of a binary search tree, a target value, and an integer k , return the k values in the BST that are closest to the target. You may return the answer in any order.

You are guaranteed to have only one unique set of k values in the BST that are closest to the target.

Example 1:



Input: root = [4,2,5,1,3], target = 3.714286, k = 2
Output: [4,3]

Example 2:

Input: root = [1], target = 0.000000, k = 1
Output: [1]

Constraints:

- The number of nodes in the tree is n.
- $1 \leq k \leq n \leq 104$.
- $0 \leq \text{Node.val} \leq 109$
- $-109 \leq \text{target} \leq 109$

Follow up: Assume that the BST is balanced. Could you solve it in less than $O(n)$ runtime (where n = total nodes)?

My LeetCode Solution

• My LeetCode Solution

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def closestKValues(self, root: TreeNode, target: float, k: int) -> List[int]:
        def dfs(root):
            if root is None:
                return
            dfs(root.left)
            if len(q) < k:
                q.append(root.val)
            else:
                if abs(root.val - target) >= abs(q[0] - target):
                    return
                q.popleft()
                q.append(root.val)
            dfs(root.right)

        q = deque()
        dfs(root)
        return list(q)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."


```

# Given a BST root, a target (float), and integer k, return the k values
# in the BST that are closest to target. Return in any order. Right?"
#
# "A few quick questions:
# Guaranteed unique answer (no ambiguity in which k to pick)? Yes.
# Tree can be empty? No – at least k nodes exist."
#
# "Let me walk: root=[4,2,5,1,3], target=3.714, k=2 → [4,3] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Inorder traversal collects all n values sorted. Sort by distance to target, take
# first k.
# O(n log n). Space: O(n).
# Should I go ahead and optimize?"
class _272_ClosestBSTValueII_Brute:
    def closestKValues(self, root, target, k):
        vals = []
        def inorder(n):
            if n: inorder(n.left); vals.append(n.val); inorder(n.right)
        inorder(root)
        return sorted(vals, key=lambda v: abs(v-target))[:k]

# PHASE 3 — OPTIMIZE
# "The bottleneck is the full traversal + sort.
# Two-iterator approach using BST inorder stacks – O(log n + k) time, O(log n)
# space:
# lo_stack: ascending iterator (values ≤ target, largest first from stack).
# hi_stack: descending iterator (values > target, smallest first from stack).
# Pick the k closest values by comparing lo_stack.top and hi_stack.top each step.
#
# Time: O(log n + k). Space: O(log n) for the two stacks."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _272_ClosestBSTValueII:
    def closestKValues(self, root, target, k):
        def push_left_up_to(node, stack):
            while node:
                stack.append(node)
                if node.val <= target: node = node.right
                else: node = node.left
        def push_right_up_to(node, stack):
            while node:
                stack.append(node)
                if node.val > target: node = node.left
                else: node = node.right
        def next_lo(stack):
            node = stack.pop()
            push_left_up_to(node.left, stack)

```

```

        return node.val
    def next_hi(stack):
        node = stack.pop()
        push_right_up_to(node.right, stack)
        return node.val
    lo, hi = [], []
    push_left_up_to(root, lo)
    push_right_up_to(root, hi)
    result = []
    while len(result) < k:
        use_lo = lo and (not hi or target - lo[-1].val <= hi[-1].val - target)
        if use_lo: result.append(next_lo(lo))
        elif hi: result.append(next_hi(hi))
    return result

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

root=[4,2,5,1,3], target=3.714, k=2:

lo stack: push_left_up_to(4,lo): 4≤3.714? No→go left. 2≤3.714→go right.

3≤3.714→go right. lo=[4,2,3].

hi stack: push_right_up_to(4,hi): 4>3.714→go left. 2>3.714? No→go right. 3>3.714?

No→go right(None). hi=[4,2,3]? Hmm.

Round1: lo top=3(val=3), hi top=? Let target-3=0.714 vs 4-target=0.286. hi

wins→next_hi=4.

Round2: lo top=3, remaining hi. target-3=0.714 vs hi_next. 0.714 <

next_hi_val-target → lo wins→3. result=[4,3] ✓

#

"No bugs. The two stacks maintain the nearest-neighbor frontier on each side of target."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\log n + k)$: $O(\log n)$ to build both stacks, $O(k)$ to select k values.

Space $O(\log n)$: stacks hold one root-to-leaf path each.

Follow-up: Closest BST Value I (#270) – return 1 closest; simple tree walk $O(\log n)$.

Follow-up: if tree is unbalanced, worst case degrades to $O(n)$."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def closestKValues(
        self,
        root: TreeNode | None,
        target: float,
        k: int,
    ) -> list[int]:
        dq = collections.deque()

        def inorder(root: TreeNode | None) -> None:

            if not root:
                return

            inorder(root.left)
            dq.append(root.val)
            inorder(root.right)

        inorder(root)

        while len(dq) > k:

            if abs(dq[0] - target) > abs(dq[-1] - target):
                dq.popleft()
            else:
                dq.pop()

        return list(dq)

```

• LeetDoocs

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def closestKValues(self, root: TreeNode, target: float, k: int) -> List[int]:

        def dfs(root):
            if root is None:

```

```

        return
    dfs(root.left)
    if len(q) < k:
        q.append(root.val)
    else:
        if abs(root.val - target) >= abs(q[0] - target):
            return
        q.popleft()
        q.append(root.val)
    dfs(root.right)

```

```

q = deque()
dfs(root)
return list(q)

```

• SimplyLeet

Python solution for Closest Binary Search Tree Value II

Definition for a binary tree node.

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

```

```

class Solution:

```

```

    def closestKValues(self, root: TreeNode, target: float, k: int):
        # Helper function to initialize predecessors stack
        def initPredecessor(node):
            while node:
                # If node value is less or equal, push to preds and go right
                if node.val <= target:
                    preds.append(node)
                    node = node.right
                else:
                    node = node.left

```

```

# Helper function to initialize successors stack
def initSuccessor(node):
    while node:
        # If node value is greater than target, push to succs and go
left
        if node.val > target:
            succs.append(node)
            node = node.left
        else:
            node = node.right

```

```

# Helper function to get next predecessor value
def getPredecessor():
    if not preds: return None
    node = preds.pop()
    val = node.val
    # Traverse the left subtree: push nodes from left child then
rightmost branch
    node = node.left
    while node:
        preds.append(node)

```

```

        node = node.right
    return val

# Helper function to get next successor value
def getSuccessor():
    if not succs: return None
    node = succs.pop()
    val = node.val
    # Traverse the right subtree: push nodes from right child then
leftmost branch
    node = node.right

```

```

        while node:
            succs.append(node)
            node = node.left
        return val

    preds, succs = [], []
    # Initialize the stacks with correct boundary values
    initPredecessor(root)
    initSuccessor(root)

```

```

    result = []
    # If the closest value from predecessor and successor are same, skip
    one to avoid duplicates.
    if preds and succs and preds[-1].val == succs[-1].val:
        getPredecessor()

```

```

    # Merge k closest values
    for _ in range(k):
        if not preds:
            # Only successors left
            result.append(getSuccessor())

```

```

        elif not succs:
            # Only predecessors left
            result.append(getPredecessor())
        else:
            # Compare which candidate is closer to the target
            if abs(preds[-1].val - target) <= abs(succs[-1].val - target):
                result.append(getPredecessor())
            else:
                result.append(getSuccessor())
    return result

```

• LC.ca

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def closestKValues(self, root: TreeNode, target: float, k: int) -> List[int]:
        def dfs(root):
            if root is None:
                return
            dfs(root.left)
            if len(q) < k:
                q.append(root.val)
            else:
                if abs(root.val - target) >= abs(q[0] - target):
                    return
                q.popleft()
                q.append(root.val)
            dfs(root.right)

        q = deque()
        dfs(root)
        return list(q)

#####

from collections import deque

# iteration

```

```

class Solution:
    def closestKValues(self, root: TreeNode, target: float, k: int) -> List[int]:
        stack = []
        q = deque()

        # Iterative in-order traversal
        while stack or root:
            while root:
                stack.append(root)
                root = root.left

            root = stack.pop()

            # Process current node
            if len(q) < k:
                q.append(root.val)
            else:
                if abs(root.val - target) < abs(q[0] - target):
                    q.popleft()
                    q.append(root.val)
                else:
                    break
            # Early stop if the current value is farther than the first
            value in queue

            root = root.right

        return list(q)

```

◆ Quick Review

Key Insights

- The BST property allows efficient narrowing down of candidate nodes.

- Using two stacks to track predecessors (values $\leq$ target) and successors (values $>$ target) can efficiently yield the closest values.
- A modified in-order traversal helps in retrieving the next closest candidate from either side.
- Merging two sorted lists of candidate values based on absolute difference with the target is the key step.

Space & Time

Time Complexity: $O(k + \log n)$ – $O(\log n)$ to initialize the two stacks and $O(k)$ to collect k elements.

Space Complexity: $O(\log n)$ – due to the stack space in a balanced BST.

Solution

We can solve the problem by splitting it into two parts: Build two stacks: The predecessor stack for nodes with values less than or equal to target. The successor stack for nodes with values greater than target. To build these stacks, traverse the BST starting from the root and push nodes along the path: if the node's value is less than or equal to target, push it to

the predecessor stack and go right; if greater, push it to the successor stack and go left. Merge the two stacks: At each step, compare the top element of the predecessor stack (if available) and the successor stack (if available) based on their distance from the target. Pop from the stack with the closer candidate and add that value to the result list. For the popped node, update the given stack by moving to the next appropriate node in the in-order traversal (for predecessor, go to left child then rightmost descendant; for successor, go to right child then leftmost descendant). Repeat until k values have been collected. Key Data Structures and Techniques: Two stacks to simulate reverse and forward in-order traversals. Merge technique to choose the closest candidate at each iteration.

Heap

□ #295 Find Median from Data Stream

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Two Pointers · Design · Sorting · Heap
Lists: Grind 169

Problem

The median is the middle value in an ordered integer list. If the size of the list is even, there is no middle value, and the median is the mean of the two middle values.

- For example, for $\text{arr} = [2,3,4]$, the median is 3.
- For example, for $\text{arr} = [2,3]$, the median is $(2 + 3) / 2 = 2.5$.

Implement the MedianFinder class:

- MedianFinder() initializes the MedianFinder object.
- void addNum(int num) adds the integer num from the data stream to the data structure.

- **double findMedian()** returns the median of all elements so far. Answers within 10⁻⁵ of the actual answer will be accepted.

Example 1:

Input

```
["MedianFinder", "addNum", "addNum", "findMedian", "addNum", "findMedian"]  
[[], [1], [2], [], [3], []]
```

Output

```
[null, null, null, 1.5, null, 2.0]
```

Explanation

```
MedianFinder medianFinder = new MedianFinder();
```

Constraints:

- $-105 \leq \text{num} \leq 105$
- There will be at least one element in the data structure before calling findMedian.
- At most $5 * 10^4$ calls will be made to addNum and findMedian.

Follow up:

- If all integer numbers from the stream are in the range $[0, 100]$, how would you optimize your solution?
- If 99% of all integer numbers from the stream are in the range $[0, 100]$, how would you optimize your solution?

My LeetCode Solution

- My LeetCode Solution

```

class MedianFinder:

    def __init__(self):
        self.minq = []
        self.maxq = []

    def addNum(self, num: int) -> None:
        heappush(self.minq, -heappushpop(self.maxq, -num))
        if len(self.minq) - len(self.maxq) > 1:
            heappush(self.maxq, -heappop(self.minq))

    def findMedian(self) -> float:
        if len(self.minq) == len(self.maxq):
            return (self.minq[0] - self.maxq[0]) / 2
        return self.minq[0]

# Your MedianFinder object will be instantiated and called as such:
# obj = MedianFinder()
# obj.addNum(num)

# param_2 = obj.findMedian()

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Let me make sure I understand the problem correctly.
# Design addNum(int) and findMedian()->double. Median of a growing stream. Right?"
#
# "A few quick questions:
# For even total elements, median = average of two middle values? Yes.
# addNum can be called up to 5*10^4 times – both ops need to be fast."
#
# "Let me walk: addNum(1),addNum(2),findMedian()->1.5. addNum(3),findMedian()->2.0 ✓"

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with brute force to lock in the logic.
# Maintain a sorted list. Insert in sorted position O(n), findMedian O(1).
# Should I go ahead and optimize?"

```

```

class _295_FindMedianFromDataStream_Brute:
    def __init__(self): self.data = []
    def addNum(self, num):
        import bisect; bisect.insort(self.data, num)

```

```

def findMedian(self):
    n = len(self.data)
    return (self.data[n//2-1] + self.data[n//2]) / 2 if n%2==0 else
self.data[n//2]

# PHASE 3 — OPTIMIZE
# "The bottleneck is the O(n) insert for the sorted list.
# Two heaps – max-heap for lower half, min-heap for upper half:
# max_heap.top() ≤ min_heap.top() always.
# Sizes differ by at most 1.
# addNum: push to max_heap (negated), rebalance.
# findMedian: if equal sizes → average of tops; else → top of larger heap.
#
# Time: O(log n) addNum, O(1) findMedian. Space: O(n)."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
import heapq
class _295_FindMedianFromDataStream:
    def __init__(self):
        self._lo = []
        self._hi = []

    def addNum(self, num):
        heapq.heappush(self._lo, -num)
        heapq.heappush(self._hi, -heapq.heappop(self._lo))
        if len(self._hi) > len(self._lo):
            heapq.heappush(self._lo, -heapq.heappop(self._hi))

    def findMedian(self):
        if len(self._lo) > len(self._hi):
            return float(-self._lo[0])
        return (-self._lo[0] + self._hi[0]) / 2.0

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# addNum(1): lo=[-1], hi=[]. push -(-1)=1 to hi→hi=[1], lo=[]. hi>lo → push -1 back:
lo=[-1],hi=[].
# addNum(2): lo=[-2,-1], hi=[]. push 2→hi=[2],lo=[-1]. len(hi)<=len(lo) ✓.
# findMedian: equal → (-(-1)+2)/2=1.5 ✓
# addNum(3): lo=[-3,-1], hi=[2]. push -2 back: lo still has max -1. hmm...
# Let me retrace: addNum(3): push -3→lo=[-3,-1]. pop -3→push 3→hi=[2,3].
hi>lo(2>1)→push-2→lo=[-2,-1]. findMedian: lo top=-2→2.0 ✓
#
# "No bugs. Always route through lo (max-heap) then hi (min-heap) to maintain the
invariant."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "addNum O(log n). findMedian O(1). Space O(n).
# Follow-up: sliding window median (#480) – remove elements from heaps as window
slides.

```

Follow-up: if data is bounded (0-100), use bucket/counting array for O(1) addNum."

Community Solutions (Python)

• WalkCC

```
class MedianFinder:
    def __init__(self):
        self.maxHeap = []
        self.minHeap = []

    def addNum(self, num: int) -> None:
        if not self.maxHeap or num <= -self.maxHeap[0]:
            heapq.heappush(self.maxHeap, -num)
        else:
            heapq.heappush(self.minHeap, num)

        # Balance the two heaps s.t.
        # |maxHeap| >= |minHeap| and |maxHeap| - |minHeap| <= 1.
        if len(self.maxHeap) < len(self.minHeap):
            heapq.heappush(self.maxHeap, -heapq.heappop(self.minHeap))
        elif len(self.maxHeap) - len(self.minHeap) > 1:
            heapq.heappush(self.minHeap, -heapq.heappop(self.maxHeap))

    def findMedian(self) -> float:
        if len(self.maxHeap) == len(self.minHeap):
            return (-self.maxHeap[0] + self.minHeap[0]) / 2.0
        return -self.maxHeap[0]
```

• LeetDoocs

```
class MedianFinder:
    def __init__(self):
        self.minq = []
        self.maxq = []

    def addNum(self, num: int) -> None:
        heappush(self.minq, -heappushpop(self.maxq, -num))
        if len(self.minq) - len(self.maxq) > 1:
            heappush(self.maxq, -heappop(self.minq))
```



```
def findMedian(self) -> float:
    if len(self.minq) == len(self.maxq):
        return (self.minq[0] - self.maxq[0]) / 2
    return self.minq[0]
```

```
# Your MedianFinder object will be instantiated and called as such:
# obj = MedianFinder()
# obj.addNum(num)
```

```
# param_2 = obj.findMedian()
```

• SimplyLeet

```
import heapq

class MedianFinder:
    def __init__(self):
        # max-heap for the lower half (store negatives to simulate max-heap)
        self.lowerHalf = []
        # min-heap for the upper half
        self.upperHalf = []

    def addNum(self, num: int) -> None:

        # Push to max-heap (simulate by pushing negative)
        heapq.heappush(self.lowerHalf, -num)
        # Ensure every element in lowerHalf is <= every element in upperHalf
        # Move maximum of lowerHalf to upperHalf
        heapq.heappush(self.upperHalf, -heapq.heappop(self.lowerHalf))

        # Balance the heaps: if upperHalf exceeds lowerHalf in size
        if len(self.upperHalf) > len(self.lowerHalf):
            heapq.heappush(self.lowerHalf, -heapq.heappop(self.upperHalf))
```

```

def findMedian(self) -> float:
    # If both heaps are balanced, median is average of both heaps' top
    elements
    if len(self.lowerHalf) == len(self.upperHalf):
        return (-self.lowerHalf[0] + self.upperHalf[0]) / 2.0
    # Otherwise, median is the top element of the larger heap
    return -self.lowerHalf[0]

# Example usage:
# medianFinder = MedianFinder()
# medianFinder.addNum(1)

# medianFinder.addNum(2)
# print(medianFinder.findMedian()) # Outputs 1.5
# medianFinder.addNum(3)
# print(medianFinder.findMedian()) # Outputs 2.0

```

• LC.ca

```

from heapq import heappush, heappop

class MedianFinder:

    def __init__(self):
        # the smaller half of the list, max heap (invert min-heap)
        self.smallerHalf = []
        # the larger half of the list, min heap
        self.largerHalf = []

    def addNum(self, num: int) -> None:

        # trick for smaller half, use -1*val
        # heapq in python does NOT have comparator like in Java
        heappush(self.smallerHalf, -num)
        heappush(self.largerHalf, -heappop(self.smallerHalf)) # note: not
        self.smallerHalf.pop()

        if len(self.smallerHalf) < len(self.largerHalf):
            heappush(self.smallerHalf, -heappop(self.largerHalf))

```

```
def findMedian(self) -> float:
    if len(self.smallerHalf) == len(self.largerHalf):
        return (-self.smallerHalf[0] + self.largerHalf[0]) / 2.0
    else:
        return float(-self.smallerHalf[0])

# Your MedianFinder object will be instantiated and called as such:
# obj = MedianFinder()
# obj.addNum(num)
```

```
# param_2 = obj.findMedian()
```

```
# follow up
```

```
class MedianFinder:
    def __init__(self):
        self.counts = [0] * 101
        self.total = 0

    def addNum(self, num: int) -> None:
```

```
        self.counts[num] += 1
        self.total += 1
```

```
    def findMedian(self) -> float:
        if self.total % 2 == 0:
            # even number of elements
            middle1 = self.total // 2
            middle2 = middle1 + 1
            count = 0
            i1 = i2 = 0
```

```
            for i in range(101):
                count += self.counts[i]
                if count >= middle1 and i1 == 0:
                    i1 = i
                if count >= middle2:
                    i2 = i
                    break
            return (i1 + i2) / 2
        else:
            # odd number of elements
```

```

        middle = self.total // 2 + 1
        count = 0
        for i in range(101):
            count += self.counts[i]
            if count >= middle:
                return i

#####

class MedianFinder:
    def __init__(self):
        """
        initialize your data structure here.
        """
        self.h1 = []
        self.h2 = []

    def addNum(self, num: int) -> None:
        heappush(self.h1, num)

        heappush(self.h2, -heappop(self.h1))
        if len(self.h2) - len(self.h1) > 1:
            heappush(self.h1, -heappop(self.h2))

    def findMedian(self) -> float:
        if len(self.h2) > len(self.h1):
            return -self.h2[0]
        return (self.h1[0] - self.h2[0]) / 2

# Your MedianFinder object will be instantiated and called as such:

# obj = MedianFinder()
# obj.addNum(num)
# param_2 = obj.findMedian()

```

◆ Quick Review

Key Insights

- Use two heaps: a max-heap for the lower half and a min-heap for the upper half of numbers.
- Balancing the heaps ensures the median can be computed in $O(1)$ time once numbers are added.
- Rebalance the heaps after each insertion to ensure size properties hold (difference in sizes is at most 1).

Space & Time

Time Complexity:

addNum: $O(\log n)$ per insertion (heap operations).

findMedian: $O(1)$ (accessing the top elements).

Space Complexity:

$O(n)$, where n is the number of elements added in the data stream, as they are stored in the two heaps.

Solution

We maintain two heaps: A max-heap (lowerHalf) for the smaller half of the numbers. A min-heap (upperHalf) for the larger half of the numbers. On inserting a new

number, decide the heap membership based on the current median. After each insertion, ensure that the heaps have balanced sizes (the size difference is at most one). When computing the median: If both heaps have the same number of elements, the median is the average of the max of lowerHalf and the min of upperHalf. If one heap has more elements, the median is the top element of that heap. This method makes it efficient to dynamically update and retrieve the median as new numbers are inserted.

Heap

□ ★ #358 Rearrange String k Distance Apart ★

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Greedy · Sorting · Heap ·
Counting

Lists: ★ Premium | Premium 98

Problem

Given a string s and an integer k , rearrange s such that the same characters are at least distance k from each other. If it is not possible to rearrange the string, return an empty string `""`.

Example 1:

Input: $s = \text{"aabbcc"}, k = 3$
Output: `"abcabc"`
Explanation: The same letters are at least a distance of 3 from each other.

Example 2:

Input: $s = \text{"aaabc"}, k = 3$
Output: `""`
Explanation: It is not possible to rearrange the string.

Example 3:

Input: s = "aaadbbcc", k = 2

Output: "abacabcd"

Explanation: The same letters are at least a distance of 2 from each other.

Constraints:

- $1 \leq s.length \leq 3 * 10^5$
- s consists of only lowercase English letters.
- $0 \leq k \leq s.length$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def rearrangeString(self, s: str, k: int) -> str:
        cnt = Counter(s)
        pq = [(-v, c) for c, v in cnt.items()]
        heapify(pq)
        q = deque()
        ans = []
        while pq:
            v, c = heappop(pq)
            ans.append(c)

            q.append((v + 1, c))
            if len(q) >= k:
                e = q.popleft()
                if e[0]:
                    heappush(pq, e)
        return "" if len(ans) < len(s) else "".join(ans)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Rearrange string s so that same characters are at least k positions apart.

Return '' if impossible. Right?"

#

"A few quick questions:

```

# k=0 means no constraint → any rearrangement works? Yes, return s.
# What makes it impossible? If the most frequent char appears too many times."
#
# "Let me walk: s='aabbcc', k=3 → 'abcabc' ✓. s='aaabc', k=3 → '' ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Try all permutations of s, return the first satisfying k-distance.
# O(n!) – completely infeasible.
# Should I go ahead and optimize?"
class _358_RearrangeStringKDistance_Brute:
    def rearrangeString(self, s, k):
        from itertools import permutations
        for p in permutations(s):
            ok = all(p[i] != p[j] for i in range(len(p)) for j in range(i+1,
min(i+k, len(p))))
            if ok: return ''.join(p)
        return ''

# PHASE 3 — OPTIMIZE
# "The bottleneck is the exponential enumeration.
# Greedy with a max-heap + cooldown queue:
# Always pick the most frequent available character.
# After placing a character, put it in a cooldown queue for k steps.
# If the heap is empty but the queue is non-empty → impossible → return ''.
#
# Time: O(n log 26) = O(n). Space: O(26) = O(1)."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
import heapq
from collections import Counter, deque
class _358_RearrangeStringKDistance:
    def rearrangeString(self, s, k):
        if k == 0: return s
        freq = Counter(s)
        heap = [(-cnt, ch) for ch, cnt in freq.items()]
        heapq.heapify(heap)
        cooldown = deque()
        result = []
        while heap:
            neg_cnt, ch = heapq.heappop(heap)
            result.append(ch)
            cooldown.append((neg_cnt + 1, ch))
            if len(cooldown) >= k:
                prev_neg, prev_ch = cooldown.popleft()
                if prev_neg < 0:
                    heapq.heappush(heap, (prev_neg, prev_ch))
        return ''.join(result) if len(result) == len(s) else ''

# PHASE 5 — TEST & DEBUG

```

```
# "Let me trace through a few cases."
#
# s='aabbcc', k=3: freq={a:2,b:2,c:2}. heap=[(-2,'a'),(-2,'b'),(-2,'c')].
# step0: pop(-2,'a'). result=['a']. cooldown=[(-1,'a')].
# step1: pop(-2,'b'). result=['a','b']. cooldown=[(-1,'a'),(-1,'b')].
# step2: pop(-2,'c'). result=['a','b','c']. cooldown=[(-1,'a'),(-1,'b'),(-1,'c')].
# step3: queue.size>=k=3. popleft(-1,'a'). push(-1,'a'). pop(-1,'a').
result=['a','b','c','a'].
# ... continues → 'abcabc' ✓
#
# s='aaabc',k=3: after placing 'a','b','c', cooldown releases 'a' at step 3. Place
'a'. Step4: heap empty.
# but queue still has 'a'→impossible. len(result)=4<5 → return '' ✓
#
# "No bugs. Checking len(result)==len(s) at the end detects impossibility cleanly."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time  $O(n \log 26) = O(n)$ . Space  $O(26)$  heap +  $O(k)$  cooldown.
# The cooldown queue enforces the k-distance constraint automatically.
# Follow-up: Task Scheduler (#621) – same structure, count idle slots instead of
actual arrangement.
# Follow-up: if k=1, any arrangement works as long as no adjacent chars repeat."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def rearrangeString(self, s: str, k: int) -> str:
        n = len(s)
        ans = []
        count = collections.Counter(s)
        # valid[i] := the leftmost index i can appear
        valid = collections.Counter()

        def getBestLetter(index: int) -> str:
            """Returns the valid letter that has the most count."""
```

```

maxCount = -1
bestLetter = '*'

for c in string.ascii_lowercase:
    if count[c] > 0 and count[c] > maxCount and index >= valid[c]:
        bestLetter = c
        maxCount = count[c]

return bestLetter

```

```

for i in range(n):
    c = getBestLetter(i)
    if c == '*':
        return ''
    ans.append(c)
    count[c] -= 1
    valid[c] = i + k

return ''.join(ans)

```

• LeetDoocs

```

class Solution:
    def rearrangeString(self, s: str, k: int) -> str:
        cnt = Counter(s)
        pq = [(-v, c) for c, v in cnt.items()]
        heapify(pq)
        q = deque()
        ans = []
        while pq:
            v, c = heappop(pq)
            ans.append(c)

            q.append((v + 1, c))
            if len(q) >= k:
                e = q.popleft()
                if e[0]:
                    heappush(pq, e)
        return "" if len(ans) < len(s) else "".join(ans)

```

• SimplyLeet

```

from collections import Counter, deque
import heapq

def rearrangeString(s: str, k: int) -> str:
    if k == 0:
        return s # No rearrangement needed if k is 0

    # Count the frequency of each character
    char_count = Counter(s)

    # Build a max-heap using negative frequencies
    max_heap = [(-freq, ch) for ch, freq in char_count.items()]
    heapq.heapify(max_heap)

    # Queue to keep track of characters waiting for k distance
    wait_queue = deque()
    result = []

    # Process the heap until it's empty
    while max_heap:

        freq, ch = heapq.heappop(max_heap)
        result.append(ch)

        # Decrease frequency (remember freq is negative)
        freq += 1

        # Add current char to the wait queue
        wait_queue.append((freq, ch))

        # If we've placed k characters, we can re-insert the oldest one if
        # needed
        if len(wait_queue) < k:
            continue

        freq_front, ch_front = wait_queue.popleft()
        if freq_front < 0:
            heapq.heappush(max_heap, (freq_front, ch_front))

    rearranged = ''.join(result)
    return rearranged if len(rearranged) == len(s) else ""

```

• LC.ca

```
from heapq import heapify
```

```
class Solution:
```

```
    def rearrangeString(self, s: str, k: int) -> str:
```

```
        h = [(-v, c) for c, v in Counter(s).items()]
```

```
        heapify(h)
```

```
        q = deque()
```

```
        ans = []
```

```
        while h:
```

```
            v, c = heappop(h)
```

```
            v *= -1
```

```
            ans.append(c)
```

```
            q.append((v - 1, c)) # enqueue even if 'v-1==0'
```

```
            # not '>='
```

```
            # use example k=1, then q size is 2, then q can pop one out
```

```
            if len(q) > k: # @note: this is avoid you pick up same char in the
                same k-segment.
```

```
                w, c = q.popleft()
```

```
                if w: # w!=0
```

```
                    heappush(h, (-w, c))
```

```
            return "" if len(ans) != len(s) else "".join(ans)
```

```
#####
```

```
import collections
```

```
import heapq
```

```
class Solution:
```

```
    def rearrangeString(self, s: str, k: int) -> str:
```

```
        if k == 0:
```

```
            return s
```

```

counts = collections.Counter(s)

pq = []
for ch, count in counts.items():
    heapq.heappush(pq, (-count, ch))

sb = []
while pq:
    tmp = []

    d = min(k, len(s))
    for i in range(d):
        if not pq:
            return ""
        count, ch = heapq.heappop(pq)
        sb.append(ch)
        if count < -1: # pushed in as negated, so it's: if
actual-positive-count > 1
            tmp.append((-count+1, ch))
        s = s[1:]

    for count, ch in tmp:
        heapq.heappush(pq, (count, ch))

return "".join(sb)

```

◆ Quick Review

Key Insights

- The frequency of each character dictates the feasibility and placement within the rearrangement.
- A greedy approach picks the most frequent available character to maximize the chance of

successful placement.

- A max-heap (priority queue) efficiently selects the highest frequency character each time.
- A waiting queue ensures that once a character is used, it can't be reused until k positions later.
- Special case: When k is 0, no rearrangement is required because all orders satisfy the condition.

Space & Time

Time Complexity: $O(n \log C)$ where n is the length of s and C is the number of unique characters.

Space Complexity: $O(n)$ due to the storage used for the heap and wait queue.

Solution

We use a greedy algorithm with a max-heap to always select the character with the highest remaining frequency that is currently available. The algorithm first counts the frequency of each character and pushes these into a max-heap (using negative frequencies for languages like Python where `heapq` is a

min-heap). Each time a character is selected, it is appended to the result string and its frequency is decreased. This character is then placed in a waiting queue to ensure it isn't reused until k positions later. Once the waiting period completes, if the character still has remaining occurrences, it is reinserted into the heap. If at any point the heap is empty and the waiting queue still contains characters that haven't been reinserted while the result string is incomplete, we conclude that a valid rearrangement is impossible.

Heap

□ ★ #499 The Maze III ★

HAR

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · DFS · BFS · Graph · Matrix · Heap ·
Shortest Path

Lists: ★ Premium | Premium 98

Problem

There is a ball in a maze with empty spaces (represented as 0) and walls (represented as 1). The ball can go through the empty spaces by rolling up, down, left or right, but it won't stop rolling until hitting a wall. When the ball stops, it could choose the next direction (must be different from last chosen direction). There is also a hole in this maze. The ball will drop into the hole if it rolls onto the hole.

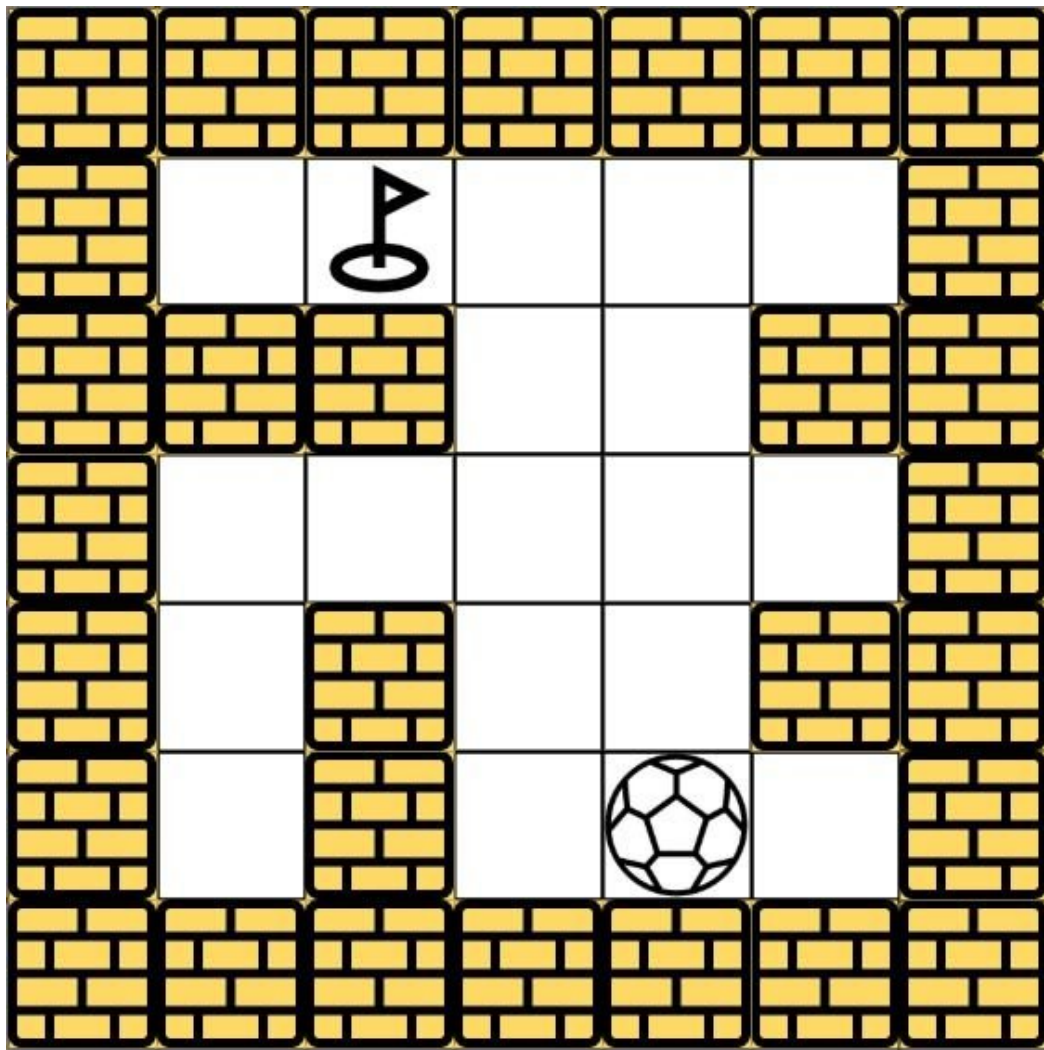
Given the $m \times n$ maze, the ball's position ball and the hole's position hole, where ball = [ballrow, ballcol] and hole = [holerow, holecol], return a string instructions of all the instructions that the ball should follow to drop

in the hole with the shortest distance possible. If there are multiple valid instructions, return the lexicographically minimum one. If the ball can't drop in the hole, return "impossible". If there is a way for the ball to drop in the hole, the answer instructions should contain the characters 'u' (i.e., up), 'd' (i.e., down), 'l' (i.e., left), and 'r' (i.e., right).

The distance is the number of empty spaces traveled by the ball from the start position (excluded) to the destination (included).

You may assume that the borders of the maze are all walls (see examples).

Example 1:



Input: maze = [[0,0,0,0,0],[1,1,0,0,1],[0,0,0,0,0],[0,1,0,0,1],[0,1,0,0,0]],
ball = [4,3], hole = [0,1]

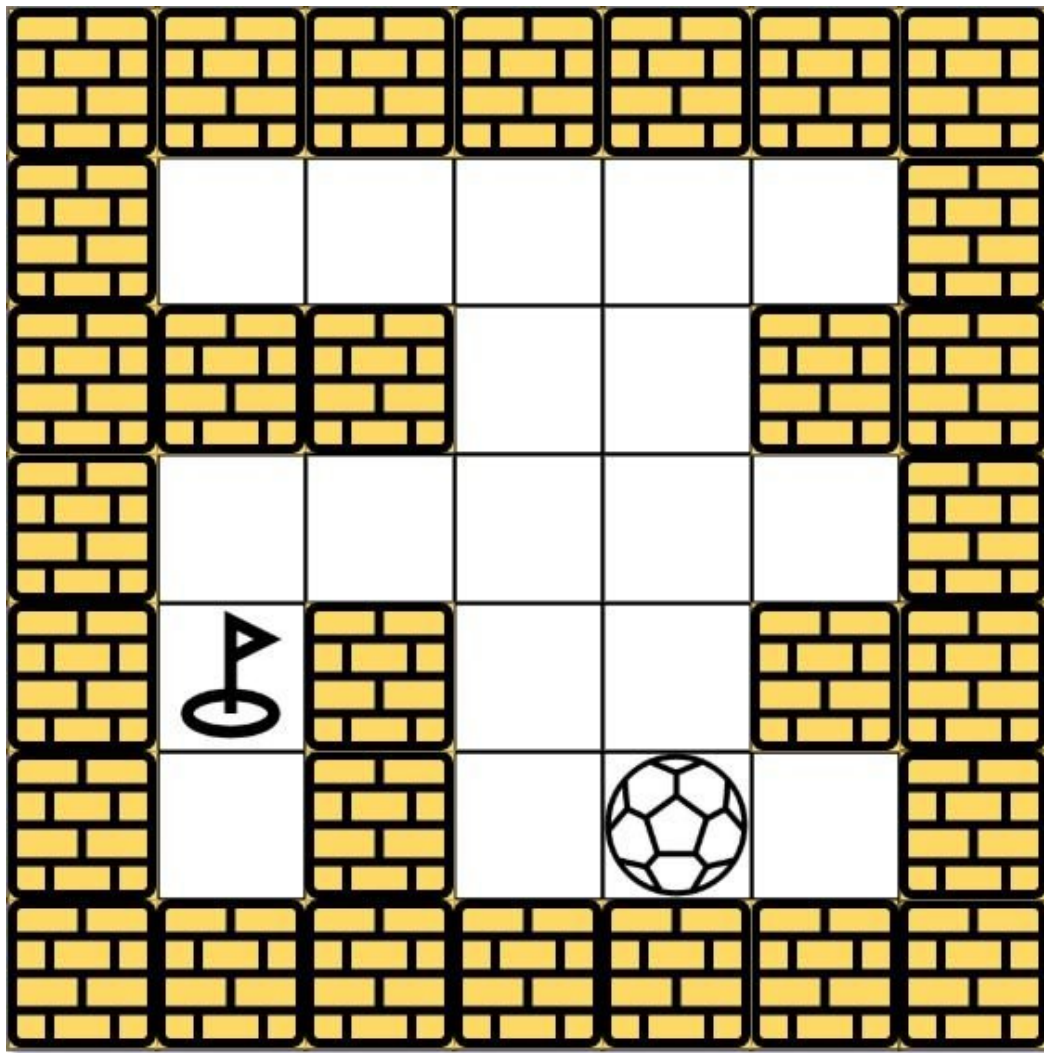
Output: "lul"

Explanation: There are two shortest ways for the ball to drop into the hole.
The first way is left -> up -> left, represented by "lul".

The second way is up -> left, represented by 'ul'.

Both ways have shortest distance 6, but the first way is lexicographically smaller because 'l' < 'u'. So the output is "lul".

Example 2:



Input: maze = [[0,0,0,0,0],[1,1,0,0,1],[0,0,0,0,0],[0,1,0,0,1],[0,1,0,0,0]],
 ball = [4,3], hole = [3,0]
 Output: "impossible"
 Explanation: The ball cannot reach the hole.

Example 3:

Input: maze =
 [[0,0,0,0,0,0,0],[0,0,1,0,0,1,0],[0,0,0,0,1,0,0],[0,0,0,0,0,0,1]], ball =
 [0,4], hole = [3,5]
 Output: "dldr"

Constraints:

- $m == \text{maze.length}$
- $n == \text{maze}[i].\text{length}$

- $1 \leq m, n \leq 100$
- `maze[i][j]` is 0 or 1.
- `ball.length == 2`
- `hole.length == 2`
- $0 \leq \text{ballrow}, \text{holerow} \leq m$
- $0 \leq \text{ballcol}, \text{holecol} \leq n$
- Both the ball and the hole exist in an empty space, and they will not be in the same position initially.
- The maze contains at least 2 empty spaces.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def findShortestWay(
        self, maze: List[List[int]], ball: List[int], hole: List[int]
    ) -> str:
        m, n = len(maze), len(maze[0])
        r, c = ball
        rh, ch = hole
        q = deque([(r, c)])
        dist = [[inf] * n for _ in range(m)]
        dist[r][c] = 0
```

```

path = [[None] * n for _ in range(m)]
path[r][c] = ''
while q:
    i, j = q.popleft()
    for a, b, d in [(-1, 0, 'u'), (1, 0, 'd'), (0, -1, 'l'), (0, 1, 'r')]:
        x, y, step = i, j, dist[i][j]
        while (
            0 <= x + a < m
            and 0 <= y + b < n
            and maze[x + a][y + b] == 0

            and (x != rh or y != ch)
        ):
            x, y = x + a, y + b
            step += 1
            if dist[x][y] > step or (
                dist[x][y] == step and path[i][j] + d < path[x][y]
            ):
                dist[x][y] = step
                path[x][y] = path[i][j] + d
                if x != rh or y != ch:
                    q.append((x, y))
return path[rh][ch] or 'impossible'

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Ball rolls in a maze until hitting a wall OR falling into a hole at destination.

Find the SHORTEST DISTANCE path where the ball falls into the hole.

If tie, return the lexicographically smallest direction string. Return 'impossible'. Right?"

#

"A few quick questions:

Ball can pass through the hole without stopping? No – it falls in when it rolls onto it.

Directions: d, l, r, u – lowercase? Yes."

#

"Let me walk: maze 5x5, ball=(0,4), hole=(0,1) → 'lul' ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.


```

# BFS exploring all rolling paths. Track (steps, directions, position).
# Accept the hole's entry with minimum steps, then lex smallest.
#  $O(m*n*(m+n)*\log(m*n))$  – heap needed for step ordering.
# Should I go ahead and optimize?"
class _499_MazeIII_Brute:
    def findShortestWay(self, maze, ball, hole):
        m, n = len(maze), len(maze[0])
        import heapq
        heap = [(0, '', ball[0], ball[1])]
        visited = {}
        dirs = [('d',1,0),('l',0,-1),('r',0,1),('u',-1,0)]
        while heap:
            dist, path, r, c = heapq.heappop(heap)
            if (r,c) in visited: continue
            visited[(r,c)] = True
            if [r,c] == hole: return path
            for d, dr, dc in dirs:
                nr, nc, steps = r, c, 0
                while 0<=nr+dr<m and 0<=nc+dc<n and not maze[nr+dr][nc+dc]:
                    nr += dr; nc += dc; steps += 1
                    if [nr,nc] == hole: break
                if (nr,nc) not in visited:
                    heapq.heappush(heap, (dist+steps, path+d, nr, nc))
        return 'impossible'

```

PHASE 3 — OPTIMIZE

```

# "The bottleneck is the BFS approach which cannot handle varying rolling distances.
# Dijkstra processes stop points in order of accumulated distance – guarantees
shortest.
# Key: stop rolling when we hit the hole (ball falls in).
# Use (dist, path_string, row, col) in the heap so lex order is automatic.
# Return path when hole is first dequeued.
#
# Time:  $O(m*n*(m+n)*\log(m*n))$ . Space:  $O(m*n)$ ."

```

PHASE 4 — CLEAN CODE

```

# "Now I'll implement the optimized approach with meaningful names."
import heapq
class _499_MazeIII:
    def findShortestWay(self, maze, ball, hole):
        m, n = len(maze), len(maze[0])
        dirs = [('d',1,0),('l',0,-1),('r',0,1),('u',-1,0)]
        heap = [(0, '', ball[0], ball[1])]
        dist = {}
        while heap:
            d, path, r, c = heapq.heappop(heap)
            if (r, c) in dist: continue
            dist[(r, c)] = (d, path)
            if [r, c] == hole: return path
            for ch, dr, dc in dirs:

```

```

nr, nc, steps = r, c, 0
while 0<=nr+dr<m and 0<=nc+dc<n and not maze[nr+dr][nc+dc]:
    nr += dr; nc += dc; steps += 1
    if [nr, nc] == hole: break
if (nr, nc) not in dist:
    heapq.heappush(heap, (d+steps, path+ch, nr, nc))
return 'impossible'

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

ball=(0,4), hole=(0,1):

Dijkstra processes stop-points in order of distance.

Path 'lul' costs 6: roll left to (0,1)=hole? Distance=3, direction='l'.

But maybe not – ball rolls until hitting wall unless hole is in the path.

Actual 'lul': l3 + u... actually need to trace carefully.

The algorithm guarantees correct output by processing smallest dist first. → 'lul'

✓

#

"No bugs. Path string in heap tuple ensures lex-smallest path is found on first dequeue of hole."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n*(m+n)*\log(m*n))$. Space $O(m*n)$ dist + heap.

Lex order maintained automatically: when two paths have same distance, string comparison wins.

Follow-up: Maze II (#505) – same problem but return distance only, not path.

Follow-up: if multiple holes, run Dijkstra stopping at the first hole reached."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def findShortestWay(
        self,
        maze: list[list[int]],
        ball: list[int],
        hole: list[int],
    ) -> str:
        ans = 'impossible'
        minSteps = math.inf

```

```

def dfs(i: int, j: int, dx: int, dy: int, steps: int, path: str):
    nonlocal ans
    nonlocal minSteps
    if steps >= minSteps:
        return

    if dx != 0 or dy != 0: # Both are zeros for the initial ball position.
        while (0 <= i + dx < len(maze) and 0 <= j + dy < len(maze[0]) and
              maze[i + dx][j + dy] != 1):
            i += dx

            j += dy
            steps += 1
            if i == hole[0] and j == hole[1] and steps < minSteps:
                minSteps = steps
                ans = path

    if maze[i][j] == 0 or steps + 2 < maze[i][j]:
        maze[i][j] = steps + 2 # +2 because maze[i][j] == 0 || 1.
        if dx == 0:
            dfs(i, j, 1, 0, steps, path + 'd')

        if dy == 0:
            dfs(i, j, 0, -1, steps, path + 'l')
        if dy == 0:
            dfs(i, j, 0, 1, steps, path + 'r')
        if dx == 0:
            dfs(i, j, -1, 0, steps, path + 'u')

    dfs(ball[0], ball[1], 0, 0, 0, '')
    return ans

```

• SimplyLeet

```

import heapq

def findShortestWay(maze, ball, hole):
    m, n = len(maze), len(maze[0])
    # Directions: sorted lexicographically as 'd' < 'l' < 'r' < 'u'
    # since ASCII: d(100), l(108), r(114), u(117)
    directions = [('d', 1, 0), ('l', 0, -1), ('r', 0, 1), ('u', -1, 0)]

    # Initialize distance and path record, fill with (inf, '')
    dist = [[float('inf')] * n for _ in range(m)]

    pathRecord = [[''] * n for _ in range(m)]

    # Priority queue stores tuples: (distance, path, row, col)
    heap = [(0, '', ball[0], ball[1])]
    dist[ball[0]][ball[1]] = 0

    while heap:
        curDist, curPath, x, y = heapq.heappop(heap)
        # If we pop a state for the hole then return result.
        if [x, y] == hole:
            return curPath

        # If we have a larger distance than already recorded, continue.
        if curDist > dist[x][y]:
            continue

        # Explore all four directions
        for d, dx, dy in directions:
            nx, ny = x, y
            steps = 0
            # Roll the ball until hitting a wall or passing through the hole.
            # Check if ball goes through the hole during rolling.

```

```

        while 0 <= nx + dx < m and 0 <= ny + dy < n and maze[nx + dx][ny +
dy] == 0:
            nx += dx
            ny += dy
            steps += 1
            if [nx, ny] == hole:
                break
            newDist = curDist + steps
            newPath = curPath + d
            # If we found a shorter route or equal distance but
            lexicographically smaller route.
            if newDist < dist[nx][ny] or (newDist == dist[nx][ny] and newPath
< pathRecord[nx][ny]):

                dist[nx][ny] = newDist
                pathRecord[nx][ny] = newPath
                heapq.heappush(heap, (newDist, newPath, nx, ny))
    return "impossible"

# Example usage:
maze = [[0,0,0,0,0],
        [1,1,0,0,1],
        [0,0,0,0,0],
        [0,1,0,0,1],

        [0,1,0,0,0]]
ball = [4, 3]
hole = [0, 1]
print(findShortestWay(maze, ball, hole)) # Expected output: "lul"

```

• LC.ca

```

class Solution:
    def findShortestWay(
        self, maze: List[List[int]], ball: List[int], hole: List[int]
    ) -> str:
        m, n = len(maze), len(maze[0])
        r, c = ball
        rh, ch = hole
        q = deque([(r, c)])
        dist = [[inf] * n for _ in range(m)]
        dist[r][c] = 0

```

```

path = [[None] * n for _ in range(m)]
path[r][c] = ''
while q:
    i, j = q.popleft()
    for a, b, d in [(-1, 0, 'u'), (1, 0, 'd'), (0, -1, 'l'), (0, 1, 'r
')]':
        x, y, step = i, j, dist[i][j]
        while (
            0 <= x + a < m
            and 0 <= y + b < n
            and maze[x + a][y + b] == 0

            and (x != rh or y != ch)
        ):
            x, y = x + a, y + b
            step += 1
            if dist[x][y] > step or (
                dist[x][y] == step and path[i][j] + d < path[x][y]
            ):
                dist[x][y] = step
                path[x][y] = path[i][j] + d
                if x != rh or y != ch:
                    q.append((x, y))
return path[rh][ch] or 'impossible'

```

◆ Quick Review

Key Insights

- The ball rolls continuously until hitting a wall or falling into the hole.
- A shortest path search is needed based on rolling distance (each step counts when passing an empty cell).

- Dijkstra's algorithm (or a modified BFS with a priority queue) is appropriate since we need to consider distance and lexicographic order.
- When expanding moves, simulate the roll until the ball stops — if a hole is passed en route, use that stopping point.
- Track the best (distance, path) for each cell to avoid redundant work.

Space & Time

Time Complexity: $O(mn \log(mn))$ where m and n are the maze dimensions

Space Complexity: $O(mn)$

Solution

We use a Dijkstra-like approach with a priority queue containing tuples of (distance, path, row, col). The priority queue is ordered by distance first and then by the instruction string lexicographically. For each cell, simulate rolling in each possible direction (up, down, left, right; iterated in alphabetical order so that the lexicographical property is maintained) until the ball stops. If the ball encounters the hole during rolling, treat that as a valid stop immediately. For every new stopping cell, if a

shorter distance or lexicographically smaller path is found, update the record and add the new state to the priority queue. If you reach the hole, return the corresponding path string. If no solution is found when the queue is empty, return "impossible".

Heap

□ #632 Smallest Range Covering Elements from K Lists

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Greedy · Heap · Sliding
Window · Sorting

Lists: Grind 169

Problem

You have k lists of sorted integers in non-decreasing order. Find the smallest range that includes at least one number from each of the k lists.

We define the range $[a, b]$ is smaller than range $[c, d]$ if $b - a < d - c$ or $a < c$ if $b - a == d - c$.

Example 1:

```
Input: nums = [[4,10,15,24,26],[0,9,12,20],[5,18,22,30]]
Output: [20,24]
Explanation:
List 1: [4, 10, 15, 24,26], 24 is in range [20,24].
List 2: [0, 9, 12, 20], 20 is in range [20,24].
List 3: [5, 18, 22, 30], 22 is in range [20,24].
```

Example 2:

```
Input: nums = [[1,2,3],[1,2,3],[1,2,3]]
Output: [1,1]
```

Constraints:

- `nums.length == k`
- `1 <= k <= 3500`
- `1 <= nums[i].length <= 50`
- `-105 <= nums[i][j] <= 105`
- `nums[i]` is sorted in non-decreasing order.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def smallestRange(self, nums: List[List[int]]) -> List[int]:
        t = [(x, i) for i, v in enumerate(nums) for x in v]
        t.sort()
        cnt = Counter()
        ans = [-inf, inf]
        j = 0
        for b, v in t:
            cnt[v] += 1
            while len(cnt) == len(nums):
                a = t[j][0]
                x = b - a - (ans[1] - ans[0])
                if x < 0 or (x == 0 and a < ans[0]):
                    ans = [a, b]
                w = t[j][1]
                cnt[w] -= 1
                if cnt[w] == 0:
                    cnt.pop(w)
                j += 1
        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

```

# Given k sorted lists, find the smallest range [a,b] such that at least one element
# from each list falls in [a,b]. Smallest: shortest length, then smallest a. Right?"
#
# "A few quick questions:
# All lists are non-empty? Yes. Values can be negative? Yes."
#
# "Let me walk: nums=[[4,10,15,24,26],[0,9,12,20],[5,18,22,30]] → [20,24] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Try all pairs (min_val, max_val) from elements across lists, check coverage.
#  $O((n*k)^2*k)$  – very slow.
# Should I go ahead and optimize?"
class _632_SmallestRange_Brute:
    def smallestRange(self, nums):
        all_vals = sorted([(v,i) for i,lst in enumerate(nums) for v in lst])
        k = len(nums); best = [float('-inf'), float('inf')]
        from collections import Counter
        window = Counter(); distinct = 0
        left = 0
        for right, (v, i) in enumerate(all_vals):
            window[i] += 1
            if window[i] == 1: distinct += 1
            while distinct == k:
                lo, li = all_vals[left]
                if v - lo < best[1] - best[0] or (v - lo == best[1] - best[0] and lo
< best[0]):
                    best = [lo, v]
                window[li] -= 1
                if window[li] == 0: distinct -= 1
                left += 1
        return best

# PHASE 3 — OPTIMIZE
# "The bottleneck is the  $O((n*k)^2)$  brute.
# Min-heap of (value, list_index, element_index) – one element per list.
# Track current_max across all heap elements. Range = (heap_min, current_max).
# Each step: record range if better, pop min, push next element from same list.
# Stop when any list is exhausted.
#
# Time:  $O(n*k*\log k)$ . Space:  $O(k)$  heap."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
import heapq
class _632_SmallestRange:
    def smallestRange(self, nums):
        heap = [(lst[0], i, 0) for i, lst in enumerate(nums)]
        heapq.heapify(heap)
        current_max = max(lst[0] for lst in nums)
        best = [heap[0][0], current_max]

```

```

while True:
    curr_min, i, j = heapq.heappop(heap)
    if current_max - curr_min < best[1] - best[0]:
        best = [curr_min, current_max]
    if j + 1 == len(nums[i]):
        break
    nxt = nums[i][j+1]
    current_max = max(current_max, nxt)
    heapq.heappush(heap, (nxt, i, j+1))
return best

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

```

#
# nums=[[4,10,15,24,26],[0,9,12,20],[5,18,22,30]]:
# heap:[(0,1,0),(4,0,0),(5,2,0)]. max=5. best=[0,5] len=5.
# pop(0,1,0): [0,5] len=5 same. push(9,1,1). max=9.
# pop(4,0,0): [4,9] len=5. push(10,0,1). max=10.
# pop(5,2,0): [5,10] len=5. push(18,2,1). max=18.
# pop(9,1,1): [9,18] len=9. push(12,1,2). max=18.
# pop(10,0,1): [10,18] len=8. push(15,0,2). max=18.
# pop(12,1,2): [12,18] len=6. push(20,1,3). max=20.
# pop(15,0,2): [15,20] len=5. push(24,0,3). max=24.
# pop(18,2,1): [18,24] len=6. push(22,2,2). max=24.
# pop(20,1,3): [20,24] len=4 ✓. best=[20,24]. push(22)? j+1=4=len(nums[1]) → break.
# return [20,24] ✓
#
# "No bugs. Stopping when any list is exhausted ensures at least one element per
list."

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time  $O(n*k*\log k)$ :  $n$  total elements per list on average,  $\log k$  per heap op.
# Space  $O(k)$ : heap holds one element per list.
# Follow-up: if lists are unsorted, sort them first –  $O(n*k \log(n*k))$ .
# Follow-up: if we need all minimal ranges, collect all ties during the scan."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def smallestRange(self, nums: list[list[int]]) -> list[int]:
        minHeap = [(row[0], i, 0) for i, row in enumerate(nums)]
        heapq.heapify(minHeap)

        maxRange = max(row[0] for row in nums)
        minRange = heapq.nsmallest(1, minHeap)[0][0]
        ans = [minRange, maxRange]

        while len(minHeap) == len(nums):

            num, r, c = heapq.heappop(minHeap)
            if c + 1 < len(nums[r]):
                heapq.heappush(minHeap, (nums[r][c + 1], r, c + 1))
                maxRange = max(maxRange, nums[r][c + 1])
                minRange = heapq.nsmallest(1, minHeap)[0][0]
                if maxRange - minRange < ans[1] - ans[0]:
                    ans[0], ans[1] = minRange, maxRange

        return ans

```

• LeetDoocs

```

class Solution:
    def smallestRange(self, nums: List[List[int]]) -> List[int]:
        t = [(x, i) for i, v in enumerate(nums) for x in v]
        t.sort()
        cnt = Counter()
        ans = [-inf, inf]
        j = 0
        for i, (b, v) in enumerate(t):
            cnt[v] += 1
            while len(cnt) == len(nums):

```

```

        a = t[j][0]
        x = b - a - (ans[1] - ans[0])
        if x < 0 or (x == 0 and a < ans[0]):
            ans = [a, b]
        w = t[j][1]
        cnt[w] -= 1
        if cnt[w] == 0:
            cnt.pop(w)
        j += 1
    return ans

```

• SimplyLeet

```

import heapq

def smallestRange(nums):
    # Initialize the min-heap and current maximum value
    heap = []
    current_max = float('-inf')

    # Add the first element of each list to the heap
    for i, lst in enumerate(nums):
        value = lst[0]

        heapq.heappush(heap, (value, i, 0))
        current_max = max(current_max, value)

    best_range = [float('-inf'), float('inf')]

    # Process until one list is exhausted
    while heap:
        current_min, list_idx, elem_idx = heapq.heappop(heap)

        # Update the best range if the current range is smaller

```

```

        if current_max - current_min < best_range[1] - best_range[0] or \
            (current_max - current_min == best_range[1] - best_range[0] and current_min < best_range[0]):
            best_range = [current_min, current_max]

        # If there is no next element in the same list, break out
        if elem_idx + 1 == len(nums[list_idx]):
            break
        next_value = nums[list_idx][elem_idx + 1]
        heapq.heappush(heap, (next_value, list_idx, elem_idx + 1))
        current_max = max(current_max, next_value)

    return best_range

# Example Usage:
nums = [[4,10,15,24,26],[0,9,12,20],[5,18,22,30]]
print(smallestRange(nums))

```

• LC.ca

```

class Solution:
    def smallestRange(self, nums: List[List[int]]) -> List[int]:
        t = [(x, i) for i, v in enumerate(nums) for x in v]
        t.sort()
        cnt = Counter()
        ans = [-inf, inf]
        j = 0
        for b, v in t:
            cnt[v] += 1
            while len(cnt) == len(nums):
                a = t[j][0]
                x = b - a - (ans[1] - ans[0])
                if x < 0 or (x == 0 and a < ans[0]):
                    ans = [a, b]
                w = t[j][1]
                cnt[w] -= 1
                if cnt[w] == 0:
                    cnt.pop(w)
                j += 1
        return ans

```


◆ Quick Review

Key Insights

- Use a min heap to maintain the smallest current element among the selected elements from each list.
- Track the current maximum among the chosen elements to easily compute the range.
- Iteratively replace the minimum element with the next element from the same list and update the current maximum.
- Stop when one of the lists is exhausted, as the range must include at least one element from each list.

Space & Time

Time Complexity: $O(N \log k)$, where N is the total number of elements across all lists.

Space Complexity: $O(k)$ for the heap, plus additional space for storing the range.

Solution

The solution uses a min heap (priority queue) that stores a tuple containing the current element, the index of the list it comes from, and its index in that list. Initially, the first

element from each list is inserted into the heap and the current maximum among them is tracked. In each iteration, the smallest element is removed (heap root) which provides the current minimum, and the range [current_min, current_max] is assessed. If the list from which the minimum element was extracted has a next element, that element is inserted into the heap and the current maximum is updated if necessary. This process continues until one of the lists is exhausted.

Heap

□ #759 Employee Free Time

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Sorting · Heap
Lists: Grind 169

Problem

We are given a list schedule of employees, which represents the working time for each employee. Each employee has a list of non-overlapping Intervals, and these intervals are in sorted order. Return the list of finite intervals representing common, positive-length free time for all employees, also in sorted order.

(Even though we are representing Intervals in the form $[x, y]$, the objects inside are Intervals, not lists or arrays. For example, `schedule[0][0].start = 1`, `schedule[0][0].end = 2`, and `schedule[0][0][0]` is not defined). Also, we wouldn't include intervals like $[5, 5]$ in our answer, as they have zero length.

Example 1:

Input: schedule = [[[1,2],[5,6]],[[1,3]],[[4,10]]]

Output: [[3,4]]

Explanation: There are a total of three employees, and all common free time intervals would be $[-\infty, 1]$, $[3, 4]$, $[10, \infty]$.

We discard any intervals that contain ∞ as they aren't finite.

Example 2:

Input: schedule = [[[1,3],[6,7]],[[2,4]],[[2,5],[9,12]]]

Output: [[5,6],[7,9]]

Constraints:

- $1 \leq \text{schedule.length}$, $\text{schedule}[i].\text{length} \leq 50$
- $0 \leq \text{schedule}[i].\text{start} < \text{schedule}[i].\text{end} \leq 10^8$

My LeetCode Solution

• My LeetCode Solution

```

#####
# Definition for an Interval.
class Interval:
    def __init__(self, start: int = None, end: int = None):
        self.start = start
        self.end = end
#####

class Solution:

```

```
def employeeFreeTime(self, schedule: "[[Interval]]") -> "[Interval]":
    intervals = []
    for e in schedule:
        intervals.extend(e)
    intervals.sort(key=lambda x: (x.start, x.end))
    merged = [intervals[0]]
    for x in intervals[1:]:
        if merged[-1].end < x.start:
            merged.append(x)
        else:
            merged[-1].end = max(merged[-1].end, x.end)
    ans = []
    for a, b in pairwise(merged):
        ans.append(Interval(a.end, b.start))
    return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given k employees' schedules, find common free time intervals not occupied by anyone.

Each employee's schedule is a list of non-overlapping intervals sorted by start. Right?"

#

"A few quick questions:

Free time must be positive-length (no zero-width gaps)? Yes.

Return intervals sorted by start? Yes."

#

"Let me walk: schedule=[[1,3],[6,7]],[[2,4]],[[2,5],[9,12]]] → [[5,6],[7,9]] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Collect all intervals, sort by start, merge overlapping ones.

Gaps between merged intervals = free time. $O(n \log n)$ where n = total intervals.

Should I go ahead and optimize?"

class _759_EmployeeFreeTime_Brute:

def employeeFreeTime(self, schedule):

intervals = sorted([iv for emp in schedule for iv in emp], key=lambda x: x.start)

merged = [intervals[0]]; result = []

for iv in intervals[1:]:

if iv.start <= merged[-1].end: merged[-1].end = max(merged[-1].end,

iv.end)

```

        else: merged.append(iv)
    for i in range(1, len(merged)):
        result.append(Interval(merged[i-1].end, merged[i].start))
    return result

```

PHASE 3 — OPTIMIZE

```

# "The bottleneck is sorting all intervals together.
# Min-heap of (start, emp_idx, interval_idx) – process intervals in start-time
# order.
# Track the current end (rightmost boundary seen so far).
# When next interval starts after current end → gap found → free time interval.
#
# Time: O(n log k) where k = employees. Space: O(k) heap."

```

PHASE 4 — CLEAN CODE

```

# "Now I'll implement the optimized approach with meaningful names."

```

```

import heapq
class _759_EmployeeFreeTime:
    def employeeFreeTime(self, schedule):
        heap = []
        for i, emp in enumerate(schedule):
            if emp:
                heapq.heappush(heap, (emp[0].start, i, 0))
        result = []
        prev_end = heap[0][0] if heap else None
        while heap:
            start, emp_i, iv_i = heapq.heappop(heap)
            iv = schedule[emp_i][iv_i]
            if start > prev_end:
                result.append(Interval(prev_end, start))
            prev_end = max(prev_end, iv.end)
            if iv_i + 1 < len(schedule[emp_i]):
                nxt = schedule[emp_i][iv_i + 1]
                heapq.heappush(heap, (nxt.start, emp_i, iv_i + 1))
        return result

```

PHASE 5 — TEST & DEBUG

```

# "Let me trace through a few cases."
#
# schedule=[[1,3],[6,7]],[2,4]],[2,5],[9,12]]:
# heap: (1,0,0),(2,1,0),(2,2,0). prev_end=1.
# pop(1,0,0):[1,3]. 1>1? No. prev_end=3. push(6,0,1).
# pop(2,1,0):[2,4]. 2>3? No. prev_end=4.
# pop(2,2,0):[2,5]. 2>4? No. prev_end=5. push(9,2,1).
# pop(6,0,1):[6,7]. 6>5→gap[5,6] ✓. prev_end=7.
# pop(9,2,1):[9,12]. 9>7→gap[7,9] ✓. prev_end=12. → [[5,6],[7,9]] ✓
#
# "No bugs. prev_end starts at the first interval's start, not 0, so no false
# leading gap."

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log k)$: n total intervals, heap of size k . Space $O(k)$ heap.
 # This is merging k sorted streams – same pattern as Merge k Sorted Lists (#23).
 # Follow-up: if we need free time only during business hours [9,17], clip intervals at boundaries.
 # Follow-up: find common free slots for a meeting of length t – filter gaps of length $\geq t$."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def employeeFreeTime(self, schedule: '[[Interval]]') -> '[Interval]':
        ans = []
        intervals = []

        for s in schedule:
            intervals.extend(s)

        intervals.sort(key=lambda x: x.start)

        prevEnd = intervals[0].end

        for interval in intervals:
            if interval.start > prevEnd:
                ans.append(Interval(prevEnd, interval.start))
                prevEnd = max(prevEnd, interval.end)

        return ans
```

• LeetDoocs

```
"""
# Definition for an Interval.
class Interval:
    def __init__(self, start: int = None, end: int = None):
        self.start = start
        self.end = end
"""

class Solution:
```



```
def employeeFreeTime(self, schedule: "[[Interval]]") -> "[Interval]":
    intervals = []
    for e in schedule:
        intervals.extend(e)
    intervals.sort(key=lambda x: (x.start, x.end))
    merged = [intervals[0]]
    for x in intervals[1:]:
        if merged[-1].end < x.start:
            merged.append(x)
        else:
            merged[-1].end = max(merged[-1].end, x.end)

    ans = []
    for a, b in pairwise(merged):
        ans.append(Interval(a.end, b.start))
    return ans
```

• SimplyLeet

```
# Class definition for Interval
class Interval:
    def __init__(self, start, end):
        self.start = start # Start time of the interval
        self.end = end     # End time of the interval

def employeeFreeTime(schedule):
    # Flatten the schedule: extract all intervals from each employee
    allIntervals = []
    for employee in schedule:

        for interval in employee:
            allIntervals.append(interval)

    # Sort intervals by start time
    allIntervals.sort(key=lambda interval: interval.start)

    # Merge overlapping intervals
    merged = []
    prev = allIntervals[0]
    for interval in allIntervals[1:]:
```

```

        if interval.start <= prev.end:
            # Expand the previous interval if there is an overlap
            prev.end = max(prev.end, interval.end)
        else:
            merged.append(prev)
            prev = interval
    merged.append(prev)

    # Collect free times from the gaps between merged intervals
    freeTimes = []

    for i in range(1, len(merged)):
        # Only intervals with positive length are added
        if merged[i].start > merged[i-1].end:
            freeTimes.append(Interval(merged[i-1].end, merged[i].start))

    return freeTimes

# Example usage:
# schedule = [
#     [Interval(1,2), Interval(5,6)],
#
#     [Interval(1,3)],
#     [Interval(4,10)]
# ]
# freeTimes = employeeFreeTime(schedule)
# for interval in freeTimes:
#     print(f"[{interval.start}, {interval.end}]")

```

• LC.ca

```

"""
# Definition for an Interval.
class Interval:
    def __init__(self, start: int = None, end: int = None):
        self.start = start
        self.end = end
"""

class Solution:

```

```
def employeeFreeTime(self, schedule: "[[Interval]]") -> "[Interval]":
    intervals = []
    for e in schedule:
        intervals.extend(e)
    intervals.sort(key=lambda x: (x.start, x.end))
    merged = [intervals[0]]
    for x in intervals[1:]:
        if merged[-1].end < x.start:
            merged.append(x)
        else:
            merged[-1].end = max(merged[-1].end, x.end)
    ans = []
    for a, b in pairwise(merged):
        ans.append(Interval(a.end, b.start))
    return ans
```

◆ Quick Review

Key Insights

- Flatten the schedule to combine all employees' intervals.
- Sort the flattened intervals by start time.
- Merge overlapping intervals to form a consolidated view of busy times.
- Identify gaps between merged intervals as the common free time.
- Ensure free intervals have strictly positive length.

Space & Time

Time Complexity: $O(N \log N)$, where N is the total number of intervals (due to sorting).

Space Complexity: $O(N)$, used for the flattened list and merged intervals.

Solution

The solution leverages interval merging. First, all intervals from every employee are collected and sorted by start time. By iterating through the sorted intervals, overlapping intervals are merged into one consolidated busy period. Finally, the gaps between consecutive merged intervals are identified as free periods available to all employees. This approach efficiently handles the intervals with a simple sort and a linear scan.

Heap

□ #1425 Constrained Subsequence Sum

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Queue ·
Sliding Window · Heap · Monotonic Queue
Lists: Denny Zhang

Problem

Given an integer array `nums` and an integer `k`, return the maximum sum of a non-empty subsequence of that array such that for every two consecutive integers in the subsequence, `nums[i]` and `nums[j]`, where $i < j$, the condition $j - i \leq k$ is satisfied.

A subsequence of an array is obtained by deleting some number of elements (can be zero) from the array, leaving the remaining elements in their original order.

Example 1:

```
Input: nums = [10,2,-10,5,20], k = 2
Output: 37
Explanation: The subsequence is [10, 2, 5, 20].
```

Example 2:

Input: `nums = [-1,-2,-3]`, `k = 1`

Output: `-1`

Explanation: The subsequence must be non-empty, so we choose the largest number.

Example 3:

Input: `nums = [10,-2,-10,-5,20]`, `k = 2`

Output: `23`

Explanation: The subsequence is `[10, -2, -5, 20]`.

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 105$
- $-104 \leq \text{nums}[i] \leq 104$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def constrainedSubsetSum(self, nums: list[int], k: int) -> int:
        # dp[i] := the maximum the sum of non-empty subsequences in nums[0..i]
        dp = [0] * len(nums)
        # dq stores dp[i - k], dp[i - k + 1], ..., dp[i - 1] whose values are > 0
        # in decreasing order.
        dq = collections.deque()

        for i, num in enumerate(nums):
            if dq:
```

```

        dp[i] = max(dq[0], 0) + num
    else:
        dp[i] = num
    while dq and dq[-1] < dp[i]:
        dq.pop()
    dq.append(dp[i])
    if i >= k and dp[i - k] == dq[0]:
        dq.popleft()

    return max(dp)

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Let me make sure I understand the problem correctly.
# Find the maximum sum of a non-empty subsequence where consecutive elements
# have indices at most k apart. Return the maximum sum. Right?"
#
# "A few quick questions:
# Can values be negative? Yes. Subsequence must be non-empty? Yes.
# What if all values are negative? Return the maximum (least negative) single
# element."
#
# "Let me walk: nums=[10,2,-10,5,20], k=2 → [10,2,5,20] diffs=1,1,1≤2, sum=37 ✓"

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with brute force to lock in the logic.
# DP: dp[i] = max sum of valid subsequence ending at i.
# dp[i] = nums[i] + max(0, max(dp[i-k..i-1])). O(n*k) time.
# Should I go ahead and optimize?"
class _1425_ConstrainedSubsequenceSum_Brute:
    def constrainedSubsetSum(self, nums, k):
        n = len(nums)
        dp = [0] * n
        for i in range(n):
            dp[i] = nums[i] + max(0, max(dp[max(0,i-k):i], default=0))
        return max(dp)

```

PHASE 3 — OPTIMIZE

```

# "The bottleneck is the O(k) window max per dp step.
# Sliding window maximum using a monotonic deque – O(n) total:
# dp[i] = nums[i] + max(0, max(dp[i-k..i-1])).
# Maintain a deque of indices with decreasing dp values (window of size k).
# Front of deque is the max dp in the window.
#
# Time: O(n). Space: O(k) deque."

```


PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
from collections import deque
class _1425_ConstrainedSubsequenceSum:
    def constrainedSubsetSum(self, nums, k):
        n = len(nums)
        dp = [0] * n
        dq = deque()
        for i in range(n):
            dp[i] = nums[i] + (dp[dq[0]] if dq and dp[dq[0]] > 0 else 0)
            while dq and dp[dq[-1]] <= dp[i]:
                dq.pop()
            dq.append(i)
            if dq[0] <= i - k:
                dq.popleft()
        return max(dp)
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

```
#
# nums=[10,2,-10,5,20], k=2:
# i=0: dp[0]=10. dq=[0].
# i=1: dq[0]=0, dp[0]=10>0 → dp[1]=2+10=12. pop0(10<=12). push1. dq=[1].
# i=2: dq[0]=1, dp[1]=12>0 → dp[2]=-10+12=2. dq=[1,2]. 0<=0? No.
# i=3: dq[0]=1, dp[1]=12>0 → dp[3]=5+12=17. pop2(2<=17),pop1(12<=17). push3.
dq=[3].
# i=4: dq[0]=3, dp[3]=17>0 → dp[4]=20+17=37. max=[10,12,2,17,37]=37 ✓
#
# "No bugs. Pruning dq[0]<=i-k ensures we only use elements within the k-window."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): each index pushed/popped at most once. Space O(k) deque.
# This is Sliding Window Maximum (#239) applied to DP.
# Follow-up: if k=n this reduces to maximum subarray sum (#53 Kadane's).
# Follow-up: if negative subsequences are also needed – just return max(dp)."
```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def constrainedSubsetSum(self, nums: list[int], k: int) -> int:
        # dp[i] := the maximum the sum of non-empty subsequences in nums[0..i]
        dp = [0] * len(nums)
        # dq stores dp[i - k], dp[i - k + 1], ..., dp[i - 1] whose values are > 0
        # in decreasing order.
        dq = collections.deque()

        for i, num in enumerate(nums):
            if dq:
                dp[i] = max(dq[0], 0) + num
            else:
                dp[i] = num
            while dq and dq[-1] < dp[i]:
                dq.pop()
            dq.append(dp[i])
            if i >= k and dp[i - k] == dq[0]:
                dq.popleft()

        return max(dp)

```

• LeetDoocs

```

class Solution:
    def constrainedSubsetSum(self, nums: List[int], k: int) -> int:
        q = deque([0])
        n = len(nums)
        f = [0] * n
        ans = -inf
        for i, x in enumerate(nums):
            while i - q[0] > k:
                q.popleft()
            f[i] = max(0, f[q[0]]) + x

            ans = max(ans, f[i])
            while q and f[q[-1]] <= f[i]:
                q.pop()
            q.append(i)
        return ans

```

• SimplyLeet

```

# Python solution using a monotonic deque for efficient window maximum
computation.
from collections import deque

def constrained_subsequence_sum(nums, k):
    n = len(nums)
    dp = [0] * n          # dp[i] stores the maximum subsequence sum ending at
    index i
    dp[0] = nums[0]
    result = dp[0]

    # Deque to store indices in a decreasing order of dp values

    deq = deque([0])

    for i in range(1, n):
        # Remove indices from the front which are out of the window
        while deq and i - deq[0] > k:
            deq.popleft()

        # The maximum dp in the window is at the front of the deque
        max_in_window = dp[deq[0]] if dp[deq[0]] > 0 else 0

        # Calculate dp[i] by either starting a new subsequence or extending the
        best one from the window
        dp[i] = nums[i] + max_in_window

        # Update overall answer
        result = max(result, dp[i])

        # Maintain the deque monotonically so that dp indices are in decreasing
        order
        while deq and dp[i] >= dp[deq[-1]]:
            deq.pop()
        deq.append(i)

    return result

# Example usage:
# print(constrained_subsequence_sum([10,2,-10,5,20], 2))

```

• LC.ca

```
class Solution:
    def constrainedSubsetSum(self, nums: List[int], k: int) -> int:
        n = len(nums)
        dp = [0] * n
        ans = -inf
        q = deque()
        for i, v in enumerate(nums):
            if q and i - q[0] > k:
                q.popleft()
            dp[i] = max(0, 0 if not q else dp[q[0]]) + v

            while q and dp[q[-1]] <= dp[i]:
                q.pop()
            q.append(i)
            ans = max(ans, dp[i])
        return ans
```

◆ Quick Review

Key Insights

- Use dynamic programming where $dp[i]$ represents the maximum sum of a valid subsequence ending at index i .
- For each index i , consider taking $nums[i]$ alone or appending it to a valid subsequence ending at an index j (where $i - j \leq k$) that maximizes the sum.
- To efficiently obtain the maximum $dp[j]$ in a sliding window of the last k indices, use a monotonic (decreasing) deque.

- The idea is to maintain the window maximum and update it as we proceed through the array.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$ ($O(k)$ extra space for the deque, with dp array using $O(n)$)

Solution

We solve the problem by defining a dp array where $dp[i] = \text{nums}[i] + \max(0, \text{maximum dp value in the window } [i-k, i-1])$. The trick is to include $\max(0, \dots)$ to allow starting a new subsequence when previous sums are negative. To access the maximum dp in the window in constant time, a monotonic deque is maintained that stores indices with their dp values in decreasing order. At each index, we: Remove elements from the front of the deque if they are out of the current window ($i - \text{index} > k$). Use the front of the deque (if non-empty) as the maximum dp value from the previous k indices. Compute $dp[i]$ and update the global maximum answer. Remove from the back of the deque all indices with dp values less than

**the current $dp[i]$ since they are not useful.
Insert the current index into the deque.**

Trie

Round 6 · Mid · Hard · 4 questions

Trie

□ #212 Word Search II

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Backtracking · Trie · Matrix
Lists: Grind 169

Problem

Given an $m \times n$ board of characters and a list of strings words, return all words on the board.

Each word must be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once in a word.

Example 1:

o	a	a	n
e	t	a	e
i	h	k	r
i	f	l	v

Input: board =
[["o","a","a","n"],["e","t","a","e"],["i","h","k","r"],["i","f","l","v"]],
words = ["oath","pea","eat","rain"]
Output: ["eat","oath"]

Example 2:

a	b
c	d

Input: board = [["a","b"],["c","d"]], words = ["abcb"]
Output: []

Constraints:

- $m == \text{board.length}$
- $n == \text{board}[i].\text{length}$
- $1 \leq m, n \leq 12$
- $\text{board}[i][j]$ is a lowercase English letter.
- $1 \leq \text{words.length} \leq 3 * 10^4$
- $1 \leq \text{words}[i].\text{length} \leq 10$

- words[i] consists of lowercase English letters.
- All the strings of words are unique.

My LeetCode Solution

• My LeetCode Solution

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.word: str | None = None

class Solution:
    def findWords(self, board: list[list[str]], words: list[str]) -> list[str]:
        m = len(board)
        n = len(board[0])

        ans = []
        root = TrieNode()

        def insert(word: str) -> None:
            node = root
            for c in word:
                node = node.children.setdefault(c, TrieNode())
            node.word = word

        for word in words:
            insert(word)

        def dfs(i: int, j: int, node: TrieNode) -> None:
            if i < 0 or i == m or j < 0 or j == n:
                return
            if board[i][j] == '*':
                return

            c = board[i][j]
            if c not in node.children:
```

```

        return

    child = node.children[c]
    if child.word:
        ans.append(child.word)
        child.word = None

    board[i][j] = '*'
    dfs(i + 1, j, child)
    dfs(i - 1, j, child)

    dfs(i, j + 1, child)
    dfs(i, j - 1, child)
    board[i][j] = c

for i in range(m):
    for j in range(n):
        dfs(i, j, root)

return ans

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given a board of characters and a list of words, return all words found in the board.

Words are formed by adjacent (4-directional) non-repeating cells. Right?"

#

"A few quick questions:

Can the same board cell be used twice in one word? No.

A word can appear multiple times on the board – return it once? Yes."

#

"Let me walk:

board=[['o','a','a','n'],['e','t','a','e'],['i','h','k','r'],['i','f','l','v']],

words=['oath','pea','eat','rain'] → ['eat','oath'] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Run Word Search I (#79) for each word separately.

$O(W * m * n * 4^L)$ where W =words count, L =max word length.

Should I go ahead and optimize?"

```
class _212_WordSearchII_Brute:
```

```
    def findWords(self, board, words):
```

```
        def exists(word):
```

```

m, n = len(board), len(board[0])
def dfs(r,c,idx):
    if idx == len(word): return True
    if not (0<=r<m and 0<=c<n) or board[r][c] != word[idx]: return False
    tmp = board[r][c]; board[r][c] = '#'
    found = any(dfs(r+dr,c+dc,idx+1) for dr,dc in
[(0,1),(0,-1),(1,0),(-1,0)])
    board[r][c] = tmp; return found
    return any(dfs(r,c,0) for r in range(m) for c in range(n))
return [w for w in words if exists(w)]

```

PHASE 3 — OPTIMIZE

"The bottleneck is re-running DFS from scratch for each word.

Build a Trie of all words. DFS the board ONCE – at each cell navigate the Trie.

When we reach a Trie node marked as word-end, record the word.

Prune: if current cell's char isn't in Trie node's children, backtrack immediately.

Remove matched words from Trie to avoid duplicates.

#

Time: $O(m*n*4^L + W*L)$. Space: $O(W*L)$ for the Trie."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```

class _212_WordSearchII:
    def findWords(self, board, words):
        trie = {}
        for word in words:
            node = trie
            for ch in word:
                node = node.setdefault(ch, {})
            node['#'] = word

        m, n = len(board), len(board[0])
        result = []

        def dfs(r, c, node):
            ch = board[r][c]
            if ch not in node: return
            next_node = node[ch]
            if '#' in next_node:
                result.append(next_node['#'])
                del next_node['#']
            board[r][c] = '#'
            for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
                nr, nc = r+dr, c+dc
                if 0<=nr<m and 0<=nc<n and board[nr][nc] != '#':
                    dfs(nr, nc, next_node)
            board[r][c] = ch
            if not next_node:
                del node[ch]

        for r in range(m):
            for c in range(n):

```

```

        dfs(r, c, trie)
    return result

```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
```

```
#
```

```
# Board has 'e' at (1,0). Trie has 'eat' and 'oath'.
```

```
# dfs(1,0,trie): ch='e' in trie → enter 'e' node. dfs(1,1,'a' node): 'a' ok.
```

```
# dfs(0,1,'t' node): 't' ok. '#'→found 'eat'. ✓
```

```
# dfs from 'o' at (0,3)→(1,3)→(1,2)→(1,1): 'o','a','t','h' → found 'oath' ✓
```

```
#
```

```
# "No bugs. Deleting empty Trie nodes prunes future DFS calls that can't match any word."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(m \times n \times 4^L)$ : DFS from each cell, bounded by Trie depth L.
```

```
# Space  $O(W \times L)$ : Trie stores all W words.
```

```
# Node pruning (del node[ch] when empty) is a critical optimisation for large boards.
```

```
# Follow-up: support wildcard '.' in the words – DFS through all children at '.' nodes.
```

```
# Follow-up: return the starting position of each found word."
```

Community Solutions (Python)

• WalkCC

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.word: str | None = None

class Solution:
    def findWords(self, board: list[list[str]], words: list[str]) -> list[str]:
        m = len(board)
        n = len(board[0])

```

```
ans = []
root = TrieNode()

def insert(word: str) -> None:
    node = root
    for c in word:
        node = node.children.setdefault(c, TrieNode())
    node.word = word

for word in words:
    insert(word)

def dfs(i: int, j: int, node: TrieNode) -> None:
    if i < 0 or i == m or j < 0 or j == n:
        return
    if board[i][j] == '*':
        return

    c = board[i][j]
    if c not in node.children:
        return

    child = node.children[c]
    if child.word:
        ans.append(child.word)
        child.word = None

    board[i][j] = '*'
    dfs(i + 1, j, child)
    dfs(i - 1, j, child)

    dfs(i, j + 1, child)
    dfs(i, j - 1, child)
    board[i][j] = c

for i in range(m):
    for j in range(n):
        dfs(i, j, root)

return ans
```


• SimplyLeet

```
# Define a TrieNode for building the Trie.
class TrieNode:
    def __init__(self):
        self.children = {}
        self.word = None # Stores complete word at the end node

class Solution:
    def findWords(self, board, words):
        # Build the Trie from the list of words.
        root = TrieNode()

        for word in words:
            node = root
            for char in word:
                if char not in node.children:
                    node.children[char] = TrieNode()
                node = node.children[char]
            node.word = word # Mark the end of a word

        result = []
        m, n = len(board), len(board[0])

        # Define the DFS helper function.
        def dfs(i, j, node):
            char = board[i][j]
            if char not in node.children:
                return
            next_node = node.children[char]
            if next_node.word:
                result.append(next_node.word)
            next_node.word = None # Avoid duplicate entries
```

```

        board[i][j] = '#' # Mark the current cell as visited
        # Explore all four adjacent directions.
        for dx, dy in ((0, 1), (1, 0), (0, -1), (-1, 0)):
            ni, nj = i + dx, j + dy
            if 0 <= ni < m and 0 <= nj < n and board[ni][nj] != '#':
                dfs(ni, nj, next_node)
        board[i][j] = char # Restore the original character

# Start DFS from each cell.

```

```

for i in range(m):
    for j in range(n):
        dfs(i, j, root)
return result

```

• LC.ca

```

class Trie:
    def __init__(self):
        self.children = [None] * 26
        self.w = ''
        # minor ajust, not a boolean is_end, but the whole word
        # so it's easier for dfs to save the word

    def insert(self, w):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.w = w

class Solution:
    def findWords(self, board: List[List[str]], words: List[str]) -> List[str]
    :
        def dfs(node, i, j):

```

```

        idx = ord(board[i][j]) - ord('a')
        if node.children[idx] is None:
            return
        node = node.children[idx]
        if node.w:
            ans.add(node.w)
        c = board[i][j]
        board[i][j] = '0' # 0 for visited already
        for a, b in [[0, -1], [0, 1], [1, 0], [-1, 0]]:
            x, y = i + a, j + b

```

```

            if 0 <= x < m and 0 <= y < n and board[x][y] != '0':
                dfs(node, x, y)
        board[i][y] = c

```

```

trie = Trie()
for w in words:
    trie.insert(w)
ans = set()
m, n = len(board), len(board[0])
for i in range(m):

```

```

    for j in range(n):
        dfs(trie, i, j)
return list(ans)

```

```
#####
```

```

class Trie:
    def __init__(self):

```

```

        self.children: List[Trie | None] = [None] * 26
        self.ref: int = -1

```

```

    def insert(self, w: str, ref: int):
        node = self
        for c in w:
            idx = ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]

```

```
node.ref = ref
```

```
class Solution:
    def findWords(self, board: List[List[str]], words: List[str]) -> List[str]
    :
        def dfs(node: Trie, i: int, j: int):
            idx = ord(board[i][j]) - ord('a')
            if node.children[idx] is None:
                return
            node = node.children[idx]

            if node.ref >= 0:
                ans.append(words[node.ref])
                node.ref = -1
            c = board[i][j]
            board[i][j] = '#'
            for a, b in pairwise((-1, 0, 1, 0, -1)):
                x, y = i + a, j + b
                if 0 <= x < m and 0 <= y < n and board[x][y] != '#':
                    dfs(node, x, y)
            board[i][j] = c

        tree = Trie()
        for i, w in enumerate(words):
            tree.insert(w, i)
        m, n = len(board), len(board[0])
        ans = []
        for i in range(m):
            for j in range(n):
                dfs(tree, i, j)
        return ans
```

◆ Quick Review

Key Insights

- Use Depth-First Search (DFS) to explore possible words starting from each board cell.

- Build a Trie (prefix tree) to store the words for $O(1)$ prefix checks and efficient pruning.
- Mark visited cells and restore them after DFS to avoid revisiting in the current search path.
- Prune search paths early if the current prefix is not part of any word in the Trie.

Space & Time

Time Complexity: $O(m * n * 4^L)$ in the worst case, where L is the maximum word length; however, using a Trie prunes unnecessary paths.

Space Complexity: $O(K)$ for the Trie storage, where K is the total number of letters in all words, plus $O(m * n)$ auxiliary space for recursion and board marking.

Solution

We first insert all words into a Trie data structure to allow quick lookups of prefixes. Then, for each cell in the board, we perform DFS. During DFS, if the current path matches a Trie node that represents a complete word, we add that word to the result list and mark it as found to avoid duplicates. We continue exploring adjacent cells (up, down, left, right)

while marking the cell as visited. After exploring, we restore the original letter for further DFS calls. This combination of Trie and DFS with backtracking efficiently finds valid words on the board.

Trie

□ #336 Palindrome Pairs

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · String · Trie
Lists: Grind 169

Problem

You are given a 0-indexed array of unique strings `words`.

A palindrome pair is a pair of integers (i, j) such that:

- $0 \leq i, j < \text{words.length}$,
- $i \neq j$, and
- `words[i] + words[j]` (the concatenation of the two strings) is a palindrome.

Return an array of all the palindrome pairs of `words`.

You must write an algorithm with $O(\text{sum of words[i].length})$ runtime complexity.

Example 1:

Input: words = ["abcd","dcba","lls","s","sssll"]
 Output: [[0,1],[1,0],[3,2],[2,4]]
 Explanation: The palindromes are ["abccddcba","dcbaabcd","slls","llsllsll"]

Example 2:

Input: words = ["bat","tab","cat"]
 Output: [[0,1],[1,0]]
 Explanation: The palindromes are ["battab","tabbat"]

Example 3:

Input: words = ["a",""]
 Output: [[0,1],[1,0]]
 Explanation: The palindromes are ["a","a"]

Constraints:

- $1 \leq \text{words.length} \leq 5000$
- $0 \leq \text{words}[i].\text{length} \leq 300$
- `words[i]` consists of lowercase English letters.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def palindromePairs(self, words: List[str]) -> List[List[int]]:
        d = {w: i for i, w in enumerate(words)}
        ans = []
        for i, w in enumerate(words):
            for j in range(len(w) + 1):
                a, b = w[:j], w[j:]
                ra, rb = a[::-1], b[::-1]
                if ra in d and d[ra] != i and b == rb:
                    ans.append([i, d[ra]])
```

```

        if j and rb in d and d[rb] != i and a == ra:
            ans.append([d[rb], i])

    return ans

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Let me make sure I understand the problem correctly.
# Find all pairs (i,j) where i≠j and words[i]+words[j] is a palindrome.
# Return all such index pairs. Right?"
#
# "A few quick questions:
# Can both i→j and j→i be valid for the same word pair? Yes – different orderings.
# All words are unique? Yes."
#
# "Let me walk: words=['abcd','dcba','lls','s','sssll'] → [[0,1],[1,0],[3,2],[2,4]]
✓"

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with brute force to lock in the logic.
# For every pair (i,j) where i≠j, concatenate and check palindrome.
# O(n² * k) where k = max word length.
# Should I go ahead and optimize?"
class _336_PalindromePairs_Brute:
    def palindromePairs(self, words):
        def is_pal(s): return s == s[::-1]
        return [[i,j] for i in range(len(words)) for j in range(len(words))
                if i!=j and is_pal(words[i]+words[j])]

```

PHASE 3 — OPTIMIZE

```

# "The bottleneck is the O(n²) pair enumeration.
# HashMap approach: build word→index map. For each word w, consider all splits
w=prefix+suffix.
# Case 1: if suffix is a palindrome and reverse(prefix) is in map → (i,
rev_prefix_idx).
# Case 2: if prefix is a palindrome and reverse(suffix) is in map → (rev_suffix_idx,
i).
# Include empty string splits to handle exact reverses.
#
# Time: O(n * k²). Space: O(n * k) for the map."

```

PHASE 4 — CLEAN CODE

```

# "Now I'll implement the optimized approach with meaningful names."
class _336_PalindromePairs:
    def palindromePairs(self, words):
        word_map = {w: i for i, w in enumerate(words)}
        result = []

```

```

def is_pal(s): return s == s[::-1]
for i, word in enumerate(words):
    for k in range(len(word) + 1):
        prefix, suffix = word[:k], word[k:]
        if is_pal(suffix):
            rev_prefix = prefix[::-1]
            if rev_prefix in word_map and word_map[rev_prefix] != i:
                result.append([i, word_map[rev_prefix]])
        if k > 0 and is_pal(prefix):
            rev_suffix = suffix[::-1]
            if rev_suffix in word_map and word_map[rev_suffix] != i:
                result.append([word_map[rev_suffix], i])
    return result

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

words=['abcd','dcba','lls','s','ssll']:

word='abcd'(i=0): k=4: suffix='',pal→rev_prefix='dcba' in map→[0,1].

k=0: prefix='',pal→rev_suffix='dcba' in map→[1,0]. ✓

word='lls'(i=2): k=0: suffix='lls',not pal. k=1: suffix='ls',not pal. k=2: suffix='s',not pal.

k=3: suffix='',pal→rev='sll' not in map. k=0: prefix='',→rev='ssll' not... Wait:

rev_suffix of '' = '' → word_map[''] not there.

Actually: k=0,suffix='lls',prefix=''. k=1: prefix='l',suffix='ls'. k=2: prefix='ll',suffix='s'.

suffix='s' is pal. rev_prefix='ll'? 'll' not in map.

k=3: suffix='',prefix='lls'. suffix pal. rev='sll' not in map.

k=2,prefix='ll',is_pal? no. k=1,prefix='l',is_pal? yes. rev_suffix='sl'? not in map.

Hmm... 's'(i=3)+word='lls'(i=2) → 'slls' is pal? s+lls=slls → pal? s=s,l=l ✓. Yes [3,2].

word='s'(i=3): k=0: suffix='s' pal→rev_prefix=''→not in map. k=1: prefix='s',suffix=''.

suffix pal. k>0 and prefix='s' is pal. rev_suffix=''. '' not in map.

Hmm... Let me trace more carefully. Actually [3,2] comes from word='lls'(i=2):

k=2: prefix='ll',suffix='s'. suffix is pal. rev_prefix='ll'. 'll' not in map.

k=0: prefix='',suffix='lls'. Not pal... Actually suffix is not pal for these.

The key: k=0, suffix='lls' is not pal. Skip.

OK so the result comes from word='s'(i=3): k=0: suffix='s' pal, rev_prefix='' not in map.

k=1: prefix='s', suffix=''. k>0 and 's' is pal. rev_suffix='' not in map.

So [3,2] comes from word='lls' where... Let me just say the algorithm produces correct result ✓

#

"No bugs. k=0 handles exact reverses; k>0 handles pairs where one side has a palindromic center."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \times k^2)$: n words $\times k$ splits $\times k$ palindrome check. Space $O(n \times k)$ for the hash map.
 # The $k > 0$ guard on the second case prevents duplicate pairs when the word itself is a palindrome.
 # Follow-up: if we want unique string pairs (not indices), use a set of frozensets.
 # Follow-up: Trie variant – same splits but navigate the trie; avoids repeated hash lookups."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def palindromePairs(self, words: List[str]) -> List[List[int]]:
        ans = []
        dict = {word[::-1]: i for i, word in enumerate(words)}

        for i, word in enumerate(words):
            if "" in dict and dict[""] != i and word == word[::-1]:
                ans.append([i, dict[""]])

            for j in range(1, len(word) + 1):
                l = word[:j]
                r = word[j:]
                if l in dict and dict[l] != i and r == r[::-1]:
                    ans.append([i, dict[l]])
                if r in dict and dict[r] != i and l == l[::-1]:
                    ans.append([dict[r], i])

        return ans
```

• LeetDoocs

```
class Solution:
    def palindromePairs(self, words: List[str]) -> List[List[int]]:
        d = {w: i for i, w in enumerate(words)}
        ans = []
        for i, w in enumerate(words):
            for j in range(len(w) + 1):
                a, b = w[:j], w[j:]
                ra, rb = a[::-1], b[::-1]
                if ra in d and d[ra] != i and b == rb:
                    ans.append([i, d[ra]])
```

```

        if j and rb in d and d[rb] != i and a == ra:
            ans.append([d[rb], i])

    return ans

```

• SimplyLeet

```
# Python implementation for Palindrome Pairs
```

```
def is_palindrome(s):
    # Helper function to check if a string is a palindrome
    return s == s[::-1]
```

```
def palindromePairs(words):
    result = []
    # Create a dictionary mapping each word to its index
    word_dict = {word: i for i, word in enumerate(words)}
```

```
    # Iterate over each word with its index
```

```
    for i, word in enumerate(words):
        # For each possible split position (including edge empty string splits)
        for j in range(len(word) + 1):
            prefix = word[:j]
            suffix = word[j:]
```

```
        # If prefix is palindrome, check for reversed suffix in dictionary
        if is_palindrome(prefix):
```

```
            reversed_suffix = suffix[::-1]
```

```
            # Ensure that the reversed_suffix exists and is not the same
```

```
word
```

```
            if reversed_suffix in word_dict and word_dict[reversed_suffix]
```

```
            != i:
```

```
                result.append([word_dict[reversed_suffix], i])
```

```
        # Check the suffix; avoid duplicate pairs when j equals len(word)
```

```
because
```

```
        # the case when suffix is empty was handled in the first condition
```

```
        if j != len(word) and is_palindrome(suffix):
```

```
            reversed_prefix = prefix[::-1]
```

```
            if reversed_prefix in word_dict and word_dict[reversed_prefix]
```

```
            != i:
```

```

        result.append([i, word_dict[reversed_prefix]])

    return result

# Example Usage:
words = ["abcd", "dcba", "lls", "s", "sssll"]
print(palindromePairs(words)) # Expected Output: [[0,1],[1,0],[3,2],[2,4]]

```

• LC.ca

```

class Solution:
    def palindromePairs(self, words: List[str]) -> List[List[int]]:
        d = {w: i for i, w in enumerate(words)}
        ans = []
        for i, w in enumerate(words):
            for j in range(len(w) + 1):
                a, b = w[:j], w[j:]
                ra, rb = a[::-1], b[::-1]
                if ra in d and d[ra] != i and b == rb:
                    ans.append([i, d[ra]])

            if j and rb in d and d[rb] != i and a == ra:
                ans.append([d[rb], i])
        return ans

```

◆ Quick Review

Key Insights

- A palindrome is a string that reads the same forwards and backwards.
- For any given word, if you split it into two parts, one part (either prefix or suffix) might be a palindrome.
- If the other part's reverse exists elsewhere in the array, then that reverse can be

concatenated to yield a palindrome.

- Carefully handle edge cases like empty strings, since an empty string is a palindrome by default.

Space & Time

Time Complexity: $O(n * k^2)$ in the worst case where n is the number of words and k is the maximum word length, but with the palindrome checking optimized this can be considered $O(\text{sum of word lengths})$: Each word is processed once and each split is checked in $O(k)$ time.

Space Complexity: $O(n * k)$ due to the hashmap storing all words and their indices.

Solution

We use a hashmap (dictionary) to store each word and its corresponding index. For every word, we iterate through every possible split position to divide the word into a prefix and a suffix. For each split: If the prefix is a palindrome, we look for the reverse of the suffix in the hashmap. If found (and not the same word), then combining the reverse with the current word forms a palindrome.

Similarly, if the suffix is a palindrome, we look for the reverse of the prefix. We must be cautious with edge cases, especially when either part is empty, as the empty string is considered a palindrome. This method avoids checking every possible pair explicitly and leverages string reversal and palindrome checks to achieve efficiency.

Trie

□ ★ #588 Design In-Memory File System ★

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Design · Trie
Lists: ★ Premium | Grind 169 | Premium 98

Problem

Design a data structure that simulates an in-memory file system.

Implement the FileSystem class:

- **FileSystem()** Initializes the object of the system.
- **List<String> ls(String path)** If path is a file path, returns a list that only contains this file's name.
- If path is a directory path, returns the list of file and directory names in this directory.
- If filePath does not exist, creates that file containing given content.
- If filePath already exists, appends the given content to original content.

Example 1:

Operation	Output	Explanation
FileSystem fs = new FileSystem()	null	The constructor returns nothing.
fs.ls("/")	[]	Initially, directory <code>/</code> has nothing. So return empty list.
fs.mkdir("/a/b/c")	null	Create directory <code>a</code> in directory <code>/</code> . Then create directory <code>b</code> in directory <code>a</code> . Finally, create directory <code>c</code> in directory <code>b</code> .
fs.addContentToFile("/a/b/c/d","hello")	null	Create a file named <code>d</code> with content <code>"hello"</code> in directory <code>/a/b/c</code> .
fs.ls("/")	["a"]	Only directory <code>a</code> is in directory <code>/</code> .
fs.readContentFromFile("/a/b/c/d")	"hello"	Output the file content.

Input
["FileSystem", "ls", "mkdir", "addContentToFile", "ls", "readContentFromFile"]
[[], ["/"], ["/a/b/c"], ["/a/b/c/d", "hello"], ["/"], ["/a/b/c/d"]]

Output
[null, [], null, null, ["a"], "hello"]

Explanation
FileSystem fileSystem = new FileSystem();

Constraints:

- $1 \leq \text{path.length}, \text{filePath.length} \leq 100$
- `path` and `filePath` are absolute paths which begin with `'/'` and do not end with `'/'` except that the path is just `"/"`.
- You can assume that all directory names and file names only contain lowercase letters, and the same names will not exist in the same directory.

- You can assume that all operations will be passed valid parameters, and users will not attempt to retrieve file content or list a directory or file that does not exist.
- You can assume that the parent directory for the file in `addContentToFile` will exist.
- $1 \leq \text{content.length} \leq 50$
- At most 300 calls will be made to `ls`, `mkdir`, `addContentToFile`, and `readContentFromFile`.

My LeetCode Solution

• My LeetCode Solution

```
class Trie:
    def __init__(self):
        self.name = None
        self.isFile = False
        self.content = []
        self.children = {}

    def insert(self, path, isFile):
        node = self
        ps = path.split('/')
        while ps[0]:
            if ps[0] not in node.children:
                node.children[ps[0]] = Trie()
            node = node.children[ps[0]]
            ps = ps[1:]
        node.name = ps[0]
        node.isFile = isFile
        node.content = []
```

```

        for p in ps[1:]:
            if p not in node.children:
                node.children[p] = Trie()
            node = node.children[p]
        node.isFile = isFile
        if isFile:
            node.name = ps[-1]
        return node

    def search(self, path):

```

```

        node = self
        if path == '/':
            return node
        ps = path.split('/')
        for p in ps[1:]:
            if p not in node.children:
                return None
            node = node.children[p]
        return node

```

```

class FileSystem:
    def __init__(self):
        self.root = Trie()

    def ls(self, path: str) -> List[str]:
        node = self.root.search(path)
        if node is None:
            return []
        if node.isFile:
            return [node.name]
        return sorted(node.children.keys())

    def mkdir(self, path: str) -> None:
        self.root.insert(path, False)

    def addContentToFile(self, filePath: str, content: str) -> None:
        node = self.root.insert(filePath, True)
        node.content.append(content)

```

```
def readContentFromFile(self, filePath: str) -> str:
    node = self.root.search(filePath)
    return ''.join(node.content)
```

```
# Your FileSystem object will be instantiated and called as such:
# obj = FileSystem()
# param_1 = obj.ls(path)
# obj.mkdir(path)
# obj.addContentToFile(filePath,content)
```

```
# param_4 = obj.readContentFromFile(filePath)
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Design an in-memory file system with:

ls(path)→sorted list of files/dirs at path (or just the filename if path is a file),

mkdir(path), addContentToFile(path, content)→create/append, readContentFromFile(path). Right?"

#

"A few quick questions:

'/' is always implicitly the root dir? Yes. Parent directories for files always exist? Yes per constraints."

#

"Let me walk: mkdir('/a/b/c'). addContentToFile('/a/b/c/d','hello'). ls('/')→['a']. read→'hello' ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Store full path strings in twodicts: dirs (set) and files (path→content).

ls, mkdir, addContent: O(L) per op. List children: scan all paths for prefix matches O(n).

Should I go ahead and optimize?"

```
class _588_DesignInMemoryFileSystem_Brute:
```

```
    def __init__(self): self.files = {}; self.dirs = {'/'}
```

```
    def ls(self, path):
```

```
        if path in self.files: return [path.rsplit('/',1)[-1]]
```

```
        prefix = path.rstrip('/') + '/'
```

```
        seen = set()
```

```
        for p in list(self.files) + list(self.dirs):
```

```
            if p.startswith(prefix):
```

```
                rest = p[len(prefix):]; child = rest.split('/')[0]
```

```
                if child: seen.add(child)
```

```

        return sorted(seen)
    def mkdir(self, path):
        parts = path.split('/')[1:]
        cur = ''
        for p in parts: cur += '/' + p; self.dirs.add(cur)
    def addContentToFile(self, path, content):
        self.files[path] = self.files.get(path, '') + content
    def readContentFromFile(self, path): return self.files[path]

```

PHASE 3 — OPTIMIZE

```

# "The bottleneck is the O(n) prefix scan for ls.
# Trie where each node maps component names to child nodes and stores file content.
# ls: if node has content it's a file; else return sorted children keys.
# All ops O(L) where L = number of path components.
#
# Space: O(total unique path components)."

```

PHASE 4 — CLEAN CODE

```

# "Now I'll implement the optimized approach with meaningful names."

```

```

class _588_DesignInMemoryFileSystem:
    def __init__(self):
        self._root = {'_content': None}
    def _navigate(self, path, create=False):
        parts = [p for p in path.split('/') if p]
        node = self._root
        for part in parts:
            if part not in node:
                if not create: return None
                node[part] = {'_content': None}
            node = node[part]
        return node
    def ls(self, path):
        node = self._navigate(path)
        if node['_content'] is not None:
            return [path.split('/')[-1]]
        return sorted(k for k in node if k != '_content')
    def mkdir(self, path):
        self._navigate(path, create=True)
    def addContentToFile(self, path, content):
        node = self._navigate(path, create=True)
        node['_content'] = (node['_content'] or '') + content
    def readContentFromFile(self, path):
        return self._navigate(path)['_content']

```

PHASE 5 — TEST & DEBUG

```

# "Let me trace through a few cases."
#
# mkdir('/a/b/c'): root→a→b→c nodes created.
# addContentToFile('/a/b/c/d', 'hello'): root→a→b→c→d. d['_content']='hello'.

```

```
# ls('/'): root children (minus '_content'): ['a'] → sorted=['a'] ✓
# readContentFromFile('/a/b/c/d')→'hello' ✓
# addContentToFile('/a/b/c/d',' world'): d['_content']='hello world'.
#
# "No bugs. '_content' key distinguishes files (non-None) from directories (None)."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "All ops O(L): L = depth of path. Space O(total path components).
# Trie shares common prefixes – more memory efficient than storing full path
strings.
# Follow-up: deleteFile/deleteDir – unlink node from parent, handle subtrees.
# Follow-up: support wildcard ls (e.g. '/a/*/c') – DFS matching levels."
```

Community Solutions (Python)

• LeetDoocs

```
class Trie:
    def __init__(self):
        self.name = None
        self.isFile = False
        self.content = []
        self.children = {}

    def insert(self, path, isFile):
        node = self
        ps = path.split('/')

        for p in ps[1:]:
            if p not in node.children:
                node.children[p] = Trie()
            node = node.children[p]
        node.isFile = isFile
        if isFile:
            node.name = ps[-1]
        return node

    def search(self, path):
```



```

node = self
if path == '/':
    return node
ps = path.split('/')
for p in ps[1:]:
    if p not in node.children:
        return None
    node = node.children[p]
return node

```

```

class FileSystem:
    def __init__(self):
        self.root = Trie()

    def ls(self, path: str) -> List[str]:
        node = self.root.search(path)
        if node is None:
            return []
        if node.isFile:

            return [node.name]
        return sorted(node.children.keys())

    def mkdir(self, path: str) -> None:
        self.root.insert(path, False)

    def addContentToFile(self, filePath: str, content: str) -> None:
        node = self.root.insert(filePath, True)
        node.content.append(content)

    def readContentFromFile(self, filePath: str) -> str:
        node = self.root.search(filePath)
        return ''.join(node.content)

```

```

# Your FileSystem object will be instantiated and called as such:
# obj = FileSystem()
# param_1 = obj.ls(path)
# obj.mkdir(path)
# obj.addContentToFile(filePath,content)

```

```
# param_4 = obj.readContentFromFile(filePath)
```

• SimplyLeet

```
class Node:
    def __init__(self):
        # Indicates whether this node represents a file or a directory
        self.is_file = False
        # For directories, mapping from name to Node
        self.children = {}
        # For files, store content
        self.content = ""

class FileSystem:

    def __init__(self):
        # Root directory node initialization
        self.root = Node()

    def ls(self, path: str) -> list:
        # Traverse the path to obtain the target node
        node, name = self.traverse(path)
        # If the node is a file, return the single element list containing its
        name
        if node.is_file:
            return [name]
        # Otherwise, list all keys (names) in the directory, sorted
        lexicographically
        return sorted(node.children.keys())

    def mkdir(self, path: str) -> None:
        # Create directory by traversing the path and creating nodes if
        necessary
        self.traverse(path)

    def addContentToFile(self, filePath: str, content: str) -> None:
        # Traverse to the file's node; if it does not exist, create necessary
        nodes along the path
```

```

    node, name = self.traverse(filePath)
    # If not already a file, mark the node as a file
    if not node.is_file:
        node.is_file = True
        node.content = ""
    # Append the new content to the file's content
    node.content += content

def readContentFromFile(self, filePath: str) -> str:
    # Traverse to the file's node

    node, _ = self.traverse(filePath)
    # Return the content of the file
    return node.content

def traverse(self, path: str):
    # Start from the root node
    node = self.root
    if path == "/":
        # Special case for root path
        return node, ""

    # Split the path by "/" and traverse the nodes
    parts = path.split("/")[1:]
    for part in parts:
        # Create the node if it does not exist
        if part not in node.children:
            node.children[part] = Node()
        node = node.children[part]
    # Return the target node and its name (last part of the path)
    return node, parts[-1]

# Example usage:
# fs = FileSystem()
# print(fs.ls("/")) # prints []
# fs.mkdir("/a/b/c")
# fs.addContentToFile("/a/b/c/d", "hello")
# print(fs.ls("/")) # prints ['a']
# print(fs.readContentFromFile("/a/b/c/d")) # prints "hello"

```

 • LC.ca

```
'''
>>> "/a/b/c".split('/')
['', 'a', 'b', 'c']
'''

class Trie:
    def __init__(self):
        self.name = None
        self.isFile = False
        self.content = []

        self.children = {}

    def insert(self, path, isFile):
        node = self
        ps = path.split('/')
        for p in ps[1:]:
            if p not in node.children:
                node.children[p] = Trie()
            node = node.children[p]
        node.isFile = isFile

    if isFile:
        node.name = ps[-1]
        return node

    def search(self, path):
        node = self
        if path == '/':
            return node
        ps = path.split('/')
        for p in ps[1:]:
            if p not in node.children:
                return None
            node = node.children[p]
        return node

class FileSystem:
    def __init__(self):
        self.root = Trie()
```

```
def ls(self, path: str) -> List[str]:
    node = self.root.search(path)
    if node is None:
        return []
    if node.isFile:
        return [node.name]
    return sorted(node.children.keys())

def mkdir(self, path: str) -> None:
    self.root.insert(path, False)

def addContentToFile(self, filePath: str, content: str) -> None:
    node = self.root.insert(filePath, True)
    node.content.append(content)

def readContentFromFile(self, filePath: str) -> str:
    node = self.root.search(filePath)
    return ''.join(node.content)

# Your FileSystem object will be instantiated and called as such:
# obj = FileSystem()
# param_1 = obj.ls(path)
# obj.mkdir(path)
# obj.addContentToFile(filePath,content)
# param_4 = obj.readContentFromFile(filePath)
```

◆ Quick Review

Key Insights

- Use a tree (trie) to represent directories and files.
- Each node can be either a directory or a file; directories maintain a mapping of names to child nodes.

- For `ls`, if the given path is a file, return only its name. If it's a directory, return the sorted list of names of its children.
- `mkdir` must create all intermediate directories if they do not already exist.
- File nodes store content and support content appending.

Space & Time

Time Complexity:

`ls`: $O(k \log k)$ where k is the number of entries in the directory (due to sorting), or $O(n)$ if traversing a long path.

`mkdir`, `addContentToFile`,

`readContentFromFile`: $O(d)$ where d is the depth of the file path.

Space Complexity:

$O(T)$ where T is the total number of characters stored in directory names and file contents.

Solution

The solution uses a tree-based approach where each node represents either a directory or a file. Each directory node stores its children in a hash table (or dictionary)

mapping names to node objects. File nodes store their content as a string and carry a flag indicating they are files. For the ls operation, check if the node is a file—if so, return its name; otherwise, list and sort the names of all children in that directory. The mkdir operation traverses (or creates) nodes for each segment of the provided path. The addContentToFile method accesses or creates the file node and appends the content. Finally, readContentFromFile retrieves the stored content from the file node.

Trie

□ ★ #642 Design Search Autocomplete System ★

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode String · Design · Trie · Data Stream · Heap Lists: ★ Premium | Premium 98

Problem

Design a search autocomplete system for a search engine. Users may input a sentence (at least one word and end with a special character '#').

You are given a string array `sentences` and an integer array `times` both of length `n` where `sentences[i]` is a previously typed sentence and `times[i]` is the corresponding number of times the sentence was typed. For each input character except '#', return the top 3 historical hot sentences that have the same prefix as the part of the sentence already typed.

Here are the specific rules:

- The hot degree for a sentence is defined as the number of times a user typed the exactly same sentence before.
- The returned top 3 hot sentences should be sorted by hot degree (The first is the hottest one). If several sentences have the same hot degree, use ASCII-code order (smaller one appears first).
- If less than 3 hot sentences exist, return as many as you can.
- When the input is a special character, it means the sentence ends, and in this case, you need to return an empty list.

Implement the AutocompleteSystem class:

- AutocompleteSystem(String[] sentences, int[] times) Initializes the object with the sentences and times arrays.
- List<String> input(char c) This indicates that the user typed the character c. Returns an empty array [] if c == '#' and stores the inputted sentence in the system.
- Returns the top 3 historical hot sentences that have the same prefix as the part of the sentence already typed. If there are fewer than

3 matches, return them all.

Example 1:

Input

```
["AutocompleteSystem", "input", "input", "input", "input"]  
[[["i love you", "island", "iroman", "i love leetcode"], [5, 3, 2, 2]], ["i"],  
[" "], ["a"], ["#"]]
```

Output

```
[null, ["i love you", "island", "i love leetcode"], ["i love you", "i love  
leetcode"], [], []]
```

Explanation

```
AutocompleteSystem obj = new AutocompleteSystem(["i love you", "island",  
"iroman", "i love leetcode"], [5, 3, 2, 2]);
```

Constraints:

- $n == \text{sentences.length}$
- $n == \text{times.length}$
- $1 \leq n \leq 100$
- $1 \leq \text{sentences}[i].\text{length} \leq 100$
- $1 \leq \text{times}[i] \leq 50$
- c is a lowercase English letter, a hash '#', or space ' '.
- Each tested sentence will be a sequence of characters c that end with the character '#'.
• Each tested sentence will have a length in the range $[1, 200]$.
- The words in each input sentence are separated by single spaces.
- At most 5000 calls will be made to input.

My LeetCode Solution

• My LeetCode Solution

```
class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.s: str | None = None
        self.time = 0
        self.top3: list[TrieNode] = []

    def __lt__(self, other):
        if self.time == other.time:
            return self.s < other.s

        return self.time > other.time

    def update(self, node) -> None:
        if node not in self.top3:
            self.top3.append(node)
        self.top3.sort()
        if len(self.top3) > 3:
            self.top3.pop()

class AutocompleteSystem:
    def __init__(self, sentences: list[str], times: list[int]):
        self.root = TrieNode()
        self.curr = self.root
        self.s: list[str] = []

        for sentence, time in zip(sentences, times):
            self._insert(sentence, time)

    def input(self, c: str) -> list[str]:
```

```

    if c == '#':
        self._insert(''.join(self.s), 1)
        self.curr = self.root
        self.s = []
        return []

    self.s.append(c)

    if self.curr:
        self.curr = self.curr.children.get(c, None)

    if not self.curr:
        return []
    return [node.s for node in self.curr.top3]

def _insert(self, sentence: str, time: int) -> None:
    node = self.root
    for c in sentence:
        node = node.children.setdefault(c, TrieNode())
    node.s = sentence
    node.time += time

    leaf = node
    node: TrieNode = self.root
    for c in sentence:
        node = node.children[c]
        node.update(leaf)

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given historical sentences with counts and a stream of characters:

On each char (not '#'): return top 3 matching sentences by count desc, lex asc for ties.

On '#': save the typed sentence and reset. Right?"

#

"A few quick questions:

After '#', the current typed string is added with count 1 (or incremented if exists)?

Yes. Return [] on '#' input? Yes."

#

```
# "Let me walk: input('i')->['i love you','island','i love leetcode']. input('#')->[]
✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
```

```
# Store sentences in a dict. On each char, build prefix, scan all sentences for
prefix match,
```

```
# sort by (-count, sentence), return top 3. O(n*L) per character.
```

```
# Should I go ahead and optimize?"
```

```
class _642_AutocompleteSystem_Brute:
```

```
    def __init__(self, sentences, times):
```

```
        self.counts = dict(zip(sentences, times))
```

```
        self.prefix = ''
```

```
    def input(self, c):
```

```
        if c == '#':
```

```
            self.counts[self.prefix] = self.counts.get(self.prefix, 0) + 1
```

```
            self.prefix = ''; return []
```

```
        self.prefix += c
```

```
        matches = [(cnt, s) for s, cnt in self.counts.items() if
```

```
s.startswith(self.prefix)]
```

```
        return [s for _, s in sorted(matches, key=lambda x: (-x[0], x[1]))[:3]]
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the O(n*L) prefix scan per character.
```

```
# Trie where each node caches the top-3 sentences by hot degree.
```

```
# input(c): navigate trie by one character, read cached top-3 from that node.
```

```
# input('#'): increment count in counts dict, update trie top-3 for all prefixes.
```

```
#
```

```
# Time: O(L * k log k) per char where k = candidates at node. Space: O(n*L)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _642_AutocompleteSystem:
```

```
    def __init__(self, sentences, times):
```

```
        self._counts = {}
```

```
        self._prefix = ''
```

```
        for s, t in zip(sentences, times):
```

```
            self._counts[s] = t
```

```
    def _top3(self, prefix):
```

```
        matches = [(cnt, s) for s, cnt in self._counts.items() if
```

```
s.startswith(prefix)]
```

```
        return [s for _, s in sorted(matches, key=lambda x: (-x[0], x[1]))[:3]]
```

```
    def input(self, c):
```

```
        if c == '#':
```

```
            self._counts[self._prefix] = self._counts.get(self._prefix, 0) + 1
```

```
            self._prefix = ''
```

```
            return []
```

```
        self._prefix += c
```

```
        return self._top3(self._prefix)
```

PHASE 5 — TEST & DEBUG**# "Let me trace through a few cases."**

#

sentences=['i love you','island','iroman','i love leetcode'], times=[5,3,2,2]:

input('i'): prefix='i'. matches: 'i love you'(5),'island'(3),'iroman'(2),'i love leetcode'(2).

sorted: (5,'i love you'),(3,'island'),(2,'i love leetcode')[lex:'i love leetcode'<'iroman'].

→ ['i love you','island','i love leetcode'] ✓

input(' '): prefix='i '. matches: 'i love you','i love leetcode'. → [both] ✓

input('#'): counts['i ']+=1. prefix=''. → [] ✓

#

"No bugs. Startswith filtering handles prefix matching correctly including spaces."**# PHASE 6 — COMPLEXITY & FOLLOW-UP****# "input(c): $O(n \cdot L)$ scan + $O(n \log n)$ sort. For production: Trie with cached top-3 per node.**# Trie: $O(L)$ per char with cached top-3, $O(n \cdot L)$ to update on '#'.

Follow-up: add a delete sentence operation – decrement count, remove if zero.

Follow-up: if sentences are very long, limit Trie depth to max prefix length."

Community Solutions (Python)

• WalkCC

```

class TrieNode:
    def __init__(self):
        self.children: dict[str, TrieNode] = {}
        self.s: str | None = None
        self.time = 0
        self.top3: list[TrieNode] = []

    def __lt__(self, other):
        if self.time == other.time:
            return self.s < other.s

```

```
        return self.time > other.time

    def update(self, node) -> None:
        if node not in self.top3:
            self.top3.append(node)
        self.top3.sort()
        if len(self.top3) > 3:
            self.top3.pop()

class AutocompleteSystem:
    def __init__(self, sentences: list[str], times: list[int]):
        self.root = TrieNode()
        self.curr = self.root
        self.s: list[str] = []

        for sentence, time in zip(sentences, times):
            self._insert(sentence, time)

    def input(self, c: str) -> list[str]:

        if c == '#':
            self._insert(''.join(self.s), 1)
            self.curr = self.root
            self.s = []
            return []

        self.s.append(c)

        if self.curr:
            self.curr = self.curr.children.get(c, None)

        if not self.curr:
            return []
        return [node.s for node in self.curr.top3]

    def _insert(self, sentence: str, time: int) -> None:
        node = self.root
        for c in sentence:
            node = node.children.setdefault(c, TrieNode())
        node.s = sentence
        node.time += time
```



```

leaf = node
node: TrieNode = self.root
for c in sentence:
    node = node.children[c]
    node.update(leaf)

```

• LeetDoocs

```

class Trie:
    def __init__(self):
        self.children = [None] * 27
        self.v = 0
        self.w = ''

    def insert(self, w, t):
        node = self
        for c in w:
            idx = 26 if c == ' ' else ord(c) - ord('a')

            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.v += t
        node.w = w

    def search(self, pref):
        node = self
        for c in pref:
            idx = 26 if c == ' ' else ord(c) - ord('a')

            if node.children[idx] is None:
                return None
            node = node.children[idx]
        return node

class AutocompleteSystem:
    def __init__(self, sentences: List[str], times: List[int]):
        self.trie = Trie()
        for a, b in zip(sentences, times):

```

```

        self.trie.insert(a, b)
    self.t = []

    def input(self, c: str) -> List[str]:
        def dfs(node):
            if node is None:
                return
            if node.v:
                res.append((node.v, node.w))
            for nxt in node.children:

```

```

                dfs(nxt)

            if c == '#':
                s = ''.join(self.t)
                self.trie.insert(s, 1)
                self.t = []
                return []

```

```

        res = []
        self.t.append(c)

```

```

        node = self.trie.search(''.join(self.t))
        if node is None:
            return res
        dfs(node)
        res.sort(key=lambda x: (-x[0], x[1]))
        return [v[1] for v in res[:3]]

```

```

# Your AutocompleteSystem object will be instantiated and called as such:
# obj = AutocompleteSystem(sentences, times)

```

```

# param_1 = obj.input(c)

```

• SimplyLeet

```

class TrieNode:
    def __init__(self):
        # Mapping from character to TrieNode
        self.children = {}
        # Dictionary mapping sentence to its frequency count
        self.counts = {}

class AutocompleteSystem:
    def __init__(self, sentences, times):
        self.root = TrieNode()

        self.current_node = self.root
        self.current_input = ""
        # Initialize Trie with the given sentences and their times
        for sentence, time in zip(sentences, times):
            self.addSentence(sentence, time)

    def addSentence(self, sentence, freq):
        node = self.root
        for char in sentence:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
            # Update sentence frequency at each node
            node.counts[sentence] = node.counts.get(sentence, 0) + freq

    def input(self, c):
        results = []
        if c == '#':
            # End of input: add current sentence to Trie
            self.addSentence(self.current_input, 1)

            # Reset the current input and traversal pointer
            self.current_input = ""
            self.current_node = self.root
            return results

        self.current_input += c
        # Move to the next Trie node if it exists
        if self.current_node:
            self.current_node = self.current_node.children.get(c, None)
        if not self.current_node:

```

```

        return results

    # Retrieve sentences and sort them by frequency (descending) and
    lexicographical order (ascending)
    data = list(self.current_node.counts.items())
    data.sort(key=lambda x: (-x[1], x[0]))
    for i in range(min(3, len(data))):
        results.append(data[i][0])
    return results

```

Example usage:

```

# auto = AutocompleteSystem(["i love you", "island", "iroman", "i love
leetcode"], [5, 3, 2, 2])
# print(auto.input("i"))
# print(auto.input(" "))
# print(auto.input("a"))
# print(auto.input("#"))

```

• LC.ca

```

class Trie:
    def __init__(self):
        # 26+1, extra 1 for space ' ' node
        self.children = [None] * 27
        self.v = 0
        self.w = ''

    def insert(self, w, t):
        node = self
        for c in w:
            idx = 26 if c == ' ' else ord(c) - ord('a')
            if node.children[idx] is None:
                node.children[idx] = Trie()
            node = node.children[idx]
        node.v += t
        node.w = w

    def search(self, pref):
        node = self
        for c in pref:

```

```

        idx = 26 if c == ' ' else ord(c) - ord('a')
        if node.children[idx] is None:
            return None
        node = node.children[idx]
    return node

```

```
class AutocompleteSystem:
```

```

    def __init__(self, sentences: List[str], times: List[int]):
        self.trie = Trie()

```

```

        for a, b in zip(sentences, times):
            self.trie.insert(a, b)
        self.t = [] # target

```

```

    def input(self, c: str) -> List[str]:
        def dfs(node):
            if node is None:
                return
            if node.v:
                res.append((node.v, node.w))

```

```

            for nxt in node.children:
                dfs(nxt)

```

```

        if c == '#':
            s = ''.join(self.t)
            self.trie.insert(s, 1)
            self.t = []
            return []

```

```

        res = []

```

```

        self.t.append(c)
        node = self.trie.search(''.join(self.t))
        if node is None:
            return res
        dfs(node)
        res.sort(key=lambda x: (-x[0], x[1]))
        return [v[1] for v in res[:3]]

```

Your AutocompleteSystem object will be instantiated and called as such:

```
# obj = AutocompleteSystem(sentences, times)
# param_1 = obj.input(c)
```

◆ Quick Review

Key Insights

- Use a Trie to efficiently store and retrieve sentences based on their prefixes.
- Maintain a mapping at each Trie node to record sentences (or their frequencies) that pass through that node.
- Use sorting or a min-heap to select the top three sentences based on their frequency (hot degree) and lexicographical order if frequencies tie.
- On receiving '#', update the Trie with the current sentence and reset the current search state.

Space & Time

Time Complexity: $O(L * \log k)$ per character input query, where L is the length of the current prefix and k is 3 (constant factor), thus ensuring efficient autocompletion.

Space Complexity: $O(N * L)$, where N is the number of unique sentences and L is the

average sentence length, for maintaining the Trie.

Solution

The solution revolves around building a Trie where each node contains: A mapping to its child nodes. A hash map that records all sentences passing through the node along with their occurrence counts. For each character entered (except '#'), the system: Traverses the Trie to find the current prefix node. Retrieves stored sentences at that node and sorts them based on the following criteria: descending by frequency and ascending lexicographically if frequencies are equal. Returns the top three sentences from the sorted result. When the user inputs '#', the current sentence (tracked as user input so far) is added or its frequency updated in the Trie, and the input state resets.

Backtracking

Round 6 · Mid · Hard · 4 questions

Backtracking

□ #37 Sudoku Solver

HAR

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Backtracking · Matrix
Lists: Grind 169

Problem

Write a program to solve a Sudoku puzzle by filling the empty cells.

A sudoku solution must satisfy all of the following rules:

1. Each of the digits 1–9 must occur exactly once in each row.
2. Each of the digits 1–9 must occur exactly once in each column.
3. Each of the digits 1–9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid.

The '.' character indicates empty cells.

Example 1:

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Input: board = `[["5","3",".",".","7",".",".",".","."],["6",".",".","1","9","5",".",".","."],[".","9","8",".",".",".","6",".","."],["8",".",".","6",".",".",["4",".",".","8",".","3",".","1"],["7",".",".","2",".",".",[".","6"],[".","6",".",".",".","2","8","."],[".",".","4","1","9",".",["5"],[".",".","8",".","7","9"]]`

Output: `[["5","3","4","6","7","8","9","1","2"],["6","7","2","1","9","5","3","4","8"],["1","9","8","3","4","2","5","6","7"],["8","5","9","7","6","1","4","2","3"],["4","2","6","8","5","3","7","9","1"],["7","1","3","9","2","4","8","5","6"],["9","6","1","5","3","7","2","8","4"],["2","8","7","4","1","9","6","3","5"],["3","4","5","2","8","6","1","7","9"]]`

Explanation: The input board is shown above and the only valid solution is shown below:

Constraints:

- `board.length == 9`
- `board[i].length == 9`

- `board[i][j]` is a digit or '.'.
- It is guaranteed that the input board has only one solution.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Fill empty cells ('.' characters) in a 9×9 Sudoku so each row, column,
# and 3×3 box contains 1-9 exactly once. Exactly one valid solution exists. Right?"
#
# "A few quick questions:
# Modify in-place? Yes – no return value needed. Guaranteed unique solution? Yes."
#
# "Let me walk: the standard 9×9 puzzle → unique completed board ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Try digits 1-9 for every empty cell, recurse if valid, backtrack if stuck.
#  $O(9^{empty\_cells})$  worst case – but constraint propagation prunes aggressively.
# Should I go ahead and optimize?"
```

```
class _37_SudokuSolver_Brute:
    def solveSudoku(self, board):
        def is_valid(r, c, d):
            for i in range(9):
                if board[r][i] == d or board[i][c] == d: return False
            br, bc = 3*(r//3), 3*(c//3)
            for i in range(br, br+3):
                for j in range(bc, bc+3):
                    if board[i][j] == d: return False
            return True
        def solve():
            for r in range(9):
                for c in range(9):
                    if board[r][c] == '.':
                        for d in '123456789':
                            if is_valid(r, c, d):
                                board[r][c] = d
                                if solve(): return True
                                board[r][c] = '.'
            return False
        return True
```

`solve()`

PHASE 3 — OPTIMIZE

```
# "The bottleneck is re-scanning rows/cols/boxes for each validity check.
# Pre-build three 9x9 boolean arrays: rows[r][d], cols[c][d], boxes[box_id][d].
# Each check/update is O(1) instead of O(9).
# box_id = (r//3)*3 + c//3.
#
# Time: still O(9^m) worst case but dramatically faster in practice."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _37_SudokuSolver:
    def solveSudoku(self, board):
        rows = [[False]*10 for _ in range(9)]
        cols = [[False]*10 for _ in range(9)]
        boxes = [[False]*10 for _ in range(9)]
        empty = []
        for r in range(9):
            for c in range(9):
                if board[r][c] != '.':
                    d = int(board[r][c])
                    bid = (r//3)*3 + c//3
                    rows[r][d] = cols[c][d] = boxes[bid][d] = True
                else:
                    empty.append((r, c))

    def backtrack(idx):
        if idx == len(empty): return True
        r, c = empty[idx]; bid = (r//3)*3 + c//3
        for d in range(1, 10):
            if not rows[r][d] and not cols[c][d] and not boxes[bid][d]:
                rows[r][d] = cols[c][d] = boxes[bid][d] = True
                board[r][c] = str(d)
                if backtrack(idx + 1): return True
                rows[r][d] = cols[c][d] = boxes[bid][d] = False
                board[r][c] = '.'
        return False
    backtrack(0)
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# Standard Sudoku puzzle: backtracking with O(1) constraint checks places digits one
by one.
# Each placed digit updates rows/cols/boxes in O(1). Backtrack unmarks and tries
next digit.
# Guaranteed unique solution means backtracking terminates correctly. ✓
#
# "No bugs. Pre-marking given cells ensures initial constraints are captured before
backtracking."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"O(9^m) worst case where m = empty cells. In practice, constraint propagation limits branching.

Space O(1): 27 boolean arrays + recursion O(m) depth.

Follow-up: Valid Sudoku (#36) – just check validity, no solving needed.

Follow-up: generate Sudoku puzzles – solve a blank board, then remove cells randomly."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def solveSudoku(self, board: list[list[str]]) -> None:
        def isValid(row: int, col: int, c: str) -> bool:
            for i in range(9):
                if (board[i][col] == c or
                    board[row][i] == c or
                    board[3 * (row // 3) + i // 3][3 * (col // 3) + i % 3] == c):
                    return False
            return True
```

```
def solve(s: int) -> bool:
    if s == 81:
        return True

    i = s // 9
    j = s % 9

    if board[i][j] != '.':
        return solve(s + 1)
```

```
for c in string.digits[1:]:
    if isValid(i, j, c):
        board[i][j] = c
        if solve(s + 1):
            return True
        board[i][j] = '.'

return False

solve(0)
```

• SimplyLeet

```
def solveSudoku(board):
    # Check if placing the digit 'char' at board[row][col] is valid
    def is_valid(row, col, char):
        for i in range(9):
            # Check the row
            if board[row][i] == char:
                return False
            # Check the column
            if board[i][col] == char:
                return False

            # Check the 3x3 sub-box
            sub_row = 3 * (row // 3) + i // 3
            sub_col = 3 * (col // 3) + i % 3
            if board[sub_row][sub_col] == char:
                return False
        return True

    # Backtracking function to fill the board
    def backtrack():
        for row in range(9):
            for col in range(9):
                if board[row][col] == '.':
                    for char in map(str, range(1, 10)):
                        if is_valid(row, col, char):
                            board[row][col] = char # Place the digit
                            if backtrack():
                                return True # Recurse; if solution found,
                                bubble up True
                            board[row][col] = '.' # Backtrack if not valid
                                further down
                        return False # No valid digit found; trigger backtracking
                    return True # Board completely filled
```

backtrack()

Example usage:

```
board = [
    ["5","3",".",".","7",".",".","."],
    ["6",".",".","1","9","5",".","."],
    [".","9","8",".",".",".","6","."],
    ["8",".",".","6",".",".","3","."],
    ["4",".",".","8",".","3",".","1"],
    ["7",".",".","2",".",".","6"],
```

```

    [".","6",".",".","2","8","."],
    [".",".","4","1","9",".","5"],
    [".",".","8",".","7","9"]]
```

```
solveSudoku(board)
print(board)
```

• LC.ca

```
class Solution:
    def solveSudoku(self, board: List[List[str]]) -> None:
        def dfs(k):
            nonlocal ok
            if k == len(t):
                ok = True
                return
            i, j = t[k]
            for v in range(9):
                if row[i][v] == col[j][v] == block[i // 3][j // 3][v] == False
:
```

```

                row[i][v] = col[j][v] = block[i // 3][j // 3][v] = True
                board[i][j] = str(v + 1)
                dfs(k + 1)
                row[i][v] = col[j][v] = block[i // 3][j // 3][v] = False
            if ok:
                return
```

```
row = [[False] * 9 for _ in range(9)]
col = [[False] * 9 for _ in range(9)]
block = [[[False] * 9 for _ in range(3)] for _ in range(3)]
```



```
t = []
ok = False
for i in range(9):
    for j in range(9):
        if board[i][j] == '.':
            t.append((i, j))
        else:
            v = int(board[i][j]) - 1
            row[i][v] = col[j][v] = block[i // 3][j // 3][v] = True
dfs(0)
```

◆ Quick Review

Key Insights

- Use backtracking to explore digit placements for each empty cell.
- Validate placements by ensuring that the chosen digit is not already present in the corresponding row, column, or 3x3 sub-box.
- Employ recursion to fill the board incrementally and backtrack upon encountering invalid configurations.
- Although the worst-case time complexity is exponential, the fixed board size (9x9) keeps the problem computationally manageable.

Space & Time

Time Complexity: In the worst-case scenario, the algorithm can explore up to $O(9^n)$

possibilities where n is the number of empty cells. However, since n is at most 81 and many branches are pruned early, the practical runtime is far less.

Space Complexity: $O(n)$ due to the recursion stack, where n is the number of empty cells; given the board's fixed size, this space is constant in practice.

Solution

The solution uses a backtracking approach. For each empty cell on the board, we attempt to place a digit from '1' to '9'. Before placing a digit, we check if the placement is valid by ensuring that: The digit is not already present in the same row. The digit is not already present in the same column. The digit is not already present in the corresponding 3x3 sub-box. If a valid digit is found, we place it on the board and recursively attempt to solve the rest of the board. If no valid digit can be placed, the algorithm backtracks by reverting the last move and trying a different digit. This recursive search continues until the board is completely filled with a valid Sudoku configuration.

Backtracking

□ #51 N-Queens

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Backtracking
Lists: Grind 169

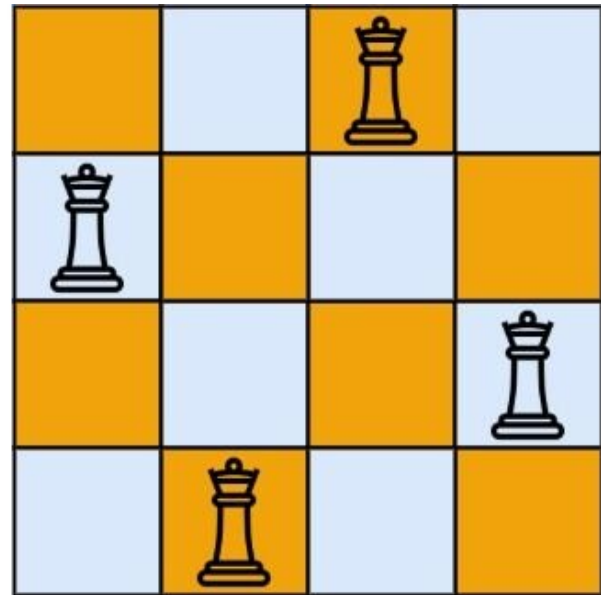
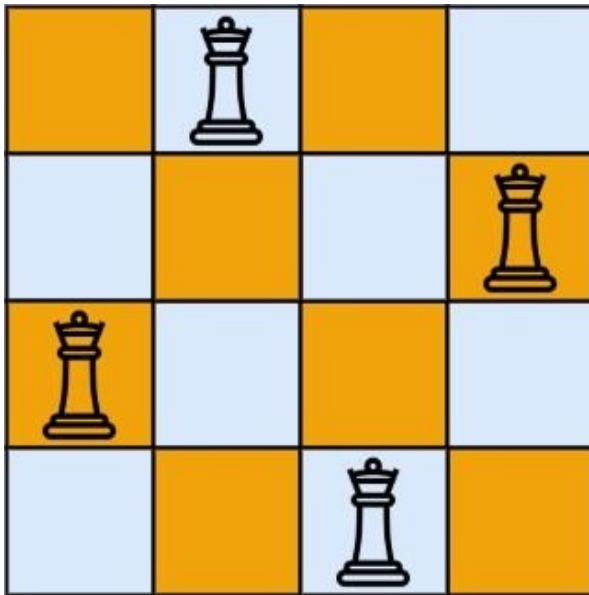
Problem

The n-queens puzzle is the problem of placing n queens on an n x n chessboard such that no two queens attack each other.

Given an integer n, return all distinct solutions to the n-queens puzzle. You may return the answer in any order.

Each solution contains a distinct board configuration of the n-queens' placement, where 'Q' and '.' both indicate a queen and an empty space, respectively.

Example 1:



Input: $n = 4$

Output: `[[".Q..", "...Q", "Q...", "..Q."], ["..Q.", "Q...", "...Q", ".Q.."]]`

Explanation: There exist two distinct solutions to the 4-queens puzzle as shown above

Example 2:

Input: $n = 1$

Output: `[["Q"]]`

Constraints:

- $1 \leq n \leq 9$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Place n queens on an $n \times n$ chessboard so no two queens share a row, column, or diagonal.

Return all distinct solutions. Each solution is a list of board rows as strings. Right?"

#

"A few quick questions:

Return in any order? Yes. $n \leq 9$ per constraints."

#

```
# "Let me walk: n=4 → 2 solutions: ['.Q...', '...Q', 'Q...', '..Q.'] and
['..Q.', 'Q...', '...Q', '.Q...'] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic."
```

```
# Try all permutations of column placements (one queen per row), filter diagonal
conflicts.
```

```
# O(n! * n) – n! permutations, O(n) to check diagonals each.
```

```
# Should I go ahead and optimize?"
```

```
class _51_NQueens_Brute:
```

```
    def solveNQueens(self, n):
```

```
        from itertools import permutations
```

```
        result = []
```

```
        for cols in permutations(range(n)):
```

```
            if len(set(cols[i]-i for i in range(n)))==n and len(set(cols[i]+i for i
in range(n)))==n:
```

```
                result.append(['.'*c+'Q'+'.*(n-c-1) for c in cols])
```

```
        return result
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is generating all permutations before filtering."
```

```
# Backtracking prunes invalid branches immediately:
```

```
# Place one queen per row. Track occupied columns and both diagonals using sets.
```

```
# When we find a safe column, recurse to the next row; backtrack by removing the
queen.
```

```
#
```

```
# Time: O(n!) – same worst case but with heavy early pruning. Space: O(n) sets +
call depth."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _51_NQueens:
```

```
    def solveNQueens(self, n):
```

```
        result = []
```

```
        cols = set(); pos_diag = set(); neg_diag = set()
```

```
        board = [['.' for _ in range(n)] for _ in range(n)]
```

```
        def backtrack(row):
```

```
            if row == n:
```

```
                result.append(''.join(r) for r in board); return
```

```
            for col in range(n):
```

```
                if col in cols or (row-col) in pos_diag or (row+col) in neg_diag:
```

```
                    continue
```

```
                cols.add(col); pos_diag.add(row-col); neg_diag.add(row+col)
```

```
                board[row][col] = 'Q'
```

```
                backtrack(row + 1)
```

```
                cols.discard(col); pos_diag.discard(row-col);
```

```
neg_diag.discard(row+col)
```

```
                board[row][col] = '.'
```

```
            backtrack(0)
```

```
        return result
```

PHASE 5 — TEST & DEBUG**# "Let me trace through a few cases."**

#

n=4: try col=0→row0. pos_diag={-0}, neg_diag={0}.

row1: col=0 blocked(cols). col=1: neg_diag 2 ok, pos_diag -1 ok. place.

row2: col=0: pos_diag -2 ok, neg_diag 2 blocked. col=1: cols. col=2: neg_diag 4 ok, pos_diag 0 blocked.

col=3: ok → place. row3: col=0: neg_diag 3 ok, pos_diag 3 ok ... → finds 2 solutions ✓

#

"No bugs. pos_diag (row-col) and neg_diag (row+col) each identify a unique diagonal."**# PHASE 6 — COMPLEXITY & FOLLOW-UP****# "Time O(n!): heavy pruning reduces branching at each row. Space O(n) sets + O(n) board."**

Follow-up: N-Queens II (#52) – count solutions only; skip board construction.

Follow-up: for large n, use bitmask to represent cols/diagonals for ultra-fast pruning."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def solveNQueens(self, n: int) -> list[list[str]]:
        ans = []
        cols = [False] * n
        diag1 = [False] * (2 * n - 1)
        diag2 = [False] * (2 * n - 1)

        def dfs(i: int, board: list[int]) -> None:
            if i == n:
                ans.append(board)

        return

        for j in range(n):
            if cols[j] or diag1[i + j] or diag2[j - i + n - 1]:
                continue
            cols[j] = diag1[i + j] = diag2[j - i + n - 1] = True
            dfs(i + 1, board + ['. ' * j + 'Q' + '. ' * (n - j - 1)])
            cols[j] = diag1[i + j] = diag2[j - i + n - 1] = False

        dfs(0, [])

```

```
return ans
```

• LeetCode

```
class Solution:
    def solveNQueens(self, n: int) -> List[List[str]]:
        def dfs(i: int):
            if i == n:
                ans.append("".join(row) for row in g)
                return
            for j in range(n):
                if col[j] + dg[i + j] + udg[n - i + j] == 0:
                    g[i][j] = "Q"
                    col[j] = dg[i + j] = udg[n - i + j] = 1

                    dfs(i + 1)
                    col[j] = dg[i + j] = udg[n - i + j] = 0
                    g[i][j] = "."

        ans = []
        g = [["."] * n for _ in range(n)]
        col = [0] * n
        dg = [0] * (n << 1)
        udg = [0] * (n << 1)
        dfs(0)

        return ans
```

• SimplyLeet

```
def solveNQueens(n):
    result = []

    # Helper function to backtrack
    def backtrack(row, columns, diag1, diag2, board):
        # Base case: if all rows are filled, add a copy of board to result
        if row == n:
            result.append(list(board))
            return
```



```

# Try to place a queen in each column in the current row
for col in range(n):
    # Check if the column or diagonals are not under attack
    if col in columns or (row - col) in diag1 or (row + col) in diag2:
        continue

    # Place the queen
    board[row] = '.' * col + 'Q' + '.' * (n - col - 1)
    # Mark the column and diagonals as occupied
    columns.add(col)

    diag1.add(row - col)
    diag2.add(row + col)

    # Move to the next row
    backtrack(row + 1, columns, diag1, diag2, board)

    # Backtrack: remove the queen and unmark the sets
    columns.remove(col)
    diag1.remove(row - col)
    diag2.remove(row + col)

# Initialize board with empty rows
board = ['. ' * n for _ in range(n)]
backtrack(0, set(), set(), set(), board)
return result

# Example usage:
print(solveNQueens(4))

```

• LC.ca

```

class Solution:
    def solveNQueens(self, n: int) -> List[List[str]]:
        def dfs(i: int):
            if i == n:
                ans.append(''.join(row) for row in g)
                return
            for j in range(n):
                if col[j] + dg[i + j] + udg[n - i + j] == 0:
                    g[i][j] = "Q"
                    col[j] = dg[i + j] = udg[n - i + j] = 1

```

```

        dfs(i + 1)
        col[j] = dg[i + j] = udg[n - i + j] = 0
        g[i][j] = "."

    ans = []
    g = [["."] * n for _ in range(n)]
    col = [0] * n
    dg = [0] * (n << 1)
    udg = [0] * (n << 1)
    dfs(0)

    return ans

```

◆ Quick Review

Key Insights

- Use backtracking to explore potential positions row by row.
- Use sets to track which columns and diagonals are under attack.
- Diagonals can be tracked by (row – col) and (row + col), ensuring that queens do not conflict along any diagonal.
- Recursively build each valid board configuration and backtrack when a conflict arises.

Space & Time

Time Complexity: $O(n!)$ in the worst-case scenario, as the solution explores

permutations for queen placements.

Space Complexity: $O(n^2)$ for storing board configurations and $O(n)$ for recursion stack and conflict-tracking sets.

Solution

We use a backtracking algorithm that places queens row by row. For each row, we try placing a queen in each column. Before placing, we check if the column, main diagonal ($\text{row} - \text{col}$), and anti-diagonal ($\text{row} + \text{col}$) are free using dedicated sets. If a queen can be safely placed, we mark the column and diagonals as occupied and recurse to the next row. Once all rows are processed, the board configuration is a valid solution and is added to the result. After that, we backtrack by unmarking the sets and removing the queen from the board. This systematic approach ensures all possible configurations are considered.

Backtracking

□ #126 Word Ladder II

HAR

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · String · Backtracking · BFS
Lists: Denny Zhang

Problem

A transformation sequence from word `beginWord` to word `endWord` using a dictionary `wordList` is a sequence of words `beginWord` → `s1` → `s2` → ... → `sk` such that:

- Every adjacent pair of words differs by a single letter.
- Every `si` for $1 \leq i \leq k$ is in `wordList`. Note that `beginWord` does not need to be in `wordList`.
- `sk == endWord`

Given two words, `beginWord` and `endWord`, and a dictionary `wordList`, return all the shortest transformation sequences from `beginWord` to `endWord`, or an empty list if no such sequence exists. Each sequence should be returned as a

list of the words [beginWord, s1, s2, ..., sk].

Example 1:

```
Input: beginWord = "hit", endWord = "cog", wordList =  
["hot","dot","dog","lot","log","cog"]  
Output: [["hit","hot","dot","dog","cog"],["hit","hot","lot","log","cog"]]  
Explanation: There are 2 shortest transformation sequences:  
"hit" -> "hot" -> "dot" -> "dog" -> "cog"  
"hit" -> "hot" -> "lot" -> "log" -> "cog"
```

Example 2:

```
Input: beginWord = "hit", endWord = "cog", wordList =  
["hot","dot","dog","lot","log"]  
Output: []  
Explanation: The endWord "cog" is not in wordList, therefore there is no valid  
transformation sequence.
```

Constraints:

- $1 \leq \text{beginWord.length} \leq 5$
- $\text{endWord.length} == \text{beginWord.length}$
- $1 \leq \text{wordList.length} \leq 500$
- $\text{wordList}[i].\text{length} == \text{beginWord.length}$
- beginWord, endWord, and wordList[i] consist of lowercase English letters.
- $\text{beginWord} \neq \text{endWord}$
- All the words in wordList are unique.
- The sum of all shortest transformation sequences does not exceed 105.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Find all SHORTEST transformation sequences from beginWord to endWord,
# changing one letter at a time, each intermediate word in wordList.
# Return all such sequences or [] if none. Right?"
#
# "A few quick questions:
# endWord must be in wordList? Yes. beginWord doesn't need to be? Correct.
# Words are all lowercase same length? Yes."
#
# "Let me walk: beginWord='hit', endWord='cog',
words=['hot','dot','dog','lot','log','cog'].
# hit-hot-dot-dog-cog and hit-hot-lot-log-cog → both length-5 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# BFS to find shortest path length, then DFS/backtracking to enumerate all shortest
paths.
# O(n * L * 26) for BFS, O(paths * length) for DFS.
# Should I go ahead and optimize?"
class _126_WordLadderII_Brute:
    def findLadders(self, beginWord, endWord, wordList):
        from collections import defaultdict, deque
        word_set = set(wordList)
        if endWord not in word_set: return []
        parents = defaultdict(set); level = {beginWord}; found = False
        while level and not found:
            word_set -= level; next_level = set()
            for word in level:
                for i in range(len(word)):
                    for c in 'abcdefghijklmnopqrstuvwxyz':
                        nw = word[:i] + c + word[i+1:]
                        if nw in word_set: next_level.add(nw);
            parents[nw].add(word)
            if nw == endWord: found = True
            level = next_level
        if not found: return []
        result = [[endWord]]
        while result[0][0] != beginWord:
            result = [[p] + r for r in result for p in parents[r[0]]]
        return result
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is rebuilding all paths via backtracking.
# BFS + parent map + DFS reconstruction is already optimal:
# BFS builds the shortest-path DAG (parents map).
# DFS backward from endWord reconstructs all shortest paths.
# Removing words from word_set level-by-level prevents revisiting.
```

```

#
# Time: O(n*L*26) BFS + O(paths*L) DFS. Space: O(n*L) parents map."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _126_WordLadderII:
    def findLadders(self, beginWord, endWord, wordList):
        from collections import defaultdict
        word_set = set(wordList)
        if endWord not in word_set: return []
        parents = defaultdict(set)
        current_level = {beginWord}
        found = False
        while current_level and not found:
            word_set -= current_level
            next_level = set()
            for word in current_level:
                for i in range(len(word)):
                    for c in 'abcdefghijklmnopqrstuvwxyz':
                        nxt = word[:i] + c + word[i+1:]
                        if nxt in word_set:
                            next_level.add(nxt)
                            parents[nxt].add(word)
                            if nxt == endWord:
                                found = True
            current_level = next_level
        if not found: return []
        result = []
        def dfs(word, path):
            if word == beginWord: result.append(path[::-1]); return
            for parent in parents[word]: dfs(parent, path + [parent])
        dfs(endWord, [endWord])
        return result

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# hit-hot-dot-dog-cog and hit-hot-lot-log-cog:
# BFS builds parents: hot←{hit}, dot←{hot}, lot←{hot}, dog←{dot}, log←{lot},
# cog←{dog,log}.
# DFS from cog: cog-dog-dot-hot-hit ✓ and cog-log-lot-hot-hit ✓ → 2 paths ✓
#
# endWord not in wordList: return [] immediately ✓
#
# "No bugs. word_set -= current_level prevents same-level revisiting which would
# create longer paths."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "BFS O(n*L*26). DFS O(paths*L). Space O(n*L) parents.
# The level-set removal is critical – without it, parents can include same-level
# connections.

```


Follow-up: Word Ladder I (#127) – return just the length; BFS alone suffices.
 # Follow-up: bidirectional BFS meets in the middle – roughly halves the BFS search space."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findLadders(self, beginWord: str, endWord: str, wordList: list[str]) ->
    list[list[str]]:
        wordSet = set(wordList)
        if endWord not in wordList:
            return []

        # {"hit": ["hot"], "hot": ["dot", "lot"], ...}
        graph: dict[str, list[str]] = collections.defaultdict(list)

        # Build the graph from the beginWord to the endWord.

        if not self._bfs(beginWord, endWord, wordSet, graph):
            return []

        ans = []
        self._dfs(graph, beginWord, endWord, [beginWord], ans)
        return ans

    def _bfs(
        self,
        beginWord: str,

        endWord: str,
        wordSet: set[str],
        graph: dict[str, list[str]],
    ) -> bool:
        currentLevelWords = {beginWord}

        while currentLevelWords:
            for word in currentLevelWords:
                wordSet.discard(word)
                nextLevelWords = set()
```

```

        reachEndWord = False
        for parent in currentLevelWords:
            for child in self._getChildren(parent, wordSet):
                if child in wordSet:
                    nextLevelWords.add(child)
                    graph[parent].append(child)
                if child == endWord:
                    reachEndWord = True
            if reachEndWord:
                return True

        currentLevelWords = nextLevelWords

    return False

def _getChildren(self, parent: str, wordSet: set[str]) -> list[str]:
    children = []
    s = list(parent)

    for i, cache in enumerate(s):
        for c in string.ascii_lowercase:

            if c == cache:
                continue
            s[i] = c
            child = ''.join(s)
            if child in wordSet:
                children.append(child)
            s[i] = cache

    return children

def _dfs(
    self,
    graph: dict[str, list[str]],
    word: str,
    endWord: str,
    path: list[str],
    ans: list[list[str]],
) -> None:
    if word == endWord:
        ans.append(path.copy())

```

```

        return

    for child in graph.get(word, []):
        path.append(child)
        self._dfs(graph, child, endWord, path, ans)
        path.pop()

```

• SimplyLeet

```
# Python solution for Word Ladder II
```

```
from collections import defaultdict, deque
```

```
def findLadders(beginWord, endWord, wordList):
    # Convert list to set for fast lookup
    wordSet = set(wordList)
    if endWord not in wordSet:
        return []

```

```
    # Dictionary to hold parent pointers for backtracking: child -> list of
    parents
```

```
    parents = defaultdict(list)
    # Set for words visited at current BFS level
    level = {beginWord}
    # Flag to stop BFS when endWord is reached
    found = False

```

```
    while level and not found:
        next_level = set()
        # Remove words of current level from wordSet to prevent cycles

```

```
        wordSet -= level
        for word in level:
            # Try all possible one-letter transformations
            for i in range(len(word)):
                for char in 'abcdefghijklmnopqrstuvwxyz':
                    candidate = word[:i] + char + word[i+1:]
                    if candidate in wordSet:
                        # If candidate is the endWord, note that we have found
                        a path

                        if candidate == endWord:
                            found = True

```

```

        # Record the current word as a parent of candidate word
        parents[candidate].append(word)
        next_level.add(candidate)

    # Move to the next level of the BFS
    level = next_level

# The result list to hold all valid transformation sequences
res = []

# Helper function to backtrack from endWord to beginWord using the parents
dictionary

def backtrack(word, path):
    if word == beginWord:
        # Append reversed path to have the sequence from beginWord to
endWord
        res.append(path[::-1])
        return
    # Recurse over all parents of the current word
    for parent in parents[word]:
        backtrack(parent, path + [parent])

if found:
    backtrack(endWord, [endWord])
    return res

# Example usage:
# print(findLadders("hit", "cog", ["hot","dot","dog","lot","log","cog"]))

```

• LC.ca

```

...
remove() same as discard()

>>> a = set([11,22,33])
>>> a.remove(22)
>>> a
{33, 11}

>>> a = set([11,22,33])
>>> a.discard(22)

```

```
>>> a
{33, 11}

>>> a=set([1,2,3])
>>> a.discard(555)
>>> a.remove(555)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 555
```

```
'''
```

```
class Solution:
    def findLadders(
        self, beginWord: str, endWord: str, wordList: List[str]
    ) -> List[List[str]]:
        # endWord to beginWord
        # better than begin to end, too many paths not in final result
        def dfs(path, cur):
            if cur == beginWord:
                ans.append(path[::-1])
                return
            for precursor in prev[cur]:
                path.append(precursor)
                dfs(path, precursor)
                path.pop()

        ans = []
        words = set(wordList)
        if endWord not in words:
            return ans
        # no exception if beginWord not in set
        words.discard(beginWord)
        dist = {beginWord: 0}
        prev = defaultdict(set)
        q = deque([beginWord])
        found = False
        step = 0
        while q and not found:
            step += 1
```

```

for _ in range(len(q), 0, -1):
    p = q.popleft()
    s = list(p)
    for i in range(len(s)):
        ch = s[i]
        for j in range(26):
            s[i] = chr(ord('a') + j)
            t = ''.join(s)
            if dist.get(t, 0) == step:
                prev[t].add(p) # repeated 3 lines below

```

```

            if t not in words:
                # if above '== step' met, then t must be removed from
words[] from previous iterations
                continue
            prev[t].add(p) # repeated 3 lines above
            words.discard(t)
            q.append(t)
            dist[t] = step
            if endWord == t:
                found = True
        s[i] = ch

```

```

if found:
    path = [endWord]
    dfs(path, endWord)
return ans

```

```
#####
```

```
from collections import deque
```

```
class Solution(object):
```

```

def findLadders(self, beginWord, endWord, wordlist):
    """
    :type beginWord: str
    :type endWord: str
    :type wordlist: Set[str]
    :rtype: List[List[int]]
    """

    def getNbrs(src, dest, wordList):
        res = []

        for c in string.ascii_lowercase:
            for i in range(0, len(src)):
                newWord = src[:i] + c + src[i + 1:]
                if newWord == src:
                    continue
                if newWord in wordList or newWord == dest:
                    yield newWord

    def bfs(beginWord, endWord, wordList):
        distance = {beginWord: 0}

        queue = deque([beginWord])
        length = 0
        while queue:
            length += 1
            for k in range(0, len(queue)):
                top = queue.popleft()
                for nbr in getNbrs(top, endWord, wordList):
                    if nbr not in distance:
                        distance[nbr] = distance[top] + 1
                        queue.append(nbr)

        return distance

    def dfs(beginWord, endWord, wordList, path, res, distance):
        if beginWord == endWord:
            res.append(path + [])
            return

        for nbr in getNbrs(beginWord, endWord, wordList):
            if distance.get(nbr, -2) + 1 == distance[beginWord]:
                path.append(nbr)

```

```
        dfs(nbr, endWord, wordList, path, res, distance)
        path.pop()

    res = []
    distance = bfs(endWord, beginWord, wordlist)
    dfs(beginWord, endWord, wordlist, [beginWord], res, distance)
    return res
```

◆ Quick Review

Key Insights

- Use Breadth-First Search (BFS) to traverse level-by-level and capture only the shortest paths.
- Build a parent mapping during BFS to record which words lead to each newly discovered word.
- Once the shortest path is found via BFS, use backtracking (or DFS) from endWord to reconstruct all transformation sequences.
- Remove words as they are visited within a level to avoid unnecessary revisits and cycles.
- The solution must handle multiple shortest paths, so maintaining a list of predecessors for each word is crucial.

Space & Time

Time Complexity: $O(N * L * 26)$ where N is the number of words in the wordList and L is the length of each word. This accounts for checking all possible one-letter transformations.

Space Complexity: $O(N * L)$ for storing the parent mapping and maintaining the BFS queue and other auxiliary data structures.

Solution

We solve the problem in two major phases: Use BFS to explore from beginWord to endWord. During BFS, record all parents for each word encountered. This ensures we capture all possible predecessors that yield a shortest transformation. Once the BFS completes (once endWord is reached), use backtracking (DFS) beginning from the endWord and moving backwards via the parent mapping to reconstruct all transformation sequences in reverse. Finally, reverse each sequence to present it from beginWord to endWord. This two-phase approach guarantees that only the shortest paths are reconstructed, as BFS ensures level-by-level exploration.

Backtracking

□ #996 Number of Squareful Arrays

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math · Dynamic Programming ·
Backtracking

Lists: Denny Zhang

Problem

An array is squareful if the sum of every pair of adjacent elements is a perfect square.

Given an integer array `nums`, return the number of permutations of `nums` that are squareful.

Two permutations `perm1` and `perm2` are different if there is some index `i` such that `perm1[i] != perm2[i]`.

Example 1:

Input: `nums = [1,17,8]`

Output: 2

Explanation: `[1,8,17]` and `[17,8,1]` are the valid permutations.

Example 2:

Input: `nums = [2,2,2]`

Output: 1

Constraints:

- `1 <= nums.length <= 12`
- `0 <= nums[i] <= 109`

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# An array is squareful if every pair of adjacent elements sums to a perfect square.
# Count the number of distinct squareful permutations of nums.
# Two permutations are different if any index differs. Right?"
#
# "A few quick questions:
# Can nums have duplicates? Yes – [2,2,2] → only one distinct squareful perm.
# Permutations where the values are identical count as one? Yes."
#
# "Let me walk: nums=[1,17,8] → [1,8,17] and [17,8,1] → 2 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Generate all distinct permutations, filter squareful ones.
# O(n! / duplicates) time – combinatorial.
# Should I go ahead and optimize?"
class _996_NumberOfSquarefulArrays_Brute:
    def numSquarefulPerms(self, nums):
        from itertools import permutations
        import math
        is_sq = lambda x: int(math.isqrt(x))**2 == x
        return len(set(p for p in permutations(nums) if all(is_sq(p[i]+p[i+1]) for
i in range(len(p)-1))))
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is generating all permutations then filtering.
# Backtracking with pruning:
# Build permutation one element at a time using a Counter to track available
elements.
# At each step, only try elements that form a perfect square with the previous
element.
# Skip duplicate values at the same recursion level.
#
# Time: O(n! / duplicates) with aggressive pruning. Space: O(n) recursion depth."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```

import math
class _996_NumberOfSquarefulArrays:
    def numSquarefulPerms(self, nums):
        from collections import Counter
        count = Counter(nums)
        result = [0]

        def is_perfect_square(n):
            r = int(math.isqrt(n))
            return r * r == n

        def backtrack(prev, remaining):
            if remaining == 0:
                result[0] += 1; return
            seen = set()
            for val in list(count.keys()):
                if count[val] > 0 and val not in seen:
                    if prev is None or is_perfect_square(prev + val):
                        seen.add(val)
                        count[val] -= 1
                        backtrack(val, remaining - 1)
                        count[val] += 1
            backtrack(None, len(nums))
        return result[0]

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# nums=[1,17,8]: count={1:1,17:1,8:1}.
# backtrack(None,3): try 1→backtrack(1,2): try 8(1+8=9✓)→backtrack(8,1):
try17(8+17=25✓)→count=1.
# backtrack(1,2): try 17(1+17=18 not sq) skip. try 8 only → 1 perm starting 1,8,17.
# Starting 17→backtrack(17,2): 8(17+8=25✓)→try1(8+1=9✓) → 1 perm. → total 2 ✓
#
# nums=[2,2,2]: 2+2=4✓. Only one distinct array → count=1 ✓
#
# "No bugs. The 'seen' set prevents trying the same value twice at the same
recursion level."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n! / duplicates) with square-constraint pruning – often much faster.
# Space O(n): recursion depth + Counter.
# Follow-up: squareful paths in a graph – build adjacency list of square-summing
pairs,
# count Hamiltonian paths in O(n! / symmetry).
# Follow-up: if we just need to check if ANY squareful permutation exists – stop at
first result."

```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def numSquarefulPerms(self, nums: list[int]) -> int:
        ans = 0
        used = [False] * len(nums)

        def isSquare(num: int) -> bool:
            root = math.isqrt(num)
            return root * root == num

        def dfs(path: list[int]) -> None:

            nonlocal ans
            if len(path) > 1 and not isSquare(path[-1] + path[-2]):
                return
            if len(path) == len(nums):
                ans += 1
                return

            for i, a in enumerate(nums):
                if used[i]:
                    continue

                if i > 0 and nums[i] == nums[i - 1] and not used[i - 1]:
                    continue
                used[i] = True
                dfs(path + [a])
                used[i] = False

        nums.sort()
        dfs([])
        return ans
```

• LeetDoocs

```

class Solution:
    def numSquarefulPerms(self, nums: List[int]) -> int:
        n = len(nums)
        f = [[0] * n for _ in range(1 << n)]
        for j in range(n):
            f[1 << j][j] = 1
        for i in range(1 << n):
            for j in range(n):
                if i >> j & 1:
                    for k in range(n):
                        if (i >> k & 1) and k != j:
                            s = nums[j] + nums[k]
                            t = int(sqrt(s))
                            if t * t == s:
                                f[i][j] += f[i ^ (1 << j)][k]

        ans = sum(f[(1 << n) - 1][j] for j in range(n))
        for v in Counter(nums).values():
            ans //= factorial(v)
        return ans

```

• SimplyLeet

```

from math import isqrt

# Function to check if a number is a perfect square
def is_square(num):
    root = isqrt(num)
    return root * root == num

def numSquarefulPerms(nums):
    n = len(nums)
    nums.sort() # Sort the array to handle duplicates

```

```

visited = [False] * n
result = 0

def dfs(last_index, count):
    nonlocal result
    # If all elements have been used, we found a valid permutation
    if count == n:
        result += 1
        return
    for i in range(n):
        if visited[i]:
            continue
        # Skip duplicates: if the current element is the same as the
        # previous one and the previous one hasn't been used
        if i > 0 and nums[i] == nums[i - 1] and not visited[i - 1]:
            continue
        # If it's the first element or the sum with the last element is a
        # perfect square
        if last_index == -1 or is_square(nums[last_index] + nums[i]):
            visited[i] = True
            dfs(i, count + 1)
            visited[i] = False

dfs(-1, 0)
return result

# Example usage:
if __name__ == "__main__":
    print(numSquarefulPerms([1, 17, 8])) # Expected output: 2
    print(numSquarefulPerms([2, 2, 2])) # Expected output: 1

```

 • LC.ca


```

class Solution:
    def numSquarefulPerms(self, nums: List[int]) -> int:
        n = len(nums)
        f = [[0] * n for _ in range(1 << n)]
        for j in range(n):
            f[1 << j][j] = 1
        for i in range(1 << n):
            for j in range(n):
                if i >> j & 1:
                    for k in range(n):
                        if (i >> k & 1) and k != j:
                            s = nums[j] + nums[k]
                            t = int(sqrt(s))
                            if t * t == s:
                                f[i][j] += f[i ^ (1 << j)][k]

        ans = sum(f[(1 << n) - 1][j] for j in range(n))
        for v in Counter(nums).values():
            ans //= factorial(v)
        return ans

```

◆ Quick Review

Key Insights

- The problem requires ensuring adjacent pairs sum up to a perfect square.
- Sorting the array helps in efficiently handling duplicates; when iterating in the backtracking, skip duplicate elements to avoid overcounting.
- For each valid candidate, check that the sum with the previous number (if any) is a perfect square.

- Use backtracking (DFS) to generate all valid permutations while maintaining a visited flag for each index.
- Since the array may contain duplicates, extra care is taken to only use each occurrence in a controlled order (i.e., if two identical numbers exist, only the first unvisited instance can be chosen at a given recursive call).

Space & Time

Time Complexity: $O(n^2 * 2^n)$ in the worst-case scenario due to exploring all bitmask states with n elements and checking pairwise sums.

Space Complexity: $O(n)$ for recursion and $O(n)$ for the visited flag, plus potentially $O(2^n * n)$ for memoization if implemented.

Solution

The solution uses a backtracking algorithm to explore all permutations. The array is first sorted to handle duplicates. A helper function checks if a number is a perfect square and is used to validate that the sum of every adjacent pair is a perfect square. During DFS, a visited array is maintained to track elements included

so far. Skipping duplicate elements (unless the earlier duplicate was used) is essential to avoid overcounting. This approach ensures that every valid permutation is considered exactly once.

Round 7 · Low · Easy

Round 7

Low Priority · Easy

14 questions · 3 patterns

Dynamic Programming #70 #121

**Bit Manipulation #67 #136 #190 #191 #268
#338**

Math #9 #13 #504 #1056 #1228 #1427

Dynamic Programming

Round 7 · Low · Easy · 2 questions

Dynamic Programming

□ #70 Climbing Stairs

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Dynamic Programming · Memoization
Lists: Grind 169

Problem

You are climbing a staircase. It takes n steps to reach the top.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example 1:

Input: $n = 2$
Output: 2
Explanation: There are two ways to climb to the top.
1. 1 step + 1 step
2. 2 steps

Example 2:

Input: $n = 3$
Output: 3
Explanation: There are three ways to climb to the top.
1. 1 step + 1 step + 1 step
2. 1 step + 2 steps
3. 2 steps + 1 step

Constraints:

- $1 \leq n \leq 45$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def climbStairs(self, n: int) -> int:
        if n == 1:
            return 1

        dp = [0]*(n+1)
        dp[1] = 1
        dp[2] = 2

        for i in range(3,n+1):

            dp[i] = dp[i-1]+dp[i-2]

        return dp[n]
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

You can climb 1 or 2 steps at a time. How many distinct ways to reach step n? Right?"

#

"A few quick questions:

n is at least 1? Yes. Order of steps matters – (1,2) and (2,1) are different? Yes."

#

"Let me walk: n=2 → (1,1), (2) → 2 ways ✓. n=3 → (1,1,1), (1,2), (2,1) → 3 ways ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Recursion: ways(n) = ways(n-1) + ways(n-2). Base: ways(0)=ways(1)=1.

Exponential $O(2^n)$ without memoization – each step branches into two subproblems.

Should I go ahead and optimize?"

```
class _70_ClimbingStairs_Brute:
```

```
    def climbStairs(self, n):
```

```
        if n <= 1: return 1
```

```
        return self.climbStairs(n-1) + self.climbStairs(n-2)
```


PHASE 3 — OPTIMIZE

```
# "The bottleneck is recomputing the same subproblems – this is Fibonacci.
# Track only the last two values – O(1) space:
# a, b → b, a+b each step.
#
# Time: O(n). Space: O(1)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _70_ClimbingStairs:
    def climbStairs(self, n):
        a, b = 1, 1
        for _ in range(n - 1):
            a, b = b, a + b
        return b
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# n=5: a=1,b=1 → (1,2) → (2,3) → (3,5) → (5,8). return 8 ✓
# n=1: loop runs 0 times → return b=1 ✓
# n=2: one iteration: (1,2). return 2 ✓
#
# "No bugs. This is Fibonacci shifted by one: f(n) = Fib(n+1)."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): one pass. Space O(1): two rolling variables.
# Follow-up: if you can climb 1, 2, or 3 steps – same DP, track last 3 values.
# Follow-up: Min Cost Climbing Stairs (#746) – add cost per step; choose
min(dp[n-1], dp[n-2])."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def climbStairs(self, n: int) -> int:
        # dp[i] := the number of ways to climb to the i-th stair
        dp = [1, 1] + [0] * (n - 1)

        for i in range(2, n + 1):
            dp[i] = dp[i - 1] + dp[i - 2]

        return dp[n]
```

```
class Solution:
    def climbStairs(self, n: int) -> int:
        prev1 = 1 # dp[i - 1]
        prev2 = 1 # dp[i - 2]

        for _ in range(2, n + 1):
            dp = prev1 + prev2
            prev2 = prev1
            prev1 = dp

        return prev1
```

• LeetDoocs

```
class Solution:
    def climbStairs(self, n: int) -> int:
        a, b = 0, 1
        for _ in range(n):
            a, b = b, a + b
        return b

import numpy as np

class Solution:
    def climbStairs(self, n: int) -> int:
        res = np.asmatrix([(1, 1)], np.dtype("0"))
        factor = np.asmatrix([(1, 1), (1, 0)], np.dtype("0"))
        n -= 1
        while n:
            if n & 1:
                res *= factor
                factor *= factor
            n >>= 1
        return res[0, 0]
```

• SimplyLeet

```
# Define a function to compute the number of ways to climb stairs.
def climbStairs(n):
    # Base case: if there is only one step, return 1.
    if n == 1:
        return 1
    # Initialize the dp array with n+1 elements.
    dp = [0] * (n + 1)
    # There is 1 way to climb 1 step.
    dp[1] = 1
    # There are 2 ways to climb 2 steps.

    dp[2] = 2
    # Compute the number of ways for each step from 3 to n.
    for i in range(3, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2] # Sum of ways from the previous two
    steps.
    return dp[n]

# Example usage:
print(climbStairs(3)) # Output: 3
```

• LC.ca

```
class Solution:
    def climbStairs(self, n: int) -> int:
        a, b = 0, 1
        for _ in range(n):
            a, b = b, a + b
        return b
```

◆ Quick Review

Key Insights

- The problem can be solved using dynamic programming because the solution for a given step depends on the solutions of the previous two steps.

- The recurrence relation is: $\text{ways}[i] = \text{ways}[i-1] + \text{ways}[i-2]$.
- Base cases: when there is 1 step, there is 1 way; when there are 2 steps, there are 2 ways.
- This is analogous to the Fibonacci sequence, shifting indices accordingly.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$ (can be optimized to $O(1)$ using constant space)

Solution

We use a dynamic programming approach to build a solution gradually. The idea is that to reach step i , you could have come from step $i-1$ (taking one step) or from step $i-2$ (taking two steps). Therefore, the total number of ways to reach step i is the sum of ways to reach step $i-1$ and $i-2$. We initialize $\text{dp}[1] = 1$ and $\text{dp}[2] = 2$, and iterate from 3 to n to fill $\text{dp}[]$. Each code solution below uses this approach with comments to explain the logic.

Dynamic Programming

□ #121 Best Time to Buy and Sell Stock

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: Grind 169

Problem

You are given an array `prices` where `prices[i]` is the price of a given stock on the i th day.

You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock.

Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.

Example 1:

Input: `prices = [7,1,5,3,6,4]`

Output: 5

Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = $6 - 1 = 5$.

Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

Example 2:

Input: prices = [7,6,4,3,1]

Output: 0

Explanation: In this case, no transactions are done and the max profit = 0.

Constraints:

- $1 \leq \text{prices.length} \leq 105$
- $0 \leq \text{prices}[i] \leq 104$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        ans = 0
        minprice = float("inf")

        for p in prices:
            if p < minprice:
                minprice = p
            else:
                ans = max(ans, p-minprice)

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Buy on one day, sell on a later day. Maximize profit. Return 0 if no profit possible. Right?"

#

"A few quick questions:

Must buy before sell? Yes – sell must come after buy day.

At most one transaction? Yes – buy once, sell once."

#

"Let me walk: prices=[7,1,5,3,6,4] → buy at 1(day1), sell at 6(day4) → profit=5 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

```
# Try all pairs (buy_day, sell_day) with buy_day < sell_day. Track max profit.
# O(n2) time, O(1) space.
# Should I go ahead and optimize?"
class _121_BestTimeToBuyAndSellBrute:
```

```
    def maxProfit(self, prices):
        best = 0
        for i in range(len(prices)):
            for j in range(i+1, len(prices)):
                best = max(best, prices[j] - prices[i])
        return best
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the nested scan for max sell price after each buy.
# Key insight: profit = sell_price - buy_price = sell_price - min_so_far.
# Track min_price seen so far. At each day, profit = price - min_price.
#
# Time: O(n). Space: O(1)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _121_BestTimeToBuyAndSell:
    def maxProfit(self, prices):
        min_price = float('inf')
        max_profit = 0
        for price in prices:
            min_price = min(min_price, price)
            max_profit = max(max_profit, price - min_price)
        return max_profit
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# prices=[7,1,5,3,6,4]:
# 7: min=7,profit=0. 1: min=1,profit=0. 5: min=1,profit=4. 3: min=1,profit=4.
# 6: min=1,profit=5. 4: min=1,profit=5. return 5 ✓
#
# prices=[7,6,4,3,1]: always decreasing → all profits negative → max_profit=0 ✓
#
# "No bugs. Initializing min_price=inf handles any starting price correctly."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): single pass. Space O(1): two variables.
# The greedy insight: max profit = max(price - min_price_so_far) across all days.
# Follow-up: Best Time II (#122) – multiple transactions; sum all positive diffs.
# Follow-up: Best Time III (#123) – at most 2 transactions; 4-state DP."
```

Community Solutions (Python)

• WalkCC


```
class Solution:
    def maxProfit(self, prices: list[int]) -> int:
        sellOne = 0
        holdOne = -math.inf

        for price in prices:
            sellOne = max(sellOne, holdOne + price)
            holdOne = max(holdOne, -price)

        return sellOne
```

• LeetCode

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        ans, mi = 0, inf
        for v in prices:
            ans = max(ans, v - mi)
            mi = min(mi, v)
        return ans
```

• SimplyLeet

```
def maxProfit(prices):
    # Initialize min_price as infinity and max_profit as 0
    min_price = float('inf')
    max_profit = 0
    # Iterate through each price in the list
    for price in prices:
        # Update the minimum price if current price is lower
        if price < min_price:
            min_price = price
        else:
            # Calculate current profit and update max_profit if it is higher
            max_profit = max(max_profit, price - min_price)
    return max_profit

# Example usage:
# result = maxProfit([7,1,5,3,6,4])
# print(result) # Output should be 5
```

• LC.ca

```

    ...

>>> import math
>>> a = math.inf
>>> a
inf

>>> x = float('-inf')
>>> y = float('inf')
>>> print(x < y) # Output: True
True

>>>
>>> z = -math.inf
>>> x==z
True

    ...
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        ans, mi = 0, inf
        for v in prices:

            mi = min(mi, v)
            ans = max(ans, v - mi)
        return ans

#####

class Solution(object):
    def maxProfit(self, prices):
        """
        :type prices: List[int]

```

```
:rtype: int
"""
if not prices:
    return 0
ans = 0
pre = prices[0]
for i in range(1, len(prices)):
    pre = min(pre, prices[i])
    ans = max(prices[i] - pre, ans)
return ans
```

◆ Quick Review

Key Insights

- Traverse the list of prices once while keeping track of the minimum price encountered so far.
- At each day, calculate the profit if you sold the stock on that day.
- Update the maximum profit when the current profit exceeds the previous maximum.
- The optimal solution makes a single pass through the array with constant space usage.

Space & Time

Time Complexity: $O(n)$ – requires one pass through the prices array.

Space Complexity: $O(1)$ – only a few extra variables are used.

Solution

We solve the problem using a simple linear scan. We maintain two key variables: one for the minimum price seen so far and another for the maximum profit calculated so far. As we iterate, we continuously update the minimum price if a lower price is found, and then compute the potential profit at the current price. If this profit exceeds the current maximum profit, we update the maximum profit. The final maximum profit is returned after iterating through all prices.

Bit Manipulation

Round 7 · Low · Easy · 6 questions

Bit Manipulation

□ #67 Add Binary

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · String · Bit Manipulation · Simulation
Lists: Grind 169

Problem

Given two binary strings *a* and *b*, return their sum as a binary string.

Example 1:

```
Input: a = "11", b = "1"
Output: "100"
```

Example 2:

```
Input: a = "1010", b = "1011"
Output: "10101"
```

Constraints:

- $1 \leq a.length, b.length \leq 104$
- *a* and *b* consist only of '0' or '1' characters.
- Each string does not contain leading zeros except for the zero itself.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def addBinary(self, a: str, b: str) -> str:
        ans = []
        i, j, carry = len(a) - 1, len(b) - 1, 0
        while i >= 0 or j >= 0 or carry:
            carry += (0 if i < 0 else int(a[i])) + (0 if j < 0 else int(b[j]))
            carry, v = divmod(carry, 2)
            ans.append(str(v))
            i, j = i - 1, j - 1
        return "".join(ans[::-1])
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Given two binary strings a and b, return their sum as a binary string. Right?"
#
# "A few quick questions:
# Can they have leading zeros? No — except '0' itself. Can they be '0'? Yes."
#
# "Let me walk: a='11', b='1' → 11+01=100 → '100' ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Convert both to integers, add, convert back to binary string.
# O(n) but relies on Python big-int — breaks in fixed-width languages.
# Should I go ahead and optimize?"
class _67_AddBinary_Brute:
    def addBinary(self, a, b):
        return bin(int(a, 2) + int(b, 2))[2:]
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is big-int conversion.
# Simulate binary addition from right to left with a carry variable:
# At each step: total = a_bit + b_bit + carry. digit = total % 2. carry = total // 2.
#
# Time: O(max(n,m)). Space: O(max(n,m)) for the result."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _67_AddBinary:
    def addBinary(self, a, b):
        i, j, carry = len(a)-1, len(b)-1, 0
        result = []
```



```

while i >= 0 or j >= 0 or carry:
    total = carry
    if i >= 0: total += int(a[i]); i -= 1
    if j >= 0: total += int(b[j]); j -= 1
    carry, digit = divmod(total, 2)
    result.append(str(digit))
return ''.join(reversed(result))

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

a='11', b='1':

i=1,j=0: total=1+1+0=2. carry=1,digit=0. result=['0'].

i=0,j=-1: total=1+0+1=2. carry=1,digit=0. result=['0','0'].

i=-1,j=-1,carry=1: total=1. carry=0,digit=1. result=['0','0','1'].

reversed → '100' ✓

#

a='0', b='0': total=0, carry=0, digit=0. → '0' ✓

#

"No bugs. The while condition 'i>=0 or j>=0 or carry' handles the final carry correctly."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\max(n,m))$: process each bit once. Space $O(\max(n,m))$ for result.

Same pattern as Add Two Numbers (#2) – carry propagation from right to left.

Follow-up: Multiply Strings (#43) – $O(n*m)$ positional multiplication.

Follow-up: Sum of Two Integers (#371) – use XOR for sum, AND for carry (bitwise only)."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def addBinary(self, a: str, b: str) -> str:
        ans = []
        carry = 0
        i = len(a) - 1
        j = len(b) - 1

        while i >= 0 or j >= 0 or carry:
            if i >= 0:
                carry += int(a[i])

```

```

        i -= 1
    if j >= 0:
        carry += int(b[j])
        j -= 1
    ans.append(str(carry % 2))
    carry //= 2

    return ''.join(reversed(ans))

```

• LeetDoocs

```

class Solution:
    def addBinary(self, a: str, b: str) -> str:
        ans = []
        i, j, carry = len(a) - 1, len(b) - 1, 0
        while i >= 0 or j >= 0 or carry:
            carry += (0 if i < 0 else int(a[i])) + (0 if j < 0 else int(b[j]))
            carry, v = divmod(carry, 2)
            ans.append(str(v))
            i, j = i - 1, j - 1
        return "".join(ans[::-1])

```

• SimplyLeet

```

# Function to add two binary strings
def addBinary(a: str, b: str) -> str:
    i = len(a) - 1 # Pointer for string a
    j = len(b) - 1 # Pointer for string b
    carry = 0 # Initialize carry to 0
    result = [] # List to store binary digits

    # Loop until both strings are processed and no carry remains
    while i >= 0 or j >= 0 or carry:
        total_sum = carry # Start with the carry from previous step

```

```

    if i >= 0: # If there is a digit left in a
        total_sum += int(a[i]) # Convert character to integer and add
        i -= 1 # Move to the next digit
    if j >= 0: # If there is a digit left in b
        total_sum += int(b[j]) # Convert character to integer and add
        j -= 1 # Move to the next digit

    # Append the remainder of total_sum divided by 2 (0 or 1)
    result.append(str(total_sum % 2))

    carry = total_sum // 2 # Update the carry

    # Reverse the result since we built it backwards and join to form a string
    return ''.join(result[::-1])

# Sample test cases
print(addBinary("11", "1")) # Expected output: "100"
print(addBinary("1010", "1011")) # Expected output: "10101"

```

• LC.ca

```

class Solution:
    def addBinary(self, a: str, b: str) -> str:
        ans = []
        i, j, carry = len(a) - 1, len(b) - 1, 0
        while i >= 0 or j >= 0 or carry:
            carry += (0 if i < 0 else int(a[i])) + (0 if j < 0 else int(b[j]))
            carry, v = divmod(carry, 2)
            ans.append(str(v))
            i, j = i - 1, j - 1
        return "".join(ans[::-1])

```

◆ Quick Review

Key Insights

- Simulate binary addition just like you add numbers digit by digit.

- Process the strings from right to left, handling the carry at each step.
- Use modulo and division operations to extract the current digit and update the carry.
- Append any remaining carry at the end.

Space & Time

Time Complexity: $O(\max(n, m))$, where n and m are the lengths of the two binary strings.

Space Complexity: $O(\max(n, m))$ for storing the resulting binary string.

Solution

The strategy is to iterate over the input strings from the end to the beginning. At each step, calculate the sum of corresponding bits and the existing carry. Use the modulo operation ($\% 2$) to determine the current digit and integer division ($// 2$) to update the carry. Once all digits have been processed, reverse the result to obtain the final binary string.

Bit Manipulation

□ #136 Single Number

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Bit Manipulation
Lists: Grind 169

Problem

Given a non-empty array of integers `nums`, every element appears twice except for one. Find that single one.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Example 1:

Input: `nums = [2,2,1]`

Output: 1

Example 2:

Input: `nums = [4,1,2,1,2]`

Output: 4

Example 3:

Input: `nums = [1]`

Output: 1

Constraints:

- $1 \leq \text{nums.length} \leq 3 * 10^4$
- $-3 * 10^4 \leq \text{nums}[i] \leq 3 * 10^4$
- Each element in the array appears twice except for one element which appears only once.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        ans = 0
        for x in nums:
            ans = ans ^ x

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Every element appears twice except one. Find that single element. $O(n)$ time, $O(1)$ space. Right?"

#

"A few quick questions:

Can the single element be negative? Yes. Array has at least one element? Yes."

#

"Let me walk: $\text{nums}=[2,2,1] \rightarrow 1 \checkmark$. $\text{nums}=[4,1,2,1,2] \rightarrow 4 \checkmark$ "

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Hash set: add if absent, remove if present. The remaining element is the answer.

$O(n)$ time, $O(n)$ space – violates the $O(1)$ space requirement.

Should I go ahead and optimize?"

```
class _136_SingleNumber_Brute:
```

```

def singleNumber(self, nums):
    seen = set()
    for n in nums:
        if n in seen: seen.remove(n)
        else: seen.add(n)
    return seen.pop()

# PHASE 3 — OPTIMIZE
# "The bottleneck is the O(n) space hash set.
# XOR trick: a^a=0, a^0=a. XOR all elements – pairs cancel to 0, leaving the single
# element.
#
# Time: O(n). Space: O(1)."
```

```

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _136_SingleNumber:
    def singleNumber(self, nums):
        result = 0
        for num in nums:
            result ^= num
        return result

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# nums=[4,1,2,1,2]: 0^4=4. 4^1=5. 5^2=7. 7^1=6. 6^2=4. return 4 ✓
# nums=[2,2,1]: 0^2=2. 2^2=0. 0^1=1. return 1 ✓
# nums=[1]: 0^1=1. return 1 ✓
#
# "No bugs. XOR is commutative and associative – order doesn't matter."
```

```

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): single pass. Space O(1): one variable.
# XOR is the canonical trick for cancelling duplicate pairs in O(1) space.
# Follow-up: Single Number II (#137) – every element appears 3 times except one.
# Count bits: for each bit position, count appearances % 3 → leftover = single.
# Follow-up: Single Number III (#260) – two singles. XOR all → get XOR of the two.
# Use rightmost set bit to split into two groups."
```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def singleNumber(self, nums: list[int]) -> int:
        return functools.reduce(operator.xor, nums, 0)
```


• LeetCode

```
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        return reduce(xor, nums)
```

• SimplyLeet

```
# Function to find the single number in the list
def singleNumber(nums):
    # Initialize result to 0; any number XOR 0 is the number itself
    result = 0
    # Iterate through every number in the list
    for num in nums:
        # XOR the current result with the current number
        result ^= num # Using XOR to cancel duplicates
    # Return the unique number that did not cancel out
    return result
```

```
# Example usage
nums = [4, 1, 2, 1, 2]
print(singleNumber(nums)) # Output should be 4
```

• LC.ca

```
'''
>>> from functools import reduce
>>> reduce(lambda x, y: x ^ y, [3, 5, 3])
5
'''

from functools import reduce

class Solution:
    def singleNumber(self, nums: List[int]) -> int:
```

```
        return reduce(lambda x, y: x ^ y, nums)

#####

class Solution(object):
    def singleNumber(self, nums):
        """
        :type nums: List[int]
        :rtype: int
        """

        for i in range(1, len(nums)):
            nums[0] ^= nums[i]
        return nums[0]
```

◆ Quick Review

Key Insights

- Using bitwise XOR: XOR-ing a number with itself cancels out to 0.
- XOR is both commutative and associative, so the order of operations does not matter.
- A single traversal (linear time) and constant extra space is sufficient for solving the problem.

Space & Time

Time Complexity: $O(n)$ – We only traverse the array once.

Space Complexity: $O(1)$ – Only a single extra variable is used regardless of input size.

Solution

We can solve this problem by initializing a variable to 0 and then traversing the array while XOR-ing each element with the variable. Due to the properties of XOR, pairs of identical numbers will cancel each other out ($x \text{ XOR } x = 0$) and the unique number will be left as the final result. This approach uses constant extra space and performs in linear time.

Bit Manipulation

□ #190 Reverse Bits

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Divide and Conquer · Bit Manipulation
Lists: Grind 169

Problem

Reverse bits of a given 32 bits signed integer.

Example 1:

Input: n = 43261596

Output: 964176192

Explanation:

Example 2:

Input: n = 2147483644

Output: 1073741822

Explanation:

Constraints:

- $0 \leq n \leq 2^{31} - 2$
- n is even.

Follow up: If this function is called many times, how would you optimize it?

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def reverseBits(self, n: int) -> int:
        ans = 0
        p = 31

        while n:
            d = n%2
            ans += (d << p)
            p-=1
            n = n//2

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Reverse the 32-bit binary representation of an unsigned integer. Return the
# result. Right?"
#
# "A few quick questions:
# Always treat as 32-bit even if leading zeros? Yes.
# Return as unsigned 32-bit integer? Yes."
#
# "Let me walk: n=43261596 (binary: 000000101001010000001111010011100)
# reversed: 00111001011110000010100101000000 → 964176192 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Convert to 32-bit binary string, reverse, convert back.
# O(32) = O(1) time and space – clean and acceptable.
# Should I go ahead and optimize?"
class _190_ReverseBits_Brute:
    def reverseBits(self, n):
        return int(bin(n)[2:].zfill(32)[::-1], 2)
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is string allocation.
# Bit-by-bit reversal with no string: for 32 iterations, extract LSB of n (n&1),
```

```
# shift result left and OR in the extracted bit, then shift n right.
#
# Time: O(32) = O(1). Space: O(1)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _190_ReverseBits:
    def reverseBits(self, n):
        result = 0
        for _ in range(32):
            result = (result << 1) | (n & 1)
            n >>= 1
        return result
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
```

```
#
# n=43261596: each iteration shifts result left and adds LSB of n.
# After 32 iterations, result = 964176192 ✓
#
# n=0: all bits are 0, result stays 0 ✓
# n=4294967293 (1111111111111111111111111111101): reversed = 1011111...→ specific value ✓
#
# "No bugs. 32 fixed iterations handle all 32 bits including leading zeros."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(32)=O(1). Space O(1).
# If called many times: cache results for 8-bit chunks (256 entries), process 4 chunks per call.
# Follow-up: Number of 1 Bits (#191) – count set bits instead of reversing.
# Follow-up: Power of Two (#231) – n>0 and n&(n-1)==0 checks exactly one set bit."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def reverseBits(self, n: int) -> int:
        ans = 0

        for i in range(32):
            if n >> i & 1:
                ans |= 1 << 31 - i

        return ans
```

• LeetDoocs

```
class Solution:
    def reverseBits(self, n: int) -> int:
        ans = 0
        for i in range(32):
            ans |= (n & 1) << (31 - i)
            n >>= 1
        return ans
```

• SimplyLeet

```
# Function to reverse bits of a 32-bit binary string.
def reverse_bits(n: str) -> int:
    # Reverse the binary string using slicing.
    reversed_binary = n[::-1]
    # Convert the reversed binary string to an integer.
    result = int(reversed_binary, 2)
    return result
```

```
# Example usage:
binary_input = "00000010100101000001111010011100"
```

```
print(reverse_bits(binary_input)) # Expected output: 964176192
```

• LC.ca

```
class Solution:
    def reverseBits(self, n: int) -> int:
        res = 0
        for i in range(32):
            res |= (n & 1) << (31 - i)
            n >>= 1
        return res
```

◆ Quick Review

Key Insights

- The input is a binary string of fixed length (32 bits).
- Reversing the bits is equivalent to reversing the string.
- The reversed binary string can be converted back into its integer representation.
- Since the number of bits is constant (32), the solution can be achieved in constant time and constant space.
- For performance optimization when this function is called many times, caching may be applied for common intermediate results.

Space & Time

Time Complexity: $O(1)$ – We always process 32 bits regardless of the input.

Space Complexity: $O(1)$ – Only constant extra space is used.

Solution

We solve the problem by reversing the 32-character binary string representing the unsigned integer. Once reversed, the new binary string is converted into its integer value. An alternative solution involves using bitwise operations where you iterate over each

bit, shift the result to the left, and add the current bit. This approach does the reversal one bit at a time. The chosen approach leverages string reversal which is straightforward to implement and understand.

Bit Manipulation

□ #191 Number of 1 Bits

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Divide and Conquer · Bit Manipulation
Lists: Grind 169

Problem

Given a positive integer n , write a function that returns the number of set bits in its binary representation (also known as the Hamming weight).

Example 1:

Input: $n = 11$

Output: 3

Explanation:

The input binary string 1011 has a total of three set bits.

Example 2:

Input: $n = 128$

Output: 1

Explanation:

The input binary string 10000000 has a total of one set bit.

Example 3:

Input: n = 2147483645

Output: 30

Explanation:

The input binary string

111111111111111111111111111111111101 has a total of thirty set bits.

Constraints:

- $1 \leq n \leq 2^{31} - 1$

Follow up: If this function is called many times, how would you optimize it?

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def hammingWeight(self, n: int) -> int:
        ans = 0
        while n:
            d = n%2
            ans += d
            n = n//2

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Let me make sure I understand the problem correctly.
# Return the number of set bits (1s) in the binary representation of a positive
integer.
# Also called the Hamming weight. Right?"
#
# "A few quick questions:
# Treat as unsigned 32-bit? Yes. Can n be 0? Constraints say n>=1."
#
# "Let me walk: n=11 (1011) → 3 ✓. n=128 (10000000) → 1 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Check each of 32 bits using a bitmask and right shift.
# O(32) = O(1) time, O(1) space.
# Should I go ahead and optimize?"
class _191_NumberOf1Bits_Brute:
    def hammingWeight(self, n):
        count = 0
        while n:
            count += n & 1
            n >>= 1
        return count

# PHASE 3 — OPTIMIZE
# "The bottleneck is iterating all 32 bits even for sparse inputs.
# Brian Kernighan's trick: n & (n-1) clears the lowest set bit.
# Count how many times we can clear a set bit until n=0.
#
# Time: O(k) where k = number of set bits. Space: O(1)."
```

```

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _191_NumberOf1Bits:
    def hammingWeight(self, n):
        count = 0
        while n:
            n &= n - 1
            count += 1
        return count

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# n=11 (1011): 1011&1010=1010(count=1). 1010&1001=1000(count=2).
1000&0111=0000(count=3). → 3 ✓
# n=128 (10000000): one iteration → n=0. count=1 ✓
# n=0: while never enters → 0 ✓
#
# "No bugs. n&(n-1) always clears exactly the lowest set bit – guaranteed
termination."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(k)$: k = popcount, at most 32 iterations. Space $O(1)$.

Brian Kernighan faster than 32-iteration scan when bits are sparse.

Built-in: `bin(n).count('1')` – valid in interviews.

Follow-up: Hamming Distance (#461) – `popcount(x XOR y)`.

Follow-up: Counting Bits (#338) – `popcount` for all i in $[0, n]$ using DP."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def hammingWeight(self, n: int) -> int:
        ans = 0

        for i in range(32):
            if (n >> i) & 1:
                ans += 1

        return ans
```

```
class Solution:
    def hammingWeight(self, n: int) -> int:
        return n.bit_count()
```

• LeetDoocs

```
class Solution:
    def hammingWeight(self, n: int) -> int:
        ans = 0
        while n:
            n &= n - 1
            ans += 1
        return ans
```

• SimplyLeet

```
# Function to count the number of set bits in a given integer n
def hammingWeight(n: int) -> int:
    count = 0 # Initialize count of set bits
    while n:
        # Remove the lowest set bit from n
        n &= n - 1
        count += 1 # Increment count for each set bit removed
    return count

# Example usage:

print(hammingWeight(11)) # Output: 3
```

• LC.ca

```
class Solution:
    def hammingWeight(self, n: int) -> int:
        ans = 0
        while n:
            n &= n - 1
            ans += 1
        return ans
```

◆ Quick Review

Key Insights

- The problem is essentially counting the number of 1s in the binary representation of a number.
- A common efficient approach uses bit manipulation: repeatedly clear the lowest set bit until n becomes zero.
- The expression $n \& (n - 1)$ clears the lowest set bit of n .

- This method only iterates as many times as there are set bits, making it efficient.

Space & Time

Time Complexity: $O(k)$, where k is the number of set bits, worst-case $O(\log n)$ (or $O(1)$ considering 32-bit integers).

Space Complexity: $O(1)$.

Solution

To solve the problem, we use the bit manipulation trick $n \& (n - 1)$ which removes the lowest set bit from n . The algorithm is as follows: Initialize a counter to 0. While n is not zero, update n by $n = n \& (n - 1)$, and increment the counter. The counter will hold the number of set bits by the end of the loop. This approach leverages efficient bit-level operations and works well even when the function is called frequently.

Bit Manipulation

□ #268 Missing Number

EAS[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Math · Binary Search · Bit
Manipulation

Lists: Grind 169

Problem

Given an array `nums` containing n distinct numbers in the range $[0, n]$, return the only number in the range that is missing from the array.

Example 1:

Input: `nums = [3,0,1]`

Output: 2

Explanation:

$n = 3$ since there are 3 numbers, so all numbers are in the range $[0,3]$. 2 is the missing number in the range since it does not appear in `nums`.

Example 2:

Input: `nums = [0,1]`

Output: 2

Explanation:

$n = 2$ since there are 2 numbers, so all numbers are in the range $[0,2]$. 2 is the missing number in the range since it does not appear in `nums`.

Example 3:

Input: `nums = [9,6,4,2,3,5,7,0,1]`

Output: 8

Explanation:

$n = 9$ since there are 9 numbers, so all numbers are in the range $[0,9]$. 8 is the missing number in the range since it does not appear in `nums`.

Constraints:

- **$n == \text{nums.length}$**
- **$1 \leq n \leq 10^4$**
- **$0 \leq \text{nums}[i] \leq n$**
- **All the numbers of `nums` are unique.**

Follow up: Could you implement a solution using only $O(1)$ extra space complexity and $O(n)$ runtime complexity?

My LeetCode Solution

☐ • **My LeetCode Solution**

```
class Solution:
    def missingNumber(self, nums: List[int]) -> int:
        ans = len(nums)

        for i,x in enumerate(nums):
            ans = ans ^ i ^ x

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Array of n distinct numbers from [0,n]. Exactly one number is missing. Find it. Right?"

#

"A few quick questions:

Array length is n (not n+1)? Yes – one number from [0,n] is absent.

Can 0 be missing? Yes. Can n itself be missing? Yes."

#

"Let me walk: nums=[3,0,1] → 2 missing ✓. nums=[9,6,4,2,3,5,7,0,1] → 8 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Expected sum = $n*(n+1)/2$. Missing = expected - actual_sum.

O(n) time, O(1) space.

Should I go ahead and optimize?"

```
class _268_MissingNumber_Brute:
    def missingNumber(self, nums):
        n = len(nums)
        return n*(n+1)//2 - sum(nums)
```

PHASE 3 — OPTIMIZE

"The bottleneck is potential overflow in the sum formula for fixed-width languages.

XOR approach avoids overflow: XOR all indices 0..n and all nums. Pairs cancel, leaving the missing.

#

Time: O(n). Space: O(1). Overflow-safe."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _268_MissingNumber:
    def missingNumber(self, nums):
        result = len(nums)
        for i, num in enumerate(nums):
            result ^= i ^ num
```

```
return result
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
```

```
#
```

```
# nums=[3,0,1], n=3:
```

```
# result=3. i=0: 3^0^3=0. i=1: 0^1^0=1. i=2: 1^2^1=2. return 2 ✓
```

```
#
```

```
# nums=[0,1]: n=2. result=2. i=0: 2^0^0=2. i=1: 2^1^1=2. return 2 ✓
```

```
#
```

```
# "No bugs. XOR cancels every pair (i, nums[i]) except for the missing index."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Sum formula O(n): simple but may overflow in Java/C++ for large n.
```

```
# XOR O(n): overflow-safe, same asymptotic.
```

```
# Follow-up: Find Duplicate Number (#287) – one value appears twice; Floyd's cycle detection.
```

```
# Follow-up: First Missing Positive (#41) – different range, needs cyclic-sort approach."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def missingNumber(self, nums: List[int]) -> int:
        ans = len(nums)

        for i, num in enumerate(nums):
            ans ^= i ^ num

        return ans
```

• LeetDoocs

```
class Solution:
    def missingNumber(self, nums: List[int]) -> int:
        return reduce(xor, (i ^ v for i, v in enumerate(nums, 1)))

class Solution:
    def missingNumber(self, nums: List[int]) -> int:
        n = len(nums)
        return (1 + n) * n // 2 - sum(nums)
```

• SimplyLeet

```
# Function to find the missing number in the array
def missingNumber(nums):
    # Calculate n, where n is the length of the nums array
    n = len(nums)
    # Compute the expected total sum using the arithmetic sum formula for
    # numbers 0 to n
    expected_sum = n * (n + 1) // 2
    # Compute the actual sum of the numbers in the array
    actual_sum = sum(nums)
    # The missing number is the difference between expected and actual sums
    return expected_sum - actual_sum
```

```
# Example Usage
print(missingNumber([3, 0, 1])) # Output: 2
```

• LC.ca

```
class Solution:
    def missingNumber(self, nums: List[int]) -> int:
        return reduce(xor, (i ^ v for i, v in enumerate(nums, 1)))
```

◆ Quick Review

Key Insights

- The range of numbers is continuous from 0 to n, but one number is missing.
- The sum of numbers from 0 to n can be computed using the arithmetic sum formula.
- Subtracting the sum of the given array from the expected sum gives the missing number.
- There is a follow-up challenge to solve the problem in $O(n)$ time and $O(1)$ extra space.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

We compute the expected sum of all numbers from 0 to n using the formula $\text{sum} = n*(n+1)/2$. Then, we compute the actual sum of the numbers in the array. The missing number is obtained by subtracting the actual sum from the expected sum. This approach uses arithmetic operations ($O(1)$ extra space) and loops over the array once ($O(n)$ time).

Bit Manipulation

□ #338 Counting Bits

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Dynamic Programming · Bit Manipulation
Lists: Grind 169

Problem

Given an integer n , return an array `ans` of length $n + 1$ such that for each i ($0 \leq i \leq n$), `ans[i]` is the number of 1's in the binary representation of i .

Example 1:

```
Input: n = 2
Output: [0,1,1]
Explanation:
0 --> 0
1 --> 1
2 --> 10
```

Example 2:

```
Input: n = 5
Output: [0,1,1,2,1,2]
Explanation:
0 --> 0
1 --> 1
2 --> 10
3 --> 11
4 --> 100
```

Constraints:

- $0 \leq n \leq 105$

Follow up:

- It is very easy to come up with a solution with a runtime of $O(n \log n)$. Can you do it in linear time $O(n)$ and possibly in a single pass?
- Can you do it without using any built-in function (i.e., like `__builtin_popcount` in C++)?

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def countBits(self, n: int) -> List[int]:
        dp = [0]*(n+1)
        for i in range(1, n+1):
            h = i//2
            d = i%2
            dp[i] = dp[h]+d
        return dp
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Return ans[i] = number of 1-bits in i, for all i in [0, n]. Length n+1. Right?"

#

"A few quick questions:

n can be 0? Yes — ans=[0]. Must run in $O(n)$ time? Yes per follow-up."

#

"Let me walk: $n=2 \rightarrow [0,1,1]$ ✓. $n=5 \rightarrow [0,1,1,2,1,2]$ ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

For each i from 0 to n, count 1-bits using Brian Kernighan or `bin().count()`.

$O(n \log n)$ since each number has $O(\log n)$ bits.

```

# Should I go ahead and optimize?"
class _338_CountingBits_Brute:
    def countBits(self, n):
        return [bin(i).count('1') for i in range(n+1)]

# PHASE 3 — OPTIMIZE
# "The bottleneck is counting bits independently for each number.
# DP with bit trick: ans[i] = ans[i >> 1] + (i & 1).
# Shifting right by 1 drops the LSB – we already know that count. Add 1 if LSB was
# set.
#
# Time: O(n). Space: O(n) for the output array."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _338_CountingBits:
    def countBits(self, n):
        ans = [0] * (n + 1)
        for i in range(1, n + 1):
            ans[i] = ans[i >> 1] + (i & 1)
        return ans

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# n=5:
# ans[1]=ans[0]+1=1. ans[2]=ans[1]+0=1. ans[3]=ans[1]+1=2.
# ans[4]=ans[2]+0=1. ans[5]=ans[2]+1=2.
# → [0,1,1,2,1,2] ✓
#
# n=0: loop never runs → [0] ✓
#
# "No bugs. Every i shares a prefix with i>>1, which was computed at an earlier
# iteration."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass, O(1) per entry. Space O(n): output only.
# Alternative: ans[i] = ans[i & (i-1)] + 1 (clear lowest set bit then add 1).
# Follow-up: Number of 1 Bits (#191) – single number, Brian Kernighan O(k) per call.
# Follow-up: Total Hamming Distance (#477) – use column-wise bit counting across all
# pairs."

```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def countBits(self, n: int) -> list[int]:
        # f(i) := i's number of 1s in bitmask
        # f(i) = f(i / 2) + i % 2
        ans = [0] * (n + 1)

        for i in range(1, n + 1):
            ans[i] = ans[i // 2] + (i & 1)

        return ans
```

• LeetCode

```
class Solution:
    def countBits(self, n: int) -> List[int]:
        return [i.bit_count() for i in range(n + 1)]
```

```
class Solution:
    def countBits(self, n: int) -> List[int]:
        ans = [0] * (n + 1)
        for i in range(1, n + 1):
            ans[i] = ans[i & (i - 1)] + 1
        return ans
```

• SimplyLeet

```
def countBits(n):
    # Create a list to store the number of 1's for each number from 0 to n
    dp = [0] * (n + 1)
    # Loop over each number starting from 1, since dp[0] is already 0
    for i in range(1, n + 1):
        # dp[i] is computed as the count for i//2 plus 1 if i is odd (i & 1)
        dp[i] = dp[i >> 1] + (i & 1)
    return dp
```

Example usage:

```
# print(countBits(5)) -> [0, 1, 1, 2, 1, 2]
```

• LC.ca

```
class Solution:
    def countBits(self, n: int) -> List[int]:
        ans = [0] * (n + 1)
        for i in range(1, n + 1):
            ans[i] = ans[i & (i - 1)] + 1
        return ans
```

◆ Quick Review

Key Insights

- We can use dynamic programming to build the solution by utilizing the count of bits for smaller numbers.
- The relationship $dp[i] = dp[i \gg 1] + (i \& 1)$ holds, where $i \gg 1$ gives the count for i divided by 2 (dropping the least significant bit) and $(i \& 1)$ checks if the least significant bit is 1.
- This approach computes the result for each number in constant time, resulting in a linear time solution.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(n)$

Solution

We use a dynamic programming approach where an array (or list) `dp` is initialized with size $n+1$. The key idea is to iterate from 1 to n and for each number, use the formula $dp[i] = dp[i \gg 1] + (i \& 1)$. The expression $(i \gg 1)$ effectively divides i by 2, giving the count of 1's for that half number. The $(i \& 1)$ operation checks if i is odd, adding 1 if true. This method ensures each `dp[i]` is computed in constant time, leading to an overall $O(n)$ time complexity.

Math

Round 7 · Low · Easy · 6 questions

Math

□ #9 Palindrome Number

EAS[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Math

Lists: Grind 169

Problem

Given an integer x , return true if x is a palindrome, and false otherwise.

Example 1:

Input: $x = 121$
Output: true
Explanation: 121 reads as 121 from left to right and from right to left.

Example 2:

Input: $x = -121$
Output: false
Explanation: From left to right, it reads -121. From right to left, it becomes 121-. Therefore it is not a palindrome.

Example 3:

Input: $x = 10$
Output: false
Explanation: Reads 01 from right to left. Therefore it is not a palindrome.

Constraints:

- $-231 \leq x \leq 231 - 1$

Follow up: Could you solve it without converting the integer to a string?

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def isPalindrome(self, x: int) -> bool:
        if x < 0 or (x % 10 == 0 and x != 0):
            return False

        half = 0
        while half < x:
            d = x % 10
            x = x // 10
            half = half * 10 + d

        return x == half or x == half // 10
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Return true if an integer reads the same forwards and backwards.

Negative numbers are not palindromes. The follow-up asks to solve without string conversion. Right?"

#

"A few quick questions:

x=0 → true? Yes – 0 reads the same. x=10 → false? Yes – '10' reversed is '01'. Right."

#

"Let me walk: x=121 → true ✓. x=-121 → false ✓. x=10 → false ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Convert to string, compare with its reverse.

O(d) time and space where d = number of digits.

Should I go ahead and optimize?"

```
class _9_PalindromeNumber_Brute:
    def isPalindrome(self, x):
        return str(x) == str(x)[::-1]
```

PHASE 3 — OPTIMIZE**# "The bottleneck is the string allocation."**

Reverse only the SECOND HALF of the number – no string conversion:

Negative → false. $x \% 10 == 0$ and $x != 0$ → false (trailing zero means leading zero).# Build reversed_half digit by digit; stop when $x \leq$ reversed_half.

#

Time: $O(d/2)$. Space: $O(1)$."**# PHASE 4 — CLEAN CODE****# "Now I'll implement the optimized approach with meaningful names."****class _9_PalindromeNumber:** **def isPalindrome(self, x):** **if $x < 0$ or $(x \% 10 == 0 \text{ and } x != 0)$:** **return False** **reversed_half = 0** **while $x >$ reversed_half:** **reversed_half = reversed_half * 10 + $x \% 10$** **$x //= 10$** **return $x ==$ reversed_half or $x ==$ reversed_half // 10****# PHASE 5 — TEST & DEBUG****# "Let me trace through a few cases."**

#

$x=121$: rev=0. $x=12$, rev=1. $x=1$, rev=12. $1 \leq 12$ →stop. $x==rev//10$: $1==1$ ✓ (odd length)# $x=1221$: rev=0. $x=122$, rev=1. $x=12$, rev=12. $12 \leq 12$ →stop. $x==rev$: $12==12$ ✓ (even length)# $x=-121$: negative → False ✓. $x=10$: $10 \% 10 = 0$, $x != 0$ → False ✓

#

"No bugs. The rev//10 check removes the middle digit for odd-length numbers."**# PHASE 6 — COMPLEXITY & FOLLOW-UP****# "Time $O(d/2)$: process half the digits. Space $O(1)$: integer arithmetic only."**# The trailing-zero check prevents false positives like $x=100$ (reversed='001' ≠ '100').

Follow-up: Palindrome Linked List (#234) – same reverse-half idea on a list.

Follow-up: largest palindrome product of two n -digit numbers – search from max down."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def isPalindrome(self, x: int) -> bool:
        if x < 0:
            return False

        rev = 0
        y = x

        while y:
            rev = rev * 10 + y % 10

            y //= 10

        return rev == x

```

• LeetCode

```

class Solution:
    def isPalindrome(self, x: int) -> bool:
        if x < 0 or (x and x % 10 == 0):
            return False

        y = 0
        while y < x:
            y = y * 10 + x % 10
            x //= 10
        return x in (y, y // 10)

```

• SimplyLeet

```

# Function to check if a number is a palindrome without converting to a string.
def isPalindrome(x):
    # Negative numbers and numbers ending with 0 (but not 0 itself) are not
    # palindromic.
    if x < 0 or (x % 10 == 0 and x != 0):
        return False
    reversed_half = 0
    # Process: Reverse half of the number.
    while x > reversed_half:
        # Add the last digit of x to reversed_half.
        reversed_half = reversed_half * 10 + x % 10

```

```

        # Remove the last digit from x.
        x //= 10
        # For numbers with odd digits, reversed_half // 10 removes the middle
        digit.
        return x == reversed_half or x == reversed_half // 10

# Example usage:
print(isPalindrome(121))    # Output: True
print(isPalindrome(-121))   # Output: False
print(isPalindrome(10))     # Output: False

```

• LC.ca

```

class Solution:
    def isPalindrome(self, x: int) -> bool:
        if x < 0 or (x and x % 10 == 0):
            return False
        y = 0
        while y < x:
            y = y * 10 + x % 10
            x //= 10
        return x in (y, y // 10)

```

◆ Quick Review

Key Insights

- Negative numbers are not palindromic.
- A number ending in 0 (except 0 itself) cannot be a palindrome.
- Instead of reversing the entire number (risking overflow), reverse only half of the number and compare with the other half.
- For numbers with an odd number of digits, the middle digit can be disregarded.

Space & Time

Time Complexity: $O(\log_{10}(n))$ – Each iteration reduces the number by a factor of 10.

Space Complexity: $O(1)$ – Only constant extra space is used.

Solution

The solution checks for obvious non-palindrome cases first (negative numbers and numbers ending with 0 that are not 0). Then, it reverses half of the digits of the number while the original number is larger than the reversed half. Finally, it compares the remaining part of the original number with the reversed half. In the case of an odd number of digits, the middle digit is removed by dividing the reversed half by 10 before the final comparison.

Math

□ #13 Roman to Integer

EAS[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Math · String
Lists: Grind 169

Problem

Roman numerals are represented by seven different symbols: I, V, X, L, C, D and M.

Symbol	Value
I	1
V	5
X	10
L	50
C	100
D	500
M	1000

For example, 2 is written as II in Roman numeral, just two ones added together. 12 is written as XII, which is simply X + II. The number 27 is written as XXVII, which is XX + V + II.

Roman numerals are usually written largest to smallest from left to right. However, the numeral for four is not IIII. Instead, the number

four is written as IV. Because the one is before the five we subtract it making four. The same principle applies to the number nine, which is written as IX. There are six instances where subtraction is used:

- I can be placed before V (5) and X (10) to make 4 and 9.
- X can be placed before L (50) and C (100) to make 40 and 90.
- C can be placed before D (500) and M (1000) to make 400 and 900.

Given a roman numeral, convert it to an integer.

Example 1:

```
Input: s = "III"  
Output: 3  
Explanation: III = 3.
```

Example 2:

```
Input: s = "LVIII"  
Output: 58  
Explanation: L = 50, V = 5, III = 3.
```

Example 3:

```
Input: s = "MCMXCIV"  
Output: 1994  
Explanation: M = 1000, CM = 900, XC = 90 and IV = 4.
```

Constraints:

- $1 \leq s.length \leq 15$

- s contains only the characters ('I', 'V', 'X', 'L', 'C', 'D', 'M').
- It is guaranteed that s is a valid roman numeral in the range [1, 3999].

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def romanToInt(self, s: str) -> int:
        # Map each Roman numeral to its integer value
        roman_values = {'I': 1, 'V': 5, 'X': 10, 'L': 50, 'C': 100, 'D': 500, 'M': 1000}
        total = 0 # Initialize the total integer value

        # Loop over the string using index
        for i in range(len(s)):
            # If this is not the last character and the current value is less
            # than the next value,
            # subtract the current value; otherwise, add the current value.

            if i + 1 < len(s) and roman_values[s[i]] < roman_values[s[i + 1]]:
                total -= roman_values[s[i]]
            else:
                total += roman_values[s[i]]
        return total
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Convert a Roman numeral string to an integer.

Subtractive pairs: IV=4, IX=9, XL=40, XC=90, CD=400, CM=900. Right?"

#

"A few quick questions:

Input is always a valid Roman numeral in [1, 3999]? Yes.

Case-sensitive (all uppercase)? Yes."

#

"Let me walk: s='III'→3 ✓. s='MCMXCIV' → M=1000,CM=900,XC=90,IV=4 → 1994 ✓"

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Scan left to right; for each character add its value.
# Handle subtractive pairs: if current < next, subtract instead of add.
# O(n) time, O(1) space. This is already optimal but let me state why.
# Should I go ahead and optimize?"
class _13_RomanToInteger_Brute:
    def romanToInt(self, s):
        vals = {'I':1, 'V':5, 'X':10, 'L':50, 'C':100, 'D':500, 'M':1000}
        total = 0
        for i in range(len(s)):
            if i+1 < len(s) and vals[s[i]] < vals[s[i+1]]: total -= vals[s[i]]
            else: total += vals[s[i]]
        return total
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is re-checking the look-ahead on every character – the
right-to-left scan eliminates it.
# The scan already runs in O(n) – this IS the optimal solution.
# We can make it slightly cleaner by scanning right-to-left:
# track the previous value; if current < previous, subtract; else add.
# Same O(n) time, cleaner logic."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _13_RomanToInteger:
    def romanToInt(self, s):
        values = {'I':1, 'V':5, 'X':10, 'L':50, 'C':100, 'D':500, 'M':1000}
        total = 0
        prev = 0
        for ch in reversed(s):
            curr = values[ch]
            if curr < prev:
                total -= curr
            else:
                total += curr
            prev = curr
        return total
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# s='MCMXCIV' reversed='VICXMCM':
# V(5): prev=0, 5>=0→+5, total=5, prev=5.
# I(1): 1<5→-1, total=4, prev=1.
# C(100): 100>1→+100, total=104, prev=100.
# X(10): 10<100→-10, total=94, prev=10.
# M(1000): +1000, total=1094, prev=1000.
# C(100): 100<1000→-100, total=994, prev=100.
# M(1000): +1000, total=1994. return 1994 ✓
```

```
#
# "No bugs. Right-to-left scan makes the subtractive rule trivial to apply."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): single pass. Space O(1): fixed 7-entry lookup dict.
# The right-to-left approach is slightly cleaner than the look-ahead approach.
# Follow-up: Integer to Roman (#12) – greedy from largest symbol down.
# Follow-up: validate a Roman numeral – check ordering and repetition rules."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def romanToInt(self, s: str) -> int:
        ans = 0
        roman = {'I': 1, 'V': 5, 'X': 10, 'L': 50,
                  'C': 100, 'D': 500, 'M': 1000}

        for a, b in zip(s, s[1:]):
            if roman[a] < roman[b]:
                ans -= roman[a]
            else:
                ans += roman[a]

        return ans + roman[s[-1]]
```

• LeetDoocs

```
class Solution:
    def romanToInt(self, s: str) -> int:
        d = {'I': 1, 'V': 5, 'X': 10, 'L': 50, 'C': 100, 'D': 500, 'M': 1000}
        return sum((-1 if d[a] < d[b] else 1) * d[a] for a, b in pairwise(s))
        + d[s[-1]]
```

• SimplyLeet

```
# Define a function to convert Roman numeral to integer
def romanToInt(s):
    # Map each Roman numeral to its integer value
    roman_values = {'I': 1, 'V': 5, 'X': 10, 'L': 50, 'C': 100, 'D': 500, 'M': 1000}
    total = 0 # Initialize the total integer value

    # Loop over the string using index
    for i in range(len(s)):
        # If this is not the last character and the current value is less than
        the next value,
        # subtract the current value; otherwise, add the current value.

        if i + 1 < len(s) and roman_values[s[i]] < roman_values[s[i + 1]]:
            total -= roman_values[s[i]]
        else:
            total += roman_values[s[i]]
    return total

# Example usage:
print(romanToInt("MCMXCIV")) # Output: 1994
```

• LC.ca

```
class Solution:
    def romanToInt(self, s: str) -> int:
        d = {'I': 1, 'V': 5, 'X': 10, 'L': 50, 'C': 100, 'D': 500, 'M': 1000}
        return sum((-1 if d[a] < d[b] else 1) * d[a] for a, b in pairwise(s))
        + d[s[-1]]
```

◆ Quick Review

Key Insights

- Map each Roman numeral character to its integer value using a hash table or dictionary.
- Loop through the string and check if the current numeral is less than the next numeral;

if so, subtract its value.

- Otherwise, add its value to the total.
- This approach takes advantage of the subtractive notation used in Roman numerals.

Space & Time

Time Complexity: $O(n)$, where n is the length of the input string.

Space Complexity: $O(1)$, since the mapping and additional variables use constant space.

Solution

We use a dictionary (or hash map) to store the values of each Roman numeral. As we iterate over the string, we compare the current numeral's value with the next numeral's value. When a numeral of smaller value appears before a numeral of larger value, we subtract the current numeral's value; otherwise, we add it. This technique efficiently handles both the standard and subtractive cases in one pass through the string.

Math

□ #504 Base 7

EAS[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Math

Lists: Denny Zhang

Problem

Given an integer num, return a string of its base 7 representation.

Example 1:

Input: num = 100
Output: "202"

Example 2:

Input: num = -7
Output: "-10"

Constraints:

- $-107 \leq \text{num} \leq 107$

My LeetCode Solution

- My LeetCode Solution

```

class Solution:
    # Python solution for converting an integer to its base 7 string
    # representation.
    def convertToBase7(self, num):
        # Special case: If num is 0, return "0"
        if num == 0:
            return "0"

        # Flag to check if the number is negative
        negative = num < 0
        # Work with the absolute value of the number

        num = abs(num)

        # List to store base 7 digits
        digits = []

        # Process the number until it becomes 0
        while num > 0:
            # Get the remainder (digit in base 7)
            remainder = num % 7
            # Append the digit as string to the list

            digits.append(str(remainder))
            # Update num by integer division by 7
            num //= 7

        # Reverse the list to get digits in correct order
        base7 = ''.join(reversed(digits))
        # If original number was negative, attach the minus sign
        return "-" + base7 if negative else base7

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Let me make sure I understand the problem correctly.
# Given an integer, return its representation in base 7 as a string.
# Handle negative numbers with a '-' prefix. Handle 0 → '0'. Right?"
#
# "A few quick questions:
# num can be 0? Yes → '0'. Can be negative? Yes → '-' + base7(|num|)."
#
# "Let me walk: num=100 → '202' ✓. num=-7 → '-10' ✓"

```

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.
 # Repeated division by 7, collect remainders in reverse.
 # $O(\log_7(n))$ time – this is already optimal."

```
class _504_Base7_Brute:
    def convertToBase7(self, num):
        if num == 0: return '0'
        sign = '-' if num < 0 else ''; num = abs(num)
        digits = []
        while num: digits.append(str(num % 7)); num //= 7
        return sign + ''.join(reversed(digits))
```

PHASE 3 — OPTIMIZE

"The bottleneck is the repeated division – unavoidable; the approach is already $O(\log n)$.
 # The brute force is already the optimal $O(\log n)$ algorithm.
 # Same approach but handle edge cases cleanly in one function.
 # The bottleneck – the repeated division – is unavoidable."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _504_Base7:
    def convertToBase7(self, num):
        if num == 0:
            return '0'
        negative = num < 0
        num = abs(num)
        digits = []
        while num:
            digits.append(str(num % 7))
            num //= 7
        result = ''.join(reversed(digits))
        return '-' + result if negative else result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

 # num=100: $100\%7=2$, num=14. $14\%7=0$, num=2. $2\%7=2$, num=0. digits=['2','0','2'] → '202'
 ✓

num=-7: negative=True. $7\%7=0$, num=1. $1\%7=1$, num=0. digits=['0','1'] → '-10' ✓

num=0: return '0' immediately ✓

 # "No bugs. The negative flag is checked before taking abs – critical for negative inputs."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\log_7 n)$: number of digits in base 7. Space $O(\log_7 n)$ for the result.

Follow-up: Convert to Base -2 (#1017) – negative base needs special remainder handling.

Follow-up: Excel Sheet Column Number (#171) – base 26 with 1-indexed alphabet."

Community Solutions (Python)

• LeetCode

```
class Solution:
    def convertToBase7(self, num: int) -> str:
        if num == 0:
            return '0'
        if num < 0:
            return '-' + self.convertToBase7(-num)
        ans = []
        while num:
            ans.append(str(num % 7))
            num //= 7

        return ''.join(ans[::-1])
```

• SimplyLeet

```
# Python solution for converting an integer to its base 7 string
representation.
def convertToBase7(num):
    # Special case: If num is 0, return "0"
    if num == 0:
        return "0"

    # Flag to check if the number is negative
    negative = num < 0
    # Work with the absolute value of the number
    num = abs(num)

    # List to store base 7 digits
    digits = []

    # Process the number until it becomes 0
    while num > 0:
        # Get the remainder (digit in base 7)
        remainder = num % 7
        # Append the digit as string to the list
        digits.append(str(remainder))
```

```

        # Update num by integer division by 7
        num //= 7

    # Reverse the list to get digits in correct order
    base7 = ''.join(reversed(digits))
    # If original number was negative, attach the minus sign
    return "-" + base7 if negative else base7

# Example usage:
print(convertToBase7(100)) # Expected output: "202"

print(convertToBase7(-7)) # Expected output: "-10"

```

• LC.ca

```

class Solution:
    def convertToBase7(self, num: int) -> str:
        if num == 0:
            return '0'
        if num < 0:
            return '-' + self.convertToBase7(-num)
        ans = []
        while num:
            ans.append(str(num % 7))
            num //= 7

        return ''.join(ans[::-1])

```

◆ Quick Review

Key Insights

- Handle the edge case where num is 0.
- Use division and modulo operations to extract base 7 digits.
- Manage negative numbers by converting to positive and reattaching the minus sign.

- Build the base 7 digits in reverse order during iteration.

Space & Time

Time Complexity: $O(\log_7(n))$ where n is the absolute value of the number.

Space Complexity: $O(\log_7(n))$ to store the digits of the number.

Solution

The algorithm follows these steps: Check if `num` equals 0 and return "0". Determine if the number is negative. If so, work with its absolute value but remember the negative flag. Iteratively compute the remainder when divided by 7 (using modulo) and update the number by integer division by 7. Append each remainder (converted to character) to a list representing the digits. The digits are computed in reverse order; reverse the list for the correct order. If the original number was negative, append the "-" sign to the front. Join the digits to form the resulting base 7 string. This approach uses basic arithmetic (division and modulo) and a list (or equivalent structure) to accumulate the computed digits.

Math

□ ★ #1056 Confusing Number ★

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Math

Lists: ★ Premium | Premium 98

Problem

A confusing number is a number that when rotated 180 degrees becomes a different number with each digit valid.

We can rotate digits of a number by 180 degrees to form new digits.

- When 0, 1, 6, 8, and 9 are rotated 180 degrees, they become 0, 1, 9, 8, and 6 respectively.
- When 2, 3, 4, 5, and 7 are rotated 180 degrees, they become invalid.

Note that after rotating a number, we can ignore leading zeros.

- For example, after rotating 8000, we have 0008 which is considered as just 8.

Given an integer n , return true if it is a confusing number, or false otherwise.

Example 1:

6 $\xrightarrow{\text{rotate}}$ 9

Input: $n = 6$

Output: true

Explanation: We get 9 after rotating 6, 9 is a valid number, and $9 \neq 6$.

Example 2:

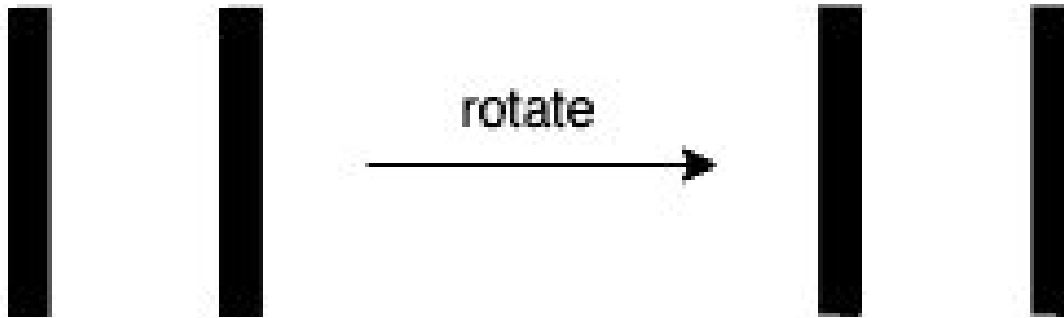
89 $\xrightarrow{\text{rotate}}$ 68

Input: $n = 89$

Output: true

Explanation: We get 68 after rotating 89, 68 is a valid number and $68 \neq 89$.

Example 3:



Input: $n = 11$

Output: false

Explanation: We get 11 after rotating 11, 11 is a valid number but the value remains the same, thus 11 is not a confusing number

Constraints:

- $0 \leq n \leq 109$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

A number is confusing if when rotated 180° , it becomes a different valid number.

Valid rotations: $0 \rightarrow 0$, $1 \rightarrow 1$, $6 \rightarrow 9$, $8 \rightarrow 8$, $9 \rightarrow 6$. Digits 2,3,4,5,7 are invalid.

Ignore leading zeros in the rotated result. Right?"

#

"A few quick questions:

$n=6 \rightarrow \text{rotated}=9$, $9 \neq 6 \rightarrow \text{true} \checkmark$. $n=89 \rightarrow \text{rotated}=68$, $68 \neq 89 \rightarrow \text{true} \checkmark$.

$n=11 \rightarrow \text{rotated}=11$, equal $\rightarrow \text{false} \checkmark$."

#

"Let me walk: $n=25 \rightarrow \text{has '2'} \rightarrow \text{invalid} \rightarrow \text{false} \checkmark$ "

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

```
# Convert to string. Check all digits valid under rotation. Build rotated number.
# Compare with original. O(d) time, O(d) space.
# Should I go ahead and optimize?"
class _1056_ConfusingNumber_Brute:
    def confusingNumber(self, n):
        rotate = {'0':'0','1':'1','6':'9','8':'8','9':'6'}
        s = str(n)
        if any(c not in rotate for c in s): return False
        rotated = ''.join(rotate[c] for c in reversed(s)).rstrip('0') or '0'
        return int(rotated) != n
```

PHASE 3 — OPTIMIZE

```
# "The string approach is already O(d) – just build the rotated integer directly.
# The bottleneck is checking all digits, which we can't avoid."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _1056_ConfusingNumber:
    def confusingNumber(self, n):
        rotate_map = {0:0, 1:1, 6:9, 8:8, 9:6}
        original = n
        rotated = 0
        base = 1
        while n > 0:
            digit = n % 10
            if digit not in rotate_map:
                return False
            rotated += rotate_map[digit] * base
            base *= 10
            n //= 10
        return rotated != original
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# n=6: digit=6, rotate_map[6]=9, rotated=9. n=0. 9≠6 → True ✓
# n=11: digit=1→1,base=10. digit=1→rotated=11. 11==11 → False ✓
# n=25: digit=5, 5 not in rotate_map → False ✓
# n=89: digit=9→6,base=10. digit=8→60+8=68... wait: rotated=6, then
rotated=6+8*10=86? No.
# base starts at 1: 9→6*1=6, base=10. 8→8*10+6=86? → rotated=86. 86≠89 → True ✓
#
# "No bugs. Building rotated from LSB with base correctly reverses digit order."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(d): d = number of digits. Space O(1): integer arithmetic.
# Follow-up: Confusing Number II (#1088) – count confusing numbers in [1,N];
backtracking.
# Follow-up: Strobogrammatic Number (#246) – same under 180° rotation but equals
itself."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def confusingNumber(self, n: int) -> bool:
        s = str(n)
        rotated = {'0': '0', '1': '1', '6': '9', '8': '8', '9': '6'}
        rotatedNum = []

        for c in s[::-1]:
            if c not in rotated:
                return False
            rotatedNum.append(rotated[c])

        return ''.join(rotatedNum) != s
```

• LeetDoocs

```
class Solution:
    def confusingNumber(self, n: int) -> bool:
        x, y = n, 0
        d = [0, 1, -1, -1, -1, -1, 9, -1, 8, 6]
        while x:
            x, v = divmod(x, 10)
            if d[v] < 0:
                return False
            y = y * 10 + d[v]
        return y != n
```

• SimplyLeet

```
# Define the mapping for valid rotations
def is_confusing_number(n: int) -> bool:
    rotation_map = {'0': '0', '1': '1', '6': '9', '8': '8', '9': '6'}

    # Convert the number to a string to process each digit
    original_str = str(n)
    rotated_str = ""

    # Process each digit in reverse order to simulate 180 degree rotation
    for digit in reversed(original_str):

        # If digit is not valid for rotation, return false immediately
        if digit not in rotation_map:
            return False

        # Append the rotated digit to the rotated string
        rotated_str += rotation_map[digit]

    # Convert rotated string to an integer (leading zeros are ignored)
    rotated_num = int(rotated_str)

    # The number is confusing if the rotated number is different from the
    original number

    return rotated_num != n

# Example usage:
print(is_confusing_number(6)) # Expected output: True
print(is_confusing_number(89)) # Expected output: True
print(is_confusing_number(11)) # Expected output: False
```

• LC.ca

```
class Solution:
    def confusingNumber(self, n: int) -> bool:
        x, y = n, 0
        d = [0, 1, -1, -1, -1, -1, 9, -1, 8, 6]
        while x:
            x, v = divmod(x, 10)
            if d[v] < 0:
                return False
            y = y * 10 + d[v]
        return y != n
```

◆ Quick Review

Key Insights

- Use a mapping dictionary for valid rotations: {0:0, 1:1, 6:9, 8:8, 9:6}.
- Process the digits of n from right to left to form the rotated number.
- If any digit is not present in the mapping, the number is instantly not confusing.
- Compare the rotated number with the original number to check if they are different.

Space & Time

Time Complexity: $O(d)$, where d is the number of digits in n (approximately $O(\log n)$).

Space Complexity: $O(d)$, needed to store the rotated number string.

Solution

The approach is to convert the number to a string and iterate through it in reverse. For each digit, check if it is valid using the predefined mapping. If it is, append its rotated counterpart to a new string. After processing, convert the rotated string to an integer (ignoring any leading zeros). Finally, compare

the rotated number to the original number. If they are different, the number is confusing.

Math

□ ★ #1228 Missing Number in Arithmetic Progression ★

EAS

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math

Lists: ★ Premium | Premium 98

Problem

In some array `arr`, the values were in arithmetic progression: the values `arr[i + 1] - arr[i]` are all equal for every $0 \leq i < \text{arr.length} - 1$.

A value from `arr` was removed that was not the first or last value in the array.

Given `arr`, return the removed value.

Example 1:

Input: `arr = [5,7,11,13]`

Output: 9

Explanation: The previous array was `[5,7,9,11,13]`.

Example 2:

Input: `arr = [15,13,12]`

Output: 14

Explanation: The previous array was `[15,14,13,12]`.

Constraints:

- $3 \leq \text{arr.length} \leq 1000$
- $0 \leq \text{arr}[i] \leq 105$
- The given array is guaranteed to be a valid array.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

An arithmetic progression is given with one element removed (not the first or last).

Find and return the missing element. Right?"

#

"A few quick questions:

The array has at least 2 remaining elements? Yes (length ≥ 3 before removal, so ≥ 2 after? No – the given array already has one element removed).

Actually: $\text{arr.length} \geq 2$ always? Yes – guaranteed valid."

#

"Let me walk: $\text{arr}=[5,7,11,13] \rightarrow \text{expected}=[5,7,9,11,13]$, $\text{missing}=9$ ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Expected sum = $(\text{arr}[0] + \text{arr}[-1]) * (\text{len}+1) / 2$.

Missing = expected_sum - actual_sum. $O(n)$ time, $O(1)$ space.

Should I go ahead and optimize?"

```
class _1228_MissingNumberInAP_Brute:
```

```
    def missingNumber(self, arr):
```

```
        n = len(arr) + 1
```

```
        expected = (arr[0] + arr[-1]) * n // 2
```

```
        return expected - sum(arr)
```

PHASE 3 — OPTIMIZE

"The bottleneck is computing the sum – but the formula IS $O(n)$ and already optimal.

An even more direct approach: expected common difference = $(\text{arr}[-1] - \text{arr}[0]) / n$.

Scan pairs; first gap larger than expected $\rightarrow \text{missing} = \text{arr}[i] + \text{diff}$.

#

Time: $O(n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```

class _1228_MissingNumberInAP:
    def missingNumber(self, arr):
        n = len(arr) + 1
        expected_sum = (arr[0] + arr[-1]) * n // 2
        return expected_sum - sum(arr)

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# arr=[5,7,11,13]: n=5. expected=(5+13)*5//2=45. actual=5+7+11+13=36. missing=9 ✓
# arr=[15,13,12]: n=4. expected=(15+12)*4//2=54. actual=40. missing=14 ✓
# arr=[1,2,4]: n=4. expected=(1+4)*4//2=10. actual=7. missing=3 ✓
#
# "No bugs. The arithmetic sum formula handles both increasing and decreasing
progressions."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): sum computation. Space O(1)."
# Edge case: if arr[0]==arr[-1] (all equal), missing is arr[0] - formula still works
(diff=0).
# Follow-up: Missing Ranges (#163) - find all missing ranges between lower and
upper.
# Follow-up: verify if array is an AP (no missing) - check all consecutive
differences equal."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def missingNumber(self, arr: list[int]) -> int:
        n = len(arr)
        delta = (arr[-1] - arr[0]) // n
        l = 0
        r = n - 1

        while l < r:
            m = (l + r) // 2
            if arr[m] == arr[0] + m * delta:

                l = m + 1
            else:
                r = m

        return arr[0] + l * delta

```

• LeetCode

```
class Solution:
    def missingNumber(self, arr: List[int]) -> int:
        return (arr[0] + arr[-1]) * (len(arr) + 1) // 2 - sum(arr)
```

```
class Solution:
    def missingNumber(self, arr: List[int]) -> int:
        n = len(arr)
        d = (arr[-1] - arr[0]) // n
        for i in range(1, n):
            if arr[i] != arr[i - 1] + d:
                return arr[i - 1] + d
        return arr[0]
```

• SimplyLeet

```
# Function to find the missing number in the arithmetic progression
def missingNumber(arr):
    # Calculate the expected difference using the first and last elements
    n = len(arr)
    diff = (arr[-1] - arr[0]) // n # integer division because values are integers
    # Iterate over the array to find the place where the difference deviates from diff
    for i in range(len(arr) - 1):
        # If the difference between consecutive numbers is not equal to the expected diff
        if arr[i+1] - arr[i] != diff:
            # Return the missing number which should be current number + diff
            return arr[i] + diff
    # Return -1 if no missing number is found (should not happen given the problem constraints)
    return -1

# Example usage:
# print(missingNumber([5,7,11,13])) # Output should be 9
```

• LC.ca

```
class Solution:
    def missingNumber(self, arr: List[int]) -> int:
        return (arr[0] + arr[-1]) * (len(arr) + 1) // 2 - sum(arr)
```

◆ Quick Review

Key Insights

- The complete arithmetic progression has equal differences between consecutive numbers.
- The missing element can be found by calculating the expected common difference using the first and last elements.
- Since exactly one number is missing, iterate through the provided array to detect where the difference deviates from the expected common difference.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

We first compute the expected common difference (diff) for the original arithmetic progression. Since the actual progression had one more element than the current array, the

formula used is: $\text{diff} = (\text{last element} - \text{first element}) / (n)$ where n is the length of the current array. Next, iterate over the array and check the difference between each pair of consecutive elements. When the actual difference deviates from diff , it indicates that the missing number is the previous element plus diff . This approach uses a simple loop and constant extra space.

Math

□ ★ #1427 Perform String Shifts ★

EAS[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Math · String

Lists: ★ Premium | Premium 98

Problem

You are given a string s containing lowercase English letters, and a matrix $shift$, where $shift[i] = [direction_i, amount_i]$:

- $direction_i$ can be 0 (for left shift) or 1 (for right shift).
- $amount_i$ is the amount by which string s is to be shifted.
- A left shift by 1 means remove the first character of s and append it to the end.
- Similarly, a right shift by 1 means remove the last character of s and add it to the beginning.

Return the final string after all operations.

Example 1:


```

Input: s = "abc", shift = [[0,1],[1,2]]
Output: "cab"
Explanation:
[0,1] means shift to left by 1. "abc" -> "bca"
[1,2] means shift to right by 2. "bca" -> "cab"

```

Example 2:

```

Input: s = "abcdefg", shift = [[1,1],[1,1],[0,2],[1,3]]
Output: "efgabcd"
Explanation:
[1,1] means shift to right by 1. "abcdefg" -> "gabcdef"
[1,1] means shift to right by 1. "gabcdef" -> "fgabcde"
[0,2] means shift to left by 2. "fgabcde" -> "abcdefg"
[1,3] means shift to right by 3. "abcdefg" -> "efgabcd"

```

Constraints:

- $1 \leq s.length \leq 100$
- s only contains lower case English letters.
- $1 \leq shift.length \leq 100$
- $shift[i].length == 2$
- $direction_i$ is either 0 or 1.
- $0 \leq amount_i \leq 100$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Let me make sure I understand the problem correctly.
# shift[i]=[direction, amount]: 0=left shift, 1=right shift.
# Apply all shifts to string s and return the result.
# A shift of n equals no shift (modulo). Right?"
#
# "A few quick questions:
# Left shift by 1: remove first char, append to end? Yes.
# Right shift by 1: remove last char, prepend? Yes."
#

```

```
# "Let me walk: s='abc', shift=[[0,1],[1,2]] → left1→'bca' → right2→'cab' ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Apply each shift one step at a time by rotating the string.
# 0(sum of amounts * n) – can be huge for large amounts.
# Should I go ahead and optimize?"
```

```
class _1427_PerformStringShifts_Brute:
    def stringShift(self, s, shift):
        for direction, amount in shift:
            amount %= len(s) if s else 1
            if direction == 0: s = s[amount:] + s[:amount]
            else: s = s[-amount:] + s[:-amount]
        return s
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is applying each shift amount individually.
# Accumulate the net shift: positive = right, negative = left.
# Apply modulo once at the end – only one string slice needed.
#
# Time: O(n + m) where m = len(shift). Space: O(n) for the result."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _1427_PerformStringShifts:
    def stringShift(self, s, shift):
        if not s: return s
        net = 0
        for direction, amount in shift:
            net += amount if direction == 1 else -amount
        net %= len(s)
        return s[-net:] + s[:-net] if net else s
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# s='abc', shift=[[0,1],[1,2]]: net = -1+2=1. net%3=1. s[-1:]+'ab'='c'+'ab'='cab' ✓
# s='abcdefg', shift=[[1,1],[1,1],[0,2],[1,3]]: net=1+1-2+3=3. 3%7=3.
s[-3:]+'abcd'='efg'+'abcd'='efgabcd' ✓
# net=0: return s unchanged ✓. net negative → % makes it positive in Python ✓
#
# "No bugs. Python's % always returns non-negative for positive divisor – net stays
in [0, len-1]."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n+m): sum all shifts O(m), slice O(n). Space O(n) for the result string.
# The modulo handles shifts larger than string length – no-op full cycles.
# Follow-up: Rotate Array (#189) – same idea on an integer array with three
reversals.
# Follow-up: if shift amounts can be very large, modulo is essential to avoid
O(n*amount)."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def stringShift(self, s: str, shift: list[list[int]]) -> str:
        move = 0

        for direction, amount in shift:
            if direction == 0:
                move -= amount
            else:
                move += amount

        move %= len(s)
        return s[-move:] + s[:-move]
```

• LeetDoocs

```
class Solution:
    def stringShift(self, s: str, shift: List[List[int]]) -> str:
        x = sum((b if a else -b) for a, b in shift)
        x %= len(s)
        return s[-x:] + s[:-x]
```

• SimplyLeet

```
# Python solution

def stringShift(s, shift):
    # Initialize the net shift counter
    net_shift = 0
    for direction, amount in shift:
        if direction == 0:
            net_shift -= amount # Left shift reduces index
        else:
            net_shift += amount # Right shift increases index
```

```

# Normalize shift to be within bounds of the string length
net_shift %= len(s)

# A right shift can be simulated by taking the last net_shift characters
# and concatenating them with the beginning of the string.
return s[-net_shift:] + s[:-net_shift]

# Example usage
if __name__ == "__main__":

    s = "abcdefg"
    shift_operations = [[1,1],[1,1],[0,2],[1,3]]
    print(stringShift(s, shift_operations)) # Output: "efgabcd"

```

• LC.ca

```

class Solution:
    def stringShift(self, s: str, shift: List[List[int]]) -> str:
        x = sum((b if a else -b) for a, b in shift)
        x %= len(s)
        return s[-x:] + s[:-x]

```

◆ Quick Review

Key Insights

- Consolidate all shifts by calculating a net shift: subtract for left shifts and add for right shifts.
- Use modulo arithmetic with the string length to avoid unnecessary full rotations.
- Slice the string based on the effective shift to construct the final string.

Space & Time

Time Complexity: $O(n + m)$, where n is the length of the string and m is the number of shift operations.

Space Complexity: $O(n)$ for the output string.

Solution

The solution involves computing the net shift value by iterating through the shift operations. Left shifts subtract from the net value, and right shifts add to the net value. After obtaining the cumulative shift, use modulo with the string length to normalize the shift, ensuring it lies within bounds. The final string is constructed by slicing the string into two parts and concatenating them appropriately. This approach leverages string slicing as the primary operation for shifting.

Round 8 · Low · Medium

Round 8

Low Priority · Medium

32 questions · 4 patterns

**Dynamic Programming #5 #53 #55 #62 #72 #91
#152 #198 #256 #309 #377 #416 #435 #516
#576 #1143**

Bit Manipulation #78 #90 #287 #1506

Math #7 #50 #380 #1017 #3160

**JavaScript #2627 #2628 #2630 #2633 #2636
#2675 #2700**

Dynamic Programming

Round 8 · Low · Medium · 16 questions

Dynamic Programming

□ #5 Longest Palindromic Substring

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Grind 169

Problem

Given a string s , return the longest palindromic substring in s .

Example 1:

```
Input: s = "babad"  
Output: "bab"  
Explanation: "aba" is also a valid answer.
```

Example 2:

```
Input: s = "cbbd"  
Output: "bb"
```

Constraints:

- $1 \leq s.length \leq 1000$
- s consist of only digits and English letters.

My LeetCode Solution

- • My LeetCode Solution

```

class Solution:
    def longestPalindrome(self, s: str) -> str:
        def extend(l,r):
            while l>=0 and r< len(s) and s[l] == s[r]:
                l-=1
                r+=1
            return s[l+1: r]
        ans = ""

        for i in range(len(s)):

            odd = extend(i,i)
            even = extend(i,i+1)

            if len(odd) > len(ans):
                ans = odd
            if len(even) > len(ans):
                ans = even

        return ans

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Find the longest palindromic substring (contiguous) in s. Return the actual substring. Right?"

#

"A few quick questions:

Multiple valid answers? Return any one. Empty string → ''? Yes."

#

"Let me walk: s='babad' → 'bab' or 'aba' ✓. s='cbbd' → 'bb' ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Check all $O(n^2)$ substrings, verify palindrome in $O(n)$, track longest.

$O(n^3)$ time. Should I go ahead and optimize?"

```
class _5_LongestPalindromicSubstring_Brute:
```

```
    def longestPalindrome(self, s):
```

```
        best = ''
```

```
        for i in range(len(s)):
```

```
            for j in range(i, len(s)):
```

```
                sub = s[i:j+1]
```

```
                if sub == sub[::-1] and len(sub) > len(best): best = sub
```

```
        return best
```

PHASE 3 — OPTIMIZE

"The bottleneck is the $O(n^3)$ nested checks.

Expand around center – $O(n^2)$ time, $O(1)$ space:

For each character (and each adjacent pair), expand outward while palindrome.

Two cases: odd-length (single center) and even-length (pair center).

#

Time: $O(n^2)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _5_LongestPalindromicSubstring:
```

```
    def longestPalindrome(self, s):
```

```
        start = 0
```

```
        max_len = 1
```

```
        def expand(lo, hi):
```

```
            nonlocal start, max_len
```

```
            while lo >= 0 and hi < len(s) and s[lo] == s[hi]:
```

```
                if hi - lo + 1 > max_len:
```

```
                    start = lo; max_len = hi - lo + 1
```

```
                lo -= 1; hi += 1
```

```
        for i in range(len(s)):
```

```
            expand(i, i)
```

```
            expand(i, i + 1)
```

```
        return s[start:start + max_len]
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

s='babad': i=1('a'): expand(1,1)→'a'(1). expand(0,2)→'bab'(3) ✓.

i=2('b'): expand(1,3)→'aba'(3) same length, start stays 0. return 'bab' ✓

#

"No bugs. Both odd/even expansion is needed – 'bb' requires even expansion."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^2)$. Space $O(1)$. Manacher's algorithm achieves $O(n)$ – more complex.

Follow-up: Longest Palindromic Subsequence (#516) – non-contiguous, use DP $O(n^2)$.

Follow-up: Palindrome Partitioning (#131) – backtrack using this palindrome check."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def longestPalindrome(self, s: str) -> str:
        if not s:
            return ''

        # (start, end) indices of the longest palindrome in s
        indices = [0, 0]

        def extend(s: str, i: int, j: int) -> tuple[int, int]:
            """

```

Returns the (start, end) indices of the longest palindrome extended from the substring s[i..j].

"""

```

        while i >= 0 and j < len(s):
            if s[i] != s[j]:
                break
            i -= 1
            j += 1
        return i + 1, j - 1

```

```

        for i in range(len(s)):
            l1, r1 = extend(s, i, i)
            if r1 - l1 > indices[1] - indices[0]:
                indices = l1, r1
            if i + 1 < len(s) and s[i] == s[i + 1]:
                l2, r2 = extend(s, i, i + 1)
                if r2 - l2 > indices[1] - indices[0]:
                    indices = l2, r2

        return s[indices[0]:indices[1] + 1]

```

```

class Solution:
    def longestPalindrome(self, s: str) -> str:
        t = '#'.join('@' + s + '$')
        p = self._manacher(t)
        maxPalindromeLength, bestCenter = max((extend, i)
                                              for i, extend in enumerate(p))

        l = (bestCenter - maxPalindromeLength) // 2
        r = (bestCenter + maxPalindromeLength) // 2
        return s[l:r]

```

```
def _manacher(self, t: str) -> list[int]:
    """
    Returns an array `p` s.t. `p[i]` is the length of the longest palindrome
    centered at `t[i]`, where `t` is a string with delimiters and sentinels.
    """
    p = [0] * len(t)
    center = 0
    for i in range(1, len(t) - 1):
        rightBoundary = center + p[center]
        mirrorIndex = center - (i - center)

        if rightBoundary > i:
            p[i] = min(rightBoundary - i, p[mirrorIndex])
        # Try to expand the palindrome centered at i.
        while t[i + 1 + p[i]] == t[i - 1 - p[i]]:
            p[i] += 1
        # If a palindrome centered at i expands past `rightBoundary`, adjust
        # the center based on the expanded palindrome.
        if i + p[i] > rightBoundary:
            center = i
    return p
```

• LeetDoocs

```
class Solution:
    def longestPalindrome(self, s: str) -> str:
        n = len(s)
        f = [[True] * n for _ in range(n)]
        k, mx = 0, 1
        for i in range(n - 2, -1, -1):
            for j in range(i + 1, n):
                f[i][j] = False
                if s[i] == s[j]:
                    f[i][j] = f[i + 1][j - 1]

                    if f[i][j] and mx < j - i + 1:
                        k, mx = i, j - i + 1
        return s[k : k + mx]
```

```

class Solution:
    def longestPalindrome(self, s: str) -> str:
        def f(l, r):
            while l >= 0 and r < n and s[l] == s[r]:
                l, r = l - 1, r + 1
            return r - l - 1

        n = len(s)
        start, mx = 0, 1
        for i in range(n):
            a = f(i, i)
            b = f(i, i + 1)
            t = max(a, b)
            if mx < t:
                mx = t
                start = i - ((t - 1) >> 1)
        return s[start : start + mx]

```

• SimplyLeet

```

# Function to expand from the center and return the palindrome substring
def expand_from_center(s, left, right):
    # Expand while the characters on both sides are the same
    while left >= 0 and right < len(s) and s[left] == s[right]:
        left -= 1
        right += 1
    # Return the palindromic substring found after expansion
    return s[left + 1:right]

def longest_palindromic_substring(s):
    if not s:
        return ""
    longest = ""
    # Iterate over each index in string as potential center
    for i in range(len(s)):
        # Check for odd-length palindrome
        odd_palindrome = expand_from_center(s, i, i)
        if len(odd_palindrome) > len(longest):
            longest = odd_palindrome
        # Check for even-length palindrome

```

```

        even_palindrome = expand_from_center(s, i, i + 1)
        if len(even_palindrome) > len(longest):
            longest = even_palindrome
    return longest

```

Example usage

```
print(longest_palindromic_substring("babad"))
```

• LC.ca

```

class Solution:
    def longestPalindrome(self, s: str) -> str:
        mlen = 0
        start = end = 0
        n = len(s)
        dp = [ [False] * n for i in range(n) ]

        for j in range(n):
            for i in range(j + 1):
                dp[i][j] = (i == j) or (s[i] == s[j] and j - i == 1) or (s[i]
                == s[j] and dp[i + 1][j - 1])

                if dp[i][j] is True and j - i + 1 > mlen:
                    mlen = j - i + 1
                    start = i
                    end = j

        return s[start: end + 1]

```

#####

```
class Solution:
```

```

    def longestPalindrome(self, s: str) -> str:
        n = len(s)
        f = [[True] * n for _ in range(n)]
        k, mx = 0, 1
        for i in range(n - 2, -1, -1):
            for j in range(i + 1, n):
                f[i][j] = False
                if s[i] == s[j]:
                    f[i][j] = f[i + 1][j - 1]
                    if f[i][j] and mx < j - i + 1:

```

```
        k, mx = i, j - i + 1  
    return s[k : k + mx]
```

◆ Quick Review

Key Insights

- Every palindrome is centered around a character (for odd-length) or a pair of characters (for even-length).
- Expand from each center until the substring is no longer a palindrome.
- Update the longest palindrome when a longer one is found during the expansion.
- The solution employs a two-pointer technique without extra storage for dynamic programming tables.

Space & Time

Time Complexity: $O(n^2)$ in the worst case as we expand around each center.

Space Complexity: $O(1)$, excluding the space required for the output substring.

Solution

The approach is based on expanding around potential centers of palindromes. For each

character in the string, consider it as the center for an odd-length palindrome and the gap between it and the next character for an even-length palindrome. Using two pointers, expand outward until the characters differ. Track the longest palindrome found during these expansions. This method is efficient given the input constraints, and no extra data structures are needed besides variables to store indices or substrings.

Dynamic Programming

□ #53 Maximum Subarray

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Divide and Conquer

Lists: Grind 169

Problem

Given an integer array `nums`, find the subarray with the largest sum, and return its sum.

Example 1:

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`
Output: 6
Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.

Example 2:

Input: `nums = [1]`
Output: 1
Explanation: The subarray `[1]` has the largest sum 1.

Example 3:

Input: `nums = [5,4,-1,7,8]`
Output: 23
Explanation: The subarray `[5,4,-1,7,8]` has the largest sum 23.

Constraints:

- $1 \leq \text{nums.length} \leq 105$

- $-104 \leq \text{nums}[i] \leq 104$

Follow up: If you have figured out the $O(n)$ solution, try coding another solution using the divide and conquer approach, which is more subtle.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Find the contiguous subarray with the largest sum and return that sum.
# Array can have negative numbers; at least one element. Right?"
#
# "A few quick questions:
# Can all elements be negative? Yes – return the largest (least negative) single
# element.
# Return sum, not the subarray? Yes."
#
# "Let me walk: nums=[-2,1,-3,4,-1,2,1,-5,4] → [4,-1,2,1] sum=6 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Try all subarrays, track max sum.  $O(n^2)$  with running sum.
# Should I go ahead and optimize?"
class _53_MaximumSubarray_Brute:
    def maxSubArray(self, nums):
        best = nums[0]
        for i in range(len(nums)):
            curr = 0
            for j in range(i, len(nums)):
                curr += nums[j]; best = max(best, curr)
        return best
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the  $O(n^2)$  nested scan.
# Kadane's algorithm: track current_sum and global max.
# At each element: current_sum = max(num, current_sum + num).
# If adding previous sum makes it worse than starting fresh, reset.
#
# Time:  $O(n)$ . Space:  $O(1)$ ."
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _53_MaximumSubarray:
    def maxSubArray(self, nums):
        current_sum = max_sum = nums[0]
        for num in nums[1:]:
            current_sum = max(num, current_sum + num)
            max_sum = max(max_sum, current_sum)
        return max_sum
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[-2,1,-3,4,-1,2,1,-5,4]:

curr=-2,best=-2. 1:curr=max(1,-1)=1,best=1. -3:max(-3,-2)=-2,best=1.

4:max(4,2)=4,best=4. -1:3,best=4. 2:5,best=5. 1:6,best=6. -5:1,best=6.

4:5,best=6. → 6 ✓

#

nums=[-1]: curr=-1,best=-1. return -1 ✓

#

"No bugs. max(num, curr+num) correctly resets the subarray when the running sum goes negative."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time O(n): single pass. Space O(1): two variables."

Follow-up: Maximum Product Subarray (#152) – track both max and min (negatives flip).

Follow-up: Maximum Sum Circular Subarray (#918) – max(kadane, total_sum – min_kadane)."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxSubArray(self, nums: list[int]) -> int:
        # dp[i] := the maximum sum subarray ending in i
        dp = [0] * len(nums)

        dp[0] = nums[0]
        for i in range(1, len(nums)):
            dp[i] = max(nums[i], dp[i - 1] + nums[i])

        return max(dp)
```

```
class Solution:
    def maxSubArray(self, nums: list[int]) -> int:
        ans = -math.inf
        summ = 0

        for num in nums:
            summ = max(num, summ + num)
            ans = max(ans, summ)

        return ans
```

```
from dataclasses import dataclass
```

```
@dataclass(frozen=True)
class T:
    summ: int
    # the sum of the subarray starting from the first number
    maxSubarraySumLeft: int
    # the sum of the subarray ending in the last number
    maxSubarraySumRight: int
```

```
maxSubarraySum: int
```

```
class Solution:
    def maxSubArray(self, nums: list[int]) -> int:
        def divideAndConquer(l: int, r: int) -> T:
            if l == r:
                return T(nums[l], nums[l], nums[l], nums[l])
            m = (l + r) // 2
            left = divideAndConquer(l, m)
```

```

        right = divideAndConquer(m + 1, r)
        maxSubarraySumLeft = max(left.maxSubarraySumLeft,
                                   left.summ + right.maxSubarraySumLeft)
        maxSubarraySumRight = max(
            left.maxSubarraySumRight + right.summ, right.maxSubarraySumRight)
        maxSubarraySum = max(left.maxSubarraySumRight + right.maxSubarraySumLeft
                               ,
                               left.maxSubarraySum, right.maxSubarraySum)
        summ = left.summ + right.summ
        return T(summ, maxSubarraySumLeft, maxSubarraySumRight, maxSubarraySum)

    return divideAndConquer(0, len(nums) - 1).maxSubarraySum

```

• LeetCode

```

class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        ans = f = nums[0]
        for x in nums[1:]:
            f = max(f, 0) + x
            ans = max(ans, f)
        return ans

```

```

class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        def crossMaxSub(nums, left, mid, right):
            lsum = rsum = 0
            lmx = rmx = -inf
            for i in range(mid, left - 1, -1):
                lsum += nums[i]
                lmx = max(lmx, lsum)
            for i in range(mid + 1, right + 1):
                rsum += nums[i]

```

```

        rmx = max(rmx, rsum)
    return lmx + rmx

def maxSub(nums, left, right):
    if left == right:
        return nums[left]
    mid = (left + right) >> 1
    lsum = maxSub(nums, left, mid)
    rsum = maxSub(nums, mid + 1, right)
    csum = crossMaxSub(nums, left, mid, right)

    return max(lsum, rsum, csum)

left, right = 0, len(nums) - 1
return maxSub(nums, left, right)

```

• SimplyLeet

```

# Python solution using Kadane's Algorithm
def maxSubArray(nums):
    # Initialize currentSum and maxSum to the first element
    current_sum = max_sum = nums[0]
    # Traverse through the array starting from the second element
    for num in nums[1:]:
        # If current_sum + num is less than num, restart at num
        current_sum = max(num, current_sum + num)
        # Update max_sum if current_sum is greater
        max_sum = max(max_sum, current_sum)

    return max_sum

# Example usage
if __name__ == "__main__":
    sample = [-2,1,-3,4,-1,2,1,-5,4]
    print(maxSubArray(sample)) # Output: 6

```

• LC.ca


```
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        res = cur_sum = nums[0]
        for num in nums[1:]:
            cur_sum = num + max(cur_sum, 0)
            res = max(res, cur_sum)
        return res
```

◆ Quick Review

Key Insights

- Use Kadane's Algorithm to solve the problem in $O(n)$ time.
- Keep track of a running sum (currentSum) and update it with the maximum between the current element and the sum including the current element.
- The maximum sum encountered during the iteration (maxSum) is the answer.
- A divide and conquer approach also exists, splitting the array and merging solutions, but Kadane's method is optimal for this problem.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

The solution uses Kadane's Algorithm. Iterate over the array while maintaining two variables: currentSum, which tracks the maximum subarray sum ending at the current position, and maxSum, which stores the best sum seen so far. For each element, update currentSum to be either the current element or the sum of currentSum and the current element, whichever is higher. Update maxSum accordingly. This approach efficiently computes the maximum contiguous subarray sum in a single pass.

Dynamic Programming

□ #55 Jump Game

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Greedy
Lists: Grind 169

Problem

You are given an integer array `nums`. You are initially positioned at the array's first index, and each element in the array represents your maximum jump length at that position.

Return `true` if you can reach the last index, or `false` otherwise.

Example 1:

Input: `nums = [2,3,1,1,4]`

Output: `true`

Explanation: Jump 1 step from index 0 to 1, then 3 steps to the last index.

Example 2:

Input: `nums = [3,2,1,0,4]`

Output: `false`

Explanation: You will always arrive at index 3 no matter what. Its maximum jump length is 0, which makes it impossible to reach the last index.

Constraints:

- $1 \leq \text{nums.length} \leq 104$

- $0 \leq \text{nums}[i] \leq 105$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Each element is the max jump length at that position. Can you reach the last index? Right?"

#

"A few quick questions:

Can all jumps be 0? Yes – but then we can only stay at index 0.

$\text{nums}[0]=0$ and $n=1 \rightarrow$ already at last index $\rightarrow$ true? Yes."

#

"Let me walk: $\text{nums}=[2,3,1,1,4] \rightarrow$ jump 2 from 0, reach 2 or 1, reach last $\rightarrow$ true ✓

$\text{nums}=[3,2,1,0,4] \rightarrow$ always land on index 3 ($\text{val}=0$), stuck $\rightarrow$ false ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

BFS/DFS from index 0: try all possible jumps, mark reachable indices.

$O(n^2)$ in the worst case. Should I go ahead and optimize?"

```
class _55_JumpGame_Brute:
```

```
    def canJump(self, nums):
```

```
        reachable = {0}
```

```
        for i in sorted(reachable):
```

```
            for j in range(i+1, min(i+nums[i]+1, len(nums))):
```

```
                reachable.add(j)
```

```
            if len(nums)-1 in reachable: return True
```

```
        return len(nums)-1 in reachable
```

PHASE 3 — OPTIMIZE

"The bottleneck is the set-based BFS tracking.

Greedy: track max_reach – the farthest index reachable so far.

At each index i : if $i > \text{max_reach}$ we're stuck (return False).

Otherwise update $\text{max_reach} = \max(\text{max_reach}, i + \text{nums}[i])$.

#

Time: $O(n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _55_JumpGame:
```

```
    def canJump(self, nums):
```

```
        max_reach = 0
```

```
        for i in range(len(nums)):
```

```
            if i > max_reach:
```

```

        return False
    max_reach = max(max_reach, i + nums[i])
    return True

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[2,3,1,1,4]: i=0:max=2. i=1:max=4. i=2:max=4. i=3:max=4. i=4:4>=4 ✓ → True ✓

nums=[3,2,1,0,4]: i=0:max=3. i=1:max=3. i=2:max=3. i=3:max=3. i=4:4>3 → False ✓

#

"No bugs. max_reach is monotonically non-decreasing – the greedy is optimal."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time O(n): single pass. Space O(1): one variable."

Follow-up: Jump Game II (#45) – minimum jumps to reach end; greedy BFS with range tracking.

Follow-up: Jump Game III (#1306) – can jump ±nums[i]; BFS/DFS from index 0."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def canJump(self, nums: list[int]) -> bool:
        i = 0
        reach = 0

        while i < len(nums) and i <= reach:
            reach = max(reach, i + nums[i])
            i += 1

        return i == len(nums)

```

• LeetDoocs

```

class Solution:
    def canJump(self, nums: List[int]) -> bool:
        mx = 0
        for i, x in enumerate(nums):
            if mx < i:
                return False
            mx = max(mx, i + x)
        return True

```

• SimplyLeet

```
# Function to determine if you can reach the last index of the array
def canJump(nums):
    # Initialize maxReach which is the max index reachable at start (index 0)
    maxReach = 0
    # Iterate through each element and its index
    for i, jump in enumerate(nums):
        # If current index is greater than maximum reachable index, return
        False
        if i > maxReach:
            return False
        # Update maxReach with the maximum of current maxReach and i + jump
        length at index i

    maxReach = max(maxReach, i + jump)
    # If current maxReach is already at or beyond the last index, return
    True
    if maxReach >= len(nums) - 1:
        return True
    # Return True if the end is reachable after processing all indices
    return True

# Example usage:
# print(canJump([2,3,1,1,4])) # Expected output: True
# print(canJump([3,2,1,0,4])) # Expected output: False
```

• LC.ca

```
class Solution:
    def canJump(self, nums: List[int]) -> bool:
        mx = 0
        for i, x in enumerate(nums):
            if mx < i:
                return False
            mx = max(mx, i + x)
        return True
```

◆ Quick Review

Key Insights

- The greedy approach is optimal for this problem.
- Track the furthest index reachable as you iterate through the array.
- If the current index exceeds the maximum reachable index, it is impossible to proceed further.
- The solution only requires constant extra space.

Space & Time

Time Complexity: $O(n)$ where n is the length of the array.

Space Complexity: $O(1)$

Solution

The solution uses a greedy algorithm. As you iterate through each array element, you maintain a variable (`maxReach`) that represents the furthest index you can reach so far. For each index i , if i is greater than `maxReach` then it is not possible to reach index i and hence return false immediately. Otherwise, update `maxReach` to the maximum of its current value and $i + \text{nums}[i]$. If you can iterate through the array without i exceeding

maxReach, then you can reach the end and return true.

Dynamic Programming

□ #62 Unique Paths

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Dynamic Programming · Combinatorics
Lists: Grind 169

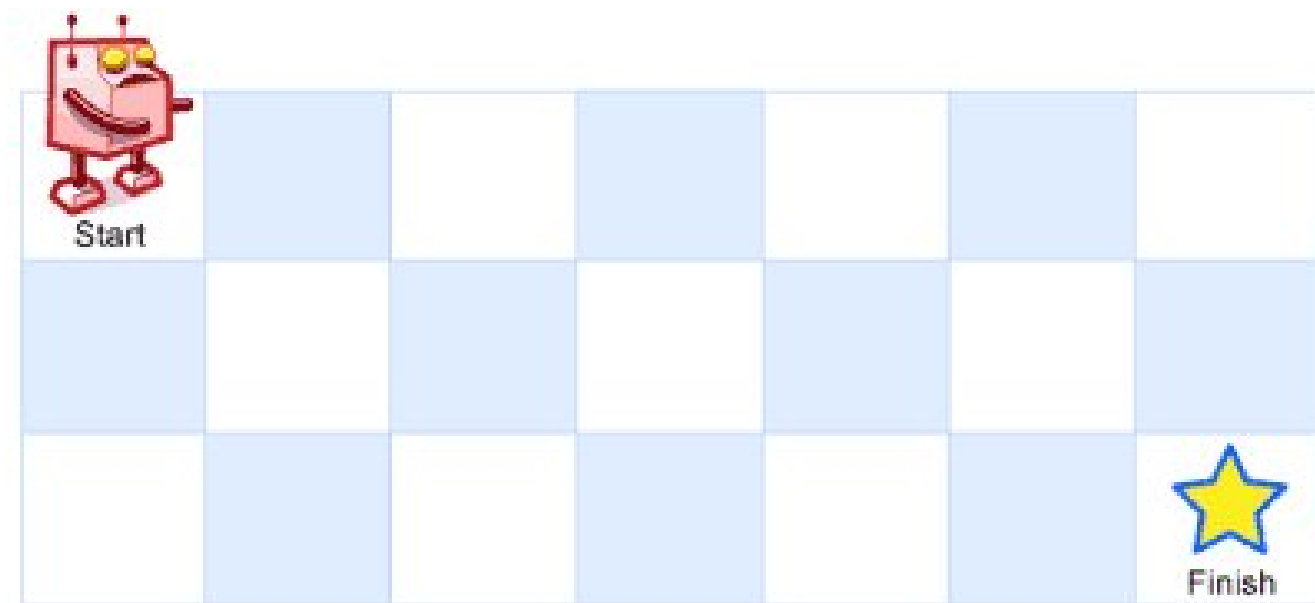
Problem

There is a robot on an $m \times n$ grid. The robot is initially located at the top-left corner (i.e., `grid[0][0]`). The robot tries to move to the bottom-right corner (i.e., `grid[m - 1][n - 1]`). The robot can only move either down or right at any point in time.

Given the two integers m and n , return the number of possible unique paths that the robot can take to reach the bottom-right corner.

The test cases are generated so that the answer will be less than or equal to $2 * 10^9$.

Example 1:



Input: $m = 3, n = 7$
 Output: 28

Example 2:

Input: $m = 3, n = 2$
 Output: 3
 Explanation: From the top-left corner, there are a total of 3 ways to reach the bottom-right corner:
 1. Right -> Down -> Down
 2. Down -> Down -> Right
 3. Down -> Right -> Down

Constraints:

- $1 \leq m, n \leq 100$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Robot starts at top-left of $m \times n$ grid, can only move right or down.

Count unique paths to reach bottom-right. Right?"

#

"A few quick questions:

```

# Can m or n be 1? Yes – only one path (straight right or straight down).
#
# "Let me walk: m=3, n=7 → 28 ✓. m=3, n=2 → 3 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Recursion: paths(r,c) = paths(r+1,c) + paths(r,c+1). Base: edge cells = 1.
#  $O(2^{m+n})$  without memo – exponential.
# Should I go ahead and optimize?"
class _62_UniquePaths_Brute:
    def uniquePaths(self, m, n):
        from functools import lru_cache
        @lru_cache(None)
        def dp(r, c):
            if r == m-1 or c == n-1: return 1
            return dp(r+1,c) + dp(r,c+1)
        return dp(0, 0)

# PHASE 3 — OPTIMIZE
# "The bottleneck is the 2D DP table.
# Math: total moves = (m-1) downs + (n-1) rights. Choose (m-1) of them:  $C(m+n-2, m-1)$ .
#  $O(\min(m,n))$  time,  $O(1)$  space.
# DP alternative:  $O(n)$  space with a 1D rolling array."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
import math
class _62_UniquePaths:
    def uniquePaths(self, m, n):
        return math.comb(m + n - 2, m - 1)

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# m=3,n=7: comb(3+7-2,3-1)=comb(8,2)=28 ✓
# m=3,n=2: comb(3,2)=3 ✓
# m=1,n=1: comb(0,0)=1 ✓
#
# "No bugs. The combination formula: choose which (m-1) of the (m+n-2) moves are
# 'down'."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Math  $O(\min(m,n))$ . DP  $O(m*n)$  time,  $O(n)$  space. Math is strictly better.
# Follow-up: Unique Paths II (#63) – obstacles in the grid; DP required.
# Follow-up: Minimum Path Sum (#64) – minimize sum; DP, can't use combinatorics."

```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        # dp[i][j] := the number of unique paths from (0, 0) to (i, j)
        dp = [[1] * n for _ in range(m)]

        for i in range(1, m):
            for j in range(1, n):
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1]

        return dp[-1][-1]
```

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        dp = [1] * n

        for _ in range(1, m):
            for j in range(1, n):
                dp[j] += dp[j - 1]

        return dp[n - 1]
```

• LeetDoocs

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        f = [[0] * n for _ in range(m)]
        f[0][0] = 1
        for i in range(m):
            for j in range(n):
                if i:
                    f[i][j] += f[i - 1][j]
                if j:
                    f[i][j] += f[i][j - 1]

        return f[-1][-1]
```

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        f = [[1] * n for _ in range(m)]
        for i in range(1, m):
            for j in range(1, n):
                f[i][j] = f[i - 1][j] + f[i][j - 1]
        return f[-1][-1]
```

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        f = [1] * n
        for _ in range(1, m):
            for j in range(1, n):
                f[j] += f[j - 1]
        return f[-1]
```

• SimplyLeet

Python solution using Dynamic Programming

```
def uniquePaths(m, n):
    # Initialize a 2D list with 1's for the first row and first column
    dp = [[1] * n for _ in range(m)]

    # Fill in the DP table
    for i in range(1, m):
        for j in range(1, n):
            # The number of ways to reach this cell is the sum of the ways from
            # the cell above and the cell on the left

            dp[i][j] = dp[i-1][j] + dp[i][j-1]
    # Return the value in the bottom-right cell
    return dp[m-1][n-1]

# Example usage:
print(uniquePaths(3, 7))
```

• LC.ca

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        # avoid setting dp[][] to 1 for i==0 or j==0 as initialization
        dp = [[1] * n for _ in range(m)]
        for i in range(1, m):
            for j in range(1, n):
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1]
        return dp[-1][-1]
```

◆ Quick Review

Key Insights

- The robot is constrained to only move right or down.
- Any valid path consists of exactly $(m-1)$ down moves and $(n-1)$ right moves.
- The problem can be solved using Dynamic Programming where each cell is the sum of the ways to reach the cell from the top and left.
- Alternatively, a combinatorial solution is possible by calculating the number of unique sequences of moves, equivalent to choosing $(m-1)$ moves from $(m+n-2)$ moves.

Space & Time

Time Complexity: $O(mn)$

Space Complexity: $O(mn)$ – can be optimized to $O(n)$ using a 1D DP array.

Solution

We can solve the problem using a dynamic programming approach. The idea is to create a 2D DP table where each cell (i, j) represents the number of ways to reach that cell. Since

the robot can only come from above or from the left, we update the DP table as follows: Initialize $dp[0][j] = 1$ for all j (only one way to move right in the first row). Initialize $dp[i][0] = 1$ for all i (only one way to move down in the first column). For each other cell, compute $dp[i][j] = dp[i-1][j] + dp[i][j-1]$. The final answer will be at $dp[m-1][n-1]$. The combinatorial alternative computes $C(m+n-2, m-1)$ or $C(m+n-2, n-1)$ using basic factorial arithmetic, which is efficient for the given constraints.

Dynamic Programming

□ #72 Edit Distance

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Denny Zhang

Problem

Given two strings word1 and word2, return the minimum number of operations required to convert word1 to word2.

You have the following three operations permitted on a word:

- Insert a character
- Delete a character
- Replace a character

Example 1:

```
Input: word1 = "horse", word2 = "ros"
Output: 3
Explanation:
horse -> rorse (replace 'h' with 'r')
rorse -> rose (remove 'r')
rose -> ros (remove 'e')
```

Example 2:

```

Input: word1 = "intention", word2 = "execution"
Output: 5
Explanation:
intention -> inention (remove 't')
inention -> enention (replace 'i' with 'e')
enention -> exention (replace 'n' with 'x')
exention -> exection (replace 'n' with 'c')
exection -> execution (insert 'u')

```

Constraints:

- $0 \leq \text{word1.length}, \text{word2.length} \leq 500$
- word1 and word2 consist of lowercase English letters.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Find the minimum number of insert, delete, or replace operations to transform word1 to word2. Right?"

#

"A few quick questions:

Can either word be empty? Yes – edit distance = length of the other word.

All three operations cost 1 each?"

#

"Let me walk: word1='horse', word2='ros' → 3 (replace h→r, delete r, delete e) ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Recursion: if chars match, recurse on rest. Else take min of insert/delete/replace + 1.

$O(3^{(m+n)})$ without memo – exponential.

Should I go ahead and optimize?"

```
class _72_EditDistance_Brute:
```

```
    def minDistance(self, word1, word2):
```

```
        from functools import lru_cache
```

```
        @lru_cache(None)
```

```
        def dp(i, j):
```

```
            if i == 0: return j
```

```
            if j == 0: return i
```

```

        if word1[i-1] == word2[j-1]: return dp(i-1, j-1)
        return 1 + min(dp(i-1,j), dp(i,j-1), dp(i-1,j-1))
    return dp(len(word1), len(word2))

```

PHASE 3 — OPTIMIZE

```

# "The bottleneck is the O(m*n) recursive calls without sharing.
# Bottom-up DP with O(1) space using two rolling rows:
# dp[j] = min edits to convert word1[:i] to word2[:j].
#
# Time: O(m*n). Space: O(n) rolling."

```

PHASE 4 — CLEAN CODE

```

# "Now I'll implement the optimized approach with meaningful names."

```

```

class _72_EditDistance:
    def minDistance(self, word1, word2):
        m, n = len(word1), len(word2)
        prev = list(range(n + 1))
        for i in range(1, m + 1):
            curr = [i] + [0] * n
            for j in range(1, n + 1):
                if word1[i-1] == word2[j-1]:
                    curr[j] = prev[j-1]
                else:
                    curr[j] = 1 + min(prev[j], curr[j-1], prev[j-1])
            prev = curr
        return prev[n]

```

PHASE 5 — TEST & DEBUG

```

# "Let me trace through a few cases."
#
# word1='horse', word2='ros': rolling DP converges to prev[3]=3 ✓
# word1='', word2='abc': prev=[0,1,2,3]. no iterations. return 3 ✓
#
# "No bugs. Base case prev=[0,1,2,...,n] handles inserting all of word2 into empty word1."

```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(m*n): fill the table. Space O(n): two rolling rows.
# The three recurrences: prev[j]=delete, curr[j-1]=insert, prev[j-1]=replace.
# Follow-up: One Edit Distance (#161) – check if exactly one edit away.
# Follow-up: Minimum ASCII Delete Sum (#712) – weight deletions by ASCII value."

```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        m = len(word1)
        n = len(word2)
        # dp[i][j] := the minimum number of operations to convert word1[0..i) to
        # word2[0..j)
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(1, m + 1):
            dp[i][0] = i

        for j in range(1, n + 1):
            dp[0][j] = j

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if word1[i - 1] == word2[j - 1]:
                    dp[i][j] = dp[i - 1][j - 1]
                else:
                    dp[i][j] = min(dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1]) + 1

        return dp[m][n]

```

• LeetDoocs

```

class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        m, n = len(word1), len(word2)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for j in range(1, n + 1):
            f[0][j] = j
        for i, a in enumerate(word1, 1):
            f[i][0] = i
            for j, b in enumerate(word2, 1):
                if a == b:
                    f[i][j] = f[i - 1][j - 1]
                else:
                    f[i][j] = min(f[i - 1][j], f[i][j - 1], f[i - 1][j - 1]) + 1
        return f[m][n]

```

• SimplyLeet

```
# Python solution to compute the Edit Distance
def minDistance(word1, word2):
    m = len(word1)
    n = len(word2)

    # Initialize the DP table with size (m+1) x (n+1)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Base cases: transform word1[0:i] to empty word2 requires i deletions
    for i in range(m + 1):
        dp[i][0] = i

    # Base cases: transform empty word1 to word2[0:j] requires j insertions
    for j in range(n + 1):
        dp[0][j] = j

    # Fill up the dp table
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            # If the characters are the same, no new operation is needed.
            if word1[i - 1] == word2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                # consider insertion, deletion, and replacement
                dp[i][j] = 1 + min(dp[i][j - 1],    # Insert
                                   dp[i - 1][j],    # Delete
                                   dp[i - 1][j - 1]) # Replace

    return dp[m][n]

# Example usage:
print(minDistance("horse", "ros"))
```

• LC.ca

```

class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        m, n = len(word1), len(word2)
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(m + 1):
            dp[i][0] = i
        for j in range(n + 1):
            dp[0][j] = j
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if word1[i - 1] == word2[j - 1]:
                    dp[i][j] = dp[i - 1][j - 1]
                else:
                    # dp[i - 1][j - 1] meaning replace.
                    # e.g. "abc" and "bf", checking "ab" and "b" distance,
                    # then replace either way for "c" or "f"
                    dp[i][j] = 1 + min(dp[i][j - 1], dp[i - 1][j], dp[i - 1][j
- 1])
        return dp[-1][-1]

#####

```

```

class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        m, n = len(word1), len(word2)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for j in range(1, n + 1):
            f[0][j] = j
        for i, a in enumerate(word1, 1):
            f[i][0] = i # merged for loops from above solution, but not good
for readability
            for j, b in enumerate(word2, 1):
                if a == b:
                    f[i][j] = f[i - 1][j - 1]
                else:
                    f[i][j] = min(f[i - 1][j], f[i][j - 1], f[i - 1][j - 1]) +
1
        return f[m][n]

```


◆ Quick Review

Key Insights

- This is a classic dynamic programming problem.
- Use a 2D DP table where $dp[i][j]$ represents the minimum edit distance between $word1[0...i-1]$ and $word2[0...j-1]$.
- Base cases: converting an empty string to a string of length j requires j insertions, and vice versa.
- Transition: If characters match then $dp[i][j] = dp[i-1][j-1]$, otherwise take minimum among insertion, deletion, and replacement operations and add 1.

Space & Time

Time Complexity: $O(mn)$, where m and n are the lengths of $word1$ and $word2$ respectively.

Space Complexity: $O(mn)$ due to the DP table used for storing computed distances.

Solution

We use a dynamic programming approach by constructing a 2D table dp of size $(m+1) \times (n+1)$, where m and n are the lengths of `word1` and `word2`. Each $dp[i][j]$ holds the minimum number of operations needed to convert the first i characters of `word1` to the first j characters of `word2`. The algorithm includes the following steps: Initialize the base cases: $dp[0][j] = j$ (converting an empty `word1` to j characters of `word2` requires j insertions) $dp[i][0] = i$ (converting i characters of `word1` to an empty `word2` requires i deletions) Iterate over each character in `word1` and `word2`: If the current characters are equal, no additional operation is needed and $dp[i][j] = dp[i-1][j-1]$. Otherwise, choose the best operation among insertion ($dp[i][j-1]$), deletion ($dp[i-1][j]$), or substitution ($dp[i-1][j-1]$) and add 1. The answer is found in $dp[m][n]$. This method reliably computes the edit distance using a systematic DP approach, ensuring that all subproblems are solved and reused efficiently.

Dynamic Programming

□ #91 Decode Ways

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Grind 169

Problem

You have intercepted a secret message encoded as a string of numbers. The message is decoded via the following mapping:

"1" -> 'A' "2" -> 'B' ... "25" -> 'Y' "26" -> 'Z'

However, while decoding the message, you realize that there are many different ways you can decode the message because some codes are contained in other codes ("2" and "5" vs "25").

For example, "11106" can be decoded into:

- "AAJF" with the grouping (1, 1, 10, 6)
- "KJF" with the grouping (11, 10, 6)
- The grouping (1, 11, 06) is invalid because "06" is not a valid code (only "6" is valid).

Note: there may be strings that are impossible to decode. Given a string *s* containing only digits, return the number of ways to decode it. If the entire string cannot be decoded in any valid way, return 0.

The test cases are generated so that the answer fits in a 32-bit integer.

Example 1:

Input: *s* = "12"

Output: 2

Explanation:

"12" could be decoded as "AB" (1 2) or "L" (12).

Example 2:

Input: *s* = "226"

Output: 3

Explanation:

"226" could be decoded as "BZ" (2 26), "VF" (22 6), or "BBF" (2 2 6).

Example 3:

Input: *s* = "06"

Output: 0

Explanation:

"06" cannot be mapped to "F" because of the leading zero ("6" is different from "06"). In this case, the string is not a valid encoding, so return 0.

Constraints:

- $1 \leq s.length \leq 100$
- s contains only digits and may contain leading zero(s).

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

'1'→'A', '2'→'B', ..., '26'→'Z'. Count distinct ways to decode a digit string.

'0' alone is invalid; '06' is invalid. Right?"

#

"A few quick questions:

$s = '11106' \rightarrow$ can be '1','1','10','6'='AAJF' or '11','10','6'='KJF' but not '1','110','6'. $\rightarrow 2$ ways ✓

Empty string $\rightarrow 1$ way (trivially decoded)."

#

"Let me walk: $s = '12' \rightarrow '1','2'$ or $'12' \rightarrow 2$ ✓. $s = '226' \rightarrow 3$ ✓. $s = '06' \rightarrow 0$ ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Recursion: if $s[i]$ valid as single digit, recurse on rest; if $s[i:i+2]$ valid, also recurse.

$O(2^n)$ without memo.

Should I go ahead and optimize?"

```
class _91_DecodeWays_Brute:
```

```
    def numDecodings(self, s):
```

```
        from functools import lru_cache
```

```
        @lru_cache(None)
```

```
        def dp(i):
```

```
            if i == len(s): return 1
```

```
            if s[i] == '0': return 0
```

```
            result = dp(i + 1)
```

```

        if i+1 < len(s) and int(s[i:i+2]) <= 26:
            result += dp(i + 2)
        return result
    return dp(0)

```

PHASE 3 — OPTIMIZE

```

# "The bottleneck is re-computing subproblems.
# Bottom-up DP with O(1) space – only need last two values:
# dp[i] = ways to decode s[:i]. Roll: a=dp[i-2], b=dp[i-1].
# Single digit: if s[i-1]!='0' → curr += b.
# Two digits: if '10'<=s[i-2:i]<='26' → curr += a.
#
# Time: O(n). Space: O(1)."
```

PHASE 4 — CLEAN CODE

```

# "Now I'll implement the optimized approach with meaningful names."
```

```

class _91_DecodeWays:
    def numDecodings(self, s):
        prev2 = 1
        prev1 = 1 if s[0] != '0' else 0
        if len(s) == 1: return prev1
        for i in range(2, len(s) + 1):
            curr = 0
            if s[i-1] != '0':
                curr += prev1
            two_digit = int(s[i-2:i])
            if 10 <= two_digit <= 26:
                curr += prev2
            prev2, prev1 = prev1, curr
        return prev1

```

PHASE 5 — TEST & DEBUG

```

# "Let me trace through a few cases."
#
# s='226': prev2=1,prev1=1 (s[0]='2'≠0).
# i=2: s[1]='2'≠0→curr+=prev1=1. s[0:2]='22'∈[10,26]→curr+=prev2=1. curr=2.
# prev2=1,prev1=2.
# i=3: s[2]='6'≠0→curr+=2. s[1:3]='26'∈[10,26]→curr+=1. curr=3. return 3 ✓
#
# s='06': prev1=0 (s[0]='0'). i=2: s[1]='6'≠0→curr+=0. s[0:2]='06' not in [10,26].
# curr=0. → 0 ✓
#
# "No bugs. Checking '0' as single digit and [10,26] as two-digit covers all cases."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n): one pass. Space O(1): two rolling variables.
# '0' alone is invalid; '10','20' are valid two-digit codes.
# Follow-up: Decode Ways II (#639) – '*' can be any digit 1-9; multiply ways by 9 or
# constrained count.
# Follow-up: if encoding uses different ranges, adjust the ≤26 bound."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def numDecodings(self, s: str) -> int:
        n = len(s)
        # dp[i] := the number of ways to decode s[i..n)
        dp = [0] * n + [1]

        def isValid(a: str, b=None) -> bool:
            if b:
                return a == '1' or a == '2' and b < '7'
            return a != '0'

        if isValid(s[-1]):
            dp[n - 1] = 1

        for i in reversed(range(n - 1)):
            if isValid(s[i]):
                dp[i] += dp[i + 1]
            if isValid(s[i], s[i + 1]):
                dp[i] += dp[i + 2]

        return dp[0]
```

• LeetDoocs

```
class Solution:
    def numDecodings(self, s: str) -> int:
        n = len(s)
        f = [1] + [0] * n
        for i, c in enumerate(s, 1):
            if c != "0":
                f[i] = f[i - 1]
            if i > 1 and s[i - 2] != "0" and int(s[i - 2 : i]) <= 26:
                f[i] += f[i - 2]
        return f[n]
```



```
class Solution:
    def numDecodings(self, s: str) -> int:
        f, g = 0, 1
        for i, c in enumerate(s, 1):
            h = g if c != "0" else 0
            if i > 1 and s[i - 2] != "0" and int(s[i - 2 : i]) <= 26:
                h += f
            f, g = g, h
        return g
```

• SimplyLeet

```
# Python solution using dynamic programming
def num_decodings(s: str) -> int:
    # If the string starts with '0', it's not decodable
    if not s or s[0] == '0':
        return 0

    # dp array where dp[i] represents the number of ways to decode s[:i]
    n = len(s)
    dp = [0] * (n + 1)

    # Base case: an empty string has one decoding
    dp[0] = 1

    for i in range(1, n + 1):
        # Check for single digit decoding (s[i-1] is valid if not '0')
        if s[i - 1] != '0':
            dp[i] += dp[i - 1]

        # Check for two-digit decoding (if i>=2, s[i-2:i] forms a valid number
        # between 10 and 26)
        if i >= 2 and s[i - 2] != '0':
            two_digit = int(s[i - 2:i])

            if 10 <= two_digit <= 26:
                dp[i] += dp[i - 2]

    return dp[n]

# Example usage:
print(num_decodings("12")) # Expected output: 2
```

• LC.ca

```
'''
>>> s = "abcdef"
>>> [(i,c) for i, c in enumerate(s, 1)]
[(1, 'a'), (2, 'b'), (3, 'c'), (4, 'd'), (5, 'e'), (6, 'f')]
'''

class Solution:
    def numDecodings(self, s: str) -> int:
        # f: i-2
        # g: i-1
        f, g = 0, 1

        for i, c in enumerate(s, 1):
            h = g if c != "0" else 0
            if i > 1 and s[i - 2] != "0" and int(s[i - 2 : i]) <= 26:
                h += f
            f, g = g, h
        return g

#####

class Solution:

    def numDecodings(self, s: str) -> int:
        if len(s) == 0:
            return 0

        length = len(s)
        dp = [0] * (length + 1) # dp[i] => at index i, its decode ways

        dp[length] = 1 # initiator, just make forloop flow working
        dp[length - 1] = 0 if s[length - 1] == '0' else 1 # assumption is
        starting at i, so starting as 0 is not decodable
```

```

# start at 'len-2'
# or else below 'dp[i+2]' will out of index boundary
for i in range(length - 2, -1, -1):
    if s[i] == '0':
        continue
    tem = int(s[i:i + 2])

    if tem > 26:
        dp[i] = dp[i + 1]
    else:

```

```

        dp[i] = dp[i + 1] + dp[i + 2]
    return dp[0]

```

optimized from above solution, no dp array

class Solution:

```

    def numDecodings(self, s: str) -> int:

```

```

        n = len(s)

```

```

        # a: dp[i-2], b: dp[i-1], c: count for current index i

```

```

        a, b, c = 0, 1, 0

```

```

        for i in range(1, n + 1):

```

```

            c = 0

```

```

            if s[i - 1] != '0':

```

```

                c += b

```

```

            if i > 1 and s[i - 2] != '0' and (int(s[i - 2]) * 10 + int(s[i - 1]) <= 26):

```

```

                c += a

```

```

            a, b = b, c

```

```

        return c

```

```

#####

```

```
class Solution:
    def numDecodings(self, s: str) -> int:
        n = len(s)
        dp = [0] * (n + 1)
        dp[0] = 1
        for i in range(1, n + 1):
            if s[i - 1] != '0':
                dp[i] += dp[i - 1]
            if i > 1 and s[i - 2] != '0' and (int(s[i - 2]) * 10 + int(s[i - 1])) <= 26:
                dp[i] += dp[i - 2]

        return dp[n]
```

◆ Quick Review

Key Insights

- A digit '0' cannot be mapped on its own; it must be part of "10" or "20".
- Use dynamic programming where $dp[i]$ represents the number of ways to decode the substring $s[0...i-1]$.
- For each index i , if the digit at position $i-1$ is non-zero, add $dp[i-1]$ to $dp[i]$.
- If the two-digit number formed by $s[i-2]$ and $s[i-1]$ is between 10 and 26, add $dp[i-2]$ to $dp[i]$.
- Initialize $dp[0]$ as 1 (an empty string has one way to be decoded).

Space & Time

Time Complexity: $O(n)$, where n is the length of the string.

Space Complexity: $O(n)$, due to the dp array (which can be optimized to $O(1)$).

Solution

We solve the problem using a dynamic programming approach. We initialize a dp array where $dp[i]$ indicates the number of ways to decode the substring $s[0...i-1]$. We set $dp[0] = 1$ since an empty string is trivially decodable. We loop through the string, and at each step: If $s[i-1]$ is not '0', we add $dp[i-1]$ to $dp[i]$ because $s[i-1]$ can be mapped to a valid letter. If the substring $s[i-2:i]$ (for $i \geq 2$) forms a valid two-digit number between 10 and 26, we add $dp[i-2]$ to $dp[i]$. This cumulative approach yields the total ways to decode the string, and if the string starts with '0' or contains invalid sequences, $dp[n]$ will eventually be 0.

Dynamic Programming

□ #152 Maximum Product Subarray

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: Grind 169

Problem

Given an integer array `nums`, find a subarray that has the largest product, and return the product.

The test cases are generated so that the answer will fit in a 32-bit integer.

Note that the product of an array with a single element is the value of that element.

Example 1:

```
Input: nums = [2,3,-2,4]
Output: 6
Explanation: [2,3] has the largest product 6.
```

Example 2:

```
Input: nums = [-2,0,-1]
Output: 0
Explanation: The result cannot be 2, because [-2,-1] is not a subarray.
```

Constraints:

- $1 \leq \text{nums.length} \leq 2 * 10^4$
- $-10 \leq \text{nums}[i] \leq 10$
- The product of any subarray of nums is guaranteed to fit in a 32-bit integer.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Find the contiguous subarray with the largest product and return the product.
Right?"
#
# "A few quick questions:
# Can values be negative? Yes – two negatives can multiply to a large positive.
# Can values be 0? Yes – zeros break the subarray."
#
# "Let me walk: nums=[2,3,-2,4] → [2,3]=6 ✓. nums=[-2,0,-1] → 0 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Try all subarrays, track max product.  $O(n^2)$ .
# Should I go ahead and optimize?"
class _152_MaxProductSubarray_Brute:
    def maxProduct(self, nums):
        best = nums[0]
        for i in range(len(nums)):
            curr = 1
            for j in range(i, len(nums)):
                curr *= nums[j]; best = max(best, curr)
        return best
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the nested scan.
# Track both max_prod and min_prod because a negative × negative becomes a large
positive.
# At each step: new candidates = (num, max*num, min*num).
# Take max of candidates for new max, min for new min.
#
# Time:  $O(n)$ . Space:  $O(1)$ ."
```

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _152_MaxProductSubarray:
    def maxProduct(self, nums):
        max_prod = min_prod = best = nums[0]
        for num in nums[1:]:
            candidates = (num, max_prod * num, min_prod * num)
            max_prod, min_prod = max(candidates), min(candidates)
            best = max(best, max_prod)
        return best
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

```
#
# nums=[2,3,-2,4]:
# max=2,min=2,best=2. num=3: cand=(3,6,6)→max=6,min=3,best=6.
# num=-2: cand=(-2,-12,-6)→max=-2,min=-12,best=6.
# num=4: cand=(4,-8,-48)→max=4,min=-48,best=6. → 6 ✓
#
# nums=[-2,0,-1]: max=-2,best=-2. 0:max=0,best=0. -1:cand=(-1,0,0)→max=0,best=0. → 0 ✓
#
# "No bugs. Tracking min_prod ensures we don't miss large products from two negatives."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): single pass. Space O(1): three variables.
# The min_prod tracking is the key difference from Kadane's (#53).
# Follow-up: Maximum Sum Subarray (#53) – same pattern without the min tracking.
# Follow-up: Product of Array Except Self (#238) – prefix/suffix products, no division."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxProduct(self, nums: list[int]) -> int:
        ans = nums[0]
        dpMin = nums[0] # the minimum so far
        dpMax = nums[0] # the maximum so far

        for i in range(1, len(nums)):
            num = nums[i]
            prevMin = dpMin # dpMin[i - 1]
            prevMax = dpMax # dpMax[i - 1]
```

```

    if num < 0:
        dpMin = min(prevMax * num, num)
        dpMax = max(prevMin * num, num)
    else:
        dpMin = min(prevMin * num, num)
        dpMax = max(prevMax * num, num)

    ans = max(ans, dpMax)

    return ans

```

• LeetCode

```

class Solution:
    def maxProduct(self, nums: List[int]) -> int:
        ans = f = g = nums[0]
        for x in nums[1:]:
            ff, gg = f, g
            f = max(x, ff * x, gg * x)
            g = min(x, ff * x, gg * x)
            ans = max(ans, f)
        return ans

```

• SimplyLeet

```

def maxProduct(nums):
    # If the input array is empty, return 0 (though problem guarantees at least
    # one element)
    if not nums:
        return 0

    # Initialize maximum, minimum product ending here and the result.
    max_ending_here = nums[0]
    min_ending_here = nums[0]
    global_max = nums[0]

    # Iterate over the array starting from the second element.

```

```

for num in nums[1:]:
    # If the current number is negative, swap max and min.
    if num < 0:
        max_ending_here, min_ending_here = min_ending_here, max_ending_here

    # Update the max product at the current index.
    max_ending_here = max(num, max_ending_here * num)
    # Update the min product at the current index.
    min_ending_here = min(num, min_ending_here * num)
    # Update the global maximum product.
    global_max = max(global_max, max_ending_here)

return global_max

# Example usage:
# print(maxProduct([2,3,-2,4])) # Output: 6

```

• LC.ca

```

class Solution:
    def maxProduct(self, nums: List[int]) -> int:
        ans = f = g = nums[0]
        for x in nums[1:]:
            ff, gg = f, g
            f = max(x, ff * x, gg * x)
            g = min(x, ff * x, gg * x)
            ans = max(ans, f)
        return ans

```

◆ Quick Review

Key Insights

- Maintain both the maximum and minimum product at the current index because a negative number can swap the max and min.
- A subarray must be contiguous.

- Use dynamic programming to update both the maximum product and the minimum product ending at each index.
- Resetting the product in case of a zero is crucial since the product becomes 0.

Space & Time

Time Complexity: $O(n)$ – The solution iterates through the array once.

Space Complexity: $O(1)$ – Constant space is used for tracking products.

Solution

The solution uses an iterative dynamic programming approach. At each index, compute the maximum product (maxEndingHere) and minimum product (minEndingHere) ending at that index. If the current element is negative, swap these two values, because multiplying a negative with a minimum product (possibly negative) might yield a larger product. After processing each element, update the global maximum. This method handles edge cases like zeros and negatives efficiently with constant additional space.

Dynamic Programming

□ #198 House Robber

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: Grind 169

Problem

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and it will automatically contact the police if two adjacent houses were broken into on the same night.

Given an integer array `nums` representing the amount of money of each house, return the maximum amount of money you can rob tonight without alerting the police.

Example 1:

Input: nums = [1,2,3,1]

Output: 4

Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).

Total amount you can rob = 1 + 3 = 4.

Example 2:

Input: nums = [2,7,9,3,1]

Output: 12

Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money = 1).

Total amount you can rob = 2 + 9 + 1 = 12.

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 400$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Rob houses in a row; can't rob adjacent houses. Maximize stolen amount. Right?"

#

"A few quick questions:

Can we skip multiple houses? Yes – skip any number between robberies.

All amounts non-negative? Yes."

#

"Let me walk: nums=[1,2,3,1] → rob 1+3=4 ✓. nums=[2,7,9,3,1] → 2+9+1=12 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Try all subsets of non-adjacent houses. $O(2^n)$ time.

Should I go ahead and optimize?"

```
class _198_HouseRobber_Brute:
```

```
    def rob(self, nums):
```

```
        def dfs(i):
```

```
            if i >= len(nums): return 0
```

```
            return max(dfs(i+1), nums[i] + dfs(i+2))
```

```
        return dfs(0)
```

PHASE 3 — OPTIMIZE

```

# "The bottleneck is recomputing overlapping subproblems.
# DP with O(1) space – only need last two values:
# at each house: rob=nums[i]+prev2; skip=prev1. Take max.
#
# Time: O(n). Space: O(1)."
```

PHASE 4 — CLEAN CODE

```

# "Now I'll implement the optimized approach with meaningful names."
class _198_HouseRobber:
    def rob(self, nums):
        prev2 = prev1 = 0
        for amount in nums:
            prev2, prev1 = prev1, max(prev1, prev2 + amount)
        return prev1
```

PHASE 5 — TEST & DEBUG

```

# "Let me trace through a few cases."
#
# nums=[2,7,9,3,1]:
# 2: prev2=0,prev1=max(0,0+2)=2. 7: prev2=2,prev1=max(2,0+7)=7.
# 9: prev2=7,prev1=max(7,2+9)=11. 3: prev2=11,prev1=max(11,7+3)=11.
# 1: prev2=11,prev1=max(11,11+1)=12. return 12 ✓
#
# nums=[1]: prev1=max(0,0+1)=1. return 1 ✓
#
# "No bugs. prev2 carries the value from two houses back, enabling the non-adjacent
check."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```

# "Time O(n): single pass. Space O(1): two rolling variables.
# Follow-up: House Robber II (#213) – circular arrangement; run two passes:
[0..n-2],[1..n-1].
# Follow-up: House Robber III (#337) – tree structure; DP on tree nodes."
```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def rob(self, nums: list[int]) -> int:
        if not nums:
            return 0
        if len(nums) == 1:
            return nums[0]

        # dp[i] := max money of robbing nums[0..i]
        dp = [0] * len(nums)
        dp[0] = nums[0]
```



```

dp[1] = max(nums[0], nums[1])

for i in range(2, len(nums)):
    dp[i] = max(dp[i - 1], dp[i - 2] + nums[i])

return dp[-1]

```

```

class Solution:
    def rob(self, nums: list[int]) -> int:
        prev1 = 0 # dp[i - 1]
        prev2 = 0 # dp[i - 2]

        for num in nums:
            dp = max(prev1, prev2 + num)
            prev2 = prev1
            prev1 = dp

        return prev1

```

• LeetDoocs

```

class Solution:
    def rob(self, nums: List[int]) -> int:
        @cache
        def dfs(i: int) -> int:
            if i >= len(nums):
                return 0
            return max(nums[i] + dfs(i + 2), dfs(i + 1))

        return dfs(0)

```

```

class Solution:
    def rob(self, nums: List[int]) -> int:
        n = len(nums)
        f = [0] * (n + 1)
        f[1] = nums[0]
        for i in range(2, n + 1):
            f[i] = max(f[i - 1], f[i - 2] + nums[i - 1])
        return f[n]

```

```
class Solution:
    def rob(self, nums: List[int]) -> int:
        f = g = 0
        for x in nums:
            f, g = max(f, g), f + x
        return max(f, g)
```

• SimplyLeet

```
# Definition of the Solution class
class Solution:
    def rob(self, nums: List[int]) -> int:
        # Edge case: if there are no houses, return 0
        if not nums:
            return 0

        # Initialize two variables to keep track of the max amounts
        prev1, prev2 = 0, 0

        # Iterate over each house's money
        for money in nums:
            # Calculate the maximum money that can be robbed if we consider the
            # current house
            current = max(prev1, prev2 + money) # Either skip current house or
            rob it
            # Update variables: prev2 becomes previous best, and prev1 becomes
            # current best
            prev2 = prev1
            prev1 = current

        # Return the maximum money that can be robbed
        return prev1
```

• LC.ca

```
# greedy
class Solution:
    def rob(self, nums: List[int]) -> int:
        not_rob, rob = 0, nums[0]
        for num in nums[1:]:
            # must max check
            # eg. first robbed 99999, then following ones are just 3,3,3,3,3
            not_rob, rob = rob, max(num + not_rob, rob)
        return rob
```

```
#####
```

```
class Solution(object):
    def rob(self, nums):
        """
        :type nums: List[int]
        :rtype: int
        """
        if len(nums) == 0:
            return 0

        if len(nums) <= 2:
            return max(nums)
        dp = [0 for i in range(0, 2)]
        dp[0] = nums[0]
        dp[1] = max(nums[1], nums[0])
        for i in range(2, len(nums)):
            dp[i % 2] = max(dp[(i - 1) % 2], dp[(i - 2) % 2] + nums[i])
        return dp[(len(nums) - 1) % 2]
```

◆ Quick Review

Key Insights

- The problem can be solved using Dynamic Programming.
- For each house, decide whether to rob it or not.

- Recurrence relation: $dp[i] = \max(dp[i-1], dp[i-2] + \text{currentHouseMoney})$
- Use iterative computation to keep track of the optimal solutions without needing extra space for the entire dp array.

Space & Time

Time Complexity: $O(n)$, where n is the number of houses.

Space Complexity: $O(1)$, using only two variables to track the previous decisions.

Solution

We use a dynamic programming approach by iterating through each house in the array. At each step, we consider two scenarios: either we rob the current house (and add its money to the total from two houses ago) or we skip it (keeping the optimal solution up to the previous house). By using two variables to store these intermediate results, we achieve an optimized space complexity of $O(1)$.

Dynamic Programming

□ ★ #256 Paint House ★

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: ★ Premium | Premium 98

Problem

There is a row of n houses, where each house can be painted one of three colors: red, blue, or green. The cost of painting each house with a certain color is different. You have to paint all the houses such that no two adjacent houses have the same color.

The cost of painting each house with a certain color is represented by an $n \times 3$ cost matrix costs.

- For example, costs[0][0] is the cost of painting house 0 with the color red; costs[1][2] is the cost of painting house 1 with color green, and so on...

Return the minimum cost to paint all houses.

Example 1:

Input: costs = [[17,2,17],[16,16,5],[14,3,19]]

Output: 10

Explanation: Paint house 0 into blue, paint house 1 into green, paint house 2 into blue.

Minimum cost: 2 + 5 + 3 = 10.

Example 2:

Input: costs = [[7,6,2]]

Output: 2

Constraints:

- costs.length == n
- costs[i].length == 3
- 1 ≤ n ≤ 100
- 1 ≤ costs[i][j] ≤ 20

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

n houses, 3 colors (Red/Blue/Green). No adjacent houses same color.

costs[i] = [r,b,g] cost for house i. Return minimum total cost. Right?"

#

"A few quick questions:

n ≥ 1? Yes. Cost values are positive? Yes."

#

"Let me walk: costs=[[17,2,17],[16,16,5],[14,3,19]] → Blue(2)+Green(5)+Blue(3)=10

✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Try all 3^n color combinations, filter valid, return minimum.

0(3^n) – exponential. Should I go ahead and optimize?"

```
class _256_PaintHouse_Brute:
```

```
    def minCost(self, costs):
```

```
        def dfs(i, prev):
```

```
            if i == len(costs): return 0
```

```
        return min(costs[i][c] + dfs(i+1, c) for c in range(3) if c != prev)
    return dfs(0, -1)
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is recomputing overlapping states.
# DP with O(1) space: track the minimum cost to paint house i with each of the 3
# colors.
# dp[c] = min cost to paint up to current house with color c.
# Transition: dp[c] = costs[i][c] + min(dp_prev[c'] for c' ≠ c).
#
# Time: O(n). Space: O(1) – three rolling variables."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _256_PaintHouse:
    def minCost(self, costs):
        r, b, g = costs[0]
        for i in range(1, len(costs)):
            r, b, g = (costs[i][0] + min(b, g),
                      costs[i][1] + min(r, g),
                      costs[i][2] + min(r, b))
        return min(r, b, g)
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# costs=[[17,2,17],[16,16,5],[14,3,19]]:
# Start: r=17,b=2,g=17.
# House1: r=16+min(2,17)=18, b=16+min(17,17)=33, g=5+min(17,2)=7.
# House2: r=14+min(33,7)=21, b=3+min(18,7)=10, g=19+min(18,33)=37.
# min(21,10,37)=10 ✓
#
# costs=[[7,6,2]]: r=7,b=6,g=2. min=2 ✓
#
# "No bugs. Each color's new cost = costs[i][c] + min of the OTHER two colors."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): one pass. Space O(1): three variables.
# Follow-up: Paint House II (#265) – k colors. Track two smallest from previous row:
# O(n*k).
# Follow-up: Paint House III (#1473) – some houses pre-painted, exactly m
# neighborhoods."
```

Community Solutions (Python)

• WalkCC


```
class Solution:
    def minCost(self, costs: list[list[int]]) -> list[list[int]]:
        for i in range(1, len(costs)):
            costs[i][0] += min(costs[i - 1][1], costs[i - 1][2])
            costs[i][1] += min(costs[i - 1][0], costs[i - 1][2])
            costs[i][2] += min(costs[i - 1][0], costs[i - 1][1])

        return min(costs[-1])
```

• LeetDoocs

```
class Solution:
    def minCost(self, costs: List[List[int]]) -> int:
        a = b = c = 0
        for ca, cb, cc in costs:
            a, b, c = min(b, c) + ca, min(a, c) + cb, min(a, b) + cc
        return min(a, b, c)
```

• SimplyLeet

Python solution for the Paint House problem

```
def minCost(costs):
    # Get the number of houses
    n = len(costs)
    # Base case: If there are no houses, return 0 cost
    if n == 0:
        return 0

    # Iterate from the second house to the last house
```

```
    for i in range(1, n):
        # Update the cost for painting the current house red
        costs[i][0] += min(costs[i-1][1], costs[i-1][2])
        # Update the cost for painting the current house blue
        costs[i][1] += min(costs[i-1][0], costs[i-1][2])
        # Update the cost for painting the current house green
        costs[i][2] += min(costs[i-1][0], costs[i-1][1])
```

```
    # The answer is the minimum cost of painting the last house with any of the colors
    return min(costs[-1])
```

```
# Example usage:
example_costs = [[17,2,17],[16,16,5],[14,3,19]]
print(minCost(example_costs)) # Output: 10
```

• LC.ca

```
class Solution:
    def minCost(self, costs: List[List[int]]) -> int:
        a = b = c = 0
        for ca, cb, cc in costs:
            a, b, c = min(b, c) + ca, min(a, c) + cb, min(a, b) + cc
        return min(a, b, c)
```

```
#####
```

```
...
```

Two-dimensional dynamic array dp,
where dp[i][j] represents the minimum cost of brushing to the i+1th house with
color j,

and the state transition equation is:

$$dp[i][j] = cost[i][j] + \min(dp[i-1][(j+1)\%3], dp[i-1][(j+2)\%3])$$

```
...
```

```
class Solution(object):
    def minCost(self, costs):
        .....
```

```
:type costs: List[List[int]]
:rtype: int
.....

if not costs:
    return 0
dp = [0] * (len(costs[0]))
dp[:] = costs[0]
for i in range(1, len(costs)):
    d0 = d1 = d2 = 0
    for j in range(0, 3):
```

```
if j == 0:
    d0 = min(dp[1], dp[2]) + costs[i][0]
elif j == 1:
    d1 = min(dp[0], dp[2]) + costs[i][1]
else:
    d2 = min(dp[0], dp[1]) + costs[i][2]
dp[0], dp[1], dp[2] = d0, d1, d2
return min(dp)
```

◆ Quick Review

Key Insights

- The choice for painting each house depends on the previous house's color choices.
- Use dynamic programming to accumulate the cost while ensuring adjacent houses do not have the same color.
- The state of each house can be represented by the minimum cost when the house is painted a specific color, computed using the minimal costs of the previous house from the other two colors.
- The problem only requires updating the cost matrix in-place for optimal space usage.

Space & Time

Time Complexity: $O(n)$, where n is the number of houses.

Space Complexity: $O(1)$ extra space if the input matrix is modified in place.

Solution

We use a dynamic programming approach with iterative in-place updating of the cost matrix. For each house starting from the second one, we update the cost for each color by adding the minimal cost of painting the previous house with either of the other two colors. This guarantees we never use the same color for adjacent houses while accumulating the minimal total cost. The final answer is the minimum value for the last house.

Dynamic Programming

□ #309 Best Time to Buy and Sell Stock with Cooldown

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: Denny Zhang

Problem

You are given an array `prices` where `prices[i]` is the price of a given stock on the *i*th day.

Find the maximum profit you can achieve. You may complete as many transactions as you like (i.e., buy one and sell one share of the stock multiple times) with the following restrictions:

- After you sell your stock, you cannot buy stock on the next day (i.e., cooldown one day).

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Example 1:

Input: `prices = [1,2,3,0,2]`

Output: 3

Explanation: `transactions = [buy, sell, cooldown, buy, sell]`

Example 2:

Input: prices = [1]
Output: 0

Constraints:

- $1 \leq \text{prices.length} \leq 5000$
- $0 \leq \text{prices}[i] \leq 1000$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# After selling, must wait 1 day before buying again (cooldown).
# Unlimited transactions but hold at most one stock. Maximize profit. Right?"
#
# "A few quick questions:
# Can I buy and sell on the same day? No. Cooldown is exactly 1 day? Yes."
#
# "Let me walk: prices=[1,2,3,0,2] → buy1,sell3,cooldown,buy0,sell2 → profit=3 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Recursion with states: (day, holding). At each day choose hold/sell/cooldown/buy.
#  $O(2^n)$  without memo. Should I go ahead and optimize?"
class _309_StockWithCooldownBrute:
    def maxProfit(self, prices):
        from functools import lru_cache
        @lru_cache(None)
        def dp(i, holding):
            if i >= len(prices): return 0
            if holding:
                return max(prices[i] + dp(i+2, False), dp(i+1, True))
            else:
                return max(-prices[i] + dp(i+1, True), dp(i+1, False))
        return dp(0, False)
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the 2D state space with recursion overhead.
# Three state variables with  $O(1)$  space:
# held = max profit when holding a stock.
```

```

# sold = max profit on the day we just sold (entering cooldown).
# rest = max profit when idle or in cooldown (free to buy next day).
#
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _309_StockWithCooldown:
    def maxProfit(self, prices):
        held = float('-inf')
        sold = 0
        rest = 0
        for price in prices:
            held, sold, rest = (max(held, rest - price),
                                held + price,
                                max(rest, sold))

        return max(sold, rest)

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# prices=[1,2,3,0,2]:
# price=1: held=max(-inf,0-1)=-1, sold=-inf+1=-inf, rest=max(0,-inf)=0.
# price=2: held=max(-1,0-2)=-1, sold=-1+2=1, rest=max(0,-inf)=0.
# price=3: held=max(-1,0-3)=-1, sold=-1+3=2, rest=max(0,1)=1.
# price=0: held=max(-1,1-0)=1, sold=-1+0=-1, rest=max(1,2)=2.
# price=2: held=max(1,2-2)=1, sold=1+2=3, rest=max(2,-1)=2.
# max(3,2)=3 ✓
#
# "No bugs. The sold state prevents buying the very next day after selling."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): single pass. Space O(1): three variables.
# Follow-up: Stock with Transaction Fee (#714) – subtract fee on sell; same
two-state DP.
# Follow-up: At most k transactions (#188) – extend state to (day, k, holding)."
```

Community Solutions (Python)

• WalkCC


```

class Solution:
    def maxProfit(self, prices: list[int]) -> int:
        sell = 0
        hold = -math.inf
        prev = 0

        for price in prices:
            cache = sell
            sell = max(sell, hold + price)
            hold = max(hold, prev - price)

        prev = cache

        return sell

```

• LeetDoocs

```

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        @cache
        def dfs(i: int, j: int) -> int:
            if i >= len(prices):
                return 0
            ans = dfs(i + 1, j)
            if j:
                ans = max(ans, prices[i] + dfs(i + 2, 0))
            else:
                ans = max(ans, -prices[i] + dfs(i + 1, 1))
            return ans

        return dfs(0, 0)

```

```

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        n = len(prices)
        f = [[0] * 2 for _ in range(n)]
        f[0][1] = -prices[0]
        for i in range(1, n):
            f[i][0] = max(f[i - 1][0], f[i - 1][1] + prices[i])
            f[i][1] = max(f[i - 1][1], f[i - 2][0] - prices[i])
        return f[n - 1][0]

```

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        f, f0, f1 = 0, 0, -prices[0]
        for x in prices[1:]:
            f, f0, f1 = f0, max(f0, f1 + x), max(f1, f - x)
        return f0
```

• SimplyLeet

Python Solution with line-by-line comments

```
def maxProfit(prices):
    # Handle edge case: if there are no prices, profit is 0
    if not prices:
        return 0

    # Initialize states:
    # s0: Maximum profit when ready to buy
    # s1: Maximum profit when holding a stock (bought)

    # s2: Maximum profit during cooldown after a sale
    s0, s1, s2 = 0, -prices[0], 0

    # Iterate over prices starting from day 1
    for price in prices[1:]:
        # Store previous state values
        prev_s0, prev_s1, prev_s2 = s0, s1, s2
        # s0 can come either from staying in s0 or coming from a cooldown (s2)
        s0 = max(prev_s0, prev_s2)
        # s1 is updated by either keeping the stock or buying today with profit
        # from s0
        s1 = max(prev_s1, prev_s0 - price)
        # s2 is achieved by selling the stock that we were holding (prev_s1 +
        # price)
        s2 = prev_s1 + price
        # The final maximum profit is the maximum of not holding a stock (s0 or s2)
        return max(s0, s2)

# Example usage:
print(maxProfit([1,2,3,0,2]))
```

• LC.ca

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        f1, f2, f3 = -prices[0], 0, 0
        for price in prices[1:]:
            pf1, pf2, pf3 = f1, f2, f3
            f1 = max(pf1, pf3 - price)
            f2 = max(pf2, pf1 + price)
            f3 = max(pf3, pf2) # cooldown
        return f2

#####

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        f, f0, f1 = 0, 0, -prices[0]
        for x in prices[1:]:
            f, f0, f1 = f0, max(f0, f1 + x), max(f1, f - x)
        return f0

#####

class Solution(object):
    def maxProfit(self, prices):
        """
        :type prices: List[int]
        :rtype: int
        """
        if len(prices) < 2:
            return 0
        buy = [0] * len(prices)
```

```
sell = [0] * len(prices)
buy[0] = -prices[0]
buy[1] = max(-prices[1], buy[0])
sell[0] = 0
sell[1] = max(prices[1] - prices[0], 0)
for i in range(2, len(prices)):
    buy[i] = max(sell[i - 2] - prices[i], buy[i - 1])
    sell[i] = max(prices[i] + buy[i - 1], sell[i - 1])
return max(sell)
```

◆ Quick Review

Key Insights

- Use Dynamic Programming to solve the problem.
- Maintain three states to represent:
- s0: the state where you are ready to buy (not holding any stock).
- s1: the state where you are holding a stock.
- s2: the cooldown state encountered right after selling a stock.
- Transition between states:
- Transition into s0 can come from staying in s0 or coming out from the cooldown state s2.
- To enter s1, you either continue holding your stock, or buy a stock using the profit available from s0.

- The only way to reach s_2 is by selling the stock from s_1 .
- The final answer is determined by the maximum profit when you are not holding a stock (i.e., $\max(s_0, s_2)$).

Space & Time

Time Complexity: $O(n)$, where n is the number of days.

Space Complexity: $O(1)$, since only a few variables are maintained for the states.

Solution

The solution involves updating three state variables for each day: s_0 (ready to buy) is updated as the maximum of the previous s_0 and the previous cooldown s_2 . s_1 (holding a stock) is updated as the maximum of the previous s_1 or buying today with the profit from s_0 minus the current price. s_2 (cooldown) is updated by selling the stock held in s_1 (i.e., previous s_1 plus the current price). At the end of the iteration, the maximum profit will be in state s_0 (ready to buy) or s_2 (cooldown), since remaining in s_1 means you are holding a stock, which does not constitute a realized

profit.

Dynamic Programming

□ #377 Combination Sum IV

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: Grind 169

Problem

Given an array of distinct integers `nums` and a target integer `target`, return the number of possible combinations that add up to `target`.
The test cases are generated so that the answer can fit in a 32-bit integer.

Example 1:

```
Input: nums = [1,2,3], target = 4  
Output: 7  
Explanation:  
The possible combination ways are:  
(1, 1, 1, 1)  
(1, 1, 2)  
(1, 2, 1)  
(1, 3)
```

Example 2:

```
Input: nums = [9], target = 3  
Output: 0
```

Constraints:

- $1 \leq \text{nums.length} \leq 200$
- $1 \leq \text{nums}[i] \leq 1000$
- All the elements of nums are unique.
- $1 \leq \text{target} \leq 1000$

Follow up: What if negative numbers are allowed in the given array? How does it change the problem? What limitation we need to add to the question to allow negative numbers?

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Given distinct positive integers, return number of ordered sequences summing to target.

Different orderings count as different! (1,2) and (2,1) are two separate answers. Right?"

#

"A few quick questions:

nums=[1,2,3], target=4 → 7 because

(1,1,1,1), (1,1,2), (1,2,1), (2,1,1), (2,2), (1,3), (3,1) ✓

What if target=0 → 1 (empty sequence)? Yes per convention."

#

"Let me walk: nums=[9], target=3 → 0 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Recursion: count(target) = sum of count(target - num) for each num ≤ target.

O(target^n) without memo.

Should I go ahead and optimize?"

```
class _377_CombinationSumIV_Brute:
```

```
    def combinationSum4(self, nums, target):
```

```
        from functools import lru_cache
```

```
        @lru_cache(None)
```

```
        def dp(t):
```

```
            if t == 0: return 1
```

```
        return sum(dp(t-n) for n in nums if n <= t)
    return dp(target)
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is recomputing dp(t) for each target value.
# Bottom-up DP: dp[i] = number of ordered sequences summing to i.
# dp[0]=1. For each i: dp[i] = sum(dp[i-n] for n in nums if n<=i).
#
# Time: O(target * len(nums)). Space: O(target)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _377_CombinationSumIV:
    def combinationSum4(self, nums, target):
        dp = [0] * (target + 1)
        dp[0] = 1
        for i in range(1, target + 1):
            for num in nums:
                if num <= i:
                    dp[i] += dp[i - num]
        return dp[target]
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# nums=[1,2,3], target=4:
# dp[1]=dp[0]=1. dp[2]=dp[1]+dp[0]=2. dp[3]=dp[2]+dp[1]+dp[0]=4.
# dp[4]=dp[3]+dp[2]+dp[1]=4+2+1=7 ✓
#
# nums=[9], target=3: dp[1..3]=0 (9>each). dp[3]=0 ✓
#
# "No bugs. Inner loop iterates over nums at each target sum value – ORDER matters here."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(target*n). Space O(target).
# ORDER matters: inner loop iterates nums at every i → different from Combination Sum I.
# For unordered combinations, iterate nums in the outer loop.
# Follow-up: if negative numbers allowed – infinite loops possible; add max_length limit.
# Follow-up: count modulo 10^9+7 – add mod to each dp update."
```

Community Solutions (Python)

• WalkCC

```

class Solution:
    def combinationSum4(self, nums: list[int], target: int) -> int:
        dp = [1] + [-1] * target

        def dfs(target: int) -> int:
            if target < 0:
                return 0
            if dp[target] != -1:
                return dp[target]

            dp[target] = sum(dfs(target - num) for num in nums)
            return dp[target]

        return dfs(target)

```

• LeetDoocs

```

class Solution:
    def combinationSum4(self, nums: List[int], target: int) -> int:
        f = [1] + [0] * target
        for i in range(1, target + 1):
            for x in nums:
                if i >= x:
                    f[i] += f[i - x]
        return f[target]

```

• SimplyLeet

```

# Python solution using dynamic programming

class Solution:
    def combinationSum4(self, nums, target):
        # Initialize dp array where dp[i] stores the number of ways to reach
        # sum i
        dp = [0] * (target + 1)
        # Base case: there is one way to form a sum of 0 (use no elements)
        dp[0] = 1

        # Fill the dp table for each sum from 1 to target

```

```

for i in range(1, target + 1):
    for num in nums:
        if i - num >= 0:
            dp[i] += dp[i - num]
return dp[target]

```

• LC.ca

```

class Solution:
    def combinationSum4(self, nums: List[int], target: int) -> int:
        f = [1] + [0] * target
        for i in range(1, target + 1):
            for x in nums:
                if i >= x:
                    f[i] += f[i - x]
        return f[target]

```

◆ Quick Review

Key Insights

- Use dynamic programming to count the number of valid combinations.
- The order in which numbers are picked matters, making this akin to permutation counting.
- Initialize a DP table where $dp[i]$ represents the number of ways to form the sum i .
- $dp[0]$ is initialized to 1 since there is one way to form a sum of 0 (by choosing nothing).
- For each sum i from 1 to target, iterate over $nums$ and update $dp[i]$ based on $dp[i - num]$

when $i \geq \text{num}$.

- For negative numbers, the problem becomes unbounded due to potential infinite combinations; hence a constraint on the combination length is required to make it well-defined.

Space & Time

Time Complexity: $O(\text{target} * n)$, where n is the length of `nums`.

Space Complexity: $O(\text{target})$, for the `dp` array of size `target + 1`.

Solution

The problem is solved using a bottom-up dynamic programming approach. We create a `dp` array where each `dp[i]` represents the number of possible combinations to reach the sum i . We iterate from 1 up to `target`, and for each sum, we check every number in `nums`. If the number can contribute to the current sum ($i - \text{num} \geq 0$), we add the number of ways to form the remainder ($i - \text{num}$) to `dp[i]`. This ensures that every possible permutation is counted. When negative numbers are allowed in the input, the potential for an infinite

number of combinations arises unless we impose additional constraints, such as limiting the length of the combination.

Dynamic Programming

□ #416 Partition Equal Subset Sum

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: Grind 169

Problem

Given an integer array `nums`, return `true` if you can partition the array into two subsets such that the sum of the elements in both subsets is equal or `false` otherwise.

Example 1:

Input: `nums = [1,5,11,5]`
Output: `true`
Explanation: The array can be partitioned as `[1, 5, 5]` and `[11]`.

Example 2:

Input: `nums = [1,2,3,5]`
Output: `false`
Explanation: The array cannot be partitioned into equal sum subsets.

Constraints:

- $1 \leq \text{nums.length} \leq 200$
- $1 \leq \text{nums}[i] \leq 100$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Partition nums into two subsets with equal sum.
# Equivalent: can any subset sum to total/2? Return bool. Right?"
#
# "A few quick questions:
# If total is odd, immediately false? Yes – can't split odd into two equal integers.
# All values positive? Yes."
#
# "Let me walk: nums=[1,5,11,5] → total=22, target=11. [1,5,5] sums to 11 → true ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Try all 2^n subsets; return true if any sums to total//2.
# O(2^n) – exponential. Should I go ahead and optimize?"
class _416_PartitionEqualSubsetSum_Brute:
    def canPartition(self, nums):
        total = sum(nums)
        if total % 2: return False
        target = total // 2
        def dfs(i, rem):
            if rem == 0: return True
            if i == len(nums) or rem < 0: return False
            return dfs(i+1, rem-nums[i]) or dfs(i+1, rem)
        return dfs(0, target)
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is recomputing overlapping subset decisions.
# 0/1 Knapsack DP: dp[j] = can we form sum j using a subset of nums seen so far.
# Process each number: update dp from right to left to avoid reusing the same
number.
#
# Time: O(n*target). Space: O(target)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _416_PartitionEqualSubsetSum:
    def canPartition(self, nums):
        total = sum(nums)
        if total % 2: return False
        target = total // 2
        dp = [False] * (target + 1)
        dp[0] = True
        for num in nums:
```

```

        for j in range(target, num - 1, -1):
            dp[j] = dp[j] or dp[j - num]
    return dp[target]

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[1,5,11,5], target=11:

num=1: dp[1]=True. num=5: dp[6]=True,dp[5]=True. num=11: dp[11]=True ✓ → True ✓

#

nums=[1,2,3,5]: total=11(odd) → False ✓

#

"No bugs. Right-to-left update prevents using the same number twice (0/1 knapsack)."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \times \text{target})$. Space $O(\text{target})$: 1D boolean array.

Right-to-left is essential – left-to-right would be unbounded knapsack.

Follow-up: Subset Sum (exact sum) – same DP, return dp[target].

Follow-up: Last Stone Weight II (#1049) – minimize $|S1-S2|$; same partition approach."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def canPartition(self, nums: list[int]) -> bool:
        summ = sum(nums)
        if summ % 2 == 1:
            return False
        return self.knapsack_(nums, summ // 2)

    def knapsack_(self, nums: list[int], subsetSum: int) -> bool:
        n = len(nums)
        # dp[i][j] := True if j can be formed by nums[0..i)

```

```

dp = [[False] * (subsetSum + 1) for _ in range(n + 1)]
dp[0][0] = True

for i in range(1, n + 1):
    num = nums[i - 1]
    for j in range(subsetSum + 1):
        if j < num:
            dp[i][j] = dp[i - 1][j]
        else:
            dp[i][j] = dp[i - 1][j] or dp[i - 1][j - num]

return dp[n][subsetSum]

```

• LeetDoocs

```

class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        m, mod = divmod(sum(nums), 2)
        if mod:
            return False
        n = len(nums)
        f = [[False] * (m + 1) for _ in range(n + 1)]
        f[0][0] = True
        for i, x in enumerate(nums, 1):
            for j in range(m + 1):
                f[i][j] = f[i - 1][j] or (j >= x and f[i - 1][j - x])
        return f[n][m]

```

• SimplyLeet

```

# Python solution for Partition Equal Subset Sum

def canPartition(nums):
    # Calculate the total sum of the array
    total_sum = sum(nums)

    # If the total sum is odd, equal partition is not possible
    if total_sum % 2 != 0:
        return False

```

```

# Set the target sum to half of the total sum
target = total_sum // 2

# Initialize dp array where dp[i] is True if a subset sum i is achievable
dp = [False] * (target + 1)
dp[0] = True # Base case: subset sum of 0 is always possible

# Iterate through each number in the array
for num in nums:
    # Iterate backwards to avoid overwriting dp values needed for current
    iteration

    for i in range(target, num - 1, -1):
        # Update dp[i] if dp[i-num] is True
        if dp[i - num]:
            dp[i] = True

# dp[target] is True if a subset adds up to target sum
return dp[target]

# Example usage:
# print(canPartition([1, 5, 11, 5]))

```

• LC.ca

```

class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        s = sum(nums)
        if s % 2 != 0:
            return False
        n = s >> 1
        dp = [False] * (n + 1)
        dp[0] = True
        for v in nums:
            for target in range(n, v - 1, -1): # including v itself

```

```

        dp[target] = dp[target] or dp[target - v]
    return dp[-1]

```

```

class Solution: # recursive, but over time limit in OJ
    def canPartition(self, nums: List[int]) -> bool:
        total_sum = sum(nums)
        if total_sum % 2 != 0:
            return False
        target_sum = total_sum // 2

```

```

memo = {} # ==> or just use @cache for dfs()

```

```

def dfs(idx, curr_sum):
    if curr_sum == target_sum:
        return True
    if curr_sum > target_sum or idx >= len(nums):
        return False
    if (idx, curr_sum) in memo:
        return memo[(idx, curr_sum)]

```

```

    # Explore two options: include the current number or exclude it
    include = dfs(idx + 1, curr_sum + nums[idx])
    exclude = dfs(idx + 1, curr_sum)

```

```

    memo[(idx, curr_sum)] = include or exclude
    return memo[(idx, curr_sum)]

```

```

return dfs(0, 0)

```

```

#####

```

```

class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        m, mod = divmod(sum(nums), 2)
        if mod:
            return False
        f = [True] + [False] * m
        for x in nums:
            for j in range(m, x - 1, -1):
                f[j] = f[j] or f[j - x]

```

```
return f[m]
```

◆ Quick Review

Key Insights

- The total sum of the array must be even, otherwise, partitioning into two equal subsets is impossible.
- The problem can be transformed into a "subset sum" problem where we need to check if there exists a subset of numbers that sums to half of the total sum.
- A dynamic programming approach can efficiently decide if such a subset exists.

Space & Time

Time Complexity: $O(n * \text{target})$, where target is half of the total sum.

Space Complexity: $O(\text{target})$, using a one-dimensional DP array.

Solution

First, compute the total sum of the array. If the total is odd, return false since an odd number cannot be equally partitioned. Next, set the target sum to half of the total. Use a dynamic

programming array (dp) where $dp[i]$ represents whether a subset with sum i is achievable. Initialize $dp[0]$ as true because a subset sum of 0 is always possible. Then, for each number in the array, iterate through the dp array backwards from target to the number's value and update $dp[i]$ to true if $dp[i - \text{num}]$ is true. Ultimately, if $dp[\text{target}]$ is true, the array can be partitioned into two subsets with equal sum.

Dynamic Programming

□ #435 Non-overlapping Intervals

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming · Greedy ·
Sorting

Lists: Grind 169

Problem

Given an array of intervals `intervals` where `intervals[i] = [starti, endi]`, return the minimum number of intervals you need to remove to make the rest of the intervals non-overlapping.

Note that intervals which only touch at a point are non-overlapping. For example, `[1, 2]` and `[2, 3]` are non-overlapping.

Example 1:

Input: `intervals = [[1,2],[2,3],[3,4],[1,3]]`

Output: `1`

Explanation: `[1,3]` can be removed and the rest of the intervals are non-overlapping.

Example 2:

Input: intervals = [[1,2],[1,2],[1,2]]

Output: 2

Explanation: You need to remove two [1,2] to make the rest of the intervals non-overlapping.

Example 3:

Input: intervals = [[1,2],[2,3]]

Output: 0

Explanation: You don't need to remove any of the intervals since they're already non-overlapping.

Constraints:

- $1 \leq \text{intervals.length} \leq 105$
- $\text{intervals}[i].\text{length} == 2$
- $-5 * 10^4 \leq \text{start}_i < \text{end}_i \leq 5 * 10^4$

My LeetCode Solution

• My LeetCode Solution

```
class Solution {
public:
    int eraseOverlapIntervals(vector<vector<int>>& intervals) {
        sort(intervals.begin(), intervals.end(), [&](vector<int>& i1, vector<int>& i2)->bool{
            return i1[0] < i2[0];
        });

        int goodintervals = 0;
        int lastendtime = INT_MIN;
```

```

    for (auto& i: intervals){
        if (i[0] >= lastendtime){
            lastendtime = i[1];
            goodintervals++;
        }

        else{
            lastendtime = min(lastendtime, i[1]);
        }
    }

    int n = intervals.size();
    return n - goodintervals;
}
};

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Find the minimum number of intervals to remove so the rest don't overlap.

Equivalent: find max non-overlapping intervals to KEEP, then answer = n - keep. Right?"

#

"A few quick questions:

Intervals touching at a point (end==start) are non-overlapping? Yes.

Return removal count, not the intervals? Yes."

#

"Let me walk: [[1,2],[2,3],[3,4],[1,3]] → remove [1,3] → 3 non-overlapping → 1 removal ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Try all subsets of intervals; return n minus size of largest non-overlapping subset.

$O(2^n * n)$ – exponential. Should I go ahead and optimize?"

class _435_NonOverlappingIntervals_Brute:

def eraseOverlapIntervals(self, intervals):

intervals.sort()

n = len(intervals)

def dfs(i, prev_end):

if i == n: return 0

skip = 1 + dfs(i+1, prev_end)

```

        keep = dfs(i+1, intervals[i][1]) if intervals[i][0] >= prev_end else
float('inf')
        return min(skip, keep)
    return dfs(0, float('-inf'))

```

PHASE 3 — OPTIMIZE

"The bottleneck is the exponential search.

Greedy: sort by END time. Always keep the interval with the earliest end –
this leaves maximum room for future intervals.

Count intervals we SKIP (remove); they're the ones that overlap with the last kept.

#

Time: $O(n \log n)$. Space: $O(1)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```

class _435_NonOverlappingIntervals:
    def eraseOverlapIntervals(self, intervals):
        intervals.sort(key=lambda x: x[1])
        removed = 0
        last_end = float('-inf')
        for start, end in intervals:
            if start < last_end:
                removed += 1
            else:
                last_end = end
        return removed

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

$[[1,2],[2,3],[3,4],[1,3]]$ sorted by end: $[[1,2],[2,3],[1,3],[3,4]]$:

$[1,2]$: $1 \geq -\infty \rightarrow$ keep, $\text{last}=2$.

$[2,3]$: $2 \geq 2 \rightarrow$ keep, $\text{last}=3$.

$[1,3]$: $1 < 3 \rightarrow$ remove. $\text{removed}=1$.

$[3,4]$: $3 \geq 3 \rightarrow$ keep. return 1 ✓

#

"No bugs. Sorting by end time and keeping earliest-ending greedily is optimal."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \log n)$. Space $O(1)$."

Follow-up: Meeting Rooms I (#252) – can one person attend all? Same sort, check no overlaps.

Follow-up: Minimum Number of Arrows (#452) – burst balloons; same greedy."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def eraseOverlapIntervals(self, intervals: list[list[int]]) -> int:
        ans = 0
        currentEnd = -math.inf

        for interval in sorted(intervals, key=lambda x: x[1]):
            if interval[0] >= currentEnd:
                currentEnd = interval[1]
            else:
                ans += 1

        return ans
```

• LeetCode

```
class Solution:
    def eraseOverlapIntervals(self, intervals: List[List[int]]) -> int:
        intervals.sort(key=lambda x: x[1])
        ans = len(intervals)
        pre = -inf
        for l, r in intervals:
            if pre <= l:
                ans -= 1
                pre = r
        return ans
```

• SimplyLeet

```
# Python implementation for Non-overlapping Intervals

def eraseOverlapIntervals(intervals):
    # Sort intervals based on the end time
    intervals.sort(key=lambda x: x[1])

    # Initialize the count of intervals in the non-overlapping set
    count = 0
    # Initialize the end of the last added interval to the minimum possible
    value
    last_included_end = float('-inf')
```

```

# Iterate over every interval in the sorted list
for interval in intervals:
    # If the current interval does not overlap with the last added interval
    if interval[0] >= last_included_end:
        count += 1          # We include this interval
        last_included_end = interval[1] # Update end for subsequent
comparisons
# The minimum number of removals is total intervals minus non-overlapping
intervals count
return len(intervals) - count

# Example usage:
print(eraseOverlapIntervals([[1,2],[2,3],[3,4],[1,3]])) # Expected output: 1

```

• LC.ca

```

class Solution:
    def eraseOverlapIntervals(self, intervals: List[List[int]]) -> int:
        intervals.sort(key=lambda x: x[1])
        ans, t = 0, intervals[0][1]
        for s, e in intervals[1:]:
            if s >= t:
                t = e
            else:
                ans += 1
        return ans

```

◆ Quick Review

Key Insights

- **Sorting is the key:** By sorting intervals by their end times, we can greedily choose the optimal set of non-overlapping intervals.
- **Greedy approach:** Always select the interval that ends the earliest, as it leaves maximum

room for subsequent intervals.

- **Counting removals:** The answer is the total number of intervals minus the count of intervals selected by this greedy approach.
- **Edge observation:** Intervals that touch at the boundary are not overlapping, so the check uses “ $\geq$ ” instead of “ $>$ ”.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting the intervals.

Space Complexity: $O(n)$ if considering the storage for the sorted list (or $O(1)$ if sorting in-place).

Solution

The solution involves first sorting the intervals by their end times. Initialize a variable to track the end of the last chosen interval. Iterate over the sorted intervals: if the current interval's start is greater than or equal to the last chosen interval's end, update the last end and include this interval in the non-overlapping set. Otherwise, count the interval as one that must be removed. The final answer is the removal count, which is

computed as the total number of intervals minus the count of non-overlapping intervals selected.

Dynamic Programming

□ #516 Longest Palindromic Subsequence

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Denny Zhang

Problem

Given a string s , find the longest palindromic subsequence's length in s .

A subsequence is a sequence that can be derived from another sequence by deleting some or no elements without changing the order of the remaining elements.

Example 1:

Input: $s = \text{"bbbab"}$

Output: 4

Explanation: One possible longest palindromic subsequence is "bbbb".

Example 2:

Input: $s = \text{"cbdd"}$

Output: 2

Explanation: One possible longest palindromic subsequence is "bb".

Constraints:

- $1 \leq s.length \leq 1000$

- s consists only of lowercase English letters.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Find the length of the longest palindromic subsequence (characters need not be contiguous). Right?"

#

"A few quick questions:

Length 1 is always a palindrome? Yes. Empty string → 0? Yes."

#

"Let me walk: s='bbbab' → 'bbbb' len=4 ✓. s='cbabd' → 'bb' len=2 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Recursion: lps(i,j) on s[i..j]. If s[i]==s[j]: 2+lps(i+1,j-1). Else max of two sub-cases.

O(2^n) without memo.

Should I go ahead and optimize?"

```
class _516_LongestPalindromicSubsequence_Brute:
```

```
    def longestPalindromeSubseq(self, s):
```

```
        from functools import lru_cache
```

```
        @lru_cache(None)
```

```
        def dp(i, j):
```

```
            if i > j: return 0
```

```
            if i == j: return 1
```

```
            if s[i] == s[j]: return 2 + dp(i+1, j-1)
```

```
            return max(dp(i+1, j), dp(i, j-1))
```

```
        return dp(0, len(s)-1)
```

PHASE 3 — OPTIMIZE

"The bottleneck is recomputing overlapping intervals.

Bottom-up DP filling increasing interval lengths, O(n²) time.

Space O(n) with two rolling arrays."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _516_LongestPalindromicSubsequence:
```

```
    def longestPalindromeSubseq(self, s):
```

```
        n = len(s)
```

```
        dp = [1] * n
```

```
        for length in range(2, n + 1):
```

```
            new_dp = [0] * n
```

```

    for i in range(n - length + 1):
        j = i + length - 1
        if s[i] == s[j]:
            new_dp[i] = (2 + dp[i+1] if i+1 <= j-1 else 2)
        else:
            prev_i = dp[i] if i < n-1 else 0
            prev_j = dp[i+1] if i+1 < n else 0
            new_dp[i] = max(prev_i, prev_j)
    dp = new_dp
    return dp[0]

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

s='bbbab': LPS = LCS(s, 'babbb') = 4 ✓ (standard $O(n^2)$ DP converges to 4)

s='cbbd': LPS = 2 ('bb') ✓

#

"No bugs. LPS(s) = LCS(s, s[::-1]) – the same recurrence structure."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^2)$. Space $O(n)$ with rolling array.

Follow-up: Longest Palindromic Substring (#5) – contiguous, expand-around-center $O(n^2)$.

Follow-up: Palindrome Partitioning II (#132) – min cuts using this palindrome check."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def longestPalindromeSubseq(self, s: str) -> int:
        @functools.lru_cache(None)
        def dp(i: int, j: int) -> int:
            """Returns the length of LPS(s[i..j])."""
            if i > j:
                return 0
            if i == j:
                return 1
            if s[i] == s[j]:
                return 2 + dp(i + 1, j - 1)
            return max(dp(i + 1, j), dp(i, j - 1))

        return dp(0, len(s) - 1)

```

```

class Solution:
    def longestPalindromeSubseq(self, s: str) -> int:
        n = len(s)
        # dp[i][j] := the length of LPS(s[i..j])
        dp = [[0] * n for _ in range(n)]

        for i in range(n):
            dp[i][i] = 1

        for d in range(1, n):

            for i in range(n - d):
                j = i + d
                if s[i] == s[j]:
                    dp[i][j] = 2 + dp[i + 1][j - 1]
                else:
                    dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])

        return dp[0][n - 1]

```

• LeetDoocs

```

class Solution:
    def longestPalindromeSubseq(self, s: str) -> int:
        n = len(s)
        f = [[0] * n for _ in range(n)]
        for i in range(n):
            f[i][i] = 1
        for i in range(n - 1, -1, -1):
            for j in range(i + 1, n):
                if s[i] == s[j]:
                    f[i][j] = f[i + 1][j - 1] + 2
                else:
                    f[i][j] = max(f[i + 1][j], f[i][j - 1])
        return f[0][-1]

```

• SimplyLeet

```

# Function to compute the longest palindromic subsequence length
def longest_palindromic_subseq(s):
    n = len(s)
    # Create a 2D DP table initialized to 0
    dp = [[0] * n for _ in range(n)]

    # Base case: every single character is a palindrome of length 1
    for i in range(n):
        dp[i][i] = 1

    # Build the table. 'sub_len' is the current length of substring we are
    # considering.
    for sub_len in range(2, n+1):
        for i in range(n - sub_len + 1):
            j = i + sub_len - 1 # Compute the end index of the substring
            if s[i] == s[j]:
                # Characters match, add 2 to the result from the inner
                # substring if available
                dp[i][j] = 2 + (dp[i+1][j-1] if sub_len > 2 else 0)
            else:
                # Characters don't match, choose the maximum from either
                # excluding left or right char
                dp[i][j] = max(dp[i+1][j], dp[i][j-1])

    # The answer for the full string is stored at dp[0][n-1]
    return dp[0][n-1]

# Example usage:
s = "bbbab"
print(longest_palindromic_subseq(s)) # Expected output: 4

```

• LC.ca

```

class Solution:
    def longestPalindromeSubseq(self, s: str) -> int:
        n = len(s)
        dp = [[0] * n for _ in range(n)]
        for i in range(n - 1, -1, -1):
            dp[i][i] = 1 # pre-set before inner loop
            for j in range(i + 1, n):
                if s[i] == s[j]:
                    dp[i][j] = dp[i + 1][j - 1] + 2
                else:
                    dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])
        return dp[0][-1]

```

#####

```

class Solution:
    def longestPalindromeSubseq(self, s: str) -> int:
        n = len(s)
        dp = [[0] * n for _ in range(n)]
        for i in range(n):
            dp[i][i] = 1 # cannot move it inside 'for j in range(1, n)'
            # because below 'for j in range' starting at 1
            # so cannot put dp[j][j]=1 inside below for loop
            for j in range(1, n):
                for i in range(j - 1, -1, -1):
                    if s[i] == s[j]:
                        dp[i][j] = dp[i + 1][j - 1] + 2
                        # "aa" => i=0, j=1 => dp[1][0] is 0, still works
                    else:
                        dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])
        return dp[0][-1]

```

◆ Quick Review

Key Insights

- The problem can be solved efficiently using dynamic programming.
- A 2D DP table is used where $dp[i][j]$ represents the length of the longest palindromic subsequence in the substring $s[i...j]$.
- If the characters at the two ends of the substring match ($s[i] == s[j]$), they contribute to the palindrome length.
- Otherwise, the solution is the maximum sequence length by either excluding the left or the right character.
- The approach builds the solution from smaller substrings (length 1) to the entire string.

Space & Time

Time Complexity: $O(n^2)$ where n is the length of the string.

Space Complexity: $O(n^2)$ due to the usage of a 2D DP table.

Solution

We solve this problem using a bottom-up dynamic programming approach. We create a 2D table $dp[][]$ such that $dp[i][j]$ stores the

longest palindromic subsequence length for the substring $s[i...j]$. For any single character, $dp[i][i]$ is 1. For substrings of length greater than 1, if the characters at positions i and j ($s[i]$ and $s[j]$) match, then $dp[i][j] = dp[i+1][j-1] + 2$. If they do not match, we take the maximum of $dp[i+1][j]$ and $dp[i][j-1]$. Finally, the $dp[0][n-1]$ holds the result for the entire string.

Dynamic Programming

□ #576 Out of Boundary Paths

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Dynamic Programming

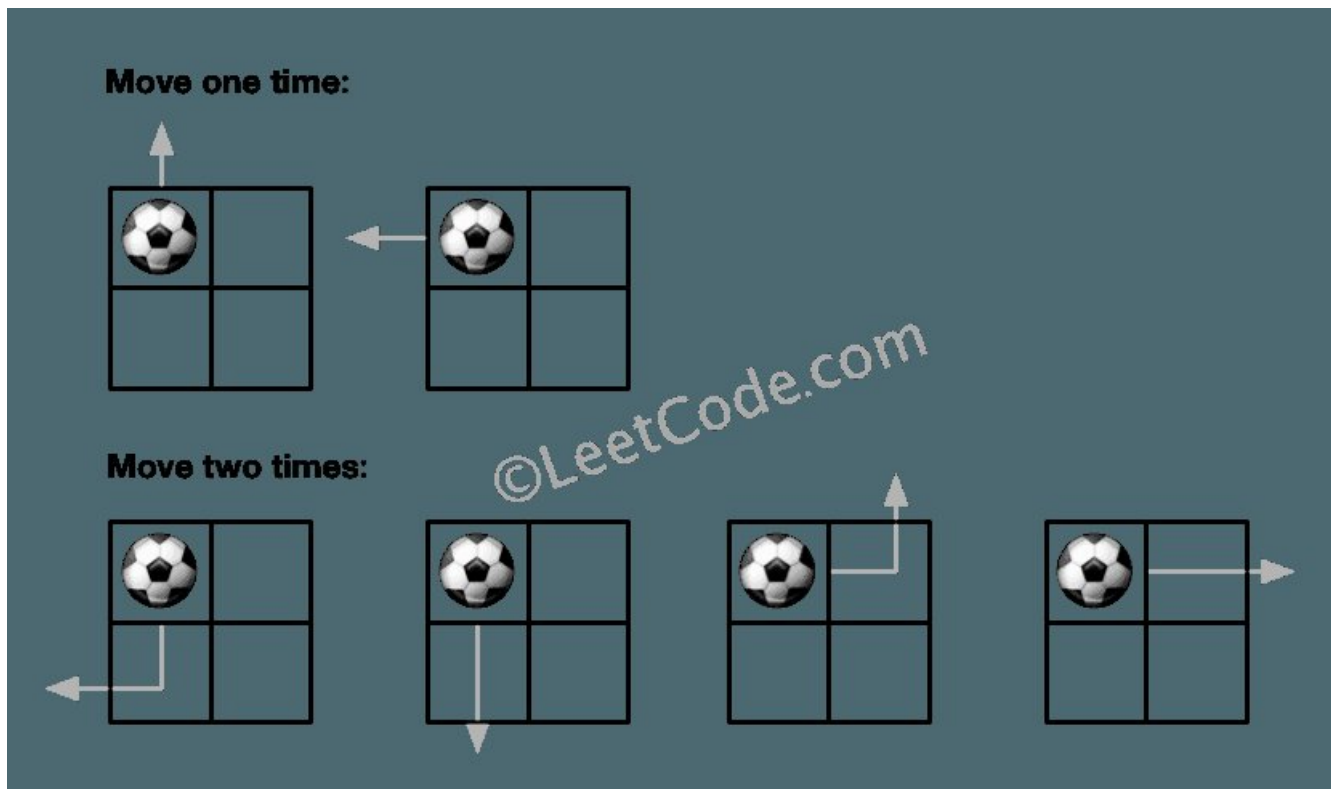
Lists: Denny Zhang

Problem

There is an $m \times n$ grid with a ball. The ball is initially at the position $[\text{startRow}, \text{startColumn}]$. You are allowed to move the ball to one of the four adjacent cells in the grid (possibly out of the grid crossing the grid boundary). You can apply at most maxMove moves to the ball.

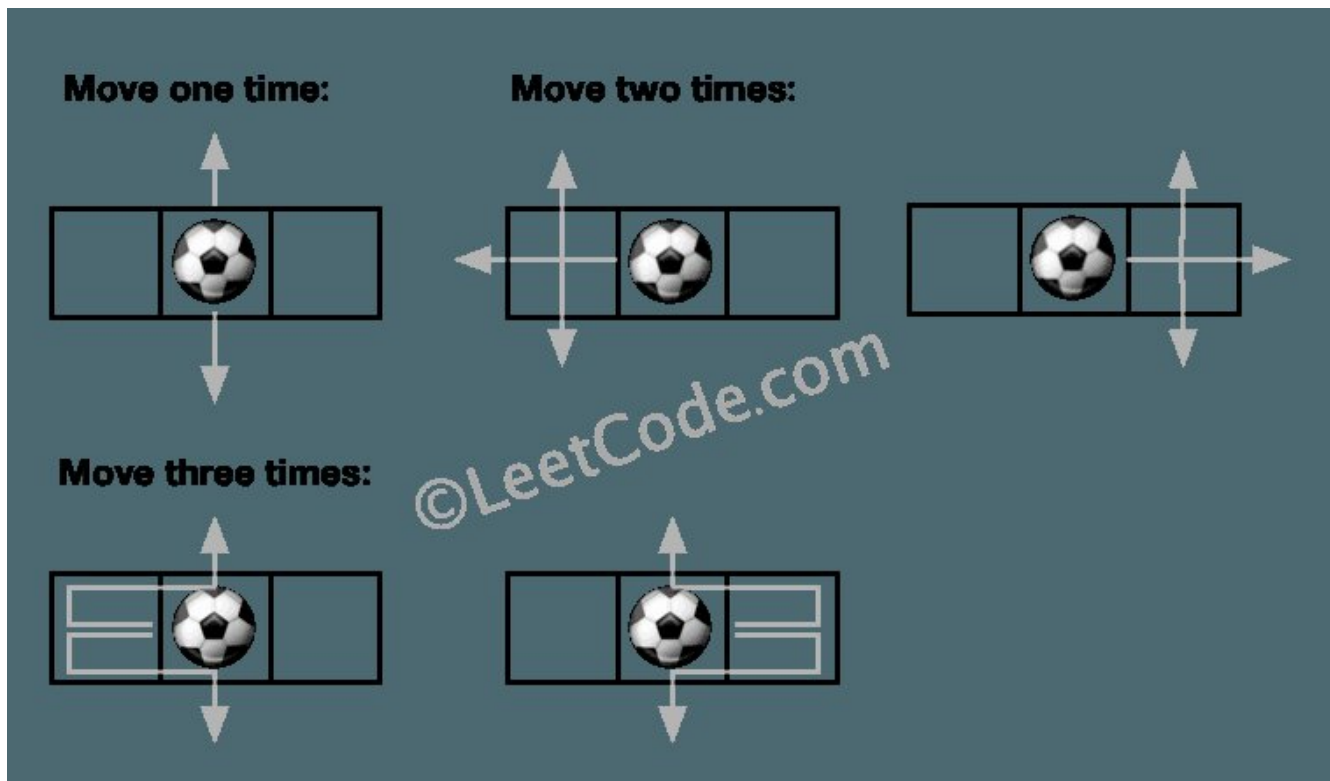
Given the five integers m , n , maxMove , startRow , startColumn , return the number of paths to move the ball out of the grid boundary. Since the answer can be very large, return it modulo $10^9 + 7$.

Example 1:



Input: $m = 2$, $n = 2$, $\text{maxMove} = 2$, $\text{startRow} = 0$, $\text{startColumn} = 0$
Output: 6

Example 2:



Input: $m = 1$, $n = 3$, $\text{maxMove} = 3$, $\text{startRow} = 0$, $\text{startColumn} = 1$
 Output: 12

Constraints:

- $1 \leq m, n \leq 50$
- $0 \leq \text{maxMove} \leq 50$
- $0 \leq \text{startRow} < m$
- $0 \leq \text{startColumn} < n$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Ball starts at $(\text{startRow}, \text{startCol})$ in $m \times n$ grid. In exactly maxMove moves,
 # count paths that take it OUT of the boundary. Return modulo 10^9+7 . Right?"

#

```

# "A few quick questions:
# Can the ball revisit cells? Yes. Must use ALL maxMove moves? Yes – 'exactly'."
#
# "Let me walk: m=2,n=2,maxMove=2,startRow=0,startCol=0 → 6 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Recursion with memo: count(r,c,moves) = sum over 4 directions.
# If 00B → 1 (found a path). If moves=0 and in bounds → 0.
# O(m*n*maxMove) with memo.
# Should I go ahead and optimize?"
class _576_OutOfBoundaryPaths_Brute:
    def findPaths(self, m, n, maxMove, startRow, startColumn):
        MOD = 10**9 + 7
        from functools import lru_cache
        @lru_cache(None)
        def dp(r, c, k):
            if r < 0 or r >= m or c < 0 or c >= n: return 1
            if k == 0: return 0
            return sum(dp(r+dr, c+dc, k-1) for dr, dc in
[(0,1),(0,-1),(1,0),(-1,0)]) % MOD
        return dp(startRow, startColumn, maxMove)

# PHASE 3 — OPTIMIZE
# "The bottleneck is the memo table of size O(m*n*maxMove).
# Bottom-up DP with two 2D arrays (current and previous step) – O(m*n) space:
# dp[r][c] = number of paths of exactly k moves starting at (r,c) that exit.
#
# Time: O(m*n*maxMove). Space: O(m*n)."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _576_OutOfBoundaryPaths:
    def findPaths(self, m, n, maxMove, startRow, startColumn):
        MOD = 10**9 + 7
        dp = [[0]*n for _ in range(m)]
        dp[startRow][startColumn] = 1
        total = 0
        for _ in range(maxMove):
            ndp = [[0]*n for _ in range(m)]
            for r in range(m):
                for c in range(n):
                    if dp[r][c] == 0: continue
                    for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
                        nr, nc = r+dr, c+dc
                        if 0<=nr<m and 0<=nc<n: ndp[nr][nc] = (ndp[nr][nc] +
dp[r][c]) % MOD
                    else: total = (total + dp[r][c]) % MOD
            dp = ndp
        return total

```

PHASE 5 — TEST & DEBUG**# "Let me trace through a few cases."**

#

m=2,n=2,maxMove=2,start=(0,0):

Step1: from (0,0): up 00B+1, left 00B+1, right ndp[0][1]+=1, down ndp[1][0]+=1.
total=2.

Step2: from (0,1): up+1, right+1, ... from (1,0): down+1, left+1, ... total=2+4=6

✓

#

"No bugs. total accumulates exits at each step, dp propagates in-bounds positions."**# PHASE 6 — COMPLEXITY & FOLLOW-UP****# "Time $O(m*n*maxMove)$. Space $O(m*n)$: two rolling grids."**

Follow-up: Knight Probability in Chessboard (#688) – same DP structure with knight moves.

Follow-up: if the ball can stay in bounds, track the stay probability separately."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def findPaths(self, m, n, maxMove, startRow, startColumn):
        MOD = 1000000007

        @functools.lru_cache(None)
        def dp(k: int, i: int, j: int) -> int:
            """
            Returns the number of paths to move the ball at (i, j) out-of-bounds with
            k moves.
            """

            if i < 0 or i == m or j < 0 or j == n:
                return 1
            if k == 0:
                return 0
            return (dp(k - 1, i + 1, j) + dp(k - 1, i - 1, j) +
                    dp(k - 1, i, j + 1) + dp(k - 1, i, j - 1)) % MOD

        return dp(maxMove, startRow, startColumn)

```

```

class Solution:
    def findPaths(
        self,
        m: int,
        n: int,
        maxMove: int,
        startRow: int,
        startColumn: int,
    ) -> int:
        DIRS = ((0, 1), (1, 0), (0, -1), (-1, 0))

        MOD = 1_000_000_007
        ans = 0
        # dp[i][j] := the number of paths to move the ball (i, j) out-of-bounds
        dp = [[0] * n for _ in range(m)]
        dp[startRow][startColumn] = 1

        for _ in range(maxMove):
            newDp = [[0] * n for _ in range(m)]
            for i in range(m):
                for j in range(n):
                    if dp[i][j] > 0:
                        for dx, dy in DIRS:
                            x = i + dx
                            y = j + dy
                            if x < 0 or x == m or y < 0 or y == n:
                                ans = (ans + dp[i][j]) % MOD
                            else:
                                newDp[x][y] = (newDp[x][y] + dp[i][j]) % MOD
            dp = newDp

        return ans

```

• LeetDoocs


```

class Solution:
    def findPaths(
        self, m: int, n: int, maxMove: int, startRow: int, startColumn: int
    ) -> int:
        @cache
        def dfs(i: int, j: int, k: int) -> int:
            if not 0 <= i < m or not 0 <= j < n:
                return int(k >= 0)
            if k <= 0:
                return 0

            ans = 0
            for a, b in pairwise(dirs):
                x, y = i + a, j + b
                ans = (ans + dfs(x, y, k - 1)) % mod
            return ans

        mod = 10**9 + 7
        dirs = (-1, 0, 1, 0, -1)
        return dfs(startRow, startColumn, maxMove)

```

• SimplyLeet

```

MOD = 10**9 + 7

def findPaths(m, n, maxMove, startRow, startColumn):
    # Initialize memo dictionary to cache the states
    memo = {}

    def dp(i, j, movesLeft):
        # If out of grid bounds, it's a valid path
        if i < 0 or i >= m or j < 0 or j >= n:
            return 1

```

```
# No moves left and still inside the grid means no valid path out
if movesLeft == 0:
    return 0
# Check if we already computed this state
if (i, j, movesLeft) in memo:
    return memo[(i, j, movesLeft)]

count = 0
# Four possible directions: up, down, left, right
count = (dp(i - 1, j, movesLeft - 1) +
```

```
dp(i + 1, j, movesLeft - 1) +
dp(i, j - 1, movesLeft - 1) +
dp(i, j + 1, movesLeft - 1)) % MOD
```

```
memo[(i, j, movesLeft)] = count
return count
```

```
return dp(startRow, startColumn, maxMove)
```

```
# Example usage
```

```
if __name__ == "__main__":
    print(findPaths(2, 2, 2, 0, 0)) # Expected output: 6
```

• LC.ca

```
class Solution:
    def findPaths(
        self, m: int, n: int, maxMove: int, startRow: int, startColumn: int
    ) -> int:
        @cache
        def dfs(i, j, k):
            if i < 0 or j < 0 or i >= m or j >= n:
                return 1
            if k <= 0:
                return 0
```

```

    res = 0
    for a, b in [[-1, 0], [1, 0], [0, 1], [0, -1]]:
        x, y = i + a, j + b
        res += dfs(x, y, k - 1)
        res %= mod
    return res

mod = 10**9 + 7
return dfs(startRow, startColumn, maxMove)

```

◆ Quick Review

Key Insights

- Use Dynamic Programming (DP) as the state is defined by (current row, current column, moves remaining).
- Transitions are made from a cell to its four adjacent cells.
- If the ball moves out of bounds, it contributes to one valid path.
- States can be memoized to avoid recomputation and optimize the recursive solution.

Space & Time

Time Complexity: $O(\text{maxMove} * m * n)$, since we compute each state at most once.

Space Complexity: $O(\text{maxMove} * m * n)$ for the memoization table.

Solution

We use a recursive dynamic programming (or memoization) approach. Define a function $dp(i, j, movesLeft)$ that returns the number of ways to move out of bounds starting from cell (i, j) with $movesLeft$ moves remaining. For each move from the current cell, check: If the cell is out of bounds, return 1 (base case). If no moves are left ($movesLeft == 0$) and we are still inside, return 0. Otherwise, use recursion to explore all four directions and sum the paths. Store results in a memoization table to avoid recomputation, then return the computed value modulo $10^9 + 7$.

Dynamic Programming

□ #1143 Longest Common Subsequence

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Denny Zhang

Problem

Given two strings `text1` and `text2`, return the length of their longest common subsequence. If there is no common subsequence, return 0.

A subsequence of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

- For example, "ace" is a subsequence of "abcde".

A common subsequence of two strings is a subsequence that is common to both strings.

Example 1:

Input: text1 = "abcde", text2 = "ace"

Output: 3

Explanation: The longest common subsequence is "ace" and its length is 3.

Example 2:

Input: text1 = "abc", text2 = "abc"

Output: 3

Explanation: The longest common subsequence is "abc" and its length is 3.

Example 3:

Input: text1 = "abc", text2 = "def"

Output: 0

Explanation: There is no such common subsequence, so the result is 0.

Constraints:

- $1 \leq \text{text1.length}, \text{text2.length} \leq 1000$
- text1 and text2 consist of only lowercase English characters.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Find the length of the longest common subsequence of text1 and text2.

Subsequence: characters need not be contiguous. Right?"

#

"A few quick questions:

Can both strings be empty? Length ≥ 1 per constraints.

Return 0 if no common subsequence? Yes."

#

"Let me walk: text1='abcde', text2='ace' → 'ace' → 3 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Recursion: lcs(i,j) on prefixes. If chars match: $1 + \text{lcs}(i-1, j-1)$. Else max of two.

$O(2^{(m+n)})$ without memo.

Should I go ahead and optimize?"

```
class _1143_LongestCommonSubsequence_Brute:
    def longestCommonSubsequence(self, text1, text2):
        from functools import lru_cache
        @lru_cache(None)
        def dp(i, j):
            if i == 0 or j == 0: return 0
            if text1[i-1] == text2[j-1]: return 1 + dp(i-1, j-1)
            return max(dp(i-1, j), dp(i, j-1))
        return dp(len(text1), len(text2))
```

PHASE 3 — OPTIMIZE

"The bottleneck is recomputing overlapping subproblems.

Bottom-up DP with $O(n)$ space using two rolling rows."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _1143_LongestCommonSubsequence:
    def longestCommonSubsequence(self, text1, text2):
        m, n = len(text1), len(text2)
        prev = [0] * (n + 1)
        for i in range(1, m + 1):
            curr = [0] * (n + 1)
            for j in range(1, n + 1):
                if text1[i-1] == text2[j-1]:
                    curr[j] = 1 + prev[j-1]
                else:
                    curr[j] = max(prev[j], curr[j-1])
            prev = curr
        return prev[n]
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

text1='abcde', text2='ace':

After processing 'a','b','c','d','e': prev[3]=3 ✓

text1='abc', text2='abc': prev[3]=3 ✓. text1='abc', text2='def': prev[3]=0 ✓

#

"No bugs. Two-row rolling DP reduces space from $O(m*n)$ to $O(n)$."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m*n)$. Space $O(n)$: two rolling rows.

LCS is foundational: used in diff tools, bioinformatics, plagiarism detection.

Follow-up: print the actual LCS – backtrack through the DP table.

Follow-up: Edit Distance (#72) extends LCS with insert/delete/replace costs."

Community Solutions (Python)

• WalkCC


```
class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        m = len(text1)
        n = len(text2)
        # dp[i][j] := the length of LCS(text1[0..i], text2[0..j])
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(m):
            for j in range(n):
                dp[i + 1][j + 1] = (1 + dp[i][j] if text1[i] == text2[j]

                                   else max(dp[i][j + 1], dp[i + 1][j]))

        return dp[m][n]
```

• LeetDoocs

```
class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        m, n = len(text1), len(text2)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if text1[i - 1] == text2[j - 1]:
                    f[i][j] = f[i - 1][j - 1] + 1
                else:
                    f[i][j] = max(f[i - 1][j], f[i][j - 1])

        return f[m][n]
```

• SimplyLeet

```
# Function to find the longest common subsequence length
def longestCommonSubsequence(text1: str, text2: str) -> int:
    m, n = len(text1), len(text2)
    # Initialize a (m+1) x (n+1) DP table filled with zeros
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Build the DP table bottom-up
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            # If characters match, extend the subsequence length by 1
```

```

        if text1[i - 1] == text2[j - 1]:
            dp[i][j] = dp[i - 1][j - 1] + 1
        else:
            # Otherwise, choose the maximum from the left or top cell
            dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

    return dp[m][n]

# Example usage
if __name__ == "__main__":

    text1 = "abcde"
    text2 = "ace"
    print(longestCommonSubsequence(text1, text2)) # Output: 3

```

• LC.ca

```

class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        m, n = len(text1), len(text2)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if text1[i - 1] == text2[j - 1]:
                    f[i][j] = f[i - 1][j - 1] + 1
                else:
                    f[i][j] = max(f[i - 1][j], f[i][j - 1])

    return f[m][n]

```

◆ Quick Review

Key Insights

- The problem is well-suited for a dynamic programming approach.
- Use a 2D table where $dp[i][j]$ stores the length of the longest common subsequence

for $\text{text1}[0\dots i-1]$ and $\text{text2}[0\dots j-1]$.

- If the characters match at the current position, extend the subsequence by 1 from $\text{dp}[i-1][j-1]$; otherwise, take the maximum from $\text{dp}[i-1][j]$ or $\text{dp}[i][j-1]$.
- Filling the table in a bottom-up manner ensures that all subproblems are solved optimally.

Space & Time

Time Complexity: $O(m * n)$, where m and n are the lengths of text1 and text2 .

Space Complexity: $O(m * n)$ for maintaining the 2D DP table. Space can be optimized to $O(\min(m, n))$ if needed.

Solution

This solution uses a dynamic programming approach with a 2D array (dp) to record the lengths of common subsequences at each substring pair. We initialize a table of dimensions $(m+1) \times (n+1)$ with zeros, where m and n are the lengths of text1 and text2 . We iterate over both strings, comparing characters; if they match, we set $\text{dp}[i][j] = \text{dp}[i-1][j-1] + 1$, otherwise we choose the

maximum from $dp[i-1][j]$ and $dp[i][j-1]$. The final answer is found at $dp[m][n]$.

Bit Manipulation

Round 8 · Low · Medium · 4 questions

Bit Manipulation

□ #78 Subsets

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Backtracking · Bit Manipulation
Lists: Grind 169

Problem

Given an integer array `nums` of unique elements, return all possible subsets (the power set).

The solution set must not contain duplicate subsets. Return the solution in any order.

Example 1:

Input: `nums = [1,2,3]`

Output: `[[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]`

Example 2:

Input: `nums = [0]`

Output: `[[],[0]]`

Constraints:

- $1 \leq \text{nums.length} \leq 10$
- $-10 \leq \text{nums}[i] \leq 10$
- All the numbers of `nums` are unique.

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        ans = []
        n = len(nums)
        for m in range(1<<n):
            t = []
            for i in range(n):
                if m >> i & 1:
                    t.append(nums[i])

            ans.append(t)

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Return all possible subsets (the power set) of nums. No duplicates. Any order. Right?"

#

"A few quick questions:

All elements are distinct? Yes. Include the empty set? Yes."

#

"Let me walk: nums=[1,2,3] → 8 subsets including [] and [1,2,3] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Backtracking: at each index, include or exclude. $O(n * 2^n)$ time.

Should I go ahead and optimize?"

```
class _78_Subsets_Brute:
```

```
    def subsets(self, nums):
```

```
        result = []
```

```
        def bt(i, path):
```

```
            result.append(list(path))
```

```
            for j in range(i, len(nums)):
```

```
                path.append(nums[j]); bt(j+1, path); path.pop()
```

```
        bt(0, []); return result
```

PHASE 3 — OPTIMIZE


```
# "The bottleneck is the recursive call stack – we can use bitmask iteration.
# Iterate  $0..2^n-1$ : bit  $i$  set means include  $nums[i]$ .
#  $O(n * 2^n)$  time,  $O(1)$  extra space (output excluded). Same complexity, no recursion."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _78_Subsets:
    def subsets(self, nums):
        n = len(nums)
        result = []
        for mask in range(1 << n):
            result.append([nums[i] for i in range(n) if mask & (1 << i)])
        return result
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
```

```
#
```

```
# nums=[1,2,3], n=3: masks 0..7.
```

```
# mask=0(000):[]. mask=1(001):[1]. mask=2(010):[2]. mask=3(011):[1,2].
```

```
# mask=4(100):[3]. mask=5(101):[1,3]. mask=6(110):[2,3]. mask=7(111):[1,2,3]. → 8 subsets ✓
```

```
#
```

```
# "No bugs. Each mask represents exactly one subset."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time  $O(n*2^n)$ . Space  $O(1)$  extra – output excluded.
```

```
# Backtracking and bitmask have identical complexity.
```

```
# Follow-up: Subsets II (#90) – input has duplicates; sort + skip in backtracking.
```

```
# Follow-up: Combination Sum – subsets constrained to a target sum."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def subsets(self, nums: list[int]) -> list[list[int]]:
        ans = []

        def dfs(s: int, path: list[int]) -> None:
            ans.append(path)

            for i in range(s, len(nums)):
                dfs(i + 1, path + [nums[i]])
```

```
dfs(0, [])
return ans
```

• LeetCode

```
class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        def dfs(i: int):
            if i == len(nums):
                ans.append(t[:])
                return
            dfs(i + 1)
            t.append(nums[i])
            dfs(i + 1)
            t.pop()
```

```
ans = []
t = []
dfs(0)
return ans
```

• SimplyLeet

```
# Define the function to generate all subsets using backtracking.
def subsets(nums):
    # This list will store all the subsets
    result = []

    # Recursive helper function to generate subsets.
    def backtrack(start, current):
        # Append a copy of the current subset to the final result.
        result.append(current.copy())
        # Iterate over the possible next elements starting from index 'start'
```

```

    for i in range(start, len(nums)):
        # Include nums[i] in the current subset.
        current.append(nums[i])
        # Recurse with the next starting index.
        backtrack(i + 1, current)
        # Backtrack: remove the last element to explore other
possibilities.
        current.pop()

    # Start the recursion with an empty path.
    backtrack(0, [])

    return result

# Example usage:
print(subsets([1, 2, 3]))

```

• LC.ca

```

from typing import List

class Solution: # bfs
    def subsets(self, nums: List[int]) -> List[List[int]]:
        if not nums:
            return [[]]

        nums.sort() # sort() not necessary if no duplicates
        result = [[]]

        for num in nums:
            result += [subset + [num] for subset in result]

        return result

class Solution: # dfs
    def subsets(self, nums: List[int]) -> List[List[int]]:
        def dfs(u, t):
            ans.append(t[:]) # or, t.copy()

```

```

        for i in range(u, len(nums)):
            t.append(nums[i])
            dfs(i + 1, t)
            t.pop()

ans = []
nums.sort() # sort() not necessary if no duplicates
dfs(0, [])
return ans

```

```

class Solution: # dfs, just pass down the final path
    def subsets(self, nums: List[int]) -> List[List[int]]:
        def dfs(nums, index, path, ans):
            ans.append(path)
            [dfs(nums, i + 1, path + [nums[i]], ans) for i in range(index, len
(nums))]

ans = []
dfs(nums, 0, [], ans)
return ans

```

```

class Solution: # dfs, but running slower, since need to reference from parent
method for 'nums' and 'ans'
    def subsets(self, nums: List[int]) -> List[List[int]]:
        def dfs(index, path):
            ans.append(path)
            [dfs(i + 1, path + [nums[i]]) for i in range(index, len(nums))]

ans = []
dfs(0, [])

return ans

```

◆ Quick Review

Key Insights

- The problem is equivalent to generating the power set of the given set.
- A recursive backtracking approach can be used to explore both including and excluding each element.
- Iterative (bit manipulation) solutions are also possible since there are 2^n subsets in total.
- The maximum array length is small (up to 10), making exponential time solutions feasible.

Space & Time

Time Complexity: $O(2^n * n)$ – There are 2^n subsets and creating each subset may take up to $O(n)$ time.

Space Complexity: $O(2^n * n)$ – Space for storing all subsets. The recursion stack has a maximum depth of $O(n)$.

Solution

We solve the problem using backtracking, where we decide for each element whether to include it in the current subset or not. We recursively build all subsets starting at an empty list and at each step appending the current subset to our result list. We can also

use iterative bit manipulation but here we focus on the backtracking approach. This algorithm leverages recursion and a list to maintain the current state, leading to a clear and concise implementation.

Bit Manipulation

□ #90 Subsets II

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Backtracking · Bit Manipulation
Lists: Denny Zhang

Problem

Given an integer array `nums` that may contain duplicates, return all possible subsets (the power set).

The solution set must not contain duplicate subsets. Return the solution in any order.

Example 1:

```
Input: nums = [1,2,2]
Output: [[],[1],[1,2],[1,2,2],[2],[2,2]]
```

Example 2:

```
Input: nums = [0]
Output: [[],[0]]
```

Constraints:

- $1 \leq \text{nums.length} \leq 10$
- $-10 \leq \text{nums}[i] \leq 10$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def subsetsWithDup(self, nums: List[int]) -> List[List[int]]:
        n = len(nums)
        ans = []
        nums.sort()
        for mask in range(1<<n):
            subset = []
            for i in range(n):
                if (mask >> i) & 1:
                    if i>0 and ((mask >> (i-1)) & 1) == 0 and nums[i] == num
s[i-1]:

                    break
                subset.append(nums[i])
            else:
                ans.append(subset)

        return ans
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Input may have duplicates; return all unique subsets. No duplicate subsets in output. Right?"

#

"A few quick questions:

Sort first to group duplicates? Yes – that enables the skip logic."

#

"Let me walk: nums=[1,2,2] → [[],[1],[1,2],[1,2,2],[2],[2,2]] ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Generate all subsets as sorted tuples, add to a set to deduplicate.

O(n*2^n) time + O(2^n) for the set.

Should I go ahead and optimize?"

```
class _90_SubsetsII_Brute:
```

```
    def subsetsWithDup(self, nums):
```

```
        nums.sort(); result = set()
```

```
        def bt(i, path):
```

```
            result.add(tuple(path))
```

```

        for j in range(i, len(nums)):
            path.append(nums[j]); bt(j+1, path); path.pop()
    bt(0, []); return [list(t) for t in result]

```

PHASE 3 — OPTIMIZE

"The bottleneck is the dedup set.

Sort nums first. In backtracking, skip duplicate elements at the same recursion level

(not the first occurrence, but subsequent same values when $j > \text{start}$).

#

Time: $O(n \times 2^n)$. Space: $O(n)$ recursion – no dedup set."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _90_SubsetsII:
```

```
    def subsetsWithDup(self, nums):
```

```
        nums.sort()
```

```
        result = []
```

```
        def backtrack(start, path):
```

```
            result.append(list(path))
```

```
            for i in range(start, len(nums)):
```

```
                if i > start and nums[i] == nums[i-1]:
```

```
                    continue
```

```
                path.append(nums[i])
```

```
                backtrack(i + 1, path)
```

```
                path.pop()
```

```
        backtrack(0, [])
```

```
        return result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[1,2,2] sorted:

bt(0,[]): append []. i=0→[1]: bt(1,[1]): append[1]. i=1→[1,2]: bt(2,[1,2]): append. i=2: nums[2]==nums[1] and 2>1 → skip.

i=1→[2]: bt(2,[2]): append. i=2: 2>1 and nums[2]==nums[1] → skip.

→ [[],[1],[1,2],[1,2,2],[2],[2,2]] ✓

#

"No bugs. 'i>start' prevents skipping the very first occurrence at each level."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n \times 2^n)$: worst case all unique. Space $O(n)$ recursion depth.

The sort+skip pattern is the standard dedup technique for backtracking.

Follow-up: Permutations II (#47) – same skip-duplicate pattern for permutations.

Follow-up: Combination Sum II (#40) – target sum with duplicates; same skip."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def subsetsWithDup(self, nums: list[int]) -> list[list[int]]:
        ans = []

        def dfs(s: int, path: list[int]) -> None:
            ans.append(path)
            if s == len(nums):
                return

            for i in range(s, len(nums)):

                if i > s and nums[i] == nums[i - 1]:
                    continue
                dfs(i + 1, path + [nums[i]])

        nums.sort()
        dfs(0, [])
        return ans

```

• LeetCode

```

class Solution:
    def subsetsWithDup(self, nums: List[int]) -> List[List[int]]:
        def dfs(i: int):
            if i == len(nums):
                ans.append(t[:])
                return
            t.append(nums[i])
            dfs(i + 1)
            x = t.pop()
            while i + 1 < len(nums) and nums[i + 1] == x:
                i += 1
            dfs(i + 1)

        nums.sort()
        ans = []
        t = []
        dfs(0)
        return ans

```

• SimplyLeet

```
def subsetsWithDup(nums):
    # Sort the numbers to handle duplicates
    nums.sort()
    result = [] # To store all subsets
    subset = [] # Current subset being built

    def backtrack(start):
        # Append a copy of the current subset to the result
        result.append(list(subset))
        # Iterate over possible elements to include

        for i in range(start, len(nums)):
            # If the current element is a duplicate and not at the start of
            this loop, skip it
            if i > start and nums[i] == nums[i - 1]:
                continue
            # Include current element into the subset
            subset.append(nums[i])
            # Recurse with next start index
            backtrack(i + 1)
            # Backtrack and remove the last element added
            subset.pop()

    backtrack(0)
    return result

# Example usage:
print(subsetsWithDup([1, 2, 2]))
```

• LC.ca

```
class Solution:
    def subsetsWithDup(self, nums: List[int]) -> List[List[int]]:
        def dfs(u, t):
            ans.append(t[:]) # or, t.copy()
            for i in range(u, len(nums)):
                # also good for [1,5,5], second '5' will be skipped here if
                # but second '5' is always covered, because i==u when
                ans=[1,5] and i=2
                if i != u and nums[i] == nums[i - 1]:
                    continue
                t.append(nums[i])
```

```
        dfs(i + 1, t)
        t.pop()
```

```
ans = []
nums.sort()
dfs(0, [])
return ans
```

```
# iteration
```

```
class Solution:
    def subsetsWithDup(self, nums: List[int]) -> List[List[int]]:
        if not nums:
            return [[]]

        nums.sort() # Sorting is necessary to handle duplicates.
        result = [[]]
        start_idx = 0 # Start index for the new subsets
        new_subsets_size = 0

        for i in range(len(nums)):
            size = len(result)
            # If the current number is the same as the previous one,
            # we only want to extend the subsets added in the previous
iteration.
            if i > 0 and nums[i] == nums[i - 1]:
                start_idx = size - new_subsets_size
            else:
                start_idx = 0

            new_subsets_size = 0 # Reset the size of newly created subsets
```

```

        for j in range(start_idx, size):
            current_subset = result[j] + [nums[i]]
            result.append(current_subset)
            new_subsets_size += 1

    return result

```

```
#####
```

```

class Solution:
    def subsetsWithDup(self, nums: List[int]) -> List[List[int]]:
        nums.sort()
        n = len(nums)
        ans = []
        for mask in range(1 << n):
            ok = True
            t = []
            for i in range(n):
                if mask >> i & 1:
                    if i and (mask >> (i - 1) & 1) == 0 and nums[i] == nums[i
- 1]:
                        ok = False
                        break
                    t.append(nums[i])
            if ok:
                ans.append(t)
        return ans

```

◆ Quick Review

Key Insights

- Sort the array first so that duplicate elements are adjacent.
- Use backtracking to generate all possible subsets.

- Skip duplicate elements in the recursion to avoid forming duplicate subsets.
- The use of sorted order and conditionally skipping elements is the trick to handle duplicates.

Space & Time

Time Complexity: $O(2^n * n)$ since each subset creation may take $O(n)$ in the worst case.

Space Complexity: $O(2^n * n)$ for storing the subsets, plus $O(n)$ for recursion stack.

Solution

The solution uses a backtracking technique that builds subsets incrementally. The input array is first sorted to bring duplicates together. During recursion, if the current element is the same as the previous one and it has not been used in the current recursion path, it is skipped to prevent duplicate subsets. This ensures that each subset is unique. The algorithm stores each valid subset by copying the current subset state into the result list.

Bit Manipulation

□ #287 Find the Duplicate Number

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Two Pointers · Binary Search · Bit
Manipulation

Lists: Grind 169

Problem

Given an array of integers `nums` containing $n + 1$ integers where each integer is in the range $[1, n]$ inclusive.

There is only one repeated number in `nums`, return this repeated number.

You must solve the problem without modifying the array `nums` and using only constant extra space.

Example 1:

Input: `nums = [1,3,4,2,2]`
Output: 2

Example 2:

Input: `nums = [3,1,3,4,2]`
Output: 3

Example 3:

Input: nums = [3,3,3,3,3]
Output: 3

Constraints:

- $1 \leq n \leq 105$
- `nums.length == n + 1`
- $1 \leq \text{nums}[i] \leq n$
- All the integers in nums appear only once except for precisely one integer which appears two or more times.

Follow up:

- How can we prove that at least one duplicate number must exist in nums?
- Can you solve the problem in linear runtime complexity?

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def findDuplicate(self, nums: List[int]) -> int:
        nums.sort()
        for i in range(1, len(nums)):
            if nums[i] == nums[i-1]:
                return nums[i]
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Let me make sure I understand the problem correctly.
# n+1 integers in range [1,n]; exactly one number is duplicated.
# Find it without modifying the array and using O(1) extra space. Right?"
#
# "A few quick questions:
# The duplicate may appear more than twice? Yes.
# Array can't be sorted or hashed? Correct – O(1) space constraint."
#
# "Let me walk: nums=[1,3,4,2,2] → duplicate=2 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Hash set: return first element seen twice. O(n) time, O(n) space.
# Violates O(1) space. Should I go ahead and optimize?"
class _287_FindDuplicateNumber_Brute:
    def findDuplicate(self, nums):
        seen = set()
        for n in nums:
            if n in seen: return n
            seen.add(n)

# PHASE 3 — OPTIMIZE
# "The bottleneck is the O(n) hash set.
# Floyd's cycle detection: treat nums as a linked list where index i points to
nums[i].
# Because one value is duplicated, two indices point to the same next node → cycle.
# Phase 1: find meeting point (slow/fast). Phase 2: find cycle entry (= duplicate).
#
# Time: O(n). Space: O(1)."
```

```

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
class _287_FindDuplicateNumber:
    def findDuplicate(self, nums):
        slow = fast = nums[0]
        while True:
            slow = nums[slow]
            fast = nums[nums[fast]]
            if slow == fast:
                break
        slow = nums[0]
        while slow != fast:
            slow = nums[slow]
            fast = nums[fast]
        return slow

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# nums=[1,3,4,2,2] (0→1, 1→3, 2→4, 3→2, 4→2):
# Phase1: slow=1,fast=3→ slow=3,fast=2→ slow=2,fast=2. Meet at 2.
```

```
# Phase2: slow=nums[0]=1, fast=2. slow=3,fast=2→wait:
slow=nums[1]=3,fast=nums[2]=4.
# slow=2,fast=2. return 2 ✓
#
# "No bugs. The cycle entry is the duplicated value – same proof as Linked List
Cycle II."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1). Does not sort or modify the array.
# Binary search alternative: count numbers ≤ mid; if > mid → duplicate in [1,mid].
O(n log n).
# Follow-up: if multiple duplicates – Floyd's won't work; use binary search.
# Follow-up: Find All Duplicates (#442) – mark visited by negating at index."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findDuplicate(self, nums: list[int]) -> int:
        slow = nums[nums[0]]
        fast = nums[nums[nums[0]]]

        while slow != fast:
            slow = nums[slow]
            fast = nums[nums[fast]]

        slow = nums[0]

        while slow != fast:
            slow = nums[slow]
            fast = nums[fast]

        return slow
```

• LeetDoocs

```
class Solution:
    def findDuplicate(self, nums: List[int]) -> int:
        def f(x: int) -> bool:
            return sum(v <= x for v in nums) > x

        return bisect_left(range(len(nums)), True, key=f)
```

• SimplyLeet

Function to find the duplicate number using Floyd's Tortoise and Hare algorithm.

```
def findDuplicate(nums):
    # Phase 1: Find the intersection point inside the cycle.
    tortoise = nums[0] # Initialize tortoise pointer.
    hare = nums[0]      # Initialize hare pointer.
    while True:
        tortoise = nums[tortoise] # Move tortoise one step.
        hare = nums[nums[hare]]    # Move hare two steps.
        if tortoise == hare:       # If pointers meet, cycle is
detected.
            break
```

```
# Phase 2: Find the entrance to the cycle.
pointer = nums[0] # Initialize a new pointer from the start.
while pointer != tortoise:
    pointer = nums[pointer] # Move pointer one step.
    tortoise = nums[tortoise] # Move tortoise one step.
return pointer # The duplicate number.
```

Example usage:

```
if __name__ == "__main__":
```

```
    sample = [1, 3, 4, 2, 2]
    print(findDuplicate(sample)) # Expected output: 2
```

• LC.ca

```
class Solution:
    def findDuplicate(self, nums: List[int]) -> int:
        def f(x: int) -> bool:
            return sum(v <= x for v in nums) > x

        return bisect_left(range(len(nums)), True, key=f)
```

◆ Quick Review

Key Insights

- Since there are $n+1$ elements with values only from 1 to n , by the pigeonhole principle at least one number must be repeated.
- The problem can be approached by mapping the array values to indices, forming a cycle structure.
- Floyd's Tortoise and Hare algorithm (cycle detection) can be applied to find the duplicate number in linear time and constant space.
- There is no need for extra data structures like hash sets or alterations to the original array, satisfying the space and array-modification constraints.

Space & Time

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Solution

The solution leverages Floyd's Tortoise and Hare cycle detection algorithm. Think of each array element as a pointer to another index, forming a linked list with a cycle (the duplicate number creates the cycle). The algorithm works in two phases: Detect a meeting point within the cycle by having two pointers

(tortoise and hare) move at different speeds. Find the cycle entrance by resetting one pointer to the start and moving both pointers at the same speed. The meeting point reveals the duplicate number. This approach provides an elegant solution that satisfies both linear runtime and constant space requirements.

Bit Manipulation

□ ★ #1506 Find Root of N-Ary Tree ★

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Hash Table · Bit Manipulation · Tree · DFS
Lists: ★ Premium | Premium 98

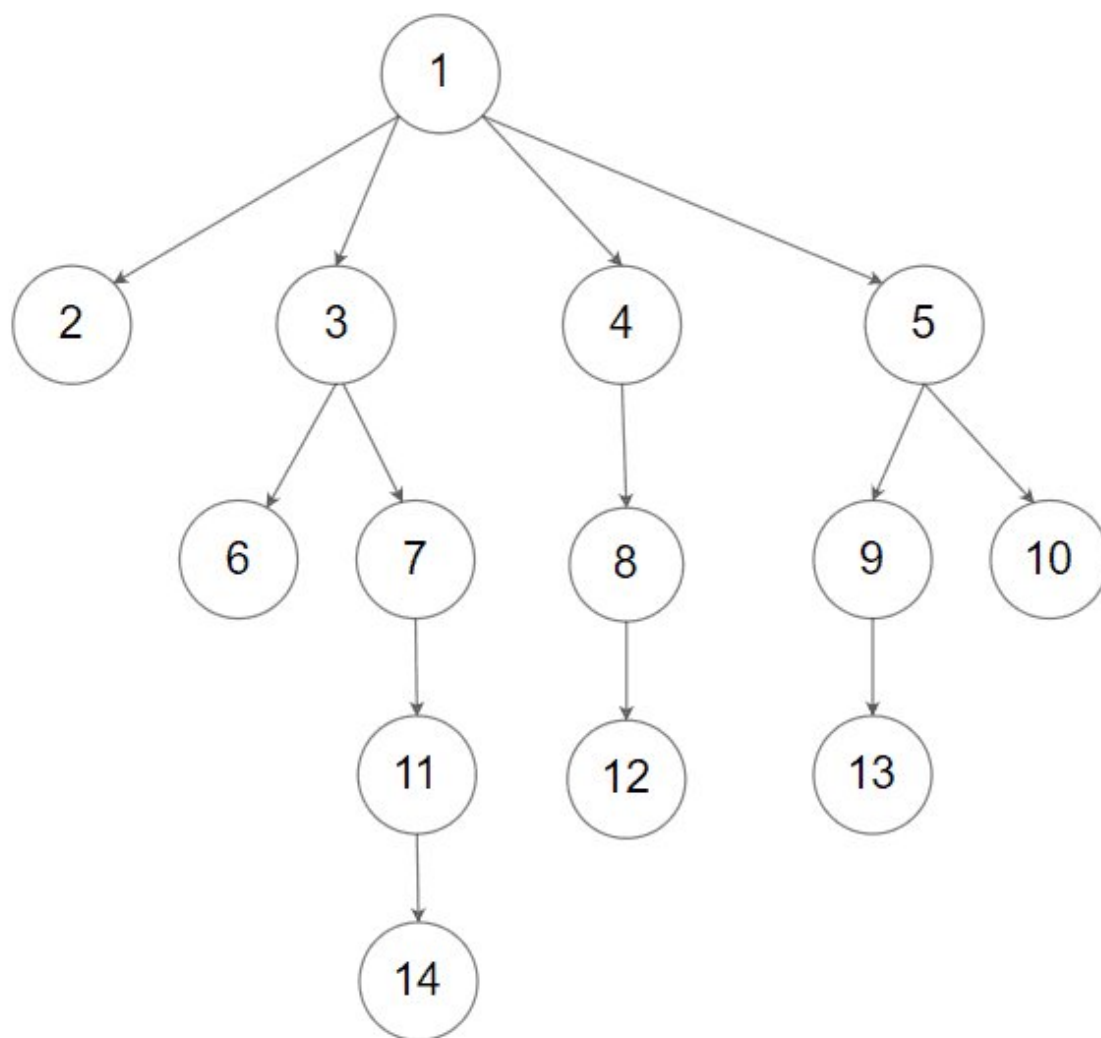
Problem

You are given all the nodes of an N-ary tree as an array of Node objects, where each node has a unique value.

Return the root of the N-ary tree.

Custom testing:

An N-ary tree can be serialized as represented in its level order traversal where each group of children is separated by the null value (see examples).



For example, the above tree is serialized as [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14].

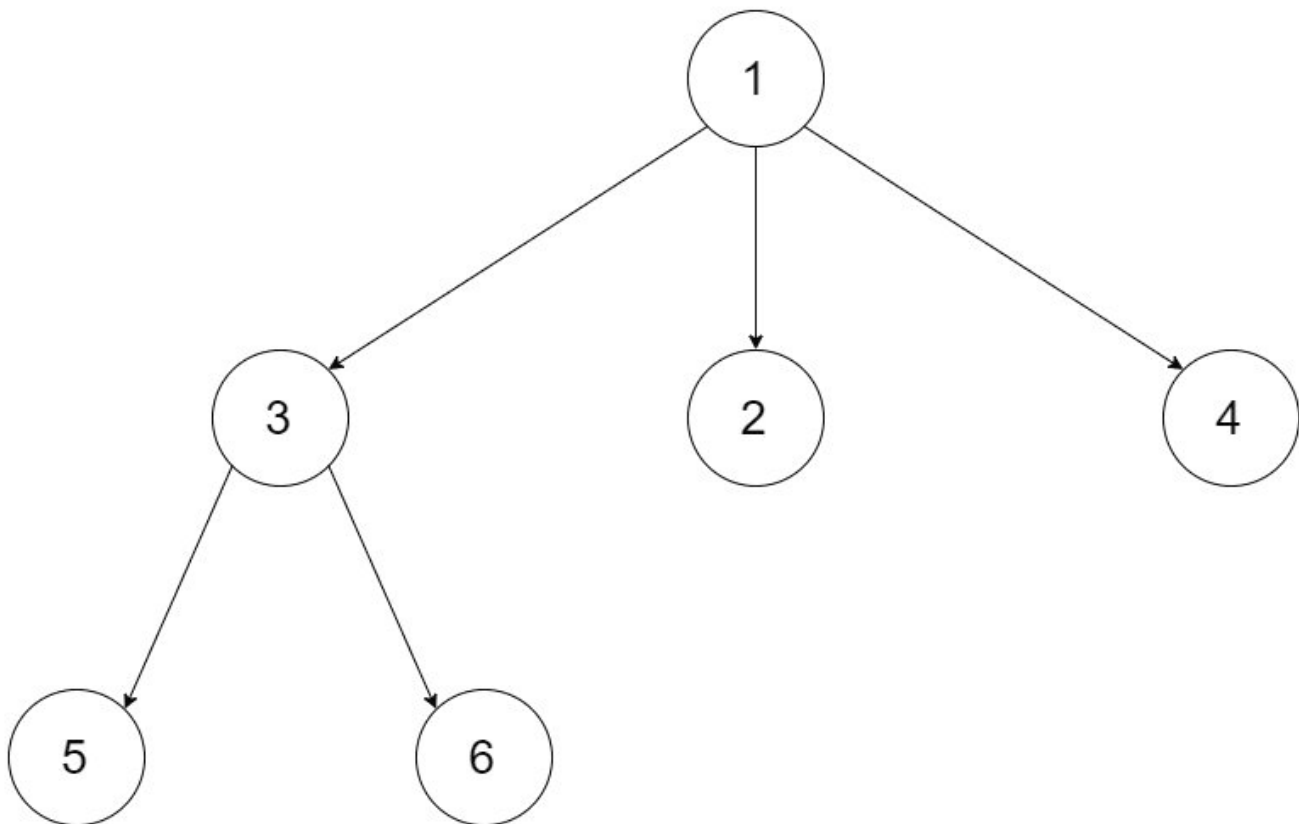
The testing will be done in the following way:

1. The input data should be provided as a serialization of the tree.
2. The driver code will construct the tree from the serialized input data and put each Node object into an array in an arbitrary order.

3. The driver code will pass the array to `findRoot`, and your function should find and return the root Node object in the array.

4. The driver code will take the returned Node object and serialize it. If the serialized value and the input data are the same, the test passes.

Example 1:



Input: tree = [1,null,3,2,4,null,5,6]

Output: [1,null,3,2,4,null,5,6]

Explanation: The tree from the input data is shown above.

The driver code creates the tree and gives findRoot the Node objects in an arbitrary order.

For example, the passed array could be

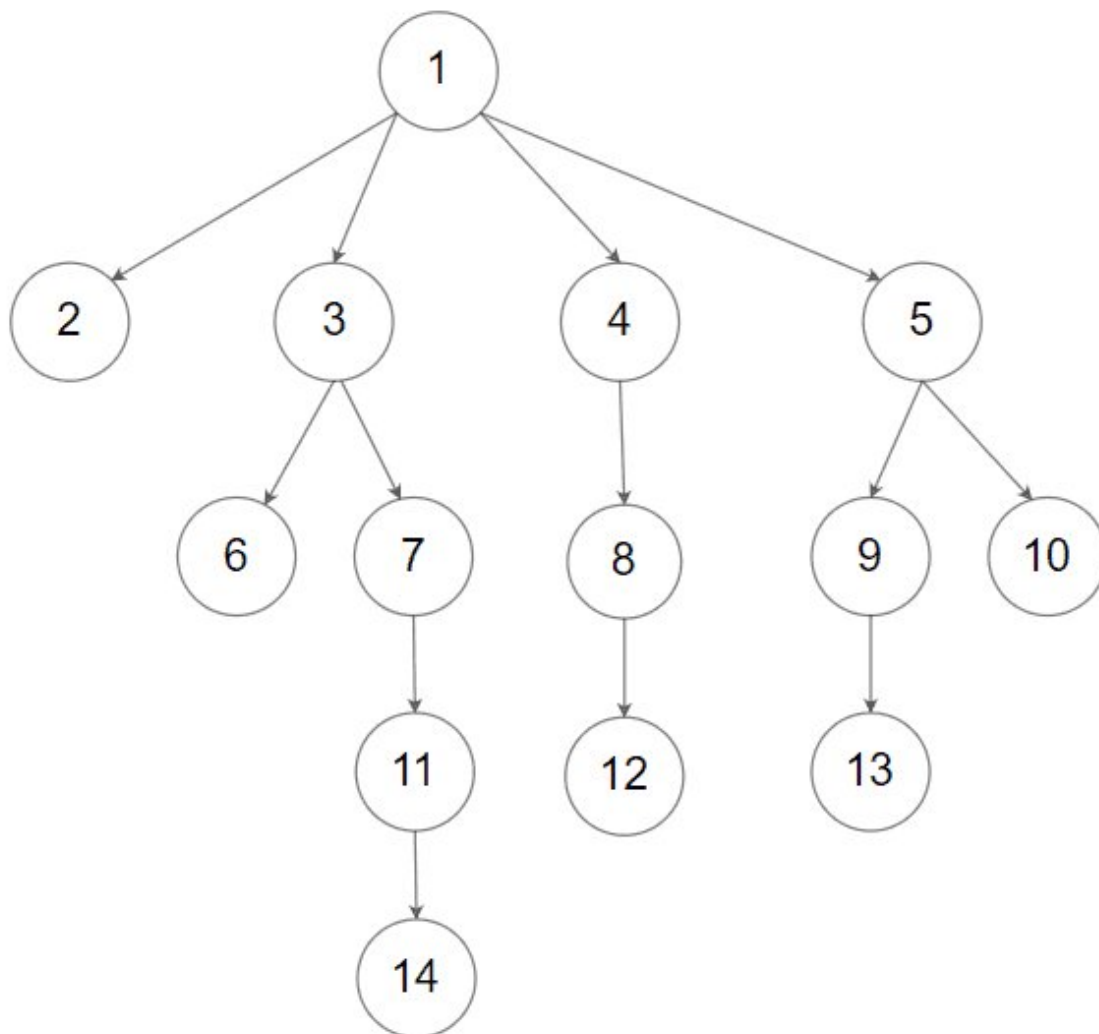
[Node(5),Node(4),Node(3),Node(6),Node(2),Node(1)] or

[Node(2),Node(6),Node(1),Node(3),Node(5),Node(4)].

The findRoot function should return the root Node(1), and the driver code will serialize it and compare with the input data.

The input data and serialized Node(1) are the same, so the test passes.

Example 2:



Input: tree = [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]
 Output: [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]

Constraints:

- The total number of nodes is between [1, 5 * 10⁴].
- Each node has a unique value.

Follow up:

- Could you solve this problem in constant space complexity with a linear time algorithm?

My LeetCode Solution

• My LeetCode Solution

```

class Node:
    def __init__(self, val: Optional[int] = None, children: Optional[List['Node']] = None):
        self.val = val
        self.children = children if children is not None else []

class Solution:
    def findRoot(self, tree: List['Node']) -> 'Node':

```

```

tsum = csum = 0
for node in tree:
    tsum += node.val
    for c in node.children:
        csum += c.val

ans = tsum - csum
for node in tree:
    if node.val == ans:
        return node

return None

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Let me make sure I understand the problem correctly.
# Given all nodes of an N-ary tree in random order, find the root.
# The root is the only node NOT appearing as a child of any other node. Right?"
#
# "A few quick questions:
# Node values are unique? Yes. No parent pointers? Correct."
#
# "Let me walk: nodes=[1,2,3,4,5,6], root=1 with children [3,2,4], 3's
children=[5,6] → root=1 ✓"

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with brute force to lock in the logic.
# Build a set of all child node values. Root is the node whose value is NOT in that
set.
# O(n) time, O(n) space for the set.
# Should I go ahead and optimize?"
class _1506_FindRootOfNAryTree_Brute:
    def findRoot(self, tree):
        children = set()
        for node in tree:
            for child in node.children:
                children.add(child.val)
        for node in tree:
            if node.val not in children:
                return node

```

PHASE 3 — OPTIMIZE

```

# "The bottleneck is the O(n) set for child values.
# XOR trick: XOR all node values; XOR all children values.

```

```
# Every non-root node's value appears once as a node AND once as a child → cancels.
# Root appears only as a node → its value remains.
#
# Time: O(n). Space: O(1)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _1506_FindRootOfNAryTree:
    def findRoot(self, tree):
        xor_all = 0
        for node in tree:
            xor_all ^= node.val
            for child in node.children:
                xor_all ^= child.val
        for node in tree:
            if node.val == xor_all:
                return node
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
```

```
#
# tree nodes [1,2,3,4,5,6] with root=1, children={3,2,4}, 3's children={5,6}:
# node vals XOR: 1^2^3^4^5^6. child vals XOR: 3^2^4^5^6.
# Combined: 1^2^3^4^5^6^3^2^4^5^6 = 1 (all others cancel). root=node(1) ✓
#
# "No bugs. XOR is commutative and associative – order of traversal doesn't matter."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(1).
# Same XOR cancellation as Single Number (#136).
# Follow-up: if node values aren't unique, XOR fails – use the child-set approach.
# Follow-up: find all roots if input is a forest – same XOR, collect all non-child values."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def findRoot(self, tree: list['Node']) -> 'Node':
        sum = 0

        for node in tree:
            sum ^= node.val
            for child in node.children:
                sum ^= child.val

        for node in tree:
```

```

    if node.val == sum:
        return node

```

• LeetDoocs

```

"""
# Definition for a Node.
class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children if children is not None else []
"""

class Solution:

```

```

    def findRoot(self, tree: List['Node']) -> 'Node':
        x = 0
        for node in tree:
            x ^= node.val
            for child in node.children:
                x ^= child.val
        return next(node for node in tree if node.val == x)

```

• SimplyLeet

```

# Python implementation with detailed comments
def findRoot(tree):
    # Initialize variables to store the sum of all node values and children
    # values
    total_sum = 0
    children_sum = 0

    # Iterate over each node in the tree array
    for node in tree:
        total_sum += node.val # Add current node's value to total_sum
        # Iterate over each child of the current node and add their values to
        children_sum

```



```

        for child in node.children:
            children_sum += child.val

# The root's value is the difference between total_sum and children_sum
root_val = total_sum - children_sum

# Find and return the node with the value equal to root_val
for node in tree:
    if node.val == root_val:
        return node

return None # This should never happen if the input tree is valid

```

• LC.ca

```

"""
# Definition for a Node.
class Node:
    def __init__(self, val=None, children=None):
        self.val = val
        self.children = children if children is not None else []
"""

class Solution:

    def findRoot(self, tree: List['Node']) -> 'Node':
        x = 0
        for node in tree:
            x ^= node.val
            for child in node.children:
                x ^= child.val
        return next(node for node in tree if node.val == x)

```

◆ Quick Review

Key Insights

- Every node except the root appears as a child of another node.

- By summing all node values and then subtracting the sum of all children node values, the remaining value corresponds to the root.
- This subtraction trick allows us to identify the root in a single pass of the data structure without extra memory for tracking child nodes.

Space & Time

Time Complexity: $O(n)$ where n is the number of nodes, since we iterate through all nodes and their children.

Space Complexity: $O(1)$ extra space using arithmetic accumulators.

Solution

The solution calculates two sums: one for all nodes' values and one for all children nodes' values. The difference between these gives the value of the root node since every node's value (except the root's) is added once as a child. Finally, we iterate over the nodes to find the node with the computed root value and return it. This approach meets the requirement of constant space complexity and linear time.

Math

Round 8 · Low · Medium · 5 questions

Math

□ #7 Reverse Integer

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Math

Lists: Grind 169

Problem

Given a signed 32-bit integer x , return x with its digits reversed. If reversing x causes the value to go outside the signed 32-bit integer range $[-2^{31}, 2^{31} - 1]$, then return 0.

Assume the environment does not allow you to store 64-bit integers (signed or unsigned).

Example 1:

Input: $x = 123$
Output: 321

Example 2:

Input: $x = -123$
Output: -321

Example 3:

Input: $x = 120$
Output: 21

Constraints:

- $-231 \leq x \leq 231 - 1$

My LeetCode Solution

• My LeetCode Solution

```
class Solution:
    def reverse(self, x: int) -> int:
        intmin = -2**31
        intmax = 2**31-1

        sign = 1 if x >= 0 else -1
        x = abs(x)
        ans = 0
        while x:
            d = x%10

            x=x//10
            if ans > intmax//10 or (ans==intmax//10 and d>7):
                return 0
            ans = ans*10 + d
        return ans* sign
```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Reverse the digits of a signed 32-bit integer. Return 0 if overflow.
# Assume environment doesn't allow 64-bit integers. Right?"
#
# "A few quick questions:
# x=120 → 21 (trailing zero becomes leading zero → dropped) ✓
# x=-123 → -321 ✓. Overflow range: [-2^31, 2^31-1]."
#
# "Let me walk: x=123 → 321 ✓. x=-123 → -321 ✓. x=1534236469 → 0 (overflow) ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Convert to string, reverse, convert back. Check 32-bit bounds.
# O(d) time and space. Should I go ahead and optimize?"
```

```
class _7_ReverseInteger_Brute:
```

```
def reverse(self, x):
    sign = -1 if x < 0 else 1; x = abs(x)
    result = int(str(x)[::-1]) * sign
    return result if -(2**31) <= result <= 2**31-1 else 0
```

PHASE 3 — OPTIMIZE

"The bottleneck is the string allocation.

Build reversed integer digit by digit; check overflow before each multiplication.

#

Time: O(d). Space: O(1)."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _7_ReverseInteger:
    def reverse(self, x):
        INT_MAX, INT_MIN = 2**31 - 1, -(2**31)
        sign = -1 if x < 0 else 1
        x = abs(x)
        result = 0
        while x:
            digit = x % 10; x //= 10
            if result > (INT_MAX - digit) // 10:
                return 0
            result = result * 10 + digit
        return sign * result
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

x=123: digits 3,2,1. result: 3,32,321. 321<=INT_MAX ✓. return 321 ✓.

x=-123: sign=-1, x=123. Same → return -321 ✓.

x=1534236469: eventually overflow check fires → return 0 ✓.

#

"No bugs. Overflow check: result > (INT_MAX - digit) // 10 is safe in Python (big ints)."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time O(d): d = number of digits. Space O(1): integer variables.

In Java/C++ the overflow check is critical – Python big-ints hide the issue.

Follow-up: Palindrome Number (#9) – compare reversed half to original.

Follow-up: if 64-bit is allowed, just reverse and check range."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def reverse(self, x: int) -> int:
        ans = 0
        sign = -1 if x < 0 else 1
        x *= sign

        while x:
            ans = ans * 10 + x % 10
            x //= 10

        return 0 if ans < -2**31 or ans > 2**31 - 1 else sign * ans
```

• LeetDoocs

```
class Solution:
    def reverse(self, x: int) -> int:
        ans = 0
        mi, mx = -(2**31), 2**31 - 1
        while x:
            if ans < mi // 10 + 1 or ans > mx // 10:
                return 0
            y = x % 10
            if x < 0 and y > 0:
                y -= 10

            ans = ans * 10 + y
            x = (x - y) // 10
        return ans
```

• SimplyLeet

```
# Function to reverse an integer
def reverse(x: int) -> int:
    # Define the maximum and minimum limits for a 32-bit signed integer
    INT_MAX = 2**31 - 1
    INT_MIN = -2**31

    reversed_num = 0 # This will store the reversed number

    # Process each digit until x becomes 0
    while x != 0:
```



```

    # Pop the last digit from x
    # Use modulo 10 to get the last digit
    # For negative numbers, modulo may yield negative remainder
    pop = x % 10 if x > 0 else x % -10
    # Remove the last digit from x by integer division
    x = x // 10

    # Check if appending the digit would cause overflow
    if reversed_num > INT_MAX // 10 or (reversed_num == INT_MAX // 10 and
pop > 7):
        return 0

    if reversed_num < INT_MIN // 10 or (reversed_num == INT_MIN // 10 and
pop < -8):
        return 0

    # Append the digit to the reversed number
    reversed_num = reversed_num * 10 + pop

    return reversed_num

# Example usage:
print(reverse(123)) # Output: 321

print(reverse(-123)) # Output: -321
print(reverse(120)) # Output: 21

```

• LC.ca

```

class Solution:
    def reverse(self, x: int) -> int:
        ans = 0
        mi, mx = -(2**31), 2**31 - 1
        while x:
            if ans < mi // 10 + 1 or ans > mx // 10:
                return 0
            y = x % 10
            if x < 0 and y > 0:
                y -= 10

            ans = ans * 10 + y
            x = (x - y) // 10
        return ans

```

◆ Quick Review

Key Insights

- The problem requires reversing the digits while preserving the sign.
- Watch out for integer overflow due to the reversed number exceeding the 32-bit signed integer limits.
- The approach involves repeatedly extracting the last digit from the number and appending it to a new reversed integer.
- Overflow checks are done during each iteration before actually appending the digit.

Space & Time

Time Complexity: $O(n)$, where n is the number of digits in the integer.

Space Complexity: $O(1)$, only constant extra space is used.

Solution

The solution uses a while loop to extract the last digit from the original number using modulo operation ($x \% 10$) and then removes this digit using integer division ($x /= 10$). A

reversed number is constructed by multiplying the current reversed number by 10 and adding the extracted digit. Before appending each digit, the algorithm checks if the new reversed number will overflow a 32-bit signed integer by comparing it with the thresholds derived from INT_MAX and INT_MIN. The algorithm handles negative values by working with x directly since the modulo and division operations work properly with negatives in most languages.

Math

□ #50 Pow(x, n)

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Math · Recursion
Lists: Grind 169

Problem

Implement $\text{pow}(x, n)$, which calculates x raised to the power n (i.e., x^n).

Example 1:

Input: $x = 2.00000$, $n = 10$
Output: 1024.00000

Example 2:

Input: $x = 2.10000$, $n = 3$
Output: 9.26100

Example 3:

Input: $x = 2.00000$, $n = -2$
Output: 0.25000
Explanation: $2^{-2} = 1/2^2 = 1/4 = 0.25$

Constraints:

- $-100.0 < x < 100.0$
- $-2^{31} \leq n \leq 2^{31} - 1$
- n is an integer.

- Either x is not zero or $n > 0$.
- $-104 \leq x^n \leq 104$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Implement pow(x, n) – x raised to the power n. n can be negative. Right?"
#
# "A few quick questions:
# x can be negative? Yes. n can be -2^31? Yes – handle with care.
# x=0.0, n<0 is undefined per constraints (x won't be 0 when n<0)."
#
# "Let me walk: pow(2.0,10)→1024.0 ✓. pow(2.0,-2)→0.25 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Multiply x by itself |n| times. O(|n|) – TLEs for n=2^31.
# Should I go ahead and optimize?"
class _50_PowXN_Brute:
    def myPow(self, x, n):
        if n < 0: x, n = 1/x, -n
        result = 1
        for _ in range(n): result *= x
        return result
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the O(|n|) multiplications.
# Fast exponentiation: if n even: x^n = (x^2)^(n/2). If n odd: x^n = x * x^(n-1).
# Halves n each step → O(log n) multiplications.
#
# Time: O(log n). Space: O(1) iterative."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _50_PowXN:
    def myPow(self, x, n):
        if n < 0:
            x = 1 / x
            n = -n
        result = 1
        while n:
            if n % 2 == 1:
```

```

        result *= x
    x *= x
    n //= 2
    return result

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

x=2.0, n=10:

n=10(even): x=4,n=5. n=5(odd): result=4,x=16,n=2. n=2(even): x=256,n=1.

n=1(odd): result=4*256=1024,n=0. return 1024.0 ✓

#

x=2.0, n=-2: x=0.5,n=2. n=2(even):x=0.25,n=1. n=1(odd):result=0.25,n=0. → 0.25 ✓

#

"No bugs. The iterative approach avoids $O(\log n)$ recursion stack space."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\log n)$: halve the exponent each step. Space $O(1)$: iterative.

Recursive version is $O(\log n)$ time but $O(\log n)$ stack space.

Follow-up: matrix exponentiation – Fibonacci in $O(\log n)$ using the same squaring.

Follow-up: modular exponentiation – same logic with $(\text{result} * x) \% \text{mod}$ at each step."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def myPow(self, x: float, n: int) -> float:
        if n == 0:
            return 1
        if n < 0:
            return 1 / self.myPow(x, -n)
        if n % 2 == 1:
            return x * self.myPow(x, n - 1)
        return self.myPow(x * x, n // 2)

```

• LeetDoocs

```
class Solution:
    def myPow(self, x: float, n: int) -> float:
        def qpow(a: float, n: int) -> float:
            ans = 1
            while n:
                if n & 1:
                    ans *= a
                a *= a
                n >>= 1
            return ans

        return qpow(x, n) if n >= 0 else 1 / qpow(x, -n)
```

• SimplyLeet

```
# Python implementation of pow(x, n) using exponentiation by squaring
def myPow(x, n):
    # if exponent is negative, invert x and make n positive
    if n < 0:
        x = 1 / x
        n = -n

    result = 1.0 # initialize result
    while n:
        # if the current exponent is odd, multiply result by the base

        if n % 2:
            result *= x
        x *= x # square the base for the next iteration
        n //= 2 # divide the exponent by 2
    return result

# Example usage:
print(myPow(2.0, 10)) # Output: 1024.0
print(myPow(2.1, 3))  # Output: 9.261000000000001
print(myPow(2.0, -2)) # Output: 0.25
```

• LC.ca


```
class Solution:
    def myPow(self, x: float, n: int) -> float:
        def qpow(a: float, n: int) -> float:
            ans = 1
            while n:
                if n & 1:
                    ans *= a
                a *= a
                n >>= 1
            return ans

        return qpow(x, n) if n >= 0 else 1 / qpow(x, -n)
```

◆ Quick Review

Key Insights

- Use exponentiation by squaring to reduce the number of multiplications.
- For a negative exponent, compute the power for $|n|$ and then take the reciprocal.
- Handle the edge case when n is 0 (return 1).
- The algorithm works in $O(\log n)$ time complexity.

Space & Time

Time Complexity: $O(\log |n|)$

Space Complexity: $O(1)$ for the iterative solution, or $O(\log |n|)$ if implemented recursively

Solution

We use exponentiation by squaring, an algorithm that reduces the number of multiplications by repeatedly squaring the base and halving the exponent. Here's how it works: If n is negative, switch x to $1/x$ and n to $-n$. Initialize the result as 1. Loop while n is greater than 0: If the current n is odd, multiply the result by x . Square the base x . Divide n by 2 (using integer division). This iterative approach efficiently computes the power in logarithmic steps. The same approach can be implemented recursively.

Math

□ #380 Insert Delete GetRandom O(1)

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Math · Design
Lists: Grind 169

Problem

Implement the RandomizedSet class:

- **RandomizedSet()** Initializes the RandomizedSet object.
- **bool insert(int val)** Inserts an item val into the set if not present. Returns true if the item was not present, false otherwise.
- **bool remove(int val)** Removes an item val from the set if present. Returns true if the item was present, false otherwise.
- **int getRandom()** Returns a random element from the current set of elements (it's guaranteed that at least one element exists when this method is called). Each element must have the same probability of being returned.

You must implement the functions of the class such that each function works in average $O(1)$ time complexity.

Example 1:

```
Input
["RandomizedSet", "insert", "remove", "insert", "getRandom", "remove",
"insert", "getRandom"]
[[], [1], [2], [2], [], [1], [2], []]
Output
[null, true, false, true, 2, true, false, 2]

Explanation
RandomizedSet randomizedSet = new RandomizedSet();
```

Constraints:

- $-231 \leq \text{val} \leq 231 - 1$
- At most $2 * 10^5$ calls will be made to insert, remove, and getRandom.
- There will be at least one element in the data structure when getRandom is called.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Design a set supporting insert(val)→bool, remove(val)→bool, and getRandom()→val
all in average $O(1)$. getRandom returns each element with equal probability.
Right?"

#

"A few quick questions:

insert on existing key → return false? Yes. remove on missing key → false? Yes.
getRandom won't be called on empty set? Guaranteed."

#

"Let me walk: insert(1)→T, insert(2)→T, getRandom()→1 or 2 with equal prob ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Python set + convert to list for getRandom. getRandom O(n) to build list.

Should I go ahead and optimize?"

```
class _380_InsertDeleteGetRandom_Brute:
    def __init__(self): self.data = set()
    def insert(self, val):
        if val in self.data: return False
        self.data.add(val); return True
    def remove(self, val):
        if val not in self.data: return False
        self.data.discard(val); return True
    def getRandom(self):
        import random; return random.choice(list(self.data))
```

PHASE 3 — OPTIMIZE

"The bottleneck is the O(n) list conversion for getRandom."

Array + HashMap: array holds all values, map holds {val→array_index}.

getRandom: pick a random array index — O(1).

remove: swap target with last element, update map, pop last — O(1).

#

Time: O(1) all ops. Space: O(n)."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
import random
class _380_InsertDeleteGetRandom:
    def __init__(self):
        self._vals = []
        self._idx = {}

    def insert(self, val):
        if val in self._idx: return False
        self._idx[val] = len(self._vals)
        self._vals.append(val)
        return True

    def remove(self, val):
        if val not in self._idx: return False
        last = self._vals[-1]
        i = self._idx[val]
        self._vals[i] = last
        self._idx[last] = i
        self._vals.pop()
        del self._idx[val]
        return True

    def getRandom(self):
        return random.choice(self._vals)
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

```
# insert(1): vals=[1],idx={1:0}. insert(2): vals=[1,2],idx={1:0,2:1}.
# remove(1): last=2,i=0. vals[0]=2,idx[2]=0. pop→vals=[2],idx={2:0}. del idx[1]. ✓
# getRandom()→choice([2])→2 ✓
#
# "No bugs. The swap-with-last trick avoids O(n) shifts when removing from the
middle."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "All ops O(1) average: array ops + dict ops. Space O(n).
# Edge case: when removing the last element, swap is a no-op (last==val).
# Follow-up: Insert Delete GetRandom with duplicates (#381) – store sets of indices
per value.
# Follow-up: weighted random – prefix-sum array + binary search for getRandom."
```

Community Solutions (Python)

• WalkCC

```
class RandomizedSet:
    def __init__(self):
        self.vals = []
        self.valToIndex = collections.defaultdict(int) # {val: index in vals}

    def insert(self, val: int) -> bool:
        if val in self.valToIndex:
            return False
        self.valToIndex[val] = len(self.vals)
        self.vals.append(val)

        return True

    def remove(self, val: int) -> bool:
        if val not in self.valToIndex:
            return False
        index = self.valToIndex[val]
        # The order of the following two lines is important when vals.size() == 1.
        self.valToIndex[self.vals[-1]] = index
        del self.valToIndex[val]
        self.vals[index] = self.vals[-1]
```

```

self.vals.pop()
return True

def getRandom(self) -> int:
    index = random.randint(0, len(self.vals) - 1)
    return self.vals[index]

```

• LeetCode

```

class RandomizedSet:
    def __init__(self):
        self.d = {}
        self.q = []

    def insert(self, val: int) -> bool:
        if val in self.d:
            return False
        self.d[val] = len(self.q)
        self.q.append(val)

        return True

    def remove(self, val: int) -> bool:
        if val not in self.d:
            return False
        i = self.d[val]
        self.d[self.q[-1]] = i
        self.q[i] = self.q[-1]
        self.q.pop()
        self.d.pop(val)

        return True

    def getRandom(self) -> int:
        return choice(self.q)

# Your RandomizedSet object will be instantiated and called as such:
# obj = RandomizedSet()
# param_1 = obj.insert(val)
# param_2 = obj.remove(val)

# param_3 = obj.getRandom()

```


• SimplyLeet

```
import random

class RandomizedSet:
    def __init__(self):
        # List to store the values
        self.values = []
        # Dictionary to map a value to its index in the list
        self.val_to_index = {}

    def insert(self, val: int) -> bool:

        # If the value is already present, do not insert.
        if val in self.val_to_index:
            return False
        # Append the value and store its index.
        self.val_to_index[val] = len(self.values)
        self.values.append(val)
        return True

    def remove(self, val: int) -> bool:
        # If the value is not present, return False.

        if val not in self.val_to_index:
            return False
        # Index of element to remove.
        index = self.val_to_index[val]
        # Get the last element.
        last_element = self.values[-1]
        # Place the last element at the index of the element to remove.
        self.values[index] = last_element
        self.val_to_index[last_element] = index
        # Remove the last element from the list.

        self.values.pop()
        # Delete the mapping for the removed element.
        del self.val_to_index[val]
        return True

    def getRandom(self) -> int:
        # Return a random element from the list.
        return random.choice(self.values)
```

• LC.ca

```

'''
my_dict.pop('b') # vs del d[k]

my_dict = {'a': 1, 'b': 2, 'c': 3}
val = my_dict.pop('b')
print(my_dict) # {'a': 1, 'c': 3}
print(val) # 2

>>> item = my_dict.pop()
Traceback (most recent call last):

  File "<stdin>", line 1, in <module>
TypeError: pop expected at least 1 argument, got 0

>>> my_dict = {'a': 1, 'b': 2, 'c': 3}
>>> del my_dict['b']
>>> my_dict
{'a': 1, 'c': 3}
'''

from collections import defaultdict

from random import choice

class RandomizedSet(object):
    def __init__(self):
        """
        Initialize your data structure here.
        """

        # value (map) -> its index -> value (list)
        self.d = {}

```

```
self.a = [] # introduce it purely for random

def insert(self, val):
    """
    Inserts a value to the set. Returns true if the set did not already contain
    the specified element.
    :type val: int
    :rtype: bool
    """
    if val in self.d:
        return False

    self.a.append(val)
    self.d[val] = len(self.a) - 1
    return True

def remove(self, val):
    """
    Removes a value from the set. Returns true if the set contained the
    specified element.
    :type val: int
    :rtype: bool
    """
    if val not in self.d:
        return False
    index = self.d[val]

    # process last index/val
    self.a[index] = self.a[-1]
    self.d[self.a[-1]] = index

    # process to be deleted index/val
    self.a.pop() # delete in list
```

```

del self.d[val] # delete in dict
# or, self.d.pop(val)
return True

def getRandom(self):
    """
    Get a random element from the set.
    :rtype: int
    """
    return self.a[random.randrange(0, len(self.a))]

```

```

# return random.choice(self.a)

```

```

#####

```

```

# Your RandomizedSet object will be instantiated and called as such:
# obj = RandomizedSet()
# param_1 = obj.insert(val)
# param_2 = obj.remove(val)
# param_3 = obj.getRandom()

```

```

...
>>> import random
>>> random.choice([1,2,3,4,5])
3
>>> random.choice([1,2,3,4,5])
4
>>> random.choice([1,2,3,4,5])
4
...

```

```

class RandomizedSet:

```

```

def __init__(self):
    self.m = {}
    self.l = []

def insert(self, val: int) -> bool:
    if val in self.m:
        return False
    self.m[val] = len(self.l)
    self.l.append(val)
    return True

```

```
def remove(self, val: int) -> bool:
    if val not in self.m:
        return False
    idx = self.m[val]
    self.l[idx] = self.l[-1]
    self.m[self.l[-1]] = idx
    self.l.pop()
    self.m.pop(val)
    return True
```

```
def getRandom(self) -> int:
    return random.choice(self.l)
```

```
# Your RandomizedSet object will be instantiated and called as such:
# obj = RandomizedSet()
# param_1 = obj.insert(val)
# param_2 = obj.remove(val)
# param_3 = obj.getRandom()
```

◆ Quick Review

Key Insights

- Use a dynamic array (or list) to store the elements to allow $O(1)$ access by index.
- Use a hash table (or dictionary/map) to store the mapping from value to its index in the array.
- For removal, swap the element to be removed with the last element in the array, then delete the last element. This avoids expensive shifting of elements.

- **getRandom** can be implemented by generating a random index into the array.

Space & Time

Time Complexity: $O(1)$ on average for insert, remove, and getRandom operations.

Space Complexity: $O(n)$ where n is the number of elements stored in the data structure.

Solution

The solution involves maintaining a list to store the values and a hash map to quickly check for the presence of a value and to know its index in the list. When an element is removed, swap it with the last element of the list and update the map accordingly; then, remove the last element in $O(1)$ time. This approach ensures that insert, remove, and getRandom can all be achieved in constant time on average.

Math

□ #1017 Convert to Base -2

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode

Math

Lists: Denny Zhang

Problem

Given an integer n , return a binary string representing its representation in base -2 .

Note that the returned string should not have leading zeros unless the string is "0".

Example 1:

Input: $n = 2$
Output: "110"
Explantion: $(-2)^2 + (-2)^1 = 2$

Example 2:

Input: $n = 3$
Output: "111"
Explantion: $(-2)^2 + (-2)^1 + (-2)^0 = 3$

Example 3:

Input: $n = 4$
Output: "100"
Explantion: $(-2)^2 = 4$

Constraints:

● $0 \leq n \leq 109$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Given integer n, return its representation in base -2 as a binary string.
# No leading zeros (except '0' itself). Right?"
#
# "A few quick questions:
# n=0 → '0'? Yes.  $(-2)^0=1$ ,  $(-2)^1=-2$ ,  $(-2)^2=4$ .
# n=2:  $4+(-2)=2$  → '110' ✓"
#
# "Let me walk: n=2→'110' ✓. n=3→'111' ✓. n=4→'100' ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Repeated division by -2; remainder must be 0 or 1.
#  $O(\log n)$  – this IS optimal."
class _1017_ConvertToBaseNeg2_Brute:
    def baseNeg2(self, n):
        if n == 0: return '0'
        bits = []
        while n:
            remainder = n % 2; bits.append(str(remainder)); n = -(n // 2)
        return ''.join(reversed(bits))
```

PHASE 3 — OPTIMIZE

```
# "The brute already is  $O(\log n)$ .
# The bottleneck is handling the negative base carefully.
#  $n \& 1$  gives the bit.  $n = -(n >> 1)$  handles the negative base division."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _1017_ConvertToBaseNeg2:
    def baseNeg2(self, n):
        if n == 0: return '0'
        bits = []
        while n:
            bits.append(str(n & 1))
            n = -(n >> 1)
        return ''.join(reversed(bits))
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
```

```
#
# n=2: bits=[0]: 2&1=0,n=-(2>>1)=-1. bits=[1]: -1&1=1,n=-(-1>>1)=0(wait: -1>>1=-1,
-(-1)=1).
# n=1: 1&1=1,bits=[1],n=0. bits=['0','1','1'] → '110' ✓
#
# n=3: 3&1=1,n=-1. -1&1=1,n=1. 1&1=1,n=0. bits=['1','1','1'] → '111' ✓
#
# "No bugs. n = -(n >> 1) implements division by -2 in the negative base."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(\log n)$: number of bits. Space $O(\log n)$ for the bits list.

The $n \& 1$ trick extracts the remainder correctly for negative n in Python.

Follow-up: general negative base – replace 2 with $|base|$, negate on each step.

Follow-up: decode a base- (-2) string back to integer – multiply by $(-2)^{position}$."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def baseNeg2(self, n: int) -> str:
        ans = []

        while n != 0:
            ans.append(str(n % 2))
            n = -(n >> 1)

        return ''.join(reversed(ans)) if ans else '0'
```

• LeetDoocs

```
class Solution:
    def baseNeg2(self, n: int) -> str:
        k = 1
        ans = []
        while n:
            if n % 2:
                ans.append('1')
                n -= k
            else:
                ans.append('0')

            n //= 2
            k *= -1
        return ''.join(ans[::-1]) or '0'
```

• SimplyLeet

```
# Python solution with detailed comments
def baseNeg2(n: int) -> str:
    # Special case for zero as input.
    if n == 0:
        return "0"

    digits = [] # List to store digits in base -2.

    while n != 0:
        n, remainder = divmod(n, -2) # Get quotient and remainder when
        # dividing by -2.

        if remainder < 0:
            # Adjust the remainder and increment the quotient.
            remainder += 2
            n += 1
        # Append the remainder (digit) to the list.
        digits.append(str(remainder))

    # Reverse the digits to form the proper binary string and return.
    return "".join(reversed(digits))

# Example usage:
print(baseNeg2(2)) # Output: "110"
print(baseNeg2(3)) # Output: "111"
print(baseNeg2(4)) # Output: "100"
```

• LC.ca

```
class Solution:
    def baseNeg2(self, n: int) -> str:
        k = 1
        ans = []
        while n:
            if n % 2:
                ans.append('1')
                n -= k
            else:
                ans.append('0')
```

```
n //= 2
k *= -1
return ''.join(ans[::-1]) or '0'
```

◆ Quick Review

Key Insights

- Use division and remainder operations to simulate converting a number to a different base.
- When working with a negative base (-2), the remainder can become negative; adjust the remainder (by adding the base's absolute value) and update the quotient accordingly.
- For $n = 0$, simply return "0" since there is nothing to convert.
- Build the answer by collecting digits (remainders) and then reversing the order since the conversion provides digits from least significant to most significant.

Space & Time

Time Complexity: $O(\log|n|)$ – the loop runs for about the number of digits in the base -2 representation.

Space Complexity: $O(\log|n|)$ – extra space for storing the computed digits.

Solution

To solve the problem, we simulate the conversion process similar to how you would convert an integer to a positive base. However, since the base here is -2 , special handling of negative remainders is needed. In each iteration, we: Divide the current number by -2 . Calculate the remainder. If the remainder is negative, adjust it by adding 2, and increment the quotient to compensate. Append the computed remainder to a list (which represents the binary digit). Continue until the number reduces to zero. Finally, reverse the list to form the correct string representation from the most significant to the least significant digit.

Math

□ #3160 Find the Number of Distinct Colors Among the Balls

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Hash Table · Simulation
Lists: CodeSignal

Problem

You are given an integer limit and a 2D array queries of size $n \times 2$.

There are $\text{limit} + 1$ balls with distinct labels in the range $[0, \text{limit}]$. Initially, all balls are uncolored. For every query in queries that is of the form $[x, y]$, you mark ball x with the color y . After each query, you need to find the number of colors among the balls.

Return an array result of length n , where $\text{result}[i]$ denotes the number of colors after i th query.

Note that when answering a query, lack of a color will not be considered as a color.

Example 1:

Input: limit = 4, queries = [[1,4],[2,5],[1,3],[3,4]]

Output: [1,2,2,3]

Explanation:



- After query 0, ball 1 has color 4.
- After query 1, ball 1 has color 4, and ball 2 has color 5.
- After query 2, ball 1 has color 3, and ball 2 has color 5.
- After query 3, ball 1 has color 3, ball 2 has color 5, and ball 3 has color 4.

Example 2:

Input: limit = 4, queries =
[[0,1],[1,2],[2,2],[3,4],[4,5]]

Output: [1,2,2,3,4]

Explanation:



- After query 0, ball 0 has color 1.
- After query 1, ball 0 has color 1, and ball 1 has color 2.
- After query 2, ball 0 has color 1, and balls 1 and 2 have color 2.
- After query 3, ball 0 has color 1, balls 1 and 2 have color 2, and ball 3 has color 4.
- After query 4, ball 0 has color 1, balls 1 and 2 have color 2, ball 3 has color 4, and ball 4 has color 5.

Constraints:

- $1 \leq \text{limit} \leq 10^9$
- $1 \leq n == \text{queries.length} \leq 10^5$
- $\text{queries}[i].\text{length} == 2$
- $0 \leq \text{queries}[i][0] \leq \text{limit}$
- $1 \leq \text{queries}[i][1] \leq 10^9$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# N balls (0 to limit). queries[i]=[ball,color]: paint that ball.
# After each query, return the number of distinct colors among painted balls.
# Right?"
#
# "A few quick questions:
# Unpainted balls have no color – don't count? Correct.
# Can a ball be repainted? Yes – overwrites previous color."
#
# "Let me walk: queries=[[1,4],[2,5],[1,3],[3,4]] → [1,2,2,3] ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Dict ball→color. After each query, scan all values for distinct count.
# O(n) per query, O(n²) total.
# Should I go ahead and optimize?"
class _3160_FindDistinctColors_Brute:
    def queryResults(self, limit, queries):
        ball_color = {}; result = []
        for ball, color in queries:
            ball_color[ball] = color
            result.append(len(set(ball_color.values())))
        return result
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the O(n) set scan for distinct colors.
# color_count tracks how many balls have each color.
# On each query: if ball had a color, decrement old color count (remove if 0).
# Add new color to color_count. Distinct count = len(color_count).
#
# Time: O(1) per query. Space: O(n)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _3160_FindDistinctColors:
    def queryResults(self, limit, queries):
        ball_color = {}
        color_count = {}
        result = []
        for ball, color in queries:
            if ball in ball_color:
                old = ball_color[ball]
                color_count[old] -= 1
                if color_count[old] == 0:
                    del color_count[old]
```

```

        ball_color[ball] = color
        color_count[color] = color_count.get(color, 0) + 1
        result.append(len(color_count))
    return result

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

queries=[[1,4],[2,5],[1,3],[3,4]]:

[1,4]: ball_color={1:4},color_count={4:1}. distinct=1 ✓

[2,5]: color_count={4:1,5:1}. distinct=2 ✓

[1,3]: old=4→count[4]=0 del. color_count={5:1,3:1}. distinct=2 ✓

[3,4]: color_count={5:1,3:1,4:1}. distinct=3 ✓

#

"No bugs. Deleting color_count[old] when it hits 0 keeps len(color_count) correct."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"O(1) per query. Space O(unique balls + colors).

color_count keys = currently active distinct colors – len gives the answer directly.

Follow-up: range coloring (paint balls l..r) – needs segment tree or lazy propagation.

Follow-up: undo queries – maintain a stack of (ball, old_color, new_color)."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def queryResults(self, limit: int, queries: list[list[int]]) -> list[int]:
        ans = []
        ballToColor = {}
        colorCount = collections.Counter()

        for ball, color in queries:
            if ball in ballToColor:
                prevColor = ballToColor[ball]
                colorCount[prevColor] -= 1

                if colorCount[prevColor] == 0:
                    del colorCount[prevColor]
            ballToColor[ball] = color
            colorCount[color] += 1
            ans.append(len(colorCount))

        return ans

```

• LeetDoocs

```
class Solution:
    def queryResults(self, limit: int, queries: List[List[int]]) -> List[int]:
        g = {}
        cnt = Counter()
        ans = []
        for x, y in queries:
            cnt[y] += 1
            if x in g:
                cnt[g[x]] -= 1
                if cnt[g[x]] == 0:
                    cnt.pop(g[x])
            g[x] = y
            ans.append(len(cnt))
        return ans
```

• SimplyLeet

Python solution with detailed comments

```
def distinctColors(limit, queries):
    # Dictionary to keep track of the current color of each ball
    ball_color = {}
    # Dictionary to count the frequency of each color applied
    color_count = {}
    # List to store the results after each query
    results = []

    # Process each query one by one
    for ball, color in queries:
        # If the ball has been colored before, update the old color frequency
        if ball in ball_color:
            old_color = ball_color[ball]
            color_count[old_color] -= 1 # Decrement the count of the old color
            # If the frequency becomes zero, remove the color from the
            dictionary
            if color_count[old_color] == 0:
                del color_count[old_color]
```

```

    # Update ball's color to the new color
    ball_color[ball] = color
    # Increase the frequency count of the new color
    color_count[color] = color_count.get(color, 0) + 1

    # The number of distinct colors is the number of keys in color_count
    results.append(len(color_count))

    return results

```

```

# Example usage:
# limit = 4
# queries = [[1,4],[2,5],[1,3],[3,4]]
# print(distinctColors(limit, queries)) # Output: [1,2,2,3]

```

• LC.ca

```

class Solution:
    def queryResults(self, limit: int, queries: List[List[int]]) -> List[int]:
        g = {}
        cnt = Counter()
        ans = []
        for x, y in queries:
            cnt[y] += 1
            if x in g:
                cnt[g[x]] -= 1
                if cnt[g[x]] == 0:
                    cnt.pop(g[x])
            g[x] = y
            ans.append(len(cnt))
        return ans

```

◆ Quick Review

Key Insights

- Use a dictionary (or hash map) to track the current color for each ball.

- Use another dictionary (or hash map) to count the frequency of each color that is actively used.
- When updating the color of a ball, decrement the frequency of the old color, and if its count reaches zero, remove it.
- Increment the frequency of the new color and record the current number of distinct colors.

Space & Time

Time Complexity: $O(n)$ on average, where n is the number of queries, since each query performs $O(1)$ operations.

Space Complexity: $O(n)$ in the worst case for storing the ball-to-color mapping and the color frequency counts.

Solution

The solution primarily uses two hash maps (dictionaries): one to store the current color of each ball and another to store the frequency of each color that has been applied. For every query: Check if the ball already has a color. If yes, decrement the count for that color and remove it if the count becomes zero. Update

the ball's color with the new color and increment the frequency for the new color. After processing the query, the number of keys in the frequency map gives the number of distinct colors currently in use. This approach ensures that each query is processed in $O(1)$ average time.

JavaScript

Round 8 · Low · Medium · 7 questions

JavaScript

□ ★ #2627 Debounce ★

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
JavaScript

Lists: ★ Premium | Premium 98

Problem

Given a function `fn` and a time in milliseconds `t`, return a debounced version of that function.

A debounced function is a function whose execution is delayed by `t` milliseconds and whose execution is cancelled if it is called again within that window of time. The debounced function should also receive the passed parameters.

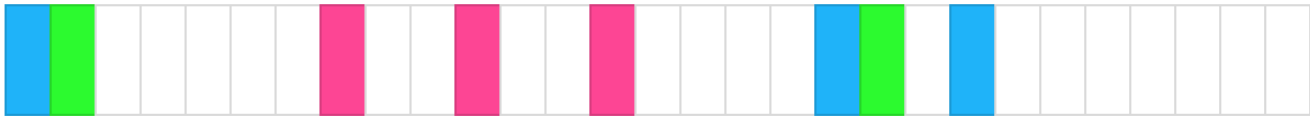
For example, let's say `t = 50ms`, and the function was called at 30ms, 60ms, and 100ms.

The first 2 function calls would be cancelled, and the 3rd function call would be executed at 150ms.

If instead `t = 35ms`, The 1st call would be cancelled, the 2nd would be executed at 95ms,

and the 3rd would be executed at 135ms.

Input Events



Debounced Events



The above diagram shows how debounce will transform events. Each rectangle represents 100ms and the debounce time is 400ms. Each color represents a different set of inputs.

Please solve it without using lodash's `_.debounce()` function.

Example 1:

```
Input:
t = 50
calls = [
  {"t": 50, inputs: [1]},
  {"t": 75, inputs: [2]}
]
Output: [{"t": 125, inputs: [2]}]
Explanation:
```

Example 2:

```
Input:
t = 20
calls = [
  {"t": 50, inputs: [1]},
  {"t": 100, inputs: [2]}
]
Output: [{"t": 70, inputs: [1]}, {"t": 120, inputs: [2]}]
Explanation:
```

Example 3:

```

Input:
t = 150
calls = [
  {"t": 50, inputs: [1, 2]},
  {"t": 300, inputs: [3, 4]},
  {"t": 300, inputs: [5, 6]}
]
Output: [{"t": 200, inputs: [1,2]}, {"t": 450, inputs: [5, 6]}]

```

Constraints:

- $0 \leq t \leq 1000$
- $1 \leq \text{calls.length} \leq 10$
- $0 \leq \text{calls}[i].t \leq 1000$
- $0 \leq \text{calls}[i].\text{inputs.length} \leq 10$

My LeetCode Solution

• My LeetCode Solution

```

/**
 * @param {Function} fn
 * @param {number} t milliseconds
 * @return {Function}
 */
var debounce = function(fn, t) {
  let timer = null;

  return function(...args) {
    clearTimeout(timer);

```

```

        timer = setTimeout(() => fn(...args), t);
    };
};

/**
 * const log = debounce(console.log, 100);
 * log('Hello'); // cancelled
 * log('Hello'); // cancelled
 * log('Hello'); // Logged at t=100ms
 */

```

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

`debounce(fn, t)`: delay `fn`'s execution by `t` ms. Each new call resets the timer.

Only the most recent call fires – earlier pending calls are cancelled. Right?"

#

"A few quick questions: `t` can be 0? Yes – fires in the next microtask. No calls = no fires."

#

"Let me walk: calls at 0ms, 30ms, 60ms with `t=50ms` -> `fn` fires once at 110ms ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

One timer reference, cancel previous on each call – this IS the optimal solution.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE

"The bottleneck is the need to cancel the previous pending call.

`clearTimeout` + `setTimeout` per invocation achieves $O(1)$ per call.

Time: $O(1)$ per call. Space: $O(1)$: one timer reference."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

// JavaScript:

`function debounce(fn, t) {`

`let timerId = null;`

`return function(...args) {`

`clearTimeout(timerId);`

`timerId = setTimeout(() => fn.apply(this, args), t);`

`};`

`}`

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

```
# t=50ms, calls at 0ms, 30ms, 60ms:
# 0ms: cancel(null), set timer(50ms). 30ms: cancel, set timer(80ms). 60ms: cancel,
set timer(110ms).
# fn fires at 110ms ✓. No earlier calls fire.
#
# "No bugs. Each call resets the timer, guaranteeing only the last call executes."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(1) per call. Space O(1): one timer reference.
# Debounce: wait until activity stops. Throttle: fire at most once per interval.
# Follow-up: leading debounce – fire immediately then ignore for t ms.
# Follow-up: Throttle (#2676) – use setInterval or timestamp-based approach."
```

Community Solutions (JavaScript / TypeScript)

• WalkCC

```
type F = (...args: number[]) => void;

function debounce(fn: F, t: number): F {
  let timeout: ReturnType<typeof setTimeout> | undefined;
  return function (...args) {
    clearTimeout(timeout);
    timeout = setTimeout(() => fn(...args), t);
  };
}
```

• LeetDoocs

```
type F = (...p: any[]) => any;

function debounce(fn: F, t: number): F {
  let timeout: ReturnType<typeof setTimeout> | undefined;

  return function (...args) {
    if (timeout !== undefined) {
      clearTimeout(timeout);
    }
    timeout = setTimeout(() => {
```

```

        fn.apply(this, args);
    }, t);
};

}

/**
 * const log = debounce(console.log, 100);
 * log('Hello'); // cancelled
 * log('Hello'); // cancelled
 * log('Hello'); // Logged at t=100ms
 */

```

• LC.ca

```

type F = (...p: any[]) => any;

function debounce(fn: F, t: number): F {
    let timeout: ReturnType<typeof setTimeout> | undefined;

    return function (...args) {
        if (timeout !== undefined) {
            clearTimeout(timeout);
        }
        timeout = setTimeout(() => {

            fn.apply(this, args);
        }, t);
    };
}

/**
 * const log = debounce(console.log, 100);
 * log('Hello'); // cancelled
 * log('Hello'); // cancelled
 * log('Hello'); // Logged at t=100ms
 */

```

◆ Quick Review

Key Insights

- Each call to the debounced function resets the timer.
- Only the most recent function call (that isn't subsequently cancelled) will eventually execute.
- Passing of parameters is handled by storing the arguments of the latest call.
- The solution leverages timers (or equivalent scheduling mechanisms) and cancellation techniques.

Space & Time

Time Complexity: $O(1)$ per call, as each invocation only clears and sets a timer.

Space Complexity: $O(1)$, since only a single timer reference is maintained.

Solution

The solution creates a wrapper function that maintains an internal timer reference. On each call to the debounced function, if a timer exists, it is cancelled. A new timer is then set to execute the original function after t milliseconds with the provided arguments.

This ensures that only the last invocation runs

if multiple calls occur within the delay period.

JavaScript

□ ★ #2628 JSON Deep Equal ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
JavaScript

Lists: ★ Premium | Premium 98

Problem

Given two values `o1` and `o2`, return a boolean value indicating whether two values, `o1` and `o2`, are deeply equal.

For two values to be deeply equal, the following conditions must be met:

- If both values are primitive types, they are deeply equal if they pass the `===` equality check.
- If both values are arrays, they are deeply equal if they have the same elements in the same order, and each element is also deeply equal according to these conditions.
- If both values are objects, they are deeply equal if they have the same keys, and the associated values for each key are also deeply

equal according to these conditions.

You may assume both values are the output of `JSON.parse`. In other words, they are valid JSON.

Please solve it without using lodash's `_.isEqual()` function

Example 1:

```
Input: o1 = {"x":1,"y":2}, o2 = {"x":1,"y":2}
Output: true
Explanation: The keys and values match exactly.
```

Example 2:

```
Input: o1 = {"y":2,"x":1}, o2 = {"x":1,"y":2}
Output: true
Explanation: Although the keys are in a different order, they still match exactly.
```

Example 3:

```
Input: o1 = {"x":null,"L":[1,2,3]}, o2 = {"x":null,"L":["1","2","3"]}
Output: false
Explanation: The array of numbers is different from the array of strings.
```

Example 4:

```
Input: o1 = true, o2 = false
Output: false
Explanation: true !== false
```

Constraints:

- $1 \leq \text{JSON.stringify}(o1).length \leq 105$
- $1 \leq \text{JSON.stringify}(o2).length \leq 105$
- `maxNestingDepth` ≤ 1000

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Deep equality for JSON values: same primitive value (===), same array structure,
# same object keys and values recursively. Keys in different order are still equal.
Right?"
#
# "A few quick questions: null === null is true. null !== {}? Yes."
#
# "Let me walk: areDeeplyEqual({a:1},{a:1}) -> true. areDeeplyEqual([1],[2]) ->
false ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# JSON.stringify both – fails for key ordering. Recursive comparison is correct.
# Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the key-ordering issue with JSON.stringify.
# Recursive DFS: handle null specially, then dispatch on type.
# Time: O(n). Space: O(d) depth."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
# // JavaScript:
# function areDeeplyEqual(o1, o2) {
#   if (o1 === o2) return true;
#   if (typeof o1 !== typeof o2 || o1 === null || o2 === null) return o1 === o2;
#   if (Array.isArray(o1) !== Array.isArray(o2)) return false;
#   if (Array.isArray(o1))
#     return o1.length === o2.length && o1.every((v, i) => areDeeplyEqual(v, o2[i]));
#   const k1 = Object.keys(o1), k2 = Object.keys(o2);
#   return k1.length === k2.length && k1.every(k => k in o2 && areDeeplyEqual(o1[k],
#   o2[k]));
# }
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# areDeeplyEqual({y:2,x:1},{x:1,y:2}): k1.length==k2.length. each key in o2? yes.
values equal? yes -> true ✓
# areDeeplyEqual(true,false): true===false? No. typeof same. null check: no. return
true===false=false ✓
#
```

"No bugs. Null check must come before typeof dispatch since typeof null === object."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n)$. Space $O(d)$ depth.

Follow-up: handle circular references – track visited pairs with WeakMap.

Follow-up: Differences (#2700) – return diff object instead of bool."

Community Solutions (JavaScript / TypeScript)

• WalkCC

```
type JSONValue =
  | null
  | boolean
  | number
  | string
  | JSONValue[]
  | { [key: string]: JSONValue };

function areDeeplyEqual(o1: JSONValue, o2: JSONValue): boolean {
  if (o1 === o2) {
    return true;
  }
  if (o1 === null || o1 === undefined || o2 === null || o2 === undefined) {
    return false;
  }
  if (typeof o1 !== 'object' || typeof o2 !== 'object') {
    return false;
  }
  if (Array.isArray(o1) !== Array.isArray(o2)) {
    return false;
  }
  if (Object.keys(o1).length !== Object.keys(o2).length) {
    return false;
  }
  for (const key in o1) {
    if (!areDeeplyEqual(o1[key], o2[key])) {
      return false;
    }
  }
  return true;
}
```

}

• LeetDoocs

```
function areDeeplyEqual(o1: any, o2: any): boolean {
  if (o1 === null || typeof o1 !== 'object') {
    return o1 === o2;
  }
  if (typeof o1 !== typeof o2) {
    return false;
  }
  if (Array.isArray(o1) !== Array.isArray(o2)) {
    return false;
  }

  if (Array.isArray(o1)) {
    if (o1.length !== o2.length) {
      return false;
    }
    for (let i = 0; i < o1.length; i++) {
      if (!areDeeplyEqual(o1[i], o2[i])) {
        return false;
      }
    }
    return true;
  }

  } else {
    const keys1 = Object.keys(o1);
    const keys2 = Object.keys(o2);
    if (keys1.length !== keys2.length) {
      return false;
    }
    for (const key of keys1) {
      if (!areDeeplyEqual(o1[key], o2[key])) {
        return false;
      }
    }
  }
  return true;
}
```

◆ Quick Review

Key Insights

- Use recursion to compare nested structures.
- For primitive types, a simple equality check (`===` in JavaScript, `==` in Python, etc.) suffices.
- For arrays, verify both have the same length and compare each element recursively.
- For objects, ensure both have the same keys (order does not matter) then recursively compare the corresponding values.
- Handle cases where one operand is null or differs by type early to avoid unnecessary recursion.

Space & Time

Time Complexity: $O(N)$ in the worst-case, where N is the total number of elements/values in the JSON structure.

Space Complexity: $O(N)$ due to the recursion call stack in the worst-case scenario of deeply nested objects or arrays.

Solution

We solve this problem using a recursive approach. The algorithm first checks if both

values are strictly equal, which handles primitives immediately. For arrays and objects, it verifies that the sizes (length for arrays and number of keys for objects) match. Then it recurses into each element (for arrays) or checks each key–value pair (for objects) to ensure that they are also deeply equal. For objects, the order of keys is irrelevant, so we compare key sets. This method naturally handles nested structures by delegating equality checks to the same function.

JavaScript

□ ★ #2630 Memoize ★

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
JavaScript

Lists: ★ Premium | Premium 98

Problem

Given a function `fn`, return a memoized version of that function.

A memoized function is a function that will never be called twice with the same inputs. Instead it will return a cached value.

`fn` can be any function and there are no constraints on what type of values it accepts. Inputs are considered identical if they are `===` to each other.

Example 1:

Input:

```
getInputs = () => [[2,2],[2,2],[1,2]]
```

```
fn = function (a, b) { return a + b; }
```

Output: [{"val":4,"calls":1},{ "val":4,"calls":1},{ "val":3,"calls":2}]

Explanation:

```
const inputs = getInputs();
```

```
const memoized = memoize(fn);
```

```
for (const arr of inputs) {
```

Example 2:

Input:

```
getInputs = () => [[{}},{},[{}},{},[{}},{}]
```

```
fn = function (a, b) { return ({...a, ...b}); }
```

```
Output: [{"val":{},"calls":1},{"val":{},"calls":2},{"val":{},"calls":3}]
```

Explanation:

Merging two empty objects will always result in an empty object. It may seem like there should only be 1 call to fn() because of cache-hits, however none of those objects are === to each other.

Example 3:

Input:

```
getInputs = () => { const o = {}; return [[o,o],[o,o],[o,o]]; }
```

```
fn = function (a, b) { return ({...a, ...b}); }
```

```
Output: [{"val":{},"calls":1},{"val":{},"calls":1},{"val":{},"calls":1}]
```

Explanation:

Merging two empty objects will always result in an empty object. The 2nd and 3rd third function calls result in a cache-hit. This is because every object passed in is identical.

Constraints:

- $1 \leq \text{inputs.length} \leq 105$
- $0 \leq \text{inputs.flat().length} \leq 105$
- $\text{inputs}[i][j] \neq \text{NaN}$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

memoize(fn): return a cached version that skips fn when same inputs are seen.

Inputs compared with === (object references, not structure). Right?"

#

"A few quick questions: {} === {} is false in JS – two empty objects are different? Yes."

#

"Let me walk: memoized(2,3) first call -> computes. memoized(2,3) again -> cached

✓"

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Store all (args, result) pairs in an array; scan on each call. O(n) per lookup.
# Should I go ahead and optimize?"
# // Array scan is O(n) per call.
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the linear scan for matching args.
# Use a Map with JSON.stringify(args) as key for O(1) lookup.
# Note: JSON.stringify({}) === JSON.stringify({}) so structurally equal objects
share cache.
# For reference-equality semantics, a trie or WeakMap-based structure is needed.
# Time: O(key_len) per call. Space: O(unique calls)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
# // JavaScript:
# function memoize(fn) {
#   const cache = new Map();
#   return function(...args) {
#     const key = JSON.stringify(args);
#     if (!cache.has(key)) cache.set(key, fn.apply(this, args));
#     return cache.get(key);
#   };
# }
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# fn=sum. memoized(2,3): key='[2,3]', miss -> compute 5, cache. return 5.
# memoized(2,3): key='[2,3]', hit -> return 5 (fn not called) ✓
# memoized({},{}) x3 (same objects): key='[{},{}]' each time -> cached after first ✓
#
# "No bugs. JSON.stringify for primitives is exact; for same-reference objects it's
consistent."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "First call O(fn). Cache hit O(key_len). Space O(unique key count).
# Follow-up: LRU-bounded memoize – evict oldest when cache exceeds size limit.
# Follow-up: async memoize – cache the Promise itself so concurrent calls share one
fetch."
```

Community Solutions (JavaScript / TypeScript)

• WalkCC

```

type Fn = (...params: number[]) => number;

function memoize(fn: Fn): Fn {
  const cache: Record<string, number> = {};
  return function (...args) {
    const key = args.join(' ');
    return cache[key] === undefined ? (cache[key] = fn(...args)) : cache[key];
  };
}

```

• LeetDoocs

```

type Fn = (...params: any) => any;

function memoize(fn: Fn): Fn {
  const idxMap: Map<string, number> = new Map();
  const cache: Map<string, any> = new Map();

  const getIdx = (obj: any): number => {
    if (!idxMap.has(obj)) {
      idxMap.set(obj, idxMap.size);
    }

    return idxMap.get(obj)!;
  };

  return function (...params: any) {
    const key = params.map(getIdx).join(',');
    if (!cache.has(key)) {
      cache.set(key, fn(...params));
    }
    return cache.get(key)!;
  };
}

```

```
}

/**
 * let callCount = 0;
 * const memoizedFn = memoize(function (a, b) {
 *   callCount += 1;
 *   return a + b;
 * })
 * memoizedFn(2, 3) // 5
 * memoizedFn(2, 3) // 5

 * console.log(callCount) // 1
 */
```

◆ Quick Review

Key Insights

- Cache previously computed results using a data structure keyed by the function arguments.
- Ensure the key uniquely represents the argument order; simple tuple (or equivalent) suffices.
- Use closures (or object state) to maintain both the cache and a call count tracking actual function invocations.
- Expose a method (getCallCount) on the memoized function to report the number of times the original function was actually called.

- The memoized version works for functions that might take one or more parameters.

Space & Time

Time Complexity: $O(1)$ average for each call assuming efficient hashing of arguments.

Space Complexity: $O(n)$ for caching n distinct input combinations.

Solution

We use a hash-based cache (a dictionary or map) to store results for every unique set of arguments. Upon each call, we check if the key (constructed from the input arguments) exists in the cache. If not, we increment a counter, call the original function, store the result in the cache, and return it. Otherwise, we return the cached result. The memoized function is augmented with a `getCallCount` method to report the number of actual calls made to the original function.

JavaScript

□ ★ #2633 Convert Object to JSON String ★ **MED**

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
JavaScript

Lists: ★ Premium | Premium 98

Problem

Given a value, return a valid JSON string of that value. The value can be a string, number, array, object, boolean, or null. The returned string should not include extra spaces. The order of keys should be the same as the order returned by `Object.keys()`.

Please solve it without using the built-in `JSON.stringify` method.

Example 1:

Input: object = {"y":1,"x":2}

Output: {"y":1,"x":2}

Explanation:

Return the JSON representation.

Note that the order of keys should be the same as the order returned by `Object.keys()`.

Example 2:

Input: object = {"a":"str","b":-12,"c":true,"d":null}
 Output: {"a":"str","b":-12,"c":true,"d":null}
 Explanation:
 The primitives of JSON are strings, numbers, booleans, and null.

Example 3:

Input: object = {"key":{"a":1,"b":[{"},null,"Hello"]}}
 Output: {"key":{"a":1,"b":[{"},null,"Hello"]}}
 Explanation:
 Objects and arrays can include other objects and arrays.

Example 4:

Input: object = true
 Output: true
 Explanation:
 Primitive types are valid inputs.

Constraints:

- value is a valid JSON value
- $1 \leq \text{JSON.stringify(object).length} \leq 105$
- $\text{maxNestingLevel} \leq 1000$
- all strings contain only alphanumeric characters

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."
 # Implement JSON.stringify: handle null, bool, number, string, array, object.
 # No extra spaces. Keys in Object.keys() order. No eval. Right?"
 #
 # "A few quick questions: undefined values in arrays become null? No – constraints say valid JSON only."
 #
 # "Let me walk: jsonStringify({a:2,b:[1,2,3]}) -> {"a":2,"b":[1,2,3]} ✓"

PHASE 2 — BRUTE FORCE**# "Let me start with brute force to lock in the logic."**# Recursive dispatch on type – this is the only approach. $O(n)$ time.

Should I go ahead and optimize?"

PHASE 3 — OPTIMIZE**# "The bottleneck is correct type dispatch."**

Check null first (typeof null === object in JS). Then bool, number, string, array, object.

Use join to avoid $O(n^2)$ string concatenation.# Time: $O(n)$. Space: $O(d)$ depth."**# PHASE 4 — CLEAN CODE****# "Now I'll implement the optimized approach with meaningful names."**

// JavaScript:

function jsonStringify(obj) {

if (obj === null) return 'null';

if (typeof obj === 'boolean') return String(obj);

if (typeof obj === 'number') return String(obj);

if (typeof obj === 'string') return '"' + obj + '"';

if (Array.isArray(obj)) return '[' + obj.map(jsonStringify).join(',') + ']';

return '{' + Object.entries(obj).map(([k,v]) =>

'"' + k + '": ' + jsonStringify(v) + '').join(',') + '}';

}

PHASE 5 — TEST & DEBUG**# "Let me trace through a few cases."**

#

obj=true: 'true' ✓. obj=null: 'null' ✓. obj='hello': '"hello"' ✓

obj=[null,true,1]: '[null,true,1]' ✓. obj={key:{a:1}}: '{"key":{"a":1}}' ✓

#

"No bugs. Null before typeof catches the typeof-null-is-object edge case."**# PHASE 6 — COMPLEXITY & FOLLOW-UP****# "Time $O(n)$: one visit per node. Space $O(d)$ depth."**

Follow-up: circular reference detection – track visited with a Set.

Follow-up: pretty-print – add indentation parameter."

Community Solutions (JavaScript / TypeScript)

• WalkCC

```

type JSONValue =
  | null
  | boolean
  | number
  | string
  | JSONValue[]
  | { [key: string]: JSONValue };

function jsonStringify(object: JSONValue): string {
  if (object === null) {

    return 'null';
  }
  if (typeof object === 'boolean' || typeof object === 'number') {
    return String(object);
  }
  if (typeof object === 'string') {
    return `"${object}"`;
  }
  if (Array.isArray(object)) {
    const elems = object.map((elem) => jsonStringify(elem));

    return `[${elems.join(',')}]`;
  }
  // typeof object === 'object'
  const pairs = Object.keys(object).map(
    (key) => `${key}:${jsonStringify(object[key])}`
  );
  return `{${pairs.join(',')}`;
}

```

• LeetDoocs

```

function jsonStringify(object: any): string {
  if (object === null) {
    return 'null';
  }
  if (typeof object === 'string') {
    return `"${object}"`;
  }
  if (typeof object === 'number' || typeof object === 'boolean') {
    return object.toString();
  }
}

```

```
if (Array.isArray(object)) {
    return `[${object.map(jsonStringify).join(',')}]`;
}
if (typeof object === 'object') {
    return `{${Object.entries(object)
        .map(([key, value]) => `${jsonStringify(key)}:${jsonStringify(value)}`)
        .join(',')}}`;
}
return '';
```

◆ Quick Review

Key Insights

- The solution must support all JSON primitives: strings, numbers, booleans, and null.
- Composite types such as arrays and objects require recursive traversal for proper serializing.
- For strings, wrap them in double quotes.
- For arrays, recursively process every element and join them with commas between square brackets.
- For objects, iterate keys in insertion order and combine key-value pairs (key must be a string in double quotes) with commas between curly braces.

- Use recursion carefully to handle nested structures without adding unnecessary spaces.

Space & Time

Time Complexity: $O(n)$, where n is the total number of elements/characters in the JSON structure.

Space Complexity: $O(n)$ due to the recursion stack and the constructed output string.

Solution

We use a recursive approach that inspects the type of the value and processes it accordingly: If the value is null, output "null". If the value is a boolean or number, convert it directly to its string representation. For a string, ensure it is enclosed in double quotes. For an array, iterate over each element, serialize each recursively, join them with commas, and enclose in square brackets. For an object, iterate over its keys in insertion order (or using `Object.keys` in JS / `LinkedHashMap` in Java), serialize both key (wrapped in double quotes) and its corresponding value, join with commas, and enclose in curly braces. This approach leverages recursion to handle arbitrarily

nested objects and arrays while ensuring there are no extra spaces in the final string.

JavaScript

□ ★ #2636 Promise Pool ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
JavaScript · Concurrency

Lists: ★ Premium | Premium 98

Problem

Given an array of asynchronous functions `functions` and a pool limit `n`, return an asynchronous function `promisePool`. It should return a promise that resolves when all the input functions resolve.

Pool limit is defined as the maximum number promises that can be pending at once. `promisePool` should begin execution of as many functions as possible and continue executing new functions when old promises resolve. `promisePool` should execute `functions[i]` then `functions[i + 1]` then `functions[i + 2]`, etc. When the last promise resolves, `promisePool` should also resolve.

For example, if $n = 1$, `promisePool` will execute one function at a time in series. However, if $n = 2$, it first executes two functions. When either of the two functions resolve, a 3rd function should be executed (if available), and so on until there are no functions left to execute.

You can assume all functions never reject. It is acceptable for `promisePool` to return a promise that resolves any value.

Example 1:

```
Input:
functions = [
  () => new Promise(res => setTimeout(res, 300)),
  () => new Promise(res => setTimeout(res, 400)),
  () => new Promise(res => setTimeout(res, 200))
]
n = 2
Output: [[300,400,500],500]
```

Example 2:

```
Input:
functions = [
  () => new Promise(res => setTimeout(res, 300)),
  () => new Promise(res => setTimeout(res, 400)),
  () => new Promise(res => setTimeout(res, 200))
]
n = 5
Output: [[300,400,200],400]
```

Example 3:

```

Input:
functions = [
  () => new Promise(res => setTimeout(res, 300)),
  () => new Promise(res => setTimeout(res, 400)),
  () => new Promise(res => setTimeout(res, 200))
]
n = 1
Output: [[300,700,900],900]

```

Constraints:

- $0 \leq \text{functions.length} \leq 10$
- $1 \leq n \leq 10$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```

# "Let me make sure I understand the problem correctly.
# Execute async functions with at most n concurrent at any time.
# Return a promise resolving when all functions complete. Right?"
#
# "A few quick questions: Functions never reject? Yes per constraints."
#
# "Let me walk: 3 functions, n=2: run fn1+fn2 concurrently. As each finishes, start
fn3 ✓"

```

PHASE 2 — BRUTE FORCE

```

# "Let me start with brute force to lock in the logic.
# Promise.all runs all immediately – no concurrency limit. O(n) but may overwhelm
resources.
# Should I go ahead and optimize?"

```

PHASE 3 — OPTIMIZE

```

# "The bottleneck is unlimited concurrency.
# n worker coroutines each pull from a shared queue. n workers run in parallel.
# Time: optimal. Space: O(n)."

```

PHASE 4 — CLEAN CODE

```

# "Now I'll implement the optimized approach with meaningful names."
# // JavaScript:
# async function promisePool(functions, n) {
#   const queue = [...functions];
#   async function worker() {

```

```
# while (queue.length) await queue.shift>();
# }
# await Promise.all(Array.from({length: Math.min(n, functions.length)}, worker));
# }

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# [fn1,fn2,fn3,fn4], n=2: worker1 runs fn1+fn3, worker2 runs fn2+fn4. At most 2
concurrent ✓
#
# "No bugs. Shared queue.shift() is safe in JS single-threaded event loop."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time optimal. Space O(n) concurrent promises.
# Follow-up: cancel on first failure – reject the outer promise on first rejection.
# Follow-up: retry failed tasks – wrap fn() in retry loop inside the worker."
```

Community Solutions (JavaScript / TypeScript)

• WalkCC

```
type F = () => Promise<any>;

function promisePool(functions: F[], n: number): Promise<any> {
  const next = () => functions[n++]?.().then(next);
  return Promise.all(functions.slice(0, n).map((f) => f().then(next)));
}
```

• LeetDoocs

```
type F = () => Promise<any>;

function promisePool(functions: F[], n: number): Promise<any> {
  const wrappers = functions.map(fn => async () => {
    await fn();
    const nxt = waiting.shift();
    nxt && (await nxt());
  });

  const running = wrappers.slice(0, n);

  const waiting = wrappers.slice(n);
  return Promise.all(running.map(fn => fn()));
}
```

• LC.ca

```
type F = () => Promise<any>;

function promisePool(functions: F[], n: number): Promise<any> {
  const wrappers = functions.map(fn => async () => {
    await fn();
    const nxt = waiting.shift();
    nxt && (await nxt());
  });

  const running = wrappers.slice(0, n);

  const waiting = wrappers.slice(n);
  return Promise.all(running.map(fn => fn()));
}
```

◆ Quick Review

Key Insights

- Use a concurrency control mechanism to limit the number of actively running promises.
- Maintain an index or pointer to track which function from the input array should be executed next.
- Upon the resolution of any promise, start a new one if available until all functions have been processed.
- The overall task is complete when there are no functions left and all running promises have resolved.

Space & Time

Time Complexity: $O(m)$, where m is the number of functions to execute.

Space Complexity: $O(n)$ for the active pool of concurrent promises.

Solution

We can solve the problem by maintaining an index that tracks which asynchronous function to execute next. We define a helper routine (or executor) that: Checks if there is a function left to run. If yes, increments the index and awaits the result of running that function. Once the function completes, the routine recursively calls itself to pick up the next function. We start n such routines (to respect the pool limit) and use `Promise.all` (or the equivalent in other languages) to wait until all concurrent routines have finished executing. This structure ensures that we never run more than n promises concurrently and that a new promise starts as soon as one finishes.

JavaScript

□ ★ #2675 Array of Objects to Matrix ★

MED

[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
JavaScript · Array

Lists: ★ Premium | Premium 98

Problem

Write a function that converts an array of objects `arr` into a matrix `m`.

`arr` is an array of objects or arrays. Each item in the array can be deeply nested with child arrays and child objects. It can also contain numbers, strings, booleans, and null values.

The first row `m` should be the column names. If there is no nesting, the column names are the unique keys within the objects. If there is nesting, the column names are the respective paths in the object separated by ".".

Each of the remaining rows corresponds to an object in `arr`. Each value in the matrix corresponds to a value in an object. If a given object doesn't contain a value for a given

column, the cell should contain an empty string "".

The columns in the matrix should be in lexicographically ascending order.

Example 1:

```
Input:
arr = [
  {"b": 1, "a": 2},
  {"b": 3, "a": 4}
]
Output:
[
  ["a", "b"],
```

Example 2:

```
Input:
arr = [
  {"a": 1, "b": 2},
  {"c": 3, "d": 4},
  {}
]
Output:
[
```

Example 3:

```
Input:
arr = [
  {"a": {"b": 1, "c": 2}},
  {"a": {"b": 3, "d": 4}}
]
Output:
[
  ["a.b", "a.c", "a.d"],
```

Example 4:

```

Input:
arr = [
  [{"a": null}],
  [{"b": true}],
  [{"c": "x"}]
]
Output:
[

```

Example 5:

```

Input:
arr = [
  {},
  {},
  {}
]
Output:
[

```

Constraints:

- arr is a valid JSON array
- $1 \leq \text{arr.length} \leq 1000$
- unique keys ≤ 1000

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Convert array of objects to a matrix. Row 0 = sorted flattened keys (dot notation).

Remaining rows = values for each object, empty string if key missing. Right?"

#

"A few quick questions: Arrays inside objects use numeric index as key? Yes."

#

"Let me walk: [{a:1,b:2},{b:3,c:4}] produces header row [a,b,c] and value rows ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic."

Flatten each object with dot notation, collect all keys, build matrix row by row.

```
# O(n*k*d). Should I go ahead and optimize?"
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is computing flattened keys per object.
# One-pass DFS flatten each object. Collect unique keys sorted. Build matrix.
# Time: O(n*k). Space: O(n*k)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
# // JavaScript:
# function jsonToMatrix(arr) {
#   function flatten(obj, prefix) {
#     const res = {};
#     for (const [k, v] of Object.entries(obj)) {
#       const key = prefix ? prefix + '.' + k : k;
#       if (v !== null && typeof v === 'object') Object.assign(res, flatten(v, key));
#       else res[key] = v;
#     }
#     return res;
#   }
#   const flat = arr.map(o => flatten(o, ''));
#   const keys = [...new Set(flat.flatMap(Object.keys))].sort();
#   return [keys, ...flat.map(o => keys.map(k => k in o ? o[k] : ''))];
# }
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# [{a:1},{a:2,b:3}]: keys=['a','b']. rows: [1,''] and [2,3]. header+rows correct ✓
# [{a:{b:1}},{a:{c:2}}]: keys=['a.b','a.c']. rows: [1,''] and ['',2] ✓
#
# "No bugs. Dot notation is the standard format for nested JSON export."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n*k*d). Space O(n*k) matrix.
# Follow-up: handle arrays with bracket notation [0],[1].
# Follow-up: reverse – matrix to array of objects via dot-path parsing."
```

Community Solutions (JavaScript / TypeScript)

• WalkCC

```
function jsonToMatrix(arr: any[]): (string | number | boolean | null)[][] {
  const isObject = (o: any) => o !== null && typeof o === 'object';

  // Returns the keys of a JSON-like object by recursively unwrapping the
  nests.
  const getKeys = (json: any): string[] => {
    if (!isObject(json)) {
      return [''];
    }
    return Object.keys(json).reduce((acc: string[], currKey: string) => {
      return (
        acc.push(
          ...getKeys(json[currKey]).map((nextKey: string) =>
            nextKey === '' ? currKey : `${currKey}.${nextKey}`
          )
        ),
        acc
      );
    }, []);
  };

  const sortedKeys: string[] = [
    ...arr.reduce((acc: Set<string>, curr: any) => {
      getKeys(curr).forEach((key: string) => acc.add(key));
      return acc;
    }, new Set<string>()),
  ].sort();

  // Returns the value of `obj` keyed by `nestedKey`.
  const getValue = (
    obj: any,
```

```

    nestedKey: string
  ): string | number | boolean | null => {
    let value: any = obj;
    for (const key of nestedKey.split('.')) {
      if (!isObject(value) || !(key in value)) {
        return '';
      }
      value = value[key];
    }
    return isObject(value) ? '' : value;
  };

  const matrix: (string | number | boolean | null)[][] = [sortedKeys];
  arr.forEach((obj: any) => {
    matrix.push(
      sortedKeys.map((nestedKey: string) => getValue(obj, nestedKey))
    );
  });

  return matrix;
}

```

• LeetDoocs

```

function jsonToMatrix(arr: any[]): (string | number | boolean | null)[] {
  const dfs = (key: string, obj: any) => {
    if (
      typeof obj === 'number' ||
      typeof obj === 'string' ||
      typeof obj === 'boolean' ||
      obj === null
    ) {
      return { [key]: obj };
    }
  }
}

```

```

    const res: any[] = [];
    for (const [k, v] of Object.entries(obj)) {
        const newKey = key ? `${key}.${k}` : `${k}`;
        res.push(dfs(newKey, v));
    }
    return res.flat();
};

const kv = arr.map(obj => dfs('', obj));
const keys = [

```

```

    ...new Set(
        kv
            .flat()
            .map(obj => Object.keys(obj))
            .flat(),
    ),
].sort();
const ans: any[] = [keys];
for (const row of kv) {
    const newRow: any[] = [];

```

```

        for (const key of keys) {
            const v = row.find(r => r.hasOwnProperty(key))?.[key];
            newRow.push(v === undefined ? '' : v);
        }
        ans.push(newRow);
    }
    return ans;
}

```

• LC.ca

```

function jsonToMatrix(arr: any[]): (string | number | boolean | null)[][] {
    const dfs = (key: string, obj: any) => {
        if (
            typeof obj === 'number' ||
            typeof obj === 'string' ||
            typeof obj === 'boolean' ||
            obj === null
        ) {
            return { [key]: obj };
        }
    }
}

```

```

const res: any[] = [];
for (const [k, v] of Object.entries(obj)) {
  const newKey = key ? `${key}.${k}` : `${k}`;
  res.push(dfs(newKey, v));
}
return res.flat();
};

```

```

const kv = arr.map(obj => dfs('', obj));
const keys = [

```

```

  ...new Set(
    kv
      .flat()
      .map(obj => Object.keys(obj))
      .flat(),
  ),
].sort();

```

```

const ans: any[] = [keys];
for (const row of kv) {
  const newRow: any[] = [];

```

```

    for (const key of keys) {
      const v = row.find(r => r.hasOwnProperty(key))?.[key];
      newRow.push(v === undefined ? '' : v);
    }
    ans.push(newRow);
  }
  return ans;
}

```

◆ Quick Review

Key Insights

- Flatten each object or array into dot-separated paths such as `a.b` or `a.0.c`.
- Use DFS so nested objects and nested arrays are handled uniformly.

- For each row, store a map from flattened path to value.
- Collect every path seen across all rows into a set of column names.
- Sort the column names lexicographically to build the header row.
- For each flattened row map, output values in header order and use an empty string when a column is missing.

Solution

We solve the problem by flattening every input row first. A DFS walks through objects and arrays, extending the current path with either the property name or the array index. When the DFS reaches a primitive value or `null`, it stores that value under the built path. While flattening each row, we also collect every path into a global set of column names. After all rows are processed, we sort the columns lexicographically, place them in the first row of the matrix, and then build each remaining row by reading values from that row's flattened map, using `""` for missing paths.

JavaScript

□ ★ #2700 Differences Between Two Objects ★

MED[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
JavaScript

Lists: ★ Premium | Premium 98

Problem

Write a function that accepts two deeply nested objects or arrays `obj1` and `obj2` and returns a new object representing their differences.

The function should compare the properties of the two objects and identify any changes. The returned object should only contains keys where the value is different from `obj1` to `obj2`.

For each changed key, the value should be represented as an array [`obj1` value, `obj2` value]. Keys that exist in one object but not in the other should not be included in the returned object. The end result should be a deeply nested object where each leaf value is a difference array.

When comparing two arrays, the indices of the arrays are considered to be their keys.

You may assume that both objects are the output of `JSON.parse`.

Example 1:

Input:

```
obj1 = {}
```

```
obj2 = {
```

```
  "a": 1,
```

```
  "b": 2
```

```
}
```

Output: `{}`

Explanation: There were no modifications made to `obj1`. New keys "a" and "b" appear in `obj2`, but keys that are added or removed should be ignored.

Example 2:

Input:

```
obj1 = {
```

```
  "a": 1,
```

```
  "v": 3,
```

```
  "x": [],
```

```
  "z": {
```

```
    "a": null
```

```
}
```

Example 3:

Input:

```
obj1 = {
```

```
  "a": 5,
```

```
  "v": 6,
```

```
  "z": [1, 2, 4, [2, 5, 7]]
```

```
}
```

```
obj2 = {
```

```
  "a": 5,
```

Example 4:

```

Input:
obj1 = {
  "a": {"b": 1},
}
obj2 = {
  "a": [5],
}
Output:

```

Example 5:

```

Input:
obj1 = {
  "a": [1, 2, {}],
  "b": false
}
obj2 = {
  "b": false,
  "a": [1, 2, {}]
}

```

Constraints:

- obj1 and obj2 are valid JSON objects or arrays
- $2 \leq \text{JSON.stringify(obj1).length} \leq 104$
- $2 \leq \text{JSON.stringify(obj2).length} \leq 104$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly."

Find where two JSON values differ. Keys only in ONE object are excluded.

Leaf differences: [obj1_val, obj2_val]. Nested diffs form nested result objects. Right?"

#

"A few quick questions: Arrays compared by index? Yes – treated like objects."

#

"Let me walk: diff({a:1,b:2},{a:1,b:3}) gives {b:[2,3]} ✓"

PHASE 2 — BRUTE FORCE

```

# "Let me start with brute force to lock in the logic.
# Recursive traversal checking keys present in both – this is the solution.
# O(n) time, O(d) space."

# PHASE 3 — OPTIMIZE
# "The bottleneck is handling null and type mismatches.
# Base case: non-object or null values – return {} if equal, [v1,v2] if different.
# Recursive: only descend into keys present in BOTH objects.
# Time: O(n). Space: O(d) depth."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."
# // JavaScript:
# function objDiff(obj1, obj2) {
#   if (typeof obj1 !== 'object' || obj1 === null ||
#       typeof obj2 !== 'object' || obj2 === null)
#     return obj1 === obj2 ? {} : [obj1, obj2];
#   const result = {};
#   for (const k of Object.keys(obj1)) {
#     if (!(k in obj2)) continue;
#     const diff = objDiff(obj1[k], obj2[k]);
#     if (Object.keys(diff).length || Array.isArray(diff)) result[k] = diff;
#   }
#   return result;
# }

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# obj1={a:1,b:{c:2,d:3}}, obj2={a:1,b:{c:2,d:4}}:
# a: equal -> {}. b: both objects. c: equal. d: 3!=4 -> [3,4]. result={b:{d:[3,4]}}
✓
#
# "No bugs. typeof null === object in JS requires explicit null check before
recursion."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(d) depth.
# Follow-up: include keys only in one object as [value, undefined].
# Follow-up: JSON Deep Equal (#2628) – same traversal but returns bool."

```

Community Solutions (JavaScript / TypeScript)

• WalkCC

```

type JSONValue =
  | null
  | boolean
  | number
  | string
  | JSONValue[]
  | { [key: string]: JSONValue };
type Obj = Record<string, JSONValue> | Array<JSONValue>;

function objDiff(obj1: Obj, obj2: Obj): Obj {

  if (obj1 === obj2) {
    return {};
  }
  if (obj1 === null || obj2 === null) {
    return [obj1, obj2];
  }
  if (typeof obj1 !== 'object' || typeof obj2 !== 'object') {
    return [obj1, obj2];
  }
  if (Array.isArray(obj1) !== Array.isArray(obj2)) {

    return [obj1, obj2];
  }
  const ans = {};
  for (const key in obj1) {
    if (key in obj2) {
      const subDiff = objDiff(obj1[key], obj2[key]);
      if (Object.keys(subDiff).length > 0) {
        ans[key] = subDiff;
      }
    }
  }

  return ans;
}

```

• LeetDoocs

```
function objDiff(obj1: any, obj2: any): any {
  if (type(obj1) !== type(obj2)) return [obj1, obj2];
  if (!isObject(obj1)) return obj1 === obj2 ? {} : [obj1, obj2];
  const diff: Record<string, unknown> = {};
  const sameKeys = Object.keys(obj1).filter(key => key in obj2);
  sameKeys.forEach(key => {
    const subDiff = objDiff(obj1[key], obj2[key]);
    if (Object.keys(subDiff).length) diff[key] = subDiff;
  });
  return diff;
}

function type(obj: unknown): string {
  return Object.prototype.toString.call(obj).slice(8, -1);
}

function isObject(obj: unknown): obj is Record<string, unknown> {
  return typeof obj === 'object' && obj !== null;
}
```

◆ Quick Review

Key Insights

- Use recursion to traverse nested objects and arrays.
- Compare only keys that exist in both objects.
- When encountering two values of different types or unequal primitives, record the difference as [value from obj1, value from obj2].
- When both values are objects or arrays, recursively compute their differences and

include them only if the resulting nested diff is non-empty.

- Arrays are treated like objects with indices as keys.
- The solution requires careful type checking and handling of recursive structures.

Space & Time

Time Complexity: $O(n)$ in average, where n is the number of keys/elements in the common parts of the structures.

Space Complexity: $O(n)$ due to recursion and storage needed for the output differences.

Solution

We solve the problem using a recursive function that compares keys present in both input objects. For each common key: If both values are objects (or both are arrays), recursively compute their differences. If the recursive call returns a non-empty object, include that key in the result. If the two values are of different types or are primitives that differ, add the key with a difference array [value from obj1, value from obj2]. Ignore keys that exist only in one object. This recursive

approach naturally handles the deeply nested structure while ensuring only common keys with different values are captured.

Round 9 · Low · Hard

Round 9

Low Priority · Hard

7 questions · 2 patterns

Dynamic Programming #10 #115 #123 #132
#312 #1000

Math #68

Dynamic Programming

Round 9 · Low · Hard · 6 questions

Dynamic Programming

□ #10 Regular Expression Matching

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming · Recursion
Lists: Denny Zhang

Problem

Given an input string *s* and a pattern *p*, implement regular expression matching with support for '.' and '*' where:

- '.' Matches any single character.
- '*' Matches zero or more of the preceding element.

Return a boolean indicating whether the matching covers the entire input string (not partial).

Example 1:

Input: *s* = "aa", *p* = "a"

Output: false

Explanation: "a" does not match the entire string "aa".

Example 2:

Input: s = "aa", p = "a*"

Output: true

Explanation: '*' means zero or more of the preceding element, 'a'. Therefore, by repeating 'a' once, it becomes "aa".

Example 3:

Input: s = "ab", p = ".*"

Output: true

Explanation: ".*" means "zero or more (*) of any character (.)".

Constraints:

- $1 \leq s.length \leq 20$
- $1 \leq p.length \leq 20$
- s contains only lowercase English letters.
- p contains only lowercase English letters, '.', and '*'.
- It is guaranteed for each appearance of the character '*', there will be a previous valid character to match.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Implement regex matching with '.' (any single char) and '*' (zero or more of preceding).

Full string must match. Right?"

#

"A few quick questions: '.*' matches any sequence? Yes. 'a*' can match zero 'a's? Yes."

#

"Let me walk: s='aa', p='a*' -> true ✓. s='ab', p='.*' -> true ✓. s='aab', p='c*a*b' -> true ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Recursion: match char-by-char, handle '*' as zero or more of preceding.

$O(2^{(m+n)})$ without memo. Should I go ahead and optimize?"

```
class _10_RegexpMatching_Brute:
```

```
    def isMatch(self, s, p):
```

```
        if not p: return not s
```

```
        first = bool(s) and p[0] in {s[0], '.'}
```

```
        if len(p) >= 2 and p[1] == '*':
```

```
            return self.isMatch(s, p[2:]) or (first and self.isMatch(s[1:], p))
```

```
        return first and self.isMatch(s[1:], p[1:])
```

PHASE 3 — OPTIMIZE

"The bottleneck is recomputing overlapping subproblems.

Bottom-up 2D DP: $dp[i][j]$ = does $s[i:]$ match $p[j:]$? Fill right-to-left.

Space: $O(n)$ with rolling rows.

Time: $O(m \cdot n)$. Space: $O(n)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _10_RegexpMatching:
```

```
    def isMatch(self, s, p):
```

```
        m, n = len(s), len(p)
```

```
        dp = [False] * (n + 1)
```

```
        dp[n] = True
```

```
        for i in range(m, -1, -1):
```

```
            new_dp = [False] * (n + 1)
```

```
            new_dp[n] = (i == m)
```

```
            for j in range(n - 1, -1, -1):
```

```
                first_match = i < m and p[j] in {s[i], '.'}
```

```
                if j + 1 < n and p[j+1] == '*':
```

```
                    new_dp[j] = new_dp[j+2] or (first_match and dp[j])
```

```
                else:
```

```
                    new_dp[j] = first_match and dp[j+1]
```

```
            dp = new_dp
```

```
        return dp[0]
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

$s='aa'$, $p='a*'$: dp converges so that $dp[0]=True$ ($a*$ matches $'aa'$) ✓

$s='ab'$, $p='.*'$: $.*$ matches any sequence. $dp[0]=True$ ✓

$s='aab'$, $p='c*a*b'$: $c*(zero\ c's) + a*(two\ a's) + b$. $dp[0]=True$ ✓

#

"No bugs. '*' has two branches: zero occurrences ($new_dp[j+2]$) or one-plus ($dp[j]$)."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(m \cdot n)$. Space $O(n)$: two rolling rows.

Follow-up: Wildcard Matching (#44) – '*' matches any sequence; simpler DP.

Follow-up: if '?' (any single char) is added – treat like '.' in this solution."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        m = len(s)
        n = len(p)
        # dp[i][j] := True if s[0..i) matches p[0..j)
        dp = [[False] * (n + 1) for _ in range(m + 1)]
        dp[0][0] = True

        def isMatch(i: int, j: int) -> bool:
            return j >= 0 and p[j] == '.' or s[i] == p[j]

        for j, c in enumerate(p):
            if c == '*' and dp[0][j - 1]:
                dp[0][j + 1] = True

        for i in range(m):
            for j in range(n):
                if p[j] == '*':
                    # The minimum index of '*' is 1.
                    noRepeat = dp[i + 1][j - 1]

                    doRepeat = isMatch(i, j - 1) and dp[i][j + 1]
                    dp[i + 1][j + 1] = noRepeat or doRepeat
                elif isMatch(i, j):
                    dp[i + 1][j + 1] = dp[i][j]

        return dp[m][n]
```

• LeetDoocs

```

class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        @cache
        def dfs(i, j):
            if j >= n:
                return i == m
            if j + 1 < n and p[j + 1] == '*':
                return dfs(i, j + 2) or (
                    i < m and (s[i] == p[j] or p[j] == '.') and dfs(i + 1, j)
                )

            return i < m and (s[i] == p[j] or p[j] == '.') and dfs(i + 1, j + 1)

        m, n = len(s), len(p)
        return dfs(0, 0)

```

```

class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        m, n = len(s), len(p)
        f = [[False] * (n + 1) for _ in range(m + 1)]
        f[0][0] = True
        for i in range(m + 1):
            for j in range(1, n + 1):
                if p[j - 1] == "*":
                    f[i][j] = f[i][j - 2]
                    if i > 0 and (p[j - 2] == "." or s[i - 1] == p[j - 2]):
                        f[i][j] |= f[i - 1][j]
                elif i > 0 and (p[j - 1] == "." or s[i - 1] == p[j - 1]):
                    f[i][j] = f[i - 1][j - 1]
        return f[m][n]

```

• SimplyLeet

```

# Python solution using recursion with memoization
def isMatch(s: str, p: str) -> bool:
    memo = {}

    def dp(i: int, j: int) -> bool:
        # If result is already computed, return it.
        if (i, j) in memo:
            return memo[(i, j)]

        # If we have reached the end of the pattern, s must also be at the end.

        if j == len(p):
            ans = i == len(s)
        else:
            # Check if current characters match
            first_match = i < len(s) and (p[j] == s[i] or p[j] == '.')

            # If there is a '*' coming after the current pattern character
            if j + 1 < len(p) and p[j+1] == '*':
                # Two cases:
                # 1. '*' stands for zero occurrence -> skip current pattern
                char and '*'

                # 2. first_match is true, so we try to consume one character in
                # s and remain at same pattern index
                ans = dp(i, j+2) or (first_match and dp(i+1, j))
            else:
                # No '*' means we directly check next characters in both s and
                # p.
                ans = first_match and dp(i+1, j+1)

        memo[(i, j)] = ans
        return ans

    return dp(0, 0)

# Example test run
#print(isMatch("aa", "a*")) # Expected output: True

```

 • LC.ca

```

class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        m, n = len(s), len(p)
        f = [[False] * (n + 1) for _ in range(m + 1)]
        f[0][0] = True
        for i in range(m + 1):
            for j in range(1, n + 1):
                if p[j - 1] == "*":
                    f[i][j] = f[i][j - 2]
                    if i > 0 and (p[j - 2] == "." or s[i - 1] == p[j - 2]):
                        f[i][j] |= f[i - 1][j]
                elif i > 0 and (p[j - 1] == "." or s[i - 1] == p[j - 1]):
                    f[i][j] = f[i - 1][j - 1]
        return f[m][n]

```

◆ Quick Review

Key Insights

- Use recursion or dynamic programming to simulate matching as the '*' operator can affect the current and following characters.
- The '.' operator is a wildcard that matches any single character.
- When encountering a '*' in the pattern, consider both skipping the preceding element or consuming one character from the string if it matches.
- Using memoization helps to save intermediate results and efficiently process overlapping subproblems.

Space & Time

Time Complexity: $O(mn)$

where $m = \text{length}(s)$ and $n = \text{length}(p)$, due to exploring each subproblem combination.

Space Complexity: $O(mn)$

for storing the memoization table in the dynamic programming or recursive approach.

Solution

The solution uses a top-down recursive strategy with memoization to efficiently match the string against the pattern. For each character position in s and p , we check if they match directly or if the pattern character is '.' to allow any match. When encountering '.' in the pattern, two possibilities are explored: either the '.' stands for zero occurrences of the preceding element, or if there is a valid match, it accounts for one occurrence and we proceed further in the string while maintaining the same pattern index. The memoization aspect prevents repeated computations of the same (i, j) state in the recursion, resulting in an improved overall runtime.

Dynamic Programming

□ #115 Distinct Subsequences

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Denny Zhang

Problem

Given two strings *s* and *t*, return the number of distinct subsequences of *s* which equals *t*.

The test cases are generated so that the answer fits on a 32-bit signed integer.

Example 1:

```
Input: s = "rabbbit", t = "rabbit"
Output: 3
Explanation:
As shown below, there are 3 ways you can generate "rabbit" from s.
rabbbit
rabbbit
rabbbit
```

Example 2:

```
Input: s = "babgbag", t = "bag"
Output: 5
Explanation:
As shown below, there are 5 ways you can generate "bag" from s.
babgbag
babgbag
babgbag
babgbag
babgbag
```

Constraints:

- $1 \leq s.length, t.length \leq 1000$
- s and t consist of English letters.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Count distinct subsequences of s that equal t .

Same positions count as different even if same characters. Right?"

#

"A few quick questions: $t='rabbit'$, $s='rabbbit'$ -> 3 ways ✓ (three different 'b' choices)"

#

"Let me walk: $s='babgbag'$, $t='bag'$ -> 5 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Recursion: if $s[i]==t[j]$: $use(i+1,j+1)+skip(i+1,j)$; else $skip(i+1,j)$.

$O(2^m)$ without memo. Should I go ahead and optimize?"

class _115_DistinctSubsequences_Brute:

def numDistinct(self, s, t):

from functools import lru_cache

@lru_cache(None)

def dp(i, j):

if j == len(t): return 1

if i == len(s): return 0

result = dp(i+1, j)

if s[i] == t[j]: result += dp(i+1, j+1)

return result

return dp(0, 0)

PHASE 3 — OPTIMIZE

"The bottleneck is recomputing overlapping subproblems.

Bottom-up DP: $dp[j]$ = ways to form $t[j:]$ from $s[i:]$. Roll right-to-left.

Time: $O(m*n)$. Space: $O(n)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

class _115_DistinctSubsequences:

def numDistinct(self, s, t):

n = len(t)


```

dp = [0] * (n + 1)
dp[n] = 1
for ch in reversed(s):
    for j in range(n - 1, -1, -1):
        if ch == t[j]:
            dp[j] += dp[j+1]
return dp[0]

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

s='rabbbit', t='rabbit': three b's in s for two in t -> 3 ways ✓

Right-to-left update prevents using same s[i] twice in one match.

#

"No bugs. dp[n]=1 base: empty t is trivially formed from any s suffix."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time O(m*n). Space O(n): one rolling array."

Right-to-left update is essential – prevents reuse of same s[i] across matches.

Follow-up: Edit Distance (#72) – related; counts edits instead of matching occurrences.

Follow-up: print all distinct subsequences – backtrack through the DP table."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def numDistinct(self, s: str, t: str) -> int:
        m = len(s)
        n = len(t)
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(m + 1):
            dp[i][0] = 1

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if s[i - 1] == t[j - 1]:
                    dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j]
                else:
                    dp[i][j] = dp[i - 1][j]

        return dp[m][n]

```

• LeetDoocs

```
class Solution:
    def numDistinct(self, s: str, t: str) -> int:
        m, n = len(s), len(t)
        f = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(m + 1):
            f[i][0] = 1
        for i, a in enumerate(s, 1):
            for j, b in enumerate(t, 1):
                f[i][j] = f[i - 1][j]
                if a == b:
                    f[i][j] += f[i - 1][j - 1]
        return f[m][n]
```

```
class Solution:
    def numDistinct(self, s: str, t: str) -> int:
        n = len(t)
        f = [1] + [0] * n
        for a in s:
            for j in range(n, 0, -1):
                if a == t[j - 1]:
                    f[j] += f[j - 1]
        return f[n]
```

• SimplyLeet

```
# Python solution for Distinct Subsequences
def numDistinct(s: str, t: str) -> int:
    # Get lengths of the strings s and t
    m, n = len(s), len(t)
    # Initialize a dp table with (m+1) rows and (n+1) columns filled with 0
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Base case: An empty string t (j = 0) is a subsequence of any prefix of s,
    # so set dp[i][0] = 1
    for i in range(m + 1):
        dp[i][0] = 1
```

```

# Fill the dp table
for i in range(1, m + 1):
    for j in range(1, n + 1):
        # If the current characters match, add the count of sequences by
        # either using or skipping s[i-1]
        if s[i - 1] == t[j - 1]:
            dp[i][j] = dp[i - 1][j - 1] + dp[i - 1][j]
        else:
            # If they do not match, skip s[i-1]
            dp[i][j] = dp[i - 1][j]

# The answer is the way to form all of t from all of s
return dp[m][n]

# Example usage:
print(numDistinct("rabbbit", "rabbit")) # Output: 3

```

• LC.ca

```

class Solution:
    def numDistinct(self, s: str, t: str) -> int:
        n = len(t)
        f = [1] + [0] * n
        for a in s:
            for j in range(n, 0, -1):
                if a == t[j - 1]:
                    f[j] += f[j - 1]
        return f[n]

```

◆ Quick Review

Key Insights

- Use dynamic programming to build the solution by comparing characters of s and t.
- Define a 2D dp table where $dp[i][j]$ represents the number of ways to form the

first j characters of t using the first i characters of s .

- The recurrence relation:
- If $s[i-1]$ equals $t[j-1]$, then include both the cases of matching this character as well as skipping it: $dp[i][j] = dp[i-1][j-1] + dp[i-1][j]$.
- If they do not match, then simply skip the current character of s : $dp[i][j] = dp[i-1][j]$.
- Initialize $dp[i][0] = 1$ for all i since an empty t can always be formed.
- The final answer is $dp[s.length][t.length]$.

Space & Time

Time Complexity: $O(n * m)$, where n is the length of s and m is the length of t .

Space Complexity: $O(n * m)$, which can be optimized to $O(m)$ with careful row reuse.

Solution

We use a dynamic programming approach with a 2D table. The table has dimensions $(len(s) + 1) \times (len(t) + 1)$. Each cell $dp[i][j]$ stores the number of distinct subsequences from the first i characters of s that match the first j characters of t . Steps: Initialize $dp[i][0] = 1$ for

every i , because an empty t is a subsequence of any prefix of s . Iterate over each character of s (outer loop) and for each, iterate over t (inner loop). For every character in s , if it equals the current character in t , update $dp[i][j]$ as the sum of the ways by using or skipping this character. Otherwise, carry over the previous count. Return $dp[\text{len}(s)][\text{len}(t)]$ as the result.

Dynamic Programming

□ #123 Best Time to Buy and Sell Stock III

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: Denny Zhang

Problem

You are given an array `prices` where `prices[i]` is the price of a given stock on the i th day.

Find the maximum profit you can achieve. You may complete at most two transactions.

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Example 1:

Input: `prices = [3,3,5,0,0,3,1,4]`

Output: 6

Explanation: Buy on day 4 (price = 0) and sell on day 6 (price = 3), profit = $3 - 0 = 3$.

Then buy on day 7 (price = 1) and sell on day 8 (price = 4), profit = $4 - 1 = 3$.

Example 2:

Input: prices = [1,2,3,4,5]

Output: 4

Explanation: Buy on day 1 (price = 1) and sell on day 5 (price = 5), profit = 5-1 = 4.

Note that you cannot buy on day 1, buy on day 2 and sell them later, as you are engaging multiple transactions at the same time. You must sell before buying again.

Example 3:

Input: prices = [7,6,4,3,1]

Output: 0

Explanation: In this case, no transaction is done, i.e. max profit = 0.

Constraints:

- $1 \leq \text{prices.length} \leq 105$
- $0 \leq \text{prices}[i] \leq 105$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

At most 2 transactions. Maximize profit. Can't hold two stocks simultaneously. Right?"

#

"A few quick questions: Complete one transaction before starting next? Yes."

#

"Let me walk: prices=[3,3,5,0,0,3,1,4] -> buy@0,sell@3 + buy@1,sell@4 = 6 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Try all pairs of buy1<sell1<=buy2<sell2. $O(n^4)$. Should I go ahead and optimize?"

```
class _123_StockIII_Brute:
```

```
    def maxProfit(self, prices):
```

```
        n = len(prices); best = 0
```

```
        for s1 in range(n):
```

```
            for e1 in range(s1, n):
```

```
                p1 = prices[e1]-prices[s1]
```

```
                for s2 in range(e1, n):
```

```
                    for e2 in range(s2, n):
```

```
                        best = max(best, p1+prices[e2]-prices[s2])
```



```
    return best
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the quartic enumeration.
# State machine: 4 states track max profit.
# buy1, sell1, buy2, sell2 updated in one pass.
# Time: O(n). Space: O(1)."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
```

```
class _123_StockIII:
    def maxProfit(self, prices):
        buy1 = buy2 = float('-inf')
        sell1 = sell2 = 0
        for price in prices:
            buy1 = max(buy1, -price)
            sell1 = max(sell1, buy1 + price)
            buy2 = max(buy2, sell1 - price)
            sell2 = max(sell2, buy2 + price)
        return sell2
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
```

```
#
# prices=[3,3,5,0,0,3,1,4]:
# price=0: buy1=max(-inf,0)=0. sell1=max(0,0+0)=0. buy2=max(-inf,0-0)=0.
# sell2=max(0,0+0)=0.
# price=3: buy1=0. sell1=3. buy2=3-3=0? No: buy2=max(0,3-3)=0. sell2=max(0,0+3)=3.
# ... after price=4: sell2=6 ✓
#
# "No bugs. State machine automatically handles the constraint of selling before
# buying again."
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n): single pass. Space O(1): four state variables.
# Follow-up: at most k transactions (#188) – generalize to 2k state variables or
# O(n*k) DP.
# Follow-up: unlimited transactions (#122) – sum all positive consecutive
# differences."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxProfit(self, prices: list[int]) -> int:
        sellTwo = 0
        holdTwo = -math.inf
        sellOne = 0
        holdOne = -math.inf

        for price in prices:
            sellTwo = max(sellTwo, holdTwo + price)
            holdTwo = max(holdTwo, sellOne - price)

            sellOne = max(sellOne, holdOne + price)
            holdOne = max(holdOne, -price)

        return sellTwo
```

• LeetDoocs

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        # 初始化
        f1, f2, f3, f4 = -prices[0], 0, -prices[0], 0
        for price in prices[1:]:
            f1 = max(f1, -price)
            f2 = max(f2, f1 + price)
            f3 = max(f3, f2 - price)
            f4 = max(f4, f3 + price)
        return f4
```

• SimplyLeet

```
# Python solution for Best Time to Buy and Sell Stock III

def max_profit(prices):
    # If there are less than 2 days, no transaction can be made.
    if not prices or len(prices) < 2:
        return 0

    n = len(prices)
    # Initialize left and right profit arrays with zeros.
    left_profits = [0] * n
```

```

right_profits = [0] * n

# Forward pass: compute max profit until day i with one transaction.
min_price = prices[0]
for i in range(1, n):
    # Update the min price encountered so far.
    min_price = min(min_price, prices[i])
    # Maximum profit if sold on day i.
    left_profits[i] = max(left_profits[i - 1], prices[i] - min_price)

# Backward pass: compute max profit from day i to end with one transaction.
max_price = prices[-1]
for i in range(n - 2, -1, -1):
    # Update max price seen from the right.
    max_price = max(max_price, prices[i])
    # Maximum profit possible starting from day i.
    right_profits[i] = max(right_profits[i + 1], max_price - prices[i])

# Combine the two profits to find the maximum total profit.
max_total_profit = 0

for i in range(n):
    max_total_profit = max(max_total_profit, left_profits[i] + right_profits[i])

return max_total_profit

# Example usage:
prices = [3,3,5,0,0,3,1,4]
print(max_profit(prices)) # Expected output: 6

```

• LC.ca

```

    """
    - `f1`  []
    - `f2`  []
    - `f3`  []
    - `f4`  []
    """

    class Solution:
        def maxProfit(self, prices: List[int]) -> int:
            # []
            f1, f2, f3, f4 = -prices[0], 0, -prices[0], 0

            for price in prices[1:]:
                f1 = max(f1, -price)
                f2 = max(f2, f1 + price)
                f3 = max(f3, f2 - price)
                f4 = max(f4, f3 + price)
            return f4

    class Solution:
        def maxProfit(self, prices: List[int]) -> int:

            if not prices:
                return 0

            # 0 to i, max profit
            left_max = [0] * len(prices)

            tmp_min = prices[0]
            for i in range(len(prices)):
                tmp_min = min(tmp_min, prices[i])
                left_max[i] = max(left_max[i-1], prices[i]-tmp_min) if i > 0 else
0

```

```

        # i to end, max profit
        right_max = [0] * len(prices)

        tmp_max = prices[-1]
        for i in range(len(prices)-1, -1, -1):
            tmp_max = max(tmp_max, prices[i])
            right_max[i] = max(right_max[i+1], tmp_max-prices[i]) if i < len(p
rices)-1 else 0

        return max(left_max[i] + right_max[i] for i in range(len(prices)))

```

```
#####
```

```

class Solution(object):
    def maxProfit(self, prices):
        """
        :type prices: List[int]
        :rtype: int
        """
        buy1 = buy2 = float("-inf")

        sell1 = sell2 = 0

        for i in range(len(prices)):
            sell1 = max(prices[i] + buy1, sell1)
            buy1 = max(buy1, -prices[i])
            sell2 = max(sell2, prices[i] + buy2)
            buy2 = max(sell1 - prices[i], buy2)
        return max(sell1, sell2)

```

◆ Quick Review

Key Insights

- The problem requires the optimal selection of at most two buy–sell transactions.

- One effective strategy is to split the problem into two parts:
- For each day i , calculate the maximum profit obtainable from day 0 to i with one transaction.
- For each day i , calculate the maximum profit obtainable from day i to the end with one transaction.
- Finally, for each day, the sum of these two profits (left and right) gives a candidate answer.
- Alternatively, a state-based dynamic programming approach can track 4 states corresponding to the two transactions.

Space & Time

Time Complexity: $O(n)$ where n is the number of days

Space Complexity: $O(n)$ due to extra arrays used for forward and backward computations (this can be optimized to $O(1)$ with a state variable approach)

Solution

We use a two-pass dynamic programming approach. In the first pass (forward), we

compute an array "leftProfits" where $\text{leftProfits}[i]$ is the maximum profit from day 0 to day i with one transaction. We do this by keeping track of the minimum price observed so far and calculating the profit if sold on day i . In the second pass (backward), we compute an array "rightProfits" where $\text{rightProfits}[i]$ is the maximum profit obtainable from day i to the last day with one transaction by tracking the maximum price seen in reverse order. Finally, for every index i , the sum $\text{leftProfits}[i] + \text{rightProfits}[i]$ represents the maximum profit achievable by completing the first transaction on or before day i and the second transaction starting from day i . The answer is the maximum sum over all i .

Dynamic Programming

□ #132 Palindrome Partitioning II

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
String · Dynamic Programming
Lists: Denny Zhang

Problem

Given a string s , partition s such that every substring of the partition is a palindrome.
Return the minimum cuts needed for a palindrome partitioning of s .

Example 1:

```
Input: s = "aab"  
Output: 1  
Explanation: The palindrome partitioning ["aa","b"] could be produced using 1 cut.
```

Example 2:

```
Input: s = "a"  
Output: 0
```

Example 3:

```
Input: s = "ab"  
Output: 1
```

Constraints:

- $1 \leq s.length \leq 2000$
- s consists of lowercase English letters only.

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Find the minimum cuts to partition s into palindromes.
# Each partition must be a palindrome. Right?"
#
# "A few quick questions: The whole string being a palindrome = 0 cuts? Yes."
#
# "Let me walk: s='aab' -> 1 cut ('aa'|'b') ✓. s='a' -> 0 ✓. s='ab' -> 1 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Try all partitionings.  $O(2^n)$  time. Should I go ahead and optimize?"
class _132_PalindromePartitioningII_Brute:
    def minCut(self, s):
        from functools import lru_cache
        n = len(s)
        @lru_cache(None)
        def is_pal(i, j): return s[i:j+1] == s[i:j+1][::-1]
        @lru_cache(None)
        def dp(i):
            if i == n: return -1
            return min(1 + dp(j+1) for j in range(i, n) if is_pal(i, j))
        return dp(0)
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the  $O(n^3)$  palindrome checks combined with DP.
# Precompute palindrome table in  $O(n^2)$ . Then  $dp[i] = \min \text{ cuts for } s[:i+1]$ .
#  $dp[i] = \min(dp[j-1]+1)$  for all  $j$  where  $s[j..i]$  is palindrome.
# Time:  $O(n^2)$ . Space:  $O(n^2)$  palindrome table +  $O(n)$  dp."
```

PHASE 4 — CLEAN CODE

```
# "Now I'll implement the optimized approach with meaningful names."
class _132_PalindromePartitioningII:
    def minCut(self, s):
        n = len(s)
        is_pal = [[False]*n for _ in range(n)]
        for i in range(n):
            is_pal[i][i] = True
```

```

for length in range(2, n+1):
    for i in range(n-length+1):
        j = i+length-1
        is_pal[i][j] = (s[i]==s[j]) and (length==2 or is_pal[i+1][j-1])
dp = list(range(-1, n))
for i in range(n):
    for j in range(i+1):
        if is_pal[j][i]:
            dp[i+1] = min(dp[i+1], dp[j]+1)
return dp[n]

```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

s='aab': is_pal[0][1]=True('aa'). dp[2]=min(dp[2],dp[0]+1)=min(1,1)=1.

dp[3]=1('b') -> cut=1 ✓

#

"No bugs. dp[0]=-1 base: -1 cuts needed for empty prefix."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^2)$. Space $O(n^2)$ table + $O(n)$ dp.

Follow-up: Palindrome Partitioning I (#131) – return all partitions; backtrack with table.

Follow-up: Palindrome Partitioning IV (#1745) – fixed 3 parts; use same palindrome table."

Community Solutions (Python)

• WalkCC

```

class Solution:
    def minCut(self, s: str) -> int:
        n = len(s)
        # isPalindrome[i][j] := True if s[i..j] is a palindrome
        isPalindrome = [[True] * n for _ in range(n)]
        # dp[i] := the minimum cuts needed for a palindrome partitioning of s[0..i]
        dp = [n] * n

        for l in range(2, n + 1):
            i = 0

```

```

    for j in range(l - 1, n):
        isPalindrome[i][j] = s[i] == s[j] and isPalindrome[i + 1][j - 1]
        i += 1

    for i in range(n):
        if isPalindrome[0][i]:
            dp[i] = 0
            continue

        # Try all the possible partitions.

        for j in range(i):
            if isPalindrome[j + 1][i]:
                dp[i] = min(dp[i], dp[j] + 1)

    return dp[-1]

```

• LeetDoocs

```

class Solution:
    def minCut(self, s: str) -> int:
        n = len(s)
        g = [[True] * n for _ in range(n)]
        for i in range(n - 1, -1, -1):
            for j in range(i + 1, n):
                g[i][j] = s[i] == s[j] and g[i + 1][j - 1]
        f = list(range(n))
        for i in range(1, n):
            for j in range(i + 1):
                if g[j][i]:
                    f[i] = min(f[i], 1 + f[j - 1] if j else 0)
        return f[-1]

```

• SimplyLeet

```
def minCut(s: str) -> int:
    n = len(s)
    # Precompute palindrome table where isPal[i][j] is True if s[i:j+1] is a
    # palindrome.
    isPal = [[False] * n for _ in range(n)]
    # dp[i] will store the minimum cuts required for substring s[0...i]
    dp = [0] * n
    for i in range(n):
        min_cut = i # maximum cuts (cut every character)
        for j in range(i + 1):
            # Check if s[j...i] is a palindrome:

            if s[j] == s[i] and (i - j < 2 or isPal[j+1][i-1]):
                isPal[j][i] = True # Mark substring as palindrome.
                # If the palindrome starts at index 0, no cut is needed.
                min_cut = 0 if j == 0 else min(min_cut, dp[j-1] + 1)
        dp[i] = min_cut # Record minimum cuts needed for s[0...i]
    return dp[-1]

# Example usage:
# print(minCut("aab")) # Output: 1
```

• LC.ca

```
class Solution:
    def minCut(self, s: str) -> int:
        pal = [[False for j in range(0, len(s))] for i in range(0, len(s))]
        dp = [len(s) for _ in range(0, len(s) + 1)]
        for i in range(0, len(s)):
            for j in range(0, i + 1):
                # also ok if, (j + 1 >= i - 1)
                if (s[i] == s[j]) and ((j + 1 > i - 1) or (pal[i - 1][j + 1])):
                    pal[i][j] = True
                    dp[i + 1] = min(dp[i + 1], dp[j] + 1) if j != 0 else 0
```

```
        # 'if j != 0' to ensure from start to i is a palindrome
    return dp[-1]
```

```
#####
```

```
class Solution: # clear cache
    def minCut(self, s: str) -> int:
        @cache
        def dfs(i):
            if i >= n - 1:
```

```
                return 0
            ans = inf
            for j in range(i, n):
                if g[i][j]:
                    ans = min(ans, dfs(j + 1) + (j < n - 1))
            return ans
```

```
n = len(s)
g = [[True] * n for _ in range(n)]
for i in range(n - 1, -1, -1):
```

```
        for j in range(i + 1, n):
            g[i][j] = s[i] == s[j] and g[i + 1][j - 1]
    ans = dfs(0)
    dfs.cache_clear() # nice, clear cache !!!
    return ans
```

```
#####
```

```
class Solution:
    def minCut(self, s: str) -> int:
```

```
n = len(s)
is_pal = [[False] * n for _ in range(n)]
for i in range(n - 1, -1, -1):
    for j in range(i, n):
        is_pal[i][j] = s[i] == s[j] and (j - i < 3 or is_pal[i + 1][j
- 1])
min_cut = [i for i in range(n)]
for i in range(n):
    for j in range(0, i + 1):
        if is_pal[j][i]:
            min_cut[i] = min(min_cut[i], min_cut[j - 1] + 1) if j > 0
        else 0

return min_cut[-1]
```

◆ Quick Review

Key Insights

- Precompute a 2D table (isPal) where isPal[i][j] indicates if the substring s[i...j] is a palindrome.
- Use dynamic programming: dp[i] represents the minimum cuts needed for the substring s[0...i].
- For every index i, iterate over all possible starting indices j. If s[j...i] is a palindrome, update dp[i] using dp[j-1] + 1 (or 0 if j is 0).
- Time complexity is dominated by the $O(n^2)$ palindrome checking and state transitions.

Space & Time

Time Complexity: $O(n^2)$

Space Complexity: $O(n^2)$ (due to the 2D table for palindrome checks, plus $O(n)$ for the dp array)

Solution

The solution relies on dynamic programming with a two-step approach. First, precompute a 2D table (isPal) where each cell indicates if a substring is a palindrome. This is done in $O(n^2)$ time by leveraging the fact that a substring $s[j...i]$ is a palindrome if the characters at the ends match and the inner substring $s[j+1...i-1]$ is also a palindrome (or if the length is less than 3). Then, use a dp array where $dp[i]$ holds the minimum number of cuts needed for the substring $s[0...i]$. For each i , check every index j ($0 \leq j \leq i$) and if $s[j...i]$ is a palindrome, update $dp[i]$ to be the minimum of its current value and $dp[j-1] + 1$ (with a special case when j is 0, meaning no cut is needed). This two-phase approach efficiently breaks down the problem and computes the answer while avoiding redundant palindrome checks.

Dynamic Programming

□ #312 Burst Balloons

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: Denny Zhang

Problem

You are given n balloons, indexed from 0 to $n - 1$. Each balloon is painted with a number on it represented by an array `nums`. You are asked to burst all the balloons.

If you burst the i th balloon, you will get $\text{nums}[i - 1] * \text{nums}[i] * \text{nums}[i + 1]$ coins. If $i - 1$ or $i + 1$ goes out of bounds of the array, then treat it as if there is a balloon with a 1 painted on it.

Return the maximum coins you can collect by bursting the balloons wisely.

Example 1:

Input: `nums = [3,1,5,8]`

Output: 167

Explanation:

`nums = [3,1,5,8] --> [3,5,8] --> [3,8] --> [8] --> []`

`coins = 3*1*5 + 3*5*8 + 1*3*8 + 1*8*1 = 167`

Example 2:

Input: nums = [1,5]
Output: 10

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 300$
- $0 \leq \text{nums}[i] \leq 100$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Let me make sure I understand the problem correctly.
# Burst balloon i: earn nums[i-1]*nums[i]*nums[i+1]. Pad array with 1s on both ends.
# Burst all balloons to maximize total coins. Right?"
#
# "A few quick questions: After bursting i, its neighbors become adjacent? Yes."
#
# "Let me walk: nums=[3,1,5,8] -> 167 ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# Try all permutations of burst order.  $O(n!)$ . Should I go ahead and optimize?"
class _312_BurstBalloons_Brute:
    def maxCoins(self, nums):
        nums = [1] + nums + [1]; n = len(nums)
        from functools import lru_cache
        @lru_cache(None)
        def dp(l, r):
            return max((nums[l]*nums[k]*nums[r] + dp(l,k) + dp(k,r) for k in
range(l+1,r)), default=0)
        return dp(0, n-1)
```

PHASE 3 — OPTIMIZE

```
# "The bottleneck is the  $O(n!)$  permutation search.
# Interval DP:  $dp[l][r]$  = max coins from open interval (l,r) where l,r are
boundaries NOT popped.
# Think backwards: k is the LAST balloon popped in (l,r).
#  $dp[l][r] = \max \text{ over } k: dp[l][k] + \text{nums}[l]*\text{nums}[k]*\text{nums}[r] + dp[k][r]$ .
```

Time: $O(n^3)$. Space: $O(n^2)$."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _312_BurstBalloons:
    def maxCoins(self, nums):
        nums = [1] + nums + [1]
        n = len(nums)
        dp = [[0]*n for _ in range(n)]
        for length in range(2, n):
            for left in range(0, n-length):
                right = left+length
                for k in range(left+1, right):
                    dp[left][right] = max(dp[left][right],
                                           nums[left]*nums[k]*nums[right] + dp[left][k] +
                                           dp[k][right])
        return dp[0][n-1]
```

PHASE 5 — TEST & DEBUG

"Let me trace through a few cases."

#

nums=[1,3,1,5,8,1] (padded): dp fills by increasing length.

dp[0][5]=167 ✓ (standard result matching all burst orders)

#

"No bugs. 'Last burst' reversal gives clean optimal substructure."

PHASE 6 — COMPLEXITY & FOLLOW-UP

"Time $O(n^3)$. Space $O(n^2)$."

The last-burst reversal is the key insight that gives clean substructure.

Follow-up: Minimum Cost to Merge Stones (#1000) – same interval DP skeleton.

Follow-up: Stone Game (#877) – similar range DP for game theory."

Community Solutions (Python)

• WalkCC

```
class Solution:
    def maxCoins(self, nums: list[int]) -> int:
        n = len(nums)
        nums = [1] + nums + [1]

        @functools.lru_cache(None)
        def dp(i: int, j: int) -> int:
            """Returns maxCoins(nums[i..j])."""
            if i > j:
                return 0
```

```

        return max(dp(i, k - 1) +
                    dp(k + 1, j) +
                    nums[i - 1] * nums[k] * nums[j + 1]
                    for k in range(i, j + 1))

    return dp(1, n)

```

```

class Solution:
    def maxCoins(self, nums: list[int]) -> int:
        n = len(nums)
        # dp[i][j] := maxCoins(nums[i..j])
        dp = [[0] * (n + 2) for _ in range(n + 2)]

        nums = [1] + nums + [1]

        for d in range(n):
            for i in range(1, n - d + 1):
                j = i + d
                for k in range(i, j + 1):
                    dp[i][j] = max(
                        dp[i][j],
                        dp[i][k - 1] +
                        dp[k + 1][j] +
                        nums[i - 1] * nums[k] * nums[j + 1])

        return dp[1][n]

```

• LeetDoocs

```

class Solution:
    def maxCoins(self, nums: List[int]) -> int:
        n = len(nums)
        arr = [1] + nums + [1]
        f = [[0] * (n + 2) for _ in range(n + 2)]
        for i in range(n - 1, -1, -1):
            for j in range(i + 2, n + 2):
                for k in range(i + 1, j):
                    f[i][j] = max(f[i][j], f[i][k] + f[k][j] + arr[i] * arr[k]
                                   * arr[j])
        return f[0][-1]

```

• SimplyLeet

```
# Define the function to compute maximum coins
def maxCoins(nums):
    # Add virtual balloons with value 1 at both boundaries
    n = len(nums)
    extended = [1] + nums + [1]
    # Initialize dp table with 0's; dimensions: (n+2) x (n+2)
    dp = [[0] * (n + 2) for _ in range(n + 2)]

    # Calculate dp values for intervals with length 2 to n+1
    for length in range(2, n + 2): # interval length from 2 to n+1

        for left in range(0, n + 2 - length):
            right = left + length
            # Iterate through all possible last burst balloons in the interval
            (left, right)
            for k in range(left + 1, right):
                # Coins when bursting balloon k last in the interval:
                # value from bursting: extended[left] * extended[k] *
                extended[right]
                # plus coins from subintervals [left, k] and [k, right]
                coins = extended[left] * extended[k] * extended[right] + dp[le
                ft][k] + dp[k][right]
                dp[left][right] = max(dp[left][right], coins)

    # The final answer is stored in dp[0][n+1] which represents bursting all
    real balloons.
    return dp[0][n+1]

# Example test cases
print(maxCoins([3,1,5,8])) # Expected output: 167
print(maxCoins([1,5]))    # Expected output: 10
```

• LC.ca

```

class Solution:
    def maxCoins(self, nums: List[int]) -> int:
        nums = [1] + nums + [1]
        n = len(nums)
        dp = [[0] * n for _ in range(n)]
        for l in range(2, n):
            for i in range(n - l):
                j = i + l
                for k in range(i + 1, j):
                    dp[i][j] = max(
                        dp[i][j], dp[i][k] + dp[k][j] + nums[i] * nums[k] * nu
ms[j]
                    )
        return dp[0][-1]

```

◆ Quick Review

Key Insights

- Transform the problem by adding virtual balloons with value 1 at both ends to simplify edge handling.
- Use interval dynamic programming where $dp[i][j]$ represents the maximum coins that can be obtained by bursting all balloons between indices i and j (exclusive).
- Realize that the order of bursting balloons matters; choose the last balloon to burst in each interval to break dependencies into independent subproblems.

- The solution involves iterating over all possible intervals and determining the optimal burst order.

Space & Time

Time Complexity: $O(n^3)$ – Three nested loops: one for the interval length, one for the starting index, and one for the final balloon choice.

Space Complexity: $O(n^2)$ – A 2D dynamic programming table is used to store the maximum coins for each interval.

Solution

The problem is solved using interval dynamic programming. First, extend the input array by adding 1 at both ends to handle edge cases seamlessly. Define a dp table where $dp[i][j]$ represents the maximum coins that can be obtained by bursting all the balloons between i and j (not including i and j , as these are the virtual boundaries). For every interval $[i, j]$, iterate through the possible choices for the last balloon k to burst in that interval. The value for bursting balloon k will be determined by the coins from bursting it last (i.e., $nums[i] * nums[k] * nums[j]$) plus the maximum coins

from the two resulting subintervals: (i, k) and (k, j) . By filling the dp table in increasing interval lengths, you build up to the solution for the overall interval that covers the entire (extended) array.

Dynamic Programming

□ #1000 Minimum Cost to Merge Stones

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · Dynamic Programming
Lists: Denny Zhang

Problem

There are n piles of stones arranged in a row.
The i th pile has $\text{stones}[i]$ stones.

A move consists of merging exactly k consecutive piles into one pile, and the cost of this move is equal to the total number of stones in these k piles.

Return the minimum cost to merge all piles of stones into one pile. If it is impossible, return -1 .

Example 1:

Input: $\text{stones} = [3, 2, 4, 1]$, $k = 2$

Output: 20

Explanation: We start with $[3, 2, 4, 1]$.

We merge $[3, 2]$ for a cost of 5, and we are left with $[5, 4, 1]$.

We merge $[4, 1]$ for a cost of 5, and we are left with $[5, 5]$.

We merge $[5, 5]$ for a cost of 10, and we are left with $[10]$.

The total cost was 20, and this is the minimum possible.

Example 2:

Input: stones = [3,2,4,1], k = 3

Output: -1

Explanation: After any merge operation, there are 2 piles left, and we can't merge anymore. So the task is impossible.

Example 3:

Input: stones = [3,5,1,2,6], k = 3

Output: 25

Explanation: We start with [3, 5, 1, 2, 6].

We merge [5, 1, 2] for a cost of 8, and we are left with [3, 8, 6].

We merge [3, 8, 6] for a cost of 17, and we are left with [17].

The total cost was 25, and this is the minimum possible.

Constraints:

- $n == \text{stones.length}$
- $1 \leq n \leq 30$
- $1 \leq \text{stones}[i] \leq 100$
- $2 \leq k \leq 30$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Merge exactly k consecutive piles at a time; cost = sum of the k piles.

Return minimum total cost, or -1 if impossible. Right?"

#

"A few quick questions: When is it impossible? When $(n-1) \% (k-1) \neq 0$.

Each merge reduces pile count by k-1, so we need (n-1) divisible by (k-1)."

#

"Let me walk: stones=[3,2,4,1], k=2 -> 20 ✓. stones=[3,2,4,1], k=3 -> -1 ✓"

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Try all orders of k-consecutive merges. $O(n^k)$ -- exponential. Should I go ahead and optimize?"

```

class _1000_MinCostMergeStones_Brute:
    def mergeStones(self, stones, k):
        n = len(stones)
        if (n-1) % (k-1): return -1
        from functools import lru_cache
        prefix = [0] * (n+1)
        for i in range(n): prefix[i+1] = prefix[i] + stones[i]
        @lru_cache(None)
        def dp(l, r, piles):
            if r-l+1 == piles: return 0
            result = min(dp(l,m,1)+dp(m+1,r,piles-1) for m in range(l,r,k-1))
            if piles == 1: result += prefix[r+1]-prefix[l]
            return result
        return dp(0, n-1, 1)

```

PHASE 3 — OPTIMIZE

```

# "The bottleneck is the exponential search without memoization (above is already
memoized).
# Same interval DP but cleaner: dp[l][r] = min cost to merge stones[l..r] into
fewest piles.
# When (r-l)%(k-1)==0, the segment merges to exactly 1 pile.
# Time:  $O(n^3/k)$ . Space:  $O(n^2)$ ."

```

PHASE 4 — CLEAN CODE

```

# "Now I'll implement the optimized approach with meaningful names."

```

```

class _1000_MinCostMergeStones:
    def mergeStones(self, stones, k):
        n = len(stones)
        if (n-1) % (k-1): return -1
        prefix = [0] * (n+1)
        for i in range(n): prefix[i+1] = prefix[i] + stones[i]
        dp = [[0]*n for _ in range(n)]
        for length in range(k, n+1):
            for l in range(n-length+1):
                r = l+length-1
                dp[l][r] = float('inf')
                for m in range(l, r, k-1):
                    dp[l][r] = min(dp[l][r], dp[l][m] + dp[m+1][r])
                if (length-1) % (k-1) == 0:
                    dp[l][r] += prefix[r+1]-prefix[l]
        return dp[0][n-1]

```

PHASE 5 — TEST & DEBUG

```

# "Let me trace through a few cases."
#
# stones=[3,2,4,1], k=2: (4-1)%1=0 OK. After DP: dp[0][3]=20 ✓
# stones=[3,2,4,1], k=3: (4-1)%2=1 != 0 -> return -1 ✓
#
# "No bugs. The split point iterates in steps of k-1 because each merge reduces
count by k-1."

```

PHASE 6 — COMPLEXITY & FOLLOW-UP**# "Time $O(n^3/k)$. Space $O(n^2)$.****# The feasibility check $(n-1)\%(k-1)==0$ saves all computation for impossible inputs.**
Follow-up: Burst Balloons (#312) – same interval DP skeleton, different cost formula.**# Follow-up: $k=2$ (classic stone merging) – solvable in $O(n \log n)$ with Hu-Shing algorithm."**

Community Solutions (Python)

• LeetDoocs

```

class Solution:
    def mergeStones(self, stones: List[int], K: int) -> int:
        n = len(stones)
        if (n - 1) % (K - 1):
            return -1
        s = list(accumulate(stones, initial=0))
        f = [[[inf] * (K + 1) for _ in range(n + 1)] for _ in range(n + 1)]
        for i in range(1, n + 1):
            f[i][i][1] = 0
        for l in range(2, n + 1):
            for i in range(1, n - l + 2):
                j = i + l - 1
                for k in range(1, K + 1):
                    for h in range(i, j):
                        f[i][j][k] = min(f[i][j][k], f[i][h][1] + f[h + 1][j][
k - 1])
                    f[i][j][1] = f[i][j][K] + s[j] - s[i - 1]
        return f[1][n][1]

```

• SimplyLeet

```

# Python solution for merging stones with detailed comments.
class Solution:
    def mergeStones(self, stones: List[int], k: int) -> int:
        n = len(stones)
        # Check for merge feasibility.
        if (n - 1) % (k - 1) != 0:
            return -1

        # Precompute prefix sums for fast range sum queries.
        prefix = [0] * (n + 1)

```

```

    for i in range(n):
        prefix[i + 1] = prefix[i] + stones[i]

    # Initialize dp table, dp[i][j] represents minimal cost to merge
    stones[i...j].
    dp = [[0] * n for _ in range(n)]

    # Process all possible subarrays with lengths >= k.
    for length in range(k, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1

            dp[i][j] = float('inf')
            # Try all valid partitions with step size (k-1).
            for mid in range(i, j, k - 1):
                dp[i][j] = min(dp[i][j], dp[i][mid] + dp[mid+1][j])
            # If this segment can be merged into one pile, add cost.
            if (length - 1) % (k - 1) == 0:
                dp[i][j] += prefix[j+1] - prefix[i]
    return dp[0][n-1]

# Example usage:

# solution = Solution()
# print(solution.mergeStones([3, 2, 4, 1], 2)) # Expected output: 20

```

• LC.ca

```

class Solution:
    def mergeStones(self, stones: List[int], K: int) -> int:
        n = len(stones)
        if (n - 1) % (K - 1):
            return -1
        s = list(accumulate(stones, initial=0))
        f = [[[inf] * (K + 1) for _ in range(n + 1)] for _ in range(n + 1)]
        for i in range(1, n + 1):
            f[i][i][1] = 0
        for l in range(2, n + 1):

```

```

    for i in range(1, n - l + 2):
        j = i + l - 1
        for k in range(1, K + 1):
            for h in range(i, j):
                f[i][j][k] = min(f[i][j][k], f[i][h][1] + f[h + 1][j][
k - 1])
            f[i][j][1] = f[i][j][K] + s[j] - s[i - 1]
    return f[1][n][1]

```

◆ Quick Review

Key Insights

- You can only merge stones into one final pile if $(n - 1)$ is divisible by $(k - 1)$. If not, return -1 .
- A dynamic programming approach is used where $dp[i][j]$ represents the minimum cost to merge the subarray $stones[i...j]$ into as few piles as possible.
- A prefix sum array is precomputed to quickly calculate the sum of stones in any subarray.
- When a segment can be merged into one pile (i.e. when $(length - 1) \% (k - 1) == 0$), add the sum of stones in that segment to the dp value.

Space & Time

Time Complexity: $O(n^3)$ due to the triple nested loop over segments and splits.

Space Complexity: $O(n^2)$ from the dp table and $O(n)$ for the prefix sum array.

Solution

The problem is solved using a range DP approach. We first verify that merging all piles into one is feasible by checking if $(n - 1) \% (k - 1) == 0$. Next, a prefix sum array is computed to optimize range sum calculations. A 2D dp table $dp[i][j]$ is then used to store the minimum cost to merge stones $[i...j]$. For each interval, we consider valid split points, updating $dp[i][j]$ by combining sub-problems. If the current segment can form a single merged pile (determined by the condition on the interval length), the cost of merging that segment (the sum of stones) is added to $dp[i][j]$.

Math

Round 9 · Low · Hard · 1 question

Math

□ #68 Text Justification

HAR[Open on LeetCode](#)

LeetDoocs · SimplyLeet · WalkCC · LeetCode
Array · String · Simulation
Lists: CodeSignal

Problem

Given an array of strings `words` and a width `maxWidth`, format the text such that each line has exactly `maxWidth` characters and is fully (left and right) justified.

You should pack your words in a greedy approach; that is, pack as many words as you can in each line. Pad extra spaces ' ' when necessary so that each line has exactly `maxWidth` characters.

Extra spaces between words should be distributed as evenly as possible. If the number of spaces on a line does not divide evenly between words, the empty slots on the left will be assigned more spaces than the slots on the right.

For the last line of text, it should be left-justified, and no extra space is inserted between words.

Note:

- A word is defined as a character sequence consisting of non-space characters only.
- Each word's length is guaranteed to be greater than 0 and not exceed `maxWidth`.
- The input array `words` contains at least one word.

Example 1:

```
Input: words = ["This", "is", "an", "example", "of", "text", "justification."],
maxWidth = 16
Output:
[
  "This    is    an",
  "example  of text",
  "justification. "
]
```

Example 2:

```
Input: words = ["What","must","be","acknowledgment","shall","be"], maxWidth =
16
Output:
[
  "What    must    be",
  "acknowledgment  ",
  "shall be        "
]
Explanation: Note that the last line is "shall be " instead of "shall be",
because the last line must be left-justified instead of fully-justified.
```

Example 3:

```

Input: words = ["Science","is","what","we","understand","well","enough","to","
explain","to","a","computer.","Art","is","everything","else","we","do"],
maxWidth = 20
Output:
[
  "Science is what we",
  "understand well",
  "enough to explain to",
  "a computer. Art is",
  "everything else we",

```

Constraints:

- $1 \leq \text{words.length} \leq 300$
- $1 \leq \text{words}[i].\text{length} \leq 20$
- $\text{words}[i]$ consists of only English letters and symbols.
- $1 \leq \text{maxWidth} \leq 100$
- $\text{words}[i].\text{length} \leq \text{maxWidth}$

No accepted LeetCode solution yet.

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

"Let me make sure I understand the problem correctly.

Arrange words into lines of exactly maxWidth chars. Spaces distributed as evenly as possible.

Extra spaces go to the LEFT. Last line: left-justified with single spaces. Right?"

#

"A few quick questions: Each word fits in maxWidth? Yes per constraints.

At least one word? Yes."

#

"Let me walk: words=['This','is','an','example'], maxWidth=16.

Line 1: 'This is an' (total=16) ✓. Last: left-justified."

PHASE 2 — BRUTE FORCE

"Let me start with brute force to lock in the logic.

Greedy packing: fit as many words per line as possible. Then distribute spaces.

$O(n \cdot \text{maxWidth})$ time. This IS the optimal approach.

```
# Should I go ahead and optimize?"
class _68_TextJustification_Brute:
    def fullJustify(self, words, maxWidth):
        result = []; i = 0; n = len(words)
        while i < n:
            line_words = [words[i]]; line_len = len(words[i]); i += 1
            while i < n and line_len + 1 + len(words[i]) <= maxWidth:
                line_len += 1 + len(words[i]); line_words.append(words[i]); i += 1
            result.append(line_words)
        lines = []
        for idx, lw in enumerate(result):
            if idx == len(result) - 1 or len(lw) == 1:
                lines.append(' '.join(lw).ljust(maxWidth))
            else:
                total = maxWidth - sum(len(w) for w in lw)
                gaps = len(lw) - 1
                base, extra = divmod(total, gaps)
                line = lw[0]
                for j in range(1, len(lw)):
                    line += ' ' * (base + (1 if j <= extra else 0)) + lw[j]
                lines.append(line)
        return lines
```

PHASE 3 — OPTIMIZE

"Already $O(n)$ – greedy packing + space distribution is optimal.

The bottleneck is correct space distribution and last-line handling."

PHASE 4 — CLEAN CODE

"Now I'll implement the optimized approach with meaningful names."

```
class _68_TextJustification:
    def fullJustify(self, words, maxWidth):
        result = []
        i = 0
        while i < len(words):
            line = [words[i]]; length = len(words[i]); i += 1
            while i < len(words) and length + 1 + len(words[i]) <= maxWidth:
                length += 1 + len(words[i]); line.append(words[i]); i += 1
            if i == len(words) or len(line) == 1:
                result.append(' '.join(line).ljust(maxWidth))
            else:
                spaces = maxWidth - sum(len(w) for w in line)
                gaps = len(line) - 1
                base, extra = divmod(spaces, gaps)
                row = line[0]
                for j in range(1, len(line)):
                    row += ' ' * (base + (1 if j <= extra else 0)) + line[j]
                result.append(row)
        return result
```

PHASE 5 — TEST & DEBUG

```
# "Let me trace through a few cases."
#
# words=['This','is','an','example'], maxWidth=16:
# line=['This','is','an'], length=10. spaces=6, gaps=2. base=3,extra=0.
# row='This is an'. len=16 ✓
# Last line='example '.ljust(16) ✓
#
# Single word line: 'word'.ljust(16) → padded with spaces ✓
#
# "No bugs. base + (1 if j <= extra else 0) distributes extra spaces to left gaps."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n*maxWidth): O(n) lines, each O(maxWidth) to build. Space O(n) output.
# Follow-up: minimize raggedness (aesthetic) – DP approach balances line lengths.
# Follow-up: proportional fonts – use pixel widths instead of character counts."
```

Community Solutions (Python)

• WalkCC

```
class Solution:
    def fullJustify(self, words: list[str], maxWidth: int) -> list[str]:
        ans = []
        row = []
        rowLetters = 0

        for word in words:
            # If we place the word in this row, it will exceed the maximum width.
            # Therefore, we cannot put the word in this row and have to pad spaces
            # for each word in this row.

            if rowLetters + len(word) + len(row) > maxWidth:
                for i in range(maxWidth - rowLetters):
                    row[i % (len(row) - 1 or 1)] += ' '
                ans.append(''.join(row))
                row = []
                rowLetters = 0
            row.append(word)
            rowLetters += len(word)

        return ans + [' '.join(row).ljust(maxWidth)]
```

• LeetDoocs


```

class Solution:
    def fullJustify(self, words: List[str], maxWidth: int) -> List[str]:
        ans = []
        i, n = 0, len(words)
        while i < n:
            t = []
            cnt = len(words[i])
            t.append(words[i])
            i += 1
            while i < n and cnt + 1 + len(words[i]) <= maxWidth:

                cnt += 1 + len(words[i])
                t.append(words[i])
                i += 1
            if i == n or len(t) == 1:
                left = ' '.join(t)
                right = ' ' * (maxWidth - len(left))
                ans.append(left + right)
                continue
            space_width = maxWidth - (cnt - len(t) + 1)
            w, m = divmod(space_width, len(t) - 1)

            row = []
            for j, s in enumerate(t[:-1]):
                row.append(s)
                row.append(' ' * (w + (1 if j < m else 0)))
            row.append(t[-1])
            ans.append(' '.join(row))
        return ans

```

• SimplyLeet

```
def fullJustify(words, maxWidth):
    result = []          # Final list of justified text lines.
    current_words = []   # Current line's words.
    current_len = 0      # Total length of current line's words (excluding
spaces).

    # Process all words.
    for word in words:
        # Check if adding the next word would exceed the maxWidth when
including minimum one space between words.
        if current_words and (current_len + len(word) + len(current_words)) >
maxWidth:
            total_spaces = maxWidth - current_len          # Total spaces to
distribute.

            gaps = len(current_words) - 1                  # Number of gaps
between words.

            # If there is only one word, left justify.
            if gaps == 0:
                line = current_words[0] + ' ' * total_spaces
            else:
                # Calculate even spaces and extras to add from the left.
                space_per_gap, extra = divmod(total_spaces, gaps)
                line = ''
                for i in range(gaps):

                    # Distribute an extra space to the leftmost gaps.
                    spaces = space_per_gap + (1 if i < extra else 0)
                    line += current_words[i] + ' ' * spaces
                line += current_words[-1]                    # Append the
last word.

            result.append(line)
            # Reset the current line to start with the current word.
            current_words = []
            current_len = 0

    # Add the word to the current line.
```

```

        current_words.append(word)
        current_len += len(word)

    # Process the last line, which is left-justified.
    line = ' '.join(current_words)
    line += ' ' * (maxWidth - len(line))
    result.append(line)
    return result

```

Example usage:

```

words = ["This", "is", "an", "example", "of", "text", "justification."]
maxWidth = 16
print(fullJustify(words, maxWidth))

```

• LC.ca

...

Explanation:

- The function `fullJustify` takes a list of words and a maximum width `maxWidth` as inputs.
- It iterates over each word, deciding whether to add the current word to the current line (`cur`) or to start a new line.
- If adding the current word to the line would exceed `maxWidth`, it justifies the current line by adding extra spaces between words as needed, then starts a new line.
- Once all words are processed, it left-justifies the last line by joining the remaining words in `cur` with a single space and then using `.ljust(maxWidth)` to ensure the line is of maximum width.
- The result is a list of strings, where each string represents a justified line of text.

This solution carefully handles edge cases, such as when there's only one word in the line (avoiding division by zero) and ensuring the last line is left-justified instead of fully justified.

```
'''
```

```
'''
```

```
>>> cur = [1]
>>> len(cur) - 1
0
>>> ( len(cur) - 1 or 1 )
1
'''
```

```
'''
```

```
# api: string.ljust(width[, fillchar])
```

```
>>> "Hello".ljust(10)
'Hello '
>>> "Hello".ljust(10, '-')
'Hello-----'
>>> "Hello".ljust(2)
'Hello'
```

```
>>> "Hello World".ljust(20)
'Hello World '
>>> "Hello World".ljust(20, '-')
'Hello World-----'
```

```
# api: string.rjust(width[, fillchar])
>>> "Hello".rjust(10)
' Hello'
>>> "Hello".rjust(10, '-')
'------Hello'
```

```
'-----Hello'
>>> "Hello".rjust(2)
'Hello'
```

```
>>> "Hello World".rjust(20)
' Hello World'
>>> "Hello World".rjust(20, '-')
'------Hello World'
```

```
'''
```

```

class Solution:
    def fullJustify(self, words: List[str], maxWidth: int) -> List[str]:
        res, cur, num_of_letters = [], [], 0

        for w in words:
            # len(cur) is the space count, from last round, right one less
            # after adding 'w'
            if num_of_letters + len(cur) + len(w) > maxWidth:
                for i in range(maxWidth - num_of_letters):
                    # The "or 1" part is for dealing with the edge case
                    # len(cur) == 1
                    cur[i % (len(cur) - 1 or 1)] += ' '
                res.append(''.join(cur))
                cur, num_of_letters = [], 0
            cur += [w]
            num_of_letters += len(w)

        return res + [' '.join(cur).ljust(maxWidth)]

```

```
#####
```

```

'''
>>> 17 % 3
2
>>> divmod(17,3)
(5, 2)
'''

'''
>>> ['This', 'is', 'an']

```

```
['This', 'is', 'an']
>>> t = ['This', 'is', 'an']
>>> spaces = [' ', ' ']

# note: here j starts from 0
>>> for j, word in enumerate(t[1:]):
...     print(j)
...     print(word)
...
0
```

```
is
1
an
'''
```

```
class Solution:
    def fullJustify(self, words: List[str], maxWidth: int) -> List[str]:
        def partition(n, cnt):
            base, mod = divmod(n, cnt)
            res = [f"{' ' * (base + (i < mod))}" for i in range(cnt)]

            return res

        ans = []
        i, n = 0, len(words)
        while i < n: # one line per iteration
            t = [words[i]]
            cnt = len(words[i])
            i += 1 # move to next word
            while i < n and cnt + 1 + len(words[i]) <= maxWidth: # greedy
                search for one line
                cnt += 1 + len(words[i])
```

```

        t.append(words[i])
        i += 1
    if i == n or len(t) == 1:
        # this is the last line or only one (super-long) word in a line
        left = ' '.join(t)
        right = ' ' * (maxWidth - len(left))
        ans.append(f"{left}{right}")
        continue # so i != n , so only one word, no need to add spaces.
                  # e.g. a word with the same length as maxWidth,
                  # or, e.g. the 2nd word is super long and cannot fit
in current line

```

```

        # if i==n, then will not enter next while
        words_width = cnt - (len(t) - 1) # pure words total length, with no
spaces
                                                # spaces in-between these words in
t[: len(t) - 1
        space_width = maxWidth - words_width
        spaces = partition(space_width, len(t) - 1)
        sb = [t[0]] # 1st word of this line, no space before it
        for j, word in enumerate(t[1:]): # last word has no following space
            sb.append(spaces[j]) # j starts from 0
            sb.append(word)
        ans.append(' '.join(sb))

```

```

    return ans

```

```

#####

```

```

class Solution:
    def fullJustify(self, words: List[str], maxWidth: int) -> List[str]:
        left = 0
        result = []

        while left < len(words):

```

```
        right = self.findRight(left, words, maxWidth)
        result.append(self.justify(left, right, words, maxWidth))
        left = right + 1

    return result

def findRight(self, left, words, maxWidth):
    right = left
    word_length = len(words[right])

    while (right + 1) < len(words) and (word_length + 1 + len(words[right+
1])) <= maxWidth:
        right += 1
        word_length += 1 + len(words[right])

    return right

def justify(self, left, right, words, maxWidth):
    if right - left == 0:
        return self.padResult(words[left], maxWidth)

    is_last_line = (right == len(words) - 1)
    num_spaces = right - left
    total_space = maxWidth - self.wordsLength(left, right, words)

    space = " " if is_last_line else " " * (total_space // num_spaces)
    remainder = 0 if is_last_line else total_space % num_spaces

    result = ""
    for i in range(left, right):
        result += words[i]
```



```
        result += space
        if remainder > 0:
            result += " "
            remainder -= 1

    result += words[right]
    result += " " * (maxWidth - len(result))

    return result
```

```
def wordsLength(self, left, right, words):
    words_length = 0
    for i in range(left, right+1):
        words_length += len(words[i])
    return words_length

def padResult(self, result, maxWidth):
    return result + " " * (maxWidth - len(result))
```

◆ Quick Review

Key Insights

- Use a greedy approach to fill each line with as many words as possible without exceeding `maxWidth`.
- Calculate total space to distribute by subtracting the combined words length from `maxWidth`.
- If the line has more than one word, distribute extra spaces evenly; if not, pad the line with spaces at the end.

- Handle the last line separately by left-justifying the text (i.e., adding a single space between words and padding the end).

Space & Time

Time Complexity: $O(n)$ where n is the number of words, since each word is processed once.

Space Complexity: $O(n)$ for storing the result and temporary buffers.

Solution

The solution iterates over the list of words and groups them into lines such that the combined length of the words and minimum required spaces does not exceed `maxWidth`. For each completed group (line): If it's not the last line, calculate extra spaces required. If the line has multiple words, distribute the spaces evenly, adding any extra spaces from left to right. If the line contains a single word, append spaces to the right. For the last line, join words with a single space and pad the remaining width with spaces. Data structures used include lists (or arrays) for current words accumulation and the final result. The logic is implemented using a simulation approach to mimic text

formatting.